

Visual Studio 2005 Visual C#

Copyright© 2016 Microsoft Corporation

The content in this document is retired and is no longer updated or supported. Some links might not work. Retired content represents the latest updated version of this content.

Visual C#

Microsoft Visual C# 2005, pronounced C sharp, is a programming language designed for building a wide range of applications that run on the .NET Framework. C# is simple, powerful, type-safe, and object-oriented. With its many innovations, C# enables rapid application development while retaining the expressiveness and elegance of C-style languages.

Visual Studio supports Visual C# with a full-featured Code Editor, project templates, designers, code wizards, a powerful and easy-to-use debugger, and other tools. The .NET Framework class library provides access to a wide range of operating system services and other useful, well-designed classes that speed up the development cycle significantly.

In This Section

[Getting Started with Visual C#](#)

Introduces the features of C# 2.0 for programmers new to the language or new to Visual Studio, and provides a roadmap for finding Help on Visual Studio. This is also where "How Do I" pages are located.

[Using the Visual C# IDE](#)

Introduces the Visual C# development environment.

[Writing Applications with Visual C#](#)

Provides a high-level orientation covering common programming tasks using C# and the .NET Framework, with links to more detailed documentation.

[Migrating to Visual C#](#)

Compares the C# language to Java and C++ and describes how to use the Java Language Conversion Assistant to convert Java and Visual J++ applications to Visual C#.

[C# Programming Guide](#)

Provides information and practical examples on how to use C# language constructs.

[C# Reference](#)

Provides detailed reference information on C# programming concepts, keywords, types, operators, attributes, preprocessor directives, compiler switches, and compiler error and warnings.

[C# Language Specification](#)

Links to the latest version of the C# Specifications in Microsoft Word format.

[Visual C# Samples](#)

Sample source code demonstrating how to program using Visual C#.

Related Sections

[What's New in the C# 2.0 Language and Compiler](#)

Describes new language features.

[What's New in Visual C# 2005](#)

Describes new Code Editor, development environment, code wizard, and debugging features.

[Upgrading Visual C# Applications to Visual Studio 2005](#)

Describes updating your existing projects to Microsoft Visual Studio 2005.

See Also

Other Resources

[Visual Studio](#)

Getting Started with Visual C#

The following topics help you start developing applications using Microsoft Visual C# 2005. These topics will also introduce you to the many new features in Microsoft Visual Studio 2005 as well as version 2.0 of the C# language.

In This Section

[Visual C# Documentation Roadmap](#)

Provides a high-level orientation of the contents of the Visual C# documentation.

[Introduction to the C# Language and the .NET Framework](#)

Provides an overview of the C# language and the .NET platform.

[What's New in Visual C# 2005](#)

What's new in Microsoft Visual C# 2005.

[What's New in the C# 2.0 Language and Compiler](#)

What's new in version 2.0 of C#.

[Upgrading Visual C# Applications to Visual Studio 2005](#)

Updating your existing projects to Microsoft Visual Studio 2005.

[Creating Your First C# Application](#)

Write, compile and run a simple C# application.

[Using C# Starter Kits](#)

Using the C# Starter Kits.

[Additional Help Resources \(Visual C#\)](#)

Links to other help resources.

[How Do I in C#](#)

Links to topics that show how to perform a wide variety of specific tasks.

Related Sections

[Using the Visual C# IDE](#)

A guide to using the Visual C# development environment.

[Migrating to Visual C#](#)

Converting Java and Visual J++ applications to Visual C#.

[Writing Applications with Visual C#](#)

Provides overviews of common programming tasks using C# and the .NET Framework, and links to more detailed documentation.

[C# Programming Guide](#)

Provides information on C# programming concepts, and describes how to perform various tasks in C#.

[C# Reference](#)

Provides detailed reference information on C# keywords, operators, preprocessor directives, compiler switches, and compiler error and warnings.

[Visual C# Samples](#)

Sample source code demonstrating how to program using Visual C#.

[C# Terminology](#)

A glossary of C# terms.

See Also

Other Resources

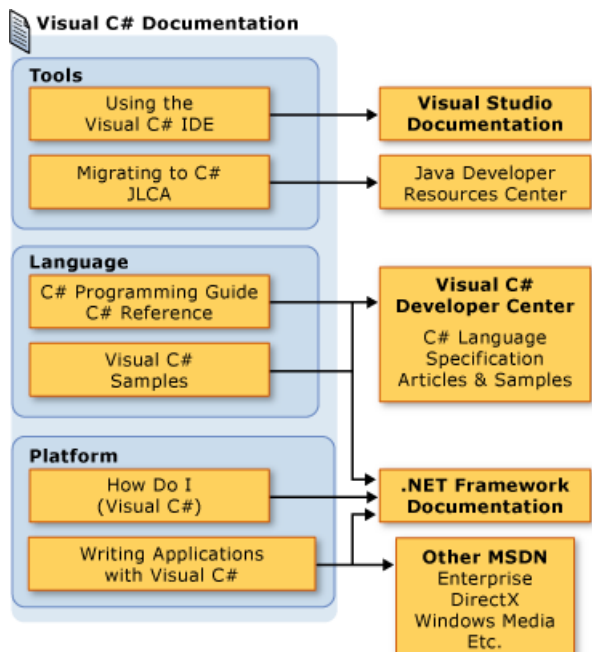
[Visual C#](#)

[Visual Studio](#)

Visual C# Documentation Roadmap

The Microsoft Visual C# 2005 documentation contains information that is specific to the C# language, such as keywords, compiler options, error messages, and programming concepts. This documentation also gives you an overview of how to use the integrated development environment (IDE). In addition, there are many links to more detailed help on .NET Framework classes, ASP.NET Web development, debugging, SQL database programming, and much more.

The following diagram provides a conceptual view of the contents of the Visual C# documentation, and the relationship of this content to other relevant sections of the Visual Studio documentation and MSDN Online.



Quick Links to C# Specific Documentation

[Getting Started with Visual C#](#)

[How Do I in C#](#)

[Migrating to Visual C#](#)

[Using the Visual C# IDE](#)

[Writing Applications with Visual C#](#)

[C# Reference](#)

[Visual C# Samples](#)

[C# Compiler Options](#)

Quick Links to Related Documentation

For more information about creating Windows applications, see [Windows-based Applications, Components, and Services](#).

For more information about creating Web applications, see [Visual Web Developer](#).

For more information about the .NET Framework class library, see [.NET Framework Class Library Overview](#).

For more information about the .NET Framework Common Language Runtime, the Common Type System, and other related concepts, see [Overview of the .NET Framework](#).

See Also

Other Resources

[Visual Studio](#)

Introduction to the C# Language and the .NET Framework

C# is an elegant and type-safe object-oriented language that enables developers to build a wide range of secure and robust applications that run on the .NET Framework. You can use C# to create traditional Windows client applications, XML Web services, distributed components, client-server applications, database applications, and much, much more. Microsoft Visual C# 2005 provides an advanced code editor, convenient user interface designers, integrated debugger, and many other tools to facilitate rapid application development based on version 2.0 of the C# language and the .NET Framework.

Note

The Visual C# documentation assumes that you have an understanding of basic programming concepts. If you are a complete beginner, you might want to explore Visual C# Express Edition, which is available on the Web. You can also take advantage of any of several excellent books and Web resources on C# to learn practical programming skills.

C# Language

C# syntax is highly expressive, yet with less than 90 keywords, it is also simple and easy to learn. The curly-brace syntax of C# will be instantly recognizable to anyone familiar with C, C++ or Java. Developers who know any of these languages are typically able to begin working productively in C# within a very short time. C# syntax simplifies many of the complexities of C++ while providing powerful features such as nullable value types, enumerations, delegates, anonymous methods and direct memory access, which are not found in Java. C# also supports generic methods and types, which provide increased type safety and performance, and iterators, which enable implementers of collection classes to define custom iteration behaviors that are simple to use by client code.

As an object-oriented language, C# supports the concepts of encapsulation, inheritance and polymorphism. All variables and methods, including the `Main` method, the application's entry point, are encapsulated within class definitions. A class may inherit directly from one parent class, but it may implement any number of interfaces. Methods that override virtual methods in a parent class require the **override** keyword as a way to avoid accidental redefinition. In C#, a struct is like a lightweight class; it is a stack-allocated type that can implement interfaces but does not support inheritance.

In addition to these basic object-oriented principles, C# facilitates the development of software components through several innovative language constructs, including:

- Encapsulated method signatures called delegates, which enable type-safe event notifications.
- Properties, which serve as accessors for private member variables.
- Attributes, which provide declarative metadata about types at run time.
- Inline XML documentation comments.

If you need to interact with other Windows software such as COM objects or native Win32 DLLs, you can do this in C# through a process called "Interop." Interop enables C# programs to do just about anything that a native C++ application can do. C# even supports pointers and the concept of "unsafe" code for those cases in which direct memory access is absolutely critical.

The C# build process is simple compared to C and C++ and more flexible than in Java. There are no separate header files, and no requirement that methods and types be declared in a particular order. A C# source file may define any number of classes, structs, interfaces, and events.

The following are additional C# resources:

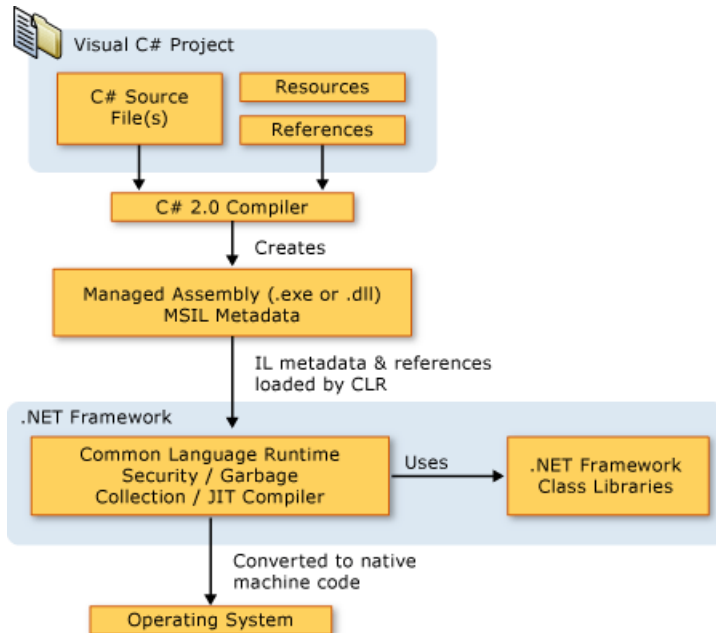
- For a good general introduction to the language, see Chapter 1 of the [C# Language Specification](#).
- For detailed information about specific aspects of the C# language, see the [C# Reference](#).
- For a comparison of C# syntax to that of Java and C++, see [The C# Programming Language for Java Developers](#) and [Comparison Between C++ and C#](#).

.NET Framework Platform Architecture

C# programs run on the .NET Framework, an integral component of Windows that includes a virtual execution system called the common language runtime (CLR) and a unified set of class libraries. The CLR is Microsoft's commercial implementation of the common language infrastructure (CLI), an international standard that is the basis for creating execution and development environments in which languages and libraries work together seamlessly.

Source code written in C# is compiled into an intermediate language (IL) that conforms to the CLI specification. The IL code, along with resources such as bitmaps and strings, is stored on disk in an executable file called an assembly, typically with an extension of .exe or .dll. An assembly contains a manifest that provides information on the assembly's types, version, culture, and security requirements.

When the C# program is executed, the assembly is loaded into the CLR, which might take various actions based on the information in the manifest. Then, if the security requirements are met, the CLR performs just in time (JIT) compilation to convert the IL code into native machine instructions. The CLR also provides other services related to automatic garbage collection, exception handling, and resource management. Code that is executed by the CLR is sometimes referred to as "managed code," in contrast to "unmanaged code" which is compiled into native machine language that targets a specific system. The following diagram illustrates the compile-time and run time relationships of C# source code files, the base class libraries, assemblies, and the CLR.



Language interoperability is a key feature of the .NET Framework. Because the IL code produced by the C# compiler conforms to the Common Type Specification (CTS), IL code generated from C# can interact with code that was generated from the .NET versions of Visual Basic, Visual C++, Visual J#, or any of more than 20 other CTS-compliant languages. A single assembly may contain multiple modules written in different .NET languages, and the types can reference each other just as if they were written in the same language.

In addition to the run time services, the .NET Framework also includes an extensive library of over 4000 classes organized into namespaces that provide a wide variety of useful functionality for everything from file input and output to string manipulation to XML parsing, to Windows Forms controls. The typical C# application uses the .NET Framework class library extensively to handle common "plumbing" chores.

For more information about the .NET Framework platform, see [Overview of the .NET Framework](#).

See Also

Other Resources

[Visual C#](#)

[Writing Applications with Visual C#](#)

What's New in Visual C# 2005

Microsoft Visual C# 2005 includes new features in the following areas:

- Language and Compiler
- Code Editor
- Development Environment
- Documentation and Language Specification
- Debugging

Language and Compiler

The C# language now supports generic types, iterators, and partial types. The latest version of the C# compiler also includes new features and options. For more information, see [What's New in the C# 2.0 Language and Compiler](#).

Code Editor

The Code Editor contains the following new features for Visual C# 2005.

Code Snippets

Code Snippets speed the entry of common code constructs by providing a template you can fill out. Snippets are stored as XML files that can be easily edited and customized.

- [Code Snippets \(C#\)](#)
- [How to: Use Code Snippets \(C#\)](#)
- [How to: Use Surround-with Code Snippets](#)

Refactoring

Refactoring tools can automatically restructure your source code, for example, by promoting local variables to parameters or converting a block of code into a method.

- [How to: Promote Local Variable to Parameter](#)
- [Extract Method](#)
- [Encapsulate Field](#)
- [Extract Interface](#)
- [Rename](#)
- [Remove Parameters](#)
- [Reorder Parameters](#)

Development Environment

The development environment includes the following enhancements for Visual C# 2005.

IntelliSense

IntelliSense has been enhanced with the following new features:

- The completion list for [List Members](#) automatically appears when you backspace the cursor to a scope operator that precedes an object, or when you undo the completion action.
- When you write error handling code, [List Members](#) helps you discover which exception to catch by filtering irrelevant members from the completion list in a [catch](#) clause.
- When you need to insert standardized code, [Automatic Code Generation](#) now allows you to prompt IntelliSense to insert the code for you.

- IntelliSense is available when authoring Web applications.

Class Designer

The Class Designer is a new editor that graphically displays classes and types, and allows methods to be added or modified. It is also possible to use refactoring tools from the **Class Designer** window.

- See [Designing and Viewing Classes and Types](#).

Object Test Bench

The Object Test Bench is designed for simple object-level testing. It allows you to create an instance of an object, and call its methods.

- See [Object Test Bench](#).

ClickOnce Deployment

ClickOnce deployment allows you to publish Windows applications to a Web server or network file share for simplified installation.

- See [ClickOnce Deployment](#).

Tools Support for Strong Named Assemblies

The **Project Properties** dialog box has been redesigned, and now includes support for signing assemblies.

- See [Project Properties](#).

Code Wizards

The following code wizards are now obsolete:

- C# Method Wizard
- C# Property Wizard
- C# Field Wizard
- C# Indexer Wizard

Documentation and Language Specification

The C# reference documentation has been extensively rewritten to provide more complete information for common as well as advanced usage questions that developers might encounter while creating applications in C#.

The C# Language Specification is no longer integrated into the Help environment, but is provided in two .doc files. These files are installed by default under `\\Microsoft Visual Studio 8\\vcsharp\\specifications\\1033\\`. The most up-to-date versions can be downloaded from the C# Developer Center on MSDN. For more information, see [C# Language Specification](#).

C# Specific Debugging Enhancements

New features, including Edit and Continue, have been added to aid the C# developer.

- See [What's New in the Visual Studio 2005 Debugger](#).

See Also

Concepts

[What's New in Visual Studio 2005](#)

Other Resources

[Visual C#](#)

[Getting Started with Visual C#](#)

[Using the Visual C# IDE](#)

What's New in the C# 2.0 Language and Compiler

With the release of Visual Studio 2005, the C# language has been updated to version 2.0, which supports the following new features:

Generics

Generic types are added to the language to enable programmers to achieve a high level of code reuse and enhanced performance for collection classes. Generic types can differ only by arity. Parameters can also be forced to be specific types. For more information, see [Generic Type Parameters](#).

Iterators

Iterators make it easier to dictate how a **foreach** loop will iterate over a collection's contents.

Partial Classes

Partial type definitions allow a single type, such as a class, to be split into multiple files. The Visual Studio designer uses this feature to separate its generated code from user code.

Nullable Types

Nullable types allow a variable to contain a value that is undefined. Nullable types are useful when working with databases and other data structures that may contain elements that contain no specific values.

Anonymous Methods

It is now possible to pass a block of code as a parameter. Anywhere a delegate is expected, a code block can be used instead: there is no need to define a new method.

Namespace alias qualifier

The namespace alias qualifier (::) provides more control over accessing namespace members. The [global ::](#) alias allows access the root namespace that may be hidden by an entity in your code.

Static Classes

Static classes are a safe and convenient way of declaring a class containing static methods that cannot be instantiated. In C# version 1.2 you would have defined the class constructor as private to prevent the class being instantiated.

External Assembly Alias

Reference different versions of the same component contained in the same assembly with this expanded use of the [extern](#) keyword.

Property Accessor Accessibility

It is now possible to define different levels of accessibility for the **get** and **set** accessors on properties.

Covariance and Contravariance in Delegates

The method passed to a delegate may now have greater flexibility in its return type and parameters.

How to: Declare, Instantiate, and Use a Delegate

Method group conversion provides a simplified syntax for declaring delegates.

Fixed Size Buffers

In an unsafe code block, it is now possible to declare fixed-size structures with embedded arrays.

Friend Assemblies

Assemblies can provide access to non-public types to other assemblies.

Inline warning control

The **#pragma** warning directive may be used to disable and enable certain compiler warnings.

volatile

The **volatile** keyword can now be applied to [IntPtr](#) and [UIntPtr](#).

The C# compiler introduces the following additions and changes for this release:

[/errorreport](#) option

Can be used to report internal compiler errors to Microsoft over the Internet.

/incremental option

Has been removed.

[/keycontainer](#) and [/keyfile](#) options

Support specifying cryptographic keys.

[/langversion](#) option

Can be used to specify compatibility with a specific version of the language.

[/linkresource](#) option

Contains additional options.

[/moduleassemblyname](#) option

Allows you to build a .netmodule file and access non-public types in an existing assembly.

[/pdb](#) option

Specifies the name and location of the .pdb file.

[/platform](#) option

Enables you to target Itanium Family (IPF) and x64 architectures.

[#pragma warning](#)

Used to disable and enable individual warnings in code.

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[C# Language Specification](#)

[C# Reference](#)

Upgrading Visual C# Applications to Visual Studio 2005

When you open a project or solution file created by an earlier version of Visual Studio, the Upgrade Wizard steps you through the process of converting your project to Visual Studio 2005. The Upgrade Wizard performs many tasks for you: creating new properties and attributes, deleting obsolete ones, and so on, but because error checking has been tightened up, you might encounter new error or warning messages that were not produced by the previous version of the compiler. Thus, the final step in upgrading an existing application is to make the code changes needed to resolve any new errors.

Often, code that produced one message in prior versions of the C# compiler now produces a different message in the current version. Usually this is because a general message has been replaced by a more specific one. Because no change in code is required, we do not document such differences.

The following are the new messages that the Upgrade Wizard generates because of stricter error checking.

New Error and Warning Messages

CS0121: Ambiguous call

Due to an implicit conversion, the compiler was not able to call one form of an overloaded method. You can resolve this error in the following ways:

- Specify the method parameters in such a way that implicit conversion does not take place.
- Remove all overloads for the method.
- Cast to proper type before calling the method.

CS0122: Method inaccessible due to its protection level

You may receive this error when referencing a type in an assembly compiled by C++ that was compiled with the **/d1PrivateNativeTypes** compiler option.

This error occurs because in the current release, a C++ assembly produces a signature that uses a type that is not marked as public.

You can work around this issue by using the **/test:AllowBadRetTypeAccess** compiler option. This option will be removed when this feature has been fixed.

CS0429: Unreachable expression code detected

This error occurs whenever part of an expression in your code is unreachable. For example, the condition `false && myTest()` meets this criteria because the `myTest()` method will never get evaluated because the left hand side of the `&&` operation is always false. To fix this, redo the logic test to eliminate the unreachable code.

CS0441: A class cannot be both static and sealed

All static classes are also sealed classes. The C# language specification prohibits specifying both modifiers on a class, and the compiler now reports this as an error.

To fix this error, remove `sealed` from the class.

CS1699: Warning on use of assembly signing attributes

The assembly attributes that specify signing have been moved from code to compiler options. Using the `AssemblyKeyFile` or `AssemblyKeyName` attributes in code now produces this warning.

Instead of these attributes, you should use the following compiler options:

- use the [/keyfile \(Specify Strong Name Key File\) \(C# Compiler Options\)](#) compiler option instead of the `AssemblyKeyFile` attribute, and use [/keycontainer \(Specify Strong Name Key Container\) \(C# Compiler Options\)](#) instead of `AssemblyKeyName`.

Not switching to the command-line options may hamper compiler diagnostics when using friend assemblies.

If you are using [/warnaserror \(Treat Warnings as Errors\) \(C# Compiler Options\)](#), you can convert this back into a warning by adding **/warnaserror-:1699** to your compiler command line. If necessary, you can shut off the warning by using **/nowarn:1699**.

Incremental compilation removed

The `/incremental` compiler option has been removed. The Edit and Continue feature replaces this functionality.

See Also

Other Resources

[C# Compiler Options](#)

Creating Your First C# Application

It only takes a minute to create a C# application. Follow these steps to create a program that opens a window and reacts to a button press.

Procedures

To create a C# application

1. On the **File** menu, point to **New**, and then click **Project**.
2. Ensure the **Windows Application** template is selected, in the **Name** field, type **MyProject**, and click **OK**.
You will see a Windows Form in the Windows Forms designer. This is the user interface for your application.
3. On the **View** menu, click **Toolbox** to make the list of controls visible.
4. Expand the **Common Controls** list, and drag the **Label** control to your form.
5. Also from the Toolbox **Common Controls** list, drag a button onto the form, near the label.
6. Double-click the new button to open the Code Editor. Visual C# has inserted a method called `button1_Click` that is executed when the button is clicked.
7. Change the method to look like this:

```
private void button1_Click(object sender, EventArgs e)
{
    label1.Text = "Hello, World!";
}
```

8. Press F5 to compile and run your application.

When you click the button, a text message is displayed. Congratulations! You've just written your first C# application.

See Also

Other Resources

[Visual C#](#)

[Getting Started with Visual C#](#)

Project Templates in Visual C# Editions

When you create a new project, icons in the **New Project** dialog box and the **New Web Site** dialog box represent the available project types and their templates. The project template you choose determines the output type and other options available for that project. Not all project templates are available in all editions of Visual C#.

Note

Documentation for features that are not available in Visual C# Express Edition or Visual C# Standard Edition may be included in the documentation set for these editions.

Visual C# Project Templates

The following table shows which Visual C# project templates are available in the different editions of Visual Studio.

Template	Microsoft Visual C# Express Edition	Visual Studio 2005 Standard	Visual Studio 2005 Professional and above
ASP.NET Web Site		X	X
ASP.NET Web Service		X	X
ASP.NET Crystal Reports Web Site			X
Class Library	X	X	X
Console Application	X	X	X
Crystal Reports Application			X
Device Application		X	X
Empty Project	X	X	X
Empty Web Site		X	X
Excel Template			X
Excel Workbook			X
Movie Collection Starter Kit	X	X	X
Outlook Add-in			X
Personal Web Site Starter Kit		X	X
Pocket PC 2003: Class Library		X	X
Pocket PC 2003: Class Library (1.0)		X	X
Pocket PC 2003: Console Application		X	X
Pocket PC 2003: Console Application (1.0)		X	X

Pocket PC 2003: Control Library		X	X
Pocket PC 2003: Device Application		X	X
Pocket PC 2003: Device Application (1.0)		X	X
Pocket PC 2003: Empty Project		X	X
Pocket PC 2003: Empty Project (1.0)		X	X
Screen Saver Starter Kit	X	X	X
SQL Server Project			X
Smartphone 2003: Class Library (1.0)		X	X
Smartphone 2003: Console Application (1.0)		X	X
Smartphone 2003: Device Application (1.0)		X	X
Smartphone 2003: Empty Project (1.0)		X	X
Test Project			X
Web Control Library		X	X
Windows Application	X	X	X
Windows CE 5.0: Class Library		X	X
Windows CE 5.0: Console Application		X	X
Windows CE 5.0: Control Library		X	X
Windows CE 5.0: Device Application		X	X
Windows CE 5.0: Empty Project		X	X
Windows Control Library		X	X
Windows Service			X
Word Document			X
Word Template			X

See Also

Concepts

[Installation and Setup Essentials](#)

[What's New in Visual Studio 2005](#)

[What's New in Visual C# 2005](#)

Other Resources

[Writing Applications with Visual C#](#)

Using C# Starter Kits

A Starter Kit is a complete, self-contained application ready for you to load and build. A Starter Kit comes with its own documentation, including descriptions of programming techniques, and suggestions for how it may be customized. Starter Kits are a great way to see a working C# application in action.

To load and build a Visual C# Starter Kit

1. On the **File** menu, click **New Project**.

The **New Project** dialog box appears. This dialog box lists the different default application types that Visual C# can create.

2. Select a **Starter Kit** application type, and click **OK**.

The Starter Kit loads into Visual C#.

3. To build and launch the Starter Kit project, press F5.

See Also

Other Resources

[Visual C#](#)

[Getting Started with Visual C#](#)

Additional Help Resources (Visual C#)

The following sites and newsgroups can help you find answers to common and not-so-common problems.

Microsoft Resources

The following are Web sites maintained by Microsoft that host articles and discussion groups on topics of interest to C# developers.

On the Web

[Microsoft Help and Support](#)

Provides access to KB articles, downloads and updates, support Webcasts, and other services.

[Microsoft Visual C# Developer Center](#)

Provides code samples, upgrade information, and technical content.

[MSDN Discussion Groups](#)

Provides a way to connect as a community with experts from around the world.

Forums

[Microsoft Technical Forums](#)

Web-based discussion forums for many Microsoft technologies including C# and the .NET Framework.

Newsgroups

microsoft.public.dotnet.languages.csharp

Provides a forum for questions and general discussion of Visual C#.

microsoft.public.vsnet.general

Provides a forum for questions and issues on Visual Studio.

microsoft.public.vsnet.ide

Provides a forum for questions about working in the Visual Studio environment.

microsoft.public.vsnet.documentation

Provides a forum for questions and issues on the Visual C# documentation.

Third-Party Resources

MSDN's Web site provides information on current third-party sites and newsgroups of interest. For the most current list of resources available, see the [MSDN Community Web site](#).

See Also

Other Resources

[Visual C#](#)

[Getting Started with Visual C#](#)

[Using Help in Visual Studio](#)

[Interacting with Other Developers](#)

[Product Support and Accessibility](#)

How Do I in C#

How Do I is your gateway to key task-based topics about C# programming and application development. The essential categories for what you can do with C# are listed in this topic. The links provide pointers to important procedure-based Help pages.

[C# Language](#)

[C# Language Specification...](#) [Threading...](#) [Generics...](#) [Code example snippets...](#) [Samples...](#) [more](#)

[.NET Framework](#)

[File I/O...](#) [Strings...](#) [Collections...](#) [Serialization...](#) [Components...](#) [Assemblies and Application Domains...](#) [more](#)

[Windows Applications](#)

[Building Windows Applications...](#) [Controls...](#) [Windows Forms...](#) [Drawing...](#) [more](#)

[Web Pages and Web Services](#)

[ASP.NET Web pages...](#) [XML Web Services...](#) [more](#)

[Debugging](#)

[Using the VS Debugger...](#) [.NET Framework Trace class...](#) [Debugging SQL Transactions...](#) [more](#)

[Data Access](#)

[Connecting to data sources...](#) [SQL Server...](#) [Data binding...](#) [more](#)

[Designing Classes](#)

[Class Designer...](#) [Working with Classes and Other Types...](#) [Create and Modify Type Members...](#) [Class Library Design Guidelines...](#) [more](#)

[Security](#)

[Code Access Security...](#) [Security Policy Best Practices...](#) [Permission Sets...](#) [more](#)

[Office Programming](#)

[Office Programming...](#) [Controls...](#) [Word...](#) [Excel...](#) [more](#)

[Smart Devices](#)

[What's New in Smart Device Projects...](#) [Programming Smart Devices...](#) [Debugging Smart Devices...](#) [more](#)

[Deployment](#)

[ClickOnce...](#) [Windows Installer](#)

Additional Resources

The following sites require an Internet connection.

[Visual Studio 2005 Developer Center](#)

Contains many articles and resources on developing applications using Visual Studio 2005. This site is updated regularly with new content.

[Visual C# Developer Center](#)

Contains many articles and resources on developing C# applications. This site is updated regularly with new content.

[Microsoft .NET Framework Developer Center](#)

Contains many articles and resources on developing and debugging .NET Framework applications. This site is updated regularly with new content.

See Also

Other Resources

[Getting Started with Visual C#](#)

C# Language (How Do I in C#)

This page links to help on widely used C# Language tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

The C# Language

[What's New in the C# 2.0 Language and Compiler](#)

Contains information regarding the new features, including generics, iterators, anonymous methods, and partial types.

[Using C# Starter Kits](#)

Explains how to load and build a Visual C# Starter Kit.

[C# Language Specification](#)

Pointers to the latest version of the specification in Microsoft Word format.

Command Line

[Main\(\) and Command Line Arguments \(C# Programming Guide\)](#)

Explains the `Main` method, the entry point of your program, where you create objects and invoke other methods. There can only be one entry point in a C# program.

[How to: Access Command-Line Arguments Using `foreach` \(C# Programming Guide\)](#)

Provides a code example that shows how access the command line parameters.

[How to: Display Command Line Arguments \(C# Programming Guide\)](#)

Explains how to display to the Command Line Arguments through the **`args`** string array.

[Main\(\) Return Values \(C# Programming Guide\)](#)

Explains the possible return values of the main method.

Classes and Inheritance

[base \(C# Reference\)](#)

Explains how to specify the base-class constructor called when creating instances of a derived class.

[How to: Know the Difference Between Passing a Struct and Passing a Class Reference to a Method \(C# Programming Guide\)](#)

Contains a code example that shows that when a struct is passed to a method, a copy of the struct is passed, but when a class instance is passed, a reference is passed.

[Instance Constructors \(C# Programming Guide\)](#)

Explains class constructors and inheritance.

[How to: Write a Copy Constructor \(C# Programming Guide\)](#)

Contains a code example that demonstrates how a constructor of a class takes another object as an argument.

[How to: Implement User-Defined Conversions Between Structs \(C# Programming Guide\)](#)

Contains a code example that defines two structs and demonstrates conversions between them.

Data Types

[Boxing Conversion \(C# Programming Guide\)](#)

Contains an example that illustrates that a value type and a boxed object can store different values.

[Unboxing Conversion \(C# Programming Guide\)](#)

Contains a code example that illustrates how to display an error message for a case of invalid boxing.

Arrays

[Arrays as Objects \(C# Programming Guide\)](#)

Contains a code example that displays the number of dimensions of an array.

[Jagged Arrays \(C# Programming Guide\)](#)

Contains a code example that builds an array whose elements are themselves arrays.

[Passing Arrays as Parameters \(C# Programming Guide\)](#)

Contains code examples that initialize a string array and pass it as a parameter to the **PrintArray** method, where its elements are displayed.

[Passing Arrays Using ref and out \(C# Programming Guide\)](#)

Contains code examples that demonstrate the difference between **out** and **ref** when used in passing arrays to methods.

Properties

[How to: Declare and Use Read/Write Properties \(C# Programming Guide\)](#)

Contains an example that shows you how to declare and use read/write properties.

[How to: Define Abstract Properties \(C# Programming Guide\)](#)

Contains a code example that shows how to define abstract properties.

Methods

[Passing Value-Type Parameters \(C# Programming Guide\)](#)

Contains code examples that demonstrate the various ways to pass value types.

[Passing Reference-Type Parameters \(C# Programming Guide\)](#)

Contains code examples that demonstrate the various ways to pass reference types.

Events

[How to: Subscribe to and Unsubscribe from Events \(C# Programming Guide\)](#)

Shows how to subscribe to events published by other classes, including Forms, buttons, List Boxes, and so on.

[How to: Publish Events that Conform to .NET Framework Guidelines \(C# Programming Guide\)](#)

Shows how to create events based on [EventHandler](#) and [EventHandler<T>](#).

[How to: Implement Interface Events \(C# Programming Guide\)](#)

Shows how to implement events that are declared in interfaces.

[How to: Use a Dictionary to Store Event Instances \(C# Programming Guide\)](#)

Explains how to use a hash table to store the event instances.

[How to: Raise Base Class Events in Derived Classes \(C# Programming Guide\)](#)

Shows how to wrap base class events in protected virtual methods in order to make them callable from derived classes.

Interfaces

[How to: Explicitly Implement Interface Members \(C# Programming Guide\)](#)

Shows how to declare a class that explicitly implements an interface and how to access the members through the interface instance.

[How to: Explicitly Implement Interface Members with Inheritance \(C# Programming Guide\)](#)

Provides an example that displays the dimensions of a box in both metric and English units.

Generics

[An Introduction to C# Generics](#)

Describes how generics allow you to define type-safe collection classes. You implement the generic class only once, but you can declare and use it with any type.

[Generics in the .NET Framework](#)

Explains the features and use of the new set of generic collections in the [System.Collections.Generic](#) namespace.

[default Keyword in Generic Code \(C# Programming Guide\)](#)

Provides a code example that demonstrates how to use the default keyword for type parameters.

[Generic Methods \(C# Programming Guide\)](#)

Introduces the syntax to declare a generic method. It also demonstrates an example on using generic methods in an application.

[Constraints on Type Parameters \(C# Programming Guide\)](#)

Shows how to constrain type parameters to enable access to methods and properties of the types used to instantiate the generic class.

[Generic Delegates \(C# Programming Guide\)](#)

Contains the syntax to declare generic delegates. It also includes important remarks on instantiating and using generic delegates as well as code examples.

Namespaces

[How to: Use the Namespace Alias Qualifier \(C# Programming Guide\)](#)

Discusses the ability to access a member in the global namespace when the member might be hidden by another entity of the same name.

Iterators

[How to: Create an Iterator Block for a Generic List \(C# Programming Guide\)](#)

Provides an example where an array of integers is used to build the list `SampleCollection`. A `for` loop iterates through the collection and yields the value of each item. Then a **foreach** loop is used to display the items of the collection.

[How to: Create an Iterator Block for a Generic List \(C# Programming Guide\)](#)

Provides an example where a generic class `Stack<T>` implements a generic interface `IEnumerator<T>`. An array of the type `T` is declared and assigned values by using the method `Push`. In the `GetEnumerator` method, the values of the array are returned using the `yield return` statement.

Delegates

[How to: Combine Delegates \(Multicast Delegates\)\(C# Programming Guide\)](#)

Provides an example that demonstrates how to compose multicast delegates.

[How to: Declare, Instantiate, and Use a Delegate \(C# Programming Guide\)](#)

Provides an example that illustrates how to declare, instantiate, and use a delegate.

Operator Overloading

[How to: Use Operator Overloading to Create a Complex Number Class \(C# Programming Guide\)](#)

Shows how you can use operator overloading to create a complex number class `Complex` that defines complex addition.

Interoperability

[How to: Use COM Interop to Check Spelling Using Word \(C# Programming Guide\)](#)

This example illustrates how to use Word's spelling checker from a C# application.

[How to: Use COM Interop to Create an Excel Spreadsheet \(C# Programming Guide\)](#)

This example illustrates how to open an existing Excel spreadsheet in C# using .NET Framework COM interop capability.

[How to: Use Managed Code as an Automation Add-In for Excel \(C# Programming Guide\)](#)

This example illustrates how to create a C# add-in for calculating income tax rate in a cell in an Excel worksheet.

[How to: Use Platform Invoke to Play a Wave File \(C# Programming Guide\)](#)

This example illustrates how to use platform invoke services to play a wave sound file on the Windows platform.

Unsafe code

[How to: Use Pointers to Copy an Array of Bytes \(C# Programming Guide\)](#)

Shows how to use pointers to copy bytes from one array to another, using pointers.

[How to: Use the Windows ReadFile Function \(C# Programming Guide\)](#)

Shows how to call the Windows `ReadFile` function, which requires the use of an unsafe context because the read buffer requires a pointer as a parameter.

Threading

[Using Threads and Threading](#)

Provides a list of topics that discuss the creation and management of managed threads and how to avoid unintended consequences.

[How to: Create and Terminate Threads \(C# Programming Guide\)](#)

Provides an example that demonstrates how to create and start a thread, and shows the interaction between two threads running simultaneously within the same process.

[How to: Synchronize a Producer and a Consumer Thread \(C# Programming Guide\)](#)

Provides an example that shows how synchronization can be accomplished using the C# lock keyword and the Pulse method of the Monitor object.

[How to: Use a Thread Pool \(C# Programming Guide\)](#)

Explains an example that shows how to use a thread pool.

Strings

[How to: Search Strings Using Regular Expressions \(C# Programming Guide\)](#)

Explains how to use the RegEx class can be used to search strings. These searches can range in complexity from very simple to making full use of regular expressions.

[How to: Join Multiple Strings \(C# Programming Guide\)](#)

Contains a code example that demonstrates how to join multiple strings.

[How to: Search Strings Using String Methods \(C# Programming Guide\)](#)

Contains a code example that demonstrates how to use String methods to search for a string.

[How to: Parse Strings Using the Split Method \(C# Programming Guide\)](#)

Contains a code example that demonstrates how a string can be parsed using the System.String.Split method.

[How to: Modify String Contents \(C# Programming Guide\)](#)

Contains a code example that extracts the contents of a string into an array and then modifies some elements of the array.

Attributes

[How to: Create a C/C++ Union Using Attributes \(C# Programming Guide\)](#)

Contains an example that uses the attribute Serializable to apply a specific characteristic to a class.

Working with DLLs

[How to: Create and Use C# DLLs \(C# Programming Guide\)](#)

Demonstrates building and using a DLL, using an example scenario.

Assemblies

[How to: Determine If a File Is an Assembly \(C# Programming Guide\)](#)

Contains an example that tests a DLL to see if it is an assembly.

[How to: Load and Unload Assemblies \(C# Programming Guide\)](#)

Explains how it is possible to load specific assemblies into the current application domain at runtime.

[How to: Share an Assembly with Other Applications \(C# Programming Guide\)](#)

Explains how to share an assembly with other applications.

Application Domains

[Executing Code in Another Application Domain \(C# Programming Guide\)](#)

Shows how to execute an assembly that has been loaded into another application domain.

[How to: Create and Use an Application Domain \(C# Programming Guide\)](#)

Shows how operator overloading can be used to implement a three-valued logical type.

Samples

[Visual C# Samples](#)

Contains links to open or copy the sample's files that range from [Hello World Sample](#) to [Generics Sample \(C#\)](#).

See Also

Concepts

[How Do I in C#](#)

.NET Framework (How Do I in C#)

This page links to Help on widely used .NET Framework tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

General

[Introduction to the C# Language and the .NET Framework](#)

Describes the relationship between the C# language and the .NET Framework class libraries and run-time execution engine.

[Overview of the .NET Framework](#)

Provides conceptual overviews of the key features of the .NET Framework, including the common language runtime, the .NET Framework class library, and cross-language interoperability.

[Quick Technology Finder](#)

Provides a quick reference to the major technology areas of the .NET Framework.

File I/O

[How to: Create a Directory Listing](#)

Create a new directory.

[How to: Read and Write to a Newly Created Data File](#)

Read and write to a newly created data file.

[How to: Open and Append to a Log File](#)

Open and append to a log file.

[How to: Write Text to a File](#)

Write text to a file.

[How to: Read Text from a File](#)

Read text from a file.

[How to: Read Characters from a String](#)

Read characters from a string.

[How to: Write Characters to a String](#)

Write characters to a string.

[How to: Add or Remove Access Control List Entries](#)

Add or remove Access Control List (ACL) entries for enhanced security.

Strings

[Creating New Strings](#)

How to create a new string.

[Trimming and Removing Characters](#)

How to remove characters from the beginning or end of a string.

[Padding Strings](#)

How to add tabs or spaces to the beginning or end of strings.

[Comparing Strings](#)

How to compare two strings for equality.

[Changing Case](#)

How to change upper-case letters to lower-case, and the reverse.

[Using the StringBuilder Class](#)

Efficient string manipulation techniques.

[How to: Perform String Manipulations by Using Basic String Operations](#)

How to split strings, append one string to another, and more.

[How to: Convert Data Types Using System.Convert](#)

Contains an example that uses the Convert class to transform a string value into a Boolean value.

[How to: Strip Invalid Characters from a String](#)

Contains an example that uses the static Regex.Replace method to strip invalid characters from a string.

[How to: Verify That Strings are in Valid E-Mail Format](#)

Contains an example that uses the static Regex.IsMatch method to verify that a string is in valid e-mail format.

Collections

[Collections and Data Structures](#)

Overview of .NET Framework collection classes.

[Selecting a Collection Class](#)

How to choose which type of collection to use.

[When to Use Generic Collections](#)

Explains the advantages of generic over non-generic collection classes.

[System.Collections.Generic](#)

Portal page to the generic collection classes.

[List](#)

Provides example code showing how to add and remove items from a **List<T>** collection.

[SortedDictionary](#)

Provides example code showing how to add and remove key/value pairs from a **SortedDictionary<K,V>** collection.

Exceptions

[How to: Use Specific Exceptions in a Catch Block](#)

Contains an example that uses a **try/catch** block to catch an `InvalidCastException`.

[How to: Use the Try/Catch Block to Catch Exceptions](#)

Contains an example that uses a **try/catch** block to catch a possible exception.

[How to: Create User-Defined Exceptions](#)

Contains an example where a new exception class, **EmployeeListNotFoundException**, is derived from **Exception**.

[How to: Use Finally Blocks](#)

Contains an example that uses a **try/catch** block to catch an **ArgumentOutOfRangeException** exception.

[How to: Explicitly Throw Exceptions](#)

Contains an example that uses a **try/catch** block to catch a possible **FileNotFoundException** exception.

Events

[How to: Consume Events in a Windows Forms Application](#)

Contains examples to show how to handle a button click event on a Windows Form.

[How to: Connect Event Handler Methods to Events](#)

Contains examples to show how to add an event handler method for an event.

[How to: Raise and Consume Events](#)

Contains an example that uses concepts described in detail in Events and Delegates and Raising an Event.

[How to: Handle Multiple Events Using Event Properties](#)

Contains an example to show how to handle multiple events using event properties.

[How to: Implement Events in Your Class](#)

Contains procedures that describe how to implement an event in a class.

Debugging

See [Debugging \(How Do I in C#\)](#).

Deployment

See [Security \(How Do I in C#\)](#).

Serviced Components

[How to: Create a Compensating Resource Manager \(CRM\)](#)

Includes code examples to show you how to create a Compensating Resource Manager

[How to: Create a Serviced Component](#)

Contains a procedure that describes how to create a new serviced component.

[How to: Apply the Description Attribute to an Assembly](#)

Shows how to apply the [DescriptionAttribute](#) attribute to set the description of an assembly.

[How To: Use the SetAbort and SetComplete Methods](#)

Shows how to use the static **SetComplete** and **SetAbort** methods of the [ContextUtil](#) class.

[How to: Apply the ApplicationID Attribute to an Assembly](#)

Shows how to apply the **ApplicationID** attribute to an assembly.

[How to: Create a Pooled Object and Set Its Size and Time-out Limits](#)

Shows how to create a pooled object and set its size and time-out limits.

[How to: Create a Web Service Method that Uses Automatic Transactions](#)

Describes how to create a Web service method that uses automatic transactions.

[How to: Set the SoapRoot Property for an Application](#)

Shows how to set the **SoapVRoot** property to "MyVRoot".

[How to: Set the Transaction Time-out](#)

Shows how to set the transaction time-out to 10 seconds.

[How to: Set the Application Name by Using the ApplicationName Attribute](#)

Shows how to supply the application name by using the assembly-level **ApplicationName** attribute.

[How to: Use the BYOT \(Bring Your Own Transaction\) Feature of COM+](#)

Contains procedures that show how a class derived from the **ServicedComponent** class can use the BYOT feature of COM+ to access a Distributed Transaction Coordinator (DTC).

[How to: Create a Private Component](#)

Shows how to use the **PrivateComponentAttribute** attribute on a class.

[How to: Set the Activation Type of an Application](#)

Shows how to set the activation type to "server."

[How to: Enable Synchronization on Instances of a Class](#)

Shows how to enable synchronization on instances of the **TestSync** class.

[How to: Use Automatic Transactions in a .NET Framework Class](#)

Describes how to prepare a class to participate in an automatic transaction.

[How to: Enable JIT Activation](#)

Shows how to enable JIT activation and deactivation on and off a class.

[How To: Set the AutoComplete Attribute on a Transaction-Aware Class](#)

Shows the placement of the **AutoComplete** attribute on a transaction-aware class.

[How to: Implement a Queued Component that Displays a Message Asynchronously](#)

Demonstrates how to implement a queued component that displays a message asynchronously.

[How to: Implement Loosely Coupled Events](#)

Contains procedures that show how to implement an event class and event sink that implement a common event interface, plus a publisher to fire an event.

[How to: Configure Object Construction](#)

Contains a procedure and an example that describe how to configure object construction and set the default initialization string of the **TestObjectConstruct** class to the string "Initial Catalog=Northwind;Data Source=.\SQLServerInstance;Trusted_Connection=yes".

[Assemblies and Application Domains](#)

[How to: Obtain Type and Member Information from an Assembly](#)

Contains an example that obtains type and member information from an assembly.

[How to: Build a Single-File Assembly](#)

Contains a procedure that shows how to create single-file assemblies using command-line compilers.

[How to: Create an Application Domain](#)

Creates a new application domain, assigns it the name MyDomain, and then prints the name of the host domain and the newly created child application domain to the console.

[How to: Determine an Assembly's Fully Qualified Name](#)

Shows how to display the fully qualified name of an assembly containing a specified class to the console.

[How to: Configure an Application Domain](#)

Creates an instance of the **AppDomainSetup** class, uses this class to create a new application domain, writes the information to console, and then unloads the application domain.

[How to: View Assembly Contents](#)

Contains an example that starts with a basic "Hello, World" program and shows how to use Ildasm.exe to disassemble the Hello.exe assembly and view the assembly manifest.

[How to: Reference a Strong-Named Assembly](#)

Creates an assembly called myAssembly.dll that references a strong-named assembly called myLibAssembly.dll from a code module called myAssembly.cs.

[How to: Unload an Application Domain](#)

Creates a new application domain called MyDomain, prints some information to the console, and then unloads the application domain.

[How to: Remove an Assembly from the Global Assembly Cache](#)

Contains an example that removes an assembly named hello.dll from the global assembly cache.

[How to: Install an Assembly into the Global Assembly Cache](#)

Contains an example that installs an assembly with the file name hello.dll into the global assembly cache.

[How to: Build a Multifile Assembly](#)

Describes the procedure used to create a multifile assembly and provides a complete example that illustrates each of the steps in the procedure.

[How to: Load Assemblies into an Application Domain](#)

Contains an example that loads an assembly into the current application domain and then executes the assembly.

[How to: Sign an Assembly with a Strong Name](#)

Contains an example that signs the assembly MyAssembly.dll with a strong name using the key file sgKey.snk.

[How to: View the Contents of the Global Assembly Cache](#)

Shows how to use the Global Assembly Cache tool (Gacutil.exe) to view the contents of the global assembly cache.

[How to: Create a Public/Private Key Pair](#)

Shows how to sign an assembly with a strong name, and how to create a key pair using the Strong Name tool (Sn.exe).

Interoperation

[How to: Embed Type Libraries as Win32 Resources in .NET-Based Applications](#)

Shows how to embed a type library as a Win32 resource in a .NET Framework-based application.

[How to: Generate Primary Interop Assemblies Using Tlbimp.exe](#)

Provides examples that generate primary interop assemblies using Tlbimp.exe.

[How to: Create Primary Interop Assemblies Manually](#)

Provides an example to create primary interop assemblies manually.

[How to: Generate Interop Assemblies from Type Libraries](#)

Provides examples to generate an interop assembly from a type library.

[How to: Raise Events Handled by a COM Sink](#)

Provides an example that shows a managed server as the event source and a COM client as the event sink.

[How to: Customize Runtime Callable Wrappers](#)

Shows how to customize runtime callable wrappers by modifying the IDL source or modifying an imported assembly.

[How to: Configure .NET-Based Components for Registration-Free Activation](#)

Explains how to configure .NET Framework-based components for registration-free activation.

[How to: Implement Callback Functions](#)

Demonstrates how a managed application, using platform invoke, can print the handle value for each window on the local computer

[How to: Map HRESULTs and Exceptions](#)

Contains an example to create a new exception class called **NoAccessException** and map it to the HRESULT E_ACCESSDENIED.

[How to: Edit Interop Assemblies](#)

Demonstrates how to specify marshaling changes in Microsoft intermediate language (MSIL).

[How to: Add References to Type Libraries](#)

Explains the steps to add a reference to a type library.

[How to: Handle Events Raised by a COM Source](#)

Includes an example to demonstrate how to open an Internet Explorer window and wire events raised by the **InternetExplorer** object to event handlers implemented in managed code.

[How to: Create Wrappers Manually](#)

Shows an example of the **ISATest** interface and **SATest** class in IDL and the corresponding types in C# source code.

[How to: Register Primary Interop Assemblies](#)

Includes an example to register the CompanyA.UtilLib.dll primary interop assembly.

[How to: Wrap Multiple Versions of Type Libraries](#)

Explains how to wrap more than one version of a type library.

Security

See [Security \(How Do I in C#\)](#).

Serialization

[How to: Deserialize an Object](#)

Provides an example that deserializes an object into a file.

[How to: Use the XML Schema Definition Tool to Generate Classes and XML Schema Documents](#)

Provides procedures to show how to use the XML schema definition tool to generate classes and XML schema documents.

[How to: Specify an Alternate Element Name for an XML Stream](#)

Shows how you can generate more than one XML stream with the same set of classes.

[How to: Control Serialization of Derived Classes](#)

Provides an example to show how to control serialization of derived classes.

[How to: Serialize an Object as a SOAP-Encoded XML Stream](#)

Provides a procedure and an example to serialize an object as a SOAP-encoded XML stream.

[How To: Chunk Serialized Data](#)

Provides a procedure and an example to implement server-side chunking and client-side processing.

[How to: Serialize an Object](#)

Provides a procedure to serialize an object.

[How to: Qualify XML Element and XML Attribute Names](#)

Provides a procedure and an example to create qualified names in an XML document.

[How to: Override Encoded SOAP XML Serialization](#)

Provides a procedure and an example to override serialization of objects as SOAP messages.

Encoding and Localization

[How to: Parse Unicode Digits](#)

Provides an example that uses the **Decimal.Parse** method to parse strings of Unicode code values that specify digits in different scripts.

[How to: Create Custom Cultures](#)

Provides a procedure to define and create a custom culture.

Advanced Programming

[How to: Define and Execute Dynamic Methods](#)

Shows how to define and execute a simple dynamic method and a dynamic method bound to an instance of a class.

[How to: Examine and Instantiate Generic Types with Reflection](#)

Provides procedures to show how to discover and manipulate generic types.

[How to: Define a Generic Method with Reflection Emit](#)

Provides procedures to show how to define a generic method with reflection emit.

[How to: Use Full Signing to Give a Dynamic Assembly a Strong Name](#)

Demonstrates using full signing to give a dynamic assembly a strong name.

[How to: Load Assemblies into the Reflection-Only Context](#)

Provides a procedure and a code example to show how to load assemblies into the reflection-only context.

[How to: Define a Generic Type with Reflection Emit](#)

Shows how to create a simple generic type with two type parameters, how to apply class constraints, interface constraints, and special constraints to the type parameters, and how to create members that use the type parameters of the class as parameter types and return types.

[.NET Framework Walkthroughs](#)

[Walkthrough: Adding Smart Tags to a Windows Forms Component](#)

Demonstrates how to add smart tags using code from a simple example control, **ColorLabel**, which is derived from the standard **Windows Forms Label** control.

[Walkthrough: Altering the SOAP Message Using SOAP Extensions](#)

Shows how to build and run a SOAP extension.

[Walkthrough: Building a Basic XML Web Service Using ASP.NET](#)

Demonstrates building a basic XML Web service using ASP.NET.

[Walkthrough: Customizing ASP.NET Mobile Web Pages for Specific Devices](#)

Demonstrates customizing for specific devices.

[Walkthrough: Customizing the Generation of Service Descriptions and Proxy Classes](#)

Demonstrates customizing the generation of Service Descriptions and proxy classes.

[Walkthrough: Deploying a ClickOnce Application Manually](#)

Describes the steps required to create a full ClickOnce deployment using the command-line or graphical version of the Manifest Generation and Editing tool (Mage).

[Walkthrough: Downloading Assemblies on Demand with the ClickOnce Deployment API](#)

Demonstrates how to mark certain assemblies in your application as "optional," and how to download those using classes in the **System.Deployment.Application** namespace when the common language runtime (CLR) demands them.

[Walkthrough: Implementing a UI Type Editor](#)

Explains how to author your own UI type editor for a custom type and display the editing interface by using a **PropertyGrid**.

Additional Resources

[Visual Studio 2005 Developer Center](#)

Contains many articles and resources on developing applications using Visual Studio 2005. This site is updated regularly with new content.

[Visual C# Developer Center](#)

Contains many articles and resources on developing C# applications. This site is updated regularly with new content.

[Microsoft .NET Framework Developer Center](#)

Contains many articles and resources on developing and debugging .NET Framework applications. This site is updated regularly with new content.

See Also

Concepts

[How Do I in C#](#)

Windows Applications (How Do I in C#)

This page links to help on widely used Windows applications tasks. To view other categories of popular tasks covered in Help, [How Do I in C#](#).

General

[Overview of Windows-based Applications](#)

Provides an overview of Windows applications that you can create using Microsoft Visual Studio 2005.

[Choosing Between Windows Forms and Web Forms](#)

Discuss the features and characteristics of each technology to help you determine which is best suited to your application.

Working with Forms

Windows Forms Designer

[How to: Choose the Startup Form in a Windows Application](#)

Provides information on how to set the startup form in a Windows application.

[How to: Connect Multiple Events to a Single Event Handler in Windows Forms](#)

Explains how to connect multiple events to a single event handler in Windows Forms application by using the Events view of the Properties window in C#.

[How to: Create a Multipane User Interface with Windows Forms](#)

Explains how to create a multipane user interface that is similar to the one used in Microsoft Outlook, with a Folder list, a Messages pane, and a Preview pane.

[How to: Add Background Images to Windows Forms](#)

Explains how to place a background image in a control or on the form itself. You can accomplish this easily by using the Properties window.

[How to: Set ToolTips for Controls on a Windows Form at Design Time](#)

Explains how you can set a ToolTip string in code or in the Windows Forms Designer.

[How to: Add ActiveX Controls to Windows Forms](#)

Explains how you can put ActiveX controls on Windows Forms.

[How to: Create Access Keys for Windows Forms Controls](#)

Explains how to create an access key in the text of a menu, menu item, or the label of a control such as a button.

Working with Windows Forms at Run Time

[How to: Add to or Remove from a Collection of Controls at Run Time](#)

Provides common tasks in application development such as adding controls to and removing controls from any container control on your forms.

[How to: Enable Windows XP Visual Styles](#)

Shows how to enable visual styles in the client area of a Windows form.

[How to: Make a Startup Windows Form Invisible](#)

Shows how to make the main form of a Windows-based application invisible when the application starts.

[How to: Keep a Windows Form on Top](#)

Shows how to make a form the top-most form in a Windows Forms application at design time or programmatically.

[How to: Display Modal and Modeless Windows Forms](#)

Shows how to display a form as a modal or modeless dialog box.

Controls

TextBox Controls

[How to: Select Text in the Windows Forms TextBox Control](#)

Shows how to select text in a Windows Forms TextBox control.

Shows you how to select text programmatically in the Windows Forms TextBox control.

[How to: Put Quotation Marks in a String \(Windows Forms\)](#)

Shows you how to place quotation marks (" ") in a string of text.

[How to: Create a Read-Only Text Box \(Windows Forms\)](#)

Shows you how to transform an editable Windows Forms text box into a read-only control.

[How to: Create a Password Text Box with the Windows Forms TextBox Control](#)

Shows you how to create a password Text box with the Windows Forms TextBox Control.

[How to: Control the Insertion Point in a Windows Forms TextBox Control](#)

Shows you how to control the insertion point in a TextBox control.

[How to: Bind Data to the MaskedTextBox Control](#)

Shows you how to bind data to a MaskedTextBox control.

[Walkthrough: Working with the MaskedTextBox Control](#)

Illustrated performing the following task:

Initializing the MaskedTextBox control.

Alerting the user when a character does not conform to the mask.

Alerting the user when the value they're attempting to commit is not valid for the type.

RichTextBox Controls

[How to: Load Files into the Windows Forms RichTextBox Control](#)

Explains how to load files into a Windows Forms RichTextBox control that can display a plain-text, Unicode plain-text, or Rich-Text-Format (RTF) file.

[How to: Display Scroll Bars in the Windows Forms RichTextBox Control](#)

Provides seven possible values for the ScrollBars property of the RichTextBox control, which are described in the table below.

[How to: Set Font Attributes for the Windows Forms RichTextBox Control](#)

Describes how you can make the selected characters bold, underlined, or italic, using the SelectionFont property.

[How to: Set Indents, Hanging Indents, and Bulleted Paragraphs with the Windows Forms RichTextBox Control](#)

Describes how you can format selected paragraphs as bulleted lists by setting the SelectionBullet property. You can also use the SelectionIndent, SelectionRightIndent, and SelectionHangingIndent properties to set the indentation of paragraphs relative to the left and right edges of the control, and the left edge of other lines of text.

[How to: Enable Drag-and-Drop Operations with the Windows Forms RichTextBox Control](#)

Describes how to enable Drag-and-drop operations with the Windows Forms RichTextBox control by handling the DragEnter and DragDrop events.

[How to: Display Web-Style Links with the Windows Forms RichTextBox Control](#)

Describes how you can write code that opens a browser window showing the Web site specified in the link text when the link is clicked.

Button Controls

[How to: Respond to Windows Forms Button Clicks](#)

Explains the most basic use of a Windows Forms Button control that is executing some code when the button is clicked..

[How to: Designate a Windows Forms Button as the Accept Button Using the Designer](#)

Explains how you can designate a Button control to be the accept button, also known as the default button. Whenever the user presses the ENTER key, the default button is clicked regardless of which other control on the form has the focus.

[How to: Designate a Windows Forms Button as the Cancel Button Using the Designer](#)

Explains how you can designate a Button control to be the cancel button. A cancel button is clicked whenever the user presses the ESC key, regardless of which other control on the form has the focus. Such a button is usually programmed to

enable the user to quickly exit an operation without committing to any action.

CheckBox Controls

[How to: Set Options with Windows Forms CheckBox Controls](#)

Provides information on how a Windows Forms CheckBox control is used to give users True/False or Yes/No options. The control displays a check mark when it is selected.

[How to: Respond to Windows Forms CheckBox Clicks](#)

Explains how you can program your application to perform some action depending upon the state of the check box.

RadioButton Control

[How to: Group Windows Forms RadioButton Controls to Function as a Set](#)

Explains how you group radio buttons by drawing them inside a container such as a Panel control, a GroupBox control, or a form.

ListBox, ComboBox and CheckedListBox Controls

[How to: Bind a Windows Forms ComboBox or ListBox Control to Data](#)

Explains how you can bind the ComboBox and ListBox to data to perform tasks such as browsing data in a database, entering new data, or editing existing data.

[How to: Create a Lookup Table for a Windows Forms ComboBox, ListBox, or CheckedListBox Control](#)

Provides tables that show an example of how to store and display order-form data for food.

[How to: Add and Remove Items from a Windows Forms ComboBox, ListBox, or CheckedListBox Control](#)

Provides an example on how to add items to a Windows Forms combo box, list box, or checked list box. However, this topic demonstrates the simplest method and requires no data binding.

[How to: Access Specific Items in a Windows Forms ComboBox, ListBox, or CheckedListBox Control](#)

Demonstrates accessing specific items in a Windows Forms combo box, list box, or checked list box. It enables you to programmatically determine what is in a list, at any given position.

[How to: Sort the Contents of a Windows Forms ComboBox, ListBox, or CheckedListBox Control](#)

Demonstrates using data sources that support sorting: data views, data view managers, and sorted arrays.

CheckedListBox Control

[How to: Determine Checked Items in the Windows Forms CheckedListBox Control](#)

Demonstrates how to determine checked items in the Windows Forms CheckedListBox Control by either iterating through the collection stored in the CheckedItems property, or step through the list using the GetItemChecked method to determine which items are checked.

DataGridView Controls

[How to: Bind Data to the Windows Forms DataGridView Control Using the Designer](#)

Explains how you can use the designer to connect the DataGridView control to data sources of several different varieties, including databases, business objects, or Web services.

[How to: Validate Data in the Windows Forms DataGridView Control](#)

Demonstrates how to validate data entered by a user into a DataGridView control.

[How to: Handle Errors That Occur During Data Entry in the Windows Forms DataGridView Control](#)

Demonstrates how to use the DataGridView control to report data entry errors to the user.

[How to: Specify Default Values for New Rows in the Windows Forms DataGridView Control](#)

Demonstrates how to specify default values for new rows using the DefaultValuesNeeded event.

[How to: Create an Unbound Windows Forms DataGridView Control](#)

Demonstrates how to populate a DataGridView control programmatically without binding it to a data source.

[How to: Add an Unbound Column to a Data-Bound Windows Forms DataGridView Control](#)

Demonstrates how to create an unbound column of Details buttons to display a child table related to a particular row in a parent table when you implement a master/detail scenario.

[How to: Display Images in Cells of the Windows Forms DataGridView Control](#)

Demonstrates how to extract an icon from an embedded resource and convert it to a bitmap for display in every cell of an image column.

[How to: Host Controls in Windows Forms DataGridView Cells](#)

Shows how to create a calendar column. The cells of this column display dates in ordinary text box cells, but when the user edits a cell, a DateTimePicker control appears.

[Walkthrough: Validating Data in the Windows Forms DataGridView Control](#)

Demonstrates how you retrieve rows from the Customers table in the Northwind sample database and display them in a DataGridView control. When you edit a cell in the CompanyName column and attempt to leave it, the new company name string will be checked to make sure it is not empty; if the new value is an empty string, the DataGridView will prevent the cursor from leaving the cell until a non-empty string is entered..

[Walkthrough: Handling Errors that Occur During Data Entry in the Windows Forms DataGridView Control](#)

Demonstrates how you retrieve rows from the Customers table in the Northwind sample database and display them in a DataGridView control. When a duplicate CustomerID value is detected in a new row or an edited existing row, the DataError event will occur, which will be handled by showing a MessageBox that displays the exception.

[Walkthrough: Creating an Unbound Windows Forms DataGridView Control](#)

Shows how to populate a DataGridView control and manage the addition and deletion of rows in "unbound" mode.

DataGridView Layout and Formatting

[How to: Make Columns Read-Only in the Windows Forms DataGridView Control Using the Designer](#)

Demonstrates a procedure to make the columns that contain the data read-only.

[How to: Enable Column Reordering in the Windows Forms DataGridView Control Using the Designer](#)

Demonstrates enabling your users to reorder the columns. When you enable column reordering, users can move a column to a new position by dragging the column header with the mouse.

[How to: Change the Order of Columns in the Windows Forms DataGridView Control Using the Designer](#)

Explains how to change the order of columns in the Windows Forms DataGridView Control using the Designer.

[How to: Add and Remove Columns in the Windows Forms DataGridView Control Using the Designer](#)

Explains how to add or remove columns in the Windows Forms DataGridView control using the Designer.

Data Binding with Controls

[How to: Handle Errors and Exceptions that Occur with Databinding](#)

Demonstrates how to handle errors and exceptions that occur during a data-binding operation.

BindingSource Controls

[How to: Bind Windows Forms Controls with the BindingSource Component Using the Designer](#)

Demonstrates how to bind a control at design time.

[How to: Create a Lookup Table with the Windows Forms BindingSource Component](#)

Demonstrates how to use a ComboBox control to display the field with the foreign-key relationship from the parent to the child table.

[How to: Reflect Data Source Updates in a Windows Forms Control with the BindingSource](#)

Demonstrates using the ResetBindings method to notify a bound control about an update in the data source.

[How to: Sort and Filter ADO.NET Data with the Windows Forms BindingSource Component](#)

Demonstrates how to sort and filter data with the BindingSource.

[How to: Bind to a Web Service Using the Windows Forms BindingSource](#)

Demonstrates how to create and bind to a client-side proxy.

Binding Navigator

[How to: Navigate Data with the Windows Forms BindingNavigator Control](#)

Explains how to set up the BindingNavigator control.

[How to: Move Through a DataSet with the Windows Forms BindingNavigator Control](#)

Demonstrates how to use a BindingNavigator control to move through the results of a database query.

ListView

[How to: Add and Remove Items with the Windows Forms ListView Control](#)

Explains the process of adding an removing an item to a Windows Forms ListView control. Adding or removing list items can be done at any time.

[How to: Add Search Capabilities to a ListView Control](#)

Demonstrated creating professional-looking Windows Forms applications in a short of amount of time.

[How to: Select an Item in the Windows Forms ListView Control](#)

Demonstrates how to programmatically select an item in a Windows Forms ListView control.

[How to: Display Icons for the Windows Forms ListView Control](#)

Demonstrates how to display images in a list view.

[How to: Display Subitems in Columns with the Windows Forms ListView Control](#)

Demonstrates how to add subitems to a list item.

TreeView

[How to: Set Icons for the Windows Forms TreeView Control](#)

Demonstrates how to display images in a tree view.

[How to: Add and Remove Nodes with the Windows Forms TreeView Control](#)

Demonstrates how to add and remove nodes programmatically to and from a tree view..

[How to: Determine Which TreeView Node Was Clicked \(Windows Forms\)](#)

Demonstrates how to determine which TreeView node was clicked.

Container Controls

[How to: Split a Window Horizontally](#)

Explains how to make the splitter that divides the SplitContainer control horizontally.

[How to: Create a Multipane User Interface with Windows Forms](#)

Demonstrates how you create a multipane user interface that is similar to the one used in Microsoft Outlook, with a Folder list, a Messages pane, and a Preview pane.

[How to: Span Rows and Columns in a TableLayoutPanel Control](#)

Demonstrates how controls in a TableLayoutPanel control can span adjacent rows and columns.

[Walkthrough: Arranging Controls on Windows Forms Using a TableLayoutPanel](#)

Illustrates performing the following tasks:

- Creating a Windows Forms project.

- Arranging Controls in Rows and Columns.

- Setting Row and Column Properties.

- Spanning Rows and Columns with a Control.

- Automatic Handling of Overflows.

- Inserting Controls by Double-clicking Them in the Toolbox.

- Inserting a Control by Drawing Its Outline.

- Reassigning Existing Controls to a Different Parent.

[Walkthrough: Arranging Controls on Windows Forms Using a FlowLayoutPanel](#)

Illustrates performing the following tasks:

Creating a Windows Forms project.

Arranging Controls Horizontally and Vertically.

Changing Flow Direction.

Inserting Flow Breaks.

Arranging Controls Using Padding and Margins.

Inserting Controls by Double-clicking Them in the Toolbox.

Inserting a Control by Drawing Its Outline.

Inserting Controls Using the Caret.

Reassigning Existing Controls to a Different Parent.

Picture and Image Controls

[How to: Load a Picture Using the Designer \(Windows Forms\)](#)

Explains how you can load and display a picture on a form at design time by setting the Image property to a valid picture.

[How to: Set Pictures at Run Time \(Windows Forms\)](#)

Explains how you can programmatically set the image displayed by a Windows Forms PictureBox control.

[How to: Modify the Size or Placement of a Picture at Run Time \(Windows Forms\)](#)

Explains how you can set the SizeMode property of a Windows Forms PictureBox control on a form to different values

DateTimePicker

[How to: Set and Return Dates with the Windows Forms DateTimePicker Control](#)

Explains how you can set the Value property before the control is displayed to determine which date will be initially selected in the control.

[How to: Display a Date in a Custom Format with the Windows Forms DateTimePicker Control](#)

Explains how to display a custom format and set the CustomFormat property to a format string.

MonthCalendar

[How to: Select a Range of Dates in the Windows Forms MonthCalendar Control](#)

Demonstrates how you can set a range of dates or get a selection range set by the user with properties of the MonthCalendar control.

[How to: Display Specific Days in Bold with the Windows Forms MonthCalendar Control](#)

Demonstrates how you make a date appear in bold type or in the regular font.

[How to: Display More than One Month in the Windows Forms MonthCalendar Control](#)

Demonstrates how you display multiple months in a Windows Forms MonthCalendar control.

[How to: Change the Windows Forms MonthCalendar Control's Appearance](#)

Demonstrates how to change the month calendar's color scheme, to display the current date at the bottom of the control and to display week numbers.

Data Access (for Windows Forms)

[Walkthrough: Passing Data Between Forms in a Windows Application](#)

Provides step by step instructions for passing data from one form into a method on a second form.

[Walkthrough: Displaying Data on a Form in a Windows Application](#)

Creates a simple form that displays data from a single table in several individual controls.

[Walkthrough: Creating a Form to Search Data in a Windows Application](#)

Demonstrates creating a Windows Form to Search Data.

ToolStrip

[How to: Position a ToolStripItem on a ToolStrip](#)

Explains how you can move or add a ToolStripItem to the left or right side of a ToolStrip.

[How to: Disable ToolStripMenuItems Using the Designer](#)

Explains how you can disable a menu item at design time.

[How to: Move ToolStripMenuItems](#)

Explains how you can move the entire top-level menus and their menu items to a different place on the MenuStrip. You can also move individual menu items between top-level menus or change the position of menu items within a menu.

[How to: Change the Appearance of ToolStrip Text and Images in Windows Forms](#)

Explains how you can control whether text and images are displayed on a ToolStripItem and how they are aligned relative to each other and the ToolStrip.

Context Menus

[How to: Associate a Shortcut Menu with a Windows Forms NotifyIcon Component](#)

Demonstrates how to: Associate a shortcut menu with a Windows Forms NotifyIcon component.

[How to: Add and Remove Menu Items with the Windows Forms ContextMenu Component](#)

Explains how to add and remove shortcut menu items in Windows Forms.

Printing

[How to: Create Standard Windows Forms Print Jobs](#)

Shows you how to specify what to print and how to print it by writing code to handle the PrintPage event.

[How to: Complete Windows Forms Print Jobs](#)

Shows you how to complete a print job by handling the EndPrint event of the PrintDocument component.

[How to: Print a Multi-Page Text File in Windows Forms](#)

Shows you how to print text in a Windows Form by using methods for drawing objects (graphics or text) to a device, such as a screen or printer.

[How to: Choose the Printers Attached to a User's Computer in Windows Forms](#)

Shows you how to choose a printer and then print a file.

[How to: Capture User Input from a PrintDialog at Run Time](#)

Shows you how change print options at run time. This is done through the PrintDialog component and the PrinterSettings class.

User Controls and Custom Controls

[Adding Controls to Your User Control](#)

Demonstrates adding controls to your User Control.

[Adding Code to Your User Control](#)

Demonstrates adding code to your User Control.

Multiple Document Interfaces (MDI)

[How to: Create MDI Parent Forms](#)

Demonstrates how to create an MDI parent form at design time.

[How to: Create MDI Child Forms](#)

Demonstrates how to create MDI child forms that display a RichTextBox control, similar to most word-processing applications.

[How to: Arrange MDI Child Forms](#)

Demonstrates how to display the child forms as cascading, as horizontally or vertically tiled, or as child form icons arranged along the lower portion of the MDI form.

[How to: Determine the Active MDI Child](#)

Demonstrates how to determine the active MDI child and copy its text to the Clipboard.

[How to: Send Data to the Active MDI Child](#)

Demonstrates how to send data to the active MDI child window from the Clipboard.

Graphics

[How to: Draw an Outlined Shape](#)

Shows how to draw outlined ellipses and rectangles on a form.

[How to: Create a Linear Gradient](#)

Shows how to fill a line, an ellipse, and a rectangle with a horizontal linear gradient brush.

[How to: Create a Path Gradient](#)

Shows you how to customize the way you fill a shape with gradually changing colors.

[How to: Create Figures from Lines, Curves, and Shapes](#)

Explains how to create a path that has a single or multiple figures.

[How to: Create Graphics Objects for Drawing](#)

Explains how to create graphics objects for drawing.

[How to: Create Thumbnail Images](#)

Demonstrates how to construct an image object from a bitmap file.

[How to: Create Vertical Text](#)

Demonstrates how to use a StringFormat object to specify that text be drawn vertically rather than horizontally..

[How to: Align Drawn Text](#)

Demonstrates how to draw text in a rectangle. Each line of text is centered, and the entire block of text is centered in the rectangle.

[How to: Draw a Line on a Windows Form](#)

Demonstrates how to draw a line on a form.

[How to: Rotate, Reflect, and Skew Images](#)

Demonstrates how to rotate, reflect, and skew an image by specifying destination points for the upper-left, upper-right, and lower-left corners of the original image.

[How to: Draw Text on a Windows Form](#)

Shows how to use the DrawString method of the Graphics to draw text on a form.

[How to: Load and Display Bitmaps](#)

Shows how to load a bitmap from a file and display that bitmap on the screen.

[How to: Load and Display Metafiles](#)

Shows how to use the methods of the Metafile class for recording, displaying, and examining vector images..

Localizing and Globalizing Windows Forms

[Walkthrough: Localizing Windows Forms](#)

Demonstrates processes of localizing a Windows Application project.

[How to: Support Localization on Windows Forms Using AutoSize and the TableLayoutPanel Control](#)

Demonstrates enabling a layout that adapts to varying string sizes.

[How to: Set the Culture and UI Culture for Windows Forms Globalization](#)

Demonstrates how to set formatting options appropriate for a specific culture.

[How to: Display Right-to-Left Text in Windows Forms for Globalization](#)

Demonstrates how to display right-to-left text.

Additional Resources

[Visual Studio 2005 Developer Center](#)

Contains many articles and resources on developing applications using Visual Studio 2005. This site is updated regularly with new content.

[Visual C# Developer Center](#)

Contains many articles and resources on developing C# applications. This site is updated regularly with new content.

[Microsoft .NET Framework Developer Center](#)

Contains many articles and resources on developing and debugging .NET Framework applications. This site is updated regularly with new content.

See Also

Concepts

[How Do I in C#](#)

Web Pages and Web Services (How Do I in C#)

This page links to help on widely used Web applications tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

Web Pages

[What's New in Web Development for Visual Studio](#)

Introduction to Visual Web Developer, the development tool for creating ASP.NET web pages.

[Introduction to ASP.NET Web Pages](#)

Provides an overview of the fundamental characteristics of how ASP.NET Web pages work in Web applications.

Web Services

[XML Documents and Data](#)

Links to articles on how to read and write XML documents and data.

[XML Web Services in Managed Code](#)

Links to articles on how to create and deploy XML Web services and how to access XML Web services in managed code.

[Getting Started with XML Web Services in Visual Basic and Visual C#](#)

Describes how Visual Studio and XML Web services provide a simple, flexible, standards-based model that allows developers to assemble applications regardless of the platform, programming language, or object model.

[How to: Access an XML Web Service in Managed Code](#)

Describes how to access Web services.

[How to: Access an XML Web Service Asynchronously in Managed Code](#)

Describes the asynchronous approach to accessing Web services.

[How to: Access XML Web Services from a Browser](#)

Describes how to access XML Web services from a browser.

[How to: Explore XML Web Service Content](#)

Describes how to explore XML Web service content.

[How to: Create ASP.NET Web Service Projects](#)

Describes how to create ASP.NET Web service projects.

[How to: Use the WebService Attribute](#)

Describes how to use the WebService attribute to specify the namespace and description text for the XML Web service

[How to: Inherit from the WebService Class](#)

Describes how to inherit from the WebService Class.

[How to: Create an XML Web Service Method](#)

Describes how to create an XML Web Service method.

[How to: Use the WebMethod Attribute](#)

Describes how to use the WebMethod Attribute to indicate that you want the method exposed as part of the XML Web service.

[How to: Debug XML Web Services in Managed Code](#)

Describes how to debug XML Web services in managed code.

[How to: Deploy XML Web Services in Managed Code](#)

Describes how to deploy XML Web services in managed code.

[How to: Generate an XML Web Service Proxy](#)

Describes how to generate an XML Web service proxy.

[How to: Create a Console Application Client](#)

Describes how to create a console application Web service client.

[How to: Handle Exceptions Thrown by a Web Service Method](#)

Describes how to handle exceptions thrown by a Web service method.

[How to: Implement an Asynchronous Web Service Client Using the Callback Technique](#)

Describes how to implement an asynchronous Web service client using the callback technique.

[How to: Implement an Asynchronous Web Service Client Using the Wait Technique](#)

Describes how to implement an asynchronous Web service client using the wait technique.

[Walkthrough: Redirecting an Application to Target a Different XML Web Service at Installation](#)

Demonstrates how to create a Web application that can be redirected to target a different XML Web service by using the URL Behavior property, an Installer class, and a Web Setup project.

[Securing XML Web Services Created Using ASP.NET](#)

Summarizes the authentication and authorization options available to Web services built using ASP.NET.

Additional Resources

These sites require an Internet connection.

[MSDN Web Services and Other Distributed Technologies Developer Center](#)

Articles, code examples and many other resources for developing and connecting to Web services.

[MSDN XML Developer Center](#)

Articles, code examples and many other resources for working with XML data.

[Microsoft ASP.NET Developer Center](#)

Articles, code examples and many other resources for creating ASP.NET web pages.

See Also

Concepts

[How Do I in C#](#)

Debugging (How Do I in C#)

This page links to help on widely used debugging tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

Using the Visual Studio Debugger

[Building in Visual Studio](#)

Discusses tools for continuously testing and debugging applications as you build them.

[Debugging in Visual Studio](#)

Discusses the fundamentals of using the Visual Studio debugger.

[Debugger Roadmap](#)

Links to articles on basic tasks in debugging and features of the debugger.

.NET Framework Trace Functionality

[How to: Add Trace Statements to Application Code](#)

Explains how to use the methods: Write, WriteIf, WriteLine, WriteLineIf, Assert, and Fail for tracing in your application.

[How to: Create and Initialize Trace Listeners](#)

Explains how to create and initialize trace listeners.

[How to: Use TraceSource and Filters with Trace Listeners](#)

Describes the use of TraceSource coupled with an application configuration file.

[How to: Create and Initialize Trace Switches](#)

Explains how to create and initialize a trace switch.

[How to: Compile Conditionally with Trace and Debug](#)

Explains how to specify the compiler settings for your application in several ways.

[How To: Create and Initialize Trace Sources](#)

Explains how to use configuration files to facilitate the reconfiguration of the traces produced by trace sources at run time.

[Enhancing Debugging with the Debugger Display Attributes](#)

Explains how to enhance debugging with the debugger display attributes.

[How to: Trace Code in an Application](#)

Explains how to use the Trace class that allows you to instrument your application.

[How to: Configure Trace Switches](#)

Explains how to use the .config file to configure switches.

Debugging Web Services

[Walkthrough: Debugging an XML Web Service](#)

Provides steps for debugging Web services.

Debugging Windows Forms

[Walkthrough: Debugging a Windows Form](#)

Describes debugging Windows forms applications.

Debugging SQL Applications

[Walkthrough: Debug a SQL CLR User-Defined Table-Valued Function](#)

Shows how to debug a SQL/CLR User Defined Table-Valued Function (UDF).

[Walkthrough: Debugging a SQL CLR Trigger](#)

Shows how to debug a SQL CLR trigger. It uses the Contact table in the **AdventureWorks** sample database, which is one of

the databases installed with SQL Server 2005. The sample creates a new insert CLR trigger on the Contact table, and then steps into it.

[Walkthrough: Debugging a SQL CLR User-Defined Type](#)

Shows how to debug a SQL/CLR user-defined type. It creates a new SQL/CLR type in the **Adventureworks** sample database. The type is then used in a table definition, an INSERT statement, and then a SELECT statement.

[Walkthrough: Debugging a SQL CLR User-Defined Scalar Function](#)

Shows how to debug a SQL CLR User Defined Function (UDF). It creates a new SQL CLR User-Defined Function in the **Adventureworks** sample database.

[Walkthrough: Debugging a SQL CLR User-Defined Aggregate,](#)

Shows how to debug a CLR SQL user-defined aggregate. It creates a new CLR SQL aggregate function named Concatenate in the **Adventureworks** sample database. When this function is invoked in a SQL statement, it will concatenate together all the values for the column specified as its input parameter.

[T-SQL Database Debugging](#)

Describes the necessary setup procedures, and provides a sample that illustrates how to debug a multi-tiered application.

[Walkthrough: Debugging a T-SQL Trigger](#)

Discusses an example that uses the **Adventureworks** database, which has a Sales.Currency table with an UPDATE trigger. The sample includes a stored procedure that updates a row in the table, thus causing the trigger to fire. Set breakpoints in the trigger, and by executing the stored procedure with different parameters, you can follow different execution paths in the trigger.

[Walkthrough: Debugging a T-SQL User-Defined Function ,](#)

Discusses an example that uses an existing User Defined Function (UDF) in the **Adventureworks** database, named ufnGetStock that returns a count of items in stock for a given ProductID.

[Walkthrough: Debug a T-SQL Stored Procedure](#)

Shows how to create and debug a T-SQL stored procedure by Direct Database Debugging, i.e. stepping into it via Server Explorer. It also illustrates different debugging techniques such as setting breakpoints, viewing data items, etc.

[Additional Resources](#)

This site requires an Internet connection.

[Visual Studio 2005 Developer Center](#)

Contains many articles and resources on developing and debugging applications. This site is updated regularly with new content.

See Also

Concepts

[How Do I in C#](#)

Data Access (How Do I in C#)

This page links to help on widely used data access tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

General

[How to: Install Sample Databases](#)

Provides steps to install a sample database such as Northwind sample database, SQL Server Express (SSE), MSDE, or an Access version of Northwind.

[Walkthrough: Creating a Simple Data Application](#)

Provides a step-by-step procedure to create a data application.

Connecting to Data in Visual Studio

[Connecting to Data in Visual Studio Overview](#)

Provides information on connecting your application to data from many different sources, such as databases, web services, and objects.

[Walkthrough: Connecting to Data in a Database](#)

Provides a procedure to connect your application to data in Visual Studio by using the Data Source Configuration Wizard.

[Walkthrough: Connecting to Data in a Web Service](#)

Provides a procedure to connect your application to data Web service by using the Data Source Configuration Wizard.

[Walkthrough: Connecting to Data in an Access Database](#)

Provides a procedure to connect your application to data in an Access database by using the Data Source Configuration Wizard.

Creating and Designing Typed Datasets

[How to: Create a Typed Dataset](#)

Explains how to create a typed DataSet using the Data Source Configuration Wizard or the Dataset Designer.

[Walkthrough: Creating a Dataset with the Dataset Designer](#)

Provides a procedure to create a dataset using the Dataset Designer.

[Walkthrough: Creating a DataTable in the Dataset Designer](#)

Provides a procedure to create a DataTable using the Dataset Designer.

[Walkthrough: Creating a Relationship between Data Tables](#)

Explains how to create two data tables without TableAdapters using the Dataset Designer and creating a relationship between them.

TableAdapters

[TableAdapter Overview](#)

Provides an overview of TableAdapters which provides communication between your application and a database.

[Walkthrough: Creating a TableAdapter with Multiple Queries](#)

Provides a procedure to create a TableAdapter in a dataset using the Data Source Configuration Wizard. The walkthrough will take you through the process of creating a second query in the TableAdapter using the TableAdapter Query Configuration Wizard within the Dataset Designer..

Filling Datasets and Executing Queries

[Filling Datasets and Querying Data Overview](#)

Explains how to execute SQL statements or stored procedures against a data source using TableAdapters or command objects.

[Walkthrough: Filling a Dataset with Data](#)

Demonstrates how to create a dataset with one data table and fills it with data from the Customers table in the Northwind

sample database.

[Walkthrough: Reading XML Data into a Dataset](#)

Demonstrates how to create a Windows application that will load XML data into a dataset.

Displaying Data on Windows Forms

[Displaying Data Overview](#)

Provides a summary of the tasks, objects, and dialog boxes involved in creating data-bound Windows applications.

[Walkthrough: Displaying Data on a Form in a Windows Application](#)

Provides a procedure to create a simple form that displays data from a single table in several individual controls.

[Walkthrough: Displaying Related Data on a Form in a Windows Application](#)

Provides a procedure to work with data that comes from more than one table, and often, data from related tables.

[Walkthrough: Creating a Form to Search Data in a Windows Application](#)

Shows how to create a query that returns customers in a specific city, and modify the user interface so users can enter a city's name and press a button to execute the query.

[Walkthrough: Creating a Lookup Table](#)

Provides a procedure to display information from one table based on the value of a foreign-key field in another table.

Databinding

[Walkthrough: Creating a User Control that Supports Simple Data Binding](#)

Shows how to create a control that implements the `DefaultBindingPropertyAttribute`. This control can contain one property that can be bound to data; similar to a `TextBox`, or `CheckBox`.

[Walkthrough: Creating a User Control that Supports Complex Data Binding](#)

Shows how to create a control that implements the `ComplexBindingPropertiesAttribute`. This control contains a `DataSource` and `DataMember` property that can be bound to data; similar to a `DataGridView`, or `ListBox`.

[Walkthrough: Creating a User Control that Supports Lookup Databinding](#)

Shows how to create a control that implements the `LookupBindingPropertiesAttribute`. This control contains three properties that can be bound to data; similar to a `ComboBox`.

[Object Binding in Visual Studio](#)

Explains design-time tools for working with custom objects (as opposed to datasets and web services) as the data source in your application.

Editing Data in Datasets (DataTables)

[Editing Data in Datasets Overview](#)

Provides a table that contains links to the common tasks associated with editing and querying data in a dataset.

Validating Data

[Data Validation Overview](#)

Provides an overview on validating data, the process of confirming that the values being entered into data objects conform to the constraints within a dataset's schema, as well as the rules established for your application.

[Walkthrough: Adding Validation to a Dataset](#)

Explains how to use the `ColumnChanging` event to verify that an acceptable value is being entered into the record.

Saving Data

[Saving Data Overview](#)

Explains how the process of writing information to the original data source is separate from the process of modifying the data in the dataset.

[Concurrency Control in ADO.NET](#)

Explains common methods of concurrency control, as well as specific ADO.NET features for handling concurrency errors.

[Walkthrough: Saving Data with the TableAdapter DBDirect Methods](#)

Provides detailed instructions for executing SQL statements directly against a database using the DbDirect methods of a TableAdapter.

[Walkthrough: Handling a Concurrency Exception](#)

Contains a procedure to create a Windows application that illustrates catching a DBConcurrencyException, locating the row that caused the error, and one strategy for handling it.

Data Resources

[Data User Interface Elements](#)

Contains information about all the dialog boxes and wizards that you use when designing data access in your applications.

[ADO.NET Data Adapters](#)

Provides information about ADO.NET data-adapter objects and how to work with them in Visual Studio.

Creating SQL Server 2005 Objects in Managed Code

[SQL Server Projects](#)

Explains how to use .NET languages in addition to the Transact-SQL programming language to create database objects such as stored procedures and triggers, and to retrieve and update data for Microsoft SQL Server 2005 databases.

[Walkthrough: Creating a Stored Procedure in Managed Code](#)

Provides step by step instructions for the following:

Creating a stored procedure in managed code.

Deploying the stored procedure to a SQL Server 2005 database.

Create a script to test the stored procedure on the database.

Query data in the database to confirm the stored procedure executed properly.

Additional Resources

The following sites require an Internet connection.

[Visual Studio 2005 Developer Center](#)

Contains many articles and resources on developing applications using Visual Studio 2005. This site is updated regularly with new content.

[Visual C# Developer Center](#)

Contains many articles and resources on developing C# applications. This site is updated regularly with new content.

[Microsoft .NET Framework Developer Center](#)

Contains many articles and resources on developing and debugging .NET Framework applications. This site is updated regularly with new content.

[Data Access and Storage Developer Center](#)

Contains many articles and resources about using Microsoft data access technologies in your applications

[SQL Server Developer Center](#)

Contains many articles and resources about using SQL Server.

See Also

Concepts

[How Do I in C#](#)

Designing Classes (How Do I in C#)

This page links to help on widely used C# Class Designer tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

Class Designer

[How to: Create Types on Class Diagrams](#)

Describes how to create new types, such as classes, enumerations, interfaces, structs, and delegates.

[Creating and Configuring Type Members](#)

Describes how to work with type members.

[How to: Inherit from a Generic Type](#)

Explains how to establish a relationship in which a class inherits from a generic.

[How to: Define Inheritance Between Types](#)

Explains how to use the Class Designer to define an inheritance relationship between two types currently displayed in a class diagram.

[How to: Define Associations Between Types](#)

Explains how to define an association. Association lines in the Class Designer show how classes in a diagram are related.

[How to: Delete Type Shapes and Associated Code from Class Diagrams](#)

Describes how to remove a shape, or both the shape and code, from the class diagram.

[How to: Apply Custom Attributes to Types or Type Members](#)

Explains how to apply a custom attribute to a type or to a member of a type.

Working with Classes and Other Types

[How to: View Inheritance Between Types](#)

Demonstrates how to display the base type of a selected type, assuming that an inheritance relationship exists between the type and its base type.

[How to: View Derived Types](#)

Displays the types derived from a selected type. It assumes that an inheritance relationship exists between the type and its base class or interface.

[How to: Remove Type Shapes from Class Diagrams](#)

Contains a procedure that demonstrates how to remove a shape from a diagram.

[How to: View Compartments in Type Shapes](#)

Contains a procedure that demonstrates how to show or hide a compartment.

[How to: View Type Details](#)

Contains a procedure that demonstrates how to display details of a type.

[How to: Change Between Member Notation and Association Notation](#)

Contains a procedure that demonstrates how to change between member notation and association.

[How to: View Type Members](#)

Contains a procedure that demonstrates how to show or hide a member within a type.

[How to: Add Class Diagrams to Projects](#)

Contains a procedure that demonstrates how to add a class diagram to a project.

[How to: View Existing Types](#)

Contains a procedure that demonstrates how to visualize existing types on the design surface.

[How to: Add Class Diagrams to Projects](#)

Contains a procedure that demonstrates how to add a class diagram to a project.

[Understanding Code You Did Not Write](#)

Explains how to use the Visual Studio Class Designer as a tool to help you understand classes and types written by others. The tool displays a graphical representation of the code. You can customize this view to suit your preferences.

[How to: Group Type Members](#)

Contains a procedure that demonstrates how to group members by kind, access modifier, or how to sort them alphabetically.

[How to: Add Comments to Class Diagrams](#)

Contains a procedure that demonstrates how to use comment shapes to annotate class diagrams.

[Customizing Class Diagrams](#)

Contains a procedure that demonstrates how to change the way class diagrams display information about your project.

[How to: Copy Class Diagram Elements to a Microsoft Office Document](#)

Contains a procedure that demonstrates how to copy one, several, or all the shapes from a class diagram into other documents.

[How to: Print Class Diagrams](#)

Contains a procedure that demonstrates how to print a class diagram using the print feature of Visual Studio.

[How to: Override Type Members](#)

Contains a procedure that demonstrates how to use Class Designer to cause a member in a child class to override (provide a new implementation for) a member inherited from a base class.

[How to: Rename Types and Type Members](#)

Contains a procedure that demonstrates how to rename a type or a member of a type using the Class Designer, the Class Details window, or the Properties window.

[How to: Move a Type Member from one Type to Another](#)

Contains a procedure that demonstrates how to move a type member from one type to another type, if both are visible in the current class diagram.

[How to: Implement an Interface](#)

Explains how to use the Class Designer to create, implement, and delete interfaces.

[How to: Implement an Abstract Class](#)

Contains a procedure that demonstrates how to use the Class Designer to implement an abstract class.

[How to: Extract to Interface \(C# Only\)](#)

Contains a procedure that demonstrates how to extract one or more public members from a type into a new interface.

[How to: Reorder Parameters \(C# Only\)](#)

Contains a procedure that demonstrates how to reorder the parameters of methods in types shown in the Class Designer.

[Create and Modify Type Members](#)

[How to: Open the Class Details Window](#)

Describes how Class Details is a window you can use to configure the members of a type.

[Class Details Window Elements](#)

Describe aspects of the rows displayed by the Class Details window.

[How to: Create a Member](#)

Explains how to create a member using any of the following tools: Class Designer, Class Details window toolbar, or Class Details window.

[How to: Add a Parameter to a Method](#)

Explains how to add a parameter to a method using the Class Details window.

[How to: Modify Type Members](#)

Describes how to modify the members of a type you have created in Class Designer by using using the Class Details window.

[Class Details Window Usage Notes](#)

Gives you tips on using the Class Details window.

[Display of Read-Only Information](#)

Explains how the Class Designer and the Class Details window can display the types (and members of types) for a project, or for a project or assembly referenced from a project.

[Class Library Design Guide](#)

[How to: Implement a Designer for a Control](#)

Describes how to implement a designer (HelpLabelDesigner) for the HelpLabel extender provider control.

[How to: Create and Configure Components in Design Mode](#)

Demonstrates how to use designer services to create and initialize components in your custom designer.

[How to: Access Design-Time Support in Windows Forms](#)

Provides steps to access the design-time support provided by the .NET Framework.

[How to: Implement a HelpLabel Extender Provider](#)

Demonstrates how to build an extender provider by creating the HelpLabel control.

[How to: Access Design-Time Services](#)

Shows you how to gain access to the rich set of .NET Framework services so that you can integrate your components and controls into the design environment.

[How to: Serialize Collections of Standard Types with the DesignerSerializationVisibilityAttribute](#)

Demonstrates how to use the [DesignerSerializationVisibilityAttribute](#) class to control how a collection is serialized at design time.

[How to: Perform Custom Initialization for Controls in Design Mode](#)

Demonstrates how to initialize a control when it is created by the design environment.

[How to: Implement a Type Converter](#)

Demonstrates how to use a type converter to convert values between data types, and to assist property configuration at design time by providing text-to-value conversion or a drop-down list of values to select from.

[How to: Implement a UI Type Editor](#)

Demonstrates how to implement a custom UI type editor for Windows Forms.

[How to: Extend the Appearance and Behavior of Controls in Design Mode](#)

Demonstrates how to create a custom designer that extends the user interface (UI) for designing a custom control.

[How to: Create a Windows Forms Control That Takes Advantage of Design-Time Features](#)

Illustrates how to create a custom control and an associated custom designer. When this library is built, you can build custom MarqueeControl implementations that run on a form.

[How to: Attach Smart Tags to a Windows Forms Component](#)

Shows how to add smart tag support to components and custom controls.

[How to: Adjust the Attributes, Events, and Properties of a Component in Design Mode](#)

Demonstrates how to create a custom designer that adjusts a component's attributes, events, and properties.

[Additional Resources](#)

The following sites require an Internet connection.

[Visual Studio 2005 Developer Center](#)

Contains many articles and resources on developing applications using Visual Studio 2005. This site is updated regularly with new content.

[Visual C# Developer Center](#)

Contains many articles and resources on developing C# applications. This site is updated regularly with new content.

[Microsoft .NET Framework Developer Center](#)

Contains many articles and resources on developing and debugging .NET Framework applications. This site is updated regularly with new content.

[Microsoft Patterns and Practices Developer Center](#)

Scenario-specific recommendations illustrating how to design, develop, deploy, and operate architecturally sound applications for the Microsoft .NET platform.

See Also

Concepts

[How Do I in C#](#)

Security (How Do I in C#)

This page links to help on widely used security and deployment tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

General

[Security In Visual Studio](#)

Provides an understanding of secure coding techniques.

[Code Access Security Basics](#)

Provides code access security concepts in order to write effective applications targeting the common language runtime.

[Microsoft Security Developer Center](#)

Provides up-to-date security issues that help you develop secure code..

[New Technologies Help You Make Your Web Services More Secure](#)

Explains the importance of good security and its effect on Web Services.

[Determining When to Use Windows Installer Versus XCOPY](#)

Examines and compares two approaches for deploying Microsoft .NET applications: the DOS XCOPY command, and the Microsoft Windows Installer technology.

[Security Policy Best Practices](#)

Explains basic administration concepts and describes some of the best practices to use when administering code access security policy.

Code Access and Permission Sets

[How to: Use Data Protection](#)

Provides procedures to encrypt or decrypt in-memory data or a file or stream using data protection.

[How to: Add Custom Permissions to Security Policy](#)

Provides a procedure to add a custom permission to security policy.

[How to: Enable Internet Explorer Security Settings for Managed Execution](#)

Provides a procedure to enable Internet Explorer security settings.

[How to: Request Minimum Permissions by Using the RequestMinimum Flag](#)

Provides an example that requests FileIOPermission using the RequestMinimum flag.

[How to: Create GenericPrincipal and GenericIdentity Objects](#)

Provides an example on how to use the GenericIdentity class in conjunction with the GenericPrincipal class to create an authorization scheme that exists independent of a Windows NT or Windows 2000 domain.

[How to: Create a WindowsPrincipal Object](#)

Provides two ways to create a WindowsPrincipal object, depending on whether code must repeatedly perform role-based validation or must perform it only once.

[How to: Perform Imperative Security Checks](#)

Provides an example that uses an imperative check to ensure that a GenericPrincipal matches the PrincipalPermission object.

[How to: Refuse Permissions by Using the RequestRefuse Flag](#)

Provides an example that uses RequestRefuse to refuse FileIOPermission from the common language runtime security system.

[How to: Request Permission to Access Unmanaged Code](#)

Provides an example that shows how to request permission to access unmanaged code.

[How to: Request Permission for a Named Permission Set](#)

Provides an example that shows the syntax to request permission for a named permission set.

[How to: Request Optional Permissions by Using the RequestOptional Flag](#)

Provides an example that requests FileIOPermission using the SecurityAction.RequestOptional flag, indirectly refusing all other permissions.

[How to: Store Asymmetric Keys in a Key Container](#)

Demonstrates how to create an asymmetric key, save it in a key container, retrieve the key at a later time, and delete the key from the container.

[How to: Add Assemblies to Security Policy Using Caspol.exe](#)

Explains how to add an assembly that implements a custom security object to the fully trusted assembly list.

[How to: View Code Groups Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to view a simple list of code groups belonging to a policy level, or a list that includes the names and descriptions of the code groups.

[How to: Modify Permissions in a Permission Set](#)

Explains how to use the .NET Framework Configuration tool (Mscorcfg.msc) to modify a permission in a permission set.

[How to: Add Permissions to a Permission Set](#)

Explains how to use the .NET Framework Configuration tool (Mscorcfg.msc) to add a permission to a permission set.

[How to: Suppress Policy Change Warnings Using Caspol.exe](#)

Explains how to Suppress Policy Change Warnings Using Caspol.exe.

[How to: Change Membership Conditions for a Code Group](#)

Explains how to use Mscorcfg.msc to change membership conditions in relation to code groups.

[How to: View Code Groups and Permission Sets Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to list all the code groups an assembly belongs to.

[How to: Administer Security Policy for Nondefault Users Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to administer a user policy for a user other than the current user.

[How to: Change Permission Sets Associated with an Existing Code Group](#)

Explains how to use Mscorcfg.msc to change permission sets.

[How to: Analyze Problems with Assembly Permissions Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to troubleshoot problems that might cause an assembly to not run or to access protected resources or run when it should not.

[How to: View Permission Sets Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to list the permission sets belonging to all policy levels or to a single policy level.

[How to: Undo Policy Changes Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to recover the last machine, user, or enterprise policy before the change was made.

[How to: Import a Permission by Using an XML File](#)

Provides an example that shows how the information for a permission might appear in the XML file.

[How to: Return to the Default Security Policy Settings Using Caspol.exe](#)

Explains how to return to the Default Security Policy Settings Using Caspol.exe.

[How to: Add Code Groups Using Caspol.exe](#)

Explains how to add code groups using Caspol.exe.

[How to: Override the Caspol.exe Self-Protection Mechanism](#)

Explains how to override the self-protection mechanism, if necessary.

[How to: Create Code Groups](#)

Explains how to create code groups using Mscorcfg.msc.

[How to: Disable Concurrent Garbage Collection](#)

Explains how to use the <gcConcurrent> element to specify how the runtime should run garbage collection.

[How to: Import a Code Group by Using an XML File](#)

Provides an example that shows how information for a code group and its associated membership condition and permission set might appear in an XML file.

[How to: Create a Publisher Policy](#)

Provides an example that shows a publisher policy file that redirects one version of myAssembly to another.

[How to: Remove Code Groups Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to remove code groups from code group hierarchies.

[How to: Create a Channel Template in a Configuration File](#)

Provides an example that shows how to create a channel template in a configuration file.

[How to: Change Permission Sets Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to replace the original permission set with the new set specified in the XML file.

[How to: Remove Permission Sets](#)

Explains how to use the .NET Framework Configuration tool (Mscorcfg.msc) to remove a permission set at a particular level.

[How to: Create Permission Sets](#)

Explains how to use the .NET Framework Configuration tool (Mscorcfg.msc) to create a permission set for a particular level and associate it with a new or existing code group.

[How to: Make Code Groups Exclusive or Level Final](#)

Explains how to use Mscorcfg.msc to make the new code group exclusive or level final.

[How to: Add an Assembly to the Policy Assemblies List](#)

Explains how to use the .NET Framework Configuration tool (Mscorcfg.msc) to add the assembly to the fully trusted assembly list.

[How to: Import a Permission Set by Using an XML File](#)

Provides an example that shows a permission set and a permission inside an XML file.

[How to: Locate Assemblies by Using DEVPATH](#)

Provides an example that shows how to cause the runtime to search for assemblies in directories specified by the DEVPATH environment variable.

[How to: Register a Server-Activated Object and a Client-Activated Object for a Host Application Domain](#)

Provides an example that shows how to register a server-activated object and a client-activated object for a host application domain.

[How to: View Security Policy Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to view the security policy code group hierarchy, and a list of known permission sets for all policy levels or for a single policy level.

[How to: Add Permission Sets Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to add permission sets to a code group.

[How to: Change Code Groups Using Caspol.exe](#)

Explains how to use the **-chgggroup** option of the Code Access Security Policy tool (Caspol.exe) to change the name, membership condition, permission set, flags, or description of a code group.

[How to: Configure Channels](#)

Provides an example that shows how to build an HttpChannel with a name that is different from "http" and use it for a server application.

[How to: Turn Security On and Off Using Caspol.exe](#)

Explains how to use the Code Access Security Policy tool (Caspol.exe) to turn security on and off.

[How to: Remove Permissions from a Permission Set](#)

Explains how to use the .NET Framework Configuration tool (Mscorcfg.msc) to remove a permission from a permission set.

[How to: Perform Common Security Policy Tasks Using the .NET Framework Configuration Tool \(Mscorcfg.msc\)](#)

Explains how to use the .NET Framework Configuration tool (Mscorcfg.msc) to configure security policy to suit your needs.

Additional Resources

[Microsoft Security Developer Center](#)

Contains many articles and resources on developing secure applications.

[Visual Studio 2005 Developer Center](#)

Contains many articles and resources on developing applications using Visual Studio 2005. This site is updated regularly with new content.

[Visual C# Developer Center](#)

Contains many articles and resources on developing C# applications. This site is updated regularly with new content.

[Microsoft .NET Framework Developer Center](#)

Contains many articles and resources on developing and debugging .NET Framework applications. This site is updated regularly with new content.

See Also

Concepts

[How Do I in C#](#)

Office Programming (How Do I in C#)

This page links to help on widely Office programming tasks. To view other categories of popular tasks covered in Help, [How Do I in C#](#).

General

[How to: Upgrade Document-Level Projects](#)

Explains the steps you must perform manually to complete the upgrade to Microsoft Visual Studio 2005 Tools for Office.

[Walkthroughs Using Excel](#)

Demonstrates three basic types of tasks: automating Microsoft Office Excel 2003, performing data analysis, and working with controls.

[Walkthroughs Using Word](#)

Demonstrates ways you can use the tools for Microsoft Office 2003 to automate Microsoft Office Word 2003 projects.

[How to: Automate Office Applications By Using Primary Interop Assemblies](#)

Provides steps to perform in your existing projects to be able to call unmanaged code using Visual Basic or C#.

[How to: Add Controls to Office Documents](#)

Explains how to add controls to Office Documents at design time or at runtime.

Word and Excel Applications

[How to: Run Excel Calculations Programmatically](#)

Explains how to run calculations programmatically for a range or for the Entire Application.

[How to: Create Office Menus Programmatically](#)

Provides an example that creates a menu called New Menu on the menu bar in Microsoft Office Excel 2003.

[How to: Create Office Toolbars Programmatically](#)

Provides an example that creates a toolbar called Test in Microsoft Office Word 2003. It appears near the middle of the document and contains two buttons. When a button is clicked, a message box appears.

[How to: Handle Errors in Office Projects](#)

Explains how to set the debugger to break on common language runtime exceptions.

See Also

Concepts

[How Do I in C#](#)

Smart Devices

This page links to Help on widely used smart devices programming tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

Getting Started with Smart Devices

[What's New in Smart Device Projects](#)

Provides the new or expanded features that are available in Visual Studio 2005.

[Device Capabilities and Required Development Tools](#)

Provides tables that contain a snapshot of the variations in smart device hardware, hardware features, and development tools.

[Remote Tools for Device Projects](#)

Provides a list of the remote tools that were available in embedded Visual C++ 4.0 and are shipped with Visual Studio 2005 to help you develop and debug device applications.

[How to: Optimize Help for Smart Device Development](#)

Shows you how to determine what Help you have installed, how to add Mobile and Embedded Development Help, and how to filter Help.

[How to: Launch the Device Emulator in Visual Studio](#)

Shows you how to launch the Device Emulator using the Connect to Device dialog box or the Device Emulator Manager.

[Updating Projects Created with Previous Tools](#)

Provides the improvements to the Visual Studio 2005 development environment.

[Selecting a Development Language](#)

Provides your options of programming languages when you develop an application, control, or library for deployment on a Smart Device.

[Walkthrough: Creating Windows Forms Applications for a Device](#)

Demonstrates the main difference between desktop and device programming; namely, that you must target a device. In this walkthrough, the device is a built-in emulator of the Pocket PC 2003.

Programming Smart Devices Using Visual Basic and C#

[Programming for Devices Using the .NET Compact Framework](#)

Provides information on developing smart device applications using the Visual Basic or C# language and the .NET Compact Framework.

[.NET Compact Framework Reference for Device Projects](#)

Provides information on the .NET Compact Framework as a subset of the .NET Framework class library.

[How to: Create Device Applications Using Visual Basic or Visual C#](#)

Describes how to create device applications, and how the process differs from creating desktop applications.

[How to: Share Source Code Across Platforms \(Devices\)](#)

Describes using compiler constants to share the same source code across different platforms.

[How to: Change Platforms in Device Projects](#)

Describes how to change back and forth between platforms in the same project.

[How to: Upgrade Projects to Later .NET Compact Framework Versions](#)

Describes how to upgrade the platform of an existing project if a later version of the platform is installed.

[Managing Code Snippets in Device Projects](#)

Describes using snippets that pertain exclusively to device projects.

[How to: Verify Platform Support for Code in Device Projects](#)

Describes how to ensure that your code is supported by the target platform.

[How to: Handle HardwareButton Events \(Devices\)](#)

Describes how to override the application keys on a Pocket PC.

[How to: Change the Orientation and Resolution of Forms \(Devices\)](#)

Describes how to change orientation and resolution if the defaults are incorrect or lacking.

[How to: Change the Default Device \(Managed Projects\)](#)

Describes how to change the target device during project development.

[How to: Optimize Help for Smart Device Development](#)

Describes how to use the Smart Device Help filter to show only those .NET Framework elements supported for device application development.

Device Connections

[How to: Connect to the Device Emulator From a Virtual PC Session](#)

Describes the technique for connecting in the absence of TCP/IP.

[How to: Access the Smartphone Emulator File System](#)

Describes how to access the file system of the Smartphone emulator, which does not have its own file viewer.

[How to: Connect Using Bluetooth](#)

Describes how to connect using Bluetooth.

[How to: Connect Using IR](#)

Describes how to connect using Infrared.

[How to: Connect to Windows CE Device Without ActiveSync](#)

Describes the steps you take to connect to a device when ActiveSync services are not available.

[How to: Access Development Computer Files from the Emulator](#)

Describes how to use a shared folder to access development computer files from the emulator.

[How to: Set Connection Options \(Devices\)](#)

Describes where to find the common dialog boxes for setting connection options.

Debugging Smart Devices

[Differences Between Device And Desktop Debuggers](#)

Describes differences between device and desktop Debuggers.

[How to: Attach to Managed Device Processes](#)

Describes how to attach to managed device processes.

[How to: Change Device Registry Settings](#)

Describes how to use the Remote Registry Editor to change registry settings on the device.

[Walkthrough: Debugging a Solution That Includes Both Managed and Native Code](#)

Describes how to debug these mixed solutions.

Data In Managed Device Projects

[How to: Generate SqlCeResultSet Code \(Devices\)](#)

Describes how to generate ResultSets instead of DataSets.

[How to: Change the Design-Time Connection String \(Devices\)](#)

Describes changing the string that Visual Studio uses to connect to a SQL Server Mobile database at design time.

[How to: Change the Run-Time Connection String \(Devices\)](#)

Describes changing the string that the application uses to connect to a SQL Server Mobile database at run time.

[How to: Add Navigation Buttons \(Devices\)](#)

Describes an alternative to the **DataNavigator** class, which is not supported in the .NET Compact Framework.

[How to: Persist Data Changes to the Database \(Devices\)](#)

Describes how to apply changes in the DataSet back to the database.

[How to: Create a Database \(Devices\)](#)

Describes how to use the Visual Studio environment to create a SQL Mobile database, inside or outside a project.

[How to: Add a Database to a Device Project](#)

Describes adding a SQL Mobile database, available in Server Explorer, as a data source for a Visual Basic or Visual C# project.

[How to: Add a SQL Server Database as a Data Source \(Devices\)](#)

Describes how to add a SQL Server database as a data source for a Visual Basic or Visual C# project.

[How to: Add a Business Object as a Data Source \(Devices\)](#)

Describes how to add a business object as a data source for a Visual Basic or Visual C# project.

[How to: Add a Web Service as a Data Source \(Devices\)](#)

Describes how to add a Web service as a data source for a Visual Basic or Visual C# project.

[How to: Manage Tables in a Database \(Devices\)](#)

Describes how to add and remove tables, and how to edit an existing table schema.

[How to: Manage Columns in a Database \(Devices\)](#)

Describes how to add and remove columns and to edit their properties.

[How to: Manage Indexes in a Database \(Devices\)](#)

Describes how to add and remove indexes, and how to change their sort-order property.

[How to: Manage Passwords for Databases \(Devices\)](#)

Describes how to set a password for a new SQL Mobile database and how to change a password for an existing database.

[How to: Shrink and Repair a Database \(Devices\)](#)

Describes how to shrink and repair SQL Mobile databases.

[How to: Create Parameterized Queries \(Devices\)](#)

Describes how to create parameterized queries.

[Walkthrough: A Parameterized Query Application](#)

Provides step-by-step instructions for an end-to-end project that includes building a parameterized query.

[How to: Create Master-Detail Applications \(Devices\)](#)

Describes how to implement a master-detail relationship.

[Walkthrough: A Database Master-Detail Application](#)

Provides step-by-step instructions for an end-to-end project that includes creating and running a master-detail application.

[How to: Preview Data in a Database \(Devices\)](#)

Describes several options for viewing data in a database.

[How to: Generate Summary and Edit Views for Data Applications \(Devices\)](#)

Describes how to use data forms to view and edit single rows of data in a datagrid.

[Packaging and Deploying Device Solutions](#)

[Walkthrough: Packaging a Smart Device Solution for Deployment](#)

Provides step-by-step instructions for packaging an application and its resources.

Security in Device Projects

[How to: Import and Apply Certificates in Device Projects](#)

Describes the effective use of the **Select Certificate** dialog box for signing device projects.

[How to: Launch Signtool.exe as a Post-Build Event \(Devices\)](#)

Describes how to sign a project when post-build events have changed the original binaries.

[How to: Query a Device For Its Security Model](#)

Describes how to determine what certificates are already installed in the device certificate store.

[How to: Sign a Visual Basic or Visual C# Application \(Devices\)](#)

Lists the steps for signing an application written against the .NET Compact Framework.

[How to: Sign a Visual Basic or Visual C# Assembly \(Devices\)](#)

Lists the steps for signing project assemblies.

[How to: Sign a CAB File \(Devices\)](#)

Lists the steps for signing a device CAB project.

[How to: Provision a Device in a Visual Basic or Visual C# Project](#)

List the steps for adding digital certificates to the device store in managed projects.

[How to: Provision a Device with a Security Model](#)

Describes using RapiConfig.exe to provision a device with a security model.

See Also

Concepts

[How Do I in C#](#)

Deployment (How Do I in C#)

This page links to help on widely used deployment tasks. To view other categories of popular tasks covered in Help, see [How Do I in C#](#).

ClickOnce

[How to: Publish a ClickOnce Application](#)

Demonstrates how to make a ClickOnce application available to users by publishing it to a Web server, file share, or removable media.

[How to: Specify a Publishing Location](#)

Demonstrates how to specify the location where the application files and manifest will be placed.

[How to: Specify an Installation URL](#)

Demonstrates how to use the **Installation URL** property to specify the Web server where users will go to download the application.

[How to: Specify a Support URL](#)

Demonstrates how to use the Support URL property to identify a Web page or file share where users can go to get information about the application.

[How to: Specify the ClickOnce Install Mode](#)

Demonstrates how to set the Install Mode to specify whether the application will be available offline or online.

[How to: Enable AutoStart for CD Installations](#)

Demonstrates how to enable AutoStart so that the ClickOnce application will be automatically launched when the media is inserted.

[How to: Set the ClickOnce Publish Version](#)

Demonstrates how to set the Publish Version property, which determines whether or not the application that you are publishing will be treated as an update.

[How to: Automatically Increment the ClickOnce Publish Version](#)

Demonstrates how to change the Publish Version property to cause the application to be published as an update.

[How to: Specify Which Files Are Published via ClickOnce](#)

Demonstrates how to exclude files, mark files as data files or prerequisites, and create groups of files for conditional installation.

[How to: Install Prerequisites with a ClickOnce Application](#)

Demonstrates how to specify a set of prerequisite components to be packaged along with your application

[How to: Manage Updates for a ClickOnce Application](#)

Demonstrates how to specify when and how update checks are performed, whether updates are mandatory, and where the application should check for updates.

[How to: Include a Data File in a ClickOnce Application](#)

Provides procedures that show you how to add a data file of any type into your ClickOnce application.

[How to: Deploy the .NET Framework by Using Systems Management Server](#)

Provides tasks that must be performed on the server running Systems Management Server.

[How to: Add a Trusted Publisher to a Client Computer for ClickOnce Applications](#)

Shows you how to use the command-line tool CertMgr.exe to add a publisher's certificate to the Trusted Publishers store on a client computer.

[How to: Specify an Alternate Location for Deployment Updates](#)

Shows you how to specify an alternate location for updates in your deployment manifest so that your application can update

itself from the Web after its initial installation.

[How to: Deploy the .NET Framework by Using Active Directory](#)

Provides a procedure to deploy the .NET Framework by using Active Directory.

[How to: Retrieve Query String Information in a ClickOnce Application](#)

Provides procedure to show you how to use a ClickOnce application to obtain query string information. It also shows how your ClickOnce application can use a small piece of code to read these values the first time the application launches.

[How to: Enable ClickOnce Security Settings](#)

Demonstrates how to temporarily disable the security settings during development.

[How to: Set a Security Zone for a ClickOnce Application](#)

Demonstrates how to set a security zone to populate the Permissions required by the application table.

[How to: Set Custom Permissions for a ClickOnce Application](#)

Demonstrates how to restrict the application to the specific permissions that it needs to operate properly.

[How to: Determine the Permissions for a ClickOnce Application](#)

Demonstrates how to analyze your application to determine which permissions it needs by running the Permission Calculator tool.

[How to: Debug a ClickOnce Application with Restricted Permissions](#)

Demonstrates how to debug the application with the same permissions as the end user.

Windows Installer

[Windows Installer Deployment](#)

Links to articles on how to use Windows Installer deployment to create installer packages to be distributed to users.

[Walkthrough: Deploying a Windows-based Application](#)

Demonstrates the process of creating an installer for a Windows application that launches Notepad.

[Walkthrough: Installing Shared Components Using Merge Modules](#)

How to use merge modules to ensure that shared components are packaged and delivered for consistent deployment.

[Walkthrough: Creating a Custom Action](#)

Demonstrates the process of creating a DLL custom action to direct a user to a Web page at the end of an installation.

[Walkthrough: Using a Custom Action to Display a Message at Installation](#)

Demonstrates how to use a custom action to take user input and pass it to a message box that appears during installation.

[Walkthrough: Using a Custom Action to Pre-Compile an Assembly at Installation](#)

Demonstrates how to pass the path name of a DLL to the CustomActionData property in order to precompile the assembly to native code during installation.

[Walkthrough: Using a Custom Action to Create a Database at Installation](#)

Demonstrates the use of a custom action and the CustomActionData property to create a database and database table during installation.

[Walkthrough: Redirecting an Application to Target a Different XML Web Service at Installation](#)

Demonstrates how to create a Web application that can be redirected to target a different XML Web service by using the URL Behavior property, an Installer class, and a Web Setup project.

[How to: Install Prerequisites in Windows Installer Deployment](#)

Demonstrates how to automatically detect the existence of components during installation and install a predetermined set of prerequisites, a process known as bootstrapping.

[How to: Create or Add Deployment Projects](#)

Demonstrates how to specify where and how your solution will be deployed both during and after development.

[How to: Create or Add a Setup Project](#)

Demonstrates how to create Windows Installer (.msi) files, which are used to distribute your application for installation on another computer or Web server.

[How to: Create or Add a Merge Module Project](#)

Demonstrates how to create a merge module project to package files or components that will be shared between multiple applications.

[How to: Create or Add a Cab Project](#)

Demonstrates how to create a CAB project to create cabinet (.cab) files that can be used to download components to a Web browser.

[How to: Set Deployment Project Properties](#)

Demonstrates how to set configuration-dependent properties using the **Deployment Properties** dialog box.

[How to: Add Items to a Deployment Project](#)

Demonstrates how to specify what needs to be included in the installer and where to install it on the target computer.

[How to: Add Merge Modules to a Deployment Project](#)

Demonstrates how to use merge modules (.msm files) to share components between multiple deployment projects.

[How to: Add and Remove Icons](#)

Demonstrates how to install and associate icons with your application on a target computer during installation

[How to: Exclude Items from a Deployment Project](#)

Demonstrates how to exclude a file or files from a deployment project.

[How to: Set Conditional Installation Based on Operating System Versions](#)

Demonstrates how to set the Condition property to add conditional logic to an installer, for example, installing different files or setting different registry values for different operating system versions.

See Also

Concepts

[How Do I in C#](#)

[Deploying C# Applications](#)

Using the Visual C# IDE

This section introduces you to the Visual C# integrated development environment (IDE) and describes how it is used in all phases of the development cycle, from setting up a project to distributing the finished application to end users.

In This Section

[Introduction to the IDE \(Visual C#\)](#)

Provides a guide to the editors and windows that make up the Visual C# integrated development environment.

[Creating a Project \(Visual C#\)](#)

Describes setting up the right project for the kind of application you want to create.

[Designing a User Interface \(Visual C#\)](#)

Explains using the designers to add controls to your application.

[Editing Code \(Visual C#\)](#)

Describes entering source code with the Code Editor and using tools such as IntelliSense, Code Snippets, and Refactoring.

[Navigating and Searching \(Visual C#\)](#)

Describes moving around in your project files and finding things fast.

[Building and Debugging \(Visual C#\)](#)

Describes how to create an executable assembly from your project files, and how to run it in the Visual Studio debugger.

[Modeling and Analyzing Code \(Visual C#\)](#)

Describes how to view class relationships with Class Designer, test objects with Object Test Bench, and run the code analysis tools.

[Adding and Editing Resources \(Visual C#\)](#)

Describes adding files such as icons, strings, tables, and other types of data to your project.

[Getting Help \(Visual C#\)](#)

Explains how to find the documentation you need.

[Deploying C# Applications](#)

Describes distributing your application to end users.

[Visual C# Code Editor Features](#)

Provides complete reference material on IDE features that are unique to Visual C#, including Code Snippets and Refactoring.

[Visual C# IDE Settings](#)

Describes how to change the default Visual C# settings.

[Visual C# Keyboard Shortcuts](#)

Explains how to speed up the way you use Visual C#.

See Also

Other Resources

[Visual C#](#)

[Getting Started with Visual C#](#)

[C# Reference](#)

[Integrated Development Environment for Visual Studio](#)

Introduction to the IDE (Visual C#)

The Visual C# integrated development environment (IDE) is a collection of development tools exposed through a common user interface. Some of the tools are shared with other Visual Studio languages, and some, such as the C# compiler, are unique to Visual C#. The documentation in this section provides an overview of how to use the most important Visual C# tools as you work in the IDE in various phases of the development process.

Note

If you are developing an ASP.NET 2.0 Web application, you will be using the Visual Web Developer IDE, which is a fully integrated part of Visual Studio 2005. However, if your code-behind pages are in Visual C#, then you will be using the Visual C# Code Editor within Visual Web Developer. Therefore, some topics in this section, such as [Designing a User Interface \(Visual C#\)](#), might not be completely applicable to Web applications.

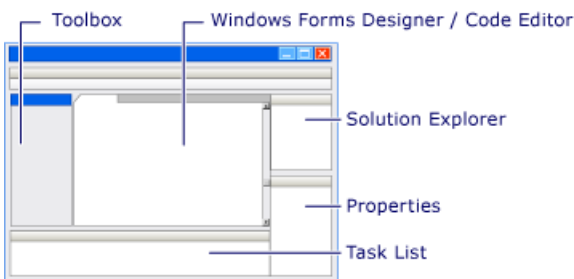
Visual C# Tools

The following are the most important tools and windows in Visual C#. The windows for most of these tools can be opened from the **View** menu.

- The Code Editor, for writing source code.
- The C# compiler, for converting C# source code into an executable program.
- The Visual Studio debugger, for testing your program.
- The **Toolbox** and **Designer**, for rapid development of user interfaces using the mouse.
- **Solution Explorer**, for viewing and managing project files and settings.
- **Project Designer**, for configuring compiler options, deployment paths, resources, and more.
- **Class View**, for navigating through source code according to types, not files.
- **Properties Window**, for configuring properties and events on controls in your user interface.
- Object Browser, for viewing the methods and classes available in dynamic link libraries including .NET Framework assemblies and COM objects.
- Document Explorer, for browsing and searching product documentation on your local machine and on the Internet.

How the IDE Exposes the Tools

You interact with the tools through windows, menus, property pages, and wizards in the IDE. The basic IDE looks something like this:



You can quickly access any open tool windows or files by pressing CTRL + TAB. For more information, see [Navigating and Searching \(Visual C#\)](#).

Editor and Windows Form Designer Windows

The large main window is used by both the Code Editor and the Windows Forms Designer. You can toggle between code view and Design view by pressing F7, or clicking **Code** or **Designer** on the **View** menu. While in Design view, you can drag controls onto the window from the **Toolbox**, which you can make visible by clicking on the **Toolbox** tab on the left margin. For more information about the Code Editor, see [Editing Code \(Visual C#\)](#). For more information about the Windows Forms Designer, see [Windows Forms Designer](#).

The **Properties** window in the lower right is populated only in Design view. It enables you to set properties and hook up

events for user interface controls such as buttons, text boxes, and so on. When you set this window to **Auto Hide**, it will collapse into the right margin whenever you switch to **Code View**. For more information about the **Properties** window and the Designer, see [Designing a User Interface \(Visual C#\)](#).

Solution Explorer and Project Designer

The window in the top right is **Solution Explorer**, which shows all the files in your project in a hierarchical tree view. When you use the **Project** menu to add new files to your project, you will see them reflected in **Solution Explorer**. In addition to files, **Solution Explorer** also displays your project settings, and references to external libraries required by your application.

The **Project Designer** property pages are accessed by right-clicking on the **Properties** node in **Solution Explorer**, and then clicking **Open**. Use these pages to modify build options, security requirements, deployment details, and many other project properties. For more information about **Solution Explorer** and the **Project Designer**, see [Creating a Project \(Visual C#\)](#).

Compiler, Debugger, and Error List Windows

The C# compiler has no window because it is not an interactive tool, but you can set compiler options in the **Project Designer**. When you click **Build** on the **Build** menu, the C# compiler is invoked by the IDE. If the build is successful, the status pane displays a Build Succeeded message. If there were build errors, the **Error List** window appears below the editor/designer window with a list of errors. Double-click an error to go to the problem line in your source code. Press F1 to see Help documentation for the highlighted error.

The debugger has various windows that display values of variables and type information as your application is running. You can use the Code Editor window while debugging to specify a line at which to pause execution in the debugger, and to step through code one line at a time. For more information, see [Building and Debugging \(Visual C#\)](#).

Customizing the IDE

All of the windows in Visual C# can be made dockable or floating, hidden or visible, or can be moved to new locations. To change the behavior of a window, click the down arrow or push-pin icons on the title bar and select from among the available options. To move a docked window to a new docked location, drag the title bar until the window dropper icons appear. While holding down the left mouse button, move the mouse pointer over the icon at the new location. Position the pointer over the left, right, top or bottom icons to dock the window on the specified side. Position the pointer over the middle icon to make the window a tabbed window. As you position the pointer, a blue semi-transparent rectangle appears, which indicates where the window will be docked in the new location.



You can customize many other aspects of the IDE by clicking **Options** on the **Tools** menu. For more information, see [Options Dialog Box \(Visual Studio\)](#).

See Also

Concepts

[Designing a User Interface \(Visual C#\)](#)

[Creating a Project \(Visual C#\)](#)

[Editing Code \(Visual C#\)](#)

[Building and Debugging \(Visual C#\)](#)

Other Resources

[Visual C#](#)

[Using the Visual C# IDE](#)

[Integrated Development Environment for Visual Studio](#)

[Visual Web Developer](#)

[Visual Web Developer User Interface Elements](#)

Creating a Project (Visual C#)

When you are ready to begin coding, the first step is to set up a project. The project contains all the raw materials for your application, including not only the source code files, but also resource files such as icons, references to external files that your program depends on, and configuration data such as compiler settings. When you build a project, Visual C# invokes the C# compiler and other internal tools to create an executable assembly using the files in your project.

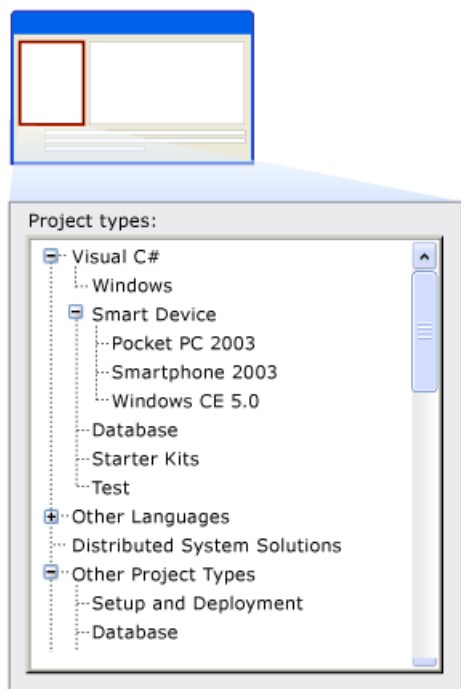
Creating a New Project

You create a new project by clicking the **File** menu, pointing to **New**, and then clicking **Project**.

Note

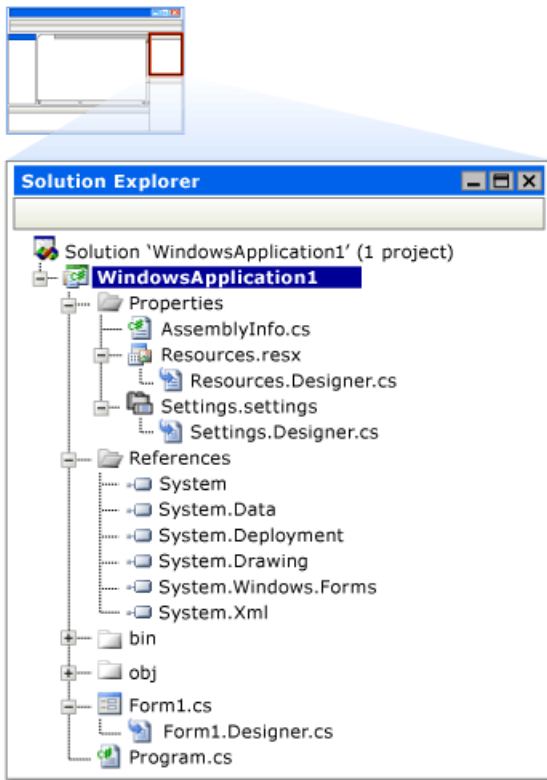
If you select **Web Site** instead of **Project**, the [Visual Web Developer](#) integrated development environment (IDE) opens. This is a separate and distinct environment within Visual Studio for creating ASP.NET Web applications. The Visual Web Developer IDE does use the Visual C# code editor for editing code-behind files in C#. If you are creating Web applications, you should use the Visual Web Developer documentation primarily, but refer to [Editing Code \(Visual C#\)](#) for information about the C# editor.

The following illustration shows the **New Project** dialog box. You can see that **Visual C#** is selected by default in the window on the left, and on the right, you have a choice of six or more project templates to choose from. If you expand the **Smart Device** or **Other Project Types** node on the left, you can see different project types appear on the right side.



Starter Kits are another type of project template. If you install a starter kit, you will see it listed in the **New Project Dialog Box**. For more information, see [Starter Kits](#).

After you select a project template and click **OK**, Visual Studio creates the project and you are ready to begin coding. The project files, references, settings, and resources are visible in the **Solution Explorer** window to the right.



What's in Your Project?

Properties

The **Properties** node represents configuration settings that apply to your entire project and are stored in the .csproj file in your solution folder. These settings include compilation options, security and deployment settings and much more. You make modifications to your project using the **Project Designer**, which is a set of **Property Pages** that you access by right-clicking **Properties**, and selecting **Open**. For more information, see [Modifying Project Properties \(Visual C#\)](#).

References

In the context of a project, a reference simply identifies a binary file that your application requires in order to run. Typically, a reference identifies a DLL file such as one of the .NET Framework class library files. It can also reference a .NET assembly (called a shim) that enables your application to call methods on a COM object or native Win32 DLL. If your program creates an instance of a class that is defined in some other assembly, then you must add a reference to that file in your project before you compile your project. To add a reference, click **Add Reference** on the **Project** menu. All C# projects by default include a reference to mscorlib.dll, which contains the core .NET Framework classes. You can add references to additional .NET Framework DLLs and other files by clicking the **Project** menu, and selecting **Add Reference**.

Note

Do not confuse the concept of a project reference with the concept of reference types in C# or other programming languages. The former refers to a file and its expected location on disk. The latter refers to C# types, which are declared using the **class** keyword.

Resources

A resource is data that is included with your application but can be stored in such a way that it can be modified independently from the other source code. For example, you can store all your strings as resources rather than hard-coding them into the source code. You can then translate the strings into different languages at some later date, and add them to the application folder that you ship to customers without having to recompile your assembly. The five types of resources defined by Visual C# are: strings, images, icons, audio, and files. You add, remove, or edit resources using the **Resource Designer**, which is accessed on the **Resources** tab in the **Project Designer**.

Forms

When you create a Windows Forms project, Visual C# adds one form to the project by default and calls it Form1. The two files that represent the form are called Form1.cs and Form1.designer.cs. You write your code in Form1.cs; the designer.cs file is where the Windows Forms Designer writes the code that implements all the actions that you performed by dragging and dropping controls from the Toolbox.

You can add a new form by clicking the **Project** menu item, and selecting **Add Windows Form**. Each form has two files

associated with it. Form1.cs, or whatever you might name it, contains the source code that you write to configure the form and its controls, such as list boxes and text boxes, and responds to events such as button clicks and keystrokes. In simple Windows Forms projects, you do most or all of your coding in this file.

The Designer.cs file contains the source code that the **Forms Designer** writes when you drag controls onto the form, set properties in the **Properties** window, and so on. Typically, you should not edit this file manually at all.

Note

Obviously, if you create a console application project, it will not contain source code files for Windows Forms.

Other Source Code Files

A project may include any number of additional .cs files that may or may not be associated with a particular Windows Form. In the previous illustration of **Solution Explorer**, program.cs contains the entry point for the application. A single .cs file can contain any number of class and struct definitions. You can add new or existing files or classes to your project by clicking **Add New Item** or **Add Existing Item** on the **Project** menu.

See Also

Tasks

[How to: Create Solution and Project Build Configurations](#)

[How to: Create a Windows Application Project](#)

Concepts

[Introduction to Solutions, Projects, and Items](#)

[Using Solution Explorer](#)

[Hidden Project Files in Solution Explorer](#)

[Controlling Projects and Solutions](#)

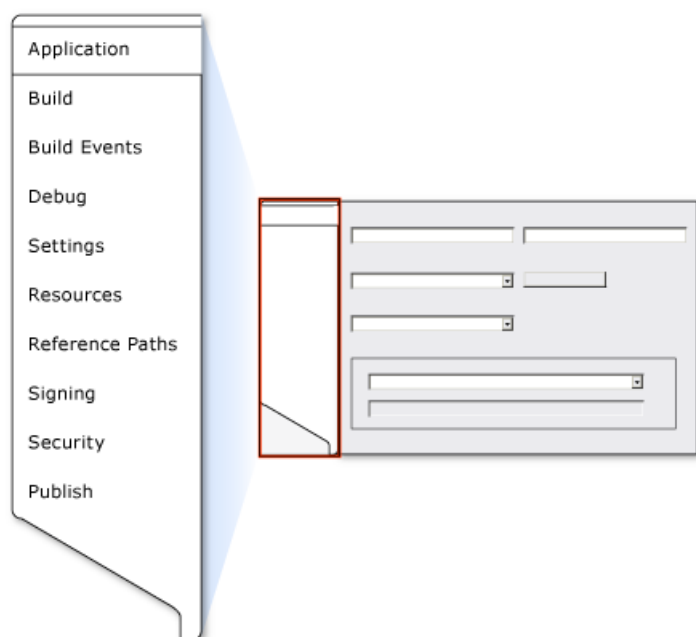
Other Resources

[Visual C#](#)

[Using the Visual C# IDE](#)

Modifying Project Properties (Visual C#)

After you create a project, you can use the **Project Designer** to perform tasks such as change the name of your executable file, customize the build process, add a reference to a DLL, or strengthen the security settings. You can access the **Project Designer** in the **Project** menu by clicking **Properties**, or by right-clicking the **Properties** item in **Solution Explorer**. The **Project Designer** will appear in the editor/designer window as shown in the following illustration:



Project properties are grouped into 10 pages in the **Project Designer**. The **Project Designer** property pages are located in the same middle pane used by the **Windows Forms Designer** and code editor.

Note

Visual Studio Team System includes an additional property page for Code Analysis.

In the illustration above, the **Application** property page is displayed. By clicking on the labels on the left tab (**Build**, **Build Events**, **Debug**, and so on) you can access the corresponding property page. The project-specific information that is entered here is stored in a .csproj file which is not visible in **Solution Explorer** but is located in the project folder on the drive. While you are working in Visual C#, you can access help for any of the property pages by positioning the mouse cursor on the page and pressing **F1**.

The following table provides a brief description of each page in the **Project Designer**:

Property Page	Description
Application	Change the name of the assembly, the project type, assembly information including version number and other resource options. For more information, see Application Page, Project Designer (C#) .
Build	Change the location in which the compiled assembly is stored, conditional compilation options, how errors and warnings are handled, and other settings. For more information, see Build Page, Project Designer (C#) .
Build Events	Create and modify custom build steps. For more information, see Build Events Page, Project Designer (C#, J#) .
Debug	Specify the command line arguments when running under the debugger, and other settings. For more information, see Debug Page, Project Designer .
Resources	Add strings, icons, images or other types of files to your project as resources. For more information, see Resources Page, Project Designer .

Settings	Store settings such as connection strings for a database or the color-scheme that a particular user wants to use. These settings can be retrieved dynamically at run time. For more information, see Settings Page, Project Designer .
Reference Paths	Specify the path where assemblies referenced in your project are located. For more information, see Reference Paths Page, Project Designer (C#, J#) .
Signing	Specify ClickOnce certificate options, and provide strong name for your assembly. For more information, see Signing Page, Project Designer and ClickOnce Deployment Overview
Security	Specify security settings that your application requires in order to run. For more information, see Security Page, Project Designer .
Publish	Specify options for distributing your application to a web site, ftp server, or file location. For more information, see Publish Page, Project Designer .
Code Analysis (Visual Studio Team System only)	Options for tools that analyze your source code for potential security issues, adherence to .NET Framework design guidelines, and more. For more information, see Code Analysis, Project Designer .

See Also

Concepts

[Introduction to the Project Designer](#)

Other Resources

[Using the Visual C# IDE](#)

[Integrated Development Environment for Visual Studio](#)

Designing a User Interface (Visual C#)

In Visual C#, the most rapid and convenient way to create your user interface (UI) is to do so visually, using the **Windows Forms Designer** and **Toolbox**. There are three basic steps in creating all user interfaces:

- Adding controls to the design surface.
- Setting initial properties for the controls.
- Writing handlers for specified events.

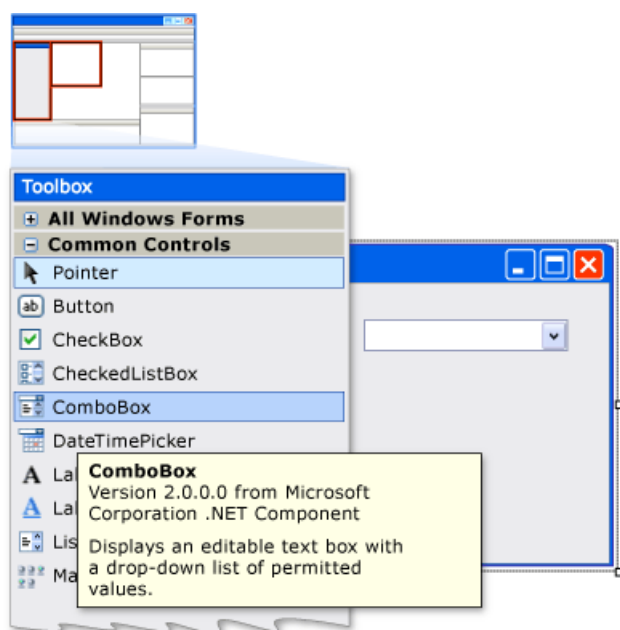
Although you can also create your UI by writing your own code, designers enable you to do this work much more rapidly than is possible by manual coding.

Note

You can also use Visual C# to create console applications that have a simple text-based UI. For more information, see [Creating Console Applications \(Visual C#\)](#).

Adding Controls

In the designer, you use the mouse to drag , such as buttons and text boxes, onto a design surface that represents your form. The following illustration shows a combo box that has been dragged from the **Toolbox** window onto a form in the **Windows Forms Designer**.



As you work visually, the designer translates your actions into C# source code and writes them into a project file called <name>.designer.cs where <name> is the name you gave to the form. When your application runs, that source code will position and size your UI elements so that they appear just as they do on the design surface. For more information, see [Windows Forms Designer](#).

Setting Properties

After you add a control to your form, you can use the **Properties** window to set its properties, such as background color and default text. The values that you specify in the **Properties** window are simply the initial values that will be assigned to that property when the control is created at run time. In many cases, those values can be accessed or changed programmatically at run time simply by getting or setting the property on the instance of the control class in your application. The **Properties** window is useful at design time because it enables you to browse all the properties, events, and methods supported on a control. For more information, see [Properties Window](#).

Handling Events

Programs with graphical user interfaces are primarily event-driven. They wait until a user does something such as entering text into a text box, clicking a button, or changing a selection in a list box. When that happens, the control, which is just an instance

of a .NET Framework class, sends an event to your application. You have the option of handling an event by writing a special method in your application that will be called when the event is received.

You can use the **Properties** window to specify which events you want to handle in your code; select a control in the designer and click the **Events** button, with the lightning bolt icon, on the **Properties** window toolbar to see its events. The following diagram shows the events button.



When you add an event handler through the **Properties** window, the designer will automatically write the empty method body for you, and it is up to you to write the code to make the method do something useful. Most controls generate a large number of events, but in most cases an application will only need to handle a few of them, or even only one. For example, you probably need to handle a button's **Click** event, but you do not need to handle its **Paint** event unless you want to customize its appearance in some advanced way.

Next Steps

For more information about Windows Forms user interfaces, see the following topics:

- [Creating Windows-based Applications](#)
- [Walkthrough: Creating a Simple Windows Form](#)
- [Windows Forms Designer User Interface Elements](#)

In the .NET Framework class library, [System.Windows.Forms](#) and related namespaces contain the classes used in Windows Forms development.

See Also

Other Resources

[Visual C#](#)

[Using the Visual C# IDE](#)

Editing Code (Visual C#)

The Visual C# Code Editor is a word processor for writing source code. Just as Microsoft Word provides extensive support for sentences, paragraphs, and grammar, the C# Code Editor does the same for C# syntax and the .NET Framework. This support can be grouped into five main categories:

- **IntelliSense:** Continually updated basic documentation on .NET Framework classes and methods as you type them in the editor, as well as automatic code generation.
- **Refactoring:** Intelligent restructuring of your code base as it evolves over the course of a development project.
- **Code snippets:** A library you can browse that contains frequently repeated code patterns.
- **Wavy underlines:** Visual notifications of misspelled words, erroneous syntax, and warning situations as you type.
- **Readability aids:** Outlining and colorization.

IntelliSense

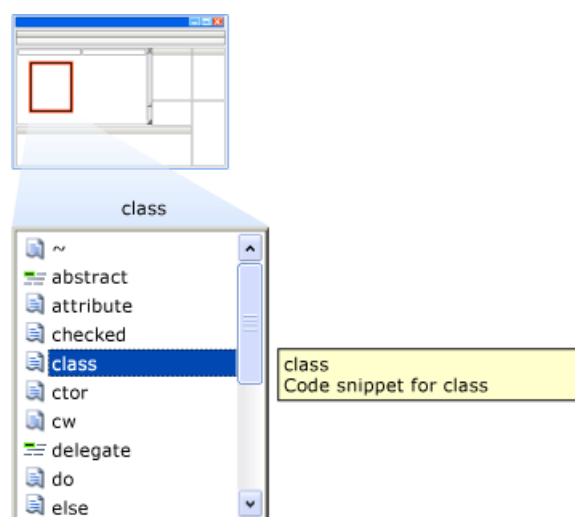
IntelliSense is the name of a set of related features that are designed to minimize the time you spend looking for help and to help you enter code more accurately and efficiently. These features all provide basic information about language keywords, .NET Framework types, and method signatures as you type them in the editor. The information is displayed in ToolTips, list boxes, and smart tags.

Note

Many of the features in IntelliSense are shared with other Visual Studio languages and are documented with illustrations in the [Coding Aids](#) node of the MSDN library. The following sections provide a brief overview of IntelliSense, with links to the more complete documentation.

Completion Lists

As you enter source code in the editor, IntelliSense displays a list box that contains all the C# keywords and .NET Framework classes. If it finds a match in the list box for the name that you are typing, it selects the item. If the selected item is what you want, you can simply hit TAB and IntelliSense will finish entering the name or keyword for you. For more information, see [Completion Lists in C#](#).



Quick Info

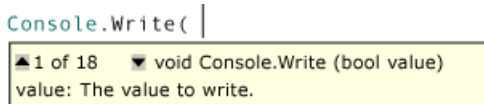
When you hover the cursor over a .NET Framework type, IntelliSense will display a Quick Info Tooltip that contains basic documentation about that type. For more information, see [Quick Info](#).

List Members

When you enter a .NET Framework type into the Code Editor, and then type the dot operator (.), IntelliSense displays a list box that contains the members of that type. When you make a selection and press TAB, IntelliSense enters the member name. For more information, see [List Members](#).

Parameter Info

When you enter a method name in the Code Editor, and then type an opening parentheses, IntelliSense will display a Parameter Info ToolTip that shows the order and types of the method's parameters. If the method is overloaded, you can scroll down through all the overloaded signatures. For more information, see [Parameter Info](#).



Add using

Sometimes you may attempt to create an instance of a .NET Framework class without a sufficiently qualified name. When this happens, IntelliSense displays a smart tag after the unresolved identifier. When you click the smart tag, IntelliSense displays a list of **using** directives that will enable the identifier to be resolved. When you select one from the list, IntelliSense adds the directive to the top of your source code file and you can continue coding at your current location. For more information, see [Add using](#).

Refactoring

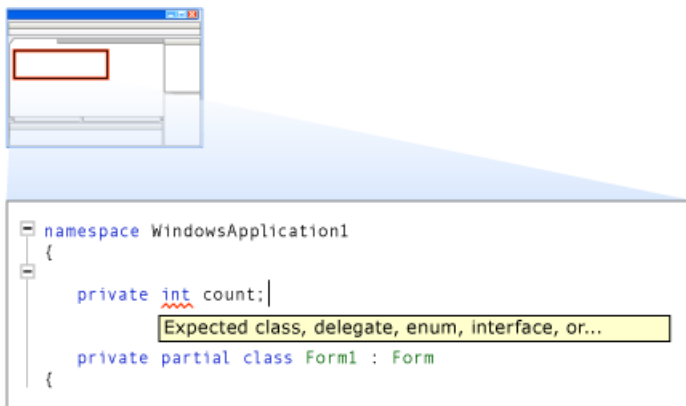
As a code base grows and evolves over the course of a development project, it is sometimes desirable to make changes in order to make it more readable to humans, or more portable. For example, you may want to split some methods up into smaller methods, or change method parameters, or rename identifiers. The Refactoring feature, which is accessible by right-clicking in the Code Editor, does all this in a way that is much more convenient, intelligent, and thorough than traditional tools such as search and replace. For more information, see [Refactoring](#).

Code Snippets

Code snippets are small units of commonly used C# source code that you can enter accurately and quickly with only a couple of keystrokes. The code snippet menu is accessed by right-clicking in the Code Editor. You can browse from among the many snippets provided with Visual C#, and you can also create your own. For more information, see [Code Snippets \(C#\)](#).

Wavy underlines

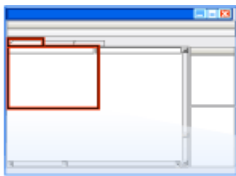
Wavy underlines give you instant feedback on errors in your code as you type. A red wavy underline identifies a syntax error such as a missing semicolon or mismatched braces. A green wavy underline identifies a potential compiler warning, and blue identifies an [Edit and Continue](#) issue. The following illustration shows a red wavy underline:



Readability Aids

Outlining

The Code Editor automatically treats namespaces, classes, and methods as regions that you can collapse in order to make other parts of the source code file easier to find and read. You can also create your own collapsible regions by surrounding code with the **#region** and **#endregion** directives.



```
public int RowHeight...  
public Font ItemFont...  
public Font TitleFont...  
public void NextArticle()...  
public void PreviousArticle()...  
public void ItemListView(string title, IList<T> items)...
```

Colorization

The editor gives different colors to the various categories of identifiers in a C# source code file. For more information, see [Code Colorization](#).

See Also

Other Resources

[Using the Visual C# IDE](#)

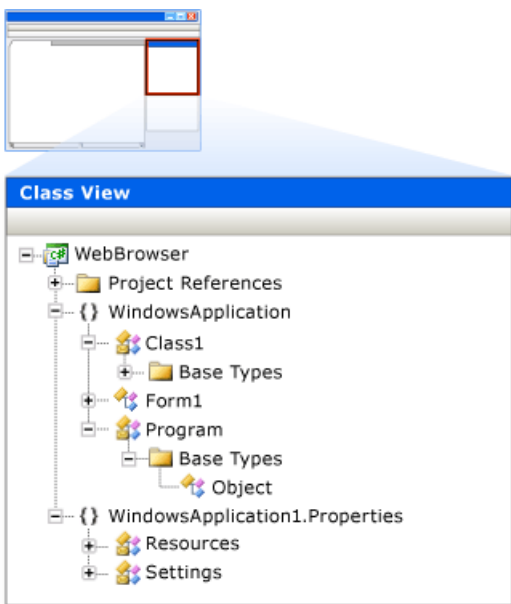
Navigating and Searching (Visual C#)

Visual C# provides the following tools to help you navigate and search through your source code, project files, and open windows.

- Class View
- Navigation Bars
- CTRL-TAB Navigation
- Find in Files

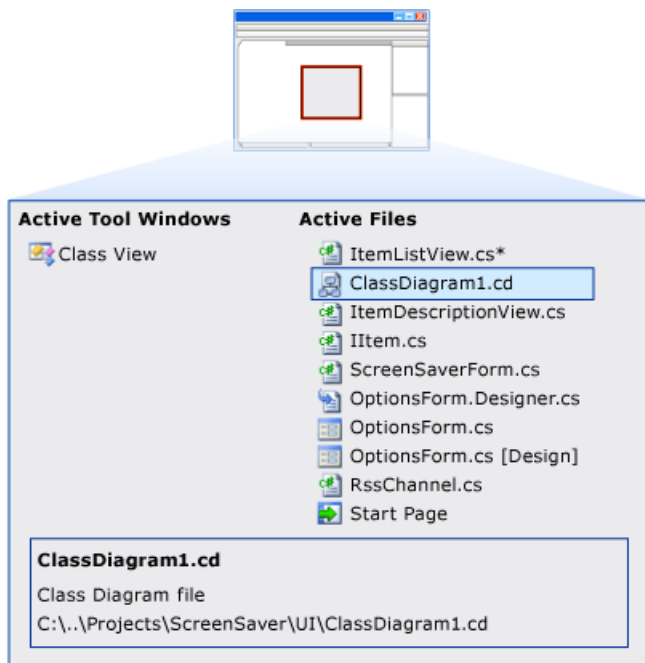
Class View

The **Class View** window provides a view of your project based on classes instead of files, as in **Solution Explorer**. You can use **Class View** to quickly navigate to any class or class member in your project. To access **Class View**, click **Class View** on the **View** menu.



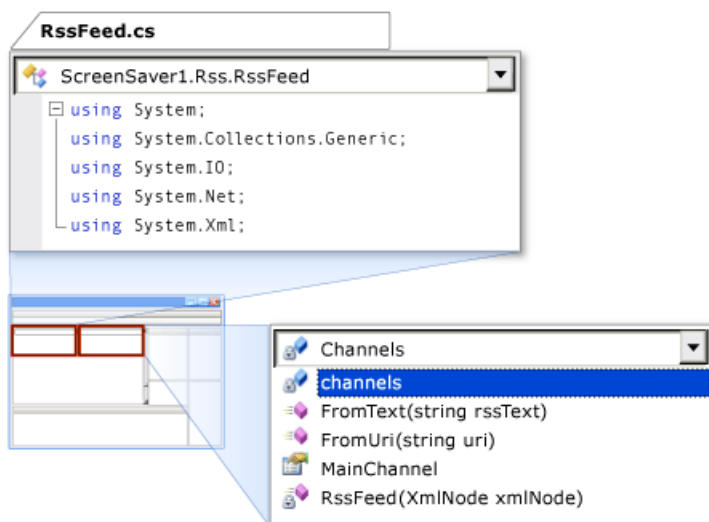
CTRL-TAB Navigation

At any given time you may have several active windows in a Visual C# project. To quickly navigate to a window, press CTRL+TAB to display a window that lists all of your active tools and source code windows. Move the arrow keys while holding down the CTRL key to select the window to display.



Navigation Bars

At the top of every code editor window is the navigation bar, which consists of two list boxes. The one on the left lists all the classes defined in the current file, and the one on the right lists all the members for the class that is selected in the left list box. You can go directly to a method by selecting it in the right list box.



Find in Files

By pressing CTRL+SHIFT+F you can open the **Find in Files** dialog box to perform search and replace operations across an entire project.

 Note
To rename methods or types, or change method parameters, use the Refactoring feature, which is more thorough and intelligent than Search and Replace. For more information, see Refactoring .

For More Information

- [How to: Navigate Code and Text](#)
- [How to: Outline and Hide Code](#)
- [How to: Search a Document Incrementally](#)

See Also

Other Resources

Building and Debugging (Visual C#)

In Visual C#, you build an executable application by clicking **Build** on the **Build** menu (or pressing CTRL+SHIFT+B). You can build and start the application in one operation by pressing F5 or clicking **Run** on the **Debug** menu.

Building involves inputting your project files into the C# compiler, which converts your source code into Microsoft intermediate language (MSIL) and then joins the MSIL with the metadata, resources, manifest and other modules, if there are any, to create an assembly. An assembly is an executable file that typically has an extension of .exe or .dll. As you develop your application, you will sometimes want to build a debug version of it in order to test it and see how it runs. Finally, when everything is correct, you will create a release version that you will deploy to customers.

For more information about assemblies, see [Assemblies Overview](#).

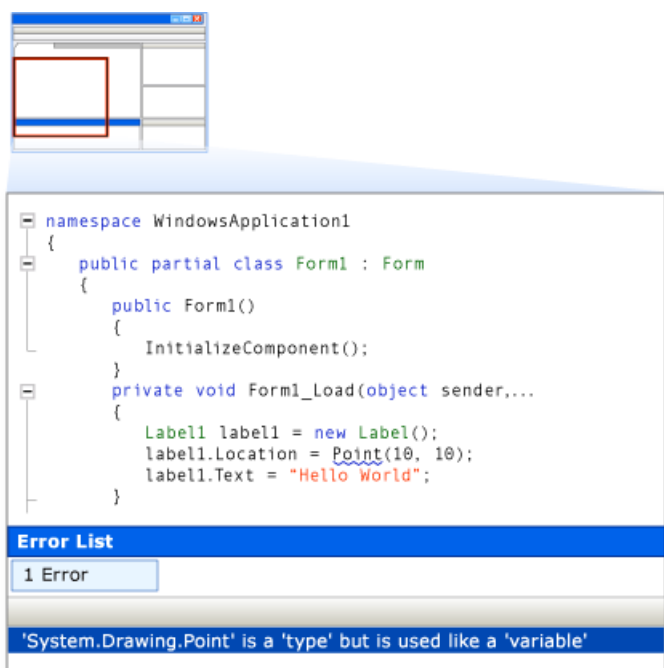
Build Settings

To specify various build settings, right-click the project item in **Solution Explorer** and then select the **Build** pane in the **Project Designer**. For more information, see [Introduction to the Project Designer](#) and [C# Compiler Options](#).

Visual Studio uses the MSBuild tool to create assemblies. MSBuild can also be run from the command line and can be customized in many ways. For more information, see [MSBuild](#).

Build Errors

If there are any errors in your C# syntax, or identifiers that cannot be resolved into a known type or member, then your build will not succeed and you will see a list of errors in the [Error List Window](#), which appears by default directly below the code editor. You can double-click on an error message to go to the line in your code where the error occurred.



C# compiler error messages are generally quite clear and descriptive, but if you cannot figure out the problem, you can go to the Help page for that message by pressing F1 with the error message selected in the error list. The Help page contains additional useful information. If you still cannot solve the problem, then the next step is to ask your question on one of the C# forums or newsgroups. To access the forums, click **Ask A Question** on the **Community** menu.

Note

If you encounter a compiler error Help page that was not helpful for your particular error, you can help Microsoft improve the documentation by sending a description of the problem. To send the e-mail, click the link at the bottom of the Help page that contains the error.

Release vs. Debug configurations

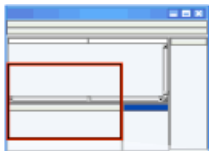
While you are still actively working on your project, you will typically build your application using the debug configuration, because this configuration enables you to view the value of variables and control execution in the debugger. You can also create and test builds in the release configuration to ensure that you have not introduced any bugs that only manifest on one

type of build or the other. In .NET Framework programming, such bugs are very rare, but they can happen.

When you are ready to distribute your application to end users, create a release build, which will be much smaller in size and will usually have much better performance than the corresponding debug configuration. You can set the build configuration in the **Build** pane of the **Project Designer**, or in the **Build** toolbar. For more information, see [Build Configurations](#).

Debugging

At any time you are working in the code editor, you can set a breakpoint on a line of code by pressing **F9**. When you press **F5** to run your application within the Visual Studio debugger, the application will stop on that line and you can examine the value of any given variable, or observe how or when execution breaks out of a loop, step through the code one line at a time by pressing **F10**, or set additional breakpoints.

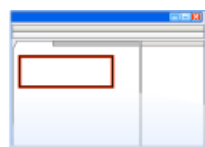


```
private void Form1_Load(object sender, EventArgs e)
{
    Label label1 = new Label();
    Point point = new Point(10, 10);
    label1.Location = point;
    label1.Text = "Hello World";
}
```

Name	Value
this	{WindowsApplication1.Form1, Text: Form1}
sender	{WindowsApplication1.Form1, Text: Form1}
e	{System.EventArgs}
label1	{System.Windows.Forms.Label, Text: }
point	{X = 10 Y = 10}
IsEmpty	false
X	10
Y	10
Static members	Static members
Non-Public members	Non-Public members

You can also set conditional breakpoints, which will only stop execution if a specified condition is met. Tracepoints are similar to breakpoints except that they do not stop execution, but simply write the value of a specified variable to the output window. For more information, see [Breakpoints and Tracepoints](#).

When execution is stopped on a breakpoint, you can hover over any variable in scope to view information about that variable. The following illustration shows a data tip in the debugger:



```
public Font TitleFont
{
    get
    {
        // Choose a font for the title text.
        // This font will be twice as big as the ItemFont
        float titleFontHeight = (float) (percentOfArti...
        if {titleFont == null || titleFont.Size !=title...
        {
            titleFont = new Font("Microsoft Sans Serif",...
        }
        titleFont {Name = "Microsoft Sans...
        return titleFont;
    }
}
```

You can step through your code one line at a time by pressing F10 after the debugger has stopped on a breakpoint. You can even fix certain kinds of errors in your code, and continue debugging without having to stop and recompile your application.

The Visual Studio debugger is a powerful tool and it is worthwhile to take the time to read the documentation in order to understand different concepts such as [Edit and Continue](#), [Viewing Data in the Debugger](#), [Visualizers](#), and [Just-In-Time Debugging](#).

See Also

Tasks

[How to: Set Debug and Release Configurations](#)

[How to: Debug Code in the Editor](#)

Reference

[System.Diagnostics](#)

Other Resources

[Visual C#](#)

[Using the Visual C# IDE](#)

[Debugging Preparation: C#, J#, and Visual Basic Project Types](#)

[Debug Settings and Preparation](#)

Modeling and Analyzing Code (Visual C#)

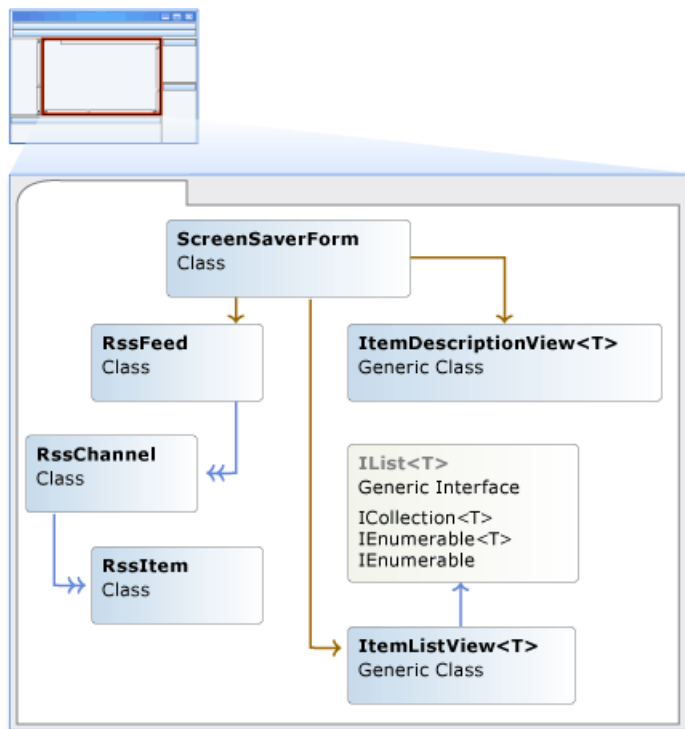
It is not uncommon for software developers to work with source code whose basic architecture is unfamiliar either because it was written by someone else, or because it was written so long ago that its original creators no longer recall completely how it works. Another common scenario is the need to understand the contents of a library that is only available in binary format. Visual C# provides the following tools to help you model, analyze, and understand types and type relationships in source code as well as in binary assemblies:

- Class Designer, for visually representing inheritance and association relationships between types.
- Object Browser, for examining the types, methods, and events exported by .NET Framework assemblies, and native DLLs including COM objects.
- Metadata as source, for viewing type information in managed assemblies as if it were source code in your own project.

In addition to the tools listed above, Visual Studio Team System includes the Code Analysis for Managed Code tool that inspects your code for a wide variety of potential problems.

Class Designer

Class Designer is a graphical tool for visually modeling the relationship between types in a software application or component; you can also use it to design new types and refactor or delete existing types. The following illustration shows a simple class design:

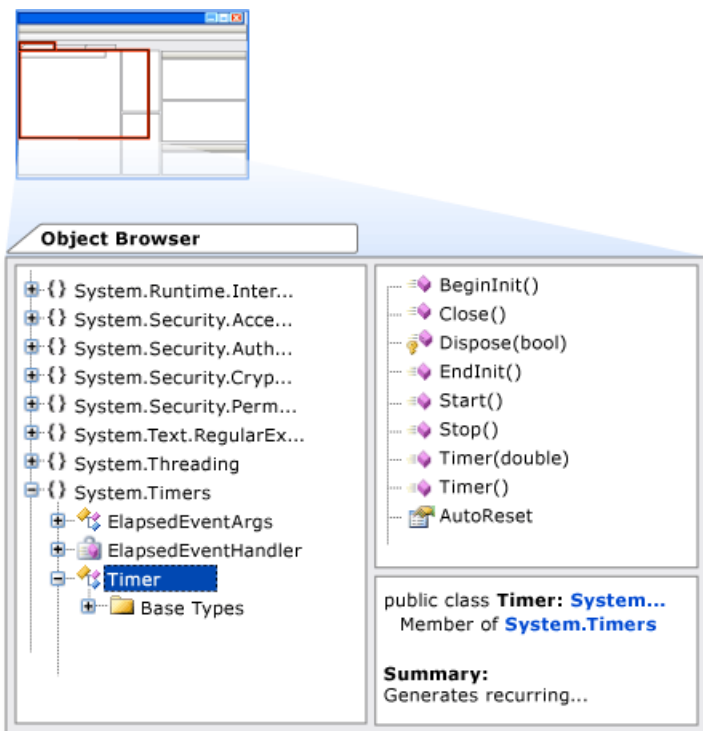


To add a class diagram to a project, click **Add New Item** on the **Project** menu, and then click **Add Class Diagram**.

For more information, see [Designing and Viewing Classes and Types](#).

Object Browser

The Object Browser enables you to view type information in both native and managed DLLs, including COM objects. Although the information you see in the Object Browser is similar to what you see in Class View, you can use Object Browser to examine any DLL on your system, not only the ones referenced in your own project. In addition, Object Browser also displays XML documentation comments for the selected type. The following illustration shows how Object Browser displays type information in binary files.



For more information, see [Object Browser](#)

Metadata as Source:

The Metadata As Source feature enables you to view type information for classes in managed assemblies as if it were source code in your own project. This is a convenient way to view the signatures for all the public methods in a class at a glance when you do not have access to the actual source code.

For example, if you enter the statement `System.Console.WriteLine()` in the code editor, place the insertion point within `Console` and then right-click and select **Go To Definition**, you will see what looks like a source code file containing the declaration of the `Console` class. This declaration is constructed from the metadata in the assembly using [Reflection](#), and although it does not expose the implementation of any methods, it does show any XML documentation comments that are present.

You can also use the Metadata As Source feature by selecting a managed type in the Object Browser, and clicking **Code Definition Window** on the **View** menu.

For more information and an illustration, see [Metadata as Source](#).

Code Analysis for Managed Code

The code analysis for managed code tool analyzes managed assemblies and reports information such as potential security problems, and violations of the programming and design rules set forth in the Microsoft .NET Framework Design Guidelines. This information is presented as warnings. You access the tool in the Project Designer by right-clicking **Properties** in **Solution Explorer**, and selecting **Open**.

For more information, see [Code Analysis, Project Designer](#) and [Code Analysis for Managed Code Overview](#).

See Also

Concepts

[Editing Code \(Visual C#\)](#)

[Reflection \(C# Programming Guide\)](#)

Other Resources

[Using the Visual C# IDE](#)

[Design Guidelines for Developing Class Libraries](#)

[Design Guidelines for Exceptions](#)

[Member Design Guidelines](#)

[Type Design Guidelines](#)

Adding and Editing Resources (Visual C#)

Visual C# applications often include data that is not source code. Such data is referred to as a *project resource* and it can include binary data, text files, audio or video files, string tables, icons, images, XML files, or any other type of data that your application requires. Project resource data is stored in XML format in the .resx file (named Resources.resx by default) which can be opened in **Solution Explorer**. For more information about project resources, see [Working with Resource Files](#).

Adding Resources to Projects

You can add resources to a project by right-clicking the **Properties** node under your project in **Solution Explorer**, clicking **Open**, and then clicking the **Add Resource** button on the **Resources** page in **Project Designer**.

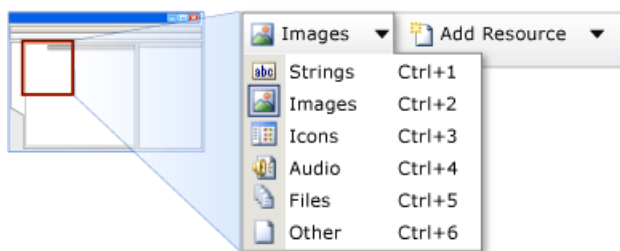
You can add resources to your project either as linked resources, which are external files, or as embedded resources, which are embedded directly into the .resx file.

- When you add a linked resource, the .resx file that stores your project resource information includes only a relative path to the resource file on disk. If you add images, videos, or other complex files as linked resources, you can edit them using a default editor that you associate with that file type in the Resource Designer.
- When you add an embedded resource, the data is stored directly in the project's resource (.resx) file. Strings can only be stored as embedded resources.

For more information, see [Linked vs. Embedded Resources](#) and [Resources in .Resx File Format](#).

Editing Resources

The Resource Designer enables you to add and modify project resources during development by associating a default application for editing each resource. You access the Resource Designer by right-clicking **Properties** in **Solution Explorer**, clicking **Open**, and then clicking the **Resources** tab in Project Designer. For more information, see [Resources Page, Project Designer](#). The following illustration shows the Resource Designer menu options:



To edit embedded resources, you must work directly in the .resx file to manipulate the individual characters or bytes. That is why it is more convenient to store complex file types as linked resources during development. You can use the [Binary Editor](#) to edit resource files, including the .resx file, at the binary level in either hexadecimal or ASCII format. You can use the [Image Editor](#) to edit icons and cursors as well as jpeg and GIF files that are stored as linked resources. You can also choose other applications as editors for these file types. For more information, see [Viewing and Editing Resources in a Resource Editor](#).

Compiling Resources into Assemblies

When you build your application, Visual Studio invokes the resgen.exe tool to convert your application resources into an internal class called `Resources`. This class is contained in the Resources.Designer.cs file which is nested under the Resources.resx file in **Solution Explorer**. The `Resources` class encapsulates all your project resources into static readonly **get** properties as a way of providing strongly-typed resources at run-time. When you build through the Visual C# IDE, all the encapsulated resource data, including both the resources that were embedded into the .resx file and the linked files, is compiled directly into the application assembly (the .exe or .dll file). In other words, the Visual C# IDE always uses the `/resource` compiler option. If you build from the command line, you can specify the `/linkresource` compiler option that will enable you to deploy resources in a separate file from the main application assembly. This is an advanced scenario and is only necessary in certain rare situations. A more common scenario for deploying resources separately from the main application assembly is to use satellite assemblies as discussed below.

Accessing Resources at Run-Time

To access a resource at run-time, simply reference it as you would any other class member. The following example shows how to retrieve a bitmap resource that you named Image01. Note that the Resources class is in a namespace called `<projectName>.Properties`, so you must either use the fully qualified name for each resource or else add the appropriate

[using](#) directive in the source file from which you are accessing the Resources class.

```
System.Drawing.Bitmap bitmap1 = myProject.Properties.Resources.Image01;
```

Internally the get property uses the [ResourceManager](#) class to create a new instance of the object.

For more information, see [Resources in Applications](#) and [Resource File Generator \(Resgen.exe\)](#).

Resources in Satellite Assemblies

If you are creating applications that will be localized (translated) into multiple languages, you can store each set of culture-specific strings as a resource in its own satellite assembly. When you distribute your application, you include the main application assembly along with any appropriate satellite assemblies. You can then add additional satellite assemblies or modify existing ones without recompiling the main application assembly. For more information, see [Creating Satellite Assemblies](#) and [Locating and Using Resources for a Specific Culture](#).

See Also

Concepts

[Introduction to the Project Designer](#)

Other Resources

[Visual C#](#)

[Getting Started with Visual C#](#)

[Assemblies in the Common Language Runtime](#)

[Globalizing and Localizing Applications](#)

Getting Help (Visual C#)

The Help documentation for Visual Studio is contained in the MSDN Library, which you can install locally on your own computer or network, and which is also available on the internet at <http://msdn.microsoft.com/library>. The local version of the library consists of a collection of compressed HTML files in the .hxs format. You can choose to install either all or part of the library on your machine; the complete MSDN installation is close to 2GB in size and it includes documentation for many Microsoft technologies. You can view both the local and the online MSDN documentation using the Visual Studio Help browser called Microsoft Document Explorer.

There are six ways to access Help while working in Visual C#:

- F1 Search
- Search
- Index
- Table of Contents
- How Do I
- Dynamic Help

Online vs. Local Help

On the **Help Options** property page under the **Options** menu, you can specify the following options for search behavior, including F1 search:

- Try the online MSDN library first, then your local documentation if no match is found.
- Try the local MSDN library first, then the online documentation if no match is found.
- Try only the local MSDN library.

These options will also appear the first time you invoke any search. The online MSDN documentation may contain more recent updates than your local documentation. Therefore, if you have an internet connection while working in Visual C#, it is recommended that you set the search option to try the online MSDN library first. From time-to-time, updates to local documentation may be made available for download. For more information on documentation updates, check the [Visual Studio Developer Center](#).

F1 Search

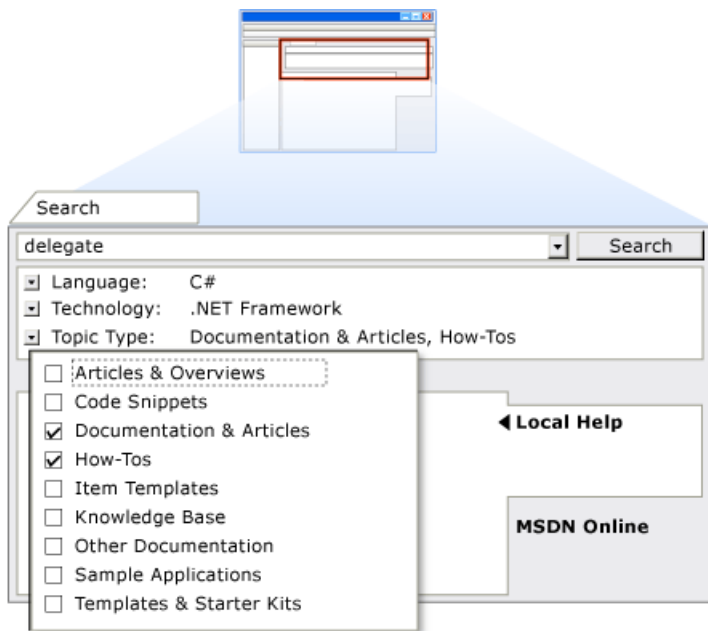
F1 provides context-sensitive search capabilities. In the code editor, you can access the Help documentation for C# keywords and .NET Framework classes by positioning the insertion point cursor on or just after the keyword or class member and pressing **F1**. When a dialog box or any other window has the focus, you can press **F1** to get Help for that window.

An **F1** search returns no more than one page. If no match is found, an informational page that provides some troubleshooting tips is displayed.

Search

Use the search interface to return all documents that match any specified term or set of terms.

The Search interface looks like this:



You can also use the **Help Options** page on the **Options** menu to specify whether you want to search Codezone Web sites in addition to the MSDN library. Codezone sites are run by Microsoft partners and provide useful information on C# and the .NET Framework. Codezone content is only available online.

The same online versus local search options apply to both Search and F1 Search.

In the Search interface you can narrow or broaden your search by specifying what kinds of documents to include. There are three options, Language, Technology, and Topic Type. You will generally obtain the best results by checking only those options that apply to your current development scenario.

Index

The index provides a quick way to locate documents in your local MSDN library. It is not a full-text search; it searches only the index keywords that have been assigned to each document. An index lookup is generally faster and more relevant than a full-text search. If more than one document contains the index keyword that you specify in the index search box, then a disambiguation window opens and enables you to select from among the possible choices.

The Index window is located by default on the left side of Document Explorer. You can access it from the Help menu in Visual C#.

Table of Contents

The MSDN library table of contents shows all the topics in the library in a hierarchical tree view structure. It is a useful tool for browsing through the documentation to get an idea of what is contained in the library, and for exploring documents that you might not find through F1, index, or search. Often, when you find a document through F1, index, or search, it is useful to know where it is located in the table of contents so that you can see what other related documentation exists for a given topic. Click the **Sync with Table of Contents** button in the Document Explorer toolbar to see where the currently displayed page is located in the MSDN library.

How Do I

How Do I is a filtered view of the MSDN library that mostly includes documents called How-to's or Walkthroughs that show you how to perform a specific task. You can access How Do I Help from the **Document Explorer** toolbar or the **Help** menu, or the Start page. Each language in Visual Studio has its own How Do I page, and the page that you see will depend on the type of project that is currently active.

Dynamic Help

The Dynamic Help window displays links to reference documentation for the .NET Framework and the C# language based on the current position of the insertion point in the code editor. For more information, see [How to: Customize Dynamic Help](#).

See Also

Other Resources

[Using the Visual C# IDE](#)

Deploying C# Applications

Deployment is the process by which you distribute a finished application or component to be installed on other computers. For console applications, or Smart Client applications based on Windows Forms, two deployment options are available: ClickOnce and Windows Installer.

ClickOnce Deployment

ClickOnce deployment allows you to publish Windows applications to a Web server or network file share for simplified installation. For most scenarios, ClickOnce is the recommended option for deployment because it enables self-updating Windows-based applications that can be installed and run with minimal user interaction.

To configure properties for ClickOnce deployment, you can use the [Publish Wizard](#) (accessible from the **Build** menu) or the **Publish** page in the **Project Designer**. For more information, see [Publish Page, Project Designer](#). For more information about ClickOnce, see [ClickOnce Deployment](#).

Windows Installer

Windows Installer deployment allows you to create installer packages to be distributed to users; the user runs the setup file and steps through a wizard to install the application. This is done by adding a Setup project to your solution; when built, it creates a setup file that you distribute to users; the user runs the setup file and steps through a wizard to install the application.

For more information about Windows Installer, see [Windows Installer Deployment](#).

See Also

Tasks

[How to: Publish a ClickOnce Application](#)

Concepts

[Deployment Alternatives](#)

[ClickOnce Deployment Overview](#)

[Deploying a Runtime Application Using Windows Installer](#)

[Setup Projects](#)

Other Resources

[Visual C#](#)

How to: Add Application Configuration Files to C# Projects

You can customize how the common language runtime locates and loads assembly files by adding application configuration files (app.config files) to your C# projects. For more information about application configuration files, see [How the Runtime Locates Assemblies](#).

When you build your project, the development environment automatically creates a copy of your app.config file, changes its file name so that it has the same file name as your executable, and then moves the new .config file in the bin directory.

To add an application configuration file to your C# project

1. On the **Project** menu, click **Add New Item**.

The **Add New Item** dialog box appears.

2. Select the **Application Configuration File** template and then click **Add**.

A file named app.config is added to your project.

See Also

Tasks

[How to: Use an Application Configuration File to Target a .NET Framework Version](#)

Visual C# Code Editor Features

Visual C# provides tools that help you edit and navigate through your code.

In This Section

[Refactoring](#)

Lists refactoring operations that help you modify your code without changing the behavior of your application.

[Code Snippets \(C#\)](#)

Provides an overview of using Code Snippets in Visual C# to automatically add common code constructs to your application.

[Code Colorization](#)

Describes how the Code Editor colorizes code constructs.

[Metadata as Source](#)

Describes how the IDE enables you to view metadata as source code.

Related Sections

[Editing Text, Code, and Markup](#)

Describes how to use the Code Editor to write and format code.

[Navigating Through Code](#)

Provides links to procedures for using the Find and Replace window, Bookmarks, and the Task List and Error List to locate lines of code.

[Visual C# IDE Settings](#)

Provides an overview of Visual Studio Settings for C# developers.

Refactoring

Refactoring is the process of improving your code after it has been written by changing the internal structure of the code without changing the external behavior of the code.

Visual C# provides the following refactoring commands on the Refactoring menu:

- [Extract Method](#)
- [Rename](#)
- [Encapsulate Field](#)
- [Extract Interface](#)
- [Promote Local Variable to Parameter](#)
- [Remove Parameters](#)
- [Reorder Parameters](#)

Multi-Project Refactoring

Visual Studio supports multi-project refactoring. All of the refactoring operations that correct references across files correct those references across all projects of the same language. This works for any project-to-project references. For example, if you have a console application that references a class library, when you rename a class library type (using the **Rename** refactoring operation), the references to the class library type in the console application are also updated.

Preview Changes Dialog Box

Many refactoring operations provide an opportunity for you to review all the reference changes that a refactoring operation would perform on your code, before committing to those changes. For these refactoring operations, a **preview reference changes** option will appear in the refactoring dialog box. After selecting that option and accepting the refactoring operation, the [Preview Changes Dialog Box](#) will appear. Notice that the **Preview Changes** dialog box has two views. The bottom view will display your code with all the reference updates due to the refactoring operation. Pressing **Cancel** on the **Preview Changes** dialog box will stop the refactoring operation, and no changes will be made to your code.

Error-Tolerant Refactoring

Refactoring is error tolerant. In other words, you can perform a refactoring in a project that cannot build. However, in these cases the refactoring process might not update ambiguous references correctly.

See Also

Tasks

[How to: Restore C# Refactoring Snippets](#)

Other Resources

[Visual C# Code Editor Features](#)

Extract Method

Extract Method is a [Refactoring](#) operation that provides an easy way to create a new method from a code fragment in an existing member.

Using **Extract Method**, you can create a new method by extracting a selection of code from inside the code block of an existing member. The new method is created containing the selected code, and the selected code in the existing member is replaced with a call to the new method. Turning a fragment of code into its own method gives you the ability to quickly and accurately reorganize code for better reuse and readability.

Extract Method has the following benefits:

- Encourages best coding practices by emphasizing discrete, reusable methods.
- Encourages self-documenting code through good organization. When descriptive names are used, high-level methods can read more like a series of comments.
- Encourages the creation of finer-grained methods to simplify overriding.
- Reduces code duplication.

Remarks

When you use the **Extract Method** command, the new method is inserted following the source member in the same class.

Partial Types

If the class is a partial type, then **Extract Method** generates the new method immediately following the source member.

Extract Method determines the signature of the new method, creating a static method when no instance data is referenced by the code within the new method.

Generic Type Parameters

When you extract a method that has an unconstrained generic type parameter, the generated code will not add the "ref" modifier to that parameter unless a value is assigned to it. If the extracted method will support reference types as the generic type argument, then you should manually add the "ref" modifier to the parameter in method signature.

Anonymous Methods

If you attempt to extract part of an anonymous method that includes a reference to a local variable that is either declared or referenced outside of the anonymous method, then Visual Studio will warn you about potential semantic changes. Specifically, the moments of when the value of the local variable is passed to the anonymous method will differ.

When an anonymous method uses the value of a local variable, the value is obtained at the moment the anonymous method is executed. When an anonymous method is extracted into another method, the value of the local variable is obtained at the moment of the call to the extracted method.

The following example illustrates this semantic change. If this code is executed, then **11** will be printed to the console. If you use Extract Method to extract the region of code that is marked by code comments into its own method and then execute the refactored code, then **10** will be printed to the console.

```
class Program
{
    delegate void D();
    D d;
    static void Main(string[] args)
    {
        Program p = new Program();
        int i = 10;
        /*begin extraction*/
        p.d = delegate { Console.WriteLine(i++); };
        /*end extraction*/
        i++;
        p.d();
    }
}
```

To work around this situation, make the local variables that are used in the anonymous method, fields of the class.

See Also

Tasks

[How to: Refactor Code with Extract Method](#)

Extract Method Dialog Box

Use this dialog box to specify the name of the new method that Extract Method will generate, and to verify that Extract Method will construct the new method signature as you intended.

To access this dialog box, select the **Extract Method** command from the **Refactor** menu. This dialog box is only available when the selection of code is valid for extracting into a method.

New method name

Type the name of the new method that Extract Method generates.

Preview method signature

Displays a preview of the new method signature. The signature is automatically determined by the refactoring engine based on the selected code, and it cannot be modified in this text box.

See Also

Tasks

[How to: Refactor Code with Extract Method](#)

Reference

[Extract Method](#)

How to: Refactor Code with Extract Method

The following procedure describes how to create a new method from a code fragment of an existing member. Use this procedure to perform the [Extract Method](#) refactoring operation.

To use Extract Method

1. Create a console application as described in the example below.

For more information, see [Console Application](#).

2. In the [Code Editor](#), select the code fragment you want to extract:

```
double area = PI * radius * radius.
```

3. Select **Extract Method** on the **Refactor** menu. The [Extract Method Dialog Box](#) appears.

You can also type the keyboard shortcut CTRL+R, CTRL+M to display the Extract Method Dialog Box.

You can also right-click the selected code, point to **Refactor** on the context menu, and then click **Extract Method** to display the Extract Method Dialog Box.

4. Specify a name for the new method in the **New Method Name** text box such as **CircleArea**. A preview of the new method signature displays under **Preview Method Signature**.

5. Click the **OK** button.

Example

To set up this example, create a console application named `ExtractMethod`, and then replace `Class1` with the following code. For more information, see [Console Application](#).

```
class A
{
    const double PI = 3.141592;

    double CalculatePaintNeeded(double paintPerUnit, double radius)
    {
        // Select any of the following:
        // 1. The entire next line of code.
        // 2. The right-hand side of the next line of code.
        // 3. Just "PI *" of the right-hand side of the next line
        //    of code (to see the prompt for selection expansion).
        // 4. All code within the method body.
        // ...Then invoke Extract Method.

        double area = PI * radius * radius;

        return area / paintPerUnit;
    }
}
```

See Also

Reference

[Extract Method](#)

Concepts

[Refactoring](#)

Rename

Rename is a refactoring operation that provides an easy way to rename identifiers for code symbols such as fields, local variables, methods, namespaces, properties, and types. Rename can be used to change the names in comments and in strings in addition to the declarations and calls of an identifier.

Note

When using [Source Control for Visual Studio](#), get the latest version of sources before attempting to perform Rename refactoring.

Rename refactoring is available from the following Visual Studio features:

Feature	Behavior of Refactoring in the Development Environment
Code Editor	In the Code Editor, rename refactoring is available when you position the cursor on the code symbol declaration. When the cursor is in this position, you can invoke the Rename command by typing the keyboard shortcut, or by selecting the Rename menu item from a smart tag, context menu, or the Refactor menu. When you select the Rename menu item, the Rename dialog box appears. For more information, see Rename Dialog Box and How to: Rename Identifiers .
Class View	When you select an identifier in Class View, rename refactoring is available from the context menu and Refactor menu.
Object Browser	When you select an identifier in Object Browser, rename refactoring is only available from the Refactor menu.
Property Grid of the Windows Forms Designer	In the Property Grid of the Windows Forms Designer, changing the name of a control will initiate a rename operation for that control. The Rename dialog box will not appear.
Solution Explorer	In Solution Explorer, a Rename command is available on the context menu. If the selected source file contains a class whose class name is the same as the file name, then you can use this command to simultaneously rename the source file and execute rename refactoring. For example, if you create a default Windows application and then rename Form1.cs to TestForm.cs, then the source file name Form1.cs will change to TestForm.cs and the class Form1 and all references to that class will be renamed to TestForm. Note The Undo command (CTRL+Z) will only undo rename refactoring within the code and will not change the file name back to the original name. If the selected source file does not contain a class whose name is the same as the file name, then the Rename command in Solution Explorer will only rename the source file and will not execute rename refactoring.

Rename Operations

When you execute **Rename**, the refactoring engine performs a rename operation specific for each code symbol, described in the following table.

Code Symbol	Rename Operation
Field	Changes the declaration and usages of the field to the new name.

Local variable	Changes the declaration and usages of the variable to the new name.
Method	Changes the name of the method and all references to that method to the new name.
Namespace	Changes the name of the namespace to the new name in the declaration, all using statements, and fully qualified names.  Note When renaming a namespace, Visual Studio also updates the Default Namespace property on the Application Page of the Project Designer. This property cannot be reset by selecting Undo from the Edit menu. To reset the Default Namespace property value, you must edit the property in the Project Designer.
Property	Changes the declaration and usages of the property to the new name.
Type	Changes all declarations and all usages of the type to the new name, including constructors and destructors. For partial types, the rename operation will propagate to all parts.

Remarks

When you Rename a member that either implements/overrides or is implemented/overridden by members in other types, Visual Studio displays a dialog box that says the rename operation will result in cascading updates. If you click continue, the refactoring engine recursively finds and renames all members in base and derived types that have implements/overrides relationship with the member being renamed.

The following code example contains members with implements/overrides relationship.

C#

```

interface IBase
{
    void Method();
}
public class Base
{
    public void Method()
    { }
    public virtual void Method(int i)
    { }
}
public class Derived : Base, IBase
{
    public new void Method()
    { }
    public override void Method(int i)
    { }
}
public class C : IBase
{
    public void Method()
    { }
}

```

In the example above, renaming `C.Method()` also renames `Ibase.Method()` because `C.Method()` implements `Ibase.Method()`. Next, the refactor engine recursively sees that `Ibase.Method()` is implemented by `Derived.Method()` and renames `Derived.Method()`. The refactor engine does not rename `Base.Method()`, because `Derived.Method()` does not override `Base.Method()`. The refactoring engine stops here unless you have **Rename overloads** checked in the **Rename** dialog box.

If **Rename overloads** is checked, the refactor engine renames `Derived.Method(int i)` because it overloads `Derived.Method()`, `Base.Method(int i)` because it is overridden by `Derived.Method(int i)`, and `Base.Method()` because it is

an overload of `Base.Method(int i)`.

Note

When you rename a member that was defined in a referenced assembly, a dialog box explains that renaming will result in build errors.

See Also

Tasks

[How to: Rename Identifiers](#)

Concepts

[Refactoring](#)

Rename Dialog Box

Use **Rename** to rename identifiers in your code for symbols such as fields, local variables, methods, namespaces, properties, and types.

New Name

Specifies the new name for an identifier, which is the code element that you intend to rename.

Location

Identifies the namespace to search when performing the rename operation.

Preview Reference Changes

Specifies to preview changes in the **Preview Changes — Rename** dialog box before your code is modified.

Search in comments

Specifies to search in comments when the check box is selected.

Search in strings

Specifies to search in strings when the check box is selected.

Rename overloads

Specifies to include method overloads in the refactoring operation. If checked, the refactoring engine only renames methods of the same name within the type - not in base or inherited types.

Note

This option is only available for methods.

Remarks

When comments and strings are searched by the **Rename** refactoring operation, the text is changed based on simple string matching in a global search and replace operation. It is recommended that you select **Preview Reference Changes** when **Search in comments** or **Search in strings** is selected.

When renaming a namespace, Visual Studio also updates the **Default Namespace** property on the [Application Page](#) of the **Project Designer**. This property cannot be reset by selecting **Undo** from the **Edit** menu. To reset the **Default Namespace** property value, you must edit the property in the **Project Designer**.

See Also

Tasks

[How to: Rename Identifiers](#)

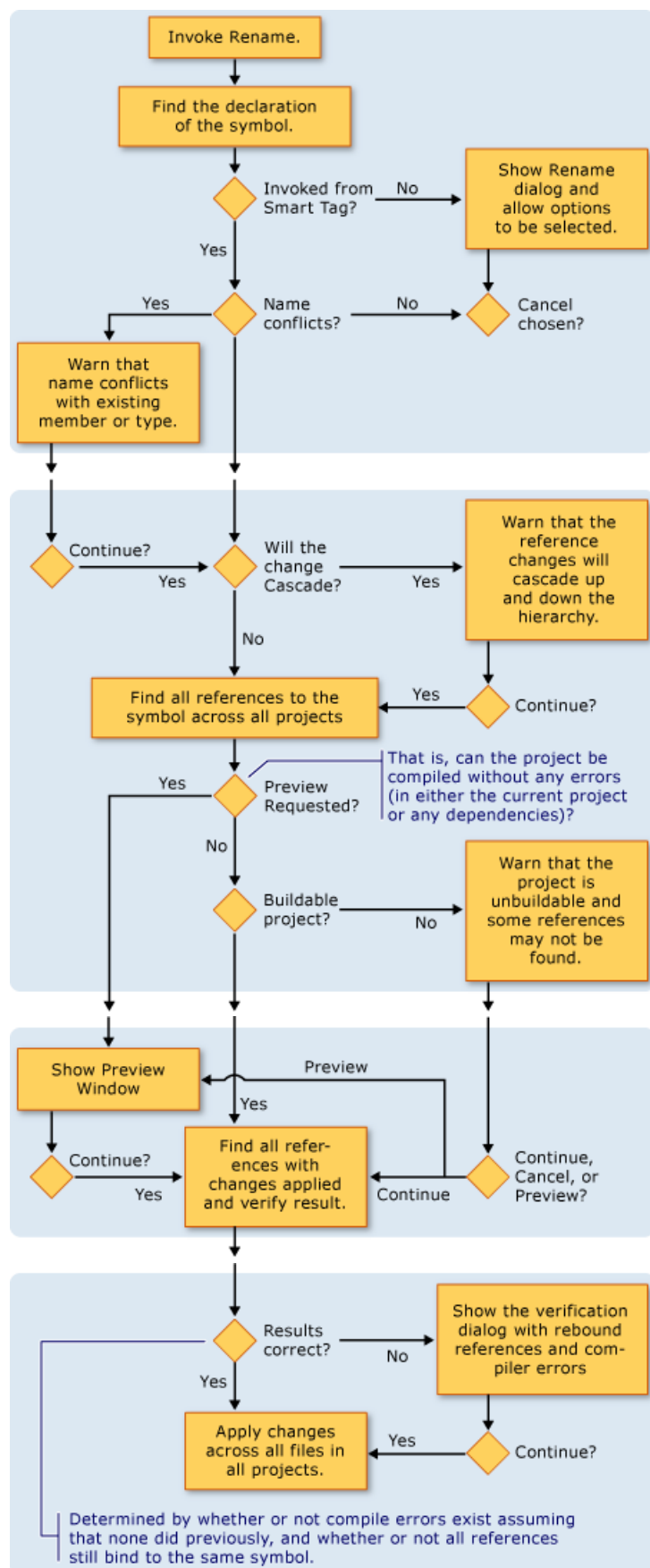
Reference

[Rename](#)

[Preview Changes Dialog Box](#)

Flow Diagram for Rename Refactoring

The following diagram illustrates the sequence of steps to complete the rename refactoring operation.



See Also
Reference
[Rename](#)
Concepts

How to: Rename Identifiers

The following procedure describes how to rename an identifier in your code. Use this procedure to perform the [Rename](#) refactoring operation.

To rename an identifier

1. Create a console application as described in the following Example section.

For more information, see [Console Application](#).

2. Place the cursor on `MethodB`, either in the method declaration or the method call.
3. From the **Refactor** menu, select **Rename**. The [Rename Dialog Box](#) appears.

You can also type the keyboard shortcut F2 to display the Rename Dialog Box.

You can also right-click the cursor, point to **Refactor** on the context menu, and then click **Rename** to display the Rename Dialog Box.

4. In the **New Name** field, type **MethodC**.
5. Select the **Search in Comments** check box.
6. Click **OK**.
7. In the **Preview Changes** dialog box, click **Apply**.

To rename an identifier using smart tags

1. Create a console application as described in the following Example section.

For more information, see [Console Application](#).

2. In the declaration for `MethodB`, type or backspace over the method identifier. A smart tag prompt will appear below this identifier.

Note

You can only invoke rename refactoring using smart tags at the declaration of an identifier.

3. Type the keyboard shortcut SHIFT+ALT+F10, and then press the DOWN ARROW to display the smart tag menu.

-or-

Move the mouse pointer over the smart tag prompt, to display the smart tag. Then move the mouse pointer over the smart tag and click the down arrow to display the smart tag menu.

4. Select the **Rename ' <identifier1> ' to ' <identifier2> '** menu item, to invoke rename refactoring without a preview of the changes to your code. All references to `<identifier1>` will automatically be updated to `<identifier2>`.

-or-

Select the **Rename with preview** menu item, to invoke rename refactoring with a preview of the changes to your code. The **Preview Changes** dialog box will appear.

Example

To set up this example, create a console application named `RenameIdentifier`, and then replace `Class1` with the following code. For more information, see [Console Application](#).

```
class ProtoClassA
{
    // Invoke on 'MethodB'.
    public void MethodB(int i, bool b) { }
}
```

```
class ProtoClassC
{
    void D()
    {
        ProtoClassA MyClassA = new ProtoClassA();

        // Invoke on 'MethodB'.
        MyClassA.MethodB(0, false);
    }
}
```

See Also

Reference

[Rename](#)

Concepts

[Refactoring](#)

Encapsulate Field

The **Encapsulate Field** refactoring operation enables you to quickly create a property from an existing field, and then seamlessly update your code with references to the new property.

When a field is [public \(C# Reference\)](#), other objects have direct access to that field and can modify it, undetected by the object owning that field. By encapsulating that field through the use of [properties](#), you can disallow direct access to fields.

To create the new property, the Encapsulate Field operation changes the access modifier for the field that you want to encapsulate to [private \(C# Reference\)](#), and then generates **get** and **set** accessors for that field. In some cases, only a **get** accessor is generated, such as when the field is declared read-only.

The refactoring engine updates your code with references to the new property in the areas specified in the **Update References** section of the [Encapsulate Field Dialog Box](#).

Remarks

The Encapsulate Field operation is only possible when the cursor is positioned on the same line as the field declaration.

For declarations that declare multiple fields, Encapsulate Field uses the comma as a boundary between fields, and initiates refactoring on the field that is nearest the cursor, on the same line as the cursor. You can also specify which field you want to encapsulate by selecting the name of that field in the declaration.

The code that is generated by this refactoring operation is modeled by the Encapsulate Field Code Snippets. Code Snippets are modifiable. For more information, see [How to: Manage Code Snippets](#).

For more information on when to use fields and when to use properties, see [Property Procedures vs. Fields](#).

See Also

Tasks

[How to: Refactor Code with Encapsulate Field](#)

Concepts

[Refactoring](#)

[Code Snippets \(C#\)](#)

Encapsulate Field Dialog Box

Use this dialog box to specify settings for the [Encapsulate Field](#) refactoring operation.

Field name

Identifies the current name of the field for which a new property is generated.

Property name

Specifies the name of the new property that Encapsulate Field generates. The refactoring operation automatically generates a unique property name for you. However, you can change this name to any valid identifier.

 **Note**

If you enter a name that is an invalid identifier or conflicts with an existing name, an error will appear and refactoring will not proceed.

Update references:

Specifies where the refactoring engine automatically updates your code with references to the new property.

Option	Description
External	Specifies that each reference to the field that is outside the enclosing type is replaced with a reference to the new property. Usages of the field within the enclosing type remain the same.
All	Specifies that every reference to the field is replaced with a reference to the new property.

 **Note**

Encapsulate Field will not update field references inside constructors.

Preview Reference Changes

Specifies that changes to the code will display in the **Preview Changes — Encapsulate field** dialog box before your code is modified.

Search in comments

Specifies that the refactoring engine will search code comments for references of the existing field to update.

Search in strings

Specifies that the refactoring engine will search string values for references of the existing field to update.

Remarks

When comments and strings are searched by the Encapsulate Field refactoring operation, the text is changed based on simple string matching in a global search and replace operation. To avoid errors, select **Preview Reference Changes** when **Search in comments** or **Search in strings** is selected.

See Also

Tasks

[How to: Refactor Code with Encapsulate Field](#)

Reference

[Preview Changes Dialog Box](#)

How to: Refactor Code with Encapsulate Field

The following procedure describes how to create a property from an existing field, and then update your code with references to the new property. Use this procedure to perform the [Encapsulate Field](#) refactoring operation.

To create a property from a field

1. Create a console application as described in the example below.

For more information, see [Console Application Template](#).

2. In the [Code and Text Editor](#), place the cursor in the declaration, on the name of the field that you want to encapsulate. In the example below, place the cursor on the word `width`:

```
public int width, height;
```

3. Select **Encapsulate Field** from the **Refactor** menu. The [Encapsulate Field Dialog Box](#) appears.

You can also type the keyboard shortcut CTRL+R, CTRL+F to display the Encapsulate Field Dialog Box.

You can also right-click the cursor, point to **Refactor** on the context menu, and then click **Encapsulate Field** to display the Encapsulate Field Dialog Box.

4. Specify settings.
5. Press ENTER, or click the **OK** button.
6. If you selected the **Preview reference changes** option, then the [Preview Reference Changes](#) window opens. Click the **Apply** button.

The following **get** and **set** accessor code is displayed in your source file:

```
public int Width
{
    get
    {
        return width;
    }

    set
    {
        width = value;
    }
}
```

The code in the `Main` method is also updated to the new `Width` property name.

```
Square mySquare = new Square();
mySquare.Width = 110;
mySquare.height = 150;
// Output values for width and height.
Console.WriteLine("width = {0}", mySquare.Width);
```

Example

To set up this example, create a console application named `EncapsulateFieldExample`, and then replace `Class1` with the following code. For more information, see [Console Application](#).

```
class Square
```

```
{
    // Select the word 'width' then use Encapsulate Field.
    public int width, height;
}
class MainClass
{
    public static void Main()
    {
        Square mySquare = new Square();
        mySquare.width = 110;
        mySquare.height = 150;
        // Output values for width and height.
        Console.WriteLine("width = {0}", mySquare.width);
        Console.WriteLine("height = {0}", mySquare.height);
    }
}
```

See Also

Reference

[Encapsulate Field](#)

Concepts

[Refactoring](#)

Extract Interface

Extract Interface is a refactoring operation that provides an easy way to create a new interface with members that originate from an existing class, struct, or interface.

When several clients use the same subset of members from a class, struct, or interface, or when multiple classes, structs, or interfaces have a subset of members in common, it can be useful to embody the subset of members in an interface. For more information about using interfaces, see [Interfaces \(C# Programming Guide\)](#).

Extract Interface generates an interface in a new file and positions the cursor at the beginning of the new file. You can specify which members to extract into the new interface, the name of the new interface, and the name of the generated file using the [Extract Interface Dialog Box](#).

Remarks

This feature is only accessible when the cursor is positioned in the class, struct, or interface that contains the members that you would like to extract. When the cursor is in this position, invoke the Extract Interface refactoring operation.

When you invoke extract interface on a class or on a struct, the bases and interfaces list is modified to include the new interface name. When you invoke extract interface on an interface, the bases and interfaces list is not modified.

See Also

Tasks

[How to: Refactor Code with Extract Interface](#)

Concepts

[Refactoring](#)

Extract Interface Dialog Box

Use this dialog box to specify settings for the [Extract Interface](#) refactoring operation. This dialog box is available from the **Refactor** menu.

New interface name

Specifies the name for the interface to be generated.

Generated name

Displays the name that is added to the bases and interfaces list of the type (class or struct) from which the interface was extracted.

New file name

Specifies the name of the new file that will contain the new interface.

Select public members to form interface

Lists all the valid members that you can use to populate the new interface. These include methods, indexers, properties, and events.

Select each member that you want to extract into the new interface. You can use the buttons at the right to **Select All** or **Deselect All** members to form the interface.

See Also

Tasks

[How to: Refactor Code with Extract Interface](#)

Concepts

[Refactoring](#)

How to: Refactor Code with Extract Interface

Use this procedure to perform the [Extract Interface](#) refactoring operation.

To use Extract Interface

1. Create a console application as described in the example below.

For more information, see [Console Application](#).

2. With the cursor positioned in MethodB, select **Extract Interface** on the **Refactor** menu. The [Extract Interface Dialog Box](#) appears.

You can also type the keyboard shortcut CTRL+R, CTRL+I to display the Extract Interface Dialog Box.

You can also right-click the cursor, point to **Refactor** on the context menu, and then click **Extract Interface** to display the Extract Interface Dialog Box.

3. Click **Select All**.
4. Click the **OK** button.

You see the new file, IProtoA.cs, and the following code:

```
using System;
namespace TopThreeRefactorings
{
    interface IProtoA
    {
        void MethodB(string s);
    }
}
```

Example

To set up this example, create a console application named `ExtractInterface`, and then replace `Class1` with the following code. For more information, see [Console Application](#).

```
// Invoke Extract Interface on ProtoA.
// Note: the extracted interface will be created in a new file.
class ProtoA
{
    public void MethodB(string s) { }
}
```

See Also

Reference

[Extract Interface](#)

Concepts

[Refactoring](#)

Promote Local Variable to Parameter

Promote Local Variable to Parameter is a Visual C# refactoring operation that provides an easy way to move a variable from a local usage to a method, indexer, or constructor parameter while updating the call sites correctly.

Perform the Promote Local Variable to Parameter operation by first positioning the cursor on the variable you wish to promote. The statement declaring the variable must also assign a value or expression to the variable. When the cursor is in position, invoke the Promote Local Variable to Parameter operation by typing the keyboard shortcut, or by selecting the command from the shortcut menu.

When you invoke the Promote Local Variable to Parameter operation, the variable is added to the end of the parameter list for the member. Any calls to the modified member are immediately updated with the new parameter as the expression originally assigned to the variable, leaving the code so that it functions the same as before the variable promotion. For more information, see [How to: Promote Local Variable to Parameter](#).

Remarks

This refactoring works best when the variable being promoted is assigned a constant value. The variable must be declared and initialized, not just a declaration nor just an assignment.

See Also

Tasks

[How to: Promote Local Variable to Parameter](#)

Concepts

[Refactoring](#)

How to: Promote Local Variable to Parameter

Use this procedure to perform the Promote Local Variable to Parameter refactoring operation. For more information, see [Promote Local Variable to Parameter](#).

To promote a local variable to parameter

1. Create a console application, and set it up as described in the following example. For more information, see [Console Application Template](#).
2. Place the cursor on `i` in `MethodB`.
3. From the **Refactor** menu, select **Promote Local Variable to Parameter**.

You can also type the keyboard shortcut CTRL+R, CTRL+P to complete the refactoring operation.

You can also right-click the cursor, point to **Refactor** on the context menu, and then click **Promote Local Variable to Parameter** to complete the refactoring operation.

The `MethodB` should now have a parameter `int i`, while the call `ProtoA.MethodB` will now pass `0` as a value.

Example

To set up this example, create a console application named `PromoteLocal`, and then replace `Class1` with the following code. For more information, see [Console Application Template](#).

```
class ProtoA
{
    public static void MethodB()
    {
        // Invoke on 'i'
        int i = 0;
    }
}

class ProtoC
{
    void MethodD()
    {
        ProtoA.MethodB();
    }
}
```

See Also

Concepts

[Refactoring](#)

Remove Parameters

Remove Parameters is a refactoring operation that provides an easy way to remove parameters from methods, indexers, or delegates. **Remove Parameters** changes the declaration; at any locations where the member is called, the parameter is removed to reflect the new declaration.

You perform the **Remove Parameters** operation by first positioning the cursor on a method, indexer, or delegate. When the cursor is in position, you can invoke the **Remove Parameters** operation by selecting it from the **Refactor** menu, typing the keyboard shortcut, or by selecting the command from a context menu.

When you invoke the **Remove Parameters** command, the **Remove Parameters** dialog box appears. For more information, see [Remove Parameters Dialog Box](#) or [How to: Remove Parameters](#).

Remarks

You can remove parameters from a method declaration or a method call. Position the cursor in the method declaration or delegate name and invoke **Remove Parameters**.

⚠Caution

Remove Parameters allows you to remove a parameter that is referenced within the body of the member, but it does not remove the references to that parameter in the method body. This can introduce build errors into your code. However, you can use the [Preview Changes Dialog Box](#) to review your code, before executing the refactoring operation.

If a parameter being removed is modified during the call to a method, the removal of the parameter will also remove the modification. For example, if a method call is changed from:

```
MyMethod(param1++, param2);
```

to

```
MyMethod(param2);
```

by the refactoring operation, **param1** will not be incremented.

See Also

Tasks

[How to: Remove Parameters](#)

Concepts

[Refactoring](#)

Remove Parameters Dialog Box

Use this dialog box to identify the parameters to remove during the [Remove Parameters](#) refactoring operation.

Parameters

Allows you to remove a parameter from a list.

Preview method signature

Displays the new method signature with the parameter removed.

Preview reference changes

Specifies to show the **Preview Changes — Remove parameters** dialog box when the check box is selected.

Remarks

If a parameter being removed is modified during the call to a method, the removal of the parameter will also remove the modification. For example, if a method call is changed from:

```
MyMethod(param1++, param2);
```

To:

```
MyMethod(param2);
```

by the refactoring operation, **param1** will not be incremented. Previewing reference changes is recommended.

See Also

Tasks

[How to: Remove Parameters](#)

Reference

[Remove Parameters](#)

[Preview Changes Dialog Box](#)

How to: Remove Parameters

Use this procedure to perform the **Remove Parameters** refactoring operation. For more information, see [Remove Parameters](#).

To remove parameters

1. Create a console application, and set up the following example.

For more information, see [Console Application Template](#).

2. Place the cursor on method `A`, either in the method declaration or the method call.
3. From the **Refactor** menu, select **Remove Parameters** to display the [Remove Parameters Dialog Box](#).

You can also type the keyboard shortcut CTRL+R, CTRL+V to display the Remove Parameters Dialog Box.

You can also right-click the cursor, point to **Refactor** on the context menu, and then click **Remove Parameters** to display the Remove Parameters Dialog Box.

4. Using the **Parameters** field, place the cursor on `int i`, then click the **Remove** button.
5. Click **OK**.
6. In the **Preview Changes — Remove Parameters** dialog box, click **Apply**.

Example

To set up this example, create a console application named `RemoveParameters`, and then replace `Class1` with the following code. For more information, see [Console Application Template](#).

```
class A
{
    // Invoke on 'A'.
    public A(string s, int i) { }
}

class B
{
    void C()
    {
        // Invoke on 'A'.
        A a = new A("a", 2);
    }
}
```

See Also

Reference

[Remove Parameters](#)

Concepts

[Refactoring](#)

Reorder Parameters

Reorder Parameters is a Visual C# refactoring operation that provides an easy way to change the order of the parameters for methods, indexers, and delegates. Reorder Parameters changes the declaration, and at any locations where the member is called, the parameters are rearranged to reflect the new order.

You can perform the Reorder Parameters operation by first positioning the cursor on a method, indexer, or delegate. When the cursor is in position, invoke the Reorder Parameters operation by typing the keyboard shortcut, or by selecting the command from a context menu.

When you invoke Reorder Parameters, the Reorder Parameters dialog box appears. For more information, see [Reorder Parameters Dialog Box](#) and [How to: Reorder Parameters](#).

Remarks

You can reorder parameters from a method declaration or a method call. Position the cursor in the method or delegate declaration but not in the body.

See Also

Tasks

[How to: Reorder Parameters](#)

Concepts

[Refactoring](#)

Reorder Parameters Dialog Box

Use this dialog box to specify the order of parameters for the [Reorder Parameters](#) refactoring operation.

Parameters

Allows you to move a parameter up or down in a list using the arrow buttons.

Preview method signature

Displays a facsimile of the method signature with the parameter reordered.

Preview reference changes

Specifies to show the **Preview Changes — Reorder Parameters** dialog box when the check box is selected.

See Also

Tasks

[How to: Reorder Parameters](#)

Reference

[Reorder Parameters](#)

[Preview Changes Dialog Box](#)

How to: Reorder Parameters

You can change the order of parameters for methods, indexers, constructors and delegates, and automatically update their call sites using the [Reorder Parameters](#) refactoring operation.

To Reorder Parameters

1. Create a [Class Library](#) and set it up as described in the example section below.
2. Place the cursor on `MethodB`, either in the method declaration or the method call.
3. On the **Refactor** menu, click **Reorder Parameters**.

— or —

Type the keyboard shortcut CTRL+R, CTRL+O to display the Reorder Parameters Dialog Box.

— or —

Right-click the cursor, point to **Refactor** on the context menu, and then click **Reorder Parameters** to display the Reorder Parameters Dialog Box.

The [Reorder Parameters Dialog Box](#) appears.

4. In the Reorder Parameters dialog box, select `int i` in the **Parameters** list. Then click the down button.

— or —

Drag `int i` below `bool b` in the **Parameters** list.

5. In the Reorder Parameters dialog box, click **OK**.

If the **Preview reference changes** option is selected in the Reorder Parameters dialog box, then the **Preview Changes - Reorder Parameters** dialog box will appear. It provides a preview of the changes in the parameter list for `MethodB` in both the signature and the method call.

- a. If the **Preview Changes - Reorder Parameters** dialog box appears, then click **Apply**.

In this example, the method declaration and all of the method call sites for `MethodB` are updated.

Example

To set up this example, create a class library named `ReorderParameters`, and then replace `Class1` with the following code.

```
class ProtoClassA
{
    // Invoke on 'MethodB'.
    public void MethodB(int i, bool b) { }
}

class ProtoClassC
{
    void D()
    {
        ProtoClassA MyClassA = new ProtoClassA();

        // Invoke on 'MethodB'.
        MyClassA.MethodB(0, false);
    }
}
```

See Also

Reference

[Reorder Parameters](#)

Concepts

[Refactoring](#)

Preview Changes Dialog Box

The **Preview Changes** dialog box enables you to review all the reference changes that a refactoring operation would perform on your code, before executing that refactoring operation.

Previewing reference changes is an optional refactoring process. To preview reference changes, check the **Preview reference changes** option, which is available in the following dialog boxes:

- [Rename Dialog Box](#)
- [Encapsulate Field Dialog Box](#)
- [Remove Parameters Dialog Box](#)
- [Reorder Parameters Dialog Box](#)

Note

Rename refactoring from the Property Grid of the Windows Forms Designer directly modifies your code. It does not display the **Rename** dialog box or the **Preview Changes** dialog box. For more information, see [Rename](#) refactoring.

<Refactor> <Code> as <RefactoredCode>

Shows where the code in your project will change. Code statements appear as nodes under source file nodes. When you select a code statement node, the corresponding reference appears refactored and highlighted in the **Preview code changes** text box.

Preview code changes

Displays your program, with all reference changes incorporated into your program by the refactoring operation.

See Also

Tasks

[How to: Rename Identifiers](#)

[How to: Refactor Code with Encapsulate Field](#)

[How to: Remove Parameters](#)

[How to: Reorder Parameters](#)

Refactoring Warning Dialog Box

This warning dialog box indicates that the compiler does not have a complete understanding of your program, and that it is possible that the refactoring engine might not update all the appropriate references. This warning dialog box also provides an opportunity for you to preview your code in the [Preview Changes Dialog Box](#) before you commit changes.

Note

If a method contains a syntax error (which the IDE indicates with a red wavy underline), then the refactoring engine will not update any references to an element within that method. The example below illustrates this behavior.

By default, if you execute a refactoring operation without previewing reference changes and a compilation error is detected in your program, then the development environment displays this warning dialog box.

If you execute a refactoring operation that has **Preview reference changes** enabled and a compilation error is detected in your program, then the development environment will display the following warning message at the bottom of the Preview Changes dialog box, in lieu of displaying the Refactoring Warning dialog box:

Your project or one of its dependencies does not currently build. References may not be updated.

This refactoring warning is only available for refactoring operations that provide the **Preview reference changes** option, which is available on the following refactoring dialog boxes:

- [Rename Dialog Box](#)
- [Encapsulate Field Dialog Box](#)
- [Remove Parameters Dialog Box](#)
- [Reorder Parameters Dialog Box](#)

Show this dialog every time

This option is selected by default. When it is selected, the **Refactoring Warning** dialog box continues to appear when compilation errors have been detected during a refactoring operation.

Clearing this check box disables this warning dialog box for future refactoring operations. If you clear this check box, then to re-enable this warning dialog box for future refactoring operations, select the **Warn if build errors exist when refactoring** option in the [Advanced, C#/J#, Text Editor, Options Dialog Box](#).

Continue

Continues the current refactoring operation without a preview of the reference changes.

Preview

Opens the [Preview Changes Dialog Box](#) so that you can preview your code.

Cancel

Cancels the current refactoring operation. No changes in your code will occur.

Example

The following code example illustrates where the refactoring engine will not update references. If you use refactoring to rename **example** to some other name, then the reference in **ContainsSyntaxError** will not be updated, where as the other two references will be updated.

```
public class Class1
{
    static int example;

    static void ContainsSyntaxError()
    {
        example = 20
    }
}
```



```
static void ContainsSemanticError()
{
    example = "Three";
}

static void ContainsNoError()
{
    example = 1;
}
}
```

See Also

Concepts

[Refactoring](#)

Verification Results Dialog Box

This dialog box appears when the refactoring engine detects compile errors or rebinding issues, during the verification process of refactoring.

Verification Results

Identifies the statements that would contain compile errors or rebinding issues as a result of a refactoring operation.

Preview Issues

Shows a preview of the reference changes (or refactored code) that would introduce compile errors or rebinding issues in your program.

Remarks

This dialog box will not appear in the following circumstances:

- Your code generates unresolved compile errors before starting a refactoring operation and no rebinding issues are introduced as a result of the refactoring operation.
- You clear all references in the **Preview Changes** dialog box.
- You click **Yes** when prompted with a warning dialog box that indicates that a naming conflict exists and that the refactoring operation will skip the verification process.

Rebinding Issues

A rebinding issue occurs when a refactoring operation inadvertently causes a code reference to bind to something different from what it was originally bound to. The **Verification Results** dialog box distinguishes the difference between two kinds of rebinding issues.

References whose definition will no longer be the renamed symbol

This kind of rebinding issue occurs when a reference no longer refers to a renamed symbol. For example, consider the following code:

```
class Example
{
    private int a;
    public Example(int b)
    {
        a = b;
    }
}
```

If you use refactoring to rename `a` to `b`, this dialog box appears. The reference to the renamed variable `a` now binds to the parameter that is passed to the constructor instead of binding to the field.

References whose definition will now become the renamed symbol

This kind of rebinding issue occurs when a reference that previously did not refer to the renamed symbol now does refer to the renamed symbol. For example, consider the following code:

```
class Example
{
    private static void Method(object a)
    {
    }
    private static void OtherMethod(int a)
    {
    }
    static void Main(string[] args)
    {
        Method(5);
    }
}
```

```
}
```

If you use refactoring to rename `OtherMethod` to `Method`, this dialog box appears. The reference in `Main` now refers to the overloaded method that accepts an **int** parameter instead of the overloaded method that accepts an **object** parameter.

See Also

Reference

[Preview Changes Dialog Box](#)

Concepts

[Refactoring](#)

Code Snippets (C#)

Visual Studio provides a new feature called code snippets. You can use code snippets to type a short alias, and then expand it into a common programming construct. For example, the **for** code snippet creates an empty **for** loop. Some code snippets are surround-with code snippets, which enable you to select lines of code, then choose a code snippet which will incorporate the selected lines of code. For example, selecting lines of code then activating the **for** code snippet creates a **for** loop with those lines of code inside the loop block. Code snippets can make writing program code quicker, easier, and more reliable.

Using Code Snippets

Code snippets are normally used in the Code Editor by typing a short name for the alias — a code snippet shortcut — then pressing TAB. The IntelliSense menu also offers an **Insert Code Snippet** menu command, providing a list of available code snippets for insertion into the Code Editor. You can activate the code snippet list by typing CTRL+K, then X. For more information, see [How to: Use Code Snippets \(C#\)](#) and [How to: Use Surround-with Code Snippets](#).

Once a code snippet has been chosen, the text of the code snippet is inserted automatically at the cursor position. At this point, any editable fields in the code snippet are highlighted in yellow, and the first editable field is selected automatically. The currently selected field is boxed in red. For example, in the **for** code snippet, the editable fields are the initializer variable (`i` by default) and the length expression (`length` by default).

When a field is selected, users can type a new value for the field. Pressing TAB cycles through the editable fields of the code snippet; pressing SHIFT+TAB cycles through them in reverse order. Clicking on a field places the cursor in the field, and double-clicking on a field selects it. When a field is highlighted, a tooltip might be displayed, offering a description of the field.

Only the first instance of a given field is editable; when that field is highlighted, the other instances of that field are outlined. When you change the value of an editable field, that field is changed everywhere it is used in the code snippet.

Pressing ENTER or ESC will cancel field editing and return the Code Editor to normal.

The default colors for editable code snippet fields can be changed by modifying the **Code Snippet Field** setting in the **Fonts and Colors** pane of the **Options** dialog box. For more information, see [How to: Change the Font Face, Size, and Colors Used in the Editor](#).

Creating Code Snippets

You can create and utilize custom code snippets, in addition to the code snippets that are included with Visual Studio by default. For more information on creating custom code snippets, see [Creating Code Snippets](#).

Note

For C# code snippets, the characters that are valid for specifying the **<Shortcut>** field are: alphanumeric characters, the number sign (#), the tilde character (~), the underscore character (_), and the en dash character (-).

For more information on code snippets that are included in Visual C# by default, see [Default Code Snippets](#).

See Also

Reference

[Code Snippet Picker](#)

Default Code Snippets

The Code Snippet Inserter inserts a code snippet at the cursor location, or inserts a surround-with code snippet around the currently selected code. The Code Snippet Inserter is invoked through the **Insert Code Snippet** or **Surround With** commands on the **IntelliSense** menu, or by using the keyboard shortcuts CTRL+K, then X and CTRL+K, then S respectively.

The Code Snippet Inserter displays the code snippet name for all of the available code snippets. The Code Snippet Inserter also includes an input dialog box where you can type the name of the code snippet, or part of the code snippet name. The Code Snippet Inserter highlights the closest match to a code snippet name. Pressing TAB at any time will dismiss the Code Snippet Inserter and insert the currently selected code snippet. Typing ESC or clicking the mouse in the Code Editor will dismiss the Code Snippet Inserter without inserting a code snippet.

Default Code Snippets

The following code snippets are included in Visual Studio by default.

Name (or shortcut)	Description	Valid locations to insert snippet
#if	Creates a #if directive and a #endif directive.	Anywhere.
#region	Creates a #region directive and a #endregion directive.	Anywhere.
~	Creates a destructor for the containing class.	Inside a class.
attribute	Creates a declaration for a class that derives from Attribute .	Inside a namespace (including the global namespace), a class, or a struct.
checked	Creates a checked block.	Inside a method, an indexer, a property accessor, or an event accessor.
class	Creates a class declaration.	Inside a namespace (including the global namespace), a class, or a struct.
ctor	Creates a constructor for the containing class.	Inside a class.
cw	Creates a call to WriteLine .	Inside a method, an indexer, a property accessor, or an event accessor.
do	Creates a do while loop.	Inside a method, an indexer, a property accessor, or an event accessor.
else	Creates an else block.	Inside a method, an indexer, a property accessor, or an event accessor.
enum	Creates an enum declaration.	Inside a namespace (including the global namespace), a class, or a struct.
equals	Creates a method declaration that overrides the Equals method defined in the Object class.	Inside a class or a struct.
exception	Creates a declaration for a class that derives from an exception (Exception by default).	Inside a namespace (including the global namespace), a class, or a struct.
for	Creates a for loop.	Inside a method, an indexer, a property accessor, or an event accessor.

foreach	Creates a foreach loop.	Inside a method, an indexer, a property accessor, or an event accessor.
forr	Creates a for loop that decrements the loop variable after each iteration.	Inside a method, an indexer, a property accessor, or an event accessor.
if	Creates an if block.	Inside a method, an indexer, a property accessor, or an event accessor.
indexer	Creates an indexer declaration.	Inside a class or a struct.
interface	Creates an interface declaration.	Inside a namespace (including the global namespace), a class, or a struct.
invoke	Creates a block that safely invokes an event.	Inside a method, an indexer, a property accessor, or an event accessor.
iterator	Creates an iterator.	Inside a class or a struct.
iterindex	Creates a "named" iterator and indexer pair by using a nested class.	Inside a class or a struct.
lock	Creates a lock block.	Inside a method, an indexer, a property accessor, or an event accessor.
mbox	Creates a call to System.Windows.Forms.MessageBox.Show . You may need to add a reference to System.Windows.Forms.dll.	Inside a method, an indexer, a property accessor, or an event accessor.
namespace	Creates a namespace declaration.	Inside a namespace (including the global namespace).
prop	Creates a property declaration and a backing field.	Inside a class or a struct.
propg	Creates a property declaration with only a "get" accessor and a backing field.	Inside a class or a struct.
sim	Creates a static int Main method declaration.	Inside a class or a struct.
struct	Creates a struct declaration.	Inside a namespace (including the global namespace), a class, or a struct.
svm	Creates a static void Main method declaration.	Inside a class or a struct.
switch	Creates a switch block.	Inside a method, an indexer, a property accessor, or an event accessor.
try	Creates a try-catch block.	Inside a method, an indexer, a property accessor, or an event accessor.
tryf	Creates a try-finally block.	Inside a method, an indexer, a property accessor, or an event accessor.
unchecked	Creates an unchecked block.	Inside a method, an indexer, a property accessor, or an event accessor.

unsafe	Creates an unsafe block.	Inside a method, an indexer, a property accessor, or an event accessor.
using	Creates a using directive.	Inside a namespace (including the global namespace).
while	Creates a while loop.	Inside a method, an indexer, a property accessor, or an event accessor.

Remarks

Shortcuts enable IntelliSense to automatically fill in code snippets in the Code Editor without using a menu. For more information, see [How to: Use Code Snippets \(C#\)](#).

See Also

Tasks

[How to: Use Surround-with Code Snippets](#)

Reference

[Code Snippet Picker](#)

Concepts

[Code Snippets \(C#\)](#)

How to: Use Code Snippets (C#)

The following procedures describe how to use code snippets. Code snippets are available in five ways: through a keyboard shortcut, through IntelliSense auto-completion, through the IntelliSense complete word list, through the Edit menu, and through the context menu.

To use code snippets through keyboard shortcut

1. In the Visual Studio IDE, open the file that you intend to edit.
2. In the Code Editor, place the cursor where you would like to insert the code snippet.
3. Type CTRL+K, CTRL+X.
4. Select the code snippet from the code snippet inserter and then press TAB or ENTER.

Alternatively, you can type the name of the code snippet, and then press TAB or ENTER.

To use code snippets through IntelliSense auto-completion

1. In the Visual Studio IDE, open the file that you intend to edit.
2. In the Code Editor, place the cursor where you would like to insert the code snippet.
3. Type the shortcut for the code snippet that you want to add to your code.
4. Type TAB, TAB to invoke the code snippet.

To use code snippets through the IntelliSense Complete Word list

1. In the Visual Studio IDE, open the file that you intend to edit.
2. In the Code Editor, place the cursor where you would like to insert the code snippet.
3. Begin typing the shortcut for the code snippet that you want to add to your code. If automatic completion is turned on, then the IntelliSense complete word list will be displayed. If it does not appear, then press CTRL+SPACE to activate it.
4. Select the code snippet from the complete word list.
5. Type TAB, TAB to invoke the code snippet.

To use code snippets through the Edit menu

1. In the Visual Studio IDE, open the file that you intend to edit.
2. In the Code Editor, place the cursor where you would like to insert the code snippet.
3. From the **Edit** menu, select **IntelliSense** and then select the **Insert Snippet** command.
4. Select the code snippet from the code snippet inserter and then press TAB or ENTER.

Alternatively, you can type the name of the code snippet, and then press TAB or ENTER.

To use code snippets through the context menu

1. In the Visual Studio IDE, open the file that you intend to edit.
2. In the Code Editor, place the cursor where you would like to insert the code snippet.
3. Right-click the cursor and then select the **Insert Snippet** command from the context menu.
4. Select the code snippet from the code snippet inserter and then press TAB or ENTER.

Alternatively, you can type the name of the code snippet, and then press TAB or ENTER.

See Also

Tasks

[How to: Use Surround-with Code Snippets](#)

Reference

[Default Code Snippets](#)

[Code Snippet Picker](#)

Concepts

[Code Snippets \(C#\)](#)

How to: Use Surround-with Code Snippets

The following procedures describe how to use surround-with code snippets. Surround-with code snippets are available three ways: through a keyboard shortcut, through the Edit menu, and through the context menu.

To use surround-with code snippets through keyboard shortcut

1. In the Visual Studio IDE, open the file that you intend to edit.
2. In the Code Editor, select text to surround.
3. Type CTRL+K, CTRL+S.
4. Select the code snippet from the code snippet list using the mouse, or by typing the name of the code snippet and pressing TAB or ENTER.

To use surround-with code snippets through the Edit menu

1. In the Visual Studio IDE, open the file that you intend to edit.
2. In the Code Editor, select text to surround.
3. From the **Edit** menu, select **IntelliSense** and then select the **Surround With** command.
4. Select the code snippet from the code snippet inserter and then press TAB or ENTER.

Alternatively, you can type the name of the code snippet, and then press TAB or ENTER.

To use surround-with code snippets through the context menu

1. In the Visual Studio IDE, open the file that you intend to edit.
2. In the Code Editor, select text to surround.
3. Right-click the selected text and then select the **Surround With** command from the context menu.
4. Select the code snippet from the code snippet inserter and then press TAB or ENTER.

Alternatively, you can type the name of the code snippet, and then press TAB or ENTER.

See Also

Tasks

[How to: Use Code Snippets \(C#\)](#)

Reference

[Default Code Snippets](#)

[Code Snippet Picker](#)

Concepts

[Code Snippets \(C#\)](#)

How to: Restore C# Refactoring Snippets

C# refactoring operations rely on code snippets found in the following directory:

Installation directory\Microsoft Visual Studio 8\VC#\Snippets\language ID\Refactoring

If this Refactoring directory, or any files in this directory are deleted or corrupted, then C# refactoring operations may not work in the IDE. The following procedures can help you restore C# refactoring code snippets.

To verify C# refactoring snippets are available through the Code Snippet Manager

1. In the **Tools** menu, select **Code Snippet Manager**.
2. In the **Code Snippet Manager** dialog box, select **Visual C#** from the **Language** drop-down list.
A **Refactoring** folder should appear in the tree view folder list.

To restore refactoring snippets in Code Snippet Manager

1. In the **Tools** menu, select **Code Snippet Manager**.
2. In the **Code Snippet Manager** dialog box, select **Visual C#** from the **Language** drop-down list.
3. Click **Add**. The **Code Snippets Directory** dialog box, which helps you locate and specify the directory to add back into the Code Snippet Manager, appears.
4. Locate the **Refactoring** folder whose directory path is:
Installation directory\Microsoft Visual Studio 8\VC#\Snippets\language ID\Refactoring
5. Click **Open** in the **Code Snippets Directory** dialog box, and then click OK in the Code Snippets Manager.

To repair Refactoring code snippets directory

1. In the **Code Snippet Manager** dialog box, click **Search Online**.
2. Enter refactoring, and then click **Search**.

Search results should include a website that allows you to download the .vsi file that you can use to reinstall the Refactoring folder.

See Also

Reference

[Code Snippets Manager](#)

Concepts

[Refactoring](#)

Code Colorization

The code editor parses tokens and code constructs so they are easily recognizable and distinguishable from other code content in the code editor. After the code editor parses your code, it colorizes code constructs appropriately.

Tokens

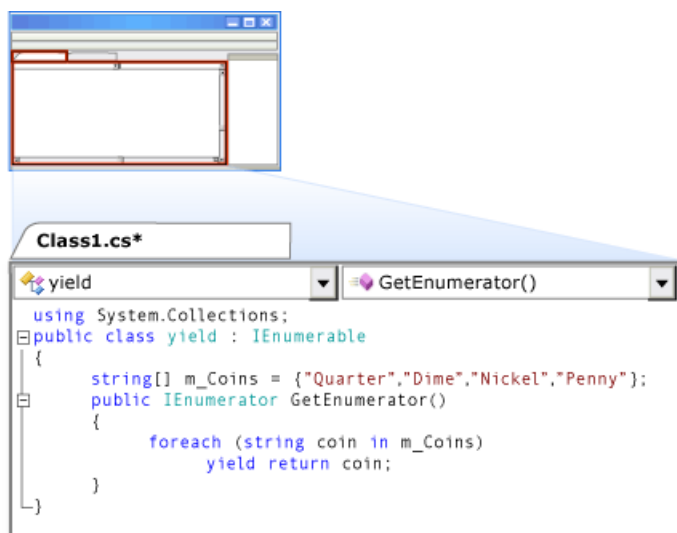
The code editor colorizes the following token types.

- Comment
- Excluded Code
- Identifier
- Keyword
- Number
- Operator
- Preprocessor Keyword
- String
- String (C# @ Verbatim)
- User Types
- User Types (Value Types)
- User Types (Enums)
- User Types (Delegates)
- XML CData Section
- XML Doc Attribute
- XML Doc Comment
- XML Doc Tag

You can modify the default colorization setting using the [Fonts and Colors, Environment, Options Dialog Box](#).

Contextual Keywords

The code editor colorizes contextual keywords appropriately. In the following example, the type **yield** is colorized teal, while the keyword **yield** is colorized blue.



Brace Matching Colorization

The code editor facilitates bold colorization or highlight colorization for brace matching.

Bold Colorization

When you edit any of the following code construct pairs, the string or code construct pairs are briefly displayed in bold to indicate an association between them:

" "	A string
@" "	A verbatim string
#if, #endif	Preprocessor directives for conditional sections
#region, #endregion	Preprocessor directives for conditional sections
case, break	Control statement keywords
default, break	Control statement keywords
for, break	Evaluation expression keywords
for, continue	Evaluation expression keywords
foreach, break	Evaluation expression keywords
foreach, continue	Evaluation expression keywords
while, break	Evaluation expression keywords
while, continue	Evaluation expression keywords

You can disable this feature by unselecting the **Automatic delimiter highlighting** property in the [General, Text Editor, Options Dialog Box](#).

Highlight Colorization

When the cursor is positioned immediately before a starting delimiter, or immediately after an ending delimiter, gray rectangles appear to highlight both starting and ending delimiters to indicate an association between them. This feature is available for the following matching pairs:

{ }	braces
[]	brackets
()	parenthesis

Example

To illustrate brace matching colorization, type (do not copy and paste) the following code in the code editor.

```
class A
{
    public A()
    {
        if(true)
            int x =0;
        else
            int x =1;
    }
}
```

Colorization Settings

Colorization settings are persisted through [Visual Studio Settings](#).

See Also

Reference

[Automatic Brace Matching](#)

Metadata as Source

Metadata as source enables you to view metadata that appears as C# source code in a read-only buffer. This enables a view of the declarations of the types and members (with no implementations). You can view metadata as source by running the **Go To Definition** command for types or members whose source code is not available from your project or solution.

Note

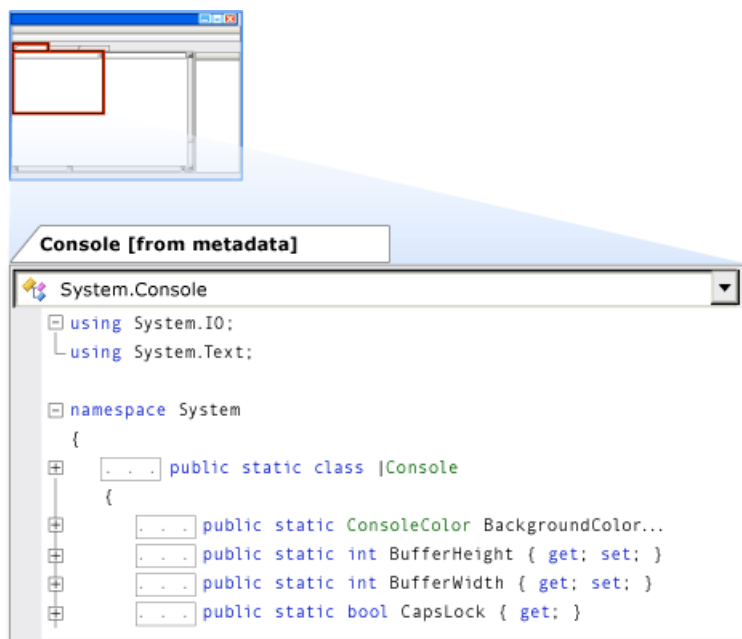
When you attempt to run the **Go To Definition** command for types or members that are marked as internal, the integrated development environment (IDE) does not display their metadata as source, regardless of whether the referencing assembly is a friend or not.

You can view metadata as source in either the Code Editor or the **Code Definition** window.

Viewing Metadata as Source in the Code Editor

When you run the **Go To Definition** command for an item whose source code is unavailable, a tabbed document that contains a view of that item's metadata, displayed as source, appears in the Code Editor. The name of the type, followed by **[from metadata]**, appears on the document's tab.

For example, if you run the **Go To Definition** command for [Console](#), metadata for **Console** appears in the Code Editor as C# source code that looks like its declaration, but with no implementation.



Viewing Metadata as Source in the Code Definition Window

When the **Code Definition** window is active or visible, the IDE automatically executes the **Go To Definition** command for items under the cursor in the Code Editor and for items that are selected in **Class View** or the **Object Browser**. If the source code is not available for that item, the IDE displays the item's metadata as source in the **Code Definition** window.

For example, if you put your cursor inside the word **Console** in the Code Editor, metadata for **Console** appears as source in the **Code Definition** window. The source looks something like the **Console** declaration, but with no implementation.

If you want to see the declaration of an item that appears in the **Code Definition** window, you must explicitly use the **Go To Definition** command because the **Code Definition** window is only one level deep.

See Also

Reference

[Code Definition Window](#)

[Find Symbol Results Window](#)

Visual C# Keyboard Shortcuts

Visual C# provides a number of keyboard shortcuts that you can use to perform actions without using the mouse or menus.

In This Section

[Shortcut Keys](#)

[General Development Settings](#) [Default Shortcut Keys](#)

See Also

Other Resources

[Visual C#](#)

[Using the Visual C# IDE](#)

Visual C# IDE Settings

Visual C# settings are a predefined configuration of tool windows, menus, and keyboard shortcuts. These settings are part of the [Visual Studio Settings](#) feature, which you can customize to fit your work habits.

Windows and Views

Feature	Displayed by default?	Notes
Class View	No	- Class View is available on the View menu. - Filtering is enabled.
Command Window	No	
Dynamic Help Window	No	Pressing the F1 key does not display the Dynamic Help window. For more information on the Dynamic Help Window, see How to: Customize Dynamic Help or How to: Control the Dynamic Help Window .
Object Browser	No	Does not display inherited members by default.
Output Window	No	
Solution Explorer	Yes	Solution Explorer appears docked on the right side of the IDE.
Start Page	Yes, when you start the IDE	The Start Page displays articles from the MSDN RSS feed for Visual C#.
Task List (Visual Studio)	No	
Toolbox	Yes, when you create a Windows Forms application	The Toolbox appears as a collapsed window that is docked on the left side of the IDE.

Keyboard

Feature	Behavior
Shortcut Keys	Visual C# supports these shortcut key settings: Visual C# 2005 Default Shortcut Keys Brief Default Shortcut Keys Emacs Default Shortcut Keys Visual C++ 2.0 Default Shortcut Keys Visual Studio 6.0 Default Shortcut Keys

See Also

Other Resources

[Visual C#](#)

[Using the Visual C# IDE](#)

Visual C# 2005 Default Shortcut Keys

The integrated development environment (IDE) provides several pre-defined keyboard binding schemes. To switch to the **Visual C# 2005** keyboard mapping scheme, on the **Tools** menu, click **Options**, expand **Environment**, and then click **Keyboard**.

Also, the Visual C# 2005 keyboard mapping scheme is the default keyboard mapping scheme when you select **Visual C# Development Settings** in the **Import and Export Settings Wizard**. For more information, see [How to: Change Select Settings](#).

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

The following information explains the default key combinations available for the **Visual C# 2005** keyboard mapping scheme.

- [Global Shortcut Keys, Visual C# 2005 Scheme](#)
- [HTML Designer Shortcut Keys, Visual C# 2005 Scheme](#)
- [XML Designer Shortcut Keys, Visual C# 2005 Scheme](#)
- [Control Manipulation Shortcut Keys, Visual C# 2005 Scheme](#)
- [Debugging Shortcut Keys, Visual C# 2005 Scheme](#)
- [Search and Replace Shortcut Keys, Visual C# 2005 Scheme](#)
- [Data Shortcut Keys, Visual C# 2005 Scheme](#)
- [Text Navigation Shortcut Keys, Visual C# 2005 Scheme](#)
- [Text Selection Shortcut Keys, Visual C# 2005 Scheme](#)
- [Text Manipulation Shortcut Keys, Visual C# 2005 Scheme](#)
- [Window Management Shortcut Keys, Visual C# 2005 Scheme](#)
- [Integrated Help Shortcut Keys, Visual C# 2005 Scheme](#)
- [Object Browser Shortcut Keys, Visual C# 2005 Scheme](#)
- [Macro Shortcut Keys, Visual C# 2005 Scheme](#)
- [Tool Window Shortcut Keys, Visual C# 2005 Scheme](#)
- [Project Shortcut Keys, Visual C# 2005 Scheme](#)
- [Image Editor Shortcut Keys, Visual C# 2005 Scheme](#)
- [Dialog Editor Shortcut Keys, Visual C# 2005 Scheme](#)
- [Refactoring Shortcut Keys, Visual C# 2005 Scheme](#)
- [Managed Resources Editor Shortcut Keys, Visual C# 2005 Scheme](#)
- [Code Snippet Shortcut Keys, Visual C# 2005 Scheme](#)
- [Class Diagram Shortcut Keys, Visual C# 2005 Scheme](#)
- [Bookmark Window Shortcut Keys, Visual C# 2005 Scheme](#)
- [Accelerator and String Editor Shortcut Keys, Visual C# 2005 Scheme](#)

See Also

Tasks

[How to: Work with Shortcut Key Combinations](#)

Other Resources

[Shortcut Keys](#)

Global Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used in various locations in the integrated development environment (IDE).

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Diagram.Properties	ALT + ENTER	Switches focus from the diagram to the Properties window.
Edit.Copy	CTRL + C	Copies the selected item to the Clipboard.
Edit.Cut	CTRL + X	Deletes the selected item from the file and copies it to the Clipboard.
Edit.CycleClipboardRing	CTRL + SHIFT + V	Pastes an item from the Clipboard ring to the insertion point in the file and automatically selects the pasted item. You can review each item on the Clipboard ring by repeatedly pressing the shortcut keys.
Edit.Delete	DELETE	Deletes one character to the right of the insertion point.
Edit.OpenFile	CTRL + SHIFT + G	Displays the Open File dialog box where you can select a file to open.
Edit.Paste	CTRL + V	Inserts the Clipboard contents at the insertion point.
Edit.Redo	CTRL + Y	Restores the previously undone action.
Edit.Undo	CTRL + Z	Reverses the last editing action.
File.Print	CTRL + P	Displays the Print dialog box where you can select printer settings.
File.SaveAll	CTRL + SHIFT + S	Saves all documents in the current solution and all files in the external files project.
File.SaveSelectedItems	CTRL + S	Saves the selected items in the current project.
Tools.GoToCommandLine	CTRL + /	Puts the pointer in the Find/Command box on the Standard toolbar.
View.Backward	ALT + LEFT ARROW	Displays the previous page in the viewing history. Available only in the Web Browser window.
View.EditLabel	F2	Lets you change the name of the selected item in Solution Explorer .
View.Forward	ALT + RIGHT ARROW	Displays the next page in the viewing history. Available only in the Web Browser window.

View.ViewCode	F7	Displays the selected item in Code view of the editor.
View.ViewDesigner	SHIFT + F7	Displays the selected item in Designer view

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Other Resources

[Shortcut Keys](#)

HTML Designer Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used only when you modify files in the HTML Designer. Certain key combinations are available only in a specific view of that designer. Other key combinations that you can use in the HTML Designer include [Text Navigation Shortcut Keys](#), [General Development Settings](#), [Text Selection Shortcut Keys](#), [\[Default Settings\] Scheme](#), and [Text Manipulation Shortcut Keys](#), [\[Default Settings\] Scheme](#).

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Format.Bold	CTRL + B	Toggles the selected text between bold and plain. Available only in Design view.
Format.ConvertToHyperlink	CTRL + L	When text is selected, displays the Hyperlink dialog box. Available only in Design view.
Format.InsertBookmark	CTRL + SHIFT + L	Displays the Bookmark dialog box. Available only in Design view.
Format.Italic	CTRL + I	Toggles the selected text between italic and plain. Available only in Design view.
Format.Underline	CTRL + U	Toggles the selected text between underlined and plain. Available only in Design view.
Layout.InsertColumnToLeft	CTRL + ALT + LEFT ARROW	Adds one column to the left of the current column in the table. Available only in Design view.
Layout.InsertColumnToTheRight	CTRL + ALT + RIGHT ARROW	Adds one column to the right of the current column in the table. Available only in Design view.
Layout.InsertRowAbove	CTRL + ALT + UP ARROW	Adds one row above the current row in the table. Available only in Design view.
Layout.InsertRowBelow	CTRL + ALT + DOWN ARROW	Adds one row below the current row in the table. Available only in Design view.
Project.AddContentPage	CTRL + M, CTRL + C	Adds a new *.aspx file to the Web site and opens the file in the HTML Designer. Available only in Design view.
View.AutoCloseTagOverride	CTRL + SHIFT + PERIOD	Temporarily overrides the default close tag behavior for the current tag. For more information, see Tag Specific Options . Available only in Source view.
View.Details	CTRL + SHIFT + Q	Displays icons for HTML elements that do not have a visual representation, such as comments, scripts, and anchors for absolutely positioned elements. Available only in Design view.
View.EditMaster	CTRL + M, CTRL + M	Opens the *.master file in Source view. Available only in Design view.

View.NextView	CTRL + PAGE DOWN	Switches among Design view, Source view, and Server Code view for the current document. Available in all views.
View.NonVisualControls	CTRL + ALT + Q	Displays symbols for non-graphical elements such as div, span, form, and script elements. Available only in Design view.
View.ShowSmartTag	SHIFT + ALT + F10	Displays a smart tag menu of common commands for Web server controls. Available only in Design view.
View.ViewDesigner	SHIFT + F7	Switches to Design view for the current document. Available only in Source view.
View.ViewMarkup	SHIFT + F7	Switches to Source view for the current document. Available only in Design view.
View.VisibleBorders	CTRL + Q	Displays a 1-pixel border around HTML elements that support a BORDER attribute that is set to zero. Examples of such HTML elements are tables, table cells, and divisions. Available only in Design view.
Window.PreviousTab	CTRL + PAGE UP	Switches among Design view, Source view, and Server Code view for the current document. Available in all views.

See Also

Reference

[HTML Designer](#)

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

XML Designer Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used when you work in the XML Designer.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Schema.Collapse	CTRL + MINUS (-)	Collapses nested elements. Available only in Schema View of the XML Designer .
Schema.Expand	CTRL + EQUALS (=)	Expands nested elements. Available only in Schema View of the XML Designer .

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Control Manipulation Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used to move, select, and change the size of controls on design surfaces.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.MoveControlDown	DOWN ARROW	Moves the selected control down in increments of 1 pixel on the design surface.
Edit.MoveControlDownGrid	CTRL + DOWN ARROW	Moves the selected control down in increments of 8 pixels on the design surface.
Edit.MoveControlLeft	LEFT ARROW	Moves the control to the left in increments of 1 pixel on the design surface.
Edit.MoveControlLeftGrid	CTRL + LEFT ARROW	Moves the control to the left in increments of 8 pixels on the design surface.
Edit.MoveControlRight	RIGHT ARROW	Moves the control to the right in increments of 1 pixel on the design surface.
Edit.MoveControlRightGrid	CTRL + RIGHT ARROW	Moves the control to the right in increments of 8 pixels on the design surface.
Edit.MoveControlUp	UP ARROW	Moves the control up in increments of 1 pixel on the design surface.
Edit.MoveControlUpGrid	CTRL + UP ARROW	Moves the control up in increments of 8 pixels on the design surface.
Edit.SelectNextControl	TAB	Moves to the next control on the page based on the TabIndex property of the control.
Edit.SelectPreviousControl	SHIFT + TAB	Moves back to the previously selected control on the page.
Edit.ShowTileGrid	ENTER	Displays the grid on the design surface.
Edit.SizeControlDown	SHIFT + DOWN ARROW	Increases the height of the control in increments of 1 pixel on the design surface.
Edit.SizeControlDownGrid	CTRL + SHIFT + DOWN ARROW	Increases the height of the control in increments of 8 pixels on the design surface.
Edit.SizeControlLeft	SHIFT + LEFT ARROW	Reduces the width of the control in increments of 1 pixel on the design surface.
Edit.SizeControlLeftGrid	CTRL + SHIFT + LEFT ARROW	Reduces the width of the control in increments of 8 pixels on the design surface.

Edit.SizeControlRight	SHIFT + RIGHT ARROW	Increases the width of the control in increments of 1 pixel on the design surface.
Edit.SizeControlRightGrid	CTRL + SHIFT + RIGHT ARROW	Increases the width of the control in increments of 8 pixels on the design surface.
Edit.SizeControlUp	SHIFT + UP ARROW	Decreases the height of the control in increments of 1 pixel on the design surface.
Edit.SizeControlUpGrid	CTRL + SHIFT + UP ARROW	Decreases the height of the control in increments of 8 pixels on the design surface.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Debugging Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used when you debug code.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Debug.ApplyCodeChanges	ALT + F10	Starts a build that lets you use the Edit and Continue feature to apply changes to code that is being debugged.
Debug.Autos	CTRL + D, CTRL + A	Displays the Auto window to view the values of variables currently in the scope of the current line of execution of the current procedure.
Debug.BreakAll	CTRL + ALT + Break	Temporarily stops execution of all processes in a debugging session. Available only in Run mode.
Debug.BreakAtFunction	CTRL + D, CTRL + N	Displays the New Breakpoint dialog box.
Debug.Breakpoints	CTRL + D, CTRL + B	Displays the Breakpoints dialog box, where you can add and modify breakpoints.
Debug.CallStack	CTRL + D, CTRL + C	Displays the Call Stack window to display a list of all active procedures or stack frames for the current thread of execution. Available only in Run mode.
Debug.DeleteAllBreakpoints	CTRL + SHIFT + F9	Clears all the breakpoints in the project.
Debug.Disassembly	CTRL + ALT + D	Displays the Disassembly window.
Debug.EnableBreakpoint	CTRL + F9	Switches the breakpoint from disabled to enabled.
Debug.Exceptions	CTRL + D, CTRL + E	Displays the Exceptions dialog box.
Debug.Immediate	CTRL + D, CTRL + I	Displays the Immediate window, where you can evaluate expressions and execute individual commands.

Debug.Locals	CTRL + D, CTRL + L	Displays the Locals window to view the variables and their values for each procedure in the current stack frame.
Debug.Memory1	CTRL + ALT + M, 1	Displays the Memory 1 window to view large buffers, strings, and other data that do not display clearly in the Watch or Variables windows.
Debug.Memory2	CTRL + ALT + M, 2	Displays the Memory 2 window to view large buffers, strings, and other data that do not display clearly in the Watch or Variables windows.
Debug.Memory3	CTRL + ALT + M, 3	Displays the Memory 3 window to view large buffers, strings, and other data that do not display clearly in the Watch or Variables windows.
Debug.Memory4	CTRL + ALT + M, 4	Displays the Memory 4 window to view large buffers, strings, and other data that do not display clearly in the Watch or Variables windows.
Debug.Modules	CTRL + D, CTRL + M	Displays the Modules window, which lets you view the .dll or .exe files used by the program. In multiprocess debugging, you can right-click and then click Show Modules for all Programs .
Debug.Processes	CTRL + D, CTRL + P	Displays the Processes window. Available in Run mode.
Debug.QuickWatch	CTRL + D, CTRL + Q	Displays the QuickWatch dialog box that has the current value of the selected expression. Available only in Break mode. Use this command to examine the current value of a variable, property, or other expression for which you have not defined a watch expression.
Debug.Registers	CTRL + D, CTRL + R	Displays the Registers window, which displays registers content for debugging native code applications.
Debug.Restart	CTRL + SHIFT + F5	Ends a debugging session, rebuilds, and then starts running the application from the beginning. Available in Break and Run modes.
Debug.RunToCursor	CTRL + F10	In Break mode, resumes execution of your code from the current statement to the selected statement. The Current Line of Execution margin indicator appears in the Margin Indicator bar. In Design mode, starts the debugger and executes your code to the cursor location.
Debug.ScriptExplorer	CTRL + ALT + N	Displays the Script Explorer window which lists the set of documents that you are debugging. Available in Run mode.
Debug.SetNextStatement	CTRL + SHIFT + F10	Sets the execution point to the line of code you choose.
Debug.ShowNextStatement	ALT + NUM *	Highlights the next statement to be executed.
Debug.Start	F5	Automatically attaches the debugger and runs the application from the startup project specified in the <Project> Properties dialog box. Changes to Continue if in Break mode.

Debug.StartWithoutDebugging	CTRL + F5	Runs the code without invoking the debugger.
Debug.StepInto	F11	Executes code one statement at a time, following execution into function calls.
Debug.StepIntoCurrentProcess	CTRL + ALT + F11	Available from the Processes window.
Debug.StepOut	SHIFT + F11	Executes the remaining lines of a function in which the current execution point is located.
Debug.StepOutCurrentProcess	CTRL + SHIFT + ALT + F11	Available from the Processes window.
Debug.StepOver	F10	Executes the next line of code, but does not follow execution through any function calls.
Debug.SetpOverCurrentProcess	CTRL + ALT + F10	Available from the Processes window.
Debug.StopDebugging	SHIFT + F5	Stops running the current application in the program. Available in Break and Run modes.
Debug.Threads	CTRL + D, CTRL + T	Displays the Threads window to view all the threads for the current process and information about them.
Debug.ToggleBreakpoint	F9	Sets or removes a breakpoint at the current line.
Debug.ToggleDisassembly	CTRL + D, CTRL + D	Displays the disassembly information for the current source file. Available only in Break mode.
Debug.Watch	CTRL + ALT + W, 1	Displays the Watch1 window to view the values of selected variables or watch expressions.
Debug.Watch2	CTRL + ALT + W, 2	Displays the Watch2 window to view the values of selected variables or watch expressions.
Debug.Watch3	CTRL + ALT + W, 3	Displays the Watch3 window to view the values of selected variables or watch expressions.
Debug.Watch4	CTRL + ALT + W, 4	Displays the Watch4 window to view the values of selected variables or watch expressions.
DebuggerContextMenu.BreakpointsWindow.Delete	ALT + F9, D	Removes the selected breakpoint. Available within the Breakpoints window only.

DebuggerContextMenu.BreakpointsWindow.GoToDisassembly	ALT + F9, A	Displays the Disassembly window. Available within the Breakpoints window only.
DebuggerContextMenu.BreakpointsWindow.GoToSourceCode	ALT + F9, S	Goes to the location of the selected breakpoint in the code file. Available within the Breakpoints window only.
Tools.AttachToProcesses	CTRL + ALT + P	Displays the Attach To Process dialog box, which lets you debug multiple programs at the same time in a single solution.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Data Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used work with data in the integrated development environment (IDE).

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Data.Column	CTRL + L	Adds a new column to the bottom of the data set. Available only in the Dataset Editor .
Data.Execute	CTRL + ALT + F5	Runs the currently active database object.
Data.InsertColumn	INSERT	Inserts a new column above the selected column in the data set. Available only in the Dataset Editor .
Data.RunSelection	CTRL + Q	Runs the current selection in the SQL editor.
Data.ShowDataSources	SHIFT + ALT + D	Displays the Data Sources window.
Data.StepInto	ALT + F5	Steps into debug mode for the currently active database object.
QueryDesigner.CancelRetrievingData	CTRL + T	Cancels or stops the currently running query. Available only in the Query and View Designer .
QueryDesigner.Criteria	CTRL + 2	Displays the Criteria pane of the Query and View Designer . Available only in the Query and View Designer .
QueryDesigner.Diagram	CTRL + 1	Displays the Diagram pane of the Query and View Designer . Available only in the Query and View Designer .
QueryDesigner.ExecuteSQL	CTRL + R	Executes the query. Available only in the Query and View Designer .
QueryDesigner.GoToRow	CTRL + G	When in the Results pane, moves focus to the tool strip docked at the bottom of the designer. Available only in the Query and View Designer .
QueryDesigner.JoinMode	CTRL + SHIFT + J	Enables JOIN mode. Available only in the Query and View Designer .
QueryDesigner.Results	CTRL + 4	Displays the Results pane of the Query and View Designer . Available only in the Query and View Designer .
QueryDesigner.SQL	CTRL + 3	Displays the SQL pane of the Query and View Designer . Available only in the Query and View Designer .
View.Datasets	CTRL + ALT + D	Displays the Report Datasets window of the Report Designer .

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Text Navigation Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used in text editors to move throughout an open document.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.CharLeft	LEFT ARROW	Moves the insertion point one character to the left.
Edit.CharRight	RIGHT ARROW	Moves the insertion point one character to the right.
Edit.ClearBookmarks	CTRL + B, CTRL + C	Removes all unnamed bookmarks in the current document.
Edit.DocumentEnd	CTRL + END	Moves the insertion point to the last line of the document.
Edit.DocumentStart	CTRL + HOME	Moves the insertion point to the first line of the document.
Edit.GoTo	CTRL + G	Displays the Go To Line dialog box.
Edit.GoToBrace	CTRL +]	Moves the insertion point to the next brace in the document.
Edit.LineDown	DOWN ARROW	Moves the insertion point down one line.
Edit.LineEnd	END	Moves the insertion point to the end of the current line.
Edit.LineStart	HOME	Moves the insertion point to the start of the line.
Edit.LineUp	UP ARROW	Moves the insertion point up one line.
Edit.NextBookmark	CTRL + B, CTRL + N	Moves the insertion point to the location of the next bookmark.
Edit.NextError	CTRL + SHIFT + F12	Moves to the next error entry in the Error List window, which automatically scrolls to the affected section of text in the editor.
Edit.PageDown	PAGE DOWN	Scrolls down one screen in the editor window.
Edit.PageUp	PAGE UP	Scrolls up one screen in the editor window.
Edit.PreviousBookmark	CTRL + B, CTRL + P	Moves the insertion point to the location of the previous bookmark.
Edit.QuickInfo	CTRL + K, CTRL + I	Displays Quick Info , based on the current language.
Edit.ScrollLineDown	CTRL + DOWN ARROW	Scrolls text down one line. Available in text editors only.

Edit.ScrollLineUp	CTRL + UP ARROW	Scrolls text up one line. Available in text editors only.
Edit.ToggleBookmark	CTRL + K, CTRL + K - or - CTRL + B, CTRL + T	Sets or removes a bookmark at the current line.
Edit.ViewBottom	CTRL + PAGE DOWN	Moves to the last visible line of the active window.
Edit.ViewTop	CTRL + PAGE UP	Moves to the first visible line of the active window.
Edit.WordNext	CTRL + RIGHT ARROW	Moves the insertion point to the right one word.
Edit.WordPrevious	CTRL + LEFT ARROW	Moves the insertion point to the left one word.
View.BrowseNext	CTRL + SHIFT + 1	Moves to the next definition, declaration, or reference of an item. Available in the Object Browser and Class View window.
View.BrowsePrevious	CTRL + SHIFT + 2	Moves to the previous definition, declaration, or reference of an item. Available in the Object Browser and Class View window.
View.NavigateBackward	CTRL + MINUS SIGN (-)	Moves to the previously browsed line of code.
View.NavigateForward	CTRL + SHIFT + MINUS SIGN (-)	Moves to the next browsed line of code.
View.PopBrowseContext	CTRL + SHIFT + 8	Moves to the previous item called in code in the current file.
View.ForwardBrowseContext	CTRL + SHIFT + 7	Moves to the next item called in code in the current file.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Text Selection Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used in text editors to select text in an open document.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.CharLeftExtend	SHIFT + LEFT ARROW	Moves the cursor to the left one character, extending the selection.
Edit.CharLeftExtendColumn	SHIFT + ALT + LEFT ARROW	Moves the cursor to the left one character, extending the column selection.
Edit.CharRightExtend	SHIFT + RIGHT ARROW	Moves the cursor to the right one character, extending the selection.
Edit.CharRightExtendColumn	SHIFT + ALT + RIGHT ARROW	Moves the cursor to the right one character, extending the column selection.
Edit.DocumentEndExtend	CTRL + SHIFT + END	Selects the text from the cursor to the last line of the document.
Edit.DocumentStartExtend	CTRL + SHIFT + HOME	Selects the text from the cursor to the first line of the document.
Edit.GoToBraceExtend	CTRL + SHIFT +]	Moves the cursor to the next brace, extending the selection.
Edit.LineDownExtend	SHIFT + DOWN ARROW	Extends text selection down one line, starting at the location of the cursor.
Edit.LineDownExtendColumn	SHIFT + ALT + DOWN ARROW	Moves the pointer down one line, extending the column selection.
Edit.LineEndExtend	SHIFT + END	Selects text from the cursor to the end of the current line.
Edit.LineEndExtendColumn	SHIFT + ALT + END	Moves the cursor to the end of the line, extending the column selection.
Edit.LineStartExtend	SHIFT + HOME	Selects text from the cursor to the start of the line.
Edit.LineStartExtendColumn	SHIFT + ALT + HOME	Moves the cursor to the start of the line, extending the column selection.
Edit.LineUpExtend	SHIFT + UP ARROW	Selects text up, line by line, starting from the location of the cursor.
Edit.LineUpExtendColumn	SHIFT + ALT + UP ARROW	Moves the cursor up one line, extending the column selection.
Edit.PageDownExtend	SHIFT + PAGE DOWN	Extends selection down one page.
Edit.PageUpExtend	SHIFT + PAGE UP	Extends selection up one page.
Edit.SelectAll	CTRL + A	Selects everything in the current document.

Edit.SelectCurrentWord	CTRL + SHIFT + W	Selects the word that contains the cursor or the word to the right of the cursor.
Edit.SelectToLastGoBack	CTRL + EQUALS (=)	Select from the current location in the Editor back to the previous location in the Editor.
Edit.ViewBottomExtend	CTRL + SHIFT + PAGE DOWN	Moves the cursor to the last line in view, extending the selection.
Edit.ViewTopExtend	CTRL + SHIFT + PAGE UP	Extends the selection to the top of the active window.
Edit.WordNextExtend	CTRL + SHIFT + RIGHT ARROW	Extends the selection one word to the right.
Edit.WordNextExtendColumn	CTRL + SHIFT + ALT + RIGHT ARROW	Moves the cursor to the right one word, extending the column selection.
Edit.WordPreviousExtend	CTRL + SHIFT + LEFT ARROW	Extends the selection one word to the left.
Edit.WordPreviousExtendColumn	CTRL + SHIFT + ALT + LEFT ARROW	Moves the cursor to the left one word, extending the column selection.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Text Manipulation Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used in text editors to delete, move, or format text in an open document.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.BreakLine	ENTER	Inserts a new line. Note In some editors, such as Design view of the HTML Designer, ENTER behaves differently depending on the context. For more information, see the documentation for the editor you are using.
Edit.CharTranspose	CTRL + T	Swaps the characters on either side of the cursor. For example, AC BD becomes AB CD. Available only in text editors.
Edit.CollapseTag	CTRL + M, CTRL + T	Hides the selected HTML tag and displays an ellipsis (. . .) instead. You can view the complete tag as a tooltip by putting the mouse pointer over the . . .
Edit.CollapseToDefinitions	CTRL + M, CTRL + O	Automatically determines logical boundaries for creating regions in code, such as procedures, and then hides them.
Edit.CommentSelection	CTRL + E, CTRL + C	Marks the current line of code as a comment by using the correct comment syntax for the programming language.
Edit.CompleteWord	CTRL + K, CTRL + W	Displays Word Completion based on the current language.
Edit.CopyParameterTip	CTRL + SHIFT + ALT + C	Copies the parameter information displayed by IntelliSense to the Clipboard.
Edit.DeleteBackwards	BACKSPACE	Deletes one character to the left of the cursor.
Edit.DeleteHorizontalWhitespace	CTRL + E, CTRL + \	Collapses white space in the selection, or deletes white space adjacent to the cursor if there is no selection.
Edit.FormatDocument	CTRL + E, CTRL + D	Applies the indenting and space formatting for the language as specified on the Formatting pane of the language in the Text Editor section of the Options dialog box.
Edit.FormatSelection	CTRL + E, CTRL + F	Correctly indents the selected lines of code based on the surrounding lines of code.
Edit.GenerateMethodStub	CTRL + K, CTRL + M	Creates a new method declaration for the method call that the cursor is resting in. For more information, see Generate Method Stub.

Edit.HideSelection	CTRL + M, CTRL + H	Hides the selected text. A signal icon marks the location of the hidden text in the file.
Edit.InsertTab	TAB	Indents the line of text a specified number of spaces.
Edit.InsertSnippet	CTRL + K, CTRL + X	Insert Code Snippet. For more information, see Code Snippets (C#) .
Edit.LineCut	CTRL + L	Cuts all selected lines, or the current line if nothing has been selected, to the Clipboard.
Edit.LineDelete	CTRL + SHIFT + L	Deletes all selected lines, or the current line if no selection has been made.
Edit.LineOpenAbove	CTRL + ENTER	Inserts a blank line above the cursor.
Edit.LineOpenBelow	CTRL + SHIFT + ENTER	Inserts a blank line below the cursor.
Edit.LineTranspose	SHIFT + ALT + T	Moves the line that contains the cursor below the next line.
Edit.ListMembers	CTRL + J	Lists members of the current class for statement completion when you are modifying code.
Edit.MakeLowercase	CTRL + U	Changes the selected text to lowercase characters.
Edit.MakeUppercase	CTRL + SHIFT + U	Changes the selected text to uppercase characters.
Edit.OverTypeMode	INSERT	Toggles between insert and overtype insertion modes. Available only when you are working in text editors.
Edit.ParameterInfo	CTRL + SHIFT + SPACEBAR	Displays a tooltip that contains information about the current parameter, based on the current language. Available only in Source view of the HTML Designer .
Edit.PasteParameterTip	CTRL + SHIFT + ALT + P	Pastes the previously copied parameter information from IntelliSense to the location indicated by the cursor.
Edit.StopHidingCurrent	CTRL + M, CTRL + U	Removes the outlining information for the currently selected region.
Edit.StopOutlining	CTRL + M, CTRL + P	Removes all outlining information from the whole document.
Edit.SwapAnchor	CTRL + E, CTRL + A	Swaps the anchor and end points of the current selection.
Edit.TabLeft	SHIFT + TAB	Moves selected lines to the left one tab stop.
Edit.ToggleAllOutlining	CTRL + M, CTRL + L	Toggles all previously marked hidden text sections between hidden and display states.

Edit.ToggleOutliningExpansion	CTRL + M, CTRL + M	Toggles the currently selected hidden text section between the hidden and display state.
Edit.ToggleTaskListShortcut	CTRL + E, CTRL + T	Sets or removes a shortcut at the current line.
Edit.ToggleWordWrap	CTRL + E, CTRL + W	Enables or disables wordwrap in an editor.
Edit.UncommentSelection	CTRL + E, CTRL + U	Removes the comment syntax from the current line of code.
Edit.ViewWhiteSpace	CTRL + E, CTRL + S - or - CTRL + R, CTRL + W	Shows or hides spaces and tab marks.
Edit.WordDeleteToEnd	CTRL + DELETE	Deletes the word to the right of the cursor.
Edit.WordDeleteToStart	CTRL + BACKSPACE	Deletes the word to the left of the cursor.
Edit.WordTranspose	CTRL + SHIFT + T	Transposes the words on either side of the cursor. For example, [End Sub would be changed to read Sub End].

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Window Management Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used to move, close, or navigate within tool and document windows.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
View.FullScreen	SHIFT + ALT + ENTER	Toggles Full Screen mode on and off.
View.NavigateBackward	CTRL + MINUS (-)	Goes back to the previous document or window in the navigation history.
View.NavigateForward	CTRL + EQUALS (=)	Moves forward to the document or window next in the navigation history.
Window.ActivateDocumentWindow	ESC	Closes a menu or dialog box, cancels an operation in progress, or puts focus in the current document window.
Window.CloseDocumentWindow	CTRL + F4	Closes the current MDI child window.
Window.CloseToolWindow	SHIFT + ESC	Closes the current tool window.
Window.MoveToNavigationBar	CTRL + F2	Moves the pointer to the drop-down bar located at the top of the code editor when the editor is in Code view or Server Code view.
Window.NextDocumentWindow	CTRL + F6	Cycles through the MDI child windows one window at a time.
Window.NextDocumentWindowNav	CTRL + TAB	Displays the IDE Navigator, with the first document window selected.
Window.NextPane	ALT + F6	Moves to the next tool window.
Window.NextTab	CTRL + PAGE DOWN	Moves to the next tab in the document or window.
Window.NextToolWindowNav	ALT + F7	Displays the IDE Navigator, with the first tool window selected.
Window.PreviousDocumentWindow	CTRL + SHIFT + F6	Moves to the previous document in the editor or designer.
Window.PreviousDocumentWindowNav	CTRL + SHIFT + TAB	Displays the IDE Navigator, with the previous document window selected.
Window.PreviousPane	SHIFT + ALT + F6	Moves to the previously selected window.

Window.PreviousSplitPane	SHIFT + F6	Moves to the previous pane of a document in split-pane view.
Window.PreviousTab	CTRL + PAGE UP	Moves to the previous tab in the document or window.
Window.PreviousToolWindowNav	SHIFT + ALT + F7	Displays the IDE Navigator with the previous tool window selected.
Window.ShowEzMDIFileList	CTRL + ALT + DOWN ARROW	Displays a pop-up listing all open documents only.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Integrated Help Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used to view and move among topics in Help.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Help.Contents	CTRL + F1, CTRL + C	Displays the Contents window for the documentation that is contained in MSDN.
Help.DynamicHelp	Ctrl + F1, CTRL + D	Displays Dynamic Help window.
Help.F1Help	F1	Displays a topic from Help that corresponds to the current user interface selected.
Help.HelpFavorites	CTRL + F1, CTRL + F	Displays Help Favorites .
Help.HowDoI	CTRL + F1, CTRL + h	Displays How Do I page that corresponds to the selected user settings.
Help.Index	CTRL + F1, I	Displays the Index window for the documentation that is contained in MSDN.
Help.IndexResults	Ctrl + F1, Ctrl + T	Displays the Index Results window
Help.NextTopic	ALT + DOWN ARROW -or- ALT + RIGHT ARROW	Displays the next topic in the table of contents. Available only in the Help (Web) browser window.
Help.PreviousTopic	ALT + UP ARROW OR ALT + LEFT ARROW	Displays the previous topic in the table of contents. Available only in the Help (Web) browser window.
Help.Search	CTRL + F1, CTRL + S	Displays the Visual Studio Help page with the Search tab active. This page enables you to search for words or phrases in the documentation that is contained in MSDN.
Help.Searchresults	CTRL + F1, CTRL + R	Displays the Visual Studio Help page with the Search tab and with the focus in the list of topics that the most recent search produced.
Help.WindowHelp	SHIFT + F1	Displays a topic from Help that corresponds to the current user interface.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

[Visual Studio Settings](#)

Object Browser Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used in the **Object Browser**.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.FindSymbol	ALT + F12	Displays the Find Symbol pane of the Find and Replace dialog box.
Edit.GoToDeclaration	CTRL + F12	Displays the definition of the selected symbol in the code.
Edit.GoToDefinition	F12	Displays the declaration for the selected symbol in code.
Edit.QuickFindSymbol	SHIFT + ALT + F12	Searches for the object or member selected in the file and displays the matches in the Find Symbol Results window.
View.ObjectBrowser	CTRL + ALT + J	Displays the Object Browser to view the classes, properties, methods, events, and constants available for packages, and the object libraries and procedures in your project.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Macro Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used when you work with macros.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
View.MacroExplorer	ALT + F8	Displays Macro Explorer , which lists all available macros in the current solution.
Tools.MacrosIDE	ALT + F11	Starts the Macros IDE, Visual Studio Macros.
Tools.RecordTemporaryMacro	CTRL + SHIFT + R	Puts the Visual Studio IDE in macro record mode.
Tools.RunTemporaryMacro	CTRL + SHIFT + P	Plays back a recorded macro.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Tool Window Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations let you display specific tool windows.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Tools.CodeSnippetManager	CTRL + K, CTRL + B	Displays the Code Snippets Manager , which lets you search for and insert code snippets in files.
View.BookmarkWindow	CTRL + W, CTRL + B	Displays the Bookmark window.
View.ClassView	CTRL + W, CTRL + C	Displays the Class View window.
View.ClassViewGoToSearchCombo	CTRL + K, CTRL + V	Changes focus in the Class View Search box.
View.CodeDefinitionWindow	CTRL + W, CTRL + D	Displays the Code Definition window.
View.CommandWindow	CTRL + W, CTRL + A	Displays the Command window, which lets you type commands that manipulate the integrated development environment (IDE).
View.DocumentOutline	CTRL + W, CTRL + U	Displays the Document Outline window to view the flat or hierarchical outline of the current document.
View.ErrorList	CTRL + W, CTRL + E	Displays the Error List window.
View.FindSymbolResults	CTRL + W, CTRL + Q	
View.ObjectBrowser	CTRL + W, CTRL + J	
View.Output	CTRL + W, CTRL + O	Displays the Output window to view status messages at run time.
View.PendingChecks	CTRL + W, CTRL + G	
View.PropertiesWindow	CTRL + W, CTRL + P	Displays the Properties window, which lists the design-time properties and events for the currently selected item.
View.PropertyPages	SHIFT + F4	Displays the property pages for the item currently selected.
View.ResourceView	CTRL + W, CTRL + R	Displays the Resource View window.

View.ServerExplorer	CTRL + W, CTRL + L	Displays Server Explorer , which lets you view and manipulate database servers, event logs, message queues, Web services, and other operating system services.
View.SolutionExplorer	CTRL + W, CTRL + S	Displays Solution Explorer , which lists the projects and files in the current solution.
View.TaskList	CTRL + W, CTRL + T	Displays the Task List window where you customize, categorize, and manage tasks, comments, shortcuts, warnings and error messages.
View.Toolbox	CTRL + W, CTRL + X	Displays the Toolbox , which contains controls and other items that can be included or used with your code.
View.WebBrowser	CTRL + W, CTRL + W	Displays the Web Browser window, which lets you view pages on the Internet.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Project Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used to add new items to a project, build a project, open files, or open projects.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Build.BuildSolution	F6	Builds the solution.
Build.Cancel	CTRL + BREAK	Stops the current build.
Build.Compile	CTRL + F7	Creates an object file that contains machine code, linker directives, sections, external references, and function/data names for the selected file.
File.NewFile	CTRL + N	Displays the New File dialog box where you can select a new file to add to the current project.
File.NewProject	CTRL + SHIFT + N	Displays the New Project dialog box.
File.OpenFile	CTRL + O	Displays the Open File dialog box.
File.OpenProject	CTRL + SHIFT + O	Displays the Open Project dialog box where you can add existing projects to your solution.
Project.AddClasses	SHIFT + ALT + C	Displays the Add New Item dialog box and selects Class template as default.
Project.AddExistingItem	SHIFT + ALT + A	Displays the Add Existing Item dialog box, which lets you add an existing file to the current project.
Project.AddNewItem	CTRL + SHIFT + A	Displays the Add New Item dialog box, which lets you add a new file to the current project.
Project.Override	CTRL + ALT + INSERT	Lets you override base class methods in a derived class.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Image Editor Shortcut Keys, Visual C# 2005 Scheme

The following table includes shortcut keys for the Image editor commands that are bound to keys by default. To change shortcut keys, click **Options** on the **Tools** menu, expand **Environment**, and then click **Keyboard**. For more information, see [How to: Work with Shortcut Key Combinations](#).

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command	Keys	Description
Image.AirBrushTool	CTRL + A	Draws using an airbrush with the selected size and color.
Image.BrushTool	CTRL + B	Draws using a brush with the selected shape, size, and color.
Image.CopyAndOutlineSelection	CTRL + SHIFT + U	Creates a copy of the current selection and outlines it. If the background color is in the current selection, it will be excluded if you have transparent selected.
Image.DrawOpaque	CTRL + J	Makes the current selection either opaque or transparent .
Image.EllipseTool	CTRL + P	Draws an ellipse with the selected line width and color.
Image.EraserTool	CTRL + SHIFT + I	Erases a part of the image (with the current background color).
Image.FilledEllipseTool	CTRL + SHIFT + ALT + P	Draws a filled ellipse.
Image.FilledRectangleTool	CTRL + SHIFT + ALT + R	Draws a filled rectangle.
Image.FilledRoundRectangleTool	CTRL + SHIFT + ALT + W	Draws a filled round rectangle.
Image.FillTool	CTRL + F	Fills an area.
Image.FlipHorizontal	CTRL + H	Flips the image or selection horizontally.
Image.FlipVertical	SHIFT+ ALT + H	Flips the image or selection vertically.
Image.LargerBrush	CTRL + =	Increases the brush size by one pixel in each direction. To decrease the brush size, see Image.SmallerBrush in this table.
Image.LineTool	CTRL + L	Draws a straight line with the selected shape, size, and color.
Image.MagnificationTool	CTRL + M	Switches to the Magnify tool, which lets you to magnify specific sections of your image.
Image.Magnify	CTRL + SHIFT + M	Toggles between the current magnification and 1:1 magnification.

Image.NewImageType	INSERT	Opens the New <Device> Image Type dialog box with which you can create an image or a different image type.
Image.NextColor	CTRL +] - or - CTRL + RIGHT ARROW	Changes the drawing foreground color to the next palette color.
Image.NextRightColor	CTRL + SHIFT +] - or - SHIFT + CTRL + RIGHT ARROW	Changes the drawing background color to the next palette color.
Image.OutlinedEllipseTool	SHIFT + ALT + P	Draws a filled ellipse with an outline.
Image.OutlinedRectangleTool	SHIFT + ATL + R	Draws a filled rectangle with an outline
Image.OutlinedRoundRectangleTool	SHIFT + ALT + W	Draws a filled round rectangle with an outline.
Image.PencilTool	CTRL + I	Draws using a single-pixel pencil.
Image.PreviousColor	CTRL + [- or - CTRL + LEFT ARROW	Changes the drawing foreground color to the previous palette color.
Image.PreviousRightColor	CTRL + SHIFT + [- or - SHIFT + CTRL + LEFT ARROW	Changes the drawing background color to the previous palette color.
Image.RectangleSelectionTool	SHIFT + ALT + S	Selects a rectangular part of the image to move, copy, or edit.
Image.RectangleTool	ATL + R	Draws a rectangle with the selected line width and color.
Image.Rotate90Degrees	CTRL + SHIFT + H	Rotates the image or selection 90 degrees.
Image.RoundedRectangleTool	ALT + W	Draws a round rectangle with the selected line width and color.
Image.ShowGrid	CTRL + ALT + S	Toggles the pixel grid (selects or clears the Pixel grid option in the Grid Settings dialog box).
Image.ShowTileGrid	CTRL + SHIFT + ALT + S	Toggles the tile grid (selects or clears the Tile grid option in the Grid Settings dialog box).

Image.SmallBrush	CTRL + . (PERIOD)	Reduces the brush size to one pixel. (See also Image.LargerBrush and Image.SmallerBrush in this table.)
Image.SmallerBrush	CTRL + - (MINUS)	Reduces the brush size by one pixel in each direction. To expand the brush size again, see Image.LargerBrush in this table.
Image.TextTool	CTRL + T	Opens the Text Tool dialog box .
Image.UseSelectionAsBrush	CTRL + U	Draws using the current selection as a brush.
Image.ZoomIn	CTRL + SHIFT + . (period) - or - CTRL + UP ARROW	Increases the magnification for the current view.
Image.ZoomOut	CTRL + , (comma) - or - CTRL + DOWN ARROW	Reduces the magnification of the current view.

For information about how to add resources to managed projects, see [Resources in Applications](#) in the .NET Framework Developer's Guide. For information about manually adding resource files to managed projects, accessing resources, displaying static resources, and assigning resource strings to properties, see [Walkthrough: Localizing Windows Forms](#) and [Walkthrough: Using Resources for Localization with ASP.NET](#).

Requirements

None

See Also

Reference

[Image Editor](#)

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Dialog Editor Shortcut Keys, Visual C# 2005 Scheme

The following table includes the default keyboard shortcuts for the Dialog editor commands. To change shortcut keys, click **Options** on the **Tools** menu, expand **Environment**, and then click **Keyboard**. For more information, see [How to: Work with Shortcut Key Combinations](#).

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command	Keys	Description
Format.AlignBottoms	CTRL + SHIFT + DOWN ARROW	Aligns the bottom edges of the selected controls with the dominant control.
Format.AlignCenters	SHIFT + F9	Aligns the vertical centers of the selected controls with the dominant control.
Format.AlignLefts	CTRL + SHIFT + LEFT ARROW	Aligns the left edges of the selected controls with the dominant control.
Format.AlignMiddles	F9	Aligns the horizontal centers of the selected controls with the dominant control.
Format.AlignRights	CTRL + SHIFT + RIGHT ARROW	Aligns the right edges of the selected controls with the dominant control.
Format.AlignTops	CTRL + SHIFT + UP ARROW	Aligns the top edges of the selected controls with the dominant control.
Format.ButtonBottom	CTRL + B	Positions the selected buttons along the bottom-center of the dialog box.
Format.ButtonRight	CTRL + R	Positions the selected buttons in the top-right corner of the dialog box.
Format.CenterHorizontal	CTRL + SHIFT + F9	Centers the controls horizontally in the dialog box.
Format.CenterVertical	CTRL + F9	Centers the controls vertically in the dialog box.
Format.CheckMnemonics	CTRL + M	Checks uniqueness of mnemonics.
Format.SizeToContent	SHIFT + F7	Resizes the selected control(s) to fit the caption text.
Format.SpaceAcross	ALT + RIGHT ARROW	Evenly spaces the selected controls horizontally.
Format.SpaceDown	ALT + DOWN ARROW	Evenly spaces the selected controls vertically.
Format.TabOrder	CTRL + D	Sets the order of controls in the dialog box.
Format.TestDialog	CTRL + T	Runs the dialog box to test appearance and behavior.
Format.ToggleGuides	CTRL + G	Cycles among no grid, guidelines, and grid for dialog editing.

For information about how to add resources to managed projects, see [Resources in Applications](#) in the .NET Framework Developer's Guide. For information about how to manually add resource files to managed projects, access resources, display static resources, and assign resource strings to properties, see [Walkthrough: Localizing Windows Forms](#) and [Walkthrough: Using Resources for Localization with ASP.NET](#).

See Also

Reference

[Dialog Editor](#)

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Search and Replace Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used to search for text in a single file, to search for text in multiple files, and to search for objects and members.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.Find	CTRL + F	Displays the Quick tab of the Find and Replace dialog box.
Edit.FindAllReferences	CTRL + K, CTRL + R	Displays the list of where to find all symbol references.
Edit.FindInFiles	CTRL + SHIFT + F	Displays the In Files tab of the Find and Replace dialog box.
Edit.FindNext	F3	Finds the next occurrence of the previous search text.
Edit.FindNextSelected	CTRL + F3	Finds the next occurrence of the currently selected text in the document.
Edit.FindPrevious	SHIFT + F3	Finds the previous occurrence of the search text.
Edit.FindPreviousSelected	CTRL + SHIFT + F3	Finds the previous occurrence of the currently selected text, or the word at the insertion point.
Edit.GoToFindCommand	CTRL + /	Puts the insertion point in the Find/Command box on the Standard toolbar.
Edit.Incremental Search	CTRL + I	Starts incremental search. If incremental search is started, but you have not typed any characters, recalls the previous pattern. If text has been found, searches for the next occurrence.
Edit.Replace	CTRL + H	Displays the replace options in the Quick tab of the Find and Replace dialog box.
Edit.ReplaceInFiles	CTRL + SHIFT + H	Displays the replace options on the In Files tab of the Find and Replace dialog box.
Edit.ReverseIncrementalSearch	CTRL + SHIFT + I	Changes the direction of incremental search to start at the bottom of the file and progress toward the top.
Edit.StopSearch	ALT + F3, S	Stops the current Find in Files operation.
View.FindSymbolResults	CTRL + W, CTRL + Q	Displays the Find Symbol Results window, which displays matches for symbols searches.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Refactoring Shortcut Keys, Visual C# 2005 Scheme

The following key combinations are shortcuts for performing [Refactoring](#) operations.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Refactor.EncapsulateField	CTRL + R, CTRL + E	Displays the Encapsulate Field Dialog Box , which lets you create a property from an existing field and then updates your code to refer to the new property.
Refactor.ExtractInterface	CTRL + R, CTRL + I	Displays the Extract Interface Dialog Box , which lets you create a new interface with members derived from an existing class, struct, or interface.
Refactor.ExtractMethod	CTRL + R, CTRL + M	Displays the Extract Method Dialog Box , which lets you create a new method from a code fragment of an existing method.
Refactor.PromoteLocalVariabletoParameter	CTRL + R, CTRL + P	Moves a variable from a local usage to a method, indexer, or constructor parameter and updates the call sites correctly. For more information, see Promote Local Variable to Parameter .
Refactor.RemoveParameters	CTRL + R, CTRL + V	Displays the Remove Parameters dialog box, which removes parameters from methods, indexers, or delegates by changing the declaration at any locations where the member is called. For more information, see Remove Parameters .
Refactor.Rename	F2 - or - CTRL + R, CTRL + R	Displays the Rename Dialog Box , which lets you rename identifiers for code symbols such as fields, local variables, methods, namespaces, properties, and types.
Refactor.ReorderParameters	CTRL + R, CTRL + O	Displays the Reorder Parameters Dialog Box , which lets you change the order of the parameters for methods, indexers, and delegates.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Managed Resources Editor Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used only when editing in the Managed Resources editor. For more information, see [Resources Page](#), [Project Designer](#).

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.EditCell	F2	Switches to edit mode in the selected cell in Other view and Strings view.
Edit.Remove	DELETE	Removes the selected file in Files view, Images view, Icons view, and Audio view.
Edit.RemoveRow	CTRL + DELETE	Deletes the selected row in Other view and Strings view.
Resources.Audio	CTRL + 4	Switches the Managed Resources editor to Audio view, which displays sound files in the current project. Formats of displayed files include .wav, .wma, and .mp3.
Resources.Files	CTRL + 5	Switches the Managed Resources editor to Files view, which displays files that are not found in the other views.
Resources.Icons	CTRL + 3	Switches the Managed Resources editor to Icons view, which displays icon (*.ico) files in the current project.
Resources.Images	CTRL + 2	Switches the Managed Resources editor to Images view, which displays image files in the current project. Formats of displayed files include .bmp, .jpg, and .gif.
Resources.Other	CTRL + 6	Switches the Managed Resources editor to Other view, which displays a settings grid for adding other types that support string serialization.
Resources.Strings	CTRL + 1	Switches the Managed Resources editor to Strings view, which displays strings in a grid with columns for the Name, Value, and Comment of the string resource.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)
[Resources in .Resx File Format](#)

Code Snippet Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used to work with code snippets.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.InsertSnippet	CTRL + K, CTRL + X	Displays the Code Snippet Picker , which lets you select a snippet by using IntelliSense and then inserts the code snippet at the cursor position.
Edit.SurroundWith	CTRL + K, CTRL + S	Displays the Code Snippet Picker , which lets you select a snippet by using IntelliSense and then wraps the snippet around the selected text.
Tools.CodeSnippetsManager	CTRL + K, CTRL + B	Displays the Code Snippets Manager , which lets you search for and insert code snippets into files.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Class Diagram Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used only when you work with class diagrams.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
ClassDiagram.Collapse	NUM - (MINUS SIGN)	Collapses expanded nodes in the Class Details window or collapses the selected shape compartment in the diagram.
ClassDiagram.Expand	NUM + (PLUS SIGN)	Expands collapsed nodes in the Class Details window or expands the selected shape compartment in the diagram.
Edit.Delete	CTRL + DELETE	Removes the selected item from the class diagram.
Edit.ExpandCollapseBaseTypeList	SHIFT + ALT + B	Expands or collapses base types in the selected shape compartment. For example, if Interface1 inherits from Interface2, Interface3, and Interface4, the parental interfaces are listed on the shape compartment for Interface1. With this command, you can collapse the list of inherited interfaces to show only summary information about the number of base interfaces inherited by Interface1.
Edit.NavigateToLollipop	SHIFT + ALT + L	Selects the Interface Lollipop for a shape compartment. The lollipop appears on shapes that implement one or more interfaces.
Edit.RemoveFromDiagram	DELETE	Removes the selected shape compartment from the diagram.
View.ViewCode	ENTER - or - F7	For the selected item, opens the corresponding file and puts the insertion point in the correct location.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Other Resources

[Working with Class Diagrams](#)

Bookmark Window Shortcut Keys, Visual C# 2005 Scheme

The following shortcut key combinations can be used only when you are working with bookmarks, either in the [Bookmark Window](#) or in the editor.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command Name	Shortcut Keys	Description
Edit.ClearBookmarks	CTRL + B, CTRL + C	Removes all bookmarks in the document
Edit.EnableBookmark	CTRL + B, CTRL + E	Enables bookmark usage in current document.
Edit.NextBookmark	CTRL + B, CTRL + N	Moves to the next bookmark in the document.
Edit.PreviousBookmark	CTRL + B, CTRL + P	Moves to the previous bookmark.
Edit.ToggleBookmark	CTRL + B, CTRL + T	Toggles a bookmark on the current line in the document.
View.BookmarkWindow	CTRL + W, CTRL + B	Displays the Bookmark window.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Accelerator and String Editor Shortcut Keys, Visual C# 2005 Scheme

Use the following shortcut key combinations in either the Accelerator editor or String editor.

Note

The options available in dialog boxes, and the names and locations of menu commands you see, might differ from what is described in Help depending on your active settings or edition. To change your settings, click **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Command	Shortcut keys	Description
Edit.NewAccelerator	INSERT	Adds a new entry for a keyboard shortcut. Available only in the Accelerator editor.
Edit.NewString	INSERT	Adds a new entry in the string table. Available only in the String editor.
Edit.NextKeyTyped	CTRL + W	Displays the Capture Next Key message box, which prompts you to press the keys you intend to use as keyboard shortcuts. Available only in the Accelerator Editor.

See Also

Concepts

[Visual C# 2005 Default Shortcut Keys](#)

Migrating to Visual C#

This section introduces C# syntax and concepts to developers who are migrating from other programming languages. It also contains the reference documentation for the Java Language Conversion Assistant, which you can use to convert Java language source to C# source code.

In This Section

[C# for Java Developers](#)

Compares the C# language syntax and constructs to the Java language.

[Converting Java Applications to Visual C#](#)

Describes the Java Language Conversion Assistant, a tool for porting Java projects to Visual C#.

[C# for C++ Developers](#)

Compares the C# language to features of the C++ language.

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C#](#)

[Getting Started with Visual C#](#)

C# for Java Developers

The topics in this section provide an introduction to the C# language and the .NET Framework.

In This Section

- [The C# Programming Language for Java Developers](#)
- [C# Code Examples for Java Developers](#)
- [C# Application Types for Java Developers](#)

See Also

Concepts

[Java Language Conversion Assistant](#)

Other Resources

[Getting Started with Visual C#](#)

The C# Programming Language for Java Developers

This section discusses the similarities and differences in between the C# and Java programming languages.

In This Section

[Source File Conventions \(C# vs. Java\)](#)

[Data Types \(C# vs. Java\)](#)

[Operators \(C# vs. Java\)](#)

[Flow Control \(C# vs. Java\)](#)

[Looping Statements \(C# vs Java\)](#)

[Class Fundamentals \(C# vs Java\)](#)

[Main \(\) and Other Methods \(C# vs Java\)](#)

[Using an Indeterminate Number of Parameters \(C# vs Java\)](#)

[Properties \(C# vs Java\)](#)

[Structs \(C# vs Java\)](#)

[Arrays](#)

[Inheritance and Derived Classes \(C# vs Java\)](#)

[Events](#)

[Operator Overloading \(C# vs Java\)](#)

[Exceptions \(C# vs Java\)](#)

[Advanced C# Techniques \(C# vs Java\)](#)

[Garbage Collection \(C# vs Java\)](#)

[Safe and Unsafe Code \(C# vs Java\)](#)

[Summary \(C# vs Java\)](#)

Related Sections

[C# Programming Guide](#)

[Visual C#](#)

[Migrating to Visual C#](#)

[The C# Programming Language for Java Developers](#)

Source File Conventions (C# vs. Java)

The naming convention for files containing C# classes is a little different from Java. In Java, all source files have a .java extension. Each source file contains one top-level public class declaration, and the class name must match the file name. In other words, a class called **Customer** declared with a public scope must be defined in a source file with the name Customer.java.

C# source code is denoted by the .cs extension. Unlike Java, source files can contain more than one top-level public class declaration, and the file name does not need to match any of the classes' names.

Top-Level Declarations

In both Java and C#, source code begins with a few top-level declarations in a certain sequence. There are only a few differences between the declarations made in Java and C# programs.

Top-Level Declarations in Java

In Java, you can group classes together with the **package** keyword. A packaged class must use the **package** keyword in the first executable line of the source file. Any import statements required to access classes in other packages are next, and then the class declaration, as follows:

```
package Acme;
import java.io.*;
class Customer
{
    ...
}
```

Top-Level Declarations in C#

C# uses the concept of namespaces to group logically related classes through the [namespace](#) keyword. These act similarly to Java packages, and a class with the same name might appear within two different namespaces. To access classes defined in a namespace external to the current one, use the [using](#) directive followed by the namespace name, as follows:

C#

```
using System.IO;

namespace Acme
{
    class Customer
    {
        // ...
    }
}
```

Note that **using** directives can be placed inside a namespace declaration, in which case such imported namespaces form part of the containing namespace.

Java does not allow multiple packages in the same source file. However, C# does allow multiple namespaces in a single .cs file, as follows:

C#

```
namespace AcmeAccounting
{
    public class GetDetails
    {
        // ...
    }
}

namespace AcmeFinance
{
    public class ShowDetails
```

```
{  
    // ...  
}
```

Fully Qualified Names and Namespace Aliases

As with Java, you can access classes in both the .NET Framework or in user-defined namespaces without a `using` reference for that namespace by providing the fully qualified name for the class, such as `DataSet`, or `AcmeAccounting.GetDetails` in the previous example.

Fully qualified names can get long and unwieldy, and in such cases, you can use the **using** keyword to specify a short name, or alias, to make your code more readable.

In the following code, an alias is created to refer to code written by a fictional company:

C#

```
using DataTier = Acme.SQLCode.Client;  
  
class OutputSales  
{  
    static void Main()  
    {  
        int sales = DataTier.GetSales("January");  
        System.Console.WriteLine("January's Sales: {0}", sales);  
    }  
}
```

Note that in the syntax for `WriteLine`, with `{x}` in the format string, the `x` denotes the position in the argument list of the value to insert at that position. Assuming the `GetSales` method returned 500, the output of the application would be as follows:

January's Sales: 500

Preprocessing Directives

Similar to C and C++, C# includes preprocessing directives that provide the ability to conditionally skip sections of source files, report error and warning conditions, and to delineate distinct regions of source code. The term "pre-processing directives" is used only for consistency with the C and C++ programming languages, as C# does not include a separate preprocessing step. For more information, see [C# Preprocessor Directives](#).

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Data Types (C# vs. Java)

This topic discusses some of the primary similarities and differences in how data is represented, allocated, and garbage-collected in Java and in C#.

Compound data types

The concept of a class as a compound data type with fields, methods, and events is similar in Java and C#. (Class inheritance is discussed separately in the topic entitled [Inheritance and Derived Classes \(C# vs Java\)](#).) C# introduces the concept of a struct as a stack-allocated compound data type that does not support inheritance. In most other respects, structs are very similar to classes. Structs provide a lightweight way of grouping together related fields and methods for use in tight loops and other scenarios where performance is critical.

C# enables you to create a destructor method that is called before instances of a class are garbage-collected. In Java, a **finalize** method can be used to contain code that cleans up resources before the object is garbage-collected. In C#, this function is performed by the class destructor. The destructor resembles a constructor with no arguments and a preceding tilde character (~).

Built-In Data Types

C# provides all the data types that are available in Java, and adds support for unsigned numerals and a new 128-bit high-precision floating-point type.

For each primitive data type in Java, the core class library provides a wrapper class that represents it as a Java object. For example, the [Int32](#) class wraps the **int** data type, and the [Double](#) class wraps the **double** data type.

On the other hand, all primitive data types in C# are objects in the [System](#) namespace. For each data type, a short name, or alias, is provided. For instance, **int** is the short name for **System.Int32** and **double** is the short form of **System.Double**.

The list of C# data types and their aliases is provided in the following table. As you can see, the first eight of these correspond to the primitive types available in Java. Note, however, that Java's **boolean** is called **bool** in C#.

Short Name	.NET Class	Type	Width	Range (bits)
byte	Byte	Unsigned integer	8	0 to 255
sbyte	SByte	Signed integer	8	-128 to 127
int	Int32	Signed integer	32	-2,147,483,648 to 2,147,483,647
uint	UInt32	Unsigned integer	32	0 to 4294967295
short	Int16	Signed integer	16	-32,768 to 32,767
ushort	UInt16	Unsigned integer	16	0 to 65535
long	Int64	Signed integer	64	-922337203685477508 to 922337203685477507
ulong	UInt64	Unsigned integer	64	0 to 18446744073709551615
float	Single	Single-precision floating point type	32	-3.402823e38 to 3.402823e38
double	Double	Double-precision floating point type	64	-1.79769313486232e308 to 1.79769313486232e308
char	Char	A single Unicode character	16	Unicode symbols used in text

bool	Boolean	Logical Boolean type	8	True or false
object	Object	Base type of all other types		
string	String	A sequence of characters		
decimal	Decimal	Precise fractional or integral type that can represent decimal numbers with 29 significant digits	128	$\pm 1.0 \times 10e-28$ to $\pm 7.9 \times 10e28$

Because C# represents all primitive data types as objects, it is possible to call an object method on a primitive data type. For example:

```
C#
static void Main()
{
    int i = 10;
    object o = i;
    System.Console.WriteLine(o.ToString());
}
```

This is achieved with the help of automatic boxing and unboxing. For more information, see [Boxing and Unboxing \(C# Programming Guide\)](#).

Constants

Both Java and C# provide the ability to declare a variable whose value is specified at compile time and cannot be changed at runtime. Java uses the **final** field modifier to declare such a variable, while C# uses the [const](#) keyword. In addition to **const**, C# provides the [readonly](#) keyword to declare variables that can be assigned a value once at runtime--either in the declaration statement or else in the constructor. After initialization, the value of a **readonly** variable cannot change. One scenario in which **readonly** variables are useful is when modules that have been compiled separately need to share data such as a version number. If module A is updated and recompiled with a new version number, module B can be initialized with that new constant value without having to be recompiled.

Enumerations

Enumerations, or enums, are used to group named constants similar to how they are used in C and C++; they are not available in Java. The following example defines a simple `Color` enumeration.

```
C#
public enum Color
{
    Green,    //defaults to 0
    Orange,   //defaults to 1
    Red,      //defaults to 2
    Blue     //defaults to 3
}
```

Integral values can also be assigned to enumerations as shown in the following enum declaration:

```
C#
public enum Color2
{
    Green = 10,
    Orange = 20,
    Red = 30,
    Blue = 40
}
```

The following code example calls the [GetNames](#) method of the [Enum](#) type to display the available constants for an enumeration. It then assigns a value to an enumeration and displays the value.

```
C#  
  
class TestEnums  
{  
    static void Main()  
    {  
        System.Console.WriteLine("Possible color choices: ");  
  
        //Enum.GetNames returns a string array of named constants for the enum.  
        foreach(string s in System.Enum.GetNames(typeof(Color)))  
        {  
            System.Console.WriteLine(s);  
        }  
  
        Color favorite = Color.Blue;  
  
        System.Console.WriteLine("Favorite Color is {0}", favorite);  
        System.Console.WriteLine("Favorite Color value is {0}", (int) favorite);  
    }  
}
```

Output

```
Possible color choices:  
  
Green  
Orange  
Red  
Blue  
  
Favorite Color is Blue  
Favorite Color value is 3
```

Strings

String types in both Java and C# exhibit similar behavior with slight differences. Both string types are immutable, meaning that the values of the strings cannot be changed once the strings have been created. In both instances, methods that appear to modify the actual content of a string actually create a new string to return, leaving the original string unchanged. The process of comparing string values is different in C# and Java. To compare string values in Java, developers need to call the **equals** method on a string type as the `==` operator compares reference types by default. In C#, developers can use the `==` or `!=` operators to compare string values directly. Even though a string is a reference type in C#, the `==` and `!=` operator will, by default, compare the string values rather than references.

Just like in Java, C# developers should not use the string type for concatenating strings to avoid the overhead of creating new string classes every time the string is concatenated. Instead, developers can use the [StringBuilder](#) class, which is functionally equivalent to the Java **StringBuffer** class.

String Literals

C# provides the ability to avoid the usage of escape sequences like `"\t"` for tab or `"\"` for backslash characters within string constants. To do this, simply declare the verbatim string using the `@` symbol to precede the assignment of the string value. The following examples show how to use escape characters and how to assign string literals:

```
C#  
  
static void Main()  
{  
    //Using escaped characters:  
    string path1 = "\\\\FileShare\\Directory\\file.txt";  
    System.Console.WriteLine(path1);  
  
    //Using String Literals:  
    string path2 = @"\\FileShare\Directory\file.txt";  
}
```

```
    System.Console.WriteLine(path2);  
}
```

Converting and Casting

Both Java and C# follow similar rules for the automatic conversion and casting of data types.

Like Java, C# supports both implicit and explicit type conversions. In the case of widening conversions, the conversions are implicit. For example, the following conversion from **int** to **long** is implicit, as in Java:

C#

```
int int1 = 5;  
long long1 = int1; //implicit conversion
```

The following is a list of implicit conversions between .NET Framework data types:

Source Type	Target Type
Byte	short, ushort, int, uint, long, ulong, float, double, or decimal
Sbyte	short, int, long, float, double, or decimal
Int	long, float, double, or decimal
UInt	long, ulong, float, double, or decimal
Short	int, long, float, double, or decimal
Ushort	int, uint, long, ulong, float, double, or decimal
Long	float, double, or decimal
Ulong	float, double, or decimal
Float	double
Char	ushort, int, uint, long, ulong, float, double, or decimal

You cast expressions that you want to explicitly convert using the same syntax as Java:

C#

```
long long2 = 5483;  
int int2 = (int)long2; //explicit conversion
```

The following table lists explicit conversions.

Source Type	Target Type
Byte	sbyte or char
Sbyte	byte, ushort, uint, ulong, or char
Int	sbyte, byte, short, ushort, uint, ulong, or char
UInt	sbyte, byte, short, ushort, int, or char

Short	sbyte, byte, ushort, uint, ulong, or char
Ushort	sbyte, byte, short, or char
Long	sbyte, byte, short, ushort, int, uint, ulong, or char
Ulong	sbyte, byte, short, ushort, int, uint, long, or char
Float	sbyte, byte, short, ushort, int, uint, long, ulong, char, or decimal
Double	sbyte, byte, short, ushort, int, uint, long, ulong, char, float, or decimal
Char	sbyte, byte, or short
Decimal	sbyte, byte, short, ushort, int, uint, long, ulong, char, float, or double

Value and Reference Types

C# supports two kinds of variable types:

- Value types

These are the built-in primitive data types, such as char, int, and float, as well as user-defined types declared with struct.

- Reference types

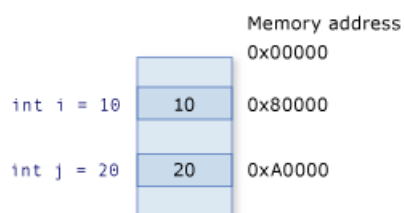
Classes and other complex data types that are constructed from the primitive types. Variables of such types do not contain an instance of the type, but merely a reference to an instance.

If you create two value-type variables, i and j, as follows, then i and j are completely independent of each other:

C#

```
int i = 10;
int j = 20;
```

They are given separate memory locations:



If you change the value of one of these variables, the other will naturally not be affected. For instance, if you have an expression such as the following, then there is still no connection between the variables:

C#

```
int k = i;
```

That is, if you change the value of i, k will remain at the value that i had at the time of the assignment.

C#

```
i = 30;

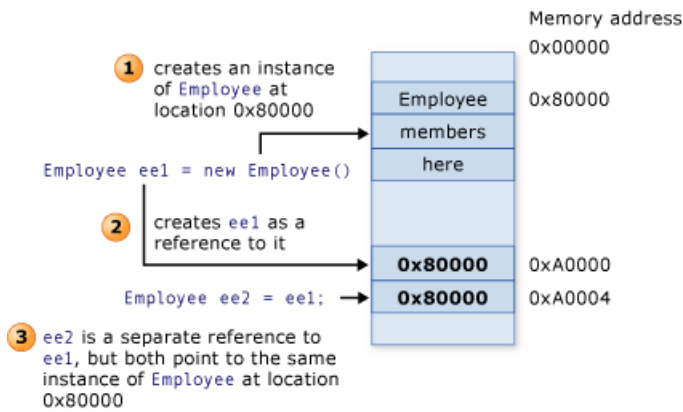
System.Console.WriteLine(i.ToString()); // 30
System.Console.WriteLine(k.ToString()); // 10
```

Reference types, however, act differently. For instance, you could declare two variables as follows:

C#

```
Employee ee1 = new Employee();  
Employee ee2 = ee1;
```

Now, because classes are reference types in C#, `ee1` is known as a reference to `Employee`. The first of the previous two lines creates an instance of `Employee` in memory, and sets `ee1` to reference it. Thus, when you set `ee2` to equal `ee1`, it contains a duplicate of the reference to the class in memory. If you now change properties on `ee2`, properties on `ee1` reflect these changes, because both point to the same object in memory, as shown in the following:



Boxing and Unboxing

The process of converting a value type to a reference type is called boxing. The inverse process, converting a reference type to a value type, is called unboxing. This is illustrated in the following code:

C#

```
int i = 123;           // a value type  
object o = i;          // boxing  
int j = (int)o;        // unboxing
```

Java requires you to perform such conversions manually. Primitive data types can be converted into objects of wrapper classes by constructing such objects, or boxing. Similarly, the values of primitive data types can be extracted from the objects of wrapper classes by calling an appropriate method on such objects, or unboxing. For more information about boxing and unboxing, see [Boxing Conversion \(C# Programming Guide\)](#) or [Unboxing Conversion \(C# Programming Guide\)](#).

See Also

Reference

[Data Types \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C#](#)

[The C# Programming Language for Java Developers](#)

Operators (C# vs. Java)

C# offers all applicable operators supported by Java, as listed in the following table. At the end of the table, you will see some new operators available in C# but not Java:

Category	Symbol
Unary	<code>++ -- + - ! ~ ()</code>
Multiplicative	<code>* / %</code>
Additive	<code>+ -</code>
Shift	<code><< >></code>
Relational	<code>< > <= >= instanceof</code>
Equality	<code>== !=</code>
Logical AND	<code>&</code>
Logical XOR	<code>^</code>
Logical OR	<code> </code>
Conditional AND	<code>&&</code>
Conditional OR	<code> </code>
Conditional	<code>? :</code>
Assignment	<code>= *= /= %= += -= <<= >>= &= ^= =</code>
Type of Operand	<code>typeof</code>
Size of Operand	<code>sizeof</code>
Enforce Overflow Checking	<code>checked</code>
Suppress Overflow Checking	<code>unchecked</code>

The only Java operator not available in C# is the shift operator (`>>>`). This operator is present in Java because of the lack of unsigned variables in that language, for cases when right-shifting is required to insert a 1 in the most significant bits.

C# supports unsigned variables, and therefore, C# only needs the standard `>>` operator. This operator produces different results depending on whether the operand is signed or unsigned. Right-shifting an unsigned number inserts 0 in the most significant bit while right-shifting a signed number copies the previous most significant bit.

Checked and Unchecked Operators

Arithmetic operations will result in overflow if the result is too large for the number of bits allocated to the data type in use. Such overflow can be checked or ignored for a given integral arithmetic operation using the **checked** and **unchecked** keywords. If the expression is a constant expression using **checked**, an error is generated at compile time.

The following is a simple example to illustrate these operators:

C#

```

class TestCheckedAndUnchecked
{
    static void Main()
    {
        short a = 10000;
        short b = 10000;

        short c = (short)(a * b);           // unchecked by default
        short d = unchecked((short)(10000 * 10000)); // unchecked
        short e = checked((short)(a * b));   // checked - run-time error

        System.Console.WriteLine(10000 * 10000); // 100000000
        System.Console.WriteLine(c);             // -7936
        System.Console.WriteLine(d);             // -7936
        System.Console.WriteLine(e);             // no result
    }
}

```

In this code, the unchecked operator circumvents the compile-time error that would otherwise be caused by the following statement:

C#

```

short d = unchecked((short)(10000 * 10000)); // unchecked

```

The next expression is unchecked by default, so the value silently overflows:

C#

```

short c = (short)(a * b);           // unchecked by default

```

We can force the expression to be checked for overflow at run time with the checked operator:

C#

```

short e = checked((short)(a * b));   // checked - run-time error

```

Assigning the first two values to `d` and `c` silently overflows with a value of -7936 when the program is run, but when attempting to multiply the value for `e` using `checked()`, the program will throw a [OverflowException](#).

Note

You can also control whether to check for arithmetic overflow in a block of code by using the command-line compiler switch **(/checked)** or directly in Visual Studio on a per-project basis.

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Flow Control (C# vs. Java)

Flow control statements, such as **if else** and **switch** statements, are very similar in both Java and C#.

Branching Statements

Branching statements change the flow of program execution at run time according to certain conditions.

if, else, and else if

These statements are identical in both languages.

The switch Statement

In both languages, the **switch** statement provides conditional multiple branching operations. There is a difference though in that Java enables you to "fall through" a case and execute the next case unless you use a **break** statement at the end of the case. C#, however, requires the use of either a **break** or a **goto** statement at the end of each case, and if neither is present, the compiler produces the following error:

Control cannot fall through from one case label to another.

Note that where a case does not specify any code to execute when that case is matched, control will fall through to the subsequent case. When using **goto** in a **switch** statement, you can only jump to another case block in the same switch. If you want to jump to the default case, you would use `goto default`. Otherwise, you would use `goto case cond`, where `cond` is the matching condition of the case you intend to jump to. Another difference from Java's **switch** is that in Java, you can only switch on integer types, while C# enables you to switch on a string variable.

For example, the following would be valid in C#, but not in Java:

C#

```
static void Main(string[] args)
{
    switch (args[0])
    {
        case "copy":
            //...
            break;

        case "move":
            //...
            goto case "delete";

        case "del":
        case "remove":
        case "delete":
            //...
            break;

        default:
            //...
            break;
    }
}
```

The Return of goto

In Java, **goto** is a reserved keyword that is not implemented. However, you can use labeled statements with **break** or **continue** to achieve a similar purpose as **goto**.

C# does allow the **goto** statement to jump to a labeled statement. Note, though, that in order to jump to a particular label, the **goto** statement must be within the scope of the label. In other words, **goto** may not be used to jump into a statement block, although it can jump out of one, to jump out of a class, or to exit the **finally** block in **try...catch** statements. The use of **goto** is discouraged in most cases, as it contravenes good object-oriented programming practice.

See Also

Concepts[C# Programming Guide](#)**Other Resources**[Visual C#](#)[The C# Programming Language for Java Developers](#)

Looping Statements (C# vs Java)

Looping statements repeat a specified block of code until a given condition is met.

for Loops

The syntax and operation of [for](#) loops is the same in both C# and Java:

C#

```
for (int i = 0; i<=9; i++)
{
    System.Console.WriteLine(i);
}
```

foreach Loops

C# introduces a new loop type called the [foreach](#) loop, which is similar to Visual Basic's **For Each**. The **foreach** loop enables iterating through each item in a container class, such as an array, that supports the [IEnumerable](#) interface. The following code illustrates the use of the **foreach** statement to output the contents of an array:

C#

```
static void Main()
{
    string[] arr= new string[] { "Jan", "Feb", "Mar" };

    foreach (string s in arr)
    {
        System.Console.WriteLine(s);
    }
}
```

For more information, see [Arrays \(C# vs Java\)](#).

while and do...while Loops

The syntax and operation of [while](#) and [do...while](#) statements are the same in both languages:

C#

```
while (condition)
{
    // statements
}
```

C#

```
do
{
    // statements
}
while(condition); // Don't forget the trailing ; in do...while loops
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Class Fundamentals (C# vs Java)

The following sections compare C# and Java modifiers.

Access Modifiers

C# modifiers are quite similar to those in Java, with several small differences. Each member of a class, or the class itself, can be declared with an access modifier to define the scope of permitted access. Classes that are not declared inside other classes can only specify the public or internal modifiers. Nested classes, like other class members, can specify any of the following five access modifiers:

- **public**
Visible to all.
- **protected**
Visible only from derived classes.
- **private**
Visible only within the given class.
- **internal**
Visible only within the same assembly.
- **protected internal**
Visible only to the current assembly or types derived from the containing class.

The public, protected, and private Modifiers

A **public** modifier makes the member available anywhere, both inside and outside of the class. A **protected** modifier indicates that access is limited to within the containing class or classes derived from it. A **private** modifier means that access is only possible from within the containing type. In C#, the default access modifier is private, while in Java, access defaults to anywhere from within the containing package.

The internal Modifier

An **internal** item may only be accessed within the current assembly. An assembly in the .NET Framework equates roughly to Java's JAR file; it represents the building blocks from which other programs can be constructed.

The protected internal Modifier

A **protected internal** item is visible only to the current assembly or types derived from the containing class.

The sealed Modifier

A class with the **sealed** modifier on its class declaration is the opposite of an abstract class: it cannot be inherited. You can mark a class as sealed to prevent other classes from overriding its functionality. Naturally, a sealed class cannot be abstract. Also note that a **struct** is implicitly sealed; therefore, it cannot be inherited. The **sealed** modifier is equivalent to marking a class with the **final** keyword in Java.

The readonly Modifier

To define a constant in C#, use the **const** or **readonly** modifier in place of Java's **final** keyword. The distinguishing factor between the two modifiers in C# is that **const** items are dealt with at compile-time, while the values of **readonly** fields are specified at run time. This means that assignment to **readonly** fields may occur in the class constructor as well as in the declaration. For example, the following class declares a **readonly** variable called IntegerVariable that is initialized in the class constructor:

C#

```
public class SampleClass
{
    private readonly int intConstant;

    public SampleClass () //constructor
```

```

{
    // You are allowed to set the value of the readonly variable
    // inside the constructor
    intConstant = 5;
}

public int IntegerConstant
{
    set
    {
        // You are not allowed to set the value of the readonly variable
        // anywhere else but inside the constructor

        // intConstant = value; // compile-time error
    }
    get
    {
        return intConstant;
    }
}
}
class TestSampleClass
{
    static void Main()
    {
        SampleClass obj= new SampleClass();

        // You cannot perform this operation on a readonly field.
        obj.IntegerConstant = 100;

        System.Console.WriteLine("intConstant is {0}", obj.IntegerConstant); // 5
    }
}

```

If a **readonly** modifier is applied to a static field, it should be initialized in the static constructor of the class.

See Also

Reference

[Access Modifiers \(C# Programming Guide\)](#)

[Constants \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Main () and Other Methods (C# vs Java)

This section discusses methods and how method parameters are passed by reference and by value.

The Main () Method

Every C# application must contain a single **Main** method specifying where program execution is to begin. In C#, **Main** is capitalized, while Java uses lowercase **main**.

Main can only return **int** or **void**, and has an optional string array argument to represent command-line parameters:

C#

```
static int Main(string[] args)
{
    //...
    return 0;
}
```

The string array parameter that contains any command-line arguments passed in works just as in Java. Thus, `args[0]` specifies the first command-line parameter, `args[1]` denotes the second parameter, and so on. Unlike C++, the `args` array does not contain the name of the EXE file.

Other Methods

When you pass parameters to a method, they can be passed by value or by reference. Value parameters simply take the value of any variable for use in the method. Therefore, the variable value in the calling code is not affected by actions performed on the parameters within a method.

Reference parameters, however, point to a variable declared in the calling code, and therefore, methods will modify the contents of that variable when passed by reference.

Passing by Reference

In both Java and C#, method parameters that refer to an object are always passed by reference, while primitive data type parameters are passed by value.

In C#, all parameters are passed by value by default. To pass by reference, you need to specify one of the keywords **ref** or **out**. The difference between these two keywords is in the parameter initialization. A **ref** parameter must be initialized before use, while an **out** parameter does not have to be explicitly initialized before being passed and any previous value is ignored.

The ref Keyword

Specify this keyword on a parameter when you want the called method to permanently change the value of variables used as parameters. This way, rather than passing the value of a variable used in the call, a reference to the variable itself is passed. The method then works on the reference, so that changes to the parameter during the method's execution are persisted to the original variable used as a parameter to the method.

The following code illustrates this in the `Add` method, where the second `int` parameter is passed by reference with the **ref** keyword:

C#

```
class TestRef
{
    private static void Add(int i, ref int result)
    {
        result += i;
        return;
    }

    static void Main()
    {
        int total = 20;
        System.Console.WriteLine("Original value of 'total': {0}", total);
    }
}
```



```

        Add(10, ref total);
        System.Console.WriteLine("Value after calling Add(): {0}", total);
    }
}

```

The output of this simple example demonstrates that changes made to the result parameter are reflected in the variable, `total`, used in the `Add` method call:

Original value of 'total': 20

Value after calling Add(): 30

This is because the result parameter references the actual memory location occupied by the `total` variable in the calling code. A property of a class is not a variable, and cannot be used directly as a *ref* parameter.

The **ref** keyword must precede the parameter when the method is called, as well as in the method declaration.

The out Keyword

The **out** keyword has a very similar effect to the **ref** keyword, and modifications made to a parameter declared using **out** will be visible outside the method. The two differences from **ref** are that any initial value of an *out* parameter is ignored within the method, and secondly that an *out* parameter must be assigned to during the method:

C#

```

class TestOut
{
    private static void Add(int i, int j, out int result)
    {
        // The following line would cause a compile error:
        // System.Console.WriteLine("Initial value inside method: {0}", result);

        result = i + j;
        return;
    }

    static void Main()
    {
        int total = 20;
        System.Console.WriteLine("Original value of 'total': {0}", total);

        Add(33, 77, out total);
        System.Console.WriteLine("Value after calling Add(): {0}", total);
    }
}

```

In this case, the third parameter to the `Add` method is declared with the **out** keyword, and calls to the method also require the **out** keyword for that parameter. The output will be:

Original value of 'total': 20

Value after calling Add(): 110

So, to sum up, use the **ref** keyword when you want a method to modify an existing variable, and use the **out** keyword to return a value produced inside the method. It is generally used in conjunction with the method's return value when the method produces more than one result value for the calling code.

See Also

Reference

[Main\(\) and Command Line Arguments \(C# Programming Guide\)](#)

[Passing Parameters \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Using an Indeterminate Number of Parameters (C# vs Java)

C# allows you to send a variable number of parameters to a method by specifying the [params](#) keyword when the method is declared. The argument list can contain regular parameters also, but note that the parameter declared with the **params** keyword must come last. It takes the form of a variable length array, and there can be only one *params* parameter per method.

When the compiler tries to resolve a method call, it looks for a method whose argument list matches the method called. If no method overload that matches the argument list can be found, but there is a matching version with a *params* parameter of the appropriate type, then that method will be called, placing the extra arguments in an array.

The following example demonstrates this idea:

C#

```
class TestParams
{
    private static void Average(string title, params int[] values)
    {
        int sum = 0;
        System.Console.Write("Average of {0} (", title);

        for (int i = 0; i < values.Length; i++)
        {
            sum += values[i];
            System.Console.Write(values[i] + ", ");
        }
        System.Console.WriteLine("): {0}", (float)sum/values.Length);
    }
    static void Main()
    {
        Average ("List One", 5, 10, 15);
        Average ("List Two", 5, 10, 15, 20, 25, 30);
    }
}
```

In the previous example, the method `Average` is declared with a *params* parameter of type integer array, letting you call it with any number of arguments. The output is shown here:

```
Average of List One (5, 10, 15, ): 10
```

```
Average of List Two (5, 10, 15, 20, 25, 30, ): 17.5
```

You can specify a *params* parameter of type [Object](#) if you want to allow indeterminate parameters of different types.

See Also

Reference

[Passing Arrays as Parameters \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Properties (C# vs Java)

In C#, a property is a named member of a class, struct, or interface offering a neat way to access private fields through what are called the **get** and **set** accessor methods.

The following code example declares a property called `Species` for the class `Animal`, which abstracts access to the private variable called `name`:

C#

```
public class Animal
{
    private string name;

    public string Species
    {
        get
        {
            return name;
        }
        set
        {
            name = value;
        }
    }
}
```

Often, the property will have the same name as the internal member that it accesses, but with a capital initial letter, such as `Name` in the above case, or the internal member will have an `_` prefix. Also, note the implicit parameter called `value` used in the **set** accessor; this has the type of the underlying member variable.

Accessors are in fact represented internally as `get_X()` and `set_X()` methods in order to maintain compatibility with the .NET Framework-based languages, which do not support accessors. Once a property is defined, it is then very easy to get or set its value:

C#

```
class TestAnimal
{
    static void Main()
    {
        Animal animal = new Animal();
        animal.Species = "Lion";           // set accessor
        System.Console.WriteLine(animal.Species); // get accessor
    }
}
```

If a property only has a **get** accessor, it is a read-only property. If it only has a **set** accessor, it is a write-only property. If it has both, it is a read-write property.

See Also

Reference

[Properties \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Structs (C# vs Java)

C# supports the **struct** keyword, another item that originates in C but is not available in Java. You can think of a **struct** as a lightweight class. Although **structs** can contain constructors, constants, fields, methods, properties, indexers, operators, and nested types, they are mostly used simply to encapsulate groups of related fields. Because structs are value types, they can be allocated slightly more efficiently than classes. **structs** differ from classes in that they cannot be abstract and do not support implementation inheritance.

In the following example, you initialize a **struct** with the **new** keyword, calling the default no-parameter constructor, and then set the members of the instance.

C#

```
public struct Customer
{
    public int ID;
    public string Name;

    public Customer(int customerID, string customerName)
    {
        ID = customerID;
        Name = customerName;
    }
}

class TestCustomer
{
    static void Main()
    {
        Customer c1 = new Customer(); //using the default constructor

        System.Console.WriteLine("Struct values before initialization:");
        System.Console.WriteLine("ID = {0}, Name = {1}", c1.ID, c1.Name);
        System.Console.WriteLine();

        c1.ID = 100;
        c1.Name = "Robert";

        System.Console.WriteLine("Struct values after initialization:");
        System.Console.WriteLine("ID = {0}, Name = {1}", c1.ID, c1.Name);
    }
}
```

Output

When we compile and run the previous code, its output shows that **struct** variables are initialized by default. The **int** variable is initialized to 0, and the **string** variable to an empty string:

```
Struct values before initialization:
```

```
ID = 0, Name =
```

```
Struct values after initialization:
```

```
ID = 100, Name = Robert
```

See Also

Tasks

[Structs Sample](#)

Concepts

[C# Programming Guide](#)

[Structs \(C# Programming Guide\)](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Arrays (C# vs Java)

Arrays are ordered collections of items of the same data type that are accessed using the array name in conjunction with the offset from the start of the array of the desired item. There are some important differences in how arrays are declared and used in C# compared to Java.

The One-Dimensional Array

A one-dimensional array stores a fixed number of items in a linear fashion, requiring just a single index value to identify any one item. In C#, the square brackets in the array declaration must follow the data type, and cannot be placed after the variable name as is permitted in Java. Thus, an array of type `integers` is declared using the following syntax:

C#

```
int[] arr1;
```

The following declaration is invalid in C#:

C#

```
//int arr2[]; //compile error
```

Once you have declared an array, you use the **new** keyword to set its size, just as in Java. The following declares the array reference:

C#

```
int[] arr;  
arr = new int[5]; // create a 5 element integer array
```

You then access elements in a one-dimensional array using identical syntax to Java. C# array indices are also zero-based. The following accesses the last element of the previous array:

C#

```
System.Console.WriteLine(arr[4]); // access the 5th element
```

Initialization

C# array elements can be initialized at creation using the same syntax as in Java:

C#

```
int[] arr2Lines;  
arr2Lines = new int[5] {1, 2, 3, 4, 5};
```

Unlike Java, the number of C# initializers must match the array size exactly. You can use this feature to declare and initialize a C# array in a single line:

C#

```
int[] arr1Line = {1, 2, 3, 4, 5};
```

This syntax creates an array of size equal to the number of initializers.

Initializing in a Program Loop

The other way to initialize an array in C# is to use the **for** loop. The following loop sets each element of an array to zero:

C#

```
int[] TaxRates = new int[5];

for (int i=0; i<TaxRates.Length; i++)
{
    TaxRates[i] = 0;
}
```

Jagged Arrays

Both C# and Java support creating jagged, or non-rectangular, arrays in which each row contains a different number of columns. For instance, the following jagged array has four entries in the first row, and three in the second:

C#

```
int[][] jaggedArray = new int[2][];
jaggedArray[0] = new int[4];
jaggedArray[1] = new int[3];
```

Multi-Dimensional Arrays

With C#, you can create regular multi-dimensional arrays that are like a matrix of values of the same type. While both Java and C# support jagged arrays, C# also supports multi-dimensional arrays or arrays of arrays.

Declare a multi-dimensional rectangular array using following syntax:

C#

```
int[,] arr2D;      // declare the array reference
float[,,] arr4D;   // declare the array reference
```

Once declared, you allocate memory to the array as follows:

C#

```
arr2D = new int[5,4]; // allocate space for 5 x 4 integers
```

Elements of the array are then accessed using the following syntax:

C#

```
arr2D[4,3] = 906;
```

Because arrays are zero-based, this line sets the element in the fifth column of the fourth row to 906.

Initialization

Multi-dimensional arrays can be created, set up, and initialized in a single statement by any of the following methods:

C#

```
int[,] arr4 = new int [2,3] { {1,2,3}, {4,5,6} };
int[,] arr5 = new int [,]   { {1,2,3}, {4,5,6} };
int[,] arr6 =                { {1,2,3}, {4,5,6} };
```

Initializing in a Program Loop

All the elements of an array can be initialized using a nested loop as shown here:

C#

```
int[,] arr7 = new int[5,4];

for(int i=0; i<5; i++)
{
    for(int j=0; i<4; j++)
    {
        arr7[i,j] = 0; // initialize each element to zero
    }
}
```

The System.Array Class

In the .NET Framework, arrays are implemented as instances of the [Array](#) class. This class provides several useful methods, such as [Sort](#) and [Reverse](#).

The following example demonstrates how easy these methods are to work with. First, you reverse the elements of an array using the **Reverse** method, and then you sort them with the **Sort** method:

C#

```
class ArrayMethods
{
    static void Main()
    {
        // Create a string array of size 5:
        string[] employeeNames = new string[5];

        // Read 5 employee names from user:
        System.Console.WriteLine("Enter five employee names:");
        for(int i=0; i<employeeNames.Length; i++)
        {
            employeeNames[i]= System.Console.ReadLine();
        }

        // Print the array in original order:
        System.Console.WriteLine("\nArray in Original Order:");
        foreach(string employeeName in employeeNames)
        {
            System.Console.Write("{0} ", employeeName);
        }

        // Reverse the array:
        System.Array.Reverse(employeeNames);

        // Print the array in reverse order:
        System.Console.WriteLine("\n\nArray in Reverse Order:");
        foreach(string employeeName in employeeNames)
        {
            System.Console.Write("{0} ", employeeName);
        }

        // Sort the array:
        System.Array.Sort(employeeNames);

        // Print the array in sorted order:
        System.Console.WriteLine("\n\nArray in Sorted Order:");
        foreach(string employeeName in employeeNames)
        {
            System.Console.Write("{0} ", employeeName);
        }
    }
}
```

Output

Enter five employee names:


```
Luca
```

```
Angie
```

```
Brian
```

```
Kent
```

```
Beatriz
```

```
Array in Original Order:
```

```
Luca Angie Brian Kent Beatriz
```

```
Array in Reverse Order:
```

```
Beatriz Kent Brian Angie Luca
```

```
Array in Sorted Order:
```

```
Angie Beatriz Brian Kent Luca
```

See Also

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Inheritance and Derived Classes (C# vs Java)

You can extend the functionality of an existing class by creating a new class that derives from the existing class. The derived class inherits the properties of the base class, and you can add or override methods and properties as required.

In C#, both inheritance and interface implementation are defined by the `:` operator, equivalent to **extends** and **implements** in Java. The base class should always be leftmost in the class declaration.

Like Java, C# does not support multiple inheritance, meaning that classes cannot inherit from more than one class. You can, however, use interfaces for that purpose in the same way as in Java.

The following code defines a class called `CoOrds` with two private member variables `x` and `y` representing the position of the point. These variables are accessed through properties called `X` and `Y` respectively:

C#

```
public class CoOrds
{
    private int x, y;

    public CoOrds() // constructor
    {
        x = 0;
        y = 0;
    }

    public int X
    {
        get { return x; }
        set { x = value; }
    }

    public int Y
    {
        get { return y; }
        set { y = value; }
    }
}
```

You derive a new class, called `ColorCoOrds`, from the `CoOrds` class, as follows:

C#

```
public class ColorCoOrds : CoOrds
```

`ColorCoOrds` then inherits all the fields and methods of the base class, to which you can add new ones to provide extra features in the derived class according to our needs. In this example, you add a private member and accessors to add color to the class:

C#

```
public class ColorCoOrds : CoOrds
{
    private System.Drawing.Color screenColor;

    public ColorCoOrds() // constructor
    {
        screenColor = System.Drawing.Color.Red;
    }

    public System.Drawing.Color ScreenColor
    {
        get { return screenColor; }
        set { screenColor = value; }
    }
}
```

```
}  
}
```

The constructor of the derived class implicitly calls the constructor for the base class, or the superclass in Java terminology. In inheritance, all base class constructors are called before the derived class's constructors in the order that the classes appear in the class hierarchy.

Typecasting to a Base Class

As in Java, you cannot use a reference to a base class to access the members and methods of a derived class even if the base class reference may contain a valid reference to an object of the derived type.

You can reference a derived class with a reference to the derived type implicitly:

C#

```
ColorCoOrds color1 = new ColorCoOrds();  
CoOrds coords1 = color1;
```

In this code, the base class reference, `coords1`, contains a copy of the `color1` reference.

The base Keyword

You can access base class members in a subclass even when those base members are overridden in the superclass using the `base` keyword. For instance, you can create a derived class which contains a method with the same signature as in the base class. If you prefaced that method with the `new` keyword, you indicate that this is an all-new method belonging to the derived class. You could still provide a method for accessing the original method in the base class with the `base` keyword.

For instance, say your base `CoOrds` class had a method called `Invert()`, which swaps the `x` and `y` coordinates. You could provide a substitute for this method in your derived `ColorCoOrds` class with code like this:

C#

```
public new void Invert()  
{  
    int temp = X;  
    X = Y;  
    Y = temp;  
    screenColor = System.Drawing.Color.Gray;  
}
```

As you can see, this method swaps `x` and `y`, and then sets the coordinates' color to gray. You could provide access to the base implementation for this method by creating another method in `ColorCoOrds`, such as this one:

C#

```
public void BaseInvert()  
{  
    base.Invert();  
}
```

You then invoke the base method on a `ColorCoOrds` object by calling the `BaseInvert()` method.

C#

```
ColorCoOrds color1 = new ColorCoOrds();  
color1.BaseInvert();
```

Remember that you would get the same effect if you assigned a reference to the base class to an instance of `ColorCoOrds`, and then accessed its methods:

C#

```
CoOrds coords1 = color1;
coords1.Invert();
```

Selecting Constructors

Base class objects are always constructed before any deriving class. Thus the constructor for the base class is executed before the constructor of the derived class. If the base class has more than one constructor, the derived class can decide the constructor to be called. For example, you could modify your `CoOrds` class to add a second constructor, as follows:

C#

```
public class CoOrds
{
    private int x, y;

    public CoOrds()
    {
        x = 0;
        y = 0;
    }

    public CoOrds(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}
```

You could then change the `ColorCoOrds` class to use a particular one of the available constructors using the `base` keyword:

C#

```
public class ColorCoOrds : CoOrds
{
    public System.Drawing.Color color;

    public ColorCoOrds() : base ()
    {
        color = System.Drawing.Color.Red;
    }

    public ColorCoOrds(int x, int y) : base (x, y)
    {
        color = System.Drawing.Color.Red;
    }
}
```

In Java, this functionality is implemented using the **super** keyword.

Method Overriding

A derived class may override the method of a base class by providing a new implementation for the declared method. An important distinction between Java and C# is that by default, Java methods are marked as `virtual`, while in C#, methods must be explicitly marked as `virtual` using the `virtual` modifier. Property accessors, as well as methods, can be overridden in much the same way.

Virtual Methods

A method that is to be overridden in a derived class is declared with the **virtual** modifier. In a derived class, the overridden method is declared using the `override` modifier.

The **override** modifier denotes a method or a property of a derived class that replaces one with the same name and signature in the base class. The base method, which is to be overridden, must be declared as **virtual**, **abstract**, or **override**: it is not possible to override a non-virtual or static method in this way. Both the overridden and the overriding method or property

must have the same access-level modifiers.

The following example shows a virtual method called `StepUp` that is overridden in a derived class with the `override` modifier:

```
C#  
  
public class CountClass  
{  
    public int count;  
  
    public CountClass(int startValue) // constructor  
    {  
        count = startValue;  
    }  
  
    public virtual int StepUp()  
    {  
        return ++count;  
    }  
}  
  
class Count100Class : CountClass  
{  
    public Count100Class(int x) : base(x) // constructor  
    {  
    }  
  
    public override int StepUp()  
    {  
        return ((base.count) + 100);  
    }  
}  
  
class TestCounters  
{  
    static void Main()  
    {  
        CountClass counter1 = new CountClass(1);  
        CountClass counter100 = new Count100Class(1);  
  
        System.Console.WriteLine("Count in base class = {0}", counter1.StepUp());  
        System.Console.WriteLine("Count in derived class = {0}", counter100.StepUp());  
    }  
}
```

When you run this code, you see that the derived class's constructor uses the method body given in the base class, letting you initialize the count member without duplicating that code. Here is the output:

```
Count in base class = 2
```

```
Count in derived class = 101
```

Abstract Classes

An abstract class declares one or more methods or properties as abstract. Such methods do not have an implementation provided in the class that declares them, although an abstract class can also contain non-abstract methods, that is, methods for which an implementation has been provided. An abstract class cannot be instantiated directly, but only as a derived class. Such derived classes must provide implementations for all abstract methods and properties, using the **override** keyword, unless the derived member is itself declared abstract.

The following example declares an abstract `Employee` class. You also create a derived class called `Manager` that provides an implementation of the abstract `Show()` method defined in the `Employee` class:

```
C#  
  
public abstract class Employee  
{  
    protected string name;
```

```

public Employee(string name) // constructor
{
    this.name = name;
}

public abstract void Show(); // abstract show method
}

public class Manager: Employee
{
    public Manager(string name) : base(name) {} // constructor

    public override void Show() //override the abstract show method
    {
        System.Console.WriteLine("Name : " + name);
    }
}

class TestEmployeeAndManager
{
    static void Main()
    {
        // Create an instance of Manager and assign it to a Manager reference:
        Manager m1 = new Manager("H. Ackerman");
        m1.Show();

        // Create an instance of Manager and assign it to an Employee reference:
        Employee ee1 = new Manager("M. Knott");
        ee1.Show(); //call the show method of the Manager class
    }
}

```

This code invokes the implementation of `Show()` provided by the `Manager` class, and prints the employee names on the screen. Here is the output:

Name : H. Ackerman

Name : M. Knott

Interfaces

An interface is a sort of skeleton class, containing method signatures but no method implementations. In this way, interfaces are like abstract classes that contain only abstract methods. C# interfaces are very similar to Java interfaces, and work in very much the same way.

All the members of an interface are public by definition, and an interface cannot contain constants, fields (private data members), constructors, destructors, or any type of static member. The compiler will generate an error if any modifier is specified for the members of an interface.

You can derive classes from an interface in order to implement that interface. Such derived classes must provide implementations for all the interface's methods unless the derived class is declared abstract.

An interface is declared identically to Java. In an interface definition, a property indicates only its type, and whether it is read-only, write-only, or read/write by `get` and `set` keywords alone. The interface below declares one read-only property:

C#

```

public interface ICDPlayer
{
    void Play(); // method signature
    void Stop(); // method signature

    int FastForward(float numberOfSeconds);

    int CurrentTrack // read-only property
    {
        get;
    }
}

```

```
}
```

A class can inherit from this interface using a colon in place of Java's **implements** keyword. The implementing class must provide definitions for all methods, and any required property accessors, as follows:

C#

```
public class CDPlayer : ICDPlayer
{
    private int currentTrack = 0;

    // implement methods defined in the interface
    public void Play()
    {
        // code to start CD...
    }

    public void Stop()
    {
        // code to stop CD...
    }

    public int FastForward(float numberOfSeconds)
    {
        // code to fast forward CD using numberOfSeconds...

        return 0; //return success code
    }

    public int CurrentTrack // read-only property
    {
        get
        {
            return currentTrack;
        }
    }

    // Add additional methods if required...
}
```

Implementing Multiple Interfaces

A class can implement multiple interfaces using the following syntax:

C#

```
public class CDAndDVDComboPlayer : ICDPlayer, IDVDPlayer
```

If a class implements more than one interface where there is ambiguity in the names of members, it is resolved using the full qualifier for the property or method name. In other words, the derived class can resolve the conflict by using the fully qualified name for the method to indicate to which interface it belongs, as in `ICDPlayer.Play()`.

See Also

Reference

[Inheritance \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Events (C# vs. Java)

An event is a way for a class to notify the users of an object when something of interest happens to this object, such as, clicking a control in a graphical user interface. This notification is called raising an event. An object raising an event is referred to as the source or sender of the event.

Unlike event handling in Java, which is performed by implementing custom listener classes, C# developers can use delegates for event handling. A delegate is a type that references a method. Once a delegate is assigned a method, it behaves exactly like that method. It is similar to a C++ function pointer, but it is type-safe.

The delegate method can be used like any other method, with parameters and a return value, such as this example:

```
public delegate int ReturnResult(int x, int y);
```

For more information on delegates, see [Delegates \(C# Programming Guide\)](#).

Events, like methods, have a signature that includes a name and a parameter list. This signature is defined by a delegate type, for example:

```
public delegate void MyEventHandler(object sender, System.EventArgs e);
```

It is common to have the first parameter as the object referring to the source of the event, and the second parameter as the object carrying data related to the event. However, this design is not required or enforced by the C# language; an event signature can be the same as any valid delegate signature, as long as it returns void.

An event can be declared by using the **event** keyword like this example:

```
public event MyEventHandler TriggerIt;
```

To trigger the event, define the method to be invoked when the event is raised like this example:

```
public void Trigger()
{
    TriggerIt();
}
```

To raise an event, call the delegate, and pass the parameters related to the event. The delegate then calls all the handlers that have been added to the event. Each event can have more than one handler assigned to receive the event. In this case, the event calls each receiver automatically. Raising an event requires only one call to the event regardless of the number of receivers.

If you want a class to receive an event, subscribe to that event by adding the delegate to the event using the **+=** operator, for example:

```
myEvent.TriggerIt += new MyEventHandler(myEvent.MyMethod);
```

To unsubscribe from an event, remove the delegate from the event by using the **-=** operator, for example:

```
myEvent.TriggerIt -= new MyEventHandler(myEvent.MyMethod);
```

For more information on events, see [Events \(C# Programming Guide\)](#).

Note

In C# 2.0, delegates can encapsulate both named methods and anonymous methods. For more information on anonymous methods, see [Anonymous Methods \(C# Programming Guide\)](#).

Example

The following example defines an event with three methods associated with it. When the event is triggered the methods are

executed. One method is then removed from the event and the event is triggered again.

```
// Declare the delegate handler for the event:
public delegate void MyEventHandler();

class TestEvent
{
    // Declare the event implemented by MyEventHandler.
    public event MyEventHandler TriggerIt;

    // Declare a method that triggers the event:
    public void Trigger()
    {
        TriggerIt();
    }
    // Declare the methods that will be associated with the TriggerIt event.
    public void MyMethod1()
    {
        System.Console.WriteLine("Hello!");
    }
    public void MyMethod2()
    {
        System.Console.WriteLine("Hello again!");
    }
    public void MyMethod3()
    {
        System.Console.WriteLine("Good-bye!");
    }

    static void Main()
    {
        // Create an instance of the TestEvent class.
        TestEvent myEvent = new TestEvent();

        // Subscribe to the event by associating the handlers with the events:
        myEvent.TriggerIt += new MyEventHandler(myEvent.MyMethod1);
        myEvent.TriggerIt += new MyEventHandler(myEvent.MyMethod2);
        myEvent.TriggerIt += new MyEventHandler(myEvent.MyMethod3);
        // Trigger the event:
        myEvent.Trigger();

        // Unsubscribe from the the event by removing the handler from the event:
        myEvent.TriggerIt -= new MyEventHandler(myEvent.MyMethod2);
        System.Console.WriteLine("\"Hello again!\" unsubscribed from the event.");

        // Trigger the new event:
        myEvent.Trigger();
    }
}
```

Output

```
Hello!
Hello again!
Good-bye!
"Hello again!" unsubscribed from the event.
Hello!
Good-bye!
```

See Also

Reference

[event \(C# Reference\)](#)

[delegate \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# for Java Developers](#)

Operator Overloading (C# vs Java)

Like C++, C# allows you to overload operators for use on your own classes. This makes it possible for a user-defined data type to look as natural and be as logical to use as a fundamental data type. For example, you might create a new data type called `ComplexNumber` to represent a complex number, and provide methods that perform mathematical operations on such numbers using the standard arithmetic operators, such as using the `+` operator to add two complex numbers.

To overload an operator, you write a function that has the name operator followed by the symbol for the operator to be overloaded. For instance, this is how you would overload the `+` operator:

C#

```
public static ComplexNumber operator+(ComplexNumber a, ComplexNumber b)
```

All operator overloads are static methods of the class. Also be aware that if you overload the equality (`==`) operator, you must overload the inequality operator (`!=`) as well. The `<` and `>` operators, and the `<=` and `>=` operators should also be overloaded in pairs.

The full list of operators that can be overloaded is:

- Unary operators: `+`, `-`, `!`, `~`, `++`, `--`, **true**, **false**
- Binary operators: `+`, `-`, `*`, `/`, `%`, `&`, `|`, `^`, `<<`, `>>`, `==`, `!=`, `>`, `<`, `>=`, `<=`

The following code example creates a `ComplexNumber` class that overloads the `+` and `-` operators:

C#

```
public class ComplexNumber
{
    private int real;
    private int imaginary;

    public ComplexNumber() : this(0, 0) // constructor
    {
    }

    public ComplexNumber(int r, int i) // constructor
    {
        real = r;
        imaginary = i;
    }

    // Override ToString() to display a complex number in the traditional format:
    public override string ToString()
    {
        return(System.String.Format("{0} + {1}i", real, imaginary));
    }

    // Overloading '+' operator:
    public static ComplexNumber operator+(ComplexNumber a, ComplexNumber b)
    {
        return new ComplexNumber(a.real + b.real, a.imaginary + b.imaginary);
    }

    // Overloading '-' operator:
    public static ComplexNumber operator-(ComplexNumber a, ComplexNumber b)
    {
        return new ComplexNumber(a.real - b.real, a.imaginary - b.imaginary);
    }
}
```

This class enables you to create and manipulate two complex numbers with code such as this:

C#

```
class TestComplexNumber
{
    static void Main()
    {
        ComplexNumber a = new ComplexNumber(10, 12);
        ComplexNumber b = new ComplexNumber(8, 9);

        System.Console.WriteLine("Complex Number a = {0}", a.ToString());
        System.Console.WriteLine("Complex Number b = {0}", b.ToString());

        ComplexNumber sum = a + b;
        System.Console.WriteLine("Complex Number sum = {0}", sum.ToString());

        ComplexNumber difference = a - b;
        System.Console.WriteLine("Complex Number difference = {0}", difference.ToString());
    }
}
```

As the program demonstrates, you can now use the plus and minus operators on objects belonging to your `ComplexNumber` class quite intuitively. Here is the output you would get:

Complex Number a = 10 + 12i

Complex Number b = 8 + 9i

Complex Number sum = 18 + 21i

Complex Number difference = 2 + 3i

Java does not support operator overloading, although internally it overloads the `+` operator for string concatenation.

See Also

Tasks

[Operator Overloading Sample](#)

Reference

[Overloadable Operators \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Exceptions (C# vs Java)

Exception handling in C# is very similar to that of Java.

Whenever something goes critically wrong during the execution of a program, the .NET Framework common language runtime (CLR) creates an [Exception](#) object detailing the error. In the .NET Framework, **Exception** is the base class for all the exception classes. There are two categories of exceptions that derive from the **Exception** class, [SystemException](#) and [ApplicationException](#). All types in the [System](#) namespace derive from **SystemException** while user-defined exceptions should derive from **ApplicationException** to differentiate between run-time and application errors. Some common **System** exceptions include:

- [IndexOutOfRangeException](#): an index greater than the size of an array or collection is used.
- [NullReferenceException](#): a property or method of a reference has been used before that reference has been set to a valid instance.
- [ArithmeticException](#): an operation results in overflow or underflow.
- [FormatException](#): an argument or operand is in an incorrect format.

As in Java, when you have code that is liable to cause an exception, you place that code within a **try** block. One or more **catch** blocks immediately after provide the error handling, and you can also use a **finally** block for any code that you want to execute, whether an exception is thrown or not. For more information, see [try-catch \(C# Reference\)](#), and [try-catch-finally \(C# Reference\)](#).

When using multiple **catch** blocks, the exceptions caught must be placed in order of increasing generality as only the first **catch** block that matches the thrown exception will be executed. The C# compiler will enforce this while the Java compiler will not.

Also, C# does not require an argument for a **catch** block as Java does; in the absence of an argument, the **catch** block applies to any **Exception** class.

For example, while reading from a file, you might encounter a [FileNotFoundException](#) or an [IOException](#), and you would want to place the more specific **FileNotFoundException** handler first, as the following code shows:

C#

```
try
{
    // code to open and read a file
}
catch (System.IO.FileNotFoundException e)
{
    // handle the file not found exception first
}
catch (System.IO.IOException e)
{
    // handle any other IO exceptions second
}
catch
{
    // a catch block without a parameter
    // handle all other exceptions last
}
finally
{
    // this is executed whether or not an exception occurs
    // use to release any external resources
}
```

You can create your own exception classes by deriving from **Exception**. For example, the following code creates an `InvalidDepartmentException` class that you might throw if, say, the department given for a new `Employee` is invalid. The class constructor for your user-defined exception calls the base class constructor using the **base** keyword, sending an appropriate message:

C#

```
public class InvalidDepartmentException : System.Exception
{
    public InvalidDepartmentException(string department) : base("Invalid Department: " + department)
    {
    }
}
```

You could then throw an exception with code like the following:

C#

```
class Employee
{
    private string department;

    public Employee(string department)
    {
        if (department == "Sales" || department == "Marketing")
        {
            this.department = department;
        }
        else
        {
            throw new InvalidDepartmentException(department);
        }
    }
}
```

C# does not support checked exceptions. In Java, these are declared using the **throws** keyword, to specify that a method can throw a particular type of exception that must be handled by the calling code.

See Also

Reference

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[The C# Programming Language for Java Developers](#)

Advanced C# Techniques (C# vs Java)

C# provides some useful language features, such as indexers, attributes, and delegates, which enable advanced programming techniques.

Indexers

Indexers provide a way to access a [class](#) or [struct](#) in the same way as an array. For example, you can have a class that represents a single department in a company. The class could contain the names of all employees in the department, and indexers could allow you to access these names as follows:

C#

```
sales[0] = "Nikki";  
sales[1] = "Becky";
```

Indexers are enabled by defining a property with the following signature, for example, in the class definition:

C#

```
public string this [int index] //indexer
```

You then provide [get](#) and [set](#) methods as you would for a normal property, and it is these accessors that specify what internal member is referred to when the indexer is used.

In the following simple example, you create a class called `Department` that uses indexers to access the employees in that department, internally represented as an array of strings:

C#

```
public class Department  
{  
    private string name;  
    private const int MAX_EMPLOYEES = 10;  
    private string[] employees = new string[MAX_EMPLOYEES]; //employee array  
  
    public Department(string departmentName) //constructor  
    {  
        name = departmentName;  
    }  
  
    public string this [int index] //indexer  
    {  
        get  
        {  
            if (index >= 0 && index < MAX_EMPLOYEES)  
            {  
                return employees[index];  
            }  
            else  
            {  
                throw new System.IndexOutOfRangeException();  
            }  
        }  
        set  
        {  
            if (index >= 0 && index < MAX_EMPLOYEES)  
            {  
                employees[index] = value;  
            }  
            else  
            {  
                throw new System.IndexOutOfRangeException();  
            }  
        }  
    }  
}
```

```

    }
}

// code for the rest of the class...
}

```

You can then create an instance of this class and access it as shown in the following code example:

C#

```

class TestDepartment
{
    static void Main()
    {
        Department sales = new Department("Sales");

        sales[0] = "Nikki";
        sales[1] = "Becky";

        System.Console.WriteLine("The sales team is {0} and {1}", sales[0], sales[1]);
    }
}

```

The output is:

The sales team is Nikki and Becky

For more information, see [Indexers \(C# Programming Guide\)](#).

Attributes

C# provides a mechanism, called an *attribute*, for adding declarative information about types. Attributes are somewhat similar to the concept of annotations in Java. Extra information about a type is placed inside declarative tags that precede the type definition. The following examples show how you can use .NET Framework attributes to decorate a class or method.

In the example below, the `GetTime` method is marked as an XML Web service by adding the [WebMethodAttribute](#) attribute.

C#

```

public class Utilities : System.Web.Services.WebService
{
    [System.Web.Services.WebMethod] // Attribute
    public string GetTime()
    {
        return System.DateTime.Now.ToShortTimeString();
    }
}

```

Adding the **WebMethod** attribute makes the .NET Framework automatically take care of the XML/SOAP interchange necessary to call this function. Calling this Web service retrieves the following value:

```

<?xml version="1.0" encoding="utf-8" ?>
<string xmlns="http://tempuri.org/">7:26 PM</string>

```

In the following example, the `Employee` class is marked as serializable by adding the [SerializableAttribute](#) attribute. While the `Salary` field is marked as public, it will not be serialized as it is marked with the [NonSerializedAttribute](#) attribute.

C#

```

[System.Serializable()]
public class Employee
{
    public int ID;
    public string Name;
    [System.NonSerialized()] public int Salary;
}

```


For more information, see [Creating Custom Attributes \(C# Programming Guide\)](#).

Delegates

Languages such as C++, Pascal, and others support the concept of function pointers that permit you to choose which function you want to call at run time.

Java does not provide any construct with the functionality of a function pointer, but C# does. Through the use of the [Delegate](#) class, a [delegate](#) instance encapsulates a method that is a callable entity.

For instance methods, the delegate consists of an instance of the containing class and a method on the instance. For static methods, a callable entity consists of a class and a static method on the class. Thus, a delegate can be used to invoke a function of any object, and delegates are object-oriented, type- safe, and secure.

There are three steps for defining and using delegates:

- Declaration
- Instantiation
- Invocation

You declare a delegate with the following syntax:

C#

```
delegate void Del1();
```

This delegate can then be used to reference any function that returns void and does not take any arguments.

Similarly, to create a delegate for any function that takes a string parameter and returns a long, you would use the following syntax:

C#

```
delegate long Del2(string s);
```

You could then assign this delegate to any method with this signature, like so:

C#

```
Del2 d; // declare the delegate variable
d = DoWork; // set the delegate to refer to the DoWork method
```

Where the signature of `DoWork` is:

C#

```
public static long DoWork(string name)
```

Reassigning Delegates

Delegate objects are immutable; that is, the signature they match cannot be changed once set. However, you can point to another method as long as both have the same signature. In this example, you reassign `d` to a new delegate object so that `d` then invokes the `DoMoreWork` method. You can only do this if both `DoWork` and `DoMoreWork` have the same signature.

C#

```
Del2 d; // declare the delegate variable
d = DoWork; // set the delegate to refer to the DoWork method
d = DoMoreWork; // reassign the delegate to refer to the DoMoreWork method
```

Invoking Delegates

Invoking a delegate is fairly straightforward. You simply substitute the name of the delegate variable for the method name. This invokes the `Add` method with values 11 and 22, and returns a long result that is assigned to variable `sum`:

C#

```
Del operation;                // declare the delegate variable
operation = Add;              // set the delegate to refer to the Add method
long sum = operation(11, 22); // invoke the delegate
```

The following illustrates the creation, instantiation, and invocation of a delegate:

C#

```
public class MathClass
{
    public static long Add(int i, int j)    // static
    {
        return (i + j);
    }

    public static long Multiply (int i, int j) // static
    {
        return (i * j);
    }
}

class TestMathClass
{
    delegate long Del(int i, int j); // declare the delegate type

    static void Main()
    {
        Del operation; // declare the delegate variable

        operation = MathClass.Add;    // set the delegate to refer to the Add method
        long sum = operation(11, 22); // use the delegate to call the Add method

        operation = MathClass.Multiply; // change the delegate to refer to the Multiply method
        long product = operation(30, 40); // use the delegate to call the Multiply method

        System.Console.WriteLine("11 + 22 = " + sum);
        System.Console.WriteLine("30 * 40 = " + product);
    }
}
```

Output

11 + 22 = 33

30 * 40 = 1200

A delegate instance must contain an object reference. The previous example gets around this by declaring methods as static, which means there is no need to specify an object reference. If a delegate refers to an instance method, however, the object reference must be given as follows:

C#

```
Del operation;                // declare the delegate variable
MathClass m1 = new MathClass(); // declare the MathClass instance
operation = m1.Add;           // set the delegate to refer to the Add method
```

In this example, `Add` and `Multiply` are instance methods of `MathClass`. If the methods of `MathClass` are not declared as static, you invoke them through the delegate by using an instance of the `MathClass`, as follows:

C#

```
public class MathClass
{
    public long Add(int i, int j)          // not static
    {
        return (i + j);
    }

    public long Multiply (int i, int j)  // not static
    {
        return (i * j);
    }
}

class TestMathClass
{
    delegate long Del(int i, int j);  // declare the delegate type

    static void Main()
    {
        Del operation;                // declare the delegate variable
        MathClass m1 = new MathClass(); // declare the MathClass instance

        operation = m1.Add;           // set the delegate to refer to the Add method
        long sum = operation(11, 22); // use the delegate to call the Add method

        operation = m1.Multiply;      // change the delegate to refer to the Multiply method
        long product = operation(30, 40); // use the delegate to call the Multiply method

        System.Console.WriteLine("11 + 22 = " + sum);
        System.Console.WriteLine("30 * 40 = " + product);
    }
}
```

Output

This example provides the same output as the previous example in which the methods were declared as static.

11 + 22 = 33

30 * 40 = 1200

Delegates and Events

The .NET Framework also uses delegates extensively for event-handling tasks like a button click event in a Windows or Web application. While event handling in Java is typically done by implementing custom listener classes, C# developers can take advantage of delegates for event handling. An [event](#) is declared like a field with a delegate type, except that the keyword `event` precedes the event declaration. Events are typically declared `public`, but any accessibility modifier is allowed. The following example shows the declaration of a **delegate** and **event**.

C#

```
// Declare the delegate type:
public delegate void CustomEventHandler(object sender, System.EventArgs e);

// Declare the event variable using the delegate type:
public event CustomEventHandler CustomEvent;
```

Event delegates are multicast, which means that they can hold references to more than one event handling method. A delegate acts as an event dispatcher for the class that raises the event by maintaining a list of registered event handlers for the event. The following example shows how you can subscribe multiple functions to an event. The class `EventClass` contains the delegate, the event, and a method to invoke the event. Note that invoking an event can only be done from within the class that declared the event. The class `TestEvents` can then subscribe to the event using the `+=` operator and unsubscribe using the `-=`

operator. When the `InvokeEvent` method is called, it fires the event and any functions that have subscribed to the event will fire synchronously as shown in the following example.

```
C#

public class EventClass
{
    // Declare the delegate type:
    public delegate void CustomEventHandler(object sender, System.EventArgs e);

    // Declare the event variable using the delegate type:
    public event CustomEventHandler CustomEvent;

    public void InvokeEvent()
    {
        // Invoke the event from within the class that declared the event:
        CustomEvent(this, System.EventArgs.Empty);
    }
}

class TestEvents
{
    private static void CodeToRun(object sender, System.EventArgs e)
    {
        System.Console.WriteLine("CodeToRun is executing");
    }

    private static void MoreCodeToRun(object sender, System.EventArgs e)
    {
        System.Console.WriteLine("MoreCodeToRun is executing");
    }

    static void Main()
    {
        EventClass ec = new EventClass();

        ec.CustomEvent += new EventClass.CustomEventHandler(CodeToRun);
        ec.CustomEvent += new EventClass.CustomEventHandler(MoreCodeToRun);

        System.Console.WriteLine("First Invocation:");
        ec.InvokeEvent();

        ec.CustomEvent -= new EventClass.CustomEventHandler(MoreCodeToRun);

        System.Console.WriteLine("\nSecond Invocation:");
        ec.InvokeEvent();
    }
}
```

Output

```
First Invocation:
CodeToRun is executing
MoreCodeToRun is executing
Second Invocation:
CodeToRun is executing
```

See Also

Tasks

[Delegates Sample](#)

Concepts

[C# Programming Guide](#)

[Delegates \(C# Programming Guide\)](#)

[Events \(C# Programming Guide\)](#)

Other Resources

Garbage Collection (C# vs Java)

In C and C++, many objects require the programmer to allocate their resources once declared, before the objects can be safely used. Releasing these resources back to the free memory pool once the object has been used is also the responsibility of the programmer. If resources are not released, the code is said to leak memory, as more and more resources are consumed needlessly. On the other hand, if resources are released prematurely, loss of data, the corruption of other memory areas, and null pointer exceptions can occur.

Both Java and C# prevent these dangers by independently managing the lifetime of all objects in use by an application.

In Java, the JVM takes care of releasing unused memory by keeping track of the references to allocated resources. Whenever the JVM detects that a resource is no longer referenced by a valid reference, the resource is garbage-collected.

In C#, garbage collection is handled by the common language runtime (CLR) with similar functionality to that of the JVM. The CLR garbage collector periodically checks the memory heap for any unreferenced objects, and releases the resources held by these objects.

See Also

Concepts

[C# Programming Guide](#)

[Automatic Memory Management](#)

Other Resources

[C# Code Examples for Java Developers](#)

Safe and Unsafe Code (C# vs Java)

A particularly interesting feature of C# is its support for non-type-safe code. Normally, the common language runtime (CLR) takes on the responsibility for overseeing the behavior of Microsoft intermediate language (MSIL) code, and prevents any questionable operations. However, there are times when you wish to directly access low-level functionality such as Win32 API calls, and you are permitted to do this, as long as you take responsibility for ensuring such code operates correctly. Such code must be placed inside unsafe blocks in our source code.

The unsafe Keyword

C# code that makes low-level API calls, uses pointer arithmetic, or carries out some other unsavory operation, has to be placed inside blocks marked with the **unsafe** keyword. Any of the following can be marked as unsafe:

- An entire method.
- A code block in braces.
- An individual statement.

The following example demonstrates the use of **unsafe** in all three of the above situations:

C#

```
class TestUnsafe
{
    unsafe static void PointyMethod()
    {
        int i=10;

        int *p = &i;
        System.Console.WriteLine("*p = " + *p);
        System.Console.WriteLine("Address of p = {0:X2}\n", (int)p);
    }

    static void StillPointy()
    {
        int i=10;

        unsafe
        {
            int *p = &i;
            System.Console.WriteLine("*p = " + *p);
            System.Console.WriteLine("Address of p = {0:X2}\n", (int)p);
        }
    }

    static void Main()
    {
        PointyMethod();
        StillPointy();
    }
}
```

In this code, the entire `PointyMethod()` method is marked unsafe because the method declares and uses pointers. The `StillPointy()` method marks a block of code as unsafe as this block once again uses pointers.

The fixed Keyword

In safe code, the garbage collector is quite free to move an object during its lifetime in its mission to organize and condense free resources. However, if your code uses pointers, this behavior could easily cause unexpected results, so you can instruct the garbage collector not to move certain objects using the **fixed** statement.

The following code shows the **fixed** keyword being used to ensure that an array is not moved by the system during the execution of a block of code in the `PointyMethod()` method. Note that **fixed** is only used within unsafe code:

C#

```
class TestFixed
{
    public static void PointyMethod(char[] array)
    {
        unsafe
        {
            fixed (char *p = array)
            {
                for (int i=0; i<array.Length; i++)
                {
                    System.Console.Write(*(p+i));
                }
            }
        }

        static void Main()
        {
            char[] array = { 'H', 'e', 'l', 'l', 'o' };
            PointyMethod(array);
        }
    }
}
```

See Also

Tasks

[Unsafe Code Sample](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

Other Resources

[The C# Programming Language for Java Developers](#)

C# Code Examples for Java Developers

C# is an elegant, simple, type-safe, object-oriented language that allows programmers to build a breadth of applications. Combined with the .NET Framework, Visual C# allows the creation of Windows applications, web services, database tools, components, controls and more.

In This Section

[Console Application Development \(C# vs Java\)](#)

Describes console applications in C# for the Java developer.

[File I/O \(C# vs Java\)](#)

Describes file I/O operations in C# for the Java developer and links to XML classes for XML file loading and saving.

[Database Access \(C# vs Java\)](#)

Compares data access in both languages.

[User Interface Development \(C# vs Java\)](#)

Describes creating Windows Forms application in C# for the Java developer.

[Resource Management \(C# vs Java\)](#)

Provides topics on Windows resource files in C# the Java developer.

[Web Services Applications \(C# vs Java\)](#)

Describes XML Web service application development in C# for the Java developer.

[Mobile Devices and Data \(C# vs Java\)](#)

Describes interacting with databases using C# and .NET Framework Windows Forms, as well as SQL Server, for the Java developer.

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Migrating to Visual C#](#)

[Writing Applications with Visual C#](#)

Console Application Development (C# vs Java)

Console applications read and write to and from the standard input and output (I/O) without any graphical user interface. The anatomy of a console application is similar in Java and C#, and similar classes are used for console I/O.

While the details of the classes and their method signatures might vary, C# and Java use similar concepts to perform a console I/O operation. Both C# and Java have the concept of a main entry point for the console application and associated console read and write methods. In C#, this is `Main`, and in Java, it is `main`.

Java "Hello World" Example

In the Java example code that follows, a `static void main()` routine accepts a `String` reference to the application's arguments. The `main` routine then prints a line to the console.

```
/* A Java Hello World Console Application */
public class Hello {
    public static void main (String args[]) {
        System.out.println ("Hello World");
    }
}
```

C# "Hello World" Example

In the C# example code that follows, a `static void Main()` routine accepts a `string` reference to the application's arguments. The `Main` routine then writes a line to the console.

C#

```
// A C# Hello World Console Application.
public class Hello
{
    static void Main()
    {
        System.Console.WriteLine("Hello World");
    }
}
```

Compiling the Code

If you are using Visual C#, you can compile and run the code in a single step by pressing F5. If you are using the command line and your file is named "Hello.cs," you invoke the C# compiler like this:

csc Hello.cs

For More Information

For more information about creating a console application, see [Creating Console Applications \(Visual C#\)](#).

For more information about .NET Framework console classes, see:

- [Console](#) class, [WriteLine](#) and [ReadLine](#) methods.
- [C# Compiler Options Listed by Category](#).
- [What's New in the C# 2.0 Language and Compiler](#)

For more information about automated conversion from Java to C#, see [What's New in Java Language Conversion Assistant](#).

See Also

Reference

[Main\(\) and Command Line Arguments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# for Java Developers](#)

File I/O (C# vs Java)

While the details of the classes and method signatures may vary, C# and Java use similar concepts for performing a file I/O operation. Both C# and Java have the concept of a file class and associated file read and write methods. Similar Document Object Models (DOM) exist for handling XML content.

Java File Operations Example

In Java, you can use the **File** object to perform basic file I/O operations, such as creating, opening, closing, reading, and writing a file. For example, you can use the methods of the **File** class to perform file I/O operations, such as using the **createNewFile** or **delete** methods of the **File** class to create or delete a file. You can use the **BufferedReader** and **BufferedWriter** classes to read and write the contents of files.

The following code example shows how to create a new file, delete a file, read text from a file, and write to a file.

```
// Java example code to create a new file
try
{
    File file = new File("path and file_name");
    boolean success = file.createNewFile();
}
catch (IOException e)    {    }

// Java example code to delete a file.
try
{
    File file = new File("path and file_name");
    boolean success = file.delete();
}
catch (IOException e)    {    }

// Java example code to read text from a file.
try
{
    BufferedReader infile = new BufferedReader(new FileReader("path and file_name "));
    String str;
    while ((str = in.readLine()) != null)
    {
        process(str);
    }
    infile.close();
}
catch (IOException e)
{
    // Exceptions ignored.
}

// Java example code to writing to a file.
try
{
    BufferedWriter outfile =
        new BufferedWriter(new FileWriter("path and file_name "));
    outfile.write("a string");
    outfile.close();
}
catch (IOException e)    {    }
```

C# File Operations Example

In C#, to perform a file I/O operation, you can use the same familiar basics of creating, opening, closing, reading, and writing using .NET Framework equivalent classes and methods. For example, you can use the methods of the [File](#) class of the .NET Framework to perform file I/O operations. For example, you can check for the existence of a file using the [Exists](#) method. You can use the [Create](#) method to create a file, optionally overwriting an existing file as illustrated in the following code example, and you can read and write using the [FileStream](#) class and the [BufferedStream](#) object.

The following code example shows how to delete a file, create a file, write to a file, and read from it.

C#

```
// sample C# code for basic file I/O operations
// exceptions ignored for code simplicity

class TestFileIO
{
    static void Main()
    {
        string fileName = "test.txt"; // a sample file name

        // Delete the file if it exists.
        if (System.IO.File.Exists(fileName))
        {
            System.IO.File.Delete(fileName);
        }

        // Create the file.
        using (System.IO.FileStream fs = System.IO.File.Create(fileName, 1024))
        {
            // Add some information to the file.
            byte[] info = new System.Text.UTF8Encoding(true).GetBytes("This is some text in
the file.");
            fs.Write(info, 0, info.Length);
        }

        // Open the file and read it back.
        using (System.IO.StreamReader sr = System.IO.File.OpenText(fileName))
        {
            string s = "";
            while ((s = sr.ReadLine()) != null)
            {
                System.Console.WriteLine(s);
            }
        }
    }
}
```

Related Sections

.NET Framework classes useful for creating, reading, and writing to streams include [StreamReader](#) and [StreamWriter](#) classes. Other .NET Framework classes useful for handling and processing files include:

- [FileAccess](#) and [FileAttribute](#) classes.
- [Directory](#) and [DirectoryInfo](#), [Path](#), [FileInfo](#), and [DriveInfo](#) classes.
- [BinaryReader](#) and [BinaryWriter](#) classes.
- [StringReader](#) and [StringWriter](#) classes.
- [TextReader](#) and [TextWriter](#) classes
- [XmlReader](#) and [XmlWriter](#) classes.
- [ToBase64Transform](#) class.
- **FileStream**, **BufferedStream**, and [MemoryStream](#) classes.

For more information about automated conversion from Java to C# see [What's New in Java Language Conversion Assistant 3.0](#).

For more information about .NET Framework security, see [NET Security](#).

See Also

Concepts

[C# Programming Guide](#)

[Asynchronous File I/O](#)

Other Resources

[C# for Java Developers](#)

Database Access (C# vs Java)

C# and Java use similar means for accessing database data. Both C# and Java require a database driver to perform the actual database operations. In addition, both require a database connection, a SQL query to execute against the database connection, and a result set from the execution of the query.

Comparing Database Drivers

Database drivers such as JDBC or ODBC can be used to access data in Java and C#. The Java Database Connectivity (JDBC) driver is used from a program written in Java. Open Database Connectivity (ODBC) is Microsoft's database programming interface for accessing a variety of relational databases on a number of platforms. There is also a JDBC-ODBC bridge standard on both the Solaris and Windows versions of the Java platform so you can also use ODBC from a Java program.

In Java, the connection string information is supplied to the driver for a connection handle, as follows:

```
final static private String url = "jdbc:oracle:server,user,pass, ...");
```

In C#, using the .NET Framework, you do not have to load ODBC or JDBC drivers in order to access the database. Simply set the connection string for the database connection object, as follows:

C#

```
static string connectionString = "Initial Catalog=northwind;Data Source=(local);Integrated Security=SSPI;";
static SqlConnection cn = new SqlConnection(connectionString);
```

- For more information about the ODBC driver for Oracle databases, see [ODBC Driver for Oracle](#). For additional information on OLE DB provider for DB2 databases, [Microsoft Host Integration Server 2000 Developer's Guide and Administration and Management of Data Access Using the OLE DB Provider for DB2](#).
- The [Microsoft® SQL Server™ 2000 Driver for JDBC](#) is a Type 4 JDBC driver that provides access to SQL Server 2000 through from any Java-enabled applet, application, or application server.

Java Read Database Example

In Java, to perform a database read operation, you can use a **ResultSet** object created by the **executeQuery** method of the **Statement** object. The **ResultSet** object contains the data returned by the query. You can then iterate through the **ResultSet** object to access the data.

The following example provides the Java code to read from a database.

```
Connection c;
try
{
    Class.forName (_driver);
    c = DriverManager.getConnection(url, user, pass);
}
catch (Exception e)
{
    // Handle exceptions for DriverManager
    // and Connection creation:
}
try
{
    Statement stmt = c.createStatement();
    ResultSet results = stmt.executeQuery(
        "SELECT TEXT FROM dba ");
    while(results.next())
    {
        String s = results.getString("ColumnName");
        // Display each ColumnName value in the ResultSet:
    }
    stmt.close();
}
```

```

    }
    catch(java.sql.SQLException e)
    {
        // Handle exceptions for executeQuery and getString:
    }

```

Similarly, to perform a database write operation, a **Statement** object is created from the **Connection** object. The **Statement** object has methods for executing SQL queries and updates against a database. Updates and queries are in the form of a string containing the SQL command of a write operation used in the **executeUpdate** method of the **Statement** object to return a **ResultSet** object.

C# Read Database Example

In C#, using the .NET Framework, accessing data is further simplified through the set of classes provided by ADO.NET, which supports database access using ODBC drivers as well as through OLE DB providers. C# applications can interact with SQL databases for reading, writing, and searching data using .NET Framework's [ADO.NET](#) classes, and through a Microsoft Data Access Component (MDAC). The .NET Framework's [System.Data.SqlClient](#) namespace and classes make accessing SQL server databases easier.

In C#, to perform a database read operation, you can use a connection, a command, and a data table. For example, to connect to a SQL Server database using the **System.Data.SqlClient** namespace, you can use the following:

- A [SqlConnection](#) class.
- A query such as a [SqlCommand](#) class.
- A result set such as a [DataTable](#) class.

The .NET Framework provides the [DataAdapter](#), which brings these three objects together, as follows:

- The **SqlConnection** object is set using the **DataAdapter** object's connection property.
- The query to execute is specified using the **DataAdapter**'s [SelectCommand](#) property.
- The **DataTable** object is created using the [Fill](#) method of the **DataAdapter** object. The **DataTable** object contains the result set data returned by the query. You can iterate through the **DataTable** object to access the data rows using rows collection.

To compile and run the code, you need the following; otherwise, the line `databaseConnection.Open()`; fails and throws an exception.

- Microsoft Data Access Components (MDAC) version 2.7 or later.

If you are using Microsoft Windows XP or Windows Server 2003, you already have MDAC 2.7. However, if you are using Microsoft Windows 2000, you may need to upgrade the MDAC already installed on your computer. For more information, see [MDAC Installation](#).

- Access to the SQL Server Northwind database and integrated security privileges for the current user name running the code on a local SQL Server with the Northwind sample database installed.

C#

```

// Sample C# code accessing a sample database

// You need:
//  A database connection
//  A command to execute
//  A data adapter that understands SQL databases
//  A table to hold the result set

namespace DataAccess
{
    using System.Data;
    using System.Data.SqlClient;

    class DataAccess
    {
        //This is your database connection:
        static string connectionString = "Initial Catalog=northwind;Data Source=(local);Int

```



```

egrated Security=SSPI;";
    static SqlConnection cn = new SqlConnection(connectionString);

    // This is your command to execute:
    static string sCommand = "SELECT TOP 10 Lastname FROM Employees ORDER BY EmployeeID
";

    // This is your data adapter that understands SQL databases:
    static SqlDataAdapter da = new SqlDataAdapter(sCommand, cn);

    // This is your table to hold the result set:
    static DataTable dataTable = new DataTable();

    static void Main()
    {
        try
        {
            cn.Open();

            // Fill the data table with select statement's query results:
            int recordsAffected = da.Fill(dataTable);

            if (recordsAffected > 0)
            {
                foreach (DataRow dr in dataTable.Rows)
                {
                    System.Console.WriteLine(dr[0]);
                }
            }
            catch (SqlException e)
            {
                string msg = "";
                for (int i=0; i < e.Errors.Count; i++)
                {
                    msg += "Error #" + i + " Message: " + e.Errors[i].Message + "\n";
                }
                System.Console.WriteLine(msg);
            }
            finally
            {
                if (cn.State != ConnectionState.Closed)
                {
                    cn.Close();
                }
            }
        }
    }
}

```

For more information about ADO.NET, see:

- [ADO.NET](#)
- [Accessing Data \(Visual Studio\)](#)
- [ADO.NET Sample Application](#)
- [ADO.NET for the Java Programmer](#)

For more information on .NET Framework database access classes, see:

- [System.Data.OracleClient](#)
- [System.Data.SqlClient](#)
- [System.Data.Odbc](#)

- [System.Data.OleDb](#)
- [ADO.NET DataSet](#)
- [Using DataSets in ADO.NET](#)

For more information on the Java Language Conversion Assistant, see [What's New in Java Language Conversion Assistant 3.0](#).

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[C# for Java Developers](#)

User Interface Development (C# vs Java)

You can use the .NET Framework's rich set of windows forms components in C# for programming client-side forms applications.

Java

Most java applications use Abstract Windowing ToolKit (AWT) or Swing which uses the AWT infrastructure, including the AWT event model, for forms programming. AWT provides all the basic GUI capabilities and classes.

Java Example

A frame, a window with a title and border, is typically used for adding your components.

```
JFrame aframe = new JFrame();
```

The **Component** class, an object with graphical representation, is typically extended and the methods inherited are used or typically over-ridden, such as the **paint** method of a **Shape** component in the code illustrated.

```
import java.awt.*;
import javax.swing.*;

class aShape extends JComponent {
    public void paint(Graphics g) {
        Graphics2D g2d = (Graphics2D)g;

        // Draw the shape.
    }

    public static void main(String[] args) {
        JFrame aframe = new JFrame();
        frame.getContentPane().add(new aShape ());
        int frameWidth = 300;
        int frameHeight = 300;
        frame.setSize(frameWidth, frameHeight);
        frame.setVisible(true);
    }
}
```

You can register to listen on an action event for a component to handle events. For example, when a button is pressed and released, AWT sends an instance of **ActionEvent** to that button, by calling **processEvent** on the button. The button's **processEvent** method receives all events for the button; it passes an action event along by calling its own **processActionEvent** method. The latter method passes the action event on to any action listeners that have registered an interest in action events generated by this button.

C#

In C#, the [System.Windows.Forms](#) namespace and classes of the .NET Framework provide a comprehensive set of components for windows forms development. For example, the following code uses [Label](#), [Button](#), and [MenuStrip](#).

C# Example

Simply derive from the [Form](#) class, as follows:

C#

```
public partial class Form1 : System.Windows.Forms.Form
```

And add your components:

C#

```
this.button1 = new System.Windows.Forms.Button();
```

```
this.Controls.Add(this.button1);
```

The following code illustrates how to add a label, a button, and a menu to a form.

C#

```
namespace WindowsFormApp
{
    public partial class Form1 : System.Windows.Forms.Form
    {
        private System.ComponentModel.IContainer components = null;

        private System.Windows.Forms.Label label1;
        private System.Windows.Forms.Button button1;
        private System.Windows.Forms.MenuStrip menu1;

        public Form1()
        {
            InitializeComponent();
        }

        private void InitializeComponent()
        {
            this.components = new System.ComponentModel.Container();

            this.label1 = new System.Windows.Forms.Label();
            this.Controls.Add(this.label1);

            this.button1 = new System.Windows.Forms.Button();
            this.Controls.Add(this.button1);

            this.menu1 = new System.Windows.Forms.MenuStrip();
            this.Controls.Add(this.menu1);
        }

        static void Main()
        {
            System.Windows.Forms.Application.Run(new Form1());
        }
    }
}
```

Like Java, in C# you can register to listen on an event for a component. For example, when a button is pressed and released, the run time sends a [Click](#) event to any listeners that have registered an interest in the **Click** event of this button.

C#

```
private void button1_Click(object sender, System.EventArgs e)
{
}
```

You can use the following code to register `button1_Click` to handle the **Click** event of an instance of **Button** called `button1`.

C#

```
// this code can go in InitializeComponent()
button1.Click += button1_Click;
```

For more information, see [Creating ASP.NET Web Applications \(Visual C#\)](#).

For more information about automated conversion from Java to C#, see [What's New in Java Language Conversion Assistant 3.0](#).

For more information about the **Forms** classes, see [Windows Forms Controls by Function](#) and **System.Windows.Forms**.

See Also

Concepts

[C# Programming Guide](#)

[Designing a User Interface \(Visual C#\)](#)

Other Resources

[C# for Java Developers](#)

Resource Management (C# vs Java)

In C# using Visual Studio, managing resources is simplified.

Java

Java applications are typically bundled in a JAR file, along with an application's various resources, such as class files, sound files, and image files. You'll most likely use JBuilder or Eclipse which manage JAR files in much the same way that Visual Studio manages solutions and projects.

C#

In C# projects, you can simply open your resources from Visual Studio's [Solution Explorer](#).

You can also use the [Image Editor](#) and the [Binary Editor](#) to work with resource files in managed projects.

For more information about adding resources to managed projects, see:

- [Adding and Editing Resources \(Visual C#\)](#)
- [Walkthrough: Localizing Windows Forms](#)
- [Walkthrough: Using Resources for Localization with ASP.NET](#)

You can read these resources in your application as external content or embedded resources. For example, the following lines of code use the classes in the [System.Reflection](#) namespace and classes such as [Assembly](#) to read an embedded resource file from your assembly. In this case, the file is `assemblyname.file.ext`.

C#

```
static void Main()
{
    System.Reflection.Assembly asm =
        System.Reflection.Assembly.GetExecutingAssembly();

    System.Drawing.Bitmap tiles = new System.Drawing.Bitmap
        (asm.GetManifestResourceStream("assemblyname.file.ext"));
}
```

- For more information, see [Reflection \(C# Programming Guide\)](#).
- For more information on application resources, see [Managing Application Resources](#).
- For information on how the general resource editor works, see [Resource Editors](#).
- For more information about editing .Resx formatted resource files, see [Resources in Applications](#).
- For more information about handling XML and Simplified API for XML (SAX2) see [SAX2 Developer Guide](#) and [XML Developer Center](#).

See Also

Concepts

[C# Programming Guide](#)

[Adding and Editing Resources \(Visual C#\)](#)

Other Resources

[C# for Java Developers](#)

Web Services Applications (C# vs Java)

The .NET Framework provides extensive support for interoperability through Web services. In C#, using the .NET Framework, Visual Studio, and ASP.NET, creating a Web service is as simple as creating a Web service project and adding an attribute **WebMethod** to any public method that you want to expose.

Java

In Java you can use a Web service package to implement an application such as the Java Web Services Developer Pack or Apache SOAP. For example, in Java you can create a Web service and Apache SOAP using the following steps.

To create a Web service in Java using Apache SOAP

1. Write a Web service method, as follows:

```
public class HelloWorld
{
    public String sayHelloWorld()
    {
        return "HelloWorld ";
    }
}
```

2. Create the Apache SOAP deployment descriptor. This may be similar to the descriptor shown:

```
<dd:service xmlns:dd="http://xml.apache.org/xml-soap/deployment"
            id="urn:HelloWorld">

    <dd:provider type="java"
                scope="Application"
                methods="sayHelloWorld">

        <dd:java class="HelloWorld" static="false" />

    </dd:provider>

    <dd:faultListener>org.apache.soap.server.DOMFaultListener</dd:faultListener>

    <dd:mappings />

</dd:service>
```

3. Compile class `HelloWorld` and move it to your Web server's classpath.
4. Deploy the Web service using the command line tool.

C#

Creating a Web Service is simpler in C# using .NET Framework classes and the Visual Studio IDE.

To create a Web service in C# using the .NET Framework and Visual Studio

1. Create a Web service application in Visual Studio. For more information, see [C# Application Types for Java Developers](#). The code generated is illustrated below.

C#

```
using System;
```

```

using System.Web;
using System.Web.Services;
using System.Web.Services.Protocols;

[WebService(Namespace = "http://tempuri.org/")]
[WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]
public class Service : System.Web.Services.WebService
{
    public Service () {

    }

    [WebMethod]
    public string HelloWorld() {
        return "Hello World";
    }
}

```

2. Find the line `[WebService(Namespace = "http://tempuri.org/")]` and change `"http://tempuri.org/"` to `"http://tempuri.org/"`.

To run your C# Web service

1. Compile and run the service. Type <http://localhost/Website1/Service.asmx> in your Web browser, where **localhost** is the name of your IIS Web Server and **Service** is the name of your service, in this case `Service`.
2. Output is:

The following operations are supported. For a formal definition, please review the Service Description.
HelloWorld

3. Click on the HelloWorld link to call the `HelloWorld` method of `Service1`. The output is:

Click here for a complete list of operations.
HelloWorld
Test
To test the operation using the HTTP POST protocol, click the 'Invoke' button.

SOAP 1.1
...
SOAP 1.2
...
HTTP POST
...

4. Click on the **Invoke** button to call the `HelloWorld` method of `Service1`. The output is:

```

<?xml version="1.0" encoding="utf-8" ?>
  <string xmlns="http://HowToDevelopWebServicesTest/">Hello World</string>

```

For more information on Web services see:

- [Building XML Web Service Clients](#)
- [XML Web Service Description](#)

- [How to: Create an XML Web Service Method](#)
- [Walkthrough: Creating an XML Web Service Using Visual Basic or Visual C#](#)
- [Walkthrough: Creating and Using an ASP.NET Web Service in Visual Web Developer](#)

For more information about automated conversion from Java to C#, see [What's New in Java Language Conversion Assistant 3.0](#).

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[C# for Java Developers](#)

Mobile Devices and Data (C# vs Java)

With C# and the .NET Compact Framework, you can access and manage database data on a mobile device using the same concepts and similar APIs that you would use for desktop database programming. On mobile devices, ADO.NET provides a subset of the desktop API targeting Windows CE devices including Pocket PC and Smartphones. For more information, see [Database Access \(C# vs Java\)](#).

Java

In Java you can use J2ME and JDBC to access a database from a mobile device. For more information, see [Database Access \(C# vs Java\)](#). J2ME does not represent a single API on all devices and does not have a single development environment. In addition, J2ME has to run within a Virtual Machine, either KVM or JVM depending on configuration.

C#

In C#, to perform a database read operation, you can use the familiar concepts of a connection, a command, a data table on the desktop or the mobile device. You simply use the [System.Data.SqlServerCe](#) namespace and classes. For example you can do the following:

- Use [SqlCeConnection](#) for your database connection.
- Use [SqlCeCommand](#) for your SQL command object.
- Use a result set object such as a [DataTable](#) for your data table object.

The .NET Framework provides [DataAdapter](#) to allow the previously mentioned classes to be used together easily. The **SqlCeConnection** object can be set using the [SqlCeDataAdapter](#) object's connection property.

The query to execute is specified using the DataAdapter's [SelectCommand](#) property or simply passed to the DataAdapter's constructor, along with the connection object.

C#

```
da = new SqlCeDataAdapter("SELECT * FROM Users", cn);
```

The **DataTable** object is created using the [Fill](#) method of the DataAdapter object. The **DataTable** object contains the result set data returned by the query. You can iterate through the **DataTable** object to access the data rows using [Rows](#) collection.

The following code illustrates how to access the rows of a table in a SQL Server CE (SQLCE) database on a mobile device.

C#

```
namespace DataAccessCE
{
    using System.Data;
    using System.Data.SqlServerCe;

    class DataAccessCE
    {
        public static string connectionString = "";
        public static SqlCeConnection cn = null;
        public static SqlCeDataAdapter da = null;
        public static DataTable dt = new DataTable();

        static void Main()
        {
            connectionString = "Data Source=\\My Documents\\Database.sdf" ;
            cn = new SqlCeConnection(connectionString);

            da = new SqlCeDataAdapter("SELECT * FROM Users", cn);
            da.Fill(dt);

            foreach (DataRow dr in dt.Rows)
            {
                System.Console.WriteLine(dr[0]);
            }
        }
    }
}
```

```
}  
    }  
}  
}
```

For more information, see:

- [Overview of ADO.NET](#)
- [Accessing Data \(Visual Studio\)](#)
- [ADO.NET Sample Application](#)
- [Replication](#)
- [Synchronizing a Subscription](#)

For more information about automated conversion from Java to C#, see [What's New in Java Language Conversion Assistant 3.0](#).

Compiling the Code

Before you can work with a SQLCE database from an application, you need to add a reference to **System.Data.SqlServerCe** to your project. This is accomplished by selecting **Add Reference** on the **Project** menu in the development environment. Then, select the **System.Data.SqlServerCe** component from the **Add Reference** dialog box.

Note

The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Robust Programming

To compile and run the code, you need the following; otherwise, the line `da.Fill(dt);` fails and throws an exception.

- SQL Server CE installed on the device.
- A database table with some data present for testing on an SQLCE database called Database.sdf. You can build this table on the device using SQL CE tools, or replicated from an SQL Server desktop to generate the .sdf file. You can add the .sdf file to your project or manually copy it to the directory specified in the connection string.

See Also

Reference

[Smart Devices](#)

[SqlConnection](#)

[SqlCommand](#)

Concepts

[C# Programming Guide](#)

[ADO.NET DataSet](#)

Other Resources

[C# for Java Developers](#)

[Smart Device Development](#)

[Using DataSets in ADO.NET](#)

[ADO.NET for the Java Programmer](#)

C# Application Types for Java Developers

C# application types include Windows Console applications, Windows Forms applications, ASP.NET Web applications, ASP.NET Web Service applications, Smart Device applications, ActiveX applications, and setup and deployment applications.

Console Applications

Console applications use standard command-line input and output for input and output rather than a form. Console applications use the [System.IO](#) class for handling input and output. You can use the class name in front of methods, such as `System.IO.Console.WriteLine()`, or include a **using** statement at the start of your program. Console applications are easy to create using Visual Studio and other development environments including any text editor, such as Microsoft® Notepad. For more information, see [Introducing Visual Studio](#), [Creating Console Applications \(Visual C#\)](#), [Hello World -- Your First Program \(C# Programming Guide\)](#), or [Main\(\) and Command Line Arguments \(C# Programming Guide\)](#).

Forms Applications

Forms applications have the familiar graphical user interface of Windows with controls such as buttons and list boxes for input. Forms applications use classes in the [System.Windows.Forms](#) namespace. Forms applications are easy to create using Visual Studio and other development environments including any text editor, such as Microsoft® Notepad. For more information on creating a Windows application, see [How to: Create a Windows Application Project](#), [Creating ASP.NET Web Applications \(Visual C#\)](#), or [Creating ASP.NET Web Applications \(Visual C#\)](#).

ASP.NET Web Applications

ASP.NET applications are Web applications displayed in a Web browser, rather than on a console or in a forms application. ASP.NET applications use the [System.Web](#) namespace and classes such as [System.Web.UI](#) for handling input and output to and from the browser. You can use the class name in front of methods such as `using System.Web.UI.HtmlControls;` or include a **using** statement at the start of your program. ASP.NET applications are easy to create using Visual Studio and other development environments including any text editor, such as Microsoft® Notepad. For more information on creating an ASP.NET application, see [Visual Web Developer](#). For more information on creating ASP.NET applications using Visual Studio .NET, see [Overview of ASP.NET Applications on Application Diagrams](#). For more information on the ASP.NET, see [ASP.NET Web Applications in the .NET Framework](#). For more information on how to debug an ASP.NET application, see [Debugging ASP.NET Web Applications](#) and [Debugging Preparation: ASP.NET Web Applications](#).

ASP.NET Web Service Application

ASP.NET Web services are accessible using URLs, HTTP, and XML so that programs running on any platform and in any language can access ASP.NET Web services. ASP.NET Web service applications can be displayed on a console in a form, or in a Web browser or a smart device. ASP.NET Web services applications use the **System.Web** and [System.Web.Services](#) namespace and classes. ASP.NET applications Web services are easy to create using Visual Studio and other development environments including any text editor, such as Microsoft® Notepad. For more information on creating a Web services application, see [Accessing and Displaying Data \(Visual C#\)](#) and [How to: Create ASP.NET Web Service Projects](#). For more information on adding ASP.NET Web Services to an existing project, see [How to: Add an XML Web Service to an Existing Web Project in Managed Code](#). For more information on the ASP.NET Web services, see [Walkthrough: Creating and Using an ASP.NET Web Service in Visual Web Developer](#) and [Walkthrough: Creating an XML Web Service Using Visual Basic or Visual C#](#). For more information on how to debug an ASP.NET Web services application, see [Debugging Preparation: XML Web Service Projects](#).

Other topics related to ASP.NET Web services include:

- [Building XML Web Service Clients](#)
- [XML Web Service Description](#)
- [How to: Create an XML Web Service Method](#)
- [Walkthrough: Creating an XML Web Service Using Visual Basic or Visual C#](#)
- [Walkthrough: Creating and Using an ASP.NET Web Service in Visual Web Developer](#)
- [How to use Visual Studio .NET 2003 to build and test an XML Web service](#)

Smart Device Applications

Smart device applications run on mobile devices, such as PDAs and Smartphones. Smart device applications can be console applications, Windows Forms applications, or ASP.NET and Web clients and are displayed on a console, in a form, or in a Web browser. Smart device applications use the same namespaces and classes as the desktop applications. However they use the [Compact Framework](#) rather than .NET Framework. For more information on developing on a Windows mobile device versus developing for the desktop, see [Developing Device vs. Desktop Applications](#). Some versions of the development environment may support development for some or all of the C# application types on the mobile device. For more information on creating an ASP.NET application, see [How to create a Smart Device Application](#) and [Smart Device Application Wizard](#).

Other topics related to ASP.NET Web services include:

- [Use Visual Studio .NET 2003 to build and test a mobile Web application](#)
- [Developing Smart Device Applications](#)
- [Smart Device Programmability Features of Visual Studio .NET](#)
- [Smart Device Hardware Considerations](#)
- [ASP.NET Mobile Controls QuickStart](#)
- [ASP.NET Mobile Controls Samples](#)
- [Smart device Walkthroughs](#)
- [Walkthrough: Creating Windows Forms Applications for a Device](#)
- [Walkthrough: Debugging Windows Forms in Device Projects](#)

ActiveX Controls

Similar to Java Beans, an ActiveX control is a component and equivalent to an "OLE Object" and Component Object Model (COM) Object. An ActiveX control, in the very simplest form, is a COM object that supports the [IUnknown](#) interface. ActiveX Controls are the primary architecture for developing programmable software components for reuse in a variety of different containers, ranging from Internet Explorer to software development tools, and end-user productivity tools. For more information on ActiveX Controls, see:

- [Introduction to ActiveX](#)
- [Packaging ActiveX Controls](#)
- [Using ActiveX Controls to Automate Your Web](#)
- [Designing Secure ActiveX Controls](#)
- [Using ActiveX Controls with Windows Forms in Visual Studio .NET](#)
- [Debugging an ActiveX Control](#)

Setup and Deployment Applications

Visual Studio provides templates for deployment desktop, Web, and smart device setup and deployment projects. Different versions of the development environment may support setup and deployment for some or all of the C# application types on the desktop, on the Web, and for mobile devices. For more information, see:

- [Deploying .NET Framework Applications.](#)
- [Distributing Device Applications](#)
- [Application Installation on the Device](#)
- [Walkthrough: Generating Custom CAB files for Device projects](#)
- [Cabinet Packaging : Internet Explorer Code Download and the Java Package Manager](#)

Related Topics

- [Basics of .NET Framework Network Operations](#)
- [Windows Forms Application Basics](#)
- [What's New in Java Language Conversion Assistant 3.0](#)

- [.NET Framework Samples](#)

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Migrating to Visual C#](#)

[C# Code Examples for Java Developers](#)

[The C# Programming Language for Java Developers](#)

[Getting Started with Visual C#](#)

[Using the Visual C# IDE](#)

Converting Java Applications to Visual C#

Microsoft Visual Studio 2005 provides the ability to convert projects created in Visual J++® version 6.0 or written in the Java language to Visual C#® so that you can take advantage of the .NET Framework. Java Language Conversion Assistant generates a new Visual C# project from an existing Visual J++ 6.0 project or Java-language files. The Java Language Conversion Assistant wizard makes it easy to convert your existing files.

In This Section

[What's New in Java Language Conversion Assistant 3.0](#)

Lists the features included with this version of Java Language Conversion Assistant.

[Converting Visual J++ or Java-Language Projects to Visual C#](#)

Describes how to use Java Language Conversion Assistant.

[Converting Web Applications](#)

Describes how to adapt Web applications when converting them with Java Language Conversion Assistant.

[Conversion of Various Types of Java-Language Applications](#)

Discusses the processes to follow and issues that may arise as you convert various types of applications using Java Language Conversion Assistant.

[JLCA Diagnostic Messages Listed by Package](#)

Provides lists of error messages generated by Java Language Conversion Assistant.

Related Sections

[Java Language Conversion Assistant Wizard](#)

Describes how to use the wizard.

[Conversion of JSP Custom Tag Libraries](#)

Provides a list of Java-language classes for which support classes are created to emulate custom tag runtime behavior.

[Compile Converted Servlet Classes Before Opening Them](#)

Explains how to open Web Forms for converted servlet classes.

[Troubleshooting: Matching Code Pages](#)

Describes what to do when you have missing or mismatched code pages.

[Manually Upgrading Unconverted Code](#)

Explains how to upgrade code that could not be converted automatically.

[C# Reference](#)

Provides an overview of the Visual C# language.

What's New in Java Language Conversion Assistant 3.0

The following features are included with this version of Java Language Conversion Assistant:

- Support for conversion of EJB applications
- Support for conversion of CORBA applications
- Support for conversion of RMI applications
- Support for conversion of JMS applications
- Support for conversion of serialized applications
- Support for conversion of applications that use JNDI
- Support for conversion of JAAS applications
- Support for conversion of JCE applications
- Support for conversion of Java Swing applications
- Support for conversion of JAXP applications
- Support for conversion of TRAX applications
- Support for conversion of applications that use JavaMail
- A command-line property switch

EJB

Enterprise JavaBeans are converted to the classes in the [System.EnterpriseServices](#) namespace for support for all types of enterprise JavaBeans: message-driven beans, session beans, and entity beans.

CORBA

Common Request Broker Architecture (CORBA) classes are converted to the classes in the [System.Runtime.Remoting](#) namespace to provide support for distributed computing.

RMI

Remote Method Invocation (RMI) classes are converted to the classes in the **System.Runtime.Remoting** namespace to provide support for distributed applications and remote objects.

JMS

The classes of the Java Message Service (JMS) are converted to the classes in the [System.Messaging](#) namespace, which use Windows message queuing.

Serialized Applications

The classes that implement the **java.io.Serializable** interface are converted to implement the [System.Runtime.Serialization.ISerializable](#) interface.

Applications That Use JNDI

The classes of the Java Naming and Directory Interface (JNDI) are converted to the classes in the [System.DirectoryServices](#) namespace to provide naming and directory services. Some methods are converted to **System.Runtime.Remoting** methods to support remote operations for RMI and CORBA.

JAAS

The classes of the Java Authentication and Authorization Service (JAAS) are converted to the classes in the [System.Security](#) namespace to provide all security and authentication services.

JCE

The classes of the Java Cryptography Extension (JCE) are converted to the classes in the [System.Security.Cryptography](#)

namespace to provide encryption and decryption of messages for greater security.

Java Swing

The classes of the **javax.swing** package are converted to the classes in the [System.Windows.Forms](#) namespace to provide user interface components and controls.

JAXP

The classes of the Java API for XML processing are converted to the classes in the [System.Xml](#) namespace. Both SAX and DOM models are supported.

TRAX

The classes of the Transformation API for XML are converted to the classes in the [System.Xml.Xsl](#) namespace.

Applications That Use JavaMail

The classes of the JavaMail API are converted to the classes of [System.Web.Mail](#) namespace to provide support to construct and send messages using the Simple Mail Transfer Protocol (SMTP).

Property Switch

A command-line switch (**/ProcessGetSetOff**) is provided so that you can choose whether to leave **get/set/is** methods as methods or convert them to properties. By default, JLCA converts these methods to properties. If you prefer, you can toggle the command-line switch and leave most of these as methods. For compatibility with other conversions, the following patterns are always converted to properties:

- ActiveX methods are converted to specific properties in the .NET Framework.
- CORBA attributes.
- Accessor, mutator, and state-checking methods required by the .NET Framework.
- Automatically generated .NET Framework properties.

JSP pages that use the **setProperty** method with the property set to a wildcard character (*) are not converted correctly because the SupportClass code used to emulate the setting of bean properties expects a property, rather than a **get/set** method.

Taglib handler class properties are converted to **get/set** methods, and the code used in the generated ASPX page to set the property values will be broken because custom controls use properties to map to ASPX attributes in the custom HTML tags.

If an accessor method of a visual component is called in a different thread from the one that owns the component, it is converted to the [Invoke](#) method, which returns an **object** type when the **ProcessGetSetOff** flag is turned on. The return value must be cast to the same type as the return value of the original method.

Unsupported Java-Language Packages

Because of differences between the architecture of CORBA and .NET Framework Remoting, the following CORBA packages and classes are not supported:

- **org.omg.CosNaming**
- **org.omg.CosNamingContextExtPackage**
- **org.omg.Dynamic**
- **org.omg.IOP**
- **org.omg.IOP.CodecFactoryPackage**
- **org.omg.IOP.CodecPackage**
- **org.omg.Messaging**
- **org.omg.PortableInterceptor**
- **org.omg.PortableInterceptor.ORBInitInfoPackage**

Because of differences between the architecture of Swing and Windows Forms, the following Swing packages are not

supported:

- **javax.swing.plaf**
- **javax.swing.plaf.basic**
- **javax.swing.plaf.metal**
- **javax.swing.plaf.multi**

Because it contains interfaces for implementing third-party drivers, the following package is not supported:

- **javax.sql**

See Also

Reference

[Command-Line Switches](#)

Other Resources

[Converting Java Applications to Visual C#](#)

[Conversion of Various Types of Java-Language Applications](#)

Converting Visual J++ or Java-Language Projects to Visual C#

Use Java Language Conversion Assistant to convert Visual J++ 6.0 projects and Java-language files for further development in Visual C#. Java Language Conversion Assistant does not modify your existing project, but rather creates a new Visual C# project based upon the original project. Any errors, warnings, or issues generated in the course of conversion are displayed in a conversion report after the new project has been generated. These errors, warnings, and issues are also noted in comments in the converted code to help you manually convert the parts of the project that cannot be converted automatically.

Note

If you are new to Visual C#, take some time to familiarize yourself with the language before attempting this process. For more information, see [Introduction to the C# Language and the .NET Framework](#).

Note

The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Visual J++ Projects

To convert a Visual J++ project

1. Start Visual Studio.
2. On the **File** menu, point to **Open**, and click **Convert**.
3. Select **Java Language Conversion Assistant**, and click **OK**.
4. On the **Source files** page, select **A Visual J++ 6.0 project**.
5. On the **Select a project** page, click **Browse**.
6. Browse to the correct .vjp file, and select it.

Note

If the CLASSPATH directive of your .vjp file specifies .jar or .class files, they are ignored.

7. On the **Specify a directory for your new project** page, specify the name and directory of the new project to be created.
8. On the **Begin your conversion** page, click **Next**.

Java-Language Projects

To convert a Java-language project

1. Start Visual Studio.
2. On the **File** menu, point to **Open**, and click **Convert**.
3. Select **Java Language Conversion Assistant**, and click **OK**.
4. On the **Source files** page, click **A directory containing the project's files**.
5. On the **Select source directory** page, click **Browse**.
6. Browse to the correct project, and select it.

Note

You will not see the files in the directory you select, but all .jav and .java files in it will be converted. The Java Language Conversion Assistant will ignore most irrelevant files, but the source folders should be cleaned before beginning the conversion.

7. Add any other files needed for your project in the second text box.
8. On the **Configure your new project** page, specify the following:
 - Name of the project to be created.
 - Output type of the project.
9. On the **Specify a directory for your new project** page, specify the name and directory of the new project to be created.
10. On the **Begin your conversion** page, click **Next**.
11. Fix code that could not be converted automatically. For more information, see [Manually Upgrading Unconverted Code](#).

See Also

Tasks

[Manually Upgrading Unconverted Code](#)

[Java Language Conversion Assistant Wizard](#)

[Troubleshooting: Matching Code Pages](#)

Reference

[Convert Dialog Box](#)

[Command-Line Switches](#)

Concepts

[Java Language Conversion Assistant](#)

Java Language Conversion Assistant Wizard

Use the Java Language Conversion Assistant wizard to convert Visual J++ and Java-language projects to Visual C#. With the wizard, you can create a new project and copy files from your original project for conversion from Visual J++ or the Java language to Visual C#. A report detailing any errors, warnings, or issues encountered in the conversion process is generated. Errors, warnings, and issues are also noted as comments in the code of the new project and can be viewed on the task list.

Note

The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

To access the Java Language Conversion Assistant wizard

1. On the **File** menu, point to **Open**, and click **Convert**.
2. Click **Java Language Conversion Assistant**, and click **OK**.

Security Note You should review any C# code converted from Java using the Java Language Conversion Assistant tool for security issues. Any insecure Java code is converted into insecure C# code. In addition, the tool does not migrate code for certain Java classes, including some related to security, for example, authentication. In these cases, the upgrade report notes what code has not been migrated. It is important to review the report and to provide mitigation for any security issues.

See Also

Tasks

[Converting Visual J++ or Java-Language Projects to Visual C#](#)

[Manually Upgrading Unconverted Code](#)

Concepts

[Java Language Conversion Assistant](#)

Java Language Conversion Assistant

Java Language Conversion Assistant is a tool that converts Visual J++ 6.0 projects and Java-language files to Visual C#. By converting these files to Visual C#, you can leverage your existing code base and take advantage of the benefits of the .NET Framework.

The new Visual C# project contains all the new Visual C# code that can be generated automatically from the existing Visual J++ or Java-language code. For more information, see [Converting Visual J++ or Java-Language Projects to Visual C#](#).

You can use Java Language Conversion Assistant to convert Visual J++ or Java-language applications and applet projects. They are converted in the following way:

Before Conversion	After Conversion
Applications	Windows Forms applications
Applets	Web user controls
JSP pages or servlets	Web applications

You can host converted Web user controls in a browser just as you would an applet. Hosted controls are declared in HTML pages with the `OBJECT` tag, instead of the `APPLET` tag. Use the `classid` attribute to identify the control by specifying the path to the control and the fully qualified name of the control, separated by the pound sign (#), as shown in the following example:

```
<OBJECT id="myControl" classid="http:ControlLibrary1.dll#ControlLibrary1.myControl" VIEWASTEXT></OBJECT>
```

For the control to be displayed properly, the .dll file containing the control must either be in the same virtual directory as the Web page that is displaying it or be installed in the global assembly cache.

Support Classes

To convert functionality in the original project that is not available in Visual C#, Java Language Conversion Assistant creates *support classes* (also called *managers*) that duplicate the original functionality. Support classes are sometimes substantially different architecturally from the classes they emulate. Although every effort is made to preserve the original architecture of your application in the converted project, the primary goal of these support classes is to duplicate the original functionality.

Conversion Report

There might be some code in your project that could not be converted automatically. After you run the Java Language Conversion Assistant wizard, you can view the conversion report that details all errors, warnings, and issues encountered during the conversion process. Unconverted code is noted in the code of the new project by comments labeled `UPGRADE_TODO`. You can view conversion comments on the task list. Each conversion comment contains a link to a Help topic about how to convert that code manually. For more information, see [Manually Upgrading Unconverted Code](#).

 Security Note
You should review any C# code converted from Java using the Java Language Conversion Assistant tool for security issues. Any insecure Java code is converted into insecure C# code. In addition, the tool does not migrate code for certain Java classes, including some related to security, for example, authentication. In these cases, the upgrade report notes what code has not been migrated. It is important to review the report and to provide mitigation for any security issues.

See Also

Tasks

[Converting Visual J++ or Java-Language Projects to Visual C#](#)

[Manually Upgrading Unconverted Code](#)

Other Resources

[Converting Java Applications to Visual C#](#)

Manually Upgrading Unconverted Code

After Java Language Conversion Assistant converts your Visual J++ project or Java-language files, the new Visual C# project might contain code that could not be converted automatically. Comments are inserted in the code of the new project to help you convert that code to Visual C# manually.

Comment Type	Description
UPGRADE_TODO	Code that could not be converted automatically
UPGRADE_WARNING	Code that might be problematic
UPGRADE_ISSUE	Code that is problematic.
UPGRADE_NOTE	Code that might exhibit a different behavior from the original code

There might also be some compiler errors that must be corrected before the application compiles. Each comment contains a brief description of the issue and a link to a Help topic about how to convert the code.

 Note
The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose Import and Export Settings on the Tools menu. For more information, see Visual Studio Settings .

To upgrade unconverted code manually

- After running the Java Language Conversion Assistant wizard, open your project in Visual Studio.

See Also

Tasks

[Converting Visual J++ or Java-Language Projects to Visual C#](#)

Concepts

[Java Language Conversion Assistant](#)

Troubleshooting: Matching Code Pages

If you find that your code after conversion has strange characters, the most likely cause is a missing or mismatched code page for character encoding. Java Language Conversion Assistant matches encoding in the following manner:

- Files that are encoded in the current system ANSI code page keep this encoding after conversion.
- Any other encoding is converted to UTF-8 encoding.

Unicode files that start with a Unicode byte-order mark are identified automatically as Unicode. If a file does not start with a byte-order mark and the encoding switch is not specified, the file is assumed to be in the current ANSI code page of the system.

To change the system ANSI code page you are currently using

1. Go to **Control Panel**, and double-click **Regional Options** (in Windows 2000) or **Regional and Language Options** (in Windows XP).
2. Click **Advanced**, and select the desired code page.

If your source files are in a non-ANSI character encoding or you have Unicode source files that do not start with a byte-order mark, you must use the encoding switch.

The following character encodings can be specified in the encoding switch:

Encoding	Switch
Latin alphabet No. 2 (Central European)	ISO-8859-2
Latin alphabet No. 3	ISO-8859-3
Latin alphabet No. 4 (Baltic States)	ISO-8859-4
Latin/Cyrillic alphabet	ISO-8859-5
Latin/Arabic alphabet	ISO-8859-6
Latin/Greek alphabet	ISO-8859-7
Latin/Hebrew alphabet	ISO-8859-8
Latin alphabet No. 9	ISO-8859-15
Japanese	EUC-JP
Korean	EUC-KR
Chinese Simplified	EUC-CN
Chinese National Standard	GB18030
UTF-8 without byte-order mark	UTF-8

To use a specific encoding, you must have support for it installed on your system, for example: to use GB18030 encoding on a Windows 2000 Server computer, you must download and install the GB18030 support package. See your system documentation for details. When using the encoding switch, all files in the project must be in the same encoding.

If you are using an encoding system that is not supported in JLCA, you might lose characters that cannot be converted. In some cases, a file might be lost.

See Also

Concepts

[Changing Response Encoding](#)

Command-Line Switches

The following table shows the command-line switches available in Java Language Conversion Assistant 3.0.

Switch	Function
/?	Prints the message.
/Out	Specifies the target directory. The default is \OutDir.
/Verbose	Specifies that everything be output.
/NoLogo	Specifies that the copyright banner not be displayed.
/NoLog	Specifies that a log not be written.
/LogFile	Specifies the log file name.
/Encoding	Specifies the encoding for input files.
/ProjectName	Specifies the project name for directory conversion.
/ProjectType	Specifies the project type. Options are EXE, WinExe, Library, and ASP.NET. The default is WinExe.
/VRoot	Specifies the virtual root for Internet Information Server.
/ContextPath	Specifies the context path to be replaced by the virtual root.
/Domain	Specifies the domain to be replaced for URLs.
/NoExtensibility	Disables extensibility mappings.
/JDK	Specifies the JDK to use for the conversion. Options are VJ and J2EE.
/ProcessGetSetOff	Disables property conversion.

Property Switch

A command-line switch (**/ProcessGetSetOff**) is provided so that you can choose whether to leave **get/set/is** methods as methods or convert them to properties. By default, JLCA converts these methods to properties. If you prefer, you can toggle the command-line switch and leave most of these as methods. For compatibility with other conversions, the following patterns are always converted to properties:

- ActiveX methods are converted to specific properties in the .NET Framework.
- CORBA attributes.
- Accessor, mutator, and state-checking methods required by the .NET Framework.
- Automatically generated .NET Framework properties.

JSP pages that use the **setProperty** method with the property set to a wildcard character (*) are not converted correctly because the SupportClass code used to emulate the setting of bean properties expects a property, rather than a **get/set** method.

Taglib handler class properties are converted to **get/set** methods, and the code used in the generated ASPX page to set the property values will be broken because custom controls use properties to map to ASPX attributes in the custom HTML tags.

If an accessor method of a visual component is called in a different thread from the one that owns the component, it is converted to the [System.Windows.Forms.Control.Invoke](#) method, which returns an **object** type when the **ProcessGetSetOff**

flag is turned on. The return value must be cast to the same type as the return value of the original method.

See Also

Other Resources

[Converting Java Applications to Visual C#](#)

Converting Web Applications

When converting Web applications from JSP to ASP.NET, additional information is needed. Java Language Conversion Assistant establishes the application root directory, moves servlet classes to a different directory, replaces links, and resolves tag library descriptors defined in the Web.xml file. You can set the context path and application domain for your Web application to enhance URL conversion.

In This Section

[Choosing the Right Type of Conversion for a Web Application](#)

Describes the types of conversions and how to choose the correct one for Web applications.

[Application Domain and Context Path Settings](#)

Describes the context path and application domain settings for a Web application conversion.

[Request Parameter Encoding](#)

Explains the conversion of Java-language request parameter encoding to ASP.NET.

[Directory Structure Differences in Web Applications](#)

Describes how Java Language Conversion Assistant modifies the directory structure for Web applications.

[Link Replacement](#)

Describes how Java Language Conversion Assistant replaces links in your converted Web application.

[Resolution of Tag Library Descriptors](#)

Describes the conversion of tag library descriptors included in the Web.xml file.

[Changing Response Encoding](#)

Explains how to change response encoding for applications that use encodings other than UTF-8.

[Conversion of JSP Custom Tag Libraries](#)

Explains how the Java Language Conversion Assistant handles JSP Custom Tag Libraries.

[Compile Converted Servlet Classes Before Opening Them](#)

Troubleshoots converting Servlet classes.

[Web Tag Samples](#)

Illustrates the methods of various HTML classes.

Choosing the Right Type of Conversion for a Web Application

To convert a Web application that contains JSP pages and/or servlet classes, you must choose the correct type of conversion and output when setting up the conversion. Select **Directory of Java-language files**, and set the output type to Web application.

See Also

Other Resources

[Converting Web Applications](#)

Application Domain and Context Path Settings

When converting a Web application, you must first select the correct output type: Web application.

Once you have entered the virtual root for your Web application, you have the option of specifying an application domain to enhance the way in which links are converted in your application. If you do not specify an application domain, all links, including external links, will be converted to ASP.NET links.

You can also specify the context path for your application.

See Also

Concepts

[Choosing the Right Type of Conversion for a Web Application](#)

Other Resources

[Converting Web Applications](#)

Request Parameter Encoding

In the Java language, servlet containers encode request parameters, using ISO 8859-1 encoding. If your Web application deals with a different encoding, it will usually contain the code in one of the examples.

Example

```
String text = request.getParameter("text");
text = new String( text.getBytes("ISO-8559-1"), charset )
```

The following example shows another version:

```
String text = request.getParameter("text");
BufferedReader reader = new BufferedReader(new InputStreamReader(new StringBufferInputStrea
m(text), charset));
text = reader.readLine();
```

In both cases, the *charset* parameter is a string representing the encoding of the request that you want to decode.

In the .NET Framework, this code is unnecessary because the decoding is performed automatically by ASP.NET with the parameters set in the Web.xml file, as shown in the following example.

Example

```
<globalization requestEncoding="euc-jp" responseEncoding="euc-jp"/>
```

When you convert Web applications, you can remove the converted Java-language code that accomplishes this function because it is no longer necessary.

See Also

Other Resources

[Converting Web Applications](#)

Directory Structure Differences in Web Applications

When converting a Web application, Java Language Conversion Assistant establishes a different directory structure for the new application.

Any servlet classes are moved to the Web application root directory. The directory structure created in the process of conversion does not necessarily correspond to the directory in which the servlet was located in the original Java-language project.

Example

In the following example, the Web application root directory is `\website\AppName`. The servlet classes are moved to a different directory in the converted directory structure.

Original Directory Structure

```
mt\src
mt\src\com\ais\servlets
mt\src\com\ais\servlets\Servlet1.java
mt\src\com\ais\servlets\Servlet2.java
mt\src\com\ais\beans
mt\src\com\ais\beans\CatalogBean.java
mt\src\com\ais\customtags
mt\src\com\ais\customtags\TableTag.java
mt\misc\pack1\myclasses
mt\misc\pack1\myclasses\DbHelper.java
mt\website
mt\website\AppName
mt\website\AppName\jsp\
mt\website\AppName\jsp\Search.jsp
mt\website\AppName\jsp\content\Catalog.jsp
mt\website\AppName\html\Index.html
mt\website\AppName\images\Logo.gif
mt\website\AppName\WEB-INF
mt\website\AppName\WEB-INF\Web.xml
mt\website\AppName\WEB-INF\MyTags.tld
```

Equivalent Converted Directory

```
outdir\src
outdir\src\com\ais\beans
outdir\src\com\ais\beans\CatalogBean.cs
outdir\src\com\ais\customtags
outdir\src\com\ais\customtags\TableTag.cs
outdir\misc\pack1\myclasses
outdir\misc\pack1\myclasses\Dbhelper.cs
outdir\website
outdir\website\AppName
outdir\website\AppName\com\ais\servlets\Servlet1.aspx
outdir\website\AppName\com\ais\servlets\Servlet2.aspx
outdir\website\AppName\jsp\
outdir\website\AppName\jsp\Search.aspx
outdir\website\AppName\jsp\content\Catalog.aspx
outdir\website\AppName\html\Index.html
outdir\website\AppName\images\Logo.gif
outdir\website\AppName\WEB-INF
outdir\website\AppName\WEB-INF\Web.xml
outdir\website\AppName\WEB-INF\MyTags.tld
```

See Also

Other Resources

[Converting Web Applications](#)

Link Replacement

Java Language Conversion Assistant replaces all links to work with the converted directory structure. This section assumes the directory structure defined in [Directory Structure Differences in Web Applications](#).

URLS and How They Are Changed

There are basically three types of URLs:

- Absolute URLs with domain information

```
http://www.example.com/servlet/pack1/Servlet1
http://www.example.com/page.jsp
```

- Absolute URLs without domain information

```
/servlet/pack1/Servlet1
/directory/page2.jsp
```

- Relative URLs

```
pack1.Servlet1
page3.jsp
```

The URL that a client uses to access a Web application has the following form:

```
http://hoststring/ContextPath/resource
```

In this example, the elements are defined as follows:

hoststring

Host name that is mapped either to a virtual host or to *hostname:portNumber*.

ContextPath

Directory structure to access the application.

resource

Reference to a file, a JSP page, or a servlet as defined in the Web.xml configuration file.

Java Language Conversion Assistant makes the following changes:

- Changes the context path (if present) to the name of the corresponding virtual root defined in the conversion, plus the Web application root.
- If the context path is not present, the VROOT is inserted in the link.
- If the file extension is .jsp, it is changed to .aspx.
- Converts servlet references by locating them, using the Web.xml configuration file.

Example

Original URL

```
http://hoststring/ContextPath/dir/JSP1.jsp?param1=1
http://HostNameProvided/dir/JSP1.jsp?param1=1
```

Equivalent Converted URL

```
http://hoststring/VRoot/dir/JSP1.aspx?param1=1
```

```
http://HostNameProvided/VRoot/dir/JSP1.aspx?param1=1
```

If the link contains the servlet token and the complete name of a servlet, the servlet token is eliminated, and dots (.) in the complete name are changed to slashes (/). If the link to the servlet is relative, the reference is changed to include the VROOT and the converted servlet directory structure.

Example

Original URL

```
http://hoststring/ContextPath/servlet/com.ais.servlets.Servlet1  
com.ais.servlets.Servlet1
```

Equivalent Converted URL

```
http://hoststring/VRoot/com/ais/servlets/Servlet1.aspx  
/VRoot/com/ais/servlets/Servlet1.aspx
```

If the servlet token is followed by a servlet name found in the Web.xml file, the complete name of the servlet is inserted in the link.

Example

Original XML

```
<servlet>  
  <servlet-name>test</servlet-name>  
  <servlet-class>com.ais.servlets.Servlet1</servlet-class>  
</servlet>
```

Original URL

```
http://hoststring/ContextPath/servlets/test
```

Equivalent Converted URL

```
http://hoststring/VRoot/com/ais/servlets/Servlet1.aspx
```

If the Web application name is followed by a pattern defined in the servlet mapping in the Web.xml file, then the name of the servlet is determined and inserted in the link. The pattern can contain wildcards (*).

Example

Original XML

```
<servlet>  
  <servlet-name>test</servlet-name>  
  <servlet-class>com.ais.servlets.Servlet1</servlet-class>  
</servlet>  
<servlet-mapping>  
  <servlet-name>test</servlet-name>  
  <url-pattern>/*.asp</url-pattern>  
</servlet-mapping>
```

Original URL

```
http://hoststring/ContextPath/abc.asp
```

Equivalent Converted URL

```
http://hoststring/VRoot/com/ais/servlets/Servlet1.aspx
```

Links calculated at run time are not converted by Java Language Conversion Assistant. Instead, you must manually change the code to generate proper ASP.NET links. This applies to the following cases:

- Links with the expressions `<%=%>`, `jsp:forward`, `jsp:include`, or `jsp:execute` and the HTML tags `<a href>` and `<form action>`, where the URL parameter value is calculated at runtime.

- Any **javax.servlet** method whose parameters represent a path to a Web resource, for example, the **Response.redirect** method, where the URL parameter value is calculated at run time.

Example

Original Java-Language Code

```
<jsp:include page="<%=value%>">
```

Equivalent Visual C# Code

```
<%-- //UPGRADE_TODO: Expected format of parameters of action jsp:include are different in t  
he equivalent in .NET Framework --%>  
<% Server.Execute(value)%>
```

See Also

Concepts

[Directory Structure Differences in Web Applications](#)

Other Resources

[Converting Web Applications](#)

Resolution of Tag Library Descriptors

When the **uri** attribute of the **taglib** directive refers to a **taglib-uri** defined in the Web.xml file, the location of the tag library descriptor (TLD) is taken from that file.

Example

Original XML

```
<taglib>
  <taglib-uri>/MYTEST</taglib-uri>
  <taglib-location>WEB-INF/myTagLib.tld</taglib-location>
</taglib>
```

Original JSP

```
<%taglib uri="/MYTEST" prefix="myTag"%>
```

Equivalent TLD to be parsed

```
/WEB-INF/myTagLib.tld
```

See Also

Other Resources

[Converting Web Applications](#)

Changing Response Encoding

When you are converting a Java-language Web application that contains JSP and/or HTML pages with an output encoding other than **UTF-8**, you can change the default response encoding of the generated ASP.NET Web application by modifying the Web.config file. Look for the **Globalization** element, and change the **responseEncoding** attribute to the desired encoding.

See Also

Tasks

[Troubleshooting: Matching Code Pages](#)

Conversion of JSP Custom Tag Libraries

Due to the differences between JSP custom tags and .NET Framework Web user controls, a set of support classes that emulate JSP custom tag processing are created during conversion. Those classes are the base of converted JSP custom tag class handlers. Support classes are created for the following classes in the **javax.servlet.jsp.tagext** package:

Original Java-Language Class	Generated Support Class
BodyContent	WCBodyContent
Tag	WCBase
IterationTag	WCIterationBase
BodyTag	WCBodyBase
BodyTagSupport	WCBodyImpl
TagSupport	WCIterationImpl

See Also

Other Resources

[Converting Web Applications](#)

Compile Converted Servlet Classes Before Opening Them

When converting servlet classes, you must resolve all conversion issues and compile your code before you can open a Web Forms page for the application. The Web Forms page cannot be opened in the designer view until the code is compiled. If you try to open it before compiling, you receive an error.

See Also

Other Resources

[Converting Web Applications](#)

Web Tag Samples

The following code example illustrates the methods of the [System.Web.UI.HtmlTextWriter](#) class and shows how to create multiple HTML elements with a single control:

Example

```
using System;
using System.Web;
using System.Web.UI;
using System.Collections.Specialized;
namespace CustomControls
{
    public class Rendered2 : Control, IPostBackDataHandler, IPostBackEventHandler
    {
        private String text1;
        private String text2;
        private String text = "Press button to see if you won.";
        private int number = 100;
        private int Sum
        {
            get
            {
                return Int32.Parse(text1) + Int32.Parse(text2);
            }
        }
        public int Number
        {
            get
            {
                return number;
            }
            set
            {
                number = value;
            }
        }
        public String Text {
            get
            {
                return text;
            }
            set
            {
                text = value;
            }
        }
        public event CheckEventHandler Check;
        protected virtual void OnCheck(CheckEventArgs ce)
        {
            if (Check != null)
            {
                Check(this, ce);
            }
        }
        public virtual bool LoadPostData(string postDataKey, NameValueCollection values)
        {
            text1 = values[UniqueID + "t1"];
            text2 = values[UniqueID + "t2"];
            Page.RegisterRequiresRaiseEvent(this);
            return false;
        }
        public virtual void RaisePostDataChangedEvent() {
        }
        public void RaisePostBackEvent(string eventArgument)
        {

```



```

        OnCheck(new CheckEventArgs(Sum - Number));
    }
    protected override void Render(HtmlTextWriter writer)
    {
        writer.RenderBeginTag(HtmlTextWriterTag.H3);
        writer.Write("Enter a number:");
        writer.RenderEndTag();
        writer.AddAttribute(HtmlTextWriterAttribute.Type, "Text");
        writer.AddAttribute(HtmlTextWriterAttribute.Name, this.UniqueID + "t1");
        writer.AddAttribute(HtmlTextWriterAttribute.Value, "0");
        writer.RenderBeginTag(HtmlTextWriterTag.Input);
        writer.RenderEndTag();
        writer.RenderBeginTag(HtmlTextWriterTag.H3);
        writer.Write("Enter another number:");
        writer.RenderEndTag();
        writer.AddAttribute(HtmlTextWriterAttribute.Type, "Text");
        writer.AddAttribute(HtmlTextWriterAttribute.Name, this.UniqueID + "t2");
        writer.AddAttribute(HtmlTextWriterAttribute.Value, "0");
        writer.RenderBeginTag(HtmlTextWriterTag.Input);
        writer.RenderEndTag();
        writer.RenderBeginTag(HtmlTextWriterTag.Br);
        writer.RenderEndTag();
        writer.AddAttribute(HtmlTextWriterAttribute.Type, "Submit");
        writer.AddAttribute(HtmlTextWriterAttribute.Name, this.UniqueID);
        writer.AddAttribute(HtmlTextWriterAttribute.Value, "Submit");
        writer.AddStyleAttribute(HtmlTextWriterStyle.Height, "25 px");
        writer.AddStyleAttribute(HtmlTextWriterStyle.Width, "100 px");
        writer.RenderBeginTag(HtmlTextWriterTag.Input);
        writer.RenderEndTag();
        writer.RenderBeginTag(HtmlTextWriterTag.Br);
        writer.RenderEndTag();
        writer.RenderBeginTag(HtmlTextWriterTag.Span);
        writer.Write(this.Text);
        writer.RenderEndTag();
    }
}
}
//CheckEvent.cs.
//Contains the code for the custom event data class CheckEventArgs.
//Also defines the event handler for the Check event.
using System;
namespace CustomControls
{
    public class CheckEventArgs : EventArgs
    {
        private bool match = false;
        public CheckEventArgs (int difference)
        {
            if (difference == 0)
            {
                match = true;
            }
        }
        public bool Match
        {
            get
            {
                return match;
            }
        }
    }
    public delegate void CheckEventHandler(object sender, CheckEventArgs ce);
}

```

The following code example demonstrates how to use the constructor of the [System.Web.UI.WebControls.WebControl](#) class to create an HTML `TextArea` element and display it on a Web Forms page:

```

<font face="Courier New" size="2" color="#000080">
<%@ Page Language="C#" %>
<html>
<head>
<script runat="server">
void Button1_Click(Object sender, EventArgs e)
{
    WebControl wc = new WebControl(HtmlTextWriterTag.Textarea);
    Placeholder1.Controls.Add(wc);
}
</script>
</head>
<body>
<form runat="server">
<h3>WebControl Constructor Example</h3>
<p>
<asp:Placeholder id="Placeholder1" runat="Server" />
<br>
<asp:Button id="Button1" Text="Click to create a new TextArea" OnClick="Button1_Click" runa
t="Server" />
<p>
</form>
</body>
</html>

```

Use the [WriteBeginTag](#) method to write any tab spacing and the opening tag of the specified HTML element to the **HtmlTextWriter** output stream.

This method does not write the closing character (>) of the HTML tag so that HTML attributes can be added to the element. Use the [TagRightChar](#) constant to close the tag. Use **WriteBeginTag** with the [SelfClosingTagEnd](#) constant when you write HTML elements that are self-closing.

This method is used by custom server controls that do not allow tag or attribute mapping and render HTML elements in the same way for each request.

```

// Create a manually rendered tag.
writer.WriteBeginTag("img");
writer.WriteAttribute("alt", "AtlValue");
writer.WriteAttribute("myattribute", "No "encoding " required", false);
writer.Write(HtmlTextWriter.TagRightChar);
writer.WriteEndTag("img");
writer.WriteLine();
writer.Indent--;
writer.RenderEndTag();

```

It is possible to obtain an instance, as shown in the following example:

```

HtmlTextWriterTag myTag;
myTag = HtmlTextWriterTag.Area;

```

See Also

Reference

[System.Web](#)

[HtmlTextWriter](#)

[WebControl](#)

Conversion of Various Types of Java-Language Applications

This section discusses the processes to follow and issues that may arise as you convert various types of applications using Java Language Conversion Assistant.

In This Section

[Conversion of JavaBeans Applications](#)

Explains the conversion of JavaBeans, including required manual processing.

[Conversion of EJB Applications](#)

Discusses the major issues dealing with conversion of applications that use Enterprise JavaBeans technology.

[Conversion of CORBA Applications](#)

Explains the conversion process for CORBA applications, including the differences in architecture between CORBA and .NET Framework Remoting.

[Conversion of RMI Applications](#)

Describes the conversion of Remote Method Invocation (RMI) applications.

[Conversion of JMS Applications](#)

Discusses the issues encountered in converting Java Message Service (JMS) applications.

[Conversion of Serialized Applications](#)

Explains the process of converting applications that employ serialization.

[Conversion of Applications That Use JNDI](#)

Describes conversion issues for applications that use the Java Naming and Directory Interface (JNDI).

[Conversion of the Data Packages](#)

Discusses the conversion of database applications.

[Conversion of JAAS Applications](#)

Describes conversion issues relating to Java Authentication and Authorization Service (JAAS) applications.

[Conversion of JCE Applications](#)

Describes conversion issues for cryptographic applications.

[Conversion of Java Swing Applications](#)

Describes conversion issues relating to Java Swing applications.

[Conversion of ActiveX Controls](#)

Describes conversion issues relating to Microsoft ActiveX controls and ActiveX Automation objects.

[Javax.swing Samples](#)

Provides code samples for conversion of Java Swing applications, including use of the **ExtenderProvider** interface, user-defined events, and the **DataSource** property of Windows Forms controls.

[How to: Set the Security Level for Remoting](#)

Describes how to adjust the default security level to allow serialization of objects.

Related Sections

[Convert Dialog Box](#)

Describes the **Convert** dialog box in which you can select the conversion tool for your project.

[Web Tag Samples](#)

Provides code samples for conversion of Web applications, using the **System.Web.UI.HtmlTextWriter** and **System.Web.UI.WebControl** classes.

Command-Line Switches

Describes the command-line switches available in Java Language Conversion Assistant, particularly the property switch.

Conversion of JavaBeans Applications

Before you convert a JavaBeans application, make sure that it has a manifest file. Otherwise, it will be treated like any other file. The manifest file must be named MANIFEST.MF and placed in a folder named META-INF. You can rename the existing file if necessary.

Only JavaBeans identified in the manifest file by the standard convention are converted as such. For example:

```
Name: x\y class
Java-Bean: True | False
```

In this example, *x* represents the package and *y* the bean class component.

The correct identification of beans affects both the class to which they are converted and the application of information contained in an associated **BeanInfo** class in the conversion process. For example, a **JPanel** object is normally converted to a [System.Windows.Forms.Panel](#) object. However, if it is identified in the manifest file as a JavaBeans, it is converted to a [System.Windows.Forms.UserControl](#) object.

Visual JavaBeans that inherit from either the **java.awt.Panel** or the **javax.swing.JPanel** class are converted to inherit from the **System.Windows.Forms.UserControl** class.

Information in associated **BeanInfo** classes is applied during the conversion process, but there is no direct equivalent in the .NET Framework to these classes.

See Also

Reference

[UserControl](#)

Conversion of EJB Applications

When converting an enterprise JavaBeans (EJB) application, only EJB-related class and interface files are included in the converted project. If your application has dependencies on other utility classes, you must add them to the project manually. All utility classes are included in the client project by default and can be manually excluded when needed.

Naming lookup is handled differently in the .NET Framework, resulting in compile errors related to the naming context. Related code can be commented out in the Visual C# files.

You must manually set up component security after conversion.

Since message-driven beans are converted to components, you must verify any code that writes output to the console.

When deploying a component, the DLL must be signed with a strong name. You must generate a key file and integrate it into the assembly manually.

Review the transaction settings for your converted component after deployment.

If your component is published to a Windows 2000 Server, you must remove the **PrivateComponent** tag for all serviced components.

After installing the client project, you must add a reference to the serviced component manually.

Transaction Settings

A serviced component can be associated with one transaction attribute. In the Java language, transaction attributes can be applied at the method level. The following table shows transaction attribute equivalents:

EJB	.NET Framework
NotSupported	NotSupported
Supports	Supported
Required	Required
RequiresNew	RequiresNew
Mandatory	No .NET Framework equivalent
Never	No .NET Framework equivalent

During conversion, Java Language Conversion Assistant attempts to determine the most representative .NET Framework transaction attribute for a component, based on the transaction attributes specified in the EJB deployment descriptor and the number of times each is associated with an EJB method. If the EJB transaction attribute specified for a method is not equivalent to the .NET Framework transaction attribute selected for the component, a warning is generated in the Visual C# code. Look for these warning messages.

Security Settings

The following .NET Framework security settings must be set, either programmatically or through the Component Services explorer:

- Authorization
- Security level
- Authentication level
- Impersonation level

Authorization must be enabled to perform security checks. Set the [System.EnterpriseServices.ApplicationAccessControlAttribute](#) class properties to enable security checking. These properties are used to configure the rest of the settings (security, authentication, and impersonation levels).

Security can be enforced at process level or at both process and component level. The [System.EnterpriseServices.AccessChecksLevelOption](#) enumeration defines the following options:

- [Application](#)
- [ApplicationComponent](#)

When security checks are performed at process level only, role-based security settings at the interface, component, and method levels is ignored and disabled. Set converted applications to **ApplicationComponent** security level.

Authentication level controls the way in which user identities are authenticated in relation to a .NET Framework application. Use one of the following values from the [System.EnterpriseServices.AuthenticationOption](#) enumeration:

- [System.EnterpriseServices.AuthenticationOption.None](#)
- [Connect](#)
- [Call](#)
- [Packet](#)
- [Integrity](#)
- [Privacy](#)

The default authentication level is **Packet**. At this level, every packet that comes from a user is authenticated. Set converted applications to the **Packet** authentication level.

Impersonation refers to how the security identity of a client is propagated when accessing another application or secured resources from the original application. Thus, a server can impersonate a client to some degree. The [System.EnterpriseServices.ImpersonationLevelOption](#) enumeration provides the following options:

- [Anonymous](#)
- [Default](#)
- [Delegate](#)
- [Identify](#)
- [Impersonate](#)

The default impersonation level for .NET Framework applications is **Impersonate**. This level allows a server to act as the client. However, a server cannot access objects or resources located on other computers on the client's behalf. Set converted applications to the **Impersonate** level.

Issues

JLCA converts EJB projects into COM applications. If the EJB source file folder contains both a JAR file and a deployment descriptor Ejb-jar.xml file, two applications are generated in the converted project because the deployment descriptor included in the JAR file is also parsed.

When an EJB is converted, an attempt is made to assign a .NET transaction attribute to its serviced component. If different transaction attributes are assigned globally to the local and remote interfaces for one EJB, no .NET transaction attribute is assigned. Likewise, if one interface is globally assigned a transaction attribute and the other interface only has specific method assignments, no .NET transaction attribute is assigned. Specific method transaction attribute assignments are only used to determine the .NET transaction attribute if there are no global transaction attribute assignments. Because .NET serviced components handle transaction attributes differently from the Java language, review the settings and make any necessary adjustments.

During conversion, default security checking is added, which might not be necessary in your project. If your Java-language source code sets security roles to specific methods or excludes certain methods, review the settings after conversion.

See Also

Reference

[System.EnterpriseServices](#)

Conversion of CORBA Applications

Before you convert a CORBA application, make sure that it has an OMG IDL file. Otherwise, the Java files will be treated as any other class.

Conversion

Make sure that no necessary code is included in the code for skeletons or stubs in your source files. This code will be lost in conversion.

The directory that you specify as the target must be included in the CLASSPATH.

CORBA applications are converted to .NET Framework Remoting over HTTP by default. This means that the converted application no longer uses CORBA clients and servers.

.NET Framework Remoting uses direct client/server connections, with no intervening naming service or middle tier. Therefore, there is no object reference broker (ORB).

Much of the code needed to create the CORBA naming service is unnecessary in .NET Framework Remoting. This includes the classes and collections typically packed into a hash table or array of properties.

Comment out all references to classes in the following packages:

- **org.omg.CosNaming**
- **org.omg.CosNamingContextPackage**

CORBA applications must first obtain the naming context. Java Language Conversion Assistant treats this as a remote lookup, and this is not needed in .NET Framework Remoting. The following line of code can be commented out:

```
NamingContext ncRef = (org.omg.CosNaming.NamingContext) Activator.GetObject(typeof(org.omg.CosNaming.NamingContext), "http://<Host>:<Port>/<Name>");
```

Security Issues

In the .NET Framework, the default security level for distributed communication (remoting) is **low**. This affects passing the user-defined object type to remote methods. For more information, see [How to: Set the Security Level for Remoting](#).

See Also

Reference

[System.Runtime.Remoting](#)

Concepts

[Conversion of Applications That Use JNDI](#)

Conversion of RMI Applications

In the Java language, Remote Method Invocation (RMI) involves three parts: a server, a client, and an object registry.

When the server-side application is compiled, it generates two files: a "skeleton" file on the server side and a "stub" file on the client side. Both are used internally to manage method invocation and data transfer, the skeleton file on the server and the stub file on the client side.

Converting RMI clients and servers to the .NET Framework does not result in Visual C# RMI clients and servers. Instead, .NET Framework Remoting is used over HTTP by default. No extra files (comparable to the skeleton and stub) are generated because method invocation and data transfer are handled internally.

The RMI client requests a remote object by sending a string URL. This string has the following form: `rmi://host:port/name`, where *port* is the port used by the server to register the remote object. (The default is 1099.)

In the .NET Framework, the string URL identifies the channel protocol to be used and has the following form: `protocol://host:port/name`. The *port* is the port registered by the server to listen for client requests.

In the Java language, an object is identified as remote by the fact that it extends the `java.rmi.server.UnicastRemoteObject` class and all its methods raise **RemoteException** errors. Nearly all exceptions that inherit from **RemoteException** are converted to [System.Runtime.Remoting.RemotingException](#) errors. If your application handles specific **RemotingException** subclasses differently, you must adjust them manually.

In the .NET Framework, remote objects can be identified by the fact that they derive from the [System.MarshalByRefObject](#) class.

Each RMI interface is placed in a separate Visual Studio project during conversion. You must update project references to include the remote interface assemblies. If the client and server entry points are included in the converted source tree, the output code must be separated into separate Visual Studio projects. Both client and server projects need a reference to the shared DLL assembly.

Security Issues

In the .NET Framework, the default security level for distributed communication (remoting) is **low**. This affects passing the user-defined object type to remote methods. For more information, see [How to: Set the Security Level for Remoting](#).

See Also

Reference

[RemotingException](#)

[MarshalByRefObject](#)

Concepts

[Conversion of Applications That Use JNDI](#)

Conversion of JMS Applications

Java Message Services (JMS) are converted to use Windows message queuing. Therefore, code to create connections, connection factories, and sessions is commented out during conversion. You must remove the related variables in other places in your code. Ignore the code relating to properties and JNDI in the Visual C# code.

Publisher-subscriber mode is converted in exactly the same way as point-to-point mode.

The following elements of JMS are not supported:

- Topics
- Durable subscribers
- Message selectors
- Exception listeners

You must choose one of the following mechanisms to use multiple-destination messaging in the .NET Framework:

- Distribution lists
- Multiple-element format names
- Multicast addresses

Each of these mechanisms uses a string value to refer to the destination queue and is set in the [System.Messaging.MessageQueue.Path](#) property.

You must change strings that refer to a queue or topic manually to use Windows message queuing. The **TemporaryQueue** object is converted to a physical queue, which must be deleted explicitly.

Transaction queues are handled differently in the .NET Framework. You must set up transactional queues and rewrite the transaction management, using [System.Messaging.MessageQueueTransaction](#) objects.

Message acknowledgment is managed automatically in the .NET Framework. If you have manual message acknowledgment in your code, it must be removed.

If a connection is not available, Windows message queuing does not attempt to connect again.

XA objects are not converted. Distributed transactions must be rewritten in the .NET Framework.

Converted applications cannot interact with JMS-compliant MOMs.

See Also Reference

[System.Messaging](#)
[MessageQueueTransaction](#)

Conversion of Serialized Applications

In the Java language, serializable objects are identified by the fact that they implement the **java.io.Serializable** interface, directly or through inheritance. The .NET Framework uses the [System.Runtime.Serialization.ISerializable](#) interface in a similar way, but the [SerializableAttribute](#) attribute can also be used. Therefore, it is not possible to determine whether an object can be serialized programmatically at run time. Take care with any method that receives or returns the **Serializable** interface in your Java-language code. It is not interchangeable with the equivalent in the .NET Framework, and they cannot read from or write to each other.

In the Java language, serializable classes usually implement the **readObject** and **writeObject** methods. The **writeObject** method is converted to an overloaded constructor, and **readObject** to the `GetObjectData` method. In most cases, the converted code compiles and runs without any further modification. However, it does represent a major change in the definition of classes with custom marshaling and demarshaling code.

See Also

Reference

[ISerializable](#)

Conversion of Applications That Use JNDI

The same service providers must be available in both environments for the converted application to retain functionality. Only LDAP/ActiveDirectory is supported in both the Java language and the .NET Framework.

JNDI methods that typically support naming and directory services are converted to methods in the [System.DirectoryServices](#) namespace. Methods that typically support RMI and CORBA are converted to the [System.Runtime.Remoting](#) namespace. Since some methods can serve a dual purpose, some **System.Runtime.Remoting** methods may need to be changed to **System.DirectoryServices** methods in the converted code.

In the Java language, the context can be instantiated using a constructor that accepts a hash table defining the environment. The [System.DirectoryServices.DirectoryEntry](#) class has no such constructor.

The directory entry path that identifies the provider is case-sensitive and must be changed, for example, from `ldap` to `LDAP`.

In the Java language, search parameters are defined independently of the context, and then passed to the context search method. In the .NET Framework, search parameters are defined in conjunction with the directory entry to be searched by creating a [System.DirectoryServices.DirectorySearcher](#) object. Modifications in your code will be necessary to accommodate this difference.

Instead of the **DirContext.bind** and **DirContext.rebind** methods, use the **System.DirectoryServices.DirectoryEntry** class and its associated classes to add or replace objects in the directory.

Instead of the **Context.createSubcontext** method, use the [System.DirectoryServices.DirectoryEntries.Add\(System.String,System.String\)](#) method, and set the properties of the new entry.

All **Context** methods that have an equivalent in the **javax.rmi.naming** package are converted to [System.Runtime.Remoting.RemotingServices](#) methods. These include **Context.bind**, **Context.lookup**, **Context.rebind**, and **Context.unbind**.

The **javax.naming.Ldap** package is not supported.

See Also

Concepts

[Conversion of CORBA Applications](#)

[Conversion of RMI Applications](#)

Conversion of the Data Packages

Applications that use the **java.sql** and **javax.sql** packages are converted to the [System.Data.OleDb](#) namespace. If you use a different database system, you can convert these references to the client of your system, for example, [System.Data.SqlClient](#) or [System.Data.ODBC](#).

The .NET Framework Data Providers automatically pool connections. The connection pooling can be configured for each Data Provider, i.e. ODBC, OleDb and SQL Server.

You must convert all connection strings to the .NET Framework format. The following examples show how to convert the various types of database connection.

Example

The following example shows how to use the JDBC-ODBC bridge:

Original Visual J++ Code

```
String dbUrl = "jdbc:odbc:BiblioODBC";
Connection c = DriverManager.getConnection(dbUrl);
```

Equivalent Visual C# Code

```
String dbUrl = "Provider = MSDASQL; DSN = BiblioODBC";
ODBCConnection c = "DriverManager.getConnection(dbUrl);
```

The following example show how to use a URL from Microsoft SQL Server:

Original Visual J++ Code

```
String dbUrl = "jdbc:microsoft:sqlserver://MySQLServer:1433; DatabaseName=pubs";
Connection c = "DriverManager.getConnection(dbUrl, username, pwd);
```

Equivalent Visual C# Code

```
String dbUrl = "Provider = SQLOLEDB; DataSource = MySQLServer; Initial Catalog = pubs";
SQLConnection c = DriverManager.getConnection (dbUrl + "; User ID = " + username + "; PWD = " + pwd);
```

Original Oracle Connection String

The following example shows how to change an Oracle database connection string:

```
String jdbcUrl = "jdbc:oracle:thin:@localhost:1521:ORCL";
conn = DriverManager.getConnection(jdbcUrl, user, password);
```

Equivalent Visual C# Code

```
String jdbcUrl = "Provider = MSDAORA; Data Source = localhost:1521;";
OracleConnection conn = DriverManager.getConnection(jdbcUrl + "User ID = " user + "; Passwo" + "rd = " + password);
```

Updatable, scrollable result sets are not reliably converted. A support class is provided, but it will require some manual adjustment. If your application relies heavily on such result sets, use the [System.Data.DataView](#) or the [System.Data.DataTable](#) class, rather than the [System.Data.OleDb.OleDbDataReader](#) class, to which they are automatically converted.

Clob and **Blob** data types are converted to character and byte arrays, respectively, which have different behavior and might use more memory. The converted code fetches the data sequentially and creates the array. You might want to write the data stream to a disk file. You can also use the [System.Data.OracleClient.OracleLob](#) class, which is used to represent the **Large Object Binary** data type on an Oracle server. Some new data types such as **STRUCT**, **ARRAY**, or user-defined types are not supported in the .NET Framework. The managed [System.Data.OracleClient.OracleBFile](#) class works with the Oracle **BFile** data type.

Data access types are specific to database type. All JDBC code is converted to OleDb types because OleDb drivers are available

for most databases. In some instances, you might want to change this after conversion for better performance or support. Other data adapters available in the .NET Framework include ODBC, SQL, and Oracle.

DataSources are not mapped; however, in some cases [OleDbConnection](#) or any other client connection classes under [System.Data](#) namespace can be used to represent some of the functionality. Logging features of the **javax.sql.DataSource** interface can be implemented using the [System.Diagnostics.EventLog](#) class.

The **javax.sql** package is not reliably converted. The interfaces in that package must be implemented by third-party drivers.

The **Driver** interface and the **DriverManager** class are deprecated and are no longer needed. Some of their methods can be converted, but they are unnecessary.

See Also

Reference

[System.Data.OleDb](#)

[System.Data.SqlClient](#)

[System.Data.Odbc](#)

[DataAdapter](#)

[DataTable](#)

[OleDbDataReader](#)

Concepts

[Understanding Connection Pooling](#)

[Using Connection Pooling with SQL Server](#)

Conversion of JAAS Applications

The .NET Framework uses role-based security with principal, identity, and permission classes to handle security. You can choose one of the built-in security modules within the .NET Framework, rather than building one yourself. When you convert Java Authentication and Authorization Service (JAAS) applications to the .NET Framework, you must take into account the differences between the two approaches to security.

All JAAS configuration files must be renamed as JAAS.config to be processed by Java Language Conversion Assistant. These are converted to App.config files, which can be used by support-class methods to obtain authentication modules and register them with the authentication manager.

The **LoginContext** class is converted to the static **System.Security.AuthenticationModule** class, which has different behavior.

The **LoginModule** class is converted to the [IAuthenticationModule](#) interface. In the Java language, the **LoginContext** object registers a **LoginModule** object, which uses callback handlers to request input from the user and login module to authenticate users. In the .NET Framework, authentication modules are registered with the authentication manager, which loops through registered authentication modules to return authorization information.

A generic [System.Security.SecurityException](#) is used instead of the different JAAS exceptions.

The **Subject** class is converted to the [System.Security.Principal.GenericPrincipal](#) class, which has different behavior. There is also a [System.Security.Principal.WindowsPrincipal](#) class for use with Windows authentication.

See Also

Reference

[System.Security](#)

[System.Security.Principal](#)

[GenericPrincipal](#)

[WindowsPrincipal](#)

Conversion of JCE Applications

In the Java language, generic classes or interfaces are used as base classes for all encryption classes. In the .NET Framework, the [System.Security.Cryptography.AsymmetricAlgorithm](#) class is the base class for all public-key algorithms. During conversion, the **CryptoSupport** support class is generated as a wrapper around the [System.Security.Cryptography.SymmetricAlgorithm](#) class. You must re-implement asymmetric algorithms manually.

In the Java language, all encryption algorithm types are generated by the **Cipher.getInstance** method with a string literal indicating the type of encryption. Each type of encryption uses a separate class in the .NET Framework. A relatively limited set of encryption classes is provided.

See Also

Reference

[System.Security.Cryptography](#)

[SymmetricAlgorithm](#)

[AsymmetricAlgorithm](#)

Conversion of Java Swing Applications

Swing applications are converted to the [System.Windows.Forms](#) namespace. Therefore, the following Swing elements have no direct equivalent in Visual C# and cannot be converted.

- The **javax.swing.plaf** package is not supported. There is no equivalent in the .NET Framework to the classes that encapsulate Swing pluggable look-and-feel or methods that refer to it.
- Renderers and editors are not supported.
- Java AWT layouts are not supported. Although these are not Swing-specific, they affect most Swing applications.

The following elements have limited support or require significant manual adaptation after conversion with the Java Language Conversion Assistant:

- Models are only partially supported.
- Classes such as **JTree** and **JTable** cannot be converted directly to their equivalents in the .NET Framework because of their dependence on models. Support classes are implemented to help match these classes, but not all items can be mapped.
- The **ContentPane** class and other intermediate-level containers cannot be converted directly. For example, the **JFrame** object contains controls in its content pane, but the .NET Framework [System.Windows.Forms.Form](#) class contains controls in itself, without any intermediate object. This results in inheritance issues, and conversion requires significant user input.
- Classes whose behavior is specified by constant values cannot be converted completely automatically. For example, the **FileDialog** class has a constant that specifies whether it opens or saves a file. This is handled by two separate classes in Visual C#.
- Event handling is built on a different model in the .NET Framework, so all Swing events require manual adaptation after conversion. A sample of event handling conversion is included in the [Javax.swing Samples](#) topic.

See Also

Reference

[Javax.swing Error Messages](#)

[System.Windows.Forms](#)

Concepts

[Javax.swing Samples](#)

Conversion of ActiveX Controls

The **com.ms.wfc.ax** package contains classes and interfaces for Microsoft ActiveX controls and ActiveX Automation objects.

In the .NET Framework, Windows Forms can host only Windows Forms controls. ActiveX controls must be wrapped to appear like Windows Forms controls in a wrapper class that derives from the [System.Windows.Forms.AxHost](#) class.

You can convert ActiveX components in any of the following ways:

- Visual Studio automatically converts COM types in a type library to metadata in an assembly.
- Import type libraries provide command-line switches to adjust metadata and generate an interoperation assembly and a namespace.
- Custom wrappers are the most labor-intensive option, but can be individualized to your application needs.

See Also

Reference

[AxHost](#)

Javax.swing Samples

The following example shows how to use extenders to provide properties to controls, using the [System.ComponentModel.IExtenderProvider](#) interface:

```
using System;
using System.Collections;
using System.ComponentModel;
using System.Drawing;
using System.Data;
using System.Windows.Forms;
using System.Globalization;
namespace Extenders
{
    [ProvideProperty("MyString", typeof(Control))]
    public class UserControl1 : System.Windows.Forms.UserControl, IExtenderProvider
    {
        private System.ComponentModel.Container components = null;
        public UserControl1()
        {
            InitializeComponent();
        }
        protected override void Dispose( bool disposing )
        {
            if( disposing )
            {
                if(components != null)
                {
                    components.Dispose();
                }
            }
            base.Dispose( disposing );
        }
        #region Component Designer generated code
        private void InitializeComponent()
        {
            this.Name = "UserControl1";
            this.Size = new System.Drawing.Size(170, 160);
        }
        #endregion
        [Browsable(true)]
        public String GetMyString(Control control)
        {
            return "This is my string.";
        }
        public void SetMyString(Control container, String value)
        {
        }
        public bool CanExtend(Object control)
        {
            return true;
        }
    }
}
```

The following example shows how to implement handling and raising user-defined events:

```
namespace EventSample
{
    using System;
    using System.ComponentModel;
    //Class that contains the data for
    //the alarm event. Derives from System.EventArgs.
    public class AlarmEventArgs : EventArgs
```

```

{
    private readonly bool snoozePressed;
    private readonly int nrings;
    public AlarmEventArgs(bool snoozePressed, int nrings)
    {
        this.snoozePressed = snoozePressed;
        this.nrings = nrings;
    }
    //The NumRings property returns the number of rings
    //that the alarm clock has sounded when the alarm event
    //is generated.
    public int NumRings
    {
        get { return nrings;
        }
    }
}
//The SnoozePressed property indicates whether the snooze
//button is pressed on the alarm when the alarm event is generated.
public bool SnoozePressed
{
    get {return snoozePressed;}
}
//The AlarmText property that contains the wake-up message.
public string AlarmText
{
    get
    {
        if (snoozePressed)
        {
            return ("Wake up! Snooze time is over.");
        }
        else
        {
            return ("Wake Up!");
        }
    }
}
}

//Delegate declaration.
public delegate void AlarmEventHandler(object sender, AlarmEventArgs e);
//The Alarm class that raises the alarm event.
public class AlarmClock
{
    private bool snoozePressed = false;
    private int nrings = 0;
    private bool stop = false;
    //The Stop property indicates whether the
    //alarm should be turned off.
    public bool Stop
    {
        get {return stop;}
        set {stop = value;}
    }
    //The SnoozePressed property indicates whether the snooze
    //button is pressed on the alarm when the alarm event is
    //generated.
    public bool SnoozePressed
    {
        get {return snoozePressed;}
        set {snoozePressed = value;}
    }
    //The event member that is of type AlarmEventHandler.
    public event AlarmEventHandler Alarm;
    //The protected OnAlarm method raises the event by invoking
    //the delegates. The sender is always this, the current instance
    //of the class.
    protected virtual void OnAlarm(AlarmEventArgs e)
    {

```

```

        if (Alarm != null)
        {
            //Invokes the delegates.
            Alarm(this, e);
        }
    }
    //This alarm clock does not have
    //a user interface.
    //To simulate the alarm mechanism, it has a loop
    //that raises the alarm event at every iteration
    //with a time delay of 300 milliseconds,
    //if snooze is not pressed. If snooze is pressed,
    //the time delay is 1000 milliseconds.
    public void Start()
    {
        for (;;)
        {
            nrings++;
            if (stop)
            {
                break;
            }
            else if (snoozePressed)
            {
                System.Threading.Thread.Sleep(1000);
                {
                    AlarmEventArgs e = new AlarmEventArgs(snoozePressed,
                    nrings);
                    OnAlarm(e);
                }
            }
            else
            {
                System.Threading.Thread.Sleep(300);
                AlarmEventArgs e = new AlarmEventArgs(snoozePressed, nrings);
                OnAlarm(e);
            }
        }
    }
}
//The WakeMeUp class that has a method AlarmRang that handles the
//alarm event.
public class WakeMeUp
{
    public void AlarmRang(object sender, AlarmEventArgs e)
    {
        Console.WriteLine(e.AlarmText + "\n");
        if (!(e.SnoozePressed))
        {
            if (e.NumRings % 10 == 0)
            {
                Console.WriteLine("Let alarm ring? Enter Y");
                Console.WriteLine(" Press Snooze? Enter N");
                Console.WriteLine(" Stop Alarm? Enter Q");
                String input = Console.ReadLine();
                if (input.Equals("Y") || input.Equals("y")) return;
                else if (input.Equals("N") || input.Equals("n"))
                {
                    ((AlarmClock)sender).SnoozePressed = true;
                    return;
                }
            }
            else
            {
                ((AlarmClock)sender).Stop = true;
                return;
            }
        }
    }
}
}

```

```

        else
        {
            Console.WriteLine(" Let alarm ring? Enter Y");
            Console.WriteLine(" Stop alarm? Enter Q");
            String input = Console.ReadLine();
            if (input.Equals("Y") || input.Equals("y")) return;
            else
            {
                ((AlarmClock)sender).Stop = true;
                return;
            }
        }
    }
}

//The driver class that hooks up the event handling method of
//WakeMeUp to the alarm event of an Alarm object using a delegate.
//In a forms-based application, the driver class is the
//form.
public class AlarmDriver
{
    public static void Main (string[] args)
    {
        //Instantiates the event receiver.
        WakeMeUp w= new WakeMeUp();
        //Instantiates the event source.
        AlarmClock clock = new AlarmClock();
        //Connects the AlarmRang method to the Alarm event.
        clock.Alarm += new AlarmEventHandler(w.AlarmRang);
        clock.Start();
    }
}

```

The following example shows how to use the `DataSource` property of a control to emulate model characteristics.

```

public class DataBinding : System.Collections.ArrayList
{
    public DataBinding()
    {
    }
    public void AddNew(int index, object obj)
    {
        this.Add(obj);
    }
}

DataBinding s_model = new DataBinding();
DataBinding i_model = new DataBinding();
System.Windows.Forms.ListBox listBox1 = new System.Windows.Forms.ListBox();
public void SetModel(int whichModel)
{
    if (whichModel==0)
    {
        i_model.Add("i_model Value1");
        i_model.Add("i_model Value2");
        listBox1.DataSource = i_model;
    }
    else
    {
        s_model.Add("s_model Value1");
        s_model.Add("s_model Value2");
        listBox1.DataSource = s_model;
    }
}

```


com.ms.wfc.data.Connection example

The following example uses the methods of the **com.ms.wfc.data.Connection** methods:

1. Add a reference to the Microsoft ActiveX Data Objects (ADO) 2.7 Library (Adodb.dll).
 2. Create a connection to the database with the **Connection** object (**ConnectionClass** class).
- Call the **ConnectionClass.Open** method.

The *Provider* parameter indicates the Jet 4 OLE DB provider, and the *Data Source* parameter points to the physical location of the database.

See Also

Other Resources

[ADO API Reference](#)

How to: Set the Security Level for Remoting

In the .NET Framework, the default security level for distributed communication (remoting) is **low**. This affects passing the user-defined object type to remote methods. You might need to adjust the default security level to **full** in some cases to allow serialization of objects.

To adjust the security level

- Modify the configuration file(s) generated in conversion.
- or -
- Edit the converted code.

Whether you need to make this adjustment will not become apparent until your application is running and a [System.Runtime.Serialization.SerializationException](#) is raised with the following description:

Because of security restrictions, the type [System.Runtime.Remoting.ObjRef](#) cannot be accessed.

See Also

Concepts

[Conversion of CORBA Applications](#)

[Conversion of RMI Applications](#)

JLCA Diagnostic Messages Listed by Package

This section contains links to all the diagnostic messages for the Java Language Conversion Assistant. The pages containing the links are organized by package.

[Java Language Errors, Warnings, and Issues](#)

[Com.ms.activex Error Messages](#)

[Com.ms.awt Error Messages](#)

[Com.ms.com Error Messages](#)

[Com.ms.directx Error Messages](#)

[Com.ms.dll Error Messages](#)

[Com.ms.dxmedia Error Messages](#)

[Com.ms.fx Error Messages](#)

[Com.ms.io Error Messages](#)

[Com.ms.jdbc.odbc Error Messages](#)

[Com.ms.lang Error Messages](#)

[Com.ms.mtx Error Messages](#)

[Com.ms.object Error Messages](#)

[Com.ms.ui Error Messages](#)

[Com.ms.util Error Messages](#)

[Com.ms.wfc Error Messages](#)

[Com.ms.wfc.app Error Messages](#)

[Com.ms.wfc.ax Error Messages](#)

[Com.ms.wfc.core Error Messages](#)

[Com.ms.wfc.data Error Messages](#)

[Com.ms.wfc.data.adodb Error Messages](#)

[Com.ms.wfc.data.ui Error Messages](#)

[Com.ms.wfc.io Error Messages](#)

[Com.ms.wfc.ole32 Error Messages](#)

[Com.ms.wfc.ui Error Messages](#)

[Com.ms.wfc.util Error Messages](#)

[Com.ms.wfc.win32 Error Messages](#)

[Com.ms.win32 Error Messages](#)

[Com.sun.image.codec Error Messages](#)

[Java.applet Error Messages](#)

[Java.awt Error Messages](#)

[Java.awt.color Error Messages](#)

[Java.awt.datatransfer Error Messages](#)

[Java.awt.dnd Error Messages](#)

[Java.awt.event Error Messages](#)

[Java.awt.font Error Messages](#)
[Java.awt.geom Error Messages](#)
[Java.awt.im Error Messages](#)
[Java.awt.image Error Messages](#)
[Java.awt.peer Error messages](#)
[Java.awt.print Error Messages](#)
[Java.beans Error Messages](#)
[Java.io Error Messages](#)
[Java.lang Error Messages](#)
[Java.math Error Messages](#)
[Java.net Error Messages](#)
[Java.rmi Error Messages](#)
[Java.security Error Messages](#)
[Java.sql Error Messages](#)
[Java.text Error Messages](#)
[Java.text.resources Error Messages](#)
[Java.util Error Messages](#)
[Java.util.cab Error Messages](#)
[Java.util.jar Error Messages](#)
[Java.util.zip Error Messages](#)
[Javax.accessibility Error Messages](#)
[Javax.crypto Error Messages](#)
[Javax.ejb Error Messages](#)
[Javax.jms Error Messages](#)
[Javax.mail Error Messages](#)
[Javax.naming Error Messages](#)
[Javax.rmi Error Messages](#)
[Javax.security Error Messages](#)
[Javax.servlet Error Messages](#)
[Javax.servlet.http Error Messages](#)
[Javax.servlet.jsp Error Messages](#)
[Javax.sound Error Messages](#)
[Javax.sql Error Messages](#)
[Javax.swing Error Messages](#)
[Javax.swing.beaninfo Error Messages](#)
[Javax.swing.border Error Messages](#)
[Javax.swing.colorchooser Error Messages](#)
[Javax.swing.event Error Messages](#)
[Javax.swing.filechooser Error Messages](#)
[Javax.swing.table Error Messages](#)

[Javax.swing.text Error Messages](#)

[Javax.swing.tree Error Messages](#)

[Javax.swing.undo Error Messages](#)

[Javax.transaction Error Messages](#)

[Javax.xml Error Messages](#)

[Org.omg Error Messages](#)

[Org.w3c Error Messages](#)

[Org.xml Error Messages](#)

Java Language Errors, Warnings, and Issues

- (1002) Compile Error: Invalid statement
- (1003) Note: Declaration changed from final to variable
- (1005) Note: Initialize expression was moved to method or constructor
- (1011) Note: New code element generated
- (1012) Note: Break statement with label is not supported
- (1014) Note: Labeled statement moved by Java Language Conversion Assistant
- (1015) Note: Continue statement with label is not supported
- (1019) Note: Inner classes not provided with enclosing instance
- (1021) Note: Inner class member field this\$0 is not supported
- (1022) Note: Local class declaration is not supported
- (1023) Note: Final variable copied to inner class as member field
- (1024) Global Note: Anonymous class declaration is not supported
- (1025) Note: Interfaces nested within interfaces are not allowed in C#
- (1027) Note: Synchronized methods are not supported in the .NET Framework
- (1042) Global Warning: Data types in Visual C# might be different
- (1043) To Do: Method or property returns a different value in the .NET Framework
- (1045) Note: Interfaces cannot contain fields in the .NET Framework
- (1062) To Do: NULL value of database columns can be checked by using OleDbDataReader.IsDBNull
- (1063) To Do: Modify connection string(s) to match the .NET Framework format
- (1064) To Do: Modify the string connection to match the .NET Framework format and add the java.util.Properties information to it
- (1066) To Do: setLoginTimeout must be a parameter to connection string for OleDbConnection constructor
- (1075) Note: Font constructor might generate non-equivalent instances
- (1077) To Do: Inheritance for class not supported
- (1078) To Do: Make sure that resources used in this class are valid resources files
- (1079) Compile Error: All the methods inherited from an abstract class must be implemented
- (1083) Note: Bit arrays must be of the same size to allow the operation
- (1086) Compile Error: Field equivalent cannot be assigned to an integer variable
- (1088) To Do: There is no NULL value defined for a .NET structure
- (1089) To Do: Access and file format of configuration settings are different
- (1091) Runtime Warning: The equivalent constructor has a different number of parameters
- (1092) To Do: Expected value or type of a parameter is different
- (1093) To Do: Change code to access value of a .NET class member
- (1095) To Do: Code element not mapped in the current release
- (1096) Note: Fields mapped to enumerations initialized with integer values
- (1099) Note: Exceptions thrown by method might be different
- (1100) Note: Catch clauses in a try statement with the same exception
- (1101) Runtime Warning: Mapped parameters might throw an exception

(1102) Compile Error: Class must be modified to call superclass

(1103) Compile Error: Field has a different type

(1104) To Do: Use `ConnectionString` property to set logon and password

(1105) To Do: Method invocation is not necessary

(1107) Global Note: Conversion of `setAllowAddNew`, `setAllowDelete`, and `setAllowUpdate` of the `DataGrid` class

(1108) To Do: Property or method must be set or called before an invocation

(1109) Compile Error: Converted parameter not compatible with parameter type of method

(1113) Compile Error: Member is reserved for internal use of its package

(1114) To Do: Parameter of `setFilter` in the `OpenFileDialog` and `SaveFileDialog` classes must be changed

(1115) Global Warning: Unsafe code requires a compiler switch to compile

(1116) To Do: COM Custom marshaller might become invalid

(1118) To Do: Check boxes do not provide mutual exclusion when grouped

(1119) Note: User control event attribute not valid

(1123) Note: Class maps to interface

(1124) Note: Code was removed from method body

(1128) Compile Error: Break statement at finally clause level is invalid

(1130) To Do: Equivalent of `java.sql.DatabaseMetaData` handles query results in `System.Data.DataTable`

(1132) To Do: Equivalent class does not throw exception for malformed URL

(1133) Global Warning: `Equals` method can return a different value

(1135) Global Error: The `System.Windows.Forms.Application.Run` method must be called for the main window

(1137) To Do: User get or set method converted to property has name collisions

(1139) Global Warning: Directory was not converted

(1140) Global Warning: Thread name can be set just once

(1141) Note: Java Language Conversion Assistant commented code from Java Callable Wrapper class for a COM coclass

(1142) To Do: The event method needs to be overloaded

(1143) To Do: Method is not an override method

(1144) Runtime Warning: Add extra logic into `componentHidden`

(1145) Runtime Warning: Add extra logic into `componentShown`

(1146) To Do: Declaration and construction for class instance in different lines cannot be converted

(1147) To Do: The `<classname>` class is marked as sealed

(1148) Note: Synchronized keyword has been removed

(1151) To Do: Inaccessible constructor

(1152) To Do: You must manually convert access to the registry

(1153) To Do: Untranslated statement

(1154) Note: Parameter to constructor of equivalent of class `com.ms.wfc.data.ui.DataNavigator`

(1155) To Do: Class that derives from `Delegate` or `MulticastDelegate`

(1156) Compile Error: Wrong statements

(1157) Compile Error: Method with the same parameter types

(1158) To Do: The way of accessing the file was modified

(1159) Global Warning: Path of archives that depend on the working directory

(1160) To Do: One interface includes the functionality of another one

(1167) Note: `getParameter` calls without a string literal

(1168) To Do: `getParameter` that requires rename of parameters

(1169) Note: The default value of property has been changed

(1170) Global Warning: Class mapped to a struct; cannot contain or return a null value

(1171) Note: `getSource` method must be in an event handling routine

(1172) Global Error: Input project file is corrupt

(1173) To Do: Hexadecimal literal returns a different value

(1174) Note: The form to load a binary file in the .NET Framework is different

(1175) Global Warning: Unsigned long detected

(1177) Runtime Warning: Implementation of interface was converted to a single class

(1178) To Do: Method needs to be modified

(1179) Global Warning: Request the `OleDbManager` connection string

(1180) Note: Only one transactional attribute is supported

(1181) Runtime Warning: Expressions used more than once in C# code

(1182) To Do: Session parameters are stored in a different order

(1183) Runtime Warning: Init functionality has been converted

(1184) To Do: <Target Name> requires changing the type of the receiving instance

(1185) To Do: Queue access is obtained differently

(1186) Compile Error: Class hierarchy differences might cause compile errors

(1187) Runtime Warning: <Member type> might be executed every time the ASP.NET page loads

(1188) Note: You can choose a protocol for converted RMI methods

(1189) To Do: Multiple-destination messaging requires special destination formatting

(1190) To Do: Activation of Remote objects

(1191) Note: Instance field was converted to static

(1192) To Do: Serialization code may not be converted correctly

(1193) Compile Error: Serialization code was not used in the conversion of the method

(1194) Runtime Warning: A key for a serialized value must be used only once and must match the corresponding key

(1195) To Do: Change in override modifier for nonvirtual method

(1196) Note: Remote interface file conversion

(1197) To Do: Delegate instantiated with String value for function name

(1198) Note: Inheritance of servlet classes can cause wrong manipulation of variables

(1199) Note: Respective javadoc comments were merged

(1200) Note: Base class shared fields have been redeclared with the new modifier

(1201) To Do: Multiple channel registration

(1202) To Do: Reference conversion needs user modification

(1203) Note: Using the class interface to expose public members

(1204) Note: Access modifiers were changed

(1205) To Do: Methods with parameters of primitive type overloaded with parameters of wrapper class type

(1206) Global Note: Converted project must be compiled before opening in Visual Studio designer view

(1207) To Do: Types extending from COM objects should override all methods

(1208) Global Note: System.Data.OleDb namespace can be replaced with another namespace

(1209) Note: Method was not converted to property

(1210) Global Warning: Deserialization level might have to be changed

(1211) Runtime Warning: Channel registration can fail without notice

(1212) Note: Array bounds were not validated

(1213) Global Note: Configuration file was added during the conversion

(1214) Compile Error: Server address must be completed by the user

(1215) Compile Error: Server port must be completed by the user

(1216) Compile Error: Remote object name must be completed by the user

(1217) Global Warning: Languages other than Java are not supported

(1218) Global Warning: Java-language non-native definitions converted to Visual C# native constructions

(1219) Global Note: System.MarshalByRefObject.InitializeLifetimeService overridden

(1220) To Do: Reset method must be implemented

(1221) To Do: Parent invocations in methods from applets

(1222) Runtime Warning: Equivalent constructor has more parameters

(1223) Global Warning: Output of bidirectional languages

(1224) To Do: Method returns different type

(1225) Global Warning: Primitive type castings have different behavior

(1227) Note: Code copied for visual layout

(1228) Note: Commented code was moved to the InitializeComponent method

(1230) Runtime Warning: Add a ContainerControl before other forms

(1231) To Do: More than one Java class member is converted to the same member in Visual C#

(1232) To Do: The method or property must be implemented in order to preserve the class logic

(1233) Note: In a converted servlet class, the keyword "this" may behave differently

(1234) Note: Inner class is now serializable

(1235) Compile Error: There is no equivalent base class method in the .NET Framework

(1236) To Do: Verify the list of servlet filters

(1237) Note: EJB was converted to serviced component

(1238) To Do: EJB callback method not supported

(1239) Global Warning: Class or interface used by more than one EJB

(1240) To Do: Reimplement data binding for EJB beans with container-managed persistence

(1241) To Do: Move and adapt EJBHome interface methods

(1242) To Do: Reimplement EJB 2.0 CMP finder method

(1243) To Do: Reimplement EJB 1.1 CMP finder method

(1244) To Do: Reimplement the EJB 2.0 CMP relationship

(1245) To Do: EJB transaction attribute not supported at method level

(1246) To Do: EJB transaction attribute not supported

(1247) Global Warning: Exceptions thrown within transaction contexts always roll back the current transaction

(1248) Warning: EJB method cannot be excluded

(1249) Warning: Security access checks cannot be disabled for unchecked methods

(1250) Global Warning: Security level needs to be changed

(1251) Note: Custom marshaling and unmarshaling is ignored

(1252) Global Warning: System includes are ignored

(1253) To Do: Replace EJB resource references

(1254) To Do: Replace EJB references with .NET equivalents

(1255) Global Note: EJB environment entries must be available to the current component

(1256) Global Note: EJB reentrance not supported

(1257) Global Warning: JNDI security

(1258) To Do: Adjust remoting context initialization manually

(1259) To Do: Remote object registration uses only the default constructor

(1260) To Do: IDL value type default factories are not needed in Visual C#

(1261) Note: Servlet destroy methods will not be called by the runtime

(1262) To Do: Import declarations of unavailable types or packages may cause compile errors

(1263) To Do: Web application filter behavior is different.

(1264) Global Warning: Events behave differently in the .NET Framework

(1265) Runtime Warning: Equivalent method has fewer parameters

(1266) Runtime Warning: Objects derived from System.MarshalByRefObject can be serialized only in their own domain

(1267) Note: Portions of code for the visual layout were moved to InitializeComponent

(1268) Note: A parameterless constructor was added for a serializable class

(1269) To Do: Members of classes that have more than one equivalent in Visual C# may not work

(1270) To Do: Constructor needs to be replaced

(1271) Note: This method can be removed

(1272) To Do: Unsupported IDL preprocessor directives were not converted

(1273) Note: Context was not converted

(1274) Global Warning: Member execution sequence may differ

(1275) Note: RMI server applications wait for client requests before terminating

(1277) To Do: Equivalent class is not serializable

(1278) Note: Out parameters must be assigned before the method returns

(1279) Runtime Warning: Method name passed to delegate creation may not be valid

(1280) To Do: A third-party package was found, but not necessarily converted

(1281) Global Warning: Remoting exceptions might occur due to invalid XML characters

(1282) Global Warning: Visual C# might not resolve ambiguous namespaces properly

(1283) Global Warning: Move converted ASP projects or JSP applets

(1284) Note: The getModifiers method simulation might not work for some controls

(1285) Global Warning: The converted implementation of a property editor could have different behavior

(1286) Note: Value type implementation was moved to the shared assembly

(1287) To Do: Transformation string might not be supported

(1288) Note: Cryptographic classes that handle keys behave differently

(1290) Global Warning: DirectX must be installed to support sound

(1291) To Do: Structures do not have an equivalent to NULL in Visual C#

(1292) Note: UI scroll fields have no effect on scrollbar conversion

(1293) Global Warning: All JNDI exceptions are mapped to System.Exception

(1294) Compile Error: The com.ms.wfc.html package is not supported

(1295) To Do: Visual C# has no constructor with an object as a parameter

(1296) Note: Change in access modifier

(1297) Global Warning: Enumerators must be started before accessing their data

(1298) Note: BeanInfo functionality transferred to the corresponding Visual C# component

(1299) To Do: Returned expression must be converted manually

(1300) To Do: TypeConverter needs a proper System.ComponentModel.Design.Serialization.InstanceDescriptor

(1301) Global Warning: JavaBeans sources must be in the project directory

(1302) Global Error: Conversion of ValueBase can cause a type mismatch

(1303) Note: The ref keyword was added to structure parameters

(1304) Note: Conversion of JavaBeans could generate deployment issues

(1305) Global Warning: Event-handler links for AwtUI components might be lost

(1306) Note: Method or property implementation has been added

(1307) To Do: Statement cannot be moved to InitializeComponent

(1308) Note: Methods in listener are converted, but not used

(1309) Note: Delegate might have a different return value

(1310) To Do: EJB class has multiple-inheritance issues

(1311) To Do: Possible EJB is missing a deployment descriptor

(1312) Runtime Warning: There is no equivalent to the Mandatory transaction attribute

(1313) Note: Statement is unnecessary if there are no references to it

(1314) Global Warning: Converted EJB has default security features enabled

(1315) To Do: Parameter cannot be passed by reference

(1316) Global Warning: EJB transaction attribute was not applied

(1317) Compile Error: ValueType arrays cannot be assigned, compared, or passed as parameters

(1318) Global Warning: You need to rename the JAAS configuration file

(2001) Runtime Warning: Some server-side tags that must be unique could be repeated when including files

(2002) Runtime Warning: Dynamic file inclusion is not allowed in Aspx

(2003) Note: Attribute "buffer" can have only two valid values in ASP.NET

(2004) Note: Request scope was changed to Session scope

(2005) Global Error: Web.Config customErrors Mode property must be set to "On"

(2006) To Do: File will not be converted

(2007) Note: Script variables can be out of scope when moving blocks inside custom tags to methods

(2008) Note: Control structures will not work properly when moving blocks of code inside custom tags

(4010) Global Error: ActiveX reference is different

(4011) Global Error: Invalid character found in file

(4012) Global Warning: Code page not available on system

(4013) Note: Specified TLD file could not be found or parsed

(4014) Global Warning: Typelib not the same as the registered one

(4015) Note: User control ID has been changed

(4020) Global Error: Archive file could not be decompressed

(4021) Global Error: Archive file could not be read

(5000) To Do: Method is not marked as virtual

See Also

Other Resources

[JLCA Diagnostic Messages Listed by Package](#)

[Converting Java Applications to Visual C#](#)

Com.ms.activex Error Messages

com.ms.activex could not be converted

com.ms.activex.ActiveXControl could not be converted

com.ms.activex.ActiveXControlListener could not be converted

com.ms.activex.ActiveXControlServices could not be converted

com.ms.activex.ActiveXInputStream could not be converted

com.ms.activex.ActiveXOutputStream could not be converted

com.ms.activex.ActiveXToolkit could not be converted

com.ms.activex.PropertyDialogThread could not be converted

Com.ms.awt Error Messages

com.ms.awt.AccessibleWrapper could not be converted

com.ms.awt.AWTFinalizeable could not be converted

com.ms.awt.AWTFinalizer could not be converted

com.ms.awt.AWTPermission.check could not be converted

com.ms.awt.AWTPermission.pid could not be converted

com.ms.awt.CaretX could not be converted

com.ms.awt.CharsetString could not be converted

com.ms.awt.CharToByteSymbol could not be converted

com.ms.awt.ColorX.ColorX(float, float, float) could not be converted

com.ms.awt.ColorX.ColorX(int) could not be converted

com.ms.awt.ColorX.ColorX(int, int, int) could not be converted

com.ms.awt.ColorX.getHilight could not be converted

com.ms.awt.ColorX.getShadow could not be converted

com.ms.awt.Device.Device could not be converted

com.ms.awt.Device.getBasics could not be converted

com.ms.awt.Device.getDisplayContext could not be converted

com.ms.awt.DrawingSurface could not be converted

com.ms.awt.DrawingSurfaceInfo could not be converted

com.ms.awt.EventFilterListener could not be converted

com.ms.awt.FocusEvent.getOtherComponent could not be converted

com.ms.awt.FocusingTextField could not be converted

com.ms.awt.FontDescriptor could not be converted

com.ms.awt.FontMetricsX.bytesWidth could not be converted

com.ms.awt.FontMetricsX.CHAR_KERNING could not be converted

com.ms.awt.FontMetricsX.charsWidth could not be converted

com.ms.awt.FontMetricsX.FontMetricsX could not be converted

com.ms.awt.FontMetricsX.getAveCharWidth could not be converted

com.ms.awt.FontMetricsX.getFace could not be converted

com.ms.awt.FontMetricsX.getFontMetrics could not be converted

com.ms.awt.FontMetricsX.getLeading could not be converted

com.ms.awt.FontMetricsX.getMaxAdvance could not be converted

com.ms.awt.FontMetricsX.getWidths could not be converted

com.ms.awt.FontMetricsX.stringWidth could not be converted

com.ms.awt.FontX.chooseFont could not be converted

com.ms.awt.FontX.EMBEDDED could not be converted

com.ms.awt.FontX.getAttributeList could not be converted

com.ms.awt.FontX.getFlags could not be converted

com.ms.awt.FontX.getFlagsVal could not be converted
com.ms.awt.FontX.getFont could not be converted
com.ms.awt.FontX.getFontList could not be converted
com.ms.awt.FontX.getFontNativeData could not be converted
com.ms.awt.FontX.getNativeData could not be converted
com.ms.awt.FontX.getStyleVal could not be converted
com.ms.awt.FontX.isTypeable could not be converted
com.ms.awt.FontX.matchFace could not be converted
com.ms.awt.FontX.OUTLINE could not be converted
com.ms.awt.FontX.USEDFONT could not be converted
com.ms.awt.GenericEvent could not be converted
com.ms.awt.GraphicsX.bitBlt could not be converted
com.ms.awt.GraphicsX.cng could not be converted
com.ms.awt.GraphicsX.comp could not be converted
com.ms.awt.GraphicsX.copyArea could not be converted
com.ms.awt.GraphicsX.drawBezier could not be converted
com.ms.awt.GraphicsX.drawChars could not be converted
com.ms.awt.GraphicsX.drawCharsWithoutFxFont could not be converted
com.ms.awt.GraphicsX.drawOutlineChar could not be converted
com.ms.awt.GraphicsX.drawOutlinePolygon could not be converted
com.ms.awt.GraphicsX.drawPixels could not be converted
com.ms.awt.GraphicsX.drawRoundRect could not be converted
com.ms.awt.GraphicsX.drawScanLines could not be converted
com.ms.awt.GraphicsX.drawT2Curve could not be converted
com.ms.awt.GraphicsX.fill3DRect could not be converted
com.ms.awt.GraphicsX.fillRoundRect could not be converted
com.ms.awt.GraphicsX.gdc could not be converted
com.ms.awt.GraphicsX.getColorType could not be converted
com.ms.awt.GraphicsX.getGlyphOutline could not be converted
com.ms.awt.GraphicsX.go could not be converted
com.ms.awt.GraphicsX.GraphicsX could not be converted
com.ms.awt.GraphicsX.image could not be converted
com.ms.awt.GraphicsX.originX could not be converted
com.ms.awt.GraphicsX.originY could not be converted
com.ms.awt.GraphicsX.resetSurfaceParams could not be converted
com.ms.awt.GraphicsX.setClip could not be converted
com.ms.awt.GraphicsX.setPaintMode could not be converted
com.ms.awt.GraphicsX.setSurfaceOffset could not be converted
com.ms.awt.GraphicsX.setSurfaceOwner could not be converted
com.ms.awt.GraphicsX.setSurfaceVisRgn could not be converted

com.ms.awt.GraphicsX.setXORMode could not be converted

com.ms.awt.GraphicsXConstants.BDR_VALID could not be converted

com.ms.awt.HeavyComponent could not be converted

com.ms.awt.HorizBagLayout could not be converted

com.ms.awt.ImageX could not be converted

com.ms.awt.ListLayout could not be converted

com.ms.awt.MenuBarX.getItemID could not be converted

com.ms.awt.MenuBarX.MenuBarX(int) could not be converted

com.ms.awt.MenuBarX.MenuBarX(int, Applet, String) could not be converted

com.ms.awt.MenuBarX.MenuBarX(int, String) could not be converted

com.ms.awt.MenuItemX.addNotify could not be converted

com.ms.awt.MenuItemX.getID could not be converted

com.ms.awt.MenuX.CheckMenuItem could not be converted

com.ms.awt.MenuX.getItemID could not be converted

com.ms.awt.MenuXConstants could not be converted

com.ms.awt.OrientableFlowLayout could not be converted

com.ms.awt.PhysicalDrawingSurface could not be converted

com.ms.awt.PlatformFont could not be converted

com.ms.awt.VariableGridLayout could not be converted

com.ms.awt.VerticalBagLayout could not be converted

com.ms.awt.WClipboard.lostClipboard could not be converted

com.ms.awt.WClipboard.lostSelectionOwnership could not be converted

com.ms.awt.WClipboard.setToolkit could not be converted

com.ms.awt.WClipboard.WClipboard could not be converted

com.ms.awt.WComponentPeer could not be converted

com.ms.awt.WDragSession could not be converted

com.ms.awt.WEventQueue could not be converted

com.ms.awt.WFileDialogPeer could not be converted

com.ms.awt.WGuiCallback could not be converted

com.ms.awt.WHeavyPeer could not be converted

com.ms.awt.Win32SystemResourceDecoder could not be converted

com.ms.awt.WinEvent.notify could not be converted

com.ms.awt.WPrintGraphics could not be converted

com.ms.awt.WPrintJob could not be converted

com.ms.awt.WToolkit could not be converted

Com.ms.com Error Messages

com.ms.com.AnsiStringMarshaller could not be converted

com.ms.com.AnsiStringRef could not be converted

com.ms.com.COAUTHIDENTITY.cbByValSize could not be converted

com.ms.com.COAUTHIDENTITY.domain could not be converted

com.ms.com.COAUTHIDENTITY.flags could not be converted

com.ms.com.COAUTHIDENTITY.fromPtr could not be converted

com.ms.com.COAUTHIDENTITY.password could not be converted

com.ms.com.COAUTHIDENTITY.toExternal could not be converted

com.ms.com.COAUTHIDENTITY.toJava could not be converted

com.ms.com.COAUTHIDENTITY.toPtr could not be converted

com.ms.com.COAUTHIDENTITY.user could not be converted

com.ms.com.COAUTHINFO.pAuthIdentityData could not be converted

com.ms.com.COAUTHINFO.pwszServerPrincName could not be converted

com.ms.com.ComContext could not be converted

com.ms.com.ComException.ComException could not be converted

com.ms.com.ComException.getHelpContext could not be converted

com.ms.com.ComException.getHResult could not be converted

com.ms.com.ComException.hr could not be converted

com.ms.com.ComException.m_helpContext could not be converted

com.ms.com.ComException.m_helpFile could not be converted

com.ms.com.ComException.m_source could not be converted

com.ms.com.ComFailException.ComFailException could not be converted

com.ms.com.ComLib.ComLib could not be converted

com.ms.com.ComLib.declareMessagePumpThread could not be converted

com.ms.com.ComLib.executeOnContext could not be converted

com.ms.com.ComLib.freeUnusedLibraries could not be converted

com.ms.com.ComLib.IENVNextMarshalerC2J could not be converted

com.ms.com.ComLib.IENVNextMarshalerJ2C could not be converted

com.ms.com.ComLib.IID_IDispatch could not be converted

com.ms.com.ComLib.IID_IUnknown could not be converted

com.ms.com.ComLib.isEqualUnknown could not be converted

com.ms.com.ComLib.jcdwClassOffsetOf could not be converted

com.ms.com.ComLib.jcdwOffsetOf could not be converted

com.ms.com.ComLib.makeProxyRef could not be converted

com.ms.com.ComLib.ownsCleanup could not be converted

com.ms.com.ComLib.ptrToUnknown could not be converted

com.ms.com.ComLib.setDataWrapperSize could not be converted

com.ms.com.ComLib.startMTAThread could not be converted
com.ms.com.ComLib.supportsInterface could not be converted
com.ms.com.ComLib.threadStartMTA could not be converted
com.ms.com.ComLib.unknownToPtr could not be converted
com.ms.com.ComSuccessException.ComSuccessException could not be converted
com.ms.com.CONNECTDATA.pUnk could not be converted
com.ms.com.COSERVERINFO.pAuthInfo could not be converted
com.ms.com.COSERVERINFO.pwszName could not be converted
com.ms.com.CUnknown could not be converted
com.ms.com.CustomLib could not be converted
com.ms.com.Dispatch could not be converted
com.ms.com.DispatchProxy could not be converted
com.ms.com.Generic could not be converted
com.ms.com.Guid.Guid could not be converted
com.ms.com.Guid.setByStr could not be converted
com.ms.com.IAccessible.iid could not be converted
com.ms.com.IAccessibleDefault.iid could not be converted
com.ms.com.IBindCtx.iid could not be converted
com.ms.com.IBindCtx.RegisterObjectBound could not be converted
com.ms.com.IBindCtx.RevokeObjectBound could not be converted
com.ms.com.IBindCtx.RevokeObjectParam could not be converted
com.ms.com.IClassFactory could not be converted
com.ms.com.IClassFactory2 could not be converted
com.ms.com.IConnectionPoint.Advise could not be converted
com.ms.com.IConnectionPoint.iid could not be converted
com.ms.com.IConnectionPointContainer.iid could not be converted
com.ms.com.IEnumConnectionPoints.iid could not be converted
com.ms.com.IEnumConnectionPoints.Next could not be converted
com.ms.com.IEnumConnections.iid could not be converted
com.ms.com.IEnumConnections.Next could not be converted
com.ms.com.IEnumConnections.Skip could not be converted
com.ms.com.IEnumMoniker.iid could not be converted
com.ms.com.IEnumMoniker.Next could not be converted
com.ms.com.IEnumSTATSTG could not be converted
com.ms.com.IEnumString.iid could not be converted
com.ms.com.IEnumString.Next could not be converted
com.ms.com.IEnumUnknown could not be converted
com.ms.com.IEnumVariant.Clone could not be converted
com.ms.com.IExternalConnectionSink could not be converted
com.ms.com.IIDIsMarshaler could not be converted

com.ms.com.ILicenseMgr could not be converted

com.ms.com.ILockBytes could not be converted

com.ms.com.IMarshal.GetMarshalSizeMax could not be converted

com.ms.com.IMarshal.GetUnmarshalClass could not be converted

com.ms.com.IMarshal.iid could not be converted

com.ms.com.IMoniker.BindToObject could not be converted

com.ms.com.IMoniker.iid could not be converted

com.ms.com.IMoniker.IsDirty could not be converted

com.ms.com.IMoniker.IsEqual could not be converted

com.ms.com.IMoniker.IsRunning could not be converted

com.ms.com.IParseDisplayName could not be converted

com.ms.com.IPersist.iid could not be converted

com.ms.com.IPersistFile.iid could not be converted

com.ms.com.IPersistFile.IsDirty could not be converted

com.ms.com.IPersistFile.Save could not be converted

com.ms.com.IPersistStorage could not be converted

com.ms.com.IPersistStream.GetClassID could not be converted

com.ms.com.IPersistStream.iid could not be converted

com.ms.com.IPersistStreamInit could not be converted

com.ms.com.IPropertyNotifySink could not be converted

com.ms.com.IROTDData could not be converted

com.ms.com.IRunningObjectTable.GetObject could not be converted

com.ms.com.IRunningObjectTable.GetTimeOfLastChange could not be converted

com.ms.com.IRunningObjectTable.iid could not be converted

com.ms.com.IRunningObjectTable.IsRunning could not be converted

com.ms.com.IRunningObjectTable.NoteChangeTime could not be converted

com.ms.com.IRunningObjectTable.Register could not be converted

com.ms.com.ISequentialStream could not be converted

com.ms.com.ISequentialStream.iid could not be converted

com.ms.com.IServiceProvider could not be converted

com.ms.com.IStorage could not be converted

com.ms.com.IStream.CopyTo could not be converted

com.ms.com.IStream.iid could not be converted

com.ms.com.IStream.LOCK_EXCLUSIVE could not be converted

com.ms.com.IStream.LOCK_ONLYONCE could not be converted

com.ms.com.IStream.LOCK_WRITE could not be converted

com.ms.com.IStream.Read could not be converted

com.ms.com.IStream.Seek could not be converted

com.ms.com.IStream.STATFLAG_DEFAULT could not be converted

com.ms.com.IStream.STATFLAG_NONAME could not be converted

com.ms.com.IStream.STATFLAG_NOOPEN could not be converted

com.ms.com.IStream.STGC_DANGEROUSLYCOMMITMERELYTODISKCACHE could not be converted

com.ms.com.IStream.STGC_DEFAULT could not be converted

com.ms.com.IStream.STGC_ONLYIFCURRENT could not be converted

com.ms.com.IStream.STGC_OVERWRITE could not be converted

com.ms.com.IStream.STREAM_SEEK_CUR could not be converted

com.ms.com.IStream.STREAM_SEEK_END could not be converted

com.ms.com.IStream.STREAM_SEEK_SET could not be converted

com.ms.com.IStream.Write could not be converted

com.ms.com.LicenseMgr could not be converted

com.ms.com.LICINFO could not be converted

com.ms.com.MULTI_QI could not be converted

com.ms.com.NoAutoMarshaling could not be converted

com.ms.com.NoAutoScripting could not be converted

com.ms.com.SafeArray.destroy could not be converted

com.ms.com.SafeArray.getFeatures could not be converted

com.ms.com.SafeArray.getNumLocks could not be converted

com.ms.com.SafeArray.getPhysicalSafeArray could not be converted

com.ms.com.SafeArray.getvt could not be converted

com.ms.com.SafeArray.reinit could not be converted

com.ms.com.SafeArray.reinterpretType could not be converted

com.ms.com.SizelsMarshaller could not be converted

com.ms.com.STATSTG.clsid_data1 could not be converted

com.ms.com.STATSTG.clsid_data2 could not be converted

com.ms.com.STATSTG.clsid_data3 could not be converted

com.ms.com.STATSTG.STGTY_LOCKBYTES could not be converted

com.ms.com.STATSTG.STGTY_PROPERTY could not be converted

com.ms.com.STATSTG.STGTY_STORAGE could not be converted

com.ms.com.STATSTG.STGTY_STREAM could not be converted

com.ms.com.StdCOMClassObject could not be converted

com.ms.com.UniStringMarshaller could not be converted

com.ms.com.UniStringRef could not be converted

com.ms.com.Variant.changeType could not be converted

com.ms.com.Variant.clone could not be converted

com.ms.com.Variant.cloneIndirect could not be converted

com.ms.com.Variant.getDate could not be converted

com.ms.com.Variant.getEmpty could not be converted

com.ms.com.Variant.getErrorRef could not be converted

com.ms.com.Variant.getNull could not be converted

com.ms.com.Variant.getVariantArray could not be converted

com.ms.com.Variant.getVariantArrayRef could not be converted
com.ms.com.Variant.getvt could not be converted
com.ms.com.Variant.noParam could not be converted
com.ms.com.Variant.putError could not be converted
com.ms.com.Variant.putSafeArrayRefHelper could not be converted
com.ms.com.Variant.toError could not be converted
com.ms.com.Variant.toScriptObject could not be converted
com.ms.com.Variant.toVariantArray could not be converted
com.ms.com.Variant.Variant could not be converted
com.ms.com.Variant.VariantByref could not be converted
com.ms.com.Variant.VariantClear could not be converted
com.ms.com.Variant.VariantCurrency could not be converted
com.ms.com.Variant.VariantEmpty could not be converted
com.ms.com.Variant.VariantError could not be converted
com.ms.com.Variant.VariantNull could not be converted
com.ms.com.Variant.VariantTypeMask could not be converted
com.ms.com.Variant.VariantVariant could not be converted

Com.ms.directx Error Messages

com.ms.directX.D3dFindDeviceResult.GetGuid could not be converted

com.ms.directX.D3dFindDeviceSearch.getGuid could not be converted

com.ms.directX.D3dFindDeviceSearch.setGuid could not be converted

com.ms.directX.Direct3dRMFace.getVertices could not be converted

com.ms.directX.Direct3dRMMesh.getVertices could not be converted

com.ms.directX.Direct3dRMMesh.setVertices could not be converted

com.ms.directX.Direct3dRMMeshBuilder.addFaces

com.ms.directX.Direct3dRMMeshBuilder.getVertices

com.ms.directX.DirectDraw.createPalette could not be converted

com.ms.directX.DirectDraw.setCooperativeLevel could not be converted

com.ms.directX.DirectDrawClipper.resetSurfaceParams could not be converted

com.ms.directX.DirectDrawClipper.setComponent could not be converted

com.ms.directX.DirectDrawClipper.setSurfaceOffset could not be converted

com.ms.directX.DirectDrawClipper.setSurfaceOwner could not be converted

com.ms.directX.DirectDrawClipper.setSurfaceVisRgn could not be converted

com.ms.directX.DirectDrawPalette.getColorEntries could not be converted

com.ms.directX.DirectDrawPalette.getPaletteEntries could not be converted

com.ms.directX.DirectDrawPalette.setEntries could not be converted

com.ms.directX.DirectSound.createSoundBuffer could not be converted

com.ms.directX.DirectSound.setCooperativeLevel could not be converted

com.ms.directX.DirectSoundBuffer.getFormat could not be converted

com.ms.directX.DirectSoundBuffer.initialize could not be converted

com.ms.directX.DirectSoundBuffer.setFormat could not be converted

com.ms.directX.DirectSoundResource.loadWaveFile could not be converted

com.ms.directX.DirectSoundResource.loadWaveResource could not be converted

com.ms.directX.DirectXConstants could not be converted

com.ms.directX.PaletteEntry could not be converted

com.ms.directX.WaveFormatEx could not be converted

Com.ms.dll Error Messages

`com.ms.dll.Callback` could not be converted

`com.ms.dll.DllLib.addrOf` could not be converted

`com.ms.dll.DllLib.addrOfPinnedObject` could not be converted

`com.ms.dll.DllLib.copy` could not be converted

`com.ms.dll.DllLib.DllLib` could not be converted

`com.ms.dll.DllLib.freePinnedHandle` could not be converted

`com.ms.dll.DllLib.getPinnedHandle` could not be converted

`com.ms.dll.DllLib.getPinnedObject` could not be converted

`com.ms.dll.DllLib.isStruct` could not be converted

`com.ms.dll.DllLib.propagateStructFields` could not be converted

`com.ms.dll.DllLib.resize` could not be converted

`com.ms.dll.ParameterCountMismatchError` could not be converted

`com.ms.dll.StringMarshaler` could not be converted

Com.ms.dxmedia Error Messages

com.ms.dxmedia.AppTriggeredEvent could not be converted

com.ms.dxmedia.ArrayBvr.ArrayBvr could not be converted

com.ms.dxmedia.ArrayBvr.getCOMPtr could not be converted

com.ms.dxmedia.ArrayBvr.newUninitBvr could not be converted

com.ms.dxmedia.ArrayBvr.setCOMBvr could not be converted

com.ms.dxmedia.Bbox2Bvr.Bbox2Bvr could not be converted

com.ms.dxmedia.Bbox2Bvr.getCOMPtr could not be converted

com.ms.dxmedia.Bbox2Bvr.getMax could not be converted

com.ms.dxmedia.Bbox2Bvr.getMin could not be converted

com.ms.dxmedia.Bbox2Bvr.newUninitBehavior could not be converted

com.ms.dxmedia.Bbox2Bvr.newUninitBvr could not be converted

com.ms.dxmedia.Bbox2Bvr.setCOMBvr could not be converted

com.ms.dxmedia.Bbox3Bvr.Bbox3Bvr could not be converted

com.ms.dxmedia.Bbox3Bvr.getCOMPtr could not be converted

com.ms.dxmedia.Bbox3Bvr.getMax could not be converted

com.ms.dxmedia.Bbox3Bvr.getMin could not be converted

com.ms.dxmedia.Bbox3Bvr.newUninitBehavior could not be converted

com.ms.dxmedia.Bbox3Bvr.newUninitBvr could not be converted

com.ms.dxmedia.Bbox3Bvr.setCOMBvr could not be converted

com.ms.dxmedia.Behavior.Behavior could not be converted

com.ms.dxmedia.Behavior.debug could not be converted

com.ms.dxmedia.Behavior.extract could not be converted

com.ms.dxmedia.Behavior.getCOMBvr could not be converted

com.ms.dxmedia.Behavior.newUninitBehavior could not be converted

com.ms.dxmedia.BooleanBvr.BooleanBvr could not be converted

com.ms.dxmedia.BooleanBvr.getCOMPtr could not be converted

com.ms.dxmedia.BooleanBvr.newUninitBehavior could not be converted

com.ms.dxmedia.BooleanBvr.setCOMBvr could not be converted

com.ms.dxmedia.CallbackNotifier could not be converted

com.ms.dxmedia.CameraBvr.CameraBvr could not be converted

com.ms.dxmedia.CameraBvr.getCOMPtr could not be converted

com.ms.dxmedia.CameraBvr.newUninitBehavior could not be converted

com.ms.dxmedia.CameraBvr.newUninitBvr could not be converted

com.ms.dxmedia.CameraBvr.setCOMBvr could not be converted

com.ms.dxmedia.ColorBvr.ColorBvr could not be converted

com.ms.dxmedia.ColorBvr.getCOMPtr could not be converted

com.ms.dxmedia.ColorBvr.newUninitBehavior could not be converted

com.ms.dxmedia.ColorBvr.NewUninitBvr could not be converted

com.ms.dxmedia.ColorBvr.setCOMBvr could not be converted

com.ms.dxmedia.Cycler could not be converted

com.ms.dxmedia.DashStyleBvr.DashStyleBvr could not be converted

com.ms.dxmedia.DashStyleBvr.getCOMPtr could not be converted

com.ms.dxmedia.DashStyleBvr.newUninitBehavior could not be converted

com.ms.dxmedia.DashStyleBvr.newUninitBvr could not be converted

com.ms.dxmedia.DashStyleBvr.setCOMBvr could not be converted

com.ms.dxmedia.DefaultErrReceiver could not be converted

com.ms.dxmedia.DXMApplet could not be converted

com.ms.dxmedia.DXMCanvas could not be converted

com.ms.dxmedia.DXMCanvasBase could not be converted

com.ms.dxmedia.DXMDebugCallback could not be converted

com.ms.dxmedia.DXMEvent.DXMEvent could not be converted

com.ms.dxmedia.DXMEvent.getCOMPtr could not be converted

com.ms.dxmedia.DXMEvent.newUninitBehavior could not be converted

com.ms.dxmedia.DXMEvent.registerCallback could not be converted

com.ms.dxmedia.DXMEvent.setCOMBvr could not be converted

com.ms.dxmedia.DXMException.DXMException could not be converted

com.ms.dxmedia.EndStyleBvr.EndStyleBvr could not be converted

com.ms.dxmedia.EndStyleBvr.getCOMPtr could not be converted

com.ms.dxmedia.EndStyleBvr.newUninitBehavior could not be converted

com.ms.dxmedia.EndStyleBvr.newUninitBvr could not be converted

com.ms.dxmedia.EndStyleBvr.setCOMBvr could not be converted

com.ms.dxmedia.ErrorAndWarningReceiver could not be converted

com.ms.dxmedia.EventCallbackObject could not be converted

com.ms.dxmedia.FontStyleBvr.FontStyleBvr could not be converted

com.ms.dxmedia.FontStyleBvr.getCOMPtr could not be converted

com.ms.dxmedia.FontStyleBvr.newUninitBehavior could not be converted

com.ms.dxmedia.FontStyleBvr.NewUninitBvr could not be converted

com.ms.dxmedia.FontStyleBvr.setCOMBvr could not be converted

com.ms.dxmedia.GeometryBvr.GeometryBvr could not be converted

com.ms.dxmedia.GeometryBvr.getCOMPtr could not be converted

com.ms.dxmedia.GeometryBvr.newUninitBehavior could not be converted

com.ms.dxmedia.GeometryBvr.setCOMBvr could not be converted

com.ms.dxmedia.ImageBvr.getCOMPtr could not be converted

com.ms.dxmedia.ImageBvr.ImageBvr could not be converted

com.ms.dxmedia.ImageBvr.newUninitBehavior could not be converted

com.ms.dxmedia.ImageBvr.setCOMBvr could not be converted

com.ms.dxmedia.JoinStyleBvr.getCOMPtr could not be converted

com.ms.dxmedia.JoinStyleBvr.JoinStyleBvr could not be converted

com.ms.dxmedia.JoinStyleBvr.newUninitBehavior could not be converted

com.ms.dxmedia.JoinStyleBvr.setCOMBvr could not be converted

com.ms.dxmedia.LineStyleBvr.getCOMPtr could not be converted

com.ms.dxmedia.LineStyleBvr.LineStyleBvr could not be converted

com.ms.dxmedia.LineStyleBvr.newUninitBehavior could not be converted

com.ms.dxmedia.LineStyleBvr.setCOMBvr could not be converted

com.ms.dxmedia.MatteBvr.getCOMPtr could not be converted

com.ms.dxmedia.MatteBvr.MatteBvr could not be converted

com.ms.dxmedia.MatteBvr.newUninitBehavior could not be converted

com.ms.dxmedia.MatteBvr.setCOMBvr could not be converted

com.ms.dxmedia.MicrophoneBvr.getCOMPtr could not be converted

com.ms.dxmedia.MicrophoneBvr.MicrophoneBvr could not be converted

com.ms.dxmedia.MicrophoneBvr.newUninitBehavior could not be converted

com.ms.dxmedia.MicrophoneBvr.setCOMBvr could not be converted

com.ms.dxmedia.Model.cleanup could not be converted

com.ms.dxmedia.Model.createModel could not be converted

com.ms.dxmedia.Model.getImportBase could not be converted

com.ms.dxmedia.Model.modifyPreferences could not be converted

com.ms.dxmedia.Model.receiveInputImages could not be converted

com.ms.dxmedia.Model.setImportBase could not be converted

com.ms.dxmedia.ModelMakerApplet could not be converted

com.ms.dxmedia.ModifiableBehavior could not be converted

com.ms.dxmedia.MontageBvr.getCOMPtr could not be converted

com.ms.dxmedia.MontageBvr.MontageBvr could not be converted

com.ms.dxmedia.MontageBvr.newUninitBehavior could not be converted

com.ms.dxmedia.MontageBvr.setCOMBvr could not be converted

com.ms.dxmedia.NumberBvr.getCOMPtr could not be converted

com.ms.dxmedia.NumberBvr.newUninitBehavior could not be converted

com.ms.dxmedia.NumberBvr.NumberBvr could not be converted

com.ms.dxmedia.NumberBvr.setCOMBvr could not be converted

com.ms.dxmedia.PairObject.PairObject could not be converted

com.ms.dxmedia.Path2Bvr.getCOMPtr could not be converted

com.ms.dxmedia.Path2Bvr.newUninitBehavior could not be converted

com.ms.dxmedia.Path2Bvr.Path2Bvr could not be converted

com.ms.dxmedia.Path2Bvr.setCOMBvr could not be converted

com.ms.dxmedia.PickableGeometry.PickableGeometry could not be converted

com.ms.dxmedia.PickableImage.PickableImage could not be converted

com.ms.dxmedia.Point2Bvr.getCOMPtr could not be converted

com.ms.dxmedia.Point2Bvr.newUninitBehavior could not be converted

com.ms.dxmedia.Point2Bvr.Point2Bvr could not be converted
com.ms.dxmedia.Point2Bvr.setCOMBvr could not be converted
com.ms.dxmedia.Point3Bvr.getCOMPtr could not be converted
com.ms.dxmedia.Point3Bvr.newUninitBehavior could not be converted
com.ms.dxmedia.Point3Bvr.Point3Bvr could not be converted
com.ms.dxmedia.Point3Bvr.setCOMBvr could not be converted
com.ms.dxmedia.Preferences.COLOR_KEY_BLUE could not be converted
com.ms.dxmedia.Preferences.COLOR_KEY_GREEN could not be converted
com.ms.dxmedia.Preferences.COLOR_KEY_RED could not be converted
com.ms.dxmedia.Preferences.DITHERING could not be converted
com.ms.dxmedia.Preferences.ENGINE_OPTIMIZATIONS could not be converted
com.ms.dxmedia.Preferences.FILL_MODE could not be converted
com.ms.dxmedia.Preferences.FILL_MODE_POINT could not be converted
com.ms.dxmedia.Preferences.FILL_MODE_SOLID could not be converted
com.ms.dxmedia.Preferences.FILL_MODE_WIREFRAME could not be converted
com.ms.dxmedia.Preferences.MAX_FRAMES_PER_SEC could not be converted
com.ms.dxmedia.Preferences.OVERRIDE_APPLICATION_PREFERENCES could not be converted
com.ms.dxmedia.Preferences.PERSPECTIVE_CORRECT could not be converted
com.ms.dxmedia.Preferences.RGB_LIGHTING_MODE could not be converted
com.ms.dxmedia.Preferences.SHADE_MODE could not be converted
com.ms.dxmedia.Preferences.SHADE_MODE_FLAT could not be converted
com.ms.dxmedia.Preferences.SHADE_MODE_GOURAUD could not be converted
com.ms.dxmedia.Preferences.SHADE_MODE_PHONG could not be converted
com.ms.dxmedia.Preferences.TEXTURE_QUALITY could not be converted
com.ms.dxmedia.Preferences.TEXTURE_QUALITY_LINEAR could not be converted
com.ms.dxmedia.Preferences.TEXTURE_QUALITY_NEAREST could not be converted
com.ms.dxmedia.Preferences.USE_3D_HW could not be converted
com.ms.dxmedia.Preferences.USE_VIDEOMEM could not be converted
com.ms.dxmedia.PropertyDispatcher could not be converted
com.ms.dxmedia.SoundBvr.getCOMPtr could not be converted
com.ms.dxmedia.SoundBvr.newUninitBehavior could not be converted
com.ms.dxmedia.SoundBvr.setCOMBvr could not be converted
com.ms.dxmedia.SoundBvr.SoundBvr could not be converted
com.ms.dxmedia.Statics.makeBvrFromInterface could not be converted
com.ms.dxmedia.StaticsBase._site could not be converted
com.ms.dxmedia.StaticsBase.BvrHook could not be converted
com.ms.dxmedia.StaticsBase.checkRead could not be converted
com.ms.dxmedia.StaticsBase.cm could not be converted
com.ms.dxmedia.StaticsBase.foot could not be converted
com.ms.dxmedia.StaticsBase.getCOMPtr could not be converted

com.ms.dxmedia.StaticsBase.handleError could not be converted
com.ms.dxmedia.StaticsBase.importGeometry could not be converted
com.ms.dxmedia.StaticsBase.importImage could not be converted
com.ms.dxmedia.StaticsBase.importMovie (asynchronous) could not be converted
com.ms.dxmedia.StaticsBase.importMovie (synchronous) could not be converted
com.ms.dxmedia.StaticsBase.importSound (asynchronous) could not be converted
com.ms.dxmedia.StaticsBase.importSound (synchronous) could not be converted
com.ms.dxmedia.StaticsBase.inch could not be converted
com.ms.dxmedia.StaticsBase.meter could not be converted
com.ms.dxmedia.StaticsBase.mm could not be converted
com.ms.dxmedia.StaticsBase.registerErrorAndWarningReceiver could not be converted
com.ms.dxmedia.StaticsBase.unregisterCallback could not be converted
com.ms.dxmedia.StringBvr.getCOMPtr could not be converted
com.ms.dxmedia.StringBvr.newUninitBehavior could not be converted
com.ms.dxmedia.StringBvr.setCOMBvr could not be converted
com.ms.dxmedia.StringBvr.StringBvr could not be converted
com.ms.dxmedia.Transform2Bvr.getCOMPtr could not be converted
com.ms.dxmedia.Transform2Bvr.newUninitBehavior could not be converted
com.ms.dxmedia.Transform2Bvr.setCOMBvr could not be converted
com.ms.dxmedia.Transform2Bvr.Transform2Bvr could not be converted
com.ms.dxmedia.Transform3Bvr.getCOMPtr could not be converted
com.ms.dxmedia.Transform3Bvr.newUninitBehavior could not be converted
com.ms.dxmedia.Transform3Bvr.setCOMBvr could not be converted
com.ms.dxmedia.Transform3Bvr.Transform3Bvr could not be converted
com.ms.dxmedia.TupleBvr.getCOMPtr could not be converted
com.ms.dxmedia.TupleBvr.setCOMBvr could not be converted
com.ms.dxmedia.TupleBvr.TupleBvr could not be converted
com.ms.dxmedia.UntilNotifierCB could not be converted
com.ms.dxmedia.Vector2Bvr.getCOMPtr could not be converted
com.ms.dxmedia.Vector2Bvr.newUninitBehavior could not be converted
com.ms.dxmedia.Vector2Bvr.setCOMBvr could not be converted
com.ms.dxmedia.Vector2Bvr.Vector2Bvr could not be converted
com.ms.dxmedia.Vector3Bvr.getCOMPtr could not be converted
com.ms.dxmedia.Vector3Bvr.newUninitBehavior could not be converted
com.ms.dxmedia.Vector3Bvr.setCOMBvr could not be converted
com.ms.dxmedia.Vector3Bvr.Vector3Bvr could not be converted
com.ms.dxmedia.Viewer.getCurrentTickTime could not be converted
com.ms.dxmedia.Viewer.registerErrorAndWarningReceiver could not be converted
com.ms.dxmedia.Viewer.startModel could not be converted
com.ms.dxmedia.Viewer.tick could not be converted

Com.ms.fx Error Messages

com.ms.fx.BaseColor.BaseColor could not be converted

com.ms.fx.fullTxtRun could not be converted

com.ms.fx.FxBrushPen.drawBytesCallback could not be converted

com.ms.fx.FxBrushPen.drawCharsCallback could not be converted

com.ms.fx.FxBrushPen.drawRoundRectCallback could not be converted

com.ms.fx.FxBrushPen.drawScanLinesCallback could not be converted

com.ms.fx.FxBrushPen.drawStringCallback could not be converted

com.ms.fx.FxBrushPen.fill3DRectCallback could not be converted

com.ms.fx.FxBrushPen.fillRoundRectCallback could not be converted

com.ms.fx.FxCaret could not be converted

com.ms.fx.FxColor.brightenColor could not be converted

com.ms.fx.FxColor.darkenColor could not be converted

com.ms.fx.FxColor.FxColor could not be converted

com.ms.fx.FxColor.getHilight could not be converted

com.ms.fx.FxColor.getShadow could not be converted

com.ms.fx.FxComponentImage could not be converted

com.ms.fx.FxComponentTexture.FxComponentTexture could not be converted

com.ms.fx.FxCurve could not be converted

com.ms.fx.FxEllipse could not be converted

com.ms.fx.FxFont.ANTIALIAS could not be converted

com.ms.fx.FxFont.drawEffects could not be converted

com.ms.fx.FxFont.FONTXFONT could not be converted

com.ms.fx.FxFont.FxFont could not be converted

com.ms.fx.FxFont.getAttributeList could not be converted

com.ms.fx.FxFont.getEmboldenedFont could not be converted

com.ms.fx.FxFont.getFlags could not be converted

com.ms.fx.FxFont.getFlagsVal could not be converted

com.ms.fx.FxFont.getFont could not be converted

com.ms.fx.FxFont.getFontList could not be converted

com.ms.fx.FxFont.getStyleVal could not be converted

com.ms.fx.FxFont.matchFace could not be converted

com.ms.fx.FxFont.STRIKEOUT could not be converted

com.ms.fx.FxFont.UNDERLINE could not be converted

com.ms.fx.FxFont.USEDFONT could not be converted

com.ms.fx.FxFontMetrics could not be converted

com.ms.fx.FxFontMetricsOther could not be converted

com.ms.fx.FxFormattedText could not be converted

com.ms.fx.FxGraphicMetaFile.addObjectToTable could not be converted

com.ms.fx.FxGraphicMetaFile.debugging could not be converted

com.ms.fx.FxGraphicMetaFile.enumerate could not be converted

com.ms.fx.FxGraphicMetaFile.FxGraphicMetaFile could not be converted

com.ms.fx.FxGraphicMetaFile.hdc could not be converted

com.ms.fx.FxGraphicMetaFile.hdcStates could not be converted

com.ms.fx.FxGraphicMetaFile.play could not be converted

com.ms.fx.FxGraphicMetaFile.records could not be converted

com.ms.fx.FxGraphicMetaFile.removeObjectFromTable could not be converted

com.ms.fx.FxGraphics.drawBezier could not be converted

com.ms.fx.FxGraphics.drawBorder could not be converted

com.ms.fx.FxGraphics.drawChars could not be converted

com.ms.fx.FxGraphics.drawCharsWithoutFxFont could not be converted

com.ms.fx.FxGraphics.drawOutlineChar could not be converted

com.ms.fx.FxGraphics.drawOutlinePolygon could not be converted

com.ms.fx.FxGraphics.drawPixels could not be converted

com.ms.fx.FxGraphics.drawScanLines could not be converted

com.ms.fx.FxGraphics.drawString could not be converted

com.ms.fx.FxGraphics.drawString(int, int) could not be converted

com.ms.fx.FxGraphics.drawStringFormatted could not be converted

com.ms.fx.FxGraphics.drawStringWithoutFxFont could not be converted

com.ms.fx.FxGraphics.drawT2Curve could not be converted

com.ms.fx.FxGraphics.fill3DRect could not be converted

com.ms.fx.FxGraphics.getClip could not be converted

com.ms.fx.FxGraphics.getExtendedGraphics could not be converted

com.ms.fx.FxGraphics.getGlyphOutline could not be converted

com.ms.fx.FxGraphics.intelliFont could not be converted

com.ms.fx.FxGraphics.setClip could not be converted

com.ms.fx.FxGraphicsOVM.baseGraphics could not be converted

com.ms.fx.FxGraphicsOVM.clearRect could not be converted

com.ms.fx.FxGraphicsOVM.clipRect could not be converted

com.ms.fx.FxGraphicsOVM.copyArea could not be converted

com.ms.fx.FxGraphicsOVM.create could not be converted

com.ms.fx.FxGraphicsOVM.dispose could not be converted

com.ms.fx.FxGraphicsOVM.drawArc could not be converted

com.ms.fx.FxGraphicsOVM.drawBezier could not be converted

com.ms.fx.FxGraphicsOVM.drawBorder could not be converted

com.ms.fx.FxGraphicsOVM.drawBytes could not be converted

com.ms.fx.FxGraphicsOVM.drawChars could not be converted

com.ms.fx.FxGraphicsOVM.drawChars(char[], int, int, int, int) could not be converted

com.ms.fx.FxGraphicsOVM.drawChars(char[], int, int, int, int, Rectangle, int, int[], int[]) could not be converted

com.ms.fx.FxGraphicsOVM.drawCharsWithoutFxFont could not be converted

com.ms.fx.FxGraphicsOVM.drawLine could not be converted

com.ms.fx.FxGraphicsOVM.drawOutlinePolygon could not be converted

com.ms.fx.FxGraphicsOVM.drawOval could not be converted

com.ms.fx.FxGraphicsOVM.drawPixels could not be converted

com.ms.fx.FxGraphicsOVM.drawPolygon could not be converted

com.ms.fx.FxGraphicsOVM.drawPolyline could not be converted

com.ms.fx.FxGraphicsOVM.drawRect could not be converted

com.ms.fx.FxGraphicsOVM.drawRoundRect could not be converted

com.ms.fx.FxGraphicsOVM.drawScanLines could not be converted

com.ms.fx.FxGraphicsOVM.drawString could not be converted

com.ms.fx.FxGraphicsOVM.drawStringWithoutFxFont could not be converted

com.ms.fx.FxGraphicsOVM.excludeClip could not be converted

com.ms.fx.FxGraphicsOVM.fillArc could not be converted

com.ms.fx.FxGraphicsOVM.fillOval could not be converted

com.ms.fx.FxGraphicsOVM.fillPolygon could not be converted

com.ms.fx.FxGraphicsOVM.fillRect could not be converted

com.ms.fx.FxGraphicsOVM.fillRoundRect could not be converted

com.ms.fx.FxGraphicsOVM.FxGraphicsOVM could not be converted

com.ms.fx.FxGraphicsOVM.getBaseGraphics could not be converted

com.ms.fx.FxGraphicsOVM.getClipBounds could not be converted

com.ms.fx.FxGraphicsOVM.getClipRect could not be converted

com.ms.fx.FxGraphicsOVM.getClipRegion could not be converted

com.ms.fx.FxGraphicsOVM.getColor could not be converted

com.ms.fx.FxGraphicsOVM.getFont could not be converted

com.ms.fx.FxGraphicsOVM.getFontMetrics could not be converted

com.ms.fx.FxGraphicsOVM.getGlyphOutline could not be converted

com.ms.fx.FxGraphicsOVM.getTranslation could not be converted

com.ms.fx.FxGraphicsOVM.hardClipRect could not be converted

com.ms.fx.FxGraphicsOVM.intersectClip could not be converted

com.ms.fx.FxGraphicsOVM.nativeSetFont could not be converted

com.ms.fx.FxGraphicsOVM.originX could not be converted

com.ms.fx.FxGraphicsOVM.originY could not be converted

com.ms.fx.FxGraphicsOVM.runningAFC could not be converted

com.ms.fx.FxGraphicsOVM.setClip could not be converted

com.ms.fx.FxGraphicsOVM.setClip(int, int, int) could not be converted

com.ms.fx.FxGraphicsOVM.setClip(Region) could not be converted

com.ms.fx.FxGraphicsOVM.setClip(Shape) could not be converted

com.ms.fx.FxGraphicsOVM.setColor could not be converted

com.ms.fx.FxGraphicsOVM.setFont could not be converted

com.ms.fx.FxGraphicsOVM.setPaintMode could not be converted

com.ms.fx.FxGraphicsOVM.setXORMode could not be converted

com.ms.fx.FxGraphicsOVM.systemInterface could not be converted

com.ms.fx.FxGraphicsOVM11 could not be converted

com.ms.fx.FxGraphicsOVM11.clearRect could not be converted

com.ms.fx.FxGraphicsOVM11.clipRect could not be converted

com.ms.fx.FxGraphicsOVM11.copyArea could not be converted

com.ms.fx.FxGraphicsOVM11.dispose could not be converted

com.ms.fx.FxGraphicsOVM11.drawArc could not be converted

com.ms.fx.FxGraphicsOVM11.drawBezier could not be converted

com.ms.fx.FxGraphicsOVM11.drawBorder could not be converted

com.ms.fx.FxGraphicsOVM11.drawBytes could not be converted

com.ms.fx.FxGraphicsOVM11.drawChars could not be converted

com.ms.fx.FxGraphicsOVM11.drawCharsWithoutFxFont could not be converted

com.ms.fx.FxGraphicsOVM11.drawLine could not be converted

com.ms.fx.FxGraphicsOVM11.drawOutlinePolygon could not be converted

com.ms.fx.FxGraphicsOVM11.drawOval could not be converted

com.ms.fx.FxGraphicsOVM11.drawPixels could not be converted

com.ms.fx.FxGraphicsOVM11.drawPolygon could not be converted

com.ms.fx.FxGraphicsOVM11.drawPolyline could not be converted

com.ms.fx.FxGraphicsOVM11.drawRect could not be converted

com.ms.fx.FxGraphicsOVM11.drawRoundRect could not be converted

com.ms.fx.FxGraphicsOVM11.drawScanLines could not be converted

com.ms.fx.FxGraphicsOVM11.drawString could not be converted

com.ms.fx.FxGraphicsOVM11.drawStringWithoutFxFont could not be converted

com.ms.fx.FxGraphicsOVM11.excludeClip could not be converted

com.ms.fx.FxGraphicsOVM11.fillArc could not be converted

com.ms.fx.FxGraphicsOVM11.fillOval could not be converted

com.ms.fx.FxGraphicsOVM11.fillPolygon could not be converted

com.ms.fx.FxGraphicsOVM11.fillRect could not be converted

com.ms.fx.FxGraphicsOVM11.fillRoundRect could not be converted

com.ms.fx.FxGraphicsOVM11.getBaseGraphics could not be converted

com.ms.fx.FxGraphicsOVM11.getClipBounds could not be converted

com.ms.fx.FxGraphicsOVM11.getClipRect could not be converted

com.ms.fx.FxGraphicsOVM11.getClipRegion could not be converted

com.ms.fx.FxGraphicsOVM11.getColor could not be converted

com.ms.fx.FxGraphicsOVM11.getFont could not be converted

com.ms.fx.FxGraphicsOVM11.getFontMetrics could not be converted

com.ms.fx.FxGraphicsOVM11.getGlyphOutline could not be converted

com.ms.fx.FxGraphicsOVM11.getTranslation could not be converted
com.ms.fx.FxGraphicsOVM11.intersectClip could not be converted
com.ms.fx.FxGraphicsOVM11.nativeSetFont could not be converted
com.ms.fx.FxGraphicsOVM11.originX could not be converted
com.ms.fx.FxGraphicsOVM11.originY could not be converted
com.ms.fx.FxGraphicsOVM11.runningAFC could not be converted
com.ms.fx.FxGraphicsOVM11.setClip could not be converted
com.ms.fx.FxGraphicsOVM11.setColor could not be converted
com.ms.fx.FxGraphicsOVM11.setFont could not be converted
com.ms.fx.FxGraphicsOVM11.setPaintMode could not be converted
com.ms.fx.FxGraphicsOVM11.setXORMode could not be converted
com.ms.fx.FxGraphicsOVM11.systemInterface could not be converted
com.ms.fx.FxOutlineFont could not be converted
com.ms.fx.FxPen.calcLineValues could not be converted
com.ms.fx.FxPen.drawRoundRectCallback could not be converted
com.ms.fx.FxPen.drawScanLinesCallback could not be converted
com.ms.fx.FxPen.fillRectCallback could not be converted
com.ms.fx.FxPen.FxPen could not be converted
com.ms.fx.FxPen.myDrawOval could not be converted
com.ms.fx.FxRubberPen could not be converted
com.ms.fx.FxStateConfigurableImage could not be converted
com.ms.fx.FxStateConfigurableUIImage.FxStateConfigurableUIImage could not be converted
com.ms.fx.FxStateConfigurableUIImage.getImageState could not be converted
com.ms.fx.FxStyledPen could not be converted
com.ms.fx.FxStyledPen.FxStyledPen could not be converted
com.ms.fx.FxSystemFont could not be converted
com.ms.fx.FxSystemIcon could not be converted
com.ms.fx.FxText.buffer could not be converted
com.ms.fx.FxText.getChar could not be converted
com.ms.fx.FxText.getWordBreak could not be converted
com.ms.fx.FxText.isDelimiter could not be converted
com.ms.fx.FxText.isWhite could not be converted
com.ms.fx.FxText.nChars could not be converted
com.ms.fx.FxText.setText could not be converted
com.ms.fx.FxTexture.DRAW_BL could not be converted
com.ms.fx.FxTexture.DRAW_BOTTOM could not be converted
com.ms.fx.FxTexture.DRAW_BR could not be converted
com.ms.fx.FxTexture.DRAW_CENTER could not be converted
com.ms.fx.FxTexture.DRAW_LEFT could not be converted
com.ms.fx.FxTexture.DRAW_RIGHT could not be converted

com.ms.fx.FxTexture.DRAW_TL could not be converted
com.ms.fx.FxTexture.DRAW_TOP could not be converted
com.ms.fx.FxTexture.DRAW_TR could not be converted
com.ms.fx.FxTexture.drawScanLinesCallback could not be converted
com.ms.fx.FxTexture.FxTexture could not be converted
com.ms.fx.FxTexture.getBottomAxis could not be converted
com.ms.fx.FxTexture.getInner could not be converted
com.ms.fx.FxTexture.getLeftAxis could not be converted
com.ms.fx.FxTexture.getPinOrigin could not be converted
com.ms.fx.FxTexture.getRightAxis could not be converted
com.ms.fx.FxTexture.getSnapDraw could not be converted
com.ms.fx.FxTexture.getStretch could not be converted
com.ms.fx.FxTexture.getTopAxis could not be converted
com.ms.fx.FxTexture.getUpdatedAreasMask could not be converted
com.ms.fx.FxTexture.imageUpdate could not be converted
com.ms.fx.FxTexture.REPEAT_PIN could not be converted
com.ms.fx.FxTexture.setAxis could not be converted
com.ms.fx.FxTexture.setPinOrigin could not be converted
com.ms.fx.FxTexture.setSnapDraw could not be converted
com.ms.fx.FxTexture.setStretch could not be converted
com.ms.fx.FxTexture.setUpdateCallback could not be converted
com.ms.fx.FxTexture.setUpdatedAreasMask could not be converted
com.ms.fx.FxTexture.size could not be converted
com.ms.fx.FxTexture.SNAP_EDGES could not be converted
com.ms.fx.FxTexture.STRETCH_MIDDLE could not be converted
com.ms.fx.FxTexture.STRETCH_OUTER could not be converted
com.ms.fx.FxToolkit could not be converted
com.ms.fx.GlyphMetrics could not be converted
com.ms.fx.GlyphOutline could not be converted
com.ms.fx.IFxShape could not be converted
com.ms.fx.IFxSystemInterface could not be converted
com.ms.fx.IFxTextCallback could not be converted
com.ms.fx.IFxTextConstants could not be converted
com.ms.fx.IFxTextConstants.DIRLAYOUT could not be converted
com.ms.fx.IFxTextConstants.htaCenter could not be converted
com.ms.fx.IFxTextConstants.htaJustified could not be converted
com.ms.fx.IFxTextConstants.htaLeft could not be converted
com.ms.fx.IFxTextConstants.htaRight could not be converted
com.ms.fx.IFxTextConstants.htaScriptDefault could not be converted
com.ms.fx.IFxTextConstants.htaStretch could not be converted

com.ms.fx.IFxTextConstants.MOVE_DOWN could not be converted
com.ms.fx.IFxTextConstants.MOVE_LEFT could not be converted
com.ms.fx.IFxTextConstants.MOVE_RIGHT could not be converted
com.ms.fx.IFxTextConstants.MOVE_UP could not be converted
com.ms.fx.IFxTextConstants.NEXT_DOWN could not be converted
com.ms.fx.IFxTextConstants.NEXT_LEFT could not be converted
com.ms.fx.IFxTextConstants.NEXT_RIGHT could not be converted
com.ms.fx.IFxTextConstants.NEXT_UP could not be converted
com.ms.fx.IFxTextConstants.OPAQUE_BODY could not be converted
com.ms.fx.IFxTextConstants.OPAQUE_POST could not be converted
com.ms.fx.IFxTextConstants.OPAQUE_PRIOR could not be converted
com.ms.fx.IFxTextConstants.SCRIPT_DEF could not be converted
com.ms.fx.IFxTextConstants.tdBT_LR could not be converted
com.ms.fx.IFxTextConstants.tdBT_RL could not be converted
com.ms.fx.IFxTextConstants.tdHebrewNormal could not be converted
com.ms.fx.IFxTextConstants.tdJapanTradNormal could not be converted
com.ms.fx.IFxTextConstants.tdLatinNormal could not be converted
com.ms.fx.IFxTextConstants.tdLeftToRightReading could not be converted
com.ms.fx.IFxTextConstants.tdLR_BT could not be converted
com.ms.fx.IFxTextConstants.tdLR_TB could not be converted
com.ms.fx.IFxTextConstants.tdMongolianNormal could not be converted
com.ms.fx.IFxTextConstants.tdRightToLeftReading could not be converted
com.ms.fx.IFxTextConstants.tdRL_BT could not be converted
com.ms.fx.IFxTextConstants.tdRL_TB could not be converted
com.ms.fx.IFxTextConstants.tdScriptDefault could not be converted
com.ms.fx.IFxTextConstants.tdTB_LR could not be converted
com.ms.fx.IFxTextConstants.tdTB_RL could not be converted
com.ms.fx.IFxTextConstants.tdVisualLayout could not be converted
com.ms.fx.IFxTextConstants.VISUAL_DEF could not be converted
com.ms.fx.IFxTextConstants.vtaBaseline could not be converted
com.ms.fx.IFxTextConstants.vtaBottom could not be converted
com.ms.fx.IFxTextConstants.vtaCenter could not be converted
com.ms.fx.IFxTextConstants.vtaScriptDefault could not be converted
com.ms.fx.IFxTextConstants.vtaStretch could not be converted
com.ms.fx.IFxTextConstants.vtaTop could not be converted
com.ms.fx.IFxTextConstants.wwCleanEdges could not be converted
com.ms.fx.IFxTextConstants.wwKeepWordIntact could not be converted
com.ms.fx.IFxTextConstants.wwMask could not be converted
com.ms.fx.IFxTextConstants.wwNone could not be converted
com.ms.fx.IFxTextConstants.wwTypeMask could not be converted

com.ms.fx.IFxTextConstants.wwVirtualRectEnd could not be converted
com.ms.fx.IFxTextConstants.wwVirtualRectSide could not be converted
com.ms.fx.IFxTextConstants.wwWrap could not be converted
com.ms.fx.IFxTextureUpdate could not be converted
com.ms.fx.OutlineCurve could not be converted
com.ms.fx.OutlinePolygon could not be converted
com.ms.fx.PeerConstants could not be converted
com.ms.fx.Region.clone could not be converted
com.ms.fx.Region.COMPLEX could not be converted
com.ms.fx.Region.complexity could not be converted
com.ms.fx.Region.copy could not be converted
com.ms.fx.Region.EMPTY~ could not be converted
com.ms.fx.Region.equals could not be converted
com.ms.fx.Region.fillBounds could not be converted
com.ms.fx.Region.getBounds could not be converted
com.ms.fx.Region.getGeometry could not be converted
com.ms.fx.Region.invert could not be converted
com.ms.fx.Region.isEmpty could not be converted
com.ms.fx.Region.Region could not be converted
com.ms.fx.Region.set could not be converted
com.ms.fx.Region.SIMPLE could not be converted
com.ms.fx.RegionConverter could not be converted
com.ms.fx.txtRun could not be converted
com.ms.fx.Version could not be converted

Com.ms.io Error Messages

`com.ms.io.console.Console` could not be converted

`com.ms.io.console.DefaultConsole` could not be converted

`com.ms.io.ObjectInputStreamWithLoader` could not be converted

`com.ms.io.OffsetInputStreamFilter.mark` could not be converted

`com.ms.io.OffsetInputStreamFilter.reset` could not be converted

`com.ms.io.Path.hasExecExtensionType` could not be converted

`com.ms.io.Path.isRoot` could not be converted

`com.ms.io.Path.validateFilename` could not be converted

`com.ms.io.SystemInputStream` could not be converted

`com.ms.io.SystemOutputStream` could not be converted

`com.ms.io.UserFileDialog` could not be converted

Com.ms.jdbc.odbc Error Messages

[com.ms.jdbc.odbc.JdbcOdbcConnection.setAutoCommit could not be converted](#)

[com.ms.jdbc.odbc.JdbcOdbcConnection.getMetaData could not be converted](#)

Com.ms.lang Error Messages

com.ms.lang.MulticastDelegate.invokeHelperMulticast could not be converted

com.ms.lang.RegKey.enumKey could not be converted

com.ms.lang.RegKey.finalize could not be converted

com.ms.lang.RegKey.getBinaryValue could not be converted

com.ms.lang.RegKey.KEYOPEN_ALL could not be converted

com.ms.lang.RegKey.KEYOPEN_CREATE could not be converted

com.ms.lang.RegKey.KEYOPEN_READ could not be converted

com.ms.lang.RegKey.KEYOPEN_WRITE could not be converted

com.ms.lang.RegKey.loadKey could not be converted

com.ms.lang.RegKey.queryInfo could not be converted

com.ms.lang.RegKey.replace could not be converted

com.ms.lang.RegKey.restore could not be converted

com.ms.lang.RegKey.unload could not be converted

com.ms.lang.RegQueryInfo could not be converted

com.ms.lang.SystemThread could not be converted

com.ms.lang.SystemX.arrayCompare could not be converted

com.ms.lang.SystemX.blockcopy could not be converted

com.ms.lang.SystemX.exitProcessAfterMainThreadReturns could not be converted

com.ms.lang.SystemX.getDeclaredMethodFromSignature could not be converted

com.ms.lang.SystemX.getDefaultInputManager could not be converted

com.ms.lang.SystemX.getInputManager could not be converted

com.ms.lang.SystemX.getKeyboardLanguageName could not be converted

com.ms.lang.SystemX.getKeyboardLanguages could not be converted

com.ms.lang.SystemX.getMethodFromSignature could not be converted

com.ms.lang.SystemX.getNativeServices could not be converted

com.ms.lang.SystemX.isLocalCharDBCSLeadByte could not be converted

com.ms.lang.SystemX.JavaStringToLocalString could not be converted

com.ms.lang.SystemX.LocalStringToJavaString could not be converted

com.ms.lang.SystemX.setInputManager could not be converted

com.ms.lang.SystemX.setKeyboardLanguage could not be converted

com.ms.lang.VerifyErrorEx could not be converted

Com.ms.mtx Error Messages

com.ms.mtx.AppServer could not be converted

com.ms.mtx.Context.createObject could not be converted

com.ms.mtx.Context.disableCommit could not be converted

com.ms.mtx.Context.enableCommit could not be converted

com.ms.mtx.Context.getContextId could not be converted

com.ms.mtx.Context.getDeactivateOnReturn could not be converted

com.ms.mtx.Context.getDirectCallerName could not be converted

com.ms.mtx.Context.getDirectCreatorName could not be converted

com.ms.mtx.Context.getMyTransactionVote could not be converted

com.ms.mtx.Context.getObjectContext could not be converted

com.ms.mtx.Context.getOriginalCallerName could not be converted

com.ms.mtx.Context.getOriginalCreatorName could not be converted

com.ms.mtx.Context.getProperty could not be converted

com.ms.mtx.Context.getPropertyNames could not be converted

com.ms.mtx.Context.getSafeRef could not be converted

com.ms.mtx.Context.getTransaction could not be converted

com.ms.mtx.Context.getTransactionId could not be converted

com.ms.mtx.Context.isCallerInRole could not be converted

com.ms.mtx.Context.isInTransaction could not be converted

com.ms.mtx.Context.isSecurityEnabled could not be converted

com.ms.mtx.Context.setAbort could not be converted

com.ms.mtx.Context.setComplete could not be converted

com.ms.mtx.Context.setDeactivateOnReturn could not be converted

com.ms.mtx.Context.setMyTransactionVote could not be converted

com.ms.mtx.Context.TxAbort could not be converted

com.ms.mtx.Context.TxCommit could not be converted

com.ms.mtx.IContextState.GetDeactivateOnReturn could not be converted

com.ms.mtx.IContextState.GetMyTransactionVote could not be converted

com.ms.mtx.IContextState.iid could not be converted

com.ms.mtx.IContextState.SetDeactivateOnReturn could not be converted

com.ms.mtx.IContextState.SetMyTransactionVote could not be converted

com.ms.mtx.IEnumNames could not be converted

com.ms.mtx.IGetContextProperties could not be converted

com.ms.mtx.IMTxAS could not be converted

com.ms.mtx.IMTxAS.GetObjectContext could not be converted

com.ms.mtx.IMTxAS.iid could not be converted

com.ms.mtx.IObjectContext.CreateInstance could not be converted

com.ms.mtx.IObjectContext.iid could not be converted

com.ms.mtx.IObjectContextInfo.iid could not be converted

com.ms.mtx.IObjectControl.iid could not be converted

com.ms.mtx.IObjectControl could not be converted

com.ms.mtx.ISecurityCallContext.getItem could not be converted

com.ms.mtx.ISecurityCallContext.iid could not be converted

com.ms.mtx.ISecurityCallersColl.iid could not be converted

com.ms.mtx.ISecurityIdentityColl.getItem could not be converted

com.ms.mtx.ISecurityIdentityColl.iid could not be converted

com.ms.mtx.SecurityProperty.GetDirectCallerName could not be converted

com.ms.mtx.SecurityProperty.GetOriginalCallerName could not be converted

com.ms.mtx.ISharedProperty.iid could not be converted

com.ms.mtx.ISharedPropertyGroup.iid could not be converted

com.ms.mtx.ISharedPropertyGroupManager.iid could not be converted

com.ms.mtx.ITransactionContextEx could not be converted

com.ms.mtx.MTx could not be converted

com.ms.mtx.ObjectContext.CreateInstance could not be converted

com.ms.mtx.ObjectContext.DisableCommit could not be converted

com.ms.mtx.ObjectContext.EnableCommit could not be converted

com.ms.mtx.ObjectContext.get_NewEnum could not be converted

com.ms.mtx.ObjectContext.getCount could not be converted

com.ms.mtx.ObjectContext.getItem could not be converted

com.ms.mtx.ObjectContext.getSecurity could not be converted

com.ms.mtx.ObjectContext.iid could not be converted

com.ms.mtx.ObjectContext.IsCallerInRole could not be converted

com.ms.mtx.ObjectContext.IsInTransaction could not be converted

com.ms.mtx.ObjectContext.IsSecurityEnabled could not be converted

com.ms.mtx.ObjectContext.SetAbort could not be converted

com.ms.mtx.ObjectContext.SetComplete could not be converted

com.ms.mtx.SecurityCallContext.getCallers could not be converted

com.ms.mtx.SecurityCallContext.getDirectCaller could not be converted

com.ms.mtx.SecurityCallContext.getNumCallers could not be converted

com.ms.mtx.SecurityCallContext.getProperty could not be converted

com.ms.mtx.SecurityCallContext.getPropertyNames could not be converted

com.ms.mtx.SecurityCallContext.isUserInRole could not be converted

com.ms.mtx.SecurityCaller could not be converted

com.ms.mtx.SecurityProperty.GetDirectCreatorName could not be converted

com.ms.mtx.SecurityProperty.GetOriginalCreatorName could not be converted

com.ms.mtx.SecurityProperty.iid could not be converted

com.ms.mtx.SharedPropertyGroupManager.clsid could not be converted

com.ms.mtx.SharedPropertyGroupManager.get_NewEnum could not be converted

com.ms.mtx.TransactionContextEx could not be converted

Com.ms.object Error Messages

com.ms.object.Category could not be converted

com.ms.object.dragdrop.DragHandler could not be converted

com.ms.object.dragdrop.DragHelper could not be converted

com.ms.object.dragdrop.DragProxy could not be converted

com.ms.object.dragdrop.DragSession.getDragModifiers could not be converted

com.ms.object.dragdrop.DragSource.DEFAULT_ACTION could not be converted

com.ms.object.dragdrop.DragSource.queryDragCursor could not be converted

com.ms.object.dragdrop.DragSource.queryDragStatus could not be converted

com.ms.object.IServiceObjectProvider could not be converted

com.ms.object.ISite could not be converted

com.ms.object.ISiteable could not be converted

com.ms.object.MetaObject could not be converted

com.ms.object.ObjectBag could not be converted

com.ms.object.SimpleTransferSession could not be converted

com.ms.object.TransferSession could not be converted

Com.ms.ui Error Messages

com.ms.ui.<ClassName>.add*Listener could not be converted
com.ms.ui.<ClassName>.addNotify could not be converted
com.ms.ui.<ClassName>.getPeer could not be converted
com.ms.ui.<ClassName>.handleEvent could not be converted
com.ms.ui.<ClassName>.isNotified could not be converted
com.ms.ui.<ClassName>.process*Event could not be converted
com.ms.ui.<ClassName>.remove*Listener could not be converted
com.ms.ui.<ClassName>.removeNotify could not be converted
com.ms.ui.<ClassName>.setListenerTracker could not be converted
com.ms.ui.<ClassName>BeanInfo could not be converted
com.ms.ui.AwtUIApplet.add(UIComponent) could not be converted
com.ms.ui.AwtUIApplet.add(UIComponent, int) could not be converted
com.ms.ui.AwtUIApplet.add(UIComponent, Object) could not be converted
com.ms.ui.AwtUIApplet.add(UIComponent, Object, int) could not be converted
com.ms.ui.AwtUIApplet.add(String, UIComponent) could not be converted
com.ms.ui.AwtUIApplet.AwtUIApplet(UIComponent) could not be converted
com.ms.ui.AwtUIApplet.AwtUIApplet(UIApplet) could not be converted
com.ms.ui.AwtUIApplet.destroy could not be converted
com.ms.ui.AwtUIApplet.getComponent could not be converted
com.ms.ui.AwtUIApplet.getHeader could not be converted
com.ms.ui.AwtUIApplet.getRoot could not be converted
com.ms.ui.AwtUIApplet.getTaskManager could not be converted
com.ms.ui.AwtUIApplet.getUIComponentCount could not be converted
com.ms.ui.AwtUIApplet.setHeader could not be converted
com.ms.ui.AwtUIApplet.setLayout could not be converted
com.ms.ui.AwtUIBand could not be converted
com.ms.ui.AwtUIBandBeanInfo.AwtUIBandBeanInfo could not be converted
com.ms.ui.AwtUIBandBeanInfo.getIcon could not be converted
com.ms.ui.AwtUIBandBox could not be converted
com.ms.ui.AwtUIBandBoxBeanInfo.AwtUIBandBoxBeanInfo could not be converted
com.ms.ui.AwtUIBandBoxBeanInfo.getIcon could not be converted
com.ms.ui.AwtUIButton._btn could not be converted
com.ms.ui.AwtUIButton.getStyle could not be converted
com.ms.ui.AwtUIButton.keyDown could not be converted
com.ms.ui.AwtUIButton.keyUp could not be converted
com.ms.ui.AwtUIButton.mouseUp could not be converted
com.ms.ui.AwtUIButton.setStyle could not be converted

com.ms.ui.AwtUICheckBox.getBase could not be converted

com.ms.ui.AwtUICheckBox.getSelectedObjects could not be converted

com.ms.ui.AwtUIChoice.add could not be converted

com.ms.ui.AwtUIChoice.addSelectedIndex could not be converted

com.ms.ui.AwtUIChoice.addSelectedIndices could not be converted

com.ms.ui.AwtUIChoice.addSelectedItem could not be converted

com.ms.ui.AwtUIChoice.addSelectedItems could not be converted

com.ms.ui.AwtUIChoice.getAnchorItem could not be converted

com.ms.ui.AwtUIChoice.getBase could not be converted

com.ms.ui.AwtUIChoice.getExtensionItem could not be converted

com.ms.ui.AwtUIChoice.getSelectedItem could not be converted

com.ms.ui.AwtUIChoice.getSelectedItems could not be converted

com.ms.ui.AwtUIChoice.getSelectedObjects could not be converted

com.ms.ui.AwtUIChoice.getSelectionMode could not be converted

com.ms.ui.AwtUIChoice.getStyle could not be converted

com.ms.ui.AwtUIChoice.removeSelectedIndex could not be converted

com.ms.ui.AwtUIChoice.removeSelectedIndices could not be converted

com.ms.ui.AwtUIChoice.removeSelectedItem could not be converted

com.ms.ui.AwtUIChoice.removeSelectedItems could not be converted

com.ms.ui.AwtUIChoice.setAnchorItem could not be converted

com.ms.ui.AwtUIChoice.setExtensionItem could not be converted

com.ms.ui.AwtUIChoice.setSelectedIndex(int) could not be converted

com.ms.ui.AwtUIChoice.setSelectedIndex(int, boolean) could not be converted

com.ms.ui.AwtUIChoice.setSelectedIndices could not be converted

com.ms.ui.AwtUIChoice.setSelectedItem could not be converted

com.ms.ui.AwtUIChoice.setSelectedItems could not be converted

com.ms.ui.AwtUIChoice.setSelectionMode could not be converted

com.ms.ui.AwtUIChoice.setStyle could not be converted

com.ms.ui.AwtUIColumnViewer could not be converted

com.ms.ui.AwtUIControl.add(Component, Object) could not be converted

com.ms.ui.AwtUIControl.add(Component, Object, int) could not be converted

com.ms.ui.AwtUIControl.add(UIComponent) could not be converted

com.ms.ui.AwtUIControl.add(UIComponent, int) could not be converted

com.ms.ui.AwtUIControl.add(UIComponent, Object) could not be converted

com.ms.ui.AwtUIControl.add(UIComponent, Object, int) could not be converted

com.ms.ui.AwtUIControl.add(String, Component) could not be converted

com.ms.ui.AwtUIControl.add(String, UIComponent) could not be converted

com.ms.ui.AwtUIControl.AwtUIControl could not be converted

com.ms.ui.AwtUIControl.getID could not be converted

com.ms.ui.AwtUIControl.isSelected could not be converted

com.ms.ui.AwtUIControl.postEvent could not be converted
com.ms.ui.AwtUIControl.setChecked could not be converted
com.ms.ui.AwtUIControl.setHot could not be converted
com.ms.ui.AwtUIControl.setID could not be converted
com.ms.ui.AwtUIControl.setIndeterminate could not be converted
com.ms.ui.AwtUIControl.setLayout could not be converted
com.ms.ui.AwtUIControl.setPressed could not be converted
com.ms.ui.AwtUIControl.setReparent could not be converted
com.ms.ui.AwtUIControl.setSelected could not be converted
com.ms.ui.AwtUIDialog.add(UIComponent) could not be converted
com.ms.ui.AwtUIDialog.add(UIComponent, int) could not be converted
com.ms.ui.AwtUIDialog.add(UIComponent, Object) could not be converted
com.ms.ui.AwtUIDialog.add(UIComponent, Object, int) could not be converted
com.ms.ui.AwtUIDialog.add(String, UIComponent) could not be converted
com.ms.ui.AwtUIDialog.getComponent could not be converted
com.ms.ui.AwtUIDialog.position could not be converted
com.ms.ui.AwtUIDialog.setLayout could not be converted
com.ms.ui.AwtUIDrawText.getCharFromScreen could not be converted
com.ms.ui.AwtUIDrawText.getCharLocation could not be converted
com.ms.ui.AwtUIDrawText.getOutline could not be converted
com.ms.ui.AwtUIDrawText.isAutoResizable could not be converted
com.ms.ui.AwtUIDrawText.setAutoResizable could not be converted
com.ms.ui.AwtUIDrawText.setHorizAlign could not be converted
com.ms.ui.AwtUIDrawText.setOutline could not be converted
com.ms.ui.AwtUIDrawText.setRefresh could not be converted
com.ms.ui.AwtUIDrawText.setVertAlign could not be converted
com.ms.ui.AwtUIDrawText.setWordWrap could not be converted
com.ms.ui.AwtUIEdit.getCharFromScreen could not be converted
com.ms.ui.AwtUIEdit.getCharLocation could not be converted
com.ms.ui.AwtUIEdit.getOutline could not be converted
com.ms.ui.AwtUIEdit.getWordWrap could not be converted
com.ms.ui.AwtUIEdit.isAutoResizable could not be converted
com.ms.ui.AwtUIEdit.setAutoResizable could not be converted
com.ms.ui.AwtUIEdit.setHorizAlign could not be converted
com.ms.ui.AwtUIEdit.setOutline could not be converted
com.ms.ui.AwtUIEdit.setRefresh could not be converted
com.ms.ui.AwtUIEdit.setVertAlign could not be converted
com.ms.ui.AwtUIEdit.setWordWrap could not be converted
com.ms.ui.AwtUIEdit.showCaret could not be converted
com.ms.ui.AwtUIFrame.add(UIComponent) could not be converted

com.ms.ui.AwtUIFrame.add(UIComponent, int) could not be converted
com.ms.ui.AwtUIFrame.add(UIComponent, Object) could not be converted
com.ms.ui.AwtUIFrame.add(UIComponent, Object, int) could not be converted
com.ms.ui.AwtUIFrame.add(String, UIComponent) could not be converted
com.ms.ui.AwtUIFrame.requestFocus could not be converted
com.ms.ui.AwtUIFrame.setLayout could not be converted
com.ms.ui.AwtUIGraphic.AwtUIGraphic could not be converted
com.ms.ui.AwtUIGraphic.getContentBounds could not be converted
com.ms.ui.AwtUIGraphic.imageUpdate could not be converted
com.ms.ui.AwtUIHost could not be converted
com.ms.ui.AwtUIHost.add(UIComponent) could not be converted
com.ms.ui.AwtUIHost.add(UIComponent, int) could not be converted
com.ms.ui.AwtUIHost.add(UIComponent, Object) could not be converted
com.ms.ui.AwtUIHost.add(UIComonent, Object, int) could not be converted
com.ms.ui.AwtUIHost.add(String, UIComponent) could not be converted
com.ms.ui.AwtUIHost.disableHostEvents could not be converted
com.ms.ui.AwtUIHost.enableHostEvents could not be converted
com.ms.ui.AwtUIHost.getComponent could not be converted
com.ms.ui.AwtUIHost.getHeader could not be converted
com.ms.ui.AwtUIHost.getPreferredSize could not be converted
com.ms.ui.AwtUIHost.getRoot could not be converted
com.ms.ui.AwtUIHost.getUIComponent could not be converted
com.ms.ui.AwtUIHost.invalidate could not be converted
com.ms.ui.AwtUIHost.layout could not be converted
com.ms.ui.AwtUIHost.listenerTracker could not be converted
com.ms.ui.AwtUIHost.obtainListenerTracker could not be converted
com.ms.ui.AwtUIHost.paint could not be converted
com.ms.ui.AwtUIHost.paintAll could not be converted
com.ms.ui.AwtUIHost.preferredSize could not be converted
com.ms.ui.AwtUIHost.preProcessHostEvent could not be converted
com.ms.ui.AwtUIHost.root could not be converted
com.ms.ui.AwtUIHost.setHeader could not be converted
com.ms.ui.AwtUIHost.setLayout could not be converted
com.ms.ui.AwtUIHost.setListenerHost could not be converted
com.ms.ui.AwtUIHost.show could not be converted
com.ms.ui.AwtUIHost.usingNewEvents could not be converted
com.ms.ui.AwtUIHost.validate could not be converted
com.ms.ui.AwtUIHost.validateTree could not be converted
com.ms.ui.AwtUIList.addSelectedIndex(int) could not be converted
com.ms.ui.AwtUIList.addSelectedIndex(int, boolean) could not be converted

com.ms.ui.AwtUIList.addSelectedIndices(int[]) could not be converted
com.ms.ui.AwtUIList.addSelectedIndices(int[], boolean) could not be converted
com.ms.ui.AwtUIList.addSelectedItem could not be converted
com.ms.ui.AwtUIList.addSelectedItems could not be converted
com.ms.ui.AwtUIList.AwtUIList(int) could not be converted
com.ms.ui.AwtUIList.AwtUIList(int, int) could not be converted
com.ms.ui.AwtUIList.find could not be converted
com.ms.ui.AwtUIList.getSelectedIndex could not be converted
com.ms.ui.AwtUIList.getSelectedItem could not be converted
com.ms.ui.AwtUIList.remove could not be converted
com.ms.ui.AwtUIList.removeSelectedIndex(int) could not be converted
com.ms.ui.AwtUIList.removeSelectedIndex(int, boolean) could not be converted
com.ms.ui.AwtUIList.removeSelectedIndices(int[]) could not be converted
com.ms.ui.AwtUIList.removeSelectedIndices(int[], boolean) could not be converted
com.ms.ui.AwtUIList.removeSelectedItem could not be converted
com.ms.ui.AwtUIList.removeSelectedItems could not be converted
com.ms.ui.AwtUIList.setSelectedIndex(int) could not be converted
com.ms.ui.AwtUIList.setSelectedIndex(int, boolean) could not be converted
com.ms.ui.AwtUIList.setSelectedIndices(int[]) could not be converted
com.ms.ui.AwtUIList.setSelectedIndices(int[], boolean) could not be converted
com.ms.ui.AwtUIList.setSelectedItem could not be converted
com.ms.ui.AwtUIList.setSelectedItems could not be converted
com.ms.ui.AwtUIList.setSelectionMode could not be converted
com.ms.ui.AwtUIMarquee could not be converted
com.ms.ui.AwtUIMenuList.AwtUIMenuList could not be converted
com.ms.ui.AwtUIMenuList.getSelectedObjects could not be converted
com.ms.ui.AwtUIMessageBox could not be converted
com.ms.ui.AwtUIMessageBox.action could not be converted
com.ms.ui.AwtUIMessageBox.AwtUIMessageBox could not be converted
com.ms.ui.AwtUIMessageBox.doModal could not be converted
com.ms.ui.AwtUIMessageBox.doModalIndex could not be converted
com.ms.ui.AwtUIMessageBox.getButtonAlignment could not be converted
com.ms.ui.AwtUIMessageBox.getButtons could not be converted
com.ms.ui.AwtUIMessageBox.getDefaultButton could not be converted
com.ms.ui.AwtUIMessageBox.getFrame could not be converted
com.ms.ui.AwtUIMessageBox.getImage could not be converted
com.ms.ui.AwtUIMessageBox.getText could not be converted
com.ms.ui.AwtUIMessageBox.getTimeout could not be converted
com.ms.ui.AwtUIMessageBox.insets could not be converted
com.ms.ui.AwtUIMessageBox.keyDown could not be converted

com.ms.ui.AwtUIMessageBox.keyUp could not be converted

com.ms.ui.AwtUIMessageBox.preferredSize could not be converted

com.ms.ui.AwtUIMessageBox.setButtonAlignment could not be converted

com.ms.ui.AwtUIMessageBox.setButtons could not be converted

com.ms.ui.AwtUIMessageBox.setDefaultButton could not be converted

com.ms.ui.AwtUIMessageBox.setImage could not be converted

com.ms.ui.AwtUIMessageBox.setText could not be converted

com.ms.ui.AwtUIMessageBox.setTimeout could not be converted

com.ms.ui.AwtUIMessageBox.timeTriggered could not be converted

com.ms.ui.AwtUIPanel.setLayout could not be converted

com.ms.ui.AwtUIProgress.AwtUIProgress could not be converted

com.ms.ui.AwtUIProgress.getBase could not be converted

com.ms.ui.AwtUIPushButton.getBase could not be converted

com.ms.ui.AwtUIRadioButton.AwtUIRadioButton(UIComponent, int) could not be converted

com.ms.ui.AwtUIRadioButton.AwtUIRadioButton(String, int) could not be converted

com.ms.ui.AwtUIRadioButton.getBase could not be converted

com.ms.ui.AwtUIRadioButton.getSelectedObjects could not be converted

com.ms.ui.AwtUIRepeatButton.AwtUIRepeatButton could not be converted

com.ms.ui.AwtUIScrollBar could not be converted

com.ms.ui.AwtUIScrollBar.AwtUIScrollBar could not be converted

com.ms.ui.AwtUIScrollBar.getBase could not be converted

com.ms.ui.AwtUIScrollBar.getStyle could not be converted

com.ms.ui.AwtUIScrollBar.scrollLineDown could not be converted

com.ms.ui.AwtUIScrollBar.scrollLineUp could not be converted

com.ms.ui.AwtUIScrollBar.scrollPageDown could not be converted

com.ms.ui.AwtUIScrollBar.scrollPageUp could not be converted

com.ms.ui.AwtUIScrollBar.setScrollInfo could not be converted

com.ms.ui.AwtUIScrollBar.setScrollLine could not be converted

com.ms.ui.AwtUIScrollBar.setStyle could not be converted

com.ms.ui.AwtUIScrollBar.setUnitIncrement could not be converted

com.ms.ui.AwtUIScrollViewer could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(Component) could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(int) could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(int, int, int) could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(int, int, int, int) could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(int, int, int, int, int) could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(UIComponent) could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(UIComponent, int) could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(UIComponent, int, int) could not be converted

com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(UIComponent, int, int, int) could not be converted
com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(UIComponent, int, int, int, int) could not be converted
com.ms.ui.AwtUIScrollViewer.AwtUIScrollViewer(UIComponent, int, int, int, int, int) could not be converted
com.ms.ui.AwtUIScrollViewer.getContent could not be converted
com.ms.ui.AwtUIScrollViewer.getHLine could not be converted
com.ms.ui.AwtUIScrollViewer.getLine could not be converted
com.ms.ui.AwtUIScrollViewer.getVLine could not be converted
com.ms.ui.AwtUIScrollViewer.setContent could not be converted
com.ms.ui.AwtUIScrollViewer.setHLine could not be converted
com.ms.ui.AwtUIScrollViewer.setLine could not be converted
com.ms.ui.AwtUIScrollViewer.setVLine could not be converted
com.ms.ui.AwtUISplitViewer could not be converted
com.ms.ui.AwtUISplitViewer.add could not be converted
com.ms.ui.AwtUISplitViewer.AwtUISplitViewer could not be converted
com.ms.ui.AwtUISplitViewer.getComponent could not be converted
com.ms.ui.AwtUISplitViewer.getPos could not be converted
com.ms.ui.AwtUISplitViewer.getStyle could not be converted
com.ms.ui.AwtUISplitViewer.remove could not be converted
com.ms.ui.AwtUISplitViewer.setPos could not be converted
com.ms.ui.AwtUIStatus.getBase could not be converted
com.ms.ui.AwtUITabList could not be converted
com.ms.ui.AwtUITabList.add(UIComponent) could not be converted
com.ms.ui.AwtUITabList.add(UIComponent, int) could not be converted
com.ms.ui.AwtUITabList.add(String) could not be converted
com.ms.ui.AwtUITabViewer.add(String, Component) could not be converted
com.ms.ui.AwtUITabViewer.add(String, UIComponent) could not be converted
com.ms.ui.AwtUITabViewer.addTab could not be converted
com.ms.ui.AwtUITree could not be converted
com.ms.ui.AwtUITree.add(Component) could not be converted
com.ms.ui.AwtUITree.add(Component, int) could not be converted
com.ms.ui.AwtUITree.add(Image, String, int) could not be converted
com.ms.ui.AwtUITree.add(UIComponent) could not be converted
com.ms.ui.AwtUITree.add(UIComponent, int) could not be converted
com.ms.ui.AwtUITree.add(String, int) could not be converted
com.ms.ui.AwtUITree.addSelectedIndex(int) could not be converted
com.ms.ui.AwtUITree.addSelectedIndex(int, boolean) could not be converted
com.ms.ui.AwtUITree.addSelectedIndices(int[]) could not be converted
com.ms.ui.AwtUITree.addSelectedIndices(int[], boolean) could not be converted
com.ms.ui.AwtUITree.addSelectedItem(UIComponent) could not be converted
com.ms.ui.AwtUITree.addSelectedItem(UIComponent, boolean) could not be converted

com.ms.ui.AwtUITree.addSelectedItems(UIComponent[]) could not be converted

com.ms.ui.AwtUITree.addSelectedItems(UIComponent[], boolean) could not be converted

com.ms.ui.AwtUITree.getExpander could not be converted

com.ms.ui.AwtUITree.getSelectedIndex could not be converted

com.ms.ui.AwtUITree.getSelectedIndices could not be converted

com.ms.ui.AwtUITree.getSelectedItems could not be converted

com.ms.ui.AwtUITree.getSelectedObjects could not be converted

com.ms.ui.AwtUITree.getSelectionMode could not be converted

com.ms.ui.AwtUITree.remove could not be converted

com.ms.ui.AwtUITree.removeSelectedIndex(int) could not be converted

com.ms.ui.AwtUITree.removeSelectedIndex(int, boolean) could not be converted

com.ms.ui.AwtUITree.removeSelectedIndices(int[]) could not be converted

com.ms.ui.AwtUITree.removeSelectedIndices(int[], boolean) could not be converted

com.ms.ui.AwtUITree.removeSelectedItem(UIComponent) could not be converted

com.ms.ui.AwtUITree.removeSelectedItem(UIComponent, boolean) could not be converted

com.ms.ui.AwtUITree.removeSelectedItems(UIComponent[]) could not be converted

com.ms.ui.AwtUITree.removeSelectedItems(UIComponent[], boolean) could not be converted

com.ms.ui.AwtUITree.setSelectedIndex(int) could not be converted

com.ms.ui.AwtUITree.setSelectedIndex(int, boolean) could not be converted

com.ms.ui.AwtUITree.setSelectedIndices(int[]) could not be converted

com.ms.ui.AwtUITree.setSelectedIndices(int[], boolean) could not be converted

com.ms.ui.AwtUITree.setSelectedItem could not be converted

com.ms.ui.AwtUITree.setSelectedItems(UIComponent[]) could not be converted

com.ms.ui.AwtUITree.setSelectedItems(UIComponent[], boolean) could not be converted

com.ms.ui.AwtUITree.setSelectionMode could not be converted

com.ms.ui.AwtUIWindow.add(UIComponent) could not be converted

com.ms.ui.AwtUIWindow.add(UIComponent, Object) could not be converted

com.ms.ui.AwtUIWindow.add(UIComponent, Object, int) could not be converted

com.ms.ui.AwtUIWindow.add(String, UIComponent) could not be converted

com.ms.ui.AwtUIWindow.AwtUIWindow could not be converted

com.ms.ui.AwtUIWindow.requestFocus could not be converted

com.ms.ui.AwtUIWindow.setLayout could not be converted

com.ms.ui.ButtonFlowLayout could not be converted

com.ms.ui.ButtonFlowLayout.ButtonFlowLayout could not be converted

com.ms.ui.ButtonFlowLayout.computeUnitDimension could not be converted

com.ms.ui.ButtonFlowLayout.getHeightPad could not be converted

com.ms.ui.ButtonFlowLayout.getMinHeight could not be converted

com.ms.ui.ButtonFlowLayout.getMinWidth could not be converted

com.ms.ui.ButtonFlowLayout.getWidthPad could not be converted

com.ms.ui.ButtonFlowLayout.setHeightPad could not be converted

com.ms.ui.ButtonFlowLayout.setMinHeight could not be converted
com.ms.ui.ButtonFlowLayout.setMinWidth could not be converted
com.ms.ui.ButtonFlowLayout.setWidthPad could not be converted
com.ms.ui.ButtonPanel.ButtonPanel could not be converted
com.ms.ui.ButtonPanel.getDefaultButton could not be converted
com.ms.ui.event.UIActionEvent.<EventType> could not be converted
com.ms.ui.event.UIActionEvent.getActionCommand could not be converted
com.ms.ui.event.UIAdjustmentEvent.<EventType> could not be converted
com.ms.ui.event.UIBaseEvent.getID could not be converted
com.ms.ui.event.UIContainerEvent.<EventType> could not be converted
com.ms.ui.event.UIEvent could not be converted
com.ms.ui.event.UIFocusEvent.<EventType> could not be converted
com.ms.ui.event.UIFocusEvent.getArg could not be converted
com.ms.ui.event.UIFocusEvent.isTemporary could not be converted
com.ms.ui.event.UInputEvent could not be converted
com.ms.ui.event.UItemEvent.<EventType> could not be converted
com.ms.ui.event.UItemEvent.getItem could not be converted
com.ms.ui.event.UItemEvent.getStateChange could not be converted
com.ms.ui.event.UIKeyEvent.CHAR_UNDEFINED could not be converted
com.ms.ui.event.UIKeyEvent.getKeyChar could not be converted
com.ms.ui.event.UIKeyEvent.getKeyCode could not be converted
com.ms.ui.event.UIKeyEvent.getOldEventKey could not be converted
com.ms.ui.event.UIKeyEvent.KEY_EVENT_BASE could not be converted
com.ms.ui.event.UIKeyEvent.KEY_PRESSED could not be converted
com.ms.ui.event.UIKeyEvent.KEY_RELEASED could not be converted
com.ms.ui.event.UIKeyEvent.KEY_TYPED could not be converted
com.ms.ui.event.UIKeyEvent.VK_BACK_QUOTE could not be converted
com.ms.ui.event.UIKeyEvent.VK_EQUALS could not be converted
com.ms.ui.event.UIKeyEvent.VK_META could not be converted
com.ms.ui.event.UIKeyEvent.VK_UNDEFINED could not be converted
com.ms.ui.event.UIMouseEvent.<EventType> could not be converted
com.ms.ui.event.UIMouseEvent.getClickCount could not be converted
com.ms.ui.event.UIMouseEvent.getPoint could not be converted
com.ms.ui.event.UIMouseEvent.getX could not be converted
com.ms.ui.event.UIMouseEvent.getY could not be converted
com.ms.ui.event.UIMouseEvent.isPopupTrigger could not be converted
com.ms.ui.event.UINotifyEvent could not be converted
com.ms.ui.event.UITextEvent.<EventType> could not be converted
com.ms.ui.event.UIWindowEvent.<EventType> could not be converted
com.ms.ui.IAwtUIAdjustable could not be converted

com.ms.ui.IAwTUIItemSelectable could not be converted
com.ms.ui.IUIAccessible could not be converted
com.ms.ui.IUIAccessible.<ErrorCode> could not be converted
com.ms.ui.IUIAccessible.getBounds could not be converted
com.ms.ui.IUIAccessible.navigate could not be converted
com.ms.ui.IUIBand could not be converted
com.ms.ui.IUIComponent.action could not be converted
com.ms.ui.IUIComponent.adjustLayoutSize could not be converted
com.ms.ui.IUIComponent.deliverEvent could not be converted
com.ms.ui.IUIComponent.ensureVisible could not be converted
com.ms.ui.IUIComponent.getBounds could not be converted
com.ms.ui.IUIComponent.getCachedPreferredSize could not be converted
com.ms.ui.IUIComponent.getID could not be converted
com.ms.ui.IUIComponent.getLocation could not be converted
com.ms.ui.IUIComponent.getMaximumSize could not be converted
com.ms.ui.IUIComponent.getMinimumSize could not be converted
com.ms.ui.IUIComponent.getPreferredSize could not be converted
com.ms.ui.IUIComponent.getToolkit could not be converted
com.ms.ui.IUIComponent.isChecked could not be converted
com.ms.ui.IUIComponent.isHeightRelative could not be converted
com.ms.ui.IUIComponent.isHot could not be converted
com.ms.ui.IUIComponent.isIndeterminate could not be converted
com.ms.ui.IUIComponent.isInvalidating could not be converted
com.ms.ui.IUIComponent.isKeyable could not be converted
com.ms.ui.IUIComponent.isKeyable(boolean) could not be converted
com.ms.ui.IUIComponent.isPressed could not be converted
com.ms.ui.IUIComponent.isRedrawing could not be converted
com.ms.ui.IUIComponent.isSelected could not be converted
com.ms.ui.IUIComponent.isValid could not be converted
com.ms.ui.IUIComponent.isWidthRelative could not be converted
com.ms.ui.IUIComponent.keyDown could not be converted
com.ms.ui.IUIComponent.keyUp could not be converted
com.ms.ui.IUIComponent.lostFocus could not be converted
com.ms.ui.IUIComponent.mouseClicked could not be converted
com.ms.ui.IUIComponent.mouseDown could not be converted
com.ms.ui.IUIComponent.mouseEnter could not be converted
com.ms.ui.IUIComponent.mouseExit could not be converted
com.ms.ui.IUIComponent.mouseMove could not be converted
com.ms.ui.IUIComponent.mouseUp could not be converted
com.ms.ui.IUIComponent.paint could not be converted

com.ms.ui.IUIComponent.paintAll could not be converted
com.ms.ui.IUIComponent.prepareImage could not be converted
com.ms.ui.IUIComponent.print could not be converted
com.ms.ui.IUIComponent.printAll could not be converted
com.ms.ui.IUIComponent.recalcPreferredSize could not be converted
com.ms.ui.IUIComponent.setBounds could not be converted
com.ms.ui.IUIComponent.setChecked could not be converted
com.ms.ui.IUIComponent.setFlags could not be converted
com.ms.ui.IUIComponent.setHot could not be converted
com.ms.ui.IUIComponent.setID could not be converted
com.ms.ui.IUIComponent.setIndeterminate could not be converted
com.ms.ui.IUIComponent.setInvalidating could not be converted
com.ms.ui.IUIComponent.setLocation(int, int) could not be converted
com.ms.ui.IUIComponent.setLocation(Point) could not be converted
com.ms.ui.IUIComponent.setPressed could not be converted
com.ms.ui.IUIComponent.setRedrawing could not be converted
com.ms.ui.IUIComponent.setSelected could not be converted
com.ms.ui.IUIComponent.setValid could not be converted
com.ms.ui.IUIComponent.setVisible could not be converted
com.ms.ui.IUIComponent.validate could not be converted
com.ms.ui.IUIContainer.add(IUIComponent) could not be converted
com.ms.ui.IUIContainer.add(IUIComponent, int) could not be converted
com.ms.ui.IUIContainer.add(IUIComponent, Object) could not be converted
com.ms.ui.IUIContainer.add(IUIComponent, Object, int) could not be converted
com.ms.ui.IUIContainer.add(String, IUIComponent) could not be converted
com.ms.ui.IUIContainer.adjustLayoutSize could not be converted
com.ms.ui.IUIContainer.continueInvalidate could not be converted
com.ms.ui.IUIContainer.ensureVisible could not be converted
com.ms.ui.IUIContainer.getChildBounds could not be converted
com.ms.ui.IUIContainer.getChildLocation could not be converted
com.ms.ui.IUIContainer.getChildSize could not be converted
com.ms.ui.IUIContainer.getComponentFromID could not be converted
com.ms.ui.IUIContainer.getEdge could not be converted
com.ms.ui.IUIContainer.getLayout could not be converted
com.ms.ui.IUIContainer.isOverlapping could not be converted
com.ms.ui.IUIContainer.navigate could not be converted
com.ms.ui.IUIContainer.paintComponents could not be converted
com.ms.ui.IUIContainer.passFocus could not be converted
com.ms.ui.IUIContainer.replace could not be converted
com.ms.ui.IUIContainer.setChildBounds could not be converted

com.ms.ui.UIContainer.setChildLocation could not be converted

com.ms.ui.UIContainer.setChildSize could not be converted

com.ms.ui.UIContainer.setEdge could not be converted

com.ms.ui.UIContainer.setHeader could not be converted

com.ms.ui.UIContainer.setLayout could not be converted

com.ms.ui.UILayoutManager could not be converted

com.ms.ui.UIMenuLauncher could not be converted

com.ms.ui.UIPosition could not be converted

com.ms.ui.UIPropertyPage could not be converted

com.ms.ui.UIRootContainer.componentMoved could not be converted

com.ms.ui.UIRootContainer.endMenu could not be converted

com.ms.ui.UIRootContainer.endTooltip could not be converted

com.ms.ui.UIRootContainer.getFocus could not be converted

com.ms.ui.UIRootContainer.getLaunchedMenu could not be converted

com.ms.ui.UIRootContainer.launchMenu could not be converted

com.ms.ui.UIRootContainer.launchTooltip could not be converted

com.ms.ui.UIRootContainer.needsValidating could not be converted

com.ms.ui.UIRootContainer.setFocus could not be converted

com.ms.ui.UIScroll could not be converted

com.ms.ui.UISelector could not be converted

com.ms.ui.UISpinnerBuddy could not be converted

com.ms.ui.UITree could not be converted

com.ms.ui.UIWizardStep could not be converted

com.ms.ui.IWinEvent could not be converted

com.ms.ui.resource.DataBoundInputStream could not be converted

com.ms.ui.resource.ResourceDecoder could not be converted

com.ms.ui.resource.ResourceFormattingException could not be converted

com.ms.ui.resource.ResourceTypeListener could not be converted

com.ms.ui.resource.UIDialogLayout could not be converted

com.ms.ui.resource.Win32ResourceDecoder could not be converted

com.ms.ui.UIApplet.destroy could not be converted

com.ms.ui.UIApplet.getAppletContext could not be converted

com.ms.ui.UIApplet.getAudioClip(URL) could not be converted

com.ms.ui.UIApplet.getAudioClip(URL, String) could not be converted

com.ms.ui.UIApplet.getDocumentBase could not be converted

com.ms.ui.UIApplet.isActive could not be converted

com.ms.ui.UIApplet.play(URL) could not be converted

com.ms.ui.UIApplet.play(URL, String) could not be converted

com.ms.ui.UIApplet.setStub could not be converted

com.ms.ui.UIApplet.showStatus could not be converted

com.ms.ui.UIAwthost.forwardEvents could not be converted

com.ms.ui.UIAwthost.getCachedPreferredSize could not be converted

com.ms.ui.UIAwthost.getMinimumSize could not be converted

com.ms.ui.UIAwthost.getPreferredSize could not be converted

com.ms.ui.UIAwthost.notifyEvent could not be converted

com.ms.ui.UIAwthost.setFocused could not be converted

com.ms.ui.UIAwthost.setValid could not be converted

com.ms.ui.UIBand could not be converted

com.ms.ui.UIBandBox could not be converted

com.ms.ui.UIBandThumb could not be converted

com.ms.ui.UIBarLayout could not be converted

com.ms.ui.UIBorderLayout could not be converted

com.ms.ui.UIButton.doDefaultAction could not be converted

com.ms.ui.UIButton.getStyle could not be converted

com.ms.ui.UIButton.keyDown could not be converted

com.ms.ui.UIButton.keyUp could not be converted

com.ms.ui.UIButton.mouseClicked could not be converted

com.ms.ui.UIButton.setHot could not be converted

com.ms.ui.UIButton.setStyle could not be converted

com.ms.ui.UIButtonBar could not be converted

com.ms.ui.UIButtonBar.<ButtonBarType> could not be converted

com.ms.ui.UIButtonBar.add could not be converted

com.ms.ui.UIButtonBar.addTo could not be converted

com.ms.ui.UIButtonBar.UIButtonBar could not be converted

com.ms.ui.UICanvas.getID could not be converted

com.ms.ui.UICanvas.setID could not be converted

com.ms.ui.UICardLayout could not be converted

com.ms.ui.UICheckButton.getCheckImageSize could not be converted

com.ms.ui.UICheckButton.getConvertedEvent could not be converted

com.ms.ui.UICheckButton.getMinimumSize could not be converted

com.ms.ui.UICheckButton.getPreferredSize could not be converted

com.ms.ui.UICheckButton.paint could not be converted

com.ms.ui.UICheckButton.setCheckImageSize could not be converted

com.ms.ui.UICheckButton.setHot could not be converted

com.ms.ui.UICheckButton.setID could not be converted

com.ms.ui.UICheckButton.setPressed could not be converted

com.ms.ui.UICheckButton.setSelected could not be converted

com.ms.ui.UICheckGroup.add(UIComponent) could not be converted

com.ms.ui.UICheckGroup.add(UIComponent, int) could not be converted

com.ms.ui.UICheckGroup.add(UIComponent, Object, int) could not be converted

com.ms.ui.UICheckGroup.add(String, Object, int) could not be converted
com.ms.ui.UICheckGroup.setHeader could not be converted
com.ms.ui.UICheckGroup.UICheckGroup could not be converted
com.ms.ui.UICheckGroup.add could not be converted
com.ms.ui.UICheckGroup.addSelectedIndex(int) could not be converted
com.ms.ui.UICheckGroup.addSelectedIndex(int, boolean) could not be converted
com.ms.ui.UICheckGroup.addSelectedIndices could not be converted
com.ms.ui.UICheckGroup.addSelectedItem could not be converted
com.ms.ui.UICheckGroup.addSelectedItems could not be converted
com.ms.ui.UICheckGroup.adjustPopupListSize could not be converted
com.ms.ui.UICheckGroup.getAnchorItem could not be converted
com.ms.ui.UICheckGroup.getConvertedEvent could not be converted
com.ms.ui.UICheckGroup.getExtensionItem could not be converted
com.ms.ui.UICheckGroup.getMenu could not be converted
com.ms.ui.UICheckGroup.getPopupListMaxStringCount could not be converted
com.ms.ui.UICheckGroup.getPreferredSize could not be converted
com.ms.ui.UICheckGroup.getSelectedIndices could not be converted
com.ms.ui.UICheckGroup.getSelectedItem could not be converted
com.ms.ui.UICheckGroup.getSelectedItems could not be converted
com.ms.ui.UICheckGroup.getSelectionMode could not be converted
com.ms.ui.UICheckGroup.getStyle could not be converted
com.ms.ui.UICheckGroup.keyDown could not be converted
com.ms.ui.UICheckGroup.mouseDown could not be converted
com.ms.ui.UICheckGroup.remove could not be converted
com.ms.ui.UICheckGroup.removeSelectedIndex could not be converted
com.ms.ui.UICheckGroup.removeSelectedIndices could not be converted
com.ms.ui.UICheckGroup.removeSelectedItem could not be converted
com.ms.ui.UICheckGroup.removeSelectedItems could not be converted
com.ms.ui.UICheckGroup.setAnchorItem could not be converted
com.ms.ui.UICheckGroup.setExtensionItem could not be converted
com.ms.ui.UICheckGroup.setPopupListMaxStringCount could not be converted
com.ms.ui.UICheckGroup.setSelectedIndex(int) could not be converted
com.ms.ui.UICheckGroup.setSelectedIndex(int, boolean) could not be converted
com.ms.ui.UICheckGroup.setSelectedIndices(int[]) could not be converted
com.ms.ui.UICheckGroup.setSelectedIndices(int[], boolean) could not be converted
com.ms.ui.UICheckGroup.setSelectedItem could not be converted
com.ms.ui.UICheckGroup.setSelectionMode could not be converted
com.ms.ui.UICheckGroup.setStyle could not be converted
com.ms.ui.UICheckGroup.setUsingWindowsLook could not be converted
com.ms.ui.UICheckGroup.THICK could not be converted

com.ms.ui.UIColorDialog.UIColorDialog could not be converted
com.ms.ui.UIColumnHeader.isMoving could not be converted
com.ms.ui.UIColumnHeader.isSizing could not be converted
com.ms.ui.UIColumnHeader.isSizingLeft could not be converted
com.ms.ui.UIColumnHeader.isSizingRight could not be converted
com.ms.ui.UIColumnHeader.mouseDown could not be converted
com.ms.ui.UIColumnViewer could not be converted
com.ms.ui.UIComponent.action could not be converted
com.ms.ui.UIComponent.adjustLayoutSize could not be converted
com.ms.ui.UIComponent.clone could not be converted
com.ms.ui.UIComponent.contains could not be converted
com.ms.ui.UIComponent.deliverEvent could not be converted
com.ms.ui.UIComponent.doDefaultAction could not be converted
com.ms.ui.UIComponent.doLayout could not be converted
com.ms.ui.UIComponent.ensureVisible could not be converted
com.ms.ui.UIComponent.getBounds could not be converted
com.ms.ui.UIComponent.getCachedPreferredSize could not be converted
com.ms.ui.UIComponent.getComponentAt could not be converted
com.ms.ui.UIComponent.getDefaultAction could not be converted
com.ms.ui.UIComponent.getDescription could not be converted
com.ms.ui.UIComponent.getGraphics could not be converted
com.ms.ui.UIComponent.getHelp could not be converted
com.ms.ui.UIComponent.getKeyboardShortcut could not be converted
com.ms.ui.UIComponent.getLocation could not be converted
com.ms.ui.UIComponent.getLocationOnScreen could not be converted
com.ms.ui.UIComponent.getMaximumSize could not be converted
com.ms.ui.UIComponent.getMinimumSize could not be converted
com.ms.ui.UIComponent.getPreferredSize could not be converted
com.ms.ui.UIComponent.getRoleCode could not be converted
com.ms.ui.UIComponent.getRoot could not be converted
com.ms.ui.UIComponent.getSize could not be converted
com.ms.ui.UIComponent.getStateCode could not be converted
com.ms.ui.UIComponent.getToolkit could not be converted
com.ms.ui.UIComponent.getTreeLock could not be converted
com.ms.ui.UIComponent.getValueText could not be converted
com.ms.ui.UIComponent.imageUpdate could not be converted
com.ms.ui.UIComponent.invalidate could not be converted
com.ms.ui.UIComponent.isChecked could not be converted
com.ms.ui.UIComponent.isEnabled could not be converted
com.ms.ui.UIComponent.isHeightRelative could not be converted

com.ms.ui.UIComponent.isHot could not be converted
com.ms.ui.UIComponent.isIndeterminate could not be converted
com.ms.ui.UIComponent.isInvalidating could not be converted
com.ms.ui.UIComponent.isKeyable could not be converted
com.ms.ui.UIComponent.isKeyable(boolean) could not be converted
com.ms.ui.UIComponent.isPressed could not be converted
com.ms.ui.UIComponent.isRedrawing could not be converted
com.ms.ui.UIComponent.isSelectable could not be converted
com.ms.ui.UIComponent.isShowing could not be converted
com.ms.ui.UIComponent.isValid could not be converted
com.ms.ui.UIComponent.isVisible could not be converted
com.ms.ui.UIComponent.isWidthRelative could not be converted
com.ms.ui.UIComponent.keyDown could not be converted
com.ms.ui.UIComponent.keyUp could not be converted
com.ms.ui.UIComponent.lostFocus could not be converted
com.ms.ui.UIComponent.mouseClicked could not be converted
com.ms.ui.UIComponent.mouseDown could not be converted
com.ms.ui.UIComponent.mouseDrag could not be converted
com.ms.ui.UIComponent.mouseEnter could not be converted
com.ms.ui.UIComponent.mouseExit could not be converted
com.ms.ui.UIComponent.mouseMove could not be converted
com.ms.ui.UIComponent.mouseUp could not be converted
com.ms.ui.UIComponent.navigate could not be converted
com.ms.ui.UIComponent.notifyEvent could not be converted
com.ms.ui.UIComponent.paint could not be converted
com.ms.ui.UIComponent.paintAll could not be converted
com.ms.ui.UIComponent.prepareImage(Image, ImageObserver) could not be converted
com.ms.ui.UIComponent.prepareImage(int, int, ImageObserver) could not be converted
com.ms.ui.UIComponent.print could not be converted
com.ms.ui.UIComponent.printAll could not be converted
com.ms.ui.UIComponent.recalcPreferredSize could not be converted
com.ms.ui.UIComponent.requestFocus could not be converted
com.ms.ui.UIComponent.repaint could not be converted
com.ms.ui.UIComponent.requestFocus could not be converted
com.ms.ui.UIComponent.setBounds could not be converted
com.ms.ui.UIComponent.setChecked could not be converted
com.ms.ui.UIComponent.setFlags could not be converted
com.ms.ui.UIComponent.setHot could not be converted
com.ms.ui.UIComponent.setID could not be converted
com.ms.ui.UIComponent.setIndeterminate could not be converted

com.ms.ui.UIComponent.setInvalidating could not be converted

com.ms.ui.UIComponent.setLocation(int, int) could not be converted

com.ms.ui.UIComponent.setLocation(Point) could not be converted

com.ms.ui.UIComponent.setPressed could not be converted

com.ms.ui.UIComponent.setRedrawing could not be converted

com.ms.ui.UIComponent.setSelected could not be converted

com.ms.ui.UIComponent.setSize could not be converted

com.ms.ui.UIComponent.setUsingWindowsLook could not be converted

com.ms.ui.UIComponent.setValid could not be converted

com.ms.ui.UIComponent.setValueText could not be converted

com.ms.ui.UIComponent.setVisible could not be converted

com.ms.ui.UIComponent.validate could not be converted

com.ms.ui.UIContainer.add(UIComponent) could not be converted

com.ms.ui.UIContainer.add(UIComponent, int) could not be converted

com.ms.ui.UIContainer.add(UIComponent, Object) could not be converted

com.ms.ui.UIContainer.add(UIComponent, Object, int) could not be converted

com.ms.ui.UIContainer.add(String, UIComponent) could not be converted

com.ms.ui.UIContainer.adjustLayoutSize could not be converted

com.ms.ui.UIContainer.continueInvalidate could not be converted

com.ms.ui.UIContainer.ensureVisible could not be converted

com.ms.ui.UIContainer.getChildBounds could not be converted

com.ms.ui.UIContainer.getChildIndex could not be converted

com.ms.ui.UIContainer.getChildLocation could not be converted

com.ms.ui.UIContainer.getChildSize could not be converted

com.ms.ui.UIContainer.getClientRect could not be converted

com.ms.ui.UIContainer.getComponent could not be converted

com.ms.ui.UIContainer.getComponentFromID could not be converted

com.ms.ui.UIContainer.getComponentIndex could not be converted

com.ms.ui.UIContainer.getComponents could not be converted

com.ms.ui.UIContainer.getEdge could not be converted

com.ms.ui.UIContainer.getFocusComponent could not be converted

com.ms.ui.UIContainer.getID could not be converted

com.ms.ui.UIContainer.getInsets could not be converted

com.ms.ui.UIContainer.getLayout could not be converted

com.ms.ui.UIContainer.getMinimumSize could not be converted

com.ms.ui.UIContainer.getName could not be converted

com.ms.ui.UIContainer.getPreferredSize could not be converted

com.ms.ui.UIContainer.gotFocus could not be converted

com.ms.ui.UIContainer.invalidateAll could not be converted

com.ms.ui.UIContainer.isHeightRelative could not be converted

com.ms.ui.UIContainer.isOverlapping could not be converted
com.ms.ui.UIContainer.isWidthRelative could not be converted
com.ms.ui.UIContainer.keyDown could not be converted
com.ms.ui.UIContainer.lostFocus could not be converted
com.ms.ui.UIContainer.mouseEnter could not be converted
com.ms.ui.UIContainer.mouseExit could not be converted
com.ms.ui.UIContainer.move(int, int) could not be converted
com.ms.ui.UIContainer.move(UIComponent, UIComponent) could not be converted
com.ms.ui.UIContainer.navigate could not be converted
com.ms.ui.UIContainer.notifyEvent could not be converted
com.ms.ui.UIContainer.paint could not be converted
com.ms.ui.UIContainer.paintAll could not be converted
com.ms.ui.UIContainer.paintComponents could not be converted
com.ms.ui.UIContainer.passFocus could not be converted
com.ms.ui.UIContainer.printAll could not be converted
com.ms.ui.UIContainer.remove could not be converted
com.ms.ui.UIContainer.removeAll could not be converted
com.ms.ui.UIContainer.removeAllChildren could not be converted
com.ms.ui.UIContainer.replace could not be converted
com.ms.ui.UIContainer.setChildBounds could not be converted
com.ms.ui.UIContainer.setChildLocation could not be converted
com.ms.ui.UIContainer.setChildSize could not be converted
com.ms.ui.UIContainer.setComponent could not be converted
com.ms.ui.UIContainer.setEdge could not be converted
com.ms.ui.UIContainer.setID could not be converted
com.ms.ui.UIContainer.setLayout could not be converted
com.ms.ui.UIContainer.setLocation could not be converted
com.ms.ui.UIContainer.setSize could not be converted
com.ms.ui.UIContextMenu.action could not be converted
com.ms.ui.UIContextMenu.ended could not be converted
com.ms.ui.UIContextMenu.getPlacement could not be converted
com.ms.ui.UIContextMenu.getRoot could not be converted
com.ms.ui.UIContextMenu.getUserItem could not be converted
com.ms.ui.UIContextMenu.UIContextMenu could not be converted
com.ms.ui.UIDialog.adjustLayoutSize could not be converted
com.ms.ui.UIDialog.isAutoPack could not be converted
com.ms.ui.UIDialog.keyDown could not be converted
com.ms.ui.UIDialog.keyUp could not be converted
com.ms.ui.UIDialog.position could not be converted
com.ms.ui.UIDialog.setAutoPack could not be converted

com.ms.ui.UIDialog.setModal could not be converted

com.ms.ui.UIDialog.UIDialog(UIFrame, boolean) could not be converted

com.ms.ui.UIDialog.UIDialog(UIFrame, String, boolean) could not be converted

com.ms.ui.UIDialogMapping could not be converted

com.ms.ui.UIDragDrop could not be converted

com.ms.ui.UIDrawText.ensureNotDirty could not be converted

com.ms.ui.UIDrawText.ensureVisible(Point) could not be converted

com.ms.ui.UIDrawText.ensureVisible(Rectangle) could not be converted

com.ms.ui.UIDrawText.eoln could not be converted

com.ms.ui.UIDrawText.getCachedGraphics could not be converted

com.ms.ui.UIDrawText.getCaretWidth could not be converted

com.ms.ui.UIDrawText.getCharFromScreen(int, int) could not be converted

com.ms.ui.UIDrawText.getCharFromScreen(Point) could not be converted

com.ms.ui.UIDrawText.getCharLocation could not be converted

com.ms.ui.UIDrawText.getMinimumSize could not be converted

com.ms.ui.UIDrawText.getOutline could not be converted

com.ms.ui.UIDrawText.getPreferredSize could not be converted

com.ms.ui.UIDrawText.getStartPoint could not be converted

com.ms.ui.UIDrawText.getVertAlign could not be converted

com.ms.ui.UIDrawText.getWordEdge could not be converted

com.ms.ui.UIDrawText.gotFocus could not be converted

com.ms.ui.UIDrawText.hideCaret could not be converted

com.ms.ui.UIDrawText.isAutoResizable could not be converted

com.ms.ui.UIDrawText.keyDown could not be converted

com.ms.ui.UIDrawText.lostFocus could not be converted

com.ms.ui.UIDrawText.mouseDown could not be converted

com.ms.ui.UIDrawText.mouseDrag could not be converted

com.ms.ui.UIDrawText.paint(FxGraphics) could not be converted

com.ms.ui.UIDrawText.paint(FxGraphics, int, int, boolean) could not be converted

com.ms.ui.UIDrawText.setAutoResizable could not be converted

com.ms.ui.UIDrawText.setCaretWidth could not be converted

com.ms.ui.UIDrawText.setCurrIndex could not be converted

com.ms.ui.UIDrawText.setHorizAlign could not be converted

com.ms.ui.UIDrawText.setInputMethod could not be converted

com.ms.ui.UIDrawText.setOutline could not be converted

com.ms.ui.UIDrawText.setRefresh could not be converted

com.ms.ui.UIDrawText.setTabs could not be converted

com.ms.ui.UIDrawText.setTextCallback could not be converted

com.ms.ui.UIDrawText.setUnderlined could not be converted

com.ms.ui.UIDrawText.setVertAlign could not be converted

com.ms.ui.UIDrawText.setWordWrap could not be converted

com.ms.ui.UIDrawText.showCaret could not be converted

com.ms.ui.UIEdit.allowUndo could not be converted

com.ms.ui.UIEdit.append(char, boolean) could not be converted

com.ms.ui.UIEdit.append(char[], boolean) could not be converted

com.ms.ui.UIEdit.append(String, boolean) could not be converted

com.ms.ui.UIEdit.clear could not be converted

com.ms.ui.UIEdit.EXCLUDE could not be converted

com.ms.ui.UIEdit.getConvertedEvent could not be converted

com.ms.ui.UIEdit.getMaskChars could not be converted

com.ms.ui.UIEdit.getMaskMode could not be converted

com.ms.ui.UIEdit.INCLUDE could not be converted

com.ms.ui.UIEdit.insert(char, int, boolean) could not be converted

com.ms.ui.UIEdit.insert(char[], int, boolean) could not be converted

com.ms.ui.UIEdit.insert(String, int, boolean) could not be converted

com.ms.ui.UIEdit.isRedoable could not be converted

com.ms.ui.UIEdit.isUndoable could not be converted

com.ms.ui.UIEdit.isUndoAllowed could not be converted

com.ms.ui.UIEdit.keyDown could not be converted

com.ms.ui.UIEdit.READONLY could not be converted

com.ms.ui.UIEdit.redo could not be converted

com.ms.ui.UIEdit.redo(boolean) could not be converted

com.ms.ui.UIEdit.remove could not be converted

com.ms.ui.UIEdit.setMaskChars could not be converted

com.ms.ui.UIEdit.setMaskMode could not be converted

com.ms.ui.UIEditChoice.action could not be converted

com.ms.ui.UIEditChoice.getEditComponent could not be converted

com.ms.ui.UIEditChoice.keyDown could not be converted

com.ms.ui.UIEditChoice.lostFocus could not be converted

com.ms.ui.UIExpandButton could not be converted

com.ms.ui.UIFindReplaceDialog could not be converted

com.ms.ui.UIFixedFlowLayout could not be converted

com.ms.ui.UIFixedFlowLayout.computeUnitDimension could not be converted

com.ms.ui.UIFixedFlowLayout.getAlignment could not be converted

com.ms.ui.UIFixedFlowLayout.getMinimumSize could not be converted

com.ms.ui.UIFixedFlowLayout.getPreferredSize could not be converted

com.ms.ui.UIFixedFlowLayout.isOverlapping could not be converted

com.ms.ui.UIFixedFlowLayout.isWidthRelative could not be converted

com.ms.ui.UIFixedFlowLayout.layout could not be converted

com.ms.ui.UIFixedFlowLayout.navigate could not be converted

com.ms.ui.UIFixedFlowLayout.setAlignment could not be converted

com.ms.ui.UIFixedFlowLayout.UIFixedFlowLayout could not be converted

com.ms.ui.UIFixedFlowLayout.UIFixedFlowLayout(int, int) could not be converted

com.ms.ui.UIFixedFlowLayout.UIFixedFlowLayout(int, int, int) could not be converted

com.ms.ui.UIFlowLayout could not be converted

com.ms.ui.UIFlowLayout.setAlignment could not be converted

com.ms.ui.UIFlowLayout.getMinimumSize could not be converted

com.ms.ui.UIFlowLayout.getPreferredSize could not be converted

com.ms.ui.UIFlowLayout.isOverlapping could not be converted

com.ms.ui.UIFlowLayout.isWidthRelative could not be converted

com.ms.ui.UIFlowLayout.layout could not be converted

com.ms.ui.UIFlowLayout.navigate could not be converted

com.ms.ui.UIFlowLayout.setAlignment could not be converted

com.ms.ui.UIFlowLayout.setWrap could not be converted

com.ms.ui.UIFlowLayout.UIFlowLayout could not be converted

com.ms.ui.UIFlowLayout.UIFlowLayout(int) could not be converted

com.ms.ui.UIFlowLayout.UIFlowLayout(int, int, int) could not be converted

com.ms.ui.UIFontDialog.UIFontDialog(UIFrame) could not be converted

com.ms.ui.UIFontDialog.UIFontDialog(UIFrame, String[]) could not be converted

com.ms.ui.UIFrame.setMenuBar could not be converted

com.ms.ui.UIGraphic.getPreferredSize could not be converted

com.ms.ui.UIGraphic.imageUpdate could not be converted

com.ms.ui.UIGraphic.paint could not be converted

com.ms.ui.UIGraphic.UIGraphic could not be converted

com.ms.ui.UIGridBagConstraints could not be converted

com.ms.ui.UIGridBagLayout could not be converted

com.ms.ui.UIGridLayout could not be converted

com.ms.ui.UIGridLayout.continueInvalidate could not be converted

com.ms.ui.UIGridLayout.getMinimumSize could not be converted

com.ms.ui.UIGridLayout.getPreferredSize could not be converted

com.ms.ui.UIGridLayout.isOverlapping could not be converted

com.ms.ui.UIGridLayout.layout could not be converted

com.ms.ui.UIGridLayout.navigate could not be converted

com.ms.ui.UIGridLayout.UIGridLayout(int, int) could not be converted

com.ms.ui.UIGridLayout.UIGridLayout(int, int, int, int) could not be converted

com.ms.ui.UIGroup.adjustLayoutSize could not be converted

com.ms.ui.UIGroup.paint could not be converted

com.ms.ui.UIGroup.UIGroup could not be converted

com.ms.ui.UIHeaderRow.mouseDown could not be converted

com.ms.ui.UIHeaderRow.mouseDrag could not be converted

com.ms.ui.UIHeaderRow.mouseUp could not be converted
com.ms.ui.UIHeaderRow.navigate could not be converted
com.ms.ui.UIItem.<Position> could not be converted
com.ms.ui.UIItem.getGap could not be converted
com.ms.ui.UIItem.getImagePos could not be converted
com.ms.ui.UIItem.getPreferredSize could not be converted
com.ms.ui.UIItem.imageUpdate could not be converted
com.ms.ui.UIItem.paint could not be converted
com.ms.ui.UIItem.setFocused could not be converted
com.ms.ui.UIItem.setGap could not be converted
com.ms.ui.UIItem.setHot could not be converted
com.ms.ui.UIItem.setImagePos could not be converted
com.ms.ui.UIItem.setSelected could not be converted
com.ms.ui.UIItem.UIItem(Image, String, int) could not be converted
com.ms.ui.UIItem.UIItem(Image, String, int, int) could not be converted
com.ms.ui.UILayoutManager could not be converted
com.ms.ui.UILine.getMinimumSize could not be converted
com.ms.ui.UILine.getPreferredSize could not be converted
com.ms.ui.UILine.paint could not be converted
com.ms.ui.UIList.UIList(int) could not be converted
com.ms.ui.UIList.UIList(int, int) could not be converted
com.ms.ui.UIMarquee could not be converted
com.ms.ui.UIMenuButton.action could not be converted
com.ms.ui.UIMenuButton.ended could not be converted
com.ms.ui.UIMenuButton.getPlacement could not be converted
com.ms.ui.UIMenuButton.keyDown could not be converted
com.ms.ui.UIMenuButton.launch could not be converted
com.ms.ui.UIMenuButton.mouseDown could not be converted
com.ms.ui.UIMenuButton.requestFocus could not be converted
com.ms.ui.UIMenuButton.UIMenuButton(UIComponent, int, UIMenuList) could not be converted
com.ms.ui.UIMenuButton.UIMenuButton(UIComponent, UIMenuList) could not be converted
com.ms.ui.UIMenuButton.UIMenuButton(String, int, UIMenuList) could not be converted
com.ms.ui.UIMenuItem.UIMenuItem could not be converted
com.ms.ui.UIMenuItem.action could not be converted
com.ms.ui.UIMenuItem.getInsets could not be converted
com.ms.ui.UIMenuItem.keyDown could not be converted
com.ms.ui.UIMenuItem.paint could not be converted
com.ms.ui.UIMenuItem.UIMenuItem(UIComponent) could not be converted
com.ms.ui.UIMenuItem.UIMenuItem(UIComponent, UIMenuList) could not be converted
com.ms.ui.UIMenuLauncher.cancel could not be converted

com.ms.ui.UIMenuLauncher.ended could not be converted

com.ms.ui.UIMenuLauncher.fitToScreen could not be converted

com.ms.ui.UIMenuLauncher.getDisplayer could not be converted

com.ms.ui.UIMenuLauncher.getPlacement could not be converted

com.ms.ui.UIMenuLauncher.isLaunched could not be converted

com.ms.ui.UIMenuLauncher.launch could not be converted

com.ms.ui.UIMenuLauncher.raiseEvent could not be converted

com.ms.ui.UIMenuLauncher.setFocused could not be converted

com.ms.ui.UIMenuLauncher.setHot could not be converted

com.ms.ui.UIMenuLauncher.setSelected could not be converted

com.ms.ui.UIMenuLauncher.UIMenuLauncher(UIComponent) could not be converted

com.ms.ui.UIMenuLauncher.UIMenuLauncher(UIComponent, UIMenuList) could not be converted

com.ms.ui.UIMenuLauncher.UIMenuLauncher(UIMenuList) could not be converted

com.ms.ui.UIMenuList could not be converted

com.ms.ui.UIMenuList.action could not be converted

com.ms.ui.UIMenuList.getMenuLauncher could not be converted

com.ms.ui.UIMenuList.gotFocus could not be converted

com.ms.ui.UIMenuList.keyDown could not be converted

com.ms.ui.UIMenuList.lostFocus could not be converted

com.ms.ui.UIMenuList.mouseClicked could not be converted

com.ms.ui.UIMenuList.mouseEnter could not be converted

com.ms.ui.UIMenuList.requestFocus could not be converted

com.ms.ui.UIMenuList.setMenuLauncher could not be converted

com.ms.ui.UIMenuList.UIMenuList could not be converted

com.ms.ui.UIMessageBox could not be converted

com.ms.ui.UIMessageBox.action could not be converted

com.ms.ui.UIMessageBox.doModal could not be converted

com.ms.ui.UIMessageBox.doModalIndex could not be converted

com.ms.ui.UIMessageBox.getButtonAlignment could not be converted

com.ms.ui.UIMessageBox.getButtons could not be converted

com.ms.ui.UIMessageBox.getDefaultButton could not be converted

com.ms.ui.UIMessageBox.getFrame could not be converted

com.ms.ui.UIMessageBox.getImage could not be converted

com.ms.ui.UIMessageBox.getInsets could not be converted

com.ms.ui.UIMessageBox.getPreferredSize could not be converted

com.ms.ui.UIMessageBox.getText could not be converted

com.ms.ui.UIMessageBox.getTimeout could not be converted

com.ms.ui.UIMessageBox.insets could not be converted

com.ms.ui.UIMessageBox.keyDown could not be converted

com.ms.ui.UIMessageBox.keyUp could not be converted

com.ms.ui.UIMessageBox.setButtonAlignment could not be converted

com.ms.ui.UIMessageBox.setButtons could not be converted

com.ms.ui.UIMessageBox.setDefaultButton could not be converted

com.ms.ui.UIMessageBox.setImage could not be converted

com.ms.ui.UIMessageBox.setText could not be converted

com.ms.ui.UIMessageBox.setTimeout could not be converted

com.ms.ui.UIMessageBox.timeTriggered could not be converted

com.ms.ui.UIMessageBox.UIMessageBox(UIFrame) could not be converted

com.ms.ui.UIMessageBox.UIMessageBox(UIFrame, String, String, int, int) could not be converted

com.ms.ui.UIMSVMPopup could not be converted

com.ms.ui.UIOldEvent could not be converted

com.ms.ui.UIPanel.add could not be converted

com.ms.ui.UIPanel.getChild could not be converted

com.ms.ui.UIPanel.getChildCount could not be converted

com.ms.ui.UIPanel.getHeader could not be converted

com.ms.ui.UIPanel.getID could not be converted

com.ms.ui.UIPanel.getLayout could not be converted

com.ms.ui.UIPanel.paintAll could not be converted

com.ms.ui.UIPanel.setID could not be converted

com.ms.ui.UIPanel.setLayout could not be converted

com.ms.ui.UIPanel.setRedrawing could not be converted

com.ms.ui.UIPanel.UIPanel could not be converted

com.ms.ui.UIProgress.paint could not be converted

com.ms.ui.UIProgress.UIProgress could not be converted

com.ms.ui.UIProgress.UIProgress(int) could not be converted

com.ms.ui.UIProgress.UIProgress(int, int) could not be converted

com.ms.ui.UIProgress.update could not be converted

com.ms.ui.UIPropertyDialog could not be converted

com.ms.ui.UIPropertyPage could not be converted

com.ms.ui.UIPushButton.getConvertedEvent could not be converted

com.ms.ui.UIPushButton.paint could not be converted

com.ms.ui.UIPushButton.setChecked could not be converted

com.ms.ui.UIPushButton.setFocused could not be converted

com.ms.ui.UIPushButton.setPressed could not be converted

com.ms.ui.UIRadioButton.UIRadioButton(UIComponent, int) could not be converted

com.ms.ui.UIRadioButton.UIRadioButton(String, int) could not be converted

com.ms.ui.UIRadioGroup.add could not be converted

com.ms.ui.UIRadioGroup.UIRadioGroup could not be converted

com.ms.ui.UIRepeatButton.mouseClicked could not be converted

com.ms.ui.UIRepeatButton.setPressed could not be converted

com.ms.ui.UIRepeatButton.timeTriggered could not be converted
com.ms.ui.UIRepeatButton.UIRepeatButton could not be converted
com.ms.ui.UIRepeatButton.UIRepeatButton(UIComponent) could not be converted
com.ms.ui.UIRepeatButton.UIRepeatButton(UIComponent, int) could not be converted
com.ms.ui.UIRepeatButton.UIRepeatButton(String) could not be converted
com.ms.ui.UIRepeatButton.UIRepeatButton(String, int) could not be converted
com.ms.ui.UIRow.getName could not be converted
com.ms.ui.UIRow.requestFocus could not be converted
com.ms.ui.UIRow.setSelected could not be converted
com.ms.ui.UIRowLayout could not be converted
com.ms.ui.UIScroll could not be converted
com.ms.ui.UIScroll.scrollLineDown could not be converted
com.ms.ui.UIScroll.scrollLineUp could not be converted
com.ms.ui.UIScroll.scrollPageDown could not be converted
com.ms.ui.UIScroll.scrollPageUp could not be converted
com.ms.ui.UIScroll.setScrollInfo could not be converted
com.ms.ui.UIScroll.setScrollLine could not be converted
com.ms.ui.UIScrollBar could not be converted
com.ms.ui.UIScrollBar.<Direction> could not be converted
com.ms.ui.UIScrollBar.add could not be converted
com.ms.ui.UIScrollBar.getLayoutComponent could not be converted
com.ms.ui.UIScrollBar.setStyle could not be converted
com.ms.ui.UIScrollBar.setUsingWindowsLook could not be converted
com.ms.ui.UIScrollBar.UIScrollBar could not be converted
com.ms.ui.UIScrollBar.UIScrollBar(int) could not be converted
com.ms.ui.UIScrollBar.UIScrollBar(int, int, int, int, int, int) could not be converted
com.ms.ui.UIScrollThumb could not be converted
com.ms.ui.UIScrollView could not be converted
com.ms.ui.UIScrollView.<Position> could not be converted
com.ms.ui.UIScrollView.add could not be converted
com.ms.ui.UIScrollView.CONT could not be converted
com.ms.ui.UIScrollView.CONTENT could not be converted
com.ms.ui.UIScrollView.ensureVisible could not be converted
com.ms.ui.UIScrollView.getContent could not be converted
com.ms.ui.UIScrollView.getHLine could not be converted
com.ms.ui.UIScrollView.getLayoutComponent could not be converted
com.ms.ui.UIScrollView.getLine could not be converted
com.ms.ui.UIScrollView.getMinimumSize could not be converted
com.ms.ui.UIScrollView.getPosition could not be converted
com.ms.ui.UIScrollView.getPreferredSize could not be converted

com.ms.ui.UIScrollView.getRoleCode could not be converted

com.ms.ui.UIScrollView.getVLine could not be converted

com.ms.ui.UIScrollView.getXPosition could not be converted

com.ms.ui.UIScrollView.getYPosition could not be converted

com.ms.ui.UIScrollView.isKeyable could not be converted

com.ms.ui.UIScrollView.keyDown could not be converted

com.ms.ui.UIScrollView.remove could not be converted

com.ms.ui.UIScrollView.replace could not be converted

com.ms.ui.UIScrollView.setContent could not be converted

com.ms.ui.UIScrollView.setHLine could not be converted

com.ms.ui.UIScrollView.setLine could not be converted

com.ms.ui.UIScrollView.setScrollStyle could not be converted

com.ms.ui.UIScrollView.setUsingWindowsLook could not be converted

com.ms.ui.UIScrollView.setVLine could not be converted

com.ms.ui.UIScrollView.setXPosition could not be converted

com.ms.ui.UIScrollView.setYPosition could not be converted

com.ms.ui.UIScrollView.UIScrollView could not be converted

com.ms.ui.UIScrollView.UIScrollView(UIComponent) could not be converted

com.ms.ui.UIScrollView.UIScrollView(UIComponent, int) could not be converted

com.ms.ui.UIScrollView.UIScrollView(UIComponent, int, int) could not be converted

com.ms.ui.UIScrollView.UIScrollView(UIComponent, int, int, int) could not be converted

com.ms.ui.UIScrollView.UIScrollView(UIComponent, int, int, int, int) could not be converted

com.ms.ui.UIScrollView.UIScrollView(UIComponent, int, int, int, int, int) could not be converted

com.ms.ui.UISelector could not be converted

com.ms.ui.UISelector.addSelectedIndex(int) could not be converted

com.ms.ui.UISelector.addSelectedIndex(int, boolean) could not be converted

com.ms.ui.UISelector.addSelectedIndices(int[]) could not be converted

com.ms.ui.UISelector.addSelectedIndices(int[], boolean) could not be converted

com.ms.ui.UISelector.find could not be converted

com.ms.ui.UISelector.getSelectedIndex could not be converted

com.ms.ui.UISelector.getSelectedItem could not be converted

com.ms.ui.UISelector.remove could not be converted

com.ms.ui.UISelector.removeSelectedIndex(int) could not be converted

com.ms.ui.UISelector.removeSelectedIndex(int, boolean) could not be converted

com.ms.ui.UISelector.removeSelectedIndices(int[]) could not be converted

com.ms.ui.UISelector.removeSelectedIndices(int[], boolean) could not be converted

com.ms.ui.UISelector.setSelectedIndex(int) could not be converted

com.ms.ui.UISelector.setSelectedIndex(int, boolean) could not be converted

com.ms.ui.UISelector.setSelectedIndices(int[]) could not be converted

com.ms.ui.UISelector.setSelectedIndices(int[], boolean) could not be converted

com.ms.ui.UISelector.setSelectedItem could not be converted
com.ms.ui.UISelector.setSelectionMode could not be converted
com.ms.ui.UISingleContainer.add could not be converted
com.ms.ui.UISingleContainer.getChildBounds could not be converted
com.ms.ui.UISingleContainer.getChildLocation could not be converted
com.ms.ui.UISingleContainer.getChildSize could not be converted
com.ms.ui.UISingleContainer.getComponent could not be converted
com.ms.ui.UISingleContainer.getHeader could not be converted
com.ms.ui.UISingleContainer.layout could not be converted
com.ms.ui.UISingleContainer.mouseExit could not be converted
com.ms.ui.UISingleContainer.relayout could not be converted
com.ms.ui.UISingleContainer.remove could not be converted
com.ms.ui.UISingleContainer.setHeader could not be converted
com.ms.ui.UISingleContainer.setID could not be converted
com.ms.ui.UISingleContainer.setName could not be converted
com.ms.ui.UISingleContainer.UISingleContainer could not be converted
com.ms.ui.UISlider.add could not be converted
com.ms.ui.UISlider.clearSelection could not be converted
com.ms.ui.UISlider.getLayoutComponent could not be converted
com.ms.ui.UISlider.getSelectionEnd could not be converted
com.ms.ui.UISlider.getSelectionStart could not be converted
com.ms.ui.UISlider.setSelection could not be converted
com.ms.ui.UISlider.setSelectionEnd could not be converted
com.ms.ui.UISlider.setSelectionStart could not be converted
com.ms.ui.UISpinner could not be converted
com.ms.ui.UISpinner.<Direction> could not be converted
com.ms.ui.UISpinner.<Type> could not be converted
com.ms.ui.UISpinner.add could not be converted
com.ms.ui.UISpinner.getLayoutComponent could not be converted
com.ms.ui.UISpinner.getMinimumSize could not be converted
com.ms.ui.UISpinner.getPreferredSize could not be converted
com.ms.ui.UISpinner.getRoleCode could not be converted
com.ms.ui.UISpinner.getScrollPos could not be converted
com.ms.ui.UISpinner.keyDown could not be converted
com.ms.ui.UISpinner.layout could not be converted
com.ms.ui.UISpinner.RAISED could not be converted
com.ms.ui.UISpinner.requestFocus could not be converted
com.ms.ui.UISpinner.scrollLineDown could not be converted
com.ms.ui.UISpinner.scrollLineUp could not be converted
com.ms.ui.UISpinner.scrollPageDown could not be converted

com.ms.ui.UISpinner.scrollPageUp could not be converted
com.ms.ui.UISpinner.setScrollInfo could not be converted
com.ms.ui.UISpinner.setStyle could not be converted
com.ms.ui.UISpinner.UISpinner(int) could not be converted
com.ms.ui.UISpinner.UISpinner(int, int, int, int, int, int) could not be converted
com.ms.ui.UISpinnerEdit.BORDER could not be converted
com.ms.ui.UISpinnerEdit.CENTER could not be converted
com.ms.ui.UISpinnerEdit.getEditStyle could not be converted
com.ms.ui.UISpinnerEdit.setEditStyle could not be converted
com.ms.ui.UISpinnerEdit.UISpinnerEdit(int, int) could not be converted
com.ms.ui.UISpinnerEdit.UISpinnerEdit(int, int, int, int, int, int, int) could not be converted
com.ms.ui.UISplitLayout could not be converted
com.ms.ui.UISplitViewer could not be converted
com.ms.ui.UISplitViewer.<Type> could not be converted
com.ms.ui.UISplitViewer.getComponent could not be converted
com.ms.ui.UISplitViewer.getFloater could not be converted
com.ms.ui.UISplitViewer.getPos could not be converted
com.ms.ui.UISplitViewer.getStyle could not be converted
com.ms.ui.UISplitViewer.mouseDown could not be converted
com.ms.ui.UISplitViewer.mouseDrag could not be converted
com.ms.ui.UISplitViewer.mouseUp could not be converted
com.ms.ui.UISplitViewer.setPos could not be converted
com.ms.ui.UISplitViewer.UISplitViewer could not be converted
com.ms.ui.UIShadowComponent.adjustLayoutSize could not be converted
com.ms.ui.UIShadowComponent.disableEvents could not be converted
com.ms.ui.UIShadowComponent.enableEvents could not be converted
com.ms.ui.UIShadowComponent.getCachedPreferredSize could not be converted
com.ms.ui.UIShadowComponent.getConvertedEvent could not be converted
com.ms.ui.UIShadowComponent.getFlags could not be converted
com.ms.ui.UIShadowComponent.HEADER could not be converted
com.ms.ui.UIShadowComponent.isInvalidating could not be converted
com.ms.ui.UIShadowComponent.isRedrawing could not be converted
com.ms.ui.UIShadowComponent.isSelected could not be converted
com.ms.ui.UIShadowComponent.isValid could not be converted
com.ms.ui.UIShadowComponent.listeners could not be converted
com.ms.ui.UIShadowComponent.obtainListenerTracker could not be converted
com.ms.ui.UIShadowComponent.recalcPreferredSize could not be converted
com.ms.ui.UIShadowComponent.setChecked could not be converted
com.ms.ui.UIShadowComponent.setFlags could not be converted
com.ms.ui.UIShadowComponent.setHot could not be converted

com.ms.ui.UIStateComponent.setIndeterminate could not be converted
com.ms.ui.UIStateComponent.setInvalidating could not be converted
com.ms.ui.UIStateComponent.setPressed could not be converted
com.ms.ui.UIStateComponent.setRedrawing could not be converted
com.ms.ui.UIStateComponent.setReparent could not be converted
com.ms.ui.UIStateComponent.setSelected could not be converted
com.ms.ui.UIStateComponent.setUsingWindowsLook could not be converted
com.ms.ui.UIStateComponent.setValid could not be converted
com.ms.ui.UIStateContainer.add could not be converted
com.ms.ui.UIStateContainer.adjustLayoutSize could not be converted
com.ms.ui.UIStateContainer.disableEvents could not be converted
com.ms.ui.UIStateContainer.enableEvents could not be converted
com.ms.ui.UIStateContainer.getBackground could not be converted
com.ms.ui.UIStateContainer.getCachedPreferredSize could not be converted
com.ms.ui.UIStateContainer.getConvertedEvent could not be converted
com.ms.ui.UIStateContainer.getCursor could not be converted
com.ms.ui.UIStateContainer.getEdge could not be converted
com.ms.ui.UIStateContainer.getFlags could not be converted
com.ms.ui.UIStateContainer.getFont could not be converted
com.ms.ui.UIStateContainer.getForeground could not be converted
com.ms.ui.UIStateContainer.getIndex could not be converted
com.ms.ui.UIStateContainer.getParent could not be converted
com.ms.ui.UIStateContainer.isChecked could not be converted
com.ms.ui.UIStateContainer.isEnabled could not be converted
com.ms.ui.UIStateContainer.isFocused could not be converted
com.ms.ui.UIStateContainer.isInvalidating could not be converted
com.ms.ui.UIStateContainer.isRedrawing could not be converted
com.ms.ui.UIStateContainer.isSelected could not be converted
com.ms.ui.UIStateContainer.isValid could not be converted
com.ms.ui.UIStateContainer.isVisible could not be converted
com.ms.ui.UIStateContainer.listeners could not be converted
com.ms.ui.UIStateContainer.obtainConvertedEvent could not be converted
com.ms.ui.UIStateContainer.obtainListenerTracker could not be converted
com.ms.ui.UIStateContainer.recalcPreferredSize could not be converted
com.ms.ui.UIStateContainer.setBackground could not be converted
com.ms.ui.UIStateContainer.setChecked could not be converted
com.ms.ui.UIStateContainer.setCursor could not be converted
com.ms.ui.UIStateContainer.setEdge could not be converted
com.ms.ui.UIStateContainer.setEnabled could not be converted
com.ms.ui.UIStateContainer.setFlags could not be converted

com.ms.ui.UIStateContainer.setFont could not be converted

com.ms.ui.UIStateContainer.setForeground could not be converted

com.ms.ui.UIStateContainer.setHot could not be converted

com.ms.ui.UIStateContainer.setIndeterminate could not be converted

com.ms.ui.UIStateContainer.setIndex could not be converted

com.ms.ui.UIStateContainer.setInvalidating could not be converted

com.ms.ui.UIStateContainer.setParent could not be converted

com.ms.ui.UIStateContainer.setPressed could not be converted

com.ms.ui.UIStateContainer.setRedrawing could not be converted

com.ms.ui.UIStateContainer.setReparent could not be converted

com.ms.ui.UIStateContainer.setSelected could not be converted

com.ms.ui.UIStateContainer.setUsingWindowsLook could not be converted

com.ms.ui.UIStateContainer.setValid could not be converted

com.ms.ui.UIStateContainer.setVisible could not be converted

com.ms.ui.UIStateContainer.UIStateContainer could not be converted

com.ms.ui.UIStatic could not be converted

com.ms.ui.UIStatic.HCENTER could not be converted

com.ms.ui.UIStatic.setFlags could not be converted

com.ms.ui.UIStatic.VCENTER could not be converted

com.ms.ui.UISystem could not be converted

com.ms.ui.UITab.paint could not be converted

com.ms.ui.UITab.setSelected could not be converted

com.ms.ui.UITab.UITab could not be converted

com.ms.ui.UITabLayout could not be converted

com.ms.ui.UITabList could not be converted

com.ms.ui.UITabList.add(UIComponent) could not be converted

com.ms.ui.UITabList.add(UIComponent, int) could not be converted

com.ms.ui.UITabList.add(String) could not be converted

com.ms.ui.UITabList.add(String, int) could not be converted

com.ms.ui.UITabList.layout could not be converted

com.ms.ui.UITabList.paint could not be converted

com.ms.ui.UITabList.passFocus could not be converted

com.ms.ui.UITabList.setSelectedItem could not be converted

com.ms.ui.UITabListLayout could not be converted

com.ms.ui.UITabViewer.add could not be converted

com.ms.ui.UITabViewer.addTab(UIComponent, UIComponent) could not be converted

com.ms.ui.UITabViewer.getConvertedEvent could not be converted

com.ms.ui.UITabViewer.paint could not be converted

com.ms.ui.UITabViewer.removeTab could not be converted

com.ms.ui.UITabViewer.requestFocus could not be converted

com.ms.ui.UITabViewer.setSelectedItem could not be converted

com.ms.ui.UIText.getPreferredSize could not be converted

com.ms.ui.UIText.paint could not be converted

com.ms.ui.UIText.setFocused could not be converted

com.ms.ui.UIText.setHot could not be converted

com.ms.ui.UIText.setSelected could not be converted

com.ms.ui.UIThreePanelLayout could not be converted

com.ms.ui.UIThumb could not be converted

com.ms.ui.UITree could not be converted

com.ms.ui.UITree.action could not be converted

com.ms.ui.UITree.add(Component) could not be converted

com.ms.ui.UITree.add(Component, int) could not be converted

com.ms.ui.UITree.add(Image, String, int) could not be converted

com.ms.ui.UITree.add(String, int) could not be converted

com.ms.ui.UITree.add(UIComponent) could not be converted

com.ms.ui.UITree.getAttachRect could not be converted

com.ms.ui.UITree.getExpander could not be converted

com.ms.ui.UITree.keyDown could not be converted

com.ms.ui.UITree.remove could not be converted

com.ms.ui.UITree.setChecked could not be converted

com.ms.ui.UITree.setExpanded could not be converted

com.ms.ui.UITree.setExpander could not be converted

com.ms.ui.UITree.setLayout could not be converted

com.ms.ui.UITreeLayout could not be converted

com.ms.ui.UIVerticalFlowLayout could not be converted

com.ms.ui.UIViewer.ensureVisible could not be converted

com.ms.ui.UIViewer.isOverlapping could not be converted

com.ms.ui.UIViewer.mouseDown could not be converted

com.ms.ui.UIViewer.mouseDrag could not be converted

com.ms.ui.UIViewer.mouseUp could not be converted

com.ms.ui.UIViewer.requestFocus could not be converted

com.ms.ui.UIViewer.timeTriggered could not be converted

com.ms.ui.UIWindow.keyDown could not be converted

com.ms.ui.UIWindow.pack could not be converted

com.ms.ui.UIWindow.setSize could not be converted

com.ms.ui.UIWinEvent could not be converted

com.ms.ui.UIWizard could not be converted

com.ms.ui.UIWizardStep could not be converted

com.ms.ui.Version could not be converted

Com.ms.util Error Messages

com.ms.util.ArraySort.compare could not be converted
com.ms.util.ArraySort.sort could not be converted
com.ms.util.ArraySort.swap could not be converted
com.ms.util.cab.CabEnumerator could not be converted
com.ms.util.cab.CabException could not be converted
com.ms.util.cab.CabFileEntry could not be converted
com.ms.util.cab.CabFolderEntry could not be converted
com.ms.util.cab.CabProgressInterface could not be converted
com.ms.util.EventLog.reportEvent could not be converted
com.ms.util.HTMLTokenizer could not be converted
com.ms.util.IntRangeComparator could not be converted
com.ms.util.IncludeExcludeIntRanges could not be converted
com.ms.util.IncludeExcludeWildcards could not be converted
com.ms.util.ini.IniFile could not be converted
com.ms.util.ini.IniSection could not be converted
com.ms.util.ini.IniSyntaxErrorException could not be converted
com.ms.util.IntRanges.compare could not be converted
com.ms.util.IntRanges.compareSet could not be converted
com.ms.util.IntRanges.ComparisonResultToString could not be converted
com.ms.util.IntRanges.condense could not be converted
com.ms.util.IntRanges.contains could not be converted
com.ms.util.IntRanges.indexOf could not be converted
com.ms.util.IntRanges.intersect could not be converted
com.ms.util.IntRanges.IntRanges could not be converted
com.ms.util.IntRanges.invertComparisonResult could not be converted
com.ms.util.IntRanges.lock could not be converted
com.ms.util.IntRanges.parse could not be converted
com.ms.util.IntRanges.removeRange could not be converted
com.ms.util.IntRanges.removeRanges could not be converted
com.ms.util.IntRanges.removeRanges(int, int) could not be converted
com.ms.util.IntRanges.removeRanges(int, int, IntRangeComparator) could not be converted
com.ms.util.IntRanges.removeSingleton could not be converted
com.ms.util.IntRanges.size could not be converted
com.ms.util.IntRanges.sort could not be converted
com.ms.util.IntRanges.unlock could not be converted
com.ms.util.IWildcardExpressionComparator could not be converted
com.ms.util.OrdinalMap.insertInCognate could not be converted

com.ms.util.OrdinalMap.moveCognate could not be converted
com.ms.util.OrdinalMap.newCognate could not be converted
com.ms.util.OrdinalMap.removeFromCognate could not be converted
com.ms.util.ProvideSetComparisonInfo could not be converted
com.ms.util.Queue.capacity could not be converted
com.ms.util.Queue.elementFromHead could not be converted
com.ms.util.Queue.elementFromTail could not be converted
com.ms.util.Queue.hasMoreElements could not be converted
com.ms.util.Queue.nextElement could not be converted
com.ms.util.Queue.setCapacity could not be converted
com.ms.util.Queue.toString could not be converted
com.ms.util.SetComparer could not be converted
com.ms.util.SetComparison could not be converted
com.ms.util.SetTemplate(foundAt could not be converted
com.ms.util.SetTemplate(foundCognate could not be converted
com.ms.util.SetTemplate(foundIn could not be converted
com.ms.util.SetTemplate.generation could not be converted
com.ms.util.SetTemplate.insertInCognate could not be converted
com.ms.util.SetTemplate.keys could not be converted
com.ms.util.SetTemplate.locate could not be converted
com.ms.util.SetTemplate.moveCognate could not be converted
com.ms.util.SetTemplate.newCognate could not be converted
com.ms.util.SetTemplate.rehash could not be converted
com.ms.util.SetTemplate.removeFromCognate could not be converted
com.ms.util.SetTemplate.reportFinds could not be converted
com.ms.util.SetTemplate.reportRequests could not be converted
com.ms.util.SetTemplate.reportUnits could not be converted
com.ms.util.SetTemplate.SetTemplate could not be converted
com.ms.util.Sort.compare could not be converted
com.ms.util.Sort.doSort could not be converted
com.ms.util.Sort.swap could not be converted
com.ms.util.StringComparison.ascending could not be converted
com.ms.util.StringComparison.descending could not be converted
com.ms.util.SystemVersionManager could not be converted
com.ms.util.Task could not be converted
com.ms.util.TaskManager could not be converted
com.ms.util.ThreadLocalStorage could not be converted
com.ms.util.Timer.getRepeat could not be converted
com.ms.util.Timer.getUserID could not be converted
com.ms.util.Timer.Timer could not be converted

com.ms.util.TimerEvent.getSource could not be converted
com.ms.util.TimerEvent.getTime could not be converted
com.ms.util.TimerEvent.getUserID could not be converted
com.ms.util.TimerListener could not be converted
com.ms.util.UnsignedIntRanges.contains could not be converted
com.ms.util.UnsignedIntRanges.indexOf could not be converted
com.ms.util.UnsignedIntRanges.intersect could not be converted
com.ms.util.UnsignedIntRanges.removeRange could not be converted
com.ms.util.UnsignedIntRanges.UnsignedIntRanges could not be converted
com.ms.util.VectorSort.compare could not be converted
com.ms.util.VectorSort.swap could not be converted
com.ms.util.WildcardExpression could not be converted
com.ms.util.zip.ZipInputStreamEx could not be converted
com.ms.util.zip.ZipOutputStreamEx could not be converted

Com.ms.wfc Error Messages

[com.ms.wfc.Rectangle.Editor](#) could not be converted

Com.ms.wfc.app Error Messages

com.ms.wfc.app.Application.addOnSettingChange could not be converted

com.ms.wfc.app.Application.addOnSystemShutdown could not be converted

com.ms.wfc.app.Application.allocThreadStorage could not be converted

com.ms.wfc.app.Application.createThread could not be converted

com.ms.wfc.app.Application.doEvents could not be converted

com.ms.wfc.app.Application.freeThreadStorage could not be converted

com.ms.wfc.app.Application.getParkingForm could not be converted

com.ms.wfc.app.Application.getThreadStorage could not be converted

com.ms.wfc.app.Application.removeOnSettingChange could not be converted

com.ms.wfc.app.Application.removeOnSystemShutdown could not be converted

com.ms.wfc.app.Application.runDialog could not be converted

com.ms.wfc.app.Application.setThreadStorage could not be converted

com.ms.wfc.app.CharacterSet could not be converted

com.ms.wfc.app.Clipboard.Clipboard could not be converted

com.ms.wfc.app.DataFormats.CF_CSV could not be converted

com.ms.wfc.app.DataFormats.CF_WFCOBJECT could not be converted

com.ms.wfc.app.DataFormats.DataFormats could not be converted

com.ms.wfc.app.DataFormats.Format.Format could not be converted

com.ms.wfc.app.DataFormats.Format.win32Handle could not be converted

com.ms.wfc.app.DataObject.OleDAdvise could not be converted

com.ms.wfc.app.DataObject.OleDUnadvise could not be converted

com.ms.wfc.app.DataObject.OleEnumDAdvise could not be converted

com.ms.wfc.app.DataObject.OleEnumFormatEtc could not be converted

com.ms.wfc.app.DataObject.OleGetCanonicalFormatEtc could not be converted

com.ms.wfc.app.DataObject.OleGetData could not be converted

com.ms.wfc.app.DataObject.OleGetDataHere could not be converted

com.ms.wfc.app.DataObject.OleQueryGetData could not be converted

com.ms.wfc.app.DataObject.OleSetData could not be converted

com.ms.wfc.app.IMessageFilter.postFilterMessage could not be converted

com.ms.wfc.app.KeywordVk could not be converted

com.ms.wfc.app.Languages.Languages could not be converted

com.ms.wfc.app.Locale.CalendarType could not be converted

com.ms.wfc.app.Locale.compareStrings could not be converted

com.ms.wfc.app.Locale.DateFormatOrder could not be converted

com.ms.wfc.app.Locale.getCalendarType could not be converted

com.ms.wfc.app.Locale.getCenturyFormat could not be converted

com.ms.wfc.app.Locale.getCharacterSet could not be converted

com.ms.wfc.app.Locale.getCompareIgnoreCase could not be converted
com.ms.wfc.app.Locale.getCompareIgnoreKana could not be converted
com.ms.wfc.app.Locale.getCompareIgnoreKashida could not be converted
com.ms.wfc.app.Locale.getCompareIgnoreNonSpace could not be converted
com.ms.wfc.app.Locale.getCompareIgnoreSymbols could not be converted
com.ms.wfc.app.Locale.getCompareIgnoreWidth could not be converted
com.ms.wfc.app.Locale.getCountryCode could not be converted
com.ms.wfc.app.Locale.getDefaultCountry could not be converted
com.ms.wfc.app.Locale.getEnglishCurrencyName could not be converted
com.ms.wfc.app.Locale.getLeadingZero could not be converted
com.ms.wfc.app.Locale.getNativeDigits could not be converted
com.ms.wfc.app.Locale.getSortId could not be converted
com.ms.wfc.app.Locale.getSupportedLocales could not be converted
com.ms.wfc.app.Locale.Languages.Locale.Languages could not be converted
com.ms.wfc.app.Locale.Languages.RHAETO_ROMAN could not be converted
com.ms.wfc.app.Locale.LeadngZeros could not be converted
com.ms.wfc.app.Locale.Locale could not be converted
com.ms.wfc.app.Locale.MeasurementSystem could not be converted
com.ms.wfc.app.Locale.NegativeNumberMode.Locale.NegativeNumberMode could not be converted
com.ms.wfc.app.Locale.OptionalCalendarType could not be converted
com.ms.wfc.app.Locale.PositiveCurrencyMode.Locale.PositiveCurrencyMode could not be converted
com.ms.wfc.app.Locale.setCalendarType could not be converted
com.ms.wfc.app.Locale.setCompareIgnoreCase could not be converted
com.ms.wfc.app.Locale.setCompareIgnoreKana could not be converted
com.ms.wfc.app.Locale.setCompareIgnoreKashida could not be converted
com.ms.wfc.app.Locale.setCompareIgnoreNonSpace could not be converted
com.ms.wfc.app.Locale.setCompareIgnoreSymbols could not be converted
com.ms.wfc.app.Locale.setCompareIgnoreWidth could not be converted
com.ms.wfc.app.Locale.setFirstDayOfWeek could not be converted
com.ms.wfc.app.Locale.setFirstWeekOfYear could not be converted
com.ms.wfc.app.Locale.setLeadingZero could not be converted
com.ms.wfc.app.Locale.setListSeparator could not be converted
com.ms.wfc.app.Locale.setMeasurementSystem could not be converted
com.ms.wfc.app.Locale.Sort could not be converted
com.ms.wfc.app.Locale.SubLanguages.CHINESE_SIMPLIFIED could not be converted
com.ms.wfc.app.Locale.SubLanguages.CHINESE_TRADITIONAL could not be converted
com.ms.wfc.app.Locale.SubLanguages.DUTCH_BELGIAN could not be converted
com.ms.wfc.app.Locale.SubLanguages.ENGLISH_AUS could not be converted
com.ms.wfc.app.Locale.SubLanguages.ENGLISH_CAN could not be converted
com.ms.wfc.app.Locale.SubLanguages.ENGLISH_EIRE could not be converted

com.ms.wfc.app.Locale.SubLanguages.ENGLISH_NZ could not be converted
com.ms.wfc.app.Locale.SubLanguages.ENGLISH_UK could not be converted
com.ms.wfc.app.Locale.SubLanguages.ENGLISH_US could not be converted
com.ms.wfc.app.Locale.SubLanguages.FRENCH_BELGIAN could not be converted
com.ms.wfc.app.Locale.SubLanguages.FRENCH_CANADIAN could not be converted
com.ms.wfc.app.Locale.SubLanguages.FRENCH_SWISS could not be converted
com.ms.wfc.app.Locale.SubLanguages.GERMAN_AUSTRIAN could not be converted
com.ms.wfc.app.Locale.SubLanguages.GERMAN_SWISS could not be converted
com.ms.wfc.app.Locale.SubLanguages.ITALIAN_SWISS could not be converted
com.ms.wfc.app.Locale.SubLanguages.Locale.SubLanguages could not be converted
com.ms.wfc.app.Locale.SubLanguages.NORWEGIAN_BOKMAL could not be converted
com.ms.wfc.app.Locale.SubLanguages.NORWEGIAN_NYNORSK could not be converted
com.ms.wfc.app.Locale.SubLanguages.PORTUGUESE_BRAZILIAN could not be converted
com.ms.wfc.app.Locale.SubLanguages.SERBO_CROATIAN_CYRILLIC could not be converted
com.ms.wfc.app.Locale.SubLanguages.SERBO_CROATIAN_LATIN could not be converted
com.ms.wfc.app.Locale.SubLanguages.SPANISH_MEXICAN could not be converted
com.ms.wfc.app.Locale.SubLanguages.SPANISH_MODERN could not be converted
com.ms.wfc.app.Locale.SubLanguages.SubLanguages could not be converted
com.ms.wfc.app.Locale.SubLanguages.SYS_DEFAULT could not be converted
com.ms.wfc.app.Message.free could not be converted
com.ms.wfc.app.MethodInvoker.MethodInvoker could not be converted
com.ms.wfc.app.NegativeNumberMode.NegativeNumberMode could not be converted
com.ms.wfc.app.PositiveCurrencyMode.PositiveCurrencyMode could not be converted
com.ms.wfc.app.Registry.Registry could not be converted
com.ms.wfc.app.RegistryKey.getBaseKey could not be converted
com.ms.wfc.app.SendKeysHookProc could not be converted
com.ms.wfc.app.SKEvent could not be converted
com.ms.wfc.app.SubLanguages.SubLanguages could not be converted
com.ms.wfc.app.SystemInformation.getArrange could not be converted
com.ms.wfc.app.ThreadExceptionHandler.ThreadExceptionHandler could not be converted
com.ms.wfc.app.Time.getConstructorArgs
com.ms.wfc.app.Time.getExtension could not be converted
com.ms.wfc.app.Time.save could not be converted
com.ms.wfc.app.Time.Time could not be converted
com.ms.wfc.app.Time.toSystemTime could not be converted
com.ms.wfc.app.Time.toVariant could not be converted
com.ms.wfc.app.Window.callback could not be converted

Com.ms.wfc.ax Error Messages

com.ms.wfc.ax could not be converted

com.ms.wfc.ax._POINTL could not be converted

com.ms.wfc.ax.ActiveX could not be converted

com.ms.wfc.ax.Ambients could not be converted

com.ms.wfc.ax.IAdviseSink could not be converted

com.ms.wfc.ax.ICategorizeProperties could not be converted

com.ms.wfc.ax.IClassFactory could not be converted

com.ms.wfc.ax.IClassFactory2 could not be converted

com.ms.wfc.ax.IDDispatch could not be converted

com.ms.wfc.ax.IEDispatch could not be converted

com.ms.wfc.ax.IEnumOLEVERB could not be converted

com.ms.wfc.ax.IEnumUnknown could not be converted

com.ms.wfc.ax.IExtender could not be converted

com.ms.wfc.ax.IFont could not be converted

com.ms.wfc.ax.IFontDisp could not be converted

com.ms.wfc.ax.IGetOleObject could not be converted

com.ms.wfc.ax.IGetVBAObject could not be converted

com.ms.wfc.ax.IMyDispatch could not be converted

com.ms.wfc.ax.IObjectIdentity could not be converted

com.ms.wfc.ax.IObjectWithSite could not be converted

com.ms.wfc.ax.IOleClientSite could not be converted

com.ms.wfc.ax.IOleContainer could not be converted

com.ms.wfc.ax.IOleControl could not be converted

com.ms.wfc.ax.IOleControlSite could not be converted

com.ms.wfc.ax.IOleInPlaceActiveObject could not be converted

com.ms.wfc.ax.IOleInPlaceFrame could not be converted

com.ms.wfc.ax.IOleInPlaceObject could not be converted

com.ms.wfc.ax.IOleInPlaceSite could not be converted

com.ms.wfc.ax.IOleInPlaceUIWindow could not be converted

com.ms.wfc.ax.IOleWindow could not be converted

com.ms.wfc.ax.IParseDisplayName could not be converted

com.ms.wfc.ax.IPerPropertyBrowsing could not be converted

com.ms.wfc.ax.IPersist could not be converted

com.ms.wfc.ax.IPersist.iid could not be converted

com.ms.wfc.ax.IPersistPropertyBag could not be converted

com.ms.wfc.ax.IPersistStorage could not be converted

com.ms.wfc.ax.IPersistStream could not be converted

com.ms.wfc.ax.IPersistStreamInit could not be converted
com.ms.wfc.ax.IPicture could not be converted
com.ms.wfc.ax.IPictureDisp could not be converted
com.ms.wfc.ax.IPropertyBag could not be converted
com.ms.wfc.ax.IQuickActivate could not be converted
com.ms.wfc.ax.ISimpleFrameSite could not be converted
com.ms.wfc.ax.ISpecifyPropertyPages could not be converted
com.ms.wfc.ax.IVBFormat could not be converted
com.ms.wfc.ax.IVBGetControl could not be converted
com.ms.wfc.ax.tagCADWORD could not be converted
com.ms.wfc.ax.tagCALPOLESTR could not be converted
com.ms.wfc.ax.tagCAUUID could not be converted
com.ms.wfc.ax.tagCONTROLINFO could not be converted
com.ms.wfc.ax.tagDISPPARAMS could not be converted
com.ms.wfc.ax.tagLICINFO could not be converted
com.ms.wfc.ax.tagLOGPALETTE could not be converted
com.ms.wfc.ax.tagMSG could not be converted
com.ms.wfc.ax.tagOIFI could not be converted
com.ms.wfc.ax.tagOleMenuGroupWidths could not be converted
com.ms.wfc.ax.tagOLEVERB could not be converted
com.ms.wfc.ax.tagPOINTF could not be converted
com.ms.wfc.ax.tagQACONTAINER could not be converted
com.ms.wfc.ax.tagQACONTROL could not be converted
com.ms.wfc.ax.tagRECT could not be converted
com.ms.wfc.ax.tagSIZE could not be converted
com.ms.wfc.ax.tagSIZEL could not be converted

Com.ms.wfc.core Error Messages

com.ms.wfc.core.ArrayDialog could not be converted

com.ms.wfc.core.ArrayEditor.createNewInstance could not be converted

com.ms.wfc.core.ArrayEditor.defaultPropInfo could not be converted

com.ms.wfc.core.ArrayEditor.editValue could not be converted

com.ms.wfc.core.ArrayEditor.getBaseName could not be converted

com.ms.wfc.core.ArrayEditor.getDisplayText could not be converted

com.ms.wfc.core.ArrayEditor.getTextFromValue could not be converted

com.ms.wfc.core.ArrayEditor.getTypeDescription could not be converted

com.ms.wfc.core.BooleanEditor.getStyle could not be converted

com.ms.wfc.core.BooleanEditor.getValueFromText could not be converted

com.ms.wfc.core.CancelEventHandler.CancelEventHandler could not be converted

com.ms.wfc.core.CategoryAttribute.Position could not be converted

com.ms.wfc.core.Component.appendEventHandlers could not be converted

com.ms.wfc.core.Component.componentChanged could not be converted

com.ms.wfc.core.Component.getDisposing could not be converted

com.ms.wfc.core.Component.getResource could not be converted

com.ms.wfc.core.Component.getService could not be converted

com.ms.wfc.core.ComponentInfo.getClassInfo could not be converted

com.ms.wfc.core.ComponentInfo.getDefaultEventInfo could not be converted

com.ms.wfc.core.ComponentInfo.getExtenders could not be converted

com.ms.wfc.core.ComponentManager.createClassInfo could not be converted

com.ms.wfc.core.ComponentManager.createValueEditor could not be converted

com.ms.wfc.core.ComponentManager.getComponentInfo could not be converted

com.ms.wfc.core.ComponentManager.getValueEditor could not be converted

com.ms.wfc.core.ComponentManager.registerClassInfo could not be converted

com.ms.wfc.core.ComponentManager.registerValueEditorClass could not be converted

com.ms.wfc.core.ConstructorArg could not be converted

com.ms.wfc.core.ConstructorArg.ConstructorArg could not be converted

com.ms.wfc.core.CustomizerVerb.addOnVerbExecute could not be converted

com.ms.wfc.core.CustomizerVerb.CustomizerVerb could not be converted

com.ms.wfc.core.CustomizerVerb.getBitmap could not be converted

com.ms.wfc.core.CustomizerVerb.getData could not be converted

com.ms.wfc.core.CustomizerVerb.onVerbExecute could not be converted

com.ms.wfc.core.CustomizerVerb.performVerbExecute could not be converted

com.ms.wfc.core.CustomizerVerb.removeOnVerbExecute could not be converted

com.ms.wfc.core.CustomizerVerb.setBitmap could not be converted

com.ms.wfc.core.CustomizerVerb.setData could not be converted

com.ms.wfc.core.DescriptionAttribute.getDescription could not be converted
com.ms.wfc.core.DesignForm.DesignForm could not be converted
com.ms.wfc.core.DesignForm.getPage could not be converted
com.ms.wfc.core.DesignForm.getPageCount could not be converted
com.ms.wfc.core.DesignForm.showForm could not be converted
com.ms.wfc.core.DesignPage.getPageMessage could not be converted
com.ms.wfc.core.DesignPage.NULLVALUE could not be converted
com.ms.wfc.core.DesignPage.objects could not be converted
com.ms.wfc.core.DesignPage.onReadProperty could not be converted
com.ms.wfc.core.DesignPage.onWriteProperty could not be converted
com.ms.wfc.core.DesignPage.properties could not be converted
com.ms.wfc.core.DesignPage.setObjects could not be converted
com.ms.wfc.core.DoubleEditor.getValueFromText could not be converted
com.ms.wfc.core.Enum.Enum could not be converted
com.ms.wfc.core.Event.Event could not be converted
com.ms.wfc.core.Event.extendedInfo could not be converted
com.ms.wfc.core.EventHandler.EventHandler could not be converted
com.ms.wfc.core.EventInfo.addEventHandler could not be converted
com.ms.wfc.core.EventInfo.getAddMethod could not be converted
com.ms.wfc.core.EventInfo.getMulticast could not be converted
com.ms.wfc.core.EventInfo.getRemoveMethod could not be converted
com.ms.wfc.core.EventInfo.getType could not be converted
com.ms.wfc.core.EventInfo.removeEventHandler could not be converted
com.ms.wfc.core.ExtenderInfo could not be converted
com.ms.wfc.core.FieldsEditor could not be converted
com.ms.wfc.core.FloatEditor.getValueFromText could not be converted
com.ms.wfc.core.IAttributes could not be converted
com.ms.wfc.core.IClassInfo could not be converted
com.ms.wfc.core.IComponent.dispose could not be converted
com.ms.wfc.core.IComponent.getChildOf could not be converted
com.ms.wfc.core.IComponentSite.componentChanged could not be converted
com.ms.wfc.core.IComponentSite.getDisposing could not be converted
com.ms.wfc.core.IConstructable could not be converted
com.ms.wfc.core.IContainer.dispose could not be converted
com.ms.wfc.core.IContainer.getComponent could not be converted
com.ms.wfc.core.IContainer.getComponents could not be converted
com.ms.wfc.core.ICustomizer.getDesignPages could not be converted
com.ms.wfc.core.ICustomizer.getHitTest could not be converted
com.ms.wfc.core.ICustomizer.getVerbs could not be converted
com.ms.wfc.core.IDesignPage.getSize could not be converted

com.ms.wfc.core.IDesignPage.setObjects could not be converted

com.ms.wfc.core.IDesignPage.setSize could not be converted

com.ms.wfc.core.IEditorHost.getHostHandle could not be converted

com.ms.wfc.core.IEditorSite.getType could not be converted

com.ms.wfc.core.IEditorSite.getValueOwner could not be converted

com.ms.wfc.core.IEditorSite.getValueOwners could not be converted

com.ms.wfc.core.IResourceLoader could not be converted

com.ms.wfc.core.IResourceManager.containsProperty could not be converted

com.ms.wfc.core.IResourceManager.getNames could not be converted

com.ms.wfc.core.IResourceManager.getProperties could not be converted

com.ms.wfc.core.IResourceManager.saveAllProperties could not be converted

com.ms.wfc.core.IResourceManager.setProperty could not be converted

com.ms.wfc.core.IResourceManager.setResourceSet could not be converted

com.ms.wfc.core.IValueEditor.getConstantName could not be converted

com.ms.wfc.core.IValueEditor.getValueFromSubPropertyValues could not be converted

com.ms.wfc.core.IValueEditor.STYLE_IMMEDIATE could not be converted

com.ms.wfc.core.IValueEditor.STYLE_NOARRAYEXPANSION could not be converted

com.ms.wfc.core.IValueEditor.STYLE_NOARRAYMULTISELECT could not be converted

com.ms.wfc.core.IValueEditor.STYLE_NOEDITABLETEXT could not be converted

com.ms.wfc.core.IValueEditor.STYLE_PAINTVALUE could not be converted

com.ms.wfc.core.IValueEditor.STYLE_PROPERTIES could not be converted

com.ms.wfc.core.IValueEditor.STYLE_SHOWFIELDS could not be converted

com.ms.wfc.core.IValueEditor.STYLE_VALUES could not be converted

com.ms.wfc.core.MemberAttribute.MemberAttribute could not be converted

com.ms.wfc.core.NonEditableReferenceEditor.getStyle could not be converted

com.ms.wfc.core.NonEditableReferenceEditor.NonEditableReferenceEditor could not be converted

com.ms.wfc.core.NonPersistableAttribute could not be converted

com.ms.wfc.core.PropertyInfo.canResetValue could not be converted

com.ms.wfc.core.PropertyInfo.getGetMethod could not be converted

com.ms.wfc.core.PropertyInfo.getIsReadOnly could not be converted

com.ms.wfc.core.PropertyInfo.getResetMethod could not be converted

com.ms.wfc.core.PropertyInfo.getSetMethod could not be converted

com.ms.wfc.core.PropertyInfo.getShouldPersistMethod could not be converted

com.ms.wfc.core.PropertyInfo.getType could not be converted

com.ms.wfc.core.PropertyInfo.getValue could not be converted

com.ms.wfc.core.PropertyInfo.getValueEditor could not be converted

com.ms.wfc.core.PropertyInfo.resetValue could not be converted

com.ms.wfc.core.PropertyInfo.setValue could not be converted

com.ms.wfc.core.PropertyInfo.shouldPersistValue could not be converted

com.ms.wfc.core.ReferenceEditor.container could not be converted

com.ms.wfc.core.ReferenceEditor.getStyle could not be converted

com.ms.wfc.core.ReferenceEditor.getTextFromValue could not be converted

com.ms.wfc.core.ReferenceEditor.getValueFromText could not be converted

com.ms.wfc.core.ReferenceEditor.getValues could not be converted

com.ms.wfc.core.ReferenceEditor.none could not be converted

com.ms.wfc.core.ReferenceEditor.ReferenceEditor could not be converted

com.ms.wfc.core.ReferenceEditor.type could not be converted

com.ms.wfc.core.ResourceManager.containsProperty could not be converted

com.ms.wfc.core.ResourceManager.EXTENSION could not be converted

com.ms.wfc.core.ResourceManager.getLocaleFileNameFromBase could not be converted

com.ms.wfc.core.ResourceManager.getNames could not be converted

com.ms.wfc.core.ResourceManager.getProperties could not be converted

com.ms.wfc.core.ResourceManager.getResource could not be converted

com.ms.wfc.core.ResourceManager.getResourceLoader could not be converted

com.ms.wfc.core.ResourceManager.ResourceManager could not be converted

com.ms.wfc.core.ResourceManager.save could not be converted

com.ms.wfc.core.ResourceManager.saveAllProperties could not be converted

com.ms.wfc.core.ResourceManager.setBaseName could not be converted

com.ms.wfc.core.ResourceManager.setProperty could not be converted

com.ms.wfc.core.ResourceManager.setResourceLoader could not be converted

com.ms.wfc.core.ResourceManager.setResourceSet could not be converted

com.ms.wfc.core.ResourceReader could not be converted

com.ms.wfc.core.ResourceWriter could not be converted

com.ms.wfc.core.ShowInToolboxAttribute.NO could not be converted

com.ms.wfc.core.ShowInToolboxAttribute.YES could not be converted

com.ms.wfc.core.StringListDialog could not be converted

com.ms.wfc.core.StringListEditor could not be converted

com.ms.wfc.core.Sys could not be converted

com.ms.wfc.core.ValueEditor.editValue could not be converted

com.ms.wfc.core.ValueEditor.getValueFromSubPropertyValues could not be converted

com.ms.wfc.core.ValueEditorAttribute.valueEditorType could not be converted

com.ms.wfc.core.VerbExecuteEvent could not be converted

com.ms.wfc.core.VerbExecuteEventHandler could not be converted

com.ms.wfc.core.WFCException.getBaseException could not be converted

com.ms.wfc.core.WFCException.printStackTrace could not be converted

Com.ms.wfc.data Error Messages

com.ms.wfc.data.AdoEnums could not be converted

com.ms.wfc.data.AdoEvent could not be converted

com.ms.wfc.data.AdoException.AdoException could not be converted

com.ms.wfc.data.AdoProperties.AdoProperties could not be converted

com.ms.wfc.data.BooleanDataFormat could not be converted

com.ms.wfc.data.Connection.addOnBeginTransComplete could not be converted

com.ms.wfc.data.Connection.addOnCommitTransComplete could not be converted

com.ms.wfc.data.Connection.addOnConnectComplete could not be converted

com.ms.wfc.data.Connection.addOnDisconnect could not be converted

com.ms.wfc.data.Connection.addOnExecuteComplete could not be converted

com.ms.wfc.data.Connection.addOnInfoMessage could not be converted

com.ms.wfc.data.Connection.addOnRollbackTransComplete could not be converted

com.ms.wfc.data.Connection.addOnWillConnect could not be converted

com.ms.wfc.data.Connection.addOnWillExecute could not be converted

com.ms.wfc.data.Connection.Connection could not be converted

com.ms.wfc.data.Connection.getPeer could not be converted

com.ms.wfc.data.Connection.openSchema could not be converted

com.ms.wfc.data.Connection.removeOnBeginTransComplete could not be converted

com.ms.wfc.data.Connection.removeOnCommitTransComplete could not be converted

com.ms.wfc.data.Connection.removeOnConnectComplete could not be converted

com.ms.wfc.data.Connection.removeOnDisconnect could not be converted

com.ms.wfc.data.Connection.removeOnExecuteComplete could not be converted

com.ms.wfc.data.Connection.removeOnInfoMessage could not be converted

com.ms.wfc.data.Connection.removeOnRollbackTransComplete could not be converted

com.ms.wfc.data.Connection.removeOnWillConnect could not be converted

com.ms.wfc.data.Connection.removeOnWillExecute could not be converted

com.ms.wfc.data.Connection.setConnectionString could not be converted

com.ms.wfc.data.ConnectionEvent could not be converted

com.ms.wfc.data.ConnectionEventHandler could not be converted

com.ms.wfc.data.DataFormat could not be converted

com.ms.wfc.data.DataFormat.FormatWrapper could not be converted

com.ms.wfc.data.DateDataFormat could not be converted

com.ms.wfc.data.DateFormat.valid could not be converted

com.ms.wfc.data.dsl.<ClassName>.clsid could not be converted

com.ms.wfc.data.dsl.<ClassName>.iid could not be converted

com.ms.wfc.data.dsl._COAUTHIDENTITY._COAUTHIDENTITY could not be converted

com.ms.wfc.data.dsl._COAUTHINFO._COAUTHINFO could not be converted

com.ms.wfc.data.dsl._COSERVERINFO._COSERVERINFO could not be converted

com.ms.wfc.data.dsl._RemotableHandle._RemotableHandle could not be converted

com.ms.wfc.data.dsl.tagMULTI_QI could not be converted

com.ms.wfc.data.EventHandlerList.clear could not be converted

com.ms.wfc.data.EventHandlerList.getList could not be converted

com.ms.wfc.data.EventsListener could not be converted

com.ms.wfc.data.Field.getDataTimestamp could not be converted

com.ms.wfc.data.Field.getDispatch could not be converted

com.ms.wfc.data.Field.getGuid could not be converted

com.ms.wfc.data.Field.getInt could not be converted could not be converted

com.ms.wfc.data.Field.getObject could not be converted

com.ms.wfc.data.Field.getTime could not be converted

com.ms.wfc.data.Field.getTimestamp could not be converted

com.ms.wfc.data.Field.setDataDate could not be converted

com.ms.wfc.data.Field.setDispatch could not be converted

com.ms.wfc.data.Field.setGuid could not be converted

com.ms.wfc.data.Field.setTime could not be converted

com.ms.wfc.data.Field.setTimestamp could not be converted

com.ms.wfc.data.IDataFormat could not be converted

com.ms.wfc.data.IDataFormat.Editor could not be converted

com.ms.wfc.data.IDataSource.addDataSourceListener could not be converted

com.ms.wfc.data.IDataSource.getDataMember could not be converted

com.ms.wfc.data.IDataSource.getDataMemberCount could not be converted

com.ms.wfc.data.IDataSource.getDataMemberName could not be converted

com.ms.wfc.data.IDataSource.iid could not be converted

com.ms.wfc.data.IDataSource.removeDataSourceListener could not be converted

com.ms.wfc.data.IDataSourceListener could not be converted

com.ms.wfc.data.IMarshal.iid could not be converted

com.ms.wfc.data.IPersist.iid could not be converted

com.ms.wfc.data.IPersistStream.GetSizeMax could not be converted

com.ms.wfc.data.IPersistStream.iid could not be converted

com.ms.wfc.data.IPersistStream.Load could not be converted

com.ms.wfc.data.NumberDataFormat could not be converted

com.ms.wfc.data.ObjectProxy.call could not be converted

com.ms.wfc.data.rds.<ClassName>.clsid could not be converted

com.ms.wfc.data.rds.<ClassName>.iid could not be converted

com.ms.wfc.data.Recordset.addDataSourceListener could not be converted

com.ms.wfc.data.Recordset.addNew could not be converted

com.ms.wfc.data.Recordset.addOnEndOfRecordset could not be converted

com.ms.wfc.data.Recordset.addOnFetchComplete could not be converted

com.ms.wfc.data.Recordset.addOnFetchProgress could not be converted

com.ms.wfc.data.Recordset.addOnFieldChangeComplete could not be converted

com.ms.wfc.data.Recordset.addOnMoveComplete could not be converted

com.ms.wfc.data.Recordset.addOnRecordChangeComplete could not be converted

com.ms.wfc.data.Recordset.addOnRecordsetChangeComplete could not be converted

com.ms.wfc.data.Recordset.addOnWillChangeField could not be converted

com.ms.wfc.data.Recordset.addOnWillChangeRecord could not be converted

com.ms.wfc.data.Recordset.addOnWillChangeRecordset could not be converted

com.ms.wfc.data.Recordset.addOnWillMove could not be converted

com.ms.wfc.data.Recordset.DisconnectObject could not be converted

com.ms.wfc.data.Recordset.GetClassID could not be converted

com.ms.wfc.data.Recordset.getCommand could not be converted

com.ms.wfc.data.Recordset.getDataMember could not be converted

com.ms.wfc.data.Recordset.getDataMemberCount could not be converted

com.ms.wfc.data.Recordset.getDataMemberName could not be converted

com.ms.wfc.data.Recordset.GetMarshalSizeMax could not be converted

com.ms.wfc.data.Recordset.getRecordset could not be converted

com.ms.wfc.data.Recordset.getRows could not be converted

com.ms.wfc.data.Recordset.GetSizeMax could not be converted

com.ms.wfc.data.Recordset.GetUnmarshalClass could not be converted

com.ms.wfc.data.Recordset.IsDirty could not be converted

com.ms.wfc.data.Recordset.Load could not be converted

com.ms.wfc.data.Recordset.MarshalInterface could not be converted

com.ms.wfc.data.Recordset.move could not be converted

com.ms.wfc.data.Recordset.nextRecordset could not be converted

com.ms.wfc.data.Recordset.open could not be converted

com.ms.wfc.data.Recordset.open(Object) could not be converted

com.ms.wfc.data.Recordset.Recordset(IDataSource) could not be converted

com.ms.wfc.data.Recordset.Recordset(IDataSource, String) could not be converted

com.ms.wfc.data.Recordset.Recordset(Object) could not be converted

com.ms.wfc.data.Recordset.release could not be converted

com.ms.wfc.data.Recordset.ReleaseMarshalData could not be converted

com.ms.wfc.data.Recordset.removeDataSourceListener could not be converted

com.ms.wfc.data.Recordset.removeOnEndOfRecordset could not be converted

com.ms.wfc.data.Recordset.removeOnFetchComplete could not be converted

com.ms.wfc.data.Recordset.removeOnFetchProgress could not be converted

com.ms.wfc.data.Recordset.removeOnFieldChangeComplete could not be converted

com.ms.wfc.data.Recordset.removeOnMoveComplete could not be converted

com.ms.wfc.data.Recordset.removeOnRecordChangeComplete could not be converted

com.ms.wfc.data.Recordset.removeOnRecordsetChangeComplete could not be converted

com.ms.wfc.data.Recordset.removeOnWillChangeField could not be converted
com.ms.wfc.data.Recordset.removeOnWillChangeRecord could not be converted
com.ms.wfc.data.Recordset.removeOnWillChangeRecordset could not be converted
com.ms.wfc.data.Recordset.removeOnWillMove could not be converted
com.ms.wfc.data.Recordset.setCommand could not be converted
com.ms.wfc.data.Recordset.UnmarshalInterface could not be converted
com.ms.wfc.data.RecordsetEvent could not be converted
com.ms.wfc.data.RecordsetEventHandler could not be converted
com.ms.wfc.data.StringManager.getString could not be converted

Com.ms.wfc.data.adodb Error Messages

com.ms.wfc.data.adodb.<ClassName>.clsid could not be converted

com.ms.wfc.data.adodb.<ClassName>.iid could not be converted

com.ms.wfc.data.adodb._Command.setActiveConnection could not be converted

com.ms.wfc.data.adodb._Connection.setConnectionString could not be converted

com.ms.wfc.data.adodb._Recordset.getCollect could not be converted

com.ms.wfc.data.adodb._Recordset.setCollect could not be converted

com.ms.wfc.data.adodb.Command.setActiveConnection could not be converted

com.ms.wfc.data.adodb.Connection.setConnectionString could not be converted

com.ms.wfc.data.adodb.Recordset.getCollect could not be converted

com.ms.wfc.data.adodb.Recordset.setCollect could not be converted

Com.ms.wfc.data.ui Error Messages

com.ms.wfc.data.ui.Column.getAllowSizing could not be converted
com.ms.wfc.data.ui.Column.getBackColor could not be converted
com.ms.wfc.data.ui.Column.getDataFormat could not be converted
com.ms.wfc.data.ui.Column.getDataType could not be converted
com.ms.wfc.data.ui.Column.getFont could not be converted
com.ms.wfc.data.ui.Column.getForeColor could not be converted
com.ms.wfc.data.ui.Column.getSelectedBackColor could not be converted
com.ms.wfc.data.ui.Column.getSelectedForeColor could not be converted
com.ms.wfc.data.ui.Column.getVisible could not be converted
com.ms.wfc.data.ui.Column.getWriteAllowed could not be converted
com.ms.wfc.data.ui.Column.setAlignment could not be converted
com.ms.wfc.data.ui.Column.setAllowSizing could not be converted
com.ms.wfc.data.ui.Column.setBackColor could not be converted
com.ms.wfc.data.ui.Column.setDataFormat could not be converted
com.ms.wfc.data.ui.Column.setFont could not be converted
com.ms.wfc.data.ui.Column.setForeColor could not be converted
com.ms.wfc.data.ui.Column.setIndex could not be converted
com.ms.wfc.data.ui.Column.setSelectedBackColor could not be converted
com.ms.wfc.data.ui.Column.setSelectedForeColor could not be converted
com.ms.wfc.data.ui.Column.setValue could not be converted
com.ms.wfc.data.ui.Column.setVisible could not be converted
com.ms.wfc.data.ui.Column.shouldPersistBackColor could not be converted
com.ms.wfc.data.ui.Column.shouldPersistFont could not be converted
com.ms.wfc.data.ui.Column.shouldPersistForeColor could not be converted
com.ms.wfc.data.ui.Column.shouldPersistSelectedBackColor could not be converted
com.ms.wfc.data.ui.Column.shouldPersistSelectedForeColor could not be converted
com.ms.wfc.data.ui.ColumnEditingEvent could not be converted
com.ms.wfc.data.ui.ColumnEditingEventHandler could not be converted
com.ms.wfc.data.ui.ColumnEvent could not be converted
com.ms.wfc.data.ui.ColumnEventHandler could not be converted
com.ms.wfc.data.ui.ColumnResizeEvent could not be converted
com.ms.wfc.data.ui.ColumnResizeEventHandler could not be converted
com.ms.wfc.data.ui.ColumnsEditor could not be converted
com.ms.wfc.data.ui.ColumnsEditorDialog could not be converted
com.ms.wfc.data.ui.ColumnUpdatingEvent could not be converted
com.ms.wfc.data.ui.ColumnUpdatingEventHandler could not be converted
com.ms.wfc.data.ui.ConnectionStringEditor could not be converted

com.ms.wfc.data.ui.DataBinder could not be converted

com.ms.wfc.data.ui.DataBinder.Customizer could not be converted

com.ms.wfc.data.ui.DataBinder.GeneralPage could not be converted

com.ms.wfc.data.ui.DataBinding.bindTarget could not be converted

com.ms.wfc.data.ui.DataBinding.CallbackType could not be converted

com.ms.wfc.data.ui.DataBinding.commitChange could not be converted

com.ms.wfc.data.ui.DataBinding.DataBinding could not be converted

com.ms.wfc.data.ui.DataBinding.FalseArgument could not be converted

com.ms.wfc.data.ui.DataBinding.getConstructorArgs could not be converted

com.ms.wfc.data.ui.DataBinding.getDataFormat could not be converted

com.ms.wfc.data.ui.DataBinding.getFieldName could not be converted

com.ms.wfc.data.ui.DataBinding.getPropertyValue could not be converted

com.ms.wfc.data.ui.DataBinding.getTarget could not be converted

com.ms.wfc.data.ui.DataBinding.m_propertyChange could not be converted

com.ms.wfc.data.ui.DataBinding.onChanged could not be converted

com.ms.wfc.data.ui.DataBinding.onChanging could not be converted

com.ms.wfc.data.ui.DataBinding.onTargetDispose could not be converted

com.ms.wfc.data.ui.DataBinding.propertyChange could not be converted

com.ms.wfc.data.ui.DataBinding.refreshPropertyValue could not be converted

com.ms.wfc.data.ui.DataBinding.setDataFormat could not be converted

com.ms.wfc.data.ui.DataBinding.setFieldName could not be converted

com.ms.wfc.data.ui.DataBinding.setPropertyValue could not be converted

com.ms.wfc.data.ui.DataBinding.setTarget could not be converted

com.ms.wfc.data.ui.DataGrid.addColumn could not be converted

com.ms.wfc.data.ui.DataGrid.addColumnEdited could not be converted

com.ms.wfc.data.ui.DataGrid.addColumnEditing could not be converted

com.ms.wfc.data.ui.DataGrid.addColumnResize could not be converted

com.ms.wfc.data.ui.DataGrid.addColumnUpdated could not be converted

com.ms.wfc.data.ui.DataGrid.addColumnUpdating could not be converted

com.ms.wfc.data.ui.DataGrid.addOnDeleted could not be converted

com.ms.wfc.data.ui.DataGrid.addOnDeleting could not be converted

com.ms.wfc.data.ui.DataGrid.addOnError could not be converted

com.ms.wfc.data.ui.DataGrid.addOnHeaderClick could not be converted

com.ms.wfc.data.ui.DataGrid.addOnInserted could not be converted

com.ms.wfc.data.ui.DataGrid.addOnInserting could not be converted

com.ms.wfc.data.ui.DataGrid.addOnPositionChange could not be converted

com.ms.wfc.data.ui.DataGrid.addOnRowResize could not be converted

com.ms.wfc.data.ui.DataGrid.addOnScroll could not be converted

com.ms.wfc.data.ui.DataGrid.addOnSelChange could not be converted

com.ms.wfc.data.ui.DataGrid.addOnUpdated could not be converted

com.ms.wfc.data.ui.DataGrid.addOnUpdating could not be converted
com.ms.wfc.data.ui.DataGrid.Customizer could not be converted
com.ms.wfc.data.ui.DataGrid.DataGrid could not be converted
com.ms.wfc.data.ui.DataGrid.DataGridErrorDialog could not be converted
com.ms.wfc.data.ui.DataGrid.fireErrorEvent could not be converted
com.ms.wfc.data.ui.DataGrid.getAllowAddNew could not be converted
com.ms.wfc.data.ui.DataGrid.getAllowArrows could not be converted
com.ms.wfc.data.ui.DataGrid.getAllowDelete could not be converted
com.ms.wfc.data.ui.DataGrid.getAllowRowSizing could not be converted
com.ms.wfc.data.ui.DataGrid.getAllowUpdate could not be converted
com.ms.wfc.data.ui.DataGrid.getCurrentRow could not be converted
com.ms.wfc.data.ui.DataGrid.getCurrentRowModified could not be converted
com.ms.wfc.data.ui.DataGrid.getDisplayIndex could not be converted
com.ms.wfc.data.ui.DataGrid.getDynamicColumns could not be converted
com.ms.wfc.data.ui.DataGrid.getEnterAction could not be converted
com.ms.wfc.data.ui.DataGrid.getFirstRow could not be converted
com.ms.wfc.data.ui.DataGrid.getHeaderLineCount could not be converted
com.ms.wfc.data.ui.DataGrid.getRowHeight could not be converted
com.ms.wfc.data.ui.DataGrid.getScrollbars could not be converted
com.ms.wfc.data.ui.DataGrid.getSelectedColumns could not be converted
com.ms.wfc.data.ui.DataGrid.getSelectedRows could not be converted
com.ms.wfc.data.ui.DataGrid.getShowPhantom could not be converted
com.ms.wfc.data.ui.DataGrid.getTabAction could not be converted
com.ms.wfc.data.ui.DataGrid.getWrapCellPointer could not be converted
com.ms.wfc.data.ui.DataGrid.makeCurrentCellVisible could not be converted
com.ms.wfc.data.ui.DataGrid.MIN_COLUMN_WIDTH could not be converted
com.ms.wfc.data.ui.DataGrid.MIN_ROW_HEIGHT could not be converted
com.ms.wfc.data.ui.DataGrid.onColumnEdited could not be converted
com.ms.wfc.data.ui.DataGrid.onColumnEditing could not be converted
com.ms.wfc.data.ui.DataGrid.onColumnResize could not be converted
com.ms.wfc.data.ui.DataGrid.onColumnUpdated could not be converted
com.ms.wfc.data.ui.DataGrid.onColumnUpdating could not be converted
com.ms.wfc.data.ui.DataGrid.onDeleted could not be converted
com.ms.wfc.data.ui.DataGrid.onDeleting could not be converted
com.ms.wfc.data.ui.DataGrid.onError could not be converted
com.ms.wfc.data.ui.DataGrid.onHeaderClick could not be converted
com.ms.wfc.data.ui.DataGrid.onInserted could not be converted
com.ms.wfc.data.ui.DataGrid.onInserting could not be converted
com.ms.wfc.data.ui.DataGrid.onPositionChange could not be converted
com.ms.wfc.data.ui.DataGrid.onRowResize could not be converted

com.ms.wfc.data.ui.DataGrid.onScroll could not be converted

com.ms.wfc.data.ui.DataGrid.onSelChange could not be converted

com.ms.wfc.data.ui.DataGrid.onUpdated could not be converted

com.ms.wfc.data.ui.DataGrid.onUpdating could not be converted

com.ms.wfc.data.ui.DataGrid.rebind could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnColumnEdited could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnColumnEditing could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnColumnResize could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnColumnUpdated could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnColumnUpdating could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnDeleted could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnDeleting could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnError could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnHeaderClick could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnInserted could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnInserting could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnPositionChange could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnRowResize could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnScroll could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnSelChange could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnUpdated could not be converted

com.ms.wfc.data.ui.DataGrid.removeOnUpdating could not be converted

com.ms.wfc.data.ui.DataGrid.rowBookmark could not be converted

com.ms.wfc.data.ui.DataGrid.rowTop could not be converted

com.ms.wfc.data.ui.DataGrid.scroll could not be converted

com.ms.wfc.data.ui.DataGrid.setAllowAddNew could not be converted

com.ms.wfc.data.ui.DataGrid.setAllowArrows could not be converted

com.ms.wfc.data.ui.DataGrid.setAllowDelete could not be converted

com.ms.wfc.data.ui.DataGrid.setAllowRowSizing could not be converted

com.ms.wfc.data.ui.DataGrid.setAllowUpdate could not be converted

com.ms.wfc.data.ui.DataGrid.setCurrentRow could not be converted

com.ms.wfc.data.ui.DataGrid.setDynamicColumns could not be converted

com.ms.wfc.data.ui.DataGrid.setEnterAction could not be converted

com.ms.wfc.data.ui.DataGrid.setFirstRow could not be converted

com.ms.wfc.data.ui.DataGrid.setHeaderLineCount could not be converted

com.ms.wfc.data.ui.DataGrid.setLeftColumn could not be converted

com.ms.wfc.data.ui.DataGrid.setRowHeight could not be converted

com.ms.wfc.data.ui.DataGrid.setScrollbars could not be converted

com.ms.wfc.data.ui.DataGrid.setSelectedColumns could not be converted

com.ms.wfc.data.ui.DataGrid.setSelectedRows could not be converted

com.ms.wfc.data.ui.DataGrid.setTabAction could not be converted

com.ms.wfc.data.ui.DataGrid.setWrapCellPointer could not be converted

com.ms.wfc.data.ui.DataMemberEditor could not be converted

com.ms.wfc.data.ui.DataNavigator.Customizer could not be converted

com.ms.wfc.data.ui.DataNavigator.getDataMember could not be converted

com.ms.wfc.data.ui.DataNavigator.moveFirst could not be converted

com.ms.wfc.data.ui.DataNavigator.moveLast could not be converted

com.ms.wfc.data.ui.DataNavigator.moveNext could not be converted

com.ms.wfc.data.ui.DataNavigator.movePrevious could not be converted

com.ms.wfc.data.ui.DataNavigator.propertyChanged could not be converted

com.ms.wfc.data.ui.DataNavigator.setDataMember could not be converted

com.ms.wfc.data.ui.DataNavigator.setDataSource could not be converted

com.ms.wfc.data.ui.DataSource.addDataSourceListener could not be converted

com.ms.wfc.data.ui.DataSource.addOnBeginTransComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnCommitTransComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnConnectComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnDisconnect could not be converted

com.ms.wfc.data.ui.DataSource.addOnEndOfRecordset could not be converted

com.ms.wfc.data.ui.DataSource.addOnExecuteComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnFetchComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnFetchProgress could not be converted

com.ms.wfc.data.ui.DataSource.addOnFieldChangeComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnInfoMessage could not be converted

com.ms.wfc.data.ui.DataSource.addOnMoveComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnRecordChangeComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnRecordsetChangeComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnRollbackTransComplete could not be converted

com.ms.wfc.data.ui.DataSource.addOnWillChangeField could not be converted

com.ms.wfc.data.ui.DataSource.addOnWillChangeRecord could not be converted

com.ms.wfc.data.ui.DataSource.addOnWillChangeRecordset could not be converted

com.ms.wfc.data.ui.DataSource.addOnWillConnect could not be converted

com.ms.wfc.data.ui.DataSource.addOnWillExecute could not be converted

com.ms.wfc.data.ui.DataSource.addOnWillMove could not be converted

com.ms.wfc.data.ui.DataSource.dataMemberAdded could not be converted

com.ms.wfc.data.ui.DataSource.dataMemberChanged could not be converted

com.ms.wfc.data.ui.DataSource.dataMemberRemoved could not be converted

com.ms.wfc.data.ui.DataSource.DataSource could not be converted

com.ms.wfc.data.ui.DataSource.getAsyncConnect could not be converted

com.ms.wfc.data.ui.DataSource.getAsyncExecute could not be converted

com.ms.wfc.data.ui.DataSource.getAsyncFetch could not be converted

com.ms.wfc.data.ui.DataSource.getCacheSize could not be converted

com.ms.wfc.data.ui.DataSource.getCursorLocation could not be converted

com.ms.wfc.data.ui.DataSource.getCursorType could not be converted

com.ms.wfc.data.ui.DataSource.getDataMember could not be converted

com.ms.wfc.data.ui.DataSource.getDataMemberCount could not be converted

com.ms.wfc.data.ui.DataSource.getDataMemberName could not be converted

com.ms.wfc.data.ui.DataSource.getDesignTimeData could not be converted

com.ms.wfc.data.ui.DataSource.getIsolationLevel could not be converted

com.ms.wfc.data.ui.DataSource.getLockType could not be converted

com.ms.wfc.data.ui.DataSource.getMaxRecords could not be converted

com.ms.wfc.data.ui.DataSource.getMode could not be converted

com.ms.wfc.data.ui.DataSource.getParentDataSource could not be converted

com.ms.wfc.data.ui.DataSource.getParentFieldName could not be converted

com.ms.wfc.data.ui.DataSource.getPassword could not be converted

com.ms.wfc.data.ui.DataSource.getPrepared could not be converted

com.ms.wfc.data.ui.DataSource.getRecordset could not be converted

com.ms.wfc.data.ui.DataSource.getSort could not be converted

com.ms.wfc.data.ui.DataSource.getStayInSync could not be converted

com.ms.wfc.data.ui.DataSource.getUserId could not be converted

com.ms.wfc.data.ui.DataSource.isChildDataSource could not be converted

com.ms.wfc.data.ui.DataSource.removeDataSourceListener could not be converted

com.ms.wfc.data.ui.DataSource.removeOnBeginTransComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnCommitTransComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnConnectComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnDisconnect could not be converted

com.ms.wfc.data.ui.DataSource.removeOnEndOfRecordset could not be converted

com.ms.wfc.data.ui.DataSource.removeOnExecuteComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnFetchComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnFetchProgress could not be converted

com.ms.wfc.data.ui.DataSource.removeOnFieldChangeComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnInfoMessage could not be converted

com.ms.wfc.data.ui.DataSource.removeOnMoveComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnRecordChangeComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnRecordsetChangeComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnRollbackTransComplete could not be converted

com.ms.wfc.data.ui.DataSource.removeOnWillChangeField could not be converted

com.ms.wfc.data.ui.DataSource.removeOnWillChangeRecord could not be converted

com.ms.wfc.data.ui.DataSource.removeOnWillChangeRecordset could not be converted

com.ms.wfc.data.ui.DataSource.removeOnWillConnect could not be converted

com.ms.wfc.data.ui.DataSource.removeOnWillExecute could not be converted

com.ms.wfc.data.ui.DataSource.removeOnWillMove could not be converted

com.ms.wfc.data.ui.DataSource.setAsyncConnect could not be converted

com.ms.wfc.data.ui.DataSource.setAsyncExecute could not be converted

com.ms.wfc.data.ui.DataSource.setAsyncFetch could not be converted

com.ms.wfc.data.ui.DataSource.setCacheSize could not be converted

com.ms.wfc.data.ui.DataSource.setConnectionString could not be converted

com.ms.wfc.data.ui.DataSource.setConnectionTimeout could not be converted

com.ms.wfc.data.ui.DataSource.setCursorLocation could not be converted

com.ms.wfc.data.ui.DataSource.setCursorType could not be converted

com.ms.wfc.data.ui.DataSource.setDesignTimeData could not be converted

com.ms.wfc.data.ui.DataSource.setFilter could not be converted

com.ms.wfc.data.ui.DataSource.setIsolationLevel could not be converted

com.ms.wfc.data.ui.DataSource.setLockType could not be converted

com.ms.wfc.data.ui.DataSource.setMaxRecords could not be converted

com.ms.wfc.data.ui.DataSource.setMode could not be converted

com.ms.wfc.data.ui.DataSource.setParentDataSource could not be converted

com.ms.wfc.data.ui.DataSource.setParentFieldName could not be converted

com.ms.wfc.data.ui.DataSource.setPassword could not be converted

com.ms.wfc.data.ui.DataSource.setPrepared could not be converted

com.ms.wfc.data.ui.DataSource.setSort could not be converted

com.ms.wfc.data.ui.DataSource.setStayInSync could not be converted

com.ms.wfc.data.ui.DataSource.setUserId could not be converted

com.ms.wfc.data.ui.DataSource.unrealize could not be converted

com.ms.wfc.data.ui.EnterAction could not be converted

com.ms.wfc.data.ui.ErrorEvent could not be converted

com.ms.wfc.data.ui.ErrorEventHandler could not be converted

com.ms.wfc.data.ui.FieldNameEditor could not be converted

com.ms.wfc.data.ui.GridLineStyle.GridLineStyle could not be converted

com.ms.wfc.data.ui.GridLineStyle.RAISED3D could not be converted

com.ms.wfc.data.ui.GridLineStyle.SUNKEN3D could not be converted

com.ms.wfc.data.ui.PositionChangeEvent.lastColumn could not be converted

com.ms.wfc.data.ui.PositionChangeEvent.lastRow could not be converted

com.ms.wfc.data.ui.PositionChangeEvent.PositionChangeEvent could not be converted

com.ms.wfc.data.ui.PositionChangeEventHandler.PositionChangeEventHandler could not be converted

com.ms.wfc.data.ui.PropertyNameEditor could not be converted

com.ms.wfc.data.ui.RecordsetProvider could not be converted

com.ms.wfc.data.ui.TabAction could not be converted

Com.ms.wfc.io Error Messages

com.ms.wfc.io.BufferedStream.getStream could not be converted

com.ms.wfc.io.BufferedStream.readCore could not be converted

com.ms.wfc.io.BufferedStream.readStringCharsAnsi could not be converted

com.ms.wfc.io.BufferedStream.readStringNull could not be converted

com.ms.wfc.io.BufferedStream.readStringNullAnsi could not be converted

com.ms.wfc.io.BufferedStream.writeCore could not be converted

com.ms.wfc.io.CodePage.ANSI could not be converted

com.ms.wfc.io.CodePage.CodePage could not be converted

com.ms.wfc.io.CodePage.MAC could not be converted

com.ms.wfc.io.CodePage.OEM could not be converted

com.ms.wfc.io.DataStream.comStream could not be converted

com.ms.wfc.io.DataStream.DataStream could not be converted

com.ms.wfc.io.DataStream.fromComStream could not be converted

com.ms.wfc.io.DataStream.getComStream could not be converted

com.ms.wfc.io.DataStream.readCore could not be converted

com.ms.wfc.io.DataStream.readStringNull could not be converted

com.ms.wfc.io.DataStream.readStringNullAnsi could not be converted

com.ms.wfc.io.DataStream.readUTF could not be converted

com.ms.wfc.io.DataStream.toComStream could not be converted

com.ms.wfc.io.DataStream.writeCore could not be converted

com.ms.wfc.io.DataStream.writeStringChars could not be converted

com.ms.wfc.io.DataStream.writeStringCharsAnsi could not be converted

com.ms.wfc.io.DataStream.writeStringNull could not be converted

com.ms.wfc.io.DataStream.writeStringNullAnsi could not be converted

com.ms.wfc.io.DataStream.writeUTF could not be converted

com.ms.wfc.io.DataStreamFromComStream could not be converted

com.ms.wfc.io.File.createDirectory could not be converted

com.ms.wfc.io.File.File could not be converted

com.ms.wfc.io.File.getDirectory could not be converted

com.ms.wfc.io.File.GetFiles could not be converted

com.ms.wfc.io.File.getName could not be converted

com.ms.wfc.io.File.handle could not be converted

com.ms.wfc.io.File.openStandardError could not be converted

com.ms.wfc.io.File.openStandardInput could not be converted

com.ms.wfc.io.File.openStandardOutput could not be converted

com.ms.wfc.io.File.readCore could not be converted

com.ms.wfc.io.File.writeCore could not be converted

com.ms.wfc.io.FileEnumerator.close could not be converted
com.ms.wfc.io.FileEnumerator.FileEnumerator could not be converted
com.ms.wfc.io.FileEnumerator.finalize could not be converted
com.ms.wfc.io.FileEnumerator.getAttributes could not be converted
com.ms.wfc.io.IDataStream.getComStream could not be converted
com.ms.wfc.io.FileEnumerator.getSize could not be converted
com.ms.wfc.io.IDataStream.readStringNull could not be converted
com.ms.wfc.io.IDataStream.readStringNullAnsi could not be converted
com.ms.wfc.io.IDataStream.readUTF could not be converted
com.ms.wfc.io.IDataStream.writeStringChars could not be converted
com.ms.wfc.io.IDataStream.writeStringCharsAnsi could not be converted
com.ms.wfc.io.IDataStream.writeStringNull could not be converted
com.ms.wfc.io.IDataStream.writeStringNullAnsi could not be converted
com.ms.wfc.io.IDataStream.writeUTF could not be converted
com.ms.wfc.io.IDataStreamProvider could not be converted
com.ms.wfc.io.MemoryStream.MemoryStream could not be converted
com.ms.wfc.io.MemoryStream.readCore could not be converted
com.ms.wfc.io.MemoryStream.readStringCharsAnsi could not be converted
com.ms.wfc.io.MemoryStream.readStringNull could not be converted
com.ms.wfc.io.MemoryStream.readStringNullAnsi could not be converted
com.ms.wfc.io.MemoryStream.writeCore could not be converted
com.ms.wfc.io.MemoryStream.writeTo could not be converted
com.ms.wfc.io.Reader.Reader could not be converted
com.ms.wfc.io.TextReader.TextReader(ByteStream) could not be converted
com.ms.wfc.io.TextReader.TextReader(ByteStream, int) could not be converted
com.ms.wfc.io.TextReader.TextReader(ByteStream, int, int) could not be converted
com.ms.wfc.io.WinIOException.getErrorCode could not be converted
com.ms.wfc.io.WinIOException.WinIOException could not be converted
com.ms.wfc.io.Writer.Writer could not be converted

Com.ms.wfc.ole32 Error Messages

[com.ms.wfc.ole32.DBBINDING.pTypeInfo could not be converted](#)

[com.ms.wfc.ole32.DBCOLUMNINFO.pTypeInfo could not be converted](#)

[com.ms.wfc.ole32.IEnumFORMATETC.iid could not be converted](#)

[com.ms.wfc.ole32.IEnumSTATDATA.iid could not be converted](#)

[com.ms.wfc.ole32.ILockBytes.iid could not be converted](#)

[com.ms.wfc.ole32.IMalloc.iid could not be converted](#)

[com.ms.wfc.ole32.IOleDataFormat.iid could not be converted](#)

[com.ms.wfc.ole32.IOleDataObject.iid could not be converted](#)

[com.ms.wfc.ole32.IOleDataObjectWithIStream.iid could not be converted](#)

[com.ms.wfc.ole32.IOleDropSource.iid could not be converted](#)

[com.ms.wfc.ole32.IOleDropTarget.iid could not be converted](#)

[com.ms.wfc.ole32.IPersist.iid could not be converted](#)

[com.ms.wfc.ole32.IPersistStream.iid could not be converted](#)

[com.ms.wfc.ole32.IPicture.iid could not be converted](#)

[com.ms.wfc.ole32.IStorage.iid could not be converted](#)

Com.ms.wfc.ui Error Messages

com.ms.wfc.ui.AnchorEditor.editValue could not be converted

com.ms.wfc.ui.AnchorEditor.getConstantName could not be converted

com.ms.wfc.ui.AnchorEditor.getTextFromValue could not be converted

com.ms.wfc.ui.Animation could not be converted

com.ms.wfc.ui.AnimationFileNameEditor could not be converted

com.ms.wfc.ui.AutoSizeEvent could not be converted

com.ms.wfc.ui.AutoSizeEventHandler could not be converted

com.ms.wfc.ui.AxHost.AboutBoxDelegate could not be converted

com.ms.wfc.ui.AxHost.AxPropertyInfo could not be converted

com.ms.wfc.ui.AxHost.begin could not be converted

com.ms.wfc.ui.AxHost.DispidAttribute could not be converted

com.ms.wfc.ui.AxHost.fireOnClick could not be converted

com.ms.wfc.ui.AxHost.fireOnDbClick could not be converted

com.ms.wfc.ui.AxHost.fireOnKeyDown could not be converted

com.ms.wfc.ui.AxHost.fireOnKeyPress could not be converted

com.ms.wfc.ui.AxHost.fireOnKeyUp could not be converted

com.ms.wfc.ui.AxHost.fireOnMouseDown could not be converted

com.ms.wfc.ui.AxHost.fireOnMouseMove could not be converted

com.ms.wfc.ui.AxHost.fireOnMouseUp could not be converted

com.ms.wfc.ui.AxHost.Flags could not be converted

com.ms.wfc.ui.AxHost.getAmbientProperty could not be converted

com.ms.wfc.ui.AxHost.getClientAttributes could not be converted

com.ms.wfc.ui.AxHost.getColorFromOleColor could not be converted

com.ms.wfc.ui.AxHost.getFontFromIFont could not be converted

com.ms.wfc.ui.AxHost.getFontFromIFontDisp could not be converted

com.ms.wfc.ui.AxHost.getIFontDispFromFont could not be converted

com.ms.wfc.ui.AxHost.getIFontFromFont could not be converted

com.ms.wfc.ui.AxHost.getIPictureDispFromPicture could not be converted

com.ms.wfc.ui.AxHost.getIPictureFromPicture could not be converted

com.ms.wfc.ui.AxHost.getOleColorFromColor could not be converted

com.ms.wfc.ui.AxHost.getPictureFromIPicture could not be converted

com.ms.wfc.ui.AxHost.getPictureFromIPictureDisp could not be converted

com.ms.wfc.ui.AxHost.propertyChanged could not be converted

com.ms.wfc.ui.AxHost.setAboutBoxDelegate could not be converted

com.ms.wfc.ui.AxHost.setTopLevel could not be converted

com.ms.wfc.ui.AxHost.shouldPersistContainingForm could not be converted

com.ms.wfc.ui.AxHost.State.save could not be converted

com.ms.wfc.ui.AxHost.State.State could not be converted

com.ms.wfc.ui.AxPersist could not be converted

com.ms.wfc.ui.Bitmap.Bitmap could not be converted

com.ms.wfc.ui.Bitmap.Bitmap(Bitmap, Bitmap) could not be converted

com.ms.wfc.ui.Bitmap.Bitmap(int) could not be converted

com.ms.wfc.ui.Bitmap.Bitmap(int, int, int, int, short[]) could not be converted

com.ms.wfc.ui.Bitmap.Bitmap(IPicture) could not be converted

com.ms.wfc.ui.Bitmap.Bitmap(Palette) could not be converted

com.ms.wfc.ui.Bitmap.copyHandle could not be converted

com.ms.wfc.ui.Bitmap.destroyHandle could not be converted

com.ms.wfc.ui.Bitmap.drawStretchTo could not be converted

com.ms.wfc.ui.Bitmap.drawTo could not be converted

com.ms.wfc.ui.Bitmap.getColorMask could not be converted

com.ms.wfc.ui.Bitmap.getGraphics could not be converted

com.ms.wfc.ui.Bitmap.getHandle could not be converted

com.ms.wfc.ui.Bitmap.getMonochromeMask could not be converted

com.ms.wfc.ui.Bitmap.getPICTDESC could not be converted

com.ms.wfc.ui.Bitmap.getTransparent could not be converted

com.ms.wfc.ui.Bitmap.getTransparentColor could not be converted

com.ms.wfc.ui.Bitmap.initialize could not be converted

com.ms.wfc.ui.Brush.Brush(Bitmap) could not be converted

com.ms.wfc.ui.Brush.Brush(int) could not be converted

com.ms.wfc.ui.Brush.copyHandle could not be converted

com.ms.wfc.ui.Brush.destroyHandle could not be converted

com.ms.wfc.ui.Brush.getHandle could not be converted

com.ms.wfc.ui.Brush.HALFTONE could not be converted

com.ms.wfc.ui.BrushStyle.HOLLOW could not be converted

com.ms.wfc.ui.BrushStyle.PATTERN could not be converted

com.ms.wfc.ui.Button.propertyChanged could not be converted

com.ms.wfc.ui.CheckBox.getGroupValue could not be converted

com.ms.wfc.ui.CheckBox.setGroupValue could not be converted

com.ms.wfc.ui.CheckBox.setTextAlign could not be converted

com.ms.wfc.ui.CheckedListBox.propertyChanged could not be converted

com.ms.wfc.ui.Color.Color could not be converted

com.ms.wfc.ui.Color.Editor.ColorPicker could not be converted

com.ms.wfc.ui.Color.Editor.ColorPicker.Palette could not be converted

com.ms.wfc.ui.Color.Editor.ColorPicker.Palette.CustomColorDialog could not be converted

com.ms.wfc.ui.Color.fromCMYK could not be converted

com.ms.wfc.ui.Color.getConstructorArgs could not be converted

com.ms.wfc.ui.Color.toString could not be converted

com.ms.wfc.ui.ColumnClickEventHandler.ColumnClickEventHandler could not be converted

com.ms.wfc.ui.ComboBox.propertyChanged could not be converted

com.ms.wfc.ui.ContentAlignment.toTextFormat could not be converted

com.ms.wfc.ui.Control.getTopLevel could not be converted

com.ms.wfc.ui.Control.getVisible could not be converted

com.ms.wfc.ui.Control.invokeAsync could not be converted

com.ms.wfc.ui.Control.PROP_BACKCOLOR could not be converted

com.ms.wfc.ui.Control.PROP_CURSOR could not be converted

com.ms.wfc.ui.Control.PROP_ENABLED could not be converted

com.ms.wfc.ui.Control.PROP_FONT could not be converted

com.ms.wfc.ui.Control.PROP_FORECOLOR could not be converted

com.ms.wfc.ui.Control.PROP_LOCATION could not be converted

com.ms.wfc.ui.Control.PROP_PARENTBACKCOLOR could not be converted

com.ms.wfc.ui.Control.PROP_PARENTFONT could not be converted

com.ms.wfc.ui.Control.PROP_PARENTFORECOLOR could not be converted

com.ms.wfc.ui.Control.PROP_SIZE could not be converted

com.ms.wfc.ui.Control.PROP_TEXT could not be converted

com.ms.wfc.ui.Control.PROP_VISIBLE could not be converted

com.ms.wfc.ui.Control.propertyChanged could not be converted

com.ms.wfc.ui.Control.sendMessage could not be converted

com.ms.wfc.ui.Control.setTabIndex could not be converted

com.ms.wfc.ui.Control.setTopLevel could not be converted

com.ms.wfc.ui.Control.STYLE_ACCEPTSCHILDREN could not be converted

com.ms.wfc.ui.CoordinateSystem could not be converted

com.ms.wfc.ui.Cursor.Cursor could not be converted

com.ms.wfc.ui.Cursor.destroyHandle could not be converted

com.ms.wfc.ui.Cursor.getPICTDESC could not be converted

com.ms.wfc.ui.Cursor.initialize could not be converted

com.ms.wfc.ui.DateBoldEventHandler.DateBoldEventHandler could not be converted

com.ms.wfc.ui.DateRangeEventHandler.DateRangeEventHandler could not be converted

com.ms.wfc.ui.DateTimeFormatEvent could not be converted

com.ms.wfc.ui.DateTimeFormatEventHandler could not be converted

com.ms.wfc.ui.DateTimeFormatQueryEvent could not be converted

com.ms.wfc.ui.DateTimeFormatQueryEventHandler could not be converted

com.ms.wfc.ui.DateTimePicker.addOnFormat could not be converted

com.ms.wfc.ui.DateTimePicker.addOnFormatQuery could not be converted

com.ms.wfc.ui.DateTimePicker.addOnUserString could not be converted

com.ms.wfc.ui.DateTimePicker.getAllowUserString could not be converted

com.ms.wfc.ui.DateTimePicker.getMaxDate could not be converted

com.ms.wfc.ui.DateTimePicker.getMinDate could not be converted

com.ms.wfc.ui.DateTimePicker.onFormat could not be converted

com.ms.wfc.ui.DateTimePicker.onFormatQuery could not be converted

com.ms.wfc.ui.DateTimePicker.onUserString could not be converted

com.ms.wfc.ui.DateTimePicker.removeOnFormat could not be converted

com.ms.wfc.ui.DateTimePicker.removeOnFormatQuery could not be converted

com.ms.wfc.ui.DateTimePicker.removeOnUserString could not be converted

com.ms.wfc.ui.DateTimePicker.setAllowUserString could not be converted

com.ms.wfc.ui.DateTimeUserStringEvent could not be converted

com.ms.wfc.ui.DateTimeUserStringEventHandler could not be converted

com.ms.wfc.ui.DateTimeWmKeyDownEvent.DateTimeWmKeyDownEvent could not be converted

com.ms.wfc.ui.DateTimeWmKeyDownEvent.format could not be converted

com.ms.wfc.ui.DateTimeWmKeyDownEvent.time could not be converted

com.ms.wfc.ui.DateTimeWmKeyDownEventHandler.DateTimeWmKeyDownEventHandler could not be converted

com.ms.wfc.ui.Dimensions.Editor could not be converted

com.ms.wfc.ui.Dimensions.getConstructorArgs could not be converted

com.ms.wfc.ui.Dimensions.save could not be converted

com.ms.wfc.ui.DockEditor.editValue could not be converted

com.ms.wfc.ui.DockEditor.getConstantName could not be converted

com.ms.wfc.ui.DockEditor.getTextFromValue could not be converted

com.ms.wfc.ui.DocumentReadyEvent could not be converted

com.ms.wfc.ui.DocumentReadyEventHandler could not be converted

com.ms.wfc.ui.DocumentReadyEventHandler.DocumentReadyEventHandler could not be converted

com.ms.wfc.ui.DragEventHandler.DragEventHandler could not be converted

com.ms.wfc.ui.DrawItemEventHandler.DrawItemEventHandler could not be converted

com.ms.wfc.ui.Edit.propertyChanged could not be converted

com.ms.wfc.ui.Edit.setTextAlign could not be converted

com.ms.wfc.ui.Edit.undo could not be converted

com.ms.wfc.ui.Editor.createExtensionsString could not be converted

com.ms.wfc.ui.Editor.createFilterEntry could not be converted

com.ms.wfc.ui.Editor.getExtensions could not be converted

com.ms.wfc.ui.Editor.getFileDialogDescription could not be converted

com.ms.wfc.ui.Editor.loadFromStream could not be converted

com.ms.wfc.ui.EraseBackgroundEvent could not be converted

com.ms.wfc.ui.FileDialog.promptFileCreate could not be converted

com.ms.wfc.ui.FileDialog.promptFileNotFound could not be converted

com.ms.wfc.ui.FileDialog.promptFileOverwrite could not be converted

com.ms.wfc.ui.FloodFillType could not be converted

com.ms.wfc.ui.Font.ANSI_VAR could not be converted

com.ms.wfc.ui.Font.destroyHandle could not be converted

com.ms.wfc.ui.Font.DEVICE_DEFAULT could not be converted

com.ms.wfc.ui.Font.equalsBase could not be converted
com.ms.wfc.ui.Font.Font could not be converted
com.ms.wfc.ui.Font.getCharacterSet could not be converted
com.ms.wfc.ui.Font.getConstructorArgs could not be converted
com.ms.wfc.ui.Font.getFamily could not be converted
com.ms.wfc.ui.Font.getFontMetrics could not be converted
com.ms.wfc.ui.Font.getOrientation could not be converted
com.ms.wfc.ui.Font.getPitch could not be converted
com.ms.wfc.ui.Font.getStock could not be converted
com.ms.wfc.ui.Font.OEM_FIXED could not be converted
com.ms.wfc.ui.Font.save could not be converted
com.ms.wfc.ui.Font.SYSTEM could not be converted
com.ms.wfc.ui.Font.SYSTEM_FIXED could not be converted
com.ms.wfc.ui.FontDescriptor could not be converted
com.ms.wfc.ui.FontDevice could not be converted
com.ms.wfc.ui.FontDialog.getFontDevice could not be converted
com.ms.wfc.ui.FontDialog.getPrinterDC could not be converted
com.ms.wfc.ui.FontDialog.getScalableOnly could not be converted
com.ms.wfc.ui.FontDialog.getTrueTypeOnly could not be converted
com.ms.wfc.ui.FontDialog.getWysiwyg could not be converted
com.ms.wfc.ui.FontDialog.hookProc could not be converted
com.ms.wfc.ui.FontDialog.setFontDevice could not be converted
com.ms.wfc.ui.FontDialog.setPrinterDC could not be converted
com.ms.wfc.ui.FontDialog.setScalableOnly could not be converted
com.ms.wfc.ui.FontDialog.setTrueTypeOnly could not be converted
com.ms.wfc.ui.FontDialog.setWysiwyg could not be converted
com.ms.wfc.ui.FontFamily could not be converted
com.ms.wfc.ui.FontMetrics could not be converted
com.ms.wfc.ui.FontMetrics.ascent could not be converted
com.ms.wfc.ui.FontMetrics.charSet could not be converted
com.ms.wfc.ui.FontMetrics.descent could not be converted
com.ms.wfc.ui.FontMetrics.FontMetrics could not be converted
com.ms.wfc.ui.FontMetrics.height could not be converted
com.ms.wfc.ui.FontPitch could not be converted
com.ms.wfc.ui.FontSize.CELLHEIGHT could not be converted
com.ms.wfc.ui.FontSize.CENTIMETERS could not be converted
com.ms.wfc.ui.FontSize.CHARACTERHEIGHT could not be converted
com.ms.wfc.ui.FontSize.EM could not be converted
com.ms.wfc.ui.FontSize.EX could not be converted
com.ms.wfc.ui.FontType could not be converted

com.ms.wfc.ui.FontWeight.EXTRALIGHT could not be converted

com.ms.wfc.ui.FontWeight.LIGHT could not be converted

com.ms.wfc.ui.FontWeight.THIN could not be converted

com.ms.wfc.ui.Form.checkCloseDialog could not be converted

com.ms.wfc.ui.Form.FORMSTATE_AUTOSCALING could not be converted

com.ms.wfc.ui.Form.FORMSTATE_AUTOSCROLLING could not be converted

com.ms.wfc.ui.Form.FORMSTATE_BORDERSTYLE could not be converted

com.ms.wfc.ui.Form.FORMSTATE_CONTROLBOX could not be converted

com.ms.wfc.ui.Form.FORMSTATE_HELPBUTTON could not be converted

com.ms.wfc.ui.Form.FORMSTATE_HSCROLLVISIBLE could not be converted

com.ms.wfc.ui.Form.FORMSTATE_KEYPREVIEW could not be converted

com.ms.wfc.ui.Form.FORMSTATE_MAXIMIZEBOX could not be converted

com.ms.wfc.ui.Form.FORMSTATE_MINIMIZEBOX could not be converted

com.ms.wfc.ui.Form.FORMSTATE_PALETTEMODE could not be converted

com.ms.wfc.ui.Form.FORMSTATE_SETCLIENTSIZE could not be converted

com.ms.wfc.ui.Form.FORMSTATE_SHOWWINDOWONCREATE could not be converted

com.ms.wfc.ui.Form.FORMSTATE_STARTPOS could not be converted

com.ms.wfc.ui.Form.FORMSTATE_TASKBAR could not be converted

com.ms.wfc.ui.Form.FORMSTATE_TOPMOST could not be converted

com.ms.wfc.ui.Form.FORMSTATE_USERHASSCROLLED could not be converted

com.ms.wfc.ui.Form.FORMSTATE_VSCROLLVISIBLE could not be converted

com.ms.wfc.ui.Form.FORMSTATE_WINDOWSTATE could not be converted

com.ms.wfc.ui.Form.getFormState could not be converted

com.ms.wfc.ui.Form.getPalette could not be converted

com.ms.wfc.ui.Form.getPaletteMode could not be converted

com.ms.wfc.ui.Form.getPaletteSource could not be converted

com.ms.wfc.ui.Form.hasFormState could not be converted

com.ms.wfc.ui.Form.notifyPaletteChange could not be converted

com.ms.wfc.ui.Form.onMDIChildActivate could not be converted

com.ms.wfc.ui.Form.onNewPalette could not be converted

com.ms.wfc.ui.Form.propertyChanged could not be converted

com.ms.wfc.ui.Form.setAutoScaleBaseSize could not be converted

com.ms.wfc.ui.Form.setBorderStyle could not be converted

com.ms.wfc.ui.Form.setControlBox could not be converted

com.ms.wfc.ui.Form.setFormState could not be converted

com.ms.wfc.ui.Form.setMaximizeBox could not be converted

com.ms.wfc.ui.Form.setMinimizeBox could not be converted

com.ms.wfc.ui.Form.setNewControls could not be converted

com.ms.wfc.ui.Form.setPaletteMode could not be converted

com.ms.wfc.ui.Form.setPaletteSource could not be converted

com.ms.wfc.ui.Form.setShowInTaskbar could not be converted

com.ms.wfc.ui.Form.setStartPosition could not be converted

com.ms.wfc.ui.FormPaletteMode could not be converted

com.ms.wfc.ui.GiveFeedbackEventHandler.GiveFeedbackEventHandler could not be converted

com.ms.wfc.ui.Graphics.drawArc could not be converted

com.ms.wfc.ui.Graphics.drawChord could not be converted

com.ms.wfc.ui.Graphics.drawPie could not be converted

com.ms.wfc.ui.Graphics.drawString could not be converted

com.ms.wfc.ui.Graphics.floodFill could not be converted

com.ms.wfc.ui.Graphics.getCoordinateSystem could not be converted

com.ms.wfc.ui.Graphics.getDeviceOrigin could not be converted

com.ms.wfc.ui.Graphics.getDevicePoint could not be converted

com.ms.wfc.ui.Graphics.getDeviceScale could not be converted

com.ms.wfc.ui.Graphics.getFontDescriptors could not be converted

com.ms.wfc.ui.Graphics.getHandle could not be converted

com.ms.wfc.ui.Graphics.getLogicalPoint could not be converted

com.ms.wfc.ui.Graphics.getLogicalSizeX could not be converted

com.ms.wfc.ui.Graphics.getLogicalSizeY could not be converted

com.ms.wfc.ui.Graphics.getOpaque could not be converted

com.ms.wfc.ui.Graphics.getPageOrigin could not be converted

com.ms.wfc.ui.Graphics.getPageScale could not be converted

com.ms.wfc.ui.Graphics.getPhysicalSizeX could not be converted

com.ms.wfc.ui.Graphics.getPhysicalSizeY could not be converted

com.ms.wfc.ui.Graphics.getPixel could not be converted

com.ms.wfc.ui.Graphics.getTextSize could not be converted

com.ms.wfc.ui.Graphics.getTextSpace could not be converted

com.ms.wfc.ui.Graphics.Graphics could not be converted

com.ms.wfc.ui.Graphics.invert could not be converted

com.ms.wfc.ui.Graphics.renderPalette could not be converted

com.ms.wfc.ui.Graphics.scroll could not be converted

com.ms.wfc.ui.Graphics.setCoordinateOrigin could not be converted

com.ms.wfc.ui.Graphics.setCoordinateScale could not be converted

com.ms.wfc.ui.Graphics.setCoordinateSystem could not be converted

com.ms.wfc.ui.Graphics.setHandle could not be converted

com.ms.wfc.ui.Graphics.setOpaque could not be converted

com.ms.wfc.ui.Graphics.setPixel could not be converted

com.ms.wfc.ui.Graphics.setTextSpace could not be converted

com.ms.wfc.ui.Help.Help could not be converted

com.ms.wfc.ui.HelpEvent.component could not be converted

com.ms.wfc.ui.HelpEvent.contextId could not be converted

com.ms.wfc.ui.HelpEvent.contextType could not be converted

com.ms.wfc.ui.HelpEvent.controlId could not be converted

com.ms.wfc.ui.HelpEvent.HelpEvent(int, int, IComonent, int, Point) could not be converted

com.ms.wfc.ui.HelpEvent.HelpEvent(Object, int, int, IComonent, Int, Point) could not be converted

com.ms.wfc.ui.HelpEventHandler.HelpEventHandler could not be converted

com.ms.wfc.ui.HelpFileFileNameEditor could not be converted

com.ms.wfc.ui.HelpProvider.shouldPersistShowHelp could not be converted

com.ms.wfc.ui.HTMLControl could not be converted

com.ms.wfc.ui.HTMLControl.add could not be converted

com.ms.wfc.ui.HTMLControl.addOnDocumentReady could not be converted

com.ms.wfc.ui.HTMLControl.getAmbientProperty could not be converted

com.ms.wfc.ui.HTMLControl.HTMLControl could not be converted

com.ms.wfc.ui.HTMLControl.onDocumentReady could not be converted

com.ms.wfc.ui.HTMLControl.propertyChanged could not be converted

com.ms.wfc.ui.HTMLControl.removeOnDocumentReady could not be converted

com.ms.wfc.ui.HTMLControl.setBoundElements could not be converted

com.ms.wfc.ui.HTMLControl.setNewHTMLElements could not be converted

com.ms.wfc.ui.HTMLControl.setURL could not be converted

com.ms.wfc.ui.IActiveXCustomPropertyDialog could not be converted

com.ms.wfc.ui.Icon.getPCTDESC could not be converted

com.ms.wfc.ui.Icon.Icon could not be converted

com.ms.wfc.ui.Icon.initialize could not be converted

com.ms.wfc.ui.Icon.loadPicture could not be converted

com.ms.wfc.ui.IHandleHook could not be converted

com.ms.wfc.ui.Image.copyHandle could not be converted

com.ms.wfc.ui.Image.dirty could not be converted

com.ms.wfc.ui.Image.drawStretchTo could not be converted

com.ms.wfc.ui.Image.drawTo could not be converted

com.ms.wfc.ui.Image.getExtension could not be converted

com.ms.wfc.ui.Image.getHandle could not be converted

com.ms.wfc.ui.Image.getPCTDESC could not be converted

com.ms.wfc.ui.Image.getPicture could not be converted

com.ms.wfc.ui.Image.Image could not be converted

com.ms.wfc.ui.Image.initialize could not be converted

com.ms.wfc.ui.Image.loadImage could not be converted

com.ms.wfc.ui.Image.loadPicture could not be converted

com.ms.wfc.ui.Image.PictureList could not be converted

com.ms.wfc.ui.ImageIndexEditor could not be converted

com.ms.wfc.ui.ImageList.createHandle could not be converted

com.ms.wfc.ui.ImageList.destroyHandle could not be converted

com.ms.wfc.ui.ImageList.getBackColor could not be converted

com.ms.wfc.ui.ImageList.getIcon could not be converted

com.ms.wfc.ui.ImageList.getMaskColor could not be converted

com.ms.wfc.ui.ImageList.getUseMask could not be converted

com.ms.wfc.ui.ImageList.ImageList could not be converted

com.ms.wfc.ui.ImageList.recreateHandle could not be converted

com.ms.wfc.ui.ImageList.SetBackColor could not be converted

com.ms.wfc.ui.ImageList.setImages could not be converted

com.ms.wfc.ui.ImageList.setMaskColor could not be converted

com.ms.wfc.ui.ImageList.setUseMask could not be converted

com.ms.wfc.ui.ImageListStreamer.getExtension could not be converted

com.ms.wfc.ui.ImageListStreamer.ImageListStreamer could not be converted

com.ms.wfc.ui.ImageListStreamer.save could not be converted

com.ms.wfc.ui.InputLangChangeEventHandler.InputLangChangeEventHandler could not be converted

com.ms.wfc.ui.InputLangChangeRequestEvent.InputLangChangeRequestEvent could not be converted

com.ms.wfc.ui.InputLangChangeRequestEventHandler.InputLangChangeRequestEventHandler could not be converted

com.ms.wfc.ui.ISelectionService.setSelectionStyle could not be converted

com.ms.wfc.ui.ItemCheckEventHandler.ItemCheckEventHandler could not be converted

com.ms.wfc.ui.ItemDragEventHandler.ItemDragEventHandler could not be converted

com.ms.wfc.ui.KeyEvent.KeyEvent could not be converted

com.ms.wfc.ui.KeyEventHandler.KeyEventHandler could not be converted

com.ms.wfc.ui.KeyPressEvent.KeyPressEvent could not be converted

com.ms.wfc.ui.KeyPressEventHandler.KeyPressEventHandler could not be converted

com.ms.wfc.ui.Label.propertyChanged could not be converted

com.ms.wfc.ui.Label.setTextAlign could not be converted

com.ms.wfc.ui.LabelEditEventHandler.LabelEditEventHandler could not be converted

com.ms.wfc.ui.LayoutEventHandler.LayoutEventHandler could not be converted

com.ms.wfc.ui.ListItem.getConstructorArgs could not be converted

com.ms.wfc.ui.ListItem.ListItem could not be converted

com.ms.wfc.ui.ListItem.ListItem(Stream) could not be converted

com.ms.wfc.ui.ListItem.ListItem(String, int, String[]) could not be converted

com.ms.wfc.ui.ListItem.ListItem(String, String[]) could not be converted

com.ms.wfc.ui.ListItem.save could not be converted

com.ms.wfc.ui.ListItem.setSubItem could not be converted

com.ms.wfc.ui.MDIWindowDialog could not be converted

com.ms.wfc.ui.MeasureItemEventHandler.MeasureItemEventHandler could not be converted

com.ms.wfc.ui.Menu.Menu could not be converted

com.ms.wfc.ui.Menu.mergeMenu could not be converted

com.ms.wfc.ui.MenuItem.setChecked could not be converted

com.ms.wfc.ui.Metafile.copyHandle could not be converted

com.ms.wfc.ui.Metafile.destroyHandle could not be converted

com.ms.wfc.ui.Metafile.drawStretchTo could not be converted

com.ms.wfc.ui.Metafile.drawTo could not be converted

com.ms.wfc.ui.Metafile.getHandle could not be converted

com.ms.wfc.ui.Metafile.getPICTDESC could not be converted

com.ms.wfc.ui.Metafile.getRenderedSize could not be converted

com.ms.wfc.ui.Metafile.initialize could not be converted

com.ms.wfc.ui.Metafile.Metafile could not be converted

com.ms.wfc.ui.MonthCalendar.propertyChanged could not be converted

com.ms.wfc.ui.MouseEvent.MouseEvent could not be converted

com.ms.wfc.ui.MouseEventHandler.MouseEventHandler could not be converted

com.ms.wfc.ui.NodeLabelEditEventHandler.NodeLabelEditEventHandler could not be converted

com.ms.wfc.ui.PaintEvent.graphics could not be converted

com.ms.wfc.ui.PaintEventHandler.PaintEventHandler could not be converted

com.ms.wfc.ui.Palette could not be converted

com.ms.wfc.ui.Palette.clone could not be converted

com.ms.wfc.ui.Palette.copyHandle could not be converted

com.ms.wfc.ui.Palette.destroyHandle could not be converted

com.ms.wfc.ui.Palette.dispose could not be converted

com.ms.wfc.ui.Palette.equals could not be converted

com.ms.wfc.ui.Palette.finalize could not be converted

com.ms.wfc.ui.Palette.getHalftonePalette could not be converted

com.ms.wfc.ui.Palette.getHandle could not be converted

com.ms.wfc.ui.Palette.getPaletteSupported could not be converted

com.ms.wfc.ui.Palette.Palette could not be converted

com.ms.wfc.ui.Pen.copyHandle could not be converted

com.ms.wfc.ui.Pen.destroyHandle could not be converted

com.ms.wfc.ui.Pen.getHandle could not be converted

com.ms.wfc.ui.Pen.Pen could not be converted

com.ms.wfc.ui.PenEntry could not be converted

com.ms.wfc.ui.PenStyle.INSIDEFRAME could not be converted

com.ms.wfc.ui.PenStyle.NULL could not be converted

com.ms.wfc.ui.PictureBox.setImage could not be converted

com.ms.wfc.ui.Point.Editor could not be converted

com.ms.wfc.ui.Point.getConstructorArgs could not be converted

com.ms.wfc.ui.Point.Point could not be converted

com.ms.wfc.ui.Point.save could not be converted

com.ms.wfc.ui.PopulatedMenusEditor could not be converted

com.ms.wfc.ui.ProgressBar.addOnValueChanged could not be converted

com.ms.wfc.ui.ProgressBar.removeOnValueChanged could not be converted

com.ms.wfc.ui.QueryContinueDragEventHandler.QueryContinueDragEventHandler could not be converted

com.ms.wfc.ui.RadioButton.setTextAlign could not be converted

com.ms.wfc.ui.Radix could not be converted

com.ms.wfc.ui.RasterOp could not be converted

com.ms.wfc.ui.ReadyStateEvent could not be converted

com.ms.wfc.ui.ReadyStateEventHandler could not be converted

com.ms.wfc.ui.Rebar.addBand could not be converted

com.ms.wfc.ui.Rebar.addOnAutoSize could not be converted

com.ms.wfc.ui.Rebar.addOnHeightChange could not be converted

com.ms.wfc.ui.Rebar.addOnLayoutChange could not be converted

com.ms.wfc.ui.Rebar.applyAutoSize could not be converted

com.ms.wfc.ui.Rebar.getBandBorders could not be converted

com.ms.wfc.ui.Rebar.getBands could not be converted

com.ms.wfc.ui.Rebar.getDoubleClickToggle could not be converted

com.ms.wfc.ui.Rebar.getFixedOrder could not be converted

com.ms.wfc.ui.Rebar.getOrientation could not be converted

com.ms.wfc.ui.Rebar.onAutoSize could not be converted

com.ms.wfc.ui.Rebar.onHeightChange could not be converted

com.ms.wfc.ui.Rebar.onLayoutChange could not be converted

com.ms.wfc.ui.Rebar.removeAllBands could not be converted

com.ms.wfc.ui.Rebar.removeBand could not be converted

com.ms.wfc.ui.Rebar.removeOnAutoSize could not be converted

com.ms.wfc.ui.Rebar.removeOnHeightChange could not be converted

com.ms.wfc.ui.Rebar.removeOnLayoutChange could not be converted

com.ms.wfc.ui.Rebar.setBandBorders could not be converted

com.ms.wfc.ui.Rebar.setBands could not be converted

com.ms.wfc.ui.Rebar.setComponentSite could not be converted

com.ms.wfc.ui.Rebar.setDoubleClickToggle could not be converted

com.ms.wfc.ui.Rebar.setFixedOrder could not be converted

com.ms.wfc.ui.Rebar.setNewControls could not be converted

com.ms.wfc.ui.Rebar.setOrientation could not be converted

com.ms.wfc.ui.RebarBand.getAllowVariableHeight could not be converted

com.ms.wfc.ui.RebarBand.getAllowVertical could not be converted

com.ms.wfc.ui.RebarBand.getBandBreak could not be converted

com.ms.wfc.ui.RebarBand.getChildControl could not be converted

com.ms.wfc.ui.RebarBand.getChildEdge could not be converted

com.ms.wfc.ui.RebarBand.getFixedBitmap could not be converted

com.ms.wfc.ui.RebarBand.getGrowBy could not be converted

com.ms.wfc.ui.RebarBand.getHeaderWidth could not be converted

com.ms.wfc.ui.RebarBand.getIdealWidth could not be converted

com.ms.wfc.ui.RebarBand.getImageIndex could not be converted
com.ms.wfc.ui.RebarBand.getIndex could not be converted
com.ms.wfc.ui.RebarBand.getMaxInitialHeight could not be converted
com.ms.wfc.ui.RebarBand.getMinChildHeight could not be converted
com.ms.wfc.ui.RebarBand.getMinChildWidth could not be converted
com.ms.wfc.ui.RebarBand.getVisibleGripper could not be converted
com.ms.wfc.ui.RebarBand.maximize could not be converted
com.ms.wfc.ui.RebarBand.minimize could not be converted
com.ms.wfc.ui.RebarBand.setAllowVariableHeight could not be converted
com.ms.wfc.ui.RebarBand.setAllowVertical could not be converted
com.ms.wfc.ui.RebarBand.setBandBreak could not be converted
com.ms.wfc.ui.RebarBand.setChildControl could not be converted
com.ms.wfc.ui.RebarBand.setChildEdge could not be converted
com.ms.wfc.ui.RebarBand.setFixedBitmap could not be converted
com.ms.wfc.ui.RebarBand.setGrowBy could not be converted
com.ms.wfc.ui.RebarBand.setHeaderWidth could not be converted
com.ms.wfc.ui.RebarBand.setIdealWidth could not be converted
com.ms.wfc.ui.RebarBand.setImageIndex could not be converted
com.ms.wfc.ui.RebarBand.setIndex could not be converted
com.ms.wfc.ui.RebarBand.setMaxInitialHeight could not be converted
com.ms.wfc.ui.RebarBand.setMinChildHeight could not be converted
com.ms.wfc.ui.RebarBand.setMinChildWidth could not be converted
com.ms.wfc.ui.RebarBand.setVisibleGripper could not be converted
com.ms.wfc.ui.RebarBand.updateStyle could not be converted
com.ms.wfc.ui.Rectangle.Editor.Editor could not be converted
com.ms.wfc.ui.Rectangle.getConstructorArgs could not be converted
com.ms.wfc.ui.Rectangle.Rectangle could not be converted
com.ms.wfc.ui.Rectangle.save could not be converted
com.ms.wfc.ui.Rectangle.setBounds could not be converted
com.ms.wfc.ui.Rectangle.toRECT could not be converted
com.ms.wfc.ui.Region.copyHandle could not be converted
com.ms.wfc.ui.Region.createPolygonal could not be converted
com.ms.wfc.ui.Region.destroyHandle could not be converted
com.ms.wfc.ui.Region.getHandle could not be converted
com.ms.wfc.ui.RequestResizeEventHandler.RequestResizeEventHandler could not be converted
com.ms.wfc.ui.RichEdit.DLL_RICHEDIT could not be converted
com.ms.wfc.ui.RichEdit.getDelimiter could not be converted
com.ms.wfc.ui.RichEdit.getFollowPunctuation could not be converted
com.ms.wfc.ui.RichEdit.getIMEColor could not be converted
com.ms.wfc.ui.RichEdit.getIMEOptions could not be converted

com.ms.wfc.ui.RichEdit.getLeadPunctuation could not be converted
com.ms.wfc.ui.RichEdit.getOnHScroll could not be converted
com.ms.wfc.ui.RichEdit.getOnIMEChange could not be converted
com.ms.wfc.ui.RichEdit.getOnProtected could not be converted
com.ms.wfc.ui.RichEdit.getOnRequestResize could not be converted
com.ms.wfc.ui.RichEdit.getOnSelChange could not be converted
com.ms.wfc.ui.RichEdit.getOnVScroll could not be converted
com.ms.wfc.ui.RichEdit.getWordBreak could not be converted
com.ms.wfc.ui.RichEdit.getWordPunctuation could not be converted
com.ms.wfc.ui.RichEdit.moveInsertionPoint could not be converted
com.ms.wfc.ui.RichEdit.RICHEDIT_CLASS10A could not be converted
com.ms.wfc.ui.RichEdit.RICHEDIT_CLASSA could not be converted
com.ms.wfc.ui.RichEdit.RICHEDIT_CLASSW could not be converted
com.ms.wfc.ui.RichEdit.RICHEDIT_DLL10 could not be converted
com.ms.wfc.ui.RichEdit.RICHEDIT_DLL20 could not be converted
com.ms.wfc.ui.RichEdit.setFollowPunctuation could not be converted
com.ms.wfc.ui.RichEdit.setIMEColor could not be converted
com.ms.wfc.ui.RichEdit.setIMEOptions could not be converted
com.ms.wfc.ui.RichEdit.setLeadPunctuation could not be converted
com.ms.wfc.ui.RichEdit.setWordBreak could not be converted
com.ms.wfc.ui.RichEdit.setWordPunctuation could not be converted
com.ms.wfc.ui.RichEdit.span could not be converted
com.ms.wfc.ui.RichEdit.WC_RICHEDIT could not be converted
com.ms.wfc.ui.RichEditIMEColor could not be converted
com.ms.wfc.ui.RichEditIMEOptions could not be converted
com.ms.wfc.ui.SameParentReferenceEditor could not be converted
com.ms.wfc.ui.SaveFileDialog.runFileDialog could not be converted
com.ms.wfc.ui.SaveFileDialog.setCreatePrompt could not be converted
com.ms.wfc.ui.Screen.MonitorEnumProc could not be converted
com.ms.wfc.ui.Screen.MONITORINFO could not be converted
com.ms.wfc.ui.Screen.MONITORINFOEX could not be converted
com.ms.wfc.ui.ScrollBar.propertyChanged could not be converted
com.ms.wfc.ui.ScrollEvent.ScrollEvent could not be converted
com.ms.wfc.ui.ScrollEventHandler.ScrollEventHandler could not be converted
com.ms.wfc.ui.SelectionChangedEventHandler.SelectionChangedEventHandler could not be converted
com.ms.wfc.ui.SelectionRange.Editor could not be converted
com.ms.wfc.ui.SelectionRange.getConstructorArgs could not be converted
com.ms.wfc.ui.SelectionStyle could not be converted
com.ms.wfc.ui.Shortcut.Editor could not be converted
com.ms.wfc.ui.Splitter.postFilterMessage could not be converted

com.ms.wfc.ui.SplitterEventHandler.SplitterEventHandler could not be converted

com.ms.wfc.ui.State.save could not be converted

com.ms.wfc.ui.State.State could not be converted

com.ms.wfc.ui.StatusBarDrawItemEventHandler.StatusBarDrawItemEventHandler could not be converted

com.ms.wfc.ui.StatusBarPanelClickEventHandler.StatusBarPanelClickEventHandler could not be converted

com.ms.wfc.ui.TabBase.getTCITEM could not be converted

com.ms.wfc.ui.TabBase.propertyChanged could not be converted

com.ms.wfc.ui.TabBase.setSelectedIndex could not be converted

com.ms.wfc.ui.TabStrip.insertTab could not be converted

com.ms.wfc.ui.TextFormat.BOTTOM could not be converted

com.ms.wfc.ui.TextFormat.EDITCONTROL could not be converted

com.ms.wfc.ui.TextFormat.ENDELLIPSIS could not be converted

com.ms.wfc.ui.TextFormat.EXPANDTABS could not be converted

com.ms.wfc.ui.TextFormat.HORIZONTALCENTER could not be converted

com.ms.wfc.ui.TextFormat.LEFT could not be converted

com.ms.wfc.ui.TextFormat.NOPREFIX could not be converted

com.ms.wfc.ui.TextFormat.PATHELLIPSIS could not be converted

com.ms.wfc.ui.TextFormat.RIGHT could not be converted

com.ms.wfc.ui.TextFormat.RIGHTTOLEFT could not be converted

com.ms.wfc.ui.TextFormat.SINGLELINE could not be converted

com.ms.wfc.ui.TextFormat.TextFormat could not be converted

com.ms.wfc.ui.TextFormat.TOP could not be converted

com.ms.wfc.ui.TextFormat.valid could not be converted

com.ms.wfc.ui.TextFormat.VERTICALCENTER could not be converted

com.ms.wfc.ui.TextFormat.WORDBREAK could not be converted

com.ms.wfc.ui.Toolbar.propertyChanged could not be converted

com.ms.wfc.ui.ToolBarTextAlign.RIGHT could not be converted

com.ms.wfc.ui.ToolBarTextAlign.UNDERNEATH could not be converted

com.ms.wfc.ui.ToolTip.shouldPersistAutomaticDelay could not be converted

com.ms.wfc.ui.ToolTip.shouldPersistAutoPopDelay could not be converted

com.ms.wfc.ui.ToolTip.shouldPersistInitialDelay could not be converted

com.ms.wfc.ui.ToolTip.shouldPersistReshowDelay could not be converted

com.ms.wfc.ui.TrackBar.propertyChanged could not be converted

com.ms.wfc.ui.TreeNode.getConstructorArgs could not be converted

com.ms.wfc.ui.TreeNode.save could not be converted

com.ms.wfc.ui.TreeNode.TreeNode could not be converted

com.ms.wfc.ui.TreeNodeNodesEditDialog could not be converted

com.ms.wfc.ui.TreeNodeNodesEditor could not be converted

com.ms.wfc.ui.TreeView.shouldPersistIndent could not be converted

com.ms.wfc.ui.TreeViewCancelEventHandler.TreeViewCancelEventHandler could not be converted

com.ms.wfc.ui.TreeViewEventHandler.TreeViewEventHandler could not be converted

com.ms.wfc.ui.UpDown.getAcceleration could not be converted

com.ms.wfc.ui.UpDown.getAutoBuddy could not be converted

com.ms.wfc.ui.UpDown.getAutoSize could not be converted

com.ms.wfc.ui.UpDown.getBuddyControl could not be converted

com.ms.wfc.ui.UpDown.getHorizontal could not be converted

com.ms.wfc.ui.UpDown.getModifyBuddy could not be converted

com.ms.wfc.ui.UpDown.getRadix could not be converted

com.ms.wfc.ui.UpDown.getWrap could not be converted

com.ms.wfc.ui.UpDown.onCreateHandle could not be converted

com.ms.wfc.ui.UpDown.setAcceleration could not be converted

com.ms.wfc.ui.UpDown.setAutoBuddy could not be converted

com.ms.wfc.ui.UpDown.setAutoSize could not be converted

com.ms.wfc.ui.UpDown.setBuddyControl could not be converted

com.ms.wfc.ui.UpDown.setHorizontal could not be converted

com.ms.wfc.ui.UpDown.setModifyBuddy could not be converted

com.ms.wfc.ui.UpDown.setRadix could not be converted

com.ms.wfc.ui.UpDown.setWrap could not be converted

com.ms.wfc.ui.UpDown.shouldPersistBuddyControl could not be converted

com.ms.wfc.ui.UpDownAcceleration could not be converted

com.ms.wfc.ui.UpDownAcceleration.Editor could not be converted

com.ms.wfc.ui.UpDownAlignment.MANUAL could not be converted

com.ms.wfc.ui.UpDownChangedEvent.delta could not be converted

com.ms.wfc.ui.UpDownChangedEvent.value could not be converted

com.ms.wfc.ui.UpDownChangedEventHandler.UpDownChangedEventHandler could not be converted

Com.ms.wfc.util Error Messages

com.ms.wfc.util.ArrayEnumerator.hasMoreItems could not be converted

com.ms.wfc.util.Debug.addOnDisplayAssert could not be converted

com.ms.wfc.util.Debug.addOnDisplayMessage could not be converted

com.ms.wfc.util.Debug.Debug could not be converted

com.ms.wfc.util.Debug.displaySwitches could not be converted

com.ms.wfc.util.Debug.getSwitchListText could not be converted

com.ms.wfc.util.Debug.printEnumIf could not be converted

com.ms.wfc.util.Debug.printExceptionIf could not be converted

com.ms.wfc.util.Debug.printf could not be converted

com.ms.wfc.util.Debug.printlnIf could not be converted

com.ms.wfc.util.Debug.printObjectIf could not be converted

com.ms.wfc.util.Debug.printSwitchList could not be converted

com.ms.wfc.util.Debug.printStackTraceIf could not be converted

com.ms.wfc.util.Debug.removeOnDisplayAssert could not be converted

com.ms.wfc.util.Debug.removeOnDisplayMessage could not be converted

com.ms.wfc.util.DebugMessageEventHandler could not be converted

com.ms.wfc.util.HandleCollector could not be converted

com.ms.wfc.util.HashTable.addSlot could not be converted

com.ms.wfc.util.HashTable.buckets could not be converted

com.ms.wfc.util.HashTable.findItem could not be converted

com.ms.wfc.util.HashTable.free could not be converted

com.ms.wfc.util.HashTable.getKeys could not be converted

com.ms.wfc.util.HashTable.getValues could not be converted

com.ms.wfc.util.HashTable.growHashTable could not be converted

com.ms.wfc.util.HashTable.HashTable could not be converted

com.ms.wfc.util.HashTable.hashValues could not be converted

com.ms.wfc.util.HashTable.iMax could not be converted

com.ms.wfc.util.HashTable.keys could not be converted

com.ms.wfc.util.HashTable.maxAverageDepth could not be converted

com.ms.wfc.util.HashTable.rehash could not be converted

com.ms.wfc.util.HashTable.removeSlot could not be converted

com.ms.wfc.util.HashTable.removeValue could not be converted

com.ms.wfc.util.HashTable.sizes could not be converted

com.ms.wfc.util.HashTable.values could not be converted

com.ms.wfc.util.IEnumerator.hasMoreItems could not be converted

com.ms.wfc.util.IEnumerator.nextItem could not be converted

com.ms.wfc.util.List.capInc could not be converted

com.ms.wfc.util.List.clone could not be converted
com.ms.wfc.util.List.getVersion could not be converted
com.ms.wfc.util.List.version could not be converted
com.ms.wfc.util.NumberFormat could not be converted
com.ms.wfc.util.Root.checkLeaks could not be converted
com.ms.wfc.util.Root.get could not be converted
com.ms.wfc.util.StringSorter.compare could not be converted
com.ms.wfc.util.StringSorter.DECENDING could not be converted
com.ms.wfc.util.StringSorter.sort could not be converted
com.ms.wfc.util.StringSorter.sort(Locale, String, Object, int, int, int) could not be converted
com.ms.wfc.util.Utls.getJavalIdentifier could not be converted
com.ms.wfc.util.Utls.getPrimitiveWord could not be converted
com.ms.wfc.util.Utls.getReservedWord could not be converted
com.ms.wfc.util.Utls.validateIdentifier could not be converted
com.ms.wfc.util.Value.format could not be converted
com.ms.wfc.util.Value.formatCurrency could not be converted
com.ms.wfc.util.Value.formatNumber could not be converted
com.ms.wfc.util.Value.SysFreeString could not be converted

Com.ms.wfc.win32 Error Messages

com.ms.wfc.win32.CharBuffer could not be converted

com.ms.wfc.win32.Windows.ACCELERATORHANDLE could not be converted

com.ms.wfc.win32.Windows.BeginPaint could not be converted

com.ms.wfc.win32.Windows.CloseEnhMetaFile could not be converted

com.ms.wfc.win32.Windows.CloseHandle could not be converted

com.ms.wfc.win32.Windows.CopyEnhMetaFile could not be converted

com.ms.wfc.win32.Windows.CopyImage could not be converted

com.ms.wfc.win32.Windows.CreateAcceleratorTable could not be converted

com.ms.wfc.win32.Windows.CreateBitmap could not be converted

com.ms.wfc.win32.Windows.CreateBitmap(int, int, int, int, int[]) could not be converted

com.ms.wfc.win32.Windows.CreateBitmap(int, int, int, int, short[]) could not be converted

com.ms.wfc.win32.Windows.CreateBrushIndirect could not be converted

com.ms.wfc.win32.Windows.CreateCompatibleBitmap could not be converted

com.ms.wfc.win32.Windows.CreateCompatibleDC could not be converted

com.ms.wfc.win32.Windows.CreateDC could not be converted

com.ms.wfc.win32.Windows.CreateDIBitmap could not be converted

com.ms.wfc.win32.Windows.CreateEllipticRgn could not be converted

com.ms.wfc.win32.Windows.CreateEllipticRgnIndirect could not be converted

com.ms.wfc.win32.Windows.CreateEnhMetaFile could not be converted

com.ms.wfc.win32.Windows.CreateFile could not be converted

com.ms.wfc.win32.Windows.CreateFont could not be converted

com.ms.wfc.win32.Windows.CreateFontIndirect could not be converted

com.ms.wfc.win32.Windows.CreateHalftonePalette could not be converted

com.ms.wfc.win32.Windows.CreateHatchBrush could not be converted

com.ms.wfc.win32.Windows.CreateIC could not be converted

com.ms.wfc.win32.Windows.CreateILockBytesOnHGlobal could not be converted

com.ms.wfc.win32.Windows.CreateMappedBitmap could not be converted

com.ms.wfc.win32.Windows.CreateMenu could not be converted

com.ms.wfc.win32.Windows.CreatePalette could not be converted

com.ms.wfc.win32.Windows.CreatePatternBrush could not be converted

com.ms.wfc.win32.Windows.CreatePen could not be converted

com.ms.wfc.win32.Windows.CreatePenIndirect could not be converted

com.ms.wfc.win32.Windows.CreatePolygonRgn could not be converted

com.ms.wfc.win32.Windows.CreatePolyPolygonRgn could not be converted

com.ms.wfc.win32.Windows.CreatePopupMenu could not be converted

com.ms.wfc.win32.Windows.CreateProcess could not be converted

com.ms.wfc.win32.Windows.CreateRectRgn could not be converted

com.ms.wfc.win32.Windows.CreateRectRgnIndirect could not be converted
com.ms.wfc.win32.Windows.CreateRoundRectRgn could not be converted
com.ms.wfc.win32.Windows.CreateSolidBrush could not be converted
com.ms.wfc.win32.Windows.CreateWindowEx could not be converted
com.ms.wfc.win32.Windows.CURSORHANDLE could not be converted
com.ms.wfc.win32.Windows.DeleteDC could not be converted
com.ms.wfc.win32.Windows.DeleteEnhMetaFile could not be converted
com.ms.wfc.win32.Windows.DeleteObject could not be converted
com.ms.wfc.win32.Windows.DestroyAcceleratorTable could not be converted
com.ms.wfc.win32.Windows.DestroyCursor could not be converted
com.ms.wfc.win32.Windows.DestroyIcon could not be converted
com.ms.wfc.win32.Windows.DestroyMenu could not be converted
com.ms.wfc.win32.Windows.DestroyWindow could not be converted
com.ms.wfc.win32.Windows.DoDragDrop could not be converted
com.ms.wfc.win32.Windows.DrawState could not be converted
com.ms.wfc.win32.Windows.DuplicateHandle could not be converted
com.ms.wfc.win32.Windows.EMFHANDLE could not be converted
com.ms.wfc.win32.Windows.EndPaint could not be converted
com.ms.wfc.win32.Windows.EnumObjects could not be converted
com.ms.wfc.win32.Windows.EnumThreadWindows could not be converted
com.ms.wfc.win32.Windows.FindClose could not be converted
com.ms.wfc.win32.Windows.FindFirstFile could not be converted
com.ms.wfc.win32.Windows.FINDHANDLE could not be converted
com.ms.wfc.win32.Windows.GDIHANDLE could not be converted
com.ms.wfc.win32.Windows.GetDC could not be converted
com.ms.wfc.win32.Windows.GetDCEx could not be converted
com.ms.wfc.win32.Windows.GetEnhMetaFile could not be converted
com.ms.wfc.win32.Windows.GetHGlobalFromLockBytes could not be converted
com.ms.wfc.win32.Windows.GetWindowDC could not be converted
com.ms.wfc.win32.Windows.HDCHANDLE could not be converted
com.ms.wfc.win32.Windows.ICONHANDLE could not be converted
com.ms.wfc.win32.Windows.InvalidateRect could not be converted
com.ms.wfc.win32.Windows.KERNELHANDLE could not be converted
com.ms.wfc.win32.Windows.MENUHANDLE could not be converted
com.ms.wfc.win32.Windows.OleCreatePictureIndirect could not be converted
com.ms.wfc.win32.Windows.PathToRegion could not be converted
com.ms.wfc.win32.Windows.RegisterDragDrop could not be converted
com.ms.wfc.win32.Windows.ReleaseDC could not be converted
com.ms.wfc.win32.Windows.SetEnhMetaFileBits could not be converted
com.ms.wfc.win32.Windows.SetMetaFileBitsEx could not be converted

com.ms.wfc.win32.Windows.SetWindowRgn could not be converted

com.ms.wfc.win32.Windows.SetWinMetaFileBits could not be converted

com.ms.wfc.win32.Windows.StgCreateDocfileOnILockBytes could not be converted

com.ms.wfc.win32.Windows.StgOpenStorageOnILockBytes could not be converted

com.ms.wfc.win32.Windows.WINDOWHANDLE could not be converted

Com.ms.win32 Error Messages

[com.ms.win32.Kernel32.CreateFiber could not be converted](#)

[com.ms.win32.Ole32.CoCreateInstanceEx could not be converted](#)

[com.ms.win32.Ole32.CoGetInstanceFromFile could not be converted](#)

[com.ms.win32.Ole32.CoGetInstanceFromIStorage could not be converted](#)

[com.ms.win32.Ole32.CreateILockBytesOnHGlobal could not be converted](#)

[com.ms.win32.Ole32.GetHGlobalFromILockBytes could not be converted](#)

[com.ms.win32.Ole32.StgCreateDocfile could not be converted](#)

[com.ms.win32.Ole32.StgCreateStorageEx could not be converted](#)

[com.ms.win32.Ole32.StgOpenLayoutDocfile could not be converted](#)

[com.ms.win32.Ole32.StgOpenStorage could not be converted](#)

[com.ms.win32.Spoolss.EnumPrinters could not be converted](#)

Com.sun.image.codec Error Messages

com.sun.image.codec.jpeg.ImageFormatException could not be converted

com.sun.image.codec.jpeg.JPEGCodec could not be converted

com.sun.image.codec.jpeg.JPEGCodec.createJPGDecoder could not be converted

com.sun.image.codec.jpeg.JPEGCodec.createJPGEncoder could not be converted

com.sun.image.codec.jpeg.JPEGCodec.getDefaultJPGEncodeParam(BufferedImage) could not be converted

com.sun.image.codec.jpeg.JPEGCodec.getDefaultJPGEncodeParam(int,int) could not be converted

com.sun.image.codec.jpeg.JPEGCodec.getDefaultJPGEncodeParam(JPGDecodeParam) could not be converted

com.sun.image.codec.jpeg.JPEGCodec.getDefaultJPGEncodeParam(JPGEncodeParam) could not be converted

com.sun.image.codec.jpeg.JPEGCodec.getDefaultJPGEncodeParam(Raster, int) could not be converted

com.sun.image.codec.jpeg.JPGDecodeParam could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.addMarkerData could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setACHuffmanComponentMapping could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setACHuffmanTable could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setDCHuffmanComponentMapping could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setDCHuffmanTable could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setDensityUnit could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setHorizontalSubsampling could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setImageInfoValid could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setMarkerData could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setQTable could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setQTableComponentMapping could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setQuality could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setRestartInterval could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setTableInfoValid could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setVerticalSubsampling could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setXDensity could not be converted

com.sun.image.codec.jpeg.JPGEncodeParam.setYDensity could not be converted

com.sun.image.codec.jpeg.JPGHuffmanTable could not be converted

com.sun.image.codec.jpeg.JPGImageDecoder could not be converted

com.sun.image.codec.jpeg.JPGImageDecoder.decodeAsBufferedImage could not be converted

com.sun.image.codec.jpeg.JPGImageDecoder.decodeAsRaster could not be converted

com.sun.image.codec.jpeg.JPGImageDecoder.getInputStream could not be converted

com.sun.image.codec.jpeg.JPGImageDecoder.getJPGDecodeParam could not be converted

com.sun.image.codec.jpeg.JPGImageEncoder could not be converted

com.sun.image.codec.jpeg.JPGImageEncoder.encode(BufferedImage) could not be converted

com.sun.image.codec.jpeg.JPGImageEncoder.encode(BufferedImage, JPGEncodeParam) could not be converted

com.sun.image.codec.jpeg.JPGImageEncoder.encode(Raster) could not be converted

com.sun.image.codec.jpeg.JPGImageEncoder.encode(Raster, JPGEncodeParam) could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.getDefaultColorId could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.getDefaultJPEGEncodeParam(JPEGDecodeParam) could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.getDefaultJPEGEncodeParam(JPEGEncodeParam) could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.getDefaultJPEGEncodeParam(JPEGEncodeParam, BufferedImage) could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.getDefaultJPEGEncodeParam(JPEGEncodeParam, int, int) could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.getDefaultJPEGEncodeParam(JPEGEncodeParam, Raster,int) could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.getJPEGEncodeParam could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.getOutputStream could not be converted

com.sun.image.codec.jpeg.JPEGImageEncoder.setJPEGEncodeParam could not be converted

com.sun.image.codec.jpeg.JPEGQTable could not be converted

com.sun.image.codec.jpeg.TruncatedFileException could not be converted

Java.applet Error Messages

java.applet.Applet could not be converted

java.applet.Applet.destroy could not be converted

java.applet.Applet.getAccessibleContext could not be converted

java.applet.Applet.getAppletContext could not be converted

java.applet.Applet.getAppletInfo could not be converted

java.applet.Applet.getAudioClip could not be converted

java.applet.Applet.getCodeBase could not be converted

java.applet.Applet.getDocumentBase could not be converted

java.applet.Applet.getImage could not be converted

java.applet.Applet.getLocale could not be converted

java.applet.Applet.getParameter could not be converted

java.applet.Applet.getParameterInfo could not be converted

java.applet.Applet.init could not be converted

java.applet.Applet.isActive could not be converted

java.applet.Applet.newAudioClip could not be converted

java.applet.Applet.play could not be converted

java.applet.Applet.resize could not be converted

java.applet.Applet.setStub could not be converted

java.applet.Applet.showStatus could not be converted

java.applet.Applet.start could not be converted

java.applet.Applet.stop could not be converted

java.applet.AppletContext could not be converted

java.applet.AppletStub could not be converted

java.applet.AudioClip could not be converted

Java.awt Error Messages

java.awt.<ClassName>.add*Listener could not be converted

java.awt.<ClassName>.addNotify could not be converted

java.awt.<ClassName>.getPeer could not be converted

java.awt.<ClassName>.print* could not be converted

java.awt.<ClassName>.processActionEvent could not be converted

java.awt.<ClassName>.processEvent could not be converted

java.awt.<ClassName>.processItemEvent could not be converted

java.awt.<ClassName>.remove*Listener could not be converted

java.awt.<ClassName>.removeNotify could not be converted

java.awt.ActiveEvent could not be converted

java.awt.Adjustable could not be converted

java.awt.AlphaComposite could not be converted

java.awt.AWTError could not be converted

java.awt.AWTEvent could not be converted

java.awt.AWTEvent.<EventType> could not be converted

java.awt.AWTEvent.AWTEvent could not be converted

java.awt.AWTEvent.consume could not be converted

java.awt.AWTEvent.consumed could not be converted

java.awt.AWTEvent.getID could not be converted

java.awt.AWTEvent.id could not be converted

java.awt.AWTEvent.isConsumed could not be converted

java.awt.AWTEventMulticaster could not be converted

java.awt.AWTPermission.AWTPermission could not be converted

java.awt.BasicStroke.BasicStroke could not be converted

java.awt.BasicStroke.createStrokedShape could not be converted

java.awt.BorderLayout could not be converted

java.awt.Button.getActionCommand could not be converted

java.awt.Button.getListeners could not be converted

java.awt.Button.setActionCommand could not be converted

java.awt.Canvas could not be converted

java.awt.Canvas.Canvas could not be converted

java.awt.Canvas.paint could not be converted

java.awt.CardLayout could not be converted

java.awt.Checkbox.getListeners could not be converted

java.awt.CheckboxGroup.getCurrent could not be converted

java.awt.CheckboxGroup.getSelectedCheckbox could not be converted

java.awt.CheckboxMenuItem.getAccessibleContext could not be converted

java.awt.CheckboxMenuItem.getListeners could not be converted

java.awt.Choice could not be converted

java.awt.Choice.getListeners could not be converted

java.awt.Choice.getSelectedIndex could not be converted

java.awt.Choice.getSelectedItem could not be converted

java.awt.Color.Color(ColorSpace, float[], float) could not be converted

java.awt.Color.Color(float, float, float, float) could not be converted

java.awt.Color.Color(int) could not be converted

java.awt.Color.Color(int, boolean) could not be converted

java.awt.Color.Color(int, int, int,int) could not be converted

java.awt.Color.createContext could not be converted

java.awt.Color.decode could not be converted

java.awt.Color.getColor could not be converted

java.awt.Color.getColorComponents(ColorSpace, float[]) could not be converted

java.awt.Color.getColorComponents(float[]) could not be converted

java.awt.Color.getColorSpace could not be converted

java.awt.Color.getComponents(ColorSpace, float[]) could not be converted

java.awt.Color.getComponents(float[]) could not be converted

java.awt.Color.getHSBColor could not be converted

java.awt.Color.getRGBColorComponents could not be converted

java.awt.Color.getRGBComponents could not be converted

java.awt.Color.getTransparency could not be converted

java.awt.Color.HSBtoRGB could not be converted

java.awt.Color.RGBtoHSB could not be converted

java.awt.Component.action could not be converted

java.awt.Component.addHierarchyBoundsListener could not be converted

java.awt.Component.addHierarchyListener could not be converted

java.awt.Component.addInputMethodListener could not be converted

java.awt.Component.addPropertyChangeListener could not be converted

java.awt.Component.BOTTOM_ALIGNMENT could not be converted

java.awt.Component.CENTER_ALIGNMENT could not be converted

java.awt.Component.checkImage could not be converted

java.awt.Component.coalesceEvents could not be converted

java.awt.Component.createImage(ImageProducer) could not be converted

java.awt.Component.createImage(int, int) could not be converted

java.awt.Component.deliverEvent could not be converted

java.awt.Component.disable could not be converted

java.awt.Component.disableEvents could not be converted

java.awt.Component.dispatchEvent could not be converted

java.awt.Component.enableEvents could not be converted

java.awt.Component.enableInputMethods could not be converted
java.awt.Component.firePropertyChange could not be converted
java.awt.Component.getAlignmentX could not be converted
java.awt.Component.getAlignmentY could not be converted
java.awt.Component.getColorModel could not be converted
java.awt.Component.getDropTarget could not be converted
java.awt.Component.getGraphicsConfiguration could not be converted
java.awt.Component.getInputContext could not be converted
java.awt.Component.getInputMethodRequests could not be converted
java.awt.Component.listeners could not be converted
java.awt.Component.getLocale could not be converted
java.awt.Component.getMaximumSize could not be converted
java.awt.Component.getMinimumSize could not be converted
java.awt.Component.getPreferredSize could not be converted
java.awt.Component.getSize could not be converted
java.awt.Component.getToolkit could not be converted
java.awt.Component.getTreeLock could not be converted
java.awt.Component.handleEvent could not be converted
java.awt.Component.hasFocus could not be converted
java.awt.Component.imageUpdate could not be converted
java.awt.Component.isDisplayable could not be converted
java.awt.Component.isDoubleBuffered could not be converted
java.awt.Component.isLightweight could not be converted
java.awt.Component.isOpaque could not be converted
java.awt.Component.isValid could not be converted
java.awt.Component.isVisible could not be converted
java.awt.Component.keyDown could not be converted
java.awt.Component.keyUp could not be converted
java.awt.Component.LEFT_ALIGNMENT could not be converted
java.awt.Component.lostFocus could not be converted
java.awt.Component.minimumSize could not be converted
java.awt.Component.mouseDown could not be converted
java.awt.Component.mouseDrag could not be converted
java.awt.Component.mouseEnter could not be converted
java.awt.Component.mouseExit could not be converted
java.awt.Component.mouseMove could not be converted
java.awt.Component.mouseUp could not be converted
java.awt.Component.move could not be converted
java.awt.Component.nextFocus could not be converted
java.awt.Component.paint could not be converted

java.awt.Component.paintAll could not be converted

java.awt.Component.postEvent could not be converted

java.awt.Component.preferredSize could not be converted

java.awt.Component.prepareImage could not be converted

java.awt.Component.process*Event could not be converted

java.awt.Component.remove could not be converted

java.awt.Component.removeInputMethodListener could not be converted

java.awt.Component.removePropertyChangeListener could not be converted

java.awt.Component.repaint could not be converted

java.awt.Component.repaint(int, int, int, int) could not be converted

java.awt.Component.repaint(long) could not be converted

java.awt.Component.repaint(long, int, int, int, int) could not be converted

java.awt.Component.RIGHT_ALIGNMENT could not be converted

java.awt.Component.setBounds could not be converted

java.awt.Component.setComponentOrientation could not be converted

java.awt.Component.setDropTarget could not be converted

java.awt.Component.setEnabled could not be converted

java.awt.Component.setLocale could not be converted

java.awt.Component.setLocation could not be converted

java.awt.Component.setSize could not be converted

java.awt.Component.setVisible could not be converted

java.awt.Component.TOP_ALIGNMENT could not be converted

java.awt.Component.transferFocus could not be converted

java.awt.Component.validate could not be converted

java.awt.ComponentOrientation.getOrientation(Locale) could not be converted

java.awt.ComponentOrientation.getOrientation(ResourceBundle) could not be converted

java.awt.ComponentOrientation.isHorizontal could not be converted

java.awt.Composite could not be converted

java.awt.CompositeContext could not be converted

java.awt.Container.add(Component) could not be converted

java.awt.Container.add(Component, int) could not be converted

java.awt.Container.add(Component, Object) could not be converted

java.awt.Container.add(Component, Object, int) could not be converted

java.awt.Container.add(String, Component) could not be converted

java.awt.Container.deliverEvent could not be converted

java.awt.Container.getComponent could not be converted

java.awt.Container.getComponentAt(int, int) could not be converted

java.awt.Container.getComponentAt(Point) could not be converted

java.awt.Container.getLayout could not be converted

java.awt.Container.listeners could not be converted

java.awt.Container.paint* could not be converted

java.awt.Container.processContainerEvent could not be converted

java.awt.Container.setLayout could not be converted

java.awt.Container.validate could not be converted

java.awt.Container.validateTree could not be converted

java.awt.Cursor.Cursor could not be converted

java.awt.Cursor.CUSTOM_CURSOR could not be converted

java.awt.Cursor.getName could not be converted

java.awt.Cursor.getSystemCustomCursor could not be converted

java.awt.Cursor.name could not be converted

java.awt.Cursor.predefined could not be converted

java.awt.Dialog.Dialog could not be converted

java.awt.Dialog.hide could not be converted

java.awt.Dialog.setModal could not be converted

java.awt.Dialog.show could not be converted

java.awt.Dimension.setSize could not be converted

java.awt.Dimension.toString could not be converted

java.awt.Event could not be converted

java.awt.EventQueue could not be converted

java.awt.FileDialog.*FilenameFilter could not be converted

java.awt.FileDialog.FileDialog(Frame) could not be converted

java.awt.FileDialog.FileDialog(Frame, String) could not be converted

java.awt.FileDialog.FileDialog(Frame, String, int) could not be converted

java.awt.FileDialog.getFile could not be converted

java.awt.FileDialog.setMode could not be converted

java.awt.FlowLayout could not be converted

java.awt.FlowLayout.addLayoutComponent could not be converted

java.awt.FlowLayout.FlowLayout could not be converted

java.awt.FlowLayout.FlowLayout(int) could not be converted

java.awt.FlowLayout.FlowLayout(int, int, int) could not be converted

java.awt.FlowLayout.getAlignment could not be converted

java.awt.FlowLayout.getHgap could not be converted

java.awt.FlowLayout.getVgap could not be converted

java.awt.FlowLayout.layoutContainer could not be converted

java.awt.FlowLayout.minimumLayoutSize could not be converted

java.awt.FlowLayout.preferredLayoutSize could not be converted

java.awt.FlowLayout.removeLayoutComponent could not be converted

java.awt.FlowLayout.setAlignment could not be converted

java.awt.FlowLayout.setHgap could not be converted

java.awt.FlowLayout.setVgap could not be converted

java.awt.FlowLayout.toString could not be converted

java.awt.Font.canDisplay could not be converted

java.awt.Font.canDisplayUpTo could not be converted

java.awt.Font.CENTER_BASELINE could not be converted

java.awt.Font.createFont could not be converted

java.awt.Font.createGlyphVector could not be converted

java.awt.Font.deriveFont(AffineTransform) could not be converted

java.awt.Font.deriveFont(int, AffineTransform) could not be converted

java.awt.Font.deriveFont(Map) could not be converted

java.awt.Font.Font could not be converted

java.awt.Font.getAttributes could not be converted

java.awt.Font.getAvailableAttributes could not be converted

java.awt.Font.getBaselineFor could not be converted

java.awt.Font.getFont could not be converted

java.awt.Font.getFont(Map) could not be converted

java.awt.Font.getItalicAngle could not be converted

java.awt.Font.getLineMetrics could not be converted

java.awt.Font.getLineMetrics(CharacterIterator, int, int, FontRenderContext) could not be converted

java.awt.Font.getMaxCharBounds could not be converted

java.awt.Font.getMissingGlyphCode could not be converted

java.awt.Font.getNumGlyphs could not be converted

java.awt.Font.getPSName could not be converted

java.awt.Font.getStringBounds(char[], int, int, FontRenderContext) could not be converted

java.awt.Font.getStringBounds(CharacterIterator, int, int, FontRenderContext) could not be converted

java.awt.Font.getStringBounds(String, FontRenderContext) could not be converted

java.awt.Font.getStringBounds(String, int, int, FontRenderContext) could not be converted

java.awt.Font.getTransform could not be converted

java.awt.Font.HANGING_BASELINE could not be converted

java.awt.Font.hasUniformLineMetrics could not be converted

java.awt.Font.PLAIN could not be converted

java.awt.Font.ROMAN_BASELINE could not be converted

java.awt.Font.TRUETYPE_FONT could not be converted

java.awt.FontMetrics.*Width could not be converted

java.awt.FontMetrics.FontMetrics could not be converted

java.awt.FontMetrics.getLeading could not be converted

java.awt.FontMetrics.getLineMetrics could not be converted

java.awt.FontMetrics.getMaxAdvance could not be converted

java.awt.FontMetrics.getMaxCharBounds could not be converted

java.awt.FontMetrics.getStringBounds(char[], int, int, Graphics) could not be converted

java.awt.FontMetrics.getStringBounds(CharacterIterator, int, int, Graphics) could not be converted

java.awt.FontMetrics.getStringBounds(String, Graphics) could not be converted
java.awt.FontMetrics.getStringBounds(String, int, int, Graphics) could not be converted
java.awt.FontMetrics.getWidths could not be converted
java.awt.FontMetrics.hasUniformLineMetrics could not be converted
java.awt.Frame could not be converted
java.awt.Frame.Frame(GraphicsConfiguration) could not be converted
java.awt.Frame.Frame(String, GraphicsConfiguration) could not be converted
java.awt.Frame.getCursorType could not be converted
java.awt.Frame.getFrames could not be converted
java.awt.Frame.getIconImage could not be converted
java.awt.Frame.setIconImage could not be converted
java.awt.Frame.setState could not be converted
java.awt.GradientPaint.createContext could not be converted
java.awt.GradientPaint.getTransparency could not be converted
java.awt.GradientPaint.GradientPaint could not be converted
java.awt.GradientPaint.isCyclic could not be converted
java.awt.Graphics.copyArea could not be converted
java.awt.Graphics.drawBytes could not be converted
java.awt.Graphics.drawChars could not be converted
java.awt.Graphics.drawRoundRect could not be converted
java.awt.Graphics.drawString could not be converted
java.awt.Graphics.drawString(AttributedCharacterIterator, int, int) could not be converted
java.awt.Graphics.drawString(int, int) could not be converted
java.awt.Graphics.fill3DRect could not be converted
java.awt.Graphics.fillRoundRect could not be converted
java.awt.Graphics.finalize could not be converted
java.awt.Graphics.getClip could not be converted
java.awt.Graphics.setClip could not be converted
java.awt.Graphics.setPaintMode could not be converted
java.awt.Graphics.setXORMode could not be converted
java.awt.Graphics2D.addRenderingHints could not be converted
java.awt.Graphics2D.clip could not be converted
java.awt.Graphics2D.draw could not be converted
java.awt.Graphics2D.drawGlyphVector could not be converted
java.awt.Graphics2D.drawImage could not be converted
java.awt.Graphics2D.drawRenderableImage could not be converted
java.awt.Graphics2D.drawRenderedImage could not be converted
java.awt.Graphics2D.drawString could not be converted
java.awt.Graphics2D.fill could not be converted
java.awt.Graphics2D.fill3DRect could not be converted

java.awt.Graphics2D.getComposite could not be converted

java.awt.Graphics2D.getDeviceConfiguration could not be converted

java.awt.Graphics2D.getPaint could not be converted

java.awt.Graphics2D.getRenderingHint could not be converted

java.awt.Graphics2D.getRenderingHints could not be converted

java.awt.Graphics2D.getStroke could not be converted

java.awt.Graphics2D.Graphics2D could not be converted

java.awt.Graphics2D.hit could not be converted

java.awt.Graphics2D.rotate could not be converted

java.awt.Graphics2D.setComposite could not be converted

java.awt.Graphics2D.setPaint could not be converted

java.awt.Graphics2D.setRenderingHint could not be converted

java.awt.Graphics2D.setRenderingHints could not be converted

java.awt.Graphics2D.setStroke could not be converted

java.awt.Graphics2D.shear could not be converted

java.awt.Graphics2D.transform could not be converted

java.awt.GraphicsConfigTemplate could not be converted

java.awt.GraphicsConfiguration could not be converted

java.awt.GraphicsDevice could not be converted

java.awt.GraphicsEnvironment could not be converted

java.awt.GraphicsEnvironment.getAllFonts could not be converted

java.awt.GraphicsEnvironment.getAvailableFontFamilyNames could not be converted

java.awt.GraphicsEnvironment.getAvailableFontFamilyNames(Locale) could not be converted

java.awt.GraphicsEnvironment.getDefaultScreenDevice could not be converted

java.awt.GraphicsEnvironment.getScreenDevices could not be converted

java.awt.GridBagConstraints could not be converted

java.awt.GridBagLayout could not be converted

java.awt.GridLayout could not be converted

java.awt.GridLayout.addLayoutComponent could not be converted

java.awt.GridLayout.GridLayout could not be converted

java.awt.GridLayout.GridLayout(int, int) could not be converted

java.awt.GridLayout.GridLayout(int, int, int, int) could not be converted

java.awt.GridLayout.layoutContainer could not be converted

java.awt.GridLayout.minimumLayoutSize could not be converted

java.awt.GridLayout.preferredLayoutSize could not be converted

java.awt.GridLayout.removeLayoutComponent could not be converted

java.awt.Image.getProperty could not be converted

java.awt.Image.SCALE_<Algorithm> could not be converted

java.awt.Image.UndefinedProperty could not be converted

java.awt.ItemSelectable could not be converted

java.awt.JobAttributes.DestinationType could not be converted
java.awt.JobAttributes.DialogType could not be converted
java.awt.JobAttributes.getDefaultSelection could not be converted
java.awt.JobAttributes.getDestination could not be converted
java.awt.JobAttributes.getDialog could not be converted
java.awt.JobAttributes.getFileName could not be converted
java.awt.JobAttributes.getMultipleDocumentHandling could not be converted
java.awt.JobAttributes.getPageRanges could not be converted
java.awt.JobAttributes.getSides could not be converted
java.awt.JobAttributes.JobAttributes could not be converted
java.awt.JobAttributes.MultipleDocumentHandlingType could not be converted
java.awt.JobAttributes.setDefaultSelection could not be converted
java.awt.JobAttributes.setDestination could not be converted
java.awt.JobAttributes.setDialog could not be converted
java.awt.JobAttributes.setFileName could not be converted
java.awt.JobAttributes.setMultipleDocumentHandling could not be converted
java.awt.JobAttributes.setMultipleDocumentHandlingToDefault could not be converted
java.awt.JobAttributes.setPageRanges could not be converted
java.awt.JobAttributes.setSides could not be converted
java.awt.JobAttributes.setSidesToDefault could not be converted
java.awt.Label.setAlignment could not be converted
java.awt.LayoutManager could not be converted
java.awt.LayoutManager2 could not be converted
java.awt.List.delItems could not be converted
java.awt.List.getListeners could not be converted
java.awt.MediaTracker could not be converted
java.awt.Menu.getAccessibleContext could not be converted
java.awt.Menu.isTearOff could not be converted
java.awt.MenuBar.deleteShortcut could not be converted
java.awt.MenuBar.getAccessibleContext could not be converted
java.awt.MenuBar.getHelpMenu could not be converted
java.awt.MenuBar.getShortcutMenuItem could not be converted
java.awt.MenuBar.setHelpMenu could not be converted
java.awt.MenuBar.shortcuts could not be converted
java.awt.MenuComponent.dispatchEvent could not be converted
java.awt.MenuComponent.getAccessibleContext could not be converted
java.awt.MenuComponent.getTreeLock could not be converted
java.awt.MenuComponent.MenuComponent could not be converted
java.awt.MenuComponent.postEvent could not be converted
java.awt.MenuComponent.setFont could not be converted

java.awt.MenuContainer.postEvent could not be converted

java.awt.MenuItem.disableEvents could not be converted

java.awt.MenuItem.enableEvents could not be converted

java.awt.MenuItem.getAccessibleContext could not be converted

java.awt.MenuItem.getActionCommand could not be converted

java.awt.MenuItem.getListeners could not be converted

java.awt.MenuItem.setActionCommand could not be converted

java.awt.MenuShortcut.MenuShortcut could not be converted

java.awt.MenuShortcut.usesShiftModifier could not be converted

java.awt.PageAttributes.getOrigin could not be converted

java.awt.PageAttributes.MediaType.<PaperKind> could not be converted

java.awt.PageAttributes.PageAttributes could not be converted

java.awt.PageAttributes.setMedia could not be converted

java.awt.PageAttributes.setMediaToDefault could not be converted

java.awt.PageAttributes.setOrigin could not be converted

java.awt.PageAttributes.setPrinterResolution(int) could not be converted

java.awt.PageAttributes.setPrinterResolution(int[]) could not be converted

java.awt.PageAttributes.setPrintQuality(int) could not be converted

java.awt.PageAttributes.setPrintQuality(PageAttributesPrintQualityType) could not be converted

java.awt.Paint.createContext could not be converted

java.awt.PaintContext could not be converted

java.awt.Panel.Panel could not be converted

java.awt.Point.getX could not be converted

java.awt.Point.getY could not be converted

java.awt.Point.setLocation could not be converted

java.awt.Polygon.addPoint could not be converted

java.awt.Polygon.bounds could not be converted

java.awt.Polygon.contains(double, double) could not be converted

java.awt.Polygon.contains(double, double, double, double) could not be converted

java.awt.Polygon.contains(Rectangle2D) could not be converted

java.awt.Polygon.getPathIterator could not be converted

java.awt.Polygon.intersects(double, double, double, double) could not be converted

java.awt.Polygon.intersects(Rectangle2D) could not be converted

java.awt.Polygon.npoints could not be converted

java.awt.Polygon.Polygon could not be converted

java.awt.PopupMenu.getAccessibleContext could not be converted

java.awt.PopupMenu.show could not be converted

java.awt.PrintGraphics could not be converted

java.awt.PrintJob could not be converted

java.awt.Rectangle.outcode could not be converted

java.awt.Rectangle.setRect could not be converted

java.awt.RenderingHints.<PropertySetting> could not be converted

java.awt.RenderingHints.add could not be converted

java.awt.RenderingHints.Key could not be converted

java.awt.RenderingHints.putAll could not be converted

java.awt.RenderingHints.RenderingHints could not be converted

java.awt.Robot could not be converted

java.awt.Scrollbar could not be converted

java.awt.Scrollbar.addAdjustmentListener could not be converted

java.awt.Scrollbar.getListeners could not be converted

java.awt.Scrollbar.getVisible could not be converted

java.awt.Scrollbar.getVisibleAmount could not be converted

java.awt.Scrollbar.paramString could not be converted

java.awt.Scrollbar.processAdjustmentEvent could not be converted

java.awt.Scrollbar.Scrollbar could not be converted

java.awt.Scrollbar.Scrollbar could not be converted

java.awt.Scrollbar.setOrientation could not be converted

java.awt.Scrollbar.setVisibleAmount could not be converted

java.awt.ScrollPane.get*Adjustable could not be converted

java.awt.ScrollPane.printComponents could not be converted

java.awt.ScrollPane.ScrollPane could not be converted

java.awt.ScrollPane.setLayout could not be converted

java.awt.Shape could not be converted

java.awt.Shape.contains(double,double) could not be converted

java.awt.Shape.contains(double, double, double, double) could not be converted

java.awt.Shape.contains(Rectangle2D) could not be converted

java.awt.Shape.getPathIterator could not be converted

java.awt.Shape.intersects(double,double,double,double) could not be converted

java.awt.Shape.intersects(Rectangle2D) could not be converted

java.awt.Stroke could not be converted

java.awt.Stroke.createStrokedShape could not be converted

java.awt.SystemColor.createContext could not be converted

java.awt.SystemColor.NUM_COLORS could not be converted

java.awt.SystemColor.TEXT could not be converted

java.awt.SystemColor.TEXT_HIGHLIGHT could not be converted

java.awt.SystemColor.TEXT_HIGHLIGHT_TEXT could not be converted

java.awt.SystemColor.TEXT_INACTIVE_TEXT could not be converted

java.awt.SystemColor.TEXT_TEXT could not be converted

java.awt.SystemColor.textHighlight could not be converted

java.awt.SystemColor.textHighlightText could not be converted

java.awt.SystemColor.textInactiveText could not be converted
java.awt.SystemColor.textText could not be converted
java.awt.TextArea.*Columns could not be converted
java.awt.TextArea.*Rows could not be converted
java.awt.TextComponent.enableInputMethods could not be converted
java.awt.TextComponent.getBackground could not be converted
java.awt.TextComponent.getListeners could not be converted
java.awt.TextComponent.processTextEvent could not be converted
java.awt.TextComponent.setText could not be converted
java.awt.TextComponent.textListener could not be converted
java.awt.TextField.*Columns could not be converted
java.awt.TextField.setColumns could not be converted
java.awt.TextField.getColumns could not be converted
java.awt.TextField.getListeners could not be converted
java.awt.TextField.getPreferredSize could not be converted
java.awt.TextField.preferredSize could not be converted
java.awt.TextField.setColumns could not be converted
java.awt.TextField.TextField could not be converted
java.awt.TexturePaint.createContext could not be converted
java.awt.TexturePaint.getAnchorRect could not be converted
java.awt.TexturePaint.getTransparency could not be converted
java.awt.Toolkit could not be converted
java.awt.Transparency could not be converted
java.awt.Window.applyResourceBundle could not be converted
java.awt.Window.getGraphicsConfiguration could not be converted
java.awt.Window.getInputContext could not be converted
java.awt.Window.getListeners could not be converted
java.awt.Window.getToolkit could not be converted
java.awt.Window.getWarningString could not be converted
java.awt.Window.pack could not be converted
java.awt.Window.postEvent could not be converted
java.awt.Window.processWindowEvent could not be converted
java.awt.Window.Window could not be converted

Java.awt.color Error Messages

[java.awt.color.ColorSpace could not be converted](#)

[java.awt.color.ICC_ColorSpace could not be converted](#)

[java.awt.color.ICC_Profile could not be converted](#)

[java.awt.color.ICC_ProfileGray could not be converted](#)

[java.awt.color.ICC_ProfileRGB could not be converted](#)

Java.awt.datatransfer Error Messages

java.awt.datatransfer.Clipboard.Clipboard could not be converted

java.awt.datatransfer.Clipboard.contents could not be converted

java.awt.datatransfer.Clipboard.getName could not be converted

java.awt.datatransfer.Clipboard.owner could not be converted

java.awt.datatransfer.ClipboardOwner could not be converted

java.awt.datatransfer.DataFlavor.clone could not be converted

java.awt.datatransfer.DataFlavor.DataFlavor could not be converted

java.awt.datatransfer.DataFlavor.DataFlavor(Class, String) could not be converted

java.awt.datatransfer.DataFlavor.DataFlavor(DataFlavor) could not be converted

java.awt.datatransfer.DataFlavor.DataFlavor(String) could not be converted

java.awt.datatransfer.DataFlavor.DataFlavor(String, String) could not be converted

java.awt.datatransfer.DataFlavor.DataFlavor(String, String, ClassLoader) could not be converted

java.awt.datatransfer.DataFlavor.DataFlavor(String, String, MimeTypeParameterList, class, j) could not be converted

java.awt.datatransfer.DataFlavor.equals could not be converted

java.awt.datatransfer.DataFlavor.getDefaultRepresentationClass could not be converted

java.awt.datatransfer.DataFlavor.getDefaultRepresentationClassAsString could not be converted

java.awt.datatransfer.DataFlavor.getHumanPresentableName could not be converted

java.awt.datatransfer.DataFlavor.getMimeType could not be converted

java.awt.datatransfer.DataFlavor.getParameter could not be converted

java.awt.datatransfer.DataFlavor.getPrimaryType could not be converted

java.awt.datatransfer.DataFlavor.getReaderForText could not be converted

java.awt.datatransfer.DataFlavor.getRepresentationClass could not be converted

java.awt.datatransfer.DataFlavor.getSubType could not be converted

java.awt.datatransfer.DataFlavor.isFlavorJavaFileListType could not be converted

java.awt.datatransfer.DataFlavor.isFlavorRemoteObjectType could not be converted

java.awt.datatransfer.DataFlavor.isMimeTypeEqual could not be converted

java.awt.datatransfer.DataFlavor.isMimeTypeSerializedObject could not be converted

java.awt.datatransfer.DataFlavor.isRepresentationClassInputStream could not be converted

java.awt.datatransfer.DataFlavor.isRepresentationClassRemote could not be converted

java.awt.datatransfer.DataFlavor.isRepresentationClassSerializable could not be converted

java.awt.datatransfer.DataFlavor.javaFileListFlavor could not be converted

java.awt.datatransfer.DataFlavor.javaJVMLocalObjectMimeType could not be converted

java.awt.datatransfer.DataFlavor.javaRemoteObjectMimeType could not be converted

java.awt.datatransfer.DataFlavor.normalizeMimeType could not be converted

java.awt.datatransfer.DataFlavor.normalizeMimeTypeParameter could not be converted

java.awt.datatransfer.DataFlavor.readExternal could not be converted

java.awt.datatransfer.DataFlavor.selectBestTextFlavor could not be converted

java.awt.datatransfer.DataFlavor.setHumanPresentableName could not be converted
java.awt.datatransfer.DataFlavor.tryToLoadClass could not be converted
java.awt.datatransfer.DataFlavor.writeExternal could not be converted
java.awt.datatransfer.FlavorMap could not be converted
java.awt.datatransfer.StringSelection.getTransferDataFlavors could not be converted
java.awt.datatransfer.StringSelection.isDataFlavorSupported could not be converted
java.awt.datatransfer.StringSelection.lostOwnership could not be converted
java.awt.datatransfer.SystemFlavorMap could not be converted
java.awt.datatransfer.Transferable.getTransferDataFlavors could not be converted
java.awt.datatransfer.Transferable.isDataFlavorSupported could not be converted

Java.awt.dnd Error Messages

java.awt.dnd.Autoscroll could not be converted

java.awt.dnd.DragGestureEvent.DragGestureEvent could not be converted

java.awt.dnd.DragGestureEvent.getDragAction could not be converted

java.awt.dnd.DragGestureEvent.getDragSource could not be converted

java.awt.dnd.DragGestureEvent.getSourceAsDragGestureRecognizer could not be converted

java.awt.dnd.DragGestureEvent.getTriggerEvent could not be converted

java.awt.dnd.DragGestureEvent.iterator could not be converted

java.awt.dnd.DragGestureEvent.toArray could not be converted

java.awt.dnd.DragGestureRecognizer could not be converted

java.awt.dnd.DragSource.createDragSourceContext could not be converted

java.awt.dnd.DragSource.DefaultCopyDrop could not be converted

java.awt.dnd.DragSource.DefaultCopyNoDrop could not be converted

java.awt.dnd.DragSource.DefaultLinkDrop could not be converted

java.awt.dnd.DragSource.DefaultLinkNoDrop could not be converted

java.awt.dnd.DragSource.DefaultMoveDrop could not be converted

java.awt.dnd.DragSource.DefaultMoveNoDrop could not be converted

java.awt.dnd.DragSource.DragSource could not be converted

java.awt.dnd.DragSource.getDefaultDragSource could not be converted

java.awt.dnd.DragSource.getFlavorMap could not be converted

java.awt.dnd.DragSource.isDragImageSupported could not be converted

java.awt.dnd.DragSourceContext could not be converted

java.awt.dnd.DragSourceDragEvent.DragSourceDragEvent could not be converted

java.awt.dnd.DragSourceDropEvent.DragSourceDropEvent could not be converted

java.awt.dnd.DragSourceDropEvent.getDropSuccess could not be converted

java.awt.dnd.DragSourceEvent.DragSourceEvent could not be converted

java.awt.dnd.DragSourceEvent.getDragSourceContext could not be converted

java.awt.dnd.DragSourceListener.dropActionChanged could not be converted

java.awt.dnd.DropTarget could not be converted

java.awt.dnd.DropTarget.setComponent could not be converted

java.awt.dnd.DropTargetContext could not be converted

java.awt.dnd.DropTargetDragEvent.DropTargetDragEvent could not be converted

java.awt.dnd.DropTargetDragEvent.getCurrentDataFlavors could not be converted

java.awt.dnd.DropTargetDragEvent.getCurrentDataFlavorsAsList could not be converted

java.awt.dnd.DropTargetDragEvent.isDataFlavorSupported could not be converted

java.awt.dnd.DropTargetDragEvent.rejectDrag could not be converted

java.awt.dnd.DropTargetDropEvent.dropComplete could not be converted

java.awt.dnd.DropTargetDropEvent.DropTargetDropEvent(DropTargetContext, Point, int, int) could not be converted

java.awt.dnd.DropTargetDropEvent.DropTargetDropEvent(DropTargetContext, Point, int, int, boolean) could not be converted
java.awt.dnd.DropTargetDropEvent.getCurrentDataFlavors could not be converted
java.awt.dnd.DropTargetDropEvent.getCurrentDataFlavorsAsList could not be converted
java.awt.dnd.DropTargetDropEvent.isDataFlavorSupported could not be converted
java.awt.dnd.DropTargetDropEvent.isLocalTransfer could not be converted
java.awt.dnd.DropTargetDropEvent.rejectDrop could not be converted
java.awt.dnd.DropTargetEvent could not be converted
java.awt.dnd.InvalidDnDOperationException could not be converted
java.awt.dnd.MouseDragGestureRecognizer could not be converted
java.awt.dnd.peer.DragSourceContextPeer could not be converted
java.awt.dnd.peer.DropTargetContextPeer could not be converted
java.awt.dnd.peer.DropTargetPeer could not be converted

Java.awt.event Error Messages

java.awt.event.<ClassName>.<EventType>_FIRST could not be converted

java.awt.event.<ClassName>.<EventType>_LAST could not be converted

java.awt.event.ActionEvent.<KeyName>_MASK could not be converted

java.awt.event.ActionEvent.ACTION_PERFORMED could not be converted

java.awt.event.ActionEvent.getActionCommand could not be converted

java.awt.event.AWTEventListener could not be converted

java.awt.event.ComponentEvent.COMPONENT_<EventType> could not be converted

java.awt.event.FocusEvent.FOCUS_<EventType> could not be converted

java.awt.event.FocusEvent.isTemporary could not be converted

java.awt.event.HierarchyBoundsAdapter could not be converted

java.awt.event.HierarchyBoundsListener could not be converted

java.awt.event.HierarchyEvent could not be converted

java.awt.event.HierarchyListener could not be converted

java.awt.event.InputEvent could not be converted

java.awt.event.InputMethodEvent could not be converted

java.awt.event.InputMethodListener could not be converted

java.awt.event.InvocationEvent could not be converted

java.awt.event.ItemEvent.<EventType> could not be converted

java.awt.event.ItemEvent.getStateChange could not be converted

java.awt.event.KeyEvent.<VirtualKey> could not be converted

java.awt.event.KeyEvent.CHAR_UNDEFINED could not be converted

java.awt.event.KeyEvent.getKeyChar could not be converted

java.awt.event.KeyEvent.getKeyCode could not be converted

java.awt.event.KeyEvent.getKeyModifiersText could not be converted

java.awt.event.KeyEvent.getKeyText could not be converted

java.awt.event.KeyEvent.isActionKey could not be converted

java.awt.event.KeyEvent.KEY_<EventType> could not be converted

java.awt.event.KeyEvent.setKeyChar could not be converted

java.awt.event.KeyEvent.setKeyCode could not be converted

java.awt.event.KeyEvent.setModifiers could not be converted

java.awt.event.KeyEvent.setSource could not be converted

java.awt.event.KeyEvent.VK.<VirtualKey> could not be converted

java.awt.event.KeyEvent.VK_<CharacterName> could not be converted

java.awt.event.MouseEvent.getClickCount could not be converted

java.awt.event.MouseEvent.getPoint could not be converted

java.awt.event.MouseEvent.getX could not be converted

java.awt.event.MouseEvent.getY could not be converted

java.awt.event.MouseEvent.isPopupTrigger could not be converted

java.awt.event.MouseEvent.MOUSE_<EventType> could not be converted

java.awt.event.MouseEvent.translatePoint could not be converted

java.awt.event.PaintEvent.<EventType> could not be converted

java.awt.event.PaintEvent.setUpdateRect could not be converted

java.awt.event.TextEvent.TEXT_VALUE_CHANGED could not be converted

java.awt.event.WindowEvent.WINDOW_<EventType> could not be converted

Java.awt.font Error Messages

java.awt.font.FontRenderContext could not be converted

java.awt.font.FontRenderContext.FontRenderContext could not be converted

java.awt.font.FontRenderContext.getTransform could not be converted

java.awt.font.FontRenderContext.isAntiAliased could not be converted

java.awt.font.FontRenderContext.usesFractionalMetrics could not be converted

java.awt.font.GlyphJustificationInfo could not be converted

java.awt.font.GlyphMetrics could not be converted

java.awt.font.GlyphVector could not be converted

java.awt.font.GraphicAttribute could not be converted

java.awt.font.GraphicAttribute.<AttributeName> could not be converted

java.awt.font.GraphicAttribute.draw could not be converted

java.awt.font.GraphicAttribute.getAdvance could not be converted

java.awt.font.GraphicAttribute.getAlignment could not be converted

java.awt.font.GraphicAttribute.getAscent could not be converted

java.awt.font.GraphicAttribute.getBounds could not be converted

java.awt.font.GraphicAttribute.getDescent could not be converted

java.awt.font.GraphicAttribute.getJustificationInfo could not be converted

java.awt.font.GraphicAttribute.GraphicAttribute could not be converted

java.awt.font.ImageGraphicAttribute could not be converted

java.awt.font.ImageGraphicAttribute.draw could not be converted

java.awt.font.ImageGraphicAttribute.equals could not be converted

java.awt.font.ImageGraphicAttribute.getAdvance could not be converted

java.awt.font.ImageGraphicAttribute.getAscent could not be converted

java.awt.font.ImageGraphicAttribute.getBounds could not be converted

java.awt.font.ImageGraphicAttribute.getDescent could not be converted

java.awt.font.ImageGraphicAttribute.hashCode could not be converted

java.awt.font.ImageGraphicAttribute.ImageGraphicAttribute could not be converted

java.awt.font.LineBreakMeasurer could not be converted

java.awt.font.LineBreakMeasurer.deleteChar could not be converted

java.awt.font.LineBreakMeasurer.getPosition could not be converted

java.awt.font.LineBreakMeasurer.insertChar could not be converted

java.awt.font.LineBreakMeasurer.LineBreakMeasurer could not be converted

java.awt.font.LineBreakMeasurer.nextLayout could not be converted

java.awt.font.LineBreakMeasurer.nextOffset could not be converted

java.awt.font.LineBreakMeasurer.setPosition could not be converted

java.awt.font.LineMetrics could not be converted

java.awt.font.LineMetrics.getAscent could not be converted

java.awt.font.LineMetrics.getBaselineIndex could not be converted

java.awt.font.LineMetrics.getBaselineOffsets could not be converted

java.awt.font.LineMetrics.getDescent could not be converted

java.awt.font.LineMetrics.getHeight could not be converted

java.awt.font.LineMetrics.getLeading could not be converted

java.awt.font.LineMetrics.getNumChars could not be converted

java.awt.font.LineMetrics.getStrikethroughOffset could not be converted

java.awt.font.LineMetrics.getStrikethroughThickness could not be converted

java.awt.font.LineMetrics.getUnderlineOffset could not be converted

java.awt.font.LineMetrics.getUnderlineThickness could not be converted

java.awt.font.LineMetrics.LineMetrics could not be converted

java.awt.font.MultipleMaster could not be converted

java.awt.font.OpenType could not be converted

java.awt.font.OpenType.<TagType> could not be converted

java.awt.font.OpenType.getFontTable(int) could not be converted

java.awt.font.OpenType.getFontTable(int, int, int) could not be converted

java.awt.font.OpenType.getFontTable(String) could not be converted

java.awt.font.OpenType.getFontTable(String, int, int) could not be converted

java.awt.font.OpenType.getFontTableSize could not be converted

java.awt.font.OpenType.getVersion could not be converted

java.awt.font.ShapeGraphicAttribute could not be converted

java.awt.font.ShapeGraphicAttribute.draw could not be converted

java.awt.font.ShapeGraphicAttribute.equals could not be converted

java.awt.font.ShapeGraphicAttribute.FILL could not be converted

java.awt.font.ShapeGraphicAttribute.getAdvance could not be converted

java.awt.font.ShapeGraphicAttribute.getAscent could not be converted

java.awt.font.ShapeGraphicAttribute.getBounds could not be converted

java.awt.font.ShapeGraphicAttribute.getDescent could not be converted

java.awt.font.ShapeGraphicAttribute.hashCode could not be converted

java.awt.font.ShapeGraphicAttribute.ShapeGraphicAttribute could not be converted

java.awt.font.ShapeGraphicAttribute.STROKE could not be converted

java.awt.font.TextAttribute could not be converted

java.awt.font.TextHitInfo could not be converted

java.awt.font.TextHitInfo.afterOffset could not be converted

java.awt.font.TextHitInfo.beforeOffset could not be converted

java.awt.font.TextHitInfo.equals could not be converted

java.awt.font.TextHitInfo.getCharIndex could not be converted

java.awt.font.TextHitInfo.getInsertionIndex could not be converted

java.awt.font.TextHitInfo.getOffsetHit could not be converted

java.awt.font.TextHitInfo.getOtherHit could not be converted

java.awt.font.TextHitInfo.hashCode could not be converted

java.awt.font.TextHitInfo.isLeadingEdge could not be converted

java.awt.font.TextHitInfo.leading could not be converted

java.awt.font.TextHitInfo.toString could not be converted

java.awt.font.TextHitInfo.trailing could not be converted

java.awt.font.TextLayout could not be converted

java.awt.font.TextLayout.CaretPolicy could not be converted

java.awt.font.TextLayout.CaretPolicy.CaretPolicy could not be converted

java.awt.font.TextLayout.CaretPolicy.getStrongCaret could not be converted

java.awt.font.TextLayout.clone could not be converted

java.awt.font.TextLayout.DEFAULT_CARET_POLICY could not be converted

java.awt.font.TextLayout.draw could not be converted

java.awt.font.TextLayout.equals could not be converted

java.awt.font.TextLayout.getAdvance could not be converted

java.awt.font.TextLayout.getAscent could not be converted

java.awt.font.TextLayout.getBaseline could not be converted

java.awt.font.TextLayout.getBaselineOffsets could not be converted

java.awt.font.TextLayout.getBlackBoxBounds could not be converted

java.awt.font.TextLayout.getBounds could not be converted

java.awt.font.TextLayout.getCaretInfo could not be converted

java.awt.font.TextLayout.getCaretShape could not be converted

java.awt.font.TextLayout.getCaretShapes could not be converted

java.awt.font.TextLayout.getCharacterCount could not be converted

java.awt.font.TextLayout.getCharacterLevel could not be converted

java.awt.font.TextLayout.getDescent could not be converted

java.awt.font.TextLayout.getJustifiedLayout could not be converted

java.awt.font.TextLayout.getLeading could not be converted

java.awt.font.TextLayout.getLogicalHighlightShape could not be converted

java.awt.font.TextLayout.getLogicalRangesForVisualSelection could not be converted

java.awt.font.TextLayout.getNextLeftHit could not be converted

java.awt.font.TextLayout.getNextRightHit could not be converted

java.awt.font.TextLayout.getOutline could not be converted

java.awt.font.TextLayout.getVisibleAdvance could not be converted

java.awt.font.TextLayout.getVisualHighlightShape could not be converted

java.awt.font.TextLayout.getVisualOtherHit could not be converted

java.awt.font.TextLayout.handleJustify could not be converted

java.awt.font.TextLayout.hashCode could not be converted

java.awt.font.TextLayout.hitTestChar could not be converted

java.awt.font.TextLayout.isLeftToRight could not be converted

java.awt.font.TextLayout.isVertical could not be converted

java.awt.font.TextLayout.TextLayout could not be converted

java.awt.font.TextLayout.toString could not be converted

java.awt.font.TextMeasurer could not be converted

java.awt.font.TransformAttribute could not be converted

Java.awt.geom Error Messages

java.awt.geom.AffineTransform.<Type> could not be converted

java.awt.geom.AffineTransform.clone could not be converted

java.awt.geom.AffineTransform.concatenate could not be converted

java.awt.geom.AffineTransform.createTransformedShape could not be converted

java.awt.geom.AffineTransform.deltaTransform(double[], int, double[], int, int) could not be converted

java.awt.geom.AffineTransform.deltaTransform(Point2D, Point2D) could not be converted

java.awt.geom.AffineTransform.getType could not be converted

java.awt.geom.AffineTransform.inverseTransform(double[], int, double[], int, int) could not be converted

java.awt.geom.AffineTransform.inverseTransform(Point2D, Point2D) could not be converted

java.awt.geom.AffineTransform.preConcatenate could not be converted

java.awt.geom.AffineTransform.setTransform(AffineTransform) could not be converted

java.awt.geom.AffineTransform.setTransform(double, double, double, double, double, double) could not be converted

java.awt.geom.AffineTransform.transform could not be converted

java.awt.geom.Arc2D.Arc2D could not be converted

java.awt.geom.Arc2D.containsAngle could not be converted

java.awt.geom.Arc2D.Double.Double could not be converted

java.awt.geom.Arc2D.Double.extent could not be converted

java.awt.geom.Arc2D.Double.getAngleExtent could not be converted

java.awt.geom.Arc2D.Double.getAngleStart could not be converted

java.awt.geom.Arc2D.Double.getHeight could not be converted

java.awt.geom.Arc2D.Double.getWidth could not be converted

java.awt.geom.Arc2D.Double.getX could not be converted

java.awt.geom.Arc2D.Double.getY could not be converted

java.awt.geom.Arc2D.Double.height could not be converted

java.awt.geom.Arc2D.Double.makeBounds could not be converted

java.awt.geom.Arc2D.Double.setAngleExtent could not be converted

java.awt.geom.Arc2D.Double.setAngleStart could not be converted

java.awt.geom.Arc2D.Double.start could not be converted

java.awt.geom.Arc2D.Double.width could not be converted

java.awt.geom.Arc2D.Double.x could not be converted

java.awt.geom.Arc2D.Double.y could not be converted

java.awt.geom.Arc2D.Float.extent could not be converted

java.awt.geom.Arc2D.Float.Float could not be converted

java.awt.geom.Arc2D.Float.getAngleExtent could not be converted

java.awt.geom.Arc2D.Float.getAngleStart could not be converted

java.awt.geom.Arc2D.Float.getHeight could not be converted

java.awt.geom.Arc2D.Float.getWidth could not be converted

java.awt.geom.Arc2D.Float.getX could not be converted
java.awt.geom.Arc2D.Float.getY could not be converted
java.awt.geom.Arc2D.Float.height could not be converted
java.awt.geom.Arc2D.Float.makeBounds could not be converted
java.awt.geom.Arc2D.Float.setAngleExtent could not be converted
java.awt.geom.Arc2D.Float.setAngleStart could not be converted
java.awt.geom.Arc2D.Float.start could not be converted
java.awt.geom.Arc2D.Float.width could not be converted
java.awt.geom.Arc2D.Float.x could not be converted
java.awt.geom.Arc2D.Float.y could not be converted
java.awt.geom.Arc2D.getAngleExtent could not be converted
java.awt.geom.Arc2D.getAngleStart could not be converted
java.awt.geom.Arc2D.getArcType could not be converted
java.awt.geom.Arc2D.makeBounds could not be converted
java.awt.geom.Arc2D.setAngleExtent could not be converted
java.awt.geom.Arc2D.setAngles(double, double, double, double) could not be converted
java.awt.geom.Arc2D.setAngles(Point2D) could not be converted
java.awt.geom.Arc2D.setAngleStart(double) could not be converted
java.awt.geom.Arc2D.setAngleStart(Point2D) could not be converted
java.awt.geom.Arc2D.setArcByCenter could not be converted
java.awt.geom.Arc2D.setArcByTangent could not be converted
java.awt.geom.Arc2D.setArcType could not be converted
java.awt.geom.Arc2D.setFrame could not be converted
java.awt.geom.Area.add could not be converted
java.awt.geom.Area.clone could not be converted
java.awt.geom.Area.contains could not be converted
java.awt.geom.Area.equals could not be converted
java.awt.geom.Area.exclusiveOr could not be converted
java.awt.geom.Area.getBounds could not be converted
java.awt.geom.Area.getBounds2D could not be converted
java.awt.geom.Area.getPathIterator could not be converted
java.awt.geom.Area.intersects could not be converted
java.awt.geom.Area.isEmpty could not be converted
java.awt.geom.Area.isPolygonal could not be converted
java.awt.geom.Area.isRectangular could not be converted
java.awt.geom.Area.isSingular could not be converted
java.awt.geom.Area.subtract could not be converted
java.awt.geom.CubicCurve2D.contains could not be converted
java.awt.geom.CubicCurve2D.CubicCurve2D could not be converted
java.awt.geom.CubicCurve2D.Double.Double could not be converted

java.awt.geom.CubicCurve2D.Float.Float could not be converted
java.awt.geom.CubicCurve2D.getBounds could not be converted
java.awt.geom.CubicCurve2D.getFlatness could not be converted
java.awt.geom.CubicCurve2D.getFlatnessSq could not be converted
java.awt.geom.CubicCurve2D.getPathIterator could not be converted
java.awt.geom.CubicCurve2D.solveCubic(double[]) could not be converted
java.awt.geom.CubicCurve2D.solveCubic(double[], double[]) could not be converted
java.awt.geom.CubicCurve2D.subdivide could not be converted
java.awt.geom.Dimension2D.clone could not be converted
java.awt.geom.Dimension2D.Dimension2D could not be converted
java.awt.geom.Ellipse2D.Double.Double could not be converted
java.awt.geom.Ellipse2D.Ellipse2D could not be converted
java.awt.geom.Ellipse2D.Float.Float could not be converted
java.awt.geom.FlatteningPathIterator could not be converted
java.awt.geom.GeneralPath.append could not be converted
java.awt.geom.GeneralPath.GeneralPath could not be converted
java.awt.geom.GeneralPath.GeneralPath(int) could not be converted
java.awt.geom.GeneralPath.GeneralPath(int, int) could not be converted
java.awt.geom.GeneralPath.GeneralPath(Shape) could not be converted
java.awt.geom.GeneralPath.getBounds could not be converted
java.awt.geom.GeneralPath.getBounds2D could not be converted
java.awt.geom.GeneralPath.getCurrentPoint could not be converted
java.awt.geom.GeneralPath.getPathIterator could not be converted
java.awt.geom.GeneralPath.moveTo could not be converted
java.awt.geom.GeneralPath.quadTo could not be converted
java.awt.geom.Line2D.Double.Double(double, double, double, double) could not be converted
java.awt.geom.Line2D.Double.Double(Point2D, Point2D) could not be converted
java.awt.geom.Line2D.Float.Float(float, float, float, float) could not be converted
java.awt.geom.Line2D.Float.Float(Point2D, Point2D) could not be converted
java.awt.geom.Line2D.getBounds could not be converted
java.awt.geom.Line2D.getPathIterator could not be converted
java.awt.geom.Line2D.intersectsLine could not be converted
java.awt.geom.Line2D.Line2D could not be converted
java.awt.geom.Line2D.linesIntersect could not be converted
java.awt.geom.Line2D.ptLineDist could not be converted
java.awt.geom.Line2D.ptLineDistSq could not be converted
java.awt.geom.Line2D.ptSegDist could not be converted
java.awt.geom.Line2D.ptSegDistSq could not be converted
java.awt.geom.Line2D.relativeCCW could not be converted
java.awt.geom.PathIterator could not be converted

java.awt.geom.PathIterator.<SegmentType> could not be converted
java.awt.geom.PathIterator.currentSegment could not be converted
java.awt.geom.PathIterator.getWindingRule could not be converted
java.awt.geom.PathIterator.isDone could not be converted
java.awt.geom.PathIterator.next could not be converted
java.awt.geom.Point2D.Point2D could not be converted
java.awt.geom.QuadCurve2D could not be converted
java.awt.geom.QuadCurve2D.Double could not be converted
java.awt.geom.QuadCurve2D.Float could not be converted
java.awt.geom.Rectangle2D.<Position> could not be converted
java.awt.geom.Rectangle2D.add could not be converted
java.awt.geom.Rectangle2D.createIntersection could not be converted
java.awt.geom.Rectangle2D.Double.createIntersection could not be converted
java.awt.geom.Rectangle2D.Double.outcode could not be converted
java.awt.geom.Rectangle2D.Float.createIntersection could not be converted
java.awt.geom.Rectangle2D.Float.outcode could not be converted
java.awt.geom.Rectangle2D.getPathIterator could not be converted
java.awt.geom.Rectangle2D.intersect could not be converted
java.awt.geom.Rectangle2D.intersectsLine could not be converted
java.awt.geom.Rectangle2D.outcode could not be converted
java.awt.geom.Rectangle2D.Rectangle2D could not be converted
java.awt.geom.RectangularShape.clone could not be converted
java.awt.geom.RectangularShape.getBounds could not be converted
java.awt.geom.RectangularShape.getPathIterator could not be converted
java.awt.geom.RectangularShape setFrame(double, double, double, double) could not be converted
java.awt.geom.RectangularShape setFrame(Point2D, Dimension2D) could not be converted
java.awt.geom.RectangularShape setFrame(Rectangle2D) could not be converted
java.awt.geom.RectangularShape setFrameFromCenter could not be converted
java.awt.geom.RectangularShape setFrameFromDiagonal(double, double, double, double) could not be converted
java.awt.geom.RectangularShape setFrameFromDiagonal(Point2D, Point2D) could not be converted
java.awt.geom.RoundRectangle2D.Double could not be converted
java.awt.geom.RoundRectangle2D.Double.archeight could not be converted
java.awt.geom.RoundRectangle2D.Double.arcwidth could not be converted
java.awt.geom.RoundRectangle2D.Double.getArcHeight could not be converted
java.awt.geom.RoundRectangle2D.Double.getArcWidth could not be converted
java.awt.geom.RoundRectangle2D.Double.height could not be converted
java.awt.geom.RoundRectangle2D.Double.setRoundRect could not be converted
java.awt.geom.RoundRectangle2D.Double.width could not be converted
java.awt.geom.RoundRectangle2D.Double.x could not be converted
java.awt.geom.RoundRectangle2D.Double.y could not be converted

java.awt.geom.RoundRectangle2D.Float could not be converted
java.awt.geom.RoundRectangle2D.Float.archeight could not be converted
java.awt.geom.RoundRectangle2D.Float.arcwidth could not be converted
java.awt.geom.RoundRectangle2D.Float.getArcHeight could not be converted
java.awt.geom.RoundRectangle2D.Float.getArcWidth could not be converted
java.awt.geom.RoundRectangle2D.Float.height could not be converted
java.awt.geom.RoundRectangle2D.Float.setRoundRect could not be converted
java.awt.geom.RoundRectangle2D.Float.width could not be converted
java.awt.geom.RoundRectangle2D.Float.x could not be converted
java.awt.geom.RoundRectangle2D.Float.y could not be converted
java.awt.geom.RoundRectangle2D.getArcHeight could not be converted
java.awt.geom.RoundRectangle2D.getArcWidth could not be converted
java.awt.geom.RoundRectangle2D.RoundRectangle2D could not be converted
java.awt.geom.RoundRectangle2D.setFrame could not be converted
java.awt.geom.RoundRectangle2D.setRoundRect could not be converted

Java.awt.im Error Messages

[java.awt.im.InputContext could not be converted](#)

[java.awt.im.InputMethodHighlight could not be converted](#)

[java.awt.im.InputMethodRequests could not be converted](#)

[java.awt.im.InputSubset could not be converted](#)

[java.awt.im.spi.InputMethod could not be converted](#)

[java.awt.im.spi.InputMethodContext could not be converted](#)

[java.awt.im.spi.InputMethodDescriptor could not be converted](#)

Java.awt.image Error Messages

java.awt.image.<ClassName>.*Consumer could not be converted

java.awt.image.<ClassName>.*TopDownLeftRight* could not be converted

java.awt.image.<ClassName>.imageComplete could not be converted

java.awt.image.<ClassName>.setHints could not be converted

java.awt.image.<ClassName>.setPixels could not be converted

java.awt.image.<ClassName>.setProperties could not be converted

java.awt.image.AffineTransformOp could not be converted

java.awt.image.BandCombineOp could not be converted

java.awt.image.BandedSampleModel could not be converted

java.awt.image.BandedSampleModel.BandedSampleModel could not be converted

java.awt.image.BandedSampleModel.createCompatibleSampleModel could not be converted

java.awt.image.BandedSampleModel.createSubsetSampleModel could not be converted

java.awt.image.BandedSampleModel.getDataElements could not be converted

java.awt.image.BandedSampleModel.getPixel could not be converted

java.awt.image.BandedSampleModel.getPixels could not be converted

java.awt.image.BandedSampleModel.getSample could not be converted

java.awt.image.BandedSampleModel.getSampleDouble could not be converted

java.awt.image.BandedSampleModel.getSampleFloat could not be converted

java.awt.image.BandedSampleModel.getSamples could not be converted

java.awt.image.BandedSampleModel.setDataElements could not be converted

java.awt.image.BandedSampleModel.setSample could not be converted

java.awt.image.BandedSampleModel.setSamples could not be converted

java.awt.image.BufferedImage.addTileObserver could not be converted

java.awt.image.BufferedImage.BufferedImage(ColorModel, Raster, Hashtable) could not be converted

java.awt.image.BufferedImage.BufferedImage(int, int, int) could not be converted

java.awt.image.BufferedImage.BufferedImage(int, int, int, ColorModel) could not be converted

java.awt.image.BufferedImage.coerceData could not be converted

java.awt.image.BufferedImage.copyData could not be converted

java.awt.image.BufferedImage.getAlphaRaster could not be converted

java.awt.image.BufferedImage.getColorModel could not be converted

java.awt.image.BufferedImage.getHeight could not be converted

java.awt.image.BufferedImage.getMinTileX could not be converted

java.awt.image.BufferedImage.getMinTileY could not be converted

java.awt.image.BufferedImage.getNumXTiles could not be converted

java.awt.image.BufferedImage.getNumYTiles could not be converted

java.awt.image.BufferedImage.getProperty could not be converted

java.awt.image.BufferedImage.getPropertyNames could not be converted

java.awt.image.BufferedImage.getRGB could not be converted

java.awt.image.BufferedImage.getSources could not be converted

java.awt.image.BufferedImage.getTile could not be converted

java.awt.image.BufferedImage.getTileGridXOffset could not be converted

java.awt.image.BufferedImage.getTileGridYOffset could not be converted

java.awt.image.BufferedImage.getTileHeight could not be converted

java.awt.image.BufferedImage.getTileWidth could not be converted

java.awt.image.BufferedImage.getWidth could not be converted

java.awt.image.BufferedImage.getWritableTile could not be converted

java.awt.image.BufferedImage.getWritableTileIndices could not be converted

java.awt.image.BufferedImage.hasTileWriters could not be converted

java.awt.image.BufferedImage.isAlphaPremultiplied could not be converted

java.awt.image.BufferedImage.isTileWritable could not be converted

java.awt.image.BufferedImage.releaseWritableTile could not be converted

java.awt.image.BufferedImage.removeTileObserver could not be converted

java.awt.image.BufferedImage.setRGB could not be converted

java.awt.image.BufferedImage.TYPE_BYTE_BINARY could not be converted

java.awt.image.BufferedImage.TYPE_BYTE_GRAY could not be converted

java.awt.image.BufferedImageFilter could not be converted

java.awt.image.BufferedImageOp could not be converted

java.awt.image.ByteLookupTable could not be converted

java.awt.image.ColorConvertOp could not be converted

java.awt.image.ConvolveOp could not be converted

java.awt.image.ColorModel.coerceData could not be converted

java.awt.image.ColorModel.ColorModel could not be converted

java.awt.image.ColorModel.createCompatibleSampleModel could not be converted

java.awt.image.ColorModel.createCompatibleWritableRaster could not be converted

java.awt.image.ColorModel.finalize could not be converted

java.awt.image.ColorModel.getAlpha could not be converted

java.awt.image.ColorModel.getAlphaRaster could not be converted

java.awt.image.ColorModel.getBlue could not be converted

java.awt.image.ColorModel.getColorSpace could not be converted

java.awt.image.ColorModel.getComponents(int, int[], int) could not be converted

java.awt.image.ColorModel.getComponents(object, int[], int) could not be converted

java.awt.image.ColorModel.getComponentSize could not be converted

java.awt.image.ColorModel.getComponentSize(int) could not be converted

java.awt.image.ColorModel.getDataElement could not be converted

java.awt.image.ColorModel.getDataElements(int, Object) could not be converted

java.awt.image.ColorModel.getDataElements(int[], int, Object) could not be converted

java.awt.image.ColorModel.getGreen could not be converted

java.awt.image.ColorModel.getNormalizedComponents could not be converted

java.awt.image.ColorModel.getNumColorComponents could not be converted

java.awt.image.ColorModel.getNumComponents could not be converted

java.awt.image.ColorModel.getRed could not be converted

java.awt.image.ColorModel.getRGB could not be converted

java.awt.image.ColorModel.getTransferType could not be converted

java.awt.image.ColorModel.getTransparency could not be converted

java.awt.image.ColorModel.getUnnormalizedComponents could not be converted

java.awt.image.ComponentColorModel.coerceData could not be converted

java.awt.image.ComponentColorModel.ComponentColorModel could not be converted

java.awt.image.ComponentColorModel.createCompatibleSampleModel could not be converted

java.awt.image.ComponentColorModel.createCompatibleWritableRaster could not be converted

java.awt.image.ComponentColorModel.getAlpha could not be converted

java.awt.image.ComponentColorModel.getAlphaRaster could not be converted

java.awt.image.ComponentColorModel.getBlue could not be converted

java.awt.image.ComponentColorModel.getComponents could not be converted

java.awt.image.ComponentColorModel.getDataElement could not be converted

java.awt.image.ComponentColorModel.getDataElements(int[], int) could not be converted

java.awt.image.ComponentColorModel.getDataElements(int, Object) could not be converted

java.awt.image.ComponentColorModel.getGreen could not be converted

java.awt.image.ComponentColorModel.getRed could not be converted

java.awt.image.ComponentColorModel.getRGB could not be converted

java.awt.image.ColorModel.isAlphaPremultiplied could not be converted

java.awt.image.ColorModel.isCompatibleRaster could not be converted

java.awt.image.ColorModel.isCompatibleSampleModel could not be converted

java.awt.image.ColorModel.transferType could not be converted

java.awt.image.ComponentColorModel.isCompatibleRaster could not be converted

java.awt.image.ComponentColorModel.isCompatibleSampleModel could not be converted

java.awt.image.ComponentSampleModel could not be converted

java.awt.image.ComponentSampleModel.bandOffsets could not be converted

java.awt.image.ComponentSampleModel.bankIndices could not be converted

java.awt.image.ComponentSampleModel.ComponentSampleModel could not be converted

java.awt.image.ComponentSampleModel.createCompatibleSampleModel could not be converted

java.awt.image.ComponentSampleModel.createSubsetSampleModel could not be converted

java.awt.image.ComponentSampleModel.getBandOffsets could not be converted

java.awt.image.ComponentSampleModel.getBankIndices could not be converted

java.awt.image.ComponentSampleModel.getDataElements could not be converted

java.awt.image.ComponentSampleModel.getNumDataElements could not be converted

java.awt.image.ComponentSampleModel.getOffset(int, int) could not be converted

java.awt.image.ComponentSampleModel.getOffset(int, int, int) could not be converted

java.awt.image.ComponentSampleModel.getPixel could not be converted
java.awt.image.ComponentSampleModel.getPixels could not be converted
java.awt.image.ComponentSampleModel.getSample could not be converted
java.awt.image.ComponentSampleModel.getSampleDouble could not be converted
java.awt.image.ComponentSampleModel.getSampleFloat could not be converted
java.awt.image.ComponentSampleModel.getSamples could not be converted
java.awt.image.ComponentSampleModel.getSampleSize could not be converted
java.awt.image.ComponentSampleModel.numBands could not be converted
java.awt.image.ComponentSampleModel.numBanks could not be converted
java.awt.image.ComponentSampleModel.pixelStride could not be converted
java.awt.image.ComponentSampleModel.scanlineStride could not be converted
java.awt.image.ComponentSampleModel.setDataElements could not be converted
java.awt.image.ComponentSampleModel.setSample could not be converted
java.awt.image.ComponentSampleModel.setSamples could not be converted
java.awt.image.DataBuffer.<DataType> could not be converted
java.awt.image.DataBuffer.DataBuffer(int, int, int, int) could not be converted
java.awt.image.DataBuffer.DataBuffer(int, int, int, int[]) could not be converted
java.awt.image.DataBuffer.dataType could not be converted
java.awt.image.DataBuffer.getDataType could not be converted
java.awt.image.DataBuffer.getDataTypeSize could not be converted
java.awt.image.DataBuffer.getElem could not be converted
java.awt.image.DataBuffer.getElemDouble could not be converted
java.awt.image.DataBuffer.getElemFloat could not be converted
java.awt.image.DataBuffer.getOffset could not be converted
java.awt.image.DataBuffer.getOffsets could not be converted
java.awt.image.DataBuffer.offset could not be converted
java.awt.image.DataBuffer.offsets could not be converted
java.awt.image.DataBuffer.setElem could not be converted
java.awt.image.DataBuffer.setElemDouble could not be converted
java.awt.image.DataBuffer.setElemFloat could not be converted
java.awt.image.DataBufferByte.DataBufferByte could not be converted
java.awt.image.DataBufferByte.getBankData could not be converted
java.awt.image.DataBufferInt.DataBufferInt could not be converted
java.awt.image.DataBufferInt.getBankData could not be converted
java.awt.image.DataBufferShort.DataBufferShort could not be converted
java.awt.image.DataBufferShort.getBankData could not be converted
java.awt.image.DataBufferUShort.DataBufferUShort could not be converted
java.awt.image.DataBufferUShort.getBankData could not be converted
java.awt.image.DirectColorModel.coerceData could not be converted
java.awt.image.DirectColorModel.createCompatibleWritableRaster could not be converted

java.awt.image.DirectColorModel.DirectColorModel(ColorSpace, int, int, int, int, int, boolean, int) could not be converted

java.awt.image.DirectColorModel.DirectColorModel(int, int, int, int) could not be converted

java.awt.image.DirectColorModel.DirectColorModel(int, int, int, int, int) could not be converted

java.awt.image.DirectColorModel.getAlpha could not be converted

java.awt.image.DirectColorModel.getBlue could not be converted

java.awt.image.DirectColorModel.getComponents could not be converted

java.awt.image.DirectColorModel.getDataElement could not be converted

java.awt.image.DirectColorModel.getDataElements(int, Object) could not be converted

java.awt.image.DirectColorModel.getDataElements(int[], int) could not be converted

java.awt.image.DirectColorModel.getGreen could not be converted

java.awt.image.DirectColorModel.getRed could not be converted

java.awt.image.DirectColorModel.getRGB could not be converted

java.awt.image.DirectColorModel.isCompatibleRaster could not be converted

java.awt.image.FilteredImageSource.startProduction could not be converted

java.awt.image.ImageConsumer could not be converted

java.awt.image.ImageFilter.setColorModel could not be converted

java.awt.image.ImageObserver could not be converted

java.awt.image.ImageProducer.startProduction could not be converted

java.awt.image.ImagingOpException could not be converted

java.awt.image.IndexColorModel.convertToIntDiscrete could not be converted

java.awt.image.IndexColorModel.createCompatibleSampleModel could not be converted

java.awt.image.IndexColorModel.createCompatibleWritableRaster could not be converted

java.awt.image.IndexColorModel.finalize could not be converted

java.awt.image.IndexColorModel.getComponents could not be converted

java.awt.image.IndexColorModel.getComponentSize could not be converted

java.awt.image.IndexColorModel.getDataElement could not be converted

java.awt.image.IndexColorModel.getDataElements could not be converted

java.awt.image.IndexColorModel.getRGBs could not be converted

java.awt.image.IndexColorModel.getTransparency could not be converted

java.awt.image.IndexColorModel.getValidPixels could not be converted

java.awt.image.IndexColorModel.IndexColorModel(int, int, int[], int, boolean,int,int) could not be converted

java.awt.image.IndexColorModel.IndexColorModel(int, int, int[], int, int, BigInteger) could not be converted

java.awt.image.IndexColorModel.isCompatibleRaster could not be converted

java.awt.image.IndexColorModel.isCompatibleSampleModel could not be converted

java.awt.image.Kernel could not be converted

java.awt.image.LookupOp could not be converted

java.awt.image.LookupTable could not be converted

java.awt.image.MemoryImageSource could not be converted

java.awt.image.MultiPixelPackedSampleModel could not be converted

java.awt.image.MultiPixelPackedSampleModel.createCompatibleSampleModel could not be converted

java.awt.image.MultiPixelPackedSampleModel.createSubsetSampleModel could not be converted

java.awt.image.MultiPixelPackedSampleModel.getBitOffset could not be converted

java.awt.image.MultiPixelPackedSampleModel.getDataBitOffset could not be converted

java.awt.image.MultiPixelPackedSampleModel.getDataElements could not be converted

java.awt.image.MultiPixelPackedSampleModel.getNumDataElements could not be converted

java.awt.image.MultiPixelPackedSampleModel.getOffset could not be converted

java.awt.image.MultiPixelPackedSampleModel.getPixel could not be converted

java.awt.image.MultiPixelPackedSampleModel.getSample could not be converted

java.awt.image.MultiPixelPackedSampleModel.getSampleSize could not be converted

java.awt.image.MultiPixelPackedSampleModel.getSampleSize(int) could not be converted

java.awt.image.MultiPixelPackedSampleModel.getTransferType could not be converted

java.awt.image.MultiPixelPackedSampleModel.MultiPixelPackedSampleModel(int, int, int, int) could not be converted

java.awt.image.MultiPixelPackedSampleModel.MultiPixelPackedSampleModel(int, int, int, int, int, int) could not be converted

java.awt.image.MultiPixelPackedSampleModel.setDataElements could not be converted

java.awt.image.MultiPixelPackedSampleModel.setSample could not be converted

java.awt.image.PackedColorModel.createCompatibleSampleModel could not be converted

java.awt.image.PackedColorModel.getAlphaRaster could not be converted

java.awt.image.PackedColorModel.getMask could not be converted

java.awt.image.PackedColorModel.getMasks could not be converted

java.awt.image.PackedColorModel.isCompatibleSampleModel could not be converted

java.awt.image.PackedColorModel.PackedColorModel could not be converted

java.awt.image.PixelGrabber.abortGrabbing could not be converted

java.awt.image.PixelGrabber.getStatus could not be converted

java.awt.image.PixelGrabber.setColorModel could not be converted

java.awt.image.PixelGrabber.startGrabbing could not be converted

java.awt.image.PixelGrabber.status could not be converted

java.awt.image.PixelInterleavedSampleModel could not be converted

java.awt.image.PixelInterleavedSampleModel.createCompatibleSampleModel could not be converted

java.awt.image.PixelInterleavedSampleModel.createSubsetSampleModel could not be converted

java.awt.image.PixelInterleavedSampleModel.PixelInterleavedSampleModel could not be converted

java.awt.image.Raster could not be converted

java.awt.image.Raster.createChild could not be converted

java.awt.image.Raster.createCompatibleWritableRaster could not be converted

java.awt.image.Raster.createRaster could not be converted

java.awt.image.Raster.createTranslatedChild could not be converted

java.awt.image.Raster.createWritableRaster could not be converted

java.awt.image.Raster.getDataBuffer could not be converted

java.awt.image.Raster.getDataElements(int, int, int, int, object) could not be converted

java.awt.image.Raster.getDataElements(int, int, object) could not be converted

java.awt.image.Raster.getNumBands could not be converted

java.awt.image.Raster.getNumDataElements could not be converted

java.awt.image.Raster.getParent could not be converted

java.awt.image.Raster.getPixel(int, int, double[]) could not be converted

java.awt.image.Raster.getPixel(int, int, float[]) could not be converted

java.awt.image.Raster.getPixels(int, int, int, int, double[]) could not be converted

java.awt.image.Raster.getPixels(int, int, int, int, float[]) could not be converted

java.awt.image.Raster.getPixels(int, int, int, int, int[]) could not be converted

java.awt.image.Raster.getPixel(int, int, int[]) could not be converted

java.awt.image.Raster.getSample could not be converted

java.awt.image.Raster.getSampleDouble could not be converted

java.awt.image.Raster.getSampleFloat could not be converted

java.awt.image.Raster.getSampleModelTranslateX could not be converted

java.awt.image.Raster.getSampleModelTranslateY could not be converted

java.awt.image.Raster.getSamples could not be converted

java.awt.image.Raster.getSamples(int, int, int, int, int, double[]) could not be converted

java.awt.image.Raster.getSamples(int, int, int, int, int, float[]) could not be converted

java.awt.image.Raster.getTransferType could not be converted

java.awt.image.Raster.numBands could not be converted

java.awt.image.Raster.numDataElements could not be converted

java.awt.image.Raster.parent could not be converted

java.awt.image.Raster.Raster(SampleModel, DataBuffer, Point) could not be converted

java.awt.image.Raster.Raster(SampleModel, DataBuffer, Rectangle, Point, Raster) could not be converted

java.awt.image.Raster.Raster(SampleModel, Point) could not be converted

java.awt.image.Raster.sampleModelTranslateX could not be converted

java.awt.image.Raster.sampleModelTranslateY could not be converted

java.awt.image.RasterOp could not be converted

java.awt.image.renderable.ContextualRenderedImageFactory could not be converted

java.awt.image.renderable.RenderableImage.createDefaultRendering could not be converted

java.awt.image.renderable.RenderableImage.createRendering could not be converted

java.awt.image.renderable.RenderableImage.createScaledRendering could not be converted

java.awt.image.renderable.RenderableImage.getMinX could not be converted

java.awt.image.renderable.RenderableImage.getMinY could not be converted

java.awt.image.renderable.RenderableImage.getProperty could not be converted

java.awt.image.renderable.RenderableImage.getPropertyNames could not be converted

java.awt.image.renderable.RenderableImage.getSources could not be converted

java.awt.image.renderable.RenderableImage.HINTS_OBSERVED could not be converted

java.awt.image.renderable.RenderableImage.isDynamic could not be converted

java.awt.image.renderable.RenderableImageOp could not be converted

java.awt.image.renderable.RenderableImageProducer.addConsumer could not be converted

java.awt.image.renderable.RenderableImageProducer.isConsumer could not be converted

java.awt.image.renderable.RenderableImageProducer.removeConsumer could not be converted

java.awt.image.renderable.RenderableImageProducer.RenderableImageProducer could not be converted

java.awt.image.renderable.RenderableImageProducer.requestTopDownLeftRightResend could not be converted

java.awt.image.renderable.RenderableImageProducer.run could not be converted

java.awt.image.renderable.RenderableImageProducer.setRenderContext could not be converted

java.awt.image.renderable.RenderableImageProducer.startProduction could not be converted

java.awt.image.renderable.RenderContext could not be converted

java.awt.image.renderable.RenderedImageFactory could not be converted

java.awt.image.RenderedImage.copyData could not be converted

java.awt.image.RenderedImage.getColorModel could not be converted

java.awt.image.RenderedImage.getMinTileX could not be converted

java.awt.image.RenderedImage.getMinTileY could not be converted

java.awt.image.RenderedImage.getNumXTiles could not be converted

java.awt.image.RenderedImage.getNumYTiles could not be converted

java.awt.image.RenderedImage.getProperty could not be converted

java.awt.image.RenderedImage.getPropertyNames could not be converted

java.awt.image.RenderedImage.getSources could not be converted

java.awt.image.RenderedImage.getTile could not be converted

java.awt.image.RenderedImage.getTileGridXOffset could not be converted

java.awt.image.RenderedImage.getTileGridYOffset could not be converted

java.awt.image.RenderedImage.getTileHeight could not be converted

java.awt.image.RenderedImage.getTileWidth could not be converted

java.awt.image.ReplicateScaleFilter.outpixmap could not be converted

java.awt.image.ReplicateScaleFilter.src* could not be converted

java.awt.image.RescaleOp could not be converted

java.awt.image.RGBImageFilter could not be converted

java.awt.image.SampleModel could not be converted

java.awt.image.SampleModel.createCompatibleSampleModel could not be converted

java.awt.image.SampleModel.createSubsetSampleModel could not be converted

java.awt.image.SampleModel.dataType could not be converted

java.awt.image.SampleModel.getDataElements(int, int, int, int, Object, DataBuffer) could not be converted

java.awt.image.SampleModel.getDataElements(int, int, Object, DataBuffer) could not be converted

java.awt.image.SampleModel.getDataType could not be converted

java.awt.image.SampleModel.getNumBands could not be converted

java.awt.image.SampleModel.getNumDataElements could not be converted

java.awt.image.SampleModel.getPixel(int, int, double[], DataBuffer) could not be converted

java.awt.image.SampleModel.getPixel(int, int, float[], DataBuffer) could not be converted

java.awt.image.SampleModel.getPixel(int, int, int[], DataBuffer) could not be converted

java.awt.image.SampleModel.getPixels(int, int, int, int, double[], DataBuffer) could not be converted

java.awt.image.SampleModel.getPixels(int, int, int, int, float[], DataBuffer) could not be converted

java.awt.image.SampleModel.getPixels(int, int, int, int, int[], DataBuffer) could not be converted

java.awt.image.SampleModel.getSample could not be converted

java.awt.image.SampleModel.getSampleDouble could not be converted

java.awt.image.SampleModel.getSampleFloat could not be converted

java.awt.image.SampleModel.getSamples(int, int, int, int, int, double[], DataBuffer) could not be converted

java.awt.image.SampleModel.getSamples(int, int, int, int, int, float[], DataBuffer) could not be converted

java.awt.image.SampleModel.getSamples(int, int, int, int, int, int[], DataBuffer) could not be converted

java.awt.image.SampleModel.getSampleSize could not be converted

java.awt.image.SampleModel.getSampleSize(int) could not be converted

java.awt.image.SampleModel.getTransferType could not be converted

java.awt.image.SampleModel.numBands could not be converted

java.awt.image.SampleModel.SampleModel could not be converted

java.awt.image.SampleModel.setDataElements(int, int, int, int, Object, DataBuffer) could not be converted

java.awt.image.SampleModel.setDataElements(int, int, Object, DataBuffer) could not be converted

java.awt.image.SampleModel.setPixel could not be converted

java.awt.image.SampleModel.setSample could not be converted

java.awt.image.SampleModel.setSamples could not be converted

java.awt.image.ShortLookupTable could not be converted

java.awt.image.SinglePixelPackedSampleModel could not be converted

java.awt.image.SinglePixelPackedSampleModel.createCompatibleSampleModel could not be converted

java.awt.image.SinglePixelPackedSampleModel.createSubsetSampleModel could not be converted

java.awt.image.SinglePixelPackedSampleModel.getBitMasks could not be converted

java.awt.image.SinglePixelPackedSampleModel.getBitOffsets could not be converted

java.awt.image.SinglePixelPackedSampleModel.getDataElements could not be converted

java.awt.image.SinglePixelPackedSampleModel.getNumDataElements could not be converted

java.awt.image.SinglePixelPackedSampleModel.getOffset could not be converted

java.awt.image.SinglePixelPackedSampleModel.getPixel could not be converted

java.awt.image.SinglePixelPackedSampleModel.getPixels could not be converted

java.awt.image.SinglePixelPackedSampleModel.getSample could not be converted

java.awt.image.SinglePixelPackedSampleModel.getSamples could not be converted

java.awt.image.SinglePixelPackedSampleModel.getSampleSize could not be converted

java.awt.image.SinglePixelPackedSampleModel.getSampleSize(int) could not be converted

java.awt.image.SinglePixelPackedSampleModel.setDataElements could not be converted

java.awt.image.SinglePixelPackedSampleModel.setSample could not be converted

java.awt.image.SinglePixelPackedSampleModel.setSamples could not be converted

java.awt.image.SinglePixelPackedSampleModel.SinglePixelPackedSampleModel(int, int, int, int, int[]) could not be converted

java.awt.image.SinglePixelPackedSampleModel.SinglePixelPackedSampleModel(int, int, int, int[]) could not be converted

java.awt.image.TileObserver could not be converted

java.awt.Image.UndefinedProperty could not be converted

java.awt.image.WritableRaster could not be converted

java.awt.image.WritableRaster.createWritableChild could not be converted

java.awt.image.WritableRaster.createWritableTranslatedChild could not be converted

java.awt.image.WritableRaster.getWritableParent could not be converted

java.awt.image.WritableRaster.setDataElements(int, int, int, int, Object) could not be converted

java.awt.image.WritableRaster.setDataElements(int, int, Object) could not be converted

java.awt.image.WritableRaster.setDataElements(int, int, Raster) could not be converted

java.awt.image.WritableRaster.setPixel(int, int, int, double) could not be converted

java.awt.image.WritableRaster.setPixel(int, int, int, float) could not be converted

java.awt.image.WritableRaster.setPixel(int, int, int, int) could not be converted

java.awt.image.WritableRaster.setRect could not be converted

java.awt.image.WritableRaster.setSample(int, int, int, double) could not be converted

java.awt.image.WritableRaster.setSample(int, int, int, float) could not be converted

java.awt.image.WritableRaster.setSample(int, int, int, int) could not be converted

java.awt.image.WritableRaster.setSamples(int, int, int, int, int, double[]) could not be converted

java.awt.image.WritableRaster.setSamples(int, int, int, int, int, float[]) could not be converted

java.awt.image.WritableRaster.setSamples(int, int, int, int, int, int[]) could not be converted

java.awt.image.WritableRaster.WritableRaster(Rectangle, Point, WritableRaster) could not be converted

java.awt.image.WritableRaster.WritableRaster(SampleModel, DataBuffer, Point) could not be converted

java.awt.image.WritableRaster.WritableRaster(SampleModel, Point) could not be converted

java.awt.image.WritableRenderedImage could not be converted

Java.awt.peer Error messages

java.awt.peer.CanvasPeer could not be converted

java.awt.peer.ComponentPeer.checkImage could not be converted

java.awt.peer.ComponentPeer.coalescePaintEvent could not be converted

java.awt.peer.ComponentPeer.createImage(ImageProducer) could not be converted

java.awt.peer.ComponentPeer.createImage(int, int) could not be converted

java.awt.peer.ComponentPeer.getColorModel could not be converted

java.awt.peer.ComponentPeer.getGraphicsConfiguration could not be converted

java.awt.peer.ComponentPeer.getMinimumSize could not be converted

java.awt.peer.ComponentPeer.getPreferredSize could not be converted

java.awt.peer.ComponentPeer.getToolkit could not be converted

java.awt.peer.ComponentPeer.handleEvent could not be converted

java.awt.peer.ComponentPeer.minimumSize could not be converted

java.awt.peer.ComponentPeer.paint could not be converted

java.awt.peer.ComponentPeer.preferredSize could not be converted

java.awt.peer.ComponentPeer.prepareImage could not be converted

java.awt.peer.ComponentPeer.print could not be converted

java.awt.peer.ComponentPeer.setBounds could not be converted

java.awt.peer.ComponentPeer.setVisible could not be converted

java.awt.peer.ContainerPeer.beginValidate could not be converted

java.awt.peer.ContainerPeer.endValidate could not be converted

java.awt.peer.FileDialogPeer.setFilenameFilter could not be converted

java.awt.peer.FramePeer.setIconImage could not be converted

java.awt.peer.LightweightPeer could not be converted

java.awt.peer.ListPeer.delItems could not be converted

java.awt.peer.ListPeer.makeVisible could not be converted

java.awt.peer.MenuBarPeer.addHelpMenu could not be converted

java.awt.peer.PopupMenuPeer.show could not be converted

java.awt.peer.RobotPeer could not be converted

java.awt.peer.ScrollbarPeer.setValues could not be converted

java.awt.peer.ScrollPanePeer.setUnitIncrement could not be converted

java.awt.peer.ScrollPanePeer.setValue could not be converted

java.awt.peer.TextComponentPeer.filterEvents could not be converted

java.awt.peer.TextComponentPeer.getCharacterBounds could not be converted

java.awt.peer.TextComponentPeer.getIndexAtPoint could not be converted

java.awt.peer.TextFieldPeer.getPreferredSize could not be converted

java.awt.peer.TextFieldPeer.preferredSize could not be converted

java.awt.peer.WindowPeer.handleFocusTraversalEvent could not be converted

Java.awt.print Error Messages

java.awt.print.Book could not be converted

java.awt.print.Pageable could not be converted

java.awt.print.PageFormat.<Format> could not be converted

java.awt.print.PageFormat.getMatrix could not be converted

java.awt.print.PageFormat.setOrientation could not be converted

java.awt.print.Paper.clone could not be converted

java.awt.print.Paper.getImageableHeight could not be converted

java.awt.print.Paper.getImageableWidth could not be converted

java.awt.print.Paper.getImageableX could not be converted

java.awt.print.Paper.getImageableY could not be converted

java.awt.print.Paper.Paper could not be converted

java.awt.print.Paper.setImageableArea could not be converted

java.awt.print.Paper.setSize could not be converted

java.awt.print.Printable could not be converted

java.awt.print.PrinterGraphics could not be converted

java.awt.print.PrinterJob.cancel could not be converted

java.awt.print.PrinterJob.defaultPage could not be converted

java.awt.print.PrinterJob.isCancelled could not be converted

java.awt.print.PrinterJob.setPageable could not be converted

java.awt.print.PrinterJob.setPrintable(Printable) could not be converted

java.awt.print.PrinterJob.setPrintable(Printable, PageFormat) could not be converted

java.awt.print.PrinterJob.validatePage could not be converted

Java.beans Error Messages

java.beans.AppletInitializer could not be converted

java.beans.BeanDescriptor could not be converted

java.beans.BeanDescriptor.BeanDescriptor could not be converted

java.beans.BeanDescriptor.getCustomizerClass could not be converted

java.beans.BeanInfo could not be converted

java.beans.BeanInfo.ICON_COLOR_16x16 could not be converted

java.beans.BeanInfo.ICON_COLOR_32x32 could not be converted

java.beans.BeanInfo.ICON_MONO_16x16 could not be converted

java.beans.BeanInfo.ICON_MONO_32x32 could not be converted

java.beans.Beans could not be converted

java.beans.Customizer could not be converted

java.beans.DesignMode could not be converted

java.beans.EventSetDescriptor could not be converted

java.beans.EventSetDescriptor.EventSetDescriptor could not be converted

java.beans.EventSetDescriptor.getAddListenerMethod could not be converted

java.beans.EventSetDescriptor.getListenerMethodDescriptors could not be converted

java.beans.EventSetDescriptor.getListenerMethods could not be converted

java.beans.EventSetDescriptor.getListenerType could not be converted

java.beans.EventSetDescriptor.getRemoveListenerMethod could not be converted

java.beans.EventSetDescriptor.isInDefaultEventSet could not be converted

java.beans.EventSetDescriptor.isUnicast could not be converted

java.beans.EventSetDescriptor.setInDefaultEventSet could not be converted

java.beans.EventSetDescriptor.setUnicast could not be converted

java.beans.FeatureDescriptor could not be converted

java.beans.FeatureDescriptor.attributeNames could not be converted

java.beans.FeatureDescriptor.FeatureDescriptor could not be converted

java.beans.FeatureDescriptor.getDisplayName could not be converted

java.beans.FeatureDescriptor.getName could not be converted

java.beans.FeatureDescriptor.getShortDescription could not be converted

java.beans.FeatureDescriptor.getValue could not be converted

java.beans.FeatureDescriptor.isExpert could not be converted

java.beans.FeatureDescriptor.isHidden could not be converted

java.beans.FeatureDescriptor.isPreferred could not be converted

java.beans.FeatureDescriptor.setDisplayName could not be converted

java.beans.FeatureDescriptor.setExpert could not be converted

java.beans.FeatureDescriptor.setHidden could not be converted

java.beans.FeatureDescriptor.setName could not be converted

java.beans.FeatureDescriptor.setPreferred could not be converted
java.beans.FeatureDescriptor.setShortDescription could not be converted
java.beans.FeatureDescriptor.setValue could not be converted
java.beans.IndexedPropertyDescriptor could not be converted
java.beans.IntrospectionException could not be converted
java.beans.Introspector could not be converted
java.beans.MethodDescriptor could not be converted
java.beans.ParameterDescriptor could not be converted
java.beans.PropertyChangeEvent.getPropagationId could not be converted
java.beans.PropertyChangeEvent.PropertyChangeEvent could not be converted
java.beans.PropertyChangeEvent.setPropagationId could not be converted
java.beans.PropertyChangeListener could not be converted
java.beans.PropertyChangeSupport could not be converted
java.beans.PropertyChangeSupport.addPropertyChangeListener could not be converted
java.beans.PropertyChangeSupport.firePropertyChange could not be converted
java.beans.PropertyChangeSupport.hasListeners could not be converted
java.beans.PropertyChangeSupport.PropertyChangeSupport could not be converted
java.beans.PropertyChangeSupport.removePropertyChangeListener could not be converted
java.beans.PropertyDescriptor could not be converted
java.beans.PropertyDescriptor.getPropertyEditorClass could not be converted
java.beans.PropertyDescriptor.getReadMethod could not be converted
java.beans.PropertyDescriptor.getWriteMethod could not be converted
java.beans.PropertyDescriptor.isBound could not be converted
java.beans.PropertyDescriptor.isConstrained could not be converted
java.beans.PropertyDescriptor.PropertyDescriptor could not be converted
java.beans.PropertyDescriptor.setBound could not be converted
java.beans.PropertyDescriptor.setConstrained could not be converted
java.beans.PropertyDescriptor.setPropertyEditorClass could not be converted
java.beans.PropertyDescriptor.setReadMethod could not be converted
java.beans.PropertyDescriptor.setWriteMethod could not be converted
java.beans.PropertyEditor could not be converted
java.beans.PropertyEditorManager could not be converted
java.beans.PropertyEditorSupport could not be converted
java.beans.PropertyVetoException.getPropertyChangeEvent could not be converted
java.beans.SimpleBeanInfo could not be converted
java.beans.VetoableChangeListener could not be converted
java.beans.VetoableChangeSupport could not be converted
java.beans.VetoableChangeSupport.addVetoableChangeListener could not be converted
java.beans.VetoableChangeSupport.fireVetoableChange could not be converted
java.beans.VetoableChangeSupport.hasListeners could not be converted

java.beans.VetoableChangeSupport.removeVetoableChangeListener could not be converted

java.beans.VetoableChangeSupport.VetoableChangeSupport could not be converted

java.beans.Visibility could not be converted

Java.io Error Messages

java.io.BufferedInputStream.available could not be converted

java.io.BufferedInputStream.buf could not be converted

java.io.BufferedInputStream.marklimit could not be converted

java.io.BufferedInputStream.markpos could not be converted

java.io.BufferedOutputStream.buf could not be converted

java.io.BufferedReader.BufferedReader could not be converted

java.io.BufferedReader.mark could not be converted

java.io.BufferedReader.reset could not be converted

java.io.BufferedWriter.BufferedWriter could not be converted

java.io.BufferedWriter.write could not be converted

java.io.ByteArrayInputStream.buf could not be converted

java.io.ByteArrayInputStream.mark~ could not be converted

java.io.ByteArrayOutputStream.buf could not be converted

java.io.ByteArrayOutputStream.reset could not be converted

java.io.ByteArrayOutputStream.toString could not be converted

java.io.CharArrayReader.buf could not be converted

java.io.CharArrayReader.count could not be converted

java.io.CharArrayReader.mark could not be converted

java.io.CharArrayReader.markedPos could not be converted

java.io.CharArrayReader.pos could not be converted

java.io.CharArrayReader.reset could not be converted

java.io.CharArrayReader.skip could not be converted

java.io.CharArrayWriter.reset could not be converted

java.io.CharArrayWriter.size could not be converted

java.io.CharArrayWriter.toCharArray could not be converted

java.io.CharArrayWriter.toString could not be converted

java.io.CharArrayWriter.write could not be converted

java.io.CharArrayWriter.writeTo could not be converted

java.io.DataInput could not be converted

java.io.DataInput.readLine could not be converted

java.io.DataInput.readUTF could not be converted

java.io.DataInputStream could not be converted

java.io.DataInputStream.readLine could not be converted

java.io.DataInputStream.readUTF could not be converted

java.io.DataOutput could not be converted

java.io.DataOutput.writeBytes could not be converted

java.io.DataOutput.writeChars could not be converted

java.io.DataOutput.writeUTF could not be converted

java.io.DataOutputStream could not be converted

java.io.DataOutputStream.writeBytes could not be converted

java.io.DataOutputStream.writeChars could not be converted

java.io.DataOutputStream.writeUTF could not be converted

java.io.DataOutputStream.written could not be converted

java.io.Externalizable could not be converted

java.io.File.canRead could not be converted

java.io.File.createTempFile(String, String) could not be converted

java.io.File.createTempFile(String, String, File) could not be converted

java.io.File.deleteOnExit could not be converted

java.io.File.isAbsolute could not be converted

java.io.File.list could not be converted

java.io.File.listFiles could not be converted

java.io.File.mkdir could not be converted

java.io.File.mkdirs could not be converted

java.io.File.renameTo could not be converted

java.io.File.setLastModified could not be converted

java.io.FileDescriptor could not be converted

java.io.FileFilter could not be converted

java.io.FileInputStream.available could not be converted

java.io.FileInputStream.FileInputStream could not be converted

java.io.FileInputStream.getFD could not be converted

java.io.FileNameFilter could not be converted

java.io.FileOutputStream.FileOutputStream could not be converted

java.io.FileOutputStream.getFD could not be converted

java.io.FileOutputStream.write could not be converted

java.io.FilePermission.FilePermission could not be converted

java.io.FileReader.FileReader could not be converted

java.io.FileWriter.FileWriter could not be converted

java.io.FilterInputStream.available could not be converted

java.io.FilterInputStream.close could not be converted

java.io.FilterInputStream.FileInputStream could not be converted

java.io.FilterInputStream.mark could not be converted

java.io.FilterInputStream.markSupported could not be converted

java.io.FilterInputStream.reset could not be converted

java.io.FilterReader.FilterReader could not be converted

java.io.FilterReader.in could not be converted

java.io.FilterReader.mark could not be converted

java.io.FilterReader.markSupported could not be converted

java.io.FilterReader.reset could not be converted

java.io.FileWriter could not be converted

java.io.FileWriter.FileWriter could not be converted

java.io.FilterWriter.write could not be converted

java.io.InputStream.available could not be converted

java.io.InputStream.InputStream could not be converted

java.io.InputStream.mark could not be converted

java.io.InputStream.markSupported could not be converted

java.io.InputStream.read could not be converted

java.io.InputStream.reset could not be converted

java.io.InputStreamReader.InputStreamReader could not be converted

java.io.InputStreamReader.read could not be converted

java.io.InputStreamReader.read(char[], int, int) could not be converted

java.io.InterruptedIOException.bytesTransferred could not be converted

java.io.InvalidClassException.classname could not be converted

java.io.InvalidClassException.InvalidClassException could not be converted

java.io.LineNumberInputStream.available could not be converted

java.io.LineNumberInputStream.getLineNumber could not be converted

java.io.LineNumberInputStream.mark could not be converted

java.io.LineNumberInputStream.reset could not be converted

java.io.LineNumberInputStream.setLineNumber could not be converted

java.io.LineNumberReader.getLineNumber could not be converted

java.io.LineNumberReader.LineNumberReader could not be converted

java.io.LineNumberReader.mark could not be converted

java.io.LineNumberReader.reset could not be converted

java.io.LineNumberReader.setLineNumber could not be converted

java.io.ObjectInput could not be converted

java.io.ObjectInput.available could not be converted

java.io.ObjectInput.readObject could not be converted

java.io.ObjectInputStream could not be converted

java.io.ObjectInputStream.available could not be converted

java.io.ObjectInputStream.defaultReadObject could not be converted

java.io.ObjectInputStream.enableResolveObject could not be converted

java.io.ObjectInputStream.GetField could not be converted

java.io.ObjectInputStream.ObjectInputStream could not be converted

java.io.ObjectInputStream.readClassDescriptor could not be converted

java.io.ObjectInputStream.readFields could not be converted

java.io.ObjectInputStream.readLine could not be converted

java.io.ObjectInputStream.readObjectOverride could not be converted

java.io.ObjectInputStream.readStreamHeader could not be converted

java.io.ObjectInputStream.readUTF could not be converted

java.io.ObjectInputStream.registerValidation could not be converted

java.io.ObjectInputStream.resolveClass could not be converted

java.io.ObjectInputStream.resolveObject could not be converted

java.io.ObjectInputStream.resolveProxyClass could not be converted

java.io.ObjectInputValidation could not be converted

java.io.ObjectOutput could not be converted

java.io.ObjectOutput.writeObject could not be converted

java.io.ObjectOutputStream could not be converted

java.io.ObjectOutputStream.annotateClass could not be converted

java.io.ObjectOutputStream.annotateProxyClass could not be converted

java.io.ObjectOutputStream.baseWriteHandle could not be converted

java.io.ObjectOutputStream.defaultWriteObject could not be converted

java.io.ObjectOutputStream.enableReplaceObject could not be converted

java.io.ObjectOutputStream.ObjectOutputStream could not be converted

java.io.ObjectOutputStream.PutField could not be converted

java.io.ObjectOutputStream.putFields could not be converted

java.io.ObjectOutputStream.replaceObject could not be converted

java.io.ObjectOutputStream.reset could not be converted

java.io.ObjectOutputStream.SC_EXTERNALIZABLE could not be converted

java.io.ObjectOutputStream.SC_SERIALIZABLE could not be converted

java.io.ObjectOutputStream.SC_WRITE_METHOD could not be converted

java.io.ObjectOutputStream.useProtocolVersion could not be converted

java.io.ObjectOutputStream.writeBytes could not be converted

java.io.ObjectOutputStream.writeChars could not be converted

java.io.ObjectOutputStream.writeClassDescriptor could not be converted

java.io.ObjectOutputStream.writeFields could not be converted

java.io.ObjectOutputStream.writeObject could not be converted

java.io.ObjectOutputStream.writeObjectOverride could not be converted

java.io.ObjectOutputStream.writeStreamHeader could not be converted

java.io.ObjectOutputStream.writeUTF could not be converted

java.io.ObjectStreamClass could not be converted

java.io.OptionalDataException could not be converted

java.io.OutputStreamWriter.getEncoding could not be converted

java.io.OutputStreamWriter.OutputStreamWriter could not be converted

java.io.OutputStreamWriter.write could not be converted

java.io.PipedInputStream.available could not be converted

java.io.PipedInputStream.buffer could not be converted

java.io.PipedInputStream.connect could not be converted

java.io.PipedInputStream.in could not be converted

java.io.PipedInputStream.out could not be converted
java.io.PipedInputStream.PIPE_SIZE could not be converted
java.io.PipedInputStream.PipedInputStream could not be converted
java.io.PipedInputStream.read could not be converted
java.io.PipedInputStream.receive could not be converted
java.io.PipedOutputStream.connect could not be converted
java.io.PipedOutputStream.PipedOutputStream could not be converted
java.io.PipedOutputStream.write could not be converted
java.io.PipedReader.connect could not be converted
java.io.PipedWriter.connect could not be converted
java.io.PipedWriter.write could not be converted
java.io.PrintStream.checkError could not be converted
java.io.PrintStream.print could not be converted
java.io.PrintStream.println could not be converted
java.io.PrintStream.setError could not be converted
java.io.PrintStream.write could not be converted
java.io.PrintWriter.checkError could not be converted
java.io.PrintWriter.print could not be converted
java.io.PrintWriter.PrintWriter could not be converted
java.io.PrintWriter.setError could not be converted
java.io.PrintWriter.write could not be converted
java.io.PushbackInputStream.available could not be converted
java.io.RandomAccessFile could not be converted
java.io.RandomAccessFile.getFD could not be converted
java.io.RandomAccessFile.RandomAccessFile(File, String) could not be converted
java.io.RandomAccessFile.RandomAccessFile(File, String, String) could not be converted
java.io.RandomAccessFile.readUTF could not be converted
java.io.RandomAccessFile.writeUTF could not be converted
java.io.Reader.lock could not be converted
java.io.Reader.mark could not be converted
java.io.Reader.markSupported could not be converted
java.io.Reader.read could not be converted
java.io.Reader.Reader could not be converted
java.io.Reader.reset could not be converted
java.io.SequenceInputStream.available could not be converted
java.io.SequenceInputStream.SequenceInputStream could not be converted
java.io.SerializablePermission could not be converted
java.io.StreamTokenizer could not be converted
java.io.StreamTokenizer.StreamTokenizer could not be converted
java.io.StringBufferInputStream.available could not be converted

java.io.StringBufferInputStream.buffer could not be converted
java.io.StringBufferInputStream.count could not be converted
java.io.StringBufferInputStream.pos could not be converted
java.io.StringBufferInputStream.read could not be converted
java.io.StringBufferInputStream.reset could not be converted
java.io.StringBufferInputStream.skip could not be converted
java.io.StringReader.mark could not be converted
java.io.StringReader.markSupported could not be converted
java.io.StringReader.ready could not be converted
java.io.StringReader.reset could not be converted
java.io.StringReader.skip could not be converted
java.io.StringWriter.getBuffer could not be converted
java.io.StringWriter.write could not be converted
java.io.Writer.lock could not be converted
java.io.Writer.write could not be converted

Java.lang Error Messages

java.lang.Boolean.getBoolean could not be converted

java.lang.Character.forDigit could not be converted

java.lang.Character.isDefined could not be converted

java.lang.Character.isIdentifierIgnorable could not be converted

java.lang.Character.isJavaIdentifierPart could not be converted

java.lang.Character.isUnicodeIdentifierPart could not be converted

java.lang.Character.isUnicodeIdentifierStart could not be converted

java.lang.Class.forName could not be converted

java.lang.Class.getClassLoader could not be converted

java.lang.Class.getDeclaredMethod could not be converted

java.lang.Class.getModifiers could not be converted

java.lang.Class.getPackage could not be converted

java.lang.Class.getProtectionDomain could not be converted

java.lang.Class.getResource could not be converted

java.lang.Class.getResourceAsStream could not be converted

java.lang.Class.getSigners could not be converted

java.lang.Class.newInstance could not be converted

java.lang.ClassLoader could not be converted

java.lang.ClassNotFoundException.printStackTrace could not be converted

java.lang.Compiler could not be converted

java.lang.Double.doubleToLongBits could not be converted

java.lang.Double.doubleToRawLongBits could not be converted

java.lang.Double.longBitsToDouble could not be converted

java.lang.ExceptionInInitializerError.getException could not be converted

java.lang.ExceptionInInitializerError.printStackTrace could not be converted

java.lang.Float.floatToIntBits could not be converted

java.lang.Float.floatToRawIntBits could not be converted

java.lang.Float.intBitsToFloat could not be converted

java.lang.InheritableThreadLocal could not be converted

java.lang.Integer.getInteger could not be converted

java.lang.Integer.TYPE could not be converted

java.lang.Long.getLong could not be converted

java.lang.Math.round could not be converted

java.lang.Number could not be converted

java.lang.Number.Number could not be converted

java.lang.Object.class could not be converted

java.lang.Object.clone could not be converted

java.lang.Package could not be converted

java.lang.ref.PhantomReference could not be converted

java.lang.ref.Reference could not be converted

java.lang.ref.ReferenceQueue could not be converted

java.lang.ref.SoftReference could not be converted

java.lang.ref.WeakReference.WeakReference could not be converted

java.lang.reflect.AccessibleObject could not be converted could not be converted

java.lang.reflect.Constructor.getExceptionTypes could not be converted

java.lang.reflect.Constructor.newInstance could not be converted

java.lang.reflect.Field.getModifiers could not be converted

java.lang.reflect.Field.setByte could not be converted

java.lang.reflect.Field.setChar could not be converted

java.lang.reflect.Field.setShort could not be converted

java.lang.reflect.InvocationHandler could not be converted could not be converted

java.lang.reflect.InvocationTargetException.InvocationTargetException could not be converted

java.lang.reflect.Member.DECLARED could not be converted

java.lang.reflect.Member.getModifiers could not be converted

java.lang.reflect.Member.PUBLIC could not be converted

java.lang.reflect.Method.getExceptionTypes could not be converted

java.lang.reflect.Method.getModifiers could not be converted

java.lang.reflect.Modifier could not be converted

java.lang.reflect.Proxy could not be converted could not be converted

java.lang.reflect.ReflectPermission could not be converted could not be converted

java.lang.Runtime.addShutdownHook could not be converted

java.lang.Runtime.exec could not be converted

java.lang.Runtime.freeMemory could not be converted

java.lang.Runtime.getLocalizedInputStream could not be converted

java.lang.Runtime.getLocalizedOutputStream could not be converted

java.lang.Runtime.halt could not be converted

java.lang.Runtime.load could not be converted

java.lang.Runtime.loadLibrary could not be converted

java.lang.Runtime.removeShutdownHook could not be converted

java.lang.Runtime.runFinalizersOnExit could not be converted

java.lang.Runtime.Runtime could not be converted

java.lang.Runtime.totalMemory could not be converted

java.lang.Runtime.traceInstructions could not be converted

java.lang.Runtime.traceMethodCalls could not be converted

java.lang.RuntimePermission could not be converted

java.lang.SecurityManager.checkAccess could not be converted

java.lang.SecurityManager.checkAwtEventQueueAccess could not be converted

java.lang.SecurityManager.checkConnect could not be converted
java.lang.SecurityManager.checkCreateClassLoader could not be converted
java.lang.SecurityManager.checkExec could not be converted
java.lang.SecurityManager.checkExit could not be converted
java.lang.SecurityManager.checkLink could not be converted
java.lang.SecurityManager.checkListen could not be converted
java.lang.SecurityManager.checkMemberAccess could not be converted
java.lang.SecurityManager.checkMulticast could not be converted
java.lang.SecurityManager.checkPackageAccess could not be converted
java.lang.SecurityManager.checkPackageDefinition could not be converted
java.lang.SecurityManager.checkPermission(Permission) could not be converted
java.lang.SecurityManager.checkPermission(Permission, Object) could not be converted
java.lang.SecurityManager.checkPrintJobAccess could not be converted
java.lang.SecurityManager.checkPropertiesAccess could not be converted
java.lang.SecurityManager.checkPropertyAccess could not be converted
java.lang.SecurityManager.checkRead could not be converted
java.lang.SecurityManager.checkSecurityAccess could not be converted
java.lang.SecurityManager.checkSetFactory could not be converted
java.lang.SecurityManager.checkTopLevelWindow could not be converted
java.lang.SecurityManager.checkWrite could not be converted
java.lang.SecurityManager.classDepth could not be converted
java.lang.SecurityManager.classLoaderDepth could not be converted
java.lang.SecurityManager.currentClassLoader could not be converted
java.lang.SecurityManager.currentLoadedClass could not be converted
java.lang.SecurityManager.getClassContext could not be converted
java.lang.SecurityManager.getInCheck could not be converted
java.lang.SecurityManager.getSecurityContext could not be converted
java.lang.SecurityManager.getThreadGroup could not be converted
java.lang.SecurityManager.inCheck could not be converted
java.lang.SecurityManager.inClass could not be converted
java.lang.SecurityManager.inClassLoader could not be converted
java.lang.SecurityManager.SecurityManager could not be converted
java.lang.StackOverflowError could not be converted
java.lang.StrictMath.round could not be converted
java.lang.String.CASE_INSENSITIVE_ORDER could not be converted
java.lang.String.getBytes could not be converted
java.lang.String.String(byte[]) could not be converted
java.lang.String.String(byte[], int, int, int) could not be converted
java.lang.String.String(byte[], int, int, String) could not be converted
java.lang.String.String(byte[], String) could not be converted

java.lang.System could not be converted

java.lang.System.currentTimeMillis could not be converted

java.lang.System.getProperties could not be converted

java.lang.System.getProperty could not be converted

java.lang.System.getSecurityManager could not be converted

java.lang.System.load could not be converted

java.lang.System.loadLibrary could not be converted

java.lang.System.mapLibraryName could not be converted

java.lang.System.runFinalization could not be converted

java.lang.System.runFinalizersOnExit could not be converted

java.lang.System.setErr could not be converted

java.lang.System.setIn could not be converted

java.lang.System.setOut could not be converted

java.lang.System.setProperty could not be converted

java.lang.System.setProperties could not be converted

java.lang.System.setSecurityManager could not be converted

java.lang.Thread.activeCount could not be converted

java.lang.Thread.checkAccess could not be converted

java.lang.Thread.countStackFrames could not be converted

java.lang.Thread.destroy could not be converted

java.lang.Thread.enumerate could not be converted

java.lang.Thread.getContextClassLoader could not be converted

java.lang.Thread.getThreadGroup could not be converted

java.lang.Thread.interrupted could not be converted

java.lang.Thread.isInterrupted could not be converted

java.lang.Thread.run could not be converted

java.lang.Thread.setContextClassLoader could not be converted

java.lang.Thread.sleep(long) could not be converted

java.lang.Thread.sleep(long, int) could not be converted

java.lang.Thread.Thread could not be converted

java.lang.Thread.yield could not be converted

java.lang.ThreadGroup could not be converted

java.lang.ThreadLocal.initialValue could not be converted

java.lang.Throwable.fillInStackTrace could not be converted

java.lang.UnsupportedClassVersionError could not be converted

Java.math Error Messages

java.math.BigDecimal could not be converted

java.math.BigDecimal.BigDecimal could not be converted

java.math.BigDecimal.divide could not be converted

java.math.BigDecimal.movePointLeft could not be converted

java.math.BigDecimal.movePointRight could not be converted

java.math.BigDecimal.ROUND_CEILING could not be converted

java.math.BigDecimal.ROUND_DOWN could not be converted

java.math.BigDecimal.ROUND_FLOOR could not be converted

java.math.BigDecimal.ROUND_HALF_DOWN could not be converted

java.math.BigDecimal.ROUND_HALF_EVEN could not be converted

java.math.BigDecimal.ROUND_HALF_UP could not be converted

java.math.BigDecimal.ROUND_UNNECESSARY could not be converted

java.math.BigDecimal.ROUND_UP could not be converted

java.math.BigDecimal.scale could not be converted

java.math.BigDecimal.setScale could not be converted

java.math.BigDecimal.toBigInteger could not be converted

java.math.BigDecimal.unscaledValue could not be converted

java.math.BigDecimal.valueOf could not be converted

java.math.BigInteger could not be converted

java.math.BigInteger.and could not be converted

java.math.BigInteger.andNot could not be converted

java.math.BigInteger.BigInteger(byte[]) could not be converted

java.math.BigInteger.BigInteger(int, byte[]) could not be converted

java.math.BigInteger.BigInteger(int, int, Random) could not be converted

java.math.BigInteger.BigInteger(int, Random) could not be converted

java.math.BigInteger.BigInteger(String, int) could not be converted

java.math.BigInteger.bitCount could not be converted

java.math.BigInteger.bitLength could not be converted

java.math.BigInteger.clearBit could not be converted

java.math.BigInteger.flipBit could not be converted

java.math.BigInteger.gcd could not be converted

java.math.BigInteger.getLowestSetBit could not be converted

java.math.BigInteger.isProbablePrime could not be converted

java.math.BigInteger.modInverse could not be converted

java.math.BigInteger.modPow could not be converted

java.math.BigInteger.not could not be converted

java.math.BigInteger.or could not be converted

java.math.BigInteger.pow could not be converted

java.math.BigInteger.setBit could not be converted

java.math.BigInteger.shiftLeft could not be converted

java.math.BigInteger.shiftRight could not be converted

java.math.BigInteger.testBit could not be converted

java.math.BigInteger.toByteArray could not be converted

java.math.BigInteger.toString could not be converted

java.math.BigInteger.xor could not be converted

Java.net Error Messages

java.net.Authenticator could not be converted

java.net.ContentHandler could not be converted

java.net.ContentHandlerFactory could not be converted

java.net.DatagramPacket.DatagramPacket could not be converted

java.net.DatagramPacket.getOffset could not be converted

java.net.DatagramSocket.getLocalAddress could not be converted

java.net.DatagramSocket.getLocalPort could not be converted

java.net.DatagramSocket.getReceiveBufferSize could not be converted

java.net.DatagramSocket.getSendBufferSize could not be converted

java.net.DatagramSocket.getSoTimeout could not be converted

java.net.DatagramSocket.setDatagramSocketImplFactory could not be converted

java.net.DatagramSocket.setReceiveBufferSize could not be converted

java.net.DatagramSocket.setSendBufferSize could not be converted

java.net.DatagramSocket.setSoTimeout could not be converted

java.net.DatagramSocketImpl could not be converted

java.net.DatagramSocketImplFactory could not be converted

java.net.FileNameMap could not be converted

java.net.HttpURLConnection.getErrorStream could not be converted

java.net.HttpURLConnection.getFollowRedirects could not be converted

java.net.HttpURLConnection.getPermission could not be converted

java.net.HttpURLConnection.getResponseCode could not be converted

java.net.HttpURLConnection.getResponseMessage could not be converted

java.net.HttpURLConnection.HTTP_MOVED_TEMP could not be converted

java.net.HttpURLConnection.responseCode could not be converted

java.net.HttpURLConnection.responseMessage could not be converted

java.net.HttpURLConnection.usingProxy could not be converted

java.net.InetAddress.getAddress could not be converted

java.net.InetAddress.getAllByName could not be converted

java.net.InetAddress.getByName could not be converted

java.net.InetAddress.getLocalHost could not be converted

java.net.InetAddress.isMulticastAddress could not be converted

java.net.JarURLConnection could not be converted

java.net.MulticastSocket.getInterface could not be converted

java.net.MulticastSocket.getTimeToLive could not be converted

java.net.MulticastSocket.getTTL could not be converted

java.net.MulticastSocket.send could not be converted

java.net.MulticastSocket.setInterface could not be converted

java.net.MulticastSocket.setTimeToLive could not be converted

java.net.MulticastSocket.setTTL could not be converted

java.net.NetPermission.NetPermission could not be converted

java.net.NoRouteToHostException.NoRouteToHostException could not be converted

java.net.PlainDatagramSocketImpl could not be converted

java.net.PlainSocketImpl could not be converted

java.net.ServerSocket.getSoTimeout could not be converted

java.net.ServerSocket.implAccept could not be converted

java.net.ServerSocket.ServerSocket could not be converted

java.net.ServerSocket.setSocketFactory could not be converted

java.net.ServerSocket.setSoTimeout could not be converted

java.net.Socket.getInetAddress could not be converted

java.net.Socket.getKeepAlive could not be converted

java.net.Socket.getLocalAddress could not be converted

java.net.Socket.getLocalPort could not be converted

java.net.Socket.getPort could not be converted

java.net.Socket.setKeepAlive could not be converted

java.net.Socket.setSocketImplFactory could not be converted

java.net.Socket.shutdownInput could not be converted

java.net.Socket.shutdownOutput could not be converted

java.net.Socket.Socket could not be converted

java.net.SocketException.SocketException could not be converted

java.net.SocketImpl could not be converted

java.net.SocketImplFactory could not be converted

java.net.SocketInputStream could not be converted

java.net.SocketOptions could not be converted

java.net.SocketOutputStream could not be converted

java.net.SocketPermission.SocketPermission could not be converted

java.net.UnknownContentHandler could not be converted

java.net.URL.getContent could not be converted

java.net.URL.getContent(Class[]) could not be converted

java.net.URL.getPort could not be converted

java.net.URL.openStream could not be converted

java.net.URL.set could not be converted

java.net.URL.setURLStreamHandlerFactory could not be converted

java.net.URL.URL could not be converted

java.net.URLClassLoader could not be converted

java.net.URLConnection.allowUserInteraction could not be converted

java.net.URLConnection.connect could not be converted

java.net.URLConnection.dolInput could not be converted

java.net.URLConnection.doOutput could not be converted

java.net.URLConnection.fileNameMap could not be converted

java.net.URLConnection.getAllowUserInteraction could not be converted

java.net.URLConnection.getContent could not be converted

java.net.URLConnection.getContentEncoding could not be converted

java.net.URLConnection.getDate could not be converted

java.net.URLConnection.getDefaultAllowUserInteraction could not be converted

java.net.URLConnection.getDefaultRequestProperty could not be converted

java.net.URLConnection.getDefaultUseCaches could not be converted

java.net.URLConnection.getDoInput could not be converted

java.net.URLConnection.getDoOutput could not be converted

java.net.URLConnection.getFileNameMap could not be converted

java.net.URLConnection.getHeaderField could not be converted

java.net.URLConnection.getHeaderFieldKey could not be converted

java.net.URLConnection.getIfModifiedSince could not be converted

java.net.URLConnection.getLastModified could not be converted

java.net.URLConnection.getPermission could not be converted

java.net.URLConnection.getRequestProperty could not be converted

java.net.URLConnection.useCaches could not be converted

java.net.URLConnection.guessContentTypeFromName could not be converted

java.net.URLConnection.guessContentTypeFromStream could not be converted

java.net.URLConnection.setAllowUserInteraction could not be converted

java.net.URLConnection.setContentHandlerFactory could not be converted

java.net.URLConnection.setDefaultAllowUserInteraction could not be converted

java.net.URLConnection.setDefaultRequestProperty could not be converted

java.net.URLConnection.setDefaultUseCaches could not be converted

java.net.URLConnection.setDoInput could not be converted

java.net.URLConnection.setDoOutput could not be converted

java.net.URLConnection.setFileNameMap could not be converted

java.net.URLConnection.setUseCaches could not be converted

java.net.URLConnection.toString could not be converted

java.net.URLConnection.url could not be converted

java.net.URLConnection.useCaches could not be converted

java.net.URLDecoder could not be converted

java.net.URLEncoder could not be converted

java.net.URLStreamHandler could not be converted

java.net.URLStreamHandlerFactory could not be converted

Java.rmi Error Messages

java.rmi.AccessException could not be converted

java.rmi.activation.Activatable.Activatable could not be converted

java.rmi.activation.Activatable.exportObject could not be converted

java.rmi.activation.Activatable.getID could not be converted

java.rmi.activation.Activatable.inactive could not be converted

java.rmi.activation.Activatable.register could not be converted

java.rmi.activation.Activatable.unexportObject could not be converted

java.rmi.activation.Activatable.unregister could not be converted

java.rmi.activation.ActivateFailedException could not be converted

java.rmi.activation.ActivationDesc could not be converted

java.rmi.activation.ActivationException could not be converted

java.rmi.activation.ActivationGroup could not be converted

java.rmi.activation.ActivationGroup_Stub could not be converted

java.rmi.activation.ActivationGroupDesc could not be converted

java.rmi.activation.ActivationGroupDesc.CommandEnvironment could not be converted

java.rmi.activation.ActivationGroupID could not be converted

java.rmi.activation.ActivationID could not be converted

java.rmi.activation.ActivationInstantiator could not be converted

java.rmi.activation.ActivationMonitor could not be converted

java.rmi.activation.ActivationSystem could not be converted

java.rmi.activation.Activator.activate could not be converted

java.rmi.activation.CommandEnvironment.CommandEnvironment could not be converted

java.rmi.activation.CommandEnvironment.equals could not be converted

java.rmi.activation.CommandEnvironment.getCommandOptions could not be converted

java.rmi.activation.CommandEnvironment.getCommandPath could not be converted

java.rmi.activation.CommandEnvironment.hashCode could not be converted

java.rmi.activation.UnknownGroupException could not be converted

java.rmi.activation.UnknownObjectException could not be converted

java.rmi.AlreadyBoundException could not be converted

java.rmi.ConnectException could not be converted

java.rmi.ConnectIOException could not be converted

java.rmi.dgc.DGC could not be converted

java.rmi.dgc.Lease.getVMID could not be converted

java.rmi.dgc.VMID could not be converted

java.rmi.MarshalException could not be converted

java.rmi.MarshalledObject could not be converted

java.rmi.Naming could not be converted

java.rmi.Naming.bind could not be converted
java.rmi.Naming.lookup could not be converted
java.rmi.Naming.rebind could not be converted
java.rmi.Naming.unbind could not be converted
java.rmi.NoSuchObjectException could not be converted
java.rmi.NotBoundException could not be converted
java.rmi.registry.LocateRegistry could not be converted
java.rmi.registry.Registry could not be converted
java.rmi.registry.Registry.bind could not be converted
java.rmi.registry.Registry.lookup could not be converted
java.rmi.registry.Registry.rebind could not be converted
java.rmi.registry.Registry.REGISTRY_PORT could not be converted
java.rmi.registry.Registry.unbind could not be converted
java.rmi.registry.RegistryHandler could not be converted
java.rmi.RemoteException.detail could not be converted
java.rmi.RMISecurityException could not be converted
java.rmi.RMISecurityManager could not be converted
java.rmi.RMISecurityManager.RMISecurityManager could not be converted
java.rmi.server.ExportException could not be converted
java.rmi.server.LoaderHandler could not be converted
java.rmi.server.LogStream could not be converted
java.rmi.server.ObjID could not be converted
java.rmi.server.Operation could not be converted
java.rmi.server.RemoteCall could not be converted
java.rmi.server.RemoteRef could not be converted
java.rmi.server.RemoteServer could not be converted
java.rmi.server.RemoteStub could not be converted
java.rmi.server.RMIClassLoader could not be converted
java.rmi.server.RMIFailureHandler could not be converted
java.rmi.server.RMISocketFactory.getDefaultSocketFactory could not be converted
java.rmi.server.RMISocketFactory.getFailureHandler could not be converted
java.rmi.server.RMISocketFactory.getSocketFactory could not be converted
java.rmi.server.RMISocketFactory.setFailureHandler could not be converted
java.rmi.server.RMISocketFactory.setSocketFactory could not be converted
java.rmi.server.RemoteObject.getRef could not be converted
java.rmi.server.RemoteObject.ref could not be converted
java.rmi.server.RemoteObject.RemoteObject could not be converted
java.rmi.server.RemoteObject.toStub could not be converted
java.rmi.server.RemoteServer.getClientHost could not be converted
java.rmi.server.RemoteServer.getLog could not be converted

java.rmi.server.RemoteServer.RemoteServer could not be converted
java.rmi.server.ServerCloneException could not be converted
java.rmi.server.ServerNotActiveException could not be converted
java.rmi.server.ServerRef could not be converted
java.rmi.server.Skeleton could not be converted
java.rmi.server.SkeletonMismatchException could not be converted
java.rmi.server.SkeletonNotFoundException could not be converted
java.rmi.server.SocketSecurityException could not be converted
java.rmi.server.UID could not be converted
java.rmi.server.UnicastRemoteObject.clone could not be converted
java.rmi.server.UnicastRemoteObject.exportObject could not be converted
java.rmi.server.UnicastRemoteObject.unexportObject could not be converted
java.rmi.server.UnicastRemoteObject.UnicastRemoteObject could not be converted
java.rmi.server.Unreferenced could not be converted
java.rmi.ServerRuntimeException could not be converted
java.rmi.StubNotFoundException could not be converted
java.rmi.UnexpectedException could not be converted
java.rmi.UnknownHostException could not be converted
java.rmi.UnmarshalException could not be converted

Java.security Error Messages

java.security.AccessControlContext could not be converted

java.security.AccessControlException.getPermission could not be converted

java.security.AccessController could not be converted

java.security.acl could not be converted

java.security.acl.Acl could not be converted

java.security.acl.AclEntry could not be converted

java.security.acl.AclNotFoundException could not be converted

java.security.acl.Group could not be converted

java.security.acl.LastOwnerException could not be converted

java.security.acl.NotOwnerException could not be converted

java.security.acl.Owner could not be converted

java.security.acl.Permission could not be converted

java.security.AlgorithmParameterGenerator could not be converted

java.security.AlgorithmParameterGeneratorSpi could not be converted

java.security.AlgorithmParameters could not be converted

java.security.AlgorithmParametersSpi could not be converted

java.security.AllPermission could not be converted

java.security.BasicPermission.newPermissionCollection could not be converted

java.security.cert.Certificate.Certificate could not be converted

java.security.cert.Certificate.getEncoded could not be converted

java.security.cert.Certificate.verify(PublicKey) could not be converted

java.security.cert.Certificate.verify(PublicKey, String) could not be converted

java.security.cert.Certificate.writeReplace could not be converted

java.security.cert.CertificateFactory.CertificateFactory could not be converted

java.security.cert.CertificateFactory.generateCertificate could not be converted

java.security.cert.CertificateFactory.generateCertificates could not be converted

java.security.cert.CertificateFactory.generateCRL could not be converted

java.security.cert.CertificateFactory.generateCRLs could not be converted

java.security.cert.CertificateFactory.getInstance could not be converted

java.security.cert.CertificateFactory.getProvider could not be converted

java.security.cert.CertificateFactorySpi could not be converted

java.security.cert.CRL could not be converted

java.security.cert.X509Certificate.getBasicConstraints could not be converted

java.security.cert.X509Certificate.getCriticalExtensionOIDs could not be converted

java.security.cert.X509Certificate.getExtensionValue could not be converted

java.security.cert.X509Certificate.getIssuerUniqueID could not be converted

java.security.cert.X509Certificate.getKeyUsage could not be converted

java.security.cert.X509Certificate.getNonCriticalExtensionOIDs could not be converted

java.security.cert.X509Certificate.getNotAfter could not be converted

java.security.cert.X509Certificate.getNotBefore could not be converted

java.security.cert.X509CRLEntry.getRevocationDate could not be converted

java.security.cert.X509Certificate.getSigAlgName could not be converted

java.security.cert.X509Certificate.getSignature could not be converted

java.security.cert.X509Certificate.getSubjectUniqueID could not be converted

java.security.cert.X509Certificate.getTBSCertificate could not be converted

java.security.cert.X509Certificate.getVersion could not be converted

java.security.cert.X509Certificate.hasUnsupportedCriticalExtension could not be converted

java.security.cert.X509CRL could not be converted

java.security.cert.X509CRLEntry.getCriticalExtensionOIDs could not be converted

java.security.cert.X509CRLEntry.getEncoded could not be converted

java.security.cert.X509CRLEntry.getExtensionValue could not be converted

java.security.cert.X509CRLEntry.getNonCriticalExtensionOIDs could not be converted

java.security.cert.X509CRLEntry.hasExtensions could not be converted

java.security.cert.X509CRLEntry.hasUnsupportedCriticalExtension could not be converted

java.security.cert.X509Extension could not be converted

java.security.Certificate could not be converted

java.security.CodeSource could not be converted

java.security.DomainCombiner could not be converted

java.security.Guard could not be converted

java.security.GuardedObject could not be converted

java.security.Identity could not be converted

java.security.IdentityScope could not be converted

java.security.interfaces.DSAKeyPairGenerator could not be converted

java.security.interfaces.DSAParams.getG could not be converted

java.security.interfaces.DSAParams.getP could not be converted

java.security.interfaces.DSAParams.getQ could not be converted

java.security.interfaces.DSAPrivateKey could not be converted

java.security.interfaces.DSAPublicKey could not be converted

java.security.interfaces.RSAPrivateCrtKey could not be converted

java.security.interfaces.RSAPrivateKey could not be converted

java.security.interfaces.RSAPublicKey could not be converted

java.security.Key.getFormat could not be converted

java.security.KeyFactory could not be converted

java.security.KeyFactorySpi could not be converted

java.security.KeyPairGenerator could not be converted

java.security.KeyPairGeneratorSpi could not be converted

java.security.KeyStore could not be converted

java.security.KeyStoreSpi could not be converted

java.security.MessageDigest.clone could not be converted

java.security.MessageDigest.getInstance could not be converted

java.security.MessageDigestSpi.clone could not be converted

java.security.MessageDigestSpi.MessageDigestSpi could not be converted

java.security.Permission.checkGuard could not be converted

java.security.Permission.getName could not be converted

java.security.PermissionCollection.implies could not be converted

java.security.PermissionCollection.setReadOnly could not be converted

java.security.Policy could not be converted

java.security.ProtectionDomain could not be converted

java.security.Provider could not be converted

java.security.SecureClassLoader could not be converted

java.security.SecureRandom.getProvider could not be converted

java.security.SecureRandom.next could not be converted

java.security.SecureRandomSpi.SecureRandomSpi could not be converted

java.security.Security could not be converted

java.security.SecurityPermission.SecurityPermission could not be converted

java.security.Signature.clone could not be converted

java.security.Signature.engineGetParameter could not be converted

java.security.Signature.engineInitSign could not be converted

java.security.Signature.engineInitVerify could not be converted

java.security.Signature.engineSetParameter could not be converted

java.security.Signature.engineSign could not be converted

java.security.Signature.engineVerify could not be converted

java.security.Signature.getInstance could not be converted

java.security.Signature.getParameter could not be converted

java.security.Signature.initSign could not be converted

java.security.Signature.initVerify could not be converted

java.security.Signature.setParameter could not be converted

java.security.Signature.sign could not be converted

java.security.Signature.SIGN~ could not be converted

java.security.Signature.state could not be converted

java.security.Signature.UNINITIALIZED could not be converted

java.security.Signature.verify could not be converted

java.security.Signature.VERIFY~ could not be converted

java.security.SignatureSpi.appRandom could not be converted

java.security.SignatureSpi.clone could not be converted

java.security.SignatureSpi.engineGetParameter could not be converted

java.security.SignatureSpi.engineInitSign could not be converted

java.security.SignatureSpi.engineSetParameter could not be converted
java.security.SignatureSpi.engineSign could not be converted
java.security.SignatureSpi.engineSign(byte[], int, int) could not be converted
java.security.SignatureSpi.engineVerify could not be converted
java.security.SecureRandomSpi.SecureRandomSpi could not be converted
java.security.SignedObject could not be converted
java.security.Signer could not be converted
java.security.spec.DSAPrivateKeySpec.DSAPrivateKeySpec could not be converted
java.security.spec.DSAPrivateKeySpec.DSAPrivateKeySpec could not be converted
java.security.spec.DSAPublicKeySpec.DSAPublicKeySpec could not be converted
java.security.spec.EncodedKeySpec could not be converted
java.security.spec.PKCS8EncodedKeySpec could not be converted
java.security.spec.RSAKeyGenParameterSpec could not be converted
java.security.spec.RSAPrivateCrtKeySpec could not be converted
java.security.spec.RSAPrivateKeySpec.RSAPrivateKeySpec could not be converted
java.security.spec.RSAPublicKeySpec.RSAPublicKeySpec could not be converted
java.security.spec.X509EncodedKeySpec could not be converted
java.security.UnresolvedPermission could not be converted

Java.sql Error Messages

java.sql.Array could not be converted

java.sql.BatchUpdateException could not be converted

java.sql.Blob could not be converted

java.sql.Blob.getBinaryStream could not be converted

java.sql.Blob.getBytes could not be converted

java.sql.Blob.position could not be converted

java.sql.CallableStatement.getArray could not be converted

java.sql.CallableStatement.getBigDecimal could not be converted

java.sql.CallableStatement.getBlob could not be converted

java.sql.CallableStatement.getClob could not be converted

java.sql.CallableStatement.getDate could not be converted

java.sql.CallableStatement.getObject could not be converted

java.sql.CallableStatement.getRef could not be converted

java.sql.CallableStatement.getTime could not be converted

java.sql.CallableStatement.getTimestamp could not be converted

java.sql.CallableStatement.registerOutParameter could not be converted

java.sql.CallableStatement.wasNull could not be converted

java.sql.Clob could not be converted

java.sql.Clob.getAsciiStream could not be converted

java.sql.Clob.getCharacterStream could not be converted

java.sql.Clob.getSubString could not be converted

java.sql.Clob.position could not be converted

java.sql.Connection.clearWarnings could not be converted

java.sql.Connection.createStatement could not be converted

java.sql.Connection.getTypeMap could not be converted

java.sql.Connection.getWarnings could not be converted

java.sql.Connection.isReadOnly could not be converted

java.sql.Connection.nativeSQL could not be converted

java.sql.Connection.prepareCall could not be converted

java.sql.Connection.prepareStatement could not be converted

java.sql.Connection.setCatalog could not be converted

java.sql.Connection.setReadOnly could not be converted

java.sql.Connection.setTypeMap could not be converted

java.sql.DatabaseMetaData.allProceduresAreCallable could not be converted

java.sql.DatabaseMetaData.allTablesAreSelectable could not be converted

java.sql.DatabaseMetaData.bestRowNotPseudo could not be converted

java.sql.DatabaseMetaData.bestRowPseudo could not be converted

java.sql.DatabaseMetaData.bestRowSession could not be converted

java.sql.DatabaseMetaData.bestRowTemporary could not be converted

java.sql.DatabaseMetaData.bestRowTransaction could not be converted

java.sql.DatabaseMetaData.bestRowUnknown could not be converted

java.sql.DatabaseMetaData.columnNoNulls could not be converted

java.sql.DatabaseMetaData.columnNullable could not be converted

java.sql.DatabaseMetaData.columnNullableUnknown could not be converted

java.sql.DatabaseMetaData.dataDefinitionCausesTransactionCommit could not be converted

java.sql.DatabaseMetaData.dataDefinitionIgnoredInTransactions could not be converted

java.sql.DatabaseMetaData.deletesAreDetected could not be converted

java.sql.DatabaseMetaData.doesMaxRowSizeIncludeBlobs could not be converted

java.sql.DatabaseMetaData.getBestRowIdentifier could not be converted

java.sql.DatabaseMetaData.getCatalogTerm could not be converted

java.sql.DatabaseMetaData.getColumnPrivileges could not be converted

java.sql.DatabaseMetaData.getColumns could not be converted

java.sql.DatabaseMetaData.getCrossReference could not be converted

java.sql.DatabaseMetaData.getDatabaseProductName could not be converted

java.sql.DatabaseMetaData.getDriverMajorVersion could not be converted

java.sql.DatabaseMetaData.getDriverMinorVersion could not be converted

java.sql.DatabaseMetaData.getDriverName could not be converted

java.sql.DatabaseMetaData.getDriverVersion could not be converted

java.sql.DatabaseMetaData.getExportedKeys could not be converted

java.sql.DatabaseMetaData.getExtraNameCharacters could not be converted

java.sql.DatabaseMetaData.getIdentiferQuoteString could not be converted

java.sql.DatabaseMetaData.getImportedKeys could not be converted

java.sql.DatabaseMetaData.getIndexInfo could not be converted

java.sql.DatabaseMetaData.getMaxColumnsInGroupBy could not be converted

java.sql.DatabaseMetaData.getMaxColumnsInIndex could not be converted

java.sql.DatabaseMetaData.getMaxColumnsInOrderBy could not be converted

java.sql.DatabaseMetaData.getMaxColumnsInSelect could not be converted

java.sql.DatabaseMetaData.getMaxColumnsInTable could not be converted

java.sql.DatabaseMetaData.getMaxConnections could not be converted

java.sql.DatabaseMetaData.getMaxIndexLength could not be converted

java.sql.DatabaseMetaData.getMaxRowSize could not be converted

java.sql.DatabaseMetaData.getMaxStatementLength could not be converted

java.sql.DatabaseMetaData.getMaxStatements could not be converted

java.sql.DatabaseMetaData.getMaxTablesInSelect could not be converted

java.sql.DatabaseMetaData.getNumericFunctions could not be converted

java.sql.DatabaseMetaData.getPrimaryKeys could not be converted

java.sql.DatabaseMetaData.getProcedureColumns could not be converted

java.sql.DatabaseMetaData.getProcedures could not be converted
java.sql.DatabaseMetaData.getProcedureTerm could not be converted
java.sql.DatabaseMetaData.getSchemaTerm could not be converted
java.sql.DatabaseMetaData.getSearchStringEscape could not be converted
java.sql.DatabaseMetaData.getSQLKeywords could not be converted
java.sql.DatabaseMetaData.getStringFunctions could not be converted
java.sql.DatabaseMetaData.getSystemFunctions could not be converted
java.sql.DatabaseMetaData.getTablePrivileges could not be converted
java.sql.DatabaseMetaData.getTables could not be converted
java.sql.DatabaseMetaData.getTimeDateFunctions could not be converted
java.sql.DatabaseMetaData.getUDTs could not be converted
java.sql.DatabaseMetaData.getURL could not be converted
java.sql.DatabaseMetaData.getUserName could not be converted
java.sql.DatabaseMetaData.getVersionColumns could not be converted
java.sql.DatabaseMetaData.importedKeyCascade could not be converted
java.sql.DatabaseMetaData.importedKeyInitiallyDeferred could not be converted
java.sql.DatabaseMetaData.importedKeyInitiallyImmediate could not be converted
java.sql.DatabaseMetaData.importedKeyNoAction could not be converted
java.sql.DatabaseMetaData.importedKeyNotDeferrable could not be converted
java.sql.DatabaseMetaData.importedKeyRestrict could not be converted
java.sql.DatabaseMetaData.importedKeySetDefault could not be converted
java.sql.DatabaseMetaData.importedKeySetNull could not be converted
java.sql.DatabaseMetaData.insertsAreDetected could not be converted
java.sql.DatabaseMetaData.isCatalogAtStart could not be converted
java.sql.DatabaseMetaData.isReadOnly could not be converted
java.sql.DatabaseMetaData.nullPlusNonNullsNull could not be converted
java.sql.DatabaseMetaData.nullsAreSortedAtEnd could not be converted
java.sql.DatabaseMetaData.nullsAreSortedAtStart could not be converted
java.sql.DatabaseMetaData.nullsAreSortedHigh could not be converted
java.sql.DatabaseMetaData.nullsAreSortedLow could not be converted
java.sql.DatabaseMetaData.othersDeletesAreVisible could not be converted
java.sql.DatabaseMetaData.othersInsertsAreVisible could not be converted
java.sql.DatabaseMetaData.othersUpdatesAreVisible could not be converted
java.sql.DatabaseMetaData.ownDeletesAreVisible could not be converted
java.sql.DatabaseMetaData.ownInsertsAreVisible could not be converted
java.sql.DatabaseMetaData.ownUpdatesAreVisible could not be converted
java.sql.DatabaseMetaData.procedureColumnIn could not be converted
java.sql.DatabaseMetaData.procedureColumnInOut could not be converted
java.sql.DatabaseMetaData.procedureColumnOut could not be converted
java.sql.DatabaseMetaData.procedureColumnResult could not be converted

java.sql.DatabaseMetaData.procedureColumnReturn could not be converted

java.sql.DatabaseMetaData.procedureColumnUnknown could not be converted

java.sql.DatabaseMetaData.procedureNoNulls could not be converted

java.sql.DatabaseMetaData.procedureNoResult could not be converted

java.sql.DatabaseMetaData.procedureNullable could not be converted

java.sql.DatabaseMetaData.procedureNullableUnknown could not be converted

java.sql.DatabaseMetaData.procedureResultUnknown could not be converted

java.sql.DatabaseMetaData.procedureReturnsResult could not be converted

java.sql.DatabaseMetaData.storesLowerCaseIdentifiers could not be converted

java.sql.DatabaseMetaData.storesLowerCaseQuotedIdentifiers could not be converted

java.sql.DatabaseMetaData.storesMixedCaseIdentifiers could not be converted

java.sql.DatabaseMetaData.storesMixedCaseQuotedIdentifiers could not be converted

java.sql.DatabaseMetaData.storesUpperCaseIdentifiers could not be converted

java.sql.DatabaseMetaData.storesUpperCaseQuotedIdentifiers could not be converted

java.sql.DatabaseMetaData.supportsAlterTableWithAddColumn could not be converted

java.sql.DatabaseMetaData.supportsAlterTableWithDropColumn could not be converted

java.sql.DatabaseMetaData.supportsANSI92EntryLevelSQL could not be converted

java.sql.DatabaseMetaData.supportsANSI92FullSQL could not be converted

java.sql.DatabaseMetaData.supportsANSI92IntermediateSQL could not be converted

java.sql.DatabaseMetaData.supportsBatchUpdates could not be converted

java.sql.DatabaseMetaData.supportsCatalogsInDataManipulation could not be converted

java.sql.DatabaseMetaData.supportsCatalogsInIndexDefinitions could not be converted

java.sql.DatabaseMetaData.supportsCatalogsInPrivilegeDefinitions could not be converted

java.sql.DatabaseMetaData.supportsCatalogsInProcedureCalls could not be converted

java.sql.DatabaseMetaData.supportsCatalogsInTableDefinitions could not be converted

java.sql.DatabaseMetaData.supportsColumnAliasing could not be converted

java.sql.DatabaseMetaData.supportsConvert could not be converted

java.sql.DatabaseMetaData.supportsCoreSQLGrammar could not be converted

java.sql.DatabaseMetaData.supportsCorrelatedSubqueries could not be converted

java.sql.DatabaseMetaData.supportsDataDefinitionAndDataManipulationTransactions could not be converted

java.sql.DatabaseMetaData.supportsDataManipulationTransactionsOnly could not be converted

java.sql.DatabaseMetaData.supportsDifferentTableCorrelationNames could not be converted

java.sql.DatabaseMetaData.supportsExpressionsInOrderBy could not be converted

java.sql.DatabaseMetaData.supportsExtendedSQLGrammar could not be converted

java.sql.DatabaseMetaData.supportsFullOuterJoins could not be converted

java.sql.DatabaseMetaData.supportsGroupBy could not be converted

java.sql.DatabaseMetaData.supportsGroupByBeyondSelect could not be converted

java.sql.DatabaseMetaData.supportsGroupByUnrelated could not be converted

java.sql.DatabaseMetaData.supportsIntegrityEnhancementFacility could not be converted

java.sql.DatabaseMetaData.supportsLikeEscapeClause could not be converted

java.sql.DatabaseMetaData.supportsLimitedOuterJoins could not be converted

java.sql.DatabaseMetaData.supportsMinimumSQLGrammar could not be converted

java.sql.DatabaseMetaData.supportsMixedCaseIdentifiers could not be converted

java.sql.DatabaseMetaData.supportsMixedCaseQuotedIdentifiers could not be converted

java.sql.DatabaseMetaData.supportsMultipleResultSets could not be converted

java.sql.DatabaseMetaData.supportsMultipleTransactions could not be converted

java.sql.DatabaseMetaData.supportsNonNullableColumns could not be converted

java.sql.DatabaseMetaData.supportsOpenCursorsAcrossCommit could not be converted

java.sql.DatabaseMetaData.supportsOpenCursorsAcrossRollback could not be converted

java.sql.DatabaseMetaData.supportsOpenStatementsAcrossCommit could not be converted

java.sql.DatabaseMetaData.supportsOpenStatementsAcrossRollback could not be converted

java.sql.DatabaseMetaData.supportsOrderByUnrelated could not be converted

java.sql.DatabaseMetaData.supportsOuterJoins could not be converted

java.sql.DatabaseMetaData.supportsPositionedDelete could not be converted

java.sql.DatabaseMetaData.supportsPositionedUpdate could not be converted

java.sql.DatabaseMetaData.supportsResultSetConcurrency could not be converted

java.sql.DatabaseMetaData.supportsResultSetType could not be converted

java.sql.DatabaseMetaData.supportsSchemasInDataManipulation could not be converted

java.sql.DatabaseMetaData.supportsSchemasInIndexDefinitions could not be converted

java.sql.DatabaseMetaData.supportsSchemasInPrivilegeDefinitions could not be converted

java.sql.DatabaseMetaData.supportsSchemasInProcedureCalls could not be converted

java.sql.DatabaseMetaData.supportsSchemasInTableDefinitions could not be converted

java.sql.DatabaseMetaData.supportsSelectForUpdate could not be converted

java.sql.DatabaseMetaData.supportsStoredProcedures could not be converted

java.sql.DatabaseMetaData.supportsSubqueriesInComparisons could not be converted

java.sql.DatabaseMetaData.supportsSubqueriesInExists could not be converted

java.sql.DatabaseMetaData.supportsSubqueriesInIns could not be converted

java.sql.DatabaseMetaData.supportsSubqueriesInQuantifieds could not be converted

java.sql.DatabaseMetaData.supportsTableCorrelationNames could not be converted

java.sql.DatabaseMetaData.supportsTransactionIsolationLevel could not be converted

java.sql.DatabaseMetaData.supportsTransactions could not be converted

java.sql.DatabaseMetaData.supportsUnion could not be converted

java.sql.DatabaseMetaData.supportsUnionAll could not be converted

java.sql.DatabaseMetaData.tableIndexClustered could not be converted

java.sql.DatabaseMetaData.tableIndexHashed could not be converted

java.sql.DatabaseMetaData.tableIndexOther could not be converted

java.sql.DatabaseMetaData.tableIndexStatistic could not be converted

java.sql.DatabaseMetaData.typeNoNulls could not be converted

java.sql.DatabaseMetaData.typeNullable could not be converted

java.sql.DatabaseMetaData.typeNullableUnknown could not be converted

java.sql.DatabaseMetaData.typePredBasic could not be converted
java.sql.DatabaseMetaData.typePredChar could not be converted
java.sql.DatabaseMetaData.typePredNone could not be converted
java.sql.DatabaseMetaData.typeSearchable could not be converted
java.sql.DatabaseMetaData.updatesAreDetected could not be converted
java.sql.DatabaseMetaData.usesLocalFilePerTable could not be converted
java.sql.DatabaseMetaData.usesLocalFiles could not be converted
java.sql.DatabaseMetaData.versionColumnNotPseudo could not be converted
java.sql.DatabaseMetaData.versionColumnPseudo could not be converted
java.sql.DatabaseMetaData.versionColumnUnknown could not be converted
java.sql.DataTruncation could not be converted
java.sql.Date.Date could not be converted
java.sql.Date.getHours could not be converted
java.sql.Date.getMinutes could not be converted
java.sql.Date.getSeconds could not be converted
java.sql.Date.setHours could not be converted
java.sql.Date.setMinutes could not be converted
java.sql.Date.setSeconds could not be converted
java.sql.DriverInfo could not be converted
java.sql.DriverManager could not be converted
java.sql.DriverPropertyInfo could not be converted
java.sql.PreparedStatement.addBatch could not be converted
java.sql.PreparedStatement.getMetaData could not be converted
java.sql.PreparedStatement.setArray could not be converted
java.sql.PreparedStatement.setBlob could not be converted
java.sql.PreparedStatement.setClob could not be converted
java.sql.PreparedStatement.setDate could not be converted
java.sql.PreparedStatement.setNull could not be converted
java.sql.PreparedStatement.setRef could not be converted
java.sql.PreparedStatement.setTime could not be converted
java.sql.PreparedStatement.setTimestamp could not be converted
java.sql.Ref could not be converted
java.sql.ResultSet could not be converted
java.sql.ResultSet.<ConcurrencyMode> could not be converted
java.sql.ResultSet.absolute could not be converted
java.sql.ResultSet.afterLast could not be converted
java.sql.ResultSet.beforeFirst could not be converted
java.sql.ResultSet.cancelRowUpdates could not be converted
java.sql.ResultSet.clearWarnings could not be converted
java.sql.ResultSet.close could not be converted

java.sql.ResultSet.deleteRow could not be converted
java.sql.ResultSet.findColumn could not be converted
java.sql.ResultSet.first could not be converted
java.sql.ResultSet.getArray could not be converted
java.sql.ResultSet.getAsciiStream could not be converted
java.sql.ResultSet.getBigDecimal could not be converted
java.sql.ResultSet.getBinaryStream could not be converted
java.sql.ResultSet.getBlob could not be converted
java.sql.ResultSet.getBoolean could not be converted
java.sql.ResultSet.getBytes could not be converted
java.sql.ResultSet.getBytes could not be converted
java.sql.ResultSet.getCharacterStream could not be converted
java.sql.ResultSet.getClob could not be converted
java.sql.ResultSet.getConcurrency could not be converted
java.sql.ResultSet.getCursorName could not be converted
java.sql.ResultSet.getDate could not be converted
java.sql.ResultSet.getDouble could not be converted
java.sql.ResultSet.getFetchDirection could not be converted
java.sql.ResultSet.getFetchSize could not be converted
java.sql.ResultSet.getFloat could not be converted
java.sql.ResultSet.getInt could not be converted
java.sql.ResultSet.getLong could not be converted
java.sql.ResultSet.getMetaData could not be converted
java.sql.ResultSet.getObject could not be converted
java.sql.ResultSet.getRef could not be converted
java.sql.ResultSet.getRow could not be converted
java.sql.ResultSet.getShort could not be converted
java.sql.ResultSet.getStatement could not be converted
java.sql.ResultSet.getString could not be converted
java.sql.ResultSet.getTime could not be converted
java.sql.ResultSet.getTimestamp could not be converted
java.sql.ResultSet.getType could not be converted
java.sql.ResultSet.getUnicodeStream could not be converted
java.sql.ResultSet.getWarnings could not be converted
java.sql.ResultSet.insertRow could not be converted
java.sql.ResultSet.isAfterLast could not be converted
java.sql.ResultSet.isBeforeFirst could not be converted
java.sql.ResultSet.isFirst could not be converted
java.sql.ResultSet.isLast could not be converted
java.sql.ResultSet.last could not be converted

java.sql.ResultSet.moveToCurrentRow could not be converted
java.sql.ResultSet.moveToInsertRow could not be converted
java.sql.ResultSet.next could not be converted
java.sql.ResultSet.previous could not be converted
java.sql.ResultSet.refreshRow could not be converted
java.sql.ResultSet.relative could not be converted
java.sql.ResultSet.rowDeleted could not be converted
java.sql.ResultSet.rowInserted could not be converted
java.sql.ResultSet.rowUpdated could not be converted
java.sql.ResultSet.setFetchDirection could not be converted
java.sql.ResultSet.setFetchSize could not be converted
java.sql.ResultSet.TYPE_FORWARD_ONLY could not be converted
java.sql.ResultSet.updateAsciiStream could not be converted
java.sql.ResultSet.updateBigDecimal could not be converted
java.sql.ResultSet.updateBinaryStream could not be converted
java.sql.ResultSet.updateBoolean could not be converted
java.sql.ResultSet.updateByte could not be converted
java.sql.ResultSet.updateBytes could not be converted
java.sql.ResultSet.updateCharacterStream could not be converted
java.sql.ResultSet.updateDate could not be converted
java.sql.ResultSet.updateDouble could not be converted
java.sql.ResultSet.updateFloat could not be converted
java.sql.ResultSet.updateInt could not be converted
java.sql.ResultSet.updateLong could not be converted
java.sql.ResultSet.updateNull could not be converted
java.sql.ResultSet.updateObject could not be converted
java.sql.ResultSet.updateRow could not be converted
java.sql.ResultSet.updateShort could not be converted
java.sql.ResultSet.updateString could not be converted
java.sql.ResultSet.updateTime could not be converted
java.sql.ResultSet.updateTimestamp could not be converted
java.sql.ResultSet.wasNull could not be converted
java.sql.ResultSetMetaData.columnNoNulls could not be converted
java.sql.ResultSetMetaData.columnNullable could not be converted
java.sql.ResultSetMetaData.columnNullableUnknown could not be converted
java.sql.ResultSetMetaData.getPrecision could not be converted
java.sql.ResultSetMetaData.isCaseSensitive could not be converted
java.sql.ResultSetMetaData.isCurrency could not be converted
java.sql.ResultSetMetaData.isDefinitelyWritable could not be converted
java.sql.ResultSetMetaData.isSearchable could not be converted

java.sql.ResultSetMetaData.isSigned could not be converted
java.sql.ResultSetMetaData.isWritable could not be converted
javax.sql.RowSetMetaData.setNullable could not be converted
java.sql.SQLData could not be converted
java.sql.SQLException.getNextException could not be converted
java.sql.SQLException.setNextException could not be converted
java.sql.SQLException.SQLException could not be converted
java.sql.SQLInput could not be converted
java.sql.SQLOutput could not be converted
java.sql.SQLPermission could not be converted
java.sql.SQLWarning could not be converted
java.sql.Statement.clearWarnings could not be converted
java.sql.Statement.close could not be converted
java.sql.Statement.execute could not be converted
java.sql.Statement.executeBatch could not be converted
java.sql.Statement.getFetchDirection could not be converted
java.sql.Statement.getFetchSize could not be converted
java.sql.Statement.getMaxFieldSize could not be converted
java.sql.Statement.getMaxRows could not be converted
java.sql.Statement.getMoreResults could not be converted
java.sql.Statement.getResultSet could not be converted
java.sql.Statement.getResultSetConcurrency could not be converted
java.sql.Statement.getResultSetType could not be converted
java.sql.Statement.getUpdateCount could not be converted
java.sql.Statement.getWarnings could not be converted
java.sql.Statement.setCursorName could not be converted
java.sql.Statement.setEscapeProcessing could not be converted
java.sql.Statement.setFetchDirection could not be converted
java.sql.Statement.setFetchSize could not be converted
java.sql.Statement.setMaxFieldSize could not be converted
java.sql.Statement.setMaxRows could not be converted
java.sql.Struct could not be converted
java.sql.Timestamp.getNanos could not be converted
java.sql.Timestamp.setNanos could not be converted
java.sql.Timestamp.Timestamp could not be converted
java.sql.Types.<TypeName> could not be converted

Java.text Error Messages

java.text.Annotation could not be converted

java.text.AttributedCharacterIterator could not be converted

java.text.AttributedCharacterIterator.Attribute could not be converted

java.text.AttributedString could not be converted

java.text.BreakDictionary could not be converted

java.text.BreakIterator could not be converted

java.text.CharacterIterator.clone could not be converted

java.text.ChoiceFormat could not be converted

java.text.CollationElementIterator could not be converted

java.text.CollationKey.compareTo could not be converted

java.text.Collator.CANONICAL_DECOMPOSITION could not be converted

java.text.Collator.Collator could not be converted

java.text.Collator.compare could not be converted

java.text.Collator.FULL_DECOMPOSITION could not be converted

java.text.Collator.getDecomposition could not be converted

java.text.Collator.getStrength could not be converted

java.text.Collator.IDENTICAL could not be converted

java.text.Collator.NO_DECOMPOSITION could not be converted

java.text.Collator.PRIMARY could not be converted

java.text.Collator.SECONDARY could not be converted

java.text.Collator.setDecomposition could not be converted

java.text.Collator.setStrength could not be converted

java.text.Collator.TERTIARY could not be converted

java.text.DateFormat.format could not be converted

java.text.DateFormat.getNumberFormat could not be converted

java.text.DateFormat.getTimeZone could not be converted

java.text.DateFormat.isLenient could not be converted

java.text.DateFormat.numberFormat could not be converted

java.text.DateFormat.parse could not be converted

java.text.DateFormat.parseObject could not be converted

java.text.DateFormat.setLenient could not be converted

java.text.DateFormat.setNumberFormat could not be converted

java.text.DateFormat.setTimeZone could not be converted

java.text.DateFormatSymbols.DateFormatSymbols could not be converted

java.text.DateFormatSymbols.getEras could not be converted

java.text.DateFormatSymbols.getLocalPatternChars could not be converted

java.text.DateFormatSymbols.getZoneStrings could not be converted

java.text.DateFormatSymbols.setEras could not be converted

java.text.DateFormatSymbols.setLocalPatternChars could not be converted

java.text.DateFormatSymbols.setZoneStrings could not be converted

java.text.DecimalFormat could not be converted

java.text.DecimalFormatSymbols.DecimalFormatSymbols could not be converted

java.text.DecimalFormatSymbols.getDigit could not be converted

java.text.DecimalFormatSymbols.getInternationalCurrencySymbol could not be converted

java.text.DecimalFormatSymbols.getPatternSeparator could not be converted

java.text.DecimalFormatSymbols.getZeroDigit could not be converted

java.text.DecimalFormatSymbols.setDigit could not be converted

java.text.DecimalFormatSymbols.setInternationalCurrencySymbol could not be converted

java.text.DecimalFormatSymbols.setPatternSeparator could not be converted

java.text.DecimalFormatSymbols.setZeroDigit could not be converted

java.text.DictionaryBasedBreakIterator could not be converted

java.text.FieldPosition could not be converted

java.text.Format could not be converted

java.text.Format.format could not be converted

java.text.MessageFormat could not be converted

java.text.NumberFormat.format could not be converted

java.text.NumberFormat.FRACTION_FIELD could not be converted

java.text.NumberFormat.INTEGER_FIELD could not be converted

java.text.NumberFormat.isParseIntegerOnly could not be converted

java.text.NumberFormat.parse could not be converted

java.text.NumberFormat.parseObject could not be converted

java.text.NumberFormat.setParseIntegerOnly could not be converted

java.text.ParseException could not be converted

java.text.ParseException.getErrorOffset could not be converted

java.text.ParsePosition.getErrorIndex could not be converted

java.text.ParsePosition.setErrorIndex could not be converted

java.text.ParsePosition.toString could not be converted

java.text.RuleBasedBreakIterator could not be converted

java.text.RuleBasedCollator could not be converted

java.text.SimpleDateFormat could not be converted

java.text.StringCharacterIterator.clone could not be converted

java.text.StringCharacterIterator.setText could not be converted

Java.text.resources Error Messages

java.text.resources.BreakIteratorRules.BreakIteratorRules could not be converted

java.text.resources.BreakIteratorRules.getContents could not be converted

java.text.resources.DateFormatZoneData.DateFormatZoneData could not be converted

java.text.resources.DateFormatZoneData.getContents could not be converted

java.text.resources.DateFormatZoneData.getKeys could not be converted

java.text.resources.DateFormatZoneData_ar could not be converted

java.text.resources.DateFormatZoneData_ar.DateFormatZoneData_ar could not be converted

java.text.resources.DateFormatZoneData_ar.getContents could not be converted

java.text.resources.DateFormatZoneData_be could not be converted

java.text.resources.DateFormatZoneData_be.DateFormatZoneData_be could not be converted

java.text.resources.DateFormatZoneData_be.getContents could not be converted

java.text.resources.DateFormatZoneData_bg could not be converted

java.text.resources.DateFormatZoneData_bg.DateFormatZoneData_bg could not be converted

java.text.resources.DateFormatZoneData_bg.getContents could not be converted

java.text.resources.DateFormatZoneData_ca could not be converted

java.text.resources.DateFormatZoneData_ca.DateFormatZoneData_ca could not be converted

java.text.resources.DateFormatZoneData_ca.getContents could not be converted

java.text.resources.DateFormatZoneData_cs.DateFormatZoneData_cs could not be converted

java.text.resources.DateFormatZoneData_cs.getContents could not be converted

java.text.resources.DateFormatZoneData_da.DateFormatZoneData_da could not be converted

java.text.resources.DateFormatZoneData_da.getContents could not be converted

java.text.resources.DateFormatZoneData_de could not be converted

java.text.resources.DateFormatZoneData_de.DateFormatZoneData_de could not be converted

java.text.resources.DateFormatZoneData_de.getContents could not be converted

java.text.resources.DateFormatZoneData_de_AT could not be converted

java.text.resources.DateFormatZoneData_de_AT.DateFormatZoneData_de_AT could not be converted

java.text.resources.DateFormatZoneData_de_AT.getContents could not be converted

java.text.resources.DateFormatZoneData_de_CH could not be converted

java.text.resources.DateFormatZoneData_de_CH.DateFormatZoneData_de_CH could not be converted

java.text.resources.DateFormatZoneData_de_CH.getContents could not be converted

java.text.resources.DateFormatZoneData_el could not be converted

java.text.resources.DateFormatZoneData_el.DateFormatZoneData_el could not be converted

java.text.resources.DateFormatZoneData_el.getContents could not be converted

java.text.resources.DateFormatZoneData_en could not be converted

java.text.resources.DateFormatZoneData_en.DateFormatZoneData_en could not be converted

java.text.resources.DateFormatZoneData_en.getContents could not be converted

java.text.resources.DateFormatZoneData_en_CA could not be converted

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

java.text.resources.LocaleElements_sr_YU.LocaleElements_sr_YU could not be converted

java.text.resources.LocaleElements_sv could not be converted

java.text.resources.LocaleElements_sv.LocaleElements_sv could not be converted

java.text.resources.LocaleElements_sv_SE could not be converted

java.text.resources.LocaleElements_sv_SE.getContents could not be converted

java.text.resources.LocaleElements_sv_SE.LocaleElements_sv_SE could not be converted

java.text.resources.LocaleElements_th could not be converted

java.text.resources.LocaleElements_th.getContents could not be converted

java.text.resources.LocaleElements_th.LocaleElements_th could not be converted

java.text.resources.LocaleElements_th_TH could not be converted

java.text.resources.LocaleElements_th_TH.getContents could not be converted

java.text.resources.LocaleElements_th_TH.LocaleElements_th_TH could not be converted

java.text.resources.LocaleElements_tr could not be converted

java.text.resources.LocaleElements_tr.LocaleElements_tr could not be converted

java.text.resources.LocaleElements_tr_TR could not be converted

java.text.resources.LocaleElements_tr_TR.getContents could not be converted

java.text.resources.LocaleElements_tr_TR.LocaleElements_tr_TR could not be converted

java.text.resources.LocaleElements_uk could not be converted

java.text.resources.LocaleElements_uk.LocaleElements_uk could not be converted

java.text.resources.LocaleElements_uk_UA could not be converted

java.text.resources.LocaleElements_uk_UA.getContents could not be converted

java.text.resources.LocaleElements_uk_UA.LocaleElements_uk_UA could not be converted

java.text.resources.LocaleElements_zh could not be converted

java.text.resources.LocaleElements_zh.LocaleElements_zh could not be converted

java.text.resources.LocaleElements_zh_CN could not be converted

java.text.resources.LocaleElements_zh_CN.getContents could not be converted

java.text.resources.LocaleElements_zh_CN.LocaleElements_zh_CN could not be converted

java.text.resources.LocaleElements_zh_HK could not be converted

java.text.resources.LocaleElements_zh_HK.getContents could not be converted

java.text.resources.LocaleElements_zh_HK.LocaleElements_zh_HK could not be converted

java.text.resources.LocaleElements_zh_SG could not be converted

java.text.resources.LocaleElements_zh_SG.LocaleElements_zh_SG could not be converted

java.text.resources.LocaleElements_zh_TW could not be converted

java.text.resources.LocaleElements_zh_TW.LocaleElements_zh_TW could not be converted

Java.util Error Messages

java.util.AbstractCollection.AbstractCollection could not be converted

java.util.AbstractCollection.add could not be converted

java.util.AbstractCollection.addAll could not be converted

java.util.AbstractCollection.contains could not be converted

java.util.AbstractCollection.containsAll could not be converted

java.util.AbstractCollection.iterator could not be converted

java.util.AbstractCollection.remove could not be converted

java.util.AbstractCollection.removeAll could not be converted

java.util.AbstractCollection.retainAll could not be converted

java.util.AbstractCollection.toArray could not be converted

java.util.AbstractCollection.toArray(Object[]) could not be converted

java.util.AbstractList.AbstractList could not be converted

java.util.AbstractList.add could not be converted

java.util.AbstractList.addAll could not be converted

java.util.AbstractList.lastIndexOf could not be converted

java.util.AbstractList.listIterator could not be converted

java.util.AbstractList.listIterator(int) could not be converted

java.util.AbstractList.modCount could not be converted

java.util.AbstractList.remove could not be converted

java.util.AbstractList.removeRange could not be converted

java.util.AbstractList.subList could not be converted

java.util.AbstractMap could not be converted

java.util.AbstractMap.AbstractMap could not be converted

java.util.AbstractMap.clear could not be converted

java.util.AbstractMap.containsKey could not be converted

java.util.AbstractMap.containsValue could not be converted

java.util.AbstractMap.entrySet could not be converted

java.util.AbstractMap.equals could not be converted

java.util.AbstractMap.get could not be converted

java.util.AbstractMap.hashCode could not be converted

java.util.AbstractMap.keySet could not be converted

java.util.AbstractMap.put could not be converted

java.util.AbstractMap.putAll could not be converted

java.util.AbstractMap.remove could not be converted

java.util.AbstractMap.size could not be converted

java.util.AbstractMap.values could not be converted

java.util.AbstractSequentialList.AbstractSequentialList could not be converted

java.util.AbstractSequentialList.addAll could not be converted
java.util.AbstractSequentialList.listIterator could not be converted
java.util.AbstractSequentialList.remove could not be converted
java.util.AbstractSequentialList.set could not be converted
java.util.AbstractSet.AbstractSet could not be converted
java.util.AbstractSet.removeAll could not be converted
java.util.ArrayList.add could not be converted
java.util.ArrayList.toArray could not be converted
java.util.Arrays.asList could not be converted
java.util.Arrays.sort could not be converted
java.util.BitSet.andNot could not be converted
java.util.BitSet.equals could not be converted
java.util.BitSet.length could not be converted
java.util.Calendar.add could not be converted
java.util.Calendar.after could not be converted
java.util.Calendar.AM could not be converted
java.util.Calendar.AM_PM could not be converted
java.util.Calendar.APRIL could not be converted
java.util.Calendar.areFieldsSet could not be converted
java.util.Calendar.AUGUST could not be converted
java.util.Calendar.before could not be converted
java.util.Calendar.Calendar could not be converted
java.util.Calendar.clear could not be converted
java.util.Calendar.clone could not be converted
java.util.Calendar.complete could not be converted
java.util.Calendar.computeFields could not be converted
java.util.Calendar.computeTime could not be converted
java.util.Calendar.DAY_OF_MONTH could not be converted
java.util.Calendar.DAY_OF_WEEK could not be converted
java.util.Calendar.DAY_OF_WEEK_IN_MONTH could not be converted
java.util.Calendar.DAY_OF_YEAR could not be converted
java.util.Calendar.DECEMBER could not be converted
java.util.Calendar.DST_OFFSET could not be converted
java.util.Calendar.ERA could not be converted
java.util.Calendar.FEBRUARY could not be converted
java.util.Calendar.FIELD_COUNT could not be converted
java.util.Calendar.fields could not be converted
java.util.Calendar.get could not be converted
java.util.Calendar.getActualMaximum could not be converted
java.util.Calendar.getActualMinimum could not be converted

java.util.Calendar.getAvailableLocales could not be converted
java.util.Calendar.getGreatestMinimum could not be converted
java.util.Calendar.getInstance could not be converted
java.util.Calendar.getLeastMaximum could not be converted
java.util.Calendar.getMaximum could not be converted
java.util.Calendar.getMinimalDaysInFirstWeek could not be converted
java.util.Calendar.getMinimum could not be converted
java.util.Calendar.getTimeInMillis could not be converted
java.util.Calendar.getTimeZone could not be converted
java.util.Calendar.HOUR could not be converted
java.util.Calendar.HOUR_OF_DAY could not be converted
java.util.Calendar.internalGet could not be converted
java.util.Calendar.isLenient could not be converted
java.util.Calendar.isSet could not be converted
java.util.Calendar.isSet~ could not be converted
java.util.Calendar.isTimeSet could not be converted
java.util.Calendar.JANUARY could not be converted
java.util.Calendar.JULY could not be converted
java.util.Calendar.JUNE could not be converted
java.util.Calendar.MARCH could not be converted
java.util.Calendar.MAY could not be converted
java.util.Calendar.NOVEMBER could not be converted
java.util.Calendar.OCTOBER could not be converted
java.util.Calendar.PM could not be converted
java.util.Calendar.roll could not be converted
java.util.Calendar.roll(int, boolean) could not be converted
java.util.Calendar.roll(int, int) could not be converted
java.util.Calendar.SEPTEMBER could not be converted
java.util.Calendar.setLenient could not be converted
java.util.Calendar.setMinimalDaysInFirstWeek could not be converted
java.util.Calendar.setTimeInMillis could not be converted
java.util.Calendar.setTimeZone could not be converted
java.util.Calendar.time could not be converted
java.util.Calendar.UNDECIMBER could not be converted
java.util.Calendar.WEEK_OF_MONTH could not be converted
java.util.Calendar.WEEK_OF_YEAR could not be converted
java.util.Calendar.ZONE_OFFSET could not be converted
java.util.Collection.add could not be converted
java.util.Collection.addAll could not be converted
java.util.Collection.clear could not be converted

java.util.Collection.contains could not be converted

java.util.Collection.containsAll could not be converted

java.util.Collection.remove could not be converted

java.util.Collection.removeAll could not be converted

java.util.Collection.retainAll could not be converted

java.util.Collection.toArray could not be converted

java.util.Collections could not be converted

java.util.Collections.EMPTY_MAP could not be converted

java.util.Collections.EMPTY_SET could not be converted

java.util.Collections.max(Collection) could not be converted

java.util.Collections.max(Collection, Comparator) could not be converted

java.util.Collections.min(Collection) could not be converted

java.util.Collections.min(Collection, Comparator) could not be converted

java.util.Collections.reverseOrder could not be converted

java.util.Collections.sort(Comparator) could not be converted

java.util.Collections.sort(List) could not be converted

java.util.Collections.synchronizedMap could not be converted

java.util.Collections.synchronizedSortedSet could not be converted

java.util.Collections.unmodifiableMap could not be converted

java.util.Collections.unmodifiableSortedMap could not be converted

java.util.Collections.unmodifiableSortedSet could not be converted

java.util.Date.clone could not be converted

java.util.Date.Date(int, int, int) could not be converted

java.util.Date.Date(int, int, int, int, int) could not be converted

java.util.Date.Date(int, int, int, int, int, int) could not be converted

java.util.Date.Date(long) could not be converted

java.util.Date.Date(String) could not be converted

java.util.Date.getMonth could not be converted

java.util.Date.getTime could not be converted

java.util.Date.getTimezoneOffset could not be converted

java.util.Date.getYear could not be converted

java.util.Date.hashCode could not be converted

java.util.Date.parse could not be converted

java.util.Date.setTime could not be converted

java.util.Date.toGMTString could not be converted

java.util.Date.toLocaleString could not be converted

java.util.Date.toString could not be converted

java.util.Date.UTC could not be converted

java.util.Dictionary.Dictionary could not be converted

java.util.Dictionary.get could not be converted

java.util.Dictionary.isEmpty could not be converted

java.util.Dictionary.keys could not be converted

java.util.Dictionary.put could not be converted

java.util.Dictionary.remove could not be converted

java.util.Enumeration.hasMoreElements could not be converted

java.util.Enumeration.nextElement could not be converted

java.util.Entry.setValue could not be converted

java.util.EventListener could not be converted

java.util.EventObject could not be converted

java.util.EventObject.source could not be converted

java.util.GregorianCalendar.add could not be converted

java.util.GregorianCalendar.after could not be converted

java.util.GregorianCalendar.before could not be converted

java.util.GregorianCalendar.computeFields could not be converted

java.util.GregorianCalendar.computeTime could not be converted

java.util.GregorianCalendar.getActualMaximum could not be converted

java.util.GregorianCalendar.getActualMinimum could not be converted

java.util.GregorianCalendar.getGreatestMinimum could not be converted

java.util.GregorianCalendar.getGregorianChange could not be converted

java.util.GregorianCalendar.getLeastMaximum could not be converted

java.util.GregorianCalendar.getMaximum could not be converted

java.util.GregorianCalendar.getMinimum could not be converted

java.util.GregorianCalendar.GregorianCalendar could not be converted

java.util.GregorianCalendar.roll could not be converted

java.util.GregorianCalendar.roll(int, boolean) could not be converted

java.util.GregorianCalendar.setGregorianChange could not be converted

java.util.HashMap could not be converted

java.util.HashMap.entrySet could not be converted

java.util.HashMap.get could not be converted

java.util.HashMap.keySet could not be converted

java.util.HashMap.putAll could not be converted

java.util.HashSet could not be converted

java.util.HashSet.add could not be converted

java.util.HashSet.clear could not be converted

java.util.HashSet.clone could not be converted

java.util.HashSet.contains could not be converted

java.util.HashSet.HashSet could not be converted

java.util.HashSet.HashSet(Collection) could not be converted

java.util.HashSet.HashSet(int) could not be converted

java.util.HashSet.HashSet(int, float) could not be converted

java.util.HashSet.remove could not be converted
java.util.Hashtable.entrySet could not be converted
java.util.Hashtable.putAll could not be converted
java.util.Hashtable.rehash could not be converted
java.util.HashtableEntry could not be converted
java.util.HashtableEnumerator could not be converted
java.util.Iterator.hasNext could not be converted
java.util.Iterator.next could not be converted
java.util.Iterator.remove could not be converted
java.util.LinkedList could not be converted
java.util.LinkedList.add could not be converted
java.util.LinkedList.listIterator could not be converted
java.util.List.add could not be converted
java.util.List.addAll could not be converted
java.util.List.containsAll could not be converted
java.util.List.lastIndexOf could not be converted
java.util.List.listIterator could not be converted
java.util.List.remove could not be converted
java.util.List.removeAll could not be converted
java.util.List.retainAll could not be converted
java.util.List.subList could not be converted
java.util.List.toArray could not be converted
java.util.ListIterator.add could not be converted
java.util.ListIterator.hasNext could not be converted
java.util.ListIterator.hasPrevious could not be converted
java.util.ListIterator.next could not be converted
java.util.ListIterator.nextIndex could not be converted
java.util.ListIterator.previous could not be converted
java.util.ListIterator.previousIndex could not be converted
java.util.ListIterator.remove could not be converted
java.util.ListIterator.set could not be converted
java.util.ListResourceBundle.getContents could not be converted
java.util.ListResourceBundle.getKeys could not be converted
java.util.ListResourceBundle.handleGetObject could not be converted
java.util.ListResourceBundle.ListResourceBundle could not be converted
java.util.Locale.CHINESE could not be converted
java.util.Locale.getDefault could not be converted
java.util.Locale.getDisplayName could not be converted
java.util.Locale.getDisplayVariant could not be converted
java.util.Locale.getISOCountries could not be converted

java.util.Locale.getLanguage could not be converted
java.util.Locale.getVariant could not be converted
java.util.Locale.hashCode could not be converted
java.util.Locale.setDefault could not be converted
java.util.Map.containsKey could not be converted
java.util.Map.entrySet could not be converted
java.util.Map.get could not be converted
java.util.Map.isEmpty could not be converted
java.util.Map.keySet could not be converted
java.util.Map.putAll could not be converted
java.util.MissingResourceException.getClassName could not be converted
java.util.MissingResourceException.getKey could not be converted
java.util.MissingResourceException.MissingResourceException could not be converted
java.util.Properties.defaults could not be converted
java.util.Properties.list could not be converted
java.util.Properties.load could not be converted
java.util.Properties.save could not be converted
java.util.Properties.setProperty could not be converted
java.util.Properties.store could not be converted
java.util.PropertyPermission could not be converted
java.util.PropertyResourceBundle.getKeys could not be converted
java.util.PropertyResourceBundle.PropertyResourceBundle could not be converted
java.util.Random.nextLong could not be converted
java.util.Random.nextBoolean could not be converted
java.util.Random.nextGaussian could not be converted
java.util.ResourceBundle.getKeys could not be converted
java.util.ResourceBundle.getObject could not be converted
java.util.ResourceBundle.getString could not be converted
java.util.ResourceBundle.getStringArray could not be converted
java.util.ResourceBundle.handleGetObject could not be converted
java.util.ResourceBundle.parent could not be converted
java.util.ResourceBundle.ResourceBundle could not be converted
java.util.ResourceBundle.setParent could not be converted
java.util.Set.add could not be converted
java.util.Set.addAll could not be converted
java.util.Set.clear could not be converted
java.util.Set.contains could not be converted
java.util.Set.containsAll could not be converted
java.util.Set.remove could not be converted
java.util.Set.removeAll could not be converted

java.util.Set.retainAll could not be converted

java.util.Set.toArray could not be converted

java.util.Set.toArray(Object[]) could not be converted

java.util.SimpleTimeZone could not be converted

java.util.SimpleTimeZone.clone could not be converted

java.util.SimpleTimeZone.getRawOffset could not be converted

java.util.SimpleTimeZone.setEndRule could not be converted

java.util.SimpleTimeZone.setRawOffset could not be converted

java.util.SimpleTimeZone.setStartRule could not be converted

java.util.SimpleTimeZone.setStartYear could not be converted

java.util.SimpleTimeZone.SimpleTimeZone could not be converted

java.util.SortedMap could not be converted

java.util.SortedMap.comparator could not be converted

java.util.SortedMap.headMap could not be converted

java.util.SortedMap.subMap could not be converted

java.util.SortedMap.tailMap could not be converted

java.util.SortedSet.comparator could not be converted

java.util.Stack.addElement could not be converted

java.util.Stack.elements could not be converted

java.util.StringTokenizer.StringTokenizer could not be converted

java.util.SystemClassLoader could not be converted

java.util.Timer.cancel could not be converted

java.util.Timer.schedule(TimerTask, Date) could not be converted

java.util.Timer.schedule(TimerTask, Date, long) could not be converted

java.util.Timer.schedule(TimerTask, long) could not be converted

java.util.Timer.schedule(TimerTask, long, long) could not be converted

java.util.Timer.scheduleAtFixedRate(TimerTask, Date, long) could not be converted

java.util.Timer.scheduleAtFixedRate(TimerTask, long, long) could not be converted

java.util.Timer.Timer could not be converted

java.util.TimerTask could not be converted

java.util.TimeZone.clone could not be converted

java.util.TimeZone.getAvailableIDs could not be converted

java.util.TimeZone.getDisplayName(boolean, int) could not be converted

java.util.TimeZone.getDisplayName(boolean, int, Locale) could not be converted

java.util.TimeZone.getDisplayName(Locale) could not be converted

java.util.TimeZone.getRawOffset could not be converted

java.util.TimeZone.getTimeZone could not be converted

java.util.TimeZone.hasSameRules could not be converted

java.util.TimeZone.setDefault could not be converted

java.util.TimeZone.setID could not be converted

java.util.TimeZone.setRawOffset could not be converted
java.util.TimeZone.TimeZone could not be converted
java.util.TreeMap.comparator could not be converted
java.util.TreeMap.entrySet could not be converted
java.util.TreeMap.headMap could not be converted
java.util.TreeMap.keySet could not be converted
java.util.TreeMap.putAll could not be converted
java.util.TreeMap.subMap could not be converted
java.util.TreeMap.tailMap could not be converted
java.util.TreeMap.TreeMap could not be converted
java.util.TreeMap.TreeMap(Map) could not be converted
java.util.TreeMap.TreeMap(SortedMap) could not be converted
java.util.TreeMap.values could not be converted
java.util.TreeSet could not be converted
java.util.TreeSet.comparator could not be converted
java.util.TreeSet.TreeSet(Comparator) could not be converted
java.util.TreeSet.TreeSet(SortedMap) could not be converted
java.util.TreeSet.TreeSet(SortedSet) could not be converted
java.util.Vector.add could not be converted
java.util.Vector.capacityIncrement could not be converted
java.util.Vector.containsAll could not be converted
java.util.Vector.elementData could not be converted
java.util.Vector.removeAll could not be converted
java.util.Vector.retainAll could not be converted
java.util.Vector.toArray could not be converted
java.util.Vector.Vector could not be converted
java.util.VectorEnumerator could not be converted
java.util.WeakHashMap could not be converted
java.util.WeakHashMap.entrySet could not be converted
java.util.WeakHashMap.WeakHashMap could not be converted

Java.util.cab Error Messages

[com.ms.util.cab.CabConstants could not be converted](#)

[com.ms.util.cab.CabCorruptException could not be converted](#)

[com.ms.util.cab.CabCreator could not be converted](#)

[com.ms.util.cab.CabDecoder could not be converted](#)

[com.ms.util.cab.CabDecoderInterface could not be converted](#)

Java.util.jar Error Messages

[java.util.jar.Attributes could not be converted](#)

[java.util.jar.Attributes.Name could not be converted](#)

[java.util.jar.JarEntry could not be converted](#)

[java.util.jar.JarException could not be converted](#)

[java.util.jar.JarFile could not be converted](#)

[java.util.jar.JarInputStream could not be converted](#)

[java.util.jar.JarOutputStream could not be converted](#)

[java.util.jar.Manifest could not be converted](#)

Java.util.zip Error Messages

java.util.zip.Adler32 could not be converted

java.util.zip.CheckedInputStream could not be converted

java.util.zip.CheckedInputStream.CheckedInputStream could not be converted

java.util.zip.CheckedInputStream.getChecksum could not be converted

java.util.zip.CheckedOutputStream could not be converted

java.util.zip.CheckedOutputStream.CheckedOutputStream could not be converted

java.util.zip.CheckedOutputStream.getChecksum could not be converted

java.util.zip.Checksum could not be converted

java.util.zip.CRC32 could not be converted

java.util.zip.DataFormatException could not be converted

java.util.zip.Deflater could not be converted

java.util.zip.DeflaterOutputStream could not be converted

java.util.zip.GZIPInputStream could not be converted

java.util.zip.GZIPOutputStream could not be converted

java.util.zip.Inflater could not be converted

java.util.zip.InflaterInputStream could not be converted

java.util.zip.ZipEntry could not be converted

java.util.zip.ZipFile could not be converted

java.util.zip.ZipInputStream could not be converted

java.util.zip.ZipOutputStream could not be converted

Javax.accessibility Error Messages

javax.accessibility.AccessibleAction could not be converted

javax.accessibility.AccessibleAction.doAccessibleAction could not be converted

javax.accessibility.AccessibleAction.getAccessibleActionCount could not be converted

javax.accessibility.AccessibleAction.getAccessibleActionDescription could not be converted

javax.accessibility.AccessibleBundle could not be converted

javax.accessibility.AccessibleComponent could not be converted

javax.accessibility.AccessibleComponent.addFocusListener could not be converted

javax.accessibility.AccessibleComponent.isVisible could not be converted

javax.accessibility.AccessibleComponent.removeFocusListener could not be converted

javax.accessibility.AccessibleComponent.setBounds could not be converted

javax.accessibility.AccessibleContext.<PropertyChangeEvent> could not be converted

javax.accessibility.AccessibleContext.accessibleDescription could not be converted

javax.accessibility.AccessibleContext.accessibleName could not be converted

javax.accessibility.AccessibleContext.accessibleParent could not be converted

javax.accessibility.AccessibleContext.addPropertyChangeListener could not be converted

javax.accessibility.AccessibleContext.firePropertyChange could not be converted

javax.accessibility.AccessibleContext.getAccessibleIcon could not be converted

javax.accessibility.AccessibleContext.getAccessibleIndexInParent could not be converted

javax.accessibility.AccessibleContext.getAccessibleParent could not be converted

javax.accessibility.AccessibleContext.getAccessibleRelationSet could not be converted

javax.accessibility.AccessibleContext.getAccessibleSelection could not be converted

javax.accessibility.AccessibleContext.getAccessibleTable could not be converted

javax.accessibility.AccessibleContext.getAccessibleText could not be converted

javax.accessibility.AccessibleContext.removePropertyChangeListener could not be converted

javax.accessibility.AccessibleContext.setAccessibleParent could not be converted

javax.accessibility.AccessibleHyperlink could not be converted

javax.accessibility.AccessibleHypertext could not be converted

javax.accessibility.AccessibleIcon could not be converted

javax.accessibility.AccessibleRelation could not be converted

javax.accessibility.AccessibleRelationSet could not be converted

javax.accessibility.AccessibleResourceBundle could not be converted

javax.accessibility.AccessibleRole.AccessibleRole could not be converted

javax.accessibility.AccessibleSelection could not be converted

javax.accessibility.AccessibleSelection.addAccessibleSelection could not be converted

javax.accessibility.AccessibleSelection.clearAccessibleSelection could not be converted

javax.accessibility.AccessibleSelection.getAccessibleSelection could not be converted

javax.accessibility.AccessibleSelection.getAccessibleSelectionCount could not be converted

javax.accessibility.AccessibleSelection.isAccessibleChildSelected could not be converted

javax.accessibility.AccessibleSelection.removeAccessibleSelection could not be converted

javax.accessibility.AccessibleSelection.selectAllAccessibleSelection could not be converted

javax.accessibility.AccessibleState.<StateName> could not be converted

javax.accessibility.AccessibleState.AccessibleState could not be converted

javax.accessibility.AccessibleStateSet.states could not be converted

javax.accessibility.AccessibleStateSet.toArray could not be converted

javax.accessibility.AccessibleTable could not be converted

javax.accessibility.AccessibleTableModelChange could not be converted

javax.accessibility.AccessibleText could not be converted

javax.accessibility.AccessibleText.<TextType> could not be converted

javax.accessibility.AccessibleText.getAfterIndex could not be converted

javax.accessibility.AccessibleText.getAtIndex could not be converted

javax.accessibility.AccessibleText.getBeforeIndex could not be converted

javax.accessibility.AccessibleText.getCaretPosition could not be converted

javax.accessibility.AccessibleText.getCharacterAttribute could not be converted

javax.accessibility.AccessibleText.getCharacterBounds could not be converted

javax.accessibility.AccessibleText.getCharCount could not be converted

javax.accessibility.AccessibleText.getIndexAtPoint could not be converted

javax.accessibility.AccessibleText.getSelectedText could not be converted

javax.accessibility.AccessibleText.getSelectionEnd could not be converted

javax.accessibility.AccessibleText.getSelectionStart could not be converted

javax.accessibility.AccessibleValue could not be converted

javax.accessibility.AccessibleValue.getCurrentAccessibleValue could not be converted

javax.accessibility.AccessibleValue.getMaximumAccessibleValue could not be converted

javax.accessibility.AccessibleValue.getMinimumAccessibleValue could not be converted

javax.accessibility.AccessibleValue.setCurrentAccessibleValue could not be converted

Javax.crypto Error Messages

javax.crypto.Cipher.<KeyType> could not be converted

javax.crypto.Cipher.Cipher could not be converted

javax.crypto.Cipher.doFinal could not be converted

javax.crypto.Cipher.getExemptionMechanism could not be converted

javax.crypto.Cipher.getInstance could not be converted

javax.crypto.Cipher.getParameters could not be converted

javax.crypto.Cipher.getProvider could not be converted

javax.crypto.Cipher.init could not be converted

javax.crypto.Cipher.t could not be converted

javax.crypto.Cipher.unwrap could not be converted

javax.crypto.Cipher.UNWRAP_MODE could not be converted

javax.crypto.Cipher.wrap could not be converted

javax.crypto.Cipher.WRAP_MODE could not be converted

javax.crypto.CipherInputStream.available could not be converted

javax.crypto.CipherInputStream.CipherInputStream(Stream) could not be converted

javax.crypto.CipherInputStream.CipherInputStream(Stream, Cipher) could not be converted

javax.crypto.CipherInputStream.skip could not be converted

javax.crypto.CipherOutputStream.CipherOutputStream(OutputStream) could not be converted

javax.crypto.CipherOutputStream.CipherOutputStream(OutputStream, Cipher) could not be converted

javax.crypto.CipherSpi could not be converted

javax.crypto.ExemptionMechanism could not be converted

javax.crypto.ExemptionMechanismSpi could not be converted

javax.crypto.interfaces.DHKey could not be converted

javax.crypto.interfaces.DHPrivateKey could not be converted

javax.crypto.interfaces.DHPublicKey could not be converted

javax.crypto.KeyAgreement could not be converted

javax.crypto.KeyAgreementSpi could not be converted

javax.crypto.KeyGenerator.generateKey could not be converted

javax.crypto.KeyGenerator.getAlgorithm could not be converted

javax.crypto.KeyGenerator.getProvider could not be converted

javax.crypto.KeyGenerator.init could not be converted

javax.crypto.KeyGeneratorSpi.engineGenerateKey could not be converted

javax.crypto.KeyGeneratorSpi.engineInit could not be converted

javax.crypto.Mac.clone could not be converted

javax.crypto.Mac.doFinal could not be converted

javax.crypto.Mac.getInstance could not be converted

javax.crypto.Mac.getProvider could not be converted

javax.crypto.Mac.Mac could not be converted

javax.crypto.MacSpi.clone could not be converted

javax.crypto.MacSpi.engineDoFinal could not be converted

javax.crypto.MacSpi.MacSpi could not be converted

javax.crypto.NullCipher.NullCipher could not be converted

javax.crypto.SealedObject could not be converted

javax.crypto.SealedObject.encodedParams could not be converted

javax.crypto.SealedObject.getAlgorithm could not be converted

javax.crypto.SealedObject.getObject could not be converted

javax.crypto.SealedObject.SealedObject could not be converted

javax.crypto.SecretKey could not be converted

javax.crypto.SecretKeyFactory.generateSecret could not be converted

javax.crypto.SecretKeyFactory.getAlgorithm could not be converted

javax.crypto.SecretKeyFactory.getKeySpec could not be converted

javax.crypto.SecretKeyFactory.getProvider could not be converted

javax.crypto.SecretKeyFactory.translateKey could not be converted

javax.crypto.SecretKeyFactorySpi.engineGenerateSecret could not be converted

javax.crypto.SecretKeyFactorySpi.engineGetKeySpec could not be converted

javax.crypto.SecretKeyFactorySpi.engineTranslateKey could not be converted

javax.crypto.ShortBufferException.a could not be converted

javax.crypto.spec.DESedeKeySpec.isParityAdjusted could not be converted

javax.crypto.spec.DESKeySpec.isParityAdjusted could not be converted

javax.crypto.spec.DESKeySpec.isWeak could not be converted

javax.crypto.spec.DHGenParameterSpec could not be converted

javax.crypto.spec.DHParameterSpec could not be converted

javax.crypto.spec.DHPrivateKeySpec could not be converted

javax.crypto.spec.DHPublicKeySpec could not be converted

javax.crypto.spec.IvParameterSpec.IvParameterSpec could not be converted

javax.crypto.spec.PBEKeySpec.getPassword could not be converted

javax.crypto.spec.PBEParameterSpec could not be converted

javax.crypto.spec.RC5ParameterSpec could not be converted

javax.crypto.spec.SecretKeySpec.getAlgorithm could not be converted

Javax.ejb Error Messages

[javax.ejb.deployment.AccessControlEntry could not be converted](#)

[javax.ejb.deployment.ControlDescriptor could not be converted](#)

[javax.ejb.deployment.DeploymentDescriptor could not be converted](#)

[javax.ejb.deployment.EntityDescriptor could not be converted](#)

[javax.ejb.deployment.SessionDescriptor could not be converted](#)

[javax.ejb.EJBContext could not be converted](#)

[javax.ejb.EJBContext.getCallerIdentity could not be converted](#)

[javax.ejb.EJBContext.getCallerPrincipal could not be converted](#)

[javax.ejb.EJBContext.getEJBHome could not be converted](#)

[javax.ejb.EJBContext.getEJBLocalHome could not be converted](#)

[javax.ejb.EJBContext.getEnvironment could not be converted](#)

[javax.ejb.EJBContext.getRollbackOnly could not be converted](#)

[javax.ejb.EJBContext.getUserTransaction could not be converted](#)

[javax.ejb.EJBContext.isCallerInRole could not be converted](#)

[javax.ejb.EJBContext.setRollbackOnly could not be converted](#)

[javax.ejb.EJBHome could not be converted](#)

[javax.ejb.EJBHome.getEJBMetaData could not be converted](#)

[javax.ejb.EJBHome.getHomeHandle could not be converted](#)

[javax.ejb.EJBLocalHome could not be converted](#)

[javax.ejb.EJBLocalObject could not be converted](#)

[javax.ejb.EJBLocalObject.getEJBLocalHome could not be converted](#)

[javax.ejb.EJBMetaData could not be converted](#)

[javax.ejb.EJBObject could not be converted](#)

[javax.ejb.EJBObject.getEJBHome could not be converted](#)

[javax.ejb.EJBObject.getHandle could not be converted](#)

[javax.ejb.EntityBean.setEntityContext could not be converted](#)

[javax.ejb.EntityBean.unsetEntityContext could not be converted](#)

[javax.ejb.EntityContext could not be converted](#)

[javax.ejb.EntityContext.getEJBLocalObject could not be converted](#)

[javax.ejb.EntityContext.getEJBObject could not be converted](#)

[javax.ejb.Handle could not be converted](#)

[javax.ejb.HomeHandle could not be converted](#)

[javax.ejb.MessageDrivenBean.setMessageDrivenContext could not be converted](#)

[javax.ejb.MessageDrivenContext could not be converted](#)

[javax.ejb.SessionBean.setSessionContext could not be converted](#)

[javax.ejb.SessionContext could not be converted](#)

[javax.ejb.SessionContext.getEJBLocalObject could not be converted](#)

javax.ejb.SessionContext.getEJBObject could not be converted

javax.ejb.SessionSynchronization could not be converted

javax.ejb.spi.HandleDelegate could not be converted

Javax.jms Error Messages

[javax.jms.BytesMessage.readBytes could not be converted](#)

[javax.jms.BytesMessage.readUTF could not be converted](#)

[javax.jms.BytesMessage.reset could not be converted](#)

[javax.jms.BytesMessage.writeBytes could not be converted](#)

[javax.jms.BytesMessage.writeObject could not be converted](#)

[javax.jms.BytesMessage.writeUTF could not be converted](#)

[javax.jms.Connection could not be converted](#)

[javax.jms.Connection.close could not be converted](#)

[javax.jms.Connection.getClientID could not be converted](#)

[javax.jms.Connection.getExceptionListener could not be converted](#)

[javax.jms.Connection.getMetaData could not be converted](#)

[javax.jms.Connection.setClientID could not be converted](#)

[javax.jms.Connection.setExceptionListener could not be converted](#)

[javax.jms.Connection.start could not be converted](#)

[javax.jms.Connection.stop could not be converted](#)

[javax.jms.ConnectionConsumer could not be converted](#)

[javax.jms.ConnectionFactory could not be converted](#)

[javax.jms.ConnectionMetaData could not be converted](#)

[javax.jms.DeliveryMode could not be converted](#)

[javax.jms.ExceptionListener could not be converted](#)

[javax.jms.IllegalStateException.IllegalStateException could not be converted](#)

[javax.jms.InvalidClientIDException could not be converted](#)

[javax.jms.InvalidDestinationException.InvalidDestinationException\(String\) could not be converted](#)

[javax.jms.InvalidDestinationException.InvalidDestinationException\(String, String\) could not be converted](#)

[javax.jms.InvalidSelectorException could not be converted](#)

[javax.jms.JMSException.getLinkedException could not be converted](#)

[javax.jms.JMSException.JMSException\(String\) could not be converted](#)

[javax.jms.JMSException.JMSException\(String, String\) could not be converted](#)

[javax.jms.JMSException.setLinkedException could not be converted](#)

[javax.jms.JMSSecurityException.JMSSecurityException\(String\) could not be converted](#)

[javax.jms.JMSSecurityException.JMSSecurityException\(String, String\) could not be converted](#)

[javax.jms.MapMessage.getFloat could not be converted](#)

[javax.jms.MapMessage.setBytes could not be converted](#)

[javax.jms.Message.acknowledge could not be converted](#)

[javax.jms.Message.clearProperties could not be converted](#)

[javax.jms.Message.getBooleanProperty could not be converted](#)

[javax.jms.Message.getBytesProperty could not be converted](#)

javax.jms.Message.getDoubleProperty could not be converted
javax.jms.Message.getFloatProperty could not be converted
javax.jms.Message.getIntProperty could not be converted
javax.jms.Message.getJMSCorrelationIDAsBytes could not be converted
javax.jms.Message.getJMSDestination could not be converted
javax.jms.Message.getJMSPriority could not be converted
javax.jms.Message.getJMSRedelivered could not be converted
javax.jms.Message.getJMSReplyTo could not be converted
javax.jms.Message.getJMSTimestamp could not be converted
javax.jms.Message.getJMSType could not be converted
javax.jms.Message.getLongProperty could not be converted
javax.jms.Message.getObjectProperty could not be converted
javax.jms.Message.getPropertyNames could not be converted
javax.jms.Message.getShortProperty could not be converted
javax.jms.Message.getStringProperty could not be converted
javax.jms.Message.propertyExists could not be converted
javax.jms.Message.setBooleanProperty could not be converted
javax.jms.Message.setByteProperty could not be converted
javax.jms.Message.setDoubleProperty could not be converted
javax.jms.Message.setFloatProperty could not be converted
javax.jms.Message.setIntProperty could not be converted
javax.jms.Message.setJMSCorrelationID could not be converted
javax.jms.Message.setJMSCorrelationIDAsBytes could not be converted
javax.jms.Message.setJMSDestination could not be converted
javax.jms.Message.setJMSMessageID could not be converted
javax.jms.Message.setJMSPriority could not be converted
javax.jms.Message.setJMSRedelivered could not be converted
javax.jms.Message.setJMSTimestamp could not be converted
javax.jms.Message.setJMSType could not be converted
javax.jms.Message.setLongProperty could not be converted
javax.jms.Message.setObjectProperty could not be converted
javax.jms.Message.setShortProperty could not be converted
javax.jms.Message.setStringProperty could not be converted
javax.jms.MessageConsumer.setMessageListener could not be converted
javax.jms.MessageConsumer.getMessageSelector could not be converted
javax.jms.MessageEOFException.MessageEOFException could not be converted
javax.jms.MessageFormatException could not be converted
javax.jms.MessageListener could not be converted
javax.jms.MessageNotReadableException could not be converted
javax.jms.MessageNotWritableException could not be converted

javax.jms.MessageProducer.getDisableMessageID could not be converted

javax.jms.MessageProducer.getDisableMessageTimestamp could not be converted

javax.jms.MessageProducer.setDeliveryMode could not be converted

javax.jms.MessageProducer.setDisableMessageID could not be converted

javax.jms.MessageProducer.setDisableMessageTimestamp could not be converted

javax.jms.MessageProducer.setPriority could not be converted

javax.jms.MessageProducer.setTimeToLive could not be converted

javax.jms.QueueConnection could not be converted

javax.jms.QueueConnection.createConnectionConsumer could not be converted

javax.jms.QueueConnection.createQueueSession could not be converted

javax.jms.QueueConnectionFactory could not be converted

javax.jms.QueueConnectionFactory.createQueueConnection could not be converted

javax.jms.QueueConnectionFactory.createQueueConnection(String, String) could not be converted

javax.jms.QueueRequestor could not be converted

javax.jms.QueueSession could not be converted

javax.jms.QueueSession.createBrowser could not be converted

javax.jms.QueueSession.createReceiver could not be converted

javax.jms.QueueBrowser.getMessageSelector could not be converted

javax.jms.QueueConnection could not be converted

javax.jms.QueueConnection.createQueueSession could not be converted

javax.jms.QueueConnectionFactory could not be converted

javax.jms.QueueConnectionFactory.createQueueConnection could not be converted

javax.jms.QueueConnectionFactory.createQueueConnection(String, String) could not be converted

javax.jms.QueueSession could not be converted

javax.jms.ResourceAllocationException.ResourceAllocationException could not be converted

javax.jms.ServerSession could not be converted

javax.jms.ServerSessionPool could not be converted

javax.jms.Session could not be converted

javax.jms.Session.AUTO_ACKNOWLEDGE could not be converted

javax.jms.Session.CLIENT_ACKNOWLEDGE could not be converted

javax.jms.Session.close could not be converted

javax.jms.Session.commit could not be converted

javax.jms.Session.DUPS_OK_ACKNOWLEDGE could not be converted

javax.jms.Session.getMessageListener could not be converted

javax.jms.Session.getTransacted could not be converted

javax.jms.Session.recover could not be converted

javax.jms.Session.rollback could not be converted

javax.jms.Session.run could not be converted

javax.jms.Session.setMessageListener could not be converted

javax.jms.StreamMessage could not be converted

javax.jms.StreamMessage.readBytes could not be converted
javax.jms.StreamMessage.readObject could not be converted
javax.jms.StreamMessage.reset could not be converted
javax.jms.StreamMessage.writeBytes could not be converted
javax.jms.StreamMessage.writeObject could not be converted
javax.jms.TemporaryQueue could not be converted
javax.jms.TemporaryTopic could not be converted
javax.jms.TopicConnection could not be converted
javax.jms.TopicConnection.createConnectionConsumer could not be converted
javax.jms.TopicConnection.createDurableConnectionConsumer could not be converted
javax.jms.TopicConnection.createTopicSession could not be converted
javax.jms.TopicConnectionFactory could not be converted
javax.jms.TopicConnectionFactory.createTopicConnection could not be converted
javax.jms.TopicRequestor could not be converted
javax.jms.TopicSession could not be converted
javax.jms.TopicSession.createDurableSubscriber could not be converted
javax.jms.TopicSession.createSubscriber could not be converted
javax.jms.TopicSession.createTemporaryTopic could not be converted
javax.jms.TopicSession.unsubscribe could not be converted
javax.jms.TopicSubscriber.getNoLocal could not be converted
javax.jms.TransactionInProgressException.TransactionInProgressException could not be converted
javax.jms.TransactionRolledBackException.TransactionRolledBackException could not be converted
javax.jms.XAConnection could not be converted
javax.jms.XAConnectionFactory could not be converted
javax.jms.XAQueueConnection could not be converted
javax.jms.XAQueueConnectionFactory could not be converted
javax.jms.XAQueueSession could not be converted
javax.jms.XASession could not be converted
javax.jms.XATopicConnection could not be converted
javax.jms.XATopicConnectionFactory could not be converted
javax.jms.XATopicSession could not be converted

Javax.mail Error Messages

[javax.mail.Address.getType could not be converted](#)

[javax.mail.AuthenticationFailedException could not be converted](#)

[javax.mail.Authenticator could not be converted](#)

[javax.mail.BodyPart.addHeader could not be converted](#)

[javax.mail.BodyPart.BodyPart could not be converted](#)

[javax.mail.BodyPart.getAllHeaders could not be converted](#)

[javax.mail.BodyPart.getContent could not be converted](#)

[javax.mail.BodyPart.getContentType could not be converted](#)

[javax.mail.BodyPart.getDataHandler could not be converted](#)

[javax.mail.BodyPart.getDescription could not be converted](#)

[javax.mail.BodyPart.getDisposition could not be converted](#)

[javax.mail.BodyPart.getHeader could not be converted](#)

[javax.mail.BodyPart.getLineCount could not be converted](#)

[javax.mail.BodyPart.getMatchingHeaders could not be converted](#)

[javax.mail.BodyPart.getParent could not be converted](#)

[javax.mail.BodyPart.getSize could not be converted](#)

[javax.mail.BodyPart.isMimeType could not be converted](#)

[javax.mail.BodyPart.parent could not be converted](#)

[javax.mail.BodyPart.removeHeader could not be converted](#)

[javax.mail.BodyPart.setContent\(Multipart\) could not be converted](#)

[javax.mail.BodyPart.setContent\(Object, String\) could not be converted](#)

[javax.mail.BodyPart.setDataHandler could not be converted](#)

[javax.mail.BodyPart.setDescription could not be converted](#)

[javax.mail.BodyPart.setDisposition could not be converted](#)

[javax.mail.BodyPart.setFileName could not be converted](#)

[javax.mail.BodyPart.setHeader could not be converted](#)

[javax.mail.BodyPart.writeTo could not be converted](#)

[javax.mail.event.ConnectionAdapter could not be converted](#)

[javax.mail.event.ConnectionEvent could not be converted](#)

[javax.mail.event.ConnectionListener could not be converted](#)

[javax.mail.event.FolderAdapter could not be converted](#)

[javax.mail.event.FolderEvent could not be converted](#)

[javax.mail.event.FolderListener could not be converted](#)

[javax.mail.event.MailEvent could not be converted](#)

[javax.mail.event.MessageChangedEvent could not be converted](#)

[javax.mail.event.MessageChangedListener could not be converted](#)

[javax.mail.event.MessageCountAdapter could not be converted](#)

javax.mail.event.MessageCountEvent could not be converted

javax.mail.event.MessageCountListener could not be converted

javax.mail.event.StoreEvent could not be converted

javax.mail.event.StoreListener could not be converted

javax.mail.event.TransportAdapter could not be converted

javax.mail.event.TransportEvent could not be converted

javax.mail.event.TransportListener could not be converted

javax.mail.FetchProfile could not be converted

javax.mail.FetchProfile.Item could not be converted

javax.mail.Flags could not be converted

javax.mail.Flags.Flag could not be converted

javax.mail.Folder could not be converted

javax.mail.FolderClosedException could not be converted

javax.mail.FolderNotFoundException could not be converted

javax.mail.Header.Header could not be converted

javax.mail.IllegalWriteException could not be converted

javax.mail.internet.AddressException could not be converted

javax.mail.internet.AddressException.getPos could not be converted

javax.mail.internet.AddressException.getRef could not be converted

javax.mail.internet.AddressException.pos could not be converted

javax.mail.internet.AddressException.ref could not be converted

javax.mail.internet.ContentDisposition could not be converted

javax.mail.internet.ContentType could not be converted

javax.mail.internet.HeaderTokenizer could not be converted

javax.mail.internet.HeaderTokenizer.Token could not be converted

javax.mail.internet.InternetAddress.clone could not be converted

javax.mail.internet.InternetAddress.encodedPersonal could not be converted

javax.mail.internet.InternetHeaders.getHeader could not be converted

javax.mail.internet.InternetAddress.getLocalAddress could not be converted

javax.mail.internet.InternetAddress.getPersonal could not be converted

javax.mail.internet.InternetAddress.InternetAddress could not be converted

javax.mail.internet.InternetAddress.personal could not be converted

javax.mail.internet.InternetAddress.setAddress could not be converted

javax.mail.internet.InternetAddress.setPersonal could not be converted

javax.mail.internet.InternetHeaders.addHeader could not be converted

javax.mail.internet.InternetHeaders.getMatchingHeaderLines could not be converted

javax.mail.internet.InternetHeaders.getMatchingHeaders could not be converted

javax.mail.internet.InternetHeaders.getNonMatchingHeaderLines could not be converted

javax.mail.internet.InternetHeaders.getNonMatchingHeaders could not be converted

javax.mail.internet.InternetHeaders.InternetHeaders could not be converted

javax.mail.internet.InternetHeaders.load could not be converted

javax.mail.internet.MailDateFormat could not be converted

javax.mail.internet.MimeBodyPart.addHeader could not be converted

javax.mail.internet.MimeBodyPart.addHeaderLine could not be converted

javax.mail.internet.MimeBodyPart.content could not be converted

javax.mail.internet.MimeBodyPart.contentStream could not be converted

javax.mail.internet.MimeBodyPart.dh could not be converted

javax.mail.internet.MimeBodyPart.getAllHeaderLines could not be converted

javax.mail.internet.MimeBodyPart.getAllHeaders could not be converted

javax.mail.internet.MimeBodyPart.getContent could not be converted

javax.mail.internet.MimeBodyPart.getContentID could not be converted

javax.mail.internet.MimeBodyPart.getContentLanguage could not be converted

javax.mail.internet.MimeBodyPart.getContentMD5 could not be converted

javax.mail.internet.MimeBodyPart.getContentStream could not be converted

javax.mail.internet.MimeBodyPart.getContentType could not be converted

javax.mail.internet.MimeBodyPart.getDataHandler could not be converted

javax.mail.internet.MimeBodyPart.getDescription could not be converted

javax.mail.internet.MimeBodyPart.getDisposition could not be converted

javax.mail.internet.MimeBodyPart.getEncoding could not be converted

javax.mail.internet.MimeBodyPart.getHeader(String) could not be converted

javax.mail.internet.MimeBodyPart.getHeader(String, String) could not be converted

javax.mail.internet.MimeBodyPart.getLineCount could not be converted

javax.mail.internet.MimeBodyPart.getMatchingHeaderLines could not be converted

javax.mail.internet.MimeBodyPart.getMatchingHeaders could not be converted

javax.mail.internet.MimeBodyPart.getNonMatchingHeaderLines could not be converted

javax.mail.internet.MimeBodyPart.getNonMatchingHeaders could not be converted

javax.mail.internet.MimeBodyPart.getRawInputStream could not be converted

javax.mail.internet.MimeBodyPart.getSize could not be converted

javax.mail.internet.MimeBodyPart.headers could not be converted

javax.mail.internet.MimeBodyPart.isMimeType could not be converted

javax.mail.internet.MimeBodyPart.MimeBodyPart could not be converted

javax.mail.internet.MimeBodyPart.MimeBodyPart(InputStream) could not be converted

javax.mail.internet.MimeBodyPart.MimeBodyPart(InternetHeaders, byte[]) could not be converted

javax.mail.internet.MimeBodyPart.removeHeader could not be converted

javax.mail.internet.MimeBodyPart.setContent(Multipart) could not be converted

javax.mail.internet.MimeBodyPart.setContent(Object, String) could not be converted

javax.mail.internet.MimeBodyPart.setContentLanguage could not be converted

javax.mail.internet.MimeBodyPart.setContentMD5 could not be converted

javax.mail.internet.MimeBodyPart.setDataHandler could not be converted

javax.mail.internet.MimeBodyPart.setDescription could not be converted

javax.mail.internet.MimeBodyPart.setDisposition could not be converted
javax.mail.internet.MimeBodyPart.setHeader could not be converted
javax.mail.internet.MimeBodyPart.setText could not be converted
javax.mail.internet.MimeBodyPart.updateHeaders could not be converted
javax.mail.internet.MimeBodyPart.writeTo could not be converted
javax.mail.internet.MimeMessage.addHeader could not be converted
javax.mail.internet.MimeMessage.addHeaderLine could not be converted
javax.mail.internet.MimeMessage.addRecipients could not be converted
javax.mail.internet.MimeMessage.content could not be converted
javax.mail.internet.MimeMessage.contentStream could not be converted
javax.mail.internet.MimeMessage.createInternetHeaders could not be converted
javax.mail.internet.MimeMessage.dh could not be converted
javax.mail.internet.MimeMessage.flags could not be converted
javax.mail.internet.MimeMessage.getAllHeaderLines could not be converted
javax.mail.internet.MimeMessage.getAllRecipients could not be converted
javax.mail.internet.MimeMessage.getContentID could not be converted
javax.mail.internet.MimeMessage.getContentLanguage could not be converted
javax.mail.internet.MimeMessage.getContentMD5 could not be converted
javax.mail.internet.MimeMessage.getContentStream could not be converted
javax.mail.internet.MimeMessage.getContentType could not be converted
javax.mail.internet.MimeMessage.getDataHandler could not be converted
javax.mail.internet.MimeMessage.getDescription could not be converted
javax.mail.internet.MimeMessage.getDisposition could not be converted
javax.mail.internet.MimeMessage.getEncoding could not be converted
javax.mail.internet.MimeMessage.getFileName could not be converted
javax.mail.internet.MimeMessage.getFlags could not be converted
javax.mail.internet.MimeMessage.getFrom could not be converted
javax.mail.internet.MimeMessage.getHeader could not be converted
javax.mail.internet.MimeMessage.getInputStream could not be converted
javax.mail.internet.MimeMessage.getLineCount could not be converted
javax.mail.internet.MimeMessage.getMatchingHeaderLines could not be converted
javax.mail.internet.MimeMessage.getMatchingHeaders could not be converted
javax.mail.internet.MimeMessage.getMessageID could not be converted
javax.mail.internet.MimeMessage.getNonMatchingHeaderLines could not be converted
javax.mail.internet.MimeMessage.getRawInputStream could not be converted
javax.mail.internet.MimeMessage.getReceivedDate could not be converted
javax.mail.internet.MimeMessage.getRecipients could not be converted
javax.mail.internet.MimeMessage.getReplyTo could not be converted
javax.mail.internet.MimeMessage.getSentDate could not be converted
javax.mail.internet.MimeMessage.getSize could not be converted

javax.mail.internet.MimeMessage.headers could not be converted

javax.mail.internet.MimeMessage.isMimeType could not be converted

javax.mail.internet.MimeMessage.isSet could not be converted

javax.mail.internet.MimeMessage.MimeMessage could not be converted

javax.mail.internet.MimeMessage.modified could not be converted

javax.mail.internet.MimeMessage.parse could not be converted

javax.mail.internet.MimeMessage.RecipientType could not be converted

javax.mail.internet.MimeMessage.reply could not be converted

javax.mail.internet.MimeMessage.saveChanges could not be converted

javax.mail.internet.MimeMessage.saved could not be converted

javax.mail.internet.MimeMessage.setContent could not be converted

javax.mail.internet.MimeMessage.setContentID could not be converted

javax.mail.internet.MimeMessage.setContentLanguage could not be converted

javax.mail.internet.MimeMessage.setContentMD5 could not be converted

javax.mail.internet.MimeMessage.setDataHandler could not be converted

javax.mail.internet.MimeMessage.setDescription could not be converted

javax.mail.internet.MimeMessage.setDisposition could not be converted

javax.mail.internet.MimeMessage.setFileName could not be converted

javax.mail.internet.MimeMessage.setFlags could not be converted

javax.mail.internet.MimeMessage.setFrom could not be converted

javax.mail.internet.MimeMessage.setRecipients could not be converted

javax.mail.internet.MimeMessage.setSentDate could not be converted

javax.mail.internet.MimeMessage.setSubject could not be converted

javax.mail.internet.MimeMessage.setText could not be converted

javax.mail.internet.MimeMessage.updateHeaders could not be converted

javax.mail.internet.MimeMessage.writeTo could not be converted

javax.mail.internet.MimeMultipart.createInternetHeaders could not be converted

javax.mail.internet.MimeMultipart.createMimeBodyPart could not be converted

javax.mail.internet.MimeMultipart.ds could not be converted

javax.mail.internet.MimeMultipart.getBodyPart could not be converted

javax.mail.internet.MimeMultipart.MimeMultipart(DataSource) could not be converted

javax.mail.internet.MimeMultipart.MimeMultipart(String) could not be converted

javax.mail.internet.MimeMultipart.parse could not be converted

javax.mail.internet.MimeMultipart.parsed could not be converted

javax.mail.internet.MimeMultipart.setSubType could not be converted

javax.mail.internet.MimeMultipart.updateHeaders could not be converted

javax.mail.internet.MimeMultipart.writeTo could not be converted

javax.mail.internet.MimePart.getContentID could not be converted

javax.mail.internet.MimePart.getContentLanguage could not be converted

javax.mail.internet.MimePart.getContentMD5 could not be converted

javax.mail.internet.MimePart.getMatchingHeaderLines could not be converted

javax.mail.internet.MimePart.getNonMatchingHeaderLines could not be converted

javax.mail.internet.MimePart.setContentLanguage could not be converted

javax.mail.internet.MimePart.setContentMD5 could not be converted

javax.mail.internet.MimePart.setText could not be converted

javax.mail.internet.MimePartDataSource could not be converted

javax.mail.internet.MimeUtility could not be converted

javax.mail.internet.NewsAddress could not be converted

javax.mail.internet.ParameterList could not be converted

javax.mail.internet.ParseException could not be converted

javax.mail.internet.SharedInputStream could not be converted

javax.mail.Message.addHeader could not be converted

javax.mail.Message.expunged could not be converted

javax.mail.Message.folder could not be converted

javax.mail.Message.getAllRecipients could not be converted

javax.mail.Message.getContentType could not be converted

javax.mail.Message.getDataHandler could not be converted

javax.mail.Message.getDescription could not be converted

javax.mail.Message.getDisposition could not be converted

javax.mail.Message.getFileName could not be converted

javax.mail.Message.getFlags could not be converted

javax.mail.Message.getFolder could not be converted

javax.mail.Message.getFrom could not be converted

javax.mail.Message.getInputStream could not be converted

javax.mail.Message.getLineCount could not be converted

javax.mail.Message.getMatchingHeaders could not be converted

javax.mail.Message.getMessageNumber could not be converted

javax.mail.Message.getNonMatchingHeaders could not be converted

javax.mail.Message.getReceivedDate could not be converted

javax.mail.Message.getRecipients could not be converted

javax.mail.Message.getReplyTo could not be converted

javax.mail.Message.getSentDate could not be converted

javax.mail.Message.getSize could not be converted

javax.mail.Message.isExpunged could not be converted

javax.mail.Message.isMimeType could not be converted

javax.mail.Message.isSet could not be converted

javax.mail.Message.match could not be converted

javax.mail.Message.Message could not be converted

javax.mail.Message.msgnum could not be converted

javax.mail.Message.RecipientType.readResolve could not be converted

javax.mail.Message.RecipientType.type could not be converted

javax.mail.Message.reply could not be converted

javax.mail.Message.saveChanges could not be converted

javax.mail.Message.session could not be converted

javax.mail.Message.setContent could not be converted

javax.mail.Message.setDataHandler could not be converted

javax.mail.Message.setDescription could not be converted

javax.mail.Message.setDisposition could not be converted

javax.mail.Message.setExpunged could not be converted

javax.mail.Message.setFileName could not be converted

javax.mail.Message.setFlag could not be converted

javax.mail.Message.setFlags could not be converted

javax.mail.Message.setFrom could not be converted

javax.mail.Message.setMessageNumber could not be converted

javax.mail.Message.setSentDate could not be converted

javax.mail.Message.writeTo could not be converted

javax.mail.MessageAware could not be converted

javax.mail.MessageContext could not be converted

javax.mail.MessageRemovedException could not be converted

javax.mail.MessagingException.setNextException could not be converted

javax.mail.MethodNotSupportedException could not be converted

javax.mail.Multipart.contentType could not be converted

javax.mail.Multipart.getContentType could not be converted

javax.mail.Multipart.getParent could not be converted

javax.mail.Multipart.parent could not be converted

javax.mail.Multipart.setMultipartDataSource could not be converted

javax.mail.Multipart.setParent could not be converted

javax.mail.Multipart.writeTo could not be converted

javax.mail.MultipartDataSource could not be converted

javax.mail.NoSuchProviderException could not be converted

javax.mail.Part.addHeader could not be converted

javax.mail.Part.ATTACHMENT could not be converted

javax.mail.Part.getContentType could not be converted

javax.mail.Part.getDataHandler could not be converted

javax.mail.Part.getDescription could not be converted

javax.mail.Part.getDisposition could not be converted

javax.mail.Part.getFileName could not be converted

javax.mail.Part.getInputStream could not be converted

javax.mail.Part.getLineCount could not be converted

javax.mail.Part.getMatchingHeaders could not be converted

javax.mail.Part.getNonMatchingHeaders could not be converted

javax.mail.Part.getSize could not be converted

javax.mail.Part.INLINE could not be converted

javax.mail.Part.isMimeType could not be converted

javax.mail.Part.setContent could not be converted

javax.mail.Part.setDataHandler could not be converted

javax.mail.Part.setDescription could not be converted

javax.mail.Part.setDisposition could not be converted

javax.mail.Part.setFileName could not be converted

javax.mail.Part.writeTo could not be converted

javax.mail.PasswordAuthentication could not be converted

javax.mail.Provider could not be converted

javax.mail.Provider.Type could not be converted

javax.mail.ReadOnlyFolderException could not be converted

javax.mail.search.AddressStringTerm could not be converted

javax.mail.search.AddressTerm could not be converted

javax.mail.search.AndTerm could not be converted

javax.mail.search.BodyTerm could not be converted

javax.mail.search.ComparisonTerm could not be converted

javax.mail.search.DateTerm could not be converted

javax.mail.search.FlagTerm could not be converted

javax.mail.search.FromStringTerm could not be converted

javax.mail.search.FromTerm could not be converted

javax.mail.search.HeaderTerm could not be converted

javax.mail.search.IntegerComparisonTerm could not be converted

javax.mail.search.MessageIDTerm could not be converted

javax.mail.search.MessageNumberTerm could not be converted

javax.mail.search.NotTerm could not be converted

javax.mail.search.OrTerm could not be converted

javax.mail.search.ReceivedDateTerm could not be converted

javax.mail.search.RecipientStringTerm could not be converted

javax.mail.search.RecipientTerm could not be converted

javax.mail.search.SearchException could not be converted

javax.mail.search.SearchTerm could not be converted

javax.mail.search.SentDateTerm could not be converted

javax.mail.search.SizeTerm could not be converted

javax.mail.search.StringTerm could not be converted

javax.mail.search.SubjectTerm could not be converted

javax.mail.SendFailedException.getInvalidAddresses could not be converted

javax.mail.SendFailedException.getValidSentAddresses could not be converted

javax.mail.SendFailedException.getValidUnsentAddresses could not be converted
javax.mail.SendFailedException.invalid could not be converted
javax.mail.SendFailedException.validSent could not be converted
javax.mail.SendFailedException.validUnsent could not be converted
javax.mail.Service could not be converted
javax.mail.Session could not be converted
javax.mail.Store could not be converted
javax.mail.StoreClosedException could not be converted
javax.mail.Transport.addTransportListener could not be converted
javax.mail.Transport.notifyTransportListeners could not be converted
javax.mail.Transport.removeTransportListener could not be converted
javax.mail.Transport.send could not be converted
javax.mail.Transport.Transport could not be converted
javax.mail.UIDFolder could not be converted
javax.mail.UIDFolder.FetchProfileItem could not be converted
javax.mail.URLName.fullURL could not be converted
javax.mail.URLName.getFile could not be converted
javax.mail.URLName.getRef could not be converted
javax.mail.URLName.parseString could not be converted
javax.mail.URLName.URLName could not be converted

Javax.naming Error Messages

javax.naming.BinaryRefAddr could not be converted

javax.naming.BinaryRefAddr.BinaryRefAddr(String, byte[]) could not be converted

javax.naming.BinaryRefAddr.BinaryRefAddr(String, byte[], int, int) could not be converted

javax.naming.Binding could not be converted

javax.naming.Binding.Binding(String, Object) could not be converted

javax.naming.Binding.Binding(String, Object, boolean) could not be converted

javax.naming.Binding.Binding(String, String, Object) could not be converted

javax.naming.Binding.Binding(String, String, Object, boolean) could not be converted

javax.naming.Binding.setObject could not be converted

javax.naming.CannotProceedException.altName could not be converted

javax.naming.CannotProceedException.altNameCtx could not be converted

javax.naming.CannotProceedException.environment could not be converted

javax.naming.CannotProceedException.getAltName could not be converted

javax.naming.CannotProceedException.getAltNameCtx could not be converted

javax.naming.CannotProceedException.getEnvironment could not be converted

javax.naming.CannotProceedException.getRemainingNewName could not be converted

javax.naming.CannotProceedException.remainingNewName could not be converted

javax.naming.CannotProceedException.setAltName could not be converted

javax.naming.CannotProceedException.setAltNameCtx could not be converted

javax.naming.CannotProceedException.setEnvironment could not be converted

javax.naming.CannotProceedException.setRemainingNewName could not be converted

javax.naming.CompositeName could not be converted

javax.naming.CompositeName.CompositeName could not be converted

javax.naming.CompoundName could not be converted

javax.naming.CompoundName.CompoundName(Enumeration, Properties) could not be converted

javax.naming.CompoundName.CompoundName(String, Properties) could not be converted

javax.naming.CompoundName.impl could not be converted

javax.naming.CompoundName.mySyntax could not be converted

javax.naming.Context.<EnvironmentProperty> could not be converted

javax.naming.Context.addToEnvironment could not be converted

javax.naming.Context.bind(Name, Object) could not be converted

javax.naming.Context.bind(String, Object) could not be converted

javax.naming.Context.composeName could not be converted

javax.naming.Context.createSubcontext could not be converted

javax.naming.Context.destroySubcontext could not be converted

javax.naming.Context.getEnvironment could not be converted

javax.naming.Context.getNamelInNamespace could not be converted

javax.naming.Context.getNameParser could not be converted

javax.naming.Context.list could not be converted

javax.naming.Context.listBindings could not be converted

javax.naming.Context.lookup could not be converted

javax.naming.Context.lookupLink could not be converted

javax.naming.Context.rebind could not be converted

javax.naming.Context.removeFromEnvironment could not be converted

javax.naming.Context.rename could not be converted

javax.naming.Context.unbind could not be converted

javax.naming.directory.Attribute could not be converted

javax.naming.directory.Attribute.clone could not be converted

javax.naming.directory.Attribute.get could not be converted

javax.naming.directory.Attribute.getAttributeDefinition could not be converted

javax.naming.directory.Attribute.getAttributeSyntaxDefinition could not be converted

javax.naming.directory.Attribute.getID could not be converted

javax.naming.directory.Attribute.isOrdered could not be converted

javax.naming.directory.Attribute.serialVersionUID could not be converted

javax.naming.directory.AttributeModificationException.getUnexecutedModifications could not be converted

javax.naming.directory.AttributeModificationException.setUnexecutedModifications could not be converted

javax.naming.directory.Attributes could not be converted

javax.naming.directory.Attributes.clone could not be converted

javax.naming.directory.Attributes.isCaseIgnored could not be converted

javax.naming.directory.Attributes.put could not be converted

javax.naming.directory.Attributes.remove could not be converted

javax.naming.directory.BasicAttribute could not be converted

javax.naming.directory.BasicAttribute.attrID could not be converted

javax.naming.directory.BasicAttribute.BasicAttribute(String) could not be converted

javax.naming.directory.BasicAttribute.BasicAttribute(String, boolean) could not be converted

javax.naming.directory.BasicAttribute.BasicAttribute(String, Object) could not be converted

javax.naming.directory.BasicAttribute.BasicAttribute(String, Object, boolean) could not be converted

javax.naming.directory.BasicAttribute.clone could not be converted

javax.naming.directory.BasicAttribute.get could not be converted

javax.naming.directory.BasicAttribute.getAttributeDefinition could not be converted

javax.naming.directory.BasicAttribute.getAttributeSyntaxDefinition could not be converted

javax.naming.directory.BasicAttribute.getID could not be converted

javax.naming.directory.BasicAttribute.isOrdered could not be converted

javax.naming.directory.BasicAttribute.ordered could not be converted

javax.naming.directory.BasicAttribute.values could not be converted

javax.naming.directory.BasicAttributes could not be converted

javax.naming.directory.BasicAttributes.BasicAttributes could not be converted

javax.naming.directory.BasicAttributes.clone could not be converted

javax.naming.directory.BasicAttributes.isCaseIgnored could not be converted

javax.naming.directory.BasicAttributes.put could not be converted

javax.naming.directory.BasicAttributes.remove could not be converted

javax.naming.directory.DirContext could not be converted

javax.naming.directory.DirContext.<OperationType> could not be converted

javax.naming.directory.DirContext.bind(Name, Object, Attributes) could not be converted

javax.naming.directory.DirContext.bind(String, Object, Attributes) could not be converted

javax.naming.directory.DirContext.createSubcontext could not be converted

javax.naming.directory.DirContext.getAttributes could not be converted

javax.naming.directory.DirContext.getSchema could not be converted

javax.naming.directory.DirContext.getSchemaClassDefinition could not be converted

javax.naming.directory.DirContext.modifyAttributes could not be converted

javax.naming.directory.DirContext.rebind could not be converted

javax.naming.directory.DirContext.search could not be converted

javax.naming.directory.InitialDirContext.bind could not be converted

javax.naming.directory.InitialDirContext.createSubcontext could not be converted

javax.naming.directory.InitialDirContext.getAttributes could not be converted

javax.naming.directory.InitialDirContext.getSchema could not be converted

javax.naming.directory.InitialDirContext.getSchemaClassDefinition could not be converted

javax.naming.directory.InitialDirContext.InitialDirContext(boolean) could not be converted

javax.naming.directory.InitialDirContext.InitialDirContext(Hashtable) could not be converted

javax.naming.directory.InitialDirContext.modifyAttributes could not be converted

javax.naming.directory.InitialDirContext.rebind could not be converted

javax.naming.directory.InitialDirContext.search could not be converted

javax.naming.directory.SearchControls.getDerefLinkFlag could not be converted

javax.naming.directory.SearchControls.getReturningObjFlag could not be converted

javax.naming.directory.SearchControls.SearchControls could not be converted

javax.naming.directory.SearchControls.setDerefLinkFlag could not be converted

javax.naming.directory.SearchControls.setReturningObjFlag could not be converted

javax.naming.directory.SearchResult.getAttributes could not be converted

javax.naming.directory.SearchResult.SearchResult(String, Object, Attributes) could not be converted

javax.naming.directory.SearchResult.SearchResult(String, Object, Attributes, boolean) could not be converted

javax.naming.directory.SearchResult.SearchResult(String, String, Object, Attributes) could not be converted

javax.naming.directory.SearchResult.SearchResult(String, String, Object, Attributes, boolean) could not be converted

javax.naming.directory.SearchResult.setAttributes could not be converted

javax.naming.event.EventContext could not be converted

javax.naming.event.EventDirContext could not be converted

javax.naming.event.NamespaceChangeListener could not be converted

javax.naming.event.NamingEvent could not be converted

javax.naming.event.NamingExceptionEvent could not be converted

javax.naming.event.NamingListener could not be converted

javax.naming.event.ObjectChangeListener could not be converted

javax.naming.InitialContext.addToEnvironment could not be converted

javax.naming.InitialContext.bind could not be converted

javax.naming.InitialContext.composeName could not be converted

javax.naming.InitialContext.createSubcontext could not be converted

javax.naming.InitialContext.destroySubcontext could not be converted

javax.naming.InitialContext.getDefaultInitCtx could not be converted

javax.naming.InitialContext.getEnvironment could not be converted

javax.naming.InitialContext.getNameInNamespace could not be converted

javax.naming.InitialContext.getNameParser could not be converted

javax.naming.InitialContext.getURLOrDefaultInitCtx could not be converted

javax.naming.InitialContext.getDefault could not be converted

javax.naming.InitialContext.init could not be converted

javax.naming.InitialContext.InitialContext could not be converted

javax.naming.InitialContext.InitialContext(boolean) could not be converted

javax.naming.InitialContext.InitialContext(Hashtable) could not be converted

javax.naming.InitialContext.list could not be converted

javax.naming.InitialContext.listBindings could not be converted

javax.naming.InitialContext.lookup could not be converted

javax.naming.InitialContext.lookupLink could not be converted

javax.naming.InitialContext.myProps could not be converted

javax.naming.InitialContext.rebind could not be converted

javax.naming.InitialContext.removeFromEnvironment could not be converted

javax.naming.InitialContext.rename could not be converted

javax.naming.InitialContext.unbind could not be converted

javax.naming.Ldap.Control could not be converted

javax.naming.Ldap.ControlFactory could not be converted

javax.naming.Ldap.ExtendedRequest could not be converted

javax.naming.Ldap.ExtendedResponse could not be converted

javax.naming.Ldap.HasControls could not be converted

javax.naming.Ldap.InitialLdapContext could not be converted

javax.naming.Ldap.InitialLdapContext.extendedOperation could not be converted

javax.naming.Ldap.InitialLdapContext.getConnectControls could not be converted

javax.naming.Ldap.InitialLdapContext.getRequestControls could not be converted

javax.naming.Ldap.InitialLdapContext.getResponseControls could not be converted

javax.naming.Ldap.InitialLdapContext.InitialLdapContext could not be converted

javax.naming.Ldap.InitialLdapContext.newInstance could not be converted

javax.naming.Ldap.InitialLdapContext.reconnect could not be converted

javax.naming ldap.InitialLdapContext.setRequestControls could not be converted

javax.naming ldap.LdapContext could not be converted

javax.naming ldap.LdapContext.CONTROL_FACTORIES could not be converted

javax.naming ldap.LdapContext.extendedOperation could not be converted

javax.naming ldap.LdapContext.getConnectControls could not be converted

javax.naming ldap.LdapContext.getRequestControls could not be converted

javax.naming ldap.LdapContext.getResponseControls could not be converted

javax.naming ldap.LdapContext.newInstance could not be converted

javax.naming ldap.LdapContext.reconnect could not be converted

javax.naming ldap.LdapContext.setRequestControls could not be converted

javax.naming ldap.LdapReferralException could not be converted

javax.naming ldap.UnsolicitedNotification could not be converted

javax.naming ldap.UnsolicitedNotificationEvent could not be converted

javax.naming ldap.UnsolicitedNotificationListener could not be converted

javax.naming LinkException.getLinkRemainingName could not be converted

javax.naming LinkException.getLinkResolvedName could not be converted

javax.naming LinkException.getLinkResolvedObj could not be converted

javax.naming LinkException.linkRemainingName could not be converted

javax.naming LinkException.linkResolvedName could not be converted

javax.naming LinkException.linkResolvedObj could not be converted

javax.naming LinkException.setLinkRemainingName could not be converted

javax.naming LinkException.setLinkResolvedName could not be converted

javax.naming LinkException.setLinkResolvedObj could not be converted

javax.naming LinkException.toString could not be converted

javax.naming LinkRef could not be converted

javax.naming Name could not be converted

javax.naming NameClassPair.isRelative could not be converted

javax.naming NameClassPair.NameClassPair(String, String) could not be converted

javax.naming NameClassPair.NameClassPair(String, String, boolean) could not be converted

javax.naming NameClassPair.setClassName could not be converted

javax.naming NameClassPair.setRelative could not be converted

javax.naming NameParser could not be converted

javax.naming NamingEnumeration.close could not be converted

javax.naming NamingEnumeration.hasMore could not be converted

javax.naming NamingEnumeration.next could not be converted

javax.naming NamingException.appendRemainingComponent could not be converted

javax.naming NamingException.appendRemainingName could not be converted

javax.naming NamingException.getRemainingName could not be converted

javax.naming NamingException.getResolvedName could not be converted

javax.naming NamingException.getResolvedObj could not be converted

javax.naming.NamingException.printStackTrace could not be converted
javax.naming.NamingException.remainingName could not be converted
javax.naming.NamingException.resolvedName could not be converted
javax.naming.NamingException.resolvedObj could not be converted
javax.naming.NamingException.rootException could not be converted
javax.naming.NamingException.setRemainingName could not be converted
javax.naming.NamingException.setResolvedName could not be converted
javax.naming.NamingException.setResolvedObj could not be converted
javax.naming.NamingException.setRootCause could not be converted
javax.naming.NamingException.toString could not be converted
javax.naming.RefAddr could not be converted
javax.naming.RefAddr.addrType could not be converted
javax.naming.RefAddr.RefAddr could not be converted
javax.naming.Reference.addrs could not be converted
javax.naming.Reference.classFactory could not be converted
javax.naming.Reference.classFactoryLocation could not be converted
javax.naming.Reference.className could not be converted
javax.naming.Reference.get could not be converted
javax.naming.Reference.getClassName could not be converted
javax.naming.Reference.getFactoryClassLocation could not be converted
javax.naming.Reference.getFactoryClassName could not be converted
javax.naming.Reference.Reference could not be converted
javax.naming.Referenceable could not be converted
javax.naming.ReferralException.getReferralContext could not be converted
javax.naming.ReferralException.getReferralInfo could not be converted
javax.naming.ReferralException.retryReferral could not be converted
javax.naming.ReferralException.skipReferral could not be converted
javax.naming.spi.DirectoryManager could not be converted
javax.naming.spi.DirObjectFactory could not be converted
javax.naming.spi.DirStateFactory could not be converted
javax.naming.spi.DirStateFactory.Result could not be converted
javax.naming.spi.InitialContextFactory could not be converted
javax.naming.spi.InitialContextFactoryBuilder could not be converted
javax.naming.spi.NamingManager could not be converted
javax.naming.spi.ObjectFactory could not be converted
javax.naming.spi.ObjectFactoryBuilder could not be converted
javax.naming.spi.Resolver could not be converted
javax.naming.spi.ResolveResult could not be converted
javax.naming.spi.StateFactory could not be converted
javax.naming.StringRefAddr could not be converted

javax.naming.StringRefAddr.StringRefAddr could not be converted

Javax.rmi Error Messages

[javax.rmi.CORBA.ClassDesc could not be converted](#)

[javax.rmi.CORBA.PortableRemoteObjectDelegate could not be converted](#)

[javax.rmi.CORBA.Stub could not be converted](#)

[javax.rmi.CORBA.StubDelegate could not be converted](#)

[javax.rmi.CORBA.Tie could not be converted](#)

[javax.rmi.CORBA.Util could not be converted](#)

[javax.rmi.CORBA.UtilDelegate could not be converted](#)

[javax.rmi.CORBA.ValueHandler could not be converted](#)

[javax.rmi.PortableRemoteObject could not be converted](#)

[javax.rmi.PortableRemoteObject.narrow could not be converted](#)

Javax.security Error Messages

javax.security.auth.AuthPermission could not be converted

javax.security.auth.callback.ChoiceCallback.ChoiceCallback could not be converted

javax.security.auth.callback.ChoiceCallback.getPrompt could not be converted

javax.security.auth.callback.LanguageCallback.LanguageCallback could not be converted

javax.security.auth.callback.NameCallback.getDefaultName could not be converted

javax.security.auth.callback.NameCallback.getPrompt could not be converted

javax.security.auth.callback.NameCallback.NameCallback(String) could not be converted

javax.security.auth.callback.NameCallback.NameCallback(String, String) could not be converted

javax.security.auth.callback.PasswordCallback.getPrompt could not be converted

javax.security.auth.callback.PasswordCallback.isEchoOn could not be converted

javax.security.auth.callback.PasswordCallback.PasswordCallback could not be converted

javax.security.auth.callback.UnsupportedCallbackException could not be converted

javax.security.auth.callback.UnsupportedCallbackException.getCallback could not be converted

javax.security.auth.callback.UnsupportedCallbackException.UnsupportedCallbackException could not be converted

javax.security.auth.Destroyable.isDestroyed could not be converted

javax.security.auth.DestroyFailedException could not be converted

javax.security.auth.DestroyFailedException.DestroyFailedException could not be converted

javax.security.auth.login.AccountExpiredException could not be converted

javax.security.auth.login.AccountExpiredException.AccountExpiredException could not be converted

javax.security.auth.login.AppConfigurationEntry could not be converted

javax.security.auth.login.AppConfigurationEntry.AppConfigurationEntry could not be converted

javax.security.auth.login.AppConfigurationEntry.getControlFlag could not be converted

javax.security.auth.login.AppConfigurationEntry.getLoginModuleName could not be converted

javax.security.auth.login.Configuration could not be converted

javax.security.auth.login.Configuration.Configuration could not be converted could not be converted

javax.security.auth.login.Configuration.refresh could not be converted could not be converted

javax.security.auth.login.Configuration.setConfiguration could not be converted could not be converted

javax.security.auth.login.CredentialExpiredException could not be converted

javax.security.auth.login.CredentialExpiredException.CredentialExpiredException could not be converted

javax.security.auth.login.FailedLoginException could not be converted

javax.security.auth.login.FailedLoginException.FailedLoginException could not be converted

javax.security.auth.login.LoginContext.getSubject could not be converted

javax.security.auth.login.LoginContext.login could not be converted

javax.security.auth.login.LoginContext.LoginContext could not be converted

javax.security.auth.login.LoginContext.logout could not be converted

javax.security.auth.login.LoginException could not be converted

javax.security.auth.login.LoginException.LoginException could not be converted

javax.security.auth.Policy could not be converted

javax.security.auth.Policy.getPolicy could not be converted

javax.security.auth.Policy.Policy could not be converted

javax.security.auth.Policy.refresh could not be converted

javax.security.auth.Policy.setPolicy could not be converted

javax.security.auth.PrivateCredentialPermission could not be converted

javax.security.auth.PrivateCredentialPermission.getActions could not be converted

javax.security.auth.PrivateCredentialPermission.getCredentialClass could not be converted

javax.security.auth.PrivateCredentialPermission.getPrincipals could not be converted

javax.security.auth.PrivateCredentialPermission.implies could not be converted

javax.security.auth.PrivateCredentialPermission.newPermissionCollection could not be converted

javax.security.auth.PrivateCredentialPermission.PrivateCredentialPermission could not be converted

javax.security.auth.Refreshable could not be converted

javax.security.auth.RefreshFailedException could not be converted

javax.security.auth.RefreshFailedException.RefreshFailedException could not be converted

javax.security.auth.spi.LoginModule.abort could not be converted

javax.security.auth.spi.LoginModule.commit could not be converted

javax.security.auth.spi.LoginModule.initialize could not be converted

javax.security.auth.spi.LoginModule.login could not be converted

javax.security.auth.spi.LoginModule.logout could not be converted

javax.security.auth.Subject.doAs could not be converted

javax.security.auth.Subject.doAsPrivileged could not be converted

javax.security.auth.Subject.getPrincipals could not be converted

javax.security.auth.Subject.getPrivateCredentials could not be converted

javax.security.auth.Subject.getPublicCredentials could not be converted

javax.security.auth.Subject.getSubject could not be converted

javax.security.auth.Subject.isReadOnly could not be converted

javax.security.auth.Subject.setReadOnly could not be converted

javax.security.auth.Subject.Subject could not be converted

javax.security.auth.SubjectDomainCombiner could not be converted

javax.security.cert.Certificate.Certificate could not be converted

javax.security.cert.Certificate.getEncoded could not be converted

javax.security.cert.Certificate.verify(PublicKey) could not be converted

javax.security.cert.Certificate.verify(PublicKey, String) could not be converted

javax.security.cert.X509Certificate.getInstance could not be converted

javax.security.cert.X509Certificate.getNotAfter could not be converted

javax.security.cert.X509Certificate.getNotBefore could not be converted

javax.security.cert.X509Certificate.getSigAlgName could not be converted

javax.security.cert.X509Certificate.getVersion could not be converted

javax.security.cert.X509Certificate.X509Certificate could not be converted

Javax.servlet Error Messages

javax.servlet.FilterConfig could not be converted

javax.servlet.GenericServlet.destroy could not be converted

javax.servlet.GenericServlet.getInitParameter could not be converted

javax.servlet.GenericServlet.getInitParameterNames could not be converted

javax.servlet.GenericServlet.getServletConfig could not be converted

javax.servlet.GenericServlet.getServletContext could not be converted

javax.servlet.GenericServlet.getServletInfo could not be converted

javax.servlet.GenericServlet.getServletName could not be converted

javax.servlet.GenericServlet.init could not be converted

javax.servlet.GenericServlet.log(String) could not be converted

javax.servlet.GenericServlet.log(String, Throwable) could not be converted

javax.servlet.GenericServlet.service could not be converted

javax.servlet.HttpDummyBase.Config could not be converted

javax.servlet.HttpDummyBase.HttpDummyBase could not be converted

javax.servlet.RequestDispatcher could not be converted

javax.servlet.RequestDispatcher.forward could not be converted

javax.servlet.RequestDispatcher.include could not be converted

javax.servlet.Servlet.destroy could not be converted

javax.servlet.Servlet.getServletConfig could not be converted

javax.servlet.Servlet.getServletInfo could not be converted

javax.servlet.Servlet.init could not be converted

javax.servlet.Servlet.service could not be converted

javax.servlet.ServletConfig could not be converted

javax.servlet.ServletContext.getContext could not be converted

javax.servlet.ServletContext.getInitParameter could not be converted

javax.servlet.ServletContext.getInitParameterNames could not be converted

javax.servlet.ServletContext.getMajorVersion could not be converted

javax.servlet.ServletContext.getMimeType could not be converted

javax.servlet.ServletContext.getMinorVersion could not be converted

javax.servlet.ServletContext.getNamedDispatcher could not be converted

javax.servlet.ServletContext.getRequestDispatcher could not be converted

javax.servlet.ServletContext.getResource could not be converted

javax.servlet.ServletContext.getResourceAsStream could not be converted

javax.servlet.ServletContext.getResourcePaths could not be converted

javax.servlet.ServletContext.getServerInfo could not be converted

javax.servlet.ServletContext.getServlet could not be converted

javax.servlet.ServletContext.getServletContextName could not be converted

javax.servlet.ServletContext.getServletNames could not be converted

javax.servlet.ServletContext.getServlets could not be converted

javax.servlet.ServletContext.log could not be converted

javax.servlet.ServletContext.setAttribute could not be converted

javax.servlet.ServletContextAttributeEvent could not be converted

javax.servlet.ServletContextAttributeListener could not be converted

javax.servlet.ServletContextEvent could not be converted

javax.servlet.ServletContextListener could not be converted

javax.servlet.ServletInputStream could not be converted

javax.servlet.ServletInputStream.readLine could not be converted

javax.servlet.ServletRequest.getAttribute could not be converted

javax.servlet.ServletRequest.getAttributeNames could not be converted

javax.servlet.ServletRequest.getLocale could not be converted

javax.servlet.ServletRequest.getLocales could not be converted

javax.servlet.ServletRequest.getParameter could not be converted

javax.servlet.ServletRequest.getParameterMap could not be converted

javax.servlet.ServletRequest.getParameterNames could not be converted

javax.servlet.ServletRequest.getParameterValues could not be converted

javax.servlet.ServletRequest.getProtocol could not be converted

javax.servlet.ServletRequest.getReader could not be converted

javax.servlet.ServletRequest.getRealPath could not be converted

javax.servlet.ServletRequest.getRequestDispatcher could not be converted

javax.servlet.ServletRequest.getScheme could not be converted

javax.servlet.ServletRequest.getServerPort could not be converted

javax.servlet.ServletRequest.removeAttribute could not be converted

javax.servlet.ServletRequest.setAttribute could not be converted

javax.servlet.ServletRequest.setCharacterEncoding could not be converted

javax.servlet.ServletRequestWrapper.getAttribute could not be converted

javax.servlet.ServletRequestWrapper.getAttributeNames could not be converted

javax.servlet.ServletRequestWrapper.getLocale could not be converted

javax.servlet.ServletRequestWrapper.getLocales could not be converted

javax.servlet.ServletRequestWrapper.getParameter could not be converted

javax.servlet.ServletRequestWrapper.getParameterMap could not be converted

javax.servlet.ServletRequestWrapper.getReader could not be converted

javax.servlet.ServletRequestWrapper.getRequestDispatcher could not be converted

javax.servlet.ServletRequestWrapper.getScheme could not be converted

javax.servlet.ServletResponse could not be converted

javax.servlet.ServletResponse.flushBuffer could not be converted

javax.servlet.ServletResponse.getBufferSize could not be converted

javax.servlet.ServletResponse.getLocale could not be converted

javax.servlet.ServletResponse.isCommitted could not be converted
javax.servlet.ServletResponse.resetBuffer could not be converted
javax.servlet.ServletResponse.setBufferSize could not be converted
javax.servlet.ServletResponse.setContentLength could not be converted
javax.servlet.ServletResponseWrapper.flushBuffer could not be converted
javax.servlet.ServletResponseWrapper.getBufferSize could not be converted
javax.servlet.ServletResponseWrapper.getLocale could not be converted
javax.servlet.ServletResponseWrapper.isCommitted could not be converted
javax.servlet.ServletResponseWrapper.setBufferSize could not be converted
javax.servlet.ServletResponseWrapper.setContentLength could not be converted
javax.servlet.ServletResponseWrapper.setLocale could not be converted
javax.servlet.UnavailableException.getServlet could not be converted
javax.servlet.UnavailableException.getUnavailableSeconds could not be converted
javax.servlet.UnavailableException.isPermanent could not be converted
javax.servlet.UnavailableException.UnavailableException could not be converted

Javax.servlet.http Error Messages

[javax.servlet.http.Cookie.clone could not be converted](#)

[javax.servlet.http.Cookie.getComment could not be converted](#)

[javax.servlet.http.Cookie.getMaxAge could not be converted](#)

[javax.servlet.http.Cookie.getSecure could not be converted](#)

[javax.servlet.http.Cookie.getVersion could not be converted](#)

[javax.servlet.http.Cookie.setComment could not be converted](#)

[javax.servlet.http.Cookie.setMaxAge could not be converted](#)

[javax.servlet.http.Cookie.setSecure could not be converted](#)

[javax.servlet.http.Cookie.setVersion could not be converted](#)

[javax.servlet.http.HttpServlet.getLastModified could not be converted](#)

[javax.servlet.http.HttpServlet.HttpServlet could not be converted](#)

[javax.servlet.http.HttpServlet.service\(HttpServletRequest, HttpServletResponse\) could not be converted](#)

[javax.servlet.http.HttpServlet.service\(ServletRequest, SeervletResponse\) could not be converted](#)

[javax.servlet.http.HttpServletRequest.BASIC_AUTH could not be converted](#)

[javax.servlet.http.HttpServletRequest.CLIENT_CERT_AUTH could not be converted](#)

[javax.servlet.http.HttpServletRequest.DIGEST_AUTH could not be converted](#)

[javax.servlet.http.HttpServletRequest.FORM_AUTH could not be converted](#)

[javax.servlet.http.HttpServletRequest.getQueryString could not be converted](#)

[javax.servlet.http.HttpServletRequest.getSession could not be converted](#)

[javax.servlet.http.HttpServletRequest.isRequestedSessionIdFromCookie could not be converted](#)

[javax.servlet.http.HttpServletRequest.isRequestedSessionIdFromURL could not be converted](#)

[javax.servlet.http.HttpServletRequest.isRequestedSessionIdValid could not be converted](#)

[javax.servlet.http.HttpServletRequestWrapper.getSession could not be converted](#)

[javax.servlet.http.HttpServletRequestWrapper.isRequestedSessionIdFromCookie could not be converted](#)

[javax.servlet.http.HttpServletRequestWrapper.isRequestedSessionIdFromURL could not be converted](#)

[javax.servlet.http.HttpServletRequestWrapper.isRequestedSessionIdValid could not be converted](#)

[javax.servlet.http.HttpServletResponse.addDateHeader could not be converted](#)

[javax.servlet.http.HttpServletResponse.containsHeader could not be converted](#)

[javax.servlet.http.HttpServletResponse.encodeRedirectURL could not be converted](#)

[javax.servlet.http.HttpServletResponse.encodeURL could not be converted](#)

[javax.servlet.http.HttpServletResponse.SC_TEMPORARY_REDIRECT could not be converted](#)

[javax.servlet.http.HttpServletResponse.setDateHeader could not be converted](#)

[javax.servlet.http.HttpServletResponseWrapper.containsHeader could not be converted](#)

[javax.servlet.http.HttpServletResponseWrapper.setDateHeader could not be converted](#)

[javax.servlet.http.HttpServletResponseWrapper.setHeader could not be converted](#)

[javax.servlet.http.HttpServletResponseWrapper.setIntHeader could not be converted](#)

[javax.servlet.http.HttpSession.getCreationTime could not be converted](#)

javax.servlet.http.HttpSession.getLastAccessedTime could not be converted
javax.servlet.http.HttpSession.getMaxInactiveInterval could not be converted
javax.servlet.http.HttpSession.getSessionContext could not be converted
javax.servlet.http.HttpSession.getValue could not be converted
javax.servlet.http.HttpSession.invalidate could not be converted
javax.servlet.http.HttpSession.putValue could not be converted
javax.servlet.http.HttpSession.removeValue could not be converted
javax.servlet.http.HttpSession.setMaxInactiveInterval could not be converted
javax.servlet.http.HttpSessionActivationListener could not be converted
javax.servlet.http.HttpSessionAttributeListener could not be converted
javax.servlet.http.HttpSessionBindingEvent could not be converted
javax.servlet.http.HttpSessionBindingListener could not be converted
javax.servlet.http.HttpSessionContext could not be converted
javax.servlet.http.HttpSessionEvent could not be converted
javax.servlet.http.HttpSessionListener could not be converted
javax.servlet.http.HttpUtils could not be converted

Javax.servlet.jsp Error Messages

javax.servlet.jsp.HttpJspPage could not be converted

javax.servlet.jsp.JspEngineInfo could not be converted

javax.servlet.jsp.JspFactory could not be converted

javax.servlet.jsp.JspPage could not be converted

javax.servlet.jsp.JspWriter.bufferSize could not be converted

javax.servlet.jsp.JspWriter.DEFAULT_BUFFER could not be converted

javax.servlet.jsp.JspWriter.getBufferSize could not be converted

javax.servlet.jsp.JspWriter.getRemaining could not be converted

javax.servlet.jsp.JspWriter.NO_BUFFER could not be converted

javax.servlet.jsp.JspWriter.UNBOUNDED_BUFFER could not be converted

javax.servlet.jsp.PageContext.APPLICATION could not be converted

javax.servlet.jsp.PageContext.APPLICATION_SCOPE could not be converted

javax.servlet.jsp.PageContext.CONFIG could not be converted

javax.servlet.jsp.PageContext.EXCEPTION could not be converted

javax.servlet.jsp.PageContext.findAttribute could not be converted

javax.servlet.jsp.PageContext.getAttribute could not be converted

javax.servlet.jsp.PageContext.getAttributeNamesInScope could not be converted

javax.servlet.jsp.PageContext.getAttributesScope could not be converted

javax.servlet.jsp.PageContext.getServletConfig could not be converted

javax.servlet.jsp.PageContext.handlePageException could not be converted

javax.servlet.jsp.PageContext.initialize could not be converted

javax.servlet.jsp.PageContext.OUT could not be converted

javax.servlet.jsp.PageContext.PAGE could not be converted

javax.servlet.jsp.PageContext.PageContext could not be converted

javax.servlet.jsp.PageContext.PAGECONTEXT could not be converted

javax.servlet.jsp.PageContext.PAGE_SCOPE could not be converted

javax.servlet.jsp.PageContext.popBody could not be converted

javax.servlet.jsp.PageContext.pushBody could not be converted

javax.servlet.jsp.PageContext.release could not be converted

javax.servlet.jsp.PageContext.removeAttribute could not be converted

javax.servlet.jsp.PageContext.REQUEST could not be converted

javax.servlet.jsp.PageContext.REQUEST_SCOPE could not be converted

javax.servlet.jsp.PageContext.RESPONSE could not be converted

javax.servlet.jsp.PageContext.SESSION could not be converted

javax.servlet.jsp.PageContext.SESSION_SCOPE could not be converted

javax.servlet.jsp.PageContext.setAttribute could not be converted

javax.servlet.jsp.tagext.BodyContent.flush could not be converted

javax.servlet.jsp.tagext.PageData could not be converted
javax.servlet.jsp.tagext.Tag.setParent could not be converted
javax.servlet.jsp.tagext.TagAttributeInfo could not be converted
javax.servlet.jsp.tagext.TagData could not be converted
javax.servlet.jsp.tagext.TagExtraInfo could not be converted
javax.servlet.jsp.tagext.TagInfo could not be converted
javax.servlet.jsp.tagext.TagLibraryInfo could not be converted
javax.servlet.jsp.tagext.TagLibraryValidator could not be converted
javax.servlet.jsp.tagext.TagSupport.setParent could not be converted
javax.servlet.jsp.tagext.TagVariableInfo could not be converted
javax.servlet.jsp.tagext.TryCatchFinally could not be converted
javax.servlet.jsp.tagext.ValidationMessage could not be converted
javax.servlet.jsp.tagext.VariableInfo could not be converted

Javax.sound Error Messages

[javax.sound.midi.ControllerEventListener](#) could not be converted

[javax.sound.midi.Instrument](#) could not be converted

[javax.sound.midi.InvalidMidiDataException](#) could not be converted

[javax.sound.midi.MetaEventListener](#) could not be converted

[javax.sound.midi.MetaMessage](#) could not be converted

[javax.sound.midi.MidiChannel](#) could not be converted

[javax.sound.midi.MidiDevice](#) could not be converted

[javax.sound.midi.MidiDevice.Info](#) could not be converted

[javax.sound.midi.MidiEvent](#) could not be converted

[javax.sound.midi.MidiFileFormat](#) could not be converted

[javax.sound.midi.MidiMessage](#) could not be converted

[javax.sound.midi.MidiSystem](#) could not be converted

[javax.sound.midi.MidiUnavailableException](#) could not be converted

[javax.sound.midi.Patch](#) could not be converted

[javax.sound.midi.Receiver](#) could not be converted

[javax.sound.midi.Sequence](#) could not be converted

[javax.sound.midi.Sequencer](#) could not be converted

[javax.sound.midi.Sequencer.SyncMode](#) could not be converted

[javax.sound.midi.ShortMessage](#) could not be converted

[javax.sound.midi.Soundbank](#) could not be converted

[javax.sound.midi.SoundbankResource](#) could not be converted

[javax.sound.midi.spi.MidiDeviceProvider](#) could not be converted

[javax.sound.midi.spi.MidiFileReader](#) could not be converted

[javax.sound.midi.spi.MidiFileWriter](#) could not be converted

[javax.sound.midi.spi.SoundbankReader](#) could not be converted

[javax.sound.midi.Synthesizer](#) could not be converted

[javax.sound.midi.SysexMessage](#) could not be converted

[javax.sound.midi.Track](#) could not be converted

[javax.sound.midi.Transmitter](#) could not be converted

[javax.sound.midi.VoiceStatus](#) could not be converted

[javax.sound.sampled.AudioFileFormat](#) could not be converted

[javax.sound.sampled.AudioFileFormat.Type](#) could not be converted

[javax.sound.sampled.AudioFormat.bigEndian](#) could not be converted

[javax.sound.sampled.AudioFormat.Encoding.<EncodingType>](#) could not be converted

[javax.sound.sampled.AudioFormat.Encoding.Encoding](#) could not be converted

[javax.sound.sampled.AudioFormat.isBigEndian](#) could not be converted

[javax.sound.sampled.AudioFormat.matches](#) could not be converted

javax.sound.sampled.AudioInputStream.AudioInputStream could not be converted

javax.sound.sampled.AudioInputStream.available could not be converted

javax.sound.sampled.AudioInputStream.format could not be converted

javax.sound.sampled.AudioInputStream.frameLength could not be converted

javax.sound.sampled.AudioInputStream.framePos could not be converted

javax.sound.sampled.AudioInputStream.frameSize could not be converted

javax.sound.sampled.AudioInputStream.getFormat could not be converted

javax.sound.sampled.AudioInputStream.getFrameLength could not be converted

javax.sound.sampled.AudioInputStream.mark could not be converted

javax.sound.sampled.AudioInputStream.markSupported could not be converted

javax.sound.sampled.AudioPermission could not be converted

javax.sound.sampled.AudioSystem could not be converted

javax.sound.sampled.BooleanControl could not be converted

javax.sound.sampled.BooleanControl.Type could not be converted

javax.sound.sampled.Clip.getFrameLength could not be converted

javax.sound.sampled.Clip.getMicrosecondLength could not be converted

javax.sound.sampled.Clip.loop could not be converted

javax.sound.sampled.Clip.open could not be converted

javax.sound.sampled.Clip.setLoopPoints could not be converted

javax.sound.sampled.Clip.setMicrosecondPosition could not be converted

javax.sound.sampled.CompoundControl could not be converted

javax.sound.sampled.CompoundControl.Type could not be converted

javax.sound.sampled.Control could not be converted

javax.sound.sampled.Control.Type could not be converted

javax.sound.sampled.DataLine.available could not be converted

javax.sound.sampled.DataLine.drain could not be converted

javax.sound.sampled.DataLine.flush could not be converted

javax.sound.sampled.DataLine.getMicrosecondPosition could not be converted

javax.sound.sampled.DataLine.Info could not be converted

javax.sound.sampled.DataLine.start could not be converted

javax.sound.sampled.EnumControl could not be converted

javax.sound.sampled.EnumControl.Type could not be converted

javax.sound.sampled.FloatControl could not be converted

javax.sound.sampled.FloatControl.Type could not be converted

javax.sound.sampled.Line could not be converted

javax.sound.sampled.Line.Info could not be converted

javax.sound.sampled.LineEvent could not be converted

javax.sound.sampled.LineEvent.Type could not be converted

javax.sound.sampled.LineListener could not be converted

javax.sound.sampled.Mixer could not be converted

javax.sound.sampled.Mixer.Info could not be converted

javax.sound.sampled.Port could not be converted

javax.sound.sampled.Port.Info could not be converted

javax.sound.sampled.ReverbType could not be converted

javax.sound.sampled.SourceDataLine could not be converted

javax.sound.sampled.SourceDataLine.open could not be converted

javax.sound.sampled.SourceDataLine.write could not be converted

javax.sound.sampled.spi.AudioFileReader could not be converted

javax.sound.sampled.spi.AudioFileWriter could not be converted

javax.sound.sampled.spi.FormatConversionProvider could not be converted

javax.sound.sampled.spi.MixerProvider could not be converted

javax.sound.sampled.TargetDataLine could not be converted

javax.sound.sampled.TargetDataLine.open could not be converted

javax.sound.sampled.TargetDataLine.read could not be converted

Javax.sql Error Messages

[javax.sql.ConnectionEvent could not be converted](#)

[javax.sql.ConnectionEventListener could not be converted](#)

[javax.sql.ConnectionPoolDataSource could not be converted](#)

[javax.sql.DataSource could not be converted](#)

[javax.sql.PooledConnection.addConnectionEventListener could not be converted](#)

[javax.sql.PooledConnection.removeConnectionEventListener could not be converted](#)

[javax.sql.RowSet.addRowSetListener could not be converted](#)

[javax.sql.RowSet.getEscapeProcessing could not be converted](#)

[javax.sql.RowSet.getMaxFieldSize could not be converted](#)

[javax.sql.RowSet.getMaxRows could not be converted](#)

[javax.sql.RowSet.getPassword could not be converted](#)

[javax.sql.RowSet.getTypeMap could not be converted](#)

[javax.sql.RowSet.getUrl could not be converted](#)

[javax.sql.RowSet.getUsername could not be converted](#)

[javax.sql.RowSet.isReadOnly could not be converted](#)

[javax.sql.RowSet.removeRowSetListener could not be converted](#)

[javax.sql.RowSet.setAsciiStream could not be converted](#)

[javax.sql.RowSet.setBinaryStream could not be converted](#)

[javax.sql.RowSet.setBlob could not be converted](#)

[javax.sql.RowSet.setCharacterStream could not be converted](#)

[javax.sql.RowSet.setClob could not be converted](#)

[javax.sql.RowSet.setConcurrency could not be converted](#)

[javax.sql.RowSet.setDataSourceName could not be converted](#)

[javax.sql.RowSet.setEscapeProcessing could not be converted](#)

[javax.sql.RowSet.setMaxFieldSize could not be converted](#)

[javax.sql.RowSet.setMaxRows could not be converted](#)

[javax.sql.RowSet.setObject could not be converted](#)

[javax.sql.RowSet.setPassword could not be converted](#)

[javax.sql.RowSet.setReadOnly could not be converted](#)

[javax.sql.RowSet.setType could not be converted](#)

[javax.sql.RowSet.setTypeMap could not be converted](#)

[javax.sql.RowSet.setUrl could not be converted](#)

[javax.sql.RowSet.setUsername could not be converted](#)

[javax.sql.RowSetEvent could not be converted](#)

[javax.sql.RowSetInternal could not be converted](#)

[javax.sql.RowSetListener could not be converted](#)

[javax.sql.RowSetMetaData.setCatalogName could not be converted](#)

javax.sql.RowSetMetaData.setColumnCount could not be converted

javax.sql.RowSetMetaData.setCurrency could not be converted

javax.sql.RowSetMetaData.setPrecision could not be converted

javax.sql.RowSetMetaData.setScale could not be converted

javax.sql.RowSetMetaData.setSearchable could not be converted

javax.sql.RowSetMetaData.setSigned could not be converted

javax.sql.RowSetReader could not be converted

javax.sql.RowSetWriter could not be converted

javax.sql.XAConnection could not be converted

javax.sql.XAConnection.getXAResource could not be converted

javax.sql.XADataSource could not be converted

Javax.swing Error Messages

javax.swing.<ClassName>.addNotify could not be converted

javax.swing.<ClassName>.process*Event could not be converted

javax.swing.<ClassName>.removeNotify could not be converted

javax.swing.<ClassName>.setLayout could not be converted

javax.swing.<ClassName>.BeanInfo could not be converted

javax.swing.AbstractAction.AbstractAction(String) could not be converted

javax.swing.AbstractAction.AbstractAction(String, Icon) could not be converted

javax.swing.AbstractAction.addPropertyChangeListener could not be converted

javax.swing.AbstractAction.changeSupport could not be converted

javax.swing.AbstractAction.enabled could not be converted

javax.swing.AbstractAction.firePropertyChange could not be converted

javax.swing.AbstractAction.getKeys could not be converted

javax.swing.AbstractAction.getValue could not be converted

javax.swing.AbstractAction.isEnabled could not be converted

javax.swing.AbstractAction.putValue could not be converted

javax.swing.AbstractAction.removePropertyChangeListener could not be converted

javax.swing.AbstractAction.setEnabled could not be converted

javax.swing.AbstractButton.<ChangedProperty> could not be converted

javax.swing.AbstractButton.AbstractButton could not be converted

javax.swing.AbstractButton.actionListener could not be converted

javax.swing.AbstractButton.addChangeListener could not be converted

javax.swing.AbstractButton.changeEvent could not be converted

javax.swing.AbstractButton.changeListener could not be converted

javax.swing.AbstractButton.checkHorizontalKey could not be converted

javax.swing.AbstractButton.checkVerticalKey could not be converted

javax.swing.AbstractButton.configurePropertiesFromAction could not be converted

javax.swing.AbstractButton.createActionListener could not be converted

javax.swing.AbstractButton.createActionPropertyChangeListener could not be converted

javax.swing.AbstractButton.createChangeListener could not be converted

javax.swing.AbstractButton.createItemListener could not be converted

javax.swing.AbstractButton.doClick could not be converted

javax.swing.AbstractButton.fireItemStateChanged could not be converted

javax.swing.AbstractButton.fireStateChanged could not be converted

javax.swing.AbstractButton.get<State>Icon could not be converted

javax.swing.AbstractButton.getAction could not be converted

javax.swing.AbstractButton.getActionCommand could not be converted

javax.swing.AbstractButton.getHorizontalAlignment could not be converted

javax.swing.AbstractButton.getHorizontalTextPosition could not be converted

javax.swing.AbstractButton.getIcon could not be converted

javax.swing.AbstractButton.getMargin could not be converted

javax.swing.AbstractButton.getMnemonic could not be converted

javax.swing.AbstractButton.getModel could not be converted

javax.swing.AbstractButton.getSelectedObjects could not be converted

javax.swing.AbstractButton.getUI could not be converted

javax.swing.AbstractButton.getVerticalAlignment could not be converted

javax.swing.AbstractButton.getVerticalTextPosition could not be converted

javax.swing.AbstractButton.imageUpdate could not be converted

javax.swing.AbstractButton.init could not be converted

javax.swing.AbstractButton.isBorderPainted could not be converted

javax.swing.AbstractButton.isContentAreaFilled could not be converted

javax.swing.AbstractButton.isFocusPainted could not be converted

javax.swing.AbstractButton.isRolloverEnabled could not be converted

javax.swing.AbstractButton.itemListener could not be converted

javax.swing.AbstractButton.model could not be converted

javax.swing.AbstractButton.paintBorder could not be converted

javax.swing.AbstractButton.removeChangeListener could not be converted

javax.swing.AbstractButton.set<State>Icon could not be converted

javax.swing.AbstractButton.setActionCommand could not be converted

javax.swing.AbstractButton.setBorderPainted could not be converted

javax.swing.AbstractButton.setContentAreaFilled could not be converted

javax.swing.AbstractButton.setFocusPainted could not be converted

javax.swing.AbstractButton.setHorizontalAlignment could not be converted

javax.swing.AbstractButton.setHorizontalTextPosition could not be converted

javax.swing.AbstractButton.setIcon could not be converted

javax.swing.AbstractButton.setMargin could not be converted

javax.swing.AbstractButton.setMnemonic could not be converted

javax.swing.AbstractButton.setModel could not be converted

javax.swing.AbstractButton.setRolloverEnabled could not be converted

javax.swing.AbstractButton.setSelected could not be converted

javax.swing.AbstractButton.setUI could not be converted

javax.swing.AbstractButton.setVerticalAlignment could not be converted

javax.swing.AbstractButton.setVerticalTextPosition could not be converted

javax.swing.AbstractButton.updateUI could not be converted

javax.swing.AbstractCellEditor could not be converted

javax.swing.AbstractListModel could not be converted

javax.swing.AbstractListModel.addListDataListener could not be converted

javax.swing.AbstractListModel.fireContentsChanged could not be converted

javax.swing.AbstractListModel.fireIntervalAdded could not be converted

javax.swing.AbstractListModel.fireIntervalRemoved could not be converted

javax.swing.AbstractListModel.listeners could not be converted

javax.swing.AbstractListModel.listenerList could not be converted

javax.swing.AbstractListModel.removeListDataListener could not be converted

javax.swing.Action.ACCELERATOR_KEY could not be converted

javax.swing.Action.ACTION_COMMAND_KEY could not be converted

javax.swing.Action.addPropertyChangeListener could not be converted

javax.swing.Action.DEFAULT could not be converted

javax.swing.Action.getValue could not be converted

javax.swing.Action.isEnabled could not be converted

javax.swing.Action.LONG_DESCRIPTION could not be converted

javax.swing.Action.MNEMONIC_KEY could not be converted

javax.swing.Action.NAME could not be converted

javax.swing.Action.putValue could not be converted

javax.swing.Action.removePropertyChangeListener could not be converted

javax.swing.Action.setEnabled could not be converted

javax.swing.Action.SHORT_DESCRIPTION could not be converted

javax.swing.Action.SMALL_ICON could not be converted

javax.swing.ActionMap could not be converted

javax.swing.ActionMap.allKeys could not be converted

javax.swing.ActionMap.get could not be converted

javax.swing.ActionMap.getParent could not be converted

javax.swing.ActionMap.keys could not be converted

javax.swing.ActionMap.setParent could not be converted

javax.swing.ActionMap.size could not be converted

javax.swing.BorderFactory could not be converted

javax.swing.BorderFactory.BorderFactory could not be converted

javax.swing.BorderFactory.createCompoundBorder could not be converted

javax.swing.BorderFactory.createEtchedBorder could not be converted

javax.swing.BorderFactory.createMatteBorder could not be converted

javax.swing.BorderFactory.createTitledBorder could not be converted

javax.swing.BoundedRangeModel could not be converted

javax.swing.Box could not be converted

javax.swing.Box.Filler could not be converted

javax.swing.BoxLayout could not be converted

javax.swing.ButtonGroup could not be converted

javax.swing.ButtonGroup.add could not be converted

javax.swing.ButtonModel could not be converted

javax.swing.ButtonModel.addActionListener could not be converted

javax.swing.ButtonModel.addChangeListener could not be converted

javax.swing.ButtonModel.addItemListener could not be converted

javax.swing.ButtonModel.getActionCommand could not be converted

javax.swing.ButtonModel.getMnemonic could not be converted

javax.swing.ButtonModel.isArmed could not be converted

javax.swing.ButtonModel.isPressed could not be converted

javax.swing.ButtonModel.isRollover could not be converted

javax.swing.ButtonModel.isSelected could not be converted

javax.swing.ButtonModel.removeActionListener could not be converted

javax.swing.ButtonModel.removeChangeListener could not be converted

javax.swing.ButtonModel.removeItemListener could not be converted

javax.swing.ButtonModel.setActionCommand could not be converted

javax.swing.ButtonModel.setArmed could not be converted

javax.swing.ButtonModel.setGroup could not be converted

javax.swing.ButtonModel.setMnemonic could not be converted

javax.swing.ButtonModel.setPressed could not be converted

javax.swing.ButtonModel.setRollover could not be converted

javax.swing.ButtonModel.setSelected could not be converted

javax.swing.CellEditor could not be converted

javax.swing.CellRendererPane could not be converted

javax.swing.ComboBoxEditor could not be converted

javax.swing.ComboBoxModel.getSelectedItem could not be converted

javax.swing.ComboBoxModel.setSelectedItem could not be converted

javax.swing.ComponentInputMap could not be converted

javax.swing.DebugGraphics could not be converted

javax.swing.DebugGraphics.BUFFERED_OPTION could not be converted

javax.swing.DebugGraphics.clearRect could not be converted

javax.swing.DebugGraphics.clipRect could not be converted

javax.swing.DebugGraphics.copyArea could not be converted

javax.swing.DebugGraphics.create could not be converted

javax.swing.DebugGraphics.DebugGraphics could not be converted

javax.swing.DebugGraphics.DebugGraphics(Graphics) could not be converted

javax.swing.DebugGraphics.DebugGraphics (Graphics, IComponent) could not be converted

javax.swing.DebugGraphics.dispose could not be converted

javax.swing.DebugGraphics.draw3DRect could not be converted

javax.swing.DebugGraphics.drawArc could not be converted

javax.swing.DebugGraphics.drawBytes could not be converted

javax.swing.DebugGraphics.drawChars could not be converted

javax.swing.DebugGraphics.drawImage could not be converted

javax.swing.DebugGraphics.drawLine could not be converted

javax.swing.DebugGraphics.drawOval could not be converted

javax.swing.DebugGraphics.drawPolygon could not be converted

javax.swing.DebugGraphics.drawPolyline could not be converted

javax.swing.DebugGraphics.drawRect could not be converted

javax.swing.DebugGraphics.drawRoundRect could not be converted

javax.swing.DebugGraphics.drawString(AttributedCharacterIterator, int, int) could not be converted

javax.swing.DebugGraphics.drawString(String, int, int) could not be converted

javax.swing.DebugGraphics.fill3DRect could not be converted

javax.swing.DebugGraphics.fillArc could not be converted

javax.swing.DebugGraphics.fillOval could not be converted

javax.swing.DebugGraphics.fillPolygon could not be converted

javax.swing.DebugGraphics.fillRect could not be converted

javax.swing.DebugGraphics.fillRoundRect could not be converted

javax.swing.DebugGraphics.FLASH_OPTION could not be converted

javax.swing.DebugGraphics.flashColor could not be converted

javax.swing.DebugGraphics.flashCount could not be converted

javax.swing.DebugGraphics.flashTime could not be converted

javax.swing.DebugGraphics.getClip could not be converted

javax.swing.DebugGraphics.getClipBounds could not be converted

javax.swing.DebugGraphics.getColor could not be converted

javax.swing.DebugGraphics.getDebugOptions could not be converted

javax.swing.DebugGraphics.getFont could not be converted

javax.swing.DebugGraphics.getFontMetrics could not be converted

javax.swing.DebugGraphics.isDrawingBuffer could not be converted

javax.swing.DebugGraphics.LOG_OPTION could not be converted

javax.swing.DebugGraphics.logStream could not be converted

javax.swing.DebugGraphics.NONE_OPTION could not be converted

javax.swing.DebugGraphics.setClip could not be converted

javax.swing.DebugGraphics.setColor could not be converted

javax.swing.DebugGraphics.setDebugOptions could not be converted

javax.swing.DebugGraphics.setFlashColor could not be converted

javax.swing.DebugGraphics.setFlashCount could not be converted

javax.swing.DebugGraphics.setFlashTime could not be converted

javax.swing.DebugGraphics.setFont could not be converted

javax.swing.DebugGraphics.setLogStream could not be converted

javax.swing.DebugGraphics.setPaintMode could not be converted

javax.swing.DebugGraphics.setXORMode could not be converted

javax.swing.DebugGraphics.translate could not be converted

javax.swing.DefaultBoundedRangeModel could not be converted

javax.swing.DefaultButtonModel could not be converted

javax.swing.DefaultButtonModel.actionCommand could not be converted
javax.swing.DefaultButtonModel.addActionListener could not be converted
javax.swing.DefaultButtonModel.addChangeListener could not be converted
javax.swing.DefaultButtonModel.addItemListener could not be converted
javax.swing.DefaultButtonModel.ARMED could not be converted
javax.swing.DefaultButtonModel.changeEvent could not be converted
javax.swing.DefaultButtonModel.DefaultButtonModel could not be converted
javax.swing.DefaultButtonModel.ENABLED could not be converted
javax.swing.DefaultButtonModel.fireActionPerformed could not be converted
javax.swing.DefaultButtonModel.fireItemStateChanged could not be converted
javax.swing.DefaultButtonModel.fireStateChanged could not be converted
javax.swing.DefaultButtonModel.getActionCommand could not be converted
javax.swing.DefaultButtonModel.getGroup could not be converted
javax.swing.DefaultButtonModel.getListeners could not be converted
javax.swing.DefaultButtonModel.getMnemonic could not be converted
javax.swing.DefaultButtonModel.getSelectedObjects could not be converted
javax.swing.DefaultButtonModel.group could not be converted
javax.swing.DefaultButtonModel.isArmed could not be converted
javax.swing.DefaultButtonModel.isPressed could not be converted
javax.swing.DefaultButtonModel.isRollover could not be converted
javax.swing.DefaultButtonModel.isSelected could not be converted
javax.swing.DefaultButtonModel.listenerList could not be converted
javax.swing.DefaultButtonModel.mnemonic could not be converted
javax.swing.DefaultButtonModel.PRESSED could not be converted
javax.swing.DefaultButtonModel.removeActionListener could not be converted
javax.swing.DefaultButtonModel.removeChangeListener could not be converted
javax.swing.DefaultButtonModel.removeItemListener could not be converted
javax.swing.DefaultButtonModel.ROLLOVER could not be converted
javax.swing.DefaultButtonModel.SELECTED could not be converted
javax.swing.DefaultButtonModel.setActionCommand could not be converted
javax.swing.DefaultButtonModel.setArmed could not be converted
javax.swing.DefaultButtonModel.setGroup could not be converted
javax.swing.DefaultButtonModel.setMnemonic could not be converted
javax.swing.DefaultButtonModel.setPressed could not be converted
javax.swing.DefaultButtonModel.setRollover could not be converted
javax.swing.DefaultButtonModel.setSelected could not be converted
javax.swing.DefaultButtonModel.stateMask could not be converted
javax.swing.DefaultCellEditor could not be converted
javax.swing.DefaultComboBoxModel could not be converted
javax.swing.DefaultComboBoxModel.DefaultComboBoxModel could not be converted

javax.swing.DefaultComboBoxModel.DefaultComboBoxModel(Object[]) could not be converted
javax.swing.DefaultComboBoxModel.DefaultComboBoxModel(Vector) could not be converted
javax.swing.DefaultComboBoxModel.getSelectedltem could not be converted
javax.swing.DefaultComboBoxModel.setSelectedltem could not be converted
javax.swing.DefaultDesktopManager could not be converted
javax.swing.DefaultDesktopManager.deiconifyFrame could not be converted
javax.swing.DefaultDesktopManager.iconifyFrame could not be converted
javax.swing.DefaultDesktopManager.maximizeFrame could not be converted
javax.swing.DefaultDesktopManager.minimizeFrame could not be converted
javax.swing.DefaultFocusManager could not be converted
javax.swing.DefaultListCellRenderer could not be converted
javax.swing.DefaultListCellRenderer.UIResource could not be converted
javax.swing.DefaultListModel could not be converted
javax.swing.DefaultListModel.capacity could not be converted
javax.swing.DefaultListModel.ensureCapacity could not be converted
javax.swing.DefaultListModel.setSize could not be converted
javax.swing.DefaultListModel.trimToSize could not be converted
javax.swing.DefaultListSelectionModel could not be converted
javax.swing.DefaultListSelectionModel.addListSelectionListener could not be converted
javax.swing.DefaultListSelectionModel.addSelectionInterval could not be converted
javax.swing.DefaultListSelectionModel.clone could not be converted
javax.swing.DefaultListSelectionModel.DefaultListSelectionModel could not be converted
javax.swing.DefaultListSelectionModel.fireValueChanged(boolean) could not be converted
javax.swing.DefaultListSelectionModel.fireValueChanged(int, int) could not be converted
javax.swing.DefaultListSelectionModel.fireValueChanged(int, int, boolean) could not be converted
javax.swing.DefaultListSelectionModel.getAnchorSelectionIndex could not be converted
javax.swing.DefaultListSelectionModel.getLeadSelectionIndex could not be converted
javax.swing.DefaultListSelectionModel.getListeners could not be converted
javax.swing.DefaultListSelectionModel.getMaxSelectionIndex could not be converted
javax.swing.DefaultListSelectionModel.getMinSelectionIndex could not be converted
javax.swing.DefaultListSelectionModel.getSelectionMode could not be converted
javax.swing.DefaultListSelectionModel.getValuelsAdjusting could not be converted
javax.swing.DefaultListSelectionModel.insertIndexInterval could not be converted
javax.swing.DefaultListSelectionModel.isLeadAnchorNotificationEnabled could not be converted
javax.swing.DefaultListSelectionModel.leadAnchorNotificationEnabled could not be converted
javax.swing.DefaultListSelectionModel.listenerList could not be converted
javax.swing.DefaultListSelectionModel.removeIndexInterval could not be converted
javax.swing.DefaultListSelectionModel.removeListSelectionListener could not be converted
javax.swing.DefaultListSelectionModel.removeSelectionInterval could not be converted
javax.swing.DefaultListSelectionModel.setAnchorSelectionIndex could not be converted

javax.swing.DefaultListSelectionModel.setLeadAnchorNotificationEnabled could not be converted

javax.swing.DefaultListSelectionModel.setLeadSelectionIndex could not be converted

javax.swing.DefaultListSelectionModel.setSelectionInterval could not be converted

javax.swing.DefaultListSelectionModel.setSelectionMode could not be converted

javax.swing.DefaultListSelectionModel.setValuesAdjusting could not be converted

javax.swing.DefaultSingleSelectionModel could not be converted

javax.swing.DefaultSingleSelectionModel.addChangeListener could not be converted

javax.swing.DefaultSingleSelectionModel.changeEvent could not be converted

javax.swing.DefaultSingleSelectionModel.DefaultSingleSelectionModel could not be converted

javax.swing.DefaultSingleSelectionModel.fireStateChanged could not be converted

javax.swing.DefaultSingleSelectionModel.listeners could not be converted

javax.swing.DefaultSingleSelectionModel.listenerList could not be converted

javax.swing.DefaultSingleSelectionModel.removeChangeListener could not be converted

javax.swing.DesktopManager could not be converted

javax.swing.DesktopManager.deiconifyFrame could not be converted

javax.swing.DesktopManager.iconifyFrame could not be converted

javax.swing.DesktopManager.maximizeFrame could not be converted

javax.swing.DesktopManager.minimizeFrame could not be converted

javax.swing.FocusManager could not be converted

javax.swing.GrayFilter could not be converted

javax.swing.GrayFilter.createDisabledImage could not be converted

javax.swing.Icon.paintIcon could not be converted

javax.swing.ImageIcon could not be converted

javax.swing.ImageIcon.component could not be converted

javax.swing.ImageIcon.getAccessibleContext could not be converted

javax.swing.ImageIcon.getDescription could not be converted

javax.swing.ImageIcon.getImageLoadStatus could not be converted

javax.swing.ImageIcon.getImageObserver could not be converted

javax.swing.ImageIcon.ImageIcon could not be converted

javax.swing.ImageIcon.ImageIcon(byte[]) could not be converted

javax.swing.ImageIcon.ImageIcon(byte[], String) could not be converted

javax.swing.ImageIcon.ImageIcon(Image) could not be converted

javax.swing.ImageIcon.ImageIcon(Image, String) could not be converted

javax.swing.ImageIcon.ImageIcon(String) could not be converted

javax.swing.ImageIcon.ImageIcon(String, String) could not be converted

javax.swing.ImageIcon.ImageIcon(URL) could not be converted

javax.swing.ImageIcon.ImageIcon(URL, String) could not be converted

javax.swing.ImageIcon.loadImage could not be converted

javax.swing.ImageIcon.paintIcon could not be converted

javax.swing.ImageIcon.setDescription could not be converted

javax.swing.ImageIcon.setImageObserver could not be converted

javax.swing.ImageIcon.tracker could not be converted

javax.swing.InputMap could not be converted

javax.swing.InputMap.allKeys could not be converted

javax.swing.InputMap.get could not be converted

javax.swing.InputMap.getParent could not be converted

javax.swing.InputMap.keys could not be converted

javax.swing.InputMap.setParent could not be converted

javax.swing.InputVerifier could not be converted

javax.swing.JApplet could not be converted

javax.swing.JApplet.accessibleContext could not be converted

javax.swing.JApplet.addImpl could not be converted

javax.swing.JApplet.createRootPane could not be converted

javax.swing.JApplet.getGlassPane could not be converted

javax.swing.JApplet.getJMenuBar could not be converted

javax.swing.JApplet.getLayeredPane could not be converted

javax.swing.JApplet.getRootPane could not be converted

javax.swing.JApplet.isRootPaneCheckingEnabled could not be converted

javax.swing.JApplet.paramString could not be converted

javax.swing.JApplet.rootPane could not be converted

javax.swing.JApplet.rootPaneCheckingEnabled could not be converted

javax.swing.JApplet.setContentPane could not be converted

javax.swing.JApplet.setGlassPane could not be converted

javax.swing.JApplet.setJMenuBar could not be converted

javax.swing.JApplet.setLayeredPane could not be converted

javax.swing.JApplet.setRootPane could not be converted

javax.swing.JApplet.setRootPaneCheckingEnabled could not be converted

javax.swing.JButton.configurePropertiesFromAction could not be converted

javax.swing.JButton.isDefaultButton could not be converted

javax.swing.JButton.isDefaultCapable could not be converted

javax.swing.JButton.JButton could not be converted

javax.swing.JButton.paramString could not be converted

javax.swing.JButton.setDefaultCapable could not be converted

javax.swing.JButton.updateUI could not be converted

javax.swing.JCheckBox.configurePropertiesFromAction could not be converted

javax.swing.JCheckBox.createActionPropertyChangeListener could not be converted

javax.swing.JCheckBox.JCheckBox could not be converted

javax.swing.JCheckBox.updateUI could not be converted

javax.swing.JCheckBoxMenuItem.getAccessibleContext could not be converted

javax.swing.JCheckBoxMenuItem.getUIClassID could not be converted

javax.swing.JCheckBoxMenuItem.JCheckBoxMenuItem(Action) could not be converted

javax.swing.JCheckBoxMenuItem.JCheckBoxMenuItem(Icon) could not be converted

javax.swing.JCheckBoxMenuItem.JCheckBoxMenuItem(String, boolean) could not be converted

javax.swing.JCheckBoxMenuItem.JCheckBoxMenuItem(String, Icon) could not be converted

javax.swing.JCheckBoxMenuItem.JCheckBoxMenuItem(String, Icon, boolean) could not be converted

javax.swing.JCheckBoxMenuItem.requestFocus could not be converted

javax.swing.JColorChooser.accessibleContext could not be converted

javax.swing.JColorChooser.addChooserPanel could not be converted

javax.swing.JColorChooser.CHOOSER_PANELS_PROPERTY could not be converted

javax.swing.JColorChooser.createDialog could not be converted

javax.swing.JColorChooser.getAccessibleContext could not be converted

javax.swing.JColorChooser.getChooserPanels could not be converted

javax.swing.JColorChooser.getPreviewPanel could not be converted

javax.swing.JColorChooser.getSelectionModel could not be converted

javax.swing.JColorChooser.getUI could not be converted

javax.swing.JColorChooser.getUIClassID could not be converted

javax.swing.JColorChooser.JColorChooser could not be converted

javax.swing.JColorChooser.PREVIEW_PANEL_PROPERTY could not be converted

javax.swing.JColorChooser.removeChooserPanel could not be converted

javax.swing.JColorChooser.SELECTION_MODEL_PROPERTY could not be converted

javax.swing.JColorChooser.setChooserPanels could not be converted

javax.swing.JColorChooser.setPreviewPanel could not be converted

javax.swing.JColorChooser.setSelectionModel could not be converted

javax.swing.JColorChooser.setUI could not be converted

javax.swing.JColorChooser.showDialog could not be converted

javax.swing.JColorChooser.updateUI could not be converted

javax.swing.JComboBox.actionCommand could not be converted

javax.swing.JComboBox.actionPerformed could not be converted

javax.swing.JComboBox.configureEditor could not be converted

javax.swing.JComboBox.configurePropertiesFromAction could not be converted

javax.swing.JComboBox.contentsChanged could not be converted

javax.swing.JComboBox.createActionPropertyChangeListener could not be converted

javax.swing.JComboBox.createDefaultKeySelectionManager could not be converted

javax.swing.JComboBox.editor could not be converted

javax.swing.JComboBox.getAction could not be converted

javax.swing.JComboBox.getActionCommand could not be converted

javax.swing.JComboBox.getEditor could not be converted

javax.swing.JComboBox.getKeySelectionManager could not be converted

javax.swing.JComboBox.getRenderer could not be converted

javax.swing.JComboBox.getUI could not be converted

javax.swing.JComboBox.getUIClassID could not be converted

javax.swing.JComboBox.installAncestorListener could not be converted

javax.swing.JComboBox.intervalAdded could not be converted

javax.swing.JComboBox.intervalRemoved could not be converted

javax.swing.JComboBox.isLightWeightPopupEnabled could not be converted

javax.swing.JComboBox.JComboBox could not be converted

javax.swing.JComboBox.keySelectionManager could not be converted

javax.swing.JComboBox.KeySelectionManager could not be converted

javax.swing.JComboBox.lightWeightPopupEnabled could not be converted

javax.swing.JComboBox.renderer could not be converted

javax.swing.JComboBox.selectedItemReminder could not be converted

javax.swing.JComboBox.selectWithKeyChar could not be converted

javax.swing.JComboBox.setActionCommand could not be converted

javax.swing.JComboBox.setEditor could not be converted

javax.swing.JComboBox.setKeySelectionManager could not be converted

javax.swing.JComboBox.setLightWeightPopupEnabled could not be converted

javax.swing.JComboBox.setModel could not be converted

javax.swing.JComboBox.setRenderer could not be converted

javax.swing.JComboBox.setUI could not be converted

javax.swing.JComboBox.updateUI could not be converted

javax.swing.JComponent.accessibleContext could not be converted

javax.swing.JComponent.AccessibleJComponent could not be converted

javax.swing.JComponent.addAncestorListener could not be converted

javax.swing.JComponent.addNotify could not be converted

javax.swing.JComponent.addPropertyChangeListener could not be converted

javax.swing.JComponent.addVetoableChangeListener could not be converted

javax.swing.JComponent.computeVisibleRect could not be converted

javax.swing.JComponent.createToolTip could not be converted

javax.swing.JComponent.disable could not be converted

javax.swing.JComponent.firePropertyChange could not be converted

javax.swing.JComponent.fireVetoableChange could not be converted

javax.swing.JComponent.getActionForKeyStroke could not be converted

javax.swing.JComponent.getActionMap could not be converted

javax.swing.JComponent.getAlignmentX could not be converted

javax.swing.JComponent.getAlignmentY could not be converted

javax.swing.JComponent.getAutoscrolls could not be converted

javax.swing.JComponent.getBorder could not be converted

javax.swing.JComponent.getClientProperty could not be converted

javax.swing.JComponent.getConditionForKeyStroke could not be converted

javax.swing.JComponent.getDebugGraphicsOptions could not be converted

javax.swing.JComponent.getInputMap could not be converted
javax.swing.JComponent.getInputVerifier could not be converted
javax.swing.JComponent.getListeners could not be converted
javax.swing.JComponent.getLocation could not be converted
javax.swing.JComponent.getMaximumSize could not be converted
javax.swing.JComponent.getMinimumSize could not be converted
javax.swing.JComponent.getNextFocusableComponent could not be converted
javax.swing.JComponent.getPreferredSize could not be converted
javax.swing.JComponent.getRegisteredKeyStrokes could not be converted
javax.swing.JComponent.getRootPane could not be converted
javax.swing.JComponent.getSize could not be converted
javax.swing.JComponent.getToolTipLocation could not be converted
javax.swing.JComponent.getToolTipText could not be converted
javax.swing.JComponent.getTopLevelAncestor could not be converted
javax.swing.JComponent.getVerifyInputWhenFocusTarget could not be converted
javax.swing.JComponent.getVisibleRect could not be converted
javax.swing.JComponent.isDoubleBuffered could not be converted
javax.swing.JComponent.isFocusCycleRoot could not be converted
javax.swing.JComponent.isLightweightComponent could not be converted
javax.swing.JComponent.isManagingFocus could not be converted
javax.swing.JComponent.isMaximumSizeSet could not be converted
javax.swing.JComponent.isMinimumSizeSet could not be converted
javax.swing.JComponent.isOpaque could not be converted
javax.swing.JComponent.isOptimizedDrawingEnabled could not be converted
javax.swing.JComponent.isPaintingTile could not be converted
javax.swing.JComponent.isPreferredSizeSet could not be converted
javax.swing.JComponent.isValidateRoot could not be converted
javax.swing.JComponent.listenerList could not be converted
javax.swing.JComponent.paint could not be converted
javax.swing.JComponent.paintBorder could not be converted
javax.swing.JComponent.paintChildren could not be converted
javax.swing.JComponent.paintComponent could not be converted
javax.swing.JComponent.paintImmediately could not be converted
javax.swing.JComponent.print could not be converted
javax.swing.JComponent.printAll could not be converted
javax.swing.JComponent.printBorder could not be converted
javax.swing.JComponent.printChildren could not be converted
javax.swing.JComponent.printComponent could not be converted
javax.swing.JComponent.processKeyBinding could not be converted
javax.swing.JComponent.putClientProperty could not be converted

javax.swing.JComponent.registerKeyboardAction could not be converted

javax.swing.JComponent.removeAncestorListener could not be converted

javax.swing.JComponent.removePropertyChangeListener could not be converted

javax.swing.JComponent.removeVetoableChangeListener could not be converted

javax.swing.JComponent.repaint could not be converted

javax.swing.JComponent.requestDefaultFocus could not be converted

javax.swing.JComponent.requestFocus could not be converted

javax.swing.JComponent.resetKeyboardActions could not be converted

javax.swing.JComponent.scrollToRectToVisible could not be converted

javax.swing.JComponent.setActionMap could not be converted

javax.swing.JComponent.setAlignmentX could not be converted

javax.swing.JComponent.setAlignmentY could not be converted

javax.swing.JComponent.setAutoscrolls could not be converted

javax.swing.JComponent.setBorder could not be converted

javax.swing.JComponent.setDebugGraphicsOptions could not be converted

javax.swing.JComponent.setDoubleBuffered could not be converted

javax.swing.JComponent.setEnabled could not be converted

javax.swing.JComponent.setInputMap could not be converted

javax.swing.JComponent.setInputVerifier could not be converted

javax.swing.JComponent.setMaximumSize could not be converted

javax.swing.JComponent.setMinimumSize could not be converted

javax.swing.JComponent.setNextFocusableComponent could not be converted

javax.swing.JComponent.setOpaque could not be converted

javax.swing.JComponent.setPreferredSize could not be converted

javax.swing.JComponent.setRequestFocusEnabled could not be converted

javax.swing.JComponent.setToolTipText could not be converted

javax.swing.JComponent.setUI could not be converted

javax.swing.JComponent.setVerifyInputWhenFocusTarget could not be converted

javax.swing.JComponent.setVisible could not be converted

javax.swing.JComponent.TOOL_TIP_TEXT_KEY could not be converted

javax.swing.JComponent.ui could not be converted

javax.swing.JComponent.unregisterKeyboardAction could not be converted

javax.swing.JComponent.updateUI could not be converted

javax.swing.JDesktopPane could not be converted

javax.swing.JDesktopPane.<DragMode> could not be converted

javax.swing.JDesktopPane.getDesktopManager could not be converted

javax.swing.JDesktopPane.getDragMode could not be converted

javax.swing.JDesktopPane.getUI could not be converted

javax.swing.JDesktopPane.getUIClassID could not be converted

javax.swing.JDesktopPane.isOpaque could not be converted

javax.swing.JDesktopPane.JDesktopPane could not be converted

javax.swing.JDesktopPane.setDesktopManager could not be converted

javax.swing.JDesktopPane.setDragMode could not be converted

javax.swing.JDesktopPane.setSelectedFrame could not be converted

javax.swing.JDesktopPane.setUI could not be converted

javax.swing.JDesktopPane.updateUI could not be converted

javax.swing.JDialog.accessibleContext could not be converted

javax.swing.JDialog.addImpl could not be converted

javax.swing.JDialog.dialogInit could not be converted

javax.swing.JDialog.getDefaultCloseOperation could not be converted

javax.swing.JDialog.getGlassPane could not be converted

javax.swing.JDialog.getLayeredPane could not be converted

javax.swing.JDialog.getRootPane could not be converted

javax.swing.JDialog.isRootPaneCheckingEnabled could not be converted

javax.swing.JDialog.JDialog could not be converted

javax.swing.JDialog.JDialog(Dialog, boolean) could not be converted

javax.swing.JDialog.JDialog(Dialog, String, boolean) could not be converted

javax.swing.JDialog.JDialog(Frame, boolean) could not be converted

javax.swing.JDialog.JDialog(Frame, String, boolean) could not be converted

javax.swing.JDialog.rootPane could not be converted

javax.swing.JDialog.rootPaneCheckingEnabled could not be converted

javax.swing.JDialog.setContentPane could not be converted

javax.swing.JDialog.setDefaultCloseOperation could not be converted

javax.swing.JDialog.setGlassPane could not be converted

javax.swing.JDialog.setLayeredPane could not be converted

javax.swing.JDialog.setLocationRelativeTo could not be converted

javax.swing.JDialog.setRootPane could not be converted

javax.swing.JDialog.setRootPaneCheckingEnabled could not be converted

javax.swing.JEditorPane could not be converted

javax.swing.JEditorPane.createDefaultEditorKit could not be converted

javax.swing.JEditorPane.createEditorKitForContentType could not be converted

javax.swing.JEditorPane.fireHyperlinkUpdate could not be converted

javax.swing.JEditorPane.getContentType could not be converted

javax.swing.JEditorPane.getEditorKit could not be converted

javax.swing.JEditorPane.getEditorKitClassNameForContentType could not be converted

javax.swing.JEditorPane.getEditorKitForContentType could not be converted

javax.swing.JEditorPane.getPage could not be converted

javax.swing.JEditorPane.getPreferredSize could not be converted

javax.swing.JEditorPane.getScrollableTracksViewportHeight could not be converted

javax.swing.JEditorPane.getScrollableTracksViewportWidth could not be converted

javax.swing.JEditorPane.getOutputStream could not be converted

javax.swing.JEditorPane.getUIClassID could not be converted

javax.swing.JEditorPane.isFocusCycleRoot could not be converted

javax.swing.JEditorPane.JEditorPane(String) could not be converted

javax.swing.JEditorPane.JEditorPane(String, String) could not be converted

javax.swing.JEditorPane.JEditorPane(URL) could not be converted

javax.swing.JEditorPane.read could not be converted

javax.swing.JEditorPane.registerEditorKitForContentType(String, String) could not be converted

javax.swing.JEditorPane.registerEditorKitForContentType(String, String, ClassLoader) could not be converted

javax.swing.JEditorPane.scrollToReference could not be converted

javax.swing.JEditorPane.setContentType could not be converted

javax.swing.JEditorPane.setEditorKit could not be converted

javax.swing.JEditorPane.setEditorKitForContentType could not be converted

javax.swing.JEditorPane.setPage could not be converted

javax.swing.JFileChooser.accept could not be converted

javax.swing.JFileChooser.accessibleContext could not be converted

javax.swing.JFileChooser.addChoosableFileFilter could not be converted

javax.swing.JFileChooser.approveSelection could not be converted

javax.swing.JFileChooser.cancelSelection could not be converted

javax.swing.JFileChooser.changeToParentDirectory could not be converted

javax.swing.JFileChooser.ensureFileIsVisible could not be converted

javax.swing.JFileChooser.fireActionPerformed could not be converted

javax.swing.JFileChooser.getAcceptAllFileFilter could not be converted

javax.swing.JFileChooser.getAccessibleContext could not be converted

javax.swing.JFileChooser.getAccessory could not be converted

javax.swing.JFileChooser.getApproveButtonMnemonic could not be converted

javax.swing.JFileChooser.getApproveButtonText could not be converted

javax.swing.JFileChooser.getApproveButtonToolTipText could not be converted

javax.swing.JFileChooser.getChoosableFileFilters could not be converted

javax.swing.JFileChooser.getCurrentDirectory could not be converted

javax.swing.JFileChooser.getDescription could not be converted

javax.swing.JFileChooser.getFileFilter could not be converted

javax.swing.JFileChooser.getFileSelectionMode could not be converted

javax.swing.JFileChooser.getFileSystemView could not be converted

javax.swing.JFileChooser.getFileView could not be converted

javax.swing.JFileChooser.getIcon could not be converted

javax.swing.JFileChooser.getSelectedFiles could not be converted

javax.swing.JFileChooser.getTypeDescription could not be converted

javax.swing.JFileChooser.getUI could not be converted

javax.swing.JFileChooser.isAcceptAllFileFilterUsed could not be converted

javax.swing.JFileChooser.isFileHidingEnabled could not be converted

javax.swing.JFileChooser.isFileSelectionEnabled could not be converted

javax.swing.JFileChooser.isTraversable could not be converted

javax.swing.JFileChooser.JFileChooser(File, FileSystemView) could not be converted

javax.swing.JFileChooser.JFileChooser(FileSystemView) could not be converted

javax.swing.JFileChooser.JFileChooser(String, FileSystemView) could not be converted

javax.swing.JFileChooser.removeChoosableFileFilter could not be converted

javax.swing.JFileChooser.rescanCurrentDirectory could not be converted

javax.swing.JFileChooser.resetChoosableFileFilters could not be converted

javax.swing.JFileChooser.setAcceptAllFileFilterUsed could not be converted

javax.swing.JFileChooser.setAccessory could not be converted

javax.swing.JFileChooser.setApproveButtonMnemonic could not be converted

javax.swing.JFileChooser.setApproveButtonText could not be converted

javax.swing.JFileChooser.setApproveButtonToolTipText could not be converted

javax.swing.JFileChooser.setControlButtonsAreShown could not be converted

javax.swing.JFileChooser.setCurrentDirectory could not be converted

javax.swing.JFileChooser.setDialogType could not be converted

javax.swing.JFileChooser.setFileFilter could not be converted

javax.swing.JFileChooser.setFileHidingEnabled could not be converted

javax.swing.JFileChooser.setFileSelectionMode could not be converted

javax.swing.JFileChooser.setFileSystemView could not be converted

javax.swing.JFileChooser.setFileView could not be converted

javax.swing.JFileChooser.setSelectedFiles could not be converted

javax.swing.JFileChooser.setup could not be converted

javax.swing.JFileChooser.showDialog could not be converted

javax.swing.JFileChooser.showSaveDialog could not be converted

javax.swing.JFileChooser.updateUI could not be converted

javax.swing.JFrame.EXIT_ON_CLOSE could not be converted

javax.swing.JFrame.frameInit could not be converted

javax.swing.JFrame.getContentPane could not be converted

javax.swing.JFrame.getDefaultCloseOperation could not be converted

javax.swing.JFrame.getGlassPane could not be converted

javax.swing.JFrame.getLayeredPane could not be converted

javax.swing.JFrame.isRootPaneCheckingEnabled could not be converted

javax.swing.JFrame.rootPane could not be converted

javax.swing.JFrame.rootPaneCheckingEnabled could not be converted

javax.swing.JFrame.setContentPane could not be converted

javax.swing.JFrame.setDefaultCloseOperation could not be converted

javax.swing.JFrame.setGlassPane could not be converted

javax.swing.JFrame.setLayeredPane could not be converted

javax.swing.JFrame.setRootPane could not be converted

javax.swing.JFrame.setRootPaneCheckingEnabled could not be converted

javax.swing.JInternalFrame could not be converted

javax.swing.JInternalFrame.<PropertyName> could not be converted

javax.swing.JInternalFrame.closable could not be converted

javax.swing.JInternalFrame.desktopIcon could not be converted

javax.swing.JInternalFrame.doDefaultCloseAction could not be converted

javax.swing.JInternalFrame.fireInternalFrameEvent could not be converted

javax.swing.JInternalFrame.getDefaultCloseOperation could not be converted

javax.swing.JInternalFrame.getDesktopIcon could not be converted

javax.swing.JInternalFrame.getGlassPane could not be converted

javax.swing.JInternalFrame.getLayer could not be converted

javax.swing.JInternalFrame.getLayeredPane could not be converted

javax.swing.JInternalFrame.getNormalBounds could not be converted

javax.swing.JInternalFrame.getUI could not be converted

javax.swing.JInternalFrame.getUIClassID could not be converted

javax.swing.JInternalFrame.iconable could not be converted

javax.swing.JInternalFrame.isClosable could not be converted

javax.swing.JInternalFrame.isClosed could not be converted

javax.swing.JInternalFrame.isIcon could not be converted

javax.swing.JInternalFrame.isMaximum could not be converted

javax.swing.JInternalFrame.isRootPaneCheckingEnabled could not be converted

javax.swing.JInternalFrame.isSelected could not be converted

javax.swing.JInternalFrame.JDesktopIcon could not be converted

javax.swing.JInternalFrame.maximizable could not be converted

javax.swing.JInternalFrame.moveToBack could not be converted

javax.swing.JInternalFrame.moveToFront could not be converted

javax.swing.JInternalFrame.pack could not be converted

javax.swing.JInternalFrame.paintComponent could not be converted

javax.swing.JInternalFrame paramString could not be converted

javax.swing.JInternalFrame.resizable could not be converted

javax.swing.JInternalFrame.restoreSubcomponentFocus could not be converted

javax.swing.JInternalFrame.rootPane could not be converted

javax.swing.JInternalFrame.rootPaneCheckingEnabled could not be converted

javax.swing.JInternalFrame.setClosable could not be converted

javax.swing.JInternalFrame.setClosed could not be converted

javax.swing.JInternalFrame.setContentPane could not be converted

javax.swing.JInternalFrame.setDefaultCloseOperation could not be converted

javax.swing.JInternalFrame.setDesktopIcon could not be converted

javax.swing.JInternalFrame.setFrameIcon could not be converted

javax.swing.JInternalFrame.setGlassPane could not be converted

javax.swing.JInternalFrame.setLayer could not be converted

javax.swing.JInternalFrame.setLayeredPane could not be converted

javax.swing.JInternalFrame.setNormalBounds could not be converted

javax.swing.JInternalFrame.setRootPane could not be converted

javax.swing.JInternalFrame.setRootPaneCheckingEnabled could not be converted

javax.swing.JInternalFrame.setSelected could not be converted

javax.swing.JInternalFrame.setUI could not be converted

javax.swing.JInternalFrame.updateUI could not be converted

javax.swing.JLabel.checkHorizontalKey could not be converted

javax.swing.JLabel.checkVerticalKey could not be converted

javax.swing.JLabel.getDisabledIcon could not be converted

javax.swing.JLabel.getDisplayedMnemonic could not be converted

javax.swing.JLabel.getIconTextGap could not be converted

javax.swing.JLabel.getLabelFor could not be converted

javax.swing.JLabel.getUI could not be converted

javax.swing.JLabel.getUIClassID could not be converted

javax.swing.JLabel.imageUpdate could not be converted

javax.swing.JLabel.labelFor could not be converted

javax.swing.JLabel.setDisabledIcon could not be converted

javax.swing.JLabel.setHorizontalAlignment could not be converted

javax.swing.JLabel.setHorizontalTextPosition could not be converted

javax.swing.JLabel.setIconTextGap could not be converted

javax.swing.JLabel.setLabelFor could not be converted

javax.swing.JLabel.setUI could not be converted

javax.swing.JLabel.setVerticalAlignment could not be converted

javax.swing.JLabel.setVerticalTextPosition could not be converted

javax.swing.JLabel.updateUI could not be converted

javax.swing.JLayeredPane could not be converted

javax.swing.JLayeredPane.addImpl could not be converted

javax.swing.JLayeredPane.getComponentCountInLayer could not be converted

javax.swing.JLayeredPane.getComponentsInLayer could not be converted

javax.swing.JLayeredPane.getComponentToLayer could not be converted

javax.swing.JLayeredPane.getIndexOf could not be converted

javax.swing.JLayeredPane.getLayer could not be converted

javax.swing.JLayeredPane.getLayeredPaneAbove could not be converted

javax.swing.JLayeredPane.getObjectForLayer could not be converted

javax.swing.JLayeredPane.getPosition could not be converted

javax.swing.JLayeredPane.highestLayer could not be converted

javax.swing.JLayeredPane.insertIndexForLayer could not be converted

javax.swing.JLayeredPane.isOptimizedDrawingEnabled could not be converted

javax.swing.JLayeredPane.JLayeredPane could not be converted

javax.swing.JLayeredPane.LAYER_PROPERTY could not be converted

javax.swing.JLayeredPane.lowestLayer could not be converted

javax.swing.JLayeredPane.moveToBack could not be converted

javax.swing.JLayeredPane.moveToFront could not be converted

javax.swing.JLayeredPane paramString could not be converted

javax.swing.JLayeredPane.putLayer could not be converted

javax.swing.JLayeredPane.remove could not be converted

javax.swing.JLayeredPane.setLayer could not be converted

javax.swing.JLayeredPane.setPosition could not be converted

javax.swing.JList.addListSelectionListener could not be converted

javax.swing.JList.addSelectionInterval could not be converted

javax.swing.JList.createSelectionModel could not be converted

javax.swing.JList.ensureIndexIsVisible could not be converted

javax.swing.JList.fireSelectionValueChanged could not be converted

javax.swing.JList.getAnchorSelectionIndex could not be converted

javax.swing.JList.getCellBounds could not be converted

javax.swing.JList.getCellRenderer could not be converted

javax.swing.JList.getFixedCellHeight could not be converted

javax.swing.JList.getFixedCellWidth could not be converted

javax.swing.JList.getLastVisibleIndex could not be converted

javax.swing.JList.getLeadSelectionIndex could not be converted

javax.swing.JList.getMaxSelectionIndex could not be converted

javax.swing.JList.getMinSelectionIndex could not be converted

javax.swing.JList.getModel could not be converted

javax.swing.JList.getPreferredScrollableViewportSize could not be converted

javax.swing.JList.getPrototypeCellValue could not be converted

javax.swing.JList.getScrollableBlockIncrement could not be converted

javax.swing.JList.getScrollableTracksViewportHeight could not be converted

javax.swing.JList.getScrollableTracksViewportWidth could not be converted

javax.swing.JList.getScrollableUnitIncrement could not be converted

javax.swing.JList.setSelectedIndex could not be converted

javax.swing.JList.getSelectedIndices could not be converted

javax.swing.JList.getSelectedValues could not be converted

javax.swing.JList.getSelectionBackground could not be converted

javax.swing.JList.getSelectionForeground could not be converted

javax.swing.JList.getSelectionModel could not be converted

javax.swing.JList.getUI could not be converted

javax.swing.JList.getUIClassID could not be converted

javax.swing.JList.getValuesAdjusting could not be converted
javax.swing.JList.getVisibleRowCount could not be converted
javax.swing.JList.indexToLocation could not be converted
javax.swing.JList.isSelectedIndex could not be converted
javax.swing.JList.JList(ListModel) could not be converted
javax.swing.JList.JList(Object[]) could not be converted
javax.swing.JList paramString could not be converted
javax.swing.JList.removeListSelectionListener could not be converted
javax.swing.JList.removeSelectionInterval could not be converted
javax.swing.JList.setCellRenderer could not be converted
javax.swing.JList.setFixedCellHeight could not be converted
javax.swing.JList.setFixedCellWidth could not be converted
javax.swing.JList.setListData could not be converted
javax.swing.JList.setModel could not be converted
javax.swing.JList.setPrototypeCellValue could not be converted
javax.swing.JList.setSelectedIndices could not be converted
javax.swing.JList.setSelectionBackground could not be converted
javax.swing.JList.setSelectionForeground could not be converted
javax.swing.JList.setSelectionInterval could not be converted
javax.swing.JList.setSelectionModel could not be converted
javax.swing.JList.setUI could not be converted
javax.swing.JList.setValuesAdjusting could not be converted
javax.swing.JList.setVisibleRowCount could not be converted
javax.swing.JList.updateUI could not be converted
javax.swing.JMenu.add(Component) could not be converted
javax.swing.JMenu.add(Component, int) could not be converted
javax.swing.JMenu.add(MenuItem) could not be converted
javax.swing.JMenu.createActionChangeListener could not be converted
javax.swing.JMenu.createWindowListener could not be converted
javax.swing.JMenu.doClick could not be converted
javax.swing.JMenu.fireMenuCanceled could not be converted
javax.swing.JMenu.fireMenuDeselected could not be converted
javax.swing.JMenu.fireMenuSelected could not be converted
javax.swing.JMenu.getAccessibleContext could not be converted
javax.swing.JMenu.getComponent could not be converted
javax.swing.JMenu.getDelay could not be converted
javax.swing.JMenu getMenuComponent could not be converted
javax.swing.JMenu getMenuComponents could not be converted
javax.swing.JMenu.getPopupMenu could not be converted
javax.swing.JMenu.getPopupMenuOrigin could not be converted

javax.swing.JMenu.getUIClassID could not be converted

javax.swing.JMenu.isMenuComponent could not be converted

javax.swing.JMenu.isPopupMenuVisible could not be converted

javax.swing.JMenu.isTearOff could not be converted

javax.swing.JMenu.isTopLevelMenu could not be converted

javax.swing.JMenu.menuSelectionChanged could not be converted

javax.swing.JMenu.popupListener could not be converted

javax.swing.JMenu.setAccelerator could not be converted

javax.swing.JMenu.setDelay could not be converted

javax.swing.JMenu.setMenuLocation could not be converted

javax.swing.JMenu.setModel could not be converted

javax.swing.JMenu.setPopupMenuVisible could not be converted

javax.swing.JMenu.updateUI could not be converted

javax.swing.JMenuBar.getAccessibleContext could not be converted

javax.swing.JMenuBar.getComponent could not be converted

javax.swing.JMenuBar.getComponentAtIndex could not be converted

javax.swing.JMenuBar.getComponentIndex could not be converted

javax.swing.JMenuBar.getHelpMenu could not be converted

javax.swing.JMenuBar.getMargin could not be converted

javax.swing.JMenuBar.getSelectionModel could not be converted

javax.swing.JMenuBar.getUI could not be converted

javax.swing.JMenuBar.getUIClassID could not be converted

javax.swing.JMenuBar.isBorderPainted could not be converted

javax.swing.JMenuBar.isManagingFocus could not be converted

javax.swing.JMenuBar.isSelected could not be converted

javax.swing.JMenuBar.menuSelectionChanged could not be converted

javax.swing.JMenuBar.paintBorder could not be converted

javax.swing.JMenuBar.processKeyBinding could not be converted

javax.swing.JMenuBar.setBorderPainted could not be converted

javax.swing.JMenuBar.setHelpMenu could not be converted

javax.swing.JMenuBar.setMargin could not be converted

javax.swing.JMenuBar.setSelected could not be converted

javax.swing.JMenuBar.setSelectionModel could not be converted

javax.swing.JMenuBar.setUI could not be converted

javax.swing.JMenuBar.updateUI could not be converted

javax.swing.JMenuItem.addMenuDragMouseListener could not be converted

javax.swing.JMenuItem.addMenuKeyListener could not be converted

javax.swing.JMenuItem.configurePropertiesFromAction could not be converted

javax.swing.JMenuItem.createActionPropertyChangeListener could not be converted

javax.swing.JMenuItem.fireMenuDragMouseDragged could not be converted

javax.swing.JMenuItem.fireMenuDragMouseEntered could not be converted

javax.swing.JMenuItem.fireMenuDragMouseExited could not be converted

javax.swing.JMenuItem.fireMenuDragMouseReleased could not be converted

javax.swing.JMenuItem.fireMenuKeyPressed could not be converted

javax.swing.JMenuItem.fireMenuKeyReleased could not be converted

javax.swing.JMenuItem.fireMenuKeyTyped could not be converted

javax.swing.JMenuItem.getAccelerator could not be converted

javax.swing.JMenuItem.getAccessibleContext could not be converted

javax.swing.JMenuItem.getComponent could not be converted

javax.swing.JMenuItem.getUIClassID could not be converted

javax.swing.JMenuItem.init could not be converted

javax.swing.JMenuItem.isArmed could not be converted

javax.swing.JMenuItem.JMenuItem could not be converted

javax.swing.JMenuItem.menuSelectionChanged could not be converted

javax.swing.JMenuItem.removeMenuDragMouseListener could not be converted

javax.swing.JMenuItem.removeMenuKeyListener could not be converted

javax.swing.JMenuItem.setAccelerator could not be converted

javax.swing.JMenuItem.setArmed could not be converted

javax.swing.JMenuItem.setUI could not be converted

javax.swing.JMenuItem.updateUI could not be converted

javax.swing.JOptionPane could not be converted

javax.swing.JOptionPane.showConfirmDialog could not be converted

javax.swing.JOptionPane.showInternalConfirmDialog could not be converted

javax.swing.JOptionPane.showInternalMessageDialog could not be converted

javax.swing.JOptionPane.showMessageDialog could not be converted

javax.swing.JPanel.JPanel could not be converted

javax.swing.JPasswordField.getUIClassID could not be converted

javax.swing.JPasswordField.JPasswordField could not be converted

javax.swing.JPasswordField.JPasswordField(Document, String, int) could not be converted

javax.swing.JPasswordField.JPasswordField(int) could not be converted

javax.swing.JPasswordField.JPasswordField(String, int) could not be converted

javax.swing.JPopupMenu.add could not be converted

javax.swing.JPopupMenu.createActionChangeListener could not be converted

javax.swing.JPopupMenu.createActionComponent could not be converted

javax.swing.JPopupMenu.firePopupMenuCanceled could not be converted

javax.swing.JPopupMenu.firePopupMenuWillBecomeInvisible could not be converted

javax.swing.JPopupMenu.firePopupMenuWillBecomeVisible could not be converted

javax.swing.JPopupMenu.getAccessibleContext could not be converted

javax.swing.JPopupMenu.getComponent could not be converted

javax.swing.JPopupMenu.getComponentAtIndex could not be converted

javax.swing.JPopupMenu.getComponentIndex could not be converted

javax.swing.JPopupMenu.getDefaultLightWeightPopupEnabled could not be converted

javax.swing.JPopupMenu.getInvoker could not be converted

javax.swing.JPopupMenu.getLabel could not be converted

javax.swing.JPopupMenu.getSelectionModel could not be converted

javax.swing.JPopupMenu.getUI could not be converted

javax.swing.JPopupMenu.getUIClassID could not be converted

javax.swing.JPopupMenu.insert could not be converted

javax.swing.JPopupMenu.isBorderPainted could not be converted

javax.swing.JPopupMenu.isLightWeightPopupEnabled could not be converted

javax.swing.JPopupMenu.isPopupTrigger could not be converted

javax.swing.JPopupMenu.isVisible could not be converted

javax.swing.JPopupMenu.menuSelectionChanged could not be converted

javax.swing.JPopupMenu.pack could not be converted

javax.swing.JPopupMenu.paintBorder could not be converted

javax.swing.JPopupMenu.Separator could not be converted

javax.swing.JPopupMenu.setBorderPainted could not be converted

javax.swing.JPopupMenu.setDefaultLightWeightPopupEnabled could not be converted

javax.swing.JPopupMenu.setLabel could not be converted

javax.swing.JPopupMenu.setLightWeightPopupEnabled could not be converted

javax.swing.JPopupMenu.setLocation could not be converted

javax.swing.JPopupMenu.setPopupSize(Dimension) could not be converted

javax.swing.JPopupMenu.setPopupSize(int, int) could not be converted

javax.swing.JPopupMenu.setSelected could not be converted

javax.swing.JPopupMenu.setSelectionModel could not be converted

javax.swing.JPopupMenu.setUI could not be converted

javax.swing.JPopupMenu.setVisible could not be converted

javax.swing.JPopupMenu.show could not be converted

javax.swing.JPopupMenu.updateUI could not be converted

javax.swing.JProgressBar.addChangeListener could not be converted

javax.swing.JProgressBar.changeEvent could not be converted

javax.swing.JProgressBar.changeListener could not be converted

javax.swing.JProgressBar.createChangeListener could not be converted

javax.swing.JProgressBar.fireStateChanged could not be converted

javax.swing.JProgressBar.getModel could not be converted

javax.swing.JProgressBar.getOrientation could not be converted

javax.swing.JProgressBar.getString could not be converted

javax.swing.JProgressBar.getUI could not be converted

javax.swing.JProgressBar.getUIClassID could not be converted

javax.swing.JProgressBar.isBorderPainted could not be converted

javax.swing.JProgressBar.isStringPainted could not be converted

javax.swing.JProgressBar.JProgressBar(Bounded RangeModel) could not be converted

javax.swing.JProgressBar.JProgressBar(int) could not be converted

javax.swing.JProgressBar.JProgressBar(int, int, int) could not be converted

javax.swing.JProgressBar.model could not be converted

javax.swing.JProgressBar.orientation could not be converted

javax.swing.JProgressBar.paintBorder could not be converted

javax.swing.JProgressBar.paintString could not be converted

javax.swing.JProgressBar.progressString could not be converted

javax.swing.JProgressBar.removeChangeListener could not be converted

javax.swing.JProgressBar.setBorderPainted could not be converted

javax.swing.JProgressBar.setModel could not be converted

javax.swing.JProgressBar.setOrientation could not be converted

javax.swing.JProgressBar.setString could not be converted

javax.swing.JProgressBar.setStringPainted could not be converted

javax.swing.JProgressBar.setUI could not be converted

javax.swing.JRadioButton.configurePropertiesFromAction could not be converted

javax.swing.JRadioButton.createActionPropertyChangeListener could not be converted

javax.swing.JRadioButton.getUIClassID could not be converted

javax.swing.JRadioButton.JRadioButton could not be converted

javax.swing.JRadioButton.updateUI could not be converted

javax.swing.JRadioButtonMenuItem.getAccessibleContext could not be converted

javax.swing.JRadioButtonMenuItem.getUIClassID could not be converted

javax.swing.JRadioButtonMenuItem.JRadioButtonMenuItem could not be converted

javax.swing.JRadioButtonMenuItem.JRadioButtonMenuItem(Action) could not be converted

javax.swing.JRadioButtonMenuItem.JRadioButtonMenuItem(Icon) could not be converted

javax.swing.JRadioButtonMenuItem.JRadioButtonMenuItem(Icon, boolean) could not be converted

javax.swing.JRadioButtonMenuItem.JRadioButtonMenuItem(String) could not be converted

javax.swing.JRadioButtonMenuItem.JRadioButtonMenuItem(String, boolean) could not be converted

javax.swing.JRadioButtonMenuItem.JRadioButtonMenuItem(String, Icon) could not be converted

javax.swing.JRadioButtonMenuItem.JRadioButtonMenuItem(String, Icon, boolean) could not be converted

javax.swing.JRadioButtonMenuItem.requestFocus could not be converted

javax.swing.JRootPane.contentPane could not be converted

javax.swing.JRootPane.createGlassPane could not be converted

javax.swing.JRootPane.createLayeredPane could not be converted

javax.swing.JRootPane.createRootLayout could not be converted

javax.swing.JRootPane.defaultButton could not be converted

javax.swing.JRootPane.defaultPressAction could not be converted

javax.swing.JRootPane.defaultReleaseAction could not be converted

javax.swing.JRootPane.getDefaultButton could not be converted

javax.swing.JRootPane.getGlassPane could not be converted
javax.swing.JRootPane.getLayeredPane could not be converted
javax.swing.JRootPane.getUI could not be converted
javax.swing.JRootPane.getUIClassID could not be converted
javax.swing.JRootPane.glassPane could not be converted
javax.swing.JRootPane.isFocusCycleRoot could not be converted
javax.swing.JRootPane.isOptimizedDrawingEnabled could not be converted
javax.swing.JRootPane.isValidateRoot could not be converted
javax.swing.JRootPane.layeredPane could not be converted
javax.swing.JRootPane.menuBar could not be converted
javax.swing.JRootPane.setDefaultButton could not be converted
javax.swing.JRootPane.setGlassPane could not be converted
javax.swing.JRootPane.setLayeredPane could not be converted
javax.swing.JRootPane.setUI could not be converted
javax.swing.JRootPane.updateUI could not be converted
javax.swing.JScrollBar could not be converted
javax.swing.JScrollBar.fireAdjustmentValueChanged could not be converted
javax.swing.JScrollBar.getBlockIncrement could not be converted
javax.swing.JScrollBar.getMaximumSize could not be converted
javax.swing.JScrollBar.getModel could not be converted
javax.swing.JScrollBar.getUI could not be converted
javax.swing.JScrollBar.getUIClassID could not be converted
javax.swing.JScrollBar.getUnitIncrement could not be converted
javax.swing.JScrollBar.getValueAdjusting could not be converted
javax.swing.JScrollBar.JScrollBar could not be converted
javax.swing.JScrollBar.model could not be converted
javax.swing.JScrollBar.orientation could not be converted
javax.swing.JScrollBar.setBlockIncrement could not be converted
javax.swing.JScrollBar.setModel could not be converted
javax.swing.JScrollBar.setOrientation could not be converted
javax.swing.JScrollBar.setUnitIncrement could not be converted
javax.swing.JScrollBar.setValueAdjusting could not be converted
javax.swing.JScrollBar.setValues could not be converted
javax.swing.JScrollBar.setVisibleAmount could not be converted
javax.swing.JScrollPane could not be converted
javax.swing.JScrollPane.columnHeader could not be converted
javax.swing.JScrollPane.createHorizontalScrollBar could not be converted
javax.swing.JScrollPane.createVerticalScrollBar could not be converted
javax.swing.JScrollPane.createViewport could not be converted
javax.swing.JScrollPane.columnHeader could not be converted

javax.swing.JScrollPane.getCorner could not be converted

javax.swing.JScrollPane.getHorizontalScrollBar could not be converted

javax.swing.JScrollPane.getHorizontalScrollBarPolicy could not be converted

javax.swing.JScrollPane.getRowHeader could not be converted

javax.swing.JScrollPane.getUI could not be converted

javax.swing.JScrollPane.getUIClassID could not be converted

javax.swing.JScrollPane.getVerticalScrollBar could not be converted

javax.swing.JScrollPane.getVerticalScrollBarPolicy could not be converted

javax.swing.JScrollPane.getViewport could not be converted

javax.swing.JScrollPane.getViewportBorder could not be converted

javax.swing.JScrollPane.horizontalScrollBar could not be converted

javax.swing.JScrollPane.horizontalScrollBarPolicy could not be converted

javax.swing.JScrollPane.isValidRoot could not be converted

javax.swing.JScrollPane.JScrollPane could not be converted

javax.swing.JScrollPane.JScrollPane(Component) could not be converted

javax.swing.JScrollPane.JScrollPane(Component, int, int) could not be converted

javax.swing.JScrollPane.JScrollPane(int, int) could not be converted

javax.swing.JScrollPane.lowerLeft could not be converted

javax.swing.JScrollPane.lowerRight could not be converted

javax.swing.JScrollPane.rowHeader could not be converted

javax.swing.JScrollPane.setColumnHeader could not be converted

javax.swing.JScrollPane.setCorner could not be converted

javax.swing.JScrollPane.setHorizontalScrollBar could not be converted

javax.swing.JScrollPane.setHorizontalScrollBarPolicy could not be converted

javax.swing.JScrollPane.setRowHeader could not be converted

javax.swing.JScrollPane.setUI could not be converted

javax.swing.JScrollPane.setVerticalScrollBar could not be converted

javax.swing.JScrollPane.setVerticalScrollBarPolicy could not be converted

javax.swing.JScrollPane.setViewport could not be converted

javax.swing.JScrollPane.setViewportBorder could not be converted

javax.swing.JScrollPane.updateUI could not be converted

javax.swing.JScrollPane.upperLeft could not be converted

javax.swing.JScrollPane.upperRight could not be converted

javax.swing.JScrollPane.verticalScrollBar could not be converted

javax.swing.JScrollPane.verticalScrollBarPolicy could not be converted

javax.swing.JScrollPane.viewport could not be converted

javax.swing.JSeparator could not be converted

javax.swing.JSlider.changeEvent could not be converted

javax.swing.JSlider.changeListener could not be converted

javax.swing.JSlider.createChangeListener could not be converted

javax.swing.JSlider.createStandardLabels(int) could not be converted

javax.swing.JSlider.createStandardLabels(int, int) could not be converted

javax.swing.JSlider.fireStateChanged could not be converted

javax.swing.JSlider.getExtent could not be converted

javax.swing.JSlider.getLabelTable could not be converted

javax.swing.JSlider.getMajorTickSpacing could not be converted

javax.swing.JSlider.getModel could not be converted

javax.swing.JSlider.getUI could not be converted

javax.swing.JSlider.getUIClassID could not be converted

javax.swing.JSlider.getValuesAdjusting could not be converted

javax.swing.JSlider.JSlider could not be converted

javax.swing.JSlider.majorTickSpacing could not be converted

javax.swing.JSlider.setExtent could not be converted

javax.swing.JSlider.setInverted could not be converted

javax.swing.JSlider.setLabelTable could not be converted

javax.swing.JSlider.setMajorTickSpacing could not be converted

javax.swing.JSlider.setModel could not be converted

javax.swing.JSlider.setPaintLabels could not be converted

javax.swing.JSlider.setPaintTrack could not be converted

javax.swing.JSlider.setSnapToTicks could not be converted

javax.swing.JSlider.setUI could not be converted

javax.swing.JSlider.setValuesAdjusting could not be converted

javax.swing.JSlider.sliderModel could not be converted

javax.swing.JSlider.snapToTicks could not be converted

javax.swing.JSlider.updateLabelUIs could not be converted

javax.swing.JSlider.updateUI could not be converted

javax.swing.JSplitPane could not be converted

javax.swing.JSplitPane.continuousLayout could not be converted

javax.swing.JSplitPane.getResizeWeight could not be converted

javax.swing.JSplitPane.getUI could not be converted

javax.swing.JSplitPane.getUIClassID could not be converted

javax.swing.JSplitPane.isContinuousLayout could not be converted

javax.swing.JSplitPane.isOneTouchExpandable could not be converted

javax.swing.JSplitPane.isValidateRoot could not be converted

javax.swing.JSplitPane.JSplitPane(int, boolean) could not be converted

javax.swing.JSplitPane.JSplitPane(int, boolean, Component, Component) could not be converted

javax.swing.JSplitPane.oneTouchExpandable could not be converted

javax.swing.JSplitPane.paintChildren could not be converted

javax.swing.JSplitPane.resetToPreferredSizes could not be converted

javax.swing.JSplitPane.setContinuousLayout could not be converted

javax.swing.JSplitPane.setOneTouchExpandable could not be converted

javax.swing.JSplitPane.setResizeWeight could not be converted

javax.swing.JSplitPane.setUI could not be converted

javax.swing.JSplitPane.updateUI could not be converted

javax.swing.JTabbedPane.add(Component) could not be converted

javax.swing.JTabbedPane.add(Component, int) could not be converted

javax.swing.JTabbedPane.add(Component, Object) could not be converted

javax.swing.JTabbedPane.add(Component, Object, int) could not be converted

javax.swing.JTabbedPane.add(String, Component) could not be converted

javax.swing.JTabbedPane.addTab(String, Component) could not be converted

javax.swing.JTabbedPane.addTab(String, Icon, Component, String) could not be converted

javax.swing.JTabbedPane.changeEvent could not be converted

javax.swing.JTabbedPane.changeListener could not be converted

javax.swing.JTabbedPane.createChangeListener could not be converted

javax.swing.JTabbedPane.fireStateChanged could not be converted

javax.swing.JTabbedPane.getBackgroundAt could not be converted

javax.swing.JTabbedPane.getBoundsAt could not be converted

javax.swing.JTabbedPane.getDisabledIconAt could not be converted

javax.swing.JTabbedPane.getForegroundAt could not be converted

javax.swing.JTabbedPane.getIconAt could not be converted

javax.swing.JTabbedPane.getModel could not be converted

javax.swing.JTabbedPane.getTabPlacement could not be converted

javax.swing.JTabbedPane.getTabRunCount could not be converted

javax.swing.JTabbedPane.getToolTipText could not be converted

javax.swing.JTabbedPane.getUI could not be converted

javax.swing.JTabbedPane.getUIClassID could not be converted

javax.swing.JTabbedPane.indexOfComponent could not be converted

javax.swing.JTabbedPane.indexOfTab(Icon) could not be converted

javax.swing.JTabbedPane.indexOfTab(String) could not be converted

javax.swing.JTabbedPane.insertTab could not be converted

javax.swing.JTabbedPane.JTabbedPane could not be converted

javax.swing.JTabbedPane.model could not be converted

javax.swing.JTabbedPane.paramString could not be converted

javax.swing.JTabbedPane.remove could not be converted

javax.swing.JTabbedPane.setBackgroundAt could not be converted

javax.swing.JTabbedPane.setComponentAt could not be converted

javax.swing.JTabbedPane.setDisabledIconAt could not be converted

javax.swing.JTabbedPane.setForegroundAt could not be converted

javax.swing.JTabbedPane.setIconAt could not be converted

javax.swing.JTabbedPane.setModel could not be converted

javax.swing.JTabbedPane.setSelectedComponent could not be converted

javax.swing.JTabbedPane.setSelectedIndex could not be converted

javax.swing.JTabbedPane.setTabPlacement could not be converted

javax.swing.JTabbedPane.setToolTipTextAt could not be converted

javax.swing.JTabbedPane.setUI could not be converted

javax.swing.JTabbedPane.tabPlacement could not be converted

javax.swing.JTabbedPane.updateUI could not be converted

javax.swing.JTable could not be converted

javax.swing.JTable.<ResizeType> could not be converted

javax.swing.JTable.addColumnSelectionInterval could not be converted

javax.swing.JTable.addRowSelectionInterval could not be converted

javax.swing.JTable.autoCreateColumnsFromModel could not be converted

javax.swing.JTable.autoResizeMode could not be converted

javax.swing.JTable.cellEditor could not be converted

javax.swing.JTable.cellSelectionEnabled could not be converted

javax.swing.JTable.changeSelection could not be converted

javax.swing.JTable.clearSelection could not be converted

javax.swing.JTable.columnAdded could not be converted

javax.swing.JTable.columnAtPoint could not be converted

javax.swing.JTable.columnMarginChanged could not be converted

javax.swing.JTable.columnMoved could not be converted

javax.swing.JTable.columnRemoved could not be converted

javax.swing.JTable.columnSelectionChanged could not be converted

javax.swing.JTable.configureEnclosingScrollPane could not be converted

javax.swing.JTable.convertColumnIndexToModel could not be converted

javax.swing.JTable.convertColumnIndexToView could not be converted

javax.swing.JTable.createDefaultColumnModel could not be converted

javax.swing.JTable.createDefaultColumnsFromModel could not be converted

javax.swing.JTable.createDefaultEditors could not be converted

javax.swing.JTable.createDefaultRenderers could not be converted

javax.swing.JTable.createDefaultSelectionModel could not be converted

javax.swing.JTable.createDefaultTableHeader could not be converted

javax.swing.JTable.createScrollPaneForTable could not be converted

javax.swing.JTable.defaultEditorsByColumnClass could not be converted

javax.swing.JTable.defaultRenderersByColumnClass could not be converted

javax.swing.JTable.doLayout could not be converted

javax.swing.JTable.editCellAt could not be converted

javax.swing.JTable.editingCanceled could not be converted

javax.swing.JTable.editingColumn could not be converted

javax.swing.JTable.editingRow could not be converted

javax.swing.JTable.editingStopped could not be converted

javax.swing.JTable.editorComp could not be converted

javax.swing.JTable.getAutoCreateColumnsFromModel could not be converted

javax.swing.JTable.getAutoResizeMode could not be converted

javax.swing.JTable.getCellEditor could not be converted

javax.swing.JTable.getCellRect could not be converted

javax.swing.JTable.getCellRenderer could not be converted

javax.swing.JTable.getCellSelectionEnabled could not be converted

javax.swing.JTable.getColumn could not be converted

javax.swing.JTable.getColumnSelectionAllowed could not be converted

javax.swing.JTable.getDefaultEditor could not be converted

javax.swing.JTable.getDefaultRenderer could not be converted

javax.swing.JTable.getEditingColumn could not be converted

javax.swing.JTable.getEditingRow could not be converted

javax.swing.JTable.getEditorComponent could not be converted

javax.swing.JTable.getInterCellSpacing could not be converted

javax.swing.JTable.getPreferredScrollableViewportSize could not be converted

javax.swing.JTable.getRowHeight could not be converted

javax.swing.JTable.getRowHeight(int) could not be converted

javax.swing.JTable.getRowMargin could not be converted

javax.swing.JTable.getRowSelectionAllowed could not be converted

javax.swing.JTable.getScrollableBlockIncrement could not be converted

javax.swing.JTable.getScrollableTracksViewportHeight could not be converted

javax.swing.JTable.getScrollableTracksViewportWidth could not be converted

javax.swing.JTable.getScrollableUnitIncrement could not be converted

javax.swing.JTable.getSelectedColumn could not be converted

javax.swing.JTable.getSelectedColumnCount could not be converted

javax.swing.JTable.getSelectedColumns could not be converted

javax.swing.JTable.getSelectedRowCount could not be converted

javax.swing.JTable.getSelectedRows could not be converted

javax.swing.JTable.getSelectionBackground could not be converted

javax.swing.JTable.getSelectionForeground could not be converted

javax.swing.JTable.getSelectionModel could not be converted

javax.swing.JTable.getShowHorizontalLines could not be converted

javax.swing.JTable.getShowVerticalLines could not be converted

javax.swing.JTable.getTableHeader could not be converted

javax.swing.JTable.getToolTipText could not be converted

javax.swing.JTable.getUI could not be converted

javax.swing.JTable.getUIClassID could not be converted

javax.swing.JTable.gridColor could not be converted

javax.swing.JTable.initializeLocalVars could not be converted

javax.swing.JTable.isCellEditable could not be converted

javax.swing.JTable.isCellSelected could not be converted

javax.swing.JTable.isColumnSelected could not be converted

javax.swing.JTable.isEditing could not be converted

javax.swing.JTable.isFocusTraversable could not be converted

javax.swing.JTable.isManagingFocus could not be converted

javax.swing.JTable.isRowSelected could not be converted

javax.swing.JTable.JTable could not be converted

javax.swing.JTable.JTable(int, int) could not be converted

javax.swing.JTable.JTable(Object[][], Object[]) could not be converted

javax.swing.JTable.JTable(TableModel) could not be converted

javax.swing.JTable.JTable(TableModel, TableColumnMode) could not be converted

javax.swing.JTable.JTable(TableModel, TableColumnModel, ListSelectionModel) could not be converted

javax.swing.JTable.JTable(Vector, Vector) could not be converted

javax.swing.JTable.moveColumn could not be converted

javax.swing.JTable.paramString could not be converted

javax.swing.JTable.preferredViewportSize could not be converted

javax.swing.JTable.prepareEditor could not be converted

javax.swing.JTable.prepareRenderer could not be converted

javax.swing.JTable.processKeyBinding could not be converted

javax.swing.JTable.removeColumnSelectionInterval could not be converted

javax.swing.JTable.removeEditor could not be converted

javax.swing.JTable.removeRowSelectionInterval could not be converted

javax.swing.JTable.rowAtPoint could not be converted

javax.swing.JTable.rowHeight could not be converted

javax.swing.JTable.rowMargin could not be converted

javax.swing.JTable.rowSelectionAllowed could not be converted

javax.swing.JTable.selectAll could not be converted

javax.swing.JTable.selectionBackground could not be converted

javax.swing.JTable.selectionForeground could not be converted

javax.swing.JTable.selectionModel could not be converted

javax.swing.JTable.setAutoCreateColumnsFromModel could not be converted

javax.swing.JTable.setAutoResizeMode could not be converted

javax.swing.JTable.setCellEditor could not be converted

javax.swing.JTable.setCellSelectionEnabled could not be converted

javax.swing.JTable.setColumnModel could not be converted

javax.swing.JTable.setColumnSelectionAllowed could not be converted

javax.swing.JTable.setColumnSelectionInterval could not be converted

javax.swing.JTable.setComponentOrientation could not be converted

javax.swing.JTable.setDefaultEditor could not be converted

javax.swing.JTable.setDefaultRenderer could not be converted

javax.swing.JTable.setEditingColumn could not be converted

javax.swing.JTable.setEditingRow could not be converted

javax.swing.JTable.setInterCellSpacing could not be converted

javax.swing.JTable.setPreferredScrollableViewportSize could not be converted

javax.swing.JTable.setRowHeight could not be converted

javax.swing.JTable.setRowMargin could not be converted

javax.swing.JTable.setRowSelectionAllowed could not be converted

javax.swing.JTable.setRowSelectionInterval could not be converted

javax.swing.JTable.setSelectionBackground could not be converted

javax.swing.JTable.setSelectionForeground could not be converted

javax.swing.JTable.setSelectionMode could not be converted

javax.swing.JTable.setSelectionModel could not be converted

javax.swing.JTable.setShowGrid could not be converted

javax.swing.JTable.setShowHorizontalLines could not be converted

javax.swing.JTable.setShowVerticalLines could not be converted

javax.swing.JTable.setTableHeader could not be converted

javax.swing.JTable.setUI could not be converted

javax.swing.JTable.showHorizontalLines could not be converted

javax.swing.JTable.showVerticalLines could not be converted

javax.swing.JTable.sizeColumnsToFit could not be converted

javax.swing.JTable.tableChanged could not be converted

javax.swing.JTable.tableHeader could not be converted

javax.swing.JTable.unconfigureEnclosingScrollPane could not be converted

javax.swing.JTable.updateUI could not be converted

javax.swing.JTable.valueChanged could not be converted

javax.swing.JTextArea.createDefaultModel could not be converted

javax.swing.JTextArea.getColumns could not be converted

javax.swing.JTextArea.getColumnWidth could not be converted

javax.swing.JTextArea.getLineEndOffset could not be converted

javax.swing.JTextArea.getLineOfOffset could not be converted

javax.swing.JTextArea.getLineStartOffset could not be converted

javax.swing.JTextArea.getPreferredScrollableViewportSize could not be converted

javax.swing.JTextArea.getRowHeight could not be converted

javax.swing.JTextArea.getRows could not be converted

javax.swing.JTextArea.getScrollableTracksViewportWidth could not be converted

javax.swing.JTextArea.getScrollableUnitIncrement could not be converted

javax.swing.JTextArea.getTabSize could not be converted

javax.swing.JTextArea.getUIClassID could not be converted

javax.swing.JTextArea.getWrapStyleWord could not be converted

javax.swing.JTextArea.isManagingFocus could not be converted

javax.swing.JTextArea.JTextArea(Document) could not be converted

javax.swing.JTextArea.JTextArea(Document, String, int, int) could not be converted

javax.swing.JTextArea.setColumns could not be converted

javax.swing.JTextArea.setRows could not be converted

javax.swing.JTextArea.setTabSize could not be converted

javax.swing.JTextField.configurePropertiesFromAction could not be converted

javax.swing.JTextField.createActionPropertyChangeListener could not be converted

javax.swing.JTextField.createDefaultModel could not be converted

javax.swing.JTextField.fireActionPerformed could not be converted

javax.swing.JTextField.getAction could not be converted

javax.swing.JTextField.getActions could not be converted

javax.swing.JTextField.setColumns could not be converted

javax.swing.JTextField.getColumnWidth could not be converted

javax.swing.JTextField.getHorizontalVisibility could not be converted

javax.swing.JTextField.getScrollOffset could not be converted

javax.swing.JTextField.getUIClassID could not be converted

javax.swing.JTextField.isValidateRoot could not be converted

javax.swing.JTextField.JTextField(Document, String, int) could not be converted

javax.swing.JTextField.JTextField(int) could not be converted

javax.swing.JTextField.JTextField(String, int) could not be converted

javax.swing.JTextField.notifyAction could not be converted

javax.swing.JTextField.postActionEvent could not be converted

javax.swing.JTextField.scrollRectToVisible could not be converted

javax.swing.JTextField.setActionCommand could not be converted

javax.swing.JTextField.setColumns could not be converted

javax.swing.JTextField.setScrollOffset could not be converted

javax.swing.JTextPane.addStyle could not be converted

javax.swing.JTextPane.createDefaultEditorKit could not be converted

javax.swing.JTextPane.getCharacterAttributes could not be converted

javax.swing.JTextPane.getInputAttributes could not be converted

javax.swing.JTextPane.getLogicalStyle could not be converted

javax.swing.JTextPane.getParagraphAttributes could not be converted

javax.swing.JTextPane.getStyle could not be converted

javax.swing.JTextPane.getStyledDocument could not be converted

javax.swing.JTextPane.getStyledEditorKit could not be converted

javax.swing.JTextPane.getUIClassID could not be converted

javax.swing.JTextPane.insertComponent could not be converted

javax.swing.JTextPane.insertIcon could not be converted

javax.swing.JTextPane.removeStylev could not be converted

javax.swing.JTextPane.setCharacterAttributes could not be converted

javax.swing.JTextPane.setDocument could not be converted

javax.swing.JTextPane.setEditorKit could not be converted

javax.swing.JTextPane.setLogicalStyle could not be converted

javax.swing.JTextPane.setParagraphAttributes could not be converted

javax.swing.JTextPane.setStyleedDocument could not be converted

javax.swing.JToggleButton.getUIClassID could not be converted

javax.swing.JToggleButton.JToggleButton could not be converted

javax.swing.JToggleButton.ToggleButtonModel could not be converted

javax.swing.JToggleButton.updateUI could not be converted

javax.swing.JToolBar.addSeparator could not be converted

javax.swing.JToolBar.addSeparator(Dimension) could not be converted

javax.swing.JToolBar.createActionChangeListener could not be converted

javax.swing.JToolBar.getComponentAtIndex could not be converted

javax.swing.JToolBar.getComponentIndex could not be converted

javax.swing.JToolBar.getMargin could not be converted

javax.swing.JToolBar.getOrientation could not be converted

javax.swing.JToolBar.getUI could not be converted

javax.swing.JToolBar.getUIClassID could not be converted

javax.swing.JToolBar.isBorderPainted could not be converted

javax.swing.JToolBar.isFloatable could not be converted

javax.swing.JToolBar.paintBorder could not be converted

javax.swing.JToolBar.Separator could not be converted

javax.swing.JToolBar.setBorderPainted could not be converted

javax.swing.JToolBar.setFloatable could not be converted

javax.swing.JToolBar.setMargin could not be converted

javax.swing.JToolBar.setOrientation could not be converted

javax.swing.JToolBar.setUI could not be converted

javax.swing.JToolBar.updateUI could not be converted

javax.swing.JToolTip.getAccessibleContext could not be converted

javax.swing.JToolTip.getComponent could not be converted

javax.swing.JToolTip.getTipText could not be converted

javax.swing.JToolTip.getUI could not be converted

javax.swing.JToolTip.getUIClassID could not be converted

javax.swing.JToolTip.paramString could not be converted

javax.swing.JToolTip.setComponent could not be converted

javax.swing.JToolTip.setTipText could not be converted

javax.swing.JToolTip.updateUI could not be converted

javax.swing.JTree could not be converted

javax.swing.JTree.<PropertyName> could not be converted
javax.swing.JTree.addSelectionInterval could not be converted
javax.swing.JTree.addSelectionPath could not be converted
javax.swing.JTree.addSelectionPaths could not be converted
javax.swing.JTree.addSelectionRow could not be converted
javax.swing.JTree.addSelectionRows could not be converted
javax.swing.JTree.cancelEditing could not be converted
javax.swing.JTree.cellEditor could not be converted
javax.swing.JTree.cellRenderer could not be converted
javax.swing.JTree.clearToggledPaths could not be converted
javax.swing.JTree.collapsePath could not be converted
javax.swing.JTree.collapseRow could not be converted
javax.swing.JTree.convertValueToText could not be converted
javax.swing.JTree.createTreeModel could not be converted
javax.swing.JTree.createTreeModelListener could not be converted
javax.swing.JTree.DynamicUtilTreeNode could not be converted
javax.swing.JTree.expandPath could not be converted
javax.swing.JTree.expandRow could not be converted
javax.swing.JTree.fireTreeCollapsed could not be converted
javax.swing.JTree.fireTreeExpanded could not be converted
javax.swing.JTree.fireTreeWillCollapse could not be converted
javax.swing.JTree.fireTreeWillExpand could not be converted
javax.swing.JTree.fireValueChanged could not be converted
javax.swing.JTree.getAnchorSelectionPath could not be converted
javax.swing.JTree.getCellEditor could not be converted
javax.swing.JTree.getCellRenderer could not be converted
javax.swing.JTree.getClosestPathForLocation could not be converted
javax.swing.JTree.getClosestRowForLocation could not be converted
javax.swing.JTree.getDefaultTreeModel could not be converted
javax.swing.JTree.getDescendantToggledPaths could not be converted
javax.swing.JTree.getEditingPath could not be converted
javax.swing.JTree.getExpandedDescendants could not be converted
javax.swing.JTree.getExpandsSelectedPaths could not be converted
javax.swing.JTree.getInvokesStopCellEditing could not be converted
javax.swing.JTree.getLeadSelectionPath could not be converted
javax.swing.JTree.getLeadSelectionRow could not be converted
javax.swing.JTree.getMaxSelectionRow could not be converted
javax.swing.JTree.getMinSelectionRow could not be converted
javax.swing.JTree.getModel could not be converted
javax.swing.JTree.getPathBetweenRows could not be converted

javax.swing.JTree.getPathBounds could not be converted

javax.swing.JTree.getPathForLocation could not be converted

javax.swing.JTree.getPathForRow could not be converted

javax.swing.JTree.getPreferredScrollableViewportSize could not be converted

javax.swing.JTree.getRowBounds could not be converted

javax.swing.JTree.getRowCount could not be converted

javax.swing.JTree.getRowForLocation could not be converted

javax.swing.JTree.getRowForPath could not be converted

javax.swing.JTree.getRowHeight could not be converted

javax.swing.JTree.getScrollableBlockIncrement could not be converted

javax.swing.JTree.getScrollableTracksViewportHeight could not be converted

javax.swing.JTree.getScrollableTracksViewportWidth could not be converted

javax.swing.JTree.getScrollableUnitIncrement could not be converted

javax.swing.JTree.getScrollsOnExpand could not be converted

javax.swing.JTree.getSelectionCount could not be converted

javax.swing.JTree.getSelectionModel could not be converted

javax.swing.JTree.getSelectionPath could not be converted

javax.swing.JTree.getSelectionPaths could not be converted

javax.swing.JTree.getSelectionRows could not be converted

javax.swing.JTree.getToggleClickCount could not be converted

javax.swing.JTree.getToolTipText could not be converted

javax.swing.JTree.getUI could not be converted

javax.swing.JTree.getUIClassID could not be converted

javax.swing.JTree.hasBeenExpanded could not be converted

javax.swing.JTree.invokesStopCellEditing could not be converted

javax.swing.JTree.isCollapsed(int) could not be converted

javax.swing.JTree.isCollapsed(TreePath) could not be converted

javax.swing.JTree.isEditing could not be converted

javax.swing.JTree.isExpanded(int) could not be converted

javax.swing.JTree.isExpanded(TreePath) could not be converted

javax.swing.JTree.isFixedRowHeight could not be converted

javax.swing.JTree.isLargeModel could not be converted

javax.swing.JTree.isPathEditable could not be converted

javax.swing.JTree.isPathSelected could not be converted

javax.swing.JTree.isRootVisible could not be converted

javax.swing.JTree.isRowSelected could not be converted

javax.swing.JTree.isVisible could not be converted

javax.swing.JTree.JTree(Hashtable) could not be converted

javax.swing.JTree.JTree(TreeNode, boolean) could not be converted

javax.swing.JTree.largeModel could not be converted

javax.swing.JTree.makeVisible could not be converted

javax.swing.JTree.removeDescendantSelectedPaths could not be converted

javax.swing.JTree.removeDescendantToggledPaths could not be converted

javax.swing.JTree.removeSelectionInterval could not be converted

javax.swing.JTree.removeSelectionPath could not be converted

javax.swing.JTree.removeSelectionPaths could not be converted

javax.swing.JTree.removeSelectionRow could not be converted

javax.swing.JTree.removeSelectionRows could not be converted

javax.swing.JTree.rootVisible could not be converted

javax.swing.JTree.scrollPathToVisible could not be converted

javax.swing.JTree.scrollRowToVisible could not be converted

javax.swing.JTree.scrollsOnExpand could not be converted

javax.swing.JTree.selectionModel could not be converted

javax.swing.JTree.selectionRedirector could not be converted

javax.swing.JTree.setAnchorSelectionPath could not be converted

javax.swing.JTree.setCellEditor could not be converted

javax.swing.JTree.setCellRenderer could not be converted

javax.swing.JTree.setExpandedState could not be converted

javax.swing.JTree.setExpandsSelectedPaths could not be converted

javax.swing.JTree.setInvokesStopCellEditing could not be converted

javax.swing.JTree.setLargeModel could not be converted

javax.swing.JTree.setLeadSelectionPath could not be converted

javax.swing.JTree.setRootVisible could not be converted

javax.swing.JTree.setRowHeight could not be converted

javax.swing.JTree.setScrollsOnExpand could not be converted

javax.swing.JTree.setSelectionInterval could not be converted

javax.swing.JTree.setSelectionModel could not be converted

javax.swing.JTree.setSelectionPath could not be converted

javax.swing.JTree.setSelectionPaths could not be converted

javax.swing.JTree.setSelectionRow could not be converted

javax.swing.JTree.setSelectionRows could not be converted

javax.swing.JTree.setToggleClickCount could not be converted

javax.swing.JTree.setUI could not be converted

javax.swing.JTree.setVisibleRowCount could not be converted

javax.swing.JTree.startEditingAtPath could not be converted

javax.swing.JTree.stopEditing could not be converted

javax.swing.JTree.toggleClickCount could not be converted

javax.swing.JTree.treeDidChange could not be converted

javax.swing.JTree.treeModel could not be converted

javax.swing.JTree.treeModelListener could not be converted

javax.swing.JTree.updateUI could not be converted

javax.swing.JTree.visibleRowCount could not be converted

javax.swing.JViewport could not be converted

javax.swing.JWindow.getGlassPane could not be converted

javax.swing.JWindow.getLayeredPane could not be converted

javax.swing.JWindow.isRootPaneCheckingEnabled could not be converted

javax.swing.JWindow.JWindow(GraphicsConfiguration) could not be converted

javax.swing.JWindow.JWindow(GraphicsConfiguration, Window) could not be converted

javax.swing.JWindow.rootPane could not be converted

javax.swing.JWindow.rootPaneCheckingEnabled could not be converted

javax.swing.JWindow.setContentPane could not be converted

javax.swing.JWindow.setGlassPane could not be converted

javax.swing.JWindow.setLayeredPane could not be converted

javax.swing.JWindow.setRootPane could not be converted

javax.swing.JWindow.setRootPaneCheckingEnabled could not be converted

javax.swing.JWindow.windowInit could not be converted

javax.swing.KeyStroke.getKeyStroke could not be converted

javax.swing.KeyStroke.getKeyStrokeForEvent could not be converted

javax.swing.KeyStroke.isOnKeyRelease could not be converted

javax.swing.KeyStroke.KeyStroke could not be converted

javax.swing.ListCellRenderer could not be converted

javax.swing.ListModel could not be converted

javax.swing.ListModel.addListDataListener could not be converted

javax.swing.ListModel.removeListDataListener could not be converted

javax.swing.ListSelectionModel could not be converted

javax.swing.ListSelectionModel.addListSelectionListener could not be converted

javax.swing.ListSelectionModel.getValuesAdjusting could not be converted

javax.swing.ListSelectionModel.insertIndexInterval could not be converted

javax.swing.ListSelectionModel.removeListSelectionListener could not be converted

javax.swing.ListSelectionModel.setValuesAdjusting could not be converted

javax.swing.LookAndFeel could not be converted

javax.swing.MenuElement.menuSelectionChanged could not be converted

javax.swing.MenuSelectionManager could not be converted

javax.swing.OverlayLayout could not be converted

javax.swing.ProgressMonitor could not be converted

javax.swing.ProgressMonitorInputStream could not be converted

javax.swing.Renderer could not be converted

javax.swing.RepaintManager could not be converted

javax.swing.RootPaneContainer.getGlassPane could not be converted

javax.swing.RootPaneContainer.getLayeredPane could not be converted

javax.swing.RootPaneContainer.setContentPane could not be converted

javax.swing.RootPaneContainer.setGlassPane could not be converted

javax.swing.RootPaneContainer.setLayeredPane could not be converted

javax.swing.Scrollable could not be converted

javax.swing.ScrollPaneConstants could not be converted

javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED could not be converted

javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER could not be converted

javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED could not be converted

javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER could not be converted

javax.swing.ScrollPaneLayout could not be converted

javax.swing.ScrollPaneLayout.UIResource could not be converted

javax.swing.SingleSelectionModel could not be converted

javax.swing.SingleSelectionModel.addChangeListener could not be converted

javax.swing.SingleSelectionModel.removeChangeListener could not be converted

javax.swing.SizeRequirements could not be converted

javax.swing.SizeSequence could not be converted

javax.swing.SizeSequence.getIndex could not be converted

javax.swing.SizeSequence.getPosition could not be converted

javax.swing.SizeSequence.getSize could not be converted

javax.swing.SizeSequence.getSizes could not be converted

javax.swing.SizeSequence.insertEntries could not be converted

javax.swing.SizeSequence.removeEntries could not be converted

javax.swing.SizeSequence.setSize could not be converted

javax.swing.SizeSequence.setSizes could not be converted

javax.swing.SizeSequence.SizeSequence could not be converted

javax.swing.SwingConstants could not be converted

javax.swing.SwingUtilities could not be converted

javax.swing.SwingUtilities.getAccessibleAt could not be converted

javax.swing.SwingUtilities.getAncestorNamed could not be converted

javax.swing.SwingUtilities.getDeepestComponentAt could not be converted

javax.swing.SwingUtilities.getLocalBounds could not be converted

javax.swing.SwingUtilities.getRoot could not be converted

javax.swing.SwingUtilities.getRootPane could not be converted

javax.swing.SwingUtilities.getWindowAncestor could not be converted

javax.swing.Timer.fireActionPerformed could not be converted

javax.swing.Timer.getInitialDelay could not be converted

javax.swing.Timer.getListeners could not be converted

javax.swing.Timer.getLogTimers could not be converted

javax.swing.Timer.isCoalesce could not be converted

javax.swing.Timer.restart could not be converted

javax.swing.Timer.setCoalesce could not be converted
javax.swing.Timer.setInitialDelay could not be converted
javax.swing.Timer.setLogTimers could not be converted
javax.swing.ToolTipManager-heavyWeightPopupEnabled could not be converted
javax.swing.ToolTipManager.isLightWeightPopupEnabled could not be converted
javax.swing.ToolTipManager.lightWeightPopupEnabled could not be converted
javax.swing.ToolTipManager.mouseDragged could not be converted
javax.swing.ToolTipManager.mouseEntered could not be converted
javax.swing.ToolTipManager.mouseExited could not be converted
javax.swing.ToolTipManager.mouseMoved could not be converted
javax.swing.ToolTipManager.mousePressed could not be converted
javax.swing.ToolTipManager.registerComponent could not be converted
javax.swing.ToolTipManager.setLightWeightPopupEnabled could not be converted
javax.swing.ToolTipManager.ToolTipManager could not be converted
javax.swing.ToolTipManager.unregisterComponent could not be converted
javax.swing.UIDefaults could not be converted
javax.swing.UIDefaults.ActiveValue could not be converted
javax.swing.UIDefaults.LazyInputMap could not be converted
javax.swing.UIDefaults.LazyValue could not be converted
javax.swing.UIDefaults.ProxyLazyValue could not be converted
javax.swing.UIManager could not be converted
javax.swing.UIManager.LookAndFeelInfo could not be converted
javax.swing.UnsupportedLookAndFeelException could not be converted
javax.swing.ViewportLayout could not be converted
javax.swing.WindowConstants could not be converted

Javax.swing.beaninfo Error Messages

[javax.swing.beaninfo.SwingBeanInfo could not be converted](#)

Javax.swing.border Error Messages

javax.swing.border.AbstractBorder could not be converted

javax.swing.border.BevelBorder could not be converted

javax.swing.border.BevelBorder.<Type> could not be converted

javax.swing.border.BevelBorder.BevelBorder could not be converted

javax.swing.border.BevelBorder.bevelType could not be converted

javax.swing.border.BevelBorder.getBevelType could not be converted

javax.swing.border.BevelBorder.getBorderInsets could not be converted

javax.swing.border.BevelBorder.getHighlightInnerColor could not be converted

javax.swing.border.BevelBorder.getHighlightOuterColor could not be converted

javax.swing.border.BevelBorder.getShadowInnerColor could not be converted

javax.swing.border.BevelBorder.getShadowOuterColor could not be converted

javax.swing.border.BevelBorder.highlightInner could not be converted

javax.swing.border.BevelBorder.highlightOuter could not be converted

javax.swing.border.BevelBorder.isBorderOpaque could not be converted

javax.swing.border.BevelBorder.paintBorder could not be converted

javax.swing.border.BevelBorder.paintLoweredBevel could not be converted

javax.swing.border.BevelBorder.paintRaisedBevel could not be converted

javax.swing.border.BevelBorder.shadowInner could not be converted

javax.swing.border.BevelBorder.shadowOuter could not be converted

javax.swing.border.Border could not be converted

javax.swing.border.Border.getBorderInsets could not be converted

javax.swing.border.Border.isBorderOpaque could not be converted

javax.swing.border.Border.paintBorder could not be converted

javax.swing.border.CompoundBorder could not be converted

javax.swing.border.EmptyBorder could not be converted

javax.swing.border.EmptyBorder.bottom could not be converted

javax.swing.border.EmptyBorder.EmptyBorder could not be converted

javax.swing.border.EmptyBorder.getBorderInsets could not be converted

javax.swing.border.EmptyBorder.isBorderOpaque could not be converted

javax.swing.border.EmptyBorder.left could not be converted

javax.swing.border.EmptyBorder.paintBorder could not be converted

javax.swing.border.EmptyBorder.right could not be converted

javax.swing.border.EmptyBorder.top could not be converted

javax.swing.border.EtchedBorder could not be converted

javax.swing.border.EtchedBorder.<Type> could not be converted

javax.swing.border.EtchedBorder.EtchedBorder could not be converted

javax.swing.border.EtchedBorder.etchType could not be converted

javax.swing.border.EtchedBorder.getBorderInsets could not be converted
javax.swing.border.EtchedBorder.getEtchType could not be converted
javax.swing.border.EtchedBorder.getHighlightColor could not be converted
javax.swing.border.EtchedBorder.getShadowColor could not be converted
javax.swing.border.EtchedBorder.highlight could not be converted
javax.swing.border.EtchedBorder.isBorderOpaque could not be converted
javax.swing.border.EtchedBorder.paintBorder could not be converted
javax.swing.border.EtchedBorder.shadow could not be converted
javax.swing.border.LineBorder could not be converted
javax.swing.border.LineBorder.createBlackLineBorder could not be converted
javax.swing.border.LineBorder.createGrayLineBorder could not be converted
javax.swing.border.LineBorder.getBorderInsets could not be converted
javax.swing.border.LineBorder.getLineColor could not be converted
javax.swing.border.LineBorder.getRoundedCorners could not be converted
javax.swing.border.LineBorder.getThickness could not be converted
javax.swing.border.LineBorder.isBorderOpaque could not be converted
javax.swing.border.LineBorder.LineBorder could not be converted
javax.swing.border.LineBorder.lineColor could not be converted
javax.swing.border.LineBorder.paintBorder could not be converted
javax.swing.border.LineBorder.roundedCorners could not be converted
javax.swing.border.LineBorder.thickness could not be converted
javax.swing.border.MatteBorder could not be converted
javax.swing.border.SoftBevelBorder could not be converted
javax.swing.border.SoftBevelBorder.getBorderInsets could not be converted
javax.swing.border.SoftBevelBorder.isBorderOpaque could not be converted
javax.swing.border.SoftBevelBorder.paintBorder could not be converted
javax.swing.border.SoftBevelBorder.SoftBevelBorder could not be converted
javax.swing.border.TitledBorder could not be converted

Javax.swing.colorchooser Error Messages

[javax.swing.colorchooser.AbstractColorChooserPanel could not be converted](#)

[javax.swing.colorchooser.ColorChooserComponentFactory could not be converted](#)

[javax.swing.colorchooser.ColorSelectionModel could not be converted](#)

[javax.swing.colorchooser.DefaultColorSelectionModel could not be converted](#)

Javax.swing.event Error Messages

[javax.swing.event.AncestorEvent could not be converted](#)

[javax.swing.event.CaretEvent could not be converted](#)

[javax.swing.event.DocumentEvent could not be converted](#)

[javax.swing.event.DocumentEvent.ElementChange could not be converted](#)

[javax.swing.event.DocumentEvent.EventType could not be converted](#)

[javax.swing.event.EventListenerList.getListenerCount could not be converted](#)

[javax.swing.event.EventListenerList.getListeners could not be converted](#)

[javax.swing.event.HyperlinkEvent.EventType could not be converted](#)

[javax.swing.event.HyperlinkEvent.getDescription could not be converted](#)

[javax.swing.event.HyperlinkEvent.getEventType could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_ACTIVATED could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_CLOSED could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_CLOSING could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_DEACTIVATED could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_DEICONIFIED could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_FIRST could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_ICONIFIED could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_LAST could not be converted](#)

[javax.swing.event.InternalFrameEvent.INTERNAL_FRAME_OPENED could not be converted](#)

[javax.swing.event.ListDataEvent could not be converted](#)

[javax.swing.event.ListSelectionEvent.getFirstIndex could not be converted](#)

[javax.swing.event.ListSelectionEvent.getLastIndex could not be converted](#)

[javax.swing.event.ListSelectionEvent.getValuesAdjusting could not be converted](#)

[javax.swing.event.MenuDragMouseEvent could not be converted](#)

[javax.swing.event.MenuKeyEvent could not be converted](#)

[javax.swing.event.SwingPropertyChangeSupport could not be converted](#)

[javax.swing.event.TableColumnModelEvent.fromIndex could not be converted](#)

[javax.swing.event.TableColumnModelEvent.getFromIndex could not be converted](#)

[javax.swing.event.TableColumnModelEvent.getToIndex could not be converted](#)

[javax.swing.event.TableColumnModelEvent.TableColumnModelEvent could not be converted](#)

[javax.swing.event.TableColumnModelEvent.toIndex could not be converted](#)

[javax.swing.event.TableModelEvent.ALL_COLUMNS could not be converted](#)

[javax.swing.event.TableModelEvent.column could not be converted](#)

[javax.swing.event.TableModelEvent.firstRow could not be converted](#)

[javax.swing.event.TableModelEvent.getColumn could not be converted](#)

[javax.swing.event.TableModelEvent.getFirstRow could not be converted](#)

[javax.swing.event.TableModelEvent.getLastRow could not be converted](#)

javax.swing.event.TableModelEvent.HEADER_ROW could not be converted
javax.swing.event.TableModelEvent.lastRow could not be converted
javax.swing.event.TableModelEvent.TableModelEvent could not be converted
javax.swing.event.TreeExpansionEvent.getPath could not be converted
javax.swing.event.TreeExpansionEvent.path could not be converted
javax.swing.event.TreeExpansionEvent.TreeExpansionEvent could not be converted
javax.swing.event.TreeModelEvent could not be converted
javax.swing.event.TreeSelectionEvent.areNew could not be converted
javax.swing.event.TreeSelectionEvent.cloneWithSource could not be converted
javax.swing.event.TreeSelectionEvent.getNewLeadSelectionPath could not be converted
javax.swing.event.TreeSelectionEvent.getOldLeadSelectionPath could not be converted
javax.swing.event.TreeSelectionEvent.getPath could not be converted
javax.swing.event.TreeSelectionEvent.getPaths could not be converted
javax.swing.event.TreeSelectionEvent.isAddedPath could not be converted
javax.swing.event.TreeSelectionEvent.newLeadSelectionPath could not be converted
javax.swing.event.TreeSelectionEvent.oldLeadSelectionPath could not be converted
javax.swing.event.TreeSelectionEvent.paths could not be converted
javax.swing.event.TreeSelectionEvent.TreeSelectionEvent could not be converted
javax.swing.event.UndoableEditEvent could not be converted

Javax.swing.filechooser Error Messages

[javax.swing.filechooser.FileFilter](#) could not be converted

[javax.swing.filechooser.FileSystemView.createNewFolder](#) could not be converted

[javax.swing.filechooser.FileSystemView.FileSystemView](#) could not be converted

[javax.swing.filechooser.FileSystemView.getFileSystemView](#) could not be converted

[javax.swing.filechooser.FileSystemView.getHomeDirectory](#) could not be converted

[javax.swing.filechooser.FileSystemView.getRoots](#) could not be converted

[javax.swing.filechooser.FileSystemView.isRoot](#) could not be converted

[javax.swing.filechooser.FileView](#) could not be converted

Javax.swing.table Error Messages

javax.swing.table.AbstractTableModel could not be converted

javax.swing.table.AbstractTableModel.addTableModelListener could not be converted

javax.swing.table.AbstractTableModel.fireTableCellUpdated could not be converted

javax.swing.table.AbstractTableModel.fireTableChanged could not be converted

javax.swing.table.AbstractTableModel.fireTableDataChanged could not be converted

javax.swing.table.AbstractTableModel.fireTableRowsDeleted could not be converted

javax.swing.table.AbstractTableModel.fireTableRowsInserted could not be converted

javax.swing.table.AbstractTableModel.fireTableRowsUpdated could not be converted

javax.swing.table.AbstractTableModel.fireTableStructureChanged could not be converted

javax.swing.table.AbstractTableModel.listeners could not be converted

javax.swing.table.AbstractTableModel.listenerList could not be converted

javax.swing.table.AbstractTableModel.removeTableModelListener could not be converted

javax.swing.table.DefaultTableCellRenderer could not be converted

javax.swing.table.DefaultTableCellRenderer.UIResource could not be converted

javax.swing.table.DefaultTableColumnModel could not be converted

javax.swing.table.DefaultTableColumnModel.addColumnModelListener could not be converted

javax.swing.table.DefaultTableColumnModel.changeEvent could not be converted

javax.swing.table.DefaultTableColumnModel.columnMargin could not be converted

javax.swing.table.DefaultTableColumnModel.columnSelectionAllowed could not be converted

javax.swing.table.DefaultTableColumnModel.createSelectionModel could not be converted

javax.swing.table.DefaultTableColumnModel.DefaultTableColumnModel could not be converted

javax.swing.table.DefaultTableColumnModel.fireColumnAdded could not be converted

javax.swing.table.DefaultTableColumnModel.fireColumnMarginChanged could not be converted

javax.swing.table.DefaultTableColumnModel.fireColumnMoved could not be converted

javax.swing.table.DefaultTableColumnModel.fireColumnRemoved could not be converted

javax.swing.table.DefaultTableColumnModel.fireColumnSelectionChanged could not be converted

javax.swing.table.DefaultTableColumnModel.getColumnIndexAtX could not be converted

javax.swing.table.DefaultTableColumnModel.getColumnMargin could not be converted

javax.swing.table.DefaultTableColumnModel.getColumnSelectionAllowed could not be converted

javax.swing.table.DefaultTableColumnModel.listeners could not be converted

javax.swing.table.DefaultTableColumnModel.getSelectedColumnCount could not be converted

javax.swing.table.DefaultTableColumnModel.getSelectedColumns could not be converted

javax.swing.table.DefaultTableColumnModel.getSelectionModel could not be converted

javax.swing.table.DefaultTableColumnModel.getTotalColumnWidth could not be converted

javax.swing.table.DefaultTableColumnModel.listenerList could not be converted

javax.swing.table.DefaultTableColumnModel.moveColumn could not be converted

javax.swing.table.DefaultTableColumnModel.propertyChange could not be converted

javax.swing.table.DefaultTableColumnModel.recalcWidthCache could not be converted

javax.swing.table.DefaultTableColumnModel.removeColumnModelListener could not be converted

javax.swing.table.DefaultTableColumnModel.selectionModel could not be converted

javax.swing.table.DefaultTableColumnModel.setColumnMargin could not be converted

javax.swing.table.DefaultTableColumnModel.setColumnSelectionAllowed could not be converted

javax.swing.table.DefaultTableColumnModel.setSelectionModel could not be converted

javax.swing.table.DefaultTableColumnModel.tableColumns could not be converted

javax.swing.table.DefaultTableColumnModel.totalColumnWidth could not be converted

javax.swing.table.DefaultTableColumnModel.valueChanged could not be converted

javax.swing.table.DefaultTableModel could not be converted

javax.swing.table.DefaultTableModel.addColumn(Object, Object[]) could not be converted

javax.swing.table.DefaultTableModel.addColumn(Object, Vector) could not be converted

javax.swing.table.DefaultTableModel.columnIdentifiers could not be converted

javax.swing.table.DefaultTableModel.dataVector could not be converted

javax.swing.table.DefaultTableModel.DefaultTableModel(int, int) could not be converted

javax.swing.table.DefaultTableModel.DefaultTableModel(Object[], int) could not be converted

javax.swing.table.DefaultTableModel.DefaultTableModel(Object[][], Object[]) could not be converted

javax.swing.table.DefaultTableModel.DefaultTableModel(Vector, int) could not be converted

javax.swing.table.DefaultTableModel.DefaultTableModel(Vector, Vector) could not be converted

javax.swing.table.DefaultTableModel.getDataVector could not be converted

javax.swing.table.DefaultTableModel.insertRow could not be converted

javax.swing.table.DefaultTableModel.isCellEditable could not be converted

javax.swing.table.DefaultTableModel.moveRow could not be converted

javax.swing.table.DefaultTableModel.newDataAvailable could not be converted

javax.swing.table.DefaultTableModel.newRowsAdded could not be converted

javax.swing.table.DefaultTableModel.rowsRemoved could not be converted

javax.swing.table.DefaultTableModel.setColumnCount could not be converted

javax.swing.table.DefaultTableModel.setColumnIdentifiers could not be converted

javax.swing.table.DefaultTableModel.setDataVector could not be converted

javax.swing.table.DefaultTableModel.setNumRows could not be converted

javax.swing.table.DefaultTableModel.setRowCount could not be converted

javax.swing.table.JTableHeader could not be converted

javax.swing.table.TableCellEditor could not be converted

javax.swing.table.TableCellRenderer could not be converted

javax.swing.table.TableColumn could not be converted

javax.swing.table.TableColumn.<PropertyName> could not be converted

javax.swing.table.TableColumn.addPropertyChangeListener could not be converted

javax.swing.table.TableColumn.disableResizedPosting could not be converted

javax.swing.table.TableColumn.enableResizedPosting could not be converted

javax.swing.table.TableColumn.getHeaderValue could not be converted

javax.swing.table.TableColumn.getIdentifier could not be converted
javax.swing.table.TableColumn.getMaxWidth could not be converted
javax.swing.table.TableColumn.getMinWidth could not be converted
javax.swing.table.TableColumn.getModelIndex could not be converted
javax.swing.table.TableColumn.getPreferredWidth could not be converted
javax.swing.table.TableColumn.getResizable could not be converted
javax.swing.table.TableColumn.getWidth could not be converted
javax.swing.table.TableColumn.headerValue could not be converted
javax.swing.table.TableColumn.identifier could not be converted
javax.swing.table.TableColumn.isResizable could not be converted
javax.swing.table.TableColumn.maxWidth could not be converted
javax.swing.table.TableColumn.minWidth could not be converted
javax.swing.table.TableColumn.modelIndex could not be converted
javax.swing.table.TableColumn.removePropertyChangeListener could not be converted
javax.swing.table.TableColumn.resizedPostingDisableCount could not be converted
javax.swing.table.TableColumn.setHeaderValue could not be converted
javax.swing.table.TableColumn.setIdentifier could not be converted
javax.swing.table.TableColumn.setMaxWidth could not be converted
javax.swing.table.TableColumn.setMinWidth could not be converted
javax.swing.table.TableColumn.setModelIndex could not be converted
javax.swing.table.TableColumn.setPreferredWidth could not be converted
javax.swing.table.TableColumn.setResizable could not be converted
javax.swing.table.TableColumn.setWidth could not be converted
javax.swing.table.TableColumn.sizeWidthToFit could not be converted
javax.swing.table.TableColumn.TableColumn could not be converted
javax.swing.table.TableColumn.width could not be converted
javax.swing.table.TableColumnModel could not be converted
javax.swing.table.TableColumnModel.getColumnIndexAtX could not be converted
javax.swing.table.TableColumnModel.getColumnModelMargin could not be converted
javax.swing.table.TableColumnModel.addColumnModelListener could not be converted
javax.swing.table.TableColumnModel.getColumnModelSelectionAllowed could not be converted
javax.swing.table.TableColumnModel.getSelectedColumnCount could not be converted
javax.swing.table.TableColumnModel.getSelectedColumns could not be converted
javax.swing.table.TableColumnModel.getSelectionModel could not be converted
javax.swing.table.TableColumnModel.getTotalColumnWidth could not be converted
javax.swing.table.TableColumnModel.moveColumn could not be converted
javax.swing.table.TableColumnModel.removeColumnModelListener could not be converted
javax.swing.table.TableColumnModel.setColumnMargin could not be converted
javax.swing.table.TableColumnModel.setColumnSelectionAllowed could not be converted
javax.swing.table.TableColumnModel.setSelectionModel could not be converted

javax.swing.table.TableModel could not be converted

javax.swing.table.TableModel.addTableModelListener could not be converted

javax.swing.table.TableModel.removeTableModelListener could not be converted

Javax.swing.text Error Messages

javax.swing.text.AbstractDocument could not be converted

javax.swing.text.AbstractDocument.AbstractElement could not be converted

javax.swing.text.AbstractDocument.AttributeContext could not be converted

javax.swing.text.AbstractDocument.BranchElement could not be converted

javax.swing.text.AbstractDocument.Content could not be converted

javax.swing.text.AbstractDocument.DefaultDocumentEvent could not be converted

javax.swing.text.AbstractDocument.ElementEdit could not be converted

javax.swing.text.AbstractDocument.LeafElement could not be converted

javax.swing.text.AbstractWriter.AbstractWriter could not be converted

javax.swing.text.AbstractWriter.decrIndent could not be converted

javax.swing.text.AbstractWriter.getCanWrapLines could not be converted

javax.swing.text.AbstractWriter.getCurrentLineLength could not be converted

javax.swing.text.AbstractWriter.getDocument could not be converted

javax.swing.text.AbstractWriter.getElementIterator could not be converted

javax.swing.text.AbstractWriter.getEndOffset could not be converted

javax.swing.text.AbstractWriter.getIndentLevel could not be converted

javax.swing.text.AbstractWriter.getIndentSpace could not be converted

javax.swing.text.AbstractWriter.getLineLength could not be converted

javax.swing.text.AbstractWriter.getStartOffset could not be converted

javax.swing.text.AbstractWriter.getText could not be converted

javax.swing.text.AbstractWriter.getWriter could not be converted

javax.swing.text.AbstractWriter.incrIndent could not be converted

javax.swing.text.AbstractWriter.indent could not be converted

javax.swing.text.AbstractWriter.inRange could not be converted

javax.swing.text.AbstractWriter.isLineEmpty could not be converted

javax.swing.text.AbstractWriter.NEWLINE could not be converted

javax.swing.text.AbstractWriter.setCanWrapLines could not be converted

javax.swing.text.AbstractWriter.setCurrentLineLength could not be converted

javax.swing.text.AbstractWriter.setIndentSpace could not be converted

javax.swing.text.AbstractWriter.setLineLength could not be converted

javax.swing.text.AbstractWriter.text could not be converted

javax.swing.text.AbstractWriter.writeAttributes could not be converted

javax.swing.text.AsyncBoxView could not be converted

javax.swing.text.AsyncBoxView.ChildLocator could not be converted

javax.swing.text.AsyncBoxView.ChildState could not be converted

javax.swing.text.AttributeSet.CharacterAttribute could not be converted

javax.swing.text.AttributeSet.ColorAttribute could not be converted

javax.swing.text.AttributeSet.copyAttributes could not be converted

javax.swing.text.AttributeSet.FontAttribute could not be converted

javax.swing.text.AttributeSet.getResolveParent could not be converted

javax.swing.text.AttributeSet.isDefined could not be converted

javax.swing.text.AttributeSet.isEqual could not be converted

javax.swing.text.AttributeSet.NameAttribute could not be converted

javax.swing.text.AttributeSet.ParagraphAttribute could not be converted

javax.swing.text.AttributeSet.ResolveAttribute could not be converted

javax.swing.text.BadLocationException could not be converted

javax.swing.text.BadLocationException.offsetRequested could not be converted

javax.swing.text.BoxView could not be converted

javax.swing.text.Caret could not be converted

javax.swing.text.ChangedCharSetException could not be converted

javax.swing.text.ChangedCharSetException.keyEqualsCharSet could not be converted

javax.swing.text.ComponentView could not be converted

javax.swing.text.CompositeView could not be converted

javax.swing.text.DefaultCaret could not be converted

javax.swing.text.DefaultEditorKit could not be converted

javax.swing.text.DefaultEditorKit.BeepAction could not be converted

javax.swing.text.DefaultEditorKit.CopyAction could not be converted

javax.swing.text.DefaultEditorKit.CutAction could not be converted

javax.swing.text.DefaultEditorKit.DefaultKeyTypedAction could not be converted

javax.swing.text.DefaultEditorKit.InsertBreakAction could not be converted

javax.swing.text.DefaultEditorKit.InsertContentAction could not be converted

javax.swing.text.DefaultEditorKit.InsertTabAction could not be converted

javax.swing.text.DefaultEditorKit.PasteAction could not be converted

javax.swing.text.DefaultHighlighter could not be converted

javax.swing.text.DefaultHighlighter.DefaultHighlightPainter could not be converted

javax.swing.text.DefaultStyledDocument could not be converted

javax.swing.text.DefaultStyledDocument.AttributeUndoableEdit could not be converted

javax.swing.text.DefaultStyledDocument.ElementBuffer could not be converted

javax.swing.text.DefaultStyledDocument.ElementSpec could not be converted

javax.swing.text.DefaultTextUI could not be converted

javax.swing.text.Document could not be converted

javax.swing.text.EditorKit could not be converted

javax.swing.text.Element could not be converted

javax.swing.text.ElementIterator could not be converted

javax.swing.text.FieldView could not be converted

javax.swing.text.FlowView could not be converted

javax.swing.text.FlowView.FlowStrategy could not be converted

javax.swing.text.GapContent could not be converted

javax.swing.text.GlyphView could not be converted

javax.swing.text.GlyphView.GlyphPainter could not be converted

javax.swing.text.Highlighter could not be converted

javax.swing.text.Highlighter.Highlight could not be converted

javax.swing.text.Highlighter.HighlightPainter could not be converted

javax.swing.text.html.BlockView could not be converted

javax.swing.text.html.CSS could not be converted

javax.swing.text.html.CSS.Attribute.<AttributeName> could not be converted

javax.swing.text.html.CSS.Attribute.getDefaultValue could not be converted

javax.swing.text.html.CSS.Attribute.isInherited could not be converted

javax.swing.text.html.FormView could not be converted

javax.swing.text.html.HTML.Attribute.<AttributeName> could not be converted

javax.swing.text.html.HTML.Tag.<ElementName> could not be converted

javax.swing.text.html.HTML.Tag.breaksFlow could not be converted

javax.swing.text.html.HTML.Tag.isBlock could not be converted

javax.swing.text.html.HTML.Tag.isPreformatted could not be converted

javax.swing.text.html.HTML.Tag.Tag could not be converted

javax.swing.text.html.HTML.UnknownTag could not be converted

javax.swing.text.html.HTMLDocument.AdditionalComments could not be converted

javax.swing.text.html.HTMLDocument.BlockElement.BlockElement could not be converted

javax.swing.text.html.HTMLDocument.BlockElement.getResolveParent could not be converted

javax.swing.text.html.HTMLDocument.create could not be converted

javax.swing.text.html.HTMLDocument.createBranchElement could not be converted

javax.swing.text.html.HTMLDocument.createDefaultRoot could not be converted

javax.swing.text.html.HTMLDocument.createLeafElement could not be converted

javax.swing.text.html.HTMLDocument.fireChangedUpdate could not be converted

javax.swing.text.html.HTMLDocument.fireUndoableEditUpdate could not be converted

javax.swing.text.html.HTMLDocument.getElement could not be converted

javax.swing.text.html.HTMLDocument.getParser could not be converted

javax.swing.text.html.HTMLDocument.getPreservesUnknownTags could not be converted

javax.swing.text.html.HTMLDocument.getReader could not be converted

javax.swing.text.html.HTMLDocument.getTokenThreshold could not be converted

javax.swing.text.html.HTMLDocument.HTMLDocument could not be converted

javax.swing.text.html.HTMLDocument.HTMLDocument(AbstractDocumentContent, StyleSheet) could not be converted

javax.swing.text.html.HTMLDocument.HTMLDocument(StyleSheet) could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.BlockAction could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.CharacterAction could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.FormAction could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.HiddenAction could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.IsindexAction could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.ParagraphAction could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.PreAction could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.SpecialAction could not be converted

javax.swing.text.html.HTMLDocument.HTMLReader.TagAction could not be converted

javax.swing.text.html.HTMLDocument.insert could not be converted

javax.swing.text.html.HTMLDocument.insertAfterEnd could not be converted

javax.swing.text.html.HTMLDocument.insertAfterStart could not be converted

javax.swing.text.html.HTMLDocument.insertBeforeEnd could not be converted

javax.swing.text.html.HTMLDocument.insertBeforeStart could not be converted

javax.swing.text.html.HTMLDocument.insertUpdate could not be converted

javax.swing.text.html.HTMLDocument.Iterator.getAttributes could not be converted

javax.swing.text.html.HTMLDocument.Iterator.getEndOffset could not be converted

javax.swing.text.html.HTMLDocument.Iterator.getStartOffset could not be converted

javax.swing.text.html.HTMLDocument.Iterator.isValid could not be converted

javax.swing.text.html.HTMLDocument.Iterator.Iterator could not be converted

javax.swing.text.html.HTMLDocument.processHTMLFrameHyperlinkEvent could not be converted

javax.swing.text.html.HTMLDocument.RunElement could not be converted

javax.swing.text.html.HTMLDocument.setInnerHTML could not be converted

javax.swing.text.html.HTMLDocument.setOuterHTML could not be converted

javax.swing.text.html.HTMLDocument.setParagraphAttributes could not be converted

javax.swing.text.html.HTMLDocument.setParser could not be converted

javax.swing.text.html.HTMLDocument.setPreservesUnknownTags could not be converted

javax.swing.text.html.HTMLDocument.setTokenThreshold could not be converted

javax.swing.text.html.HTMLEditorKit could not be converted

javax.swing.text.html.HTMLEditorKit.HTMLFactory could not be converted

javax.swing.text.html.HTMLEditorKit.HTMLTextAction could not be converted

javax.swing.text.html.HTMLEditorKit.InsertHTMLTextAction could not be converted

javax.swing.text.html.HTMLEditorKit.InsertHTMLTextAction.html could not be converted

javax.swing.text.html.HTMLEditorKit.LinkController could not be converted

javax.swing.text.html.HTMLEditorKit.Parser could not be converted

javax.swing.text.html.HTMLEditorKit.ParserCallback could not be converted

javax.swing.text.html.HTMLFrameHyperlinkEvent could not be converted

javax.swing.text.html.HTMLWriter could not be converted

javax.swing.text.html.HTMLWriter.closeOutUnwantedEmbeddedTags could not be converted

javax.swing.text.html.HTMLWriter.comment could not be converted

javax.swing.text.html.HTMLWriter.emptyTag could not be converted

javax.swing.text.html.HTMLWriter.endTag could not be converted

javax.swing.text.html.HTMLWriter.HTMLWriter could not be converted

javax.swing.text.html.HTMLWriter.isBlockTag could not be converted

javax.swing.text.html.HTMLWriter.matchNameAttribute could not be converted

javax.swing.text.html.HTMLWriter.selectContent could not be converted

javax.swing.text.html.HTMLWriter.startTag could not be converted

javax.swing.text.html.HTMLWriter.synthesizedElement could not be converted

javax.swing.text.html.HTMLWriter.text could not be converted

javax.swing.text.html.HTMLWriter.textAreaContent could not be converted

javax.swing.text.html.HTMLWriter.writeEmbeddedTags could not be converted

javax.swing.text.html.InlineView could not be converted

javax.swing.text.html.ListView could not be converted

javax.swing.text.html.MinimalHTMLWriter.endFontTag could not be converted

javax.swing.text.html.MinimalHTMLWriter.inFontTag could not be converted

javax.swing.text.html.MinimalHTMLWriter.isText could not be converted

javax.swing.text.html.MinimalHTMLWriter.MinimalHTMLWriter could not be converted

javax.swing.text.html.MinimalHTMLWriter.startFontTag could not be converted

javax.swing.text.html.MinimalHTMLWriter.text could not be converted

javax.swing.text.html.MinimalHTMLWriter.write could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeComponent could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeContent could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeHeader could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeHTMLTags could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeImage could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeLeaf could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeNonHTMLAttributes could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeStartParagraph could not be converted

javax.swing.text.html.MinimalHTMLWriter.writeStyles could not be converted

javax.swing.text.html.ObjectView could not be converted

javax.swing.text.html.Option.getAttributes could not be converted

javax.swing.text.html.Option.Option could not be converted

javax.swing.text.html.ParagraphView could not be converted

javax.swing.text.html.parser.AttributeList could not be converted

javax.swing.text.html.parser.ContentModel could not be converted

javax.swing.text.html.parser.DocumentParser could not be converted

javax.swing.text.html.parser.DTD could not be converted

javax.swing.text.html.parser.DTDConstants could not be converted

javax.swing.text.html.parser.Element could not be converted

javax.swing.text.html.parser.Entity could not be converted

javax.swing.text.html.parser.Parser could not be converted

javax.swing.text.html.parser.ParserDelegator could not be converted

javax.swing.text.html.parser.TagElement could not be converted

javax.swing.text.html.StyleSheet.addAttribute could not be converted

javax.swing.text.html.StyleSheet.addAttributes could not be converted

javax.swing.text.html.StyleSheet.addCSSAttribute could not be converted

javax.swing.text.html.StyleSheet.addCSSAttributeFromHTML could not be converted

javax.swing.text.html.StyleSheet.addStyleSheet could not be converted

javax.swing.text.html.StyleSheet.BoxPainter could not be converted

javax.swing.text.html.StyleSheet.createLargeAttributeSet could not be converted

javax.swing.text.html.StyleSheet.createSmallAttributeSet could not be converted

javax.swing.text.html.StyleSheet.getBackground could not be converted

javax.swing.text.html.StyleSheet.getBase could not be converted

javax.swing.text.html.StyleSheet.getBoxPainter could not be converted

javax.swing.text.html.StyleSheet.getDeclaration could not be converted

javax.swing.text.html.StyleSheet.getFont could not be converted

javax.swing.text.html.StyleSheet.setForeground could not be converted

javax.swing.text.html.StyleSheet.getIndexOfSize could not be converted

javax.swing.text.html.StyleSheet.getListPainter could not be converted

javax.swing.text.html.StyleSheet.getPointSize(int) could not be converted

javax.swing.text.html.StyleSheet.getPointSize(String) could not be converted

javax.swing.text.html.StyleSheet.getRule could not be converted

javax.swing.text.html.StyleSheet.getStyleSheets could not be converted

javax.swing.text.html.StyleSheet.getViewAttributes could not be converted

javax.swing.text.html.StyleSheet.importStyleSheet could not be converted

javax.swing.text.html.StyleSheet.ListPainter could not be converted

javax.swing.text.html.StyleSheet.loadRules could not be converted

javax.swing.text.html.StyleSheet.removeAttribute could not be converted

javax.swing.text.html.StyleSheet.removeAttributes could not be converted

javax.swing.text.html.StyleSheet.removeStyle could not be converted

javax.swing.text.html.StyleSheet.removeStyleSheet could not be converted

javax.swing.text.html.StyleSheet.setBase could not be converted

javax.swing.text.html.StyleSheet.setBaseFontSize(int) could not be converted

javax.swing.text.html.StyleSheet.setBaseFontSize(String) could not be converted

javax.swing.text.html.StyleSheet.stringToColor could not be converted

javax.swing.text.html.StyleSheet.StyleSheet could not be converted

javax.swing.text.html.StyleSheet.translateHTMLToCSS could not be converted

javax.swing.text.IconView could not be converted

javax.swing.text.MutableAttributeSet.addAttributes could not be converted

javax.swing.text.MutableAttributeSet.removeAttributes(AttributeSet) could not be converted

javax.swing.text.MutableAttributeSet.removeAttributes(Enumeration) could not be converted

javax.swing.text.MutableAttributeSet.setResolveParent could not be converted

javax.swing.text.JTextComponent.AccessibleJTextComponent could not be converted

javax.swing.text.JTextComponent.addCaretListener could not be converted

javax.swing.text.JTextComponent.addInputMethodListener could not be converted

javax.swing.text.JTextComponent.addKeymap could not be converted

javax.swing.text.JTextComponent.DEFAULT_KEYMAP could not be converted

javax.swing.text.JTextComponent.fireCaretUpdate could not be converted

javax.swing.text.JTextComponent.FOCUS_ACCELERATOR_KEY could not be converted

javax.swing.text.JTextComponent.getActions could not be converted

javax.swing.text.JTextComponent.getCaret could not be converted

javax.swing.text.JTextComponent.getCaretColor could not be converted

javax.swing.text.JTextComponent.getDisabledTextColor could not be converted

javax.swing.text.JTextComponent.getDocument could not be converted

javax.swing.text.JTextComponent.getFocusAccelerator could not be converted

javax.swing.text.JTextComponent.getHighlighter could not be converted

javax.swing.text.JTextComponent.getInputMethodRequests could not be converted

javax.swing.text.JTextComponent.getKeymap could not be converted

javax.swing.text.JTextComponent.getKeymap(String) could not be converted

javax.swing.text.JTextComponent.getMargin could not be converted

javax.swing.text.JTextComponent.getPreferredScrollableViewportSize could not be converted

javax.swing.text.JTextComponent.getScrollableBlockIncrement could not be converted

javax.swing.text.JTextComponent.getScrollableTracksViewportHeight could not be converted

javax.swing.text.JTextComponent.getScrollableTracksViewportWidth could not be converted

javax.swing.text.JTextComponent.getScrollableUnitIncrement could not be converted

javax.swing.text.JTextComponent.getSelectedTextColor could not be converted

javax.swing.text.JTextComponent.getSelectionColor could not be converted

javax.swing.text.JTextComponent.getUI could not be converted

javax.swing.text.JTextComponent.loadKeymap could not be converted

javax.swing.text.JTextComponent.modelToView could not be converted

javax.swing.text.JTextComponent.processInputMethodEvent could not be converted

javax.swing.text.JTextComponent.processKeyEvent could not be converted

javax.swing.text.JTextComponent.read could not be converted

javax.swing.text.JTextComponent.removeCaretListener could not be converted

javax.swing.text.JTextComponent.removeKeymap could not be converted

javax.swing.text.JTextComponent.setCaret could not be converted

javax.swing.text.JTextComponent.setCaretColor could not be converted

javax.swing.text.JTextComponent.setDisabledTextColor could not be converted

javax.swing.text.JTextComponent.setDocument could not be converted

javax.swing.text.JTextComponent.setFocusAccelerator could not be converted

javax.swing.text.JTextComponent.setHighlighter could not be converted

javax.swing.text.JTextComponent.setKeymap could not be converted

javax.swing.text.JTextComponent.setMargin could not be converted

javax.swing.text.JTextComponent.setSelectedTextColor could not be converted

javax.swing.text.JTextComponent.setSelectionColor could not be converted

javax.swing.text.JTextComponent.setText could not be converted

javax.swing.text.JTextComponent.setUI could not be converted

javax.swing.text.JTextComponent.updateUI could not be converted

javax.swing.text.JTextComponent.viewToModel could not be converted

javax.swing.text.JTextComponent.write could not be converted

javax.swing.text.JTextComponentBeanInfo could not be converted

javax.swing.text.Keymap could not be converted

javax.swing.text.LabelView could not be converted

javax.swing.text.LayeredHighlighter could not be converted

javax.swing.text.LayeredHighlighter.LayerPainter could not be converted

javax.swing.text.LayoutQueue could not be converted

javax.swing.text.MutableAttributeSet.addAttribute could not be converted

javax.swing.text.MutableAttributeSet.removeAttributes(AttributeSet) could not be converted

javax.swing.text.MutableAttributeSet.removeAttributes(Enumeration) could not be converted

javax.swing.text.MutableAttributeSet.setResolveParent could not be converted

javax.swing.text.ParagraphView could not be converted

javax.swing.text.PasswordView could not be converted

javax.swing.text.PlainDocument could not be converted

javax.swing.text.PlainView could not be converted

javax.swing.text.Position could not be converted

javax.swing.text.Position.Bias could not be converted

javax.swing.text.rtf.RTFEditorKit could not be converted

javax.swing.text.SimpleAttributeSet.addAttribute could not be converted

javax.swing.text.SimpleAttributeSet.containsAttributes could not be converted

javax.swing.text.SimpleAttributeSet.copyAttributes could not be converted

javax.swing.text.SimpleAttributeSet.EMPTY could not be converted

javax.swing.text.SimpleAttributeSet.equals could not be converted

javax.swing.text.SimpleAttributeSet.getAttributeNames could not be converted

javax.swing.text.SimpleAttributeSet.getResolveParent could not be converted

javax.swing.text.SimpleAttributeSet.isEqual could not be converted

javax.swing.text.SimpleAttributeSet.removeAttributes(AttributeSet) could not be converted

javax.swing.text.SimpleAttributeSet.removeAttributes(Enumeration) could not be converted

javax.swing.text.SimpleAttributeSet.setResolveParent could not be converted

javax.swing.text.SimpleAttributeSet.toString could not be converted

javax.swing.text.StringContent.createPosition could not be converted

javax.swing.text.StringContent.getPositionsInRange could not be converted

javax.swing.text.StringContent.updateUndoPositions could not be converted

javax.swing.text.Style could not be converted

javax.swing.text.StyleConstants could not be converted

javax.swing.text.StyleConstants.CharacterConstants could not be converted

javax.swing.text.StyleConstants.ColorConstants could not be converted

javax.swing.text.StyleConstants.FontConstants could not be converted

javax.swing.text.StyleConstants.ParagraphConstants could not be converted

javax.swing.text.StyleContext could not be converted

javax.swing.text.StyleContext.NamedStyle could not be converted

javax.swing.text.StyleContext.SmallAttributeSet could not be converted

javax.swing.text.StyledDocument could not be converted

javax.swing.text.StyledEditorKit could not be converted

javax.swing.text.StyledEditorKit.AlignmentAction could not be converted

javax.swing.text.StyledEditorKit.BoldAction could not be converted

javax.swing.text.StyledEditorKit.FontFamilyAction could not be converted

javax.swing.text.StyledEditorKit.FontSizeAction could not be converted

javax.swing.text.StyledEditorKit.ForegroundAction could not be converted

javax.swing.text.StyledEditorKit.ItalicAction could not be converted

javax.swing.text.StyledEditorKit.StyledTextAction could not be converted

javax.swing.text.StyledEditorKit.UnderlineAction could not be converted

javax.swing.text.TabableView could not be converted

javax.swing.text.TabExpander could not be converted

javax.swing.text.TableView could not be converted

javax.swing.text.TableView.TableCell could not be converted

javax.swing.text.TableView.TableRow could not be converted

javax.swing.text.TabSet could not be converted

javax.swing.text.TabStop could not be converted

javax.swing.text.TextAction could not be converted

javax.swing.text.Utilities could not be converted

javax.swing.text.View could not be converted

javax.swing.text.ViewFactory could not be converted

javax.swing.text.WrappedPlainView could not be converted

javax.swing.text.ZoneView could not be converted

Javax.swing.tree Error Messages

javax.swing.tree.AbstractLayoutCache could not be converted

javax.swing.tree.AbstractLayoutCache.NodeDimensions could not be converted

javax.swing.tree.DefaultMutableTreeNode.allowsChildren could not be converted

javax.swing.tree.DefaultMutableTreeNode.breadthFirstEnumeration could not be converted

javax.swing.tree.DefaultMutableTreeNode.children could not be converted

javax.swing.tree.DefaultMutableTreeNode.depthFirstEnumeration could not be converted

javax.swing.tree.DefaultMutableTreeNode.getAllowsChildren could not be converted

javax.swing.tree.DefaultMutableTreeNode.getDepth could not be converted

javax.swing.tree.DefaultMutableTreeNode.getLeafCount could not be converted

javax.swing.tree.DefaultMutableTreeNode.getPath could not be converted

javax.swing.tree.DefaultMutableTreeNode.getUserObject could not be converted

javax.swing.tree.DefaultMutableTreeNode.getUserObjectPath could not be converted

javax.swing.tree.DefaultMutableTreeNode.isNodeDescendant could not be converted

javax.swing.tree.DefaultMutableTreeNode.setAllowsChildren could not be converted

javax.swing.tree.DefaultMutableTreeNode.setUserObject could not be converted

javax.swing.tree.DefaultMutableTreeNode.userObject could not be converted

javax.swing.tree.DefaultTreeCellEditor could not be converted

javax.swing.tree.DefaultTreeCellEditor.DefaultTextField could not be converted

javax.swing.tree.DefaultTreeCellEditor.EditorContainer could not be converted

javax.swing.tree.DefaultTreeCellRenderer could not be converted

javax.swing.tree.DefaultTreeModel could not be converted

javax.swing.tree.DefaultTreeModel.addTreeModelListener could not be converted

javax.swing.tree.DefaultTreeModel.asksAllowsChildren could not be converted

javax.swing.tree.DefaultTreeModel.DefaultTreeModel(TreeNode) could not be converted

javax.swing.tree.DefaultTreeModel.DefaultTreeModel(TreeNode, boolean) could not be converted

javax.swing.tree.DefaultTreeModel.fireTreeNodesChanged could not be converted

javax.swing.tree.DefaultTreeModel.fireTreeNodesInserted could not be converted

javax.swing.tree.DefaultTreeModel.fireTreeNodesRemoved could not be converted

javax.swing.tree.DefaultTreeModel.fireTreeStructureChanged could not be converted

javax.swing.tree.DefaultTreeModel.getChild could not be converted

javax.swing.tree.DefaultTreeModel.getChildCount could not be converted

javax.swing.tree.DefaultTreeModel.getChildIndex could not be converted

javax.swing.tree.DefaultTreeModel.getListeners could not be converted

javax.swing.tree.DefaultTreeModel.getRoot could not be converted

javax.swing.tree.DefaultTreeModel.insertNodeInto could not be converted

javax.swing.tree.DefaultTreeModel.isLeaf could not be converted

javax.swing.tree.DefaultTreeModel.listenerList could not be converted

javax.swing.tree.DefaultTreeModel.nodeChanged could not be converted

javax.swing.tree.DefaultTreeModel.nodesChanged could not be converted

javax.swing.tree.DefaultTreeModel.nodeStructureChanged could not be converted

javax.swing.tree.DefaultTreeModel.nodesWereInserted could not be converted

javax.swing.tree.DefaultTreeModel.nodesWereRemoved could not be converted

javax.swing.tree.DefaultTreeModel.reload could not be converted

javax.swing.tree.DefaultTreeModel.removeNodeFromParent could not be converted

javax.swing.tree.DefaultTreeModel.removeTreeModelListener could not be converted

javax.swing.tree.DefaultTreeModel.root could not be converted

javax.swing.tree.DefaultTreeModel.setAsksAllowsChildren could not be converted

javax.swing.tree.DefaultTreeModel.setRoot could not be converted

javax.swing.tree.DefaultTreeModel.valueForPathChanged could not be converted

javax.swing.tree.DefaultTreeSelectionModel could not be converted

javax.swing.tree.ExpandVetoException.event could not be converted

javax.swing.tree.ExpandVetoException.ExpandVetoException(TreeExpansionEvent) could not be converted

javax.swing.tree.ExpandVetoException.ExpandVetoException(TreeExpansionEvent, String) could not be converted

javax.swing.tree.FixedHeightLayoutCache could not be converted

javax.swing.tree.MutableTreeNode.setUserObject could not be converted

javax.swing.tree.RowMapper could not be converted

javax.swing.tree.TreeCellEditor could not be converted

javax.swing.tree.TreeCellRenderer could not be converted

javax.swing.tree.TreeModel could not be converted

javax.swing.tree.TreeModel.addTreeModelListener could not be converted

javax.swing.tree.TreeModel.getChild could not be converted

javax.swing.tree.TreeModel.getChildCount could not be converted

javax.swing.tree.TreeModel.getIndexOfChild could not be converted

javax.swing.tree.TreeModel.getRoot could not be converted

javax.swing.tree.TreeModel.isLeaf could not be converted

javax.swing.tree.TreeModel.removeTreeModelListener could not be converted

javax.swing.tree.TreeModel.valueForPathChanged could not be converted

javax.swing.tree.TreeNode.getAllowsChildren could not be converted

javax.swing.tree.TreePath could not be converted

javax.swing.tree.TreeSelectionModel could not be converted

javax.swing.tree.VariableHeightLayoutCache could not be converted

Javax.swing.undo Error Messages

[javax.swing.undo.AbstractUndoableEdit could not be converted](#)

[javax.swing.undo.CompoundEdit could not be converted](#)

[javax.swing.undo.StateEdit could not be converted](#)

[javax.swing.undo.StateEditable could not be converted](#)

[javax.swing.undo.UndoableEdit could not be converted](#)

[javax.swing.undo.UndoableEditSupport could not be converted](#)

[javax.swing.undo.UndoManager could not be converted](#)

Javax.transaction Error Messages

[javax.transaction.HeuristicCommitException could not be converted](#)

[javax.transaction.HeuristicMixedException could not be converted](#)

[javax.transaction.HeuristicRollbackException could not be converted](#)

[javax.transaction.InvalidTransactionException could not be converted](#)

[javax.transaction.RollbackException could not be converted](#)

[javax.transaction.Status.STATUS_ACTIVE could not be converted](#)

[javax.transaction.Status.STATUS_COMMITTING could not be converted](#)

[javax.transaction.Status.STATUS_MARKED_ROLLBACK could not be converted](#)

[javax.transaction.Status.STATUS_PREPARED could not be converted](#)

[javax.transaction.Status.STATUS_PREPARING could not be converted](#)

[javax.transaction.Status.STATUS_UNKNOWN could not be converted](#)

[javax.transaction.SystemException.errorCode could not be converted](#)

[javax.transaction.SystemException.SystemException could not be converted](#)

[javax.transaction.Synchronization could not be converted](#)

[javax.transaction.Transaction could not be converted](#)

[javax.transaction.Transaction.commit could not be converted](#)

[javax.transaction.Transaction.delistResource could not be converted](#)

[javax.transaction.Transaction.enlistResource could not be converted](#)

[javax.transaction.Transaction.getStatus could not be converted](#)

[javax.transaction.Transaction.registerSynchronization could not be converted](#)

[javax.transaction.Transaction.rollback could not be converted](#)

[javax.transaction.Transaction.setRollbackOnly could not be converted](#)

[javax.transaction.TransactionManager could not be converted](#)

[javax.transaction.TransactionManager.setRollbackOnly could not be converted](#)

[javax.transaction.TransactionRequiredException could not be converted](#)

[javax.transaction.TransactionRolledbackException could not be converted](#)

[javax.transaction.UserTransaction could not be converted](#)

[javax.transaction.UserTransaction.begin could not be converted](#)

[javax.transaction.UserTransaction.setRollbackOnly could not be converted](#)

[javax.transaction.xa.XAException could not be converted](#)

[javax.transaction.xa.XAResource could not be converted](#)

[javax.transaction.xa.Xid could not be converted](#)

Javax.xml Error Messages

javax.xml.parsers.DocumentBuilder could not be converted

javax.xml.parsers.DocumentBuilder.isValidating could not be converted

javax.xml.parsers.DocumentBuilder.parse could not be converted

javax.xml.parsers.DocumentBuilder.setEntityResolver could not be converted

javax.xml.parsers.DocumentBuilder.setErrorHandler could not be converted

javax.xml.parsers.DocumentBuilderFactory could not be converted

javax.xml.parsers.FactoryConfigurationError could not be converted

javax.xml.parsers.ParserConfigurationException could not be converted

javax.xml.parsers.SAXParser.getProperty could not be converted

javax.xml.parsers.SAXParser.setProperty could not be converted

javax.xml.parsers.SAXParserFactory.getFeature could not be converted

javax.xml.parsers.SAXParserFactory.setFeature could not be converted

javax.xml.transform.dom.DOMLocator could not be converted

javax.xml.transform.dom.DOMResult could not be converted

javax.xml.transform.dom.DOMResult.DOMResult could not be converted

javax.xml.transform.dom.DOMSource could not be converted

javax.xml.transform.ErrorListener could not be converted

javax.xml.transform.OutputKeys could not be converted

javax.xml.transform.Result.PI_DISABLE_OUTPUT_ESCAPING could not be converted

javax.xml.transform.Result.PI_ENABLE_OUTPUT_ESCAPING could not be converted

javax.xml.transform.sax.SAXResult could not be converted

javax.xml.transform.sax.SAXResult.SAXResult could not be converted

javax.xml.transform.sax.SAXTransformerFactory could not be converted

javax.xml.transform.sax.TemplatesHandler could not be converted

javax.xml.transform.sax.TransformerHandler could not be converted

javax.xml.transform.stream.StreamResult.setSystemId could not be converted

javax.xml.transform.stream.StreamSource.getPublicId could not be converted

javax.xml.transform.stream.StreamSource.setPublicId could not be converted

javax.xml.transform.stream.StreamSource.setSystemId could not be converted

javax.xml.transform.Templates.getOutputProperties could not be converted

javax.xml.transform.Transformer.getOutputProperties could not be converted

javax.xml.transform.Transformer.getOutputProperty could not be converted

javax.xml.transform.Transformer.getURIResolver could not be converted

javax.xml.transform.Transformer.setErrorListener could not be converted

javax.xml.transform.Transformer.setOutputProperties could not be converted

javax.xml.transform.Transformer.setOutputProperty could not be converted

javax.xml.transform.Transformer.transform could not be converted

javax.xml.transform.TransformerConfigurationException.TransformerConfigurationException could not be converted
javax.xml.transform.TransformerException.initCause could not be converted
javax.xml.transform.TransformerException.setLocator could not be converted
javax.xml.transform.TransformerException.TransformerException could not be converted
javax.xml.transform.TransformerFactory.getAssociatedStylesheet could not be converted
javax.xml.transform.TransformerFactory.getAttribute could not be converted
javax.xml.transform.TransformerFactory.getURIResolver could not be converted
javax.xml.transform.TransformerFactory.newTransformer could not be converted
javax.xml.transform.TransformerFactory.setAttribute could not be converted
javax.xml.transform.TransformerFactory.setErrorListener could not be converted
javax.xml.transform.TransformerFactoryConfigurationError could not be converted
javax.xml.transform.URIResolver could not be converted
javax.xml.transform.URIResolver.resolve could not be converted

Org.omg Error Messages

org.omg.CORBA_IDLTypeStub could not be converted

org.omg.CORBA_PolicyStub could not be converted

org.omg.CORBA.Any.create_input_stream could not be converted

org.omg.CORBA.Any.create_output_stream could not be converted

org.omg.CORBA.Any.extract_Principal could not be converted

org.omg.CORBA.Any.extract_Value could not be converted

org.omg.CORBA.Any.insert_fixed could not be converted

org.omg.CORBA.Any.insert_Object could not be converted

org.omg.CORBA.Any.insert_Principal could not be converted

org.omg.CORBA.Any.insert_Streamable could not be converted

org.omg.CORBA.Any.insert_Value(Serializable) could not be converted

org.omg.CORBA.Any.insert_Value(Serializable, TypeCode) could not be converted

org.omg.CORBA.Any.read_value could not be converted

org.omg.CORBA.Any.type could not be converted

org.omg.CORBA.Any.write_value could not be converted

org.omg.CORBA.AnyHolder could not be converted

org.omg.CORBA.AnyHolder._read could not be converted

org.omg.CORBA.AnyHolder._write could not be converted

org.omg.CORBA.AnySeqHelper could not be converted

org.omg.CORBA.AnySeqHolder could not be converted

org.omg.CORBA.ARG_IN could not be converted

org.omg.CORBA.ARG_INOUT could not be converted

org.omg.CORBA.ARG_OUT could not be converted

org.omg.CORBA.BAD_CONTEXT.BAD_CONTEXT(int, CompletionStatus) could not be converted

org.omg.CORBA.BAD_CONTEXT.BAD_CONTEXT(String, int, CompletionStatus) could not be converted

org.omg.CORBA.BAD_INV_ORDER.BAD_INV_ORDER(int, CompletionStatus) could not be converted

org.omg.CORBA.BAD_INV_ORDER.BAD_INV_ORDER(String, int, CompletionStatus) could not be converted

org.omg.CORBA.BAD_OPERATION.BAD_OPERATION(int, CompletionStatus) could not be converted

org.omg.CORBA.BAD_OPERATION.BAD_OPERATION(String, int, CompletionStatus) could not be converted

org.omg.CORBA.BAD_PARAM.BAD_PARAM(int, CompletionStatus) could not be converted

org.omg.CORBA.BAD_PARAM.BAD_PARAM(String, int, CompletionStatus) could not be converted

org.omg.CORBA.BAD_POLICY could not be converted

org.omg.CORBA.BAD_POLICY_TYPE could not be converted

org.omg.CORBA.BAD_POLICY_VALUE could not be converted

org.omg.CORBA.BooleanHolder._read could not be converted

org.omg.CORBA.BooleanHolder._write could not be converted

org.omg.CORBA.BooleanSeqHelper could not be converted

org.omg.CORBA.BooleanSeqHolder could not be converted

org.omg.CORBA.ByteHolder._read could not be converted

org.omg.CORBA.ByteHolder._write could not be converted

org.omg.CORBA.CharHolder._read could not be converted

org.omg.CORBA.CharHolder._write could not be converted
org.omg.CORBA.CharSeqHelper could not be converted
org.omg.CORBA.CharSeqHolder could not be converted
org.omg.CORBA.CompletionStatus could not be converted
org.omg.CORBA.CompletionStatusHelper could not be converted
org.omg.CORBA.Context could not be converted
org.omg.CORBA.ContextList could not be converted
org.omg.CORBA.CTX_RESTRICT_SCOPE could not be converted
org.omg.CORBA.Current could not be converted
org.omg.CORBA.CurrentHelper could not be converted
org.omg.CORBA.CurrentHolder could not be converted
org.omg.CORBA.CurrentOperations could not be converted
org.omg.CORBA.CustomMarshal could not be converted
org.omg.CORBA.DataInputStream could not be converted
org.omg.CORBA.DataOutputStream could not be converted
org.omg.CORBA.DefinitionKind could not be converted
org.omg.CORBA.DefinitionKindHelper could not be converted
org.omg.CORBA.DomainManager could not be converted
org.omg.CORBA.DomainManagerOperations could not be converted
org.omg.CORBA.DoubleHolder._read could not be converted
org.omg.CORBA.DoubleHolder._write could not be converted
org.omg.CORBA.DoubleSeqHelper could not be converted
org.omg.CORBA.DoubleSeqHolder could not be converted
org.omg.CORBA.DynamicImplementation could not be converted
org.omg.CORBA.DynAny could not be converted
org.omg.CORBA.DynArray could not be converted
org.omg.CORBA.DynEnum could not be converted
org.omg.CORBA.DynFixed could not be converted
org.omg.CORBA.DynSequence could not be converted
org.omg.CORBA.DynStruct could not be converted
org.omg.CORBA.DynUnion could not be converted
org.omg.CORBA.DynValue could not be converted
org.omg.CORBA.Environment could not be converted
org.omg.CORBA.ExceptionList could not be converted
org.omg.CORBA.FieldNameHelper could not be converted
org.omg.CORBA.FixedHolder could not be converted
org.omg.CORBA.FloatHolder._read could not be converted
org.omg.CORBA.FloatHolder._write could not be converted
org.omg.CORBA.FloatSeqHelper could not be converted
org.omg.CORBA.FloatSeqHolder could not be converted
org.omg.CORBA.IdentifierHelper could not be converted
org.omg.CORBA.IDLType could not be converted
org.omg.CORBA.IDLTypeHelper could not be converted

org.omg.CORBA.IDLTypeOperations could not be converted
org.omg.CORBA.IntHolder._read could not be converted
org.omg.CORBA.IntHolder._write could not be converted
org.omg.CORBA.INV_POLICY.INV_POLICY(int, CompletionStatus) could not be converted
org.omg.CORBA.INV_POLICY.INV_POLICY(String, int, CompletionStatus) could not be converted
org.omg.CORBA.IRObject could not be converted
org.omg.CORBA.IRObjectOperations could not be converted
org.omg.CORBA.LocalObject could not be converted
org.omg.CORBA.LongHolder._read could not be converted
org.omg.CORBA.LongHolder._write could not be converted
org.omg.CORBA.LongLongSeqHelper could not be converted
org.omg.CORBA.LongLongSeqHolder could not be converted
org.omg.CORBA.LongSeqHelper could not be converted
org.omg.CORBA.LongSeqHolder could not be converted
org.omg.CORBA.MARSHAL.MARSHAL(int, CompletionStatus) could not be converted
org.omg.CORBA.MARSHAL.MARSHAL(String, int, CompletionStatus) could not be converted
org.omg.CORBA.NamedValue could not be converted
org.omg.CORBA.NameValuePair could not be converted
org.omg.CORBA.NameValuePairHelper could not be converted
org.omg.CORBA.NO_PERMISSION.NO_PERMISSION(int, CompletionStatus) could not be converted
org.omg.CORBA.NO_PERMISSION.NO_PERMISSION(String, int, CompletionStatus) could not be converted
org.omg.CORBA.NVList could not be converted
org.omg.CORBA.Object._create_request(Context, String, NVList, NamedValue) could not be converted
org.omg.CORBA.Object._create_request(Context, String, NVList, NamedValue, ExceptionList, ContextList) could not be converted
org.omg.CORBA.Object._duplicate could not be converted
org.omg.CORBA.Object._get_domain_managers could not be converted
org.omg.CORBA.Object._get_interface_def could not be converted
org.omg.CORBA.Object._get_policy could not be converted
org.omg.CORBA.Object._hash could not be converted
org.omg.CORBA.Object._is_a could not be converted
org.omg.CORBA.Object._is_equivalent could not be converted
org.omg.CORBA.Object._non_existent could not be converted
org.omg.CORBA.Object._release could not be converted
org.omg.CORBA.Object._request could not be converted
org.omg.CORBA.Object._set_policy_override could not be converted
org.omg.CORBA.ObjectHelper could not be converted
org.omg.CORBA.ObjectHolder._read could not be converted
org.omg.CORBA.ObjectHolder._write could not be converted
org.omg.CORBA.OctetSeqHelper could not be converted
org.omg.CORBA.OctetSeqHolder could not be converted
org.omg.CORBA.OMGVMCID could not be converted
org.omg.CORBA.ORB could not be converted
org.omg.CORBA.Policy could not be converted

org.omg.CORBA.PolicyError.reason could not be converted
org.omg.CORBA.PolicyHelper could not be converted
org.omg.CORBA.PolicyHolder could not be converted
org.omg.CORBA.PolicyListHelper could not be converted
org.omg.CORBA.PolicyListHolder could not be converted
org.omg.CORBA.PolicyOperations could not be converted
org.omg.CORBA.PolicyTypeHelper could not be converted
org.omg.CORBA.portable.ApplicationException.getId could not be converted
org.omg.CORBA.portable.ApplicationException.getInputStream could not be converted
org.omg.CORBA.portable.BoxedValueHelper could not be converted
org.omg.CORBA.portable.CustomValue could not be converted
org.omg.CORBA.portable.Delegate could not be converted
org.omg.CORBA.portable.IDLEntity could not be converted
org.omg.CORBA.portable.IndirectionException.IndirectionException could not be converted
org.omg.CORBA.portable.IndirectionException.offset could not be converted
org.omg.CORBA.portable.InputStream could not be converted
org.omg.CORBA.portable.InvokeHandler could not be converted
org.omg.CORBA.portable.ObjectImpl._create_request(Context, String, NVList, NamedValue) could not be converted
org.omg.CORBA.portable.ObjectImpl._create_request(Context, String, NVList, NamedValue, ExceptionList, ContextList) could not be converted
org.omg.CORBA.portable.ObjectImpl._duplicate could not be converted
org.omg.CORBA.portable.ObjectImpl._get_delegate could not be converted
org.omg.CORBA.portable.ObjectImpl._get_domain_managers could not be converted
org.omg.CORBA.portable.ObjectImpl._get_interface_def could not be converted
org.omg.CORBA.portable.ObjectImpl._get_policy could not be converted
org.omg.CORBA.portable.ObjectImpl._hash could not be converted
org.omg.CORBA.portable.ObjectImpl._ids could not be converted
org.omg.CORBA.portable.ObjectImpl._invoke could not be converted
org.omg.CORBA.portable.ObjectImpl._is_a could not be converted
org.omg.CORBA.portable.ObjectImpl._is_equivalent could not be converted
org.omg.CORBA.portable.ObjectImpl._is_local could not be converted
org.omg.CORBA.portable.ObjectImpl._non_existent could not be converted
org.omg.CORBA.portable.ObjectImpl._orb could not be converted
org.omg.CORBA.portable.ObjectImpl._release could not be converted
org.omg.CORBA.portable.ObjectImpl._releaseReply could not be converted
org.omg.CORBA.portable.ObjectImpl._request(String) could not be converted
org.omg.CORBA.portable.ObjectImpl._request(String, boolean) could not be converted
org.omg.CORBA.portable.ObjectImpl._servant_postinvoke could not be converted
org.omg.CORBA.portable.ObjectImpl._servant_preinvoke could not be converted
org.omg.CORBA.portable.ObjectImpl._set_delegate could not be converted
org.omg.CORBA.portable.ObjectImpl._set_policy_override could not be converted
org.omg.CORBA.portable.OutputStream could not be converted
org.omg.CORBA.portable.ResponseHandler could not be converted
org.omg.CORBA.portable.ServantObject could not be converted

org.omg.CORBA.portable.Streamable could not be converted
org.omg.CORBA.portable.StreamableValue could not be converted
org.omg.CORBA.portable.ValueBase could not be converted
org.omg.CORBA.portable.ValueFactory could not be converted
org.omg.CORBA.Principal could not be converted
org.omg.CORBA.PrincipalHolder could not be converted
org.omg.CORBA.PRIVATE_MEMBER could not be converted
org.omg.CORBA.PUBLIC_MEMBER could not be converted
org.omg.CORBA.RepositoryIdHelper could not be converted
org.omg.CORBA.Request could not be converted
org.omg.CORBA.ServerRequest could not be converted
org.omg.CORBA.ServiceDetail could not be converted
org.omg.CORBA.ServiceDetailHelper could not be converted
org.omg.CORBA.ServiceInformation could not be converted
org.omg.CORBA.ServiceInformationHelper could not be converted
org.omg.CORBA.ServiceInformationHolder could not be converted
org.omg.CORBA.SetOverrideType could not be converted
org.omg.CORBA.SetOverrideTypeHelper could not be converted
org.omg.CORBA.ShortHolder._read could not be converted
org.omg.CORBA.ShortHolder._write could not be converted
org.omg.CORBA.ShortSeqHelper could not be converted
org.omg.CORBA.ShortSeqHolder could not be converted
org.omg.CORBA.StringHolder._read could not be converted
org.omg.CORBA.StringHolder._write could not be converted
org.omg.CORBA.StringHolder.StringHolder could not be converted
org.omg.CORBA.StringHolder.StringHolder(String) could not be converted
org.omg.CORBA.StringValueHelper could not be converted
org.omg.CORBA.StructMember could not be converted
org.omg.CORBA.StructMemberHelper could not be converted
org.omg.CORBA.SystemException.completed could not be converted
org.omg.CORBA.SystemException.minor could not be converted
org.omg.CORBA.TCKind.<DataType> could not be converted
org.omg.CORBA.TCKind.TCKind could not be converted
org.omg.CORBA.TypeCode could not be converted
org.omg.CORBA.TypeCode.concrete_base_type could not be converted
org.omg.CORBA.TypeCode.content_type could not be converted
org.omg.CORBA.TypeCode.default_index could not be converted
org.omg.CORBA.TypeCode.discriminator_type could not be converted
org.omg.CORBA.TypeCode.equivalent could not be converted
org.omg.CORBA.TypeCode.fixed_digits could not be converted
org.omg.CORBA.TypeCode.fixed_scale could not be converted
org.omg.CORBA.TypeCode.get_compact_typecode could not be converted
org.omg.CORBA.TypeCode.id could not be converted

org.omg.CORBA.TypeCode.length could not be converted
org.omg.CORBA.TypeCode.member_count could not be converted
org.omg.CORBA.TypeCode.member_label could not be converted
org.omg.CORBA.TypeCode.member_name could not be converted
org.omg.CORBA.TypeCode.member_type could not be converted
org.omg.CORBA.TypeCode.member_visibility could not be converted
org.omg.CORBA.TypeCode.name could not be converted
org.omg.CORBA.TypeCode.type_modifier could not be converted
org.omg.CORBA.TypeCode.TypeCode could not be converted
org.omg.CORBA.TypeCodeHolder could not be converted
org.omg.CORBA.ULongLongSeqHelper could not be converted
org.omg.CORBA.ULongLongSeqHolder could not be converted
org.omg.CORBA.ULongSeqHelper could not be converted
org.omg.CORBA.ULongSeqHolder could not be converted
org.omg.CORBA.UnionMember could not be converted
org.omg.CORBA.UnionMemberHelper could not be converted
org.omg.CORBA.UnknownUserException.except could not be converted
org.omg.CORBA.UNSUPPORTED_POLICY could not be converted
org.omg.CORBA.UNSUPPORTED_POLICY_VALUE could not be converted
org.omg.CORBA.USHortSeqHelper could not be converted
org.omg.CORBA.USHortSeqHolder could not be converted
org.omg.CORBA.ValueBaseHelper could not be converted
org.omg.CORBA.ValueBaseHolder could not be converted
org.omg.CORBA.ValueBaseHolder._read could not be converted
org.omg.CORBA.ValueBaseHolder._write could not be converted
org.omg.CORBA.ValueBaseHolder.ValueBaseHolder could not be converted
org.omg.CORBA.ValueMember could not be converted
org.omg.CORBA.ValueMemberHelper could not be converted
org.omg.CORBA.VersionSpecHelper could not be converted
org.omg.CORBA.VisibilityHelper could not be converted
org.omg.CORBA.VM_ABSTRACT could not be converted
org.omg.CORBA.VM_CUSTOM could not be converted
org.omg.CORBA.VM_NONE could not be converted
org.omg.CORBA.VM_TRUNCATABLE could not be converted
org.omg.CORBA.WCharSeqHelper could not be converted
org.omg.CORBA.WCharSeqHolder could not be converted
org.omg.CORBA.WStringValueHelper could not be converted
org.omg.CORBA_2_3.ORB could not be converted
org.omg.CORBA_2_3.portable.Delegate could not be converted
org.omg.CORBA_2_3.portable.InputStream could not be converted
org.omg.CORBA_2_3.portable.ObjectImpl could not be converted
org.omg.CORBA_2_3.portable.OutputStream could not be converted
org.omg.CosNaming.BindingIteratorImplBase could not be converted

org.omg.CosNaming._BindingIteratorStub could not be converted
org.omg.CosNaming._NamingContextImplBase could not be converted
org.omg.CosNaming._NamingContextStub could not be converted
org.omg.CosNaming.Binding could not be converted
org.omg.CosNaming.BindingHelper could not be converted
org.omg.CosNaming.BindingHolder could not be converted
org.omg.CosNaming.BindingIterator could not be converted
org.omg.CosNaming.BindingIteratorHelper could not be converted
org.omg.CosNaming.BindingIteratorHolder could not be converted
org.omg.CosNaming.BindingIteratorOperations could not be converted
org.omg.CosNaming.BindingListHelper could not be converted
org.omg.CosNaming.BindingListHolder could not be converted
org.omg.CosNaming.BindingType could not be converted
org.omg.CosNaming.BindingTypeHelper could not be converted
org.omg.CosNaming.BindingTypeHolder could not be converted
org.omg.CosNaming.IstringHelper could not be converted
org.omg.CosNaming.NameComponent could not be converted
org.omg.CosNaming.NameComponentHelper could not be converted
org.omg.CosNaming.NameComponentHolder could not be converted
org.omg.CosNaming.NameHelper could not be converted
org.omg.CosNaming.NameHolder could not be converted
org.omg.CosNaming.NamingContext could not be converted
org.omg.CosNaming.NamingContextHelper could not be converted
org.omg.CosNaming.NamingContextHolder could not be converted
org.omg.CosNaming.NamingContextOperations could not be converted
org.omg.CosNaming.NamingContextPackage.AlreadyBoundHelper could not be converted
org.omg.CosNaming.NamingContextPackage.AlreadyBoundHolder could not be converted
org.omg.CosNaming.NamingContextPackage.CannotProceed.CannotProceed could not be converted
org.omg.CosNaming.NamingContextPackage.CannotProceed.cxt could not be converted
org.omg.CosNaming.NamingContextPackage.CannotProceed.rest_of_name could not be converted
org.omg.CosNaming.NamingContextPackage.CannotProceedHelper could not be converted
org.omg.CosNaming.NamingContextPackage.CannotProceedHolder could not be converted
org.omg.CosNaming.NamingContextPackage.InvalidNameHelper could not be converted
org.omg.CosNaming.NamingContextPackage.InvalidNameHolder could not be converted
org.omg.CosNaming.NamingContextPackage.NotEmptyHelper could not be converted
org.omg.CosNaming.NamingContextPackage.NotEmptyHolder could not be converted
org.omg.CosNaming.NamingContextPackage.NotFound.NotFound could not be converted
org.omg.CosNaming.NamingContextPackage.NotFound.rest_of_name could not be converted
org.omg.CosNaming.NamingContextPackage.NotFound.why could not be converted
org.omg.CosNaming.NamingContextPackage.NotFoundHelper could not be converted
org.omg.CosNaming.NamingContextPackage.NotFoundHolder could not be converted
org.omg.CosNaming.NamingContextPackage.NotFoundReason could not be converted
org.omg.CosNaming.NamingContextPackage.NotFoundReasonHelper could not be converted

org.omg.CosNaming.NamingContextPackage.NotFoundReasonHolder could not be converted

org.omg.stub.java.rmi._Remote_Stub could not be converted

Org.w3c Error Messages

org.w3c.dom.DOMException could not be converted

org.w3c.dom.DOMException.<ExceptionType> could not be converted

org.w3c.dom.DOMException.code could not be converted

org.w3c.dom.DOMImplementation.createDocumentType could not be converted

org.w3c.dom.Element.setAttribute could not be converted

org.w3c.dom.Node.getChildNodes could not be converted

org.w3c.dom.Node.setPrefix could not be converted

org.w3c.dom.ranges.DocumentRange could not be converted

org.w3c.dom.ranges.Range could not be converted

org.w3c.dom.ranges.RangeException could not be converted

org.w3c.dom.traversal.DocumentTraversal.createNodeIterator could not be converted

org.w3c.dom.traversal.DocumentTraversal.createTreeWalker could not be converted

org.w3c.dom.traversal.NodeFilter.SHOW_ALL could not be converted

org.w3c.dom.traversal.NodeIterator.getExpandEntityReferences could not be converted

org.w3c.dom.traversal.TreeWalker.getExpandEntityReferences could not be converted

Org.xml Error Messages

org.xml.sax.AttributeList.getLength could not be converted

org.xml.sax.AttributeList.getName could not be converted

org.xml.sax.AttributeList.getType could not be converted

org.xml.sax.AttributeList.getValue could not be converted

org.xml.sax.Attributes.getIndex could not be converted

org.xml.sax.Attributes.getLength could not be converted

org.xml.sax.Attributes.getLocalName could not be converted

org.xml.sax.Attributes.getQName could not be converted

org.xml.sax.Attributes.getType could not be converted

org.xml.sax.Attributes.getURI could not be converted

org.xml.sax.Attributes.getValue could not be converted

org.xml.sax.DocumentHandler could not be converted

org.xml.sax.DTDHandler could not be converted

org.xml.sax.EntityResolver could not be converted

org.xml.sax.ErrorHandler.error could not be converted

org.xml.sax.ErrorHandler.fatalError could not be converted

org.xml.sax.ErrorHandler.warning could not be converted

org.xml.sax.ext.DeclHandler could not be converted

org.xml.sax.ext.LexicalHandler could not be converted

org.xml.sax.HandlerBase could not be converted

org.xml.sax.helpers.AttributeListImpl.addAttribute could not be converted

org.xml.sax.helpers.AttributeListImpl.AttributeListImpl could not be converted

org.xml.sax.helpers.AttributeListImpl.AttributeListImpl(AttributeList) could not be converted

org.xml.sax.helpers.AttributeListImpl.clear could not be converted

org.xml.sax.helpers.AttributeListImpl.getLength could not be converted

org.xml.sax.helpers.AttributeListImpl.getName could not be converted

org.xml.sax.helpers.AttributeListImpl.getType could not be converted

org.xml.sax.helpers.AttributeListImpl.getValue could not be converted

org.xml.sax.helpers.AttributeListImpl.removeAttribute could not be converted

org.xml.sax.helpers.AttributeListImpl.setAttributeList could not be converted

org.xml.sax.helpers.AttributesImpl.addAttribute could not be converted

org.xml.sax.helpers.AttributesImpl.AttributesImpl could not be converted

org.xml.sax.helpers.AttributesImpl.AttributesImpl(Attributes) could not be converted

org.xml.sax.helpers.AttributesImpl.clear could not be converted

org.xml.sax.helpers.AttributesImpl.getIndex(String) could not be converted

org.xml.sax.helpers.AttributesImpl.getIndex(String, String) could not be converted

org.xml.sax.helpers.AttributesImpl.getLength could not be converted

org.xml.sax.helpers.AttributesImpl.getLocalName could not be converted
org.xml.sax.helpers.AttributesImpl.getQName could not be converted
org.xml.sax.helpers.AttributesImpl.getType could not be converted
org.xml.sax.helpers.AttributesImpl.getURI could not be converted
org.xml.sax.helpers.AttributesImpl.getValue could not be converted
org.xml.sax.helpers.AttributesImpl.removeAttribute could not be converted
org.xml.sax.helpers.AttributesImpl.setAttribute could not be converted
org.xml.sax.helpers.AttributesImpl.setAttributes could not be converted
org.xml.sax.helpers.AttributesImpl.setLocalName could not be converted
org.xml.sax.helpers.AttributesImpl.setQName could not be converted
org.xml.sax.helpers.AttributesImpl.setType could not be converted
org.xml.sax.helpers.AttributesImpl.setURI could not be converted
org.xml.sax.helpers.AttributesImpl.setValue could not be converted
org.xml.sax.helpers.NamespaceSupport.getDeclaredPrefixes could not be converted
org.xml.sax.helpers.NamespaceSupport.getPrefixes could not be converted
org.xml.sax.helpers.NamespaceSupport.getPrefixes(String) could not be converted
org.xml.sax.helpers.NamespaceSupport.processName could not be converted
org.xml.sax.helpers.NamespaceSupport.reset could not be converted
org.xml.sax.helpers.NamespaceSupport.XMLNS could not be converted
org.xml.sax.helpers.ParserAdapter could not be converted
org.xml.sax.helpers.ParserAdapter.getDTDHandler could not be converted
org.xml.sax.helpers.ParserAdapter.getEntityResolver could not be converted
org.xml.sax.helpers.ParserAdapter.getFeature could not be converted
org.xml.sax.helpers.ParserAdapter.getProperty could not be converted
org.xml.sax.helpers.ParserAdapter.setDTDHandler could not be converted
org.xml.sax.helpers.ParserAdapter.setErrorHandler could not be converted
org.xml.sax.helpers.ParserAdapter.setFeature could not be converted
org.xml.sax.helpers.ParserAdapter.setProperty could not be converted
org.xml.sax.helpers.XMLFilterImpl could not be converted
org.xml.sax.helpers.XMLReaderAdapter could not be converted
org.xml.sax.helpers.XMLReaderAdapter.setDTDHandler could not be converted
org.xml.sax.helpers.XMLReaderAdapter.setErrorHandler could not be converted
org.xml.sax.helpers.XMLReaderAdapter.setLocale could not be converted
org.xml.sax.helpers.XMLReaderFactory could not be converted
org.xml.sax.InputSource.getEncoding could not be converted
org.xml.sax.InputSource.getPublicId could not be converted
org.xml.sax.InputSource.setEncoding could not be converted
org.xml.sax.InputSource.setPublicId could not be converted
org.xml.sax.Parser.parse could not be converted
org.xml.sax.Parser.setDocumentHandler could not be converted

org.xml.sax.Parser.setDTDHandler could not be converted
org.xml.sax.Parser.setEntityResolver could not be converted
org.xml.sax.Parser.setErrorHandler could not be converted
org.xml.sax.Parser.setLocale could not be converted
org.xml.sax.SAXException could not be converted
org.xml.sax.SAXNotRecognizedException could not be converted
org.xml.sax.SAXNotSupportedException could not be converted
org.xml.sax.SAXParseException could not be converted
org.xml.sax.SAXParseException.SAXParseException(String, Locator) could not be converted
org.xml.sax.SAXParseException.SAXParseException(String, Locator, Exception) could not be converted
org.xml.sax.SAXParseException.SAXParseException(String, String, int, int) could not be converted
org.xml.sax.XMLFilter.getParent could not be converted
org.xml.sax.XMLFilter.setParent could not be converted
org.xml.sax.XMLReader.getContentHandler could not be converted
org.xml.sax.XMLReader.getDTDHandler could not be converted
org.xml.sax.XMLReader.getEntityResolver could not be converted
org.xml.sax.XMLReader.getErrorHandler could not be converted
org.xml.sax.XMLReader.getFeature could not be converted
org.xml.sax.XMLReader.getProperty could not be converted
org.xml.sax.XMLReader.parse could not be converted
org.xml.sax.XMLReader.setContentHandler could not be converted
org.xml.sax.XMLReader.setDTDHandler could not be converted
org.xml.sax.XMLReader.setEntityResolver could not be converted
org.xml.sax.XMLReader.setErrorHandler could not be converted
org.xml.sax.XMLReader.setFeature could not be converted
org.xml.sax.XMLReader.setProperty could not be converted

C# for C++ Developers

The following table contains important comparisons between C# and native C++, which does not use **/clr**. If you are a C++ programmer, this table will give you the most important differences between the two languages at a glance.

Note

C++ and C# projects are derived from different project models. For more information about the differences between C++ and C# projects, see [Item Management in Projects](#) and [Using Solution Explorer](#).

Feature	Refer to the topic
Inheritance: In C++, classes and structs are virtually identical whereas in C#, they are quite different. C# classes can implement any number of interfaces, but can inherit from only one base class. Furthermore, C# structs do not support inheritance, and do not support explicit default constructors (one is provided by default).	class interface struct (C# Reference)
Arrays: In C++ an array is merely a pointer. In C#, arrays are objects that include methods and properties. For example, the size of an array can be queried via the Length property. C# arrays also employ <i>indexers</i> that verify each index used to access the array. The syntax for declaring C# arrays is different from that for C++ arrays: the tokens "[]" appear following the array type in C#, not the variable.	Arrays (C# Programming Guide) Indexers (C# Programming Guide)
Booleans: In C++, the bool type is essentially an integer. In C#, there is no conversion between the bool type and other types.	bool
The long type: In C#, the long type is 64 bits, while in C++, it is 32 bits.	long
Passing parameters: In C++, all variables are passed by value unless explicitly passed with a pointer or a reference. In C#, classes are passed by reference and structs are passed by value unless explicitly passed by reference with the ref or out parameter modifiers.	struct class ref (C# Reference) out (C# Reference)
The switch statement: Unlike the C++ switch statement, C# does not support fall-through from one case label to another.	switch
Delegates: C# delegates are roughly similar to function pointers in C++, are type-safe and secure.	delegate
Base-class methods: C# supports the base keyword for calling the overridden base class members from derived classes. Also, overriding virtual or abstract methods is explicit in C#, using the override keyword.	base See also the examples for override
Method hiding: C++ supports the implicit "hiding" of method through inheritance. In C#, you must use the new modifier to explicitly hide an inherited members.	new

Preprocessor directives are used for conditional compilation. No header files are used in C#.	C# Preprocessor Directives
Exception handling: C# provides the finally keyword to provide for code that should be executed regardless of whether an exception is thrown.	try-finally try-catch-finally
C# operators: C# supports additional operators such as is and typeof . It also introduces different functionality for some logical operators.	& Operator Operator ^ Operator is typeof
The extern keyword: In C++, extern is used to import types. In C#, extern is used to create aliases for using different versions of the same assembly.	extern
The static keyword: In C++, static can be used both to declare class-level entities and to declare types that are specific to a module. In C#, static is only used to declare class-level entities.	static
The Main method in C# is declared differently from the main function in C++. In C# it is capitalized, and always static . Also, support for processing of command-line arguments is much more robust in C#.	Main() and Command Line Arguments (C# Programming Guide)
Pointers are allowed in C#, but only in unsafe mode.	unsafe
Overloading operators is performed differently in C#.	C# Operators
Strings: In C++ a string is simply an array of characters. In C#, strings are objects that support robust searching methods.	string String
The foreach keyword enables you to iterate through arrays and collections.	foreach, in
Globals: In C#, global methods and variables are not supported. Methods and variables must be contained within a class or struct .	General Structure of a C# Program
Importing types: In C++, types common to multiple modules are placed in header files. In C#, this information is available via metadata.	using Metadata Overview
Local variables in C# cannot be used before they are initialized.	Methods (C# Programming Guide)
Memory management: C++ is not a garbage collected language; memory that is not explicitly released remains allocated until the process terminates. C# is a garbage collected language.	Garbage Collection

Destructors: C# has different syntax for deterministically releasing unmanaged resources.	Destructors using Statement (C# Reference)
Constructors: Similar to C++, if you do not provide a class constructor in C#, a default constructor is automatically generated for you. The default constructor initializes all the fields to their default values.	Instance Constructors Default Values Table
C# does not support bit fields.	C++ Bit Fields
C# input/output services and formatting rely on the run-time library of the .NET Framework.	C# Language Tour Formatting Numeric Results Table
In C#, method parameters cannot have default values. Use method overloads if you want to achieve the same effect.	Compiler Error CS0241
In C#, generic types and methods provide for type parameterization in a way that is similar to C++ templates, although there are significant differences.	Generics in C#
The as keyword is similar to a standard cast, except that rather than throw an exception if the conversion fails, the return value is null. This is similar to using static_cast in C++, which, unlike dynamic_cast , performs no run-time check and hence does not throw an exception on failure.	as (C# Reference)

For more information about comparisons between keywords in C# and other programming languages, see [Language Equivalents](#). For information on the general structure of C# applications, see [General Structure of a C# Program \(C# Programming Guide\)](#).

See Also

Concepts

[C# Programming Guide](#)

[Item Management in Projects](#)

[Using Solution Explorer](#)

Writing Applications with Visual C#

C# is a type-safe, object-oriented language that is simple yet powerful, allowing programmers to build a breadth of applications. Combined with the .NET Framework, Visual C# enables the creation of Windows applications, Web services, database tools, components, controls, and more.

This section contains information on various Microsoft platform technologies upon which you can base C# applications.

In This Section

[Using the .NET Framework Class Library \(Visual C#\)](#)

Describes using the .NET Framework class library types in your Visual C# projects.

[Creating ASP.NET Web Applications \(Visual C#\)](#)

Describes using the C# code editor within Visual Web Developer to create Web applications with code-behind pages in C#.

[Creating Windows Forms Applications \(Visual C#\)](#)

Describes using Windows Forms to create Windows applications.

[Creating Console Applications \(Visual C#\)](#)

Explains creating applications that do not require a graphical user interface.

[Accessing and Displaying Data \(Visual C#\)](#)

Describes interacting with databases.

[Creating Mobile and Embedded Applications \(Visual C#\)](#)

Describes creating applications for smart devices, embedded devices, and thin mobile clients.

[Creating and Accessing Web Services \(Visual C#\)](#)

Describes interacting with XML Web services.

[Creating Components \(Visual C#\)](#)

Describes creating user controls and other components for the .NET Framework.

[Developing on the Office Platform \(Visual C#\)](#)

Describes using Visual Studio Tools for Office to create smart documents.

[Developing for the Enterprise \(Visual C#\)](#)

Describes developing applications for SQL Server, Microsoft Exchange Server, and more.

[Tablet PC Programming \(Visual C#\)](#)

Describes developing ink-based applications for the Tablet PC.

[Audio, Video, Games and Graphics \(Visual C#\)](#)

Describes using Windows Media and DirectX for Managed code.

[Creating Starter Kits \(Visual C#\)](#)

Describes creating starter kits to help others get up and running quickly with your sample code.

See Also

Other Resources

[Visual C#](#)

[C# Reference](#)

[Getting Started with Visual C#](#)

[Using the Visual C# IDE](#)

Using the .NET Framework Class Library (Visual C#)

Most Visual C# development projects use the .NET Framework Class Library extensively for everything from file system access and string manipulation to Windows Forms and ASP.NET user interface controls.

The class library is organized into namespaces, each of which contains a set of related classes and structs. For example, the [System.Drawing](#) namespace contains numerous types that represent fonts, pens, lines, shapes, colors, and so on.

using Directives and References

Before you can use the classes in a given namespace in a C# program, you must add a [using directive](#) for that namespace to your C# source file. In some cases, you must also add a reference to the DLL that contains the namespace; Visual C# automatically adds references to the most commonly used class library DLLs. You can see which references have been added in **Solution Explorer** under the **References** node. For more information, see [Creating a Project \(Visual C#\)](#).

After you have added the **using** directive for a namespace, you can create instances of its types, call methods, and respond to events just as if they were declared in your own source code. In the Visual C# code editor, you can also put the insertion point over a type or member name, and press F1 to see the Help documentation. You can also use the **Object Browser** tool and **Metadata As Source** feature to see type information on .NET Framework classes and structs. For more information, see [Modeling and Analyzing Code \(Visual C#\)](#).

For More Information

- For more information about the .NET Framework class library, see [.NET Framework Class Library Overview](#) and [.NET Framework Programming](#).
- For more information about .NET Framework architecture, see [Overview of the .NET Framework](#).
- On the Internet, the [.NET Framework Developer Center](#) has many articles and code examples about the class library.
- For information about how to perform specific tasks using the class libraries, see [How Do I in C#](#) or click **How Do I** on the **Help** menu in Visual C#.

See Also

Other Resources

[Writing Applications with Visual C#](#)

[Using the Visual C# IDE](#)

Creating ASP.NET Web Applications (Visual C#)

ASP.NET provides a unified Web development model that includes the services necessary for you to build enterprise-class Web applications. ASP.NET is part of the .NET Framework and enables you to take full advantage of the features of the common language runtime (CLR), such as type safety, inheritance, language interoperability, and versioning.

When you use Visual Studio to create ASP.NET Web sites, you are actually using a part of the integrated development environment (IDE) called Visual Web Developer. Visual Web Developer is distinct from Visual C#; it has its own designer for creating user interfaces on Web pages and other tools for Web development and Web site administration. But when you create code-behind pages in C# for your Web controls, you are using the C# code editor, and all the features of the editor are available to you in Visual Web Developer just as they are in Visual C#.

For more information about creating ASP.NET Web sites and Web pages, see [Visual Web Developer](#).

The [ASP.NET Developer Center](#) on MSDN Online has many useful articles not included in the product documentation.

See Also

Other Resources

[Writing Applications with Visual C#](#)

Creating Windows Forms Applications (Visual C#)

Windows Forms is the technology that you use in Visual C# to create Windows-based smart client applications that run on the .NET Framework. When you create a Windows Application project, you are creating an application based on Windows Forms. You will use the [Windows Forms Designer](#) to create your user interface, and you will have access to other design and run-time features including:

- [ClickOnce Deployment](#).
- Rich database support with the [DataGridView Control](#).
- Toolbars and other user interface elements that can have the appearance and behavior of Microsoft® Windows® XP, Microsoft Office, or Microsoft Internet Explorer.

For more information, see [Designing a User Interface \(Visual C#\)](#) and [Windows Forms](#).

See Also

Other Resources

[Writing Applications with Visual C#](#)

Creating Console Applications (Visual C#)

C# can be used to create applications that take input and display output at the command line console. These applications are ideal for learning C# development because the user interface is so simple. Console applications are also very useful for utility programs that require little or no user interaction.

To create a console application in Visual C#, click on **New** on the **File** menu and select **Project**. Click on the C# Console Application project template, provide the filenames that you want, and click OK.

Using the System.Console class

Use the [Console](#) class to read and write characters from the console, either individually or as an entire line. Output can be formatted in many different ways. For more information, see [Formatting Overview](#).

Command-Line Arguments can be accessed using the optional string array associated with the `Main` method. For more information, see [Command-Line Arguments \(C# Programming Guide\)](#).

Examples

- [How to: Create a Console Application Client](#)
- [Command Line Parameters Sample](#)

See Also

Other Resources

[Writing Applications with Visual C#](#)

Accessing and Displaying Data (Visual C#)

In Visual C# applications, you will typically connect to databases using [ADO.NET](#) technology as exposed in [System.Data](#) and related namespaces in the .NET Framework class library. For more information on working with data, see [Accessing Data \(Visual Studio\)](#).

In Windows Forms applications, the primary user interface control for displaying data retrieved from databases is the **DataGridView**. For more information, see [DataGridView Control \(Windows Forms\)](#). Connecting data sources to user interface controls such as text boxes and list boxes is greatly simplified through a feature known as data binding. When you bind a control to a field in a data source, changes made in one will automatically be reflected in the other. For more information, see [Windows Forms Data Binding](#).

For more information about creating and managing databases, writing stored procedures, and other related information, see [SQL Server Projects](#) and [SQL Server Tutorials](#).

The following links contain information on using Visual Studio to access data:

- [Creating Data Applications with Visual Studio](#)
- [Visual Database Tools](#)

On the internet, the [Data Access and Storage Developer Center](#) is continually updated with new articles and examples.

See Also

Other Resources

[Writing Applications with Visual C#](#)

Creating Mobile and Embedded Applications (Visual C#)

Microsoft Visual Studio 2005 offers rich, integrated support for developing software that runs on Windows CE-based smart devices such as Pocket PCs and Smartphones. You can use Visual C# to write managed applications that run on the .NET Compact Framework. You will use the same Code Editor, designers, and debugger interface as you would use when developing for the PC. Simply select from one of the **Smart Device** project templates available for the language of your choice, and begin coding.

Visual Studio also provides emulators for smart devices that enable you to run and debug your code on your development computer, and tools that simplify the process of packaging your application and its resources into CAB files for deployment to end-user devices. For more information, see [Smart Devices](#).

You can also use [Visual Web Developer](#) to develop mobile web applications based on ASP.NET. For more information, see [Introduction to Mobile Web Forms](#).

For the most up-to-date information on Windows Mobile technology, visit the [Mobile Developer Center](#).

See Also

Other Resources

[Writing Applications with Visual C#](#)

Creating and Accessing Web Services (Visual C#)

XML Web services provide the ability for applications to communicate in a loosely coupled environment using predefined message exchanges built around standard protocols such as HTTP, XML, XSD, SOAP, and WSDL. Because the protocols and specifications are public and not platform-specific, XML Web services can be used to enable applications to communicate whether they reside on the same computer or not, even if they reside on different computing platforms or devices.

You do not need to understand each specification in depth to build or consume Web services. The .NET Framework classes and Visual Studio wizards help you build or interact with XML Web services using a familiar object-oriented programming model.

For more information about XML Web services, see [XML Web Services Overview](#).

For information on creating and accessing XML Web Services using ASP.NET, see [XML Web Services Using ASP.NET](#).

For information on the tools Visual Studio provides you to help build and consume XML Web services easily, see [XML Web Services \(Visual Studio\)](#).

For more information about creating and accessing XML Web Services, see [XML Web Services in Managed Code](#).

The [Web Services Developer Center](#) on MSDN Online has additional articles and resources.

See Also

Other Resources

[Writing Applications with Visual C#](#)

Creating Components (Visual C#)

The term *component* in the software industry is often used to refer to a reusable object that exposes one or more interfaces to clients in a standardized way. A component may be implemented as a single class, or as a set of classes; the primary requirement is that the basic public interface be well-defined. For example, in the context of native Windows programming, the Component Object Model (COM) requires that all components implement the [IUnknown](#) interface in addition to any other specialized interfaces.

In the context of the .NET Framework, a component is a class or set of classes that implements the [IComponent](#) interface or one that derives directly or indirectly from a class that implements this interface. The default base class implementation of the **IComponent** interface is [Component](#).

Some of the most commonly used components in .NET Framework programming are the visual controls that you add to Windows Forms such as the [Button Control \(Windows Forms\)](#), [ComboBox Control \(Windows Forms\)](#), and so on. Non-visual components include the [Timer Control](#), [SerialPort](#), and [ServiceController](#) among others.

When you create a component in C#, it can be used by clients written in any other language that conforms to the [Common Language Specification](#).

To create your own components in Visual C#, you can use the [Component Designer](#) to assemble non-visual component classes in the same way that you assemble Windows Forms. For more information, see [Walkthrough: Creating a Windows Service Application in the Component Designer](#).

For more information about component programming using Visual Studio, see [Components in Visual Studio](#).

See Also

Other Resources

[Writing Applications with Visual C#](#)

Developing on the Office Platform (Visual C#)

Microsoft Visual Studio 2005 Tools for the Microsoft Office System enables you to customize Microsoft Office documents and Microsoft Office Outlook using managed code. Visual Studio Tools for Office adds new functionality to Visual Studio, such as hosting Microsoft Office Word and Microsoft Office Excel as designers within the Visual Studio development environment, the ability to program directly against data and presentation objects, and the ability to use Windows Forms controls on documents and within the Document Actions task pane.

For details about developing on the Office platform, see these topics:

[What's New in Visual Studio Tools for Office](#)

[Getting Started Programming Document-Level Customizations for Excel](#)

[Getting Started Programming Document-Level Customizations for Word](#)

[Getting Started Programming Application-Level Add-ins](#)

[Common Tasks in Office Programming](#)

See Also

Other Resources

[Writing Applications with Visual C#](#)

Developing for the Enterprise (Visual C#)

You can program in C# to develop a wide variety of applications for the enterprise running on server-based architectures and server products. The [Servers and Enterprise Development](#) section of the MSDN Library contains architectural guidance, patterns and practices information, and documentation and technical articles on using Microsoft's server products, including:

- Microsoft SQL Server
- Microsoft BizTalk Server
- Microsoft Commerce Server
- Microsoft Content Management Server
- Microsoft Exchange Server
- Microsoft Host Integration Server
- Microsoft Internet Security and Acceleration Server 2000
- Microsoft Business Solutions
- Microsoft MapPoint
- Microsoft Speech Server

See Also

Other Resources

[Writing Applications with Visual C#](#)

Tablet PC Programming (Visual C#)

The Tablet PC is a personal computer powered by Microsoft® Windows® XP that is geared for ink-enabled, pen-enabled, and speech-enabled applications. The combination of software and hardware in a Tablet PC enables these methods of user interaction and allows for a rich, interactive, and productive computing experience for users.

The Tablet PC platform encompasses Windows XP and its extensions that enable input and output of handwriting and speech data on a Tablet PC as well as interchange of this data with other computers.

For more information, see [Windows XP Tablet PC Edition](#) on MSDN online and the [Tablet and Mobile PC Developer Center](#).

See Also

Other Resources

[Writing Applications with Visual C#](#)

[Tablet and Mobile PC Developer Center](#)

Audio, Video, Games and Graphics (Visual C#)

You can Visual C# to create multimedia applications based on DirectX for Managed Code and Windows Media Technologies.

Managed DirectX

Microsoft® DirectX® is an advanced suite of multimedia application programming interfaces (APIs) built into Microsoft Windows® operating systems. DirectX provides a standard development platform for Windows-based PCs by enabling software developers to access specialized hardware features without having to write hardware-specific code. This technology was first introduced in 1995 and is a recognized standard for multimedia application development on the Windows platform.

Simply put, DirectX is a Windows technology that enables higher performance in graphics, including full-color graphics, video, and 3-D animation, and sound, including surround sound, when you are playing games or watching video on your PC.

For more information about using DirectX in C# applications, see [DirectX 9.0 for Managed Code](#) on MSDN Online and the [Microsoft DirectX Developer Center](#).

Windows Media Player

The Windows Media Player ActiveX control can be used in C# applications to add audio and video playback capability. The Microsoft Windows Media Player 10 Software Development Kit (SDK) provides information and tools to customize Windows Media Player and to use the Windows Media Player ActiveX control. The SDK includes documentation and code examples showing how to use the Media Player ActiveX control from a C# application.

For more information, see [Windows Media Player 10 SDK](#) on MSDN Online.

Windows Media Encoder

The Windows Media Encoder 9 Series SDK can be programmed using C# to perform the following types of tasks:

- **Broadcast live content.** A news organization can use the Automation API to schedule the automatic capture and broadcast of live content. For example, local transportation departments can stream live pictures of road conditions at multiple trouble spots, alerting drivers to traffic congestion and advising them of alternate routes.
- **Batch-process content.** A media production organization that must process a high volume of large files can create a batch process that uses the Automation API to repeatedly capture and encode streams, one after the other. A corporation can use the Automation API to manage its streaming media services with a preferred scripting language and Windows Script Host. Windows Script Host is a language-independent host that can be used to run any script engine on the Microsoft Windows® 95 or later, Windows NT, or Windows 2000 operating systems.
- **Create a custom user interface.** An Internet service provider (ISP) can build an interface that uses the functionality of the Automation API to capture, encode, and broadcast media streams. Alternatively, you can use the predefined user interfaces within the Automation API for the same purpose.
- **Remotely administer Windows Media Encoder applications.** You can use the Automation API to run, troubleshoot, and administer Windows Media Encoder applications from a remote computer.

For more information, see [Windows Media Encoder 9 Series SDK](#) on MSDN online. The [Programming C#](#) topic describes which references to include when working in C#.

Windows Media Server

The Microsoft® Windows Media® Services 9 Series Software Development Kit (SDK) is a powerful automation-based Application Programming Interface (API) designed for anyone who wants to develop a Windows Media Services 9 Series application. You can use this SDK in C# to programmatically manage a Windows Media server to send digital media content to clients on both unicast and multicast-enabled networks. For more information, see [Windows Media Services 9 Series SDK](#).

See Also

Other Resources

[Writing Applications with Visual C#](#)

Creating Starter Kits (Visual C#)

A starter kit contains code for a complete application and documentation on how to modify or expand the application. Visual C# 2005 provides two starter kits that you can access from the **New Project** dialog box. You can also create your own starter kits to help users get up and running quickly on any type of application. Creating a starter kit is fundamentally the same as creating a normal project template, the only difference being that a starter kit contains documentation files that are set to open when a project based on the starter kit is created.

For more information, see [Introduction to Starter Kits](#).

See Also

Other Resources

[Writing Applications with Visual C#](#)

C# Programming Guide

This section provides detailed information on key C# language features and features accessible to C# through the .NET Framework.

Language Sections

[Inside a C# Program](#)

[Main\(\) and Command Line Arguments \(C# Programming Guide\)](#)

[Data Types \(C# Programming Guide\)](#)

[Arrays \(C# Programming Guide\)](#)

[Strings \(C# Programming Guide\)](#)

[Statements, Expressions, and Operators \(C# Programming Guide\)](#)

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Properties \(C# Programming Guide\)](#)

[Indexers \(C# Programming Guide\)](#)

[Delegates \(C# Programming Guide\)](#)

[Events \(C# Programming Guide\)](#)

[Generics \(C# Programming Guide\)](#)

[Iterators \(C# Programming Guide\)](#)

[Namespaces \(C# Programming Guide\)](#)

[Nullable Types \(C# Programming Guide\)](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

[XML Documentation Comments \(C# Programming Guide\)](#)

Platform Sections

[Application Domains \(C# Programming Guide\)](#)

[Assemblies and the Global Assembly Cache \(C# Programming Guide\)](#)

[Attributes \(C# Programming Guide\)](#)

[Collection Classes \(C# Programming Guide\)](#)

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

[Interoperability \(C# Programming Guide\)](#)

[Threading \(C# Programming Guide\)](#)

[Performance \(C# Programming Guide\)](#)

[Reflection \(C# Programming Guide\)](#)

[C# DLLs \(C# Programming Guide\)](#)

[Security \(C# Programming Guide\)](#)

See Also

Other Resources

[C# Reference](#)

[Visual C#](#)

Inside a C# Program

The section discusses the general structure of a C# program, and includes the standard "Hello, World!" example.

In This Section

- [Hello World -- Your First Program \(C# Programming Guide\)](#)
- [General Structure of a C# Program \(C# Programming Guide\)](#)

Related Sections

- [Getting Started with Visual C#](#)
- [Migrating to Visual C#](#)
- [C# Programming Guide](#)
- [C# Reference](#)
- [Visual C# Samples](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1 Introduction

See Also

Concepts

[C# Programming Guide](#)

Hello World -- Your First Program (C# Programming Guide)

The following console program is the C# version of the traditional "Hello World!" program, which displays the string `Hello World!`.

C#

```
using System;
// A "Hello World!" program in C#
namespace HelloWorld
{
    class Hello
    {
        static void Main()
        {
            System.Console.WriteLine("Hello World!");
        }
    }
}
```

Let us now look at the important parts of this program in turn.

Comments

The first line contains a comment:

C#

```
// A "Hello World!" program in C#
```

The characters `//` convert the rest of the line to a comment. You can also comment a block of text by placing it between the characters `/*` and `*/`, for example:

C#

```
/* A "Hello World!" program in C#.
   This program displays the string "Hello World!" on the screen. */
```

The Main Method

The C# program must contain a `Main` method, in which control starts and ends. The `Main` method is where you create objects and execute other methods.

The `Main` method is a static method that resides inside a class or a struct. In the previous "Hello World!" example, it resides in a class called `Hello`. Declare the `Main` method in one of the following ways:

- It can return **void**:

C#

```
static void Main()
{
    //...
}
```

- It can also return an **int**:

C#

```
static int Main()
```

```
{  
    //...  
    return 0;  
}
```

- With both of the return types, it can take arguments:

C#

```
static void Main(string[] args)  
{  
    //...  
}
```

-or-

C#

```
static int Main(string[] args)  
{  
    //...  
    return 0;  
}
```

The parameter of the `Main` method is a `string` array that represents the command-line arguments used to invoke the program. Notice that, unlike C++, this array does not include the name of the executable (exe) file.

For more information on using command-line arguments, see the example in [Main\(\) and Command Line Arguments \(C# Programming Guide\)](#) and [How to: Create and Use C# DLLs \(C# Programming Guide\)](#).

Input and Output

C# programs generally use the input/output services provided by the run-time library of the .NET Framework. The statement, `System.Console.WriteLine("Hello World!");` uses the [WriteLine](#) method, one of the output methods of the `Console` class in the run-time library. It displays its string parameter on the standard output stream followed by a new line. Other [Console](#) methods are used for different input and output operations. If you include the `using System;` directive at the beginning of the program, you can directly use the [System](#) classes and methods without fully qualifying them. For example, you can call `Console.WriteLine` instead, without specifying `System.Console.WriteLine`:

C#

```
using System;
```

C#

```
Console.WriteLine("Hello World!");
```

For more information on input/output methods, see [System.IO](#).

Compilation and Execution

You can compile the "Hello World!" program either by creating a project in the Visual Studio IDE, or by using the command line. Use the Visual Studio Command Prompt or invoke `vsvars32.bat` to put the Visual C# tool set on the path in your command prompt.

To compile the program from the command line:

- Create the source file using any text editor and save it using a name such as `Hello.cs`. C# source code files use the

extension `.cs`.

- To invoke the compiler, enter the command:

```
csc Hello.cs
```

If your program does not contain any compilation errors, a `Hello.exe` file will be created.

- To run the program, enter the command:

```
Hello
```

For more information on the C# compiler and its options, see [C# Compiler Options](#).

See Also

Reference

[Inside a C# Program](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

[C# Reference](#)

General Structure of a C# Program (C# Programming Guide)

C# programs can consist of one or more files. Each file can contain zero or more namespaces. A namespace can contain types such as classes, structs, interfaces, enumerations, and delegates, in addition to other namespaces. The following is the skeleton of a C# program that contains all of these elements.

C#

```
// A skeleton of a C# program
using System;
namespace YourNamespace
{
    class YourClass
    {
    }

    struct YourStruct
    {
    }

    interface IYourInterface
    {
    }

    delegate int YourDelegate();

    enum YourEnum
    {
    }

    namespace YourNestedNamespace
    {
        struct YourStruct
        {
        }
    }

    class YourMainClass
    {
        static void Main(string[] args)
        {
            //Your program starts here...
        }
    }
}
```

Related Sections

For more information:

- [Classes \(C# Programming Guide\)](#)
- [Structs \(C# Programming Guide\)](#)
- [Namespaces \(C# Programming Guide\)](#)
- [Interfaces \(C# Programming Guide\)](#)
- [Delegates \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.2 Program Structure

- [9.1 Compilation Units \(Namespaces\)](#)

See Also

Reference

[Inside a C# Program](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Visual C# Samples](#)

Main() and Command Line Arguments (C# Programming Guide)

The **Main** method is the entry point of your program, where you create objects and invoke other methods. There can only be one entry point in a C# program.

C#

```
class TestClass
{
    static void Main(string[] args)
    {
        // Display the number of command line arguments:
        System.Console.WriteLine(args.Length);
    }
}
```

Overview

- The **Main** method is the entry point of your program, where the program control starts and ends.
- It is declared inside a class or struct. It must be static and it should not be public. (In the example above it receives the default access of private.)
- It can either have a void or int return type.
- The **Main** method can be declared with or without parameters.
- Parameters can be read as zero-indexed command line arguments.
- Unlike C and C++, the name of the program is not treated as the first command line argument.

In This Section

- [Command-Line Arguments \(C# Programming Guide\)](#)
- [How to: Display Command Line Arguments \(C# Programming Guide\)](#)
- [How to: Access Command-Line Arguments Using foreach \(C# Programming Guide\)](#)
- [Main\(\) Return Values \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.1 Hello World

See Also

Reference

[Inside a C# Program](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Command-Line Arguments (C# Programming Guide)

The `Main` method can use arguments, in which case, it takes one of the following forms:

C#

```
static int Main(string[] args)
```

C#

```
static void Main(string[] args)
```

The parameter of the `Main` method is a `String` array that represents the command-line arguments. Usually you check for the existence of the arguments by testing the `Length` property, for example:

C#

```
if (args.Length == 0)
{
    System.Console.WriteLine("Please enter a numeric argument.");
    return 1;
}
```

You can also convert the string arguments to numeric types by using the `Convert` class or the `Parse` method. For example, the following statement converts the string to a long number by using the `Parse` method on the `Int64` class:

```
long num = Int64.Parse(args[0]);
```

It is also possible to use the C# type `long`, which aliases `Int64`:

```
long num = long.Parse(args[0]);
```

You can also use the `Convert` class method `ToInt64` to do the same thing:

```
long num = Convert.ToInt64(s);
```

For more information, see [Parse](#) and [Convert](#).

Example

In this example, the program takes one argument at run time, converts the argument to an integer, and calculates the factorial of the number. If no arguments are supplied, the program issues a message that explains the correct usage of the program.

C#

```
// arguments: 3
```

C#

```
public class Functions
{
    public static long Factorial(int n)
    {
        if (n < 0) { return -1; }    //error result - undefined
        if (n > 256) { return -2; } //error result - input is too big

        if (n == 0) { return 1; }
```



```

        // Calculate the factorial iteratively rather than recursively:

        long tempResult = 1;
        for (int i = 1; i <= n; i++)
        {
            tempResult *= i;
        }
        return tempResult;
    }
}

```

C#

```

class MainClass
{
    static int Main(string[] args)
    {
        // Test if input arguments were supplied:
        if (args.Length == 0)
        {
            System.Console.WriteLine("Please enter a numeric argument.");
            System.Console.WriteLine("Usage: Factorial <num>");
            return 1;
        }

        try
        {
            // Convert the input arguments to numbers:
            int num = int.Parse(args[0]);

            System.Console.WriteLine("The Factorial of {0} is {1}.", num, Functions.Factori
al(num));
            return 0;
        }
        catch (System.FormatException)
        {
            System.Console.WriteLine("Please enter a numeric argument.");
            System.Console.WriteLine("Usage: Factorial <num>");
            return 1;
        }
    }
}

```

Output

The Factorial of 3 is 6.

Comments

The following are two sample runs of the program assuming that the program name is `Factorial.exe`.

Run #1:

Enter the following command line:

```
Factorial 10
```

You get the following result:

```
The Factorial of 10 is 3628800.
```

Run #2:

Enter the following command line:

```
Factorial
```

You get the following result:

```
Please enter a numeric argument.
```

Usage: Factorial <num>

For more examples on using command-line arguments, see the example in [How to: Create and Use C# DLLs \(C# Programming Guide\)](#).

See Also

Tasks

[How to: Display Command Line Arguments \(C# Programming Guide\)](#)

[How to: Access Command-Line Arguments Using foreach \(C# Programming Guide\)](#)

Reference

[Main\(\) and Command Line Arguments \(C# Programming Guide\)](#)

[Main\(\) Return Values \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

How to: Display Command Line Arguments (C# Programming Guide)

Arguments provided to an executable on the command-line are accessible through an optional parameter to `Main`. The arguments are provided in the form of an array of strings. Each element of the array contains one argument. White-space between arguments is removed. For example, consider these command-line invocations of a fictitious executable:

Input on Command-line	Array of strings passed to Main
<code>executable.exe a b c</code>	"a" "b" "c"
<code>executable.exe one two</code>	"one" "two"
<code>executable.exe "one two" three</code>	"one two" "three"

Example

This example displays the command line arguments passed to a command-line application. The output shown is for the first entry in the table above.

C#

```
class CommandLine
{
    static void Main(string[] args)
    {
        // The Length property provides the number of array elements
        System.Console.WriteLine("parameter count = {0}", args.Length);

        for (int i = 0; i < args.Length; i++)
        {
            System.Console.WriteLine("Arg[{0}] = [{1}]", i, args[i]);
        }
    }
}
```

Output

parameter count = 3 Arg[0] = [a] Arg[1] = [b] Arg[2] = [c]

See Also

Tasks

[How to: Access Command-Line Arguments Using foreach \(C# Programming Guide\)](#)

Reference

[Main\(\) and Command Line Arguments \(C# Programming Guide\)](#)

[Main\(\) Return Values \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

How to: Access Command-Line Arguments Using foreach (C# Programming Guide)

Another approach to iterating over the array is to use the [foreach](#) statement as shown in this example. The **foreach** statement can be used to iterate over an array, a .NET Framework collection class, or any class or struct that implements the [IEnumerable](#) interface.

Example

This example demonstrates how to print out the command line arguments using **foreach**.

C#

```
// arguments: John Paul Mary
```

C#

```
class CommandLine2
{
    static void Main(string[] args)
    {
        System.Console.WriteLine("Number of command line parameters = {0}", args.Length);

        foreach (string s in args)
        {
            System.Console.WriteLine(s);
        }
    }
}
```

Output

```
Number of command line parameters = 3
John
Paul
Mary
```

See Also

Tasks

[How to: Display Command Line Arguments \(C# Programming Guide\)](#)

Reference

[foreach](#), in (C# Reference)

[Main\(\) and Command Line Arguments \(C# Programming Guide\)](#)

[Main\(\) Return Values \(C# Programming Guide\)](#)

[Array](#)

[System.Collections](#)

Concepts

[C# Programming Guide](#)

Main() Return Values (C# Programming Guide)

The `Main` method can be of the type **void**:

C#

```
static void Main()
{
    //...
}
```

It can also return an **int**:

C#

```
static int Main()
{
    //...
    return 0;
}
```

If the return value from `Main` is not to be used, then returning **void** allows slightly simpler code. However, returning an integer enables the program to relate status information to other programs or scripts that invoke the executable. An example of using the return value from `Main` is shown in the following example.

Example

In this example a batch file is used to execute a program and test the return value of the `Main` function. When a program is executed in Windows, any value returned from the `Main` function is stored in an environment variable called `ERRORLEVEL`. By inspecting the `ERRORLEVEL` variable, batch files can therefore determine the outcome of execution. Traditionally, a return value of zero indicates successful execution. Below is a very simple program that returns zero from the `Main` function.

C#

```
class MainReturnValTest
{
    static int Main()
    {
        //...
        return 0;
    }
}
```

Because this example uses a batch file, it is best to compile this code from the command line, as demonstrated in [How to: Set Environment Variables](#).

Next, a batch file is used to invoke the executable resulting from the previous code example. Because the code returns zero, the batch file will report success, but if the previous code is changed to return a non-zero value, and is then re-compiled, subsequent execution of the batch file will indicate failure.

```
rem test.bat
@echo off
MainReturnValueTest
@if "%ERRORLEVEL%" == "0" goto good

:fail
    echo Execution Failed
    echo return value = %ERRORLEVEL%
    goto end

:good
    echo Execution Succeeded
```

```
    echo return value = %ERRORLEVEL%
    goto end

:end
```

Sample Output

Execution Succeeded

return value = 0

See Also

Tasks

[How to: Display Command Line Arguments \(C# Programming Guide\)](#)

[How to: Access Command-Line Arguments Using foreach \(C# Programming Guide\)](#)

Reference

[Main\(\) and Command Line Arguments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Data Types (C# Programming Guide)

C# is a strongly typed language; therefore every variable and object must have a declared type.

Data Types Overview

A data type can be described as being either:

- A built-in data type, such as an **int** or **char**, or
- A user-defined data type, such as a **class** or **interface**.
- Data types can also be defined as being either:
 - [Value Types \(C# Reference\)](#), which store values, or
 - [Reference Types \(C# Reference\)](#), which store references to the actual data.

Related Sections

For more information:

- [Casting \(C# Programming Guide\)](#)
- [Boxing and Unboxing \(C# Programming Guide\)](#)
- [Types \(C# Reference\)](#)
- [Objects, Classes and Structs \(C# Programming Guide\)](#)

C# Language Specification

For more information about types, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 3.8 Namespace and Type Names
- 4.1 Value Types
- 4.2 Reference Types
- 4.3 Boxing and Unboxing

See Also

Reference

[Data Types Compared in Different Languages](#)

[Integral Types Table \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Conversion of XML Data Types](#)

Other Resources

[C# Reference](#)

Casting (C# Programming Guide)

Converting between data types can be done explicitly using a cast, but in some cases, implicit conversions are allowed. For example:

C#

```
static void TestCasting()
{
    int i = 10;
    float f = 0;
    f = i; // An implicit conversion, no data will be lost.
    f = 0.5F;
    i = (int)f; // An explicit conversion. Information will be lost.
}
```

A cast explicitly invokes the conversion operator from one type to another. The cast will fail if no such conversion operator is defined. You can write custom conversion operators to convert between user-defined types. For more information about defining a conversion operator, see [explicit \(C# Reference\)](#) and [implicit \(C# Reference\)](#).

Example

The following program casts a [double](#) to an [int](#). The program will not compile without the cast.

C#

```
class Test
{
    static void Main()
    {
        double x = 1234.7;
        int a;
        a = (int)x; // cast double to int
        System.Console.WriteLine(a);
    }
}
```

Output

1234

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [7.6.6 Cast Expressions](#)
- [6.1 Implicit Conversions](#)
- [6.2 Explicit Conversions](#)

See Also

Reference

[Data Types \(C# Programming Guide\)](#)

[\(\) Operator \(C# Reference\)](#)

[explicit \(C# Reference\)](#)

[implicit \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Conversion Operators \(C# Programming Guide\)](#)

[Generalized Type Conversion](#)

[Explicit Conversion](#)

[Exported Type Conversion](#)

Boxing and Unboxing (C# Programming Guide)

Boxing and unboxing enable value types to be treated as objects. Boxing a value type packages it inside an instance of the [Object](#) reference type. This allows the value type to be stored on the garbage collected heap. Unboxing extracts the value type from the object. In this example, the integer variable `i` is *boxed* and assigned to object `o`.

C#

```
int i = 123;
object o = (object)i; // boxing
```

The object `o` can then be unboxed and assigned to integer variable `i`:

C#

```
o = 123;
i = (int)o; // unboxing
```

Performance

In relation to simple assignments, boxing and unboxing are computationally expensive processes. When a value type is boxed, an entirely new object must be allocated and constructed. To a lesser degree, the cast required for unboxing is also expensive computationally. For more information, see [Performance](#).

Related Sections

For more information:

- [Boxing Conversion](#)
- [Unboxing Conversion](#)
- [Reference Types](#)
- [Value Types](#)

C# Language Specification

For more information, see the following section in the [C# Language Specification](#):

- 4.3 Boxing and Unboxing

See Also

Concepts

[C# Programming Guide](#)

[Boxed Value Types](#)

Boxing Conversion (C# Programming Guide)

Boxing is used to store value types in the garbage-collected heap. Boxing is an implicit conversion of a [Value Types \(C# Reference\)](#) to the type **object** or to any interface type implemented by this value type. Boxing a value type allocates an object instance on the heap and copies the value into the new object.

Consider the following declaration of a value-type variable:

C#

```
int i = 123;
```

The following statement implicitly applies the boxing operation on the variable `i`:

C#

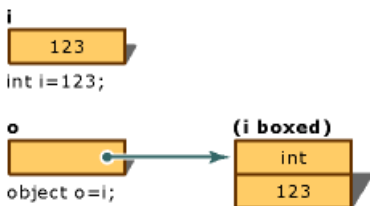
```
object o = i; // implicit boxing
```

The result of this statement is creating an object reference `o`, on the stack, that references a value of the type **int**, on the heap. This value is a copy of the value-type value assigned to the variable `i`. The difference between the two variables, `i` and `o`, is illustrated in the following figure.

Boxing Conversion

On the stack

On the heap



It also possible to perform the boxing explicitly as in the following example, but explicit boxing is never required:

C#

```
int i = 123;
object o = (object)i; // explicit boxing
```

Example

This example converts an integer variable `i` to an object `o` by means of boxing. Then, the value stored in the variable `i` is changed from `123` to `456`. The example shows that the original value type and the boxed object use separate memory locations, and therefore can store different values.

C#

```
class TestBoxing
{
    static void Main()
    {
        int i = 123;
        object o = i; // implicit boxing

        i = 456; // change the contents of i

        System.Console.WriteLine("The value-type value = {0}", i);
        System.Console.WriteLine("The object-type value = {0}", o);
    }
}
```

Output

The value-type value = 456

The object-type value = 123

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 4.3.1 Boxing Conversions

See Also

Concepts

[C# Programming Guide](#)

[Boxing and Unboxing \(C# Programming Guide\)](#)

[Unboxing Conversion \(C# Programming Guide\)](#)

[Boxed Value Types](#)

Unboxing Conversion (C# Programming Guide)

Unboxing is an explicit conversion from the type **object** to a **value type** or from an interface type to a value type that implements the interface. An unboxing operation consists of:

- Checking the object instance to make sure it is a boxed value of the given value type.
- Copying the value from the instance into the value-type variable.

The following statements demonstrate both boxing and unboxing operations:

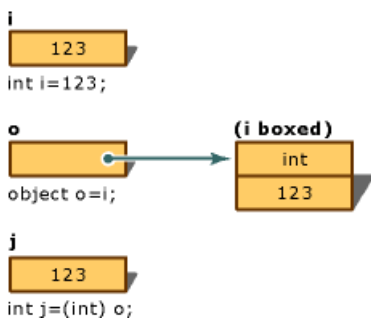
C#

```
int i = 123;      // a value type
object o = i;     // boxing
int j = (int)o;   // unboxing
```

The following figure demonstrates the result of the preceding statements.

Unboxing Conversion

On the stack On the heap



For the unboxing of value types to succeed at run time, the item being unboxed must be a reference to an object that was previously created by boxing an instance of that value type. Attempting to unbox **null** or a reference to an incompatible value type will result in an [InvalidCastException](#).

Example

The following example demonstrates a case of invalid unboxing and the resulting **InvalidCastException**. Using **try** and **catch**, an error message is displayed when the error occurs.

C#

```
class TestUnboxing
{
    static void Main()
    {
        int i = 123;
        object o = i; // implicit boxing

        try
        {
            int j = (short)o; // attempt to unbox

            System.Console.WriteLine("Unboxing OK.");
        }
        catch (System.InvalidCastException e)
        {
            System.Console.WriteLine("{0} Error: Incorrect unboxing.", e.Message);
        }
    }
}
```

Output

Specified cast is not valid. Error: Incorrect unboxing.

If you change the statement:

<code>int j = (short) o;</code>

to:

<code>int j = (int) o;</code>

the conversion will be performed, and you will get the output:

Unboxing OK.

See Also

Concepts

[C# Programming Guide](#)

[Boxing and Unboxing \(C# Programming Guide\)](#)

[Boxing Conversion \(C# Programming Guide\)](#)

Arrays (C# Programming Guide)

An array is a data structure that contains a number of variables of the same type. Arrays are declared with a type:

```
type[] arrayName;
```

The following examples create single-dimensional, multidimensional, and jagged arrays:

C#

```
class TestArraysClass
{
    static void Main()
    {
        // Declare a single-dimensional array
        int[] array1 = new int[5];

        // Declare and set array element values
        int[] array2 = new int[] { 1, 3, 5, 7, 9 };

        // Alternative syntax
        int[] array3 = { 1, 2, 3, 4, 5, 6 };

        // Declare a two dimensional array
        int[,] multiDimensionalArray1 = new int[2, 3];

        // Declare and set array element values
        int[,] multiDimensionalArray2 = { { 1, 2, 3 }, { 4, 5, 6 } };

        // Declare a jagged array
        int[][] jaggedArray = new int[6][];

        // Set the values of the first array in the jagged array structure
        jaggedArray[0] = new int[4] { 1, 2, 3, 4 };
    }
}
```

Array Overview

An array has the following properties:

- An array can be [Single-Dimensional](#), [Multidimensional](#) or [Jagged](#).
- The default value of numeric array elements are set to zero, and reference elements are set to null.
- A jagged array is an array of arrays, and therefore its elements are reference types and are initialized to **null**.
- Arrays are zero indexed: an array with *n* elements is indexed from 0 to *n*-1.
- Array elements can be of any type, including an array type.
- Array types are [reference types](#) derived from the abstract base type [Array](#). Since this type implements [IEnumerable](#) and [IEnumerable](#), you can use [foreach](#) iteration on all arrays in C#.

Related Sections

- [Arrays as Objects \(C# Programming Guide\)](#)
- [Using foreach with Arrays \(C# Programming Guide\)](#)
- [Passing Arrays as Parameters \(C# Programming Guide\)](#)
- [Passing Arrays Using ref and out \(C# Programming Guide\)](#)
- [Arrays Sample](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.8 Arrays](#)
- [12 Arrays](#)

See Also

Reference

[Array Usage Guidelines](#)

Concepts

[C# Programming Guide](#)

[Collection Classes \(C# Programming Guide\)](#)

[Array Collection Type](#)

[Arrays in the Common Type System](#)

Arrays as Objects (C# Programming Guide)

In C#, arrays are actually objects, and not just addressable regions of contiguous memory as in C and C++. [Array](#) is the abstract base type of all array types. You can use the properties, and other class members, that **Array** has. An example of this would be using the [Length](#) property to get the length of an array. The following code assigns the length of the `numbers` array, which is 5, to a variable called `lengthOfNumbers`:

C#

```
int[] numbers = { 1, 2, 3, 4, 5 };
int lengthOfNumbers = numbers.Length;
```

The **System.Array** class provides many other useful methods and properties for sorting, searching, and copying arrays.

Example

This example uses the [Rank](#) property to display the number of dimensions of an array.

C#

```
class TestArraysClass
{
    static void Main()
    {
        // Declare and initialize an array:
        int[,] theArray = new int[5, 10];
        System.Console.WriteLine("The array has {0} dimensions.", theArray.Rank);
    }
}
```

Output

The array has 2 dimensions.

See Also

Reference

[Single-Dimensional Arrays \(C# Programming Guide\)](#)

[Multidimensional Arrays \(C# Programming Guide\)](#)

[Jagged Arrays \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

Single-Dimensional Arrays (C# Programming Guide)

You can declare an array of five integers as in the following example:

C#

```
int[] array = new int[5];
```

This array contains the elements from `array[0]` to `array[4]`. The `new` operator is used to create the array and initialize the array elements to their default values. In this example, all the array elements are initialized to zero.

An array that stores string elements can be declared in the same way. For example:

C#

```
string[] stringArray = new string[6];
```

Array Initialization

It is possible to initialize an array upon declaration, in which case, the rank specifier is not needed because it is already supplied by the number of elements in the initialization list. For example:

C#

```
int[] array1 = new int[5] { 1, 3, 5, 7, 9 };
```

A string array can be initialized in the same way. The following is a declaration of a string array where each array element is initialized by a name of a day:

C#

```
string[] weekdays = new string[] { "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat" };
```

When you initialize an array upon declaration, it is possible to use the following shortcuts:

C#

```
int[] array2 = { 1, 3, 5, 7, 9 };  
string[] weekdays2 = { "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat" };
```

It is possible to declare an array variable without initialization, but you must use the **new** operator when you assign an array to this variable. For example:

C#

```
int[] array3;  
array3 = new int[] { 1, 3, 5, 7, 9 }; // OK  
//array3 = {1, 3, 5, 7, 9}; // Error
```

Value Type and Reference Type Arrays

Consider the following array declaration:

C#

```
SomeType[] array4 = new SomeType[10];
```

The result of this statement depends on whether `SomeType` is a value type or a reference type. If it is a value type, the statement

results in creating an array of 10 instances of the type `SomeType`. If `SomeType` is a reference type, the statement creates an array of 10 elements, each of which is initialized to a null reference.

For more information on value types and reference types, see [Types \(C# Reference\)](#).

See Also

Reference

[Multidimensional Arrays \(C# Programming Guide\)](#)

[Jagged Arrays \(C# Programming Guide\)](#)

[Array](#)

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

Multidimensional Arrays (C# Programming Guide)

Arrays can have more than one dimension. For example, the following declaration creates a two-dimensional array of four rows and two columns:

C#

```
int[,] array = new int[4, 2];
```

Also, the following declaration creates an array of three dimensions, 4, 2, and 3:

C#

```
int[ , , ] array1 = new int[4, 2, 3];
```

Array Initialization

You can initialize the array upon declaration as shown in the following example:

C#

```
int[,] array2D = new int[,] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };  
int[ , , ] array3D = new int[ , , ] { { { 1, 2, 3 } }, { { 4, 5, 6 } } };
```

You can also initialize the array without specifying the rank:

C#

```
int[,] array4 = { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };
```

If you choose to declare an array variable without initialization, you must use the **new** operator to assign an array to the variable. For example:

C#

```
int[,] array5;  
array5 = new int[,] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };    // OK  
//array5 = {{1,2},{3,4},{5,6},{7,8}};    // Error
```

You can also assign a value to an array element, for example:

C#

```
array5[2, 1] = 25;
```

The following code example initializes the array variables to default (except for jagged arrays):

C#

```
int[,] array6 = new int[10, 10];
```

See Also

Reference

[Single-Dimensional Arrays \(C# Programming Guide\)](#)

[Jagged Arrays \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Jagged Arrays (C# Programming Guide)

A jagged array is an array whose elements are arrays. The elements of a jagged array can be of different dimensions and sizes. A jagged array is sometimes called an "array of arrays." The following examples show how to declare, initialize, and access jagged arrays.

The following is a declaration of a single-dimensional array that has three elements, each of which is a single-dimensional array of integers:

C#

```
int[][] jaggedArray = new int[3][];
```

Before you can use `jaggedArray`, its elements must be initialized. You can initialize the elements like this:

C#

```
jaggedArray[0] = new int[5];  
jaggedArray[1] = new int[4];  
jaggedArray[2] = new int[2];
```

Each of the elements is a single-dimensional array of integers. The first element is an array of 5 integers, the second is an array of 4 integers, and the third is an array of 2 integers.

It is also possible to use initializers to fill the array elements with values, in which case you do not need the array size. For example:

C#

```
jaggedArray[0] = new int[] { 1, 3, 5, 7, 9 };  
jaggedArray[1] = new int[] { 0, 2, 4, 6 };  
jaggedArray[2] = new int[] { 11, 22 };
```

You can also initialize the array upon declaration like this:

C#

```
int[][] jaggedArray2 = new int[][]  
{  
    new int[] {1,3,5,7,9},  
    new int[] {0,2,4,6},  
    new int[] {11,22}  
};
```

You can use the following shorthand form. Notice that you cannot omit the **new** operator from the elements initialization because there is no default initialization for the elements:

C#

```
int[][] jaggedArray3 =  
{  
    new int[] {1,3,5,7,9},  
    new int[] {0,2,4,6},  
    new int[] {11,22}  
};
```

A jagged array is an array of arrays, and therefore its elements are reference types and are initialized to **null**.

You can access individual array elements like these examples:

C#

```
// Assign 77 to the second element ([1]) of the first array ([0]):
jaggedArray3[0][1] = 77;

// Assign 88 to the second element ([1]) of the third array ([2]):
jaggedArray3[2][1] = 88;
```

It is possible to mix jagged and multidimensional arrays. The following is a declaration and initialization of a single-dimensional jagged array that contains two-dimensional array elements of different sizes:

C#

```
int[,] jaggedArray4 = new int[3][,]
{
    new int[,] { {1,3}, {5,7} },
    new int[,] { {0,2}, {4,6}, {8,10} },
    new int[,] { {11,22}, {99,88}, {0,9} }
};
```

You can access individual elements as shown in this example, which displays the value of the element [1,0] of the first array (value 5):

C#

```
System.Console.WriteLine("{0}", jaggedArray4[0][1, 0]);
```

The method `Length` returns the number of arrays contained in the jagged array. For example, assuming you have declared the previous array, this line:

C#

```
System.Console.WriteLine(jaggedArray4.Length);
```

will return a value of 3.

Example

This example builds an array whose elements are themselves arrays. Each one of the array elements has a different size.

C#

```
class ArrayTest
{
    static void Main()
    {
        // Declare the array of two elements:
        int[][] arr = new int[2][];

        // Initialize the elements:
        arr[0] = new int[5] { 1, 3, 5, 7, 9 };
        arr[1] = new int[4] { 2, 4, 6, 8 };

        // Display the array elements:
        for (int i = 0; i < arr.Length; i++)
        {
            System.Console.WriteLine("Element({0}): ", i);

            for (int j = 0; j < arr[i].Length; j++)
            {
                System.Console.Write("{0}{1}", arr[i][j], j == (arr[i].Length - 1) ? "" : "
");
            }
            System.Console.WriteLine();
        }
    }
}
```

```
}  
}  
}
```

Output

Element (0) : 1 3 5 7 9

Element (1) : 2 4 6 8

See Also

Reference

[Single-Dimensional Arrays \(C# Programming Guide\)](#)

[Multidimensional Arrays \(C# Programming Guide\)](#)

[Array](#)

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

Using foreach with Arrays (C# Programming Guide)

C# also provides the [foreach](#) statement. This statement provides a simple, clean way to iterate through the elements of an array. For example, the following code creates an array called `numbers` and iterates through it with the **foreach** statement:

C#

```
int[] numbers = { 4, 5, 6, 1, 2, 3, -2, -1, 0 };
foreach (int i in numbers)
{
    System.Console.WriteLine(i);
}
```

With multidimensional arrays, you can use the same method to iterate through the elements, for example:

C#

```
int[,] numbers2D = new int[3, 2] { { 9, 99 }, { 3, 33 }, { 5, 55 } };
foreach (int i in numbers2D)
{
    System.Console.Write("{0} ", i);
}
```

The output of this example is:

9 99 3 33 5 55

However, with multidimensional arrays, using a nested [for](#) loop gives you more control over the array elements.

See Also

Reference

[Single-Dimensional Arrays \(C# Programming Guide\)](#)

[Multidimensional Arrays \(C# Programming Guide\)](#)

[Jagged Arrays \(C# Programming Guide\)](#)

[Array](#)

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

Passing Arrays as Parameters (C# Programming Guide)

Arrays may be passed to methods as parameters. As arrays are reference types, the method can change the value of the elements.

Passing single-dimensional arrays as parameters

You can pass an initialized single-dimensional array to a method. For example:

C#

```
PrintArray(theArray);
```

The method called in the line above could be defined as:

C#

```
void PrintArray(int[] arr)
{
    // method code
}
```

You can also initialize and pass a new array in one step. For example:

C#

```
PrintArray(new int[] { 1, 3, 5, 7, 9 });
```

Example 1

In the following example, a string array is initialized and passed as a parameter to the `PrintArray` method, where its elements are displayed:

C#

```
class ArrayClass
{
    static void PrintArray(string[] arr)
    {
        for (int i = 0; i < arr.Length; i++)
        {
            System.Console.Write(arr[i] + "{0}", i < arr.Length - 1 ? " " : "");
        }
        System.Console.WriteLine();
    }

    static void Main()
    {
        // Declare and initialize an array:
        string[] weekDays = new string[] { "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat" };

        // Pass the array as a parameter:
        PrintArray(weekDays);
    }
}
```

Output 1

```
Sun Mon Tue Wed Thu Fri Sat
```

Passing multidimensional arrays as parameters

You can pass an initialized multidimensional array to a method. For example, if `theArray` is a two dimensional array:

C#

```
PrintArray(theArray);
```

The method called in the line above could be defined as:

C#

```
void PrintArray(int[,] arr)
{
    // method code
}
```

You can also initialize and pass a new array in one step. For example:

C#

```
PrintArray(new int[,] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } });
```

Example 2

In this example, a two-dimensional array is initialized and passed to the `PrintArray` method, where its elements are displayed.

C#

```
class ArrayClass2D
{
    static void PrintArray(int[,] arr)
    {
        // Display the array elements:
        for (int i = 0; i < 4; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                System.Console.WriteLine("Element({0},{1})={2}", i, j, arr[i, j]);
            }
        }
    }
    static void Main()
    {
        // Pass the array as a parameter:
        PrintArray(new int[,] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } });
    }
}
```

Output 2

Element (0,0)=1

Element (0,1)=2

Element (1,0)=3

Element (1,1)=4

Element (2,0)=5

Element (2,1)=6

Element (3,0)=7

Element (3,1)=8

See Also

Reference

[Single-Dimensional Arrays \(C# Programming Guide\)](#)

[Multidimensional Arrays \(C# Programming Guide\)](#)

[Jagged Arrays \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

Passing Arrays Using ref and out (C# Programming Guide)

Like all **out** parameters, an **out** parameter of an array type must be assigned before it is used; that is, it must be assigned by the callee. For example:

C#

```
static void TestMethod1(out int[] arr)
{
    arr = new int[10];    // definite assignment of arr
}
```

Like all **ref** parameters, a **ref** parameter of an array type must be definitely assigned by the caller. Therefore, there is no need to be definitely assigned by the callee. A **ref** parameter of an array type may be altered as a result of the call. For example, the array can be assigned the **null** value or can be initialized to a different array. For example:

C#

```
static void TestMethod2(ref int[] arr)
{
    arr = new int[10];    // arr initialized to a different array
}
```

The following two examples demonstrate the difference between **out** and **ref** when used in passing arrays to methods.

Example 1

In this example, the array `theArray` is declared in the caller (the `Main` method), and initialized in the `FillArray` method. Then, the array elements are returned to the caller and displayed.

C#

```
class TestOut
{
    static void FillArray(out int[] arr)
    {
        // Initialize the array:
        arr = new int[5] { 1, 2, 3, 4, 5 };
    }

    static void Main()
    {
        int[] theArray; // Initialization is not required

        // Pass the array to the callee using out:
        FillArray(out theArray);

        // Display the array elements:
        System.Console.WriteLine("Array elements are:");
        for (int i = 0; i < theArray.Length; i++)
        {
            System.Console.Write(theArray[i] + " ");
        }
    }
}
```

Output 1

Array elements are:

1 2 3 4 5

Example 2

In this example, the array `theArray` is initialized in the caller (the `Main` method), and passed to the `FillArray` method by using the **ref** parameter. Some of the array elements are updated in the `FillArray` method. Then, the array elements are returned to the caller and displayed.

C#

```
class TestRef
{
    static void FillArray(ref int[] arr)
    {
        // Create the array on demand:
        if (arr == null)
        {
            arr = new int[10];
        }
        // Fill the array:
        arr[0] = 1111;
        arr[4] = 5555;
    }

    static void Main()
    {
        // Initialize the array:
        int[] theArray = { 1, 2, 3, 4, 5 };

        // Pass the array using ref:
        FillArray(ref theArray);

        // Display the updated array:
        System.Console.WriteLine("Array elements are:");
        for (int i = 0; i < theArray.Length; i++)
        {
            System.Console.Write(theArray[i] + " ");
        }
    }
}
```

Output 2

Array elements are:

1111 2 3 4 5555

See Also

Reference

[Single-Dimensional Arrays \(C# Programming Guide\)](#)

[Multidimensional Arrays \(C# Programming Guide\)](#)

[Jagged Arrays \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

Strings (C# Programming Guide)

The following sections discuss the **string** data type, which is an alias for the [String](#) class.

In This Section

[Using Strings \(C# Programming Guide\)](#)

[How to: Parse Strings Using the Split Method \(C# Programming Guide\)](#)

[How to: Search Strings Using String Methods \(C# Programming Guide\)](#)

[How to: Search Strings Using Regular Expressions \(C# Programming Guide\)](#)

[How to: Join Multiple Strings \(C# Programming Guide\)](#)

[How to: Modify String Contents \(C# Programming Guide\)](#)

Related Sections

[Basic String Operations](#)

[Comparing Strings](#)

See Also

Concepts

[C# Programming Guide](#)

Using Strings (C# Programming Guide)

A C# string is an array of characters declared using the **string** keyword. A string literal is declared using quotation marks, as shown in the following example:

C#

```
string s = "Hello, World!";
```

You can extract substrings, and concatenate strings, like this:

C#

```
string s1 = "orange";
string s2 = "red";

s1 += s2;
System.Console.WriteLine(s1); // outputs "orangered"

s1 = s1.Substring(2, 5);
System.Console.WriteLine(s1); // outputs "anger"
```

String objects are *immutable*, meaning that they cannot be changed once they have been created. Methods that act on strings actually return new string objects. In the previous example, when the contents of `s1` and `s2` are concatenated to form a single string, the two strings containing "orange" and "red" are both unmodified. The `+=` operator creates a new string that contains the combined contents. The result is that `s1` now refers to a different string altogether. A string containing just "orange" still exists, but is no longer referenced when `s1` is concatenated.

Note

Use caution when creating references to strings. If you create a reference to a string, and then "modify" the string, the reference will continue to point to the original object, not the new object that was created when the string was modified. The following code illustrates the danger:

```
string s1 = "Hello";
string s2 = s1;
s1 += " and goodbye.";
Console.WriteLine(s2); //outputs "Hello"
```

Because modifications to strings involve the creation of new string objects, for performance reasons, large amounts of concatenation or other involved string manipulation should be performed with the [StringBuilder](#) class, like this:

C#

```
System.Text.StringBuilder sb = new System.Text.StringBuilder();
sb.Append("one ");
sb.Append("two ");
sb.Append("three");
string str = sb.ToString();
```

The **StringBuilder** class is discussed further in the "Using Stringbuilder" section.

Working with Strings

Escape Characters

Escape characters such as `"\n"` (new line) and `"\t"` (tab) can be included in strings. The line:

C#

```
string hello = "Hello\nWorld!";
```


is the same as:

Hello

World!

If you want to include a backward slash, it must be preceded with another backward slash. The following string:

C#

```
string filePath = "\\My Documents\\";
```

is actually the same as:

\\My Documents\

The @ Symbol

The @ symbol tells the string constructor to ignore escape characters and line breaks. The following two strings are therefore identical:

C#

```
string p1 = "\\My Documents\\My Files\\";
string p2 = @"\\My Documents\\My Files\";
```

ToString()

Like all objects derived from [Object](#), strings provide the [ToString](#) method, which converts a value to a string. This method can be used to convert numeric values into strings, like this:

C#

```
int year = 1999;
string msg = "Eve was born in " + year.ToString();
System.Console.WriteLine(msg); // outputs "Eve was born in 1999"
```

Accessing Individual Characters

Individual characters contained in a string can be accessed using methods such as **SubString()**, **Replace()**, **Split()** and **Trim()**.

C#

```
string s3 = "Visual C# Express";

System.Console.WriteLine(s3.Substring(7, 2)); // outputs "C#"
System.Console.WriteLine(s3.Replace("C#", "Basic")); // outputs "Visual Basic Express"
```

It is also possible to copy the characters into a character array, like this:

C#

```
string s4 = "Hello, World";
char[] arr = s4.ToCharArray(0, s4.Length);

foreach (char c in arr)
{
    System.Console.Write(c); // outputs "Hello, World"
}
```

Individual characters from a string can be accessed with an index, like this:

C#

```
string s5 = "Printing backwards";

for (int i = 0; i < s5.Length; i++)
{
    System.Console.Write(s5[s5.Length - i - 1]); // outputs "sdrawkcab gnitnirP"
}
```

Changing Case

To change the letters in a string to upper or lower case, use **ToUpper()** or **ToLower()**, like this:

C#

```
string s6 = "Battle of Hastings, 1066";

System.Console.WriteLine(s6.ToUpper()); // outputs "BATTLE OF HASTINGS 1066"
System.Console.WriteLine(s6.ToLower()); // outputs "battle of hastings 1066"
```

Comparisons

The simplest way to compare two strings is to use the **==** and **!=** operators, which perform a case-sensitive comparison.

C#

```
string color1 = "red";
string color2 = "green";
string color3 = "red";

if (color1 == color3)
{
    System.Console.WriteLine("Equal");
}
if (color1 != color2)
{
    System.Console.WriteLine("Not equal");
}
```

String objects also have a **CompareTo()** method that returns an integer value based on whether one string is less-than (<) or greater-than (>) another. When comparing strings, the Unicode value is used, and lower case has a smaller value than upper case.

C#

```
string s7 = "ABC";
string s8 = "abc";

if (s7.CompareTo(s8) > 0)
{
    System.Console.WriteLine("Greater-than");
}
else
{
    System.Console.WriteLine("Less-than");
}
```

To search for a string inside another string, use **IndexOf()**. **IndexOf()** returns -1 if the search string is not found; otherwise, it returns the zero-based index of the first location at which it occurs.

C#

```
string s9 = "Battle of Hastings, 1066";

System.Console.WriteLine(s9.IndexOf("Hastings")); // outputs 10
System.Console.WriteLine(s9.IndexOf("1967")); // outputs -1
```

Splitting a String into Substrings

Splitting a string into substrings—such as splitting a sentence into individual words—is a common programming task. The **Split()** method takes a **char** array of delimiters, for example, a space character, and returns an array of substrings. You can access this array with **foreach**, like this:

C#

```
char[] delimit = new char[] { ' ' };
string s10 = "The cat sat on the mat.";
foreach (string substr in s10.Split(delimit))
{
    System.Console.WriteLine(substr);
}
```

This code outputs each word on a separate line, like this:

The

cat

sat

on

the

mat.

Null Strings and Empty Strings

An empty string is an instance of a [System.String](#) object that contains zero characters. Empty strings are used quite commonly in various programming scenarios to represent a blank text field. You can call methods on empty strings because they are valid **System.String** objects. Empty strings are initialized like this:

```
string s = "";
```

By contrast, a null string does not refer to an instance of a **System.String** object and any attempt to call a method on a null string results in a [NullReferenceException](#). However, you can use null strings in concatenation and comparison operations with other strings. The following examples illustrate some cases in which a reference to a null string does and does not cause an exception to be thrown:

```
string str = "hello";
string nullStr = null;
string emptyStr = "";

string tempStr = str + nullStr; // tempStr = "hello"
bool b = (emptyStr == nullStr); // b = false;
emptyStr + nullStr = ""; // creates a new empty string
int I = nullStr.Length; // throws NullReferenceException
```

Using StringBuilder

The **StringBuilder** class creates a string buffer that offers better performance if your program performs a lot of string manipulation. The **StringBuilder** string also allows you to reassign individual characters, something the built-in string data type does not support. This code, for example, changes the content of a string without creating a new string:

C#

```
System.Text.StringBuilder sb = new System.Text.StringBuilder("Rat: the ideal pet");
sb[0] = 'C';
System.Console.WriteLine(sb.ToString());
System.Console.ReadLine();
```

In this example, a **StringBuilder** object is used to create a string from a set of numeric types:

```
C#  
  
class TestStringBuilder  
{  
    static void Main()  
    {  
        System.Text.StringBuilder sb = new System.Text.StringBuilder();  
  
        // Create a string composed of numbers 0 - 9  
        for (int i = 0; i < 10; i++)  
        {  
            sb.Append(i.ToString());  
        }  
        System.Console.WriteLine(sb); // displays 0123456789  
  
        // Copy one character of the string (not possible with a System.String)  
        sb[0] = sb[9];  
  
        System.Console.WriteLine(sb); // displays 9123456789  
    }  
}
```

For More Information

From the C# Programmer's Reference:

- [string \(C# Reference\)](#)

See Also

Tasks

[How to: Search Strings Using Regular Expressions \(C# Programming Guide\)](#)

[How to: Search Strings Using String Methods \(C# Programming Guide\)](#)

[How to: Parse Strings Using the Split Method \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Parsing Strings](#)

How to: Parse Strings Using the Split Method (C# Programming Guide)

The following code example demonstrates how a string can be parsed using the [System.String.Split](#) method. This method works by returning an array of strings, where each element is a word. As input, [Split](#) takes an array of chars that indicate which characters are to be used as delimiters. In this example, spaces, commas, periods, colons, and tabs are used. An array containing these delimiters is passed to **Split**, and each word in the sentence is displayed separately using the resulting array of strings.

Example

C#

```
class TestStringSplit
{
    static void Main()
    {
        char[] delimiterChars = { ' ', ',', '.', ':', '\t' };

        string text = "one\ttwo three:four,five six seven";
        System.Console.WriteLine("Original text: '{0}'", text);

        string[] words = text.Split(delimiterChars);
        System.Console.WriteLine("{0} words in text:", words.Length);

        foreach (string s in words)
        {
            System.Console.WriteLine(s);
        }
    }
}
```

Output

```
Original text: 'one      two three:four,five six seven'
7 words in text:
one
two
three
four
five
six
seven
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Strings \(C# Programming Guide\)](#)

[.NET Framework Regular Expressions](#)

[.NET Programming in C++](#)

How to: Search Strings Using String Methods (C# Programming Guide)

The [string](#) type, which is an alias for the [System.String](#) class, provides a number of useful methods for searching the contents of a string. The following example uses the [IndexOf](#), [LastIndexOf](#), [StartsWith](#), and [EndsWith](#) methods.

Example

C#

```
class StringSearch
{
    static void Main()
    {
        string str = "A silly sentence used for silly purposes.";
        System.Console.WriteLine("{0}", str);

        bool test1 = str.StartsWith("a silly");
        System.Console.WriteLine("starts with 'a silly'? {0}", test1);

        bool test2 = str.StartsWith("a silly", System.StringComparison.OrdinalIgnoreCase);
        System.Console.WriteLine("starts with 'a silly'? {0} (ignoring case)", test2);

        bool test3 = str.EndsWith(".");
        System.Console.WriteLine("ends with '.'? {0}", test3);

        int first = str.IndexOf("silly");
        int last = str.LastIndexOf("silly");
        string str2 = str.Substring(first, last - first);
        System.Console.WriteLine("between two 'silly' words: '{0}'", str2);
    }
}
```

Output

```
'A silly sentence used for silly purposes.'
starts with 'a silly'? False
starts with 'a silly'? True (ignore case)
ends with '.'? True
between two 'silly' words: 'silly sentence used for '
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Strings \(C# Programming Guide\)](#)

How to: Search Strings Using Regular Expressions (C# Programming Guide)

The [System.Text.RegularExpressions.Regex](#) class can be used to search strings. These searches can range in complexity from very simple to making full use of regular expressions. The following are two examples of string searching using the **Regex** class. For more information, see [.NET Framework Regular Expressions](#).

Example

The following code is a console application that performs a simple case insensitive search of the strings in an array. The static method

[System.Text.RegularExpressions.Regex.IsMatch\(System.String, System.String, System.Text.RegularExpressions.RegexOptions\)](#) performs the search given the string to search and a string containing the search pattern. In this case, a third argument is used to indicate that case should be ignored. For more information, see [System.Text.RegularExpressions.RegexOptions](#).

C#

```
class TestRegularExpressions
{
    static void Main()
    {
        string[] sentences =
        {
            "cow over the moon",
            "Betsy the Cow",
            "cowering in the corner",
            "no match here"
        };

        string sPattern = "cow";

        foreach (string s in sentences)
        {
            System.Console.Write("{0,24}", s);

            if (System.Text.RegularExpressions.Regex.IsMatch(s, sPattern, System.Text.Regul
arExpressions.RegexOptions.IgnoreCase))
            {
                System.Console.WriteLine(" (match for '{0}' found)", sPattern);
            }
            else
            {
                System.Console.WriteLine();
            }
        }
    }
}
```

Output

cow over the moon	(match for 'cow' found)
Betsy the Cow	(match for 'cow' found)
cowering in the corner	(match for 'cow' found)
no match here	

The following code is a console application that uses regular expressions to validate the format of each string in an array. The validation requires that each string take the form of a telephone number in which three groups of digits are separated by dashes, the first two groups contain three digits, and the third group contains four digits. This is accomplished with the regular expression `^\d{3}-\d{3}-\d{4}$`. For more information, see [Regular Expression Language Elements](#).

C#

```
class TestRegularExpressionValidation
{
```

```

static void Main()
{
    string[] numbers =
    {
        "123-456-7890",
        "444-234-22450",
        "690-203-6578",
        "146-893-232",
        "146-839-2322",
        "4007-295-1111",
        "407-295-1111",
        "407-2-5555",
    };

    string sPattern = "^\\d{3}-\\d{3}-\\d{4}$";

    foreach (string s in numbers)
    {
        System.Console.WriteLine("{0,14}", s);

        if (System.Text.RegularExpressions.Regex.IsMatch(s, sPattern))
        {
            System.Console.WriteLine(" - valid");
        }
        else
        {
            System.Console.WriteLine(" - invalid");
        }
    }
}

```

Output

123-456-7890 - valid
444-234-22450 - invalid
690-203-6578 - valid
146-893-232 - invalid
146-839-2322 - valid
4007-295-1111 - invalid
407-295-1111 - valid
407-2-5555 - invalid

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Strings \(C# Programming Guide\)](#)

How to: Join Multiple Strings (C# Programming Guide)

There are two ways to join multiple strings: using the `+` operator that the [String](#) class overloads, and using the [StringBuilder](#) class. The `+` operator is easy to use and makes for intuitive code, but it works in series; a new string is created for each use of the operator, so chaining multiple operators together is inefficient. For example:

C#

```
string two = "two";
string str = "one " + two + " three";
System.Console.WriteLine(str);
```

Although four strings appear in the code, the three strings being joined and the final string containing all three, five strings are created in total because the first two strings are joined first, creating a string containing "one two." The third is appended separately, forming the final string stored in `str`.

Alternatively, the **StringBuilder** class can be used to add each string to an object that then creates the final string in one step. This strategy is demonstrated in the following example.

Example

The following code uses the `Append` method of the **StringBuilder** class to join three strings without the chaining effect of the `+` operator.

C#

```
class StringBuilderTest
{
    static void Main()
    {
        string two = "two";

        System.Text.StringBuilder sb = new System.Text.StringBuilder();
        sb.Append("one ");
        sb.Append(two);
        sb.Append(" three");
        System.Console.WriteLine(sb.ToString());

        string str = sb.ToString();
        System.Console.WriteLine(str);
    }
}
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Strings \(C# Programming Guide\)](#)

How to: Modify String Contents (C# Programming Guide)

Strings are *immutable*, so it is not possible to modify the contents of string. The contents of a string can, however, be extracted into a non-immutable form, modified, and then formed into a new string instance.

Example

The following example uses the [ToCharArray](#) method to extract the contents of a string into an array of the [char](#) type. Some of the elements of this array are then modified. The **char** array is then used to create a new string instance.

C#

```
class ModifyStrings
{
    static void Main()
    {
        string str = "The quick brown fox jumped over the fence";
        System.Console.WriteLine(str);

        char[] chars = str.ToCharArray();
        int animalIndex = str.IndexOf("fox");
        if (animalIndex != -1)
        {
            chars[animalIndex++] = 'c';
            chars[animalIndex++] = 'a';
            chars[animalIndex] = 't';
        }

        string str2 = new string(chars);
        System.Console.WriteLine(str2);
    }
}
```

Output

The quick brown fox jumped over the fence
The quick brown cat jumped over the fence

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Strings \(C# Programming Guide\)](#)

Statements, Expressions, and Operators (C# Programming Guide)

This section contains information regarding the fundamental elements that make up a C# program. The C# code that comprises an application consists of statements comprising C# keywords, expressions and operators.

For more information, see:

- [Statements \(C# Programming Guide\)](#)
- [Expressions \(C# Programming Guide\)](#)
- [Operators \(C# Programming Guide\)](#)
- [Overloadable Operators \(C# Programming Guide\)](#)
- [Conversion Operators \(C# Programming Guide\)](#)
 - [Using Conversion Operators \(C# Programming Guide\)](#)
 - [How to: Implement User-Defined Conversions Between Structs \(C# Programming Guide\)](#)
- [How to: Use Operator Overloading to Create a Complex Number Class \(C# Programming Guide\)](#)
- [Guidelines for Overloading Equals\(\) and Operator == \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.4 Expressions
- 1.5 Statements
- 1.6.6.5 Operators
- 5.3.3 Precise rules for determining definite assignment
- 7 Expressions
- 7.2 Operators
- 8 Statements

See Also

Reference

[Casting \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Statements (C# Programming Guide)

A statement is a procedural building-block from which all C# programs are constructed. A statement can declare a local variable or constant, call a method, create an object, or assign a value to a variable, property, or field. A control statement can create a loop, such as a [for](#) loop, or make a decision and branch to a new block of code, such as an [if](#) or [switch](#) statement. Statements are usually terminated by a semicolon. For more information, see [Statement Types \(C# Reference\)](#).

A series of statements surrounded by curly braces form a block of code. A method body is one example of a code block. Code blocks often follow a control statement. Variables or constants declared within a code block are only available to statements within the same code block. For example, the following code shows a method block and a code block following a control statement:

C#

```
bool IsPositive(int number)
{
    if (number > 0)
    {
        return true;
    }
    else
    {
        return false;
    }
}
```

Statements in C# often contain expressions. An expression in C# is a fragment of code containing a literal value, a simple name, or an operator and its operands. Most common expressions, when evaluated, yield a literal value, a variable, or an object property or object indexer access. Whenever a variable, object property or object indexer access is identified from an expression, the value of that item is used as the value of the expression. In C#, an expression can be placed anywhere that a value or object is required as long as the expression ultimately evaluates to the required type.

Some expressions evaluate to a namespace, a type, a method group, or an event access. These special-purpose expressions are only valid at certain times, usually as part of a larger expression, and will result in a compiler error when used improperly.

Related Sections

- [Statement Types \(C# Reference\)](#)
- [Expressions \(C# Programming Guide\)](#)
- [Operators \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.5 Statements
- 7 Expressions
- 8 Statements

See Also

Concepts

[C# Programming Guide](#)

Expressions (C# Programming Guide)

An expression is a fragment of code that can be evaluated to a single value, object, method, or namespace. Expressions can contain a literal value, a method invocation, an operator and its operands, or a *simple name*. Simple names can be the name of a variable, type member, method parameter, namespace or type.

Expressions can use operators that in turn use other expressions as parameters, or method calls whose parameters are in turn other method calls, so expressions can range from simple to very complex.

Literals and Simple Names

The two simplest types of expressions are literals and simple names. A literal is a constant value that has no name. For example, in the following code example, both `5` and `"Hello World"` are literal values:

C#

```
int i = 5;
string s = "Hello World";
```

For more information on literals, see [Types \(C# Reference\)](#).

In the example above, both `i` and `s` are simple names identifying local variables. When those variables are used in an expression, the value of the variable is retrieved and used for the expression. For example, in the following code example, when `DoWork` is called, the method receives the value 5 by default and is not able to access the variable `var`:

C#

```
int var = 5;
DoWork(var);
```

Invocation Expressions

In the following code example, the call to `DoWork` is another kind of expression, called an invocation expression.

C#

```
DoWork(var);
```

Specifically, calling a method is a method invocation expression. A method invocation requires the name of the method, either as a name as in the previous example, or as the result of another expression, followed by parenthesis and any method parameters. For more information, see [Methods \(C# Programming Guide\)](#). A delegate invocation uses the name of a delegate and method parameters in parenthesis. For more information, see [Delegates \(C# Programming Guide\)](#). Method invocations and delegate invocations evaluate to the return value of the method, if the method returns a value. Methods that return void cannot be used in place of a value in an expression.

Remarks

Whenever a variable, object property, or object indexer access is identified from an expression, the value of that item is used as the value of the expression. An expression can be placed anywhere in C# where a value or object is required, as long as the expression ultimately evaluates to the required type.

See Also

Reference

[Methods \(C# Programming Guide\)](#)

[Operators \(C# Programming Guide\)](#)

[Data Types \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Delegates \(C# Programming Guide\)](#)

Operators (C# Programming Guide)

In C#, an operator is a term or a symbol that takes one or more expressions, called operands, as input and returns a value. Operators that take one operand, such as the increment operator (`++`) or **new**, are called unary operators. Operators that take two operands, such as arithmetic operators (`+`, `-`, `*`, `/`) are called binary operators. One operator, the conditional operator (`?:`), takes three operands and is the sole tertiary operator in C#.

The following C# statement contains a single unary operator, and a single operand. The increment operator, `++`, modifies the value of the operand `y`:

```
C#  
  
y++;
```

The following C# statement contains two binary operators, each with two operands. The assignment operator, `=`, has the integer `y`, and the expression `2 + 3` as operands. The expression `2 + 3` itself contains the addition operator, and uses the integer values `2` and `3` as operands:

```
C#  
  
y = 2 + 3;
```

An operand can be a valid expression of any size, composed of any number of other operations.

Operators in an expression are evaluated in a specific order known as operator precedence. The following table divides the operators into categories based on the type of operation they perform. The categories are listed in order of precedence.

Primary	<code>x.y</code> , <code>f(x)</code> , <code>a[x]</code> , <code>x++</code> , <code>x--</code> , new , typeof , checked , unchecked
Unary	<code>+</code> , <code>-</code> , <code>!</code> , <code>~</code> , <code>++x</code> , <code>--x</code> , <code>(T)x</code>
Arithmetic — Multiplicative	<code>*</code> , <code>/</code> , <code>%</code>
Arithmetic — Additive	<code>+</code> , <code>-</code>
Shift	<code><<</code> , <code>>></code>
Relational and type testing	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , is , as
Equality	<code>==</code> , <code>!=</code>
Logical, in order of precedence	& , ^ ,
Conditional, in order of precedence	&& , , ?:
Assignment	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , &= , = , <code>^=</code> , <code><<=</code> , <code>>>=</code>

When two operators with the same precedence are present in an expression, they are evaluated based on associativity. Left-associative operators are evaluated in order from left to right. For example, `x * y / z` is evaluated as `(x * y) / z`. Right-associative operators are evaluated in order from right to left. The assignment operators and the tertiary operator (`?:`) are right-associative. All other binary operators are left-associative. However, C# standard does not specify when, in an expression, the "set" portion of an increment instruction is executed. For example, the output of the following example code is 6:

```
C#  
  
int num1 = 5;  
num1++;  
System.Console.WriteLine(num1);
```

However, the output of the following example code is undefined:

C#

```
int num2 = 5;
num2 = num2++; //not recommended
System.Console.WriteLine(num2);
```

Therefore, the latter example is not recommended. Parentheses can be used to surround an expression and force that expression to be evaluated before any others. For example, $2 + 3 * 2$ would normally become 8. This is because multiplicative operators take precedence over additive operators. Writing the expression as $(2 + 3) * 2$ results in 10, because it indicates to the C# compiler that the addition operator (+) must be evaluated before the multiplication operator (*).

You can change the behavior of operators for custom classes and structs. This process is called operator overloading. For more information, see [Overloadable Operators \(C# Programming Guide\)](#).

Related Sections

For more information, see [Operator Keywords \(C# Reference\)](#) and [C# Operators](#).

See Also

Reference

[Statements, Expressions, and Operators \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Overloadable Operators (C# Programming Guide)

C# allows user-defined types to overload operators by defining static member functions using the [operator](#) keyword. Not all operators can be overloaded, however, and others have restrictions, as listed in this table:

Operators	Overloadability
+ , - , ! , ~ , ++ , -- , true , false	These unary operators can be overloaded.
+ , - , * , / , % , & , , ^ , << , >>	These binary operators can be overloaded.
== , != , < , > , <= , >=	The comparison operators can be overloaded (but see the note that follows this table).
&& , 	The conditional logical operators cannot be overloaded, but they are evaluated using & and , which can be overloaded.
[]	The array indexing operator cannot be overloaded, but you can define indexers.
()	The cast operator cannot be overloaded, but you can define new conversion operators (see explicit and implicit).
+= , -= , *= , /= , %= , &= , = , ^= , <<= , >>=	Assignment operators cannot be overloaded, but += , for example, is evaluated using + , which can be overloaded.
= , , , ?: , -> , new , is , sizeof , typeof	These operators cannot be overloaded.
Note	
The comparison operators, if overloaded, must be overloaded in pairs; that is, if == is overloaded, != must also be overloaded. The reverse is also true, and similar for < and > , and for <= and >= .	

To overload an operator on a custom class requires creating a method on the class with the correct signature. The method must be named "operator X" where X is the name or symbol of the operator being overloaded. Unary operators have one parameter, and binary operators have two parameters. In each case, one parameter must be the same type as the class or struct that declares the operator, as demonstrated in the following example:

```
C#  
  
public static Complex operator +(Complex c1, Complex c2)
```

For more information, see [How to: Use Operator Overloading to Create a Complex Number Class \(C# Programming Guide\)](#).

- See Also
- Reference**
- [Statements, Expressions, and Operators \(C# Programming Guide\)](#)
 - [Operators \(C# Programming Guide\)](#)
 - [C# Operators](#)
- Concepts**
- [C# Programming Guide](#)

Conversion Operators (C# Programming Guide)

C# enables programmers to declare conversions on classes or structs so that classes or structs can be converted to and/or from other classes or structs, or basic types. Conversions are defined like operators and are named for the type to which they convert. Either the type argument to be converted, or the type of the result of the conversion, but not both, must be the containing type.

C#

```
class SampleClass
{
    public static explicit operator SampleClass(int i)
    {
        SampleClass temp = new SampleClass();
        // code to convert from int to SampleClass...

        return temp;
    }
}
```

Conversion Operators Overview

Conversion operators have the following properties:

- Conversions declared as **implicit** occur automatically when required.
- Conversions declared as **explicit** require a cast to be called.
- All conversions must be **static**.

Related Sections

For more information:

- [Using Conversion Operators \(C# Programming Guide\)](#)
- [Conversion Operator Design Guidelines](#)
- [How to: Implement User-Defined Conversions Between Structs \(C# Programming Guide\)](#)
- [explicit \(C# Reference\)](#)
- [implicit \(C# Reference\)](#)
- [static \(C# Reference\)](#)

See Also

Concepts

[C# Programming Guide](#)

Using Conversion Operators (C# Programming Guide)

Conversion operators can be **explicit** or **implicit**. Implicit conversion operators are easier to use, but explicit operators are useful when you want users of the operator to be aware that a conversion is taking place. This topic demonstrates both types.

Example 1

This is an example of an explicit conversion operator. This operator converts from the type `Byte` to a value type called `Digit`. Because not all bytes can be converted to a digit, the conversion is explicit, meaning that a cast must be used, as shown in the `Main` method.

C#

```
struct Digit
{
    byte value;

    public Digit(byte value) //constructor
    {
        if (value > 9)
        {
            throw new System.ArgumentException();
        }
        this.value = value;
    }

    public static explicit operator Digit(byte b) // explicit byte to digit conversion operator
    {
        Digit d = new Digit(b); // explicit conversion

        System.Console.WriteLine("conversion occurred");
        return d;
    }
}

class TestExplicitConversion
{
    static void Main()
    {
        try
        {
            byte b = 3;
            Digit d = (Digit)b; // explicit conversion
        }
        catch (System.Exception e)
        {
            System.Console.WriteLine("{0} Exception caught.", e);
        }
    }
}
```

Output 1

```
conversion occurred
```

Example 2

This example demonstrates an implicit conversion operator by defining a conversion operator that undoes what the previous example did: it converts from a value class called `Digit` to the integral **Byte** type. Because any digit can be converted to a **Byte**, there's no need to force users to be explicit about the conversion.

C#

```
struct Digit
{
```

```

byte value;

public Digit(byte value) //constructor
{
    if (value > 9)
    {
        throw new System.ArgumentException();
    }
    this.value = value;
}

public static implicit operator byte(Digit d) // implicit digit to byte conversion operator
{
    System.Console.WriteLine("conversion occurred");
    return d.value; // implicit conversion
}

class TestImplicitConversion
{
    static void Main()
    {
        Digit d = new Digit(3);
        byte b = d; // implicit conversion -- no cast needed
    }
}

```

Output 2

conversion occurred

See Also

Concepts

[C# Programming Guide](#)

[Conversion Operators \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

How to: Implement User-Defined Conversions Between Structs (C# Programming Guide)

This example defines two structs, `RomanNumeral` and `BinaryNumeral`, and demonstrates conversions between them.

Example

C#

```
struct RomanNumeral
{
    private int value;

    public RomanNumeral(int value) //constructor
    {
        this.value = value;
    }

    static public implicit operator RomanNumeral(int value)
    {
        return new RomanNumeral(value);
    }

    static public implicit operator RomanNumeral(BinaryNumeral binary)
    {
        return new RomanNumeral((int)binary);
    }

    static public explicit operator int(RomanNumeral roman)
    {
        return roman.value;
    }

    static public implicit operator string(RomanNumeral roman)
    {
        return ("Conversion not yet implemented");
    }
}

struct BinaryNumeral
{
    private int value;

    public BinaryNumeral(int value) //constructor
    {
        this.value = value;
    }

    static public implicit operator BinaryNumeral(int value)
    {
        return new BinaryNumeral(value);
    }

    static public explicit operator int(BinaryNumeral binary)
    {
        return (binary.value);
    }

    static public implicit operator string(BinaryNumeral binary)
    {
        return ("Conversion not yet implemented");
    }
}

class TestConversions
{

```

```

static void Main()
{
    RomanNumeral roman;
    BinaryNumeral binary;

    roman = 10;

    // Perform a conversion from a RomanNumeral to a BinaryNumeral:
    binary = (BinaryNumeral)(int)roman;

    // Perform a conversion from a BinaryNumeral to a RomanNumeral:
    // No cast is required:
    roman = binary;

    System.Console.WriteLine((int)binary);
    System.Console.WriteLine(binary);
}

```

Output

10
Conversion not yet implemented

Robust Programming

- In the previous example, the statement:

C#

```
binary = (BinaryNumeral)(int)roman;
```

performs a conversion from a `RomanNumeral` to a `BinaryNumeral`. Because there is no direct conversion from `RomanNumeral` to `BinaryNumeral`, a cast is used to convert from a `RomanNumeral` to an **int**, and another cast to convert from an **int** to a `BinaryNumeral`.

- Also the statement

C#

```
roman = binary;
```

performs a conversion from a `BinaryNumeral` to a `RomanNumeral`. Because `RomanNumeral` defines an implicit conversion from `BinaryNumeral`, no cast is required.

See Also

Concepts

[C# Programming Guide](#)

[Conversion Operators \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

How to: Use Operator Overloading to Create a Complex Number Class (C# Programming Guide)

This example shows how you can use operator overloading to create a complex number class `Complex` that defines complex addition. The program displays the imaginary and the real parts of the numbers and the addition result using an override of the **`ToString`** method.

Example

C#

```
public struct Complex
{
    public int real;
    public int imaginary;

    public Complex(int real, int imaginary) //constructor
    {
        this.real = real;
        this.imaginary = imaginary;
    }

    // Declare which operator to overload (+),
    // the types that can be added (two Complex objects),
    // and the return type (Complex):
    public static Complex operator +(Complex c1, Complex c2)
    {
        return new Complex(c1.real + c2.real, c1.imaginary + c2.imaginary);
    }

    // Override the ToString() method to display a complex number in the traditional format
    :
    public override string ToString()
    {
        return (System.String.Format("{0} + {1}i", real, imaginary));
    }
}

class TestComplex
{
    static void Main()
    {
        Complex num1 = new Complex(2, 3);
        Complex num2 = new Complex(3, 4);

        // Add two Complex objects through the overloaded plus operator:
        Complex sum = num1 + num2;

        // Print the numbers and the sum using the overridden ToString method:
        System.Console.WriteLine("First complex number: {0}", num1);
        System.Console.WriteLine("Second complex number: {0}", num2);
        System.Console.WriteLine("The sum of the two numbers: {0}", sum);
    }
}
```

Output

First complex number: 2 + 3i Second complex number: 3 + 4i The sum of the two numbers: 5 + 7i

See Also

Reference

[C# Operators](#)
[operator \(C# Reference\)](#)
Concepts
[C# Programming Guide](#)

Guidelines for Overloading Equals() and Operator == (C# Programming Guide)

In C#, there are two different kinds of equality: reference equality and value equality. Value equality is the commonly understood meaning of equality: it means that two objects contain the same values. For example, two integers with the value of 2 have value equality. Reference equality means that there are not two objects to compare. Instead, there are two object references, both of which refer to the same object. This can occur through simple assignment, as shown in the following example:

C#

```
System.Object a = new System.Object();
System.Object b = a;
System.Object.ReferenceEquals(a, b); //returns true
```

In this code, only one object exists, but there are multiple references to that object: `a` and `b`. Because they both refer to the same object, they have reference equality. If two objects have reference equality, then they also have value equality, but value equality does not guarantee reference equality.

To check for reference equality, use [ReferenceEquals](#). To check for value equality, use [Equals](#) or [Equals](#).

Overriding Equals

Equals is a virtual method, enabling any class to override its implementation. Any class that represents a value, essentially any value type, or a set of values as a group, such as a complex number class, should override **Equals**. If the type implements [IComparable](#), it should override **Equals**.

The new implementation of **Equals** should follow all the guarantees of [Equals](#):

- `x.Equals(x)` returns true.
- `x.Equals(y)` returns the same value as `y.Equals(x)`.
- if `(x.Equals(y) && y.Equals(z))` returns true, then `x.Equals(z)` returns true.
- Successive invocations of `x.Equals(y)` return the same value as long as the objects referenced by `x` and `y` are not modified.
- `x.Equals(null)` returns false.

The new implementation of **Equals** should not throw exceptions. It is recommended that any class that overrides **Equals** also override [System.Object.GetHashCode](#). It is also recommended that in addition to implementing **Equals(object)**, any class also implement **Equals(type)** for their own type, to enhance performance. For example:

C#

```
class TwoDPoint : System.Object
{
    public readonly int x, y;

    public TwoDPoint(int x, int y) //constructor
    {
        this.x = x;
        this.y = y;
    }

    public override bool Equals(System.Object obj)
    {
        // If parameter is null return false.
        if (obj == null)
        {
            return false;
        }

        // If parameter cannot be cast to Point return false.
```



```

        TwoDPoint p = obj as TwoDPoint;
        if ((System.Object)p == null)
        {
            return false;
        }

        // Return true if the fields match:
        return (x == p.x) && (y == p.y);
    }

    public bool Equals(TwoDPoint p)
    {
        // If parameter is null return false:
        if ((object)p == null)
        {
            return false;
        }

        // Return true if the fields match:
        return (x == p.x) && (y == p.y);
    }

    public override int GetHashCode()
    {
        return x ^ y;
    }
}

```

Any derived class that can call **Equals** on the base class should do so before finishing its comparison. In the following example, **Equals** calls the base class **Equals**, which checks for a null parameter and compares the type of the parameter with the type of the derived class. That leaves the implementation of **Equals** on the derived class the task of checking the new data field declared on the derived class:

C#

```

class ThreeDPoint : TwoDPoint
{
    public readonly int z;

    public ThreeDPoint(int x, int y, int z)
        : base(x, y)
    {
        this.z = z;
    }

    public override bool Equals(System.Object obj)
    {
        // If parameter cannot be cast to ThreeDPoint return false:
        ThreeDPoint p = obj as ThreeDPoint;
        if ((object)p == null)
        {
            return false;
        }

        // Return true if the fields match:
        return base.Equals(obj) && z == p.z;
    }

    public bool Equals(ThreeDPoint p)
    {
        // Return true if the fields match:
        return base.Equals((TwoDPoint)p) && z == p.z;
    }

    public override int GetHashCode()
    {
        return base.GetHashCode() ^ z;
    }
}

```

```
}  
}
```

Overriding Operator ==

By default, the operator `==` tests for reference equality by determining if two references indicate the same object, so reference types do not need to implement operator `==` in order to gain this functionality. When a type is immutable, meaning the data contained in the instance cannot be changed, overloading operator `==` to compare value equality instead of reference equality can be useful because, as immutable objects, they can be considered the same as long as they have the same value. Overriding operator `==` in non-immutable types is not recommended.

Overloaded operator `==` implementations should not throw exceptions. Any type that overloads operator `==` should also overload operator `!=`. For example:

C#

```
//add this code to class ThreeDPoint as defined previously  
//  
public static bool operator ==(ThreeDPoint a, ThreeDPoint b)  
{  
    // If both are null, or both are same instance, return true.  
    if (System.Object.ReferenceEquals(a, b))  
    {  
        return true;  
    }  
  
    // If one is null, but not both, return false.  
    if (((object)a == null) || ((object)b == null))  
    {  
        return false;  
    }  
  
    // Return true if the fields match:  
    return a.x == b.x && a.y == b.y && a.z == b.z;  
}  
  
public static bool operator !=(ThreeDPoint a, ThreeDPoint b)  
{  
    return !(a == b);  
}
```

Note

A common error in overloads of operator `==` is to use `(a == b)`, `(a == null)`, or `(b == null)` to check for reference equality. This instead results in a call to the overloaded operator `==`, causing an infinite loop. Use **ReferenceEquals** or cast the type to **Object**, to avoid the loop.

See Also

Reference

[Statements, Expressions, and Operators \(C# Programming Guide\)](#)

[Operators \(C# Programming Guide\)](#)

[Overloadable Operators \(C# Programming Guide\)](#)

[== Operator \(C# Reference\)](#)

[\(\) Operator \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Objects, Classes and Structs (C# Programming Guide)

C# is an object-oriented programming language, and uses classes and structs to implement types such as Windows Forms, user interface controls, and data structures. A typical C# application consists of classes defined by the programmer, combined with classes from the .NET Framework.

C# provides many powerful ways of defining classes, such as providing different access levels, inheriting features from other classes, and allowing the programmer to specify what happens when types are instantiated or destroyed.

Classes can also be defined as generic through the use of type parameters that enable client code to customize the class in a type-safe and efficient manner. A single generic class, for example [System.Collections.Generic.List](#) in the .NET Framework Class Library, can be used by client code to store integers, strings, or any other type of object.

Overview

Objects, classes and structs have the following properties:

- Objects are instances of a given data type. The data type provides a blueprint for the object that is created — or instantiated — when the application is executed.
- New data types are defined using classes and structs.
- Classes and structs form the building blocks of C# applications, containing code and data. A C# application will always contain at least one class.
- A struct can be considered a lightweight class, ideal for creating data types that store small amounts of data, and does not represent a type that might later be extended via inheritance.
- C# classes support inheritance, meaning they can derive from a previously defined class.

Related Sections

- [Type Design Guidelines](#)
- [Objects](#)
- [Classes](#)
- [Structs](#)
- [Inheritance](#)
- [Members](#)
- [Methods](#)
- [Constructors](#)
- [Destructors](#)
- [Using Constructors and Destructors](#)
- [Fields](#)
- [Constants](#)
- [Nested Types](#)
- [Access Modifiers](#)
- [Partial Class Definitions](#)
- [Static Classes and Static Class Members](#)
- [How to: Know the Difference Between Passing a Struct and Passing a Class Reference to a Method](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6 Classes and Objects

- [1.7 Structs](#)
- [3.4.4 Class Members](#)
- [4.2.1 Class Types](#)
- [10 Classes](#)
- [11 Structs](#)

See Also

Concepts

[C# Programming Guide](#)

Objects (C# Programming Guide)

Objects are programming constructs that have data, behavior, and identity. Object data is contained in the fields, properties, and events of the object, and object behaviors are defined by the methods and interfaces of the object.

Objects have identity — two objects with the same set of data are not necessarily the same object.

Objects in C# are defined through **classes** and **structs** — these form the single blueprint from which all objects of that type operate.

Objects Overview

Objects have the following properties:

- Everything you use in C# is an object, including Windows Forms and controls.
- Objects are instantiated; that is, they are created from templates defined by classes and structs.
- Objects use [Properties \(C# Programming Guide\)](#) to obtain and change the information they contain.
- Objects often have methods and events that allow them to perform actions.
- Visual Studio provides tools for manipulating objects: the [Properties Window](#) allows you to change the attributes of objects such as Windows Forms. The [Object Browser](#) allows you to examine the contents of an object.
- All C# objects inherit from the [Object](#).

Related Sections

For more information:

- [Classes \(C# Programming Guide\)](#)
- [Structs \(C# Programming Guide\)](#)
- [Constructors \(C# Programming Guide\)](#)
- [Destructors \(C# Programming Guide\)](#)
- [Events \(C# Programming Guide\)](#)

See Also

Reference

[object \(C# Reference\)](#)

[Inheritance \(C# Programming Guide\)](#)

[class \(C# Reference\)](#)

[struct \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[.NET Remoting](#)

Classes (C# Programming Guide)

A class is the most powerful data type in C#. Like structures, a class defines the data and behavior of the data type. Programmers can then create objects that are instances of this class. Unlike structures, classes support inheritance, a fundamental part of object-oriented programming. For more information, see [Inheritance](#).

Declaring Classes

Classes are defined using the `class` keyword, as shown in the following example:

C#

```
public class Customer
{
    //Fields, properties, methods and events go here...
}
```

The **class** keyword is preceded by the access level. In this case `public` is used, meaning anyone can create objects from this class. The name of the class follows the **class** keyword. The remainder of the definition is the class body, where the behavior and data are defined. Fields, properties, methods, and events on a class are collectively referred to as *class members*.

Creating Objects

Although they are sometimes used interchangeably, a class and an object are different things. A class defines a type of object, but it is not an object itself. An object is a concrete entity based on a class, and is sometimes called an instance of a class.

Objects can be created using the `new` keyword followed by the name of the class that the object will be based upon, like this:

C#

```
Customer object1 = new Customer();
```

When an instance of a class is created, a reference to the object is passed back to the programmer. In the example above, `object1` is a reference to an object based on `Customer`. This reference refers to the new object, but does not contain the object data itself. In fact, you can create an object reference without creating an object at all:

C#

```
Customer object2;
```

Creating object references like this one that does not refer to an object is not recommended because attempting to access an object through such a reference will fail at run time. However, such a reference can be made to refer to an object, either by creating a new object, or by assigning it to an existing object, like this:

C#

```
Customer object3 = new Customer();
Customer object4 = object3;
```

This code creates two object references that both refer to the same object. Therefore, any changes to the object made through `object3` will be reflected in subsequent uses of `object4`. It is because objects based on classes are referred to by reference that classes are known as reference types.

Class Inheritance

Inheritance is accomplished through the use of a *derivation*, which means a class is declared using a *base class* from which it inherits data and behavior. A base class is specified by appending a colon and the name of the base class following the derived class name, like this:

C#

```
public class Manager : Employee
{
    // Employee fields, properties, methods and events are inherited
    // New Manager fields, properties, methods and events go here...
}
```

When a class declares a base class, all of the class members defined for the base class become part of the new class as well. Because a base class may itself inherit from another class, which inherited from another class, and so on, a class may end up with many base classes.

Example

In the following example, a public class is defined, containing a single field, a method and a special method called a *constructor*. For more information, see [Constructors](#). The class is then instantiated with the **new** keyword.

C#

```
public class Person
{
    // Field
    public string name;

    // Constructor
    public Person()
    {
        name = "unknown";
    }

    // Method
    public void SetName(string newName)
    {
        name = newName;
    }
}

class TestPerson
{
    static void Main()
    {
        Person person1 = new Person();
        System.Console.WriteLine(person1.name);

        person1.SetName("John Smith");
        System.Console.WriteLine(person1.name);
    }
}
```

Output

unknown

John Smith

Classes Overview

Classes have the following properties:

- Unlike C++, only single inheritance is supported: a class can inherit implementation from only one base class.
- A class can implement more than one interface. For more information, see [Interfaces](#).
- Class definitions can be split between different source files. For more information, see [Partial Class Definitions](#).
- Static classes are sealed classes containing only static methods. For more information, see [Static Classes and Static Class Members](#).
-

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6 Classes and objects
- 10 Classes

See Also

Reference

[Members \(C# Programming Guide\)](#)

[Methods \(C# Programming Guide\)](#)

[Destructors \(C# Programming Guide\)](#)

[Objects \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Constructors \(C# Programming Guide\)](#)

Structs (C# Programming Guide)

Structs are defined using the [struct](#) keyword, for example:

C#

```
public struct PostalAddress
{
    // Fields, properties, methods and events go here...
}
```

Structs share almost all the same syntax as classes, although structs are more limited than classes:

- Within a struct declaration, fields cannot be initialized unless they are declared as `const` or `static`.
- A struct may not declare a default constructor —a constructor with no parameters — or a destructor.

Copies of structs are created and destroyed automatically by the compiler, so a default constructor and destructor are unnecessary. In effect, the compiler implements the default constructor by assigning all the fields of their default values (see [Default Values Table](#)). Structs cannot inherit from classes or other structs.

Structs are value types — when an object is created from a struct and assigned to a variable, the variable contains the entire value of the struct. When a variable containing a struct is copied, all of the data is copied, and any modification to the new copy does not change the data for the old copy. Because structs do not use references, they do not have identity — there is no way to distinguish between two instances of a value type with the same data. All value types in C# inherently derive from [ValueType](#), which inherits from [Object](#).

Value types can be converted to reference types by the compiler in a process known as boxing. For more information, see [Boxing and Unboxing](#).

Structs Overview

Structs have the following properties:

- Structs are value types while classes are reference types.
- Unlike classes, structs can be instantiated without using a `new` operator.
- Structs can declare constructors, but they must take parameters.
- A struct cannot inherit from another struct or class, and it cannot be the base of a class. All structs inherit directly from **System.ValueType**, which inherits from **System.Object**.
- A struct can implement interfaces.

Related Sections

For more information:

- [Using structs](#)
- [Using Constructors and Destructors](#)
- [How to: Know the Difference Between Passing a Struct and Passing a Class Reference to a Method](#)
- [How to: Implement User-Defined Conversions Between Structs](#)

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Structure Design](#)

[Classes \(C# Programming Guide\)](#)

Using Structs (C# Programming Guide)

The **struct** type is suitable for representing lightweight objects such as **Point**, **Rectangle**, and **Color**. Although it is possible to represent a point as a **class**, a **struct** is more efficient in some scenarios. For example, if you declare an array of 1000 **Point** objects, you will allocate additional memory for referencing each object; in this case, a struct would be less expensive. Since the .NET Framework contains an object called **Point**, we'll call our struct "CoOrds" instead.

C#

```
public struct CoOrds
{
    public int x, y;

    public CoOrds(int p1, int p2)
    {
        x = p1;
        y = p2;
    }
}
```

It is an error to declare a default (parameterless) constructor for a struct. A default constructor is always provided to initialize the struct members to their default values. It is also an error to initialize an instance field in a struct.

When you create a struct object using the **new** operator, it gets created and the appropriate constructor is called. Unlike classes, structs can be instantiated without using the **new** operator. If you do not use **new**, the fields will remain unassigned and the object cannot be used until all of the fields are initialized.

There is no inheritance for structs as there is for classes. A struct cannot inherit from another struct or class, and it cannot be the base of a class. Structs, however, inherit from the base class **Object**. A struct can implement interfaces, and it does that exactly as classes do.

Unlike C++, you cannot declare a class using the keyword **struct**. In C#, classes and structs are semantically different. A struct is a value type, while a class is a reference type. For more information, see [Value Types](#).

Unless you need reference-type semantics, small classes may be more efficiently handled by the system as a struct.

Example 1

This example demonstrates **struct** initialization using both default and parameterized constructors.

C#

```
public struct CoOrds
{
    public int x, y;

    public CoOrds(int p1, int p2)
    {
        x = p1;
        y = p2;
    }
}
```

C#

```
// Declare and initialize struct objects.
class TestCoOrds
{
    static void Main()
    {
        // Initialize:
        CoOrds coords1 = new CoOrds();
        CoOrds coords2 = new CoOrds(10, 10);
    }
}
```

```

        // Display results:
        System.Console.Write("CoOrds 1: ");
        System.Console.WriteLine("x = {0}, y = {1}", coords1.x, coords1.y);

        System.Console.Write("CoOrds 2: ");
        System.Console.WriteLine("x = {0}, y = {1}", coords2.x, coords2.y);
    }
}

```

Output

CoOrds 1: x = 0, y = 0

CoOrds 2: x = 10, y = 10

Example 2

This example demonstrates a feature that is unique to structs. It creates a CoOrds object without using the **new** operator. If you replace the word **struct** with the word **class**, the program will not compile.

C#

```

public struct CoOrds
{
    public int x, y;

    public CoOrds(int p1, int p2)
    {
        x = p1;
        y = p2;
    }
}

```

C#

```

// Declare a struct object without "new."
class TestCoOrdsNoNew
{
    static void Main()
    {
        // Declare an object:
        CoOrds coords1;

        // Initialize:
        coords1.x = 10;
        coords1.y = 20;

        // Display results:
        System.Console.Write("CoOrds 1: ");
        System.Console.WriteLine("x = {0}, y = {1}", coords1.x, coords1.y);
    }
}

```

Output

CoOrds 1: x = 10, y = 20

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Structs \(C# Programming Guide\)](#)

Inheritance (C# Programming Guide)

Classes can inherit from another class. This is accomplished by putting a colon after the class name when declaring the class, and naming the class to inherit from—the base class—after the colon, as follows:

C#

```
public class A
{
    public A() { }
}

public class B : A
{
    public B() { }
}
```

The new class—the derived class—then gains all the non-private data and behavior of the base class in addition to any other data or behaviors it defines for itself. The new class then has two effective types: the type of the new class and the type of the class it inherits.

In the example above, class B is effectively both B and A. When you access a B object, you can use the cast operation to convert it to an A object. The B object is not changed by the cast, but your view of the B object becomes restricted to A's data and behaviors. After casting a B to an A, that A can be cast back to a B. Not all instances of A can be cast to B—just those that are actually instances of B. If you access class B as a B type, you get both the class A and class B data and behaviors. The ability for an object to represent more than one type is called polymorphism. For more information, see [Polymorphism \(C# Programming Guide\)](#). For more information on casting, see [Casting \(C# Programming Guide\)](#).

Structs cannot inherit from other structs or classes. Both classes and structs can inherit from one or more interfaces. For more information, see [Interfaces \(C# Programming Guide\)](#)

In This Section

- [Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#)
- [Polymorphism \(C# Programming Guide\)](#)
- [Interfaces \(C# Programming Guide\)](#)

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[class \(C# Reference\)](#)

[struct \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Abstract and Sealed Classes and Class Members (C# Programming Guide)

The [abstract](#) keyword enables you to create classes and [class](#) members solely for the purpose of inheritance—to define features of derived, non-abstract classes. The [sealed](#) keyword enables you to prevent the inheritance of a class or certain class members that were previously marked [virtual](#). For more information, see [How to: Define Abstract Properties \(C# Programming Guide\)](#).

Abstract Classes and Class Members

Classes can be declared as abstract. This is accomplished by putting the keyword **abstract** before the keyword **class** in the class definition. For example:

C#

```
public abstract class A
{
    // Class members here.
}
```

An abstract class cannot be instantiated. The purpose of an abstract class is to provide a common definition of a base class that multiple derived classes can share. For example, a class library may define an abstract class that is used as a parameter to many of its functions, and require programmers using that library to provide their own implementation of the class by creating a derived class.

Abstract classes may also define abstract methods. This is accomplished by adding the keyword **abstract** before the return type of the method. For example:

C#

```
public abstract class A
{
    public abstract void DoWork(int i);
}
```

Abstract methods have no implementation, so the method definition is followed by a semicolon instead of a normal method block. Derived classes of the abstract class must implement all abstract methods. When an abstract class inherits a virtual method from a base class, the abstract class can override the virtual method with an abstract method. For example:

C#

```
// compile with: /target:library
public class D
{
    public virtual void DoWork(int i)
    {
        // Original implementation.
    }
}

public abstract class E : D
{
    public abstract override void DoWork(int i);
}

public class F : E
{
    public override void DoWork(int i)
    {
        // New implementation.
    }
}
```

If a virtual method is declared abstract, it is still virtual to any class inheriting from the abstract class. A class inheriting an abstract method cannot access the original implementation of the method—in the previous example, `DoWork` on class F cannot call `DoWork` on class D. In this way, an abstract class can force derived classes to provide new method implementations for virtual methods.

Sealed Classes and Class Members

Classes can be declared as sealed. This is accomplished by putting the keyword **sealed** before the keyword **class** in the class definition. For example:

C#

```
public sealed class D
{
    // Class members here.
}
```

A sealed class cannot be used as a base class. For this reason, it cannot also be an abstract class. Sealed classes are primarily used to prevent derivation. Because they can never be used as a base class, some run-time optimizations can make calling sealed class members slightly faster.

A class member, method, field, property, or event, on a derived class that is overriding a virtual member of the base class can declare that member as sealed. This negates the virtual aspect of the member for any further derived class. This is accomplished by putting the **sealed** keyword before the **override** keyword in the class member declaration. For example:

C#

```
public class D : C
{
    public sealed override void DoWork() { }
}
```

See Also

Tasks

[How to: Define Abstract Properties \(C# Programming Guide\)](#)

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Inheritance \(C# Programming Guide\)](#)

[Methods \(C# Programming Guide\)](#)

[Fields \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

How to: Define Abstract Properties (C# Programming Guide)

The following example shows how to define [abstract](#) properties. An abstract property declaration does not provide an implementation of the property accessors -- it declares that the class supports properties, but leaves the accessor implementation to derived classes. The following example demonstrates how to implement the abstract properties inherited from a base class.

This sample consists of three files, each of which is compiled individually and its resulting assembly is referenced by the next compilation:

- abstractshape.cs: the `Shape` class that contains an abstract `Area` property.
- shapes.cs: The subclasses of the `Shape` class.
- shapetest.cs: A test program to display the areas of some `Shape`-derived objects.

To compile the example, use the following command:

```
csc abstractshape.cs shapes.cs shapetest.cs
```

This will create the executable file `shapetest.exe`.

Example

This file declares the `Shape` class that contains the `Area` property of the type **double**.

C#

```
// compile with: csc /target:library abstractshape.cs
public abstract class Shape
{
    private string m_id;

    public Shape(string s)
    {
        // calling the set accessor of the Id property.
        Id = s;
    }

    public string Id
    {
        get
        {
            return m_id;
        }

        set
        {
            m_id = value;
        }
    }

    // Area is a read-only property - only a get accessor is needed:
    public abstract double Area
    {
        get;
    }

    public override string ToString()
    {
        return Id + " Area = " + string.Format("{0:F2}", Area);
    }
}
```

- Modifiers on the property are placed on the property declaration itself. For example:

```
public abstract double Area
```

- When declaring an abstract property (such as `Area` in this example), you simply indicate what property accessors are available, but do not implement them. In this example, only a `get` accessor is available, so the property is read-only.

The following code shows three subclasses of `Shape` and how they override the `Area` property to provide their own implementation.

C#

```
// compile with: csc /target:library /reference:abstractshape.dll shapes.cs
public class Square : Shape
{
    private int m_side;

    public Square(int side, string id)
        : base(id)
    {
        m_side = side;
    }

    public override double Area
    {
        get
        {
            // Given the side, return the area of a square:
            return m_side * m_side;
        }
    }
}

public class Circle : Shape
{
    private int m_radius;

    public Circle(int radius, string id)
        : base(id)
    {
        m_radius = radius;
    }

    public override double Area
    {
        get
        {
            // Given the radius, return the area of a circle:
            return m_radius * m_radius * System.Math.PI;
        }
    }
}

public class Rectangle : Shape
{
    private int m_width;
    private int m_height;

    public Rectangle(int width, int height, string id)
        : base(id)
    {
        m_width = width;
        m_height = height;
    }

    public override double Area
    {
        get
        {
```



```

        // Given the width and height, return the area of a rectangle:
        return m_width * m_height;
    }
}

```

The following code shows a test program that creates a number of `Shape`-derived objects and prints out their areas.

C#

```

// compile with: csc /reference:abstractshape.dll;shapes.dll shapetest.cs
class TestClass
{
    static void Main()
    {
        Shape[] shapes =
        {
            new Square(5, "Square #1"),
            new Circle(3, "Circle #1"),
            new Rectangle( 4, 5, "Rectangle #1")
        };

        System.Console.WriteLine("Shapes Collection");
        foreach (Shape s in shapes)
        {
            System.Console.WriteLine(s);
        }
    }
}

```

Output

Shapes Collection
Square #1 Area = 25.00
Circle #1 Area = 28.27
Rectangle #1 Area = 20.00

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#)

[Properties \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Polymorphism (C# Programming Guide)

Through inheritance, a [class](#) can be used as more than one type; it can be used as its own type, any base types, or any [interface](#) type if it implements interfaces. This is called polymorphism. In C#, every type is polymorphic. Types can be used as their own type or as a [Object](#) instance, because any type automatically treats **Object** as a base type.

Polymorphism is important not only to the derived classes, but to the base classes as well. Anyone using the [base](#) class could, in fact, be using an object of the derived class that has been cast to the base class type. Designers of a base class can anticipate the aspects of their base class that are likely to change for a derived type. For example, a base class for cars might contain behavior that is subject to change when the car in question is a minivan or a convertible. A base class can mark those class members as virtual, allowing derived classes representing convertibles and minivans to override that behavior.

For more information, see [Inheritance](#).

Polymorphism Overview

When a derived class inherits from a base class, it gains all the methods, fields, properties and events of the base class. To change the data and behavior of a base class, you have two choices: you can replace the base member with a new derived member, or you can override a virtual base member.

Replacing a member of a base class with a new derived member requires the [new](#) keyword. If a base class defines a method, field, or property, the **new** keyword is used to create a new definition of that method, field, or property on a derived class. The **new** keyword is placed before the return type of a class member that is being replaced. For example:

C#

```
public class BaseClass
{
    public void DoWork() { }
    public int WorkField;
    public int WorkProperty
    {
        get { return 0; }
    }
}

public class DerivedClass : BaseClass
{
    public new void DoWork() { }
    public new int WorkField;
    public new int WorkProperty
    {
        get { return 0; }
    }
}
```

When the **new** keyword is used, the new class members are called instead of the base class members that have been replaced. Those base class members are called hidden members. Hidden class members can still be called if an instance of the derived class is cast to an instance of the base class. For example:

C#

```
DerivedClass B = new DerivedClass();
B.DoWork(); // Calls the new method.

BaseClass A = (BaseClass)B;
A.DoWork(); // Calls the old method.
```

In order for an instance of a derived class to completely take over a class member from a base class, the base class has to declare that member as virtual. This is accomplished by adding the [virtual](#) keyword before the return type of the member. A derived class then has the option of using the [override](#) keyword, instead of **new**, to replace the base class implementation with its own. For example:

C#

```
public class BaseClass
{
    public virtual void DoWork() { }
    public virtual int WorkProperty
    {
        get { return 0; }
    }
}
public class DerivedClass : BaseClass
{
    public override void DoWork() { }
    public override int WorkProperty
    {
        get { return 0; }
    }
}
```

Fields cannot be virtual; only methods, properties, events and indexers can be virtual. When a derived class overrides a virtual member, that member is called even when an instance of that class is being accessed as an instance of the base class. For example:

C#

```
DerivedClass B = new DerivedClass();
B.DoWork(); // Calls the new method.

BaseClass A = (BaseClass)B;
A.DoWork(); // Also calls the new method.
```

Virtual methods and properties allow you to plan ahead for future expansion. Because a virtual member is called regardless of which type the caller is using, it gives derived classes the option to completely change the apparent behavior of the base class.

Virtual members remain virtual indefinitely, no matter how many classes have been declared between the class that originally declared the virtual member. If class A declares a virtual member, and class B derives from A, and class C derives from B, class C inherits the virtual member, and has the option to override it, regardless of whether class B declared an override for that member. For example:

C#

```
public class A
{
    public virtual void DoWork() { }
}
public class B : A
{
    public override void DoWork() { }
}
```

C#

```
public class C : B
{
    public override void DoWork() { }
}
```

A derived class can stop virtual inheritance by declaring an override as sealed. This requires putting the [sealed](#) keyword before the **override** keyword in the class member declaration. For example:

C#

```
public class C : B
```

```
{  
    public sealed override void DoWork() { }  
}
```

In the previous example, the method `DoWork` is no longer virtual to any class derived from `C`. It is still virtual for instances of `C`, even if they are cast to type `B` or type `A`. Sealed methods can be replaced by derived classes using the **new** keyword, as the following example shows:

C#

```
public class D : C  
{  
    public new void DoWork() { }  
}
```

In this case, if `DoWork` is called on `D` using a variable of type `D`, the new `DoWork` is called. If a variable of type `C`, `B`, or `A` is used to access an instance of `D`, a call to `DoWork` will follow the rules of virtual inheritance, routing those calls to the implementation of `DoWork` on class `C`.

A derived class that has replaced or overridden a method or property can still access the method or property on the base class using the `base` keyword. For example:

C#

```
public class A  
{  
    public virtual void DoWork() { }  
}  
public class B : A  
{  
    public override void DoWork() { }  
}
```

C#

```
public class C : B  
{  
    public override void DoWork()  
    {  
        // Call DoWork on B to get B's behavior:  
        base.DoWork();  
  
        // DoWork behavior specific to C goes here:  
        // ...  
    }  
}
```

For more information, see [base](#).

Note

It is recommended that virtual members use **base** to call the base class implementation of that member in their own implementation. Letting the base class behavior happen allows the derived class to concentrate on implementing behavior specific to the derived class. If the base class implementation is not called, it is up to the derived class to make their behavior compatible with the behavior of the base class.

In This Section

For more information:

- [Versioning with the Override and New Keywords \(C# Programming Guide\)](#)

- [Knowing When to Use Override and New Keywords \(C# Programming Guide\)](#)
- [How To: Override the ToString Method \(C# Programming Guide\)](#)

See Also

Reference

[Inheritance \(C# Programming Guide\)](#)

[Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#)

[Methods \(C# Programming Guide\)](#)

[Properties \(C# Programming Guide\)](#)

[Indexers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

Versioning with the Override and New Keywords (C# Programming Guide)

The C# language is designed so that versioning between [base](#) and derived classes in different libraries can evolve and maintain backwards compatibility. This means, for example, that the introduction of a new member in a base [class](#) with the same name as a member in a derived class is completely supported by C# and does not lead to unexpected behavior. It also means that a class must explicitly state whether a method is intended to override an inherited method, or whether a method is a new method that simply hides a similarly named inherited method.

C# allows derived classes to contain methods with the same name as base class methods.

- The base class method must be defined [virtual](#).
- If the method in the derived class is not preceded by [new](#) or [override](#) keywords, the compiler will issue a warning and the method will behave as if the **new** keyword were present.
- If the method in the derived class is preceded with the **new** keyword, the method is defined as being independent of the method in the base class.
- If the method in the derived class is preceded with the **override** keyword, objects of the derived class will call that method rather than the base class method.
- The base class method can be called from within the derived class using the **base** keyword.
- The **override**, **virtual**, and **new** keywords can also be applied to properties, indexers, and events.

By default, C# methods are not virtual — if a method is declared as virtual, any class inheriting the method can implement its own version. To make a method virtual, the **virtual** modifier is used in the method declaration of the base class. The derived class can then override the base virtual method by using the **override** keyword or hide the virtual method in the base class by using the **new** keyword. If neither the **override** keyword nor the **new** keyword is specified, the compiler will issue a warning and the method in the derived class will hide the method in the base class. For more information, see [Compiler Warning CS0108](#).

To demonstrate this in practice, assume for a moment that Company A has created a class entitled `GraphicsClass`, which your program uses. `GraphicsClass` looks like this:

C#

```
class GraphicsClass
{
    public virtual void DrawLine() { }
    public virtual void DrawPoint() { }
}
```

Your company uses this class, and you use it to derive your own class, adding a new method:

C#

```
class YourDerivedGraphicsClass : GraphicsClass
{
    public void DrawRectangle() { }
}
```

Your application is used without problems, until Company A releases a new version of `GraphicsClass`, which looks like this:

C#

```
class GraphicsClass
{
    public virtual void DrawLine() { }
    public virtual void DrawPoint() { }
    public virtual void DrawRectangle() { }
}
```

The new version of `GraphicsClass` now contains a method entitled `DrawRectangle`. Initially, nothing happens. The new version is still binary compatible with the old — any software you have deployed will continue to work, even if the new class is installed on those computer systems. Any existing calls to the method `DrawRectangle` will continue to reference your version, in your derived class.

However, once you recompile your application using the new version of `GraphicsClass`, you will receive a warning from the compiler. For more information, see [Compiler Warning CS0108](#).

This warning informs you that you need to consider how you want your `DrawRectangle` method to behave in your application.

If you want your method to override the new base class method, use the **override** keyword, like this:

C#

```
class YourDerivedGraphicsClass : GraphicsClass
{
    public override void DrawRectangle() { }
}
```

The **override** keyword makes sure that any objects derived from `YourDerivedGraphicsClass` will use the derived class version of `DrawRectangle`. Objects derived from `YourDerivedGraphicsClass` can still access the base class version of `DrawRectangle` using the base keyword, like this:

C#

```
base.DrawRectangle();
```

If you do not want your method to override the new base class method, the following considerations apply. To avoid confusion between the two methods, you can rename your method. This can be time-consuming and error-prone, and simply not practical in some situations. However, if your project is relatively small, you can use Visual Studio's Refactoring options to rename the method. For more information, see [Refactoring Classes and Types](#).

Alternatively, you can prevent the warning by using the keyword **new** in your derived class definition, like this:

C#

```
class YourDerivedGraphicsClass : GraphicsClass
{
    public new void DrawRectangle() { }
}
```

Using the **new** keyword tells the compiler that your definition hides the definition contained in the base class. This is the default behavior.

Override and Method Selection

When a method is named on a class, the C# compiler selects the best method to call if more than one method is compatible with the call, such as when there are two methods with the same name, and parameters that are compatible with the parameter passed. The following methods would be compatible:

C#

```
public class Derived : Base
{
    public override void DoWork(int param) { }
    public void DoWork(double param) { }
}
```

When `DoWork` is called on an instance of `Derived`, the C# compiler will first try to make the call compatible with the versions of `DoWork` declared originally on `Derived`. Override methods are not considered as declared on a class, they are new implementations of a method declared on a base class. Only if the C# compiler cannot match the method call to an original

method on `Derived` will it try to match the call to an overridden method with the same name and compatible parameters. For example:

C#

```
int val = 5;
Derived d = new Derived();
d.DoWork(val); // Calls DoWork(double).
```

Because the variable `val` can be converted to a `double` implicitly, the C# compiler calls `DoWork(double)` instead of `DoWork(int)`. There are two ways to avoid this. First, avoid declaring new methods with the same name as virtual methods. Second, you can instruct the C# compiler to call the virtual method by making it search the base class method list by casting the instance of `Derived` to `Base`. Because the method is virtual, the implementation of `DoWork(int)` on `Derived` will be called. For example:

C#

```
((Base)d).DoWork(val); // Calls DoWork(int) on Derived.
```

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Methods \(C# Programming Guide\)](#)

[Inheritance \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Knowing When to Use Override and New Keywords (C# Programming Guide)

C# enables methods in derived classes to have the same name as methods in base classes—as long as you are very specific about how the new method should be treated. The following example demonstrates the use of the [new](#) and [override](#) keywords.

First we declare three classes: a [base](#) class called `Car`, and two classes that derive from it, `ConvertibleCar` and `Minivan`. The base class contains a single method, `DescribeCar`, which sends a description of the car to the console. The derived class methods also include a method called `DescribeCar`, which displays their unique properties. These methods also call the base class `DescribeCar` method to demonstrate how they have inherited the properties of the `Car` class.

In order to highlight the difference, the `ConvertibleCar` class is defined with the **new** keyword, while the `Minivan` class is defined with **override**.

C#

```
// Define the base class
class Car
{
    public virtual void DescribeCar()
    {
        System.Console.WriteLine("Four wheels and an engine.");
    }
}

// Define the derived classes
class ConvertibleCar : Car
{
    public new virtual void DescribeCar()
    {
        base.DescribeCar();
        System.Console.WriteLine("A roof that opens up.");
    }
}

class Minivan : Car
{
    public override void DescribeCar()
    {
        base.DescribeCar();
        System.Console.WriteLine("Carries seven people.");
    }
}
```

We can now write some code that declares instances of these classes, and calls their methods so that the objects can describe themselves:

C#

```
public static void TestCars1()
{
    Car car1 = new Car();
    car1.DescribeCar();
    System.Console.WriteLine("-----");

    ConvertibleCar car2 = new ConvertibleCar();
    car2.DescribeCar();
    System.Console.WriteLine("-----");

    Minivan car3 = new Minivan();
    car3.DescribeCar();
    System.Console.WriteLine("-----");
}
```

As you might expect, the output looks like this:

```
Four wheels and an engine.
```

```
-----
```

```
Four wheels and an engine.
```

```
A roof that opens up.
```

```
-----
```

```
Four wheels and an engine.
```

```
Carries seven people.
```

```
-----
```

However, in this next section of code, we declare an array of objects derived from the `Car` base class. This array can store `Car`, `ConvertibleCar`, and `Minivan` objects. The array is declared like this:

C#

```
public static void TestCars2()
{
    Car[] cars = new Car[3];
    cars[0] = new Car();
    cars[1] = new ConvertibleCar();
    cars[2] = new Minivan();
}
```

We can then use a **foreach** loop to visit each `Car` object contained in the array, and call the `DescribeCar` method, like this:

C#

```
foreach (Car vehicle in cars)
{
    System.Console.WriteLine("Car object: " + vehicle.GetType());
    vehicle.DescribeCar();
    System.Console.WriteLine("-----");
}
```

The output from this loop is as follows:

```
Car object: YourApplication.Car
```

```
Four wheels and an engine.
```

```
-----
```

```
Car object: YourApplication.ConvertibleCar
```

```
Four wheels and an engine.
```

```
-----
```

```
Car object: YourApplication.Minivan
```

```
Four wheels and an engine.
```

```
Carries seven people.
```

```
-----
```

Notice how the `ConvertibleCar` description is not what you might expect. As the **new** keyword was used to define this method, the derived class method is not called—the base class method is called instead. The `Minivan` object correctly calls the overridden method, producing the results we expected.

If you want to enforce a rule that all classes derived from `Car` must implement the `DescribeCar` method, you should create a new base class that defines the method `DescribeCar` as **abstract**. An abstract method does not contain any code, only the method signature. Any classes derived from this base class must provide an implementation of `DescribeCar`. For more information, see [abstract](#).

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Versioning with the Override and New Keywords \(C# Programming Guide\)](#)

How To: Override the ToString Method (C# Programming Guide)

Every object in C# inherits the [ToString](#) method, which returns a string representation of that object. For example, all variables of type `int` have a **ToString** method, allowing them to return their contents as a string:

C#

```
int x = 42;
string strx = x.ToString();
System.Console.WriteLine(strx);
```

When you create a custom class or struct, you should override the **ToString** method in order to provide information about your type to client code.

Security Note

When deciding what information to provide through this method, consider whether your class or struct will ever be used by untrusted code. Be careful to ensure that you do not provide any information that could be exploited by malicious code.

To override the ToString method in your class or struct:

1. Declare a **ToString** method with the following modifiers and return type:

```
public override string ToString(){}
```

2. Implement the method so that it returns a string.

The following example returns not only the name of the class, but also the data specific to a particular instance of the class. Note that it also uses the **ToString** method on the `age` variable to convert the **int** to a string that can be output.

```
class Person
{
    string name;
    int age;
    SampleObject(string name, int age)
    {
        this.name = name;
        this.age = age;
    }
    public override string ToString()
    {
        string s = age.ToString();
        return "Person: " + name + " " + s;
    }
}
```

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[new \(C# Reference\)](#)

[override \(C# Reference\)](#)

[virtual \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Interfaces (C# Programming Guide)

Interfaces are defined using the [interface](#) keyword. For example:

C#

```
interface IComparable
{
    int CompareTo(object obj);
}
```

Interfaces describe a group of related behaviors that can belong to any [class](#) or [struct](#). Interfaces can be made up of methods, properties, events, indexers, or any combination of those four member types. An interface can not contain fields. Interfaces members are automatically public.

Classes and structs can inherit from interfaces in a manner similar to how classes can inherit a base class or struct, with two exceptions:

- A class or struct can inherit more than one interface.
- When a class or struct inherits an interface, it inherits only the method names and signatures, because the interface itself contains no implementations. For example:

C#

```
public class Minivan : Car, IComparable
{
    public int CompareTo(object obj)
    {
        //implementation of CompareTo
        return 0; //if the Minivans are equal
    }
}
```

To implement an interface member, the corresponding member on the class must be public, non-static, and have the same name and signature as the interface member. Properties and indexers on a class can define extra accessors for a property or indexer defined on an interface. For example, an interface may declare a property with a [get](#) accessor, but the class implementing the interface can declare the same property with both a **get** and [set](#) accessor. However, if the property or indexer uses explicit implementation, the accessors must match.

Interfaces and interface members are abstract; interfaces do not provide a default implementation. For more information, see [Abstract and Sealed Classes and Class Members](#).

The [IComparable](#) interface announces to the user of the object that the object can compare itself to other objects of the same type, and the user of the interface does not need to know how this is implemented.

Interfaces can inherit other interfaces. It is possible for a class to inherit an interface multiple times, through base classes or interfaces it inherits. In this case, the class can only implement the interface once, if it is declared as part of the new class. If the inherited interface is not declared as part of the new class, its implementation is provided by the base class that declared it. It is possible for a base class to implement interface members using virtual members; in that case, the class inheriting the interface can change the interface behavior by overriding the virtual members. For more information on virtual members, see [Polymorphism](#).

Interfaces Overview

An interface has the following properties:

- An interface is similar to an abstract base class: any non-abstract type inheriting the interface must implement all its members.
- An interface cannot be instantiated directly.

- Interfaces can contain events, indexers, methods and properties.
- Interfaces contain no implementation of methods.
- Classes and structs can inherit from more than one interface.
- An interface can itself inherit from multiple interfaces.

In This Section

- [Explicit Interface Implementation \(C# Programming Guide\)](#)
- [Interface Properties \(C# Programming Guide\)](#)
- [Indexers in Interfaces \(C# Programming Guide\)](#)
- [How to: Implement Interface Events \(C# Programming Guide\)](#)
- [How to: Explicitly Implement Interface Members \(C# Programming Guide\)](#)
- [How to: Explicitly Implement Interface Members with Inheritance \(C# Programming Guide\)](#)

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Inheritance \(C# Programming Guide\)](#)

[Methods \(C# Programming Guide\)](#)

[Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#)

[Methods \(C# Programming Guide\)](#)

[Properties \(C# Programming Guide\)](#)

[Indexers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Polymorphism \(C# Programming Guide\)](#)

[Events \(C# Programming Guide\)](#)

Explicit Interface Implementation (C# Programming Guide)

If a [class](#) implements two interfaces that contain a member with the same signature, then implementing that member on the class will cause both interfaces to use that member as their implementation. For example:

C#

```
interface IControl
{
    void Paint();
}
interface ISurface
{
    void Paint();
}
class SampleClass : IControl, ISurface
{
    // Both ISurface.Paint and IControl.Paint call this method.
    public void Paint()
    {
    }
}
```

If the two [interface](#) members do not perform the same function, however, this can lead to an incorrect implementation of one or both of the interfaces. It is possible to implement an interface member explicitly—creating a class member that is only called through the interface, and is specific to that interface. This is accomplished by naming the class member with the name of the interface and a period. For example:

C#

```
public class SampleClass : IControl, ISurface
{
    void IControl.Paint()
    {
        System.Console.WriteLine("IControl.Paint");
    }
    void ISurface.Paint()
    {
        System.Console.WriteLine("ISurface.Paint");
    }
}
```

The class member **IControl.Paint** is only available through the **IControl** interface, and **ISurface.Paint** is only available through **ISurface**. Both method implementations are separate, and neither is available directly on the class. For example:

C#

```
SampleClass obj = new SampleClass();
//obj.Paint(); // Compiler error.

IControl c = (IControl)obj;
c.Paint(); // Calls IControl.Paint on SampleClass.

ISurface s = (ISurface)obj;
s.Paint(); // Calls ISurface.Paint on SampleClass.
```

Explicit implementation is also used to resolve cases where two interfaces each declare different members of the same name such as a property and a method:

C#

```
interface ILeft
```



```
{
    int P { get; }
}
interface IRight
{
    int P();
}
```

To implement both interfaces, a class has to use explicit implementation either for the property P, or the method P, or both, to avoid a compiler error. For example:

C#

```
class Middle : ILeft, IRight
{
    public int P() { return 0; }
    int ILeft.P { get { return 0; } }
}
```

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Interfaces \(C# Programming Guide\)](#)

[Inheritance \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

How to: Explicitly Implement Interface Members (C# Programming Guide)

This example declares an [interface](#), `IDimensions`, and a class, `Box`, which explicitly implements the interface members `getLength` and `getWidth`. The members are accessed through the interface instance `dimensions`.

Example

C#

```
interface IDimensions
{
    float getLength();
    float getWidth();
}

class Box : IDimensions
{
    float lengthInches;
    float widthInches;

    Box(float length, float width)
    {
        lengthInches = length;
        widthInches = width;
    }
    // Explicit interface member implementation:
    float IDimensions.getLength()
    {
        return lengthInches;
    }
    // Explicit interface member implementation:
    float IDimensions.getWidth()
    {
        return widthInches;
    }

    static void Main()
    {
        // Declare a class instance box1:
        Box box1 = new Box(30.0f, 20.0f);

        // Declare an interface instance dimensions:
        IDimensions dimensions = (IDimensions)box1;

        // The following commented lines would produce compilation
        // errors because they try to access an explicitly implemented
        // interface member from a class instance:
        //System.Console.WriteLine("Length: {0}", box1.getLength());
        //System.Console.WriteLine("Width: {0}", box1.getWidth());

        // Print out the dimensions of the box by calling the methods
        // from an instance of the interface:
        System.Console.WriteLine("Length: {0}", dimensions.getLength());
        System.Console.WriteLine("Width: {0}", dimensions.getWidth());
    }
}
```

Output

Length: 30
Width: 20

Robust Programming

- Notice that the following lines, in the **Main** method, are commented out because they would produce compilation errors. An interface member that is explicitly implemented cannot be accessed from a [class](#) instance:

C#

```
//System.Console.WriteLine("Length: {0}", box1.getLength());  
//System.Console.WriteLine("Width: {0}", box1.getWidth());
```

- Notice also that the following lines, in the **Main** method, successfully print out the dimensions of the box because the methods are being called from an instance of the interface:

C#

```
System.Console.WriteLine("Length: {0}", dimensions.getLength());  
System.Console.WriteLine("Width: {0}", dimensions.getWidth());
```

See Also

Tasks

[How to: Explicitly Implement Interface Members with Inheritance \(C# Programming Guide\)](#)

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Interfaces \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

How to: Explicitly Implement Interface Members with Inheritance (C# Programming Guide)

Explicit [interface](#) implementation also allows the programmer to implement two interfaces that have the same member names and give each interface member a separate implementation. This example displays the dimensions of a box in both metric and English units. The `Box` [class](#) implements two interfaces `IEnglishDimensions` and `IMetricDimensions`, which represent the different measurement systems. Both interfaces have identical member names, `Length` and `Width`.

Example

C#

```
// Declare the English units interface:
interface IEnglishDimensions
{
    float Length();
    float Width();
}

// Declare the metric units interface:
interface IMetricDimensions
{
    float Length();
    float Width();
}

// Declare the Box class that implements the two interfaces:
// IEnglishDimensions and IMetricDimensions:
class Box : IEnglishDimensions, IMetricDimensions
{
    float lengthInches;
    float widthInches;

    public Box(float length, float width)
    {
        lengthInches = length;
        widthInches = width;
    }

    // Explicitly implement the members of IEnglishDimensions:
    float IEnglishDimensions.Length()
    {
        return lengthInches;
    }

    float IEnglishDimensions.Width()
    {
        return widthInches;
    }

    // Explicitly implement the members of IMetricDimensions:
    float IMetricDimensions.Length()
    {
        return lengthInches * 2.54f;
    }

    float IMetricDimensions.Width()
    {
        return widthInches * 2.54f;
    }

    static void Main()
    {
        // Declare a class instance box1:
        Box box1 = new Box(30.0f, 20.0f);
    }
}
```

```

// Declare an instance of the English units interface:
IEnglishDimensions eDimensions = (IEnglishDimensions)box1;

// Declare an instance of the metric units interface:
IMetricDimensions mDimensions = (IMetricDimensions)box1;

// Print dimensions in English units:
System.Console.WriteLine("Length(in): {0}", eDimensions.Length());
System.Console.WriteLine("Width (in): {0}", eDimensions.Width());

// Print dimensions in metric units:
System.Console.WriteLine("Length(cm): {0}", mDimensions.Length());
System.Console.WriteLine("Width (cm): {0}", mDimensions.Width());
}
}

```

Output

Length(in): 30
Width (in): 20
Length(cm): 76.2
Width (cm): 50.8

Robust Programming

If you want to make the default measurements in English units, implement the methods `Length` and `Width` normally, and explicitly implement the `Length` and `Width` methods from the `IMetricDimensions` interface:

C#

```

// Normal implementation:
public float Length()
{
    return lengthInches;
}
public float Width()
{
    return widthInches;
}

// Explicit implementation:
float IMetricDimensions.Length()
{
    return lengthInches * 2.54f;
}
float IMetricDimensions.Width()
{
    return widthInches * 2.54f;
}

```

In this case, you can access the English units from the class instance and access the metric units from the interface instance:

C#

```

public static void Test()
{
    Box box1 = new Box(30.0f, 20.0f);
    IMetricDimensions mDimensions = (IMetricDimensions)box1;

    System.Console.WriteLine("Length(in): {0}", box1.Length());
    System.Console.WriteLine("Width (in): {0}", box1.Width());
    System.Console.WriteLine("Length(cm): {0}", mDimensions.Length());
    System.Console.WriteLine("Width (cm): {0}", mDimensions.Width());
}

```

See Also

Tasks

[How to: Explicitly Implement Interface Members \(C# Programming Guide\)](#)

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Interfaces \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Members (C# Programming Guide)

Classes and structs have members that represent their data and behavior. Those members include:

[Fields \(C# Programming Guide\)](#)

Fields are instances of objects that are considered part of a class, normally holding class data. For example, a calendar class may have a field that contains the current date.

[Properties \(C# Programming Guide\)](#)

Properties are methods on a class that are accessed as if they were fields on that class. A property can provide protection for a class field to keep it from being changed without the object's knowledge.

[Methods \(C# Programming Guide\)](#)

Methods define the actions that a class can perform. Method can take parameters that provide input data, and can return output data through parameters. Methods can also return a value directly, without using a parameter.

[Events \(C# Programming Guide\)](#)

Events are a way of providing notifications about occurrences, such as button clicks or the successful completion of a method, to other objects. Events are defined and triggered using delegates. For more information, see [Events and Delegates](#).

[Operators \(C# Programming Guide\)](#)

Operators are terms or symbols such as +, *, <, and so on that perform operations on operands. Operators can be redefined to perform operations on custom data types. For more information, see [Overloadable Operators \(C# Programming Guide\)](#).

[Indexers \(C# Programming Guide\)](#)

Indexers allow an object to be indexed in a manner similar to arrays.

[Constructors \(C# Programming Guide\)](#)

Constructors are methods that are called when the object is first created. They are often used to initialize the object's data.

[Destructors \(C# Programming Guide\)](#)

Destructors are methods that are called by the runtime execution engine when the object is about to be removed from memory. They are generally used to make sure that any resources which need to be released are handled appropriately.

[Nested Types \(C# Programming Guide\)](#)

Nested Types are types declared within a class or struct. Nested Types are often used to describe objects used only by the types containing them.

See Also

Reference

[Methods \(C# Programming Guide\)](#)

[Methods \(C# Programming Guide\)](#)

[Destructors \(C# Programming Guide\)](#)

[Properties \(C# Programming Guide\)](#)

[Fields \(C# Programming Guide\)](#)

[Indexers \(C# Programming Guide\)](#)

[Nested Types \(C# Programming Guide\)](#)

[Operators \(C# Programming Guide\)](#)

[Overloadable Operators \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Classes \(C# Programming Guide\)](#)

[Events \(C# Programming Guide\)](#)

[Events and Delegates](#)

Other Resources

[Member Design Guidelines](#)

[Constructors \(C# Programming Guide\)](#)

Methods (C# Programming Guide)

A *method* is a code block containing a series of statements. In C#, every executed instruction is done so in the context of a method.

Methods are declared within a [class](#) or [struct](#) by specifying the access level, the return value, the name of the method, and any method parameters. Method parameters are surrounded by parentheses, and separated by commas. Empty parentheses indicate that the method requires no parameters. This class contains three methods:

C#

```
class Motorcycle
{
    public void StartEngine() { }
    public void AddGas(int gallons) { }
    public int Drive(int miles, int speed) { return 0; }
}
```

Calling a method on an object is similar to accessing a field. After the object name, add a period, the name of the method, and parentheses. Arguments are listed within the parentheses, and separated by commas. The methods of the `Motorcycle` class can therefore be called like this:

C#

```
Motorcycle moto = new Motorcycle();

moto.StartEngine();
moto.AddGas(15);
moto.Drive(5, 20);
```

Method Parameters

As shown in the previous code snippet, passing arguments to a method is simply a matter of providing them in the parentheses when calling a method. To the method being called, the incoming arguments are called *parameters*.

The parameters a method receives are also provided in a set of parentheses, but the type and a name for each parameter must be specified. The name does not have to be the same as the argument. For example:

C#

```
public static void PassesInteger()
{
    int fortyFour = 44;
    TakesInteger(fortyFour);
}
static void TakesInteger(int i)
{
    i = 33;
}
```

Here a method called `PassesInteger` passes an argument to a method called `TakesInteger`. Within `PassesInteger`, the argument is named `fortyFour`, but in `TakeInteger`, this is a parameter named `i`. This parameter exists only within the `TakesInteger` method. Any number of other variables can be named `i`, and they can be of any type, so long as they are not parameters or variables declared inside this method.

Notice that `TakesInteger` assigns a new value to the provided argument. One might expect this change to be reflected in the `PassesInteger` method once `TakeInteger` returns, but in fact the value in the variable `fortyFour` remains unchanged. This is because **int** is a *value type*. By default, when a value type is passed to a method, a copy is passed instead of the object itself. Because they are copies, any changes made to the parameter have no effect within the calling method. Value types get their name from the fact that a copy of the object is passed instead of the object itself. The value is passed, but not the same object.

For more information on passing value types, see [Passing Value-Type Parameters \(C# Programming Guide\)](#). For a list of value

types integral to C#, see [Value Types Table \(C# Reference\)](#).

This differs from *reference types*, which are passed by reference. When an object based on a reference type is passed to a method, no copy of the object is made. Instead, a reference to the object being used as a method argument is made and passed. Changes made through this reference will therefore be reflected in the calling method. A reference type is created with the **class** keyword, like this:

C#

```
public class SampleRefType
{
    public int value;
}
```

Now, if an object based on this type is passed to a method, it will be passed by reference. For example:

C#

```
public static void TestRefType()
{
    SampleRefType rt = new SampleRefType();
    rt.value = 44;
    ModifyObject(rt);
    System.Console.WriteLine(rt.value);
}
static void ModifyObject(SampleRefType obj)
{
    obj.value = 33;
}
```

This example essentially does the same thing as the previous example. But, because a reference type is used, the modification made by `ModifyObject` is made to the object created in the `TestRefType` method. The `TestRefType` method will therefore display the value 33.

For more information, see [Passing Reference-Type Parameters \(C# Programming Guide\)](#) and [Reference Types \(C# Reference\)](#).

Return Values

Methods can return a value to the caller. If the return type, the type listed before the method name, is not **void**, then the method can return the value using the **return** keyword. A statement with the keyword **return** followed by a value that matches the return type will return that value to the method caller. The **return** keyword also stops the execution of the method. If the return type is **void**, a **return** statement with no value is still useful to stop the execution of the method. Without the **return** keyword, the method will stop executing when it reaches the end of the code block. Methods with a non-void return type are required to use the **return** keyword to return a value. For example, these two methods use the **return** keyword to return integers:

C#

```
class SimpleMath
{
    public int AddTwoNumbers(int number1, int number2)
    {
        return number1 + number2;
    }

    public int SquareANumber(int number)
    {
        return number * number;
    }
}
```

To use a value returned from a method, the calling method can use the method call itself anywhere a value of the same type would suffice. You can also assign the return value to a variable. For example, the following two code examples accomplish the same goal:

C#

```
int result = obj.AddTwoNumbers(1, 2);  
obj.SquareANumber(result);
```

C#

```
obj.SquareANumber(obj.AddTwoNumbers(1, 2));
```

Using an intermediate variable, in this case, `result`, to store a value is optional. It may help the readability of the code, or it may be necessary if the value is going to be used more than once.

For more information, see [return \(C# Reference\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.6.5 Methods](#)
- [10.5 Methods](#)

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Access Modifiers \(C# Programming Guide\)](#)

[Static Classes and Static Class Members \(C# Programming Guide\)](#)

[Inheritance \(C# Programming Guide\)](#)

[params \(C# Reference\)](#)

[return \(C# Reference\)](#)

[out \(C# Reference\)](#)

[ref \(C# Reference\)](#)

[Passing Parameters \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Passing Parameters (C# Programming Guide)

In C#, parameters can be passed either by value or by reference. Passing parameters by reference allows function members, methods, properties, indexers, operators, and constructors, to change the value of the parameters and have that change persist. To pass a parameter by reference, use the **ref** or **out** keyword. For simplicity, only the **ref** keyword is used in the examples in this topic. For more information about the difference between **ref** and **out**, see [ref](#), [out](#), and [Passing Arrays Using ref and out](#). For example:

C#

```
// Passing by value
static void Square(int x)
{
    // code...
}
```

C#

```
// Passing by reference
static void Square(ref int x)
{
    // code...
}
```

This topic includes the following sections:

- [Passing Value-Type Parameters](#)
- [Passing Reference-Type Parameters](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.5.1 Parameters
- 5.1.4 Value parameters
- 5.1.5 Reference parameters
- 5.1.6 Output parameters
- 10.5.1 Method parameters

See Also

Reference

[Methods \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Passing Value-Type Parameters (C# Programming Guide)

A [value-type](#) variable contains its data directly as opposed to a [reference-type](#) variable, which contains a reference to its data. Therefore, passing a value-type variable to a method means passing a copy of the variable to the method. Any changes to the parameter that take place inside the method have no effect on the original data stored in the variable. If you want the called method to change the value of the parameter, you have to pass it by reference, using the [ref](#) or [out](#) keyword. For simplicity, the following examples use **ref**.

Example: Passing Value Types by Value

The following example demonstrates passing value-type parameters by value. The variable `n` is passed by value to the method `SquareIt`. Any changes that take place inside the method have no effect on the original value of the variable.

```
C#  
  
class PassingValByVal  
{  
    static void SquareIt(int x)  
    // The parameter x is passed by value.  
    // Changes to x will not affect the original value of x.  
    {  
        x *= x;  
        System.Console.WriteLine("The value inside the method: {0}", x);  
    }  
    static void Main()  
    {  
        int n = 5;  
        System.Console.WriteLine("The value before calling the method: {0}", n);  
  
        SquareIt(n); // Passing the variable by value.  
        System.Console.WriteLine("The value after calling the method: {0}", n);  
    }  
}
```

Output

The value before calling the method: 5

The value inside the method: 25

The value after calling the method: 5

Code Discussion

The variable `n`, being a value type, contains its data, the value 5. When `SquareIt` is invoked, the contents of `n` are copied into the parameter `x`, which is squared inside the method. In `Main`, however, the value of `n` is the same, before and after calling the `SquareIt` method. In fact, the change that takes place inside the method only affects the local variable `x`.

Example: Passing Value Types by Reference

The following example is the same as the previous example, except for passing the parameter using the **ref** keyword. The value of the parameter is changed after calling the method.

```
C#  
  
class PassingValByRef  
{  
    static void SquareIt(ref int x)  
    // The parameter x is passed by reference.  
    // Changes to x will affect the original value of x.  
    {  
        x *= x;  
        System.Console.WriteLine("The value inside the method: {0}", x);  
    }  
    static void Main()  
    {  
        int n = 5;
```

```

        System.Console.WriteLine("The value before calling the method: {0}", n);

        SquareIt(ref n); // Passing the variable by reference.
        System.Console.WriteLine("The value after calling the method: {0}", n);
    }
}

```

Output

The value before calling the method: 5

The value inside the method: 25

The value after calling the method: 25

Code Discussion

In this example, it is not the value of `n` that is passed; rather, a reference to `n` is passed. The parameter `x` is not an `int`; it is a reference to an `int`, in this case, a reference to `n`. Therefore, when `x` is squared inside the method, what actually gets squared is what `x` refers to: `n`.

Example: Swapping Value Types

A common example of changing the values of the passed parameters is the `Swap` method, where you pass two variables, `x` and `y`, and have the method swap their contents. You must pass the parameters to the `Swap` method by reference; otherwise, you will be dealing with a local copy of the parameters inside the method. The following is an example of the `Swap` method that uses reference parameters:

C#

```

static void SwapByRef(ref int x, ref int y)
{
    int temp = x;
    x = y;
    y = temp;
}

```

When you call this method, use the **ref** keyword in the call, like this:

C#

```

static void Main()
{
    int i = 2, j = 3;
    System.Console.WriteLine("i = {0} j = {1}" , i, j);

    SwapByRef (ref i, ref j);

    System.Console.WriteLine("i = {0} j = {1}" , i, j);
}

```

Output

i = 2 j = 3

i = 3 j = 2

See Also

Reference

[Passing Parameters \(C# Programming Guide\)](#)

[Passing Reference-Type Parameters \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Passing Reference-Type Parameters (C# Programming Guide)

A variable of a [reference type](#) does not contain its data directly; it contains a reference to its data. When you pass a reference-type parameter by value, it is possible to change the data pointed to by the reference, such as the value of a class member. However, you cannot change the value of the reference itself; that is, you cannot use the same reference to allocate memory for a new class and have it persist outside the block. To do that, pass the parameter using the [ref](#) or [out](#) keyword. For simplicity, the following examples use **ref**.

Example: Passing Reference Types by Value

The following example demonstrates passing a reference-type parameter, `arr`, by value, to a method, `Change`. Because the parameter is a reference to `arr`, it is possible to change the values of the array elements. However, the attempt to reassign the parameter to a different memory location only works inside the method and does not affect the original variable, `arr`.

C#

```
class PassingRefByVal
{
    static void Change(int[] pArray)
    {
        pArray[0] = 888; // This change affects the original element.
        pArray = new int[5] {-3, -1, -2, -3, -4}; // This change is local.
        System.Console.WriteLine("Inside the method, the first element is: {0}", pArray[0])
    }

    static void Main()
    {
        int[] arr = {1, 4, 5};
        System.Console.WriteLine("Inside Main, before calling the method, the first element is: {0}", arr [0]);

        Change(arr);
        System.Console.WriteLine("Inside Main, after calling the method, the first element is: {0}", arr [0]);
    }
}
```

Output

```
Inside Main, before calling the method, the first element is: 1
Inside the method, the first element is: -3
Inside Main, after calling the method, the first element is: 888
```

Code Discussion

In the preceding example, the array, `arr`, which is a reference type, is passed to the method without the `ref` parameter. In such a case, a copy of the reference, which points to `arr`, is passed to the method. The output shows that it is possible for the method to change the contents of an array element, in this case from 1 to 888. However, allocating a new portion of memory by using the [new](#) operator inside the `Change` method makes the variable `pArray` reference a new array. Thus, any changes after that will not affect the original array, `arr`, which is created inside `Main`. In fact, two arrays are created in this example, one inside `Main` and one inside the `Change` method.

Example: Passing Reference Types by Reference

This example is the same as the previous example, except for using the **ref** keyword in the method header and call. Any changes that take place in the method will affect the original variables in the calling program.

C#

```
class PassingRefByRef
{
    static void Change(ref int[] pArray)
    {
        // Both of the following changes will affect the original variables:
```

```

        pArray[0] = 888;
        pArray = new int[5] {-3, -1, -2, -3, -4};
        System.Console.WriteLine("Inside the method, the first element is: {0}", pArray[0])
    ;
    }

    static void Main()
    {
        int[] arr = {1, 4, 5};
        System.Console.WriteLine("Inside Main, before calling the method, the first element
is: {0}", arr[0]);

        Change(ref arr);
        System.Console.WriteLine("Inside Main, after calling the method, the first element
is: {0}", arr[0]);
    }
}

```

Output

Inside Main, before calling the method, the first element is: 1

Inside the method, the first element is: -3

Inside Main, after calling the method, the first element is: -3

Code Discussion

All of the changes that take place inside the method affect the original array in `Main`. In fact, the original array is reallocated using the **new** operator. Thus, after calling the `Change` method, any reference to `arr` points to the five-element array, which is created in the `Change` method.

Example: Swapping Two Strings

Swapping strings is a good example of passing reference-type parameters by reference. In the example, two strings, `str1` and `str2`, are initialized in `Main` and passed to the `SwapStrings` method as parameters modified by the **ref** keyword. The two strings are swapped inside the method and inside `Main` as well.

C#

```

class SwappingStrings
{
    static void SwapStrings(ref string s1, ref string s2)
    // The string parameter is passed by reference.
    // Any changes on parameters will affect the original variables.
    {
        string temp = s1;
        s1 = s2;
        s2 = temp;
        System.Console.WriteLine("Inside the method: {0} {1}", s1, s2);
    }

    static void Main()
    {
        string str1 = "John";
        string str2 = "Smith";
        System.Console.WriteLine("Inside Main, before swapping: {0} {1}", str1, str2);

        SwapStrings(ref str1, ref str2);    // Passing strings by reference
        System.Console.WriteLine("Inside Main, after swapping: {0} {1}", str1, str2);
    }
}

```

Output

Inside Main, before swapping: John Smith

Inside the method: Smith John

Inside Main, after swapping: Smith John

Code Discussion

In this example, the parameters need to be passed by reference to affect the variables in the calling program. If you remove the **ref** keyword from both the method header and the method call, no changes will take place in the calling program.

For more information about strings, see [string](#).

See Also

Reference

[Passing Parameters \(C# Programming Guide\)](#)

[Passing Arrays Using ref and out \(C# Programming Guide\)](#)

[ref \(C# Reference\)](#)

[Reference Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Constructors (C# Programming Guide)

Whenever a [class](#) or [struct](#) is created, its constructor is called. A class or struct may have multiple constructors that take different arguments. Constructors allow the programmer to set default values, limit instantiation, and write code that is flexible and easy to read.

If you do not provide a constructor for your object, C# will create one by default that instantiates the object and sets any member variables to the default values listed here: [Default Values Table \(C# Reference\)](#). Static classes and structs can also have constructors.

In This Section

[Using Constructors \(C# Programming Guide\)](#)

[Instance Constructors \(C# Programming Guide\)](#)

[Private Constructors \(C# Programming Guide\)](#)

[Static Constructors \(C# Programming Guide\)](#)

[How to: Write a Copy Constructor \(C# Programming Guide\)](#)

Related Sections

[C# Programming Guide](#)

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Constructor Design](#)

[Destructors \(C# Programming Guide\)](#)

[static \(C# Reference\)](#)

Using Constructors (C# Programming Guide)

Constructors are [class](#) methods that are executed when an object of a given type is created. Constructors have the same name as the class, and usually initialize the data members of the new object.

In the following example, a class called `Taxi` is defined with a simple constructor. This class is then instantiated with the [new](#) operator. The `Taxi` constructor is invoked by the **new** operator immediately after memory is allocated for the new object.

C#

```
public class Taxi
{
    public bool isInitialized;
    public Taxi()
    {
        isInitialized = true;
    }
}

class TestTaxi
{
    static void Main()
    {
        Taxi t = new Taxi();
        System.Console.WriteLine(t.isInitialized);
    }
}
```

A constructor that takes no parameters is called a *default constructor*. Default constructors are invoked whenever an object is instantiated using the **new** operator and no arguments are provided to **new**. For more information, see [Instance Constructors](#).

Unless the class is [static](#), classes without constructors are given a public default constructor by the C# compiler in order to enable class instantiation. For more information, see [Static Classes and Static Class Members](#).

You can prevent a class from being instantiated by making the constructor private, as follows:

C#

```
class NLog
{
    // Private Constructor:
    private NLog() { }

    public static double e = System.Math.E; //2.71828...
}
```

For more information, see [Private Constructors](#).

Constructors for [struct](#) types are similar to class constructors, but **structs** cannot contain an explicit default constructor because one is provided automatically by the compiler. This constructor initializes each field in the struct to the default values shown in the [Default Values Table](#). However, this default constructor is only invoked if the struct is instantiated with **new**. For example, this code uses the default constructor for [Int32](#), so you are assured that the integer is initialized:

```
int i = new int();
Console.WriteLine(i);
```

The following code, however, results in [Compiler Error CS0165](#) because it does not use **new**, and because it attempts to use an object that has not been initialized:

```
int i;
Console.WriteLine(i);
```

Alternatively, objects based on **structs** can be initialized or assigned and then used, like this:

```
int a = 44; // Initialize the value type...
int b;
b = 33;     // Or assign it before using it.
Console.WriteLine("{0}, {1}", a, b);
```

So calling the default constructor for a value type is not required.

Both classes and **structs** can define constructors that take parameters. Constructors that take parameters must be called through a **new** statement or a [base](#) statement. Classes and **structs** can also define multiple constructors, and neither is required to define a default constructor. For example:

C#

```
public class Employee
{
    public int salary;

    public Employee(int annualSalary)
    {
        salary = annualSalary;
    }

    public Employee(int weeklySalary, int numberOfWeeks)
    {
        salary = weeklySalary * numberOfWeeks;
    }
}
```

This class can be created using either of the following statements:

C#

```
Employee e1 = new Employee(30000);
Employee e2 = new Employee(500, 52);
```

A constructor can use the **base** keyword to call the constructor of a base class. For example:

C#

```
public class Manager : Employee
{
    public Manager(int annualSalary)
        : base(annualSalary)
    {
        //Add further instructions here.
    }
}
```

In this example, the constructor for the base class is called before the block for the constructor is executed. The **base** keyword can be used with or without parameters. Any parameters to the constructor can be used as parameters to **base**, or as part of an expression. For more information, see [base](#).

In a derived class, if a base-class constructor is not called explicitly using the **base** keyword, then the default constructor, if there is one, is called implicitly. This means that the following constructor declarations are effectively the same:

C#

```
public Manager(int initialdata)
{
    //Add further instructions here.
}
```

C#

```
public Manager(int initialdata) : base()
{
    //Add further instructions here.
}
```

If a base class does not offer a default constructor, the derived class must make an explicit call to a base constructor using **base**.

A constructor can invoke another constructor in the same object using the **this** keyword. Like **base**, **this** can be used with or without parameters, and any parameters in the constructor are available as parameters to **this**, or as part of an expression. For example, the second constructor in the previous example can be rewritten using **this**:

C#

```
public Employee(int weeklySalary, int numberOfWeeks)
    : this(weeklySalary * numberOfWeeks)
{
}
```

The use of the **this** keyword above causes this constructor to be called:

C#

```
public Employee(int annualSalary)
{
    salary = annualSalary;
}
```

Constructors can be marked as [public](#), [private](#), [protected](#), [internal](#), or **protected internal**. These access modifiers define how users of the class can construct the class. For more information, see [Access Modifiers](#).

A constructor can be declared static using the [static](#) keyword. Static constructors are called automatically, immediately before any static fields are accessed, and are normally used to initialize static class members. For more information, see [Static Constructors](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.6.6.1 Constructors](#)
- [10.10 Instance Constructors \(Classes\)](#)
- [10.11 Instance Constructors \(Classes\)](#)
- [11.3.8 Constructors \(Structs\)](#)
- [11.3.10 Static Constructors \(Structs\)](#)

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Destructors \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Constructors \(C# Programming Guide\)](#)

Instance Constructors (C# Programming Guide)

Instance constructors are used to create and initialize instances. The class constructor is invoked when you create a new object, for example:

C#

```
class CoOrds
{
    public int x, y;

    // constructor
    public CoOrds()
    {
        x = 0;
        y = 0;
    }
}
```

Note

For clarity, this class contains public data members. This is not a recommended programming practice because it allows any method anywhere in a program unrestricted and unverified access to an object's inner workings. Data members should generally be private, and should be accessed only through class methods and properties.

This constructor is called whenever an object based on the `CoOrds` class is created. A constructor like this one, which takes no arguments, is called a *default constructor*. However, it is often useful to provide additional constructors. For example, we can add a constructor to the `CoOrds` class that allows us to specify the initial values for the data members:

C#

```
// A constructor with two arguments:
public CoOrds(int x, int y)
{
    this.x = x;
    this.y = y;
}
```

This allows `CoOrd` objects to be created with default or specific initial values, like this:

C#

```
CoOrds p1 = new CoOrds();
CoOrds p2 = new CoOrds(5, 3);
```

If a class does not have a default constructor, one is automatically generated and default values are used to initialize the object fields, for example, an `int` is initialized to 0. For more information on default values, see [Default Values Table \(C# Reference\)](#). Therefore, because the `CoOrds` class default constructor initializes all data members to zero, it can be removed altogether without changing how the class works. A complete example using multiple constructors is provided in Example 1 later in this topic, and an example of an automatically generated constructor is provided in Example 2.

Instance constructors can also be used to call the instance constructors of base classes. The class constructor can invoke the constructor of the base class through the initializer, as follows:

C#

```
class Circle : Shape
{
    public Circle(double radius)
        : base(radius, 0)
    {
    }
```

```
}  
}
```

In this example, the `Circle` class passes values representing radius and height to the constructor provided by `Shape` from which `Circle` is derived. A complete example using `Shape` and `Circle` appears in this topic as Example 3.

Example 1

The following example demonstrates a class with two class constructors, one without arguments and one with two arguments.

C#

```
class CoOrds  
{  
    public int x, y;  
  
    // Default constructor:  
    public CoOrds()  
    {  
        x = 0;  
        y = 0;  
    }  
  
    // A constructor with two arguments:  
    public CoOrds(int x, int y)  
    {  
        this.x = x;  
        this.y = y;  
    }  
  
    // Override the ToString method:  
    public override string ToString()  
    {  
        return (System.String.Format("{0},{1}", x, y));  
    }  
}  
  
class MainClass  
{  
    static void Main()  
    {  
        CoOrds p1 = new CoOrds();  
        CoOrds p2 = new CoOrds(5, 3);  
  
        // Display the results using the overridden ToString method:  
        System.Console.WriteLine("CoOrds #1 at {0}", p1);  
        System.Console.WriteLine("CoOrds #2 at {0}", p2);  
    }  
}
```

Output

CoOrds #1 at (0,0)

CoOrds #2 at (5,3)

Example 2

In this example, the class `Person` does not have any constructors, in which case, a default constructor is automatically provided and the fields are initialized to their default values.

C#

```
public class Person  
{  
    public int age;  
    public string name;  
}
```

```

class TestPerson
{
    static void Main()
    {
        Person p = new Person();

        System.Console.WriteLine("Name: {0}, Age: {1}", p.name, p.age);
    }
}

```

Output

Name: , Age: 0

Notice that the default value of `age` is 0 and the default value of `name` is **null**. For more information on default values, see [Default Values Table \(C# Reference\)](#).

Example 3

The following example demonstrates using the base class initializer. The `Circle` class is derived from the general class `Shape`, and the `Cylinder` class is derived from the `Circle` class. The constructor on each derived class is using its base class initializer.

C#

```

abstract class Shape
{
    public const double pi = System.Math.PI;
    protected double x, y;

    public Shape(double x, double y)
    {
        this.x = x;
        this.y = y;
    }

    public abstract double Area();
}

class Circle : Shape
{
    public Circle(double radius)
        : base(radius, 0)
    {
    }
    public override double Area()
    {
        return pi * x * x;
    }
}

class Cylinder : Circle
{
    public Cylinder(double radius, double height)
        : base(radius)
    {
        y = height;
    }

    public override double Area()
    {
        return (2 * base.Area()) + (2 * pi * x * y);
    }
}

class TestShapes
{
    static void Main()

```

```
{
    double radius = 2.5;
    double height = 3.0;

    Circle ring = new Circle(radius);
    Cylinder tube = new Cylinder(radius, height);

    System.Console.WriteLine("Area of the circle = {0:F2}", ring.Area());
    System.Console.WriteLine("Area of the cylinder = {0:F2}", tube.Area());
}
```

Output

Area of the circle = 19.63

Area of the cylinder = 86.39

For more examples on invoking the base class constructors, see [virtual \(C# Reference\)](#), [override \(C# Reference\)](#), and [base \(C# Reference\)](#).

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Destructors \(C# Programming Guide\)](#)

[static \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Constructors \(C# Programming Guide\)](#)

Private Constructors (C# Programming Guide)

A private constructor is a special instance constructor. It is commonly used in classes that contain static members only. If a class has one or more private constructors and no public constructors, then other classes (except nested classes) are not allowed to create instances of this class. For example:

C#

```
class NLog
{
    // Private Constructor:
    private NLog() { }

    public static double e = System.Math.E; //2.71828...
}
```

The declaration of the empty constructor prevents the automatic generation of a default constructor. Note that if you don't use an access modifier with the constructor it will still be private by default. However, the `private` modifier is usually used explicitly to make it clear that the class cannot be instantiated.

Private constructors are used to prevent the creation of instances of a class when there are no instance fields or methods, such as the `Math` class, or when a method is called to obtain an instance of a class. If all the methods in the class are static, consider making the entire class static. For more information see [Static Classes and Static Class Members](#).

Example

The following is an example of a class using a private constructor.

C#

```
public class Counter
{
    private Counter() { }
    public static int currentCount;
    public static int IncrementCount()
    {
        return ++currentCount;
    }
}

class TestCounter
{
    static void Main()
    {
        // If you uncomment the following statement, it will generate
        // an error because the constructor is inaccessible:
        // Counter aCounter = new Counter(); // Error

        Counter.currentCount = 100;
        Counter.IncrementCount();
        System.Console.WriteLine("New count: {0}", Counter.currentCount);
    }
}
```

Output

New count: 101

Notice that if you uncomment the following statement from the example, it will generate an error because the constructor is inaccessible due to its protection level:

C#

```
// Counter aCounter = new Counter(); // Error
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [10.10.5 Private Constructors](#)
- [25.2 Static Classes](#)

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Destructors \(C# Programming Guide\)](#)

[private \(C# Reference\)](#)

[public \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Constructors \(C# Programming Guide\)](#)

Static Constructors (C# Programming Guide)

A static constructor is used to initialize any [static](#) data, or to perform a particular action that needs performed once only. It is called automatically before the first instance is created or any static members are referenced.

C#

```
class SimpleClass
{
    // Static constructor
    static SimpleClass()
    {
        //...
    }
}
```

Static constructors have the following properties:

- A static constructor does not take access modifiers or have parameters.
- A static constructor is called automatically to initialize the [class](#) before the first instance is created or any static members are referenced.
- A static constructor cannot be called directly.
- The user has no control on when the static constructor is executed in the program.
- A typical use of static constructors is when the class is using a log file and the constructor is used to write entries to this file.
- Static constructors are also useful when creating wrapper classes for unmanaged code, when the constructor can call the **LoadLibrary** method.

Example

In this example, the class `Bus` has a static constructor and one static member, `Drive()`. When `Drive()` is called, the static constructor is invoked to initialize the class.

C#

```
public class Bus
{
    // Static constructor:
    static Bus()
    {
        System.Console.WriteLine("The static constructor invoked.");
    }

    public static void Drive()
    {
        System.Console.WriteLine("The Drive method invoked.");
    }
}

class TestBus
{
    static void Main()
    {
        Bus.Drive();
    }
}
```

Output

The static constructor invoked.

The Drive method invoked.

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Destructors \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Constructors \(C# Programming Guide\)](#)

How to: Write a Copy Constructor (C# Programming Guide)

Unlike some languages, C# does not provide a copy constructor. If you create a new object and want to copy the values from an existing object, you have to write the appropriate method yourself.

Example

In this example, the **Person** class contains a constructor that takes as the argument another object of type **Person**. The contents of the fields in this object are then assigned to the fields in the new object.

C#

```
class Person
{
    private string name;
    private int age;

    // Copy constructor.
    public Person(Person previousPerson)
    {
        name = previousPerson.name;
        age = previousPerson.age;
    }

    // Instance constructor.
    public Person(string name, int age)
    {
        this.name = name;
        this.age = age;
    }

    // Get accessor.
    public string Details
    {
        get
        {
            return name + " is " + age.ToString();
        }
    }
}

class TestPerson
{
    static void Main()
    {
        // Create a new person object.
        Person person1 = new Person("George", 40);

        // Create another new object, copying person1.
        Person person2 = new Person(person1);
        System.Console.WriteLine(person2.Details);
    }
}
```

Output

George is 40

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Destructors \(C# Programming Guide\)](#)

[ICloneable](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Constructors \(C# Programming Guide\)](#)

Destructors (C# Programming Guide)

Destructors are used to destruct instances of classes.

Remarks

- Destructors cannot be defined in structs. They are only used with classes.
- A class can only have one destructor.
- Destructors cannot be inherited or overloaded.
- Destructors cannot be called. They are invoked automatically.
- A destructor does not take modifiers or have parameters.

For example, the following is a declaration of a destructor for the class `Car`:

C#

```
class Car
{
    ~ Car() // destructor
    {
        // cleanup statements...
    }
}
```

The destructor implicitly calls [Finalize](#) on the object's base class. Therefore, the preceding destructor code is implicitly translated to:

```
protected override void Finalize()
{
    try
    {
        // cleanup statements...
    }
    finally
    {
        base.Finalize();
    }
}
```

This means the `Finalize` method is called recursively for all of the instances in the inheritance chain, from the most-derived to the least-derived.

Note

Empty destructors should not be used. When a class contains a destructor, an entry is created in the `Finalize` queue. When the destructor is called, the garbage collector is invoked to process the queue. If the destructor is empty, this simply results in a needless loss of performance.

The programmer has no control over when the destructor is called because this is determined by the garbage collector. The garbage collector checks for objects that are no longer being used by the application. If it considers an object eligible for destruction, it calls the destructor (if any) and reclaims the memory used to store the object. Destructors are also called when the program exits.

It is possible to force garbage collection by calling [Collect](#), but in most cases, this should be avoided because it may result in performance issues. For more information, see [Forcing a Garbage Collection](#).

Using Destructors to Release Resources

In general, C# does not require as much memory management as is needed when developing with a language that does not target a runtime with garbage collection. This is because the .NET Framework garbage collector implicitly manages the

allocation and release of memory for your objects. However, when your application encapsulates unmanaged resources such as windows, files, and network connections, you should use destructors to free those resources. When the object is eligible for destruction, the garbage collector runs the object's `Finalize` method.

Explicit Release of Resources

If your application is using an expensive external resource, it is also recommended that you provide a way to explicitly release the resource before the garbage collector frees the object. You do this by implementing a `Dispose` method from the [IDisposable](#) interface that performs the necessary cleanup for the object. This can considerably improve the performance of the application. Even with this explicit control over resources, the destructor becomes a safeguard to clean up resources if the call to the `Dispose` method failed.

For more details on cleaning up resources, see the following topics:

- [Cleaning Up Unmanaged Resources](#)
- [Implementing a Dispose Method](#)
- [using Statement \(C# Reference\)](#)

Example

The following example creates three classes that make a chain of inheritance. The class `First` is the base class, `Second` is derived from `First`, and `Third` is derived from `Second`. All three have destructors. In `Main()`, an instance of the most-derived class is created. When the program runs, notice that the destructors for the three classes are called automatically, and in order, from the most-derived to the least-derived.

C#

```
class First
{
    ~First()
    {
        System.Console.WriteLine("First's destructor is called");
    }
}

class Second: First
{
    ~Second()
    {
        System.Console.WriteLine("Second's destructor is called");
    }
}

class Third: Second
{
    ~Third()
    {
        System.Console.WriteLine("Third's destructor is called");
    }
}

class TestDestructors
{
    static void Main()
    {
        Third t = new Third();
    }
}
```

Output

```
Third's destructor is called
Second's destructor is called
First's destructor is called
```


For more information, see the following sections in the [C# Language Specification](#):

- [1.6.6.6 Destructors](#)
- [10.2.7.4 Member names reserved for destructors](#)
- [10.12 Destructors \(Classes\)](#)
- [11.3.9 Destructors \(Structs\)](#)

See Also

Reference

[IDisposable](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Constructors \(C# Programming Guide\)](#)

[Garbage Collection](#)

Fields (C# Programming Guide)

This topic describes *fields*, which are objects or values contained in a [class](#) or [struct](#). Fields allow classes and structures to encapsulate data.

For simplicity, these examples use fields that are **public**, but this is not recommended in practice. Fields should generally be **private**. Access to fields by external classes should be indirect, by means of methods, properties, or indexers. For more information, see [Methods](#), [Properties](#) and [Indexers](#).

Fields

Fields store the data a class needs to fulfill its design. For example, a class representing a calendar date might have three integer fields: one for the month, one for the day, and one for the year. Fields are declared within the class block by specifying the access level of the field, followed by the type of the field, followed by the name of the field. For example:

C#

```
public class CalendarDate
{
    public int month;
    public int day;
    public int year;
}
```

Accessing a field in an object is done by adding a period after the object name, followed by the name of the field, as in `objectname.fieldname`. For example:

C#

```
CalendarDate birthday = new CalendarDate();
birthday.month = 7;
```

A field can be given an initial value by using the assignment operator when the field is declared. To automatically assign the `month` field to 7, for example, you would declare `month` like this:

C#

```
public class CalendarDateWithInitialization
{
    public int month = 7;
    //...
}
```

Fields are initialized immediately before the constructor for the object instance is called, so if the constructor assigns the value of a field, it will overwrite any value given during field declaration. For more information, see [Using Constructors](#).

Note

A field initializer cannot refer to other instance fields.

Fields can be marked as [public](#), [private](#), [protected](#), [internal](#), or **protected internal**. These access modifiers define how users of the class can access the fields. For more information, see [Access Modifiers](#).

A field can optionally be declared [static](#). This makes the field available to callers at any time, even if no instance of the class exists. For more information, see [Static Classes and Static Class Members](#).

A field can be declared [readonly](#). A read-only field can only be assigned a value during initialization or in a constructor. A **static readonly** field is very similar to a constant, except that the C# compiler does not have access to the value of a static read-only field at compile time, only at run time. For more information, see [Constants](#).

For more information, see the following sections in the [C# Language Specification](#):

- [1.6.4 Fields](#)
- [10.4 Fields](#)

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Using Constructors \(C# Programming Guide\)](#)

[Inheritance \(C# Programming Guide\)](#)

[Access Modifiers \(C# Programming Guide\)](#)

[Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Constants (C# Programming Guide)

Classes and structs can declare constants as members. Constants are values which are known at compile time and do not change. (To create a constant value that is initialized at runtime, use the [readonly](#) keyword.) Constants are declared as a field, using the [const](#) keyword before the type of the field. Constants must be initialized as they are declared. For example:

C#

```
class Calendar1
{
    public const int months = 12;
}
```

In this example, the constant `months` will always be 12, and cannot be changed — even by the class itself. Constants must be of an integral type (**sbyte**, **byte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**, **char**, **float**, **double**, **decimal**, **bool**, or **string**), an enumeration, or a reference to **null**.

Multiple constants of the same type can be declared at the same time, for example:

C#

```
class Calendar2
{
    const int months = 12, weeks = 52, days = 365;
}
```

The expression used to initialize a constant can refer to another constant so long as it does not create a circular reference. For example:

C#

```
class Calendar3
{
    const int months = 12;
    const int weeks = 52;
    const int days = 365;

    const double daysPerWeek = days / weeks;
    const double daysPerMonth = days / months;
}
```

Constants can be marked as [public](#), [private](#), [protected](#), [internal](#), or **protected internal**. These access modifiers define how users of the class can access the constant. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

Constants are accessed as if they were [static](#) fields, although they cannot use the **static** keyword. Expressions that are not contained within the class defining the constant must use the class name, a period, and the name of the constant to access the constant. For example:

C#

```
int birthstones = Calendar.months;
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 10.3 Constants

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Properties \(C# Programming Guide\)](#)

[Data Types \(C# Programming Guide\)](#)

[readonly \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Nested Types (C# Programming Guide)

A type defined within a [class](#) or [struct](#) is called a nested type. For example:

C#

```
class Container
{
    class Nested
    {
        Nested() { }
    }
}
```

Regardless of whether the outer types is a class or a struct, nested types default to [private](#), but can be made [public](#), protected internal, [protected](#), [internal](#), or [private](#). In the previous example, `Nested` is inaccessible to external types, but can be made public like this:

C#

```
class Container
{
    public class Nested
    {
        Nested() { }
    }
}
```

The nested, or inner type can access the containing, or outer type. To access the containing type, pass it as a constructor to the nested type. For example:

C#

```
public class Container
{
    public class Nested
    {
        private Container m_parent;

        public Nested()
        {
        }
        public Nested(Container parent)
        {
            m_parent = parent;
        }
    }
}
```

Nested types can access private and protected members of the containing type, including any inherited private or protected members.

In the previous declaration, the full name of class `Nested` is `Container.Nested`. This is the name used to create a new instance of the nested class, as follows:

C#

```
Container.Nested nest = new Container.Nested();
```

See Also
Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Access Modifiers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Constructors \(C# Programming Guide\)](#)

Access Modifiers (C# Programming Guide)

Classes and structs can be restricted so that only the program or namespace they are declared in may use them. Class members can be restricted so that only derived classes can use them, or restricted so that only classes within the current namespace or program can use them. Access modifiers are keywords added to the class, struct, or member declaration to specify these restrictions. Those keywords are [public](#), [private](#), [protected](#), and [internal](#). For example:

C#

```
public class Bicycle
{
    public void Pedal() { }
}
```

Class and Struct Accessibility

Classes and structs that are not nested within other classes or structs can be either public or internal. A type declared as public is accessible by any other type. A type declared as internal is only accessible by types within the same assembly. Classes and structs are declared as internal by default unless the keyword **public** is added to the class definition, as in the previous example. Class or struct definitions can add the **internal** keyword to make their access level explicit. Access modifiers do not affect the class or struct itself — it always has access to itself and all of its own members.

Class and Struct Member Accessibility

Class or struct members can be declared with one of five types of access. They can be public or internal, like the classes and structs themselves. A class member can be declared as protected using the **protected** keyword, meaning that only derived types using the class as a base can access the member. By combining the **protected** and **internal** keywords, a class member can be marked protected internal — only derived types or types within the same assembly can access that member. Finally, a class or struct member can be declared as private with the **private** keyword, indicating that only the class or struct declaring the member is allowed access to that member.

To set the access level for a class or struct member, add the appropriate keyword to the member declaration. Some examples:

C#

```
// public class:
public class Tricycle
{
    // protected method:
    protected void Pedal() { }

    // private field:
    private int m_wheels = 3;

    // protected internal property:
    protected internal int Wheels
    {
        get { return m_wheels; }
    }
}
```

Other Types

Interfaces, like classes, can be declared as public or internal types. Unlike classes, interfaces default to internal access. Interface members are always public, and no access modifiers can be applied.

Namespaces and enumeration members are always public, and no access modifiers can be applied.

Delegates have internal access by default.

Any types declared within a namespace or at the top level of a compilation unit (for example, not within a namespace, class, or struct) are internal by default, but can be made public.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [10.2.3 Access Modifiers](#)

See Also

Reference

[Objects, Classes and Structs \(C# Programming Guide\)](#)

[Interfaces \(C# Programming Guide\)](#)

[private \(C# Reference\)](#)

[public \(C# Reference\)](#)

[internal \(C# Reference\)](#)

[protected \(C# Reference\)](#)

[class \(C# Reference\)](#)

[struct \(C# Reference\)](#)

[interface \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Partial Class Definitions (C# Programming Guide)

It is possible to split the definition of a [class](#) or a [struct](#), or an [interface](#) over two or more source files. Each source file contains a section of the class definition, and all parts are combined when the application is compiled. There are several situations when splitting a class definition is desirable:

- When working on large projects, spreading a class over separate files allows multiple programmers to work on it simultaneously.
- When working with automatically generated source, code can be added to the class without having to recreate the source file. Visual Studio uses this approach when creating Windows Forms, Web Service wrapper code, and so on. You can create code that uses these classes without having to edit the file created by Visual Studio.
- To split a class definition, use the [partial](#) keyword modifier, as shown below:

C#

```
public partial class Employee
{
    public void DoWork()
    {
    }
}

public partial class Employee
{
    public void GoToLunch()
    {
    }
}
```

Remarks

Using the **partial** keyword indicates that other parts of the class, struct, or interface can be defined within the namespace. All the parts must use the **partial** keyword. All of the parts must be available at compile time to form the final type. All the parts must have the same accessibility, such as public, private, and so on.

If any of the parts are declared abstract, then the entire type is considered abstract. If any of the parts are declared sealed, then the entire type is considered sealed. If any of the parts declare a base type, then the entire type inherits that class.

All of the parts that specify a base class must agree, but parts that omit a base class still inherit the base type. Parts can specify different base interfaces, and the final type implements all of the interfaces listed by all of the partial declarations. Any class, struct, or interface members declared in a partial definition are available to all of the other parts. The final type is the combination of all the parts at compile time.

Note

The partial modifier is not available on delegate or enumeration declarations.

Nested types can be partial, even if the type they are nested within is not partial itself. For example:

C#

```
class Container
{
    partial class Nested
    {
        void Test() { }
    }
    partial class Nested
    {
        void Test2() { }
    }
}
```

- At compile time, attributes of partial-type definitions are merged. For example, the following declarations:

C#

```
[System.SerializableAttribute]
partial class Moon { }

[System.ObsoleteAttribute]
partial class Moon { }
```

are equivalent to:

C#

```
[System.SerializableAttribute]
[System.ObsoleteAttribute]
class Moon { }
```

- The following are merged together from all the partial-type definitions:
- XML comments
- interfaces
- generic-type parameter attributes
- class attributes
- members

For example, the following declarations:

C#

```
partial class Earth : Planet, IRotate { }
partial class Earth : IRevolve { }
```

are equivalent to:

C#

```
class Earth : Planet, IRotate, IRevolve { }
```

Restrictions

There are several rules to follow when working with partial class definitions:

- All partial-type definitions meant to be parts of the same type must be modified with **partial**. For example, the following class declarations generate an error:

C#

```
public partial class A { }
//public class A { } // Error, must also be marked partial
```

- The **partial** modifier can only appear immediately before the keywords **class**, **struct**, or **interface**.
- Nested partial types are allowed in partial-type definitions, for example:

C#

```

partial class ClassWithNestedClass
{
    partial class NestedClass { }
}

partial class ClassWithNestedClass
{
    partial class NestedClass { }
}

```

- All partial-type definitions meant to be parts of the same type must be defined in the same assembly and the same module (.exe or .dll file). Partial definitions cannot span multiple modules.
- The class name and generic-type parameters must match on all partial-type definitions. Generic types can be partial. Each partial declaration must use the same parameter names in the same order.
- The following keywords on a partial-type definition are optional, but if present on one partial-type definition, cannot conflict with the keywords specified on another partial definition for the same type:
 - `public`
 - `private`
 - `protected`
 - `internal`
 - `abstract`
 - `sealed`
 - base class
 - `new` modifier (nested parts)
 - generic constraints (For more information, see [Constraints on Type Parameters \(C# Programming Guide\)](#).)

Example 1

In the following example, the fields and the constructor of the class, `CoOrds`, are declared in one partial class definition, while the member, `PrintCoOrds`, is declared in another partial class definition.

C#

```

public partial class CoOrds
{
    private int x;
    private int y;

    public CoOrds(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}

public partial class CoOrds
{
    public void PrintCoOrds()
    {
        System.Console.WriteLine("CoOrds: {0},{1}", x, y);
    }
}

class TestCoOrds
{
    static void Main()

```

```
{  
    CoOrds myCoOrds = new CoOrds(10, 15);  
    myCoOrds.PrintCoOrds();  
}
```

Output

CoOrds: 10,15

Example 2

The following example shows that you can also develop partial structs and interfaces.

C#

```
partial interface ITest  
{  
    void Interface_Test();  
}  
  
partial interface ITest  
{  
    void Interface_Test2();  
}  
  
partial struct S1  
{  
    void Struct_Test() { }  
}  
  
partial struct S1  
{  
    void Struct_Test2() { }  
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 23 Partial Types

See Also

Reference

[Interfaces \(C# Programming Guide\)](#)

[partial \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Classes \(C# Programming Guide\)](#)

[Structs \(C# Programming Guide\)](#)

Static Classes and Static Class Members (C# Programming Guide)

Static classes and class members are used to create data and functions that can be accessed without creating an instance of the class. Static class members can be used to separate data and behavior that is independent of any object identity: the data and functions do not change regardless of what happens to the object. Static classes can be used when there is no data or behavior in the class that depends on object identity.

Static Classes

A class can be declared [static](#), indicating that it contains only static members. It is not possible to create instances of a static class using the [new](#) keyword. Static classes are loaded automatically by the .NET Framework common language runtime (CLR) when the program or namespace containing the class is loaded.

Use a static class to contain methods that are not associated with a particular object. For example, it is a common requirement to create a set of methods that do not act on instance data and are not associated to a specific object in your code. You could use a static class to hold those methods.

The main features of a static class are:

- They only contain static members.
- They cannot be instantiated.
- They are sealed.
- They cannot contain [Instance Constructors \(C# Programming Guide\)](#).

Creating a static class is therefore much the same as creating a class that contains only static members and a private constructor. A private constructor prevents the class from being instantiated.

The advantage of using a static class is that the compiler can check to make sure that no instance members are accidentally added. The compiler will guarantee that instances of this class cannot be created.

Static classes are sealed and therefore cannot be inherited. Static classes cannot contain a constructor, although it is still possible to declare a static constructor to assign initial values or set up some static state. For more information, see [Static Constructors \(C# Programming Guide\)](#).

When to Use Static Classes

Suppose you have a class `CompanyInfo` that contains the following methods to get information about the company name and address.

C#

```
class CompanyInfo
{
    public string GetCompanyName() { return "CompanyName"; }
    public string GetCompanyAddress() { return "CompanyAddress"; }
    //...
}
```

These methods do not need to be attached to a specific instance of the class. Therefore, instead of creating unnecessary instances of this class, you can declare it as a static class, like this:

C#

```
static class CompanyInfo
{
    public static string GetCompanyName() { return "CompanyName"; }
    public static string GetCompanyAddress() { return "CompanyAddress"; }
    //...
}
```

Use a static class as a unit of organization for methods not associated with particular objects. Also, a static class can make your implementation simpler and faster because you do not have to create an object in order to call its methods. It is useful to organize the methods inside the class in a meaningful way, such as the methods of the [Math](#) class in the [System](#) namespace.

Static Members

A static method, field, property, or event is callable on a class even when no instance of the class has been created. If any instances of the class are created, they cannot be used to access the static member. Only one copy of static fields and events exists, and static methods and properties can only access static fields and static events. Static members are often used to represent data or calculations that do not change in response to object state; for instance, a math library might contain static methods for calculating sine and cosine.

Static class members are declared using the **static** keyword before the return type of the member, for example:

C#

```
public class Automobile
{
    public static int NumberOfWheels = 4;
    public static int SizeOfGasTank
    {
        get
        {
            return 15;
        }
    }
    public static void Drive() { }
    public static event EventType RunOutOfGas;

    //other non-static fields and properties...
}
```

Static members are initialized before the static member is accessed for the first time, and before the static constructor, if any is called. To access a static class member, use the name of the class instead of a variable name to specify the location of the member. For example:

C#

```
Automobile.Drive();
int i = Automobile.NumberOfWheels;
```

Example

Here is an example of a static class that contains two methods that convert temperature from Celsius to Fahrenheit and vice versa:

C#

```
public static class TemperatureConverter
{
    public static double CelsiusToFahrenheit(string temperatureCelsius)
    {
        // Convert argument to double for calculations.
        double celsius = System.Double.Parse(temperatureCelsius);

        // Convert Celsius to Fahrenheit.
        double fahrenheit = (celsius * 9 / 5) + 32;

        return fahrenheit;
    }

    public static double FahrenheitToCelsius(string temperatureFahrenheit)
    {
        // Convert argument to double for calculations.
        double fahrenheit = System.Double.Parse(temperatureFahrenheit);

        // Convert Fahrenheit to Celsius.
    }
}
```

```

        double celsius = (fahrenheit - 32) * 5 / 9;

        return celsius;
    }
}

class TestTemperatureConverter
{
    static void Main()
    {
        System.Console.WriteLine("Please select the convertor direction");
        System.Console.WriteLine("1. From Celsius to Fahrenheit.");
        System.Console.WriteLine("2. From Fahrenheit to Celsius.");
        System.Console.Write(":");

        string selection = System.Console.ReadLine();
        double F, C = 0;

        switch (selection)
        {
            case "1":
                System.Console.Write("Please enter the Celsius temperature: ");
                F = TemperatureConverter.CelsiusToFahrenheit(System.Console.ReadLine());
                System.Console.WriteLine("Temperature in Fahrenheit: {0:F2}", F);
                break;

            case "2":
                System.Console.Write("Please enter the Fahrenheit temperature: ");
                C = TemperatureConverter.FahrenheitToCelsius(System.Console.ReadLine());
                System.Console.WriteLine("Temperature in Celsius: {0:F2}", C);
                break;

            default:
                System.Console.WriteLine("Please select a convertor.");
                break;
        }
    }
}

```

Input

2
98.6

Sample Output:

```

Please select the convertor
1. From Celsius to Fahrenheit.
2. From Fahrenheit to Celsius.
:2
Please enter the Fahrenheit temperature: 98.6
Temperature in Celsius: 37.00

```

Additional sample output might look as follows:

```

Please select the convertor
1. From Celsius to Fahrenheit.
2. From Fahrenheit to Celsius.
:1
Please enter the Celsius temperature: 37.00
Temperature in Fahrenheit: 98.60

```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [25.2 Static Classes](#)

See Also

Reference

[class \(C# Reference\)](#)

[Instance Constructors \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Static Class Design](#)

[Classes \(C# Programming Guide\)](#)

How to: Know the Difference Between Passing a Struct and Passing a Class Reference to a Method (C# Programming Guide)

This example shows that when a [struct](#) is passed to a method, a copy of the struct is passed, but when a [class](#) instance is passed, a reference is passed.

The output of the following example shows that only the value of the class field is changed when the class instance is passed to the `ClassTaker` method. The struct field, however, does not change by passing its instance to the `StructTaker` method. This is because a copy of the struct is passed to the `StructTaker` method, while a reference to the class is passed to the `ClassTaker` method.

Example

```
C#  
  
class TheClass  
{  
    public string willIChange;  
}  
  
struct TheStruct  
{  
    public string willIChange;  
}  
  
class TestClassAndStruct  
{  
    static void ClassTaker(TheClass c)  
    {  
        c.willIChange = "Changed";  
    }  
  
    static void StructTaker(TheStruct s)  
    {  
        s.willIChange = "Changed";  
    }  
  
    static void Main()  
    {  
        TheClass testClass = new TheClass();  
        TheStruct testStruct = new TheStruct();  
  
        testClass.willIChange = "Not Changed";  
        testStruct.willIChange = "Not Changed";  
  
        ClassTaker(testClass);  
        StructTaker(testStruct);  
  
        System.Console.WriteLine("Class field = {0}", testClass.willIChange);  
        System.Console.WriteLine("Struct field = {0}", testStruct.willIChange);  
    }  
}
```

Output

Class field = Changed Struct field = Not Changed

See Also

Reference

[Passing Parameters \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Classes \(C# Programming Guide\)](#)

[Structs \(C# Programming Guide\)](#)

Properties (C# Programming Guide)

Properties are members that provide a flexible mechanism to read, write, or compute the values of private fields. Properties can be used as though they are public data members, but they are actually special methods called *accessors*. This enables data to be accessed easily while still providing the safety and flexibility of methods.

In this example, the class `TimePeriod` stores a time period. Internally the class stores the time in seconds, but a property called `Hours` is provided that allows a client to specify a time in hours. The accessors for the `Hours` property perform the conversion between hours and seconds.

Example

C#

```
class TimePeriod
{
    private double seconds;

    public double Hours
    {
        get { return seconds / 3600; }
        set { seconds = value * 3600; }
    }
}

class Program
{
    static void Main()
    {
        TimePeriod t = new TimePeriod();

        // Assigning the Hours property causes the 'set' accessor to be called.
        t.Hours = 24;

        // Evaluating the Hours property causes the 'get' accessor to be called.
        System.Console.WriteLine("Time in hours: " + t.Hours);
    }
}
```

Output

Time in hours: 24

Properties Overview

- Properties enable a class to expose a public way of getting and setting values, while hiding implementation or verification code.
- A [get](#) property accessor is used to return the property value, and a [set](#) accessor is used to assign a new value. These accessors can have different access levels. For more information, see [Accessor Accessibility](#).
- The [value](#) keyword is used to define the value being assigned by the **set** indexer.
- Properties that do not implement a **set** method are read only.

Related Sections

- [Using Properties \(C# Programming Guide\)](#)
- [Interface Properties \(C# Programming Guide\)](#)
- [Comparison Between Properties and Indexers \(C# Programming Guide\)](#)
- [Asymmetric Accessor Accessibility \(C# Programming Guide\)](#)
- [Properties Sample](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.6.6.2 Properties](#)
- [10.2.7.1 Member names reserved for properties](#)
- [10.6 Properties](#)

See Also

Reference

[Using Properties \(C# Programming Guide\)](#)

[Indexers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Property Design](#)

Using Properties (C# Programming Guide)

Properties combine aspects of both fields and methods. To the user of an object, a property appears to be a field, accessing the property requires exactly the same syntax. To the implementer of a class, a property is one or two code blocks, representing a [get](#) accessor and/or a [set](#) accessor. The code block for the **get** accessor is executed when the property is read; the code block for the **set** accessor is executed when the property is assigned a new value. A property without a **set** accessor is considered read-only. A property without a **get** accessor is considered write-only. A property with both accessors is read-write.

Unlike fields, properties are not classified as variables. Therefore, it is not possible to pass a property as a [ref \(C# Reference\)](#) or [out \(C# Reference\)](#) parameter.

Properties have many uses: they can validate data before allowing a change; they can transparently expose data on a class where that data is actually retrieved from some other source, such as a database; they can take an action when data is changed, such as raising an event, or changing the value of other fields.

Properties are declared within the class block by specifying the access level of the field, followed by the type of the property, followed by the name of the property, then a code block declaring a **get**-accessor and/or a **set** accessor. For example:

C#

```
public class Date
{
    private int month = 7; // "backing store"

    public int Month
    {
        get
        {
            return month;
        }
        set
        {
            if ((value > 0) && (value < 13))
            {
                month = value;
            }
        }
    }
}
```

In this example, `Month` is declared as a property so that the **set** accessor can make sure the `Month` value is set between 1 and 12. The `Month` property uses a private field to track the actual value. The real location of a property's data is often referred to as the property's "backing store." It is common for properties to use private fields as a backing store. The field is marked private in order to make sure it can only be changed by calling the property. For more information about public and private access restrictions, see [Access Modifiers \(C# Programming Guide\)](#).

The get Accessor

The body of the **get** accessor is similar to that of a method. It must return a value of the property type. The execution of the **get** accessor is equivalent to reading the value of the field. For example, when you are returning the private variable from the **get** accessor and optimizations are enabled, the call to the **get** accessor method is inlined by the compiler so there is no method-call overhead. However, a virtual **get** accessor method can not be inlined since the compiler does not know at compile-time which method may actually be called at run time. The following is a **get** accessor that returns the value of a private field `name`:

C#

```
class Person
{
    private string name; // the name field
    public string Name   // the Name property
    {
        get
        {
            return name;
        }
    }
}
```

```
}  
}
```

When you reference the property, except as the target of an assignment, the **get** accessor is invoked to read the value of the property. For example:

C#

```
Person p1 = new Person();  
//...  
System.Console.WriteLine(p1.Name); // the get accessor is invoked here
```

The **get** accessor must terminate in a **return** or **throw** statement, and control cannot flow off the accessor body.

It is a bad programming style to change the state of the object by using the **get** accessor. For example, the following accessor produces the side effect of changing the state of the object each time the `number` field is accessed.

C#

```
private int number;  
public int Number  
{  
    get  
    {  
        return number++; // Don't do this  
    }  
}
```

The **get** accessor can either be used to return the field value or to compute it and return it. For example:

C#

```
class Employee  
{  
    private string name;  
    public string Name  
    {  
        get  
        {  
            return name != null ? name : "NA";  
        }  
    }  
}
```

In the preceding code segment, if you do not assign a value to the `Name` property, it will return the value NA.

The set Accessor

The **set** accessor is similar to a method whose return type is **void**. It uses an implicit parameter called *value*, whose type is the type of the property. In the following example, a **set** accessor is added to the `Name` property:

C#

```
class Person  
{  
    private string name; // the name field  
    public string Name // the Name property  
    {  
        get  
        {  
            return name;  
        }  
        set  
    }  
}
```

```

        {
            name = value;
        }
    }
}

```

When you assign a value to the property, the **set** accessor is invoked with an argument that provides the new value. For example:

C#

```

Person p1 = new Person();
p1.Name = "Joe"; // the set accessor is invoked here

System.Console.Write(p1.Name); // the get accessor is invoked here

```

It is an error to use the implicit parameter name, *value*, for a local variable declaration in a **set** accessor.

Remarks

Properties can be marked as **public**, **private**, **protected**, **internal**, or **protected internal**. These access modifiers define how users of the class can access the property. The **get** and **set** accessors for the same property may have different access modifiers. For example, the **get** may be **public** to allow read-only access from outside the type, and the **set** may be **private** or **protected**. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

A property may be declared as a static property using the **static** keyword. This makes the property available to callers at any time, even if no instance of the class exists. For more information, see [Static Classes and Static Class Members \(C# Programming Guide\)](#).

A property may be marked as a virtual property using the **virtual** keyword. This allows derived classes to override the property behavior using the **override** keyword. For more information about these options, see [Inheritance \(C# Programming Guide\)](#).

A property overriding a virtual property can also be **sealed**, specifying that for derived classes it is no longer virtual. Lastly, a property can be declared **abstract**, meaning there is no implementation in the class, and derived classes must write their own implementation. For more information about these options, see [Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#).

Note

It is an error to use a [virtual \(C# Reference\)](#), [abstract \(C# Reference\)](#), or [override \(C# Reference\)](#) modifier on an accessor of a **static** property.

Example 1

This example demonstrates instance, static, and read-only properties. It accepts the name of the employee from the keyboard, increments `NumberOfEmployees` by 1, and displays the Employee name and number.

C#

```

public class Employee
{
    public static int NumberOfEmployees;
    private static int counter;
    private string name;

    // A read-write instance property:
    public string Name
    {
        get { return name; }
        set { name = value; }
    }

    // A read-only static property:
    public static int Counter
    {
        get { return counter; }
    }
}

```



```

    }

    // A Constructor:
    public Employee()
    {
        // Calculate the employee's number:
        counter = ++counter + NumberOfEmployees;
    }
}

class TestEmployee
{
    static void Main()
    {
        Employee.NumberOfEmployees = 100;
        Employee e1 = new Employee();
        e1.Name = "Claude Vige";

        System.Console.WriteLine("Employee number: {0}", Employee.Counter);
        System.Console.WriteLine("Employee name: {0}", e1.Name);
    }
}

```

Output 1

Employee number: 101

Employee name: Claude Vige

Example 2

This example demonstrates how to access a property in a base class that is hidden by another property with the same name in a derived class.

C#

```

public class Employee
{
    private string name;
    public string Name
    {
        get { return name; }
        set { name = value; }
    }
}

public class Manager : Employee
{
    private string name;

    // Notice the use of the new modifier:
    public new string Name
    {
        get { return name; }
        set { name = value + ", Manager"; }
    }
}

class TestHiding
{
    static void Main()
    {
        Manager m1 = new Manager();

        // Derived class property.
        m1.Name = "John";

        // Base class property.
        ((Employee)m1).Name = "Mary";
    }
}

```

```

        System.Console.WriteLine("Name in the derived class is: {0}", m1.Name);
        System.Console.WriteLine("Name in the base class is: {0}", ((Employee)m1).Name);
    }
}

```

Output 2

Name in the derived class is: John, Manager

Name in the base class is: Mary

Code Discussion

The following are important points in the preceding example:

- The property `Name` in the derived class hides the property `Name` in the base class. In such a case, the **new** modifier is used in the declaration of the property in the derived class:

C#

```
public new string Name
```

- The cast `(Employee)` is used to access the hidden property in the base class:

C#

```
((Employee)m1).Name = "Mary";
```

For more information on hiding members, see the [new Modifier \(C# Reference\)](#).

Example 3

In this example, two classes, `Cube` and `Square`, implement an abstract class, `Shape`, and override its abstract `Area` property. Note the use of the [override](#) modifier on the properties. The program accepts the side as an input and calculates the areas for the square and cube. It also accepts the area as an input and calculates the corresponding side for the square and cube.

C#

```

abstract class Shape
{
    public abstract double Area
    {
        get;
        set;
    }
}

class Square : Shape
{
    public double side;

    public Square(double s) //constructor
    {
        side = s;
    }

    public override double Area
    {
        get
        {
            return side * side;
        }
        set
        {

```

```

        side = System.Math.Sqrt(value);
    }
}

class Cube : Shape
{
    public double side;

    public Cube(double s)
    {
        side = s;
    }

    public override double Area
    {
        get
        {
            return 6 * side * side;
        }
        set
        {
            side = System.Math.Sqrt(value / 6);
        }
    }
}

class TestShapes
{
    static void Main()
    {
        // Input the side:
        System.Console.Write("Enter the side: ");
        double side = double.Parse(System.Console.ReadLine());

        // Compute the areas:
        Square s = new Square(side);
        Cube c = new Cube(side);

        // Display the results:
        System.Console.WriteLine("Area of the square = {0:F2}", s.Area);
        System.Console.WriteLine("Area of the cube = {0:F2}", c.Area);
        System.Console.WriteLine();

        // Input the area:
        System.Console.Write("Enter the area: ");
        double area = double.Parse(System.Console.ReadLine());

        // Compute the sides:
        s.Area = area;
        c.Area = area;

        // Display the results:
        System.Console.WriteLine("Side of the square = {0:F2}", s.side);
        System.Console.WriteLine("Side of the cube = {0:F2}", c.side);
    }
}

```

Input

4
24

Output 3

Enter the side: 4

Area of the square = 16.00

Area of the cube = 96.00

Enter the area: 24

Side of the square = 4.90

Side of the cube = 2.00

See Also

Tasks

[Properties Sample](#)

Reference

[Properties \(C# Programming Guide\)](#)

[Interface Properties \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Interface Properties (C# Programming Guide)

Properties can be declared on an [interface \(C# Reference\)](#). The following is an example of an interface indexer accessor:

C#

```
public interface ISampleInterface
{
    // Property declaration:
    string Name
    {
        get;
        set;
    }
}
```

The accessor of an interface property does not have a body. Thus, the purpose of the accessors is to indicate whether the property is read-write, read-only, or write-only.

Example

In this example, the interface `IEmployee` has a read-write property, `Name`, and a read-only property, `Counter`. The class `Employee` implements the `IEmployee` interface and uses these two properties. The program reads the name of a new employee and the current number of employees and displays the employee name and the computed employee number.

You could use the fully qualified name of the property, which references the interface in which the member is declared. For example:

C#

```
string IEmployee.Name
{
    get { return "Employee Name"; }
    set { }
}
```

This is called [Explicit Interface Implementation \(C# Programming Guide\)](#). For example, if the class `Employee` is implementing two interfaces `ICitizen` and `IEmployee` and both interfaces have the `Name` property, the explicit interface member implementation will be necessary. That is, the following property declaration:

C#

```
string IEmployee.Name
{
    get { return "Employee Name"; }
    set { }
}
```

implements the `Name` property on the `IEmployee` interface, while the following declaration:

C#

```
string ICitizen.Name
{
    get { return "Citizen Name"; }
    set { }
}
```

implements the `Name` property on the `ICitizen` interface.

C#

```

interface IEmployee
{
    string Name
    {
        get;
        set;
    }

    int Counter
    {
        get;
    }
}

public class Employee : IEmployee
{
    public static int numberOfEmployees;

    private string name;
    public string Name // read-write instance property
    {
        get
        {
            return name;
        }
        set
        {
            name = value;
        }
    }

    private int counter;
    public int Counter // read-only instance property
    {
        get
        {
            return counter;
        }
    }

    public Employee() // constructor
    {
        counter = ++counter + numberOfEmployees;
    }
}

class TestEmployee
{
    static void Main()
    {
        System.Console.Write("Enter number of employees: ");
        Employee.numberOfEmployees = int.Parse(System.Console.ReadLine());

        Employee e1 = new Employee();
        System.Console.Write("Enter the name of the new employee: ");
        e1.Name = System.Console.ReadLine();

        System.Console.WriteLine("The employee information:");
        System.Console.WriteLine("Employee number: {0}", e1.Counter);
        System.Console.WriteLine("Employee name: {0}", e1.Name);
    }
}

```

Input

210 Hazem Abolrous

Sample Output

Enter number of employees: 210

Enter the name of the new employee: Hazem Abolrous

The employee information:

Employee number: 211

Employee name: Hazem Abolrous

See Also

Reference

[Properties \(C# Programming Guide\)](#)

[Using Properties \(C# Programming Guide\)](#)

[Comparison Between Properties and Indexers \(C# Programming Guide\)](#)

[Indexers \(C# Programming Guide\)](#)

[Interfaces \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Asymmetric Accessor Accessibility (C# Programming Guide)

The [get](#) and [set](#) portions of a property or indexer are called *accessors*. By default these accessors have the same visibility, or access level: that of the property or indexer to which they belong. For more information, see [accessibility levels](#). However, it is sometimes useful to restrict access to one of these accessors. Typically, this involves restricting the accessibility of the **set** accessor, while keeping the **get** accessor publicly accessible. For example:

C#

```
public string Name
{
    get
    {
        return name;
    }
    protected set
    {
        name = value;
    }
}
```

In this example, a property called `Name` defines a **get** and **set** accessor. The **get** accessor receives the accessibility level of the property itself, **public** in this case, while the **set** accessor is explicitly restricted by applying the [protected](#) access modifier to the accessor itself.

Restrictions on Access Modifiers on Accessors

Using the accessor modifiers on properties or indexers is subject to these conditions:

- You cannot use accessor modifiers on an interface or an explicit [interface](#) member implementation.
- You can use accessor modifiers only if the property or indexer has both **set** and **get** accessors. In this case, the modifier is permitted on one only of the two accessors.
- If the property or indexer has an [override](#) modifier, the accessor modifier must match the accessor of the overridden accessor, if any.
- The accessibility level on the accessor must be more restrictive than the accessibility level on the property or indexer itself.

Access Modifiers on Overriding Accessors

When you override a property or indexer, the overridden accessors must be accessible to the overriding code. Also, the accessibility level of both the property/indexer, and that of the accessors must match the corresponding overridden property/indexer and the accessors. For example:

C#

```
public class Parent
{
    public virtual int TestProperty
    {
        // Notice the accessor accessibility level.
        protected set { }

        // No access modifier is used here.
        get { return 0; }
    }
}
public class Kid : Parent
{
    public override int TestProperty
    {
        // Use the same accessibility level as in the overridden accessor.
        protected set { }
    }
}
```



```

        // Cannot use access modifier here.
        get { return 0; }
    }
}

```

Implementing Interfaces

When you use an accessor to implement an interface, the accessor may not have an access modifier. However, if you implement the interface using one accessor, such as **get**, the other accessor can have an access modifier, as in the following example:

C#

```

public interface ISomeInterface
{
    int TestProperty
    {
        // No access modifier allowed here
        // because this is an interface.
        get;
    }
}

public class TestClass : ISomeInterface
{
    public int TestProperty
    {
        // Cannot use access modifier here because
        // this is an interface implementation.
        get { return 10; }

        // Interface property does not have set accessor,
        // so access modifier is allowed.
        protected set { }
    }
}

```

Accessor Accessibility Domain

If you use an access modifier on the accessor, the [accessibility domain](#) of the accessor is determined by this modifier.

If you did not use an access modifier on the accessor, the accessibility domain of the accessor is determined by the accessibility level of the property or indexer.

Example

The following example contains three classes, `BaseClass`, `DerivedClass`, and `MainClass`. There are two properties on the `BaseClass`, `Name` and `Id` on both classes. The example demonstrates how the property `Id` on `DerivedClass` can be hidden by the property `Id` on `BaseClass` when you use a restrictive access modifier such as `protected` or `private`. Therefore, when you assign values to this property, the property on the `BaseClass` class is called instead. Replacing the access modifier by `public` will make the property accessible.

The example also demonstrates that a restrictive access modifier, such as **private** or **protected**, on the **set** accessor of the `Name` property in `DerivedClass` prevents access to the accessor and generates an error when you assign to it.

C#

```

public class BaseClass
{
    private string name = "Name-BaseClass";
    private string id = "ID-BaseClass";

    public string Name
    {
        get { return name; }
        set { }
    }
}

```

```

    }

    public string Id
    {
        get { return id; }
        set { }
    }
}

public class DerivedClass : BaseClass
{
    private string name = "Name-DerivedClass";
    private string id = "ID-DerivedClass";

    new public string Name
    {
        get
        {
            return name;
        }

        // Using "protected" would make the set accessor not accessible.
        set
        {
            name = value;
        }
    }

    // Using private on the following property hides it in the Main Class.
    // Any assignment to the property will use Id in BaseClass.
    new private string Id
    {
        get
        {
            return id;
        }
        set
        {
            id = value;
        }
    }
}

class MainClass
{
    static void Main()
    {
        BaseClass b1 = new BaseClass();
        DerivedClass d1 = new DerivedClass();

        b1.Name = "Mary";
        d1.Name = "John";

        b1.Id = "Mary123";
        d1.Id = "John123"; // The BaseClass.Id property is called.

        System.Console.WriteLine("Base: {0}, {1}", b1.Name, b1.Id);
        System.Console.WriteLine("Derived: {0}, {1}", d1.Name, d1.Id);
    }
}

```

Output

Name and ID in the base class: Name-BaseClass, ID-BaseClass

Name and ID in the derived class: John, ID-BaseClass

Comments

Notice that if you replace the declaration `new private string Id` by `new public string Id`, you get the output:

Name and ID in the base class: Name-BaseClass, ID-BaseClass

Name and ID in the derived class: John, John123

See Also

Reference

[Properties \(C# Programming Guide\)](#)

[Indexers \(C# Programming Guide\)](#)

[Access Modifiers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

How to: Declare and Use Read/Write Properties (C# Programming Guide)

Properties provide the convenience of public data members without the risks that come with unprotected, uncontrolled, and unverified access to an object's data. This is accomplished through *accessors*: special methods that assign and retrieve values from the underlying data member. The [set](#) accessor enables data members to be assigned, and the [get](#) accessor retrieves data member values.

This sample shows a `Person` class that has two properties: `Name` (string) and `Age` (int). Both properties provide **get** and **set** accessors, so they are considered read/write properties.

Example

C#

```
class Person
{
    private string m_name = "N/A";
    private int m_Age = 0;

    // Declare a Name property of type string:
    public string Name
    {
        get
        {
            return m_name;
        }
        set
        {
            m_name = value;
        }
    }

    // Declare an Age property of type int:
    public int Age
    {
        get
        {
            return m_Age;
        }
        set
        {
            m_Age = value;
        }
    }

    public override string ToString()
    {
        return "Name = " + Name + ", Age = " + Age;
    }
}

class TestPerson
{
    static void Main()
    {
        // Create a new Person object:
        Person person = new Person();

        // Print out the name and the age associated with the person:
        System.Console.WriteLine("Person details - {0}", person);

        // Set some values on the person object:
        person.Name = "Joe";
    }
}
```

```

        person.Age = 99;
        System.Console.WriteLine("Person details - {0}", person);

        // Increment the Age property:
        person.Age += 1;
        System.Console.WriteLine("Person details - {0}", person);
    }
}

```

Output

Person details - Name = N/A, Age = 0
Person details - Name = Joe, Age = 99
Person details - Name = Joe, Age = 100

Robust Programming

In the previous example, the `Name` and `Age` properties are **public** and include both a **get** and a **set** accessor. This allows any object to read and write these properties. It is sometimes desirable, however, to exclude one of the accessors. Omitting the **set** accessor, for example, makes the property read-only:

C#

```

public string Name
{
    get
    {
        return m_name;
    }
}

```

Alternatively, you can expose one accessor publicly but make the other private or protected. For more information, see [Asymmetric Accessor Accessibility](#).

Once the properties are declared, they can be used as if they were fields of the class. This allows for a very natural syntax when both getting and setting the value of a property, as in the following statements:

C#

```

person.Name = "Joe";
person.Age = 99;

```

Note that in a property **set** method a special `value` variable is available. This variable contains the value that the user specified, for example:

C#

```

m_name = value;

```

Notice the clean syntax for incrementing the `Age` property on a `Person` object:

C#

```

person.Age += 1;

```

If separate **set** and **get** methods were used to model properties, the equivalent code might look like this:

```

person.SetAge(person.GetAge() + 1);

```

The **ToString** method is overridden in this example:

C#

```
public override string ToString()
{
    return "Name = " + Name + ", Age = " + Age;
}
```

Notice that **ToString** is not explicitly used in the program. It is invoked by default by the **WriteLine** calls.

See Also

Reference

[Properties \(C# Programming Guide\)](#)

[Objects, Classes and Structs \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Indexers (C# Programming Guide)

Indexers permit instances of a class or struct to be indexed in the same way as arrays. Indexers are similar to [properties](#) except that their accessors take parameters.

In the following example, a generic class is defined and provided with simple [get](#) and [set](#) accessor methods as a means for assigning and retrieving values. The class `Program` creates an instance of this class for storing strings.

```
C#  
  
class SampleCollection<T>  
{  
    private T[] arr = new T[100];  
    public T this[int i]  
    {  
        get  
        {  
            return arr[i];  
        }  
        set  
        {  
            arr[i] = value;  
        }  
    }  
}  
  
// This class shows how client code uses the indexer  
class Program  
{  
    static void Main(string[] args)  
    {  
        SampleCollection<string> stringCollection = new SampleCollection<string>();  
        stringCollection[0] = "Hello, World";  
        System.Console.WriteLine(stringCollection[0]);  
    }  
}
```

Indexers Overview

- Indexers enable objects to be indexed in a similar way to arrays.
- A **get** accessor returns a value. A **set** accessor assigns a value.
- The `this` keyword is used to define the indexers.
- The `value` keyword is used to define the value being assigned by the **set** indexer.
- Indexers do not have to be indexed by an integer value; it is up to you how to define the specific look-up mechanism.
- Indexers can be overloaded.
- Indexers can have more than one formal parameter, for example, when accessing a two-dimensional array.

Related Sections

- [Using Indexers \(C# Programming Guide\)](#)
- [Indexers in Interfaces \(C# Programming Guide\)](#)
- [Comparison Between Properties and Indexers \(C# Programming Guide\)](#)
- [Asymmetric Accessor Accessibility \(C# Programming Guide\)](#)
- [Indexers Sample](#)
- [Indexed Properties Sample](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.6.3 Indexers
- 10.2.7.3 Member names reserved for indexers
- 10.8 Indexers
- 13.2.4 Interface indexers

See Also

Reference

[Properties \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Using Indexers (C# Programming Guide)

Indexers allow you to index a [class](#), [struct](#), or [interface](#) in the same way as an array. For more information about using indexers with an interface, see [Interface Indexers](#).

To declare an indexer on a class or struct, use the [this](#) keyword, as in this example:

```
public int this[int index]    // Indexer declaration
{
    // get and set accessors
}
```

Remarks

The type of an indexer and the type of its parameters must be at least as accessible as the indexer itself. For more information about accessibility levels, see [Access Modifiers](#).

The signature of an indexer consists of the number and types of its formal parameters. It does not include the indexer type or the names of the formal parameters. If you declare more than one indexer in the same class, they must have different signatures.

An indexer value is not classified as a variable; therefore, it is not possible to pass an indexer value as a [ref](#) or [out](#) parameter.

To provide the indexer with a name that other languages can use, use a `name` attribute in the declaration. For example:

```
[System.Runtime.CompilerServices.IndexerName("TheItem")]
public int this [int index]    // Indexer declaration
{
}
```

This indexer will have the name `TheItem`. Not providing the name attribute would make `Item` the default name.

Example 1

The following example shows how to declare a private array field, `arr`, and an indexer. Using the indexer enables direct access to the instance `test[i]`. The alternative to using the indexer is to declare the array as a [public](#) member and access its members, `arr[i]`, directly.

C#

```
class TempRecord
{
    // Array of temperature values
    private float[] temps = new float[10] { 56.2F, 56.7F, 56.5F, 56.9F, 58.8F,
                                             61.3F, 65.9F, 62.1F, 59.2F, 57.5F };

    // Auto-Implemented Property
    System.DateTime date { get; set; }
    // To enable client code to validate input
    // when accessing your indexer.
    public int Length
    {
        get { return temps.Length; }
    }
    // Indexer declaration.
    // Input parameter is validated by client
    // code before being passed to the indexer.
    public float this[int index]
    {
        get
        {
            return temps[index];
        }

        set
        {
        }
    }
}
```

```

        temps[index] = value;
    }
}

class MainClass
{
    static void Main()
    {
        TempRecord tempRecord = new TempRecord();
        // Use the indexer's set accessor
        tempRecord[3] = 58.3F;
        tempRecord[5] = 60.1F;

        // Use the indexer's get accessor
        for (int i = 0; i < 10; i++)
        {
            // This example validates the input on the client side. You may
            // choose to validate it in the class that implements the indexer, and throw an
            // exception or return an error code in the case of invalid input.
            if (i < tempRecord.Length)
            {
                System.Console.WriteLine("Element #{0} = {1}", i, tempRecord[i]);
            }
            else
            {
                System.Console.WriteLine("Index value of {0} is out of range", i);
            }
        }

        //Uncomment this code to see how the .NET Framework handles indexer exceptions
        //try
        //{
        //    System.Console.WriteLine("Element #{0} = {1}", tempRecord[tempRecord.Length])
        //}
        //catch (System.ArgumentOutOfRangeException e)
        //{
        //    System.Console.WriteLine(e);
        //}
    }
}

```

Output

```

Element #0 = 0
Element #1 = 0
Element #2 = 0
Element #3 = 256
Element #4 = 0
Element #5 = 1024
Element #6 = 0
Element #7 = 0
Element #8 = 0
Element #9 = 0
Element #10 = 0

```

Notice that when an indexer's access is evaluated, for example, in a **Console.Write** statement, the [get](#) accessor is invoked. Therefore, if no **get** accessor exists, a compile-time error occurs.

Indexing Using Other Values

C# does not limit the index type to integer. For example, it may be useful to use a string with an indexer. Such an indexer might

be implemented by searching for the string within the collection, and returning the appropriate value. As accessors can be overloaded, the string and integer versions can co-exist.

Example 2

In this example, a class is declared that stores the days of the week. A **get** accessor is declared that takes a string, the name of a day, and returns the corresponding integer. For example, Sunday will return 0, Monday will return 1, and so on.

C#

```
// Using a string as an indexer value
class DayCollection
{
    string[] days = { "Sun", "Mon", "Tues", "Wed", "Thurs", "Fri", "Sat" };

    // This method finds the day or returns -1
    private int GetDay(string testDay)
    {
        int i = 0;
        foreach (string day in days)
        {
            if (day == testDay)
            {
                return i;
            }
            i++;
        }
        return -1;
    }

    // The get accessor returns an integer for a given string
    public int this[string day]
    {
        get
        {
            return (GetDay(day));
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        DayCollection week = new DayCollection();
        System.Console.WriteLine(week["Fri"]);
        System.Console.WriteLine(week["Made-up Day"]);
    }
}
```

Output

5
-1

Robust Programming

There are two main ways in which the security and reliability of indexers can be improved:

- Always ensure that your code performs range and type checks when setting and retrieving values from any buffer or array accessed by the indexers.
- Set the accessibility of the **get** and **set** accessors to be as restrictive as is reasonable. This is important for the **set** accessor in particular. For more information, see [Asymmetric Accessor Accessibility \(C# Programming Guide\)](#).

See Also

Tasks

[Indexers Sample](#)

Reference[Indexers \(C# Programming Guide\)](#)[Properties \(C# Programming Guide\)](#)**Concepts**[C# Programming Guide](#)

Indexers in Interfaces (C# Programming Guide)

Indexers can be declared on an [interface \(C# Reference\)](#). Accessors of interface indexers differ from the accessors of [class](#) indexers in the following ways:

- Interface accessors do not use modifiers.
- An interface accessor does not have a body.

Thus, the purpose of the accessor is to indicate whether the indexer is read-write, read-only, or write-only.

The following is an example of an interface indexer accessor:

C#

```
public interface ISomeInterface
{
    //...

    // Indexer declaration:
    string this[int index]
    {
        get;
        set;
    }
}
```

The signature of an indexer must differ from the signatures of all other indexers declared in the same interface.

Example

The following example shows how to implement interface indexers.

C#

```
// Indexer on an interface:
public interface ISomeInterface
{
    // Indexer declaration:
    int this[int index]
    {
        get;
        set;
    }
}

// Implementing the interface.
class IndexerClass : ISomeInterface
{
    private int[] arr = new int[100];
    public int this[int index] // indexer declaration
    {
        get
        {
            // Check the index limits.
            if (index < 0 || index >= 100)
            {
                return 0;
            }
            else
            {
                return arr[index];
            }
        }
        set
        {

```

```

        if (!(index < 0 || index >= 100))
        {
            arr[index] = value;
        }
    }
}

class MainClass
{
    static void Main()
    {
        IndexerClass test = new IndexerClass();
        // Call the indexer to initialize the elements #2 and #5.
        test[2] = 4;
        test[5] = 32;
        for (int i = 0; i <= 10; i++)
        {
            System.Console.WriteLine("Element #{0} = {1}", i, test[i]);
        }
    }
}

```

Output

```

Element #0 = 0
Element #1 = 0
Element #2 = 4
Element #3 = 0
Element #4 = 0
Element #5 = 32
Element #6 = 0
Element #7 = 0
Element #8 = 0
Element #9 = 0
Element #10 = 0

```

In the preceding example, you could use the explicit interface member implementation by using the fully qualified name of the interface member. For example:

```

public string ISomeInterface.this
{
}

```

However, the fully qualified name is only needed to avoid ambiguity when the class is implementing more than one interface with the same indexer signature. For example, if an `Employee` class is implementing two interfaces, `ICitizen` and `IEmployee`, and both interfaces have the same indexer signature, the explicit interface member implementation is necessary. That is, the following indexer declaration:

```

public string IEmployee.this
{
}

```

implements the indexer on the `IEmployee` interface, while the following declaration:

```

public string ICitizen.this
{
}

```

implements the indexer on the `ICitizen` interface.

See Also

Tasks

[Indexers Sample](#)

Reference

[Indexers \(C# Programming Guide\)](#)

[Properties \(C# Programming Guide\)](#)

[Interfaces \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Comparison Between Properties and Indexers (C# Programming Guide)

Indexers are similar to properties. Except for the differences shown in the following table, all of the rules defined for property accessors apply to indexer accessors as well.

Property	Indexer
Allows methods to be called as though they were public data members.	Allows methods on an object to be called as though the object is an array.
Accessed through a simple name.	Accessed through an index.
Can be a static or an instance member.	Must be an instance member.
A get accessor of a property has no parameters.	A get accessor of an indexer has the same formal parameter list as the indexer.
A set accessor of a property contains the implicit value parameter.	A set accessor of an indexer has the same formal parameter list as the indexer, in addition to the value parameter.

See Also

Reference

[Indexers \(C# Programming Guide\)](#)

[Properties \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Delegates (C# Programming Guide)

A [delegate](#) is a type that references a method. Once a delegate is assigned a method, it behaves exactly like that method. The delegate method can be used like any other method, with parameters and a return value, as in this example:

C#

```
public delegate int PerformCalculation(int x, int y);
```

Any method that matches the delegate's signature, which consists of the return type and parameters, can be assigned to the delegate. This makes it possible to programmatically change method calls, and also plug new code into existing classes. As long as you know the delegate's signature, you can assign your own delegated method.

This ability to refer to a method as a parameter makes delegates ideal for defining callback methods. For example, a sort algorithm could be passed a reference to the method that compares two objects. Separating the comparison code allows the algorithm to be written in a more general way.

Delegates Overview

Delegates have the following properties:

- Delegates are similar to C++ function pointers, but are type safe.
- Delegates allow methods to be passed as parameters.
- Delegates can be used to define callback methods.
- Delegates can be chained together; for example, multiple methods can be called on a single event.
- Methods don't need to match the delegate signature exactly. For more information, see [Covariance and Contravariance](#)
- C# version 2.0 introduces the concept of [Anonymous Methods](#), which permit code blocks to be passed as parameters in place of a separately defined method.

In This Section

- [Overview of Delegates](#)
- [When to Use Delegates Instead of Interfaces](#)
- [Named Methods](#)
- [Anonymous Methods](#)
- [Covariance and Contravariance](#)
- [How to: Combine Delegates](#)
- [How to: Declare, Instantiate, and Use a Delegate](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.11 Delegates
- 4.2.6 Delegate types
- 7.5.5.2 Delegate invocations
- 15 Delegates

See Also

Reference

[Delegate](#)

Concepts

[C# Programming Guide](#)

Using Delegates (C# Programming Guide)

A [delegate](#) is a type that safely encapsulates a method, similar to a function pointer in C and C++. Unlike C function pointers, delegates are object-oriented, type safe, and secure. The type of a delegate is defined by the name of the delegate. The following example declares a delegate named `Del` that can encapsulate a method that takes a [string](#) as an argument and returns `void`:

C#

```
public delegate void Del(string message);
```

A delegate object is normally constructed by providing the name of the method the delegate will wrap, or with an [anonymous Method](#). Once a delegate is instantiated, a method call made to the delegate will be passed by the delegate to that method. The parameters passed to the delegate by the caller are passed to the method, and the return value, if any, from the method is returned to the caller by the delegate. This is known as invoking the delegate. An instantiated delegate can be invoked as if it were the wrapped method itself. For example:

C#

```
// Create a method for a delegate.
public static void DelegateMethod(string message)
{
    System.Console.WriteLine(message);
}
```

C#

```
// Instantiate the delegate.
Del handler = DelegateMethod;

// Call the delegate.
handler("Hello World");
```

Delegate types are derived from the [Delegate](#) class in the .NET Framework. Delegate types are [sealed](#)—they cannot be derived from—and it is not possible to derive custom classes from **Delegate**. Because the instantiated delegate is an object, it can be passed as a parameter, or assigned to a property. This allows a method to accept a delegate as a parameter, and call the delegate at some later time. This is known as an asynchronous callback, and is a common method of notifying a caller when a long process has completed. When a delegate is used in this fashion, the code using the delegate does not need any knowledge of the implementation of the method being used. The functionality is similar to the encapsulation interfaces provide. For more information, see [When to Use Delegates Instead of Interfaces](#).

Another common use of callbacks is defining a custom comparison method and passing that delegate to a sort method. It allows the caller's code to become part of the sort algorithm. The following example method uses the `Del` type as a parameter:

C#

```
public void MethodWithCallback(int param1, int param2, Del callback)
{
    callback("The number is: " + (param1 + param2).ToString());
}
```

You can then pass the delegate created above to that method:

C#

```
MethodWithCallback(1, 2, handler);
```

and receive the following output to the console:

The number is: 3

Using the delegate as an abstraction, `MethodWithCallback` does not need to call the console directly—it does not have to be designed with a console in mind. What `MethodWithCallback` does is simply prepare a string and pass the string to another method. This is especially powerful since a delegated method can use any number of parameters.

When a delegate is constructed to wrap an instance method, the delegate references both the instance and the method. A delegate has no knowledge of the instance type aside from the method it wraps, so a delegate can refer to any type of object as long as there is a method on that object that matches the delegate signature. When a delegate is constructed to wrap a static method, it only references the method. Consider the following declarations:

C#

```
public class MethodClass
{
    public void Method1(string message) { }
    public void Method2(string message) { }
}
```

Along with the static `DelegateMethod` shown previously, we now have three methods that can be wrapped by a `Del` instance.

A delegate can call more than one method when invoked. This is referred to as multicasting. To add an extra method to the delegate's list of methods—the invocation list—simply requires adding two delegates using the addition or addition assignment operators ('+' or '+='). For example:

C#

```
MethodClass obj = new MethodClass();
Del d1 = obj.Method1;
Del d2 = obj.Method2;
Del d3 = DelegateMethod;

//Both types of assignment are valid.
Del allMethodsDelegate = d1 + d2;
allMethodsDelegate += d3;
```

At this point `allMethodsDelegate` contains three methods in its invocation list—`Method1`, `Method2`, and `DelegateMethod`. The original three delegates, `d1`, `d2`, and `d3`, remain unchanged. When `allMethodsDelegate` is invoked, all three methods are called in order. If the delegate uses reference parameters, the reference is passed sequentially to each of the three methods in turn, and any changes by one method are visible to the next method. When any of the methods throws an exception that is not caught within the method, that exception is passed to the caller of the delegate and no subsequent methods in the invocation list are called. If the delegate has a return value and/or out parameters, it returns the return value and parameters of the last method invoked. To remove a method from the invocation list, use the decrement or decrement assignment operator ('-' or '-='). For example:

C#

```
//remove Method1
allMethodsDelegate -= d1;

// copy AllMethodsDelegate while removing d2
Del oneMethodDelegate = allMethodsDelegate - d2;
```

Because delegate types are derived from **System.Delegate**, the methods and properties defined by that class can be called on the delegate. For example, to find the number of methods in a delegate's invocation list, you may write:

C#

```
int invocationCount = d1.GetInvocationList().GetLength(0);
```

Delegates with more than one method in their invocation list derive from `MulticastDelegate`, which is a subclass of **System.Delegate**. The above code works in either case because both classes support **GetInvocationList**.

Multicast delegates are used extensively in event handling. Event source objects send event notifications to recipient objects that have registered to receive that event. To register for an event, the recipient creates a method designed to handle the event, then creates a delegate for that method and passes the delegate to the event source. The source calls the delegate when the event occurs. The delegate then calls the event handling method on the recipient, delivering the event data. The delegate type for a given event is defined by the event source. For more, see [Events \(C# Programming Guide\)](#).

Comparing delegates of two different types assigned at compile-time will result in a compilation error. If the delegate instances are statically of the type **System.Delegate**, then the comparison is allowed, but will return false at run time. For example:

C#

```
delegate void Delegate1();
delegate void Delegate2();

static void method(Delegate1 d, Delegate2 e, System.Delegate f)
{
    // Compile-time error.
    //Console.WriteLine(d == e);

    // OK at compile-time. False if the run-time type of f
    //is not the same as that of d.
    System.Console.WriteLine(d == f);
}
```

See Also

Reference

[Covariance and Contravariance in Delegates \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Delegates \(C# Programming Guide\)](#)

[Events \(C# Programming Guide\)](#)

Named Methods (C# Programming Guide)

A [delegate](#) can be associated with a named method. When you instantiate a delegate using a named method, the method is passed as a parameter, for example:

C#

```
// Declare a delegate:
delegate void Del(int x);

// Define a named method:
void DoWork(int k) { /* ... */ }

// Instantiate the delegate using the method as a parameter:
Del d = obj.DoWork;
```

This is called using a named method. Delegates constructed with a named method can encapsulate either a [static](#) method or an instance method. Using named methods is the only way to instantiate a delegate in previous versions of C#. However, in a situation where creating a new method is undesirable overhead, C# 2.0 allows you to instantiate a delegate and specify a code block immediately that the delegate will process when called. These are called [Anonymous Methods \(C# Programming Guide\)](#).

Remarks

The method that you pass as a delegate parameter must have the same signature as the delegate declaration.

A delegate instance may encapsulate either static or instance method.

Although the delegate can use an [out](#) parameter, it is not recommended to use it with multicast event delegates because there is no way to know which delegate will be called.

Example 1

The following is a simple example of declaring and using a delegate. Notice that both the delegate, `Del`, and the associated method, `MultiplyNumbers`, have the same signature

C#

```
// Declare a delegate
delegate void Del(int i, double j);

class MathClass
{
    static void Main()
    {
        MathClass m = new MathClass();

        // Delegate instantiation using "MultiplyNumbers"
        Del d = m.MultiplyNumbers;

        // Invoke the delegate object.
        System.Console.WriteLine("Invoking the delegate using 'MultiplyNumbers':");
        for (int i = 1; i <= 5; i++)
        {
            d(i, 2);
        }
    }

    // Declare the associated method.
    void MultiplyNumbers(int m, double n)
    {
        System.Console.Write(m * n + " ");
    }
}
```

Output

Invoking the delegate using 'MultiplyNumbers':

2 4 6 8 10

Example 2

In the following example, one delegate is mapped to both static and instance methods and returns specific information from each.

C#

```
// Declare a delegate
delegate void Del();

class SampleClass
{
    public void InstanceMethod()
    {
        System.Console.WriteLine("A message from the instance method.");
    }

    static public void StaticMethod()
    {
        System.Console.WriteLine("A message from the static method.");
    }
}

class TestSampleClass
{
    static void Main()
    {
        SampleClass sc = new SampleClass();

        // Map the delegate to the instance method:
        Del d = sc.InstanceMethod;
        d();

        // Map to the static method:
        d = SampleClass.StaticMethod;
        d();
    }
}
```

Output

A message from the instance method.

A message from the static method.

See Also

Tasks

[How to: Combine Delegates \(Multicast Delegates\)\(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Delegates \(C# Programming Guide\)](#)

[Events \(C# Programming Guide\)](#)

Anonymous Methods (C# Programming Guide)

In versions of C# previous to 2.0, the only way to declare a [delegate](#) was to use [named methods](#). C# 2.0 introduces anonymous methods.

Creating anonymous methods is essentially a way to pass a code block as a delegate parameter. For example:

C#

```
// Create a handler for a click event
button1.Click += delegate(System.Object o, System.EventArgs e)
    { System.Windows.Forms.MessageBox.Show("Click!"); };
```

or

C#

```
// Create a delegate instance
delegate void Del(int x);

// Instantiate the delegate using an anonymous method
Del d = delegate(int k) { /* ... */ };
```

By using anonymous methods, you reduce the coding overhead in instantiating delegates by eliminating the need to create a separate method.

For example, specifying a code block in the place of a delegate can be useful in a situation when having to create a method might seem an unnecessary overhead. A good example would be when launching a new thread. This class creates a thread and also contains the code that the thread executes, without the need for creating an additional method for the delegate.

C#

```
void StartThread()
{
    System.Threading.Thread t1 = new System.Threading.Thread
        (delegate()
        {
            System.Console.Write("Hello, ");
            System.Console.WriteLine("World!");
        });
    t1.Start();
}
```

Remarks

The scope of the parameters of an anonymous method is the *anonymous-method-block*.

It is an error to have a jump statement, such as [goto](#), [break](#), or [continue](#), inside the anonymous method block whose target is outside the block. It is also an error to have a jump statement, such as **goto**, **break**, or **continue**, outside the anonymous method block whose target is inside the block.

The local variables and parameters whose scope contain an anonymous method declaration are called outer or captured variables of the anonymous method. For example, in the following code segment, *n* is an outer variable:

C#

```
int n = 0;
Del d = delegate() { System.Console.WriteLine("Copy #{0}", ++n); };
```

Unlike local variables, the lifetime of the outer variable extends until the delegates that reference the anonymous methods are eligible for garbage collection. A reference to *n* is captured at the time the delegate is created.

An anonymous method cannot access the [ref](#) or [out](#) parameters of an outer scope.

No unsafe code can be accessed within the *anonymous-method-block*.

Example

The following example demonstrates the two ways of instantiating a delegate:

- Associating the delegate with an anonymous method.
- Associating the delegate with a named method (`DoWork`).

In each case, a message is displayed when the delegate is invoked.

C#

```
// Declare a delegate
delegate void Printer(string s);

class TestClass
{
    static void Main()
    {
        // Instantiate the delegate type using an anonymous method:
        Printer p = delegate(string j)
        {
            System.Console.WriteLine(j);
        };

        // Results from the anonymous delegate call:
        p("The delegate using the anonymous method is called.");

        // The delegate instantiation using a named method "DoWork":
        p = new Printer(TestClass.DoWork);

        // Results from the old style delegate call:
        p("The delegate using the named method is called.");
    }

    // The method associated with the named delegate:
    static void DoWork(string k)
    {
        System.Console.WriteLine(k);
    }
}
```

Output

The delegate using the anonymous method is called.

The delegate using the named method is called.

See Also

Reference

[Methods \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Delegates \(C# Programming Guide\)](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

[Named Methods \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

When to Use Delegates Instead of Interfaces (C# Programming Guide)

Both delegates and interfaces allow a class designer to separate type declarations and implementation. A given [interface](#) can be inherited and implemented by any [class](#) or [struct](#); a [delegate](#) can be created for a method on any class, as long as the method fits the method signature for the delegate. An interface reference or a delegate can be used by an object with no knowledge of the class that implements the interface or delegate method. Given these similarities, when should a class designer use a delegate and when should they use an interface?

Use a delegate when:

- An eventing design pattern is used.
- It is desirable to encapsulate a static method.
- The caller has no need access other properties, methods, or interfaces on the object implementing the method.
- Easy composition is desired.
- A class may need more than one implementation of the method.

Use an interface when:

- There are a group of related methods that may be called.
- A class only needs one implementation of the method.
- The class using the interface will want to cast that interface to other interface or class types.
- The method being implemented is linked to the type or identity of the class: for example, comparison methods.

One good example of using a single-method interface instead of a delegate is [IComparable](#) or [IComparable](#). **IComparable** declares the [CompareTo](#) method, which returns an integer specifying a less than, equal to, or greater than relationship between two objects of the same type. **IComparable** can be used as the basis of a sort algorithm, and while using a delegate comparison method as the basis of a sort algorithm would be valid, it is not ideal. Because the ability to compare belongs to the class, and the comparison algorithm doesn't change at run-time, a single-method interface is ideal.

See Also

Reference

[Methods \(C# Programming Guide\)](#)

[Interfaces \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

Covariance and Contravariance in Delegates (C# Programming Guide)

Covariance and *contravariance* provide a degree of flexibility when matching method signatures with [delegate](#) types. Covariance permits a method to have a more derived return type than what is defined in the delegate. Contravariance permits a method with parameter types that are less derived than in the delegate type.

Example 1 (Covariance)

This example demonstrates how delegates can be used with methods that have return types that are derived from the return type in the delegate signature. The data type returned by `SecondHandler` is of type `Dogs`, which derives from the `Mammals` type that is defined in the delegate.

C#

```
class Mammals
{
}

class Dogs : Mammals
{
}

class Program
{
    // Define the delegate.
    public delegate Mammals HandlerMethod();

    public static Mammals FirstHandler()
    {
        return null;
    }

    public static Dogs SecondHandler()
    {
        return null;
    }

    static void Main()
    {
        HandlerMethod handler1 = FirstHandler;

        // Covariance allows this delegate.
        HandlerMethod handler2 = SecondHandler;
    }
}
```

Example 2 (Contravariance)

This example demonstrates how delegates can be used with methods that have parameters of a type that are base types of the delegate signature parameter type. With contravariance, you can now use one event handler in places where, previously, you would have had to use separate handlers. For example, you can now create an event handler that accepts an `EventArgs` input parameter and use it with the `Button.MouseClick` event that sends a `MouseEventArgs` type as a parameter, and also with `TextBox.KeyDown` event that sends a `KeyEventArgs` parameter.

C#

```
System.DateTime lastActivity;
public Form1()
{
    InitializeComponent();

    lastActivity = new System.DateTime();
    this.textBox1.KeyDown += this.MultiHandler; //works with KeyEventArgs
}
```

```
    this.button1.MouseClick += this.MultiHandler; //works with MouseEventArgs
}

// Event handler for any event with an EventArgs or
// derived class in the second parameter
private void MultiHandler(object sender, System.EventArgs e)
{
    lastActivity = System.DateTime.Now;
}
```

See Also

Reference

[Generic Delegates \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

[Delegates \(C# Programming Guide\)](#)

How to: Combine Delegates (Multicast Delegates)(C# Programming Guide)

This example demonstrates how to compose multicast delegates. A useful property of [delegate](#) objects is that they can be assigned to one delegate instance to be multicast using the + operator. A composed delegate calls the two delegates it was composed from. Only delegates of the same type can be composed.

The - operator can be used to remove a component delegate from a composed delegate.

Example

C#

```
delegate void Del(string s);

class TestClass
{
    static void Hello(string s)
    {
        System.Console.WriteLine(" Hello, {0}!", s);
    }

    static void Goodbye(string s)
    {
        System.Console.WriteLine(" Goodbye, {0}!", s);
    }

    static void Main()
    {
        Del a, b, c, d;

        // Create the delegate object a that references
        // the method Hello:
        a = Hello;

        // Create the delegate object b that references
        // the method Goodbye:
        b = Goodbye;

        // The two delegates, a and b, are composed to form c:
        c = a + b;

        // Remove a from the composed delegate, leaving d,
        // which calls only the method Goodbye:
        d = c - a;

        System.Console.WriteLine("Invoking delegate a:");
        a("A");
        System.Console.WriteLine("Invoking delegate b:");
        b("B");
        System.Console.WriteLine("Invoking delegate c:");
        c("C");
        System.Console.WriteLine("Invoking delegate d:");
        d("D");
    }
}
```

Output

```
Invoking delegate a:  
    Hello, A!  
Invoking delegate b:  
    Goodbye, B!  
Invoking delegate c:  
    Hello, C!  
    Goodbye, C!  
Invoking delegate d:  
    Goodbye, D!
```

See Also

Reference

[MulticastDelegate](#)

Concepts

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

How to: Declare, Instantiate, and Use a Delegate (C# Programming Guide)

Delegates are declared as shown here:

C#

```
public delegate void Del<T>(T item);  
public void Notify(int i) { }
```

C#

```
Del<int> d1 = new Del<int>(Notify);
```

In C# 2.0, it is also possible to declare a [delegate](#) using this simplified syntax:

C#

```
Del<int> d2 = Notify;
```

The following example illustrates declaring, instantiating, and using a delegate. The `BookDB` class encapsulates a bookstore database that maintains a database of books. It exposes a method, `ProcessPaperbackBooks`, which finds all paperback books in the database and calls a delegate for each one. The **delegate** type used is called `ProcessBookDelegate`. The `Test` class uses this class to print out the titles and average price of the paperback books.

The use of delegates promotes good separation of functionality between the bookstore database and the client code. The client code has no knowledge of how the books are stored or how the bookstore code finds paperback books. The bookstore code has no knowledge of what processing is done on the paperback books after it finds them.

Example

C#

```
// A set of classes for handling a bookstore:  
namespace Bookstore  
{  
    using System.Collections;  
  
    // Describes a book in the book list:  
    public struct Book  
    {  
        public string Title;           // Title of the book.  
        public string Author;         // Author of the book.  
        public decimal Price;         // Price of the book.  
        public bool Paperback;         // Is it paperback?  
  
        public Book(string title, string author, decimal price, bool paperBack)  
        {  
            Title = title;  
            Author = author;  
            Price = price;  
            Paperback = paperBack;  
        }  
    }  
  
    // Declare a delegate type for processing a book:  
    public delegate void ProcessBookDelegate(Book book);  
  
    // Maintains a book database.  
    public class BookDB  
    {  
        // List of all books in the database:
```

```

        ArrayList list = new ArrayList();

        // Add a book to the database:
        public void AddBook(string title, string author, decimal price, bool paperBack)
        {
            list.Add(new Book(title, author, price, paperBack));
        }

        // Call a passed-in delegate on each paperback book to process it:
        public void ProcessPaperbackBooks(ProcessBookDelegate processBook)
        {
            foreach (Book b in list)
            {
                if (b.Paperback)
                {
                    // Calling the delegate:
                    processBook(b);
                }
            }
        }
    }
}

```

```

// Using the Bookstore classes:
namespace BookTestClient
{
    using Bookstore;

    // Class to total and average prices of books:
    class PriceTaller
    {
        int countBooks = 0;
        decimal priceBooks = 0.0m;

        internal void AddBookToTotal(Book book)
        {
            countBooks += 1;
            priceBooks += book.Price;
        }

        internal decimal AveragePrice()
        {
            return priceBooks / countBooks;
        }
    }

    // Class to test the book database:
    class TestBookDB
    {
        // Print the title of the book.
        static void PrintTitle(Book b)
        {
            System.Console.WriteLine("    {0}", b.Title);
        }

        // Execution starts here.
        static void Main()
        {
            BookDB bookDB = new BookDB();

            // Initialize the database with some books:
            AddBooks(bookDB);

            // Print all the titles of paperbacks:
            System.Console.WriteLine("Paperback Book Titles:");

            // Create a new delegate object associated with the static
            // method Test.PrintTitle:
            bookDB.ProcessPaperbackBooks(PrintTitle);
        }
    }
}

```



```

// Get the average price of a paperback by using
// a PriceTallier object:
PriceTallier totaller = new PriceTallier();

// Create a new delegate object associated with the nonstatic
// method AddBookToTotal on the object totaller:
bookDB.ProcessPaperbackBooks(totaller.AddBookToTotal);

System.Console.WriteLine("Average Paperback Book Price: ${0:#.##}",
    totaller.AveragePrice());
}

// Initialize the book database with some test books:
static void AddBooks(BookDB bookDB)
{
    bookDB.AddBook("The C Programming Language", "Brian W. Kernighan and Dennis M.
Ritchie", 19.95m, true);
    bookDB.AddBook("The Unicode Standard 2.0", "The Unicode Consortium", 39.95m, tr
ue);
    bookDB.AddBook("The MS-DOS Encyclopedia", "Ray Duncan", 129.95m, false);
    bookDB.AddBook("Dogbert's Clues for the Clueless", "Scott Adams", 12.00m, true)
;
}
}
}

```

Output

```

Paperback Book Titles:
  The C Programming Language
  The Unicode Standard 2.0
  Dogbert's Clues for the Clueless
Average Paperback Book Price: $23.97

```

Robust Programming

- Declaring a delegate.

The following statement:

C#

```
public delegate void ProcessBookDelegate(Book book);
```

declares a new delegate type. Each delegate type describes the number and types of the arguments, and the type of the return value of methods that it can encapsulate. Whenever a new set of argument types or return value type is needed, a new delegate type must be declared.

- Instantiating a delegate.

Once a delegate type has been declared, a delegate object must be created and associated with a particular method. In the example above, this is done by passing the `PrintTitle` method to the `ProcessPaperbackBooks` method, like this:

C#

```
bookDB.ProcessPaperbackBooks(PrintTitle);
```

This creates a new delegate object associated with the `static` method `Test.PrintTitle`. Similarly, the non-static method `AddBookToTotal` on the object `totaller` is passed like this:

C#

```
bookDB.ProcessPaperbackBooks(totaller.AddBookToTotal);
```

In both cases a new delegate object is passed to the `ProcessPaperbackBooks` method.

Once a delegate is created, the method it is associated with never changes; delegate objects are immutable.

- Calling a delegate.

Once a delegate object is created, the delegate object is typically passed to other code that will call the delegate. A delegate object is called by using the name of the delegate object, followed by the parenthesized arguments to be passed to the delegate. An example of a delegate call is:

C#

```
processBook(b);
```

A delegate can either be called synchronously, as in this example, or asynchronously by using **BeginInvoke** and **EndInvoke** methods.

See Also

Concepts

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

[Delegates \(C# Programming Guide\)](#)

Events (C# Programming Guide)

Events provide a way for a [class](#) or object to notify other classes or objects when something of interest happens. The class that sends (or *raises*) the event is called the *publisher* and the classes that receive (or *handle*) the event are called *subscribers*.

In a typical C# Windows Forms or Web application, you subscribe to events raised by controls such as buttons and list boxes. You can use the Visual C# integrated development environment (IDE) to browse the events that a control publishes and select the ones that you want to handle. The IDE automatically adds an empty event handler method and the code to subscribe to the event. For more information, see [How to: Subscribe to and Unsubscribe from Events \(C# Programming Guide\)](#).

Events Overview

Events have the following properties:

- The publisher determines when an event is raised; the subscribers determine what action is taken in response to the event.
- An event can have multiple subscribers. A subscriber can handle multiple events from multiple publishers.
- Events that have no subscribers are never called.
- Events are commonly used to signal user actions such as button clicks or menu selections in graphical user interfaces.
- When an event has multiple subscribers, the event handlers are invoked synchronously when an event is raised. To invoke events asynchronously, see [Calling Synchronous Methods Asynchronously](#).
- Events can be used to synchronize threads.
- In the .NET Framework class library, events are based on the [EventHandler](#) delegate and the [EventArgs](#) base class.

Related Sections

For more information, see:

- [How to: Subscribe to and Unsubscribe from Events \(C# Programming Guide\)](#)
- [How to: Publish Events that Conform to .NET Framework Guidelines \(C# Programming Guide\)](#)
- [How to: Raise Base Class Events in Derived Classes \(C# Programming Guide\)](#)
- [How to: Implement Interface Events \(C# Programming Guide\)](#)
- [Thread Synchronization \(C# Programming Guide\)](#)
- [How to: Use a Dictionary to Store Event Instances \(C# Programming Guide\)](#)
- [Events Sample](#)
- [Event Design](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.6.4 Events
- 10.2.7.2 Member Names Reserved for Events
- 10.7 Events
- 13.2.3 Interface Events

See Also

Reference

[EventHandler](#)

Concepts

[C# Programming Guide](#)

[Delegates \(C# Programming Guide\)](#)

Other Resources

How to: Subscribe to and Unsubscribe from Events (C# Programming Guide)

You subscribe to an event that is published by another class when you want to write custom code that is called when that event is raised. For example, you might subscribe to a button's "click" event in order to make your application do something useful when the user clicks the button.

To subscribe to events by using the Visual Studio 2005 IDE

1. If the **Properties** window is not visible, in Design view, right-click on the form or control for which you want to create an event handler, and select **Properties**.
2. On top of the **Properties** window, click the **Events** icon.
3. Double-click the event that you want to create, for example the `Load` event.

Visual C# creates an empty event handler method and adds it to your code. Alternatively you can add the code manually in Code view. For example, the following lines of code declare an event handler method that will be called when the `Form` class raises the `Load` event.

C#

```
private void Form1_Load(object sender, System.EventArgs e)
{
    // Add your form load event handling code here.
}
```

The line of code that is needed to subscribe to the event is also automatically generated in the `InitializeComponent` method in the `Form1.Designer.cs` file in your project. It looks like this:

```
this.Load += new System.EventHandler(this.Form1_Load);
```

To subscribe to events programmatically

1. Define an event handler method whose signature matches the delegate signature for the event. For example, if the event is based on the `EventHandler` delegate type, then the following code represents the method stub:

```
void HandleCustomEvent(object sender, CustomEventArgs a)
{
    // Do something useful here.
}
```

2. Use the addition assignment operator (`+=`) to attach your event handler to the event. In the following example, assume that an object named `publisher` has an event named `RaiseCustomEvent`. Note that the subscriber class needs a reference to the publisher class in order to subscribe to its events.

```
publisher.RaiseCustomEvent += HandleCustomEvent;
```

Note that the above syntax is new in C# 2.0. It is exactly equivalent to the C# 1.0 syntax in which the encapsulating delegate must be explicitly created using the `new` keyword:

```
publisher.RaiseCustomEvent += new CustomEventHandler(HandleCustomEvent);
```

To subscribe to events by using an anonymous method

- Use the addition assignment operator (`+=`) to attach your anonymous method to the event. In the following example, assume that an object named `publisher` has an event named `RaiseCustomEvent` and that a `CustomEventArgs` class has also been defined to carry some kind of specialized event information. Note that the subscriber class needs a reference to `publisher` in order to subscribe to its events.

```
publisher.RaiseCustomEvent += delegate(object o, CustomEventArgs e)
{
    string s = o.ToString() + " " + e.ToString();
    Console.WriteLine(s);
};
```

It is important to note that you cannot easily unsubscribe from an event if you used an anonymous method to subscribe to it. To unsubscribe in this scenario, go back to the code where you subscribe to the event, store the anonymous method in a delegate variable, and then add the delegate to the event.

Unsubscribing

To prevent your event handler from being invoked when the event is raised, simply unsubscribe from the event. In order to prevent resource leaks, it is important to unsubscribe from events before you dispose of a subscriber object. Until you unsubscribe from an event, the multicast delegate that underlies the event in the publishing object has a reference to the delegate that encapsulates the subscriber's event handler. As long as the publishing object holds that reference, your subscriber object will not be garbage collected.

To unsubscribe from an event

- Use the subtraction assignment operator (`-=`) to unsubscribe from an event:

```
publisher.RaiseCustomEvent -= HandleCustomEvent;
```

When all subscribers have unsubscribed from an event, the event instance in the publisher class is set to null.

See Also

Tasks

[How to: Publish Events that Conform to .NET Framework Guidelines \(C# Programming Guide\)](#)

Reference

[event \(C# Reference\)](#)

[-= Operator \(C# Reference\)](#)

[+= Operator \(C# Reference\)](#)

Concepts

[Events \(C# Programming Guide\)](#)

How to: Publish Events that Conform to .NET Framework Guidelines (C# Programming Guide)

The following procedure demonstrates how to add events that follow the standard .NET Framework pattern to your own classes and structs. All events in the .NET Framework class library are based on the [EventHandler](#) delegate, which is defined as follows:

```
public delegate void EventHandler(object sender, EventArgs e);
```

Note

The .NET Framework 2.0 introduces a generic version of this delegate, [EventHandler<T>](#). The following examples show how to use both versions.

Although events in classes that you define can be based on any valid delegate type, including delegates that return a value, it is generally recommended that you base your events on the .NET Framework pattern by using **EventHandler**, as shown in the following example:

To publish events based on the EventHandler pattern

1. (Skip this step and go directly to Step 3a if you have no need to send custom data with your event.) Declare your class at a scope visible to both your publisher and subscriber classes, and add the members needed to hold your custom event data. In this example, a simple string is returned.

```
public class CustomEventArgs : EventArgs
{
    public CustomEventArgs(string s)
    {
        msg = s;
    }
    private string msg;
    public string Message
    {
        get { return msg; }
    }
}
```

2. (Skip this step if you are using the generic version of **EventHandler**.) Declare a delegate in your publishing class. Give it a name that ends with *EventHandler*. The second parameter specifies your custom EventArgs type.

```
public delegate void CustomEventHandler(object sender, CustomEventArgs a);
```

3. Declare the event in your publishing class by using one of the following steps.
 - a. If you have no custom EventArgs class, then your Event type will be the non-generic EventHandler delegate. You do not need to declare it because it is already declared in the [System](#) namespace which is included by default in your C# project:

```
public event EventHandler RaiseCustomEvent;
```

- b. If you are using the non-generic version of **EventHandler** and you have a custom class derived from [EventArgs](#), declare your event inside your publishing class and use your delegate as the type:

```
class Publisher
{
```

```
public event CustomEventHandler RaiseCustomEvent;
}
```

- c. If you are using the generic version, you do not need a custom delegate. Instead, you specify your event type as `EventHandler<CustomEventArgs>`, substituting the name of your own class between the angle brackets.

```
public event EventHandler<CustomEventArgs> RaiseCustomEvent;
```

Example

The following example demonstrates the steps given above using a custom `EventArgs` class and `EventHandler<T>` as the event type.

C#

```
namespace DotNetEvents
{
    using System;
    using System.Collections.Generic;

    // Define a class to hold custom event info
    public class CustomEventArgs : EventArgs
    {
        public CustomEventArgs(string s)
        {
            message = s;
        }
        private string message;

        public string Message
        {
            get { return message; }
            set { message = value; }
        }
    }

    // Class that publishes an event
    class Publisher
    {
        // Declare the event using EventHandler<T>
        public event EventHandler<CustomEventArgs> RaiseCustomEvent;

        public void DoSomething()
        {
            // Write some code that does something useful here
            // then raise the event. You can also raise an event
            // before you execute a block of code.
            OnRaiseCustomEvent(new CustomEventArgs("Did something"));
        }

        // Wrap event invocations inside a protected virtual method
        // to allow derived classes to override the event invocation behavior
        protected virtual void OnRaiseCustomEvent(CustomEventArgs e)
        {
            // Make a temporary copy of the event to avoid possibility of
            // a race condition if the last subscriber unsubscribes
            // immediately after the null check and before the event is raised.
            EventHandler<CustomEventArgs> handler = RaiseCustomEvent;

            // Event will be null if there are no subscribers
            if (handler != null)
            {
                // Format the string to send inside the CustomEventArgs parameter
                e.Message += String.Format(" at {0}", DateTime.Now.ToString());
            }
        }
    }
}
```



```

        // Use the () operator to raise the event.
        handler(this, e);
    }
}

//Class that subscribes to an event
class Subscriber
{
    private string id;
    public Subscriber(string ID, Publisher pub)
    {
        id = ID;
        // Subscribe to the event using C# 2.0 syntax
        pub.RaiseCustomEvent += HandleCustomEvent;
    }

    // Define what actions to take when the event is raised.
    void HandleCustomEvent(object sender, CustomEventArgs e)
    {
        Console.WriteLine(id + " received this message: {0}", e.Message);
    }
}

class Program
{
    static void Main(string[] args)
    {
        Publisher pub = new Publisher();
        Subscriber sub1 = new Subscriber("sub1", pub);
        Subscriber sub2 = new Subscriber("sub2", pub);

        // Call the method that raises the event.
        pub.DoSomething();

        // Keep the console window open
        Console.WriteLine("Press Enter to close this window.");
        Console.ReadLine();
    }
}

```

See Also

Reference

[Delegate](#)

Concepts

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

[Event Design](#)

[Delegates \(C# Programming Guide\)](#)

How to: Raise Base Class Events in Derived Classes (C# Programming Guide)

The following simple example shows the standard way to declare events in a base class so that they can also be raised from derived classes. This pattern is used extensively in Windows Forms classes in the .NET Framework base class library.

When you create a class that can be used as a base class for other classes, you must account for the fact that events are a special type of delegate that can only be invoked from within the class that declared them. Derived classes cannot directly invoke events that are declared within the base class. Although sometimes you may want an event that can only be raised by the base class, in most cases you should enable the derived class to invoke base class events. To do this, you can create a protected invoking method in the base class that wraps the event. By calling or overriding this invoking method, derived classes can invoke the event indirectly.

Example

```
C#
namespace BaseClassEvents
{
    using System;
    using System.Collections.Generic;

    // Special EventArgs class to hold info about Shapes.
    public class ShapeEventArgs : EventArgs
    {
        private double newArea;

        public ShapeEventArgs(double d)
        {
            newArea = d;
        }
        public double NewArea
        {
            get { return newArea; }
        }
    }

    // Base class event publisher
    public abstract class Shape
    {
        protected double area;

        public double Area
        {
            get { return area; }
            set { area = value; }
        }

        // The event. Note that by using the generic EventHandler<T> event type
        // we do not need to declare a separate delegate type.
        public event EventHandler<ShapeEventArgs> ShapeChanged;

        public abstract void Draw();

        //The event-invoking method that derived classes can override.
        protected virtual void OnShapeChanged(ShapeEventArgs e)
        {
            // Make a temporary copy of the event to avoid possibility of
            // a race condition if the last subscriber unsubscribes
            // immediately after the null check and before the event is raised.
            EventHandler<ShapeEventArgs> handler = ShapeChanged;
            if (handler != null)
            {
                handler(this, e);
            }
        }
    }
}
```

```

    }
}

public class Circle : Shape
{
    private double radius;
    public Circle(double d)
    {
        radius = d;
        area = 3.14 * radius;
    }
    public void Update(double d)
    {
        radius = d;
        area = 3.14 * radius;
        OnShapeChanged(new ShapeEventArgs(area));
    }
    protected override void OnShapeChanged(ShapeEventArgs e)
    {
        // Do any circle-specific processing here.

        // Call the base class event invocation method.
        base.OnShapeChanged(e);
    }
    public override void Draw()
    {
        Console.WriteLine("Drawing a circle");
    }
}

public class Rectangle : Shape
{
    private double length;
    private double width;
    public Rectangle(double length, double width)
    {
        this.length = length;
        this.width = width;
        area = length * width;
    }
    public void Update(double length, double width)
    {
        this.length = length;
        this.width = width;
        area = length * width;
        OnShapeChanged(new ShapeEventArgs(area));
    }
    protected override void OnShapeChanged(ShapeEventArgs e)
    {
        // Do any rectangle-specific processing here.

        // Call the base class event invocation method.
        base.OnShapeChanged(e);
    }
    public override void Draw()
    {
        Console.WriteLine("Drawing a rectangle");
    }
}

// Represents the surface on which the shapes are drawn
// Subscribes to shape events so that it knows
// when to redraw a shape.
public class ShapeContainer
{
    List<Shape> _list;

```

```

public ShapeContainer()
{
    _list = new List<Shape>();
}

public void AddShape(Shape s)
{
    _list.Add(s);
    // Subscribe to the base class event.
    s.ShapeChanged += HandleShapeChanged;
}

// ...Other methods to draw, resize, etc.

private void HandleShapeChanged(object sender, ShapeEventArgs e)
{
    Shape s = (Shape)sender;

    // Diagnostic message for demonstration purposes.
    Console.WriteLine("Received event. Shape area is now {0}", e.NewArea);

    // Redraw the shape here.
    s.Draw();
}

}

class Test
{
    static void Main(string[] args)
    {
        //Create the event publishers and subscriber
        Circle c1 = new Circle(54);
        Rectangle r1 = new Rectangle(12, 9);
        ShapeContainer sc = new ShapeContainer();

        // Add the shapes to the container.
        sc.AddShape(c1);
        sc.AddShape(r1);

        // Cause some events to be raised.
        c1.Update(57);
        r1.Update(7, 7);

        // Keep the console window open.
        Console.WriteLine();
        Console.WriteLine("Press Enter to exit");
        Console.ReadLine();
    }
}

```

Output

<pre> Received event. Shape area is now 178.98 Drawing a circle Received event. Shape area is now 49 Drawing a rectangle </pre>

See Also

Reference

[Access Modifiers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

[Delegates \(C# Programming Guide\)](#)

Other Resources

[Creating Event Handlers in Windows Forms](#)

How to: Implement Interface Events (C# Programming Guide)

An [interface](#) can declare an [event](#). The following example shows how to implement interface events in a class. Basically the rules are the same as when implementing any interface method or property.

To implement interface events in a class

- Declare the event in your class and then invoke it in the appropriate places.

```
public interface IDrawingObject
{
    event EventHandler ShapeChanged;
}
public class MyEventArgs : EventArgs {...}
public class Shape : IDrawingObject
{
    event EventHandler ShapeChanged;
    void ChangeShape()
    {
        // Do something before the event...
        OnShapeChanged(new MyEventArgs(...));
        // or do something after the event.
    }
    protected virtual void OnShapeChanged(MyEventArgs e)
    {
        if(ShapeChanged != null)
        {
            ShapeChanged(this, e);
        }
    }
}
```

Example

The following example shows how to handle the less-common situation in which your class inherits from two or more interfaces and each interface has an event with the same name. In this situation, you must provide an explicit interface implementation for at least one of the events. When you write an explicit interface implementation for an event, you must also write the **add** and **remove** event accessors. Normally these are provided by the compiler, but in this case the compiler cannot provide them.

By providing your own accessors, you can specify whether the two events are represented by the same event in your class, or by different events. For example, if the events should be raised at different times according to the interface specifications, then you can associate each event with a separate implementation in your class. In the following example, subscribers determine which `OnDraw` event they will receive by casting the shape reference to either an `IShape` or an `IDrawingObject`.

C#

```
namespace WrapTwoInterfaceEvents
{
    using System;

    public interface IDrawingObject
    {
        // Raise this event before drawing
        // the object.
        event EventHandler OnDraw;
    }
    public interface IShape
    {
        // Raise this event after drawing
        // the shape.
    }
}
```

```

    event EventHandler OnDraw;
}

// Base class event publisher inherits two
// interfaces, each with an OnDraw event
public class Shape : IDrawingObject, IShape
{
    // Create an event for each interface event
    event EventHandler PreDrawEvent;
    event EventHandler PostDrawEvent;

    // Explicit interface implementation required.
    // Associate IDrawingObject's event with
    // PreDrawEvent
    event EventHandler IDrawingObject.OnDraw
    {
        add
        {
            lock (PreDrawEvent)
            {
                PreDrawEvent += value;
            }
        }
        remove
        {
            lock (PreDrawEvent)
            {
                PreDrawEvent -= value;
            }
        }
    }

    // Explicit interface implementation required.
    // Associate IShape's event with
    // PostDrawEvent
    event EventHandler IShape.OnDraw
    {
        add
        {
            lock (PostDrawEvent)
            {
                PostDrawEvent += value;
            }
        }
        remove
        {
            lock (PostDrawEvent)
            {
                PostDrawEvent -= value;
            }
        }
    }

}

// For the sake of simplicity this one method
// implements both interfaces.
public void Draw()
{
    // Raise IDrawingObject's event before the object is drawn.
    EventHandler handler = PreDrawEvent;
    if (handler != null)
    {
        handler(this, new EventArgs());
    }
    Console.WriteLine("Drawing a shape.");

    // Raise IShape's event after the object is drawn.

```

```

        handler = PostDrawEvent;
        if (handler != null)
        {
            handler(this, new EventArgs());
        }
    }
}

public class Subscriber1
{
    // References the shape object as an IDrawingObject
    public Subscriber1(Shape shape)
    {
        IDrawingObject d = (IDrawingObject)shape;
        d.OnDraw += new EventHandler(d_OnDraw);
    }

    void d_OnDraw(object sender, EventArgs e)
    {
        Console.WriteLine("Sub1 receives the IDrawingObject event.");
    }
}

// References the shape object as an IShape
public class Subscriber2
{
    public Subscriber2(Shape shape)
    {
        IShape d = (IShape)shape;
        d.OnDraw += new EventHandler(d_OnDraw);
    }

    void d_OnDraw(object sender, EventArgs e)
    {
        Console.WriteLine("Sub2 receives the IShape event.");
    }
}

public class Program
{
    static void Main(string[] args)
    {
        Shape shape = new Shape();
        Subscriber1 sub = new Subscriber1(shape);
        Subscriber2 sub2 = new Subscriber2(shape);
        shape.Draw();

        Console.WriteLine("Press Enter to close this window.");
        Console.ReadLine();
    }
}

```

Output

<pre> Sub1 receives the IDrawingObject event. Drawing a shape. Sub2 receives the IShape event. </pre>

See Also

Tasks

[How to: Raise Base Class Events in Derived Classes \(C# Programming Guide\)](#)

Reference

[Explicit Interface Implementation \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

[Delegates \(C# Programming Guide\)](#)

How to: Use a Dictionary to Store Event Instances (C# Programming Guide)

One use for **accessor-declarations** is to expose a large number of events without allocating a field for each event, but instead using a Dictionary to store the event instances. This is only useful if you have a very large number of events, but you expect most of the events will not be implemented.

Example

C#

```
public delegate void EventHandler1(int i);
public delegate void EventHandler2(string s);

public class PropertyEventsSample
{
    private System.Collections.Generic.Dictionary<string, System.Delegate> eventTable;

    public PropertyEventsSample()
    {
        eventTable = new System.Collections.Generic.Dictionary<string, System.Delegate>();
        eventTable.Add("Event1", null);
        eventTable.Add("Event2", null);
    }

    public event EventHandler1 Event1
    {
        add
        {
            lock (eventTable)
            {
                eventTable["Event1"] = (EventHandler1)eventTable["Event1"] + value;
            }
        }
        remove
        {
            lock (eventTable)
            {
                eventTable["Event1"] = (EventHandler1)eventTable["Event1"] - value;
            }
        }
    }

    public event EventHandler2 Event2
    {
        add
        {
            lock (eventTable)
            {
                eventTable["Event2"] = (EventHandler2)eventTable["Event2"] + value;
            }
        }
        remove
        {
            lock (eventTable)
            {
                eventTable["Event2"] = (EventHandler2)eventTable["Event2"] - value;
            }
        }
    }

    internal void RaiseEvent1(int i)
    {
        EventHandler1 handler1;
        if (null != (handler1 = (EventHandler1)eventTable["Event1"]))
```

```

        {
            handler1(i);
        }
    }

    internal void RaiseEvent2(string s)
    {
        EventHandler2 handler2;
        if (null != (handler2 = (EventHandler2)eventTable["Event2"]))
        {
            handler2(s);
        }
    }
}

public class TestClass
{
    public static void Delegate1Method(int i)
    {
        System.Console.WriteLine(i);
    }

    public static void Delegate2Method(string s)
    {
        System.Console.WriteLine(s);
    }

    static void Main()
    {
        PropertyEventsSample p = new PropertyEventsSample();

        p.Event1 += new EventHandler1(TestClass.Delegate1Method);
        p.Event1 += new EventHandler1(TestClass.Delegate1Method);
        p.Event1 -= new EventHandler1(TestClass.Delegate1Method);
        p.RaiseEvent1(2);

        p.Event2 += new EventHandler2(TestClass.Delegate2Method);
        p.Event2 += new EventHandler2(TestClass.Delegate2Method);
        p.Event2 -= new EventHandler2(TestClass.Delegate2Method);
        p.RaiseEvent2("TestString");

        Console.ReadLine();
    }
}

```

Output

2
TestString

See Also

Concepts

[C# Programming Guide](#)

[Events \(C# Programming Guide\)](#)

[Delegates \(C# Programming Guide\)](#)

Multithreaded Programming with the Event-based Asynchronous Pattern

There are a number of ways to expose asynchronous features to client code. The Event-based Asynchronous Pattern prescribes the recommended way for classes to present asynchronous behavior.

In This Section

[Event-based Asynchronous Pattern Overview](#)

Describes how the Event-based Asynchronous Pattern makes available the advantages of multithreaded applications while hiding many of the complex issues inherent in multithreaded design.

[Implementing the Event-based Asynchronous Pattern](#)

Describes the standardized way to package a class that has asynchronous features.

[Best Practices for Implementing the Event-based Asynchronous Pattern](#)

Describes the requirements for exposing asynchronous features according to the Event-based Asynchronous Pattern.

[Deciding When to Implement the Event-based Asynchronous Pattern](#)

Describes how to determine when you should choose to implement the Event-based Asynchronous Pattern instead of the [IAsyncResult](#) pattern.

[Walkthrough: Implementing a Component That Supports the Event-based Asynchronous Pattern](#)

Illustrates how to create a component that implements the Event-based Asynchronous Pattern. It is implemented using helper classes from the [System.ComponentModel](#) namespace, which ensures that the component works correctly under any application model.

[How to: Use Components That Support the Event-based Asynchronous Pattern](#)

Describes how to use a component that supports the Event-based Asynchronous Pattern.

Reference

[AsyncOperation](#)

Describes the **AsyncOperation** class and has links to all its members.

[AsyncOperationManager](#)

Describes the **AsyncOperationManager** class and has links to all its members.

[BackgroundWorker](#)

Describes the **BackgroundWorker** component and has links to all its members.

Related Sections

[Event-based Asynchronous Pattern Technology Sample](#)

Shows how to use the Event-based Asynchronous Pattern to perform common asynchronous operations.

[Multithreading in Visual Basic](#)

Describes multithreading features in the .NET Framework.

See Also

Concepts

[Managed Threading Best Practices](#)

[Events and Delegates](#)

Other Resources

[Multithreading in Components](#)

[Asynchronous Programming Design Patterns](#)

Generics (C# Programming Guide)

Generics are a new feature in version 2.0 of the C# language and the common language runtime (CLR). Generics introduce to the .NET Framework the concept of type parameters, which make it possible to design classes and methods that defer the specification of one or more types until the class or method is declared and instantiated by client code. For example, by using a generic type parameter T you can write a single class that other client code can use without incurring the cost or risk of runtime casts or boxing operations, as shown here:

C#

```
// Declare the generic class
public class GenericList<T>
{
    void Add(T input) { }
}
class TestGenericList
{
    private class ExampleClass { }
    static void Main()
    {
        // Declare a list of type int
        GenericList<int> list1 = new GenericList<int>();

        // Declare a list of type string
        GenericList<string> list2 = new GenericList<string>();

        // Declare a list of type ExampleClass
        GenericList<ExampleClass> list3 = new GenericList<ExampleClass>();
    }
}
```

Generics Overview

- Use generic types to maximize code reuse, type safety, and performance.
- The most common use of generics is to create collection classes.
- The .NET Framework class library contains several new generic collection classes in the [System.Collections.Generic](#) namespace. These should be used whenever possible in place of classes such as [ArrayList](#) in the [System.Collections](#) namespace.
- You can create your own generic interfaces, classes, methods, events and delegates.
- Generic classes may be constrained to enable access to methods on particular data types.
- Information on the types used in a generic data type may be obtained at run-time by means of reflection.

Related Sections

For more information:

- [Introduction to Generics \(C# Programming Guide\)](#)
- [Benefits of Generics \(C# Programming Guide\)](#)
- [Generic Type Parameters \(C# Programming Guide\)](#)
- [Constraints on Type Parameters \(C# Programming Guide\)](#)
- [Generic Classes \(C# Programming Guide\)](#)
- [Generic Interfaces \(C# Programming Guide\)](#)
- [Generic Methods \(C# Programming Guide\)](#)
- [Generic Delegates \(C# Programming Guide\)](#)

- [default Keyword in Generic Code \(C# Programming Guide\)](#)
- [Differences Between C++ Templates and C# Generics \(C# Programming Guide\)](#)
- [Generics and Reflection \(C# Programming Guide\)](#)
- [Generics in the Runtime \(C# Programming Guide\)](#)
- [Generics in the .NET Framework Class Library \(C# Programming Guide\)](#)
- [Generics Sample \(C#\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [20 Generics](#)

See Also

Reference

[Data Types \(C# Programming Guide\)](#)

[<typeparam> \(C# Programming Guide\)](#)

[<typeparamref> \(C# Programming Guide\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

Introduction to Generics (C# Programming Guide)

Generic classes and methods combine reusability, type safety and efficiency in a way that their non-generic counterparts cannot. Generics are most commonly used with collections and the methods that operate on them. Version 2.0 of the .NET Framework class library provides a new namespace, [System.Collections.Generic](#), which contains several new generic-based collection classes. It is recommended that all applications that target Version 2.0 use the new generic collection classes instead of the older non-generic counterparts such as [ArrayList](#). For more information, see [Generics in the .NET Framework Class Library \(C# Programming Guide\)](#).

Of course, you can also create custom generic types and methods to provide your own generalized solutions and design patterns that are type-safe and efficient. The following code example shows a simple generic linked-list class for demonstration purposes. (In most cases, it is recommended that you use the [List<T>](#) class provided by the .NET Framework class library, rather than create your own.) The type parameter *T* is used in several places where a concrete type would normally be used to indicate the type of the item stored in the list. It is used in the following ways:

- As the type of a method parameter in the **AddHead** method.
- As the return type of the public method **GetNext** and the **Data** property in the nested **Node** class.
- As the type of the private member data in the nested class.

Note that *T* is available to the nested **Node** class. When `GenericList<T>` is instantiated with a concrete type — for example as a `GenericList<int>` — each occurrence of *T* will be replaced with *int*.

C#

```
// type parameter T in angle brackets
public class GenericList<T>
{
    // The nested class is also generic on T
    private class Node
    {
        // T used in non-generic constructor
        public Node(T t)
        {
            next = null;
            data = t;
        }

        private Node next;
        public Node Next
        {
            get { return next; }
            set { next = value; }
        }

        // T as private member data type
        private T data;

        // T as return type of property
        public T Data
        {
            get { return data; }
            set { data = value; }
        }
    }

    private Node head;

    // constructor
    public GenericList()
    {
        head = null;
    }

    // T as method parameter type:
```

```

public void AddHead(T t)
{
    Node n = new Node(t);
    n.Next = head;
    head = n;
}

public IEnumerator<T> GetEnumerator()
{
    Node current = head;

    while (current != null)
    {
        yield return current.Data;
        current = current.Next;
    }
}
}

```

The following code example shows how client code uses the generic `GenericList<T>` class to create a list of integers. Simply by changing the type argument, the code below could easily be modified to create lists of strings or any other custom type:

C#

```

class TestGenericList
{
    static void Main()
    {
        // int is the type argument
        GenericList<int> list = new GenericList<int>();

        for (int x = 0; x < 10; x++)
        {
            list.AddHead(x);
        }

        foreach (int i in list)
        {
            System.Console.Write(i + " ");
        }
        System.Console.WriteLine("\nDone");
    }
}

```

See Also

Reference

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

[Generics \(C# Programming Guide\)](#)

Benefits of Generics (C# Programming Guide)

Generics provide the solution to a limitation in earlier versions of the common language runtime and the C# language in which generalization is accomplished by casting types to and from the universal base type [Object](#). By creating a generic class, you can create a collection that is type-safe at compile-time.

The limitations of using non-generic collection classes can be demonstrated by writing a short program that makes use of the [ArrayList](#) collection class from the .NET Framework base class library. **ArrayList** is a highly convenient collection class that can be used without modification to store any reference or value type.

C#

```
// The .NET Framework 1.1 way to create a list:
System.Collections.ArrayList list1 = new System.Collections.ArrayList();
list1.Add(3);
list1.Add(105);

System.Collections.ArrayList list2 = new System.Collections.ArrayList();
list2.Add("It is raining in Redmond.");
list2.Add("It is snowing in the mountains.");
```

But this convenience comes at a cost. Any reference or value type that is added to an **ArrayList** is implicitly upcast to **Object**. If the items are value types, they must be boxed when added to the list, and unboxed when they are retrieved. Both the casting and the boxing and unboxing operations degrade performance; the effect of boxing and unboxing can be quite significant in scenarios where you must iterate over large collections.

The other limitation is lack of compile-time type checking; since an **ArrayList** casts everything to **Object**, there is no way at compile-time to prevent client code from doing something like this:

C#

```
System.Collections.ArrayList list = new System.Collections.ArrayList();
// Add an integer to the list.
list.Add(3);
// Add a string to the list. This will compile, but may cause an error later.
list.Add("It is raining in Redmond.");

int t = 0;
// This causes an InvalidCastException to be returned.
foreach (int x in list)
{
    t += x;
}
```

Although perfectly legal and sometimes intentional if you are creating a heterogeneous collection, combining strings and **ints** in a single **ArrayList** is more likely to be a programming error, and this error will not be detected until runtime.

In versions 1.0 and 1.1 of the C# language, you could avoid the dangers of generalized code in the .NET Framework base class library collection classes only by writing your own type specific collections. Of course, since such a class is not reusable for more than one data type, you lose the benefits of generalization, and you have to rewrite the class for each type that will be stored.

What **ArrayList** and other similar classes really need is a way for client code to specify, on a per-instance basis, the particular data type that they intend to use. That would eliminate the need for the upcast to **T:System.Object** and would also make it possible for the compiler to do type checking. In other words, **ArrayList** needs a *type parameter*. That is precisely what generics provide. In the generic **List<T>** collection, in the **N:System.Collections.Generic** namespace, the same operation of adding items to the collection looks like this:

C#

```
// The .NET Framework 2.0 way to create a list
List<int> list1 = new List<int>();
```

```
// No boxing, no casting:  
list1.Add(3);  
  
// Compile-time error:  
// list1.Add("It is raining in Redmond.");
```

For client code, the only added syntax with `List<T>` compared to **ArrayList** is the type argument in the declaration and instantiation. In return for this slightly greater coding complexity, you can create a list that is not only safer than **ArrayList**, but also significantly faster, especially when the list items are value types.

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

Generic Type Parameters (C# Programming Guide)

In a generic type or method definition, a type parameter is a placeholder for a specific type that a client specifies when they instantiate a variable of the generic type. A generic class, such as `GenericList<T>` listed in [Introduction to Generics \(C# Programming Guide\)](#), cannot be used as-is because it is not really a type; it is more like a blueprint for a type. To use `GenericList<T>`, client code must declare and instantiate a constructed type by specifying a type argument inside the angle brackets. The type argument for this particular class can be any type recognized by the compiler. Any number of constructed type instances can be created, each one using a different type argument, as follows:

C#

```
GenericList<float> list1 = new GenericList<float>();
GenericList<ExampleClass> list2 = new GenericList<ExampleClass>();
GenericList<ExampleStruct> list3 = new GenericList<ExampleStruct>();
```

In each of these instances of `GenericList<T>`, every occurrence of `T` in the class will be substituted at run time with the type argument. By means of this substitution, we have created three separate type-safe and efficient objects using a single class definition. For more information on how this substitution is performed by the CLR, see [Generics in the Runtime \(C# Programming Guide\)](#).

Type Parameter Naming Guidelines

- **Do** name generic type parameters with descriptive names, unless a single letter name is completely self explanatory and a descriptive name would not add value.

C#

```
public interface ISessionChannel<TSession> { /*...*/ }
public delegate TOutput Converter<TInput, TOutput>(TInput from);
public class List<T> { /*...*/ }
```

- **Consider** using `T` as the type parameter name for types with one single letter type parameter.

C#

```
public int IComparer<T>() { return 0; }
public delegate bool Predicate<T>(T item);
public struct Nullable<T> where T : struct { /*...*/ }
```

- **Do** prefix descriptive type parameter names with "T".

C#

```
public interface ISessionChannel<TSession>
{
    TSession Session { get; }
}
```

- **Consider** indicating constraints placed on a type parameter in the name of parameter. For example, a parameter constrained to `ISession` may be called `TSession`.

See Also

Reference

[Differences Between C++ Templates and C# Generics \(C# Programming Guide\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

Constraints on Type Parameters (C# Programming Guide)

When you define a generic class, you can apply restrictions to the kinds of types that client code can use for type arguments when it instantiates your class. If client code attempts to instantiate your class with a type that is not allowed by a constraint, the result is a compile-time error. These restrictions are called constraints. Constraints are specified using the **where** contextual keyword. The following table lists the six types of constraints:

Constraint	Description
where T: struct	The type argument must be a value type. Any value type except Nullable can be specified. See Using Nullable Types (C# Programming Guide) for more information.
where T : class	The type argument must be a reference type, including any class, interface, delegate, or array type. (See note below.)
where T : new()	The type argument must have a public parameterless constructor. When used in conjunction with other constraints, the new() constraint must be specified last.
where T : <base class name>	The type argument must be or derive from the specified base class.
where T : <interface name>	The type argument must be or implement the specified interface. Multiple interface constraints can be specified. The constraining interface can also be generic.
where T : U	The type argument supplied for T must be or derive from the argument supplied for U. This is called a naked type constraint.

Note

In the .NET Framework 2.0, recursively defined generic type constraints may in some cases result in a `BadImageFormatException` at runtime. The workaround is to remove the `where T : class` constraint on the enclosing type. For more information, see Feedback ID #93389 on the Microsoft Connect web site and [Microsoft Knowledge Base article 940164](#).

Why Use Constraints

If you want to examine an item in a generic list to determine whether it is valid or to compare it to some other item, the compiler must have some guarantee that the operator or method it needs to call will be supported by any type argument that might be specified by client code. This guarantee is obtained by applying one or more constraints to your generic class definition. For example, the base class constraint tells the compiler that only objects of this type or derived from this type will be used as type arguments. Once the compiler has this guarantee, it can allow methods of that type to be called within the generic class. Constraints are applied using the contextual keyword **where**. The following code example demonstrates the functionality we can add to the `GenericList<T>` class (in [Introduction to Generics \(C# Programming Guide\)](#)) by applying a base class constraint.

C#

```
public class Employee
{
    private string name;
    private int id;

    public Employee(string s, int i)
    {
        name = s;
        id = i;
    }

    public string Name
    {
        get { return name; }
        set { name = value; }
    }
}
```

```

public int ID
{
    get { return id; }
    set { id = value; }
}
}

public class GenericList<T> where T : Employee
{
    private class Node
    {
        private Node next;
        private T data;

        public Node(T t)
        {
            next = null;
            data = t;
        }

        public Node Next
        {
            get { return next; }
            set { next = value; }
        }

        public T Data
        {
            get { return data; }
            set { data = value; }
        }
    }

    private Node head;

    public GenericList() //constructor
    {
        head = null;
    }

    public void AddHead(T t)
    {
        Node n = new Node(t);
        n.Next = head;
        head = n;
    }

    public IEnumerator<T> GetEnumerator()
    {
        Node current = head;

        while (current != null)
        {
            yield return current.Data;
            current = current.Next;
        }
    }

    public T FindFirstOccurrence(string s)
    {
        Node current = head;
        T t = null;

        while (current != null)
        {
            //The constraint enables access to the Name property.
            if (current.Data.Name == s)

```

```

        {
            t = current.Data;
            break;
        }
        else
        {
            current = current.Next;
        }
    }
    return t;
}
}

```

The constraint enables the generic class to use the `Employee.Name` property since all items of type `T` are guaranteed to be either an `Employee` object or an object that inherits from `Employee`.

Multiple constraints can be applied to the same type parameter, and the constraints themselves can be generic types, as follows:

C#

```

class EmployeeList<T> where T : Employee, IEmployee, System.IComparable<T>, new()
{
    // ...
}

```

By constraining the type parameter, you increase the number of allowable operations and method calls to those supported by the constraining type and all types in its inheritance hierarchy. Therefore, when designing generic classes or methods, if you will be performing any operation on the generic members beyond simple assignment or calling any methods not supported by **System.Object**, you will need to apply constraints to the type parameter.

When applying the `where T : class` constraint, it is recommended that you do not use the `==` and `!=` operators on the type parameter because these operators will test for reference identity only, not for value equality. This is the case even if these operators are overloaded in a type used as an argument. The following code illustrates this point; the output is false even though the `String` class overloads the `==` operator.

C#

```

public static void OpTest<T>(T s, T t) where T : class
{
    System.Console.WriteLine(s == t);
}
static void Main()
{
    string s1 = "foo";
    System.Text.StringBuilder sb = new System.Text.StringBuilder("foo");
    string s2 = sb.ToString();
    OpTest<string>(s1, s2);
}

```

The reason for this behavior is that, at compile time, the compiler only knows that `T` is a reference type, and therefore must use the default operators that are valid for all reference types. If you need to test for value equality, the recommended way is to also apply the `where T : IComparable<T>` constraint and implement that interface in any class that will be used to construct the generic class.

Unbounded Type Parameters

Type parameters that have no constraints, such as `T` in `public class SampleClass<T>{}`, are called unbounded type parameters. Unbounded type parameters have the following rules:

- The `!=` and `==` operators cannot be used because there is no guarantee that the concrete type argument will support these operators.
- They can be converted to and from **System.Object** or explicitly converted to any interface type.

- You can compare to [null](#). If an unbounded parameter is compared to **null**, the comparison will always return false if the type argument is a value type.

Naked Type Constraints

When a generic type parameter is used as a constraint, it is called a naked type constraint. Naked type constraints are useful when a member function with its own type parameter needs to constrain that parameter to the type parameter of the containing type, as shown in the following example:

C#

```
class List<T>
{
    void Add<U>(List<U> items) where U : T { /*...*/ }
}
```

In the previous example, **T** is a naked type constraint in the context of the **Add** method, and an unbounded type parameter in the context of the **List** class.

Naked type constraints can also be used in generic class definitions. Note that the naked type constraint must also have been declared within the angle brackets along with any other type parameters:

C#

```
//naked type constraint
public class SampleClass<T, U, V> where T : V { }
```

The usefulness of naked type constraints with generic classes is very limited because the compiler can assume nothing about a naked type constraint except that it derives from **System.Object**. Use naked type constraints on generic classes in scenarios in which you wish to enforce an inheritance relationship between two type parameters.

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

[new Constraint \(C# Reference\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

Generic Classes (C# Programming Guide)

Generic classes encapsulate operations that are not specific to a particular data type. The most common use for generic classes is with collections like linked lists, hash tables, stacks, queues, trees and so on where operations such as adding and removing items from the collection are performed in much the same way regardless of the type of data being stored.

For most scenarios requiring collection classes, the recommended approach is to use the ones provided in the .NET Framework 2.0 class library. For more information on using these classes, see [Generics in the .NET Framework Class Library \(C# Programming Guide\)](#).

Typically, you create generic classes by starting with an existing concrete class, and changing types into type parameters one at a time until you reach the optimal balance of generalization and usability. When creating your own generic classes, important considerations include:

- Which types to generalize into type parameters.

As a general rule, the more types you are able to parameterize, the more flexible and reusable your code becomes. However, too much generalization can result in code that is difficult for other developers to read or understand.

- What constraints, if any, to apply to the type parameters (See [Constraints on Type Parameters \(C# Programming Guide\)](#)).

A good rule is to apply the maximum constraints possible that will still let you handle the types you need to handle. For example, if you know that your generic class is intended for use only with reference types, then apply the class constraint. That will prevent unintended use of your class with value types, and will enable you to use the **as** operator on $\mathbb{T}$, and check for null values.

- Whether to factor generic behavior into base classes and subclasses.

Since generic classes can serve as base classes, the same design considerations apply here as with non-generic classes. See below for rules on inheriting from generic base classes.

- Whether to implement one or more generic interfaces.

For example, if you are designing a class that will be used to create items in a generics-based collection, you may need to implement an interface such as `IComparable<T>` where $\mathbb{T}$ is the type of your class.

For an example of a simple generic class, see [Introduction to Generics \(C# Programming Guide\)](#)

The rules for type parameters and constraints have several implications for generic class behavior, especially with respect to inheritance and member accessibility. Before proceeding, it is important to understand some terms. For a generic class `Node<T>`, client code can reference the class either by specifying a type argument, to create a closed constructed type (`Node<int>`), or it can leave the type parameter unspecified, for example when specifying a generic base class, to create an open constructed type (`Node<T>`). Generic classes can inherit from concrete, closed constructed, or open constructed base classes:

C#

```
class BaseNode { }
class BaseNodeGeneric<T> { }

// concrete type
class NodeConcrete<T> : BaseNode { }

//closed constructed type
class NodeClosed<T> : BaseNodeGeneric<int> { }

//open constructed type
class NodeOpen<T> : BaseNodeGeneric<T> { }
```

Non-generic — concrete — classes can inherit from closed constructed base classes, but not from open constructed classes or naked type parameters because there is no way at run time for client code to supply the type argument required to instantiate the base class.

C#

```
//No error
```

```

class Node1 : BaseNodeGeneric<int> { }

//Generates an error
//class Node2 : BaseNodeGeneric<T> {}

//Generates an error
//class Node3 : T {}

```

Generic classes that inherit from open constructed types must supply type arguments for any base class type parameters that are not shared by the inheriting class, as demonstrated in the following code:

C#

```

class BaseNodeMultiple<T, U> { }

//No error
class Node4<T> : BaseNodeMultiple<T, int> { }

//No error
class Node5<T, U> : BaseNodeMultiple<T, U> { }

//Generates an error
//class Node6<T> : BaseNodeMultiple<T, U> {}

```

Generic classes that inherit from open constructed types must specify constraints that are a superset of, or imply, the constraints on the base type:

C#

```

class NodeItem<T> where T : System.IComparable<T>, new() { }
class SpecialNodeItem<T> : NodeItem<T> where T : System.IComparable<T>, new() { }

```

Generic types can use multiple type parameters and constraints, as follows:

C#

```

class SuperKeyType<K, V, U>
    where U : System.IComparable<U>
    where V : new()
{ }

```

Open constructed and closed constructed types can be used as method parameters:

C#

```

void Swap<T>(List<T> list1, List<T> list2)
{
    //code to swap items
}

void Swap(List<int> list1, List<int> list2)
{
    //code to swap items
}

```

Generic classes are invariant. In other words, if an input parameter specifies a `List<BaseClass>`, you will get a compile-time error if you attempt to provide a `List<DerivedClass>`.

See Also

Reference

[System.Collections.Generic](#)

Concepts

Generic Interfaces (C# Programming Guide)

It is often useful to define interfaces either for generic collection classes, or for the generic classes that represent items in the collection. With generic classes it is preferable to use generic interfaces, such as [IComparable<T>](#) rather than [IComparable](#), in order to avoid boxing and unboxing operations on value types. The .NET Framework 2.0 class library defines several new generic interfaces for use with the new collection classes in the [System.Collections.Generic](#) namespace.

When an interface is specified as a constraint on a type parameter, only types that implement the interface can be used. The following code example shows a `SortedList<T>` class that derives from the `GenericList<T>` class. For more information, see [Introduction to Generics \(C# Programming Guide\)](#). `SortedList<T>` adds the constraint where `T : IComparable<T>`. This enables the **BubbleSort** method in `SortedList<T>` to use the generic [CompareTo](#) method on list elements. In this example, list elements are a simple class, `Person`, that implements `IComparable<Person>`.

C#

```
//Type parameter T in angle brackets.
public class GenericList<T> : System.Collections.Generic.IEnumerable<T>
{
    protected Node head;
    protected Node current = null;

    // Nested class is also generic on T
    protected class Node
    {
        public Node next;
        private T data; //T as private member datatype

        public Node(T t) //T used in non-generic constructor
        {
            next = null;
            data = t;
        }

        public Node Next
        {
            get { return next; }
            set { next = value; }
        }

        public T Data //T as return type of property
        {
            get { return data; }
            set { data = value; }
        }
    }

    public GenericList() //constructor
    {
        head = null;
    }

    public void AddHead(T t) //T as method parameter type
    {
        Node n = new Node(t);
        n.Next = head;
        head = n;
    }

    // Implementation of the iterator
    public System.Collections.Generic.IEnumerator<T> GetEnumerator()
    {
        Node current = head;
        while (current != null)
        {
            yield return current.Data;
            current = current.Next;
        }
    }
}
```

```

    }
}

// IEnumerable<T> inherits from IEnumerable, therefore this class
// must implement both the generic and non-generic versions of
// GetEnumerator. In most cases, the non-generic method can
// simply call the generic method.
System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator()
{
    return GetEnumerator();
}

}

public class SortedList<T> : GenericList<T> where T : System.IComparable<T>
{
    // A simple, unoptimized sort algorithm that
    // orders list elements from lowest to highest:

    public void BubbleSort()
    {
        if (null == head || null == head.Next)
        {
            return;
        }
        bool swapped;

        do
        {
            Node previous = null;
            Node current = head;
            swapped = false;

            while (current.next != null)
            {
                // Because we need to call this method, the SortedList
                // class is constrained on IEnumerable<T>
                if (current.Data.CompareTo(current.next.Data) > 0)
                {
                    Node tmp = current.next;
                    current.next = current.next.next;
                    tmp.next = current;

                    if (previous == null)
                    {
                        head = tmp;
                    }
                    else
                    {
                        previous.next = tmp;
                    }
                    previous = tmp;
                    swapped = true;
                }
                else
                {
                    previous = current;
                    current = current.next;
                }
            }
        } while (swapped);
    }
}

// A simple class that implements IComparable<T> using itself as the
// type argument. This is a common design pattern in objects that
// are stored in generic lists.
public class Person : System.IComparable<Person>
{

```

```

string name;
int age;

public Person(string s, int i)
{
    name = s;
    age = i;
}

// This will cause list elements to be sorted on age values.
public int CompareTo(Person p)
{
    return age - p.age;
}

public override string ToString()
{
    return name + ":" + age;
}

// Must implement Equals.
public bool Equals(Person p)
{
    return (this.age == p.age);
}
}

class Program
{
    static void Main()
    {
        //Declare and instantiate a new generic SortedList class.
        //Person is the type argument.
        SortedList<Person> list = new SortedList<Person>();

        //Create name and age values to initialize Person objects.
        string[] names = new string[]
        {
            "Franscoise",
            "Bill",
            "Li",
            "Sandra",
            "Gunnar",
            "Alok",
            "Hiroyuki",
            "Maria",
            "Alessandro",
            "Raul"
        };

        int[] ages = new int[] { 45, 19, 28, 23, 18, 9, 108, 72, 30, 35 };

        //Populate the list.
        for (int x = 0; x < 10; x++)
        {
            list.AddHead(new Person(names[x], ages[x]));
        }

        //Print out unsorted list.
        foreach (Person p in list)
        {
            System.Console.WriteLine(p.ToString());
        }
        System.Console.WriteLine("Done with unsorted list");

        //Sort the list.
        list.BubbleSort();
    }
}

```

```

        //Print out sorted list.
        foreach (Person p in list)
        {
            System.Console.WriteLine(p.ToString());
        }
        System.Console.WriteLine("Done with sorted list");
    }
}

```

Multiple interfaces can be specified as constraints on a single type, as follows:

C#

```

class Stack<T> where T : System.IComparable<T>, IEnumerable<T>
{
}

```

An interface can define more than one type parameter, as follows:

C#

```

interface IDictionary<K, V>
{
}

```

The same rules of inheritance apply to interfaces as to classes:

C#

```

interface IMonth<T> { }

interface IJanuary    : IMonth<int> { } //No error
interface IFebruary<T> : IMonth<int> { } //No error
interface IMarch<T>    : IMonth<T> { }   //No error
//interface IApril<T>  : IMonth<T, U> { } //Error

```

Generic interfaces can inherit from non-generic interfaces if the generic interface is contra-variant, which means it only uses its type parameter as a return value. In the .NET Framework class library, `IEnumerable<T>` inherits from `IEnumerable` because `IEnumerable<T>` only uses `T` in the return value of `GetEnumerator` and in the `Current` property getter.

Concrete classes can implement closed constructed interfaces, as follows:

C#

```

interface IBaseInterface<T> { }

class SampleClass : IBaseInterface<string> { }

```

Generic classes can implement generic interfaces or closed constructed interfaces as long as the class parameter list supplies all arguments required by the interface, as follows:

C#

```

interface IBaseInterface1<T> { }
interface IBaseInterface2<T, U> { }

class SampleClass1<T> : IBaseInterface1<T> { } //No error
class SampleClass2<T> : IBaseInterface2<T, string> { } //No error

```

The rules that control method overloading are the same for methods within generic classes, generic structs or generic interfaces. See [Generic Methods \(C# Programming Guide\)](#) for more information.

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

[interface](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Generics in the .NET Framework](#)

Generic Methods (C# Programming Guide)

A generic method is a method that is declared with type parameters, as follows:

C#

```
static void Swap<T>(ref T lhs, ref T rhs)
{
    T temp;
    temp = lhs;
    lhs = rhs;
    rhs = temp;
}
```

The following code example shows one way to call the method, using `int` for the type argument:

C#

```
public static void TestSwap()
{
    int a = 1;
    int b = 2;

    Swap<int>(ref a, ref b);
    System.Console.WriteLine(a + " " + b);
}
```

You can also omit the type argument and the compiler will infer it. The following call to `Swap` is equivalent to the previous call:

C#

```
Swap(ref a, ref b);
```

The same rules for type inference apply to static methods as well as instance methods. The compiler is able to infer the type parameters based on the method arguments you pass in; it cannot infer the type parameters solely from a constraint or return value. Therefore type inference does not work with methods that have no parameters. Type inference takes place at compile time before the compiler attempts to resolve any overloaded method signatures. The compiler applies type inference logic to all generic methods that share the same name. In the overload resolution step, the compiler includes only those generic methods on which type inference succeeded.

Within a generic class, non-generic methods can access the class-level type parameters, as follows:

C#

```
class SampleClass<T>
{
    void Swap(ref T lhs, ref T rhs) { }
}
```

If you define a generic method that takes the same type parameters as the containing class the compiler will generate warning CS0693 because within the method scope, the argument supplied for the inner `T` will hide the argument supplied for the outer `T`. If you require the flexibility of calling a generic class method with type arguments other than the ones provided when the class was instantiated, consider providing another identifier for the method's type parameter, as shown in `GenericList2<T>` in the following example.

C#

```
class GenericList<T>
{
    // CS0693
    void SampleMethod<T>() { }
```

```

}

class GenericList2<T>
{
    //No warning
    void SampleMethod<U>() { }
}

```

Use constraints to enable more specialized operations on type parameters in methods. This version of `Swap<T>`, now called `SwapIfGreater<T>`, can only be used with type arguments that implement `IComparable<T>`.

C#

```

void SwapIfGreater<T>(ref T lhs, ref T rhs) where T : System.IComparable<T>
{
    T temp;
    if (lhs.CompareTo(rhs) > 0)
    {
        temp = lhs;
        lhs = rhs;
        rhs = temp;
    }
}

```

Generic methods can be overloaded on a number of type parameters. For example, the following methods can all exist in the same class:

C#

```

void DoWork() { }
void DoWork<T>() { }
void DoWork<T, U>() { }

```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 20.6.4 Inference of Type Arguments.

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

[Methods \(C# Programming Guide\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

Generics and Arrays (C# Programming Guide)

In C# 2.0, single-dimensional arrays that have a lower bound of zero automatically implement [IList<T>](#). This enables you to create generic methods that can use the same code to iterate through arrays and other collection types. This technique is primarily useful for reading data in collections. The **IList<T>** interface cannot be used to add or remove elements from an array; an exception will be thrown if you attempt to call an **IList<T>** method such as [RemoveAt](#) on an array in this context.

The following code example demonstrates how a single generic method that takes an **IList<T>** input parameter can iterate through both a list and an array, in this case an array of integers.

C#

```
class Program
{
    static void Main()
    {
        int[] arr = { 0, 1, 2, 3, 4 };
        List<int> list = new List<int>();

        for (int x = 5; x < 10; x++)
        {
            list.Add(x);
        }

        ProcessItems<int>(arr);
        ProcessItems<int>(list);
    }

    static void ProcessItems<T>(IList<T> coll)
    {
        foreach (T item in coll)
        {
            System.Console.Write(item.ToString() + " ");
        }
        System.Console.WriteLine();
    }
}
```

Note

Even though the `ProcessItems` method cannot add or remove items, the **IsReadOnly** property returns false for a `T[]` within `ProcessItems` because the array itself was not declared with the [ReadOnly](#) attribute.

See Also

Reference

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

[Generics \(C# Programming Guide\)](#)

[Arrays \(C# Programming Guide\)](#)

Other Resources

[Generics in the .NET Framework](#)

Generic Delegates (C# Programming Guide)

A [delegate](#) can define its own type parameters. Code that references the generic delegate can specify the type argument to create a closed constructed type, just like when instantiating a generic class or calling a generic method, as shown in the following example:

C#

```
public delegate void Del<T>(T item);
public static void Notify(int i) { }

Del<int> m1 = new Del<int>(Notify);
```

C# version 2.0 has a new feature called method group conversion, which applies to concrete as well as generic delegate types, and enables you to write the previous line with this simplified syntax:

C#

```
Del<int> m2 = Notify;
```

Delegates defined within a generic class can use the generic class type parameters in the same way that class methods do.

C#

```
class Stack<T>
{
    T[] items;
    int index;

    public delegate void StackDelegate(T[] items);
}
```

Code that references the delegate must specify the type argument of the containing class, as follows:

C#

```
private static void DoWork(float[] items) { }

public static void TestStack()
{
    Stack<float> s = new Stack<float>();
    Stack<float>.StackDelegate d = DoWork;
}
```

Generic delegates are especially useful in defining events based on the typical design pattern because the sender argument can be strongly typed and no longer has to be cast to and from [Object](#).

C#

```
delegate void StackEventHandler<T, U>(T sender, U eventArgs);

class Stack<T>
{
    public class StackEventArgs : System.EventArgs { }
    public event StackEventHandler<Stack<T>, StackEventArgs> stackEvent;

    protected virtual void OnStackChanged(StackEventArgs a)
    {
        stackEvent(this, a);
    }
}
```

```
class SampleClass
{
    public void HandleStackChange<T>(Stack<T> stack, Stack<T>.StackEventArgs args) { }
}

public static void Test()
{
    Stack<double> s = new Stack<double>();
    SampleClass o = new SampleClass();
    s.stackEvent += o.HandleStackChange;
}
```

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

[Generic Methods \(C# Programming Guide\)](#)

[Generic Classes \(C# Programming Guide\)](#)

[Generic Interfaces \(C# Programming Guide\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

[Delegates \(C# Programming Guide\)](#)

Other Resources

[Generics in the .NET Framework](#)

default Keyword in Generic Code (C# Programming Guide)

In generic classes and methods, one issue that arises is how to assign a default value to a parameterized type T when you do not know the following in advance:

- Whether T will be a reference type or a value type.
- If T is a value type, whether it will be a numeric value or a struct.

Given a variable t of a parameterized type T, the statement t = null is only valid if T is a reference type and t = 0 will only work for numeric value types but not for structs. The solution is to use the **default** keyword, which will return null for reference types and zero for numeric value types. For structs, it will return each member of the struct initialized to zero or null depending on whether they are value or reference types. The following example from the `GenericList<T>` class shows how to use the **default** keyword. For more information, see [Generics Overview](#).

C#

```
public class GenericList<T>
{
    private class Node
    {
        //...

        public Node Next;
        public T Data;
    }

    private Node head;

    //...

    public T GetNext()
    {
        T temp = default(T);

        Node current = head;
        if (current != null)
        {
            temp = current.Data;
            current = current.Next;
        }
        return temp;
    }
}
```

See Also

Reference

[Generic Methods \(C# Programming Guide\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

[Generics \(C# Programming Guide\)](#)

Other Resources

[Generics in the .NET Framework](#)

Differences Between C++ Templates and C# Generics (C# Programming Guide)

C# Generics and C++ templates are both language features that provide support for parameterized types. However, there are many differences between the two. At the syntax level, C# generics are a simpler approach to parameterized types without the complexity of C++ templates. In addition, C# does not attempt to provide all of the functionality that C++ templates provide. At the implementation level, the primary difference is that C# generic type substitutions are performed at runtime and generic type information is thereby preserved for instantiated objects. For more information, see [Generics in the Runtime \(C# Programming Guide\)](#).

The following are the key differences between C# Generics and C++ templates:

- C# generics do not provide the same amount of flexibility as C++ templates. For example, it is not possible to call arithmetic operators in a C# generic class, although it is possible to call user defined operators.
- C# does not allow non-type template parameters, such as `template C<int i> {}`.
- C# does not support explicit specialization; that is, a custom implementation of a template for a specific type.
- C# does not support partial specialization: a custom implementation for a subset of the type arguments.
- C# does not allow the type parameter to be used as the base class for the generic type.
- C# does not allow type parameters to have default types.
- In C#, a generic type parameter cannot itself be a generic, although constructed types can be used as generics. C++ does allow template parameters.
- C++ allows code that might not be valid for all type parameters in the template, which is then checked for the specific type used as the type parameter. C# requires code in a class to be written in such a way that it will work with any type that satisfies the constraints. For example, in C++ it is possible to write a function that uses the arithmetic operators `+` and `-` on objects of the type parameter, which will produce an error at the time of instantiation of the template with a type that does not support these operators. C# disallows this; the only language constructs allowed are those that can be deduced from the constraints.

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Templates](#)

Generics in the Runtime (C# Programming Guide)

When a generic type or method is compiled into Microsoft intermediate language (MSIL), it contains metadata that identifies it as having type parameters. How the MSIL for a generic type is used differs based on whether or not the supplied type parameter is a value or reference type.

When a generic type is first constructed with a value type as a parameter, the runtime creates a specialized generic type with the supplied parameter or parameters substituted in the appropriate places in the MSIL. Specialized generic types are created once for each unique value type used as a parameter.

For example, suppose your program code declared a stack constructed of integers, like this:

C#

```
Stack<int> stack;
```

At this point, the runtime generates a specialized version of the `Stack<T>` class with the integer substituted appropriately for its parameter. Now, whenever your program code uses a stack of integers, the runtime reuses the generated specialized **Stack<T>** class. In the following example, two instances of a stack of integers are created, and they share a single instance of the `Stack<int>` code:

C#

```
Stack<int> stackOne = new Stack<int>();  
Stack<int> stackTwo = new Stack<int>();
```

However, if at another point in your program code another **Stack<T>** class is created, this time with a different value type such as a **long** or a user-defined structure as its parameter, the runtime generates another version of the generic type, this time substituting a **long** in the appropriate places in MSIL. Conversions are no longer necessary because each specialized generic class natively contains the value type.

Generics work a little differently for reference types. The first time a generic type is constructed with any reference type, the runtime creates a specialized generic type with object references substituted for the parameters in the MSIL. Then, each time a constructed type is instantiated with a reference type as its parameter, regardless of what type it is, the runtime reuses the previously created specialized version of the generic type. This is possible because all references are the same size.

For example, suppose you had two reference types, a `Customer` class and an `Order` class, and further suppose that you created a stack of `Customer` types:

C#

```
class Customer { }  
class Order { }
```

C#

```
Stack<Customer> customers;
```

At this point, the runtime generates a specialized version of the **Stack<T>** class that, instead of storing data, stores object references that will be filled in later. Suppose the next line of code creates a stack of another reference type, called `Order`:

C#

```
Stack<Order> orders = new Stack<Order>();
```

Unlike with value types, another specialized version of the **Stack<T>** class is not created for the `Order` type. Rather, an instance of the specialized version of the **Stack<T>** class is created and the `orders` variable is set to reference it. Suppose you then encountered a line of code to create a stack of a `Customer` type:

C#

```
customers = new Stack<Customer>();
```

As with the previous use of the **Stack<T>** class created with the `Order` type, another instance of the specialized **Stack<T>** class is created, and the pointers contained therein are set to reference an area of memory the size of a `Customer` type. Because the number of reference types can vary wildly from program to program, the C# implementation of generics greatly reduces code bloat by reducing to one the number of specialized classes created by the compiler for generic classes of reference types.

Moreover, when a generic C# class is instantiated with a type parameter, be it a value or reference type, it can be queried at runtime using reflection and both its actual type as well as its type parameter can be ascertained.

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Generics in the .NET Framework](#)

Generics in the .NET Framework Class Library (C# Programming Guide)

Version 2.0 of the .NET Framework class library provides a new namespace, [System.Collections.Generic](#), which includes several ready-to-use generic collection classes and associated interfaces. Other namespaces such as [System](#) also provide new generic interfaces such as [IComparable<T>](#). These classes and interfaces are more efficient and type-safe than the non-generic collection classes provided in earlier releases of the .NET Framework. Before designing and implementing your own custom collection classes, consider whether you can use or derive a class from one of the classes provided in the base class library.

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[When to Use Generic Collections](#)

Other Resources

[Collections and Data Structures](#)

Generics and Reflection (C# Programming Guide)

Because the Common Language Runtime (CLR) has access to generic type information at run time, you can use reflection to obtain information about generic types in the same way as for non-generic types. For more information, see [Generics in the Runtime \(C# Programming Guide\)](#).

In the .NET Framework 2.0 several new members are added to the [Type](#) class to enable run-time information for generic types. See the documentation on these classes for more information on how to use these methods and properties. The [System.Reflection.Emit](#) namespace also contains new members that support generics. See [How to: Define a Generic Type with Reflection Emit](#).

For a list of the invariant conditions for terms used in generic reflection, see the [IsGenericType](#) property remarks.

System.Type Member Name	Description
IsGenericType	Returns true if a type is generic.
GetGenericArguments	Returns an array of Type objects that represent the type arguments supplied for a constructed type, or the type parameters of a generic type definition.
GetGenericTypeDefinition	Returns the underlying generic type definition for the current constructed type.
GetGenericParameterConstraints	Returns an array of Type objects that represent the constraints on the current generic type parameter.
ContainsGenericParameters	Returns true if the type or any of its enclosing types or methods contain type parameters for which specific types have not been supplied.
GenericParameterAttributes	Gets a combination of GenericParameterAttributes flags that describe the special constraints of the current generic type parameter.
GenericParameterPosition	For a Type object that represents a type parameter, gets the position of the type parameter in the type parameter list of the generic type definition or generic method definition that declared the type parameter.
IsGenericParameter	Gets a value that indicates whether the current Type represents a type parameter of a generic type or method definition.
IsGenericTypeDefinition	Gets a value that indicates whether the current Type represents a generic type definition, from which other generic types can be constructed. Returns true if the type represents the definition of a generic type.
DeclaringMethod	Returns the generic method that defined the current generic type parameter, or null if the type parameter was not defined by a generic method.
MakeGenericType	Substitutes the elements of an array of types for the type parameters of the current generic type definition, and returns a Type object representing the resulting constructed type.

In addition, new members are added to the [MethodInfo](#) class to enable run-time information for generic methods. See the [IsGenericMethod](#) property remarks for a list of invariant conditions for terms used to reflect on generic methods.

System.Reflection.MethodInfo Member Name	Description
IsGenericMethod	Returns true if a method is generic.
GetGenericArguments	Returns an array of Type objects that represent the type arguments of a constructed generic method or the type parameters of a generic method definition.

GetGenericMethodDefinition	Returns the underlying generic method definition for the current constructed method.
ContainsGenericParameters	Returns true if the method or any of its enclosing types contain any type parameters for which specific types have not been supplied.
IsGenericMethodDefinition	Returns true if the current MethodInfo represents the definition of a generic method.
MakeGenericMethod	Substitutes the elements of an array of types for the type parameters of the current generic method definition, and returns a MethodInfo object representing the resulting constructed method.

See Also

Concepts

[C# Programming Guide](#)

[Generics \(C# Programming Guide\)](#)

[Overview of Reflection and Generics](#)

Other Resources

[Generics in the .NET Framework](#)

Generics and Attributes (C# Programming Guide)

Attributes can be applied to generic types in the same way as non-generic types. For more information on applying attributes, see [Attributes \(C# Programming Guide\)](#).

Custom attributes are only permitted to reference open generic types, which are generic types for which no type arguments are supplied, and closed constructed generic types, which supply arguments for all type parameters.

The following examples use this custom attribute:

C#

```
class CustomAttribute : System.Attribute
{
    public System.Object info;
}
```

An attribute can reference an open generic type:

C#

```
public class GenericClass1<T> { }

[CustomAttribute(info = typeof(GenericClass1<>))]
class ClassA { }
```

Specify multiple type parameters using the appropriate number of commas. In this example, `GenericClass2` has two type parameters:

C#

```
public class GenericClass2<T, U> { }

[CustomAttribute(info = typeof(GenericClass2<,>))]
class ClassB { }
```

An attribute can reference a closed constructed generic type:

C#

```
public class GenericClass3<T, U, V> { }

[CustomAttribute(info = typeof(GenericClass3<int, double, string>))]
class ClassC { }
```

An attribute that references a generic type parameter will cause a compile-time error:

C#

```
[[CustomAttribute(info = typeof(GenericClass3<int, T, string>))] //Error
class ClassD<T> { }
```

A generic type cannot inherit from [Attribute](#):

C#

```
//public class CustomAtt<T> : System.Attribute {} //Error
```

To obtain information about a generic type or type parameter at run time, you can use the methods of [System.Reflection](#). For

more information, see [Generics and Reflection \(C# Programming Guide\)](#)

See Also

Concepts

[C# Programming Guide](#)

[Generics \(C# Programming Guide\)](#)

[Attributes Overview](#)

Variance in Generic Types (C# Programming Guide)

One of the main benefits of the addition of generics to C# is the ability to easily create strongly typed collections using types in the [System.Collections.Generic](#) namespace. For example, you can create a variable of type `List<int>`, and the compiler will check all accesses to the variable, ensuring that only **ints** are added to the collection. This is a big usability improvement over the untyped collections available in version 1.0 of C#.

Unfortunately, strongly typed collections have drawbacks of their own. For example, suppose you have a strongly typed `List<object>` and you want to append all the elements from a `List<int>` to your `List<object>`. You would like to be able to write code like this:

C#

```
List<int> ints = new List<int>();
ints.Add(1);
ints.Add(10);
ints.Add(42);
List<object> objects = new List<object>();

// doesnt compile ints is not a IEnumerable<object>
//objects.AddRange(ints);
```

In this case, you would like to treat a `List<int>` which is also an `IEnumerable<int>`, as an `IEnumerable<object>`. This seems like a reasonable thing to do, as **int** is convertible to `object`. It is very similar to being able to treat a `string[]` as an `object[]` as you can do today. If you find yourself in this situation, the feature you are looking for is called generics variance, which treats an instantiation of a generic type, in this case `IEnumerable<int>`, as a different instantiation of that same type, in this case `IEnumerable<object>`.

C# does not support variance for generic types, so when encountering cases like this you will need to try one of a couple of techniques you can use to work around the problem. For the simplest cases, like the case of a single method like `AddRange` in the previous example, you can declare a simple helper method to do the conversion for you. For example, you could write this method:

C#

```
// Simple workaround for single method
// Variance in one direction only
public static void Add<S, D>(List<S> source, List<D> destination)
    where S : D
{
    foreach (S sourceElement in source)
    {
        destination.Add(sourceElement);
    }
}
```

Which allows you to do this:

C#

```
// does compile
VarianceWorkaround.Add<int, object>(ints, objects);
```

This example shows some characteristics of a simple variance workaround. The helper method takes two type parameters, for the source and destination, and the source type parameter *S* has a constraint which is the destination type parameter *D*. This means that the `List<>` being read from must contain elements which are convertible to the element type of the `List<>` being inserted into. This allows the compiler to enforce that **int** is convertible to an `object`. Constraining a type parameter to derive from another type parameter is called a naked type parameter constraint.

Defining a single method to work around variance problems is not too bad. Unfortunately variance issues can become quite complex quite quickly. The next level of complexity is when you want to treat an interface of one instantiation as an interface of

another instantiation. For example, you have an `IEnumerable<int>`, and you want to pass it to a method that only takes an `IEnumerable<object>`. Again, this makes some sense because you can think of an `IEnumerable<object>` as a sequence of objects, and an `IEnumerable<int>` is a sequence of **ints**. Since **ints** are objects, a sequence of **ints** should be treatable as a sequence of objects. For example:

C#

```
static void PrintObjects(IEnumerable<object> objects)
{
    foreach (object o in objects)
    {
        Console.WriteLine(o);
    }
}
```

Which you would like to call like this:

C#

```
// would like to do this, but cant ...
// ... ints is not an IEnumerable<object>
//PrintObjects(ints);
```

The workaround for the interface case, is to create a wrapper object that does the conversions for each member of the interface. This would look something like this:

C#

```
// Workaround for interface
// Variance in one direction only so type expressinos are natural
public static IEnumerable<D> Convert<S, D>(IEnumerable<S> source)
    where S : D
{
    return new EnumerableWrapper<S, D>(source);
}

private class EnumerableWrapper<S, D> : IEnumerable<D>
    where S : D
{
```

Which allows you to do this:

C#

```
PrintObjects(VarianceWorkaround.Convert<int, object>(ints));
```

Again, notice the naked type parameter constraint on the wrapper class and helper method. This machinery is getting pretty complicated, but the code in the wrapper class is pretty straightforward; it just delegates to the members of the wrapped interface doing nothing more than straightforward type conversions along the way. Why not have the compiler allow the conversion from `IEnumerable<int>` to `IEnumerable<object>` directly?

Although variance is type-safe in the case where you are looking at read-only views of your collections, variance is not type-safe in the case where both read and write operations are involved. For example, the `ICollection<>` interface could not be dealt with in this automatic way. You can still write a helper that will wrap all read operations on an `ICollection<>` in a type-safe manner, but the wrapping of write operations cannot be done so simply.

Here is part of a wrapper for dealing with variance on the `ICollection<T>` interface that shows the problems that arise with variance in both the read and write directions:

C#

```
private class ListWrapper<S, D> : CollectionWrapper<S, D>, ICollection<D>
    where S : D
```



```

{
    public ListWrapper(IList<S> source) : base(source)
    {
        this.source = source;
    }

    public int IndexOf(D item)
    {
        if (item is S)
        {
            return this.source.IndexOf((S) item);
        }
        else
        {
            return -1;
        }
    }

    // variance the wrong way ...
    // ... can throw exceptions at runtime
    public void Insert(int index, D item)
    {
        if (item is S)
        {
            this.source.Insert(index, (S)item);
        }
        else
        {
            throw new Exception("Invalid type exception");
        }
    }
}

```

The Insert method on the wrapper has a problem. It takes as an argument a D, but it must insert it into an IList<S>. Since D is a base type of S, not all Ds are Ss, so the Insert operation may fail. This example has an analogue with variance with arrays. When inserting an object into an object[], a dynamic type check is performed because the object[] may in fact be a string[] at run time. For example:

C#

```

object[] objects = new string[10];

// no problem, adding a string to a string[]
objects[0] = "hello";

// runtime exception, adding an object to a string[]
objects[1] = new object();

```

In the IList<> example, the wrapper for the Insert method can simply throw when the actual type does not match the desired type at run time. So again, you could imagine that the compiler would automatically generate the wrapper for the programmer. There are cases where this policy is not the right thing to do, however. The IndexOf method searches the collection for the item provided, and returns the index in the collection if the item is found. However, if the item is not found, the IndexOf method simply returns -1, it does not throw. This kind of wrapping cannot be provided by an automatically generated wrapper.

So far, the two simplest work arounds for generic variance issues have been described. However, variance issues can get arbitrarily complex. For example, when you treat a List<IEnumerable<int>> as a List<IEnumerable<object>>, or treating a List<IEnumerable<IEnumerable<int>>> as a List<IEnumerable<IEnumerable<object>>>.

Generating these wrappers to work around variance problems in your code can introduce a significant overhead in your code. Also, it can introduce referential identity issues, as each wrapper does not have the same identity as the original collection which can lead to subtle bugs. When using generics, you should choose your type instantiation to reduce mismatches between components that are tightly coupled. This may require some compromises in the design of your code. As always, design involves tradeoffs between conflicting requirements, and the constraints of the types system in the language should be considered in your design process.

There are type systems that include generic variance as a first class part of the language. Eiffel is the prime example of this. However, including generics variance as a first class part of the type system would dramatically increase the complexity of the type system of C#, even in relatively straightforward scenarios that do not involve variance. As a result, the C# designers felt that not including variance was the right choice for C#.

Here is the full source code for the previous examples.

C#

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Collections;

static class VarianceWorkaround
{
    // Simple workaround for single method
    // Variance in one direction only
    public static void Add<S, D>(List<S> source, List<D> destination)
        where S : D
    {
        foreach (S sourceElement in source)
        {
            destination.Add(sourceElement);
        }
    }

    // Workaround for interface
    // Variance in one direction only so type expressions are natural
    public static IEnumerable<D> Convert<S, D>(IEnumerable<S> source)
        where S : D
    {
        return new EnumerableWrapper<S, D>(source);
    }

    private class EnumerableWrapper<S, D> : IEnumerable<D>
        where S : D
    {
        public EnumerableWrapper(IEnumerable<S> source)
        {
            this.source = source;
        }

        public IEnumerator<D> GetEnumerator()
        {
            return new EnumeratorWrapper(this.source.GetEnumerator());
        }

        IEnumerator System.Collections.IEnumerable.GetEnumerator()
        {
            return this.GetEnumerator();
        }

        private class EnumeratorWrapper : IEnumerator<D>
        {
            public EnumeratorWrapper(IEnumerator<S> source)
            {
                this.source = source;
            }

            private IEnumerator<S> source;

            public D Current
            {
                get { return this.source.Current; }
            }

            public void Dispose()
            {
            }
        }
    }
}
```

```

        this.source.Dispose();
    }

    object IEnumerator.Current
    {
        get { return this.source.Current; }
    }

    public bool MoveNext()
    {
        return this.source.MoveNext();
    }

    public void Reset()
    {
        this.source.Reset();
    }
}

private IEnumerable<S> source;
}

// Workaround for interface
// Variance in both directions, causes issues
// similar to existing array variance
public static ICollection<D> Convert<S, D>(ICollection<S> source)
    where S : D
{
    return new CollectionWrapper<S, D>(source);
}

private class CollectionWrapper<S, D>
    : EnumerableWrapper<S, D>, ICollection<D>
    where S : D
{
    public CollectionWrapper(ICollection<S> source)
        : base(source)
    {
    }

    // variance going the wrong way ...
    // ... can yield exceptions at runtime
    public void Add(D item)
    {
        if (item is S)
        {
            this.source.Add((S)item);
        }
        else
        {
            throw new Exception(@"Type mismatch exception, due to type hole introduced
by variance.");
        }
    }

    public void Clear()
    {
        this.source.Clear();
    }

    // variance going the wrong way ...
    // ... but the semantics of the method yields reasonable semantics
    public bool Contains(D item)
    {
        if (item is S)
        {
            return this.source.Contains((S)item);
        }
    }
}

```

```

        }
        else
        {
            return false;
        }
    }

    // variance going the right way ...
    public void CopyTo(D[] array, int arrayIndex)
    {
        foreach (S src in this.source)
        {
            array[arrayIndex++] = src;
        }
    }

    public int Count
    {
        get { return this.source.Count; }
    }

    public bool IsReadOnly
    {
        get { return this.source.IsReadOnly; }
    }

    // variance going the wrong way ...
    // ... but the semantics of the method yields reasonable semantics
    public bool Remove(D item)
    {
        if (item is S)
        {
            return this.source.Remove((S)item);
        }
        else
        {
            return false;
        }
    }

    private ICollection<S> source;
}

// Workaround for interface
// Variance in both directions, causes issues similar to existing array variance
public static IList<D> Convert<S, D>(IList<S> source)
    where S : D
{
    return new ListWrapper<S, D>(source);
}

private class ListWrapper<S, D> : CollectionWrapper<S, D>, IList<D>
    where S : D
{
    public ListWrapper(IList<S> source) : base(source)
    {
        this.source = source;
    }

    public int IndexOf(D item)
    {
        if (item is S)
        {
            return this.source.IndexOf((S) item);
        }
        else
        {
            return -1;
        }
    }
}

```

```

    }
}

// variance the wrong way ...
// ... can throw exceptions at runtime
public void Insert(int index, D item)
{
    if (item is S)
    {
        this.source.Insert(index, (S)item);
    }
    else
    {
        throw new Exception("Invalid type exception");
    }
}

public void RemoveAt(int index)
{
    this.source.RemoveAt(index);
}

public D this[int index]
{
    get
    {
        return this.source[index];
    }
    set
    {
        if (value is S)
            this.source[index] = (S)value;
        else
            throw new Exception("Invalid type exception.");
    }
}

private IList<S> source;
}

namespace GenericVariance
{
    class Program
    {
        static void PrintObjects(IEnumerable<object> objects)
        {
            foreach (object o in objects)
            {
                Console.WriteLine(o);
            }
        }

        static void AddToObjects(IList<object> objects)
        {
            // this will fail if the collection provided is a wrapped collection
            objects.Add(new object());
        }

        static void Main(string[] args)
        {
            List<int> ints = new List<int>();
            ints.Add(1);
            ints.Add(10);
            ints.Add(42);
            List<object> objects = new List<object>();

            // doesnt compile ints is not a IEnumerable<object>
            //objects.AddRange(ints);

```

```

// does compile
VarianceWorkaround.Add<int, object>(ints, objects);

// would like to do this, but cant ...
// ... ints is not an IEnumerable<object>
//PrintObjects(ints);

PrintObjects(VarianceWorkaround.Convert<int, object>(ints));

AddToObjects(objects); // this works fine
AddToObjects(VarianceWorkaround.Convert<int, object>(ints));
}
static void ArrayExample()
{
    object[] objects = new string[10];

    // no problem, adding a string to a string[]
    objects[0] = "hello";

    // runtime exception, adding an object to a string[]
    objects[1] = new object();
}
}
}

```

See Also

Concepts

[C# Programming Guide](#)

[Generics \(C# Programming Guide\)](#)

Iterators (C# Programming Guide)

Iterators are a new feature in C# 2.0. An iterator is a method, [get](#) accessor or operator that enables you to support [foreach](#) iteration in a [class](#) or [struct](#) without having to implement the entire [IEnumerable](#) interface. Instead, you provide just an iterator, which simply traverses the data structures in your class. When the compiler detects your iterator, it will automatically generate the `Current`, `MoveNext` and `Dispose` methods of the **IEnumerator** or [IEnumerator<T>](#) interface.

Iterators Overview

- An iterator is a section of code that returns an ordered sequence of values of the same type.
- An iterator can be used as the body of a method, an operator, or a **get** accessor.
- The iterator code uses the **yield return** statement to return each element in turn. **yield break** ends the iteration. For more information, see [yield](#).
- Multiple iterators can be implemented on a class. Each iterator must have a unique name just like any class member, and can be invoked by client code in a **foreach** statement as follows: `foreach (int x in SampleClass.Iterator2) {}`
- The return type of an iterator must be **IEnumerator**, [IEnumerator](#), **IEnumerator<T>**, or [IEnumerator<T>](#).

The [yield](#) keyword is used to specify the value, or values, returned. When the **yield return** statement is reached, the current location is stored. Execution is restarted from this location the next time the iterator is called.

Iterators are especially useful with collection classes, providing an easy way to iterate non-trivial data structures such as binary trees.

Related Sections

For more information:

- [Using Iterators \(C# Programming Guide\)](#)
- [How to: Create an Iterator Block for a List of Integers \(C# Programming Guide\)](#)
- [How to: Create an Iterator Block for a Generic List \(C# Programming Guide\)](#)
- [Generic Interfaces \(C# Programming Guide\)](#)

Example

In this example, the class `DaysOfTheWeek` is a simple collection class that stores the days of the week as strings. After each iteration of a **foreach** loop, the next string in the collection is returned.

C#

```
public class DaysOfTheWeek : System.Collections.IEnumerable
{
    string[] m_Days = { "Sun", "Mon", "Tue", "Wed", "Thr", "Fri", "Sat" };

    public System.Collections.IEnumerator GetEnumerator()
    {
        for (int i = 0; i < m_Days.Length; i++)
        {
            yield return m_Days[i];
        }
    }
}

class TestDaysOfTheWeek
{
    static void Main()
    {
        // Create an instance of the collection class
        DaysOfTheWeek week = new DaysOfTheWeek();

        // Iterate with foreach
    }
}
```

```
foreach (string day in week)
{
    System.Console.Write(day + " ");
}
}
```

Output

Sun	Mon	Tue	Wed	Thr	Fri	Sat
-----	-----	-----	-----	-----	-----	-----

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 22 Iterators

See Also

Concepts

[C# Programming Guide](#)

[Generics \(C# Programming Guide\)](#)

Using Iterators (C# Programming Guide)

The most common way to create an iterator is to implement the `GetEnumerator` method on the `IEnumerable` interface, for example:

C#

```
public System.Collections.IEnumerator GetEnumerator()  
{  
    for (int i = 0; i < max; i++)  
    {  
        yield return i;  
    }  
}
```

The presence of the `GetEnumerator` method makes the type an enumerable type and allows using the `foreach` statement. If the method above was part of a class definition for `ListClass`, then it would be possible to use **foreach** on the class like this:

C#

```
static void Main()  
{  
    ListClass listClass1 = new ListClass();  
  
    foreach (int i in listClass1)  
    {  
        System.Console.WriteLine(i);  
    }  
}
```

The **foreach** statement invokes `ListClass.GetEnumerator()` and uses the returned enumerator to iterate through the values. For an example of how to create a generic iterator that returns a [IEnumerator](#) interface, see [How to: Create an Iterator Block for a Generic List \(C# Programming Guide\)](#).

It is also possible to use named iterators to support different ways of iterating through the same collection of data. For example, you could provide one iterator which returns elements in ascending order, and one which returns elements in descending order. An iterator can also have parameters to enable clients to control all or part of the iteration behavior. The following iterator implements the `IEnumerable` interface using the named iterator `SampleIterator`:

C#

```
// Implementing the enumerable pattern  
public System.Collections.IEnumerable SampleIterator(int start, int end)  
{  
    for (int i = start; i <= end; i++)  
    {  
        yield return i;  
    }  
}
```

The named iterator is invoked like this:

C#

```
ListClass test = new ListClass();  
foreach (int n in test.SampleIterator(1, 10))  
{  
    System.Console.WriteLine(n);  
}
```

You can use more than one `yield` statement in the same iterator as in the following example:

C#

```
public System.Collections.IEnumerator GetEnumerator()  
{  
    yield return "With an iterator, ";  
    yield return "more than one ";  
    yield return "value can be returned";  
    yield return ".";  
}
```

You can then print the results using the following **foreach** statement:

C#

```
foreach (string element in new TestClass())  
{  
    System.Console.Write(element);  
}
```

This example displays the following text:

With an iterator, more than one value can be returned.

On each successive iteration of the **foreach** loop (or the direct call to `IEnumerator.MoveNext`), the next iterator code body resumes after the previous **yield** statement and continues to the next until the end of the iterator body is reached, or a **yield break** statement is encountered.

See Also

Tasks

[How to: Create an Iterator Block for a List of Integers \(C# Programming Guide\)](#)

[How to: Create an Iterator Block for a Generic List \(C# Programming Guide\)](#)

Reference

[yield \(C# Reference\)](#)

[Using foreach with Arrays \(C# Programming Guide\)](#)

[foreach, in \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

How to: Create an Iterator Block for a List of Integers (C# Programming Guide)

In this example, an array of integers is used to build the list `SampleCollection`. A `for` loop iterates through the collection and yields the value of each item. Then a **foreach** loop is used to display the items of the collection.

Example

C#

```
// Declare the collection:
public class SampleCollection
{
    public int[] items;

    public SampleCollection()
    {
        items = new int[5] { 5, 4, 7, 9, 3 };
    }

    public System.Collections.IEnumerable BuildCollection()
    {
        for (int i = 0; i < items.Length; i++)
        {
            yield return items[i];
        }
    }
}

class MainClass
{
    static void Main()
    {
        SampleCollection col = new SampleCollection();

        // Display the collection items:
        System.Console.WriteLine("Values in the collection are:");
        foreach (int i in col.BuildCollection())
        {
            System.Console.Write(i + " ");
        }
    }
}
```

Output

Values in the collection are: 5 4 7 9 3
--

See Also

Tasks

[How to: Create an Iterator Block for a Generic List \(C# Programming Guide\)](#)

Reference

[Using Iterators \(C# Programming Guide\)](#)

[yield \(C# Reference\)](#)

[Array](#)

Concepts

[C# Programming Guide](#)

[Iterators \(C# Programming Guide\)](#)

How to: Create an Iterator Block for a Generic List (C# Programming Guide)

In this example the generic `class Stack<T>` implements the generic `interface IEnumerator<T>`. An array of the type `T` is declared and assigned values by using the method `Push`. In the `GetEnumerator` method, the values of the array are returned using the **yield return** statement.

The non-generic `GetEnumerator` is also implemented, because `IEnumerator<T>` inherits from `IEnumerator`. This example shows the typical implementation, which is for the non-generic method to simply forward the call to the generic method.

Example

C#

```
using System.Collections;
using System.Collections.Generic;

namespace GenericIteratorExample
{
    public class Stack<T> : IEnumerable<T>
    {
        private T[] values = new T[100];
        private int top = 0;

        public void Push(T t) { values[top++] = t; }
        public T Pop() { return values[--top]; }

        // These make Stack<T> implement IEnumerable<T> allowing
        // a stack to be used in a foreach statement.
        public IEnumerator<T> GetEnumerator()
        {
            for (int i = top; --i >= 0; )
            {
                yield return values[i];
            }
        }

        IEnumerator IEnumerable.GetEnumerator()
        {
            return GetEnumerator();
        }

        // Iterate from top to bottom.
        public IEnumerable<T> TopToBottom
        {
            get
            {
                // Since we implement IEnumerable<T>
                // and the default iteration is top to bottom,
                // just return the object.
                return this;
            }
        }

        // Iterate from bottom to top.
        public IEnumerable<T> BottomToTop
        {
            get
            {
                for (int i = 0; i < top; i++)
                {
                    yield return values[i];
                }
            }
        }
    }
}
```

```

//A parameterized iterator that return n items from the top
public IEnumerable<T> TopN(int n)
{
    // in this example we return less than N if necessary
    int j = n >= top ? 0 : top - n;

    for (int i = top; --i >= j; )
    {
        yield return values[i];
    }
}

//This code uses a stack and the TopToBottom and BottomToTop properties
//to enumerate the elements of the stack.
class Test
{
    static void Main()
    {
        Stack<int> s = new Stack<int>();
        for (int i = 0; i < 10; i++)
        {
            s.Push(i);
        }

        // Prints: 9 8 7 6 5 4 3 2 1 0
        // Foreach legal since s implements IEnumerable<int>
        foreach (int n in s)
        {
            System.Console.Write("{0} ", n);
        }
        System.Console.WriteLine();

        // Prints: 9 8 7 6 5 4 3 2 1 0
        // Foreach legal since s.TopToBottom returns IEnumerable<int>
        foreach (int n in s.TopToBottom)
        {
            System.Console.Write("{0} ", n);
        }
        System.Console.WriteLine();

        // Prints: 0 1 2 3 4 5 6 7 8 9
        // Foreach legal since s.BottomToTop returns IEnumerable<int>
        foreach (int n in s.BottomToTop)
        {
            System.Console.Write("{0} ", n);
        }
        System.Console.WriteLine();

        // Prints: 9 8 7 6 5 4 3
        // Foreach legal since s.TopN returns IEnumerable<int>
        foreach (int n in s.TopN(7))
        {
            System.Console.Write("{0} ", n);
        }
        System.Console.WriteLine();
    }
}

```

Output

9	8	7	6	5	4	3	2	1	0
9	8	7	6	5	4	3	2	1	0
0	1	2	3	4	5	6	7	8	9
9	8	7	6	5	4	3			

See Also

Tasks

[How to: Create an Iterator Block for a List of Integers \(C# Programming Guide\)](#)

Reference

[Using Iterators \(C# Programming Guide\)](#)

[System.Collections.Generic](#)

[IEnumerable](#)

Concepts

[C# Programming Guide](#)

[Iterators \(C# Programming Guide\)](#)

Namespaces (C# Programming Guide)

Namespaces are heavily used in C# programming in two ways. First, the .NET Framework uses namespaces to organize its many classes, as follows:

C#

```
System.Console.WriteLine("Hello World!");
```

System is a namespace and **Console** is a class contained within that namespace. The **using** keyword can be used so that the entire name is not required, like this:

C#

```
using System;
```

C#

```
Console.WriteLine("Hello");  
Console.WriteLine("World!");
```

For more information, see the topic [using Directive \(C# Reference\)](#).

Second, declaring your own namespaces can help you control the scope of class and method names in larger programming projects. Use the [namespace](#) keyword to declare a namespace, as in the following example:

C#

```
namespace SampleNamespace  
{  
    class SampleClass  
    {  
        public void SampleMethod()  
        {  
            System.Console.WriteLine(  
                "SampleMethod inside SampleNamespace");  
        }  
    }  
}
```

Namespaces Overview

A namespace has the following properties:

- They organize large code projects.
- They are delimited with the `.` operator.
- The **using directive** means you do not need to specify the name of the namespace for every class.
- The **global** namespace is the "root" namespace: **global::system** will always refer to the .NET Framework namespace **System**.

Related Sections

See the following topics for more information on namespaces:

- [Using Namespaces \(C# Programming Guide\)](#)
- [How to: Use the Namespace Alias Qualifier \(C# Programming Guide\)](#)
- [How to: Use the My Namespace \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [9 Namespaces](#)

See Also

Reference

[Namespace Keywords \(C# Reference\)](#)

[using Directive \(C# Reference\)](#)

[:: Operator \(C# Reference\)](#)

[. Operator \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Using Namespaces (C# Programming Guide)

Namespaces are heavily used within C# programs in two ways. Firstly, the .NET Framework classes use namespaces to organize its many classes. Secondly, declaring your own namespaces can help control the scope of class and method names in larger programming projects.

Accessing Namespaces

Most C# applications begin with a section of **using** directives. This section lists the namespaces that the application will be using frequently, and saves the programmer from specifying a fully qualified name every time a method contained within is used.

For example, by including the line:

C#

```
using System;
```

At the start of a program, the programmer can use the code:

C#

```
Console.WriteLine("Hello, World!");
```

Instead of:

C#

```
System.Console.WriteLine("Hello, World!");
```

Namespace Aliases

The [using Directive \(C# Reference\)](#) can also be used to create an alias for a [namespace](#). For example, if you are using a previously written namespace that contains nested namespaces, you might want to declare an alias to provide a shorthand way of referencing one in particular, like this:

C#

```
using Co = Company.Proj.Nested; // define an alias to represent a namespace
```

Using Namespaces to control scope

The **namespace** keyword is used to declare a scope. The ability to create scopes within your project helps organize code and provides a way to create globally-unique types. In the following example, a class entitled `SampleClass` is defined in two namespaces, one nested inside the other. The [. Operator \(C# Reference\)](#) is used to differentiate which method gets called.

C#

```
namespace SampleNamespace
{
    class SampleClass
    {
        public void SampleMethod()
        {
            System.Console.WriteLine(
                "SampleMethod inside SampleNamespace");
        }
    }

    // Create a nested namespace, and define another class.
    namespace NestedNamespace
```

```

{
    class SampleClass
    {
        public void SampleMethod()
        {
            System.Console.WriteLine(
                "SampleMethod inside NestedNamespace");
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        // Displays "SampleMethod inside SampleNamespace."
        SampleClass outer = new SampleClass();
        outer.SampleMethod();

        // Displays "SampleMethod inside SampleNamespace."
        SampleNamespace.SampleClass outer2 = new SampleNamespace.SampleClass();
        outer2.SampleMethod();

        // Displays "SampleMethod inside NestedNamespace."
        NestedNamespace.SampleClass inner = new NestedNamespace.SampleClass();
        inner.SampleMethod();
    }
}

```

Fully Qualified Names

Namespaces and types have unique titles described by fully qualified names that indicate a logical hierarchy. For example, the statement `A.B` implies that `A` is the name of the namespace or type, and `B` is nested inside it.

In the following example, there are nested classes and namespaces. The fully qualified name is indicated as a comment following each entity.

C#

```

namespace N1      // N1
{
    class C1      // N1.C1
    {
        class C2  // N1.C1.C2
        {
        }
    }
    namespace N2  // N1.N2
    {
        class C2  // N1.N2.C2
        {
        }
    }
}

```

In the preceding code segment:

- The namespace `N1` is a member of the global namespace. Its fully qualified name is `N1`.
- The namespace `N2` is a member of `N1`. Its fully qualified name is `N1.N2`.
- The class `C1` is a member of `N1`. Its fully qualified name is `N1.C1`.
- The class name `C2` is used twice in this code. However, the fully qualified names are unique. The first one is declared inside `C1`; thus, its fully qualified name is: `N1.C1.C2`. The second is declared inside a namespace `N2`; thus, its fully qualified

name is `N1.N2.C2`.

Using the preceding code segment, you can add a new class member, `C3`, to the namespace `N1.N2` as follows:

C#

```
namespace N1.N2
{
    class C3    // N1.N2.C3
    {
    }
}
```

In general, use `::` to reference a namespace alias or **global::** to reference the global namespace and `.` to qualify types or members.

It is an error to use `::` with an alias that references a type instead of a namespace. For example:

C#

```
using Alias = System.Console;
```

C#

```
class TestClass
{
    static void Main()
    {
        // Error
        //Alias::WriteLine("Hi");

        // OK
        Alias.WriteLine("Hi");
    }
}
```

Keep in mind that the word **global** is not a predefined alias; therefore, `global.X` does not have any special meaning. It acquires a special meaning only when used with `::`.

A warning (see [Compiler Warning \(level 2\) CS0440](#)) is generated if you define an alias named `global` because **global::** always references the global namespace and not an alias. For example, the following line generates the warning:

C#

```
using global = System.Collections;    // Warning
```

Using `::` with aliases is a good idea and protects against the unexpected introduction of additional types. For example, consider this example:

C#

```
using Alias = System;
```

C#

```
namespace Library
{
    public class C : Alias.Exception { }
}
```

This works, but if a type named `Alias` were to subsequently be introduced, `Alias.` would bind to that type instead. Using

`Alias::Exception` insures that `Alias` is treated as a namespace alias and not mistaken for a type.

See the topic [How to: Use the Namespace Alias Qualifier \(C# Programming Guide\)](#) for more details regarding the **global** alias.

See Also

Reference

[Namespace Keywords \(C# Reference\)](#)

[. Operator \(C# Reference\)](#)

[:: Operator \(C# Reference\)](#)

[extern \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Namespaces \(C# Programming Guide\)](#)

How to: Use the Namespace Alias Qualifier (C# Programming Guide)

The ability to access a member in the global `namespace` is useful when the member might be hidden by another entity of the same name.

For example, in the following code, `Console` resolves to `TestApp.Console` instead of to the **Console** type in the `System` namespace.

C#

```
using System;
```

C#

```
class TestApp
{
    // Define a new class called 'System' to cause problems.
    public class System { }

    // Define a constant called 'Console' to cause more problems.
    const int Console = 7;
    const int number = 66;

    static void Main()
    {
        // Error  Accesses TestApp.Console
        //Console.WriteLine(number);
    }
}
```

Using **System.Console** still results in an error because the **System** namespace is hidden by the class `TestApp.System`:

C#

```
// Error  Accesses TestApp.System
System.Console.WriteLine(number);
```

However, you can work around this error by using `global::System.Console`, like this:

C#

```
// OK
global::System.Console.WriteLine(number);
```

When the left identifier is **global**, the search for the right identifier starts at the global namespace. For example, the following declaration is referencing `TestApp` as a member of the global space.

C#

```
class TestClass : global::TestApp
```

Obviously, creating your own namespaces called `System` is not recommended, and it is unlikely you will encounter any code in which this has happened. However, in larger projects, it is a very real possibility that namespace duplication may occur in one form or another. In these situations, the global namespace qualifier is your guarantee that you can specify the root namespace.

Example

In this example, the namespace **System** is used to include the class `TestClass` therefore, `global::System.Console` must be

used to reference the **System.Console** class, which is hidden by the **System** namespace. Also, the alias `colAlias` is used to refer to the namespace **System.Collections**; therefore, the instance of a [System.Collections.Hashtable](#) was created using this alias instead of the namespace.

C#

```
using colAlias = System.Collections;
namespace System
{
    class TestClass
    {
        static void Main()
        {
            // Searching the alias:
            colAlias::Hashtable test = new colAlias::Hashtable();

            // Add items to the table.
            test.Add("A", "1");
            test.Add("B", "2");
            test.Add("C", "3");

            foreach (string name in test.Keys)
            {
                // Searching the global namespace:
                global::System.Console.WriteLine(name + " " + test[name]);
            }
        }
    }
}
```

Sample Output

A	1
B	2
C	3

See Also

Reference

[. Operator \(C# Reference\)](#)

[:: Operator \(C# Reference\)](#)

[extern \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Namespaces \(C# Programming Guide\)](#)

How to: Use the My Namespace (C# Programming Guide)

The [Microsoft.VisualBasic.MyServices](#) namespace (**My** in Visual Basic) provides easy and intuitive access to a number of .NET Framework classes, enabling you to write code that interacts with the computer, application, settings, resources, and so on. Although originally designed for use with Visual Basic, the **MyServices** namespace can be used in C# applications.

For more information about using the **MyServices** namespace from Visual Basic, see [Development with My](#).

Adding a Reference

Before you can use the **MyServices** classes in your solution, you must add a reference to the Visual Basic library.

To add a reference to the Visual Basic library

1. In **Solution Explorer**, right-click the **References** node, and select **Add Reference**.
2. When the **References** dialog box appears, scroll down the list, and select `Microsoft.VisualBasic.dll`.

You might also want to include the following line in the **using** section at the start of your program.

C#

```
using Microsoft.VisualBasic.Devices;
```

Example

This example calls various static methods contained in the **MyServices** namespace. For this code to compile, a reference to `Microsoft.VisualBasic.DLL` must be added to the project.

C#

```
using System;
using Microsoft.VisualBasic.Devices;

class TestMyServices
{
    static void Main()
    {
        // Play a sound with the Audio class:
        Audio myAudio = new Audio();
        Console.WriteLine("Playing sound...");
        myAudio.Play(@"c:\WINDOWS\Media\chimes.wav");

        // Display time information with the Clock class:
        Clock myClock = new Clock();
        Console.WriteLine("Current day of the week: ");
        Console.WriteLine(myClock.LocalTime.DayOfWeek);
        Console.WriteLine("Current date and time: ");
        Console.WriteLine(myClock.LocalTime);

        // Display machine information with the Computer class:
        Computer myComputer = new Computer();
        Console.WriteLine("Computer name: " + myComputer.Name);

        if (myComputer.Network.IsAvailable)
        {
            Console.WriteLine("Computer is connected to network.");
        }
        else
        {
            Console.WriteLine("Computer is not connected to network.");
        }
    }
}
```

Not all the classes in the **MyServices** namespace can be called from a C# application: for example, the [FileSystemProxy](#) class is not compatible. In this particular case, the static methods that are part of [FileSystem](#), which are also contained in VisualBasic.dll, can be used instead. For example, here is how to use one such method to duplicate a directory:

C#

```
// Duplicate a directory
Microsoft.VisualBasic.FileIO.FileSystem.CopyDirectory(
    @"C:\original_directory",
    @"C:\copy_of_original_directory");
```

See Also

Concepts

[C# Programming Guide](#)

[Namespaces \(C# Programming Guide\)](#)

[Using Namespaces \(C# Programming Guide\)](#)

Nullable Types (C# Programming Guide)

Nullable types are instances of the [System.Nullable](#) struct. A nullable type can represent the normal range of values for its underlying value type, plus an additional **null** value. For example, a `Nullable<Int32>`, pronounced "Nullable of Int32," can be assigned any value from -2147483648 to 2147483647, or it can be assigned the **null** value. A `Nullable<bool>` can be assigned the values **true** or **false**, or **null**. The ability to assign **null** to numeric and Boolean types is particularly useful when dealing with databases and other data types containing elements that may not be assigned a value. For example, a Boolean field in a database can store the values **true** or **false**, or it may be undefined.

C#

```
class NullableExample
{
    static void Main()
    {
        int? num = null;
        if (num.HasValue == true)
        {
            System.Console.WriteLine("num = " + num.Value);
        }
        else
        {
            System.Console.WriteLine("num = Null");
        }

        //y is set to zero
        int y = num.GetValueOrDefault();

        // num.Value throws an InvalidOperationException if num.HasValue is false
        try
        {
            y = num.Value;
        }
        catch (System.InvalidOperationException e)
        {
            System.Console.WriteLine(e.Message);
        }
    }
}
```

The above will display the output:

```
num = Null
```

Nullable object must have a value.

Nullable Types Overview

Nullable types have the following characteristics:

- Nullable types represent value-type variables that can be assigned the value of **null**. You cannot create a nullable type based on a reference type. (Reference types already support the **null** value.)
- The syntax **T?** is shorthand for `System.Nullable<T>`, where **T** is a value type. The two forms are interchangeable.
- Assign a value to a nullable type in the same way as for an ordinary value type, for example `int? x = 10;` or `double? d = 4.108;`
- Use the [System.Nullable.GetValueOrDefault](#) property to return either the assigned value, or the default value for the underlying type if the value is **null**, for example `int j = x.GetValueOrDefault();`
- Use the [HasValue](#) and [Value](#) read-only properties to test for null and retrieve the value, for example `if (x.HasValue) j = x.Value;`
 - The **HasValue** property returns true if the variable contains a value, or false if it is null.

- The **Value** property returns a value if one is assigned, otherwise a [System.InvalidOperationException](#) is thrown.
- The default value for a nullable type variable sets **HasValue** to **false**. The **Value** is undefined.
- Use the **??** operator to assign a default value that will be applied when a nullable type whose current value is null is assigned to a non-nullable type, for example `int? x = null; int y = x ?? -1;`
- Nested nullable types are not allowed. The following line will not compile: `Nullable<Nullable<int>> n;`

Related Sections

For more information:

- [Using Nullable Types \(C# Programming Guide\)](#)
- [Boxing Nullable Types \(C# Programming Guide\)](#)
- [?? Operator \(C# Reference\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 24 Nullable Types

See Also

Tasks

[Nullable Sample](#)

Reference

[Nullable](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C#](#)

[C# Reference](#)

Using Nullable Types (C# Programming Guide)

Nullable types can represent all the values of an underlying type, and an additional **null** value. Nullable types are declared in one of two ways:

```
System.Nullable<T> variable
```

-or-

```
T? variable
```

T is the underlying type of the nullable type. **T** can be any value type including **struct**; it cannot be a reference type.

For an example of when you might use a nullable type, consider how an ordinary Boolean variable can have two values: true and false. There is no value that signifies "undefined". In many programming applications, most notably database interactions, variables can exist in an undefined state. For example, a field in a database may contain the values true or false, but it may also contain no value at all. Similarly, reference types can be set to **null** to indicate that they are not initialized.

This disparity can create extra programming work, with additional variables used to store state information, the use of special values, and so on. The nullable type modifier enables C# to create value-type variables that indicate an undefined value.

Examples of Nullable Types

Any value type may be used as the basis for a nullable type. For example:

C#

```
int? i = 10;
double? d1 = 3.14;
bool? flag = null;
char? letter = 'a';
int?[] arr = new int?[10];
```

The Members of Nullable Types

Each instance of a nullable type has two public read-only properties:

- **HasValue**

HasValue is of type **bool**. It is set to **true** when the variable contains a non-null value.

- **Value**

Value is of the same type as the underlying type. If **HasValue** is **true**, **Value** contains a meaningful value. If **HasValue** is **false**, accessing **Value** will throw a [InvalidOperationException](#).

In this example, the **HasValue** member is used to test if the variable contains a value before attempting to display it.

C#

```
int? x = 10;
if (x.HasValue)
{
    System.Console.WriteLine(x.Value);
}
else
{
    System.Console.WriteLine("Undefined");
}
```

Testing for a value can also be done like this:

C#

```
int? y = 10;
if (y != null)
{
```

```
        System.Console.WriteLine(y.Value);
    }
    else
    {
        System.Console.WriteLine("Undefined");
    }
}
```

Explicit Conversions

A nullable type can be cast to a regular type, either explicitly with a cast, or by using the `Value` property. For example:

C#

```
int? n = null;

//int m1 = n;      // Will not compile.
int m2 = (int)n;    // Compiles, but will create an exception if x is null.
int m3 = n.Value;   // Compiles, but will create an exception if x is null.
```

If a user-defined conversion is defined between two data types, the same conversion can also be used with the nullable versions of these data types.

Implicit Conversions

A variable of nullable type can be set to null with the **null** keyword, as shown below:

C#

```
int? n1 = null;
```

The conversion from an ordinary type to a nullable type, is implicit.

C#

```
int? n2;
n2 = 10; // Implicit conversion.
```

Operators

The predefined unary and binary operators and any user-defined operators that exist for value types may also be used by nullable types. These operators produce a null value if the operands are null; otherwise, the operator uses the contained value to calculate the result. For example:

C#

```
int? a = 10;
int? b = null;

a++;      // Increment by 1, now a is 11.
a = a * 10; // Multiply by 10, now a is 110.
a = a + b;  // Add b, now a is null.
```

When performing comparisons with nullable types, if one of the nullable types is **null**, the comparison is always evaluated to be **false**. It is therefore important not to assume that because a comparison is **false**, the opposite case is **true**. For example:

C#

```
int? num1 = 10;
int? num2 = null;
if (num1 >= num2)
{
    System.Console.WriteLine("num1 is greater than or equal to num1");
}
```

```

else
{
    // num1 is NOT less than num2
}

```

The conclusion in the **else** statement above is not valid because `num2` is **null** and therefore doesn't contain a value.

A comparison of two nullable types which are both **null** will be evaluated to **true**.

The ?? Operator

The ?? operator defines a default value that is returned when a nullable type is assigned to a non-nullable type.

C#

```

int? c = null;

// d = c, unless c is null, in which case d = -1.
int d = c ?? -1;

```

This operator can also be used with multiple nullable types. For example:

C#

```

int? e = null;
int? f = null;

// g = e or f, unless e and f are both null, in which case g = -1.
int g = e ?? f ?? -1;

```

The bool? type

The bool? nullable type can contain three different values: **true**, **false** and **null**. As such, the cannot be used in conditionals such as with **if**, **for**, or **while**. For example, this code fails to compile with [Compiler Error CS0266](#):

```

bool? b = null;
if (b) // Error CS0266.
{
}

```

This is not allowed because it is unclear what **null** means in the context of a conditional. Nullable Booleans can be cast to a **bool** explicitly in order to be used in a conditional, but if the object has a value of **null**, **InvalidOperationException** will be thrown. It is therefore important to check the **HasValue** property before casting to **bool**.

Nullable Booleans are similar to the Boolean variable type used in SQL. To ensure that the results produced by the **&** and **|** operators are consistent with SQL's three-valued Boolean type, the following predefined operators are provided:

```
bool? operator &(bool? x, bool? y)
```

```
bool? operator |(bool? x, bool? y)
```

The results of these operators are listed in the table below:

X	y	x&y	x y
True	true	True	true
True	false	False	true
True	null	Null	true
False	true	False	true

False	false	False	false
False	null	False	null
Null	true	Null	true
Null	false	False	null
Null	null	Null	null

See Also

Reference

[Boxing Nullable Types \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Nullable Types \(C# Programming Guide\)](#)

Boxing Nullable Types (C# Programming Guide)

Objects based on nullable types are only boxed if the object is non-null. If [HasValue](#) is **false**, then, instead of boxing, the object reference is simply assigned to **null**. For example:

```
bool? b = null;
object o = b;
// Now o is null.
```

If the object is non-null -- if **HasValue** is **true** -- then boxing takes place, but only the underlying type that the nullable object is based upon is boxed. Boxing a non-null nullable value type boxes the value type itself, not the [System.Nullable](#) that wraps the value type. For example:

```
bool? b = false;
int? i = 44;
object bBoxed = b; // bBoxed contains a boxed bool.
object iBoxed = i; // iBoxed contains a boxed int.
```

The two boxed objects are identical to those created by boxing non-nullable types. And, like non-nullable boxed types, can be unboxed into nullable types, like this:

```
bool? b2 = (bool?)bBoxed;
int? i2 = (int?)iBoxed;
```

Remarks

The behavior of nullable types when boxed provides two advantages:

1. Nullable objects and their boxed counterpart can be tested for null:

```
bool? b = null;
object boxedB = b;
if (b == null)
{
    // True.
}
if (boxedB == null)
{
    // Also true.
}
```

2. Boxed nullable types fully support the functionality of the underlying type:

```
double? d = 44.4;
object iBoxed = d;
// Access IConvertible interface implemented by double.
IConvertible ic = (IConvertible)iBoxed;
int i = ic.ToInt32(null);
string str = ic.ToString();
```

For more examples of nullable types, including boxing behavior, see [Nullable Sample](#).

See Also

Tasks

[How to: Identify a Nullable Type \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Nullable Types \(C# Programming Guide\)](#)

How to: Identify a Nullable Type (C# Programming Guide)

You can use the C# [typeof](#) operator to create a [Type](#) object that represents a Nullable type:

```
System.Type type = typeof(int?);
```

You can also use the classes and methods of the [System.Reflection](#) namespace to generate **Type** objects that represent Nullable types. However, if you attempt to obtain type information from Nullable variables at runtime using the [GetType](#) method or the **is** operator, the result is a **Type** object that represents the underlying type, not the Nullable type itself.

Calling **GetType** on a Nullable type causes a boxing operation to be performed when the type is implicitly converted to [Object](#). Therefore **GetType** always returns a **Type** object that represents the underlying type, not the Nullable type.

```
int? i = 5;
Type t = i.GetType();
Console.WriteLine(t.FullName); //"System.Int32"
```

The C# **is** operator also operates on a Nullable's underlying type. Therefore you cannot use **is** to determine whether a variable is a Nullable type. The following example shows that the **is** operator treats a `Nullable<int>` variable as an `int`.

```
static void Main(string[] args)
{
    int? i = 5;
    if (i is int) // true
        //...
}
```

Example

Use the following code to determine whether a **Type** object represents a Nullable type. Remember that this code always returns false if the **Type** object was returned from a call to **GetType**, as explained above.

```
if (type.IsGenericType && type.GetGenericTypeDefinition() == typeof(Nullable<>)) {...}
```

See Also

Reference

[Boxing Nullable Types \(C# Programming Guide\)](#)

Concepts

[Nullable Types \(C# Programming Guide\)](#)

Unsafe Code and Pointers (C# Programming Guide)

To maintain type safety and security, by default C# does not support pointer arithmetic. However, by using the [unsafe](#) keyword, it is possible to define an unsafe context in which pointers can be used. For more information about pointers, see the topic [Pointer types](#).

Note

In the common language runtime (CLR), unsafe code is referred to as unverifiable code. Unsafe code in C# is not necessarily dangerous; it is simply code whose safety cannot be verified by the CLR. The CLR will therefore only execute unsafe code if it is within a fully trusted assembly. If you use unsafe code, it is your responsibility to ensure that your code does not introduce security risks or pointer errors. For more information, see [Security and C#](#).

Unsafe Code Overview

Unsafe code has the following properties:

- Methods, types, and code blocks can be defined as unsafe.
- In some cases, unsafe code may increase an application's performance by removing array bounds checks.
- Unsafe code is required when calling native functions that require pointers.
- Using unsafe code introduces security and stability risks.
- In order for C# to compile unsafe code, the application must be compiled with [/unsafe](#).

Related Sections

For more information, see:

- [Pointer types \(C# Programming Guide\)](#)
- [Fixed Size Buffers \(C# Programming Guide\)](#)
- [How to: Use Pointers to Copy an Array of Bytes \(C# Programming Guide\)](#)
- [How to: Use the Windows ReadFile Function \(C# Programming Guide\)](#)
- [unsafe \(C# Reference\)](#)
- [Unsafe Code Sample](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 18 Unsafe Code
- B 3 Grammar extensions for unsafe code

See Also

Concepts

[C# Programming Guide](#)

Fixed Size Buffers (C# Programming Guide)

In C# 2.0, you can use the [fixed](#) statement to create a with a fixed size array in a data structure. This is useful when working with existing code, such as code written in other languages, pre-existing DLLs or COM projects. The fixed array can take any of the attributes or modifiers that are allowed for regular struct members. The only restriction is that the array type must be **bool**, **byte**, **char**, **short**, **int**, **long**, **sbyte**, **ushort**, **uint**, **ulong**, **float**, or **double**.

```
private fixed char name[30];
```

Remarks

In previous versions of C#, declaring a C++ style fixed-size structure was difficult because a C# struct containing an array does not contain the array elements, but instead contains a reference to the elements.

C# 2.0 adds the ability to embed an array of fixed size in a [struct](#) when used in an [unsafe](#) code block.

For example, prior to C# 2.0, the following **struct** would be 8 bytes in size where the `pathName` array is a reference to the heap-allocated array:

C#

```
public struct MyArray
{
    public char[] pathName;
    private int reserved;
}
```

In C# 2.0, a **struct** can be declared with an embedded array:

C#

```
public struct MyArray // This code must appear in an unsafe block
{
    public fixed char pathName[128];
}
```

In this structure, the `pathName` array is of fixed size and location, and can therefore be used with other unsafe code.

The size of the 128 element **char** array is 256 bytes. Fixed size [char](#) buffers always take two bytes per character, irrespective of the encoding. This is true even when char buffers are marshaled to API methods or structs with `CharSet = CharSet.Auto` or `CharSet = CharSet.Ansi`. For more information, see [CharSet](#).

Another common fixed-size array is the [bool](#) array. The elements in a **bool** array are always one byte in size. **bool** arrays are not suitable for creating bit arrays or buffers.

Note

Except for memory created with [stackalloc](#), the C# compiler and the common language runtime (CLR) do not perform any security buffer overrun checks. As with all unsafe code, use caution.

Unsafe buffers are different from normal arrays in the following ways:

- You can only use unsafe buffers in an unsafe context.
- Unsafe buffers are always vectors, or one-dimensional arrays.
- The declaration of the array should include a count, such as `char id[8]`. You cannot use `char id[]` instead.
- Unsafe buffers can only be instance fields of structs in an unsafe context.

See Also

Tasks

[How to: Call Native DLLs from Managed Code Using PInvoke](#)

[How to: Marshal Arrays Using PInvoke](#)

[How to: Marshal Embedded Pointers Using PInvoke](#)

[How to: Marshal Function Pointers Using PInvoke](#)

[How to: Marshal Strings Using PInvoke](#)

Reference

[fixed Statement \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

[Default Marshaling for Arrays](#)

How to: Use the Windows ReadFile Function (C# Programming Guide)

This example demonstrates the Windows **ReadFile** function by reading and displaying a text file. The **ReadFile** function requires the use of **unsafe** code because it requires a pointer as a parameter.

The byte array passed into the `Read` function is a managed type. This means that the common language runtime (CLR) garbage collector could relocate the memory used by the array at will. To prevent this, **fixed** is used to get a pointer to the memory and mark it so that the garbage collector will not move it. At the end of the **fixed** block, the memory automatically returns to being subject to moving through garbage collection.

This capability is known as *declarative pinning*. The nice part about pinning is that there is very little overhead unless a garbage collection occurs in the **fixed** block, which is an unlikely occurrence.

Example

```
C#
class FileReader
{
    const uint GENERIC_READ = 0x80000000;
    const uint OPEN_EXISTING = 3;
    System.IntPtr handle;

    [System.Runtime.InteropServices.DllImport("kernel32", SetLastError = true)]
    static extern unsafe System.IntPtr CreateFile
    (
        string FileName,           // file name
        uint DesiredAccess,        // access mode
        uint ShareMode,           // share mode
        uint SecurityAttributes,   // Security Attributes
        uint CreationDisposition, // how to create
        uint FlagsAndAttributes,   // file attributes
        int hTemplateFile          // handle to template file
    );

    [System.Runtime.InteropServices.DllImport("kernel32", SetLastError = true)]
    static extern unsafe bool ReadFile
    (
        System.IntPtr hFile,       // handle to file
        void* pBuffer,             // data buffer
        int NumberOfBytesToRead,    // number of bytes to read
        int* pNumberOfBytesRead,    // number of bytes read
        int Overlapped              // overlapped buffer
    );

    [System.Runtime.InteropServices.DllImport("kernel32", SetLastError = true)]
    static extern unsafe bool CloseHandle
    (
        System.IntPtr hObject // handle to object
    );

    public bool Open(string FileName)
    {
        // open the existing file for reading
        handle = CreateFile
        (
            FileName,
            GENERIC_READ,
            0,
            0,
            OPEN_EXISTING,
            0,
            0
        );
    }
}
```

```

        if (handle != System.IntPtr.Zero)
        {
            return true;
        }
        else
        {
            return false;
        }
    }

    public unsafe int Read(byte[] buffer, int index, int count)
    {
        int n = 0;
        fixed (byte* p = buffer)
        {
            if (!ReadFile(handle, p + index, count, &n, 0))
            {
                return 0;
            }
        }
        return n;
    }

    public bool Close()
    {
        return CloseHandle(handle);
    }
}

class Test
{
    static int Main(string[] args)
    {
        if (args.Length != 1)
        {
            System.Console.WriteLine("Usage : ReadFile <FileName>");
            return 1;
        }

        if (!System.IO.File.Exists(args[0]))
        {
            System.Console.WriteLine("File " + args[0] + " not found.");
            return 1;
        }

        byte[] buffer = new byte[128];
        FileReader fr = new FileReader();

        if (fr.Open(args[0]))
        {
            // Assume that an ASCII file is being read.
            System.Text.ASCIIEncoding Encoding = new System.Text.ASCIIEncoding();

            int bytesRead;
            do
            {
                bytesRead = fr.Read(buffer, 0, buffer.Length);
                string content = Encoding.GetString(buffer, 0, bytesRead);
                System.Console.Write("{0}", content);
            }
            while (bytesRead > 0);

            fr.Close();
            return 0;
        }
        else
        {
            System.Console.WriteLine("Failed to open requested file");
        }
    }
}

```

```
    }  
    }  
    return 1;  
}
```

See Also

Reference

[Pointer types \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

[Garbage Collection](#)

Pointer types (C# Programming Guide)

In an unsafe context, a type may be a pointer type as well as a value type or a reference type. A pointer type declaration takes one of the following forms:

```
type* identifier;
void* identifier; //allowed but not recommended
```

Any of the following types may be a pointer type:

- [sbyte](#), [byte](#), [short](#), [ushort](#), [int](#), [uint](#), [long](#), [ulong](#), [char](#), [float](#), [double](#), [decimal](#), or [bool](#).
- Any [enum](#) type.
- Any pointer type.
- Any user-defined struct type that contains fields of unmanaged types only.

Pointer types do not inherit from [object](#) and no conversions exist between pointer types and **object**. Also, boxing and unboxing do not support pointers. However, you can convert between different pointer types and between pointer types and integral types.

When you declare multiple pointers in the same declaration, the `*` is written along with the underlying type only, not as a prefix to each pointer name. For example:

```
int* p1, p2, p3;    // Ok
int *p1, *p2, *p3;  // Invalid in C#
```

A pointer cannot point to a reference or to a [struct](#) that contains references because it is possible for an object reference to be garbage collected even if a pointer is pointing to it. The GC does not keep track of whether an object is being pointed to by any pointer types.

The value of the pointer variable of type `myType*` is the address of a variable of type `myType`. The following are examples of pointer type declarations:

Example	Description
<code>int* p</code>	p is a pointer to an integer
<code>int** p</code>	p is a pointer to pointer to an integer
<code>int*[] p</code>	p is a single-dimensional array of pointers to integers
<code>char* p</code>	p is a pointer to a char
<code>void* p</code>	p is a pointer to an unknown type

The pointer indirection operator `*` can be used to access the contents at the location pointed to by the pointer variable. For example, for the following declaration,

```
int* myVariable;
```

the expression `*myVariable` denotes the **int** variable found at the address contained in `myVariable`.

You cannot apply the indirection operator to a pointer of type `void*`. However, you can use a cast to convert a void pointer to any other pointer type, and vice versa.

A pointer can be **null**. Applying the indirection operator to a null pointer results in an implementation-defined behavior.

Be aware that passing pointers between methods can cause undefined behavior. Examples are returning a pointer to a local variable via an Out or Ref parameter or as the function result. If the pointer was set in a fixed block, the variable to which it

points may no longer be fixed.

The following table lists the operators and statements that can operate on pointers in an unsafe context:

Operator/Statement	Use
*	to perform pointer indirection.
->	to access a member of a struct through a pointer.
[]	to index a pointer.
&	to obtain the address of a variable.
++ and --	to increment and decrement pointers.
+ and -	to perform pointer arithmetic.
=, !=, <, >, <=, and >=	to compare pointers.
stackalloc	to allocate memory on the stack.
fixed statement	to temporarily fix a variable in order that its address may be found.

C# Language Specification

For more information, see the following section in the [C# Language Specification](#):

- 18 Unsafe Code

See Also

Reference

[Pointer Conversions \(C# Programming Guide\)](#)

[Pointer Expressions \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

[Boxing and Unboxing \(C# Programming Guide\)](#)

Other Resources

[Types \(C# Reference\)](#)

Pointer Conversions (C# Programming Guide)

The following table shows the predefined implicit pointer conversions. Implicit conversions might occur in many situations, including method invoking and assignment statements.

Implicit pointer conversions

From	To
Any pointer type	void*
null	Any pointer type

Explicit pointer conversion is used to perform conversions, for which there is no implicit conversion, by using a cast expression. The following table shows these conversions.

Explicit pointer conversions

From	To
Any pointer type	Any other pointer type
sbyte, byte, short, ushort, int, uint, long, or ulong	Any pointer type
Any pointer type	sbyte, byte, short, ushort, int, uint, long, or ulong

Example

In the following example, a pointer to **int** is converted to a pointer to **byte**. Notice that the pointer points to the lowest addressed byte of the variable. When you successively increment the result, up to the size of **int** (4 bytes), you can display the remaining bytes of the variable.

C#

```
// compile with: /unsafe
```

C#

```
class ClassConvert
{
    static void Main()
    {
        int number = 1024;

        unsafe
        {
            // Convert to byte:
            byte* p = (byte*)&number;

            System.Console.WriteLine("The 4 bytes of the integer:");

            // Display the 4 bytes of the int variable:
            for (int i = 0 ; i < sizeof(int) ; ++i)
            {
                System.Console.WriteLine(" {0:X2}", *p);
                // Increment the pointer:
                p++;
            }
            System.Console.WriteLine();
            System.Console.WriteLine("The value of the integer: {0}", number);
        }
    }
}
```

Output

The 4 bytes of the integer: 00 04 00 00

The value of the integer: 1024

See Also

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Types \(C# Reference\)](#)

Pointer Expressions (C# Programming Guide)

In this section, the following pointer expressions are discussed:

[Obtaining the Value of a Variable](#)

[Obtaining the Address of a Variable](#)

[How to: Access a Member with a Pointer \(C# Programming Guide\)](#)

[How to: Access an Array Element with a Pointer \(C# Programming Guide\)](#)

[Manipulating Pointers](#)

See Also

Reference

[Pointer Conversions \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Types \(C# Reference\)](#)

How to: Obtain the Value of a Pointer Variable (C# Programming Guide)

Use the pointer indirection operator to obtain the variable at the location pointed to by a pointer. The expression takes the following form, where `p` is a pointer type:

```
*p;
```

You cannot use the unary indirection operator on an expression of any type other than the pointer type. Also, you cannot apply it to a [void](#) pointer.

When you apply the indirection operator to a [null](#) pointer, the result depends on the implementation.

Example

In the following example, a variable of the type `char` is accessed using pointers of different types. Note that the address of `theChar` will vary from run to run, because the physical address allocated to a variable can change.

C#

```
// compile with: /unsafe
```

C#

```
unsafe class TestClass
{
    static void Main()
    {
        char theChar = 'Z';
        char* pChar = &theChar;
        void* pVoid = pChar;
        int* pInt = (int*)pVoid;

        System.Console.WriteLine("Value of theChar = {0}", theChar);
        System.Console.WriteLine("Address of theChar = {0:X2}", (int)pChar);
        System.Console.WriteLine("Value of pChar = {0}", *pChar);
        System.Console.WriteLine("Value of pInt = {0}", *pInt);
    }
}
```

Sample Output

```
Value of theChar = Z
Address of theChar = 12F718
Value of pChar = Z
Value of pInt = 90
```

See Also

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Types \(C# Reference\)](#)

How to: Obtain the Address of a Variable (C# Programming Guide)

To obtain the address of a unary expression, which evaluates to a fixed variable, use the address-of operator:

```
int number;  
int* p = &number; //address-of operator &
```

The address-of operator can only be applied to a variable. If the variable is a moveable variable, you can use the [fixed statement](#) to temporarily fix the variable before obtaining its address.

It is your responsibility to ensure that the variable is initialized. The compiler will not issue an error message if the variable is not initialized.

You cannot get the address of a constant or a value.

Example

In this example, a pointer to **int**, `p`, is declared and assigned the address of an integer variable, `number`. The variable `number` is initialized as a result of the assignment to `*p`. Commenting this assignment statement will remove the initialization of the variable `number`, but no compile-time error is issued. Notice the use of the [Member Access](#) operator `->` to obtain and display the address stored in the pointer.

C#

```
// compile with: /unsafe
```

C#

```
class AddressOfOperator  
{  
    static void Main()  
    {  
        int number;  
  
        unsafe  
        {  
            // Assign the address of number to a pointer:  
            int* p = &number;  
  
            // Commenting the following statement will remove the  
            // initialization of number.  
            *p = 0xffff;  
  
            // Print the value of *p:  
            System.Console.WriteLine("Value at the location pointed to by p: {0:X}", *p);  
  
            // Print the address stored in p:  
            System.Console.WriteLine("The address stored in p: {0}", p->ToString());  
        }  
  
        // Print the value of the variable number:  
        System.Console.WriteLine("Value of the variable number: {0:X}", number);  
    }  
}
```

Sample Output

```
Value at the location pointed to by p: FFFF  
The address stored in p: 65535  
Value of the variable number: FFFF
```

See Also

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Types \(C# Reference\)](#)

How to: Access a Member with a Pointer (C# Programming Guide)

To access a member of a struct that is declared in an unsafe context, you can use the member access operator as shown in the following example in which `p` is a pointer to a [struct](#) that contains a member `x`.

```
Coords* p = &home;  
p -> x = 25; //member access operator ->
```

Example

In this example, a [struct](#), `Coords`, that contains the two coordinates `x` and `y` is declared and instantiated. By using the member access operator `->` and a pointer to the instance `home`, `x` and `y` are assigned values.

Note

Notice that the expression `p->x` is equivalent to the expression `(*p).x`, and you can obtain the same result by using either of the two expressions.

C#

```
// compile with: /unsafe
```

C#

```
struct Coords  
{  
    public int x;  
    public int y;  
}  
  
class AccessMembers  
{  
    static void Main()  
    {  
        Coords home;  
  
        unsafe  
        {  
            Coords* p = &home;  
            p->x = 25;  
            p->y = 12;  
  
            System.Console.WriteLine("The coordinates are: x={0}, y={1}", p->x, p->y );  
        }  
    }  
}
```

See Also

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Types \(C# Reference\)](#)

How to: Access an Array Element with a Pointer (C# Programming Guide)

In an unsafe context, you can access an element in memory by using pointer element access, as shown in the following example:

```
char* charPointer = stackalloc char[123];
for (int i = 65; i < 123; i++)
{
    charPointer[i] = (char)i; //access array elements
}
```

The expression in square brackets must be implicitly convertible to **int**, **uint**, **long**, or **ulong**. The operation `p[e]` is equivalent to `*(p+e)`. Like C and C++, the pointer element access does not check for out-of-bounds errors.

Example

In this example, 123 memory locations are allocated to a character array, `charPointer`. The array is used to display the lowercase letters and the uppercase letters in two [for](#) loops.

Notice that the expression `charPointer[i]` is equivalent to the expression `*(charPointer + i)`, and you can obtain the same result by using either of the two expressions.

C#

```
// compile with: /unsafe
```

C#

```
class Pointers
{
    unsafe static void Main()
    {
        char* charPointer = stackalloc char[123];

        for (int i = 65; i < 123; i++)
        {
            charPointer[i] = (char)i;
        }

        // Print uppercase letters:
        System.Console.WriteLine("Uppercase letters:");
        for (int i = 65; i < 91; i++)
        {
            System.Console.Write(charPointer[i]);
        }
        System.Console.WriteLine();

        // Print lowercase letters:
        System.Console.WriteLine("Lowercase letters:");
        for (int i = 97; i < 123; i++)
        {
            System.Console.Write(charPointer[i]);
        }
    }
}
```

Sample Output

Uppercase letters: ABCDEFGHIJKLMNOPQRSTUVWXYZ Lowercase letters: abcdefghijklmnopqrstuvwxyz
--

See Also

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Types \(C# Reference\)](#)

Manipulating Pointers (C# Programming Guide)

This section includes the following pointer operations:

[Increment and Decrement](#)

[Arithmetic Operations](#)

[Comparison](#)

[See Also](#)

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[C# Operators](#)

[Pointer types \(C# Programming Guide\)](#)

[/unsafe \(Enable Unsafe Mode\) \(C# Compiler Options\)](#)

Concepts

[C# Programming Guide](#)

How to: Increment and Decrement Pointers (C# Programming Guide)

Use the increment and the decrement operators, `++` and `--`, to change the pointer location by `sizeof (pointer-type)` for a pointer of type `pointer-type*`. The increment and decrement expressions take the following form:

```
++p;
P++;
--p;
p--;
```

The increment and decrement operators can be applied to pointers of any type except the type `void*`.

The effect of applying the increment operator to a pointer of the type `pointer-type` is to add `sizeof (pointer-type)` to the address contained in the pointer variable.

The effect of applying the decrement operator to a pointer of the type `pointer-type` is to subtract **`sizeof (pointer-type)`** from the address contained in the pointer variable.

No exceptions are generated when the operation overflows the domain of the pointer, and the result depends on the implementation.

Example

In this example, you step through an array by incrementing the pointer by the size of `int`. With each step, you display the address and the content of the array element.

C#

```
// compile with: /unsafe
```

C#

```
class IncrDecr
{
    unsafe static void Main()
    {
        int[] numbers = {0,1,2,3,4};

        // Assign the array address to the pointer:
        fixed (int* p1 = numbers)
        {
            // Step through the array elements:
            for(int* p2=p1; p2<p1+numbers.Length; p2++)
            {
                System.Console.WriteLine("Value:{0} @ Address:{1}", *p2, (long)p2);
            }
        }
    }
}
```

Sample Output

Value:0 @ Address:12860272
Value:1 @ Address:12860276
Value:2 @ Address:12860280
Value:3 @ Address:12860284
Value:4 @ Address:12860288

See Also

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[C# Operators](#)

[Manipulating Pointers \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Types \(C# Reference\)](#)

Arithmetic Operations on Pointers (C# Programming Guide)

This topic discusses using the arithmetic operators `+` and `-` to manipulate pointers.

Note

You cannot perform any arithmetic operations on void pointers.

Adding and Subtracting Numeric Values to or From Pointers

You can add a value `n` of type `int`, `uint`, `long`, or `ulong` to a pointer, `p`, of any type except `void*`. The result `p+n` is the pointer resulting from adding `n * sizeof(p)` to the address of `p`. Similarly, `p-n` is the pointer resulting from subtracting `n * sizeof(p)` from the address of `p`.

Subtracting Pointers

You can also subtract pointers of the same type. The result is always of the type **long**. For example, if `p1` and `p2` are pointers of the type `pointer-type*`, then the expression `p1-p2` results in:

```
((long)p1 - (long)p2) / sizeof(pointer_type)
```

No exceptions are generated when the arithmetic operation overflows the domain of the pointer, and the result depends on the implementation.

Example

C#

```
// compile with: /unsafe
```

C#

```
class PointerArithmetic
{
    unsafe static void Main()
    {
        int* memory = stackalloc int[30];
        long* difference;
        int* p1 = &memory[4];
        int* p2 = &memory[10];

        difference = (long*)(p2 - p1);

        System.Console.WriteLine("The difference is: {0}", (long)difference);
    }
}
```

Output

The difference is: 6

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 18.5.6 Pointer Arithmetic

See Also

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[C# Operators](#)

[Manipulating Pointers \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

Other Resources

[Types \(C# Reference\)](#)

Pointer Comparison (C# Programming Guide)

You can apply the following operators to compare pointers of any type:

`==` `!=` `<` `>` `<=` `>=`

The comparison operators compare the addresses of the two operands as if they are unsigned integers.

Example

C#

```
// compile with: /unsafe
```

C#

```
class CompareOperators
{
    unsafe static void Main()
    {
        int x = 234;
        int y = 236;
        int* p1 = &x;
        int* p2 = &y;

        System.Console.WriteLine(p1 < p2);
        System.Console.WriteLine(p2 < p1);
    }
}
```

Sample Output

True

False

See Also

Reference

[Pointer Expressions \(C# Programming Guide\)](#)

[C# Operators](#)

[Manipulating Pointers \(C# Programming Guide\)](#)

[Pointer types \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

[stackalloc \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Types \(C# Reference\)](#)

How to: Use Pointers to Copy an Array of Bytes (C# Programming Guide)

The following example uses pointers to copy bytes from one array to another using pointers.

This example uses the `unsafe` keyword, which allows pointers to be used within the `Copy` method. The `fixed` statement is used to declare pointers to the source and destination arrays. This *pins* the location of the source and destination arrays in memory so that they will not be moved by garbage collection. These memory blocks will be unpinned when the **fixed** block completes. Because the `Copy` function in this example uses the **unsafe** keyword, it must be compiled with **/unsafe** compiler option.

Example

C#

```
// compile with: /unsafe
```

C#

```
class TestCopy
{
    // The unsafe keyword allows pointers to be used within the following method:
    static unsafe void Copy(byte[] src, int srcIndex, byte[] dst, int dstIndex, int count)
    {
        if (src == null || srcIndex < 0 ||
            dst == null || dstIndex < 0 || count < 0)
        {
            throw new System.ArgumentException();
        }

        int srcLen = src.Length;
        int dstLen = dst.Length;
        if (srcLen - srcIndex < count || dstLen - dstIndex < count)
        {
            throw new System.ArgumentException();
        }

        // The following fixed statement pins the location of the src and dst objects
        // in memory so that they will not be moved by garbage collection.
        fixed (byte* pSrc = src, pDst = dst)
        {
            byte* ps = pSrc;
            byte* pd = pDst;

            // Loop over the count in blocks of 4 bytes, copying an integer (4 bytes) at a
time:
            for (int i = 0 ; i < count / 4 ; i++)
            {
                *((int*)pd) = *((int*)ps);
                pd += 4;
                ps += 4;
            }

            // Complete the copy by moving any bytes that weren't moved in blocks of 4:
            for (int i = 0; i < count % 4 ; i++)
            {
                *pd = *ps;
                pd++;
                ps++;
            }
        }
    }

    static void Main()
    {

```

```
byte[] a = new byte[100];
byte[] b = new byte[100];

for (int i = 0; i < 100; ++i)
{
    a[i] = (byte)i;
}

Copy(a, 0, b, 0, 100);
System.Console.WriteLine("The first 10 elements are:");

for (int i = 0; i < 10; ++i)
{
    System.Console.Write(b[i] + " ");
}
System.Console.WriteLine("\n");
}
```

Output

The first 10 elements are: 0 1 2 3 4 5 6 7 8 9

See Also

Tasks

[Unsafe Code Sample](#)

Reference

[/unsafe \(Enable Unsafe Mode\) \(C# Compiler Options\)](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

Other Resources

[Garbage Collection](#)

XML Documentation Comments (C# Programming Guide)

In Visual C# you can create documentation for your code by including XML tags in special comment fields in your source code directly before the code block they refer to. For example:

```
/// <summary>
/// This class performs an important function.
/// </summary>
public class MyClass{}
```

When you compile with [/doc](#) the compiler will search for all XML tags in your source code and create an XML documentation file.

Note

The XML doc comments are not metadata; they are not included in the compiled assembly and therefore they are not accessible through reflection.

In This Section

- [Recommended Tags for Documentation Comments](#)
- [Processing the XML File](#)
- [Delimiters for Documentation Tags](#)
- [How to: Use the XML Documentation Features](#)

Related Sections

For more information, see:

- [/doc \(Process Documentation Comments\)](#)
- [XML Documentation Sample](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [Appendix A Documentation Comments](#)

See Also

Concepts

[C# Programming Guide](#)

Recommended Tags for Documentation Comments (C# Programming Guide)

The C# compiler will process documentation comments in your code to an XML file. Processing the XML file to create documentation is a detail that needs to be implemented at your site.

Tags are processed on code constructs such as types and type members.

 **Note**

Documentation comments cannot be applied to a namespace.

The compiler will process any tag that is valid XML. The following tags provide commonly used functionality in user documentation:

<c>	<para>	<see>*
<code>	<param>*	<seealso>*
<example>	<paramref>	<summary>
<exception>*	<permission>*	<typeparam>*
<include>*	<remarks>	<typeparamref>
<list>	<returns>	<value>

(* denotes that the compiler verifies syntax.)

If you want angle brackets to appear in the text of a documentation comment, use < and >. For example, <text in angle brackets>.

See Also

Tasks

[XML Documentation Sample](#)

Reference

[/doc \(Process Documentation Comments\) \(C# Compiler Options\)](#)

Concepts

[C# Programming Guide](#)

[XML Documentation Comments \(C# Programming Guide\)](#)

[XML Documentation Comments \(C# Programming Guide\)](#)

<c> (C# Programming Guide)

```
<c>text</c>
```

Parameters

text

The text you would like to indicate as code.

Remarks

The <c> tag gives you a way to indicate that text within a description should be marked as code. Use [<code>](#) to indicate multiple lines as code.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// text for class TestClass
public class TestClass
{
    /// <summary><c>DoWork</c> is a method in the <c>TestClass</c> class.
    /// </summary>
    public static void DoWork(int Int1)
    {
    }

    /// text for Main
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<code> (C# Programming Guide)

```
<code>content</code>
```

Parameters

content

The text you want marked as code.

Remarks

The <code> tag gives you a way to indicate multiple lines as code. Use [<c>](#) to indicate that text within a description should be marked as code.

Compile with [/doc](#) to process documentation comments to a file.

Example

See the [<example>](#) topic for an example of how to use the <code> tag.

See Also**Reference**

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<example> (C# Programming Guide)

<example>description</example>

Parameters

description

A description of the code sample.

Remarks

The <example> tag lets you specify an example of how to use a method or other library member. This commonly involves using the <code> tag.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// text for class TestClass
public class TestClass
{
    /// <summary>
    /// The GetZero method.
    /// </summary>
    /// <example> This sample shows how to call the GetZero method.
    /// <code>
    /// class TestClass
    /// {
    ///     static int Main()
    ///     {
    ///         return GetZero();
    ///     }
    /// }
    /// </code>
    /// </example>
    public static int GetZero()
    {
        return 0;
    }

    /// text for Main
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<exception> (C# Programming Guide)

```
<exception cref="member">description</exception>
```

Parameters

`cref` = " *member* "

A reference to an exception that is available from the current compilation environment. The compiler checks that the given exception exists and translates *member* to the canonical element name in the output XML. *member* must appear within double quotation marks (" ").

For more information on how to create a cref reference to a generic type, see [<see> \(C# Programming Guide\)](#).

description

A description of the exception.

Remarks

The <exception> tag lets you specify which exceptions can be thrown. This tag can be applied to definitions for methods, properties, events, and indexers.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// comment for class
public class EClass : System.Exception
{
    // class definition...
}

/// comment for class
class TestClass
{
    /// <exception cref="System.Exception">Thrown when...</exception>
    public void DoSomething()
    {
        try
        {
        }
        catch (EClass)
        {
        }
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<include> (C# Programming Guide)

```
<include file='filename' path='tagpath[@name="id"]' />
```

Parameters

filename

The name of the file containing the documentation. The file name can be qualified with a path. Enclose *filename* in single quotation marks (' ').

tagpath

The path of the tags in *filename* that leads to the tag *name*. Enclose the path in single quotation marks (' ').

name

The name specifier in the tag that precedes the comments; *name* will have an *id*.

id

The ID for the tag that precedes the comments. Enclose the ID in double quotation marks (" ").

Remarks

The <include> tag lets you refer to comments in another file that describe the types and members in your source code. This is an alternative to placing documentation comments directly in your source code file.

The <include> tag uses the XML XPath syntax. Refer to XPath documentation for ways to customize your <include> use.

Example

This is a multifile example. The first file, which uses <include>, is listed below:

C#

```
// compile with: /doc:DocFileName.xml

/// <include file='xml_include_tag.doc' path='MyDocs/MyMembers[@name="test"]/*' />
class Test
{
    static void Main()
    {
    }
}

/// <include file='xml_include_tag.doc' path='MyDocs/MyMembers[@name="test2"]/*' />
class Test2
{
    public void Test()
    {
    }
}
```

The second file, xml_include_tag.doc, contains the following documentation comments:

```
<MyDocs>

<MyMembers name="test">
<summary>
The summary for this type.
</summary>
</MyMembers>

<MyMembers name="test2">
<summary>
```

```
The summary for this other type.
</summary>
</MyMembers>

</MyDocs>
```

Program Output

```
<?xml version="1.0"?>
<doc>
  <assembly>
    <name>xml_include_tag</name>
  </assembly>
  <members>
    <member name="T:Test">
      <summary>
        The summary for this type.
      </summary>
    </member>
    <member name="T:Test2">
      <summary>
        The summary for this other type.
      </summary>
    </member>
  </members>
</doc>
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<list> (C# Programming Guide)

```
<list type="bullet" | "number" | "table">
  <listheader>
    <term>term</term>
    <description>description</description>
  </listheader>
  <item>
    <term>term</term>
    <description>description</description>
  </item>
</list>
```

Parameters

term

A term to define, which will be defined in *description*.

description

Either an item in a bullet or numbered list or the definition of a *term*.

Remarks

The <listheader> block is used to define the heading row of either a table or definition list. When defining a table, you only need to supply an entry for term in the heading.

Each item in the list is specified with an <item> block. When creating a definition list, you will need to specify both *term* and *description*. However, for a table, bulleted list, or numbered list, you only need to supply an entry for *description*.

A list or table can have as many <item> blocks as needed.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// text for class TestClass
public class TestClass
{
    /// <summary>Here is an example of a bulleted list:
    /// <list type="bullet">
    /// <item>
    /// <description>Item 1.</description>
    /// </item>
    /// <item>
    /// <description>Item 2.</description>
    /// </item>
    /// </list>
    /// </summary>
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<para> (C# Programming Guide)

```
<para>content</para>
```

Parameters*content*

The text of the paragraph.

Remarks

The <para> tag is for use inside a tag, such as [<summary>](#), [<remarks>](#), or [<returns>](#), and lets you add structure to the text.

Compile with [/doc](#) to process documentation comments to a file.

Example

See [<summary>](#) for an example of using <para>.

See Also**Reference**

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<param> (C# Programming Guide)

```
<param name='name'>description</param>
```

Parameters

name

The name of a method parameter. Enclose the name in double quotation marks (" ").

description

A description for the parameter.

Remarks

The <param> tag should be used in the comment for a method declaration to describe one of the parameters for the method.

The text for the <param> tag will be displayed in [IntelliSense](#), the Object Browser, and in the Code Comment Web Report.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// text for class TestClass
public class TestClass
{
    /// <param name="Int1">Used to indicate status.</param>
    public static void DoWork(int Int1)
    {
    }
    /// text for Main
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<paramref> (C# Programming Guide)

```
<paramref name="name"/>
```

Parameters

name

The name of the parameter to refer to. Enclose the name in double quotation marks (" ").

Remarks

The <paramref> tag gives you a way to indicate that a word in the code comments, for example in a <summary> or <remarks> block refers to a parameter. The XML file can be processed to format this word in some distinct way, such as with a bold or italic font.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// text for class TestClass
public class TestClass
{
    /// <summary>DoWork is a method in the TestClass class.
    /// The <paramref name="Int1"/> parameter takes a number.
    /// </summary>
    public static void DoWork(int Int1)
    {
    }

    /// text for Main
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<permission> (C# Programming Guide)

```
<permission cref="member">description</permission>
```

Parameters

`cref` = " *member* "

A reference to a member or field that is available to be called from the current compilation environment. The compiler checks that the given code element exists and translates *member* to the canonical element name in the output XML. *member* must appear within double quotation marks (" ").

For information on how to create a cref reference to a generic type, see [<see> \(C# Programming Guide\)](#).

description

A description of the access to the member.

Remarks

The <permission> tag lets you document the access of a member. The [PermissionSet](#) class lets you specify access to a member.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

class TestClass
{
    /// <permission cref="System.Security.PermissionSet">Everyone can access this method.</
    permission>
    public static void Test()
    {
    }

    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<remarks> (C# Programming Guide)

```
<remarks>description</remarks>
```

Parameters

Description

A description of the member.

Remarks

The <remarks> tag is used to add information about a type, supplementing the information specified with <summary>. This information is displayed in the [Object Browser](#).

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// <summary>
/// You may have some primary information about this class.
/// </summary>
/// <remarks>
/// You may have some additional information about this class.
/// </remarks>
public class TestClass
{
    /// text for Main
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<returns> (C# Programming Guide)

```
<returns>description</returns>
```

Parameters

description

A description of the return value.

Remarks

The <returns> tag should be used in the comment for a method declaration to describe the return value.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// text for class TestClass
public class TestClass
{
    /// <returns>Returns zero.</returns>
    public static int GetZero()
    {
        return 0;
    }

    /// text for Main
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<see> (C# Programming Guide)

```
<see cref="member"/>
```

Parameters

`cref` = " *member* "

A reference to a member or field that is available to be called from the current compilation environment. The compiler checks that the given code element exists and passes *member* to the element name in the output XML. Place *member* within double quotation marks (" ").

Remarks

The <see> tag lets you specify a link from within text. Use [<seealso>](#) to indicate that text should be placed in a See Also section.

Compile with [/doc](#) to process documentation comments to a file.

See [<summary>](#) for an example of using <see>.

Example

The following example shows how to make a **cref** reference to a generic type.

C#

```
// compile with: /doc:DocFileName.xml

// the following cref shows how to specify the reference, such that,
// the compiler will resolve the reference
/// <summary cref="C{T}">
/// </summary>
class A { }

// the following cref shows another way to specify the reference,
// such that, the compiler will resolve the reference
// <summary cref="C &lt; T &gt;">

// the following cref shows how to hard-code the reference
/// <summary cref="T:C`1">
/// </summary>
class B { }

/// <summary cref="A">
/// </summary>
/// <typeparam name="T"></typeparam>
class C<T> { }
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<seealso> (C# Programming Guide)

```
<seealso cref="member"/>
```

Parameters

cref = " *member* "

A reference to a member or field that is available to be called from the current compilation environment. The compiler checks that the given code element exists and passes *member* to the element name in the output XML. *member* must appear within double quotation marks (" ").

For information on how to create a cref reference to a generic type, see [<see> \(C# Programming Guide\)](#).

Remarks

The <seealso> tag lets you specify the text that you might want to appear in a See Also section. Use [<see>](#) to specify a link from within text.

Compile with [/doc](#) to process documentation comments to a file.

Example

See [<summary>](#) for an example of using <seealso>.

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<summary> (C# Programming Guide)

```
<summary>description</summary>
```

Parameters

description

A summary of the object.

Remarks

The <summary> tag should be used to describe a type or a type member. Use [<remarks>](#) to add supplemental information to a type description.

The text for the <summary> tag is the only source of information about the type in IntelliSense, and is also displayed in the [Object Browser](#).

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// text for class TestClass
public class TestClass
{
    /// <summary>DoWork is a method in the TestClass class.
    /// <para>Here's how you could make a second paragraph in a description. <see cref="System.Console.WriteLine(System.String)"/> for information about output statements.</para>
    /// <seealso cref="TestClass.Main"/>
    /// </summary>
    public static void DoWork(int Int1)
    {
    }

    /// text for Main
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<typeparam> (C# Programming Guide)

```
<typeparam name="name">description</typeparam>
```

Parameters

name

The name of the type parameter. Enclose the name in double quotation marks (" ").

description

A description for the type parameter.

Remarks

The **<typeparam>** tag should be used in the comment for a generic type or method declaration to describe a type parameter. Add a tag for each type parameter of the generic type or method.

For more information, see [Generics \(C# Programming Guide\)](#).

The text for the **<typeparam>** tag will be displayed in [IntelliSense](#), the [Object Browser](#) code comment web report.

Compile with [/doc](#) to process documentation comments to a file.

See [<typeparamref> \(C# Programming Guide\)](#) for a code example showing how to use **<typeparam>**.

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

<typeparamref> (C# Programming Guide)

```
<typeparamref name="name"/>
```

Parameters

name

The name of the type parameter. Enclose the name in double quotation marks (" ").

Remarks

For more information on type parameters in generic types and methods, see [Generics](#).

Use this tag to enable consumers of the documentation file to format the word in some distinct way, for example in italics.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// comment for class
public class TestClass
{
    /// <summary>
    /// Creates a new array of arbitrary type <typeparamref name="T"/>
    /// </summary>
    /// <typeparam name="T">The element type of the array</typeparam>
    public static T[] mkArray<T>(int n)
    {
        return new T[n];
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

<value> (C# Programming Guide)

```
<value>property-description</value>
```

Parameters

property-description

A description for the property.

Remarks

The <value> tag lets you describe the value that a property represents. Note that when you add a property via code wizard in the Visual Studio .NET development environment, it will add a <summary> tag for the new property. You should then manually add a <value> tag to describe the value that the property represents.

Compile with [/doc](#) to process documentation comments to a file.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// text for class Employee
public class Employee
{
    private string _name;

    /// <summary>The Name property represents the employee's name.</summary>
    /// <value>The Name property gets/sets the _name data member.</value>
    public string Name
    {
        get
        {
            return _name;
        }
        set
        {
            _name = value;
        }
    }
}

/// text for class MainClass
public class MainClass
{
    /// text for Main
    static void Main()
    {
    }
}
```

See Also

Reference

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Processing the XML File (C# Programming Guide)

The compiler generates an ID string for each construct in your code that is tagged to generate documentation. (For information on how to tag your code, see [Recommended Tags for Documentation Comments](#).) The ID string uniquely identifies the construct. Programs that process the XML file can use the ID string to identify the corresponding .NET Framework metadata/reflection item that the documentation applies to.

The XML file is not a hierarchical representation of your code; it is a flat list with a generated ID for each element.

The compiler observes the following rules when it generates the ID strings:

- No whitespace is placed in the string.
- The first part of the ID string identifies the kind of member being identified, via a single character followed by a colon. The following member types are used:

Character	Description
N	namespace You cannot add documentation comments to a namespace, but you can make cref references to them, where supported.
T	type: class, interface, struct, enum, delegate
F	field
P	property (including indexers or other indexed properties)
M	method (including such special methods as constructors, operators, and so forth)
E	event
!	error string The rest of the string provides information about the error. The C# compiler generates error information for links that cannot be resolved.

- The second part of the string is the fully qualified name of the item, starting at the root of the namespace. The name of the item, its enclosing type(s), and namespace are separated by periods. If the name of the item itself has periods, they are replaced by the hash-sign ('#'). It is assumed that no item has a hash-sign directly in its name. For example, the fully qualified name of the String constructor would be "System.String.#ctor".
- For properties and methods, if there are arguments to the method, the argument list enclosed in parentheses follows. If there are no arguments, no parentheses are present. The arguments are separated by commas. The encoding of each argument follows directly how it is encoded in a .NET Framework signature:
 - Base types. Regular types (ELEMENT_TYPE_CLASS or ELEMENT_TYPE_VALUETYPE) are represented as the fully qualified name of the type.
 - Intrinsic types (for example, ELEMENT_TYPE_I4, ELEMENT_TYPE_OBJECT, ELEMENT_TYPE_STRING, ELEMENT_TYPE_TYPEDBYREF, and ELEMENT_TYPE_VOID) are represented as the fully qualified name of the corresponding full type. For example, System.Int32 or System.TypedReference.
 - ELEMENT_TYPE_PTR is represented as a '*' following the modified type.
 - ELEMENT_TYPE_BYREF is represented as a '@' following the modified type.
 - ELEMENT_TYPE_PINNED is represented as a '^' following the modified type. The C# compiler never generates this.
 - ELEMENT_TYPE_CMOD_REQ is represented as a '|' and the fully qualified name of the modifier class, following

the modified type. The C# compiler never generates this.

- ELEMENT_TYPE_CMOD_OPT is represented as a '!' and the fully qualified name of the modifier class, following the modified type.
- ELEMENT_TYPE_SZARRAY is represented as "[]" following the element type of the array.
- ELEMENT_TYPE_GENERICARRAY is represented as "[?]" following the element type of the array. The C# compiler never generates this.
- ELEMENT_TYPE_ARRAY is represented as [*lowerbound:size,lowerbound:size*] where the number of commas is the rank - 1, and the lower bounds and size of each dimension, if known, are represented in decimal. If a lower bound or size is not specified, it is simply omitted. If the lower bound and size for a particular dimension are omitted, the ':' is omitted as well. For example, a 2-dimensional array with 1 as the lower bounds and unspecified sizes is [1;1:].
- ELEMENT_TYPE_FNPTR is represented as "=FUNC:*type(signature)*", where *type* is the return type, and *signature* is the arguments of the method. If there are no arguments, the parentheses are omitted. The C# compiler never generates this.

The following signature components are not represented because they are never used for differentiating overloaded methods:

- calling convention
- return type
- ELEMENT_TYPE_SENTINEL
- For conversion operators only (op_Implicit and op_Explicit), the return value of the method is encoded as a '~' followed by the return type, as encoded above.
- For generic types, the name of the type will be followed by a back tick and then a number that indicates the number of generic type parameters. For example,

`<member name="T:SampleClass`2">` is the tag for a type that is defined as `public class SampleClass<T, U>`.

For methods taking generic types as parameters, the generic type parameters are specified as numbers prefaced with back ticks (for example `0,`1). Each number representing a zero-based array notation for the type's generic parameters.

Examples

The following examples show how the ID strings for a class and its members would be generated:

C#

```
///
///
namespace N // "N:N"
{
    ///
    ///
    public unsafe class X // "T:N.X"
    {
        public X(){}
        //-----
        // The result of the above is:
        // "M:N.X.#ctor"

        /// <param name="i"></param>
        public X(int i){}
        //-----
        // The result of the above is:
        // "M:N.X.#ctor(System.Int32)"

        ~X(){}
        //-----
        // The result of the above is:
```

```
// "M:N.X.Finalize", destructor's representation in metadata
```

```
public string q;
```

```
//-----  
// The result of the above is:  
// "F:N.X.q"
```

```
/// <returns></returns>  
public const double PI = 3.14;  
//-----  
// The result of the above is:  
// "F:N.X.PI"
```

```
/// <param name="s"></param>  
/// <param name="y"></param>  
/// <param name="z"></param>  
/// <returns></returns>  
public int f(){return 1;}  
//-----  
// The result of the above is:  
// "M:N.X.f"
```

```
/// <param name="array1"></param>  
/// <param name="array"></param>  
/// <returns></returns>  
public int bb(string s, ref int y, void * z){return 1;}  
//-----  
// The result of the above is:  
// "M:N.X.bb(System.String,System.Int32@,System.Void*)"
```

```
/// <param name="x"></param>  
/// <param name="xx"></param>  
/// <returns></returns>  
public int gg(short[] array1, int[,] array){return 0;}  
//-----  
// The result of the above is:  
// "M:N.X.gg(System.Int16[], System.Int32[0:,0:])"
```

```
public static X operator+(X x, X xx){return x;}  
//-----  
// The result of the above is:  
// "M:N.X.op_Addition(N.X,N.X)"
```

```
public int prop {get{return 1;} set{}}  
//-----  
// The result of the above is:  
// "P:N.X.prop"
```

```
public event D d;  
//-----  
// The result of the above is:  
// "E:N.X.d"
```

```
public int this[string s]{get{return 1;}}  
//-----  
// The result of the above is:  
// "P:N.X.Item(System.String)"
```

```

public class Nested{
//-----
// The result of the above is:
// "T:N.X.Nested"

public delegate void D(int i);
//-----
// The result of the above is:
// "T:N.X.D"

/// <param name="x"></param>
/// <returns></returns>
public static explicit operator int(X x){return 1;}
//-----
// The result of the above is:
// "M:N.X.op_Explicit(N.X)~System.Int32"
}
}

```

See Also

Tasks

[XML Documentation Sample](#)

Reference

[/doc \(Process Documentation Comments\) \(C# Compiler Options\)](#)

Concepts

[C# Programming Guide](#)

[XML Documentation Comments \(C# Programming Guide\)](#)

Delimiters for Documentation Tags (C# Programming Guide)

The use of XML doc comments requires delimiters, which indicate to the compiler where a documentation comment begins and ends. You can use the following kinds of delimiters with the XML documentation tags:

```
///
```

This is the form that is shown in documentation examples and used by the Visual C# project templates.

Note

The Visual Studio IDE has a feature called Smart Comment Editing that automatically inserts the `<summary>` and `</summary>` tags and moves your cursor within these tags after you type the `///` delimiter in the Code Editor. Access this feature from the [Formatting, C#/J#, Text Editor, Options Dialog Box](#) in your project property pages.

```
/** */
```

Multiline delimiters.

There are some formatting rules when using the `/** */` delimiters:

- For the line that contains the `/**` delimiter, if the remainder of the line is white space, the line is not processed for comments. If the first character is white space, that white space character is ignored and the rest of the line is processed. Otherwise, the entire text of the line after the `/**` delimiter is processed as part of the comment.
- For the line that contains the `*/` delimiter, if there is only white space up to the `*/` delimiter, that line is ignored. Otherwise, the text on the line up to the `*/` delimiter is processed as part of the comment, subject to the pattern-matching rules described in the following bullet.
- For the lines after the one that begins with the `/**` delimiter, the compiler looks for a common pattern at the beginning of each line that consists of optional white space and an asterisk (*), followed by more optional white space. If the compiler finds a common set of characters at the beginning of each line, it will ignore that pattern for all lines after the `/**` delimiter, up to and possibly including the line that contains the `*/` delimiter.

Some examples:

- The only part of the following comment that will be processed is the line that begins with `<summary>`. The following two tag formats will produce the same comments:

```
/**
```

```
<summary>text</summary>
```

```
*/
```

```
/** <summary>text</summary> */
```

- The compiler applies a pattern of " * " to ignore at the beginning of the second and third lines.

```
/**
```

```
* <summary>
```

```
* text </summary>*/
```

- The compiler finds no pattern in this comment because there is no asterisk on the second line. Therefore, all text on the second and third lines, up till the `*/`, will be processed as part of the comment.

```
/**
```

```
* <summary>
```

```
text </summary>*/
```

- The compiler finds no pattern in this comment for two reasons. First, there is no line that begins with a consistent number of spaces before the asterisk. Second, the fifth line begins with a tab, which does not match spaces. Therefore, all text from the second line until the `*/` will be processed as part of the comment.

```
/**  
 * <summary>  
 * text  
 * text2  
 * </summary>  
 */
```

See Also

Tasks

[XML Documentation Sample](#)

Reference

[/doc \(Process Documentation Comments\) \(C# Compiler Options\)](#)

Concepts

[C# Programming Guide](#)

[XML Documentation Comments \(C# Programming Guide\)](#)

[XML Documentation Comments \(C# Programming Guide\)](#)

How to: Use the XML Documentation Features (C# Programming Guide)

The following sample provides a basic overview of a type that has been documented.

Example

C#

```
// compile with: /doc:DocFileName.xml

/// <summary>
/// Class level summary documentation goes here.</summary>
/// <remarks>
/// Longer comments can be associated with a type or member
/// through the remarks tag</remarks>
public class TestClass
{
    /// <summary>
    /// Store for the name property</summary>
    private string _name = null;

    /// <summary>
    /// The class constructor. </summary>
    public TestClass()
    {
        // TODO: Add Constructor Logic here
    }

    /// <summary>
    /// Name property </summary>
    /// <value>
    /// A value tag is used to describe the property value</value>
    public string Name
    {
        get
        {
            if (_name == null)
            {
                throw new System.Exception("Name is null");
            }
            return _name;
        }
    }

    /// <summary>
    /// Description for SomeMethod.</summary>
    /// <param name="s"> Parameter description for s goes here</param>
    /// <seealso cref="System.String">
    /// You can use the cref attribute on any tag to reference a type or member
    /// and the compiler will check that the reference exists. </seealso>
    public void SomeMethod(string s)
    {
    }

    /// <summary>
    /// Some other method. </summary>
    /// <returns>
    /// Return results are described through the returns tag.</returns>
    /// <seealso cref="SomeMethod(string)">
    /// Notice the use of the cref attribute to reference a specific method </seealso>
    public int SomeOtherMethod()
    {
        return 0;
    }
}
```

```

    /// <summary>
    /// The entry point for the application.
    /// </summary>
    /// <param name="args"> A list of command line arguments</param>
    static int Main(System.String[] args)
    {
        // TODO: Add code to start application here
        return 0;
    }
}

```

Sample Output

```

// This .xml file was generated with the previous code sample.
<?xml version="1.0"?>
<doc>
  <assembly>
    <name>xmlsample</name>
  </assembly>
  <members>
    <member name="T:SomeClass">
      <summary>
        Class level summary documentation goes here.</summary>
      <remarks>
        Longer comments can be associated with a type or member
        through the remarks tag</remarks>
    </member>
    <member name="F:SomeClass.m_Name">
      <summary>
        Store for the name property</summary>
    </member>
    <member name="M:SomeClass.#ctor">
      <summary>The class constructor.</summary>
    </member>
    <member name="M:SomeClass.SomeMethod(System.String)">
      <summary>
        Description for SomeMethod.</summary>
      <param name="s"> Parameter description for s goes here</param>
      <seealso cref="T:System.String">
        You can use the cref attribute on any tag to reference a type or member
        and the compiler will check that the reference exists. </seealso>
    </member>
    <member name="M:SomeClass.SomeOtherMethod">
      <summary>
        Some other method. </summary>
      <returns>
        Return results are described through the returns tag.</returns>
      <seealso cref="M:SomeClass.SomeMethod(System.String)">
        Notice the use of the cref attribute to reference a specific method </seea
lso>
    </member>
    <member name="M:SomeClass.Main(System.String[])">
      <summary>
        The entry point for the application.
      </summary>
      <param name="args"> A list of command line arguments</param>
    </member>
    <member name="P:SomeClass.Name">
      <summary>
        Name property </summary>
      <value>
        A value tag is used to describe the property value</value>
    </member>
  </members>
</doc>

```

Compiling the Code

To compile the example, type the following command line:

csc XMLsample.cs /doc:XMLsample.xml

This will create the XML file XMLsample.xml, which you can view in your browser or by using the TYPE command.

Robust Programming

XML documentation starts with ///. When you create a new project, the wizards put some starter /// lines in for you. The processing of these comments has some restrictions:

- The documentation must be well-formed XML. If the XML is not well-formed, a warning is generated and the documentation file will contain a comment saying that an error was encountered.
- Developers are free to create their own set of tags. There is a recommended set of tags (see the Further Reading section). Some of the recommended tags have special meanings:
 - The <param> tag is used to describe parameters. If used, the compiler will verify that the parameter exists and that all parameters are described in the documentation. If the verification failed, the compiler issues a warning.
 - The **cref** attribute can be attached to any tag to provide a reference to a code element. The compiler will verify that this code element exists. If the verification failed, the compiler issues a warning. The compiler respects any **using** statements when looking for a type described in the **cref** attribute.
 - The <summary> tag is used by IntelliSense inside Visual Studio to display additional information about a type or member.

Note

The XML file does not provide full information about the type and members (for example, it does not contain any type information). To get full information about a type or member, the documentation file must be used in conjunction with reflection on the actual type or member.

See Also

Tasks

[XML Documentation Sample](#)

Reference

[/doc \(Process Documentation Comments\) \(C# Compiler Options\)](#)

Concepts

[C# Programming Guide](#)

[XML Documentation Comments \(C# Programming Guide\)](#)

[XML Documentation Comments \(C# Programming Guide\)](#)

Application Domains (C# Programming Guide)

Application domains provide a flexible and secure method of isolating running applications.

Application domains are usually created and manipulated by run-time hosts. Occasionally, you may want your application to programmatically interact with your application domains, for example, to unload a component without having to stop your application from running.

Application domains aid security, separating applications from each other and each other's data. A single process can run several application domains, with the same level of isolation that would exist in separate processes. Running multiple applications within a single process increases server scalability.

In the following code example, you create a new application domain and then load and execute a previously built assembly, `HelloWorld.exe`, that is stored on drive C.

```
C#  
  
static void Main()  
{  
    // Create an Application Domain:  
    System.AppDomain newDomain = System.AppDomain.CreateDomain("NewApplicationDomain");  
  
    // Load and execute an assembly:  
    newDomain.ExecuteAssembly(@"c:\HelloWorld.exe");  
  
    // Unload the application domain:  
    System.AppDomain.Unload(newDomain);  
}
```

Application Domains Overview

Application domains have the following properties:

- An assembly must be loaded into an application domain before it can be executed. For more information, see [Assemblies and the Global Assembly Cache \(C# Programming Guide\)](#).
- Faults in one application domain cannot affect other code running in another application domain.
- Individual applications can be stopped and code unloaded without stopping the entire process. You cannot unload individual assemblies or types, only entire application domains.

Related Sections

- [Application Domains Overview](#)
- [Application Domains](#)
- [Application Domains and Assemblies](#)
- [Programming with Application Domains](#)
- [Programming with Application Domains and Assemblies](#)
- [Executing Code in Another Application Domain \(C# Programming Guide\)](#)
- [How to: Create and Use an Application Domain \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.1 Application Startup

See Also

Concepts

[C# Programming Guide](#)

[Assemblies and the Global Assembly Cache \(C# Programming Guide\)](#)

Executing Code in Another Application Domain (C# Programming Guide)

Once an assembly has been loaded into an application domain, the code it contains can be executed. The simplest way to do this is to use [AssemblyLoad](#) which will load the assembly into the current application domain, and begin running the code at the assembly's default entry point.

If you want to load the assembly into another application domain, use [ExecuteAssembly](#) or [ExecuteAssemblyByName](#), or one of the other overloaded versions of these methods.

If you want to execute the other assembly starting other than at the default entry point, define a new type in the remote assembly, deriving from [MarshalByRefObject](#). Then use [CreateInstance](#) to create an instance of that type in your application.

Consider the following file, which creates an assembly consisting of a single namespace and two classes. Assume this assembly has been built, and is stored on drive C with the name `HelloWorldRemote.exe`.

```
C#
// This namespace contains code to be called.
namespace HelloWorldRemote
{
    public class RemoteObject : System.MarshalByRefObject
    {
        public RemoteObject()
        {
            System.Console.WriteLine("Hello, World! (RemoteObject Constructor)");
        }
    }
    class Program
    {
        static void Main()
        {
            System.Console.WriteLine("Hello, World! (Main method)");
        }
    }
}
```

To access the code from another application, you can either load the assembly into the current application domain or create a new application domain and load the assembly into it. If you load the assembly into the current application domain with **Assembly.LoadFrom**, you can use **Assembly.CreateInstance** to instantiate an instance of the **RemoteObject** class, which causes the object constructor to be executed.

```
C#
static void Main()
{
    // Load the assembly into the current appdomain:
    System.Reflection.Assembly newAssembly = System.Reflection.Assembly.LoadFrom(@"c:\HelloWorldRemote.exe");

    // Instantiate RemoteObject:
    newAssembly.CreateInstance("HelloWorldRemote.RemoteObject");
}
```

When loading the assembly into a separate application domain, use **AppDomain.ExecuteAssembly** to access the default entry point or **AppDomain.CreateInstance** to create an instance of the **RemoteObject** class. Creating the instance causes the constructor to be executed.

```
C#
static void Main()
{
```

```
System.AppDomain NewAppDomain = System.AppDomain.CreateDomain("NewApplicationDomain");

// Load the assembly and call the default entry point:
NewAppDomain.ExecuteAssembly(@"c:\HelloWorldRemote.exe");

// Create an instance of RemoteObject:
NewAppDomain.CreateInstanceFrom(@"c:\HelloWorldRemote.exe", "HelloWorldRemote.RemoteObject");
}
```

If you do not want to load the assembly programmatically, use **Add Reference** from the **Solution Explorer** to specify the assembly HelloWorldRemote.exe. Then, add a `using HelloWorldRemote;` directive to the `using` block of your application, and use the **RemoteObject** type in your program to declare an instance of the **RemoteObject** object, like this:

C#

```
static void Main()
{
    // This code creates an instance of RemoteObject, assuming HelloWorldRemote has been added as a reference:
    HelloWorldRemote.RemoteObject o = new HelloWorldRemote.RemoteObject();
}
```

See Also

Reference

[Application Domains \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Application Domains Overview](#)

[Application Domains and Assemblies](#)

[Programming with Application Domains](#)

Other Resources

[Application Domains](#)

[Programming with Application Domains and Assemblies](#)

How to: Create and Use an Application Domain (C# Programming Guide)

Application domains provide a method of isolating your code for purposes of security and performance. For more information, see [Application Domains \(C# Programming Guide\)](#).

Using an application domain

1. Create the application domain. The common language runtime host creates an application domain automatically for your application. For information, see the topic [How to: Create an Application Domain](#).
2. Configure the application domain. For more information, see [How to: Configure an Application Domain](#).
3. Load an assembly into the application domain. For more information, see [How to: Load Assemblies into an Application Domain](#).
4. Access the contents of the other assembly. For more information, see [Executing Code in Another Application Domain \(C# Programming Guide\)](#).
5. Unload the application domain. For more information, see [How to: Unload an Application Domain](#).

See Also

Reference

[Application Domains \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Application Domains Overview](#)

[Application Domains and Assemblies](#)

[Programming with Application Domains](#)

Other Resources

[Application Domains](#)

[Programming with Application Domains and Assemblies](#)

Assemblies and the Global Assembly Cache (C# Programming Guide)

An assembly is a fundamental building block of any .NET Framework application. For example, when you build a simple C# application, Visual Studio creates an assembly in the form of a single portable executable (PE) file, specifically an EXE or DLL.

Assemblies contain metadata that describe their own internal version number and details of all the data and object types they contain. For more information see [Assembly Manifest](#).

Assemblies are only loaded as they are required. If they are not used, they are not loaded. This means that assemblies can be an efficient way to manage resources in larger projects.

Assemblies can contain one or more modules. For example, larger projects may be planned in such a way that several individual developers work on separate modules, all coming together to create a single assembly. For more information on modules, see the topic [How to: Build a Multifile Assembly](#).

Assemblies Overview

Assemblies have the following properties:

- Assemblies are implemented as .exe or .dll files.
- You can share an assembly between applications by placing it in the Global Assembly Cache.
- Assemblies must be strong-named before they can be placed in the Global Assembly Cache. For more information, see [Strong-Named Assemblies](#).
- Assemblies are only loaded into memory if they are required.
- You can programmatically obtain information about an assembly using reflection. For more information, see the topic [Reflection](#).
- If you want to load an assembly only to inspect it, use a method such as [ReflectionOnlyLoadFrom](#).
- You can use two versions of the same assembly in a single application. For more information, see [extern alias](#).

Related Sections

For more information:

- [Friend Assemblies \(C# Programming Guide\)](#)
- [How to: Share an Assembly with Other Applications \(C# Programming Guide\)](#)
- [How to: Load and Unload Assemblies \(C# Programming Guide\)](#)
- [How to: Determine If a File Is an Assembly \(C# Programming Guide\)](#)
- [extern \(C# Reference\)](#)
- [Assemblies in the Common Language Runtime](#)
- [Assemblies Overview](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.2 Program structure
- 9.1 Compilation units

See Also

Reference

[Application Domains \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Friend Assemblies (C# Programming Guide)

An internal type or internal member in an assembly can be accessed from another assembly.

Remarks

The friend assemblies feature allows you to access internal members; private types and private members will remain inaccessible.

To give an assembly (assembly B) access to another assembly's (assembly A's) [internal](#) types and members, use the [InternalsVisibleToAttribute](#) attribute in assembly A.

Note

When compiling an assembly (assembly B) that will access internal types or internal members of another assembly (assembly A), you must explicitly specify the name of the output file (.exe or .dll) with the **/out** compiler option (see [/out](#) for more information). This is required because the compiler has not yet generated the name for the assembly it is building at the time it is binding to external references.

The [StrongNameIdentityPermission](#) class also provides the ability to share types, with the following differences:

- **StrongNameIdentityPermission** applies to an individual type, while a friend assembly applies to the whole assembly.
- If there are hundreds of types in assembly A that you wish to share with assembly B, you have to decorate all of them with **StrongNameIdentityPermission**, while using a friend assembly, you only need to declare the friend relationship once.
- Using **StrongNameIdentityPermission**, the types you want to share have to be declared as public. Using a friend assembly, the shared types are declared as internal.
- For information on how you can build a .netmodule that can access non-public types in an assembly, see [/moduleassemblyname](#).

Example

In this example, the assembly makes internal types and internal members visible to the assembly called cs_friend_assemblies_2.

```
// cs_friend_assemblies.cs
// compile with: /target:library
using System.Runtime.CompilerServices;
using System;

[assembly:InternalsVisibleTo("cs_friend_assemblies_2")]

// internal by default
class Class1
{
    public void Test()
    {
        Console.WriteLine("Class1.Test");
    }
}

// public type with internal member
public class Class2
{
    internal void Test()
    {
        Console.WriteLine("Class2.Test");
    }
}
```

In this example, the assembly consumes internal types and internal members in assembly cs_friend_assemblies.dll.

Notice that you must explicitly specify the name of the output file (**/out:cs_friend_assemblies_2.exe**).

If this assembly gives another assembly (assembly C) access to its internal types and members, assembly C does not automatically become a friend of assembly cs_friend_assemblies.dll.

```
// cs_friend_assemblies_2.cs
// compile with: /reference:cs_friend_assemblies.dll /out:cs_friend_assemblies_2.exe
public class M
{
    static void Main()
    {
        // access an internal type
        Class1 a = new Class1();
        a.Test();

        Class2 b = new Class2();
        // access an internal member of a public type
        b.Test();
    }
}
```

Output

Class1.Test
Class2.Test

This example shows how to make internal types and members available for assemblies that have strong names.

To generate the keyfile and display public key, use the following sequence of sn.exe commands (for more information, see [Strong Name Tool \(Sn.exe\)](#)):

- sn -k friend_assemblies.snk // Generate strong name key
- sn -p friend_assemblies.snk key.publickey // Extract public key from key.snk into key.publickey
- sn -tp key.publickey // Display public key stored in file'key.publickey

Pass the keyfile to the compiler with [/keyfile](#).

```
// cs_friend_assemblies_3.cs
// compile with: /target:library /keyfile:friend_assemblies.snk
using System.Runtime.CompilerServices;

[assembly:InternalsVisibleTo("cs_friend_assemblies_4, PublicKey=002400000480000094000000060
2000000240000525341310004000001000100031d7b6f3abc16c7de526fd67ec2926fe68ed2f9901afbc5f1b6b4
28bf6cd9086021a0b38b76bc340dc6ab27b65e4a593fa0e60689ac98dd71a12248ca025751d135df7b98c5f9d09
172f7b62dabdd302b2a1ae688731ff3fc7a6ab9e8cf39fb73c60667e1b071ef7da5838dc009ae0119a9cbff2c58
1fc0f2d966b77114b2c4")]
class Class1
{
    public void Test()
    {
        System.Console.WriteLine("Class1.Test");
    }
}
```

This example shows how to consume internal types and members available for assemblies that have strong names.

```
// cs_friend_assemblies_4.cs
// compile with: /keyfile:friend_assemblies.snk /reference:cs_friend_assemblies_3.dll /out:
cs_friend_assemblies_4.exe
public class M
{
    static void Main()
    {
        Class1 a = new Class1();
        a.Test();
    }
}
```

```
} }
```

Output

Class1.Test

See Also

Concepts

[C# Programming Guide](#)

[Assemblies and the Global Assembly Cache \(C# Programming Guide\)](#)

How to: Determine If a File Is an Assembly (C# Programming Guide)

A file is an assembly if and only if it is managed, and contains an assembly entry in its metadata. For more information on assemblies and metadata, see the topic [Assembly Manifest](#).

How to manually determine if a file is an assembly

1. Start the [MSIL Disassembler \(Ildasm.exe\)](#).
2. Load the file you wish to test.
3. If **ILDASM** reports that the file is not a portable executable (PE) file, then it is not an assembly. For more information, see the topic [How to: View Assembly Contents](#).

How to programmatically determine if a file is an assembly

1. Call the [GetAssemblyName](#) method, passing the full file path and name of the file you are testing.
2. If a [BadImageFormatException](#) exception is thrown, the file is not an assembly.

Example

This example tests a DLL to see if it is an assembly.

C#

```
class TestAssembly
{
    static void Main()
    {
        try
        {
            System.Reflection.AssemblyName testAssembly =
                System.Reflection.AssemblyName.GetAssemblyName(@"C:\WINDOWS\system\avicap.d
ll");

            System.Console.WriteLine("Yes, the file is an Assembly.");
        }

        catch (System.IO.FileNotFoundException e)
        {
            System.Console.WriteLine("The file cannot be found.");
        }

        catch (System.BadImageFormatException e)
        {
            System.Console.WriteLine("The file is not an Assembly.");
        }

        catch (System.IO.FileLoadException e)
        {
            System.Console.WriteLine("The Assembly has already been loaded.");
        }
    }
}
```

The **GetAssemblyName** method loads the test file, and then releases it once the information is read.

Output

The file is not an Assembly.

See Also

Tasks

[Troubleshooting Exceptions: System.BadImageFormatException](#)

Reference

[AssemblyName](#)

Concepts

[C# Programming Guide](#)

[Assemblies and the Global Assembly Cache \(C# Programming Guide\)](#)

How to: Load and Unload Assemblies (C# Programming Guide)

The assemblies referenced by your program will automatically be loaded at build time, but it is also possible to load specific assemblies into the current application domain at runtime. For more information, see [Application Domains](#).

There is no way to unload an individual assembly without unloading all of the application domains that contain it. Even if the assembly goes out of scope, the actual assembly file will remain loaded until all application domains that contain it are unloaded.

If you want to unload some assemblies but not others, consider creating a new application domain, executing the code inside that domain, and then unload that application domain. For more information, see

[How to: Execute Code in Another Application Domain](#).

To load an assembly into an application domain

- Use one of the several load methods contained in the classes [AppDomain](#) and [System.Reflection](#). For more information, see [Loading Assemblies into an Application Domain](#).

To unload an application domain

- There is no way to unload an individual assembly without unloading all of the application domains that contain it. Use the `Unload` method from **AppDomain** to unload the application domains. For more information, see [Unloading an Application Domain](#).

See Also

Tasks

[How to: Load Assemblies into an Application Domain](#)

Reference

[Application Domains \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Assemblies and the Global Assembly Cache \(C# Programming Guide\)](#)

How to: Share an Assembly with Other Applications (C# Programming Guide)

Assemblies can be private or shared: by default, most simple C# programs consist of a private assembly because they are not intended to be used by other applications.

In order to share an assembly with other applications, it must be placed in the [Global Assembly Cache](#) (GAC).

Sharing an assembly

1. Create your assembly. For more information, see [Creating Assemblies](#).
2. Assign a strong name to your assembly. For more information, see [How to: Sign an Assembly with a Strong Name](#).
3. Assign version information to your assembly. For more information, see [Assembly Versioning](#).
4. Add your assembly to the Global Assembly Cache. For more information, see [How to: Install an Assembly into the Global Assembly Cache](#).
5. Access the types contained in the assembly from the other applications. For more information, see [How to: Reference a Strong-Named Assembly](#).

See Also

Concepts

[C# Programming Guide](#)

[Strong-Named Assemblies](#)

Other Resources

[Programming with Assemblies](#)

[Working with Assemblies and the Global Assembly Cache](#)

[Creating and Using Strong-Named Assemblies](#)

Attributes (C# Programming Guide)

Attributes provide a powerful method of associating declarative information with C# code (types, methods, properties, and so forth). Once associated with a program entity, the attribute can be queried at run time using a technique called [Reflection](#).

Attributes exist in two forms: attributes that are defined in the Common Language Runtime's base class library and custom attributes that you can create, to add extra information to your code. This information can later be retrieved programmatically.

In this example, the attribute [System.Reflection.TypeAttributes.Serializable](#) is used to apply a specific characteristic to a class:

C#

```
[System.Serializable]
public class SampleClass
{
    // Objects of this type can be serialized.
}
```

Attribute Overview

Attributes have the following properties:

- Attributes add metadata to your program. Metadata is information embedded in your program such as compiler instructions or descriptions of data.
- Your program can examine its own metadata using [Reflection](#). See [Accessing Attributes With Reflection](#).
- Attributes are commonly used when interacting with COM.

Related Sections

For more information, see:

- [Using Attributes \(C# Programming Guide\)](#)
- [Creating Custom Attributes \(C# Programming Guide\)](#)
- [Disambiguating Attribute Targets \(C# Programming Guide\)](#)
- [Accessing Attributes With Reflection \(C# Programming Guide\)](#)
- [How to: Create a C/C++ Union Using Attributes \(C# Programming Guide\)](#)
- [Common Attributes \(C# Programming Guide\)](#)
- [Attributes Sample](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.12 Attributes
- 17 Attributes

See Also

Concepts

[C# Programming Guide](#)

[Reflection \(C# Programming Guide\)](#)

[Attributes Overview](#)

[Common Uses for Attributes](#)

Using Attributes (C# Programming Guide)

Attributes can be placed on most any declaration, though a specific attribute might restrict the types of declarations on which it is valid. Syntactically, an attribute is specified by placing the name of the attribute, enclosed in square brackets, in front of the declaration of the entity to which it applies. For example, a method with the attribute **DllImport** is declared like this:

C#

```
[System.Runtime.InteropServices.DllImport("user32.dll")]
extern static void SampleMethod();
```

For more information, see [DllImportAttribute Class](#).

Many attributes have parameters, which can be either positional, unnamed, or named. Any positional parameters must be specified in a certain order and cannot be omitted; named parameters are optional and can be specified in any order. Positional parameters are specified first. For example, these three attributes are equivalent:

```
[DllImport("user32.dll")]
[DllImport("user32.dll", SetLastError=false, ExactSpelling=false)]
[DllImport("user32.dll", ExactSpelling=false, SetLastError=false)]
```

The first parameter, the DLL name, is positional and always comes first; the others are named. In this case, both named parameters default to false, so they can be omitted. Refer to the individual attribute's documentation for information on default parameter values.

More than one attribute can be placed on a declaration, either separately or within the same set of brackets:

C#

```
void MethodA([In][Out] ref double x) { }
void MethodB([Out][In] ref double x) { }
void MethodC([In, Out] ref double x) { }
```

Some attributes can be specified more than once for a given entity. An example of such a multiuse attribute is [Conditional](#):

C#

```
[Conditional("DEBUG"), Conditional("TEST1")]
void TraceMethod()
{
    // ...
}
```

Note

By convention, all attribute names end with the word "Attribute" to distinguish them from other items in the .NET Framework. However, you do not need to specify the attribute suffix when using attributes in code. For example, `[DllImport]` is equivalent to `[DllImportAttribute]`, but `DllImportAttribute` is the attribute's actual name in the .NET Framework.

See Also

Reference

[Disambiguating Attribute Targets \(C# Programming Guide\)](#)

[Creating Custom Attributes \(C# Programming Guide\)](#)

[Accessing Attributes With Reflection \(C# Programming Guide\)](#)

[Attribute](#)

[System.Reflection](#)

Concepts

[C# Programming Guide](#)

[Reflection \(C# Programming Guide\)](#)

[Attributes \(C# Programming Guide\)](#)

Creating Custom Attributes (C# Programming Guide)

You can create your own custom attributes by defining an attribute class, a class that derives directly or indirectly from [Attribute](#), which makes identifying attribute definitions in metadata fast and easy. Suppose you want to tag classes and structs with the name of the programmer who wrote the class or struct. You might define a custom `Author` attribute class:

C#

```
[System.AttributeUsage(System.AttributeTargets.Class |
                      System.AttributeTargets.Struct)
]
public class Author : System.Attribute
{
    private string name;
    public double version;

    public Author(string name)
    {
        this.name = name;
        version = 1.0;
    }
}
```

The class name is the attribute's name, `Author`. It is derived from **System.Attribute**, so it is a custom attribute class. The constructor's parameters are the custom attribute's positional parameters, in this case, `name`, and any public read-write fields or properties are named parameters, in this case, `version` is the only named parameter. Note the use of the **AttributeUsage** attribute to make the `Author` attribute valid only on **class** and **struct** declarations.

You could use this new attribute as follows:

C#

```
[Author("H. Ackerman", version = 1.1)]
class SampleClass
{
    // H. Ackerman's code goes here...
}
```

AttributeUsage has a named parameter, **AllowMultiple**, with which you can make a custom attribute single-use or multiuse.

C#

```
[System.AttributeUsage(System.AttributeTargets.Class |
                      System.AttributeTargets.Struct,
                      AllowMultiple = true) // multiuse attribute
]
public class Author : System.Attribute
```

C#

```
[Author("H. Ackerman", version = 1.1)]
[Author("M. Knott", version = 1.2)]
class SampleClass
{
    // H. Ackerman's code goes here...
    // M. Knott's code goes here...
}
```

See Also
Reference

[Using Attributes \(C# Programming Guide\)](#)

[Disambiguating Attribute Targets \(C# Programming Guide\)](#)

[Accessing Attributes With Reflection \(C# Programming Guide\)](#)

[System.Reflection](#)

Concepts

[C# Programming Guide](#)

[Reflection \(C# Programming Guide\)](#)

[Attributes \(C# Programming Guide\)](#)

Disambiguating Attribute Targets (C# Programming Guide)

In certain situations, the target of an attribute, that is, the entity to which the attribute applies, appears to be ambiguous. For example, in the following method declaration, the `SomeAttr` attribute could apply to the method or to the method's return value:

```
C#  
  
public class SomeAttr : System.Attribute { }  
  
[SomeAttr]  
int Method()  
{  
    return 0;  
}
```

This sort of situation arises frequently when marshaling. To resolve the ambiguity, C# has a set of default targets for each kind of declaration, which can be overridden by explicitly specifying attribute targets.

```
C#  
  
// default: applies to method  
[SomeAttr]  
int Method1() { return 0; }  
  
// applies to method  
[method: SomeAttr]  
int Method2() { return 0; }  
  
// applies to return value  
[return: SomeAttr]  
int Method3() { return 0; }
```

Note that this is independent of the targets on which `SomeAttr` is defined to be valid; that is, even if `SomeAttr` were defined to apply only to return values, the **return** target would still have to be specified. In other words, the compiler will not use **AttributeUsage** information to resolve ambiguous attribute targets. For more information, see [AttributeUsage \(C# Programming Guide\)](#).

The syntax for attribute targets is as follows:

```
[target : attribute-list]
```

Parameters

target

One of the following: assembly, field, event, method, module, param, property, return, type.

attribute-list

A list of applicable attributes.

The table below lists all declarations where attributes are allowed; for each declaration, the possible targets for attributes on the declaration are listed in the second column. Targets in bold are the defaults.

Declaration	Possible targets
assembly	assembly
module	module
class	type
struct	type

interface	type
enum	type
delegate	type , return
method	method , return
parameter	param
Field	field
property — indexer	property
property — get accessor	method , return
property — set accessor	method , param, return
event — field	event , field, method
event — property	event , property
event — add	method , param
event — remove	method , param

Assembly and module-level attributes have no default target. For more information, see [Global Attributes](#).

Example

C#

```
using System.Runtime.InteropServices;
```

C#

```
[Guid("12345678-1234-1234-1234-123456789abc"), InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]
interface ISampleInterface
{
    [DispId(17)] // set the DISPID of the method
    [return: MarshalAs(UnmanagedType.Interface)] // set the marshaling on the return type
    object DoWork();
}
```

See Also

Reference

[Using Attributes \(C# Programming Guide\)](#)

[Creating Custom Attributes \(C# Programming Guide\)](#)

[Accessing Attributes With Reflection \(C# Programming Guide\)](#)

[System.Reflection](#)

[Attribute](#)

Concepts

[C# Programming Guide](#)

[Reflection \(C# Programming Guide\)](#)

[Attributes \(C# Programming Guide\)](#)

Accessing Attributes With Reflection (C# Programming Guide)

The fact that you can define custom attributes and place them in your source code would be of little value without some way of retrieving that information and acting on it. C# has a reflection system that allows you to retrieve the information that was defined with custom attributes. The key method is **GetCustomAttributes**, which returns an array of objects that are the run-time equivalents of the source code attributes. This method has several overloaded versions. For more information, see [Attribute](#).

An attribute specification such as:

C#

```
[Author("H. Ackerman", version = 1.1)]
class SampleClass
```

is conceptually equivalent to this:

C#

```
Author anonymousAuthorObject = new Author("H. Ackerman");
anonymousAuthorObject.version = 1.1;
```

However, the code is not executed until `SampleClass` is queried for attributes. Calling **GetCustomAttributes** on `SampleClass` causes an `Author` object to be constructed and initialized as above. If the class has other attributes, other attribute objects are constructed similarly. **GetCustomAttributes** then returns the `Author` object and any other attribute objects in an array. You can then iterate over this array, determine what attributes were applied based on the type of each array element, and extract information from the attribute objects.

Example

Here is a complete example. A custom attribute is defined, applied to several entities, and retrieved via reflection.

C#

```
[System.AttributeUsage(System.AttributeTargets.Class |
                        System.AttributeTargets.Struct,
                        AllowMultiple = true) // multiuse attribute
]
public class Author : System.Attribute
{
    string name;
    public double version;

    public Author(string name)
    {
        this.name = name;
        version = 1.0; // Default value
    }

    public string GetName()
    {
        return name;
    }
}

[Author("H. Ackerman")]
private class FirstClass
{
    // ...
}

// No Author attribute
private class SecondClass
{

```

```

    // ...
}

[Author("H. Ackerman"), Author("M. Knott", version = 2.0)]
private class ThirdClass
{
    // ...
}

class TestAuthorAttribute
{
    static void Main()
    {
        PrintAuthorInfo(typeof(FirstClass));
        PrintAuthorInfo(typeof(SecondClass));
        PrintAuthorInfo(typeof(ThirdClass));
    }

    private static void PrintAuthorInfo(System.Type t)
    {
        System.Console.WriteLine("Author information for {0}", t);
        System.Attribute[] attrs = System.Attribute.GetCustomAttributes(t); // reflection

        foreach (System.Attribute attr in attrs)
        {
            if (attr is Author)
            {
                Author a = (Author)attr;
                System.Console.WriteLine("    {0}, version {1:f}", a.GetName(), a.version);
            }
        }
    }
}

```

Output

```

Author information for FirstClass
H. Ackerman, version 1.00
Author information for SecondClass
Author information for ThirdClass
H. Ackerman, version 1.00
M. Knott, version 2.00

```

See Also

Reference

[Using Attributes \(C# Programming Guide\)](#)
[Disambiguating Attribute Targets \(C# Programming Guide\)](#)
[Creating Custom Attributes \(C# Programming Guide\)](#)
[System.Reflection](#)
[Attribute](#)

Concepts

[C# Programming Guide](#)
[Reflection \(C# Programming Guide\)](#)
[Attributes \(C# Programming Guide\)](#)

How to: Create a C/C++ Union Using Attributes (C# Programming Guide)

By using attributes you can customize how structs are laid out in memory. For example, you can create what is known as a union in C/C++ by using the **StructLayout(LayoutKind.Explicit)** and **FieldOffset** attributes.

Example

In this code segment, all of the fields of `TestUnion` start at the same location in memory.

C#

```
[System.Runtime.InteropServices.StructLayout(LayoutKind.Explicit)]
struct TestUnion
{
    [System.Runtime.InteropServices.FieldOffset(0)]
    public int i;

    [System.Runtime.InteropServices.FieldOffset(0)]
    public double d;

    [System.Runtime.InteropServices.FieldOffset(0)]
    public char c;

    [System.Runtime.InteropServices.FieldOffset(0)]
    public byte b;
}
```

The following is another example where fields start at different explicitly set locations.

C#

```
[System.Runtime.InteropServices.StructLayout(LayoutKind.Explicit)]
struct TestExplicit
{
    [System.Runtime.InteropServices.FieldOffset(0)]
    public long lg;

    [System.Runtime.InteropServices.FieldOffset(0)]
    public int i1;

    [System.Runtime.InteropServices.FieldOffset(4)]
    public int i2;

    [System.Runtime.InteropServices.FieldOffset(8)]
    public double d;

    [System.Runtime.InteropServices.FieldOffset(12)]
    public char c;

    [System.Runtime.InteropServices.FieldOffset(14)]
    public byte b;
}
```

The two **int** fields, `i1` and `i2`, share the same memory locations as `lg`. This sort of control over struct layout is useful when using platform invocation.

See Also

Reference

[Using Attributes \(C# Programming Guide\)](#)

[Disambiguating Attribute Targets \(C# Programming Guide\)](#)

[Creating Custom Attributes \(C# Programming Guide\)](#)

[Accessing Attributes With Reflection \(C# Programming Guide\)](#)

[System.Reflection](#)

[Attribute](#)

Concepts

[C# Programming Guide](#)

[Reflection \(C# Programming Guide\)](#)

[Attributes \(C# Programming Guide\)](#)

Common Attributes (C# Programming Guide)

This section describes the attributes that are most commonly used in C# programs.

In This Section

- [Conditional \(C# Programming Guide\)](#)
- [Obsolete \(C# Programming Guide\)](#)
- [Global Attributes \(C# Programming Guide\)](#)
- [AttributeUsage \(C# Programming Guide\)](#)

See Also

Reference

[Using Attributes \(C# Programming Guide\)](#)

[Accessing Attributes With Reflection \(C# Programming Guide\)](#)

[System.Reflection](#)

[Attribute](#)

Concepts

[C# Programming Guide](#)

[Reflection \(C# Programming Guide\)](#)

Conditional (C# Programming Guide)

Makes the execution of a method dependent on a preprocessing identifier. The `Conditional` attribute is an alias for [ConditionalAttribute](#), and can be applied to a method or an attribute [class](#).

In this example, `Conditional` is applied to a method to enable or disable the display of program-specific diagnostic information:

```
#define TRACE_ON
using System;
using System.Diagnostics;

public class Trace
{
    [Conditional("TRACE_ON")]
    public static void Msg(string msg)
    {
        Console.WriteLine(msg);
    }
}

public class ProgramClass
{
    static void Main()
    {
        Trace.Msg("Now in Main...");
        Console.WriteLine("Done.");
    }
}
```

If the `TRACE_ON` identifier is not defined, no trace output will be displayed.

The `Conditional` attribute is often used with the `DEBUG` identifier to enable trace and logging features for debug builds but not in release builds, like this:

```
[Conditional("DEBUG")]
static void DebugMethod()
{
}
```

Remarks

When a method marked as conditional is called, the presence or absence of the specified preprocessing symbol determines whether the call is included or omitted. If the symbol is defined, the call is included; otherwise, the call is omitted. Using `Conditional` is a cleaner, more elegant, and less error-prone alternative to enclosing methods inside `#if` and `#endif`, like this:

```
#if DEBUG
void ConditionalMethod()
{
}
#endif
```

A conditional method must be a method in a class or struct declaration and must have a return type of [void](#).

Using multiple identifiers

If a method has multiple **Conditional** attributes, a call to the method is included if at least one of the conditional symbols is defined (in other words, the symbols are logically ORed together). In this example, the presence of either `A` or `B` will result in a method call:

```
[Conditional("A"), Conditional("B")]
static void DoIfAorB()
{
    // ...
}
```

To achieve the effect of logically ANDing symbols, you can define serial conditional methods. For example, the second method below will execute only if both `A` and `B` are defined:

C#

```
[Conditional("A")]
static void DoIfA()
{
    DoIfAandB();
}

[Conditional("B")]
static void DoIfAandB()
{
    // Code to execute when both A and B are defined...
}
```

Using Conditional with Attribute Classes

The **Conditional** attribute can also be applied to an attribute class definition. In this example, the custom attribute `Documentation` will only add information to the metadata if `DEBUG` is defined.

C#

```
[Conditional("DEBUG")]
public class Documentation : System.Attribute
{
    string text;

    public Documentation(string text)
    {
        this.text = text;
    }
}

class SampleClass
{
    // This attribute will only be included if DEBUG is defined.
    [Documentation("This method displays an integer.")]
    static void DoWork(int i)
    {
        System.Console.WriteLine(i.ToString());
    }
}
```

See Also

Reference

[Disambiguating Attribute Targets \(C# Programming Guide\)](#)
[Creating Custom Attributes \(C# Programming Guide\)](#)
[Accessing Attributes With Reflection \(C# Programming Guide\)](#)
[System.Reflection](#)
[Attribute](#)

Concepts

[C# Programming Guide](#)
[Reflection \(C# Programming Guide\)](#)
[Attributes \(C# Programming Guide\)](#)

Obsolete (C# Programming Guide)

The `Obsolete` attribute marks a program entity as one that is no longer recommended for use. Each use of an entity marked obsolete will subsequently generate a warning or an error, depending on how the attribute is configured. For example:

```
[System.Obsolete("use class B")]
class A
{
    public void Method() { }
}
class B
{
    [System.Obsolete("use NewMethod", true)]
    public void OldMethod() { }
    public void NewMethod() { }
}
```

In this example the `Obsolete` attribute is applied to class `A` and to method `B.OldMethod`. Because the second argument of the attribute constructor applied to `B.OldMethod` is set to **true**, using this method will result in a compiler error, whereas using class `A` will merely produce a warning. Calling `B.NewMethod`, however, produces no warning or error.

The string provided as the first argument to attribute constructor will be displayed as part of the warning or error. For example, when used with the previous definitions, the following code generates two warnings and one error:

```
// Generates 2 warnings:
A a = new A();
// Generate no errors or warnings:
B b = new B();
b.NewMethod();
// Generates an error, terminating compilation:
b.OldMethod();
```

Two warnings for class `A` are generated: one for the declaration of the class reference, and one for the class constructor.

The `Obsolete` attribute can be used without arguments, but including an explanation of why the item is obsolete and what to use instead is recommended.

The `Obsolete` attribute is a single-use attribute and can be applied to any entity that allows attributes. `Obsolete` is an alias for [ObsoleteAttribute](#).

See Also

Reference

[Disambiguating Attribute Targets \(C# Programming Guide\)](#)

[Creating Custom Attributes \(C# Programming Guide\)](#)

[Accessing Attributes With Reflection \(C# Programming Guide\)](#)

[Attribute](#)

[System.Reflection](#)

Concepts

[C# Programming Guide](#)

[Reflection \(C# Programming Guide\)](#)

[Attributes \(C# Programming Guide\)](#)

Global Attributes (C# Programming Guide)

Most attributes are applied to specific language elements such as classes or methods; however, some attributes are global—they apply to an entire assembly or module. For example, the [AssemblyVersionAttribute](#) attribute can be used to embed version information into an assembly, like this:

```
[assembly: AssemblyVersion("1.0.0.0")]
```

Global attributes appear in the source code after any top-level [using](#) directives and before any type or namespace declarations. Global attributes can appear in multiple source files, but the files must be compiled in a single compilation pass.

These are some frequently used .NET Framework assembly-level attributes:

[AssemblyCompanyAttribute](#)

[AssemblyConfigurationAttribute](#)

[AssemblyCopyrightAttribute](#)

[AssemblyCultureAttribute](#)

[AssemblyDescriptionAttribute](#)

[AssemblyProductAttribute](#)

[AssemblyTitleAttribute](#)

[AssemblyTrademarkAttribute](#)

These attributes are used in projects based on the Visual Studio [Windows Forms Application Template](#). This template includes a file called `AssemblyInfo.cs`, which includes these attribute instantiations:

```
[assembly: AssemblyTitle("WindowsApplication1")]
[assembly: AssemblyDescription("")]
[assembly: AssemblyConfiguration("")]
[assembly: AssemblyCompany("Microsoft")]
[assembly: AssemblyProduct("WindowsApplication1")]
[assembly: AssemblyCopyright("Copyright © Microsoft 2005")]
[assembly: AssemblyTrademark("")]
[assembly: AssemblyCulture("")]
```

Note

Assembly-level attributes are ignored if you are not creating an assembly.

Assembly Signing Attributes

In previous versions of Visual Studio, signing assemblies with strong names was performed with these assembly-level attributes

- [AssemblyKeyFileAttribute](#)
- [AssemblyKeyNameAttribute](#)
- [AssemblyDelaySignAttribute](#)

This is still supported, but the preferred way to sign assemblies is to use the Signing Page in the Project Designer. See [Signing Page, Project Designer](#) and [How to: Sign an Assembly \(Visual Studio\)](#) for more information.

See Also

Reference

[Creating Custom Attributes \(C# Programming Guide\)](#)

[Accessing Attributes With Reflection \(C# Programming Guide\)](#)

[System.Reflection](#)

Concepts

[C# Programming Guide](#)

[Attributes \(C# Programming Guide\)](#)

Other Resources

[Common Attributes \(C# Programming Guide\)](#)

AttributeUsage (C# Programming Guide)

Determines how a custom attribute class can be used. `AttributeUsage` is an attribute that can be applied to custom attribute definitions to control how the new attribute can be applied. The default settings look like this went applied explicitly:

```
[System.AttributeUsage(System.AttributeTargets.All,
                      AllowMultiple=false,
                      Inherited=true)]
class NewAttribute : System.Attribute { }
```

In this example, the `NewAttribute` class can be applied to any attribute-able code entity, but can be applied only once to each entity. It is inherited by derived classes when applied to a base class.

The `AllowMultiple` and `Inherited` arguments are optional, so this code has the same effect:

```
[System.AttributeUsage(System.AttributeTargets.All)]
class NewAttribute : System.Attribute { }
```

The first `AttributeUsage` argument must be one or more elements of the [AttributeTargets](#) enumeration. Multiple target types can be ORed together, like this:

```
using System;
[AttributeUsage(AttributeTargets.Property | AttributeTargets.Field)]
class NewPropertyOrFieldAttribute : Attribute { }
```

If the `AllowMultiple` argument is set to **true**, then the resulting attribute can be applied more than once to a single entity, like this:

```
using System;
[AttributeUsage(AttributeTargets.Class, AllowMultiple=true)]
class MultiUseAttr : Attribute { }

[MultiUseAttr]
[MultiUseAttr]
class Class1 { }

[MultiUseAttr, MultiUseAttr]
class Class2 { }
```

In this case `MultiUseAttr` can be applied repeatedly because `AllowMultiple` is set to **true**. Both formats shown for applying multiple attributes are valid.

If `Inherited` is set to **false**, then the attribute is not inherited by classes that are derived from a class that is attributed. For example:

```
using System;
[AttributeUsage(AttributeTargets.Class, Inherited=false)]
class Attr1 : Attribute { }

[Attr1]
class BClass { }

class DClass : BClass { }
```

In this case `Attr1` is not applied to `DClass` via inheritance.

Remarks

The `AttributeUsage` attribute is a single-use attribute--it cannot be applied more than once to the same class. `AttributeUsage` is an alias for [AttributeUsageAttribute](#).

For more information, see [Accessing Attributes With Reflection \(C# Programming Guide\)](#).

Example

The following example demonstrates the effect of the `Inherited` and `AllowMultiple` arguments to the `AttributeUsage` attribute, and how the custom attributes applied to a class can be enumerated.

```
using System;

// Create some custom attributes:
[AttributeUsage(System.AttributeTargets.Class, Inherited=false)]
class A1 : System.Attribute { }

[AttributeUsage(System.AttributeTargets.Class)]
class A2 : System.Attribute { }

[AttributeUsage(System.AttributeTargets.Class, AllowMultiple=true)]
class A3 : System.Attribute { }

// Apply custom attributes to classes:
[A1,A2]
class BaseClass { }

[A3,A3]
class DerivedClass : BaseClass { }

public class TestAttributeUsage
{
    static void Main()
    {
        BaseClass b = new BaseClass();
        DerivedClass d = new DerivedClass();

        // Display custom attributes for each class.
        Console.WriteLine("Attributes on Base Class:");
        object[] attrs = b.GetType().GetCustomAttributes(true);
        foreach (Attribute attr in attrs)
        {
            Console.WriteLine(attr);
        }

        Console.WriteLine("Attributes on Derived Class:");
        attrs = d.GetType().GetCustomAttributes(true);
        foreach (Attribute attr in attrs)
        {
            Console.WriteLine(attr);
        }
    }
}
```

Sample Output

```
Attributes on Base Class:
A1
A2
Attributes on Derived Class:
A3
A3
A2
```

See Also

Reference

[Using Attributes \(C# Programming Guide\)](#)

[Disambiguating Attribute Targets \(C# Programming Guide\)](#)

[Creating Custom Attributes \(C# Programming Guide\)](#)

[Accessing Attributes With Reflection \(C# Programming Guide\)](#)

[Attribute](#)

[System.Reflection](#)

Concepts

[C# Programming Guide](#)

[Reflection \(C# Programming Guide\)](#)

[Attributes \(C# Programming Guide\)](#)

Collection Classes (C# Programming Guide)

The .NET Framework provides specialized classes for data storage and retrieval. These classes provide support for stacks, queues, lists, and hash tables. Most collection classes implement the same interfaces, and these interfaces may be inherited to create new collection classes that fit more specialized data storage needs.

Note

Applications that target version 2.0 and later of the .NET Framework should use the generic collection classes in the [System.Collections.Generic](#) namespace, which provide greater type-safety and efficiency than their non-generic counterparts.

C#

```
ArrayList list = new ArrayList();  
list.Add(10);  
list.Add(20);
```

Collection Classes Overview

Collection Classes have the following properties

- Collection classes are defined as part of the [System.Collections](#) or **System.Collections.Generic** namespace.
- Most collection classes derive from the interfaces **ICollection**, **IEnumerator**, **IComparable**, **IEnumerable**, **IList**, **IDictionary**, and **IDictionaryEnumerator** and their generic equivalents.
- Using generic collection classes provides increased type-safety and in some cases can provide better performance, especially when storing value types. For more information, see [Benefits of Generics](#).

Related Sections

- [When to Use Generic Collections](#)
- [Collections and Data Structures](#)
- [Selecting a Collection Class](#)
- [Comparisons and Sorts Within Collections](#)
- [Creating and Manipulating Collections](#)
- [How to: Access a Collection Class with foreach \(C# Programming Guide\)](#)
- [Collection Classes Sample](#)

See Also

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

How to: Access a Collection Class with foreach (C# Programming Guide)

The following code sample illustrates how to write a non-generic collection class that can be used with **foreach**. The class is a string tokenizer, similar to the C run-time function **strtok_s**.

Note

This example represents recommended practice only in cases where you cannot use a generic collection class. Generics are supported in version 2.0 and later of the C# language and the .NET Framework. For an example of how to implement a type-safe generic collection class that supports **IEnumerable<T>** and thereby avoids the issues discussed below, see [How to: Create an Iterator Block for a Generic List \(C# Programming Guide\)](#).

In the following example, `Tokens` breaks the sentence "This is a sample sentence." into tokens using ' ' and '-' as separators, and enumerates those tokens with the **foreach** statement:

C#

```
Tokens f = new Tokens("This is a sample sentence.", new char[] { ' ', '-' });

foreach (string item in f)
{
    System.Console.WriteLine(item);
}
```

Internally, `Tokens` uses an array, which implements `IEnumerator` and `IEnumerable` itself. The code example could have used the array's enumeration methods as its own, but that would have defeated the purpose of this example.

In C#, it is not strictly necessary for a collection class to inherit from `IEnumerable` and `IEnumerator` in order to be compatible with **foreach**; as long as the class has the required `GetEnumerator`, `MoveNext`, `Reset`, and `Current` members, it will work with **foreach**. Omitting the interfaces has the advantage of allowing you to define the return type of `Current` to be more specific than **object**, thereby providing type-safety.

For example, starting with the sample code above, change the following lines:

```
// No longer inherits from IEnumerable:
public class Tokens
// Doesn't return an IEnumerator:
public TokenEnumerator GetEnumerator()
// No longer inherits from IEnumerator:
public class TokenEnumerator
// Type-safe: returns string, not object:
public string Current
```

Now, because `Current` returns a string, the compiler can detect when an incompatible type is used in a **foreach** statement:

```
// Error: cannot convert string to int:
foreach (int item in f)
```

The disadvantage of omitting **IEnumerable** and **IEnumerator** is that the collection class is no longer interoperable with the **foreach** statements, or equivalents, of other common language runtime-compatible languages.

You can have the best of both worlds, type-safety within C# and interoperability with other common language runtime-compatible languages, by inheriting from **IEnumerable** and **IEnumerator** and using explicit interface implementation as demonstrated in the following example.

Example

C#

```
using System.Collections;
```

```

// Declare the Tokens class:
public class Tokens : IEnumerable
{
    private string[] elements;

    Tokens(string source, char[] delimiters)
    {
        // Parse the string into tokens:
        elements = source.Split(delimiters);
    }

    // IEnumerable Interface Implementation:
    // Declaration of the GetEnumerator() method
    // required by IEnumerable
    public IEnumerator GetEnumerator()
    {
        return new TokenEnumerator(this);
    }

    // Inner class implements IEnumerator interface:
    private class TokenEnumerator : IEnumerator
    {
        private int position = -1;
        private Tokens t;

        public TokenEnumerator(Tokens t)
        {
            this.t = t;
        }

        // Declare the MoveNext method required by IEnumerator:
        public bool MoveNext()
        {
            if (position < t.elements.Length - 1)
            {
                position++;
                return true;
            }
            else
            {
                return false;
            }
        }

        // Declare the Reset method required by IEnumerator:
        public void Reset()
        {
            position = -1;
        }

        // Declare the Current property required by IEnumerator:
        public object Current
        {
            get
            {
                return t.elements[position];
            }
        }
    }
}

// Test Tokens, TokenEnumerator
static void Main()
{
    // Testing Tokens by breaking the string into tokens:
    Tokens f = new Tokens("This is a sample sentence.", new char[] { ' ', '-' });
}

```

```
        foreach (string item in f)
        {
            System.Console.WriteLine(item);
        }
    }
```

Output

This is a sample sentence.

See Also

Reference

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

[Collection Classes \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

Exceptions and Exception Handling (C# Programming Guide)

The C# language's exception handling features provide a way to deal with any unexpected or exceptional situations that arise while a program is running. Exception handling uses the **try**, **catch**, and **finally** keywords to attempt actions that may not succeed, to handle failures, and to clean up resources afterwards. Exceptions can be generated by the common language runtime (CLR), by third-party libraries, or by the application code using the **throw** keyword.

In this example, a method tests for a division by zero, and catches the error. Without the exception handling, this program would terminate with a **DivideByZeroException was unhandled** error.

```
int SafeDivision(int x, int y)
{
    try
    {
        return (x / y);
    }
    catch (System.DivideByZeroException dbz)
    {
        System.Console.WriteLine("Division by zero attempted!");
        return 0;
    }
}
```

Exceptions Overview

Exceptions have the following properties:

- When your application encounters an exceptional circumstance, such as a division by zero or low memory warning, an exception is generated.
- Use a **try** block around the statements that might throw exceptions.
- Once an exception occurs within the **try** block, the flow of control immediately jumps to an associated exception handler, if one is present.
- If no exception handler for a given exception is present, the program stops executing with an error message.
- If a catch block defines an exception variable, you can use it to get more information on the type of exception that occurred.
- Actions that may result in an exception are executed with the **try** keyword.
- An exception handler is a block of code that is executed when an exception occurs. In C#, the **catch** keyword is used to define an exception handler.
- Exceptions can be explicitly generated by a program using the **throw** keyword.
- Exception objects contain detailed information about the error, including the state of the call stack and a text description of the error.
- Code in a **finally** block is executed even if an exception is thrown, thus allowing a program to release resources.

Related Sections

See the following topics for more information about exceptions and exception handling:

- [Using Exceptions](#)
- [Handling Exceptions](#)
- [Creating and Throwing Exceptions](#)
- [Compiler-Generated Exceptions](#)
- [How to: Handle an exception using Try/Catch](#)
- [How to: Execute cleanup code using Try/Finally](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [8.9.5 The throw statement](#)
- [8.10 The try statement](#)
- [16 Exceptions](#)

See Also

Reference

[C# Keywords](#)

[throw \(C# Reference\)](#)

[try-catch \(C# Reference\)](#)

[try-finally \(C# Reference\)](#)

[try-catch-finally \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Exceptions Overview](#)

Other Resources

[Design Guidelines for Exceptions](#)

[Handling and Throwing Exceptions](#)

Using Exceptions (C# Programming Guide)

In C#, errors in the program at run time are propagated through the program using a mechanism called exceptions. Exceptions are thrown by code that encounters an error and caught by code that can correct the error. Exceptions can be thrown by the .NET Framework common language runtime (CLR) or by code in a program. Once an exception is thrown, it propagates up the call stack until a **catch** statement for the exception is found. Uncaught exceptions are handled by a generic exception handler provided by the system that displays a dialog box.

Exceptions are represented by classes derived from [Exception](#). This class identifies the type of exception and contains properties with details about the exception. Throwing an exception involves creating an instance of an exception-derived class, optionally configuring properties of the exception, and then throwing the object with the **throw** keyword. For example:

C#

```
private static void TestThrow()
{
    System.ApplicationException ex =
        new System.ApplicationException("Demonstration exception in TestThrow()");

    throw ex;
}
```

After an exception is thrown, the runtime checks the current statement to see if it is within a **try** block. If so, any **catch** blocks associated with the **try** block are checked to see if they can catch the exception. **Catch** blocks normally specify exception types; if the type of the **catch** block is the same type as the exception, or a base class of the exception, the **catch** block can handle the method. For example:

C#

```
static void TestCatch()
{
    try
    {
        TestThrow();
    }
    catch (System.ApplicationException ex)
    {
        System.Console.WriteLine(ex.ToString());
    }
}
```

If the statement that throws an exception is not within a **try** block or if the **try** block enclosing it has no matching **catch** block, the runtime checks the calling method for a **try** statement and **catch** blocks. The runtime continues up the calling stack, searching for a compatible **catch** block. After the **catch** block is found and executed, control is passed to the first statement after the **catch** block.

A **try** statement can contain more than one **catch** block. The first **catch** statement that can handle the exception is executed; any following **catch** statements, even if they are compatible, are ignored. For example:

C#

```
static void TestCatch2()
{
    try
    {
        TestThrow();
    }
    catch (System.ApplicationException ex)
    {
        System.Console.WriteLine(ex.ToString()); // this block will be executed
    }
    catch (System.Exception ex)
    {
    }
```

```

        System.Console.WriteLine(ex.ToString()); // this block will NOT be executed
    }

    System.Console.WriteLine("Done"); // this statement is executed after the catch block
}

```

Before the **catch** block is executed, the **try** blocks that have been evaluated by the runtime, including the **try** block containing the compatible **catch** block, are checked for **finally** blocks. **Finally** blocks allow the programmer to clean up any ambiguous state that could be left over from an aborted **try** block, or to release any external resources (such as graphics handles, database connections or file streams) without waiting for the garbage collector in the runtime to finalize the objects. For example:

C#

```

static void TestFinally()
{
    System.IO.FileStream file = null;
    System.IO.FileInfo fileInfo = new System.IO.FileInfo("C:\\file.txt");

    try
    {
        file = fileInfo.OpenWrite();
        file.WriteByte(0xF);
    }
    finally
    {
        // Closing the file allows you to reopen it immediately - otherwise IOException is
        // thrown.
        if (file != null)
        {
            file.Close();
        }
    }

    try
    {
        file = fileInfo.OpenWrite();
        System.Console.WriteLine("OpenWrite() succeeded");
    }
    catch (System.IO.IOException)
    {
        System.Console.WriteLine("OpenWrite() failed");
    }
}

```

If `WriteByte()` threw an exception, the code in the second **try** block that attempts to reopen the file would fail if `file.Close()` is not called, and the file would remain locked. Because **finally** blocks are executed whether or not an exception is thrown, the **finally** block in the previous example allows the file to be closed properly and helps avoid an error.

If no compatible **catch** block is found on the call stack after an exception is thrown, one of three things happens:

- If the exception is within a destructor, the destructor is aborted and the base destructor, if any, is called.
- If the call stack contains a static constructor, or a static field initializer, a [TypeInitializationException](#) is thrown, with the original exception assigned to the [InnerException](#) property of the new exception.
- If the beginning of the thread is reached, the thread is terminated.

See Also

Reference

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Exception Handling (C# Programming Guide)

A **try** block is used by C# programmers to partition code that may be affected by an exception, and **catch** blocks are used to handle any resulting exceptions. A **finally** block can be used to execute code regardless of whether an exception is thrown — which is sometimes necessary, as code following a try/catch construct will not be executed if an exception is thrown. A try block must be used with either a catch or a finally block, and can include multiple catch blocks. For example:

C#

```
try
{
    // Code to try here.
}
catch (System.Exception ex)
{
    // Code to handle exception here.
}
```

C#

```
try
{
    // Code to try here.
}
finally
{
    // Code to execute after try here.
}
```

C#

```
try
{
    // Code to try here.
}
catch (System.Exception ex)
{
    // Code to handle exception here.
}
finally
{
    // Code to execute after try (and possibly catch) here.
}
```

A **try** statement without a **catch** or **finally** block will result in a compiler error.

Catch Blocks

A **catch** block can specify an exception type to catch. This type is called an *exception filter*, and must either be the [Exception](#) type, or derived from this type. Application-defined exceptions should derive from [ApplicationException](#).

Multiple **catch** blocks with different exception filters can be chained together. Multiple **catch** blocks are evaluated from top to bottom, but only one **catch** block is executed for each exception thrown. The first **catch** block that species the exact type or a base class of the thrown exception will be executed. If no catch block specifies a matching exception filter, then a **catch** block with no filter (if any) will be executed. It is important to place **catch** blocks with the most specific — most derived — exception classes first.

You should catch exceptions when:

- You have a specific understanding of why the exception was thrown, and can implement a specific recovery, such as catching a [FileNotFoundException](#) object and prompting the user to enter a new file name.

- You can create and throw a new, more specific exception. For example:

C#

```
int GetInt(int[] array, int index)
{
    try
    {
        return array[index];
    }
    catch(System.IndexOutOfRangeException e)
    {
        throw new System.ArgumentOutOfRangeException(
            "Parameter index is out of range.");
    }
}
```

- To partially handle an exception. For example, a **catch** block could be used to add an entry to an error log, but then re-throw the exception to allow subsequent handling to the exception. For example:

C#

```
try
{
    // try to access a resource
}
catch (System.UnauthorizedAccessException e)
{
    LogError(e); // call a custom error logging procedure
    throw e;     // re-throw the error
}
```

Finally Blocks

A **finally** block allows clean-up of actions performed in a **try** block. If present, the **finally** block executes after the **try** and **catch** blocks execute. A **finally** block is always executed, regardless of whether an exception is thrown or whether a **catch** block matching the exception type is found.

The **finally** block can be used to release resources such as file streams, database connections, and graphics handles without waiting for the garbage collector in the runtime to finalize the objects. See [using Statement \(C# Reference\)](#) for more information.

In this example, the **finally** block is used to close a file opened in the **try** block. Notice that the state of the file handle is checked before it is closed. If the **try** block failed to open the file, the file handle will still be set to **null**. Alternatively, if the file is opened successfully and no exception is thrown, the **finally** block will still be executed and will close the open file.

C#

```
System.IO.FileStream file = null;
System.IO.FileInfo fileInfo = new System.IO.FileInfo("C:\\file.txt");
try
{
    file = fileInfo.OpenWrite();
    file.WriteByte(0xF);
}
finally
{
    // check for null because OpenWrite
    // might have failed
    if (file != null)
    {
        file.Close();
    }
}
```

```
}  
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 16 Exceptions
- 8.9.5 The throw statement
- 8.10 The try Statement

See Also

Reference

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

[try-catch \(C# Reference\)](#)

[try-finally \(C# Reference\)](#)

[try-catch-finally \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Creating and Throwing Exceptions (C# Programming Guide)

Exceptions are used to indicate that an error has occurred while running the program. Exception objects that describe an error are created and then *thrown* with the `throw` keyword. The runtime then searches for the most compatible exception handler.

Programmers should throw exceptions when:

- The method cannot complete its defined functionality. For example, if a parameter to a method has an invalid value:

C#

```
static void CopyObject(SampleClass original)
{
    if (original == null)
    {
        throw new System.ArgumentException("Parameter cannot be null", "original");
    }
}
```

- An inappropriate call to an object is made, based on the object state. For example, trying to write to a read-only file. In cases where an object state does not allow an operation, throw an instance of `InvalidOperationException` or an object based on a derivation of this class. This is an example of a method that throws an **InvalidOperationException** object:

C#

```
class ProgramLog
{
    System.IO.FileStream logFile = null;
    void OpenLog(System.IO.FileInfo fileName, System.IO.FileMode mode) {}

    void WriteLog()
    {
        if (!this.logFile.CanWrite)
        {
            throw new System.InvalidOperationException("Logfile cannot be read-only");
        }
        // Else write data to the log and return.
    }
}
```

- When an argument to a method causes an exception. In this case, the original exception should be caught and an `ArgumentException` instance should be created. The original exception should be passed to the constructor of the **ArgumentException** as the `InnerException` parameter:

C#

```
static int GetValueFromArray(int[] array, int index)
{
    try
    {
        return array[index];
    }
    catch (System.IndexOutOfRangeException ex)
    {
        System.ArgumentException argEx = new System.ArgumentException("Index is out of range", "index", ex);
        throw argEx;
    }
}
```



```
}  
}
```

Exceptions contain a property called [StackTrace](#) — this string contains the name of the methods on the current call stack, along with the file name and line number where the exception was thrown for each method. A **StackTrace** object is created automatically by the CLR from the point of the **throw** statement, so exceptions must be thrown from the point where the stack trace should begin.

All exceptions contain a property called [Message](#) — this string should be set to explain the reason for the exception. Note that security sensitive information should not be put in the message text. In addition to **Message**, **ArgumentException** contains a property called [ParamName](#) that should be set to the name of the argument that caused the exception to be thrown. In the case of a property setter, **ParamName** should be set to `value`.

Public and protected methods should throw exceptions whenever they cannot complete their intended function. The exception class thrown should be the most specific exception available that fits the error conditions. These exceptions should be documented as part of the class functionality, and derived classes or updates to the original class should retain the same behavior for backwards compatibility.

Exceptions should not be used to alter the flow of a program as part of normal execution—they should only be used to report and handle error conditions. Exceptions should not be returned as a return value or parameter instead of being thrown. Programmers should not throw **System.Exception**, **System.SystemException**, **NullReferenceException** or **IndexOutOfRangeException** intentionally.

Defining Exception Classes

Programs can throw any of the predefined exception classes in the **System** namespace (except where previously noted), or create their own exception classes by deriving from [ApplicationException](#). The derived classes should define at least four constructors — one default constructor, one that sets the message property, and one that sets both the **Message** and **InnerException** properties. The fourth constructor is used to serialize the exception — new exception classes should be serializable. For example:

C#

```
public class InvalidDepartmentException : System.ApplicationException  
{  
    public InvalidDepartmentException() {}  
    public InvalidDepartmentException(string message) {}  
    public InvalidDepartmentException(string message, System.Exception inner) {}  
  
    // Constructor needed for serialization  
    // when exception propagates from a remoting server to the client.  
    protected InvalidDepartmentException(System.Runtime.Serialization.SerializationInfo info  
o,  
        System.Runtime.Serialization.StreamingContext context) {}  
}
```

New properties should only be added to the exception class when the data they provide is useful to resolving the exception. If new properties are added to the derived exception class, `ToString()` should be overridden to return the added information.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 8.9.5 The throw Statement
- 8.10 The try Statement
- 16 Exceptions

See Also

Reference

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

[Exception Handling \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Compiler-Generated Exceptions (C# Programming Guide)

Some exceptions are thrown automatically by the .NET Framework's common language runtime (CLR) as a result of basic operations that fail. These exceptions and their error conditions are listed below.

Exception	Description
ArithmeticException	A base class for exceptions that occur during arithmetic operations, such as DivideByZeroException and OverflowException .
ArrayTypeMismatchException	Thrown when an array cannot store a given element because the actual type of the element is incompatible with the actual type of the array.
DivideByZeroException	Thrown when an attempt is made to divide an integral value by zero.
IndexOutOfRangeException	Thrown when an attempt is made to index an array when the index is less than zero or outside the bounds of the array.
InvalidCastException	Thrown when an explicit conversion from a base type to an interface or to a derived type fails at runtime.
NullReferenceException	Thrown when you attempt to reference an object whose value is null .
OutOfMemoryException	Thrown when an attempt to allocate memory using the new operator fails. This indicates that the memory available to the Common Language Runtime has been exhausted.
OverflowException	Thrown when an arithmetic operation in a checked context overflows.
StackOverflowException	Thrown when the execution stack is exhausted by having too many pending method calls; usually indicates a very deep or infinite recursion.
TypeInitializationException	Thrown when a static constructor throws an exception and no compatible catch clause exists to catch it.

See Also

Reference

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

[Exception Handling \(C# Programming Guide\)](#)

[try-catch \(C# Reference\)](#)

[try-finally \(C# Reference\)](#)

[try-catch-finally \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

How to: Handle an Exception Using try/catch (C# Programming Guide)

The purpose of a [try-catch](#) block is to catch and handle an exception generated by working code. Some exceptions can be handled in a **catch** block and the problem solved without the exception being re-thrown; however, more often the only thing you can do is make sure the appropriate exception is thrown.

Example

In this example, [IndexOutOfRangeException](#) is not the most appropriate exception: [ArgumentOutOfRangeException](#) makes more sense for the method because the error is caused by the `index` argument passed in by the caller.

C#

```
class TestTryCatch
{
    static int GetInt(int[] array, int index)
    {
        try
        {
            return array[index];
        }
        catch (System.IndexOutOfRangeException e) // CS0168
        {
            System.Console.WriteLine(e.Message);
            //set IndexOutOfRangeException to the new exception's InnerException
            throw new System.ArgumentOutOfRangeException("index parameter is out of range."
, e);
        }
    }
}
```

Comments

The code that results in an exception is enclosed in the **try** block. A **catch** statement is added immediately after to handle `IndexOutOfRangeException`, if it occurs. The **catch** block handles the `IndexOutOfRangeException` and throws the more appropriate `ArgumentOutOfRangeException` exception instead. In order to provide the caller with as much information as possible, consider specifying the original exception as the [InnerException](#) of the new exception. Because the **InnerException** property is [readonly](#), you must assign it in the constructor of the new exception.

See Also

Reference

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

[Exception Handling \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

How to: Execute Cleanup Code Using `finally` (C# Programming Guide)

The purpose of a **finally** statement is to ensure that the necessary cleanup of objects, usually objects that are holding external resources, happens immediately, even if an exception is thrown. One example of such cleanup is calling [Close](#) on a [FileStream](#) immediately after use instead of waiting for the object to be garbage collected by the common language runtime, as follows:

C#

```
static void CodeWithoutCleanup()
{
    System.IO.FileStream file = null;
    System.IO.FileInfo fileInfo = new System.IO.FileInfo("C:\\file.txt");

    file = fileInfo.OpenWrite();
    file.WriteByte(0xF);

    file.Close();
}
```

Example

To turn the previous code into a **try-catch-finally** statement, the cleanup code is separated from the working code, as follows.

C#

```
static void CodeWithCleanup()
{
    System.IO.FileStream file = null;
    System.IO.FileInfo fileInfo = null;

    try
    {
        fileInfo = new System.IO.FileInfo("C:\\file.txt");

        file = fileInfo.OpenWrite();
        file.WriteByte(0xF);
    }
    catch (System.Exception e)
    {
        System.Console.WriteLine(e.Message);
    }
    finally
    {
        if (file != null)
        {
            file.Close();
        }
    }
}
```

Because an exception can happen at any time within the **try** block before the `OpenWrite()` call, or the `OpenWrite()` call itself could fail, we are not guaranteed that the file is open when we try to close it. The **finally** block adds a check to make sure the **FileStream** object is not **null** before calling the **Close** method. Without the **null** check, the **finally** block could throw its own [NullReferenceException](#), but throwing exceptions in finally blocks should be avoided if possible.

A database connection is another good candidate for being closed in a **finally** block. Because the number of connections allowed to a database server is sometimes limited, it is important to close database connections as quickly as possible. If an exception is thrown before you can close your connection, this is another case where using the **finally** block is preferable to waiting for garbage collection.

See Also

Reference

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

[Exception Handling \(C# Programming Guide\)](#)

[using Statement \(C# Reference\)](#)

[try-catch \(C# Reference\)](#)

[try-finally \(C# Reference\)](#)

[try-catch-finally \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

How to: Catch a non-CLS Exception

Some .NET languages, including C++/CLI, allow objects to throw exceptions that do not derive from [Exception](#). Such exceptions are called *non-CLS exceptions* or *non-Exceptions*. In Visual C# you cannot throw non-CLS exceptions, but you can catch them in two ways:

- Within a `catch (Exception e)` block as a [RuntimeWrappedException](#).

By default, a Visual C# assembly catches non-CLS exceptions as wrapped exceptions. Use this method if you need access to the original exception, which is accessible through the [WrappedException](#) property. The procedure below explains how to catch exceptions in this way.

- Within a general catch block (a catch block with no exception type specified) that is placed after a `catch (Exception)` or `catch (Exception e)` block.

Use this method when you want to perform some action (such as writing to a log file) in response to non-CLS exceptions, and you do not need access to the exception information. By default the Common Language Runtime wraps all exceptions. To disable this behavior, add this assembly-level attribute to your code, typically in the AssemblyInfo.cs file:

```
[assembly: RuntimeCompatibilityAttribute(WrapNonExceptionThrows = false)].
```

To catch a non-CLS exception

1. Within a `catch (Exception e)` block, use the **as** keyword to test whether `e` can be cast to a **RuntimeWrappedException**.
2. Access the original exception through the **WrappedException** property.

Example

The following example shows how to catch a non-CLS exception that was thrown from a class library written in C++/CLR. Note that in this example, the Visual C# client code knows in advance that the exception type being thrown is a [System.String](#). You can cast the **WrappedException** property back its original type as long as that type is accessible from your code.

```
// Class library written in C++/CLR
ThrowNonCLS.Class1 myClass = new ThrowNonCLS.Class1();

try
{
    // throws gnew System::String(
    // "I do not derive from System.Exception!");
    myClass.TestThrow();
}

catch (Exception e)
{
    RuntimeWrappedException rwe = e as RuntimeWrappedException;
    if (rwe != null)
    {
        String s = rwe.WrappedException as String;
        if (s != null)
        {
            Console.WriteLine(s);
        }
    }
    else
    {
        // handle other System.Exception types
    }
}
```


Interoperability (C# Programming Guide)

Interoperability enables you to preserve and leverage existing investments in unmanaged code. Code running under the control of the common language runtime (CLR) is called "managed code", and code running outside the CLR is "unmanaged code". COM, COM+, C++ components, ActiveX components, and Win32 API are examples of unmanaged code.

The .NET Framework enables interoperability with unmanaged code through platform invoke services, the [System.Runtime.InteropServices](#) namespace, and the CLR and through COM Interoperability (COM interop).

- There are two ways to use unmanaged APIs from managed code: through platform invoke and through It Just Works (IJW) in C++. Platform invoke enables managed code to call functions exported from an unmanaged dynamic link library (DLL), such as Win32 API and custom DLLs. The CLR handles DLL loading and parameter marshaling. For performance, check if there is an equivalent function available in the .NET Framework rather than using platform invoke. For more information, see [A Closer Look at Platform Invoke](#).
- COM interop, which enables managed code to interact with COM objects through COM interfaces and COM clients. There are two ways to use COM components from managed code:
 - For calling OLE Automation compatible COM components, use COM interop or tlbimp.exe. The CLR handles COM component activation and parameter marshaling.
 - For IDL based COM components, use IJW and C++. Every public managed class that implements [IUnknown](#), [IDispatch](#), and other standard COM interfaces can be called from unmanaged code through COM interop. For more information, see [Microsoft .NET/COM Migration and Interoperability](#).

For more information, see [Interoperating with Unmanaged Code](#) and [Improving Interop Performance](#).

Both PInvoke and COM interop use marshaling to translate arguments such as integers, strings, arrays, structures, and pointers between managed and unmanaged code. For more information, see [Interop Marshaling Overview](#).

Related Sections

[Interoperability Overview \(C# Programming Guide\)](#)

[How to: Use COM Interop to Create an Excel Spreadsheet \(C# Programming Guide\)](#)

[How to: Use Platform Invoke to Play a Wave File \(C# Programming Guide\)](#)

[How to: Use COM Interop to Check Spelling Using Word \(C# Programming Guide\)](#)

[Example COM Class \(C# Programming Guide\)](#)

[How to: Use Managed Code as an Automation Add-In for Excel \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 10.5.7 External methods
- 17.5 Attributes for Interoperation
- 18.8 Dynamic memory allocation

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Interoperability in the .NET Compact Framework](#)

Interoperability Overview (C# Programming Guide)

Tools and techniques for interoperating between C# managed code and unmanaged code include Platform Invoke services and .NET Framework and COM interoperability tools.

Platform invoke

Platform invoke locates and calls unmanaged code as an exported function. It also marshals the call's arguments, such as input and output parameters, integers, strings, arrays, and structures, as needed. For more information, see [Marshaling Data with Platform Invoke](#) and [Consuming Unmanaged DLL Functions](#).

Note

The [Common Language Runtime](#) (CLR) manages access to system resources. Calling unmanaged code outside of the CLR by passes this security mechanism, and therefore presents a security risk. For example, unmanaged code might call resources in unmanaged code directly, bypassing CLR security mechanism. For more information, see [.NET Framework Security](#).

Tools for Interoperability between the .NET Framework and COM

- For calling COM APIs from managed code, use [Type Library Importer \(Tlbimp.exe\)](#) which takes a type library as input and outputs a .NET Framework assembly and associated managed metadata. You can then add this generated, .NET Framework assembly to your Visual Studio project as a Project Reference. For example, use `TlbImp comlibrary.dll /out: comlnetlibrary.dll` and add a reference to `comlnetlibrary.dll` in your project. For more information, see [Importing a Type Library as an Assembly](#) and [Calling COM Components from .NET Clients](#).
- For calling managed code from COM, use [Type Library Exporter \(Tlbexp.exe\)](#), which takes a managed assembly as input, and generates a type library containing COM definitions of all the **public** types defined in that assembly.
- For calling managed components from **COM** clients, use [Assembly Registration Tool \(Regasm.exe\)](#), which reads the metadata within a .NET Framework assembly, and adds the registry entries so that COM clients can create managed classes.
- For calling ActiveX controls, use [Windows Forms ActiveX Control Importer \(Aximp.exe\)](#), which takes an ActiveX control's type library as input and generates a wrapper control that allows the control to be hosted in Windows Forms. For example, `Aximp activex.ocx` creates two files, viz. `activex.dl` and `Axactivex.dll`, and you can use as a project reference the auto-generated `Axactivex.dll`.

Note

To use the tools above such as registering a managed assembly for **COM interop**, you must have administrator or a power user security permissions. For more information, see [.NET Framework Security](#).

Interoperability Examples and Techniques

For more information, see [Interop Marshaling Overview](#). For more information on techniques for interoperability, please see:

- [How to: Use PInvoke to play a wave file](#)
- [How to: Use managed code as an Automation Add-In for Excel](#)
- [How to: Use COM Interop to spellcheck using Word](#)
- [How to: Use COM Interop to create an Excel Spreadsheet](#)

See Also

Concepts

[C# Programming Guide](#)

[Custom Marshaling Overview](#)

Other Resources

[Marshaling Data with COM Interop](#)

[Interoperability in the .NET Compact Framework](#)

How to: Use COM Interop to Create an Excel Spreadsheet (C# Programming Guide)

The following code example illustrates how to use **COM interop** to create an **Excel** spreadsheet. For more information on **Excel**, see [Microsoft Excel Objects](#), and [Open Method](#)

This example illustrates how to open an existing **Excel** spreadsheet in C# using .NET Framework **COM interop** capability. The **Excel** assembly is used to open and enter data into a range of cells in the **Excel** spreadsheet.

Note

You must have **Excel** installed on your system for this code to run properly.

Note

The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

To create an Excel spreadsheet with COM interop

1. Create a new C# console application in Visual Studio and call it **CreateExcelWorksheet**.
2. Add the Excel assembly as a reference to the project: Right-click on the project, select **Add Reference**.
3. Click the **COM** tab of the **Add Reference** dialog box, and find **Microsoft Excel 11 Object Library**.
4. Double-click on **Microsoft Excel 11 Object Library**, and press OK.

Note

Depending on the version of Office installed the Excel Assembly may be called **Excel 10 Object Library** or **Excel 11 Object Library**.

5. Copy the following code and paste over the contents of the `Program.cs` file.

C#

```
using System;
using System.Reflection;
using Microsoft.Office.Interop.Excel;

public class CreateExcelWorksheet
{
    static void Main()
    {
        Microsoft.Office.Interop.Excel.Application xlApp = new Microsoft.Office.Interop.Excel.Application();

        if (xlApp == null)
        {
            Console.WriteLine("EXCEL could not be started. Check that your office installation and project references are correct.");
            return;
        }
        xlApp.Visible = true;

        Workbook wb = xlApp.Workbooks.Add(XlWBATemplate.xlWBATWorksheet);
        Worksheet ws = (Worksheet)wb.Worksheets[1];
```

```

        if (ws == null)
        {
            Console.WriteLine("Worksheet could not be created. Check that your office
installation and project references are correct.");
        }

        // Select the Excel cells, in the range c1 to c7 in the worksheet.
        Range aRange = ws.get_Range("C1", "C7");

        if (aRange == null)
        {
            Console.WriteLine("Could not get a range. Check to be sure you have the co
rrect versions of the office DLLs.");
        }

        // Fill the cells in the C1 to C7 range of the worksheet with the number 6.
        Object[] args = new Object[1];
        args[0] = 6;
        aRange.GetType().InvokeMember("Value", BindingFlags.SetProperty, null, aRange,
args);

        // Change the cells in the C1 to C7 range of the worksheet to the number 8.
        aRange.Value2 = 8;
    }
}

```

Security

To use **COM interop**, you must have **administrator** or **Power User** security permissions. For more information on security, see [.NET Framework Security](#).

See Also

Tasks

[How to: Use COM Interop to Check Spelling Using Word \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Interoperability Overview \(C# Programming Guide\)](#)

Other Resources

[Interoperability in the .NET Compact Framework](#)

[COM Interoperability Samples](#)

How to: Use Platform Invoke to Play a Wave File (C# Programming Guide)

The following C# code example illustrates how to use platform invoke services to play a wave sound file on the Windows platform.

Example

This example code uses **DllImport** to import **winmm.dll**'s **PlaySound** method entry point as `Form1.PlaySound()`. The example has a simple Windows Form with a button. Clicking the button opens a standard windows [OpenFileDialog](#) dialog box so you can open a file to play. When a wave file is selected, it is played using the **PlaySound()** method of the winmm.DLL assembly method. For more information on winmm.dll's **PlaySound** method, see [Using PlaySound to Play Waveform-Audio Files](#). Browse and select a file with a .wav extension, and then click **Open** to play the wave file using platform invoke. A text box shows the full path of the file selected.

The **Open Files** dialog box is filtered to show only files with a .wav extension through the filter settings:

C#

```
dialog1.Filter = "Wav Files (*.wav)|*.wav";
```

C#

```
using System.Windows.Forms;

namespace WinSound
{
    public partial class Form1 : Form
    {
        private TextBox textBox1;
        private Button button1;

        public Form1() //constructor
        {
            InitializeComponent();

            [System.Runtime.InteropServices.DllImport("winmm.DLL", EntryPoint = "PlaySound", SetLastError = true)]
            private static extern bool PlaySound(string szSound, System.IntPtr hMod, PlaySoundFlags flags);

            [System.Flags]
            public enum PlaySoundFlags : int
            {
                SND_SYNC = 0x0000,
                SND_ASYNC = 0x0001,
                SND_NODEFAULT = 0x0002,
                SND_LOOP = 0x0008,
                SND_NOSTOP = 0x0010,
                SND_NOWAIT = 0x00002000,
                SND_FILENAME = 0x00020000,
                SND_RESOURCE = 0x00040004
            }

            private void button1_Click (object sender, System.EventArgs e)
            {
                OpenFileDialog dialog1 = new OpenFileDialog();

                dialog1.Title = "Browse to find sound file to play";
                dialog1.InitialDirectory = @"c:\";
                dialog1.Filter = "Wav Files (*.wav)|*.wav";
                dialog1.FilterIndex = 2;
                dialog1.RestoreDirectory = true;
            }
        }
    }
}
```

```

        if(dialog1.ShowDialog() == DialogResult.OK)
        {
            textBox1.Text = dialog1.FileName;
            PlaySound (dialog1.FileName, new System.IntPtr(), PlaySoundFlags.SND_SYNC);
        }
    }
}

```

Compiling the Code

To compile the code

1. Create a new C# Windows Application project in Visual Studio and name it **WinSound**.
2. Copy the code above, and paste it over the contents of the `Form1.cs` file.
3. Copy the following code, and paste it in the `Form1.Designer.cs` file, in the `InitializeComponent()` method, after any existing code.

C#

```

this.button1 = new System.Windows.Forms.Button();
this.textBox1 = new System.Windows.Forms.TextBox();
this.SuspendLayout();
//
// button1
//
this.button1.Location = new System.Drawing.Point(192, 40);
this.button1.Name = "button1";
this.button1.Size = new System.Drawing.Size(88, 24);
this.button1.TabIndex = 0;
this.button1.Text = "Browse";
this.button1.Click += new System.EventHandler(this.button1_Click);
//
// textBox1
//
this.textBox1.Location = new System.Drawing.Point(8, 40);
this.textBox1.Name = "textBox1";
this.textBox1.Size = new System.Drawing.Size(168, 20);
this.textBox1.TabIndex = 1;
this.textBox1.Text = "File path";
//
// Form1
//
this.AutoScaleDimensions = new System.Drawing.SizeF(5, 13);
this.ClientSize = new System.Drawing.Size(292, 266);
this.Controls.Add(this.textBox1);
this.Controls.Add(this.button1);
this.Name = "Form1";
this.Text = "Platform Invoke WinSound C#";
this.ResumeLayout(false);
this.PerformLayout();

```

4. Compile and run the code.

Security

For more information, see [.NET Framework Security](#).

See Also

Tasks

[Platform Invoke Technology Sample](#)

Concepts

[C# Programming Guide](#)

[Interoperability Overview \(C# Programming Guide\)](#)

Other Resources

[Marshaling Data with Platform Invoke](#)

How to: Use COM Interop to Check Spelling Using Word (C# Programming Guide)

The following code example illustrates how to use COM Interoperability to use Word's spell-checking capabilities in your Visual C# application. For more information, see [ProofreadingErrors Collection Object](#) and [Microsoft Word Objects](#).

Example

This example illustrates how to use Word's spelling checker from a C# application. It creates a new **Word.application** object using COM Interoperability. It then uses the **ProofreadingErrors** collection on a **Range** object and finds the misspelled words in that range.

C#

```
using System.Reflection;
using Word = Microsoft.Office.Interop.Word;

namespace WordSpell
{
    public partial class Form1 : System.Windows.Forms.Form
    {
        private System.Windows.Forms.TextBox textBox1;
        private System.Windows.Forms.Button button1;
        private System.Windows.Forms.Label label1;

        public Form1() //constructor
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, System.EventArgs e)
        {
            Word.Application app = new Word.Application();

            int errors = 0;
            if (textBox1.Text.Length > 0)
            {
                app.Visible = false;

                // Setting these variables is comparable to passing null to the function.
                // This is necessary because the C# null cannot be passed by reference.
                object template = Missing.Value;
                object newTemplate = Missing.Value;
                object documentType = Missing.Value;
                object visible = true;

                Word._Document doc1 = app.Documents.Add(ref template, ref newTemplate, ref
documentType, ref visible);
                doc1.Words.First.InsertBefore(textBox1.Text);
                Word.ProofreadingErrors spellErrorsColl = doc1.SpellingErrors;
                errors = spellErrorsColl.Count;

                object optional = Missing.Value;

                doc1.CheckSpelling(
                    ref optional, ref optional, ref optional, ref optional, ref optional, r
ef optional,
                    ref optional, ref optional, ref optional, ref optional, ref optional, r
ef optional);

                label1.Text = errors + " errors corrected ";
                object first = 0;
                object last = doc1.Characters.Count - 1;
                textBox1.Text = doc1.Range(ref first, ref last).Text;
            }
        }
    }
}
```

```

        object saveChanges = false;
        object originalFormat = Missing.Value;
        object routeDocument = Missing.Value;

        app.Quit(ref saveChanges, ref originalFormat, ref routeDocument);
    }
}

```

Compiling the Code

This example requires Word to be installed on your system and depending on the version of Office you have installed, the **Word** assembly may be called Microsoft office 10 Object Library or Word 11 Object Library.

Note

The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

To compile the code

1. Create a new C# Windows Application project in Visual Studio, and call it **WordSpell**.
2. Copy the code above, and paste it over the contents of the `Form1.cs` file.
3. Copy the following code, and paste it in the `Form1.Designer.cs` file, in the `InitializeComponent()` method, after any existing code.

C#

```

this.textBox1 = new System.Windows.Forms.TextBox();
this.button1 = new System.Windows.Forms.Button();
this.label1 = new System.Windows.Forms.Label();
this.SuspendLayout();
//
// textBox1
//
this.textBox1.Location = new System.Drawing.Point(40, 40);
this.textBox1.Multiline = true;
this.textBox1.Name = "textBox1";
this.textBox1.ScrollBars = System.Windows.Forms.ScrollBars.Vertical;
this.textBox1.Size = new System.Drawing.Size(344, 136);
this.textBox1.TabIndex = 0;
this.textBox1.Text = "";
//
// button1
//
this.button1.Location = new System.Drawing.Point(392, 40);
this.button1.Name = "button1";
this.button1.Size = new System.Drawing.Size(96, 23);
this.button1.TabIndex = 1;
this.button1.Text = "Check Spelling";
this.button1.Click += new System.EventHandler(this.button1_Click);
//
// label1
//
this.label1.Location = new System.Drawing.Point(40, 24);
this.label1.Name = "label1";
this.label1.Size = new System.Drawing.Size(336, 16);

```

```
this.label1.TabIndex = 2;
//
// Form1
//
this.AutoScaleDimensions = new System.Drawing.SizeF(5, 13);
this.ClientSize = new System.Drawing.Size(496, 205);
this.Controls.Add(this.label1);
this.Controls.Add(this.button1);
this.Controls.Add(this.textBox1);
this.Name = "Form1";
this.Text = "SpellCheckDemo";
this.ResumeLayout(false);
```

4. Add the **Word** assembly as a reference to the project. Right-click on the project, click **Add Reference**, click the **COM** tab of the **Add Reference** dialog box. Double-click **Microsoft Office 11 Object Library**, and press OK.

Note that the dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose Import and Export Settings on the Tools menu. For more information, see Visual Studio Settings.

Security

To use COM Interoperability, you must have administrator or power-user security permissions. For more information, see [.NET Framework Security](#).

See Also

Tasks

[How to: Use COM Interop to Create an Excel Spreadsheet \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Interoperability Overview \(C# Programming Guide\)](#)

Other Resources

[COM Interoperability Samples](#)

Example COM Class (C# Programming Guide)

The following is an example of a class that you would expose as a COM object. After this code has been placed in a .cs file and added to your project, set the **Register for COM Interop** property to **True**. For more information, see [How to: Register a Component for COM Interop](#).

Exposing Visual C# objects to COM requires declaring a class interface, an events interface if it is required, and the class itself. Class members must follow these rules to be visible to COM:

- The class must be public.
- Properties, methods, and events must be public.
- Properties and methods must be declared on the class interface.
- Events must be declared in the event interface.

Other public members in the class that are not declared in these interfaces will not be visible to COM, but they will be visible to other .NET Framework objects.

To expose properties and methods to COM, you must declare them on the class interface and mark them with a **DispId** attribute, and implement them in the class. The order in which the members are declared in the interface is the order used for the COM vtable.

To expose events from your class, you must declare them on the events interface and mark them with a **DispId** attribute. The class should not implement this interface.

The class implements the class interface; it can implement more than one interface, but the first implementation will be the default class interface. Implement the methods and properties exposed to COM here. They must be marked public and must match the declarations in the class interface. Also, declare the events raised by the class here. They must be marked public and must match the declarations in the events interface.

For more information, see [COM Interop Part 1 Sample](#), [COM Interop Part 2 Sample](#), and [COM Interoperability Samples](#).

Example

C#

```
using System.Runtime.InteropServices;

namespace project_name
{
    [Guid("EAA4976A-45C3-4BC5-BC0B-E474F4C3C83F")]
    public interface ComClass1Interface
    {
    }

    [Guid("7BD20046-DF8C-44A6-8F6B-687FAA26FA71"),
     InterfaceType(ComInterfaceType.InterfaceIsIDispatch)]
    public interface ComClass1Events
    {
    }

    [Guid("0D53A3E8-E51A-49C7-944E-E72A2064F938"),
     ClassInterface(ClassInterfaceType.None),
     ComSourceInterfaces(typeof(ComClass1Events))]
    public class ComClass1 : ComClass1Interface
    {
    }
}
```

See Also

Reference

[Interoperability \(C# Programming Guide\)](#)

[Build Page, Project Designer \(C#\)](#)

Concepts

How to: Use Managed Code as an Automation Add-In for Excel (C# Programming Guide)

With Automation add-ins for Excel, you can use a public function from your COM library to be called as a cell formula. The following example illustrates how to create a C# add-in for calculating income tax rate in a cell in an Excel worksheet. [ComRegisterFunctionAttribute](#) automatically registers the add-in, and no additional tools are required to register the managed code as a COM assembly. For additional information, see [Interoperability Overview \(C# Programming Guide\)](#).

Note

The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose **Import and Export Settings** on the **Tools** menu. For more information, see [Visual Studio Settings](#).

Calculating Income Tax

A typical tax table allows for calculating the tax given a person's annual income. For example, the following shows a hypothetical tax table for individuals.

An example Tax table

1. If income is over zero dollars and under \$7,000, the tax is 10 percent of amount.
2. If income is over \$7,000 dollars and under \$28,400, the tax is 15 percent of amount over \$7,000 plus \$700.00.
3. If income is over \$28,400 dollars and under \$68,800, the tax is 25 percent of amount over \$28,400 plus 3,910.00.
4. If income is over \$68,800 dollars and under \$143,500, the tax is 28 percent of amount over \$68,800 plus \$14,010.00.
5. If income is over \$143,500 dollars and under \$311,950, the tax is 33 percent of amount over \$143,500 plus \$34,926.00.
6. If income is over \$311,950, the tax is 35 percent of amount over \$311,950 plus \$90,514.50.

Create an Automation add-in for Excel using Visual Studio and managed code

1. Create a new Visual C# **Class Library** project called **ExcelAddIn**.
2. In the **Project** menu, click **Properties**, and then click in the **Build** pane. Click the check box labeled **Register for COM Interop**. This setting automatically registers your assembly for COM interoperability.
3. Paste the following code into the class file.

C#

```
using System.Runtime.InteropServices;

namespace TaxTables
{
    [ClassInterface(ClassInterfaceType.AutoDual)]
    public class TaxTables
    {
        public static double Tax(double income)
        {
            if (income > 0 && income <= 7000) {return (.10 * income)}
        }
        if (income > 7000 && income <= 28400) {return 700.00 + (.15 * (income - 7000));}
        if (income > 28400 && income <= 68800) {return 3910.00 + (.25 * (income - 28400));}
        if (income > 68800 && income <= 143500) {return 14010.00 + (.28 * (income - 68800));}
    }
}
```

```

        if (income > 143500 && income <= 311950) {return 34926.00 + (.33 * (income
- 143500));}
        if (income > 311950) {return 90514.50 + (.35 * (income
- 311950));}
        return 0;
    }

    [ComRegisterFunctionAttribute]
    public static void RegisterFunction(System.Type t)
    {
        Microsoft.Win32.Registry.ClassesRoot.CreateSubKey
            ("CLSID\\{" + t.GUID.ToString().ToUpper() + "}\\Programmable");
    }

    [ComUnregisterFunctionAttribute]
    public static void UnregisterFunction(System.Type t)
    {
        Microsoft.Win32.Registry.ClassesRoot.DeleteSubKey
            ("CLSID\\{" + t.GUID.ToString().ToUpper() + "}\\Programmable");
    }
}
}

```

Running the Code

Running the Excel add-in

- Build the **ExcelAddIn** project, and press F5 to compile it.
- Open a new workbook in Excel.
- From the **Tools** menu, click **Add-Ins**, and then click **Automation**.
- In the Automation Servers dialog box, select **ExcelAddIn** in the list of add-ins, and click **OK**.
- In a workbook cell, type `=Tax(23500)`. The cell displays 3175.
- To register after moving the ExcelAddIn.dll to a different directory, run **regasm** with **/codebase**. You may get a warning indicating that the assembly is unsigned.

Security

To use COM interoperability, you must have administrator or power-user security permissions. For more information, see [.NET Framework Security](#).

See Also

Tasks

[How to: Use COM Interop to Create an Excel Spreadsheet \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Interoperability Overview \(C# Programming Guide\)](#)

Threading (C# Programming Guide)

Threading enables your C# program to perform concurrent processing so you can do more than one operation at a time. For example, you can use threading to monitor input from the user, perform background tasks, and handle simultaneous streams of input. The [System.Threading](#) namespace provides classes and interfaces that support multithreaded programming and enable you to easily perform tasks such as creating and starting new threads, synchronizing multiple threads, suspending threads, and aborting threads.

To incorporate threading in your C# code, simply create a function to be executed outside the main thread and point a new [Thread](#) object at it. The following code example creates a new thread in a C# application:

C#

```
System.Threading.Thread newThread;  
newThread = new System.Threading.Thread(anObject.AMethod);
```

The following code example starts a new thread in a C# application:

C#

```
newThread.Start();
```

Multithreading solves problems with responsiveness and multi-tasking, but can also introduce resource sharing and synchronization issues because threads are interrupted and resumed without warning according to a central thread scheduling mechanism. For more information, see [Thread Synchronization](#). See [Using Threads and Threading](#) for overview information.

Overview

Threads have the following properties:

- Threads enable your C# program to perform concurrent processing.
- The .NET Framework's **System.Threading** namespace makes using threads easier.
- Threads share the application's resources. For more information, see [Using Threads and Threading](#).

Related Sections

See the following topics for more information:

- [Using Threading](#)
- [How to: Create and Terminate Threads](#)
- [How to: Use a Thread Pool](#)
- [How to: Synchronize a Producer and a Consumer Thread](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.10 Execution order
- 8.12 The lock statement
- 10.4.3 Volatile fields
- 10.7.1 Field-like events

See Also

Tasks

[Monitor Synchronization Technology Sample](#)

[Wait Synchronization Technology Sample](#)

Reference

[Mutex](#)

Concepts[C# Programming Guide](#)[Mutexes](#)[Monitors](#)[Interlocked Operations](#)[AutoResetEvent](#)[Delegates \(C# Programming Guide\)](#)**Other Resources**[Thread Class](#)[HOW TO: Synchronize Access to a Shared Resource in a Multithreading Environment by Using Visual C# .NET](#)

Using Threading (C# Programming Guide)

By default, a C# program has one thread. This thread executes the code in the program starting and ending with the `Main` method. Every command executed by `Main`--either directly or indirectly--is performed by the default, or primary thread, and this thread terminates when `Main` returns. However, auxiliary threads can be created and used to execute code in parallel with the primary thread. These threads are often called *worker threads*.

Worker threads can be used to perform time consuming or time critical tasks without tying up the primary thread. For example, worker threads are often used in server applications to fulfill incoming requests without waiting for the previous request to be complete. Worker threads are also used to perform "background" tasks in desktop applications so that the main thread--which drives user interface elements--remains responsive to user actions.

Multithreading solves problems with throughput and responsiveness, but it can also introduce resource-sharing issues such as deadlocks and race conditions. Multiple threads are best for tasks that require different resources such as file handles, network connections. Assigning multiple threads to a single resource is likely to cause synchronization issues, and having threads frequently blocked while waiting for other threads defeats the purpose of using multiple threads.

A common strategy is to use worker threads to perform time-consuming or time-critical tasks that don't require many of the resources used by other threads. Naturally, some resources in your program must be accessed by multiple threads. For these cases, the [System.Threading](#) namespace provides classes for synchronizing threads. These classes include [Mutex](#), [Monitor](#), [Interlocked](#), and [AutoResetEvent](#), and [ManualResetEvent](#).

You can use some or all of these classes to synchronize the activities of multiple threads, but some support for multi-threading is supported by the C# language. For example, the C# [Lock Statement](#) provides synchronization features through implicit use of **Monitor**.

The following topics demonstrate common multithreading techniques:

- [How to: Create, Start, and Interact Between Threads](#)
- [How to: Synchronize a Producer and a Consumer Thread](#)
- [How to: Use a Thread Pool](#)

Related Sections

For more information, please see:

- [HOW TO: Create a Thread by Using Visual C# .NET](#)
- [HOW TO: Submit a Work Item to the Thread Pool by Using Visual C# .NET](#)
- [HOW TO: Synchronize Access to a Shared Resource in a Multithreading Environment by Using Visual C# .NET](#)
- [Threads and Threading](#)
- [Synchronizing Data for Multithreading](#)

See Also

Tasks

[Monitor Synchronization Technology Sample](#)

[Wait Synchronization Technology Sample](#)

[Threading Sample](#)

Reference

[Mutex](#)

Concepts

[C# Programming Guide](#)

[Monitors](#)

[Interlocked Operations](#)

[AutoResetEvent](#)

Other Resources

[Managed Threading](#)

[Using Threads and Threading](#)

[Threading Samples](#)

[Thread Class](#)

Thread Synchronization (C# Programming Guide)

The following sections describe features and classes that can be used to synchronize access to resources in multithreaded applications.

One of the benefits of using multiple threads in an application is that each thread executes asynchronously. For Windows applications, this allows time-consuming tasks to be performed in the background while the application window and controls remain responsive. For server applications, multithreading provides the ability to handle each incoming request with a different thread. Otherwise, each new request would not get serviced until the previous request had been fully satisfied.

However, the asynchronous nature of threads means that access to resources such as file handles, network connections, and memory must be coordinated. Otherwise, two or more threads could access the same resource at the same time, each unaware of the other's actions. The result is unpredictable data corruption.

For simple operations on integral numeric data types, synchronizing threads can be accomplished with members of the [Interlocked](#) class. For all other data types and non thread-safe resources, multithreading can only be safely performed using the constructs in this topic.

For background information on multithreaded programming, see:

- [Using Threading \(C# Programming Guide\)](#)
- [Managed Threading Basics](#)
- [Using Threads and Threading](#)
- [Managed Threading Best Practices](#)

The lock Keyword

The **lock** keyword can be used to ensure that a block of code runs to completion without interruption by other threads. This is accomplished by obtaining a mutual-exclusion lock for a given object for the duration of the code block.

A **lock** statement begins with the keyword **lock**, which is given an object as an argument, and followed by a code block that is to be executed by only one thread at a time. For example:

C#

```
public class TestThreading
{
    private System.Object lockThis = new System.Object();

    public void Function()
    {
        lock (lockThis)
        {
            // Access thread-sensitive resources.
        }
    }
}
```

The argument provided to the **lock** keyword must be an object based on a reference type, and is used to define the scope of the lock. In the example above, the lock scope is limited to this function because no references to the object exist outside the function. Strictly speaking, the object provided to **lock** is used solely to uniquely identify the resource being shared among multiple threads, so it can be an arbitrary class instance. In practice, however, this object usually represents the resource for which thread synchronization is necessary. For example, if a container object is to be used by multiple threads, then the container can be passed to lock, and the synchronized code block following the lock would access the container. As long as other threads locks on the same contain before accessing it, then access to the object is safely synchronized.

Generally, it is best to avoid locking on a **public** type, or on object instances beyond the control of your application. For example, `lock(this)` can be problematic if the instance can be accessed publicly, because code beyond your control may lock on the object as well. This could create deadlock situations where two or more threads wait for the release of the same object. Locking on a public data type, as opposed to an object, can cause problems for the same reason. Locking on literal strings is

especially risky because literal strings are *interned* by the common language runtime (CLR). This means that there is one instance of any given string literal for the entire program, the exact same object represents the literal in all running application domains, on all threads. As a result, a lock placed on a string with the same contents anywhere in the application process locks all instances of that string in the application. As a result, it is best to lock a private or protected member that is not interned. Some classes provide members specifically for locking. The [Array](#) type, for example, provides [SyncRoot](#). Many collection types provide a `SyncRoot` member as well.

For more information on the **lock** keyword, see:

- [lock Statement \(C# Reference\)](#)
- [How to: Synchronize a Producer and a Consumer Thread \(C# Programming Guide\)](#)

Monitors

Like the **lock** keyword, monitors prevent blocks of code from simultaneous execution by multiple threads. The [Enter](#) method allows one and only one thread to proceed into the following statements; all other threads are blocked until the executing thread calls [Exit](#). This is just like using the **lock** keyword. In fact, the **lock** keyword is implemented with the [Monitor](#) class. For example:

C#

```
lock (x)
{
    DoSomething();
}
```

This is equivalent to:

C#

```
System.Object obj = (System.Object)x;
System.Threading.Monitor.Enter(obj);
try
{
    DoSomething();
}
finally
{
    System.Threading.Monitor.Exit(obj);
}
```

Using the **lock** keyword is generally preferred over using the **Monitor** class directly, both because **lock** is more concise, and because **lock** insures that the underlying monitor is released, even if the protected code throws an exception. This is accomplished with the **finally** keyword, which executes its associated code block regardless of whether an exception is thrown.

For more information on monitors, see [Monitor Synchronization Technology Sample](#).

Synchronization Events and Wait Handles

Using a lock or monitor is useful for preventing the simultaneous execution of thread-sensitive blocks of code, but these constructs do not allow one thread to communicate an event to another. This requires *synchronization events*, which are objects that have one of two states, signaled and un-signaled, that can be used to activate and suspend threads. Threads can be suspended by being made to wait on a synchronization event that is un-signaled, and can be activated by changing the event state to signaled. If a thread attempts to wait on an event that is already signaled, then the thread continues to execute without delay.

There are two kinds of synchronization events: [AutoResetEvent](#), and [ManualResetEvent](#). They differ only in that **AutoResetEvent** changes from signaled to un-signaled automatically any time it activates a thread. Conversely, a **ManualResetEvent** allows any number of threads to be activated by its signaled state, and will only revert to an un-signaled state when its [Reset](#) method is called.

Threads can be made to wait on events by calling one of the wait methods, such as [WaitOne](#), [WaitAny](#), or [WaitAll](#). [System.Threading.WaitHandle.WaitOne](#) causes the thread to wait until a single event becomes signaled, [System.Threading.WaitHandle.WaitAny](#) blocks a thread until one or more indicated events become signaled, and [System.Threading.WaitHandle.WaitAll](#) blocks the thread until all of the indicated events become signaled. An event becomes

signaled when its [Set](#) method is called.

In the following example, a thread is created and started by the `Main` function. The new thread waits on an event using the **WaitOne** method. The thread is suspended until the event becomes signaled by the primary thread that is executing the `Main` function. Once the event becomes signaled, the auxiliary thread returns. In this case, because the event is only used for one thread activation, either the **AutoResetEvent** or **ManualResetEvent** classes could be used.

```
C#  
  
using System;  
using System.Threading;  
  
class ThreadingExample  
{  
    static AutoResetEvent autoEvent;  
  
    static void DoWork()  
    {  
        Console.WriteLine("    worker thread started, now waiting on event...");  
        autoEvent.WaitOne();  
        Console.WriteLine("    worker thread reactivated, now exiting...");  
    }  
  
    static void Main()  
    {  
        autoEvent = new AutoResetEvent(false);  
  
        Console.WriteLine("main thread starting worker thread...");  
        Thread t = new Thread(DoWork);  
        t.Start();  
  
        Console.WriteLine("main thread sleeping for 1 second...");  
        Thread.Sleep(1000);  
  
        Console.WriteLine("main thread signaling worker thread...");  
        autoEvent.Set();  
    }  
}
```

For more examples of thread synchronization event usage, see:

- [Monitor Synchronization Technology Sample](#)
- [Reader-Writer Synchronization Technology Sample](#)
- [Thread Pools Technology Sample](#)
- [Wait Synchronization Technology Sample](#)

Mutex Object

A *mutex* is similar to a monitor; it prevents the simultaneous execution of a block of code by more than one thread at a time. In fact, the name "mutex" is a shortened form of the term "mutually exclusive." Unlike monitors, however, a mutex can be used to synchronize threads across processes. A mutex is represented by the [Mutex](#) class.

When used for inter-process synchronization, a mutex is called a *named mutex* because it is to be used in another application, and therefore it cannot be shared by means of a global or static variable. It must be given a name so that both applications can access the same mutex object.

Although a mutex can be used for intra-process thread synchronization, using **Monitor** is generally preferred, because monitors were designed specifically for the .NET Framework and therefore make better use of resources. In contrast, the **Mutex** class is a wrapper to a Win32 construct. While it is more powerful than a monitor, a mutex requires interop transitions that are more computationally expensive than those required by the **Monitor** class. For an example of using a mutex, see [Mutexes](#).

Related Sections

- [How to: Create and Terminate Threads \(C# Programming Guide\)](#)

- [How to: Use a Thread Pool \(C# Programming Guide\)](#)
- [HOW TO: Synchronize Access to a Shared Resource in a Multithreading Environment by Using Visual C# .NET](#)
- [HOW TO: Create a Thread by Using Visual C# .NET](#)
- [HOW TO: Submit a Work Item to the Thread Pool by Using Visual C# .NET](#)
- [HOW TO: Synchronize Access to a Shared Resource in a Multithreading Environment by Using Visual C# .NET](#)

See Also

Reference

[Thread](#)

[WaitOne](#)

[WaitAny](#)

[WaitAll](#)

[Monitor](#)

[Mutex](#)

[AutoResetEvent](#)

[ManualResetEvent](#)

[Interlocked](#)

[WaitHandle](#)

Concepts

[C# Programming Guide](#)

How to: Create and Terminate Threads (C# Programming Guide)

This example demonstrates how an auxiliary or worker thread can be created and used to perform processing in parallel with the primary thread. Making one thread wait for another and gracefully terminating a thread are also demonstrated. For background information on multi-threading, see [Managed Threading](#) and [Using Threading \(C# Programming Guide\)](#).

The example creates a class named `Worker` that contains the method that the worker thread will execute called `DoWork`. This is essentially the `Main` function for the worker thread. The worker thread will begin execution by calling this method, and terminate automatically when this method returns. The `DoWork` method looks like this:

C#

```
public void DoWork()
{
    while (!_shouldStop)
    {
        Console.WriteLine("worker thread: working...");
    }
    Console.WriteLine("worker thread: terminating gracefully.");
}
```

The `Worker` class contains an additional method that is used to indicate to `DoWork` that it should return. This method is called `RequestStop`, and looks like this:

C#

```
public void RequestStop()
{
    _shouldStop = true;
}
```

The `RequestStop` method merely assigns the `_shouldStop` data member to **true**. Because this data member is checked by the `DoWork` method, this has the indirect effect of causing `DoWork` to return, thereby terminating the worker thread. However, it is important to note that `DoWork` and `RequestStop` will be executed by different threads. `DoWork` is executed by the worker thread, and `RequestStop` is executed by the primary thread, so the `_shouldStop` data member is declared **volatile**, like this:

C#

```
private volatile bool _shouldStop;
```

The **volatile** keyword alerts the compiler that multiple threads will access the `_shouldStop` data member, and therefore it should not make any optimization assumptions about the state of this member. For more information, see [volatile \(C# Reference\)](#).

The use of **volatile** with the `_shouldStop` data member allows us to safely access this member from multiple threads without the use of formal thread synchronization techniques, but only because `_shouldStop` is a **bool**. This means that only single, atomic operations are necessary to modify `_shouldStop`. If, however, this data member were a class, struct, or array, accessing it from multiple threads would likely result in intermittent data corruption. Consider a thread that changes the values in an array. Windows regularly interrupts threads in order to allow other threads to execute, so this thread could be halted after assigning some array elements but before assigning others. This means the array now has a state that the programmer never intended, and another thread reading this array may fail as a result.

Before actually creating the worker thread, the `Main` function creates a `Worker` object and an instance of `Thread`. The thread object is configured to use the `Worker.DoWork` method as an entry point by passing a reference to this method to the **Thread** constructor, like this:

C#

```
Worker workerObject = new Worker();
```



```
Thread workerThread = new Thread(workerObject.DoWork);
```

At this point, although the worker thread object exists and is configured, the actual worker thread has yet been created. This does not happen until `Main` calls the `Start` method:

C#

```
workerThread.Start();
```

At this point the system initiates the execution of the worker thread, but it does so asynchronously to the primary thread. This means that the `Main` function continues to execute code immediately while the worker thread simultaneously undergoes initialization. To insure that the `Main` function does not try to terminate the worker thread before it has a chance to execute, the `Main` function loops until the worker thread object's `IsAlive` property gets set to **true**:

C#

```
while (!workerThread.IsAlive);
```

Next, the primary thread is halted briefly with a call to `Sleep`. This insures that the worker thread's `DoWork` function will execute the loop inside the `DoWork` method for a few iterations before the `Main` function executes any more commands:

C#

```
Thread.Sleep(1);
```

After the 1 millisecond elapses, `Main` signals to the worker thread object that it should terminate using the `Worker.RequestStop` method introduced previously:

C#

```
workerObject.RequestStop();
```

It is also possible to terminate a thread from another thread with a call to `Abort`, but this forcefully terminates the affected thread without concern for whether it has completed its task and provides no opportunity for the cleanup of resources. The technique shown in this example is preferred.

Finally, the `Main` function calls the `Join` method on the worker thread object. This method causes the current thread to block, or wait, until the thread that the object represents terminates. Therefore **Join** will not return until the worker thread returns, thereby terminating itself:

C#

```
workerThread.Join();
```

At this point only the primary thread executing `Main` exists. It displays one final message, and then returns, terminating the primary thread as well.

The complete example appears below.

Example

C#

```
using System;
using System.Threading;

public class Worker
{
    // This method will be called when the thread is started.
    public void DoWork()
    {
```

```

        while (!_shouldStop)
        {
            Console.WriteLine("worker thread: working...");
        }
        Console.WriteLine("worker thread: terminating gracefully.");
    }
    public void RequestStop()
    {
        _shouldStop = true;
    }
    // Volatile is used as hint to the compiler that this data
    // member will be accessed by multiple threads.
    private volatile bool _shouldStop;
}

public class WorkerThreadExample
{
    static void Main()
    {
        // Create the thread object. This does not start the thread.
        Worker workerObject = new Worker();
        Thread workerThread = new Thread(workerObject.DoWork);

        // Start the worker thread.
        workerThread.Start();
        Console.WriteLine("main thread: Starting worker thread...");

        // Loop until worker thread activates.
        while (!workerThread.IsAlive);

        // Put the main thread to sleep for 1 millisecond to
        // allow the worker thread to do some work:
        Thread.Sleep(1);

        // Request that the worker thread stop itself:
        workerObject.RequestStop();

        // Use the Join method to block the current thread
        // until the object's thread terminates.
        workerThread.Join();
        Console.WriteLine("main thread: Worker thread has terminated.");
    }
}

```

Sample Output

```

main thread: starting worker thread...
worker thread: working...
worker thread: working...
worker thread: working...
worker thread: working...
worker thread: working...
worker thread: working...
worker thread: working...
worker thread: working...
worker thread: working...
worker thread: working...
worker thread: terminating gracefully...
main thread: worker thread has terminated

```

See Also

Tasks

[Threading Sample](#)

Reference

[Threading \(C# Programming Guide\)](#)

[Using Threading \(C# Programming Guide\)](#)

[volatile \(C# Reference\)](#)

[Thread](#)

[Mutex](#)

[Monitor](#)

[Start](#)

[IsAlive](#)

[Sleep](#)

[Join](#)

[Abort](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Managed Threading](#)

[Threading Samples](#)

How to: Use a Thread Pool (C# Programming Guide)

A *thread pool* is a collection of threads that can be used to perform a number of tasks in the background. (See [Using Threading](#) for background information.) This leaves the primary thread free to perform other tasks asynchronously.

Thread pools are often employed in server applications. Each incoming request is assigned to a thread from the thread pool, so the request can be processed asynchronously, without tying up the primary thread or delaying the processing of subsequent requests.

Once a thread in the pool completes its task, it is returned to a queue of waiting threads, where it can be reused. This reuse enables applications to avoid the cost of creating a new thread for each task.

Thread pools typically have a maximum number of threads. If all the threads are busy, additional tasks are placed in queue until they can be serviced as threads become available.

You can implement your own thread pool, but it is easier to use the thread pool provided by the .NET Framework through the [ThreadPool](#) class.

The following example uses the .NET Framework thread pool to calculate the `Fibonacci` result for ten numbers between 20 and 40. Each `Fibonacci` result is represented by the `Fibonacci` class, which provides a method called `ThreadPoolCallback` that performs the calculation. An object representing each Fibonacci value is created, and the `ThreadPoolCallback` method is passed to [QueueUserWorkItem](#), which assigns an available thread in the pool to execute the method.

Because each `Fibonacci` object is given a semi-random value to compute, and because each of the ten threads will be competing for processor time, there is no way to know in advance how long it will take for all ten results to be calculated. That's why each `Fibonacci` object is passed an instance of the [ManualResetEvent](#) class during construction. Each object signals the provided event object when its calculation is complete, which allows the primary thread to block execution with [WaitAll](#) until all ten `Fibonacci` objects have calculated a result. The `Main` method then displays each `Fibonacci` result.

Example

C#

```
using System;
using System.Threading;

public class Fibonacci
{
    public Fibonacci(int n, ManualResetEvent doneEvent)
    {
        _n = n;
        _doneEvent = doneEvent;
    }

    // Wrapper method for use with thread pool.
    public void ThreadPoolCallback(Object threadContext)
    {
        int threadIndex = (int)threadContext;
        Console.WriteLine("thread {0} started...", threadIndex);
        _fibOfN = Calculate(_n);
        Console.WriteLine("thread {0} result calculated...", threadIndex);
        _doneEvent.Set();
    }

    // Recursive method that calculates the Nth Fibonacci number.
    public int Calculate(int n)
    {
        if (n <= 1)
        {
            return n;
        }

        return Calculate(n - 1) + Calculate(n - 2);
    }

    public int N { get { return _n; } }
    private int _n;
}
```

```

    public int FibOfN { get { return _fibOfN; } }
    private int _fibOfN;

    private ManualResetEvent _doneEvent;
}

public class ThreadPoolExample
{
    static void Main()
    {
        const int FibonacciCalculations = 10;

        // One event is used for each Fibonacci object
        ManualResetEvent[] doneEvents = new ManualResetEvent[FibonacciCalculations];
        Fibonacci[] fibArray = new Fibonacci[FibonacciCalculations];
        Random r = new Random();

        // Configure and launch threads using ThreadPool:
        Console.WriteLine("launching {0} tasks...", FibonacciCalculations);
        for (int i = 0; i < FibonacciCalculations; i++)
        {
            doneEvents[i] = new ManualResetEvent(false);
            Fibonacci f = new Fibonacci(r.Next(20,40), doneEvents[i]);
            fibArray[i] = f;
            ThreadPool.QueueUserWorkItem(f.ThreadPoolCallback, i);
        }

        // Wait for all threads in pool to calculation...
        WaitHandle.WaitAll(doneEvents);
        Console.WriteLine("All calculations are complete.");

        // Display the results...
        for (int i = 0; i < FibonacciCalculations; i++)
        {
            Fibonacci f = fibArray[i];
            Console.WriteLine("Fibonacci({0}) = {1}", f.N, f.FibOfN);
        }
    }
}

```

Sample Output

```

launching 10 tasks...
result calculated...
result calculated...
result calculated...
result calculated...
result calculated...
result calculated...
result calculated...
result calculated...
result calculated...
result calculated...
all calculations complete
Fibonacci(22) = 17711
Fibonacci(25) = 75025
Fibonacci(32) = 2178309
Fibonacci(36) = 14930352
Fibonacci(32) = 2178309
Fibonacci(26) = 121393
Fibonacci(35) = 9227465
Fibonacci(23) = 28657
Fibonacci(39) = 63245986
Fibonacci(22) = 17711

```

See Also

Tasks

[Monitor Synchronization Technology Sample](#)

[Wait Synchronization Technology Sample](#)

Reference

[Threading \(C# Programming Guide\)](#)

[Using Threading \(C# Programming Guide\)](#)

[Mutex](#)

[WaitAll](#)

[ManualResetEvent](#)

[Set](#)

[ThreadPool](#)

[QueueUserWorkItem](#)

[ManualResetEvent](#)

Concepts

[C# Programming Guide](#)

[Monitors](#)

Other Resources

[Security in the .NET Framework](#)

[HOW TO: Synchronize Access to a Shared Resource in a Multithreading Environment by Using Visual C# .NET](#)

Performance (C# Programming Guide)

This section covers two issues that can adversely effect performance, and links to resources on performance issues.

Boxing and Unboxing

Boxing and unboxing are computationally expensive processes. When a value type is boxed, an entirely new object must be created. This can take up to 20 times longer than an assignment. When unboxing, the casting process can take four times as long as an assignment. For more information, see [Boxing and Unboxing](#).

Destructors

Empty destructors should not be used. When a class contains a destructor, an entry is created in the Finalize queue. When the destructor is called, the garbage collector is invoked to process the queue. If the destructor is empty, this simply results in a loss of performance. For more information, see [Destructors](#).

Other Resources

- [Writing Faster Managed Code: Know What Things Cost](#)
- [Writing High-Performance Managed Applications : A Primer](#)
- [Garbage Collector Basics and Performance Hints](#)
- [Performance Tips and Tricks in .NET Applications](#)

See Also

Reference

[Security \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Reflection (C# Programming Guide)

Reflection provides objects (of type [Type](#)) that encapsulate assemblies, modules and types. You can use reflection to dynamically create an instance of a type, bind the type to an existing object, or get the type from an existing object and invoke its methods or access its fields and properties. If you are using attributes in your code, Reflection enables you to access them. For more information, see [Attributes](#).

Here's a simple example of Reflection using the static method **GetType** - inherited by all types from the **Object** base class - to obtain the type of a variable:

C#

```
// Using GetType to obtain type information:
int i = 42;
System.Type type = i.GetType();
System.Console.WriteLine(type);
```

The output is:

System.Int32

In this example, Reflection is used to obtain the full name of a loaded assembly:

C#

```
// Using Reflection to get information from an Assembly:
System.Reflection.Assembly o = System.Reflection.Assembly.Load("mscorlib.dll");
System.Console.WriteLine(o.GetName());
```

The output is:

mscorlib, Version=2.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089

Reflection Overview

Reflection is useful in the following situations:

- When you need to access attributes in your program's metadata. See the topic [Accessing Attributes With Reflection](#).
- For examining and instantiating types in an assembly.
- For building new types at runtime. Use classes in [System.Reflection.Emit](#).
- For performing late binding, accessing methods on types created at run time. See the topic [Dynamically Loading and Using Types](#).

Related Sections

For more information:

- [Reflection Overview](#)
- [Viewing Type Information](#)
- [Reflection and Generic Types](#)
- [System.Reflection.Emit](#)
- [Accessing Attributes With Reflection \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.12 Attributes
- 7.5.11 The typeof operator

See Also

Reference

[Application Domains \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Assemblies and the Global Assembly Cache \(C# Programming Guide\)](#)

C# DLLs (C# Programming Guide)

You can use Visual C# to create DLLs that are callable by other .NET applications and by unmanaged code.

In This Section

[How to: Create and Use C# DLLs \(C# Programming Guide\)](#)

Shows how to create class library projects in Visual C#.

Related Sections

[/baseaddress \(Specify Base Address of DLL\) \(C# Compiler Options\)](#)

[Creating a Class to Hold DLL Functions](#)

[Why Don't My DLL Breakpoints Work?](#)

[Debugging DLL Projects](#)

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

How to: Create and Use C# DLLs (C# Programming Guide)

A dynamic linking library (DLL) is linked to your program at run time. To demonstrate building and using a DLL, consider the following scenario:

- **MathLibrary.DLL:** The library file that contains the methods to be called at run time. In this example, the DLL contains two methods, `Add` and `Multiply`.
- **Add.cs:** The source file that contains the method `Add(long i, long j)`. It returns the sum of its parameters. The class `AddClass` that contains the method `Add` is a member of the namespace `UtilityMethods`.
- **Mult.cs:** The source code that contains the method `Multiply(long x, long y)`. It returns the product of its parameters. The class `MultiplyClass` that contains the method `Multiply` is also a member of the namespace `UtilityMethods`.
- **TestCode.cs:** The file that contains the `Main` method. It uses the methods in the DLL file to calculate the sum and the product of the run-time arguments.

Example

C#

```
// File: Add.cs
namespace UtilityMethods
{
    public class AddClass
    {
        public static long Add(long i, long j)
        {
            return (i + j);
        }
    }
}
```

C#

```
// File: Mult.cs
namespace UtilityMethods
{
    public class MultiplyClass
    {
        public static long Multiply(long x, long y)
        {
            return (x * y);
        }
    }
}
```

C#

```
// File: TestCode.cs

using UtilityMethods;

class TestCode
{
    static void Main(string[] args)
    {
        System.Console.WriteLine("Calling methods from MathLibrary.DLL:");

        if (args.Length != 2)
        {
            System.Console.WriteLine("Usage: TestCode <num1> <num2>");
            return;
        }
    }
}
```

```

    }

    long num1 = long.Parse(args[0]);
    long num2 = long.Parse(args[1]);

    long sum = AddClass.Add(num1, num2);
    long product = MultiplyClass.Multiply(num1, num2);

    System.Console.WriteLine("{0} + {1} = {2}", num1, num2, sum);
    System.Console.WriteLine("{0} * {1} = {2}", num1, num2, product);
}
}

```

This file contains the algorithm that uses the DLL methods, `Add` and `Multiply`. It starts with parsing the arguments entered from the command line, `num1` and `num2`. Then it calculates the sum by using the `Add` method on the `AddClass` class, and the product by using the `Multiply` method on the `MultiplyClass` class.

Notice that the **using** directive at the beginning of the file enables you to use the unqualified class names to reference the DLL methods at compile time, as follows:

C#

```
MultiplyClass.Multiply(num1, num2);
```

Otherwise, you have to use the fully qualified names, as follows:

C#

```
UtilityMethods.MultiplyClass.Multiply(num1, num2);
```

Execution

To run the program, enter the name of the EXE file, followed by two numbers, as follows:

```
TestCode 1234 5678
```

Output

```

Calling methods from MathLibrary.DLL:
1234 + 5678 = 6912
1234 * 5678 = 7006652

```

Compiling the Code

To build the file `MathLibrary.DLL`, compile the two files `Add.cs` and `Mult.cs` using the following command line:

```
csc /target:library /out:MathLibrary.DLL Add.cs Mult.cs
```

The `/target:library` compiler option tells the compiler to output a DLL instead of an EXE file. The `/out` compiler option followed by a file name is used to specify the DLL file name. Otherwise, the compiler uses the first file (`Add.cs`) as the name of the DLL.

To build the executable file, `TestCode.exe`, use the following command line:

```
csc /out:TestCode.exe /reference:MathLibrary.DLL TestCode.cs
```

The `/out` compiler option tells the compiler to output an EXE file and specifies the name of the output file (`TestCode.exe`). This compiler option is optional. The `/reference` compiler option specifies the DLL file or files that this program uses.

See Also

Tasks

[How to: Specify a Base Address for a DLL](#)

Concepts

[C# Programming Guide](#)

[Creating a Class to Hold DLL Functions](#)

Security (C# Programming Guide)

Security is an indispensable aspect of every C# applications, and must be considered at every phase of development: not merely when design and implementation are complete.

C# Specific Security Recommendations

This list is in not an exhaustive list of potential security problems. It highlights some common issues of which C# developers need to be aware.

- Use the [checked](#) keyword to control the overflow-checking context for integral-type arithmetic operations and conversions.
- Always use the most restrictive data type for parameters. For example, when passing a value into a method that describes the size of a data structure, use unsigned integer rather than integer.
- Do not make decisions based on filenames. Filenames can be expressed in many different ways, and your test for a particular file may be bypassed.
- Never, ever hardcode passwords or other sensitive information into your application.
- Always validate input that is used to generate SQL queries.
- Validate all inputs into your methods. The regular expression methods in [System.Text.RegularExpressions](#) namespace are useful for confirming input is of the correct form, such as an email address.
- Don't display exception information: it provides any would-be attacker with valuable clues.
- Ensure your application works while running with the least possible privileges. Few applications require a user to be logged in as an administrator.
- Don't use your own encryption algorithms, use the [System.Security.Cryptography](#) classes.
- Give your Assemblies strong names.
- Don't store sensitive information in XML or other configuration files.
- Check managed code that wraps native code carefully. Confirm the native code is secure, especially with regard to buffer overruns.
- Use caution when using delegates passed from outside your application.
- Run [FxCop](#) on your assemblies to ensure compliance with Microsoft .NET Framework Design Guidelines. FxCop can also find and warn against over 200 code defects.

In This Section

The following MSDN topics provide in-depth information on creating secure, reliable software.

- [Microsoft Security Developer Center](#).
- [Defend Your Apps and Critical User Info with Defensive Coding Techniques](#).
- [An Overview of Security in the .NET Framework](#).
- [Defend Your Code with Top Ten Security Tips Every Developer Must Know](#).
- [Security Brief: Beware of Fully Trusted Code](#).
- [Programmatically check for canonicalization issues with ASP.NET](#).
- [Security in Windows Forms Overview](#)
- [Writing Secure Managed Controls](#)

See Also

Concepts

[C# Programming Guide](#)

C# Reference

This section provides reference material on C# keywords, operators and compiler errors and warnings.

In This Section

[C# Keywords](#)

C# keywords and syntax.

[C# Operators](#)

C# operators and syntax.

[C# Preprocessor Directives](#)

Compiler commands for embedding in C# source code.

[C# Compiler Options](#)

Compiler options and how to use them.

[C# Terminology](#)

A glossary of C# words and phrases.

Related Sections

[C# Language Specification](#)

Provides pointers to the latest version of the C# Language Specification in Microsoft Word format.

[C# FAQ](#)

Provides a growing list of C# Frequently Asked Questions in the C# Developer Center.

[C# KB articles in the Microsoft Knowledge Base](#)

A live search of C# related Knowledge Base articles stored on MSDN.

[Visual C#](#)

Provides a portal to Visual C# documentation.

[Visual C# Samples](#)

Provides a list of and links to Visual C# samples.

[Visual C# Code Editor Features](#)

Provides links to conceptual and task topics that describe the IDE and Editor.

[Writing Applications with Visual C#](#)

Provides links to topics that describe how to do some common programming tasks.

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C#](#)

C# Keywords

Keywords are predefined reserved identifiers that have special meanings to the compiler. They cannot be used as identifiers in your program unless they include `@` as a prefix. For example, `@if` is a legal identifier but `if` is not because it is a keyword.

abstract	event	new	struct
as	explicit	null	switch
base	extern	object	this
bool	false	operator	throw
break	finally	out	true
byte	fixed	override	try
case	float	params	typeof
catch	for	private	uint
char	foreach	protected	ulong
checked	goto	public	unchecked
class	if	readonly	unsafe
const	implicit	ref	ushort
continue	in	return	using
decimal	int	sbyte	virtual
default	interface	sealed	volatile
delegate	internal	short	void
do	is	sizeof	while
double	lock	stackalloc	
else	long	static	
enum	namespace	string	

Contextual Keywords

get	partial	set
value	where	yield

See Also

Concepts

[C# Programming Guide](#)

Other Resources

Types (C# Reference)

The C# typing system contains the following categories:

- [Value types](#)
- [Reference types](#)
- [Pointer types](#)

Variables of the value types store data, while those of the reference types store references to the actual data. Reference types are also referred to as objects. Pointer types can be used only in [unsafe](#) mode.

It is possible to convert a value type to a reference type, and back again to value types, by using [boxing and unboxing](#). With the exception of a boxed value types, you cannot convert a reference type to a value type.

This section also introduces [void](#).

Value types are also nullable, which means they can store an addition non-value state. For more information, see [Nullable Types](#).

See Also

Reference

[C# Keywords](#)

[Casting \(C# Programming Guide\)](#)

[Data Types \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Types Reference Tables \(C# Reference\)](#)

Value Types (C# Reference)

The value types consist of two main categories:

- [Structs](#)
- [Enumerations](#)

Structs fall into these categories:

- Numeric types
 - [Integral types](#)
 - [Floating-point types](#)
 - [decimal](#)
- [bool](#)
- User defined structs.

Main Features of Value Types

Variable that are based on value types directly contain a values. Assigning one value type variable to another copies the contained value. This differs from the assignment of reference type variables, which copies a reference to the object but not the object itself.

All value types are derived implicitly from the [System.ValueType](#).

Unlike reference types, it is not possible to derive a new type from a value type. However, like reference types, structs can implement interfaces.

Unlike reference types, it is not possible for a value type to contain the **null** value. However, the [nullable types](#) feature does allow values types to be assigned to **null**.

Each value type has an implicit default constructor that initializes the default value of that type. For information on default values of value types, see [Default Values Table](#).

Main Features of Simple Types

All of the simple types -- those integral to the C# language -- are aliases of the .NET Framework System types. For example, [int](#) is an alias of [System.Int32](#). For a complete list of aliases, see [Built-In Types Table \(C# Reference\)](#).

Constant expressions, whose operands are all simple type constants, are evaluated at compilation time.

Simple types can be initialized using literals. For example, 'A' is a literal of the type **char** and 2001 is a literal of the type **int**.

Initializing Value Types

Local variables in C# must be initialized before being used. Therefore, if you declare a local variable without initialization like this:

```
int myInt;
```

you cannot use it before you initialize it. You can initialize it using the following statement:

```
myInt = new int(); // Invoke default constructor for int type.
```

which is equivalent to:

```
myInt = 0; // Assign an initial value, 0 in this example.
```

You can, of course, have the declaration and the initialization in the same statement like this:

```
int myInt = new int();
```

–or–

```
int myInt = 0;
```

Using the [new](#) operator calls the default constructor of the specific type and assigns the default value to the variable. In the preceding example, the default constructor assigned the value 0 to `myInt`. For more information on values assigned by calling default constructors, see [Default Values Table](#).

With user-defined types, use [new](#) to invoke the default constructor. For example, the following statement invokes the default constructor of the `Point` struct:

```
Point p = new Point(); // Invoke default constructor for the struct.
```

After this call, the struct is considered to be definitely assigned; that is, all of its members are initialized to their default values.

For more information on the `new` operator, see [new](#).

For information on formatting the output of numeric types, see [Formatting Numeric Results Table](#).

See Also

Reference

[C# Keywords](#)

[Reference Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Types \(C# Reference\)](#)

[Types Reference Tables \(C# Reference\)](#)

bool (C# Reference)

The **bool** keyword is an alias of [System.Boolean](#). It is used to declare variables to store the Boolean values, [true](#) and [false](#).

Note

If you require a Boolean variable that can also have a value of **null**, use `bool?`. For more information, see [Nullable Types \(C# Programming Guide\)](#).

Literals

You can assign a Boolean value to a **bool** variable. You can also assign an expression that evaluates to **bool** to a **bool** variable.

```
// keyword_bool.cs
using System;
public class MyClass
{
    static void Main()
    {
        bool i = true;
        char c = '0';
        Console.WriteLine(i);
        i = false;
        Console.WriteLine(i);

        bool Alphabetic = (c > 64 && c < 123);
        Console.WriteLine(Alphabetic);
    }
}
```

Output

```
True
False
False
```

Conversions

In C++, a value of type **bool** can be converted to a value of type **int**; in other words, **false** is equivalent to zero and **true** is equivalent to nonzero values. In C#, there is no conversion between the **bool** type and other types. For example, the following **if** statement is invalid in C#, while it is legal in C++:

```
int x = 123;
if (x)    // Invalid in C#
{
    printf_s("The value of x is nonzero.");
}
```

To test a variable of the type **int**, you have to explicitly compare it to a value, such as zero, as follows:

```
int x = 123;
if (x != 0)    // The C# way
{
    Console.WriteLine("The value of x is nonzero.");
}
```

Example

In this example, you enter a character from the keyboard and the program checks if the input character is a letter. If so, it checks if it is lowercase or uppercase. These checks are performed with the [IsLetter](#), and [IsLower](#), both of which return the **bool** type:

```
// keyword_bool_2.cs
```

```

using System;
public class BoolTest
{
    static void Main()
    {
        Console.Write("Enter a character: ");
        char c = (char)Console.Read();
        if (Char.IsLetter(c))
        {
            if (Char.IsLower(c))
            {
                Console.WriteLine("The character is lowercase.");
            }
            else
            {
                Console.WriteLine("The character is uppercase.");
            }
        }
        else
        {
            Console.WriteLine("Not an alphabetic character.");
        }
    }
}

```

Input

☒ X

Sample Output

```

Enter a character: X
The character is uppercase.
Additional sample runs might look as follow:
Enter a character: x
The character is lowercase.

Enter a character: 2
The character is not an alphabetic character.

```

C# Language Specification

For more information on **bool** and related subjects, see the following sections in the [C# Language Specification](#):

- 4.1.8 The bool Type
- 7.9.4 Boolean Equality Operators
- 7.11.1 Boolean Conditional Logical Operators

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

byte (C# Reference)

The **byte** keyword denotes an integral type that stores values as indicated in the following table.

Type	Range	Size	.NET Framework type
byte	0 to 255	Unsigned 8-bit integer	System.Byte

Literals

You can declare and initialize a **byte** variable like this example:

```
byte myByte = 255;
```

In the preceding declaration, the integer literal `255` is implicitly converted from [int](#) to **byte**. If the integer literal exceeds the range of **byte**, a compilation error will occur.

Conversions

There is a predefined implicit conversion from **byte** to [short](#), [ushort](#), [int](#), [uint](#), [long](#), [ulong](#), [float](#), [double](#), or [decimal](#).

You cannot implicitly convert non-literal numeric types of larger storage size to **byte**. For more information on the storage sizes of integral types, see [Integral Types Table \(C# Reference\)](#). Consider, for example, the following two **byte** variables `x` and `y`:

```
byte x = 10, y = 20;
```

The following assignment statement will produce a compilation error, because the arithmetic expression on the right-hand side of the assignment operator evaluates to **int** by default.

```
// Error: conversion from int to byte:  
byte z = x + y;
```

To fix this problem, use a cast:

```
// OK: explicit conversion:  
byte z = (byte)(x + y);
```

It is possible though, to use the following statements where the destination variable has the same storage size or a larger storage size:

```
int x = 10, y = 20;  
int m = x + y;  
long n = x + y;
```

Also, there is no implicit conversion from floating-point types to **byte**. For example, the following statement generates a compiler error unless an explicit cast is used:

```
// Error: no implicit conversion from double:  
byte x = 3.0;  
// OK: explicit conversion:  
byte y = (byte)3.0;
```

When calling overloaded methods, a cast must be used. Consider, for example, the following overloaded methods that use **byte** and [int](#) parameters:

```
public static void SampleMethod(int i) {}  
public static void SampleMethod(byte b) {}
```

Using the **byte** cast guarantees that the correct type is called, for example:

```
// Calling the method with the int parameter:  
SampleMethod(5);  
// Calling the method with the byte parameter:  
SampleMethod((byte)5);
```

For information on arithmetic expressions with mixed floating-point types and integral types, see [float](#) and [double](#).

For more information on implicit numeric conversion rules, see the [Implicit Numeric Conversions Table \(C# Reference\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 4.1.5 Integral Types

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[Byte](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

char (C# Reference)

The **char** keyword is used to declare a Unicode character in the range indicated in the following table. Unicode characters are 16-bit characters used to represent most of the known written languages throughout the world.

Type	Range	Size	.NET Framework type
char	U+0000 to U+ffff	Unicode 16-bit character	System.Char

Literals

Constants of the **char** type can be written as character literals, hexadecimal escape sequence, or Unicode representation. You can also cast the integral character codes. All of the following statements declare a **char** variable and initialize it with the character x:

```
char char1 = 'Z';           // Character literal
char char2 = '\x0058';      // Hexadecimal
char char3 = (char)88;      // Cast from integral type
char char4 = '\u0058';      // Unicode
```

Conversions

A **char** can be implicitly converted to [ushort](#), [int](#), [uint](#), [long](#), [ulong](#), [float](#), [double](#), or [decimal](#). However, there are no implicit conversions from other types to the **char** type.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 2.4.4.4 Character Literals
- 4.1.5 Integral Types

See Also

Reference

- [C# Keywords](#)
- [Integral Types Table \(C# Reference\)](#)
- [Built-In Types Table \(C# Reference\)](#)
- [Implicit Numeric Conversions Table \(C# Reference\)](#)
- [Explicit Numeric Conversions Table \(C# Reference\)](#)

Char Structure

Concepts

- [C# Programming Guide](#)

Other Resources

- [C# Reference](#)

decimal (C# Reference)

The **decimal** keyword denotes a 128-bit data type. Compared to floating-point types, the **decimal** type has a greater precision and a smaller range, which makes it suitable for financial and monetary calculations. The approximate range and precision for the **decimal** type are shown in the following table.

Type	Approximate Range	Precision	.NET Framework type
decimal	$\pm 1.0 \times 10^{-28}$ to $\pm 7.9 \times 10^{28}$	28-29 significant digits	System.Decimal

Literals

If you want a numeric real literal to be treated as **decimal**, use the suffix m or M, for example:

```
decimal myMoney = 300.5m;
```

Without the suffix m, the number is treated as a [double](#), thus generating a compiler error.

Conversions

The integral types are implicitly converted to **decimal** and the result evaluates to **decimal**. Therefore you can initialize a decimal variable using an integer literal, without the suffix, as follows:

```
decimal myMoney = 300;
```

There is no implicit conversion between floating-point types and the **decimal** type; therefore, a cast must be used to convert between these two types. For example:

```
decimal myMoney = 99.9m;  
double x = (double)myMoney;  
myMoney = (decimal)x;
```

You can also mix **decimal** and numeric integral types in the same expression. However, mixing **decimal** and floating-point types without a cast results in a compilation error.

For more information on implicit numeric conversions, see [Implicit Numeric Conversions Table \(C# Reference\)](#).

For more information on explicit numeric conversions, see [Explicit Numeric Conversions Table \(C# Reference\)](#).

Formatting Decimal Output

You can format the results by using the **String.Format** method, or through the [System.Console.Write](#) method, which calls **String.Format()**. The currency format is specified using the standard currency format string "C" or "c," as shown in Example 2. For more information on the **String.Format** method, see [System.String.Format\(System.String,System.Object\)](#).

Example

In this example, a **decimal** and an [int](#) are mixed in the same expression. The result evaluates to the **decimal** type.

If you attempt to add the [double](#) and **decimal** variables by using a statement like this:

```
double x = 9;  
Console.WriteLine(d + x); // Error
```

you will get the following error:

Operator '+' cannot be applied to operands of type 'double' and 'decimal'

```
// keyword_decimal.cs  
// decimal conversion  
using System;
```

```
public class TestDecimal
{
    static void Main ()
    {
        decimal d = 9.1m;
        int y = 3;
        Console.WriteLine(d + y);    // Result converted to decimal
    }
}
```

Output

12.1

In this example, the output is formatted using the currency format string. Notice that `x` is rounded because the decimal places exceed \$0.99. The variable `y`, which represents the maximum exact digits, is displayed exactly in the proper format.

```
// keyword_decimal2.cs  
// Decimal type formatting  
using System;  
public class TestDecimalFormat  
{  
    static void Main ()  
    {  
        decimal x = 0.999m;  
        decimal y = 999999999999999999999999999999999999m;  
        Console.WriteLine("My amount = {0:C}", x);  
        Console.WriteLine("Your amount = {0:C}", y);  
    }  
}
```

Output

My amount = \$1.00
Your amount = \$9,999,999,999,999,999,999,999,999,999.00

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 4.1.7 The decimal Type

See Also

Reference

C# Keywords

Integral Types Table (C# Reference)

Built-In Types Table (C# Reference)

Implicit Numeric Conversions Table (C# Reference)

Explicit Numeric Conversions Table (C# Reference)

Decimal Structure

Concepts

C# Programming Guide

Other Resources

C# Reference

double (C# Reference)

The **double** keyword denotes a simple type that stores 64-bit floating-point values. The following table shows the precision and approximate range for the **double** type.

Type	Approximate range	Precision	.NET Framework type
double	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$	15-16 digits	System.Double

Literals

By default, a real numeric literal on the right-hand side of the assignment operator is treated as **double**. However, if you want an integer number to be treated as **double**, use the suffix `d` or `D`, for example:

```
double x = 3D;
```

Conversions

You can mix numeric integral types and floating-point types in an expression. In this case, the integral types are converted to floating-point types. The evaluation of the expression is performed according to the following rules:

- If one of the floating-point types is **double**, the expression evaluates to **double**, or **bool** in the case of relational or Boolean expressions.
- If there is no **double** type in the expression, it evaluates to **float**, or **bool** in the case of relational or Boolean expressions.

A floating-point expression can contain the following sets of values:

- Positive and negative zero.
- Positive and negative infinity.
- Not-a-Number value (NaN).
- The finite set of nonzero values.

For more information on these values, refer to IEEE Standard for Binary Floating-Point Arithmetic, available on the Web site <http://www.ieee.org/portal/index.jsp>.

Example

In the following example, an **int**, a **short**, a **float**, and a **double** are added together giving a **double** result.

```
// keyword_double.cs
// Mixing types in expressions
using System;
class MixedTypes
{
    static void Main()
    {
        int x = 3;
        float y = 4.5f;
        short z = 5;
        double w = 1.7E+3;
        // Result of the 2nd argument is a double:
        Console.WriteLine("The sum is {0}", x + y + z + w);
    }
}
```

Output

```
The sum is 1712.5
```

For more information, see the following sections in the [C# Language Specification](#):

- [1.3 Types and Variables](#)
- [4.1.5 Integral Types](#)

See Also

Reference

[C# Keywords](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Default Values Table \(C# Reference\)](#)

[Floating-Point Types Table \(C# Reference\)](#)

enum (C# Reference)

The **enum** keyword is used to declare an enumeration, a distinct type consisting of a set of named constants called the enumerator list. Every enumeration type has an underlying type, which can be any integral type except [char](#). The default underlying type of the enumeration elements is [int](#). By default, the first enumerator has the value 0, and the value of each successive enumerator is increased by 1. For example:

```
enum Days {Sat, Sun, Mon, Tue, Wed, Thu, Fri};
```

In this enumeration, `Sat` is 0, `Sun` is 1, `Mon` is 2, and so forth. Enumerators can have initializers to override the default values. For example:

```
enum Days {Sat=1, Sun, Mon, Tue, Wed, Thu, Fri};
```

In this enumeration, the sequence of elements is forced to start from 1 instead of 0.

A variable of type `Days` can be assigned any value in the range of the underlying type; the values are not limited to the named constants.

The default value of an **enum** `E` is the value produced by the expression `(E) 0`.

Note

An enumerator may not contain white space in its name.

The underlying type specifies how much storage is allocated for each enumerator. However, an explicit cast is needed to convert from **enum** type to an integral type. For example, the following statement assigns the enumerator `Sun` to a variable of the type [int](#) using a cast to convert from **enum** to [int](#):

```
int x = (int)Days.Sun;
```

When you apply [System.FlagsAttribute](#) to an enumeration that contains some elements combined with a bitwise OR operation, you will notice that the attribute affects the behavior of the **enum** when used with some tools. You can notice these changes when using tools such as the **Console** class methods, the Expression Evaluator, and so forth. (See example 3).

Robust Programming

Assigning additional values new versions of enums, or changing the values of the enum members in a new version, can cause problems for dependant source code. It is often the case that **enum** values are used in [switch](#) statements, and if additional elements have been added to the **enum** type, the test for default values can return true unexpectedly.

If other developers will be using your code, it is important to provide guidelines on how their code should react if new elements are added to any **enum** types.

Example

In this example, an enumeration, `Days`, is declared. Two enumerators are explicitly converted to integer and assigned to integer variables.

```
// keyword_enum.cs
// enum initialization:
using System;
public class EnumTest
{
    enum Days {Sat=1, Sun, Mon, Tue, Wed, Thu, Fri};

    static void Main()
    {
        int x = (int)Days.Sun;
        int y = (int)Days.Fri;
    }
}
```

```

        Console.WriteLine("Sun = {0}", x);
        Console.WriteLine("Fri = {0}", y);
    }
}

```

Output

Sun = 2
Fri = 7

In this example, the base-type option is used to declare an **enum** whose members are of the type **long**. Notice that even though the underlying type of the enumeration is **long**, the enumeration members must still be explicitly converted to type **long** using a cast.

```

// keyword_enum2.cs
// Using long enumerators
using System;
public class EnumTest
{
    enum Range :long {Max = 2147483648L, Min = 255L};
    static void Main()
    {
        long x = (long)Range.Max;
        long y = (long)Range.Min;
        Console.WriteLine("Max = {0}", x);
        Console.WriteLine("Min = {0}", y);
    }
}

```

Output

Max = 2147483648
Min = 255

The following code example illustrates the use and effect of the **System.FlagsAttribute** attribute on an **enum** declaration.

```

// enumFlags.cs
// Using the FlagsAttribute on enumerations.
using System;

[Flags]
public enum CarOptions
{
    SunRoof = 0x01,
    Spoiler = 0x02,
    FogLights = 0x04,
    TintedWindows = 0x08,
}

class FlagTest
{
    static void Main()
    {
        CarOptions options = CarOptions.SunRoof | CarOptions.FogLights;
        Console.WriteLine(options);
        Console.WriteLine((int)options);
    }
}

```

Output

SunRoof, FogLights
5

Comments

Notice that if you remove the initializer from `Sat=1`, the result will be:

```
Sun = 1  
Fri = 6
```

Comments

Notice that if you remove **FlagsAttribute**, the example will output the following:

5

5

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.10 Enums
- 6.2.2 Explicit Enumeration Conversions
- 14 Enums

See Also

Tasks

[Attributes Sample](#)

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Enumeration Design](#)

Other Resources

[C# Reference](#)

float (C# Reference)

The **float** keyword denotes a simple type that stores 32-bit floating-point values. The following table shows the precision and approximate range for the **float** type.

Type	Approximate range	Precision	.NET Framework type
float	$\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$	7 digits	System.Single

Literals

By default, a real numeric literal on the right-hand side of the assignment operator is treated as [double](#). Therefore, to initialize a float variable, use the suffix `F` or `F`, like this:

```
float x = 3.5F;
```

If you do not use the suffix in the previous declaration, you will get a compilation error because you are attempting to store a [double](#) value into a **float** variable.

Conversions

You can mix numeric integral types and floating-point types in an expression. In this case, the integral types are converted to floating-point types. The evaluation of the expression is performed according to the following rules:

- If one of the floating-point types is [double](#), the expression evaluates to [double](#) or [bool](#) in the case of relational or Boolean expressions.
- If there is no [double](#) type in the expression, the expression evaluates to **float** or [bool](#) in the case of relational or Boolean expressions.

A floating-point expression can contain the following sets of values:

- Positive and negative zero
- Positive and negative infinity
- Not-a-Number value (NaN)
- The finite set of nonzero values

For more information on these values, refer to IEEE Standard for Binary Floating-Point Arithmetic, available on the Web site <http://www.ieee.org/>.

Example

In the following example, an [int](#), a [short](#), and a **float** are included in a mathematical expression giving a **float** result. Notice that there is no [double](#) in the expression.

```
// keyword_float.cs
// Mixing types in expressions
using System;
class MixedTypes
{
    static void Main()
    {
        int x = 3;
        float y = 4.5f;
        short z = 5;
        Console.WriteLine("The result is {0}", x * y / z);
    }
}
```

Output

The result is 2.7

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [4.1.6 Floating Point Types](#)
- [6.2.1 Explicit Numeric Conversions](#)

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[Single Structure](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

int (C# Reference)

The **int** keyword denotes an integral type that stores values according to the size and range shown in the following table.

Type	Range	Size	.NET Framework type
int	-2,147,483,648 to 2,147,483,647	Signed 32-bit integer	System.Int32

Literals

You can declare and initialize a variable of the type **int** like this example:

```
int i = 123;
```

When an integer literal has no suffix, its type is the first of these types in which its value can be represented: **int**, [uint](#), [long](#), [ulong](#). In this example, it is of the type **int**.

Conversions

There is a predefined implicit conversion from **int** to [long](#), [float](#), [double](#), or [decimal](#). For example:

```
// '123' is an int, so an implicit conversion takes place here:  
float f = 123;
```

There is a predefined implicit conversion from [sbyte](#), [byte](#), [short](#), [ushort](#), or [char](#) to **int**. For example, the following assignment statement will produce a compilation error without a cast:

```
long aLong = 22;  
int i1 = aLong;           // Error: no implicit conversion from long.  
int i2 = (int)aLong;      // OK: explicit conversion.
```

Notice also that there is no implicit conversion from floating-point types to **int**. For example, the following statement generates a compiler error unless an explicit cast is used:

```
int x = 3.0;              // Error: no implicit conversion from double.  
int y = (int)3.0;         // OK: explicit conversion.
```

For more information on arithmetic expressions with mixed floating-point types and integral types, see [float](#) and [double](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 4.1.5 Integral Types

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[Int32 Structure](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

long (C# Reference)

The **long** keyword denotes an integral type that stores values according to the size and range shown in the following table.

Type	Range	Size	.NET Framework type
long	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Signed 64-bit integer	System.Int64

Literals

You can declare and initialize a **long** variable like this example:

```
long long1 = 4294967296;
```

When an integer literal has no suffix, its type is the first of these types in which its value can be represented: [int](#), [uint](#), **long**, [ulong](#). In the preceding example, it is of the type **long** because it exceeds the range of [uint](#) (see [Integral Types Table \(C# Reference\)](#) for the storage sizes of integral types).

You can also use the suffix L with the **long** type like this:

```
long long2 = 4294967296L;
```

When you use the suffix L, the type of the literal integer is determined to be either **long** or [ulong](#) according to its size. In the case it is **long** because it less than the range of [ulong](#).

A common use of the suffix is with calling overloaded methods. Consider, for example, the following overloaded methods that use **long** and [int](#) parameters:

```
public static void SampleMethod(int i) {}
public static void SampleMethod(long l) {}
```

Using the suffix L guarantees that the correct type is called, for example:

```
SampleMethod(5);    // Calling the method with the int parameter
SampleMethod(5L);   // Calling the method with the long parameter
```

You can use the **long** type with other numeric integral types in the same expression, in which case the expression is evaluated as **long** (or [bool](#) in the case of relational or Boolean expressions). For example, the following expression evaluates as **long**:

```
898L + 88
```

Note

You can also use the lowercase letter "l" as a suffix. However, this generates a compiler warning because the letter "l" is easily confused with the digit "1." Use "L" for clarity.

For information on arithmetic expressions with mixed floating-point types and integral types, see [float](#) and [double](#).

Conversions

There is a predefined implicit conversion from **long** to [float](#), [double](#), or [decimal](#). Otherwise a cast must be used. For example, the following statement will produce a compilation error without an explicit cast:

```
int x = 8L;           // Error: no implicit conversion from long to int
int x = (int)8L;      // OK: explicit conversion to int
```

There is a predefined implicit conversion from [sbyte](#), [byte](#), [short](#), [ushort](#), [int](#), [uint](#), or [char](#) to **long**.

Notice also that there is no implicit conversion from floating-point types to **long**. For example, the following statement generates a compiler error unless an explicit cast is used:

```
long x = 3.0;           // Error: no implicit conversion from double
long y = (long)3.0;     // OK: explicit conversion
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 4.1.5 Integral Types

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[Int64 Structure](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

sbyte (C# Reference)

The **sbyte** keyword denotes an integral type that stores values according to the size and range shown in the following table.

Type	Range	Size	.NET Framework type
sbyte	-128 to 127	Signed 8-bit integer	System.SByte

Literals

You can declare and initialize an **sbyte** variable like this example:

```
sbyte sByte1 = 127;
```

In the preceding declaration, the integer literal 127 is implicitly converted from [int](#) to **sbyte**. If the integer literal exceeds the range of **sbyte**, a compilation error will occur.

A cast must be used when calling overloaded methods. Consider, for example, the following overloaded methods that use **sbyte** and [int](#) parameters:

```
public static void SampleMethod(int i) {}  
public static void SampleMethod(sbyte b) {}
```

Using the **sbyte** cast guarantees that the correct type is called, for example:

```
// Calling the method with the int parameter:  
SampleMethod(5);  
// Calling the method with the sbyte parameter:  
SampleMethod((sbyte)5);
```

Conversions

There is a predefined implicit conversion from **sbyte** to [short](#), [int](#), [long](#), [float](#), [double](#), or [decimal](#).

You cannot implicitly convert nonliteral numeric types of larger storage size to **sbyte** (see [Integral Types Table \(C# Reference\)](#) for the storage sizes of integral types). Consider, for example, the following two **sbyte** variables *x* and *y*:

```
sbyte x = 10, y = 20;
```

The following assignment statement will produce a compilation error, because the arithmetic expression on the right-hand side of the assignment operator evaluates to [int](#) by default.

```
sbyte z = x + y;    // Error: conversion from int to sbyte
```

To fix this problem, cast the expression like this:

```
sbyte z = (sbyte)(x + y);    // OK: explicit conversion
```

It is possible though to use the following statements, where the destination variable has the same storage size or a larger storage size:

```
sbyte x = 10, y = 20;  
int m = x + y;  
long n = x + y;
```

Notice also that there is no implicit conversion from floating-point types to **sbyte**. For example, the following statement generates a compiler error unless an explicit cast is used:

```
sbyte x = 3.0;           // Error: no implicit conversion from double
sbyte y = (sbyte)3.0;    // OK: explicit conversion
```

For information on arithmetic expressions with mixed floating-point types and integral types, see [float](#) and [double](#).

For more information on implicit numeric conversion rules, see the [Implicit Numeric Conversions Table \(C# Reference\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 4.1.5 Integral Types

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[SByte Structure](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

short (C# Reference)

The **short** keyword denotes an integral data type that stores values according to the size and range shown in the following table.

Type	Range	Size	.NET Framework type
short	-32,768 to 32,767	Signed 16-bit integer	System.Int16

Literals

You can declare and initialize a **short** variable like this example:

```
short x = 32767;
```

In the preceding declaration, the integer literal `32767` is implicitly converted from [int](#) to **short**. If the integer literal does not fit into a **short** storage location, a compilation error will occur.

A cast must be used when calling overloaded methods. Consider, for example, the following overloaded methods that use **short** and [int](#) parameters:

```
public static void SampleMethod(int i) {}  
public static void SampleMethod(short s) {}
```

Using the **short** cast guarantees that the correct type is called, for example:

```
SampleMethod(5);           // Calling the method with the int parameter  
SampleMethod((short)5);    // Calling the method with the short parameter
```

Conversions

There is a predefined implicit conversion from **short** to [int](#), [long](#), [float](#), [double](#), or [decimal](#).

You cannot implicitly convert nonliteral numeric types of larger storage size to **short** (see [Integral Types Table \(C# Reference\)](#) for the storage sizes of integral types). Consider, for example, the following two **short** variables `x` and `y`:

```
short x = 5, y = 12;
```

The following assignment statement will produce a compilation error, because the arithmetic expression on the right-hand side of the assignment operator evaluates to [int](#) by default.

```
short z = x + y; // Error: no conversion from int to short
```

To fix this problem, use a cast:

```
short z = (short) (x + y); // OK: explicit conversion
```

It is possible though to use the following statements, where the destination variable has the same storage size or a larger storage size:

```
int m = x + y;  
long n = x + y;
```

There is no implicit conversion from floating-point types to **short**. For example, the following statement generates a compiler error unless an explicit cast is used:

```
short x = 3.0;           // Error: no implicit conversion from double  
short y = (short)3.0;    // OK: explicit conversion
```

For information on arithmetic expressions with mixed floating-point types and integral types, see [float](#) and [double](#).

For more information on implicit numeric conversion rules, see the [Implicit Numeric Conversions Table \(C# Reference\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.3 Types and Variables](#)
- [4.1.5 Integral Types](#)

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[Int16](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

struct (C# Reference)

A **struct** type is a value type that is typically used to encapsulate small groups of related variables, such as the coordinates of a rectangle or the characteristics of an item in an inventory. The following example shows a simple struct declaration:

```
public struct Book
{
    public decimal price;
    public string title;
    public string author;
}
```

Remarks

Structs can also contain [constructors](#), [constants](#), [fields](#), [methods](#), [properties](#), [indexers](#), [operators](#), [events](#), and [nested types](#), although if several such members are required, you should consider making your type a class instead.

Structs can implement an interface but they cannot inherit from another struct. For that reason, struct members cannot be declared as **protected**.

For more information, see [Structs \(C# Programming Guide\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 11 Structs

See Also

Reference

[C# Keywords](#)

[Built-In Types Table \(C# Reference\)](#)

[Value Types \(C# Reference\)](#)

[class \(C# Reference\)](#)

[interface \(C# Reference\)](#)

[Objects, Classes and Structs \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Default Values Table \(C# Reference\)](#)

[Types \(C# Reference\)](#)

uint (C# Reference)

The **uint** keyword denotes an integral type that stores values according to the size and range shown in the following table.

Type	Range	Size	.NET Framework type
uint	0 to 4,294,967,295	Unsigned 32-bit integer	System.UInt32

Literals

You can declare and initialize a variable of the type **uint** like this example:

```
uint myUInt = 4294967290;
```

When an integer literal has no suffix, its type is the first of these types in which its value can be represented: [int](#), **uint**, [long](#), [ulong](#). In this example, it is **uint**:

```
uint uInt1 = 123;
```

You can also use the suffix `u` or `U`, like this:

```
uint uInt2 = 123U;
```

When you use the suffix `U` or `u`, the type of the literal is determined to be either **uint** or **ulong** according to the numeric value of the literal. For example:

```
Console.WriteLine(44U.GetType());  
Console.WriteLine(323442434344U.GetType());
```

This code displays `System.UInt32`, followed by `System.UInt64` -- the underlying types for **uint** and **ulong** respectively -- because the second literal is too large to be stored by the **uint** type.

Conversions

There is a predefined implicit conversion from **uint** to [long](#), [ulong](#), [float](#), [double](#), or [decimal](#). For example:

```
float myFloat = 4294967290;    // OK: implicit conversion to float
```

There is a predefined implicit conversion from [byte](#), [ushort](#), or [char](#) to **uint**. Otherwise you must use a cast. For example, the following assignment statement will produce a compilation error without a cast:

```
long aLong = 22;  
// Error -- no implicit conversion from long:  
uint uInt1 = aLong;  
// OK -- explicit conversion:  
uint uInt2 = (uint)aLong;
```

Notice also that there is no implicit conversion from floating-point types to **uint**. For example, the following statement generates a compiler error unless an explicit cast is used:

```
// Error -- no implicit conversion from double:  
uint x = 3.0;  
// OK -- explicit conversion:  
uint y = (uint)3.0;
```

For information on arithmetic expressions with mixed floating-point types and integral types, see [float](#) and [double](#).

For more information on implicit numeric conversion rules, see the [Implicit Numeric Conversions Table \(C# Reference\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.3 Types and Variables](#)
- [4.1.5 Integral Types](#)

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[UInt32 Structure](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

ulong (C# Reference)

The **ulong** keyword denotes an integral type that stores values according to the size and range shown in the following table.

Type	Range	Size	.NET Framework type
ulong	0 to 18,446,744,073,709,551,615	Unsigned 64-bit integer	System.UInt64

Literals

You can declare and initialize a **ulong** variable like this example:

```
ulong uLong = 9223372036854775808;
```

When an integer literal has no suffix, its type is the first of these types in which its value can be represented: [int](#), [uint](#), [long](#), **ulong**. In the example above, it is of the type **ulong**.

You can also use suffixes to specify the type of the literal according to the following rules:

- If you use L or l, the type of the literal integer will be either [long](#) or **ulong** according to its size.

Note

You can use the lowercase letter "l" as a suffix. However, this generates a compiler warning because the letter "l" is easily confused with the digit "1." Use "L" for clarity.

- If you use U or u, the type of the literal integer will be either [uint](#) or **ulong** according to its size.
- If you use UL, ul, Ul, uL, LU, lu, Lu, or lU, the type of the literal integer will be **ulong**.

For example, the output of the following three statements will be the system type **UInt64**, which corresponds to the alias **ulong**:

```
Console.WriteLine(9223372036854775808L.GetType());  
Console.WriteLine(123UL.GetType());  
Console.WriteLine((123UL + 456).GetType());
```

A common use of the suffix is with calling overloaded methods. Consider, for example, the following overloaded methods that use **ulong** and [int](#) parameters:

```
public static void SampleMethod(int i) {}  
public static void SampleMethod(ulong l) {}
```

Using a suffix with the **ulong** parameter guarantees that the correct type is called, for example:

```
SampleMethod(5);    // Calling the method with the int parameter  
SampleMethod(5UL); // Calling the method with the ulong parameter
```

Conversions

There is a predefined implicit conversion from **ulong** to [float](#), [double](#), or [decimal](#).

There is no implicit conversion from **ulong** to any integral type. For example, the following statement will produce a compilation error without an explicit cast:

```
long long1 = 8UL;    // Error: no implicit conversion from ulong
```

There is a predefined implicit conversion from [byte](#), [ushort](#), [uint](#), or [char](#) to **ulong**.

Also, there is no implicit conversion from floating-point types to **ulong**. For example, the following statement generates a compiler error unless an explicit cast is used:

```
// Error -- no implicit conversion from double:  
ulong x = 3.0;  
// OK -- explicit conversion:  
ulong y = (ulong)3.0;
```

For information on arithmetic expressions with mixed floating-point types and integral types, see [float](#) and [double](#).

For more information on implicit numeric conversion rules, see the [Implicit Numeric Conversions Table \(C# Reference\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 4.1.5 Integral Types

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[UInt64 Structure](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

ushort (C# Reference)

The **ushort** keyword denotes an integral data type that stores values according to the size and range shown in the following table.

Type	Range	Size	.NET Framework type
ushort	0 to 65,535	Unsigned 16-bit integer	System.UInt16

Literals

You can declare and initialize a **ushort** variable like this example:

```
ushort myShort = 65535;
```

In the preceding declaration, the integer literal `65535` is implicitly converted from [int](#) to **ushort**. If the integer literal exceeds the range of **ushort**, a compilation error will occur.

A cast must be used when calling overloaded methods. Consider, for example, the following overloaded methods that use **ushort** and [int](#) parameters:

```
public static void SampleMethod(int i) {}  
public static void SampleMethod(ushort s) {}
```

Using the **ushort** cast guarantees that the correct type is called, for example:

```
// Calls the method with the int parameter:  
SampleMethod(5);  
// Calls the method with the ushort parameter:  
SampleMethod((ushort)5);
```

Conversions

There is a predefined implicit conversion from **ushort** to [int](#), [uint](#), [long](#), [ulong](#), [float](#), [double](#), or [decimal](#).

There is a predefined implicit conversion from [byte](#) or [char](#) to **ushort**. Otherwise a cast must be used to perform an explicit conversion. Consider, for example, the following two **ushort** variables `x` and `y`:

```
ushort x = 5, y = 12;
```

The following assignment statement will produce a compilation error, because the arithmetic expression on the right-hand side of the assignment operator evaluates to **int** by default.

```
ushort z = x + y;    // Error: conversion from int to ushort
```

To fix this problem, use a cast:

```
ushort z = (ushort)(x + y);    // OK: explicit conversion
```

It is possible though to use the following statements, where the destination variable has the same storage size or a larger storage size:

```
int m = x + y;  
long n = x + y;
```

Notice also that there is no implicit conversion from floating-point types to **ushort**. For example, the following statement generates a compiler error unless an explicit cast is used:

```
// Error -- no implicit conversion from double:  
ushort x = 3.0;  
// OK -- explicit conversion:  
ushort y = (ushort)3.0;
```

For information on arithmetic expressions with mixed floating-point types and integral types, see [float](#) and [double](#).

For more information on implicit numeric conversion rules, see the [Implicit Numeric Conversions Table \(C# Reference\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.3 Types and Variables
- 4.1.5 Integral Types

See Also

Reference

[C# Keywords](#)

[Integral Types Table \(C# Reference\)](#)

[Built-In Types Table \(C# Reference\)](#)

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

[UInt16 Structure](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Reference Types (C# Reference)

Variables of reference types, referred to as objects, store references to the actual data. This section introduces the following keywords used to declare reference types:

- [class](#)
- [interface](#)
- [delegate](#)

This section also introduces the following built-in reference types:

- [object](#)
- [string](#)

See Also

Reference

[C# Keywords](#)

[Value Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Types \(C# Reference\)](#)

class (C# Reference)

Classes are declared using the keyword **class**, as shown in the following example:

```
class TestClass
{
    // Methods, properties, fields, events, delegates
    // and nested classes go here.
}
```

Remarks

Unlike C++, only single inheritance is allowed in C#. In other words, a class can inherit implementation from one base class only. However, a class can implement more than one interface. The following table shows examples of class inheritance and interface implementation:

Inheritance	Example
None	class ClassA { }
Single	class DerivedClass: BaseClass { }
None, implements two interfaces	class ImplClass: IFace1, IFace2 { }
Single, implements one interface	class ImplDerivedClass: BaseClass, IFace1 { }

The access levels [protected](#) and [private](#) are only allowed on nested classes.

You can also declare generic classes that have type parameters; see [Generic Classes](#) for more information.

A class can contain declarations of the following members:

- [Constructors](#)
- [Destructors](#)
- [Constants](#)
- [Fields](#)
- [Methods](#)
- [Properties](#)
- [Indexers](#)
- [Operators](#)
- [Events](#)
- [Delegates](#)
- [Classes](#)
- [Interfaces](#)
- [Structs](#)

Example

The following example demonstrates declaring class fields, constructors, and methods. It also demonstrates object instantiation and printing instance data. In this example, two classes are declared, the `Kid` class, which contains two private fields (`name` and `age`) and two public methods. The second class, `MainClass`, is used to contain `Main`.

```
// keyword_class.cs
```

```
// class example
using System;
class Kid
{
    private int age;
    private string name;

    // Default constructor:
    public Kid()
    {
        name = "N/A";
    }

    // Constructor:
    public Kid(string name, int age)
    {
        this.name = name;
        this.age = age;
    }

    // Printing method:
    public void PrintKid()
    {
        Console.WriteLine("{0}, {1} years old.", name, age);
    }
}

class MainClass
{
    static void Main()
    {
        // Create objects
        // Objects must be created using the new operator:
        Kid kid1 = new Kid("Craig", 11);
        Kid kid2 = new Kid("Sally", 10);

        // Create an object using the default constructor:
        Kid kid3 = new Kid();

        // Display results:
        Console.Write("Kid #1: ");
        kid1.PrintKid();
        Console.Write("Kid #2: ");
        kid2.PrintKid();
        Console.Write("Kid #3: ");
        kid3.PrintKid();
    }
}
```

Output

```
Kid #1: Craig, 11 years old.
Kid #2: Sally, 10 years old.
Kid #3: N/A, 0 years old.
```

Comments

Notice, in the preceding example, that the private fields (`name` and `age`) can only be accessed through the public methods of the `Kid` class. For example, you cannot print the kid's name, from the `Main` method, using a statement like this:

```
Console.Write(kid1.name);    // Error
```

Accessing private members of `Kid` from `Main` would only be possible if `Main` were a member of the class.

Types declared inside a class without an [Access Modifier](#) default to **private**, so the data members in this example would still be **private** if the keyword were removed.

Finally, notice that for the object created using the default constructor (`kid3`), the `age` field was initialized to zero by default.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.6 Classes and objects](#)
- [3.4.4 Class members](#)
- [4.2.1 Class types](#)
- [10 Classes](#)

See Also

Reference

[C# Keywords](#)

[Reference Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

delegate (C# Reference)

The declaration of a delegate type takes the following form:

```
public delegate void TestDelegate(string message);
```

The **delegate** keyword is used to declare a reference type that can be used to encapsulate a named or an anonymous method. Delegates are similar to function pointers in C++; however, delegates are type-safe and secure. For applications of delegates, see [Delegates](#) and [Generic Delegates](#).

Remarks

Delegates are the basis for [Events](#).

A delegate can be instantiated by associating it either with a named or anonymous method. For more information, see [Named Methods](#) and [Anonymous Methods](#).

For use with named methods, the delegate must be instantiated with a method that has an acceptable signature. For more information on the degree of variance that is allowed in the method signature, see [Covariance and Contravariance in Delegates](#). For use with anonymous methods, the delegate and the code to be associated with it are declared together. Both ways of instantiating delegates are discussed in this section.

```
using System;
// Declare delegate -- defines required signature:
delegate void SampleDelegate(string message);

class MainClass
{
    // Regular method that matches signature:
    static void SampleDelegateMethod(string message)
    {
        Console.WriteLine(message);
    }

    static void Main()
    {
        // Instantiate delegate with named method:
        SampleDelegate d1 = SampleDelegateMethod;
        // Instantiate delegate with anonymous method:
        SampleDelegate d2 = delegate(string message)
        {
            Console.WriteLine(message);
        };

        // Invoke delegate d1:
        d1("Hello");
        // Invoke delegate d2:
        d2(" World");
    }
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.11 Delegates](#)
- [15 Delegates](#)

See Also

Reference

[C# Keywords](#)

[Reference Types \(C# Reference\)](#)

[Anonymous Methods \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Delegates \(C# Programming Guide\)](#)

[Events \(C# Programming Guide\)](#)

[Named Methods \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

interface (C# Reference)

An interface contains only the signatures of [methods](#), [delegates](#) or [events](#). The implementation of the methods is done in the class that implements the interface, as shown in the following example:

```
interface ISampleInterface
{
    void SampleMethod();
}

class ImplementationClass : ISampleInterface
{
    // Explicit interface member implementation:
    void ISampleInterface.SampleMethod()
    {
        // Method implementation.
    }

    static void Main()
    {
        // Declare an interface instance.
        ISampleInterface obj = new ImplementationClass();

        // Call the member.
        obj.SampleMethod();
    }
}
```

Remarks

An interface can be a member of a namespace or a class and can contain signatures of the following members:

- [Methods](#)
- [Properties](#)
- [Indexers](#)
- [Events](#)

An interface can inherit from one or more base interfaces.

When a base type list contains a base class and interfaces, the base class must come first in the list.

A class that implements an interface can explicitly implement members of that interface. An explicitly implemented member cannot be accessed through a class instance, but only through an instance of the interface, for example:

For more details and code examples on explicit interface implementation, see [Explicit Interface Implementation \(C# Programming Guide\)](#).

Example

The following example demonstrates interface implementation. In this example, the interface `IPoint` contains the property declaration, which is responsible for setting and getting the values of the fields. The class `Point` contains the property implementation.

```
// keyword_interface_2.cs
// Interface implementation
using System;
interface IPoint
{
    // Property signatures:
    int x
    {
        get;
    }
}
```

```

        set;
    }

    int y
    {
        get;
        set;
    }
}

class Point : IPoint
{
    // Fields:
    private int _x;
    private int _y;

    // Constructor:
    public Point(int x, int y)
    {
        _x = x;
        _y = y;
    }

    // Property implementation:
    public int x
    {
        get
        {
            return _x;
        }

        set
        {
            _x = value;
        }
    }

    public int y
    {
        get
        {
            return _y;
        }
        set
        {
            _y = value;
        }
    }
}

class MainClass
{
    static void PrintPoint(IPoint p)
    {
        Console.WriteLine("x={0}, y={1}", p.x, p.y);
    }

    static void Main()
    {
        Point p = new Point(2, 3);
        Console.Write("My Point: ");
        PrintPoint(p);
    }
}

```

Output

```
My Point: x=2, y=3
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.9 Interfaces](#)
- [3.4.5 Interface members](#)
- [4.2.4 Interface types](#)
- [10.1.2.2 Interface implementations](#)
- [11.2 Struct interfaces](#)
- [13 Interfaces](#)

See Also

Reference

[C# Keywords](#)

[Reference Types \(C# Reference\)](#)

[Using Properties \(C# Programming Guide\)](#)

[Using Indexers \(C# Programming Guide\)](#)

[class \(C# Reference\)](#)

[struct \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

object (C# Reference)

The **object** type is an alias for [Object](#) in the .NET Framework. In the unified type system of C#, all types, predefined and user-defined, reference types and value types, inherit directly or indirectly from **Object**. You can assign values of any type to variables of type **object**. When a variable of a value type is converted to object, it is said to be *boxed*. When a variable of type object is converted to a value type, it is said to be *unboxed*. For more information, see [Boxing and Unboxing](#).

Example

The following sample shows how variables of type **object** can accept values of any data type and how variables of type **object** can use methods on **Object** from the .NET Framework.

```
// keyword_object.cs
using System;
class SampleClass
{
    public int i = 10;
}

class MainClass
{
    static void Main()
    {
        object a;
        a = 1;    // an example of boxing
        Console.WriteLine(a);
        Console.WriteLine(a.GetType());
        Console.WriteLine(a.ToString());

        a = new SampleClass();
        SampleClass classRef;
        classRef = (SampleClass)a;
        Console.WriteLine(classRef.i);
    }
}
```

Output

1
System.Int32
1
10

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1 Introduction
- 4.2.2 The object type

See Also

Reference

[C# Keywords](#)

[Reference Types \(C# Reference\)](#)

[Value Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

string (C# Reference)

The **string** type represents a sequence of zero or more Unicode characters. **string** is an alias for [String](#) in the .NET Framework.

Although **string** is a reference type, the equality operators (`==` and `!=`) are defined to compare the values of **string** objects, not references. This makes testing for string equality more intuitive. For example:

```
string a = "hello";
string b = "h";
// Append to contents of 'b'
b += "ello";
Console.WriteLine(a == b);
Console.WriteLine((object)a == (object)b);
```

This displays "True" and then "False" because the content of the strings are equivalent, but `a` and `b` do not refer to the same string instance.

The `+` operator concatenates strings:

```
string a = "good " + "morning";
```

This creates a string object that contains "good morning".

Strings are *immutable*--the contents of a string object cannot be changed after the object is created, although the syntax makes it appear as if you can do this. For example, when you write this code, the compiler actually creates a new string object to hold the new sequence of characters, and the variable `b` continues to hold "h".

```
string b = "h";
b += "ello";
```

The `[]` operator can be used to access individual characters of a **string**:

```
string str = "test";
char x = str[2]; // x = 's';
```

String literals are of type **string** and can be written in two forms, quoted and `@`-quoted. Quoted string literals are enclosed in double quotation marks (`"`):

```
"good morning" // a string literal
```

String literals can contain any character literal, including escape sequences:

```
string a = "\\u0066\n";
```

This string contains a backslash, the letter `f`, and new line.

Note

The escape code `\dddd` (where `dddd` is a four-digit number) represents the Unicode character U+`dddd`. Eight-digit Unicode escape codes are also recognized: `\dddd\dddd`.

`@`-quoted string literals start with `@` and are also enclosed in double quotation marks. For example:

```
@"good morning" // a string literal
```

The advantage of @-quoting is that escape sequences are *not* processed, which makes it easy to write, for example, a fully qualified file name:

```
@"c:\Docs\Source\a.txt" // rather than "c:\\Docs\\Source\\a.txt"
```

To include a double quotation mark in an @-quoted string, double it:

```
@""""Ahoy!"" cried the captain." // "Ahoy!" cried the captain.
```

Another use of the @ symbol is to use referenced ([/reference](#)) identifiers that happen to be C# keywords.

Example

```
// keyword_string.cs
using System;
class TestClass
{
    static void Main()
    {
        string a = "\u0068ello ";
        string b = "world";
        Console.WriteLine( a + b );
        Console.WriteLine( a + b == "hello world" );
    }
}
```

Output

```
hello world
True
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 2.4.2 Identifiers
- 2.4.4.5 String literals
- 4.2.3 The string type
- 7.9.7 String equality operators

See Also

Reference

[C# Keywords](#)

[Reference Types \(C# Reference\)](#)

[Value Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Formatting Numeric Results Table \(C# Reference\)](#)

void (C# Reference)

When used as the return type for a method, **void** specifies that the method does not return a value.

void is not allowed in a method's parameter list. A method that takes no parameters and returns no value is declared as follows:

```
void SampleMethod();
```

void is also used in an unsafe context to declare a pointer to an unknown type. For more information, see [Pointer types \(C# Programming Guide\)](#).

void is an alias for the .NET Framework [System.Void](#) type.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.5 Methods

See Also

Reference

[C# Keywords](#)

[Reference Types \(C# Reference\)](#)

[Value Types \(C# Reference\)](#)

[Methods \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

Types Reference Tables (C# Reference)

The following reference tables summarize the C# types:

[Built-in Types Table](#)

[Integral types](#)

[Floating-point types](#)

[Default values](#)

[Value types](#)

[Implicit numeric conversions](#)

[Explicit Numeric Conversions Table](#)

For information on formatting the output of numeric types, see [Formatting Numeric Results Table](#).

See Also

Reference

[Reference Types \(C# Reference\)](#)

[Value Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Built-In Types Table (C# Reference)

The following table shows the keywords for built-in C# types, which are aliases of predefined types in the [System](#) namespace.

C# Type	.NET Framework Type
bool	System.Boolean
byte	System.Byte
sbyte	System.SByte
char	System.Char
decimal	System.Decimal
double	System.Double
float	System.Single
int	System.Int32
uint	System.UInt32
long	System.Int64
ulong	System.UInt64
object	System.Object
short	System.Int16
ushort	System.UInt16
string	System.String

Remarks

All of the types in the table, except **object** and **string**, are referred to as simple types.

The C# type keywords and their aliases are interchangeable. For example, you can declare an integer variable by using either of the following declarations:

```
int x = 123;
System.Int32 x = 123;
```

To display the actual type for any C# type, use the system method `GetType()`. For example, the following statement displays the system alias that represents the type of `myVariable`:

```
Console.WriteLine(myVariable.GetType());
```

You can also use the [typeof](#) operator.

See Also
Reference

- [C# Keywords](#)
- [Value Types \(C# Reference\)](#)

Concepts[C# Programming Guide](#)**Other Resources**[C# Reference](#)[Default Values Table \(C# Reference\)](#)[Formatting Numeric Results Table \(C# Reference\)](#)[Types Reference Tables \(C# Reference\)](#)

Integral Types Table (C# Reference)

The following table shows the sizes and ranges of the integral types, which constitute a subset of simple types.

Type	Range	Size
sbyte	-128 to 127	Signed 8-bit integer
byte	0 to 255	Unsigned 8-bit integer
char	U+0000 to U+ffff	Unicode 16-bit character
short	-32,768 to 32,767	Signed 16-bit integer
ushort	0 to 65,535	Unsigned 16-bit integer
int	-2,147,483,648 to 2,147,483,647	Signed 32-bit integer
uint	0 to 4,294,967,295	Unsigned 32-bit integer
long	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Signed 64-bit integer
ulong	0 to 18,446,744,073,709,551,615	Unsigned 64-bit integer

If the value represented by an integer literal exceeds the range of **ulong**, a compilation error will occur.

See Also

Reference

[C# Keywords](#)

[Built-In Types Table \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Floating-Point Types Table \(C# Reference\)](#)

[Default Values Table \(C# Reference\)](#)

[Formatting Numeric Results Table \(C# Reference\)](#)

[Types Reference Tables \(C# Reference\)](#)

Floating-Point Types Table (C# Reference)

The following table shows the precision and approximate ranges for the floating-point types.

Type	Approximate range	Precision
float	$\pm 1.5\text{e}-45$ to $\pm 3.4\text{e}38$	7 digits
double	$\pm 5.0\text{e}-324$ to $\pm 1.7\text{e}308$	15-16 digits

See Also

Reference

[Built-In Types Table \(C# Reference\)](#)

[Integral Types Table \(C# Reference\)](#)

[decimal \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Default Values Table \(C# Reference\)](#)

[Formatting Numeric Results Table \(C# Reference\)](#)

[Types Reference Tables \(C# Reference\)](#)

Default Values Table (C# Reference)

The following table shows the default values of value types returned by the default constructors. Default constructors are invoked by using the **new** operator, as follows:

```
int myInt = new int();
```

The preceding statement has the same effect as the following statement:

```
int myInt = 0;
```

Remember that using uninitialized variables in C# is not allowed.

Value type	Default value
bool	false
byte	0
char	'\0'
decimal	0.0M
double	0.0D
enum	The value produced by the expression (E)0, where E is the enum identifier.
float	0.0F
int	0
long	0L
sbyte	0
short	0
struct	The value produced by setting all value-type fields to their default values and all reference-type fields to null .
uint	0
ulong	0
ushort	0

See Also

Reference

- [Value Types \(C# Reference\)](#)
- [Built-In Types Table \(C# Reference\)](#)

Concepts

- [C# Programming Guide](#)

Other Resources

- [C# Reference](#)
- [Types Reference Tables \(C# Reference\)](#)

Value Types Table (C# Reference)

The following table lists the C# value types by category.

Value type	Category
bool	Boolean
byte	Unsigned, numeric, integral
char	Unsigned, numeric, integral
decimal	Numeric, decimal
double	Numeric, floating-point
enum	Enumeration
float	Numeric, floating-point
int	Signed, numeric, integral
long	Signed, numeric, integral
sbyte	Signed, numeric, integral
short	Signed, numeric, integral
struct	User-defined structure
uint	Unsigned, numeric, integral
ulong	Unsigned, numeric, integral
ushort	Unsigned, numeric, integral

See Also

Reference

[Value Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Formatting Numeric Results Table \(C# Reference\)](#)

[Types Reference Tables \(C# Reference\)](#)

Implicit Numeric Conversions Table (C# Reference)

The following table shows the predefined implicit numeric conversions. Implicit conversions might occur in many situations, including method invoking and assignment statements.

From	To
sbyte	short, int, long, float, double, or decimal
byte	short, ushort, int, uint, long, ulong, float, double, or decimal
short	int, long, float, double, or decimal
ushort	int, uint, long, ulong, float, double, or decimal
int	long, float, double, or decimal
uint	long, ulong, float, double, or decimal
long	float, double, or decimal
char	ushort, int, uint, long, ulong, float, double, or decimal
float	double
ulong	float, double, or decimal

Remarks

- The conversions from **int**, **uint**, or **long** to **float** and from **long** to **double** may cause a loss of precision, but not a loss of magnitude.
- There are no implicit conversions to the **char** type.
- There are no implicit conversions between floating-point types and the **decimal** type.
- A constant expression of type **int** can be converted to **sbyte**, **byte**, **short**, **ushort**, **uint**, or **ulong**, provided the value of the constant expression is within the range of the destination type.

C# Language Specification

For more information, see the C# language specification ([C# Language Specification](#)):

- 6.1 Implicit conversions
- 7.15 Constant expressions

See Also

Reference

- [Integral Types Table \(C# Reference\)](#)
- [Built-In Types Table \(C# Reference\)](#)
- [Explicit Numeric Conversions Table \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Explicit Numeric Conversions Table (C# Reference)

Explicit numeric conversion is used to convert any numeric type to any other numeric type, for which there is no implicit conversion, by using a cast expression. The following table shows these conversions.

From	To
sbyte	byte , ushort , uint , ulong , or char
byte	Sbyte or char
short	sbyte , byte , ushort , uint , ulong , or char
ushort	sbyte , byte , short , or char
int	sbyte , byte , short , ushort , uint , ulong , or char
uint	sbyte , byte , short , ushort , int , or char
long	sbyte , byte , short , ushort , int , uint , ulong , or char
ulong	sbyte , byte , short , ushort , int , uint , long , or char
char	sbyte , byte , or short
float	sbyte , byte , short , ushort , int , uint , long , ulong , char , or decimal
double	sbyte , byte , short , ushort , int , uint , long , ulong , char , float , or decimal
decimal	sbyte , byte , short , ushort , int , uint , long , ulong , char , float , or double

Remarks

- The explicit numeric conversion may cause loss of precision or result in throwing exceptions.
- When you convert a **decimal** value to an integral type, this value is rounded towards zero to the nearest integral value. If the resulting integral value is outside the range of the destination type, an [OverflowException](#) is thrown.
- When you convert from a **double** or **float** value to an integral type, the value is truncated. If the resulting integral value is outside the range of the destination value, the result depends on the overflow checking context. In a checked context, an **OverflowException** is thrown, while in an unchecked context, the result is an unspecified value of the destination type.
- When you convert **double** to **float**, the **double** value is rounded to the nearest **float** value. If the **double** value is too small or too large to fit into the destination type, the result will be zero or infinity.
- When you convert **float** or **double** to **decimal**, the source value is converted to **decimal** representation and rounded to the nearest number after the 28th decimal place if required. Depending on the value of the source value, one of the following results may occur:
 - If the source value is too small to be represented as a **decimal**, the result becomes zero.
 - If the source value is NaN (not a number), infinity, or too large to be represented as a **decimal**, an **OverflowException** is thrown.
- When you convert **decimal** to **float** or **double**, the **decimal** value is rounded to the nearest **double** or **float** value.

For more information on explicit conversion, see 6.2 Explicit in the C# Language Specification. For more information on how to access the spec, see [C# Language Specification](#).

See Also

Reference[Integral Types Table \(C# Reference\)](#)[Built-In Types Table \(C# Reference\)](#)[Implicit Numeric Conversions Table \(C# Reference\)](#)**Concepts**[C# Programming Guide](#)**Other Resources**[C# Reference](#)

Formatting Numeric Results Table (C# Reference)

You can format numeric results by using the **String.Format** method, or through the **Console.Write** method, which calls **String.Format**. The format is specified using format strings. The following table contains the supported standard format strings. The format string takes the following form: *Axx*, where *A* is the format specifier and *xx* is the precision specifier. The format specifier controls the type of formatting applied to the numerical value, and the precision specifier controls the number of significant digits or decimal places of the formatted output.

For more information on standard and custom formatting strings, see [Formatting Overview](#). For more information on the **String.Format** method, see [System.String.Format](#).

Character	Description	Examples	Output
C or c	Currency	Console.Write("{0:C}", 2.5); Console.Write("{0:C}", -2.5);	\$2.50 (\$2.50)
D or d	Decimal	Console.Write("{0:D5}", 25);	00025
E or e	Scientific	Console.Write("{0:E}", 250000);	2.500000E+005
F or f	Fixed-point	Console.Write("{0:F2}", 25); Console.Write("{0:F0}", 25);	25.00 25
G or g	General	Console.Write("{0:G}", 2.5);	2.5
N or n	Number	Console.Write("{0:N}", 2500000);	2,500,000.00
X or x	Hexadecimal	Console.Write("{0:X}", 250); Console.Write("{0:X}", 0xffff);	FA FFFF

See Also

Reference

[string \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Types Reference Tables \(C# Reference\)](#)

Modifiers (C# Reference)

Modifiers are used to modify declarations of types and type members. This section introduces the C# modifiers:

Modifier	Purpose
Access Modifiers - public - private - internal - protected	Specify the declared accessibility of types and type members.
abstract	Indicates that a class is intended only to be a base class of other classes.
const	Specify Specifies that the value of the field or the local variable cannot be modified.
event	Declares an event.
extern	Indicates that the method is implemented externally.
new	Hides an inherited member from a base class member.
override	Provides a new implementation of a virtual member inherited from a base class.
partial	Defines partial classes and structs throughout the same assembly.
readonly	Declares a field that can only be assigned values as part of the declaration or in a constructor in the same class.
sealed	Specifies that a class cannot be inherited.
static	Declares a member that belongs to the type itself rather than to a specific object.
unsafe	Declares an unsafe context.
virtual	Declares a method or an accessor whose implementation can be changed by an overriding member in a derived class.
volatile	Indicates that a field can be modified in the program by something such as the operating system, the hardware, or a concurrently executing thread.

See Also
Reference
[C# Keywords](#)
Concepts
[C# Programming Guide](#)
Other Resources
[C# Reference](#)

Access Modifiers (C# Reference)

Access modifiers are keywords used to specify the declared accessibility of a member or a type. This section introduces the four access modifiers:

- [public](#)
- [protected](#)
- [internal](#)
- [private](#)

The following five accessibility levels can be specified using the access modifiers:

public: Access is not restricted.

protected: Access is limited to the containing class or types derived from the containing class.

Internal: Access is limited to the current assembly.

protected internal: Access is limited to the current assembly or types derived from the containing class.

private: Access is limited to the containing type.

This section also introduces the following:

- [Accessibility Levels](#): Using the four access modifiers to declare five levels of accessibility.
- [Accessibility Domain](#): Specifies where, in the program sections, a member can be referenced.
- [Restrictions on Using Accessibility Levels](#): A summary of the restrictions on using declared accessibility levels.

See Also

Reference

[C# Keywords](#)

[Modifiers \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Accessibility Levels (C# Reference)

Use the access modifiers, [public](#), [protected](#), [internal](#), or [private](#) to specify one of the following declared accessibilities for members.

Declared accessibility	Meaning
public	Access is not restricted.
protected	Access is limited to the containing class or types derived from the containing class.
internal	Access is limited to the current assembly.
protected internal	Access is limited to the current assembly or types derived from the containing class.
private	Access is limited to the containing type.

Only one access modifier is allowed for a member or type, except when using the **protected internal** combination.

Access modifiers are not allowed on namespaces. Namespaces have no access restrictions.

Depending on the context in which a member declaration takes place, only certain declared accessibilities are permitted. If no access modifier is specified in a member declaration, a default accessibility is used.

Top-level types, which are not nested into other types, can only have **internal** or **public** accessibility. The default accessibility for these types is **internal**.

Nested types, which are members of other types, can have declared accessibilities as indicated in the following table.

Members of	Default member accessibility	Allowed declared accessibility of the member
enum	public	None
class	private	public protected internal private protected internal
interface	public	None
struct	private	public internal private

The accessibility of a nested type depends on its [accessibility domain](#), which is determined by both the declared accessibility of the member and the accessibility domain of the immediately containing type. However, the accessibility domain of a nested type cannot exceed that of the containing type.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.5.1 Declared Accessibility
- 3.5.3 Protected access for instance members
- 3.5.4 Accessibility constraints

- [10.2.3 Access Modifiers](#)
- [10.2.6.2 Declared Accessibility](#)

See Also

Reference

[C# Keywords](#)

[Access Modifiers \(C# Reference\)](#)

[Accessibility Domain \(C# Reference\)](#)

[Restrictions on Using Accessibility Levels \(C# Reference\)](#)

[Access Modifiers \(C# Programming Guide\)](#)

[public \(C# Reference\)](#)

[private \(C# Reference\)](#)

[protected \(C# Reference\)](#)

[internal \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Accessibility Domain (C# Reference)

The accessibility domain of a member specifies where, in the program sections, a member can be referenced. If the member is nested within another type, its accessibility domain is determined by both the [accessibility level](#) of the member and the accessibility domain of the immediately containing type.

The accessibility domain of a top-level type is at least the program text of the project in which it is declared. That is, the entire source files of this project. The accessibility domain of a nested type is at least the program text of the type in which it is declared. That is, the type body, including any nested types. The accessibility domain of a nested type never exceeds that of the containing type. These concepts are demonstrated in the following example.

Example

This example contains a top-level type, `T1`, and two nested classes, `M1` and `M2`. The classes contain fields with different declared accessibilities. In the `Main` method, a comment follows each statement to indicate the accessibility domain of each member. Notice that the statements that attempt to reference the inaccessible members are commented out. If you want to see the compiler errors caused by referencing an inaccessible member, remove the comments one at a time.

```
// cs_Accessibility_Domain.cs
using System;
namespace AccessibilityDomainNamespace
{
    public class T1
    {
        public static int publicInt;
        internal static int internalInt;
        private static int privateInt = 0; // CS0414

        public class M1
        {
            public static int publicInt;
            internal static int internalInt;
            private static int privateInt = 0; // CS0414
        }

        private class M2
        {
            public static int publicInt = 0;
            internal static int internalInt = 0;
            private static int privateInt = 0; // CS0414
        }
    }

    class MainClass
    {
        static void Main()
        {
            // Access is unlimited:
            T1.publicInt = 1;
            // Accessible only in current assembly:
            T1.internalInt = 2;
            // Error: inaccessible outside T1:
            //     T1.myPrivateInt = 3;

            // Access is unlimited:
            T1.M1.publicInt = 1;
            // Accessible only in current assembly:
            T1.M1.internalInt = 2;
            // Error: inaccessible outside M1:
            //     T1.M1.myPrivateInt = 3;

            // Error: inaccessible outside T1:
            //     T1.M2.myPublicInt = 1;
            // Error: inaccessible outside T1:
            //     T1.M2.myInternalInt = 2;
        }
    }
}
```

```
        // Error: inaccessible outside M2:  
        //     T1.M2.myPrivateInt = 3;  
    }  
}  
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.5.1 Declared Accessibility
- 3.5.2 Accessibility Domains
- 3.5.4 Accessibility constraints
- 10.2.3 Access Modifiers
- 10.2.6.2 Declared Accessibility

See Also

Reference

[C# Keywords](#)

[Access Modifiers \(C# Reference\)](#)

[Accessibility Levels \(C# Reference\)](#)

[Restrictions on Using Accessibility Levels \(C# Reference\)](#)

[Access Modifiers \(C# Programming Guide\)](#)

[public \(C# Reference\)](#)

[private \(C# Reference\)](#)

[protected \(C# Reference\)](#)

[internal \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Restrictions on Using Accessibility Levels (C# Reference)

When you declare a type, it is essential to see if that type has to be at least as accessible as another member or type. For example, the direct base class must be at least as accessible as the derived class. The following declarations will result in a compiler error, because the base class `BaseClass` is less accessible than `MyClass`:

```
class BaseClass {...}
public class MyClass: BaseClass {...} // Error
```

The following table summarizes the restrictions on using declared accessibility levels.

Context	Remarks
Classes	The direct base class of a class type must be at least as accessible as the class type itself.
Interfaces	The explicit base interfaces of an interface type must be at least as accessible as the interface type itself.
Delegates	The return type and parameter types of a delegate type must be at least as accessible as the delegate type itself.
Constants	The type of a constant must be at least as accessible as the constant itself.
Fields	The type of a field must be at least as accessible as the field itself.
Methods	The return type and parameter types of a method must be at least as accessible as the method itself.
Properties	The type of a property must be at least as accessible as the property itself.
Events	The type of an event must be at least as accessible as the event itself.
Indexers	The type and parameter types of an indexer must be at least as accessible as the indexer itself.
Operators	The return type and parameter types of an operator must be at least as accessible as the operator itself.
Constructors	The parameter types of a constructor must be at least as accessible as the constructor itself.

Example

The following example contains erroneous declarations of different types. The comment following each declaration indicates the expected compiler error.

```
// Restrictions_on_Using_Accessibility_Levels.cs
// CS0052 expected as well as CS0053, CS0056, and CS0057
// To make the program work, change access level of both class B
// and MyPrivateMethod() to public.

using System;

// A delegate:
delegate int MyDelegate();

class B
{
    // A private method:
    static int MyPrivateMethod()
    {
        return 0;
    }
}
```

```

public class A
{
    // Error: The type B is less accessible than the field A.myField.
    public B myField = new B();

    // Error: The type B is less accessible
    // than the constant A.myConst.
    public readonly B myConst = new B();

    public B MyMethod()
    {
        // Error: The type B is less accessible
        // than the method A.MyMethod.
        return new B();
    }

    // Error: The type B is less accessible than the property A.MyProp
    public B MyProp
    {
        set
        {
        }
    }

    MyDelegate d = new MyDelegate(B.MyPrivateMethod);
    // Even when B is declared public, you still get the error:
    // "The parameter B.MyPrivateMethod is not accessible due to
    // protection level."

    public static B operator +(A m1, B m2)
    {
        // Error: The type B is less accessible
        // than the operator A.operator +(A,B)
        return new B();
    }

    static void Main()
    {
        Console.Write("Compiled successfully");
    }
}

```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.5.1 Declared Accessibility
- 3.5.4 Accessibility constraints
- 10.2.3 Access Modifiers
- 10.2.6.2 Declared Accessibility
- 10.2.6.5 Access to private and protected members of the containing type

See Also

Reference

[C# Keywords](#)

[Access Modifiers \(C# Reference\)](#)

[Accessibility Domain \(C# Reference\)](#)

[Accessibility Levels \(C# Reference\)](#)

[Access Modifiers \(C# Programming Guide\)](#)

[public \(C# Reference\)](#)

[private \(C# Reference\)](#)

[protected \(C# Reference\)](#)

[internal \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

internal (C# Reference)

The **internal** keyword is an access modifier for types and type members. Internal types or members are accessible only within files in the same assembly, as in this example:

```
public class BaseClass
{
    // Only accessible within the same assembly
    internal static int x = 0;
}
```

For a comparison of **internal** with the other access modifiers, see [Accessibility Levels](#) and [Access Modifiers \(C# Programming Guide\)](#).

For more information on assemblies, see [Assemblies and the Global Assembly Cache \(C# Programming Guide\)](#).

A common use of internal access is in component-based development because it enables a group of components to cooperate in a private manner without being exposed to the rest of the application code. For example, a framework for building graphical user interfaces could provide Control and Form classes that cooperate using members with internal access. Since these members are internal, they are not exposed to code that is using the framework.

It is an error to reference a type or a member with internal access outside the assembly within which it was defined.

Note

An **internal virtual** method can be overridden in some languages, such as textual Microsoft intermediate language (MSIL) using `ilasm.exe`, even though it cannot be overridden using C#.

Example

This example contains two files, `Assembly1.cs` and `Assembly2.cs`. The first file contains an internal base class, `BaseClass`. In the second file, an attempt to instantiate `BaseClass` will produce an error.

```
// Assembly1.cs
// compile with: /target:library
internal class BaseClass
{
    public static int intM = 0;
}
```

```
// Assembly1_a.cs
// compile with: /reference:Assembly1.dll
class TestAccess
{
    static void Main()
    {
        BaseClass myBase = new BaseClass();    // CS0122
    }
}
```

In this example, use the same files you used in example 1, and change the accessibility level of `BaseClass` to **public**. Also change the accessibility level of the member `intM` to **internal**. In this case, you can instantiate the class, but you cannot access the internal member.

```
// Assembly2.cs
// compile with: /target:library
public class BaseClass
{
    internal static int intM = 0;
}
```

```
// Assembly2_a.cs
// compile with: /reference:Assembly1.dll
public class TestAccess
{
    static void Main()
    {
        BaseClass myBase = new BaseClass();    // Ok.
        BaseClass.intM = 444;    // CS0117
    }
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.5.1 Declared Accessibility
- 3.5.4 Accessibility constraints
- 10.2.3 Access Modifiers
- 10.2.6.2 Declared Accessibility

See Also

Reference

[C# Keywords](#)

[Access Modifiers \(C# Reference\)](#)

[Accessibility Levels \(C# Reference\)](#)

[Modifiers \(C# Reference\)](#)

[public \(C# Reference\)](#)

[private \(C# Reference\)](#)

[protected \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

private (C# Reference)

The **private** keyword is a member access modifier. Private access is the least permissive access level. Private members are accessible only within the body of the class or the struct in which they are declared, as in this example:

```
class Employee
{
    private int i;
    double d;    // private access by default
}
```

Nested types in the same body can also access those private members.

It is a compile-time error to reference a private member outside the class or the struct in which it is declared.

For a comparison of **private** with the other access modifiers, see [Accessibility Levels](#) and [Access Modifiers \(C# Programming Guide\)](#).

Example

In this example, the `Employee` class contains two private data members, `name` and `salary`. As private members, they cannot be accessed except by member methods, so public methods called `GetName` and `Salary` are added to allow controlled access to the private members. The `name` member is accessed via public method, and the `salary` member is accessed via a public read-only property. (See [Properties \(C# Programming Guide\)](#) for more information.)

```
// private_keyword.cs
using System;
class Employee
{
    private string name = "FirstName, LastName";
    private double salary = 100.0;

    public string GetName()
    {
        return name;
    }

    public double Salary
    {
        get { return salary; }
    }
}

class MainClass
{
    static void Main()
    {
        Employee e = new Employee();

        // The data members are inaccessible (private), so
        // then can't be accessed like this:
        //     string n = e.name;
        //     double s = e.salary;

        // 'name' is indirectly accessed via method:
        string n = e.GetName();

        // 'salary' is indirectly accessed via property
        double s = e.Salary;
    }
}
```

For more information, see the following sections in the [C# Language Specification](#):

- [3.5.1 Declared Accessibility](#)
- [3.5.4 Accessibility constraints](#)
- [10.2.3 Access Modifiers](#)
- [10.2.6.2 Declared Accessibility](#)
- [10.2.6.5 Access to private and protected members of the containing type](#)

See Also

Reference

[C# Keywords](#)

[Access Modifiers \(C# Reference\)](#)

[Accessibility Levels \(C# Reference\)](#)

[Modifiers \(C# Reference\)](#)

[public \(C# Reference\)](#)

[protected \(C# Reference\)](#)

[internal \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

protected (C# Reference)

The **protected** keyword is a member access modifier. A protected member is accessible within its class and by derived classes. For a comparison of **protected** with the other access modifiers, see [Accessibility Levels](#).

A protected member of a base class is accessible in a derived class only if the access takes place through the derived class type. For example, consider the following code segment:

```
// protected_keyword.cs
using System;
class A
{
    protected int x = 123;
}

class B : A
{
    static void Main()
    {
        A a = new A();
        B b = new B();

        // Error CS1540, because x can only be accessed by
        // classes derived from A.
        // a.x = 10;

        // OK, because this class derives from A.
        b.x = 10;
    }
}
```

The statement `a.x = 10` generates an error because A is not derived from B.

Struct members cannot be protected because the struct cannot be inherited.

Example

In this example, the class `DerivedPoint` is derived from `Point`; therefore, you can access the protected members of the base class directly from the derived class.

```
// protected_keyword_2.cs
using System;
class Point
{
    protected int x;
    protected int y;
}

class DerivedPoint: Point
{
    static void Main()
    {
        DerivedPoint dpoint = new DerivedPoint();

        // Direct access to protected members:
        dpoint.x = 10;
        dpoint.y = 15;
        Console.WriteLine("x = {0}, y = {1}", dpoint.x, dpoint.y);
    }
}
```

Output

x = 10, y = 15

Comments

If you change the access levels of `x` and `y` to [private](#), the compiler will issue the error messages:

```
'Point.y' is inaccessible due to its protection level.
```

```
'Point.x' is inaccessible due to its protection level.
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.5.1 Declared Accessibility
- 3.5.3 Protected access for instance members
- 3.5.4 Accessibility constraints
- 10.2.3 Access Modifiers

See Also

Reference

[C# Keywords](#)

[Access Modifiers \(C# Reference\)](#)

[Accessibility Levels \(C# Reference\)](#)

[Modifiers \(C# Reference\)](#)

[public \(C# Reference\)](#)

[private \(C# Reference\)](#)

[internal \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

public (C# Reference)

The **public** keyword is an access modifier for types and type members. Public access is the most permissive access level. There are no restrictions on accessing public members, as in this example:

```
class SampleClass
{
    public int x; // No access restrictions.
}
```

See [Access Modifiers \(C# Programming Guide\)](#) and [Accessibility Levels](#) for more information.

Example

In the following example, two classes are declared, `Point` and `MainClass`. The public members `x` and `y` of `Point` are accessed directly from `MainClass`.

```
// protected_public.cs
// Public access
using System;
class Point
{
    public int x;
    public int y;
}

class MainClass
{
    static void Main()
    {
        Point p = new Point();
        // Direct access to public members:
        p.x = 10;
        p.y = 15;
        Console.WriteLine("x = {0}, y = {1}", p.x, p.y);
    }
}
```

Output

x = 10, y = 15

If you change the **public** access level to [private](#) or [protected](#), you will get the error message:

'Point.y' is inaccessible due to its protection level.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.5.1 Declared Accessibility
- 3.5.4 Accessibility constraints
- 10.2.3 Access Modifiers
- 10.2.6.2 Declared Accessibility

See Also

Reference

[Access Modifiers \(C# Programming Guide\)](#)

[C# Keywords](#)

[Access Modifiers \(C# Reference\)](#)

[Accessibility Levels \(C# Reference\)](#)

[Modifiers \(C# Reference\)](#)

[private \(C# Reference\)](#)
[protected \(C# Reference\)](#)
[internal \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

abstract (C# Reference)

The **abstract** modifier can be used with classes, methods, properties, indexers, and events. Use the **abstract** modifier in a class declaration to indicate that a class is intended only to be a base class of other classes. Members marked as abstract, or included in an abstract class, must be implemented by classes that derive from the abstract class.

In this example, the class `Square` must provide an implementation of `Area` because it derives from `ShapesClass`:

```
    abstract class ShapesClass
    {
        abstract public int Area();
    }
    class Square : ShapesClass
    {
        int x, y;
        // Not providing an Area method results
        // in a compile-time error.
        public override int Area()
        {
            return x * y;
        }
    }
}
```

For more information about abstract classes, see [Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#).

Remarks

Abstract classes have the following features:

- An abstract class cannot be instantiated.
- An abstract class may contain abstract methods and accessors.
- It is not possible to modify an abstract class with the [sealed \(C# Reference\)](#) modifier, which means that the class cannot be inherited.
- A non-abstract class derived from an abstract class must include actual implementations of all inherited abstract methods and accessors.

Use the **abstract** modifier in a method or property declaration to indicate that the method or property does not contain implementation.

Abstract methods have the following features:

- An abstract method is implicitly a virtual method.
- Abstract method declarations are only permitted in abstract classes.
- Because an abstract method declaration provides no actual implementation, there is no method body; the method declaration simply ends with a semicolon and there are no curly braces (`{ }`) following the signature. For example:

```
public abstract void MyMethod();
```

- The implementation is provided by an overriding method [override \(C# Reference\)](#), which is a member of a non-abstract class.
- It is an error to use the [static](#) or [virtual](#) modifiers in an abstract method declaration.

Abstract properties behave like abstract methods, except for the differences in declaration and invocation syntax.

- It is an error to use the **abstract** modifier on a static property.
- An abstract inherited property can be overridden in a derived class by including a property declaration that uses the [override](#) modifier.

An abstract class must provide implementation for all interface members.

An abstract class that implements an interface might map the interface methods onto abstract methods. For example:

```
interface I
{
    void M();
}
abstract class C: I
{
    public abstract void M();
}
```

Example

In this example, the class `DerivedClass` is derived from an abstract class `BaseClass`. The abstract class contains an abstract method, `AbstractMethod`, and two abstract properties, `X` and `Y`.

```
// abstract_keyword.cs
// Abstract Classes
using System;
abstract class BaseClass    // Abstract class
{
    protected int _x = 100;
    protected int _y = 150;
    public abstract void AbstractMethod();    // Abstract method
    public abstract int X    { get; }
    public abstract int Y    { get; }
}

class DerivedClass : BaseClass
{
    public override void AbstractMethod()
    {
        _x++;
        _y++;
    }

    public override int X    // overriding property
    {
        get
        {
            return _x + 10;
        }
    }

    public override int Y    // overriding property
    {
        get
        {
            return _y + 10;
        }
    }

    static void Main()
    {
        DerivedClass o = new DerivedClass();
        o.AbstractMethod();
        Console.WriteLine("x = {0}, y = {1}", o.X, o.Y);
    }
}
```

Output

x = 111, y = 161

Comments

In the preceding example, if you attempt to instantiate the abstract class by using a statement like this:

```
BaseClass bc = new BaseClass();    // Error
```

you will get an error saying that the compiler cannot create an instance of the abstract class 'BaseClass'.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.5.4 Virtual, override, and abstract methods
- 10.1.1.1 Abstract classes

See Also

Reference

[Modifiers \(C# Reference\)](#)

[virtual \(C# Reference\)](#)

[override \(C# Reference\)](#)

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

const (C# Reference)

The **const** keyword is used to modify a declaration of a field or local variable. It specifies that the value of the field or the local variable is constant, which means it cannot be modified. For example:

```
const int x = 0;
public const double gravitationalConstant = 6.673e-11;
private const string productName = "Visual C#";
```

Remarks

The type of a constant declaration specifies the type of the members introduced by the declaration. A constant expression must yield a value of the target type, or of a type that can be implicitly converted to the target type.

A constant expression is an expression that can be fully evaluated at compile time. Therefore, the only possible values for constants of reference types are **string** and **null**.

The constant declaration can declare multiple constants, such as:

```
public const double x = 1.0, y = 2.0, z = 3.0;
```

The **static** modifier is not allowed in a constant declaration.

A constant can participate in a constant expression, as follows:

```
public const int c1 = 5;
public const int c2 = c1 + 100;
```

Note

The **readonly** keyword is different from the **const** keyword. A **const** field can only be initialized at the declaration of the field. A **readonly** field can be initialized either at the declaration or in a constructor. Therefore, **readonly** fields can have different values depending on the constructor used. Also, while a **const** field is a compile-time constant, the **readonly** field can be used for run-time constants, as in this line: `public static readonly uint l1 = (uint)DateTime.Now.Ticks;`

Example

```
// const_keyword.cs
using System;
public class ConstTest
{
    class SampleClass
    {
        public int x;
        public int y;
        public const int c1 = 5;
        public const int c2 = c1 + 5;

        public SampleClass(int p1, int p2)
        {
            x = p1;
            y = p2;
        }
    }

    static void Main()
    {
        SampleClass mC = new SampleClass(11, 22);
        Console.WriteLine("x = {0}, y = {1}", mC.x, mC.y);
        Console.WriteLine("c1 = {0}, c2 = {1}",
                           SampleClass.c1, SampleClass.c2 );
    }
}
```

```
}
```

Output

```
x = 11, y = 22  
c1 = 5, c2 = 10
```

This example demonstrates using constants as local variables.

```
// const_keyword_2.cs  
using System;  
public class MainClass  
{  
    static void Main()  
    {  
        const int c = 707;  
        Console.WriteLine("My local constant = {0}", c);  
    }  
}
```

Output

```
My local constant = 707
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 6.1.6 Implicit constant expression conversions
- 8.5.2 Local constant declarations

See Also

Reference

[C# Keywords](#)

[Modifiers \(C# Reference\)](#)

[readonly \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

event (C# Reference)

The **event** keyword is used to declare an event in a publisher class.

Remarks

The following example shows how to declare and raise an event that uses [EventHandler](#) as the underlying delegate type. For the complete code example that also shows how to use the generic [EventHandler<T>](#) delegate type and how to subscribe to an event and create an event handler method, see

[How to: Publish Events that Conform to .NET Framework Guidelines \(C# Programming Guide\)](#).

```
public class Publisher
{
    // Declare the delegate (if using non-generic pattern).
    public delegate void SampleEventHandler(object sender, SampleEventArgs e);

    // Declare the event.
    public event SampleEventHandler SampleEvent;

    // Wrap the event in a protected virtual method
    // to enable derived classes to raise the event.
    protected virtual void RaiseSampleEvent()
    {
        // Raise the event by using the () operator.
        SampleEvent(this, new SampleEventArgs("Hello"));
    }
}
```

Events are a special kind of multicast delegate that can only be invoked from within the class or struct where they are declared (the publisher class). If other classes or structs subscribe to the event, their event handler methods will be called when the publisher class raises the event. For more information and code examples, see [Events \(C# Programming Guide\)](#) and [Delegates \(C# Programming Guide\)](#).

Events can be marked as [public](#), [private](#), [protected](#), [internal](#), or **protected internal**. These access modifiers define how users of the class can access the event. For more information, see [Access Modifiers](#).

Keywords and Events

The following keywords apply to events.

Keyword	Description	For more information
static	Makes the event available to callers at any time, even if no instance of the class exists.	Static Classes and Static Class Members
virtual	Allows derived classes to override the event behavior by using the override keyword.	Inheritance (C# Programming Guide)
sealed	Specifies that for derived classes it is no longer virtual.	
abstract	The compiler will not generate the add and remove event accessor blocks and therefore derived classes must provide their own implementation.	

An event may be declared as a static event by using the [static](#) keyword. This makes the event available to callers at any time, even if no instance of the class exists. For more information, see [Static Classes and Static Class Members](#).

An event can be marked as a virtual event by using the [virtual](#) keyword. This allows derived classes to override the event behavior by using the [override](#) keyword. For more information, see [Inheritance \(C# Programming Guide\)](#). An event overriding a virtual event can also be [sealed](#), specifying that for derived classes it is no longer virtual. Lastly, an event can be declared [abstract](#), meaning the compiler will not generate the **add** and **remove** event accessor blocks and therefore derived classes must provide their own implementation.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.6.6.4 Events](#)
- [7.13.3 Event Assignment](#)
- [10.7 Events](#)
- [13.2.3 Interface Events](#)

See Also

Tasks

[How to: Combine Delegates \(Multicast Delegates\)\(C# Programming Guide\)](#)

Reference

[C# Keywords](#)

[Modifiers \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

extern (C# Reference)

The **extern** modifier is used to declare a method that is implemented externally. A common use of the **extern** modifier is with the **DllImport** attribute when using Interop services to call into unmanaged code; in this case, the method must also be declared as **static**, as shown in the following example:

```
[DllImport("avifil32.dll")]
private static extern void AVIFileInit();
```

Note

The **extern** keyword also can define an external assembly alias, making it possible to reference different versions of the same component from within a single assembly. For more information, see [extern alias \(C# Reference\)](#).

It is an error to use the [abstract \(C# Reference\)](#) and **extern** modifiers together to modify the same member. Using the **extern** modifier means that the method is implemented outside the C# code, while using the **abstract** modifier means that the method implementation is not provided in the class.

Note

The **extern** keyword is more limited in use than in C++. To compare with the C++ keyword, see [Using extern to Specify Linkage](#) in the C++ Language Reference.

Example

In this example, the program receives a string from the user and displays it inside a message box. The program uses the `MessageBox` method imported from the `User32.dll` library.

```
using System;
using System.Runtime.InteropServices;
class MainClass
{
    [DllImport("User32.dll")]
    public static extern int MessageBox(int h, string m, string c, int type);

    static int Main()
    {
        string myString;
        Console.WriteLine("Enter your message: ");
        myString = Console.ReadLine();
        return MessageBox(0, myString, "My Message Box", 0);
    }
}
```

This example creates a DLL from a C program that is invoked from within the C# program in the next example.

```
// cmdll.c
// compile with: /LD
int __declspec(dllexport) SampleMethod(int i)
{
    return i*10;
}
```

This example uses two files, `CM.cs` and `Cmdll.c`, to demonstrate **extern**. The C file is the external DLL created in Example 2 that is invoked from within the C# program.

```
// cm.cs
using System;
using System.Runtime.InteropServices;
public class MainClass
```

```
{
    [DllImport("Cmdll.dll")]
    public static extern int SampleMethod(int x);

    static void Main()
    {
        Console.WriteLine("SampleMethod() returns {0}.", SampleMethod(5));
    }
}
```

Output

SampleMethod() returns 50.

Remarks

To build the project:

- Compile `Cmdll.c` to a DLL using the Visual C++ command line:

```
cl /LD Cmdll.c
```

- Compile `CM.cs` using the command line:

```
csc CM.cs
```

This will create the executable file `CM.exe`. When you run this program, `SampleMethod` will pass the value 5 to the DLL file, which returns the value multiplied by 10.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 10.5.7 External methods

See Also

Reference

[C# Keywords](#)

[Modifiers \(C# Reference\)](#)

[System.Runtime.InteropServices.DllImportAttribute](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

override (C# Reference)

The **override** modifier is required to extend or modify the abstract or virtual implementation of an inherited method, property, indexer, or event.

In this example, the class `Square` must provide an overridden implementation of `Area` because `Area` is inherited from the abstract `ShapesClass`:

```
abstract class ShapesClass
{
    abstract public int Area();
}

class Square : ShapesClass
{
    int x, y;
    // Because ShapesClass.Area is abstract, failing to override
    // the Area method would result in a compilation error.
    public override int Area()
    {
        return x * y;
    }
}
```

For more information on using the **override** keyword, see [Versioning with the Override and New Keywords](#) and [Knowing when to use Override and New Keywords](#).

Remarks

An **override** method provides a new implementation of a member inherited from a base class. The method overridden by an **override** declaration is known as the overridden base method. The overridden base method must have the same signature as the **override** method. For information on inheritance, see [Inheritance](#).

You cannot override a non-virtual or static method. The overridden base method must be **virtual**, **abstract**, or **override**.

An **override** declaration cannot change the accessibility of the **virtual** method. Both the **override** method and the **virtual** method must have the same [access level modifier](#).

You cannot use the modifiers **new**, **static**, **virtual**, or **abstract** to modify an **override** method.

An overriding property declaration must specify the exact same access modifier, type, and name as the inherited property, and the overridden property must be **virtual**, **abstract**, or **override**.

Example

This example defines a base class called `Employee`, and a derived class called `SalesEmployee`. The `SalesEmployee` class includes an extra property, `salesbonus`, and overrides the method `CalculatePay` in order to take it into account

```
using System;
class TestOverride
{
    public class Employee
    {
        public string name;

        // Basepay is defined as protected, so that it may be
        // accessed only by this class and derrived classes.
        protected decimal basepay;

        // Constructor to set the name and basepay values.
        public Employee(string name, decimal basepay)
        {
            this.name = name;
            this.basepay = basepay;
        }
    }
}
```

```

    // Declared virtual so it can be overridden.
    public virtual decimal CalculatePay()
    {
        return basepay;
    }
}

// Derive a new class from Employee.
public class SalesEmployee : Employee
{
    // New field that will affect the base pay.
    private decimal salesbonus;

    // The constructor calls the base-class version, and
    // initializes the salesbonus field.
    public SalesEmployee(string name, decimal basepay,
        decimal salesbonus) : base(name, basepay)
    {
        this.salesbonus = salesbonus;
    }

    // Override the CalculatePay method
    // to take bonus into account.
    public override decimal CalculatePay()
    {
        return basepay + salesbonus;
    }
}

static void Main()
{
    // Create some new employees.
    SalesEmployee employee1 = new SalesEmployee("Alice",
        1000, 500);
    Employee employee2 = new Employee("Bob", 1200);

    Console.WriteLine("Employee " + employee1.name +
        " earned: " + employee1.CalculatePay());
    Console.WriteLine("Employee " + employee2.name +
        " earned: " + employee2.CalculatePay());
}
}

```

Output

Employee Alice earned: 1500
Employee Bob earned: 1200

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.5.4 Virtual, override, and abstract methods
- 10.5.4 Override methods

See Also

Reference

[Inheritance \(C# Programming Guide\)](#)

[C# Keywords](#)

[Modifiers \(C# Reference\)](#)

[abstract \(C# Reference\)](#)

[virtual \(C# Reference\)](#)

[new \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Polymorphism \(C# Programming Guide\)](#)

Other Resources

readonly (C# Reference)

The **readonly** keyword is a modifier that you can use on fields. When a field declaration includes a **readonly** modifier, assignments to the fields introduced by the declaration can only occur as part of the declaration or in a constructor in the same class. In this example, the value of the field `year` cannot be changed in the method `ChangeYear`, even though it is assigned a value in the class constructor:

```
class Age
{
    readonly int _year;
    Age(int year)
    {
        _year = year;
    }
    void ChangeYear()
    {
        _year = 1967; // Will not compile.
    }
}
```

Remarks

You can assign a value to a **readonly** field only in the following contexts:

- When the variable is initialized in the declaration, for example:

```
public readonly int y = 5;
```

- For an instance field, in the instance constructors of the class that contains the field declaration, or for a static field, in the static constructor of the class that contains the field declaration. These are also the only contexts in which it is valid to pass a `readonly` field as an `out` or `ref` parameter.

Note

The **readonly** keyword is different from the `const` keyword. A **const** field can only be initialized at the declaration of the field. A **readonly** field can be initialized either at the declaration or in a constructor. Therefore, **readonly** fields can have different values depending on the constructor used. Also, while a **const** field is a compile-time constant, the **readonly** field can be used for runtime constants as in the following example:

Note

```
public static readonly uint l1 = (uint)DateTime.Now.Ticks;
```

Example

```
// cs_readonly_keyword.cs
// Readonly fields
using System;
public class ReadOnlyTest
{
    class SampleClass
    {
        public int x;
        // Initialize a readonly field
        public readonly int y = 25;
        public readonly int z;

        public SampleClass()
        {
            // Initialize a readonly instance field
            z = 24;
        }
    }
}
```

```

    public SampleClass(int p1, int p2, int p3)
    {
        x = p1;
        y = p2;
        z = p3;
    }
}

static void Main()
{
    SampleClass p1 = new SampleClass(11, 21, 32);    // OK
    Console.WriteLine("p1: x={0}, y={1}, z={2}", p1.x, p1.y, p1.z);
    SampleClass p2 = new SampleClass();
    p2.x = 55;    // OK
    Console.WriteLine("p2: x={0}, y={1}, z={2}", p2.x, p2.y, p2.z);
}
}

```

Output

p1: x=11, y=21, z=32
p2: x=55, y=25, z=24

Comments

In the preceding example, if you use a statement like this:

```
p2.y = 66; // Error
```

you will get the compiler error message:

The left-hand side of an assignment must be an l-value

which is the same error you get when you attempt to assign a value to a constant.

C# Language Specification

For more information, see the following section in the [C# Language Specification](#):

- 10.4.2 Readonly fields

See Also

Reference

[C# Keywords](#)

[Modifiers \(C# Reference\)](#)

[const \(C# Reference\)](#)

[Fields \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

sealed (C# Reference)

The **sealed** modifier can be applied to classes, instance methods and properties. A sealed class cannot be inherited. A sealed method overrides a method in a base class, but itself cannot be overridden further in any derived class. When applied to a method or property, the **sealed** modifier must always be used with [override \(C# Reference\)](#).

Use the **sealed** modifier in a class declaration to prevent inheritance of the class, as in this example:

```
sealed class SealedClass
{
    public int x;
    public int y;
}
```

It is an error to use a sealed class as a base class or to use the [abstract](#) modifier with a sealed class.

Structs are implicitly sealed; therefore, they cannot be inherited.

For more information about inheritance, see [Inheritance \(C# Programming Guide\)](#).

Example

```
// cs_sealed_keyword.cs
using System;
sealed class SealedClass
{
    public int x;
    public int y;
}

class MainClass
{
    static void Main()
    {
        SealedClass sc = new SealedClass();
        sc.x = 110;
        sc.y = 150;
        Console.WriteLine("x = {0}, y = {1}", sc.x, sc.y);
    }
}
```

Output

x = 110, y = 150

In the preceding example, if you attempt to inherit from the sealed class by using a statement like this:

```
class MyDerivedC: SealedClass {} // Error
```

you will get the error message:

```
'MyDerivedC' cannot inherit from sealed class 'SealedClass'.
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 10.1.1.2 Sealed classes
- 10.5.5 Sealed methods

See Also

Reference

[C# Keywords](#)

[Static Classes and Static Class Members \(C# Programming Guide\)](#)

[Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#)

[Access Modifiers \(C# Programming Guide\)](#)

[Modifiers \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

static (C# Reference)

Use the **static** modifier to declare a static member, which belongs to the type itself rather than to a specific object. The **static** modifier can be used with classes, fields, methods, properties, operators, events and constructors, but cannot be used with indexers, destructors, or types other than classes. For example, the following class is declared as **static**, and contains only **static** methods:

```
static class CompanyEmployee
{
    public static string GetCompanyName(string name) { ... }
    public static string GetCompanyAddress(string address) { ... }
}
```

For more information, see [Static Classes and Static Class Members \(C# Programming Guide\)](#).

Remarks

- A constant or type declaration is implicitly a static member.
- A static member cannot be referenced through an instance. Instead, it is referenced through the type name. For example, consider the following class:

```
public class MyBaseC
{
    public struct MyStruct
    {
        public static int x = 100;
    }
}
```

To refer to the static member *x*, use the fully qualified name (unless it is accessible from the same scope):

```
MyBaseC.MyStruct.x
```

- While an instance of a class contains a separate copy of all instance fields of the class, there is only one copy of each static field.
- It is not possible to use [this](#) to reference static methods or property accessors.
- If the **static** keyword is applied to a class, all the members of the class must be static.
- Classes, including static classes, may have static constructors. Static constructors are called at some point between when the program begins and the class is instantiated.

Note

The **static** keyword is more limited in use than in C++. To compare with the C++ keyword, see [static](#).

To demonstrate static members, consider a class that represents a company employee. Assume that the class contains a method to count employees and a field to store the number of employees. Both the method and the field do not belong to any instance employee. Instead they belong to the company class. Therefore, they should be declared as static members of the class.

Example

This example reads the name and ID of a new employee, increments the employee counter by one, and displays the information for the new employee as well as the new number of employees. For simplicity, this program reads the current number of employees from the keyboard. In a real application, this information should be read from a file.

```
// cs_static_keyword.cs
using System;

public class Employee
{
    public string id;
    public string name;

    public Employee()
    {
    }

    public Employee(string name, string id)
    {
        this.name = name;
        this.id = id;
    }

    public static int employeeCounter;

    public static int AddEmployee()
    {
        return ++employeeCounter;
    }
}

class MainClass : Employee
{
    static void Main()
    {
        Console.Write("Enter the employee's name: ");
        string name = Console.ReadLine();
        Console.Write("Enter the employee's ID: ");
        string id = Console.ReadLine();

        // Create and configure the employee object:
        Employee e = new Employee(name, id);
        Console.Write("Enter the current number of employees: ");
        string n = Console.ReadLine();
        Employee.employeeCounter = Int32.Parse(n);
        Employee.AddEmployee();

        // Display the new information:
        Console.WriteLine("Name: {0}", e.name);
        Console.WriteLine("ID: {0}", e.id);
        Console.WriteLine("New Number of Employees: {0}",
            Employee.employeeCounter);
    }
}
```

Input

Tara Strahan
AF643G
15

Sample Output

Enter the employee's name: Tara Strahan
Enter the employee's ID: AF643G
Enter the current number of employees: 15
Name: Tara Strahan
ID: AF643G
New Number of Employees: 16

This example shows that while it is possible to initialize a static field with another static field not yet declared, the results will be undefined, until you explicitly assign a value to the static field.

```
// cs_static_keyword_2.cs
using System;
class Test
{
    static int x = y;
    static int y = 5;

    static void Main()
    {
        Console.WriteLine(Test.x);
        Console.WriteLine(Test.y);

        Test.x = 99;
        Console.WriteLine(Test.x);
    }
}
```

Output

0
5
99

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.5.3 Static and instance methods
- 5.1.1 Static variables
- 10.2.5 Static and instance members
- 10.4.1 Static and instance fields
- 10.4.5.1 Static field initialization
- 10.5.2 Static and instance methods
- 10.6.1 Static and instance properties
- 10.7.3 Static and instance events
- 10.11 Static constructors

See Also

Reference

[C# Keywords](#)

[Modifiers \(C# Reference\)](#)

[Static Classes and Static Class Members \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

unsafe (C# Reference)

The **unsafe** keyword denotes an unsafe context, which is required for any operation involving pointers. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

You can use the **unsafe** modifier in the declaration of a type or a member. The entire textual extent of the type or member is therefore considered an unsafe context. For example, the following is a method declared with the **unsafe** modifier:

```
unsafe static void FastCopy(byte[] src, byte[] dst, int count)
{
    // Unsafe context: can use pointers here.
}
```

The scope of the unsafe context extends from the parameter list to the end of the method, so pointers can also be used in the parameter list:

```
unsafe static void FastCopy ( byte* ps, byte* pd, int count ) {...}
```

You can also use an unsafe block to enable the use of an unsafe code inside this block. For example:

```
unsafe
{
    // Unsafe context: can use pointers here.
}
```

To compile unsafe code, you must specify the [/unsafe](#) compiler option. Unsafe code is not verifiable by the common language runtime.

Example

```
// cs_unsafe_keyword.cs
// compile with: /unsafe
using System;
class UnsafeTest
{
    // Unsafe method: takes pointer to int:
    unsafe static void SquarePtrParam(int* p)
    {
        *p *= *p;
    }

    unsafe static void Main()
    {
        int i = 5;
        // Unsafe method: uses address-of operator (&):
        SquarePtrParam(&i);
        Console.WriteLine(i);
    }
}
```

Output

```
25
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 18 Unsafe code

See Also

Reference

[C# Keywords](#)

[fixed Statement \(C# Reference\)](#)

[Fixed Size Buffers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

virtual (C# Reference)

The **virtual** keyword is used to modify a method, property, indexer or event declaration, and allow it to be overridden in a derived class. For example, this method can be overridden by any class that inherits it:

```
public virtual double Area()
{
    return x * y;
}
```

The implementation of a virtual member can be changed by an [overriding member](#) in a derived class. For more information on using the **virtual** keyword, see [Versioning with the Override and New Keywords \(C# Programming Guide\)](#) and [Knowing When to Use Override and New Keywords \(C# Programming Guide\)](#).

Remarks

When a virtual method is invoked, the run-time type of the object is checked for an overriding member. The overriding member in the most derived class is called, which might be the original member, if no derived class has overridden the member.

By default, methods are non-virtual. You cannot override a non-virtual method.

You cannot use the **virtual** modifier with the **static**, **abstract**, **private** or **override** modifiers.

Virtual properties behave like abstract methods, except for the differences in declaration and invocation syntax.

- It is an error to use the **virtual** modifier on a static property.
- A virtual inherited property can be overridden in a derived class by including a property declaration that uses the **override** modifier.

Example

In this example, the class `Dimensions` contains the two coordinates `x`, `y`, and the `Area()` virtual method. Different shape classes such as `Circle`, `Cylinder`, and `Sphere` inherit the `Dimensions` class, and the surface area is calculated for each figure. Each derived class has its own override implementation of `Area()`. The program calculates and displays the proper area for each figure by invoking the proper implementation of `Area()` according to the object associated with the method.

Notice that the inherited classes `Circle`, `Sphere`, and `Cylinder` are all using constructors that initialize the base class, for example:

```
public Cylinder(double r, double h): base(r, h) {}
```

This is analogous to the C++ initialization list.

```
// cs_virtual_keyword.cs
using System;
class TestClass
{
    public class Dimensions
    {
        public const double PI = Math.PI;
        protected double x, y;
        public Dimensions()
        {
        }
        public Dimensions(double x, double y)
        {
            this.x = x;
            this.y = y;
        }

        public virtual double Area()
        {
        }
    }
}
```

```

        return x * y;
    }
}

public class Circle : Dimensions
{
    public Circle(double r) : base(r, 0)
    {
    }

    public override double Area()
    {
        return PI * x * x;
    }
}

class Sphere : Dimensions
{
    public Sphere(double r) : base(r, 0)
    {
    }

    public override double Area()
    {
        return 4 * PI * x * x;
    }
}

class Cylinder : Dimensions
{
    public Cylinder(double r, double h) : base(r, h)
    {
    }

    public override double Area()
    {
        return 2 * PI * x * x + 2 * PI * x * y;
    }
}

static void Main()
{
    double r = 3.0, h = 5.0;
    Dimensions c = new Circle(r);
    Dimensions s = new Sphere(r);
    Dimensions l = new Cylinder(r, h);
    // Display results:
    Console.WriteLine("Area of Circle    = {0:F2}", c.Area());
    Console.WriteLine("Area of Sphere    = {0:F2}", s.Area());
    Console.WriteLine("Area of Cylinder = {0:F2}", l.Area());
}
}

```

Output

Area of Circle	= 28.27
Area of Sphere	= 113.10
Area of Cylinder	= 150.80

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.5.4 Virtual, override and abstract methods
- 10.5.3 Virtual methods
- 10.6.3 Virtual, sealed, override, and abstract accessors

See Also

Reference

[Modifiers \(C# Reference\)](#)

[C# Keywords](#)

[abstract \(C# Reference\)](#)

[override \(C# Reference\)](#)

[new \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Polymorphism \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

volatile (C# Reference)

The **volatile** keyword indicates that a field might be modified by multiple concurrently executing threads. Fields that are declared **volatile** are not subject to compiler optimizations that assume access by a single thread. This ensures that the most up-to-date value is present in the field at all times.

The **volatile** modifier is usually used for a field that is accessed by multiple threads without using the [lock Statement \(C# Reference\)](#) statement to serialize access. See [How to: Create and Terminate Threads \(C# Programming Guide\)](#) for an example of **volatile** in a multi-threaded scenario.

The **volatile** keyword can be applied to fields of these types:

- Reference types.
- Pointer types (in an unsafe context). Note that while the pointer itself can be volatile, the object that it points to cannot. In other words, you cannot declare a "pointer to volatile."
- Integral types such as sbyte, byte, short, ushort, int, uint, char, float, and bool.
- An enum type with an integral base type.
- Generic type parameters known to be reference types.
- [IntPtr](#) and [UIntPtr](#).

The volatile keyword can only be applied to fields of a class or struct. Local variables cannot be declared **volatile**.

Example

The following example shows how to declare a public field variable as **volatile**.

```
// csharp_volatile.cs
// compile with: /target:library
class Test
{
    public volatile int i;

    Test(int _i)
    {
        i = _i;
    }
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.10 Execution order
- 10.4.3 Volatile fields

See Also

Reference

[C# Keywords](#)

[Modifiers \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Statement Types (C# Reference)

Statements are program instructions. Except as described, statements are executed in sequence. C# has the following categories of statements.

Category	C# keywords
Selection statements	if , else , switch , case
Iteration statements	do , for , foreach , in , while
Jump statements	break , continue , default , goto , return , yield
Exception handling statements	throw , try-catch , try-finally , try-catch-finally
Checked and unchecked	checked , unchecked
fixed Statement	fixed
lock Statement	lock

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Selection Statements (C# Reference)

A selection statement causes the program control to be transferred to a specific flow based upon whether a certain condition is **true** or not.

The following keywords are used in selection statements:

- [if](#)
- [else](#)
- [switch](#)
- [case](#)
- [default](#)

See Also

Reference

[C# Keywords](#)

[Statement Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

if-else (C# Reference)

The **if** statement selects a statement for execution based on the value of a **Boolean** expression. In the following example a **Boolean** flag `flagCheck` is set to **true** and then checked in the **if** statement. The output is: The flag is set to true.

```
bool flagCheck = true;
if (flagCheck == true)
{
    Console.WriteLine("The flag is set to true.");
}
else
{
    Console.WriteLine("The flag is set to false.");
}
```

Remarks

If the expression in the parenthesis is evaluated to be true, then the `Console.WriteLine("The boolean flag is set to true.");` statement is executed. After executing the **if** statement, control is transferred to the next statement. The **else** is not executed in this example.

If you wish to execute more than one statement, multiple statements can be conditionally executed by including them into blocks using `{}` as in the example above.

The statement(s) to be executed upon testing the condition can be of any kind, including another **if** statement nested into the original **if** statement. In nested **if** statements, the **else** clause belongs to the last **if** that does not have a corresponding **else**. For example:

```
        if (x > 10)
    if (y > 20)
        Console.Write("Statement_1");
    else
        Console.Write("Statement_2");
```

In this example, `Statement_2` will be displayed if the condition `(y > 20)` evaluates to **false**. However, if you want to associate `Statement_2` with the condition `(x > 10)`, use braces:

```
        if (x > 10)
    {
        if (y > 20)
            Console.Write("Statement_1");
        }
    else
        Console.Write("Statement_2");
```

In this case, `Statement_2` will be displayed if the condition `(x > 10)` evaluates to **false**

Example 1

In this example, you enter a character from the keyboard and the program checks if the input character is an alphabetic character. If so, it checks if it is lowercase or uppercase. In each case, the proper message is displayed.

```
// statements_if_else.cs
// if-else example
using System;
class IfTest
{
    static void Main()
    {
        Console.Write("Enter a character: ");
        char c = (char)Console.Read();
```

```

        if (Char.IsLetter(c))
        {
            if (Char.IsLower(c))
            {
                Console.WriteLine("The character is lowercase.");
            }
            else
            {
                Console.WriteLine("The character is uppercase.");
            }
        }
        else
        {
            Console.WriteLine("Not an alphabetic character.");
        }
    }
}

```

Input

Sample Output

```

Enter a character: 2
The character is not an alphabetic character.

```

Additional sample might look as follows:

Run #2:

```

Enter a character: A
The character is uppercase.

```

Run #3:

```

Enter a character: h
The character is lowercase.

```

It is also possible to extend the **if** statement to handle multiple conditions using the following else-if arrangement:

```

        if (Condition_1)
        {
            // Statement_1;
        }
        else if (Condition_2)
        {
            // Statement_2;
        }
        else if (Condition_3)
        {
            // Statement_3;
        }
        else
        {
            // Statement_n;
        }
    }
}

```

Example 2

This example checks if the input character is lowercase, uppercase, or a number. Otherwise, it is not an alphanumeric character. The program makes use of the else-if ladder.

```

// statements_if_else2.cs

```

```
// else-if
using System;
public class IfTest
{
    static void Main()
    {
        Console.Write("Enter a character: ");
        char c = (char)Console.Read();

        if (Char.IsUpper(c))
        {
            Console.WriteLine("Character is uppercase.");
        }
        else if (Char.IsLower(c))
        {
            Console.WriteLine("Character is lowercase.");
        }
        else if (Char.IsDigit(c))
        {
            Console.WriteLine("Character is a number.");
        }
        else
        {
            Console.WriteLine("Character is not alphanumeric.");
        }
    }
}
```

Input

Sample Output

```
Enter a character: E
The character is uppercase.
```

Additional sample runs might look as follows:

Run #2

```
Enter a character: e
The character is lowercase.
```

Run #3:

```
Enter a character: 4
The character is a number.
```

Run #4:

```
Enter a character: $
The character is not alphanumeric.
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.5 If statements
- 8.7.1 The if statement

See Also

Reference

[C# Keywords](#)

[?: Operator \(C# Reference\)](#)

[The if-else Statement](#)

[switch \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

switch (C# Reference)

The **switch** statement is a control statement that handles multiple selections and enumerations by passing control to one of the **case** statements within its body as the following example:

```
int caseSwitch = 1;
switch (caseSwitch)
{
    case 1:
        Console.WriteLine("Case 1");
        break;
    case 2:
        Console.WriteLine("Case 2");
        break;
    default:
        Console.WriteLine("Default case");
        break;
}
```

Remarks

Control is transferred to the **case** statement which matches the value of the switch. The **switch** statement can include any number of **case** instances, but no two case statements can have the same value. Execution of the statement body begins at the selected statement and proceeds until the **break** statement transfers control out of the **case** body. A jump statement such as a **break** is required after each **case** block, including the last block whether it is a **case** statement or a **default** statement. With one exception, (unlike the C++ **switch** statement), C# does not support an implicit fall through from one case label to another. The one exception is if a **case** statement has no code.

If no case expression matches the switch value, then control is transferred to the statement(s) that follow the optional **default** label. If there is no **default** label, control is transferred outside the **switch**.

Example

```
// statements_switch.cs
using System;
class SwitchTest
{
    static void Main()
    {
        Console.WriteLine("Coffee sizes: 1=Small 2=Medium 3=Large");
        Console.Write("Please enter your selection: ");
        string s = Console.ReadLine();
        int n = int.Parse(s);
        int cost = 0;
        switch(n)
        {
            case 1:
                cost += 25;
                break;
            case 2:
                cost += 25;
                goto case 1;
            case 3:
                cost += 50;
                goto case 1;
            default:
                Console.WriteLine("Invalid selection. Please select 1, 2, or 3.");
                break;
        }
        if (cost != 0)
        {
            Console.WriteLine("Please insert {0} cents.", cost);
        }
        Console.WriteLine("Thank you for your business.");
    }
}
```

```
}
```

Input

```
2
```

Sample Output

```
Coffee sizes: 1=Small 2=Medium 3=Large
Please enter your selection: 2
Please insert 50 cents.
Thank you for your business.
```

The following sample shows that fall through from one case label to another is allowed for empty case labels.

```
// statements_switch2.cs
using System;
class SwitchTest
{
    static void Main()
    {
        int n = 2;
        switch(n)
        {
            case 1:
            case 2:
            case 3:
                Console.WriteLine("It's 1, 2, or 3.");
                break;
            default:
                Console.WriteLine("Not sure what it is.");
                break;
        }
    }
}
```

Output

```
It's 1, 2, or 3.
```

Code Discussion

- In the preceding example, an integral type variable, *n*, was used for the switch cases. Notice that you can also use the string variable, *s*, directly. In this case, you can use switch cases like these:

```
switch(s)
{
    case "1":
        // ...
    case "2":
        // ...
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.6 Switch statement
- 8.7.2 The switch statement

See Also

Reference

[C# Keywords](#)

[The switch Statement](#)

[if-else \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Iteration Statements (C# Reference)

You can create loops by using the iteration statements. Iteration statements cause embedded statements to be executed a number of times, subject to the loop-termination criteria. These statements are executed in order, except when a [jump statement](#) is encountered.

The following keywords are used in iteration statements:

- [do](#)
- [for](#)
- [foreach](#)
- [in](#)
- [while](#)

See Also

Reference

[C# Keywords](#)

[Statement Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

do (C# Reference)

The **do** statement executes a statement or a block of statements enclosed in **{ }** repeatedly until a specified expression evaluates to **false**. In the following example the **do-while** loop statements execute as long as the variable *y* is less than 5.

Example

```
// statements_do.cs
using System;
public class TestDowWhile
{
    public static void Main ()
    {
        int x = 0;
        do
        {
            Console.WriteLine(x);
            x++;
        } while (x < 5);
    }
}
```

Output

0
1
2
3
4

Remarks

Unlike the [while](#) statement, a **do-while** loop is executed once before the conditional expression is evaluated.

At any point within the **do-while** block, you can break out of the loop using the [break](#) statement. You can step directly to the **while** expression evaluation statement by using the [continue](#) statement; if the expression evaluates to true, execution continues at the first statement in the loop. If the expression evaluates to false, execution continues at the first statement after the **do-while** loop.

A **do-while** loop can also be exited by the [goto](#), [return](#), or [throw](#) statements.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.8 Do statements
- 8.8.2 The do statement

See Also

Reference

[C# Keywords](#)

[The do-while Statement \(C++\)](#)

[Iteration Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

for (C# Reference)

The **for** loop executes a statement or a block of statements repeatedly until a specified expression evaluates to **false**. The **for** loop is handy for iterating over arrays and for sequential processing. In the following example, the value of `int i` is written to the console and `i` is incremented each time through the loop by 1.

Example

```
// statements_for.cs
// for loop
using System;
class ForLoopTest
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine(i);
        }
    }
}
```

Output

```
1
2
3
4
5
```

Remarks

The **for** statement executes the enclosed statement or statements repeatedly as follows:

- First, the initial value of the variable `i` is evaluated.
- Then, while the value of `i` is less than or equal to 5, the condition evaluates to **true**, the `Console.WriteLine` statement is executed and `i` is reevaluated.
- When `i` is greater than 5, the condition becomes **false** and control is transferred outside the loop.

Because the test of conditional expression takes place before the execution of the loop, therefore, a **for** statement executes zero or more times.

All of the expressions of the **for** statement are optional; for example, the following statement is used to write an infinite loop:

```
        for (;;)
    {
        // ...
    }
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.9 For statements
- 8.8.3 The for statement

See Also

Reference

[C# Keywords](#)

[foreach, in \(C# Reference\)](#)

[The for Statement](#)

[Iteration Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

foreach, in (C# Reference)

The **foreach** statement repeats a group of embedded statements for each element in an array or an object collection. The **foreach** statement is used to iterate through the collection to get the desired information, but should not be used to change the contents of the collection to avoid unpredictable side effects.

Remarks

The embedded statements continue to execute for each element in the array or collection. After the iteration has been completed for all the elements in the collection, control is transferred to the next statement following the **foreach** block.

At any point within the **foreach** block, you can break out of the loop using the [break](#) keyword, or step directly to the next iteration in the loop by using the [continue](#) keyword.

A **foreach** loop can also be exited by the [goto](#), [return](#), or [throw](#) statements.

For more information on the **foreach** keyword and code samples, see the following topics:

[Using foreach with Arrays \(C# Programming Guide\)](#)

[How to: Access a Collection Class with foreach \(C# Programming Guide\)](#)

Example

In this example, **foreach** is used to display the contents of an array of integers.

```
// cs_foreach.cs
class ForEachTest
{
    static void Main(string[] args)
    {
        int[] fibarray = new int[] { 0, 1, 2, 3, 5, 8, 13 };
        foreach (int i in fibarray)
        {
            System.Console.WriteLine(i);
        }
    }
}
```

Output

0
1
2
3
5
8
13

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.16 Foreach statements
- 8.8.4 The foreach statement

See Also

Reference

[C# Keywords](#)

[Iteration Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

while (C# Reference)

The **while** statement executes a statement or a block of statements until a specified expression evaluates to **false**.

Example

```
// statements_while.cs
using System;
class WhileTest
{
    static void Main()
    {
        int n = 1;
        while (n < 6)
        {
            Console.WriteLine("Current value of n is {0}", n);
            n++;
        }
    }
}
```

Output

Current value of n is 1
Current value of n is 2
Current value of n is 3
Current value of n is 4
Current value of n is 5

```
// statements_while_2.cs
using System;
class WhileTest
{
    static void Main()
    {
        int n = 1;
        while (n++ < 6)
        {
            Console.WriteLine("Current value of n is {0}", n);
        }
    }
}
```

Output

Current value of n is 2
Current value of n is 3
Current value of n is 4
Current value of n is 5
Current value of n is 6

Because the test of the **while** expression takes place before each execution of the loop, a **while** loop executes zero or more times. This differs from the **do** loop, which executes one or more times.

A **while** loop can be terminated when a [break](#), [goto](#), [return](#), or [throw](#) statement transfers control outside the loop. To pass control to the next iteration without exiting the loop, use the [continue](#) statement. Notice the difference in output in the three previous examples, depending on where `int n` is incremented. In the example below no output is generated.

```
// statements_while_3.cs
// no output is generated
using System;
class WhileTest
{
    static void Main()
    {
```

```
int n = 5;
while (++n < 6)
{
    Console.WriteLine("Current value of n is {0}", n);
}
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.7 While statements
- 8.8.1 The while statement

See Also

Reference

[C# Keywords](#)

[The while Statement](#)

[Iteration Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Jump Statements (C# Reference)

Branching is performed using jump statements, which cause an immediate transfer of the program control. The following keywords are used in jump statements:

- [break](#)
- [continue](#)
- [goto](#)
- [return](#)
- [throw](#)

See Also

Reference

[C# Keywords](#)

[Statement Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

break (C# Reference)

The **break** statement terminates the closest enclosing loop or [switch](#) statement in which it appears. Control is passed to the statement that follows the terminated statement, if any.

Example

In this example, the conditional statement contains a counter that is supposed to count from 1 to 100; however, the **break** statement terminates the loop after 4 counts.

```
// statements_break.cs
using System;
class BreakTest
{
    static void Main()
    {
        for (int i = 1; i <= 100; i++)
        {
            if (i == 5)
            {
                break;
            }
            Console.WriteLine(i);
        }
    }
}
```

Output

```
1
2
3
4
```

This example demonstrates the use of **break** in a [switch](#) statement.

```
// statements_break2.cs
// break and switch
using System;
class Switch
{
    static void Main()
    {
        Console.Write("Enter your selection (1, 2, or 3): ");
        string s = Console.ReadLine();
        int n = Int32.Parse(s);

        switch (n)
        {
            case 1:
                Console.WriteLine("Current value is {0}", 1);
                break;
            case 2:
                Console.WriteLine("Current value is {0}", 2);
                break;
            case 3:
                Console.WriteLine("Current value is {0}", 3);
                break;
            default:
                Console.WriteLine("Sorry, invalid selection.");
                break;
        }
    }
}
```

Input
1**Sample Output**

```
Enter your selection (1, 2, or 3): 1
Current value is 1
```

Comments

If you entered 4, the output would be:

```
Enter your selection (1, 2, or 3): 4
Sorry, invalid selection.
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.10 Break, continue, and goto statements
- 8.9.1 The break statement

See Also**Reference**

[C# Keywords](#)

[The break Statement](#)

[switch \(C# Reference\)](#)

[Jump Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

continue (C# Reference)

The **continue** statement passes control to the next iteration of the enclosing iteration statement in which it appears.

Example

In this example, a counter is initialized to count from 1 to 10. By using the **continue** statement in conjunction with the expression `(i < 9)`, the statements between **continue** and the end of the **for** body are skipped.

```
// statements_continue.cs
using System;
class ContinueTest
{
    static void Main()
    {
        for (int i = 1; i <= 10; i++)
        {
            if (i < 9)
            {
                continue;
            }
            Console.WriteLine(i);
        }
    }
}
```

Output

9
10

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.10 Break, continue, and goto statements
- 8.9.2 The continue statement

See Also

Reference

[C# Keywords](#)

[The break Statement](#)

[Jump Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

goto (C# Reference)

The **goto** statement transfers the program control directly to a labeled statement.

Remarks

A common use of **goto** is to transfer control to a specific switch-case label or the default label in a **switch** statement.

The **goto** statement is also useful to get out of deeply nested loops.

Example

The following example demonstrates using **goto** in a [switch](#) statement.

```
// statements_goto_switch.cs
using System;
class SwitchTest
{
    static void Main()
    {
        Console.WriteLine("Coffee sizes: 1=Small 2=Medium 3=Large");
        Console.Write("Please enter your selection: ");
        string s = Console.ReadLine();
        int n = int.Parse(s);
        int cost = 0;
        switch (n)
        {
            case 1:
                cost += 25;
                break;
            case 2:
                cost += 25;
                goto case 1;
            case 3:
                cost += 50;
                goto case 1;
            default:
                Console.WriteLine("Invalid selection.");
                break;
        }
        if (cost != 0)
        {
            Console.WriteLine("Please insert {0} cents.", cost);
        }
        Console.WriteLine("Thank you for your business.");
    }
}
```

Input

Sample Output

```
Coffee sizes: 1=Small 2=Medium 3=Large
Please enter your selection: 2
Please insert 50 cents.
Thank you for your business.
```

The following example demonstrates using **goto** to break out from nested loops.

```
// statements_goto.cs
// Nested search loops
using System;
public class GotoTest1
{

```

```

static void Main()
{
    int x = 200, y = 4;
    int count = 0;
    string[,] array = new string[x, y];

    // Initialize the array:
    for (int i = 0; i < x; i++)
        for (int j = 0; j < y; j++)
            array[i, j] = (++count).ToString();

    // Read input:
    Console.Write("Enter the number to search for: ");

    // Input a string:
    string myNumber = Console.ReadLine();

    // Search:
    for (int i = 0; i < x; i++)
    {
        for (int j = 0; j < y; j++)
        {
            if (array[i, j].Equals(myNumber))
            {
                goto Found;
            }
        }
    }

    Console.WriteLine("The number {0} was not found.", myNumber);
    goto Finish;

Found:
    Console.WriteLine("The number {0} is found.", myNumber);

Finish:
    Console.WriteLine("End of search.");
}
}

```

Input

44

Sample Output

Enter the number to search for: 44
The number 44 is found.
End of search.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.10 Break, continue, and goto statements
- 8.9.3 The goto statement

See Also

Reference

[C# Keywords](#)

[The goto Statement](#)

[Jump Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

return (C# Reference)

The **return** statement terminates execution of the method in which it appears and returns control to the calling method. It can also return an optional value. If the method is a **void** type, the **return** statement can be omitted.

Example

In the following example, the method `A()` returns the variable `Area` as a [double](#) value.

```
// statements_return.cs
using System;
class ReturnTest
{
    static double CalculateArea(int r)
    {
        double area = r * r * Math.PI;
        return area;
    }

    static void Main()
    {
        int radius = 5;
        Console.WriteLine("The area is {0:0.00}", CalculateArea(radius));
    }
}
```

Output

The area is 78.54

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.12 Return statements
- 8.9.4 The return statement

See Also

Reference

[C# Keywords](#)

[The return Statement](#)

[Jump Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Exception Handling Statements (C# Reference)

C# provides built-in support for handling anomalous situations, known as exceptions, which may occur during the execution of your program. These exceptions are handled by code that is outside the normal flow of control.

The following exception handling topics are explained in this section:

- [throw](#)
- [try-catch](#)
- [try-finally](#)
- [try-catch-finally](#)

See Also

Reference

[C# Keywords](#)

[Statement Types \(C# Reference\)](#)

[Exceptions and Exception Handling \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

throw (C# Reference)

The **throw** statement is used to signal the occurrence of an anomalous situation (exception) during the program execution.

Remarks

The thrown exception is an object whose class is derived from [System.Exception](#), for example:

```
class MyException : System.Exception {}  
// ...  
throw new MyException();
```

Usually the **throw** statement is used with try-catch or try-finally statements. When an exception is thrown, the program looks for the **catch** statement that handles this exception.

You can also rethrow a caught exception using the **throw** statement. For more information and examples, see [try-catch](#) and [Throwing Exceptions](#).

Example

This example demonstrates how to throw an exception using the **throw** statement.

```
// throw example  
using System;  
public class ThrowTest  
{  
    static void Main()  
    {  
        string s = null;  
  
        if (s == null)  
        {  
            throw new ArgumentNullException();  
        }  
  
        Console.WriteLine("The string s is null"); // not executed  
    }  
}
```

Output

The [ArgumentNullException](#) exception occurs.

Code Example

See the [try-catch](#), [try-finally](#), and [try-catch-finally](#) examples.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.11 Throw statements
- 8.9.5 The throw statement

See Also

Tasks

[How to: Explicitly Throw Exceptions](#)

Reference

[The try, catch, and throw Statements](#)

[C# Keywords](#)

[Exception Handling Statements \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

try-catch (C# Reference)

The try-catch statement consists of a **try** block followed by one or more **catch** clauses, which specify handlers for different exceptions.

Remarks

The **try** block contains the guarded code that may cause the exception. The block is executed until an exception is thrown or it is completed successfully. For example, the following attempt to cast a **null** object raises the [NullReferenceException](#) exception:

```
object o2 = null;
try
{
    int i2 = (int)o2;    // Error
}
```

The **catch** clause can be used without arguments, in which case it catches any type of exception, and referred to as the general catch clause. It can also take an object argument derived from **System.Exception**, in which case it handles a specific exception. For example:

```
catch (InvalidCastException e)
{
}
```

It is possible to use more than one specific **catch** clause in the same try-catch statement. In this case, the order of the **catch** clauses is important because the **catch** clauses are examined in order. Catch the more specific exceptions before the less specific ones.

A **throw** statement can be used in the **catch** block to re-throw the exception, which has been caught by the **catch** statement. For example:

```
catch (InvalidCastException e)
{
    throw (e);    // Rethrowing exception e
}
```

If you want to re-throw the exception currently handled by a parameter-less **catch** clause, use the **throw** statement without arguments. For example:

```
catch
{
    throw;
}
```

When inside a **try** block, only initialize variables that are declared therein; otherwise, an exception can occur before the execution of the block is completed. For example, in the following code example, the variable `x` is initialized inside the **try** block. An attempt to use this variable outside the **try** block in the `Write(x)` statement will generate the compiler error:

[Use of unassigned local variable.](#)

```
static void Main()
{
    int x;
    try
    {
        // Don't initialize this variable here.
        x = 123;
    }
    catch
    {
    }
    // Error: Use of unassigned local variable 'x'.
```

```
    Console.Write(x);  
}
```

For more information on **catch**, see [try-catch-finally](#).

Example

In this example, the **try** block contains a call to the method `MyMethod()` that may cause an exception. The **catch** clause contains the exception handler that simply displays a message on the screen. When the **throw** statement is called from inside `MyMethod`, the system looks for the **catch** statement and displays the message `Exception caught`.

```
// try_catch_example.cs  
using System;  
class MainClass  
{  
    static void ProcessString(string s)  
    {  
        if (s == null)  
        {  
            throw new ArgumentNullException();  
        }  
    }  
  
    static void Main()  
    {  
        try  
        {  
            string s = null;  
            ProcessString(s);  
        }  
        catch (Exception e)  
        {  
            Console.WriteLine("{0} Exception caught.", e);  
        }  
    }  
}
```

Sample Output

```
System.ArgumentNullException: Value cannot be null.  
   at MainClass.Main() Exception caught.
```

In this example, two catch statements are used. The most specific exception, which comes first, is caught.

```
// try_catch_ordering_catch_clauses.cs  
using System;  
class MainClass  
{  
    static void ProcessString(string s)  
    {  
        if (s == null)  
        {  
            throw new ArgumentNullException();  
        }  
    }  
  
    static void Main()  
    {  
        try  
        {  
            string s = null;  
            ProcessString(s);  
        }  
        // Most specific:  
        catch (ArgumentNullException e)  
        {  
            Console.WriteLine("{0} First exception caught.", e);  
        }  
    }  
}
```

```
    }  
    // Least specific:  
    catch (Exception e)  
    {  
        Console.WriteLine("{0} Second exception caught.", e);  
    }  
}  
}
```

Sample Output

System.ArgumentNullException: Value cannot be null. at MainClass.Main() First exception caught.
--

Comments

In the preceding example, if you start with the least specific catch clause, you will get the error message: A previous catch clause already catches all exceptions of this or a super type ('System.Exception')

However, to catch the least specific exception, replace the throw statement by the following one:

```
throw new Exception();
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.13 Try-catch statements
- 8.10 The try statement
- 16 Exceptions

See Also

Tasks

[How to: Explicitly Throw Exceptions](#)

Reference

[C# Keywords](#)

[The try, catch, and throw Statements](#)

[Exception Handling Statements \(C# Reference\)](#)

[throw \(C# Reference\)](#)

[try-finally \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

try-finally (C# Reference)

The **finally** block is useful for cleaning up any resources allocated in the [try](#) block as well as running any code that must execute even if there is an exception. Control is always passed to the finally block regardless of how the try block exits.

Remarks

Whereas **catch** is used to handle exceptions that occur in a statement block, **finally** is used to guarantee a statement block of code executes regardless of how the preceding **try** block is exited.

Example

In this example, there is one invalid conversion statement that causes an exception. When you run the program, you get a run-time error message, but the **finally** clause will still be executed and display the output.

```
// try-finally
using System;
public class MainClass
{
    static void Main()
    {
        int i = 123;
        string s = "Some string";
        object o = s;

        try
        {
            // Invalid conversion; o contains a string not an int
            i = (int)o;
        }
        finally
        {
            Console.WriteLine("i = {0}", i);
        }
    }
}
```

Comments

The example above causes `System.InvalidCastException` to be thrown.

Although an exception was caught, the output statement included in the **finally** block will still be executed, that is:

```
i = 123
```

For more information on **finally**, see [try-catch-finally](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.14 Try-finally statements
- 8.11 The try statement
- 16 Exceptions

See Also

Tasks

[How to: Explicitly Throw Exceptions](#)

Reference

[C# Keywords](#)

[The try, catch, and throw Statements](#)

[Exception Handling Statements \(C# Reference\)](#)

[throw \(C# Reference\)](#)

[try-catch \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

try-catch-finally (C# Reference)

A common usage of **catch** and **finally** together is to obtain and use resources in a **try** block, deal with exceptional circumstances in a **catch** block, and release the resources in the **finally** block.

For more information and examples on re-throwing exceptions, see [try-catch](#) and [Throwing Exceptions](#).

Example

```
// try_catch_finally.cs
using System;
public class EHClass
{
    static void Main()
    {
        try
        {
            Console.WriteLine("Executing the try statement.");
            throw new NullReferenceException();
        }
        catch (NullReferenceException e)
        {
            Console.WriteLine("{0} Caught exception #1.", e);
        }
        catch
        {
            Console.WriteLine("Caught exception #2.");
        }
        finally
        {
            Console.WriteLine("Executing finally block.");
        }
    }
}
```

Sample Output

```
Executing the try statement.
System.NullReferenceException: Object reference not set to an instance of an object.
   at EHClass.Main() Caught exception #1.
Executing finally block.
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.15 Try-catch-finally statements
- 8.10 The try statement
- 16 Exceptions

See Also

Tasks

[How to: Explicitly Throw Exceptions](#)

Reference

[C# Keywords](#)

[The try, catch, and throw Statements](#)

[Exception Handling Statements \(C# Reference\)](#)

[throw \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Checked and Unchecked (C# Reference)

C# statements can execute in either checked or unchecked context. In a checked context, arithmetic overflow raises an exception. In an unchecked context, arithmetic overflow is ignored and the result is truncated.

- [checked](#) Specify checked context.
- [unchecked](#) Specify unchecked context.

If neither **checked** nor **unchecked** is specified, the default context depends on external factors such as compiler options.

The following operations are affected by the overflow checking:

- Expressions using the following predefined operators on integral types:

`++` `--` `-` (unary) `+` `-` `*` `/`

- Explicit numeric conversions between integral types.

The [/checked](#) compiler option lets you specify checked or unchecked context for all integer arithmetic statements that are not explicitly in the scope of a **checked** or **unchecked** keyword.

See Also

Reference

[C# Keywords](#)

[Statement Types \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

checked (C# Reference)

The **checked** keyword is used to explicitly enable overflow-checking for integral-type arithmetic operations and conversions.

Remarks

By default, if an expression produces a value that is outside the range of the destination type, constant expressions cause compile-time errors, and non-constant expressions are evaluated at run-time and raise exceptions. However, the **checked** keyword can be used to enable checking if it is suppressed globally by compiler options or environment configuration.

See the [unchecked](#) examples on using the **unchecked** keyword.

Example

This sample shows how to use **checked** for a non-constant expression. The overflow is reported at run-time.

```
// statements_checked.cs
using System;
class OverFlowTest
{
    static short x = 32767;    // Max short value
    static short y = 32767;

    // Using a checked expression
    static int CheckedMethod()
    {
        int z = 0;
        try
        {
            z = checked((short)(x + y));
        }
        catch (System.OverflowException e)
        {
            Console.WriteLine(e.ToString());
        }
        return z;
    }

    static void Main()
    {
        Console.WriteLine("Checked output value is: {0}",
            CheckedMethod());
    }
}
```

Sample Output

```
System.OverflowException: Arithmetic operation resulted in an overflow.
   at OverFlowTest.CheckedMethod()
Checked output value is: 0
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.2 Block statements, checked, and unchecked statements
- 7.5.12 The checked and unchecked operators
- 8.11 The checked and unchecked statements

See Also

Reference

[C# Keywords](#)

[Checked and Unchecked \(C# Reference\)](#)

[unchecked \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

unchecked (C# Reference)

The **unchecked** keyword is used to suppress overflow-checking for integral-type arithmetic operations and conversions.

Remarks

In an unchecked context, if an expression produces a value that is outside the range of the destination type, the result is truncated. For example:

```
        unchecked
    {
        int val = 2147483647 * 2;
    }
```

Because the calculation above is performed in an **unchecked** block, the fact that the result is too large for an integer is ignored, and `val` is assigned the value `-2`. By default, overflow detection is enabled, which has the same effect as using [checked](#).

In the example above, had **unchecked** been omitted, a compilation error would occur because the expression uses constants and the result is known at compile time. The **unchecked** keyword also suppresses overflow detection for non-constant expressions, which otherwise result in [OverflowException](#) at runtime.

The unchecked keyword can also be used as an operator, like this:

```
public int UncheckedAdd(int a, int b)
{
    return unchecked(a + b);
}
```

Example

This sample shows how to use the **unchecked** statement, using **unchecked** with constant expressions.

```
// statements_unchecked.cs
using System;

class TestClass
{
    const int x = 2147483647;    // Max int
    const int y = 2;

    static void Main()
    {
        int z;
        unchecked
        {
            z = x * y;
        }
        Console.WriteLine("Unchecked output value: {0}", z);
    }
}
```

Output

```
Unchecked output value: -2
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.2 Block statements, checked, and unchecked statements
- 7.5.12 The checked and unchecked operators

- 8.11 The checked and unchecked statements

See Also

Reference

[C# Keywords](#)

[Checked and Unchecked \(C# Reference\)](#)

[checked \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

fixed Statement (C# Reference)

The **fixed** statement prevents the garbage collector from relocating a movable variable. The **fixed** statement is only permitted in an [unsafe](#) context. **Fixed** can also be used to create [fixed size buffers](#).

Remarks

The **fixed** statement sets a pointer to a managed variable and "pins" that variable during the execution of *statement*. Without **fixed**, pointers to movable managed variables would be of little use since garbage collection could relocate the variables unpredictably. The C# compiler only lets you assign a pointer to a managed variable in a **fixed** statement.

```
// assume class Point { public int x, y; }
// pt is a managed variable, subject to garbage collection.
Point pt = new Point();
// Using fixed allows the address of pt members to be
// taken, and "pins" pt so it isn't relocated.
fixed ( int* p = &pt.x )
{
    *p = 1;
}
```

You can initialize a pointer with the address of an array or a string:

```
fixed (int* p = arr) ... // equivalent to p = &arr[0]
fixed (char* p = str) ... // equivalent to p = &str[0]
```

You can initialize multiple pointers, as long as they are all of the same type:

```
fixed (byte* ps = srcarray, pd = dstarray) {...}
```

To initialize pointers of different type, simply nest **fixed** statements:

```
fixed (int* p1 = &p.x)
{
    fixed (double* p2 = &array[5])
    {
        // Do something with p1 and p2.
    }
}
```

After the code in the statement is executed, any pinned variables are unpinned and subject to garbage collection. Therefore, do not point to those variables outside the **fixed** statement.

Note

Pointers initialized in fixed statements cannot be modified.

In unsafe mode, you can allocate memory on the stack, where it is not subject to garbage collection and therefore does not need to be pinned. For more information, see [stackalloc](#).

Example

```
// statements_fixed.cs
// compile with: /unsafe
using System;

class Point
{
    public int x, y;
```

```

}

class FixedTest
{
    // Unsafe method: takes a pointer to an int.
    unsafe static void SquarePtrParam (int* p)
    {
        *p *= *p;
    }

    unsafe static void Main()
    {
        Point pt = new Point();
        pt.x = 5;
        pt.y = 6;
        // Pin pt in place:
        fixed (int* p = &pt.x)
        {
            SquarePtrParam (p);
        }
        // pt now unpinned
        Console.WriteLine ("{0} {1}", pt.x, pt.y);
    }
}

```

Output

25	6
----	---

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 18.3 Fixed and moveable variables
- 18.6 The fixed statement

See Also

Reference

[C# Keywords](#)

[unsafe \(C# Reference\)](#)

[Fixed Size Buffers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

lock Statement (C# Reference)

The **lock** keyword marks a statement block as a critical section by obtaining the mutual-exclusion lock for a given object, executing a statement, and then releasing the lock. This statement takes the following form:

```
Object thisLock = new Object();
lock (thisLock)
{
    // Critical code section
}
```

For more information, see [Thread Synchronization \(C# Programming Guide\)](#).

Remarks

lock ensures that one thread does not enter a critical section of code while another thread is in the critical section. If another thread attempts to enter a locked code, it will wait, block, until the object is released.

The section [Threading \(C# Programming Guide\)](#) discusses threading.

lock calls [Enter](#) at the beginning of the block and [Exit](#) at the end of the block.

In general, avoid locking on a **public** type, or instances beyond your code's control. The common constructs `lock (this)`, `lock (typeof (MyType))`, and `lock ("myLock")` violate this guideline:

- `lock (this)` is a problem if the instance can be accessed publicly.
- `lock (typeof (MyType))` is a problem if `MyType` is publicly accessible.
- `lock ("myLock")` is a problem since any other code in the process using the same string, will share the same lock.

Best practice is to define a **private** object to lock on, or a **private static** object variable to protect data common to all instances.

Example

The following sample shows a simple use of threads in C#.

```
// statements_lock.cs
using System;
using System.Threading;

class ThreadTest
{
    public void RunMe()
    {
        Console.WriteLine("RunMe called");
    }

    static void Main()
    {
        ThreadTest b = new ThreadTest();
        Thread t = new Thread(b.RunMe);
        t.Start();
    }
}
```

Output

```
RunMe called
```

The following sample uses threads and **lock**. As long as the **lock** statement is present, the statement block is a critical section and `balance` will never become a negative number.

```
// statements_lock2.cs
```

```

using System;
using System.Threading;

class Account
{
    private Object thisLock = new Object();
    int balance;

    Random r = new Random();

    public Account(int initial)
    {
        balance = initial;
    }

    int Withdraw(int amount)
    {
        // This condition will never be true unless the lock statement
        // is commented out:
        if (balance < 0)
        {
            throw new Exception("Negative Balance");
        }

        // Comment out the next line to see the effect of leaving out
        // the lock keyword:
        lock(thisLock)
        {
            if (balance >= amount)
            {
                Console.WriteLine("Balance before Withdrawal : " + balance);
                Console.WriteLine("Amount to Withdraw          : -" + amount);
                balance = balance - amount;
                Console.WriteLine("Balance after Withdrawal  : " + balance);
                return amount;
            }
            else
            {
                return 0; // transaction rejected
            }
        }
    }

    public void DoTransactions()
    {
        for (int i = 0; i < 100; i++)
        {
            Withdraw(r.Next(1, 100));
        }
    }
}

class Test
{
    static void Main()
    {
        Thread[] threads = new Thread[10];
        Account acc = new Account(1000);
        for (int i = 0; i < 10; i++)
        {
            Thread t = new Thread(new ThreadStart(acc.DoTransactions));
            threads[i] = t;
        }
        for (int i = 0; i < 10; i++)
        {
            threads[i].Start();
        }
    }
}

```

```
}  
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.18 Lock statements
- 8.12 The lock statement

See Also

Tasks

[Monitor Synchronization Technology Sample](#)

[Wait Synchronization Technology Sample](#)

Reference

[Threading \(C# Programming Guide\)](#)

[C# Keywords](#)

[Statement Types \(C# Reference\)](#)

[MethodImplAttributes Enumeration](#)

[Thread Synchronization \(C# Programming Guide\)](#)

[Mutex](#)

Concepts

[C# Programming Guide](#)

[Monitors](#)

[Interlocked Operations](#)

[AutoResetEvent](#)

Other Resources

[C# Reference](#)

Method Parameters (C# Reference)

If a parameter is declared for a method without [ref](#) or [out](#), the parameter can have a value associated with it. That value can be changed in the method, but the changed value will not be retained when control passes back to the calling procedure. By using a method parameter keyword, you can change this behavior.

This section describes the keywords you can use when declaring method parameters:

- [params](#)
- [ref](#)
- [out](#)

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

params (C# Reference)

The **params** keyword lets you specify a [method parameter](#) that takes an argument where the number of arguments is variable.

No additional parameters are permitted after the **params** keyword in a method declaration, and only one **params** keyword is permitted in a method declaration.

Example

```
// cs_params.cs
using System;
public class MyClass
{
    public static void UseParams(params int[] list)
    {
        for (int i = 0 ; i < list.Length; i++)
        {
            Console.WriteLine(list[i]);
        }
        Console.WriteLine();
    }

    public static void UseParams2(params object[] list)
    {
        for (int i = 0 ; i < list.Length; i++)
        {
            Console.WriteLine(list[i]);
        }
        Console.WriteLine();
    }

    static void Main()
    {
        UseParams(1, 2, 3);
        UseParams2(1, 'a', "test");

        // An array of objects can also be passed, as long as
        // the array type matches the method being called.
        int[] myarray = new int[3] {10,11,12};
        UseParams(myarray);
    }
}
```

Output

1
2
3
1
a
test
10
11
12

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 10.5.1.4 Parameter arrays

See Also

Reference

[C# Keywords](#)

[Method Parameters \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

ref (C# Reference)

The **ref** keyword causes arguments to be passed by reference. The effect is that any changes made to the parameter in the method will be reflected in that variable when control passes back to the calling method. To use a **ref** parameter, both the method definition and the calling method must explicitly use the **ref** keyword. For example:

```
class RefExample
{
    static void Method(ref int i)
    {
        i = 44;
    }
    static void Main()
    {
        int val = 0;
        Method(ref val);
        // val is now 44
    }
}
```

An argument passed to a **ref** parameter must first be initialized. This differs from **out**, whose argument need not be explicitly initialized before being passed. (See [out](#).)

Although **ref** and **out** are treated differently at run-time, they are treated the same at compile time. Therefore methods cannot be overloaded if one method takes a **ref** argument and the other takes an **out** argument. These two methods, for example, are identical in terms of compilation, so this code will not compile:

```
class CS0663_Example
{
    // compiler error CS0663: "cannot define overloaded
    // methods that differ only on ref and out"
    public void SampleMethod(ref int i) { }
    public void SampleMethod(out int i) { }
}
```

Overloading can be done, however, if one method takes a **ref** or **out** argument and the other uses neither, like this:

```
class RefOutOverloadExample
{
    public void SampleMethod(int i) { }
    public void SampleMethod(ref int i) { }
}
```

Remarks

Properties are not variables and therefore cannot be passed as **ref** parameters.

For information on passing arrays, see [Passing Arrays Using ref and out](#).

Example

Passing value types by reference--as demonstrated above--is useful, but **ref** is also useful for passing reference types. This allows called methods to modify the object to which the reference refers because the reference itself is being passed by reference. The following sample shows that when a reference type is passed as a **ref** parameter, the object itself can be changed.

```
class RefRefExample
{
    static void Method(ref string s)
    {
        s = "changed";
    }
    static void Main()
    {
    }
}
```

```
{  
    string str = "original";  
    Method(ref str);  
    // str is now "changed"  
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.1.5 Reference parameters
- 10.5.1.2 Reference parameters

See Also

Reference

[Passing Parameters \(C# Programming Guide\)](#)

[Method Parameters \(C# Reference\)](#)

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

out (C# Reference)

The **out** keyword causes arguments to be passed by reference. This is similar to the [ref](#) keyword, except that [ref](#) requires that the variable be initialized before being passed. To use an **out** parameter, both the method definition and the calling method must explicitly use the **out** keyword. For example:

```
class OutExample
{
    static void Method(out int i)
    {
        i = 44;
    }
    static void Main()
    {
        int value;
        Method(out value);
        // value is now 44
    }
}
```

Although variables passed as an **out** arguments need not be initialized prior to being passed, the calling method is required to assign a value before the method returns.

The **ref** and **out** keywords are treated differently at run-time, but they are treated the same at compile time. Therefore methods cannot be overloaded if one method takes a **ref** argument and the other takes an **out** argument. These two methods, for example, are identical in terms of compilation, so this code will not compile:

```
class CS0663_Example
{
    // compiler error CS0663: "cannot define overloaded
    // methods that differ only on ref and out"
    public void SampleMethod(out int i) { }
    public void SampleMethod(ref int i) { }
}
```

Overloading can be done, however, if one method takes a **ref** or **out** argument and the other uses neither, like this:

```
class RefOutOverloadExample
{
    public void SampleMethod(int i) { }
    public void SampleMethod(out int i) { }
}
```

Remarks

Properties are not variables and therefore cannot be passed as **out** parameters.

For information on passing arrays, see [Passing Arrays Using ref and out](#).

Example

Declaring an **out** method is useful when you want a method to return multiple values. A method that uses an **out** parameter can still a variables as a return type (see [return](#)) but it can also return one or more objects to a calling method as **out** parameters. This example uses **out** to return three variables with a single method call. Note that the third argument is assigned to null. This allows methods to return values optionally.

```
class OutReturnExample
{
    static void Method(out int i, out string s1, out string s2)
    {
        i = 44;
        s1 = "I've been returned";
        s2 = null;
    }
}
```

```
}  
static void Main()  
{  
    int value;  
    string str1, str2;  
    Method(out value, out str1, out str2);  
    // value is now 44  
    // str1 is now "I've been returned"  
    // str2 is (still) null;  
}  
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.1.6 Output parameters
- 10.5.1.3 Output parameters

See Also

Reference

[C# Keywords](#)

[Method Parameters \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Namespace Keywords (C# Reference)

This section describes the keywords and operators that are associated with using namespaces:

- [namespace](#)
- [using](#)
- [. Operator](#)
- [:: Operator](#)
- [extern alias](#)

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

[Namespaces \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

namespace (C# Reference)

The **namespace** keyword is used to declare a scope. This namespace scope lets you organize code and gives you a way to create globally unique types.

```
namespace SampleNamespace
{
    class SampleClass{}
    interface SampleInterface{}
    struct SampleStruct{}
    enum SampleEnum{a,b}
    delegate void SampleDelegate(int i);
    namespace SampleNamespace.Nested
    {
        class SampleClass2{}
    }
}
```

Remarks

Within a namespace, you can declare one or more of the following types:

- another namespace
- [class](#)
- [interface](#)
- [struct](#)
- [enum](#)
- [delegate](#)

Whether or not you explicitly declare a namespace in a C# source file, the compiler adds a default namespace. This unnamed namespace, sometimes called the global namespace, is present in every file. Any identifier in the global namespace is available for use in a named namespace.

Namespaces implicitly have public access and this is not modifiable. For a discussion of the access modifiers you can assign to elements within a namespace, see [Access Modifiers \(C# Reference\)](#).

It is possible to define a namespace in two or more declarations. For example, the following example defines two classes as part of the `MyCompany` namespace:

```
// cs_namespace_keyword.cs
// compile with: /target:library
namespace MyCompany.Proj1
{
    class MyClass
    {
    }
}

namespace MyCompany.Proj1
{
    class MyClass1
    {
    }
}
```

Example

The following example shows how to call a static method in a nested namespace.

```
// cs_namespace_keyword_2.cs
```

```
using System;
namespace SomeNameSpace
{
    public class MyClass
    {
        static void Main()
        {
            Nested.NestedNameSpaceClass.SayHello();
        }
    }

    // a nested namespace
    namespace Nested
    {
        public class NestedNameSpaceClass
        {
            public static void SayHello()
            {
                Console.WriteLine("Hello");
            }
        }
    }
}
```

Output

Hello

For More Information

For more information on using namespaces, see the following topics:

- [Namespaces \(C# Programming Guide\)](#)
- [Using Namespaces \(C# Programming Guide\)](#)
- [How to: Use the Namespace Alias Qualifier \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 3.4.1 Namespace members
- 3.8 Namespace and type names
- 9 Namespaces

See Also

Reference

[C# Keywords](#)

[Namespace Keywords \(C# Reference\)](#)

[using \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

using (C# Reference)

The **using** keyword has two major uses:

- As a directive, when it is used to create an alias for a namespace or to import types defined in other namespaces. See [using Directive](#).
- As a statement, when it defines a scope at the end of which an object will be disposed. See [using Statement](#).

See Also

Reference

[C# Keywords](#)

[Namespace Keywords \(C# Reference\)](#)

[extern \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Namespaces \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

using Directive (C# Reference)

The **using** directive has two uses:

- To permit the use of types in a namespace so you do not have to qualify the use of a type in that namespace:

```
using System.Text;
```

- To create an alias for a namespace or a type.

```
using Project = PC.MyCompany.Project;
```

The **using** keyword is also be used to create *using statements*, which define when an object will be disposed. See [using Statement](#) for more information.

Remarks

The scope of a **using** directive is limited to the file in which it appears.

Create a **using** alias to make it easier to qualify an identifier to a namespace or type.

Create a **using** directive to use the types in a namespace without having to specify the namespace. A **using** directive does not give you access to any namespaces that are nested in the namespace you specify.

Namespaces come in two categories: user-defined and system-defined. User-defined namespaces are namespaces defined in your code. For a list of the system-defined namespaces, see [.NET Framework Class Library Reference](#).

For examples on referencing methods in other assemblies, see [Creating and Using C# DLLs](#).

Example 1

The following example shows how to define and use a **using** alias for a namespace:

```
namespace PC
{
    // Define an alias for the nested namespace.
    using Project = PC.MyCompany.Project;
    class A
    {
        void M()
        {
            // Use the alias
            Project.MyClass mc = new Project.MyClass();
        }
    }
    namespace MyCompany
    {
        namespace Project
        {
            public class MyClass{}
        }
    }
}
```

Example 2

The following example shows how to define a **using** directive and a **using** alias for a class:

```
// cs_using_directive2.cs
// Using directive.
using System;
// Using alias for a class.
using AliasToMyClass = NameSpace1.MyClass;
```

```

namespace NameSpace1
{
    public class MyClass
    {
        public override string ToString()
        {
            return "You are in NameSpace1.MyClass";
        }
    }
}

namespace NameSpace2
{
    class MyClass
    {
    }
}

namespace NameSpace3
{
    // Using directive:
    using NameSpace1;
    // Using directive:
    using NameSpace2;

    class MainClass
    {
        static void Main()
        {
            AliasToMyClass somevar = new AliasToMyClass();
            Console.WriteLine(somevar);
        }
    }
}

```

Output

You are in NameSpace1.MyClass

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [9.3 Using directives](#)

See Also

Reference

[C# Keywords](#)

[Namespace Keywords \(C# Reference\)](#)

[using Statement \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Namespaces \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

using Statement (C# Reference)

Defines a scope, outside of which an object or objects will be disposed.

Syntax

```
using (Font font1 = new Font("Arial", 10.0f))  
{  
}
```

Remarks

C#, through the .NET Framework common language runtime (CLR), automatically releases the memory used to store objects that are no longer required. The release of memory is non-deterministic; memory is released whenever the CLR decides to perform garbage collection. However, it is usually best to release limited resources such as file handles and network connections as quickly as possible.

The **using** statement allows the programmer to specify when objects that use resources should release them. The object provided to the **using** statement must implement the [IDisposable](#) interface. This interface provides the [Dispose](#) method, which should release the object's resources.

A **using** statement can be exited either when the end of the **using** statement is reached or if an exception is thrown and control leaves the statement block before the end of the statement.

The object can be declared in the **using** statement, as shown above, or before the using statement, like this:

```
Font font2 = new Font("Arial", 10.0f);  
using (font2)  
{  
    // use font2  
}
```

Multiple objects can be used in with a **using** statement, but they must be declared inside the **using** statement, like this:

```
using (Font font3 = new Font("Arial", 10.0f),  
        font4 = new Font("Arial", 10.0f))  
{  
    // Use font3 and font4.  
}
```

Example

The following sample shows how a user-defined class can implement its own Dispose behavior. Note that your type must inherit from **IDisposable**.

```
using System;  
  
class C : IDisposable  
{  
    public void UseLimitedResource()  
    {  
        Console.WriteLine("Using limited resource...");  
    }  
  
    void IDisposable.Dispose()  
    {  
        Console.WriteLine("Disposing limited resource.");  
    }  
}  
  
class Program  
{
```

```
static void Main()
{
    using (C c = new C())
    {
        c.UseLimitedResource();
    }
    Console.WriteLine("Now outside using statement.");
    Console.ReadLine();
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 5.3.3.17 Using statements
- 8.13 The using statement

See Also

Reference

[C# Keywords](#)

[extern \(C# Reference\)](#)

[Namespace Keywords \(C# Reference\)](#)

[using Directive \(C# Reference\)](#)

[Implementing Finalize and Dispose to Clean Up Unmanaged Resources](#)

Concepts

[C# Programming Guide](#)

[Namespaces \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

extern alias (C# Reference)

It can sometimes be necessary to reference two versions of assemblies that have the same fully-qualified type names, for example when you need to use two or more versions of an assembly in the same application. By using an external assembly alias, the namespaces from each assembly can be wrapped inside root-level namespaces named by the alias, allowing them to be used in the same file.

Note

The [extern](#) keyword is also used as a method modifier, declaring a method written in unmanaged code.

To reference two assemblies with the same fully-qualified type names, an alias must be specified on the command line, as follows:

```
/r:GridV1=grid.dll
```

```
/r:GridV2=grid20.dll
```

This creates the external aliases `GridV1` and `GridV2`. To use these aliases from within a program, reference them using the **extern** keyword. For example:

```
extern alias GridV1;
```

```
extern alias GridV2;
```

Each extern alias declaration introduces an additional root-level namespace that parallels (but does not lie within) the global namespace. Thus types from each assembly can be referred to without ambiguity using their fully qualified name, rooted in the appropriate namespace-alias

In the above example, `GridV1::Grid` would be the grid control from `grid.dll`, and `GridV2::Grid` would be the grid control from `grid20.dll`.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 25.4 Extern aliases

See Also

Reference

[C# Keywords](#)

[Namespace Keywords \(C# Reference\)](#)

[:: Operator \(C# Reference\)](#)

[/reference \(Import Metadata\) \(C# Compiler Options\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Operator Keywords (C# Reference)

Used to perform miscellaneous actions such as creating objects, checking the run-time type of an object, obtaining the size of a type, and so forth. This section introduces the following keywords:

- [as](#) Converts an object to a compatible type.
- [is](#) Checks the run-time type of an object.
- [new](#)
 - [new Operator](#) Creates objects.
 - [new Modifier](#) Hides an inherited member.
 - [new Constraint](#) Qualifies a type parameter.
- [sizeof](#) Obtains the size of a type.
- [typeof](#) Obtains the **System.Type** object for a type.
- [true](#)
 - [true Operator](#) Returns the boolean value true to indicate true and returns false otherwise.
 - [true Literal](#) Represents the boolean value true.
- [false](#)
 - [false Operator](#) Returns the Boolean value true to indicate false and returns false otherwise.
 - [false Literal](#) Represents the boolean value false.
- [stackalloc](#) Allocates a block of memory on the stack.

The following keywords, which can be used as operators and as statements, are covered in the [Statements](#) section:

- [checked](#) Specifies checked context.
- [unchecked](#) Specifies unchecked context.

See Also

Reference

[C# Keywords](#)

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

as (C# Reference)

Used to perform conversions between compatible reference types. For example:

```
string s = someObject as string;
if (s != null)
{
    // someObject is a string.
}
```

Remarks

The **as** operator is similar to a cast operation; however, if the conversion is not possible, **as** returns **null** instead of raising an exception. More formally, an expression of the form,

```
expression as type
```

is equivalent to,

```
expression is type ? (type)expression : (type)null
```

except that `expression` is evaluated only once.

Note that the **as** operator only performs reference conversions and boxing conversions. The **as** operator cannot perform other conversions, such as user-defined conversions, which should instead be performed using cast expressions.

Example

```
// cs_keyword_as.cs
// The as operator.
using System;
class Class1
{
}

class Class2
{
}

class MainClass
{
    static void Main()
    {
        object[] objArray = new object[6];
        objArray[0] = new Class1();
        objArray[1] = new Class2();
        objArray[2] = "hello";
        objArray[3] = 123;
        objArray[4] = 123.4;
        objArray[5] = null;

        for (int i = 0; i < objArray.Length; ++i)
        {
            string s = objArray[i] as string;
            Console.Write("{0}:", i);
            if (s != null)
            {
                Console.WriteLine("'" + s + "'");
            }
            else
            {
                Console.WriteLine("not a string");
            }
        }
    }
}
```

```
}  
}  
}
```

Output

0:not a string
1:not a string
2:'hello'
3:not a string
4:not a string
5:not a string

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 6 Conversions
- 7.9.10 The as operator

See Also

Reference

[C# Keywords](#)

[is \(C# Reference\)](#)

[?: Operator \(C# Reference\)](#)

[Operator Keywords \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

is (C# Reference)

Checks if an object is compatible with a given type. For example, it can be determined if an object is compatible with the **string** type like this:

```
if (obj is string)
{
}
```

Remarks

An **is** expression evaluates to **true** if the provided expression is non-null, and the provided object can be cast to the provided type without causing an exception to be thrown. For more information, see [7.6.6 Cast expressions](#).

The **is** keyword results in a compile-time warning if the expression is known to always be **true** or to always be **false**, but typically evaluates type compatibility at run time.

The **is** operator cannot be overloaded.

Note that the **is** operator only considers reference conversions, boxing conversions, and unboxing conversions. Other conversions, such as user-defined conversions, are not considered.

Example

```
// cs_keyword_is.cs
// The is operator.
using System;
class Class1
{
}
class Class2
{
}

class IsTest
{
    static void Test(object o)
    {
        Class1 a;
        Class2 b;

        if (o is Class1)
        {
            Console.WriteLine("o is Class1");
            a = (Class1)o;
            // Do something with "a."
        }
        else if (o is Class2)
        {
            Console.WriteLine("o is Class2");
            b = (Class2)o;
            // Do something with "b."
        }
        else
        {
            Console.WriteLine("o is neither Class1 nor Class2.");
        }
    }
    static void Main()
    {
        Class1 c1 = new Class1();
        Class2 c2 = new Class2();
        Test(c1);
        Test(c2);
        Test("a string");
    }
}
```

```
}
```

Output

```
o is Class1
o is Class2
o is neither Class1 nor Class2.
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [7.9.9 The is operator](#)

See Also

Reference

[C# Keywords](#)

[typeof \(C# Reference\)](#)

[as \(C# Reference\)](#)

[Operator Keywords \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

new (C# Reference)

In C#, the **new** keyword can be used as an operator, a modifier, or a constraint.

[new Operator](#)

Used to create objects and invoke constructors.

[new Modifier](#)

Used to hide an inherited member from a base class member.

[new Constraint](#)

Used to restrict types that might be used as arguments for a type parameter in a generic declaration.

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

new Operator (C# Reference)

Used to create objects and invoke constructors. For example:

```
Class1 o = new Class1();
```

The **new** operator is also used to invoke the default constructor for value types. For example:

```
int i = new int();
```

In the preceding statement, `i` is initialized to 0, which is the default value for the type **int**. The statement has the same effect as the following:

```
int i = 0;
```

For a complete list of default values, see [Default Values Table](#).

Remember that it is an error to declare a default constructor for a [struct](#) because every value type implicitly has a public default constructor. It is possible to declare parameterized constructors on a struct type to set its initial values, but this is only necessary if values other than the default are required.

Value-type objects such as structs are created on the stack, while reference-type objects such as classes are created on the heap. Both types of objects are destroyed automatically, but objects based on value types are destroyed when they go out of scope, whereas objects based on reference types are destroyed at an unspecified time after the last reference to them is removed. For reference types that consume fixed resources such as large amounts of memory, file handles, or network connections, it is sometimes desirable to employ deterministic finalization to ensure that the object is destroyed as soon as possible. For more information, see [using Statement \(C# Reference\)](#).

The **new** operator cannot be overloaded.

If the **new** operator fails to allocate memory, it throws the exception, [OutOfMemoryException](#).

Example

In the following example, a **struct** object and a class object are created and initialized by using the **new** operator and then assigned values. The default and the assigned values are displayed.

```
// cs_operator_new.cs
// The new operator.
using System;
struct SampleStruct
{
    public int x;
    public int y;

    public SampleStruct(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}

class SampleClass
{
    public string name;
    public int id;

    public SampleClass() {}

    public SampleClass(int id, string name)
    {
        this.id = id;
        this.name = name;
    }
}
```

```

    }
}

class MainClass
{
    static void Main()
    {
        // Create objects using default constructors:
        SampleStruct Location1 = new SampleStruct();
        SampleClass Employee1 = new SampleClass();

        // Display values:
        Console.WriteLine("Default values:");
        Console.WriteLine("    Struct members: {0}, {1}",
            Location1.x, Location1.y);
        Console.WriteLine("    Class members: {0}, {1}",
            Employee1.name, Employee1.id);

        // Create objects using parameterized constructors:
        SampleStruct Location2 = new SampleStruct(10, 20);
        SampleClass Employee2 = new SampleClass(1234, "John Martin Smith");

        // Display values:
        Console.WriteLine("Assigned values:");
        Console.WriteLine("    Struct members: {0}, {1}",
            Location2.x, Location2.y);
        Console.WriteLine("    Class members: {0}, {1}",
            Employee2.name, Employee2.id);
    }
}

```

Output

<pre> Default values: Struct members: 0, 0 Class members: , 0 Assigned values: Struct members: 10, 20 Class members: John Martin Smith, 1234 </pre>

Comments

Notice in the example that the default value of a string is **null**. Therefore, it is not displayed.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 7.5.10 The new operator

See Also

Reference

[C# Keywords](#)

[Operator Keywords \(C# Reference\)](#)

[new \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

new Modifier (C# Reference)

When used as a modifier, the **new** keyword explicitly hides a member inherited from a base class. Hiding an inherited member means that the derived version of the member replaces the base-class version. Hiding members without the use of the **new** modifier is allowed, but generates a warning. Using **new** to explicitly hide a member suppresses this warning, and documents the fact that the derived version is intended as a replacement.

To hide an inherited member, declare it in the derived class using the same name, and modify it with the **new** modifier. For example:

```
public class BaseC
{
    public int x;
    public void Invoke() {}
}
public class DerivedC : BaseC
{
    new public void Invoke() {}
}
```

In this example, `BaseC.Invoke` is hidden by `DerivedC.Invoke`. The field `x` is not affected because it is not hidden by a similar name.

Name hiding through inheritance takes one of the following forms:

- A constant, field, property, or type introduced in a class or struct hides all base class members with the same name.
- A method introduced in a class or struct hides properties, fields, and types, with the same name, in the base class. It also hides all base class methods with the same signature.
- An indexer introduced in a class or struct hides all base class indexers with the same signature.

It is an error to use both **new** and [override](#) on the same member, as the two modifiers have mutually exclusive meanings. Using **new** creates a new member with the same name and causes the original member to become hidden, while **override** extends the implementation for an inherited member.

Using the **new** modifier in a declaration that does not hide an inherited member generates a warning.

Example

In this example, a base class, `BaseC`, and a derived class, `DerivedC`, use the same field name `x`, thus hiding the value of the inherited field. The example demonstrates the use of the **new** modifier. It also demonstrates how to access the hidden members of the base class by using their fully qualified names.

```
// cs_modifier_new.cs
// The new modifier.
using System;
public class BaseC
{
    public static int x = 55;
    public static int y = 22;
}

public class DerivedC : BaseC
{
    // Hide field 'x'
    new public static int x = 100;

    static void Main()
    {
        // Display the new value of x:
        Console.WriteLine(x);
        // Display the hidden value of x:
        Console.WriteLine(BaseC.x);
        // Display the unhidden member y:
    }
}
```

```

    Console.WriteLine(y);
}
}

```

Output

```

100
55
22

```

In this example, a nested class hides a class with the same name in the base class. The example demonstrates using the **new** modifier to eliminate the warning message, as well as accessing the hidden class members by using their fully qualified names.

```

// cs_modifer_new_nested.cs
// Using the new modifier with nested types.
using System;
public class BaseC
{
    public class NestedC
    {
        public int x = 200;
        public int y;
    }
}

public class DerivedC : BaseC
{
    // Nested type hiding the base type members.
    new public class NestedC
    {
        public int x = 100;
        public int y;
        public int z;
    }

    static void Main()
    {
        // Creating an object from the overlapping class:
        NestedC c1 = new NestedC();

        // Creating an object from the hidden class:
        BaseC.NestedC c2 = new BaseC.NestedC();

        Console.WriteLine(c1.x);
        Console.WriteLine(c2.x);
    }
}

```

Output

```

100
200

```

Comments

If you remove the **new** modifier, the program will still compile and run, but you will get the following warning:

```

The keyword new is required on 'MyDerivedC.x' because it hides inherited member 'MyBaseC.x'
.

```

You can also use the **new** modifier to modify a nested type if the nested type is hiding another type, as demonstrated in the following example.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 10.2.2 The new modifier

See Also

Reference

[C# Keywords](#)

[Operator Keywords \(C# Reference\)](#)

[Modifiers \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

new Constraint (C# Reference)

The **new** constraint specifies that any type argument in a generic class declaration must have a public parameterless constructor. Apply this constraint to a type parameter when your generic class creates new instances of the type, as shown in the following example:

```
class ItemFactory<T> where T : new()
{
    public T GetNewItem()
    {
        return new T();
    }
}
```

When you use the **new()** constraint with other constraints, it must be specified last:

```
using System;
public class ItemFactory<T>
    where T : IComparable, new()
{
}
```

For more information, see [Constraints on Type Parameters \(C# Programming Guide\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 19.1.4 Constraints

See Also

Reference

[C# Keywords](#)

[Operator Keywords \(C# Reference\)](#)

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

[Generics \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

sizeof (C# Reference)

Used to obtain the size in bytes for a [value type](#). For example, the size of the **int** type can be retrieved like this:

```
int intSize = sizeof(int);
```

Remarks

The **sizeof** operator can be applied only to value types, not reference types.

 **Note**

From version 2.0 of C# onwards, applying **sizeof** to predefined types no longer requires that [unsafe](#) mode be used.

The **sizeof** operator may not be overloaded. The values returned by the **sizeof** operator are of the type **int**. The following table shows the constant values that represent the sizes of certain predefined types.

Expression	Result
sizeof(sbyte)	1
sizeof(byte)	1
sizeof(short)	2
sizeof(ushort)	2
sizeof(int)	4
sizeof(uint)	4
sizeof(long)	8
sizeof(ulong)	8
sizeof(char)	2 (Unicode)
sizeof(float)	4
sizeof(double)	8
sizeof(bool)	1

For all other types, including structs, the **sizeof** operator can only be used in unsafe code blocks. Although you can use the [SizeOf](#) method, the value returned by this method is not always the same as the value returned by **sizeof**. **Marshal.SizeOf** returns the size after the type has been marshaled whereas **sizeof** returns the size as it has been allocated by the Common Language Runtime, including any padding.

Example

```
// cs_operator_sizeof.cs
// compile with: /unsafe
using System;
class MainClass
{
    unsafe static void Main()
    {
```

```
        Console.WriteLine("The size of short is {0}.", sizeof(short));  
        Console.WriteLine("The size of int is {0}.", sizeof(int));  
        Console.WriteLine("The size of long is {0}.", sizeof(long));  
    }  
}
```

Output

The size of short is 2. The size of int is 4. The size of long is 8.
--

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 18.5.8 The sizeof operator

See Also

Reference

[C# Keywords](#)

[Operator Keywords \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

typeof (C# Reference)

Used to obtain the **System.Type** object for a type. A **typeof** expression takes the following form:

```
System.Type type = typeof(int);
```

Remarks

To obtain the run-time type of an expression, you can use the .NET Framework method [GetType](#), like this:

```
int i = 0;
System.Type type = i.GetType();
```

The **typeof** operator can also be used on open generic types. Types with more than one type parameter must have the appropriate number of commas in the specification. The **typeof** operator cannot be overloaded.

Example

```
// cs_operator_typeof.cs
using System;
using System.Reflection;

public class SampleClass
{
    public int sampleMember;
    public void SampleMethod() {}

    static void Main()
    {
        Type t = typeof(SampleClass);
        // Alternatively, you could use
        // SampleClass obj = new SampleClass();
        // Type t = obj.GetType();

        Console.WriteLine("Methods:");
        MethodInfo[] methodInfo = t.GetMethods();

        foreach (MethodInfo mInfo in methodInfo)
            Console.WriteLine(mInfo.ToString());

        Console.WriteLine("Members:");
        MemberInfo[] memberInfo = t.GetMembers();

        foreach (MemberInfo mInfo in memberInfo)
            Console.WriteLine(mInfo.ToString());
    }
}
```

Output

```
Methods:
Void SampleMethod()
System.Type GetType()
System.String ToString()
Boolean Equals(System.Object)
Int32 GetHashCode()
Members:
Void SampleMethod()
System.Type GetType()
System.String ToString()
Boolean Equals(System.Object)
Int32 GetHashCode()
Void .ctor()
Int32 sampleMember
```

This sample uses the **GetType** method to determine the type used to contain the result of a numeric calculation. This depends on the storage requirements of the resulting number.

```
// cs_operator_typeof2.cs
using System;
class GetTypeTest
{
    static void Main()
    {
        int radius = 3;
        Console.WriteLine("Area = {0}", radius * radius * Math.PI);
        Console.WriteLine("The type is {0}",
            (radius * radius * Math.PI).GetType()
        );
    }
}
```

Output

Area = 28.2743338823081
The type is System.Double

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 7.5.11 The typeof operator

See Also

Reference

[C# Keywords](#)

[is \(C# Reference\)](#)

[Operator Keywords \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

true (C# Reference)

Used as an overloaded operator or as a literal:

[true Operator](#)

[true Literal](#)

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

true Operator (C# Reference)

Returns the [bool](#) value `true` to indicate true and returns `false` otherwise when defined by a user-defined type. This is useful for types that represent true, false, and null (neither true nor false), as used in databases.

Such types can be used for the controlling expression in [if](#), [do](#), [while](#), and [for](#) statements and in [conditional expressions](#).

If a type defines operator **true**, it must also define operator [false](#).

A type cannot directly overload the conditional logical operators ([&&](#) and [||](#)), but an equivalent effect can be achieved by overloading the regular logical operators and operators **true** and **false**.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 10.9.1 Unary operators
- 7.11.2 User-defined conditional logical operators
- 7.16 Boolean expressions

See Also

Reference

[C# Keywords](#)

[C# Operators](#)

[false \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

true Literal (C# Reference)

Represents the boolean value true.

Example

```
// cs_keyword_true.cs
using System;
class TestClass
{
    static void Main()
    {
        bool a = true;
        Console.WriteLine( a ? "yes" : "no" );
    }
}
```

Output

yes

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 2.4.4.1 Boolean literals

See Also

Reference

[C# Keywords](#)

[false \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

false (C# Reference)

Used as an overloaded operator or as a literal:

- [false Operator](#)
- [false Literal](#)

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

false Operator (C# Reference)

Returns the [bool](#) value `true` to indicate false and returns `false` otherwise. This is useful for types that represent true, false, and null, neither true nor false, as used in databases.

Such types can be used for the controlling expression in [if](#), [do](#), [while](#), and [for](#) statements and in [conditional expressions](#).

If a type defines operator **false**, it must also define operator [true](#).

A type cannot directly overload the conditional logical operators [&&](#) and [||](#), but an equivalent effect can be achieved by overloading the regular logical operators and operators **true** and **false**.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 10.9.1 Unary operators
- 7.11.2 User-defined conditional logical operators
- 7.16 Boolean expressions

See Also

Reference

[C# Keywords](#)

[C# Operators](#)

[true \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

false Literal (C# Reference)

Represents the boolean value false.

Example

```
// cs_keyword_false.cs
using System;
class TestClass
{
    static void Main()
    {
        bool a = false;
        Console.WriteLine( a ? "yes" : "no" );
    }
}
```

Output

```
no
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 2.4.4.1 Boolean literals

See Also

Reference

[C# Keywords](#)

[true \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

stackalloc (C# Reference)

Used in an unsafe code context to allocate a block of memory on the stack.

```
int* fib = stackalloc int[100];
```

Remarks

In the example above, a block of memory of sufficient size to contain 100 elements of type **int** is allocated on the stack, not the heap; the address of the block is stored in the pointer `fib`. This memory is not subject to garbage collection and therefore does not have to be pinned (via [fixed](#)). The lifetime of the memory block is limited to the lifetime of the method in which it is defined (there is no way to free the memory before the method returns).

stackalloc is only valid in local variable initializers.

Because Pointer types are involved, **stackalloc** requires [unsafe](#) context. See [Unsafe Code and Pointers](#).

stackalloc is similar to `_alloca` in the C run-time library.

Security

Unsafe code is inherently less secure than non-unsafe alternatives. However, the use of **stackalloc** automatically enables buffer overrun detection features in the common language runtime (CLR). If a buffer overrun is detected, the process is terminated as quickly as possible to minimize the chance that malicious code is executed.

Example

```
// cs_keyword_stackalloc.cs
// compile with: /unsafe
using System;
class Test
{
    static unsafe void Main()
    {
        int* fib = stackalloc int[100];
        int* p = fib;
        *p++ = *p++ = 1;
        for (int i = 2; i < 100; ++i, ++p)
        {
            *p = p[-1] + p[-2];
        }
        for (int i = 0; i < 10; ++i)
        {
            Console.WriteLine(fib[i]);
        }
    }
}
```

Output

1
1
2
3
5
8
13
21
34
55

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 18.7 Stack allocation

See Also

Reference

[C# Keywords](#)

[Operator Keywords \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

Conversion Keywords (C# Reference)

This section describes keywords used in type conversions:

- [explicit](#)
- [implicit](#)
- [operator](#)

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

explicit (C# Reference)

The **explicit** keyword declares a user-defined type conversion operator that must be invoked with a cast. For example, this operator converts from a class called Fahrenheit to a class called Celsius:

```
// Must be defined inside a class called Farenheit:
public static explicit operator Celsius(Farenheit f)
{
    return new Celsius((5.0f/9.0f)*(f.degrees-32));
}
```

This conversion operator can be invoked like this:

```
Farenheit f = new Farenheit(100.0f);
Celsius c = (Celsius)f;
```

Remarks

The conversion operator converts from a source type to a target type. The source type provides the conversion operator. Unlike implicit conversion, explicit conversion operators must be invoked by means of a cast. If a conversion operation can cause exceptions or lose information, you should mark it **explicit**. This prevents the compiler from silently invoking the conversion operation with possibly unforeseen consequences.

Omitting the cast results in compile-time error [Compiler Error CS0266](#).

For more information, see [Using Conversion Operators \(C# Programming Guide\)](#).

Example

The following example provides a `Fahrenheit` and a `Celsius` class, each of which provides an explicit conversion operator to the other class.

```
// cs_keyword_explicit_temp.cs
using System;
class Celsius
{
    public Celsius(float temp)
    {
        degrees = temp;
    }
    public static explicit operator Fahrenheit(Celsius c)
    {
        return new Fahrenheit((9.0f / 5.0f) * c.degrees + 32);
    }
    public float Degrees
    {
        get { return degrees; }
    }
    private float degrees;
}

class Fahrenheit
{
    public Fahrenheit(float temp)
    {
        degrees = temp;
    }
    public static explicit operator Celsius(Fahrenheit f)
    {
        return new Celsius((5.0f / 9.0f) * (f.degrees - 32));
    }
    public float Degrees
    {
        get { return degrees; }
    }
}
```

```

    }
    private float degrees;
}

class MainClass
{
    static void Main()
    {
        Fahrenheit f = new Fahrenheit(100.0f);
        Console.Write("{0} fahrenheit", f.Degrees);
        Celsius c = (Celsius)f;
        Console.Write(" = {0} celsius", c.Degrees);
        Fahrenheit f2 = (Fahrenheit)c;
        Console.WriteLine(" = {0} fahrenheit", f2.Degrees);
    }
}

```

Output

```
100 fahrenheit = 37.77778 celsius = 100 fahrenheit
```

The following example defines a struct, `Digit`, that represents a single decimal digit. An operator is defined for conversions from `byte` to `Digit`, but because not all bytes can be converted to a `Digit`, the conversion is explicit.

```

// cs_keyword_explicit_2.cs
using System;
struct Digit
{
    byte value;
    public Digit(byte value)
    {
        if (value > 9)
        {
            throw new ArgumentException();
        }
        this.value = value;
    }

    // Define explicit byte-to-Digit conversion operator:
    public static explicit operator Digit(byte b)
    {
        Digit d = new Digit(b);
        Console.WriteLine("conversion occurred");
        return d;
    }
}

class MainClass
{
    static void Main()
    {
        try
        {
            byte b = 3;
            Digit d = (Digit)b; // explicit conversion
        }
        catch (Exception e)
        {
            Console.WriteLine("{0} Exception caught.", e);
        }
    }
}

```

Output

```
conversion occurred
```


C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [6.2 Explicit conversions](#)

See Also

Tasks

[How to: Implement User-Defined Conversions Between Structs \(C# Programming Guide\)](#)

Reference

[C# Keywords](#)

[implicit \(C# Reference\)](#)

[operator \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

implicit (C# Reference)

The **implicit** keyword is used to declare an implicit user-defined type conversion operator. Use it to enable implicit conversions between a user-defined type and another type, if the conversion is guaranteed not to result in a loss of data.

```
class Digit
{
    public Digit(double d) { val = d; }
    public double val;
    // ...other members

    // User-defined conversion from Digit to double
    public static implicit operator double(Digit d)
    {
        return d.val;
    }
    // User-defined conversion from double to Digit
    public static implicit operator Digit(double d)
    {
        return new Digit(d);
    }
}
class Program
{
    static void Main(string[] args)
    {
        Digit dig = new Digit(7);
        //This call invokes the implicit "double" operator
        double num = dig;
        //This call invokes the implicit "Digit" operator
        Digit dig2 = 12;
        Console.WriteLine("num = {0} dig2 = {1}", num, dig2.val);
        Console.ReadLine();
    }
}
```

Remarks

By eliminating unnecessary casts, implicit conversions can improve source code readability. However, because implicit conversions do not require programmers to explicitly cast from one type to the other, care must be taken to prevent unexpected results. In general, implicit conversion operators should never throw exceptions and never lose information so that they can be used safely without the programmer's awareness. If a conversion operator cannot meet those criteria, it should be marked **explicit**. For more information, see [Using Conversion Operators](#).

C# Language Specification

For more information, see the following section in the [C# Language Specification](#):

- 6.1 Implicit conversions
- 10.9.3 Conversion Operators

See Also

Tasks

[How to: Implement User-Defined Conversions Between Structs \(C# Programming Guide\)](#)

Reference

[C# Keywords](#)

[explicit \(C# Reference\)](#)

[operator \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

operator (C# Reference)

Use the **operator** keyword to overload a built-in operator or to provide a user-defined conversion in a class or struct declaration.

Example

The following is an extremely simplified class for fractional numbers. It overloads the + and * operators to perform fractional addition and multiplication, and also provides a conversion operator that converts a Fraction type to a double type.

```
// cs_keyword_operator.cs
using System;
class Fraction
{
    int num, den;
    public Fraction(int num, int den)
    {
        this.num = num;
        this.den = den;
    }

    // overload operator +
    public static Fraction operator +(Fraction a, Fraction b)
    {
        return new Fraction(a.num * b.den + b.num * a.den,
            a.den * b.den);
    }

    // overload operator *
    public static Fraction operator *(Fraction a, Fraction b)
    {
        return new Fraction(a.num * b.num, a.den * b.den);
    }

    // user-defined conversion from Fraction to double
    public static implicit operator double(Fraction f)
    {
        return (double)f.num / f.den;
    }
}

class Test
{
    static void Main()
    {
        Fraction a = new Fraction(1, 2);
        Fraction b = new Fraction(3, 7);
        Fraction c = new Fraction(2, 3);
        Console.WriteLine((double)(a * b + c));
    }
}
```

Output

0.880952380952381

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 7.2.2 Operator overloading
- 7.2.3 Unary operator overload resolution
- 7.2.4 Binary operator overload resolution

See Also

Tasks

[How to: Implement User-Defined Conversions Between Structs \(C# Programming Guide\)](#)

Reference

[C# Keywords](#)

[implicit \(C# Reference\)](#)

[explicit \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Access Keywords (C# Reference)

This section introduces the following access keywords:

- [base](#)

Accesses the members of the base class.

- [this](#)

Refers to the current instance of the class.

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

base (C# Reference)

The **base** keyword is used to access members of the base class from within a derived class:

- Call a method on the base class that has been overridden by another method.
- Specify which base-class constructor should be called when creating instances of the derived class.

A base class access is permitted only in a constructor, an instance method, or an instance property accessor.

It is an error to use the **base** keyword from within a static method.

Example

In this example, both the base class, `Person`, and the derived class, `Employee`, have a method named `GetInfo`. By using the **base** keyword, it is possible to call the `GetInfo` method on the base class, from within the derived class.

```
// keywords_base.cs
// Accessing base class members
using System;
public class Person
{
    protected string ssn = "444-55-6666";
    protected string name = "John L. Malgraine";

    public virtual void GetInfo()
    {
        Console.WriteLine("Name: {0}", name);
        Console.WriteLine("SSN: {0}", ssn);
    }
}
class Employee : Person
{
    public string id = "ABC567EFG";
    public override void GetInfo()
    {
        // Calling the base class GetInfo method:
        base.GetInfo();
        Console.WriteLine("Employee ID: {0}", id);
    }
}
class TestClass
{
    static void Main()
    {
        Employee E = new Employee();
        E.GetInfo();
    }
}
```

This example shows how to specify the base-class constructor called when creating instances of a derived class.

```
// keywords_base2.cs
using System;
public class BaseClass
{
    int num;

    public BaseClass()
    {
        Console.WriteLine("in BaseClass()");
    }

    public BaseClass(int i)
```

```

    {
        num = i;
        Console.WriteLine("in BaseClass(int i)");
    }

    public int GetNum()
    {
        return num;
    }
}

public class DerivedClass : BaseClass
{
    // This constructor will call BaseClass.BaseClass()
    public DerivedClass() : base()
    {
    }

    // This constructor will call BaseClass.BaseClass(int i)
    public DerivedClass(int i) : base(i)
    {
    }

    static void Main()
    {
        DerivedClass md = new DerivedClass();
        DerivedClass md1 = new DerivedClass(1);
    }
}

```

Output

Name: John L. Malgraine
SSN: 444-55-6666
Employee ID: ABC567EFG

For additional examples, see [new](#), [virtual](#), and [override](#).

Output

in BaseClass()
in BaseClass(int i)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.3 Base classes
- 7.5.8 base access

See Also

Reference

[C# Keywords](#)

[this \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

this (C# Reference)

The **this** keyword refers to the current instance of the class.

The following are common uses of **this**:

- To qualify members hidden by similar names, for example:

```
public Employee(string name, string alias)
{
    this.name = name;
    this.alias = alias;
}
```

- To pass an object as a parameter to other methods, for example:

```
CalcTax(this);
```

- To declare indexers, for example:

```
public int this [int param]
{
    get { return array[param]; }
    set { array[param] = value; }
}
```

Static member functions, because they exist at the class level and not as part of an object, do not have a **this** pointer. It is an error to refer to **this** in a static method.

Example

In this example, **this** is used to qualify the `Employee` class members, `name` and `alias`, which are hidden by similar names. It is also used to pass an object to the method `CalcTax`, which belongs to another class.

```
// keywords_this.cs
// this example
using System;
class Employee
{
    private string name;
    private string alias;
    private decimal salary = 3000.00m;

    // Constructor:
    public Employee(string name, string alias)
    {
        // Use this to qualify the fields, name and alias:
        this.name = name;
        this.alias = alias;
    }

    // Printing method:
    public void printEmployee()
    {
        Console.WriteLine("Name: {0}\nAlias: {1}", name, alias);
        // Passing the object to the CalcTax method by using this:
        Console.WriteLine("Taxes: {0:C}", Tax.CalcTax(this));
    }

    public decimal Salary
    {
```

```

        get { return salary; }
    }
}
class Tax
{
    public static decimal CalcTax(Employee E)
    {
        return 0.08m * E.Salary;
    }
}

class MainClass
{
    static void Main()
    {
        // Create objects:
        Employee E1 = new Employee("John M. Trainer", "jtrainer");

        // Display results:
        E1.printEmployee();
    }
}

```

Output

Name: John M. Trainer Alias: jtrainer Taxes: \$240.00

For additional examples, see [class](#) and [struct](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [7.5.7 this access](#)
- [10.2.6.4 this access](#)

See Also

Reference

[C# Keywords](#)

[base \(C# Reference\)](#)

[Methods \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Literal Keywords (C# Reference)

C# has the following literal keywords:

- [null](#)
- [true](#)
- [false](#)
- [default](#)

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

null (C# Reference)

The **null** keyword is a literal that represents a null reference, one that does not refer to any object. **null** is the default value of reference-type variables.

C# 2.0 introduces nullable types, which are data types that may be set to an undefined value. See [Nullable Types \(C# Programming Guide\)](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 2.4.4.6 The null literal

See Also

Reference

[C# Keywords](#)

[Literal Keywords \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Default Values Table \(C# Reference\)](#)

true (C# Reference)

Used as an overloaded operator or as a literal:

[true Operator](#)

[true Literal](#)

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

false (C# Reference)

Used as an overloaded operator or as a literal:

- [false Operator](#)
- [false Literal](#)

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

default (C# Reference)

The **default** keyword.

The **default** keyword can be used in the **switch** statement or in generic code:

- [The switch statement](#): Specifies the default label.
- [Generic code](#): Specifies the default value of the type parameter. This will be null for reference types and zero for value types.

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Contextual Keywords (C# Reference)

A contextual keyword is used to provide a specific meaning in the code, but it is not a reserved word in C#. The following contextual keywords are introduced in this section:

get Defines an accessor method for a property or an indexer.

partial Defines partial classes, structs, and interfaces throughout the same compilation unit.

set Defines an accessor method for a property or an indexer.

where Adds constraints to a generic declaration.

yield Used in an iterator block to return a value to the enumerator object or to signal the end of iteration.

value Used to set accessors and to add or remove event handlers.

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

get (C# Reference)

Defines an *accessor* method in a property or indexer that retrieves the value of the property or the indexer element. See [Properties](#) and [Indexers](#) for more information.

This is an example of a **get** accessor in a property called `Seconds`:

```
class TimePeriod
{
    private double _seconds;
    public double Seconds
    {
        get { return _seconds; }
        set { _seconds = value; }
    }
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.6.2 Properties
- 10.6.2 Accessors

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

partial (C# Reference)

Partial type definitions allow the definition of a class, struct or interface to be split into multiple files.

In File1.cs:

```
namespace PC
{
    partial class A { }
}
```

In File2.cs:

```
namespace PC
{
    partial class A { }
}
```

Remarks

Splitting a class, struct or interface type over several files can be useful when working with large projects, or with automatically generated code such as that provided by the [Windows Forms Designer](#). For more information, see [Partial Classes](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 19.4 Partial types
- 23 Partial Types

See Also

Reference

[Modifiers \(C# Reference\)](#)

[Introduction to Generics \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

set (C# Reference)

Defines an *accessor* method in a property or indexer that set the value of the property or the indexer element. See [Properties](#) and [Indexers](#) for more information.

This is an example of a **set** accessor for a property called `Seconds`:

```
class TimePeriod
{
    private double _seconds;
    public double Seconds
    {
        get { return _seconds; }
        set { _seconds = value; }
    }
}
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.6.2 Properties
- 10.6.2 Accessors

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

value (C# Reference)

The implicit parameter **value** is used in setting accessors and to add or remove event handlers.

For more information on the use of **value**, see [event](#) and [accessor accessibility](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.6.2 Properties
- 10.6.2 Accessors

See Also

Reference

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

where (C# Reference)

The **where** clause is used to specify constraints on the types that can be used as arguments for a type parameter defined in a generic declaration. For example, you can declare a generic class, `MyGenericClass`, such that the type parameter `T` implements the **`Comparable<T>`** interface:

```
public class MyGenericClass<T> where T:Comparable { }
```

In addition to interface constraints, a **where** clause can include a base class constraint, which states that a type must have the specified class as a base class (or be that class itself) in order to be used as a type argument for that generic type. If such a constraint is used, it must appear before any other constraints on that type parameter.

```
// cs_where.cs
// compile with: /target:library
using System;

class MyClass<T, U>
    where T : class
    where U : struct
{
}
```

The **where** clause may also include a constructor constraint. It is possible to create an instance of a type parameter using the new operator; however, in order to do so the type parameter must be constrained by the constructor constraint, **`new()`**. The [new\(\) Constraint](#) lets the compiler know that any type argument supplied must have an accessible parameterless--or default--constructor. For example:

```
// cs_where_2.cs
// compile with: /target:library
using System;
public class MyGenericClass <T> where T: Comparable, new()
{
    // The following line is not possible without new() constraint:
    T item = new T();
}
```

The **`new()`** constraint appears last in the **where** clause.

With multiple type parameters, use one **where** clause for each type parameter, for example:

```
// cs_where_3.cs
// compile with: /target:library
using System;
using System.Collections;

interface MyI
{
}

class Dictionary<TKey,TVal>
    where TKey: Comparable, IEnumerable
    where TVal: MyI
{
    public void Add(TKey key, TVal val)
    {
    }
}
```

You can also attach constraints to type parameters of generic methods, like this:

```
public bool MyMethod<T>(T t) where T : IMyInterface { }
```

Notice that the syntax to describe type parameter constraints on delegates is the same as that of methods:

```
delegate T MyDelegate<T>() where T : new()
```

For information on generic delegates, see [Generic Delegates](#).

For details on the syntax and use of constraints, see [Constraints on Type Parameters](#).

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 19.1.4 Constraints

See Also

Reference

[Introduction to Generics \(C# Programming Guide\)](#)

[new Constraint \(C# Reference\)](#)

[Constraints on Type Parameters \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

yield (C# Reference)

Used in an [iterator](#) block to provide a value to the enumerator object or to signal the end of iteration. It takes one of the following forms:

```
yield return <expression>;
yield break;
```

Remarks

expression is evaluated and returned as a value to the enumerator object; *expression* has to be implicitly convertible to the yield type of the iterator.

The **yield** statement can only appear inside an **iterator** block, which might be used as a body of a method, operator, or accessor. The body of such methods, operators, or accessors is controlled by the following restrictions:

- Unsafe blocks are not allowed.
- Parameters to the method, operator, or accessor cannot be [ref](#) or [out](#).

A **yield** statement cannot appear in an anonymous method. For more information, see [Anonymous Methods \(C# Programming Guide\)](#).

When used with *expression*, a **yield return** statement cannot appear in a **catch** block or in a **try** block that has one or more **catch** clauses. For more information, see [Exception Handling Statements \(C# Reference\)](#).

Example

In the following example, the **yield** statement is used inside an iterator block, which is the method `Power(int number, int power)`. When the **Power** method is invoked, it returns an enumerable object that contains the powers of a number. Notice that the return type of the **Power** method is **IEnumerable**, an iterator interface type.

```
// yield-example.cs
using System;
using System.Collections;
public class List
{
    public static IEnumerable Power(int number, int exponent)
    {
        int counter = 0;
        int result = 1;
        while (counter++ < exponent)
        {
            result = result * number;
            yield return result;
        }
    }

    static void Main()
    {
        // Display powers of 2 up to the exponent 8:
        foreach (int i in Power(2, 8))
        {
            Console.Write("{0} ", i);
        }
    }
}
```

Output

2	4	8	16	32	64	128	256
---	---	---	----	----	----	-----	-----

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [19.3 Iterators](#)
- [22 Iterators](#)

See Also

Reference

[foreach, in \(C# Reference\)](#)

[Using Iterators \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

C# Operators

C# provides a large set of operators, which are symbols that specify which operations to perform in an expression. Operations on integral types such as `==`, `!=`, `<`, `>`, `<=`, `>=`, **binary +**, **binary -**, `^`, **&**, `|`, `~`, `++`, `--`, and **sizeof()** are generally allowed on enumerations. In addition, many operators can be [overloaded](#) by the user, thus changing their meaning when applied to a user-defined type.

The following table lists the C# operators grouped in order of precedence. Operators within each group have equal precedence.

Operator category	Operators
Primary	x.y f(x) a[x] x++ x-- new typeof checked unchecked ->
Unary	+ - ! ~ ++x --x (T)x true false & sizeof
Multiplicative	* / %
Additive	+ -
Shift	<< >>

Relational and type testing	< > <= >= is as
Equality	== !=
Logical AND	&
Logical XOR	^
Logical OR	
Conditional AND	&&
Conditional OR	
Conditional	?:
Assignment	= += -= *= /= %= &= = ^= <<= >>= ??

Arithmetic Overflow

The arithmetic operators (+, -, *, /) can produce results that are outside the range of possible values for the numeric type involved. You should refer to the section on a particular operator for details, but in general:

- Integer arithmetic overflow either throws an [OverflowException](#) or discards the most significant bits of the result. Integer division by zero always throws a **DivideByZeroException**.
- Floating-point arithmetic overflow or division by zero never throws an exception, because floating-point types are based on IEEE 754 and so have provisions for representing infinity and NaN (Not a Number).
- [Decimal](#) arithmetic overflow always throws an **OverflowException**. Decimal division by zero always throws a [DivideByZeroException](#).

When integer overflow occurs, what happens depends on the execution context, which can be [checked or unchecked](#). In a

checked context, an **OverflowException** is thrown. In an unchecked context, the most significant bits of the result are discarded and execution continues. Thus, C# gives you the choice of handling or ignoring overflow.

In addition to the arithmetic operators, integral-type to integral-type casts can cause overflow, for example, casting a [long](#) to an [int](#), and are subject to checked or unchecked execution. However, bitwise operators and shift operators never cause overflow.

See Also

Tasks

[Operator Overloading Sample](#)

Reference

[Overloadable Operators \(C# Programming Guide\)](#)

[C# Keywords](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

[Visual C#](#)

[] Operator (C# Reference)

Square brackets ([]) are used for arrays, indexers, and attributes. They can also be used with pointers.

Remarks

An array type is a type followed by []:

```
int[] fib; // fib is of type int[], "array of int"
fib = new int[100]; // create a 100-element int array
```

To access an element of an array, the index of the desired element is enclosed in brackets:

```
fib[0] = fib[1] = 1;
for( int i=2; i<100; ++i ) fib[i] = fib[i-1] + fib[i-2];
```

An exception is thrown if an array index is out of range.

The array indexing operator cannot be overloaded; however, types can define indexers, and properties that take one or more parameters. Indexer parameters are enclosed in square brackets, just like array indices, but indexer parameters can be declared to be of any type, unlike array indices, which must be integral.

For example, the .NET Framework defines a **Hashtable** type that associates keys and values of arbitrary type:

```
Collections.Hashtable h = new Collections.Hashtable();
h["a"] = 123; // note: using a string as the index
```

Square brackets are also used to specify [Attributes \(C# Programming Guide\)](#):

```
[attribute(AllowMultiple=true)]
public class Attr
{
}
```

You can use square brackets to index off a pointer:

```
unsafe fixed ( int* p = fib )    // p points to fib from earlier example
{
    p[0] = p[1] = 1;
    for( int i=2; i<100; ++i ) p[i] = p[i-1] + p[i-2];
}
```

No bounds checking is performed.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- [1.6.6.5 Operators](#)
- [7.2 Operators](#)

See Also

Reference

[C# Operators](#)

[Indexers \(C# Programming Guide\)](#)

[unsafe \(C# Reference\)](#)

[fixed Statement \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

[Arrays \(C# Programming Guide\)](#)

Other Resources

() Operator (C# Reference)

In addition to being used to specify the order of operations in an expression, parentheses are used to specify casts, or type conversions:

```
double x = 1234.7;
int a;
a = (int)x; // cast double to int
```

Remarks

A cast explicitly invokes the conversion operator from one type to another; the cast will fail if no such conversion operator is defined. To define a conversion operator, see [explicit](#) and [implicit](#).

The **()** operator cannot be overloaded.

For more information, see [Casting \(C# Programming Guide\)](#).

A cast expression can lead to ambiguous syntax. For example, the expression (x)–y could either be interpreted as a cast expression (a cast of –y to type x) or as an additive expression combined with a parenthesized expression, which computes the value x – y.

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.6.5 Operators
- 7.2 Operators

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

. Operator (C# Reference)

The dot operator (.) is used for member access. The dot operator specifies a member of a type or namespace. For example, the dot operator is used to access specific methods within the .NET Framework class libraries:

```
// The class Console in namespace System:  
System.Console.WriteLine("hello");
```

Remarks

For example, consider the following class:

```
class Simple  
{  
    public int a;  
    public void b()  
    {  
    }  
}  
Simple s = new Simple();
```

The variable `s` has two members, `a` and `b`; to access them, use the dot operator:

```
s.a = 6;    // assign to field a;  
s.b();     // invoke member function b;
```

The dot is also used to form qualified names, which are names that specify the namespace or interface, for example, to which they belong.

```
// The class Console in namespace System:  
System.Console.WriteLine("hello");
```

The using directive makes some name qualification optional:

```
using System;  
// ...  
System.Console.WriteLine("hello");  
Console.WriteLine("hello");    // same thing
```

But when an identifier is ambiguous, it must be qualified:

```
using System;  
// A namespace containing another Console class:  
using OtherSystem;  
// ...  
// Must qualify Console:  
System.Console.WriteLine( "hello" );
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 7.5.4 member access

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

:: Operator (C# Reference)

The namespace alias qualifier operator.

The namespace alias qualifier (::) is used to look up identifiers. It is always placed between two identifiers, as in this example:

```
global::System.Console.WriteLine("Hello World");
```

Remarks

The namespace alias qualifier can be `global`. This invokes a lookup in the global namespace, rather than an aliased namespace.

For More Information

For an example of using the :: operator, see the following section:

- [How to: Use the Namespace Alias Qualifier \(C# Programming Guide\)](#)

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 20.9.5 Simple names
- 25.3 Namespace alias qualifiers

See Also

Reference

[C# Operators](#)

[Namespace Keywords \(C# Reference\)](#)

[. Operator \(C# Reference\)](#)

[extern alias \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

+ Operator (C# Reference)

The + operator can function as either a unary or a binary operator.

Remarks

Unary + operators are predefined for all numeric types. The result of a unary + operation on a numeric type is simply the value of the operand.

Binary + operators are predefined for numeric and string types. For numeric types, + computes the sum of its two operands. When one or both operands are of type string, + concatenates the string representations of the operands.

Delegate types also provide a binary + operator, which performs delegate concatenation.

User-defined types can overload the unary + and binary + operators (see [operator](#)). Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_plus.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(+5);           // unary plus
        Console.WriteLine(5 + 5);        // addition
        Console.WriteLine(5 + .5);       // addition
        Console.WriteLine("5" + "5");    // string concatenation
        Console.WriteLine(5.0 + "5");    // string concatenation
        // note automatic conversion from double to string
    }
}
```

Output

5
10
5.5
55
55

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.6.5 Operators
- 7.2 Operators

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

- Operator (C# Reference)

The - operator can function as either a unary or a binary operator.

Remarks

Unary - operators are predefined for all numeric types. The result of a unary - operation on a numeric type is the numeric negation of the operand.

Binary - operators are predefined for all numeric and enumeration types to subtract the second operand from the first.

Delegate types also provide a binary - operator, which performs delegate removal.

User-defined types can overload the unary - and binary - operators. For more information, see [operator](#).

Example

```
// cs_operator_minus.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 5;
        Console.WriteLine(-a);
        Console.WriteLine(a - 1);
        Console.WriteLine(a - .5);
    }
}
```

Output

-5
4
4.5

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 1.6.6.5 Operators
- 7.2 Operators

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

* Operator (C# Reference)

The multiplication operator (*), which computes the product of its operands. Also, the dereference operator, which allows reading and writing to a pointer.

Remarks

All numeric types have predefined multiplication operators.

The * operator is also used to declare pointer types and to dereference pointers. This operator can only be used in unsafe contexts, denoted by the use of the [unsafe](#) keyword, and requiring the [/unsafe](#) compiler option. The dereference operator is also known as the indirection operator.

User-defined types can overload the binary * operator (see [operator](#)). When a binary operator is overloaded, the corresponding assignment operator, if any, is also implicitly overloaded.

Example

```
// cs_operator_mult.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(5 * 2);
        Console.WriteLine(-.5 * .2);
        Console.WriteLine(-.5m * .2m); // decimal type
    }
}
```

Output

10
-0.1
-0.10

```
// cs_operator_ptr.cs
// compile with: /unsafe
public class MainClass
{
    unsafe static void Main()
    {
        int i = 5;
        int* j = &i;
        System.Console.WriteLine(*j);
    }
}
```

Output

5

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

[Unsafe Code and Pointers \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

/ Operator (C# Reference)

The division operator (/) divides its first operand by its second. All numeric types have predefined division operators.

Remarks

User-defined types can overload the / operator (see [operator](#)). Overloading the / operator implicitly overloads the /= operator.

Example

```
// cs_operator_division.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(-5/2);
        Console.WriteLine(-5.0/2);
    }
}
```

Output

-2
-2.5

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

% Operator (C# Reference)

The modulus operator (%) computes the remainder after dividing its first operand by its second. All numeric types have predefined modulus operators.

Remarks

User-defined types can overload the % operator (see [operator](#)). When a binary operator is overloaded, the corresponding assignment operator, if any, is also implicitly overloaded.

Example

```
// cs_operator_modulus.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(5 % 2);           // int
        Console.WriteLine(-5 % 2);          // int
        Console.WriteLine(5.0 % 2.2);       // double
        Console.WriteLine(5.0m % 2.2m);     // decimal
        Console.WriteLine(-5.2 % 2.0);     // double
    }
}
```

Output

1
-1
0.6
0.6
-1.2

Comments

Note the round-off errors associated with the double type.

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

& Operator (C# Reference)

The & operator can function as either a unary or a binary operator.

Remarks

The unary & operator returns the address of its operand (requires [unsafe](#) context).

Binary & operators are predefined for the integral types and **bool**. For integral types, & computes the logical bitwise AND of its operands. For **bool** operands, & computes the logical AND of its operands; that is, the result is **true** if and only if both its operands are **true**.

The **&** operator evaluates both operators regardless of the first one's value. For example:

```
int i = 0;
if (false & ++i == 1)
{
    // i is incremented, but the conditional
    // expression evaluates to false, so
    // this block does not execute.
}
```

User-defined types can overload the binary **&** operator (see [operator](#)). Operations on integral types are generally allowed on enumeration. When a binary operator is overloaded, the corresponding assignment operator, if any, is also implicitly overloaded.

Example

```
// cs_operator_ampersand.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(true & false); // logical and
        Console.WriteLine(true & true);  // logical and
        Console.WriteLine("0x{0:x}", 0xf8 & 0x3f); // bitwise and
    }
}
```

Output

False
True
0x38

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

| Operator (C# Reference)

Binary **|** operators are predefined for the integral types and **bool**. For integral types, **|** computes the bitwise OR of its operands. For **bool** operands, **|** computes the logical OR of its operands; that is, the result is **false** if and only if both its operands are **false**.

Remarks

User-defined types can overload the **|** operator (see [operator](#)).

Example

```
// cs_operator_OR.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(true | false); // logical or
        Console.WriteLine(false | false); // logical or
        Console.WriteLine("0x{0:x}", 0xf8 | 0x3f); // bitwise or
    }
}
```

Output

True
False
0xff

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

^ Operator (C# Reference)

Binary **^** operators are predefined for the integral types and **bool**. For integral types, **^** computes the bitwise exclusive-OR of its operands. For **bool** operands, **^** computes the logical exclusive-or of its operands; that is, the result is **true** if and only if exactly one of its operands is **true**.

Remarks

User-defined types can overload the **^** operator (see [operator](#)). Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_bitwise_OR.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(true ^ false); // logical exclusive-or
        Console.WriteLine(false ^ false); // logical exclusive-or
        // Bitwise exclusive-or:
        Console.WriteLine("0x{0:x}", 0xf8 ^ 0x3f);
    }
}
```

Output

True
False
0xc7

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

! Operator (C# Reference)

The logical negation operator (!) is a unary operator that negates its operand. It is defined for **bool** and returns **true** if and only if its operand is **false**.

Remarks

User-defined types can overload the ! operator (see [operator](#)).

Example

```
// cs_operator_negation.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(!true);
        Console.WriteLine(!false);
    }
}
```

Output

False
True

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

~ Operator (C# Reference)

The ~ operator performs a bitwise complement operation on its operand, which has the effect of reversing each bit. Bitwise complement operators are predefined for [int](#), [uint](#), [long](#), and [ulong](#).

Remarks

User-defined types can overload the ~ operator. For more information, see [operator](#). Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_bitwise_compl.cs
using System;
class MainClass
{
    static void Main()
    {
        int[] values = { 0, 0x111, 0xffffffff, 0x8888, 0x22000022};
        foreach (int v in values)
        {
            Console.WriteLine("~0x{0:x8} = 0x{1:x8}", v, ~v);
        }
    }
}
```

Output

~0x00000000	=	0xffffffff
~0x00000111	=	0xfffffee
~0x000fffff	=	0xfff00000
~0x00008888	=	0xffff7777
~0x22000022	=	0xddffffdd

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

= Operator (C# Reference)

The assignment operator (=) stores the value of its right-hand operand in the storage location, property, or indexer denoted by its left-hand operand and returns the value as its result. The operands must be of the same type (or the right-hand operand must be implicitly convertible to the type of the left-hand operand).

Remarks

The assignment operator cannot be overloaded.

Example

```
// cs_operator_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        double x;
        int i;
        i = 5; // int to int assignment
        x = i; // implicit conversion from int to double
        i = (int)x; // needs cast
        Console.WriteLine("i is {0}, x is {1}", i, x);
        object obj = i;
        Console.WriteLine("boxed value = {0}, type is {1}",
            obj, obj.GetType());
        i = (int)obj;
        Console.WriteLine("unboxed: {0}", i);
    }
}
```

Output

```
i is 5, x is 5
boxed value = 5, type is System.Int32
unboxed: 5
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

< Operator (C# Reference)

All numeric and enumeration types define a "less than" relational operator (<) that returns **true** if the first operand is less than the second, **false** otherwise.

Remarks

User-defined types can overload the < operator (see [operator](#)). If < is overloaded, > must also be overloaded. When a binary operator is overloaded, the corresponding assignment operator, if any, is also implicitly overloaded.

Example

```
// cs_operator_less_than.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(1 < 1.1);
        Console.WriteLine(1.1 < 1.1);
    }
}
```

Output

True
False

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

> Operator (C# Reference)

All numeric and enumeration types define a "greater than" relational operator (>) that returns **true** if the first operand is greater than the second, **false** otherwise.

Remarks

User-defined types can overload the > operator (see [operator](#)). If > is overloaded, < must also be overloaded. When a binary operator is overloaded, the corresponding assignment operator, if any, is also implicitly overloaded.

Example

```
// cs_operator_greater_than.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(1.1 > 1);
        Console.WriteLine(1.1 > 1.1);
    }
}
```

Output

True
False

See Also

Reference

[C# Operators](#)

[explicit \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

?: Operator (C# Reference)

The conditional operator (?:) returns one of two values depending on the value of a Boolean expression. The conditional operator is of the form

```
condition ? first_expression : second_expression;
```

Remarks

If condition is **true**, first expression is evaluated and becomes the result; if **false**, the second expression is evaluated and becomes the result. Only one of two expressions is ever evaluated.

Calculations that might otherwise require an if-else construction can be expressed more concisely and elegantly with the conditional operator. For example, to avoid a division by zero in the calculation of the **sin** function you could write either

```
if(x != 0.0) s = Math.Sin(x)/x; else s = 1.0;
```

or, using the conditional operator,

```
s = x != 0.0 ? Math.Sin(x)/x : 1.0;
```

The conditional operator is right-associative, so an expression of the form

```
a ? b : c ? d : e
```

is evaluated as

```
a ? b : (c ? d : e)
```

not

```
(a ? b : c) ? d : e
```

The conditional operator cannot be overloaded.

Example

```
// cs_operator_conditional.cs
using System;
class MainClass
{
    static double sinc(double x)
    {
        return x != 0.0 ? Math.Sin(x)/x : 1.0;
    }

    static void Main()
    {
        Console.WriteLine(sinc(0.2));
        Console.WriteLine(sinc(0.1));
        Console.WriteLine(sinc(0.0));
    }
}
```

Output

```
0.993346653975306
0.998334166468282
1
```

See Also

Reference

[C# Operators](#)

[if-else \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

++ Operator (C# Reference)

The increment operator (++) increments its operand by 1. The increment operator can appear before or after its operand:

Remarks

The first form is a prefix increment operation. The result of the operation is the value of the operand after it has been incremented.

The second form is a postfix increment operation. The result of the operation is the value of the operand before it has been incremented.

Numeric and enumeration types have predefined increment operators. User-defined types can overload the ++ operator. Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_increment.cs
using System;
class MainClass
{
    static void Main()
    {
        double x;
        x = 1.5;
        Console.WriteLine(++x);
        x = 1.5;
        Console.WriteLine(x++);
        Console.WriteLine(x);
    }
}
```

Output

2.5
1.5
2.5

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

-- Operator (C# Reference)

The decrement operator (`--`) decrements its operand by 1. The decrement operator can appear before or after its operand: `--variable` and `variable--`. The first form is a prefix decrement operation. The result of the operation is the value of the operand "after" it has been decremented. The second form is a postfix decrement operation. The result of the operation is the value of the operand "before" it has been decremented.

Remarks

Numeric and enumeration types have predefined decrement operators.

User-defined types can overload the `--` operator (see [operator](#)). Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_decrement.cs
using System;
class MainClass
{
    static void Main()
    {
        double x;
        x = 1.5;
        Console.WriteLine(--x);
        x = 1.5;
        Console.WriteLine(x--);
        Console.WriteLine(x);
    }
}
```

Output

0.5
1.5
0.5

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

&& Operator (C# Reference)

The conditional-AND operator (**&&**) performs a logical-AND of its **bool** operands, but only evaluates its second operand if necessary.

Remarks

The operation

`x && y`

corresponds to the operation

`x & y`

except that if `x` is **false**, `y` is not evaluated (because the result of the AND operation is `false` no matter what the value of `y` may be). This is known as "short-circuit" evaluation.

The conditional-AND operator cannot be overloaded, but overloads of the regular logical operators and operators `true` and `false` are, with certain restrictions, also considered overloads of the conditional logical operators.

Example

In the following example, observe that the expression using **&&** evaluates only the first operand.

```
// cs_operator_logical_and.cs
using System;
class MainClass
{
    static bool Method1()
    {
        Console.WriteLine("Method1 called");
        return false;
    }

    static bool Method2()
    {
        Console.WriteLine("Method2 called");
        return true;
    }

    static void Main()
    {
        Console.WriteLine("regular AND:");
        Console.WriteLine("result is {0}", Method1() & Method2());
        Console.WriteLine("short-circuit AND:");
        Console.WriteLine("result is {0}", Method1() && Method2());
    }
}
```

Output

```
regular AND:
Method1 called
Method2 called
result is False
short-circuit AND:
Method1 called
result is False
```

C# Language Specification

For more information, see the following sections in the [C# Language Specification](#):

- 7.11.2 User-defined conditional logical operators

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

|| Operator (C# Reference)

The conditional-OR operator (||) performs a logical-OR of its **bool** operands, but only evaluates its second operand if necessary.

Remarks

The operation

```
x || y
```

corresponds to the operation

```
x | y
```

except that if *x* is **true**, *y* is not evaluated (because the result of the OR operation is **true** no matter what the value of *y* might be). This is known as "short-circuit" evaluation.

The conditional-OR operator cannot be overloaded, but overloads of the regular logical operators and operators **true** and **false** are, with certain restrictions, also considered overloads of the conditional logical operators.

Example

In the following example, observe that the expression using || evaluates only the first operand.

```
// cs_operator_short_circuit_OR.cs
using System;
class MainClass
{
    static bool Method1()
    {
        Console.WriteLine("Method1 called");
        return true;
    }

    static bool Method2()
    {
        Console.WriteLine("Method2 called");
        return false;
    }

    static void Main()
    {
        Console.WriteLine("regular OR:");
        Console.WriteLine("result is {0}", Method1() | Method2());
        Console.WriteLine("short-circuit OR:");
        Console.WriteLine("result is {0}", Method1() || Method2());
    }
}
```

Output

```
regular OR:
Method1 called
Method2 called
result is True
short-circuit OR:
Method1 called
result is True
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

<< Operator (C# Reference)

The left-shift operator (<<) shifts its first operand left by the number of bits specified by its second operand. The type of the second operand must be an [int](#).

Remarks

If first operand is an [int](#) or [uint](#) (32-bit quantity), the shift count is given by the low-order five bits of second operand.

If first operand is a [long](#) or [ulong](#) (64-bit quantity), the shift count is given by the low-order six bits of second operand.

The high-order bits of first operand are discarded and the low-order empty bits are zero-filled. Shift operations never cause overflows.

User-defined types can overload the << operator (see [operator](#)); the type of the first operand must be the user-defined type, and the type of the second operand must be **int**. When a binary operator is overloaded, the corresponding assignment operator, if any, is also implicitly overloaded.

Example

```
// cs_operator_left_shift.cs
using System;
class MainClass
{
    static void Main()
    {
        int i = 1;
        long lg = 1;
        Console.WriteLine("0x{0:x}", i << 1);
        Console.WriteLine("0x{0:x}", i << 33);
        Console.WriteLine("0x{0:x}", lg << 33);
    }
}
```

Output

0x2
0x2
0x200000000

Comments

Note that `i<<1` and `i<<33` give the same result, because 1 and 33 have the same low-order five bits.

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

>> Operator (C# Reference)

The right-shift operator (>>) shifts its first operand right by the number of bits specified by its second operand.

Remarks

If the first operand is an [int](#) or [uint](#) (32-bit quantity), the shift count is given by the low-order five bits of the second operand (second operand & 0x1f).

If the first operand is a [long](#) or [ulong](#) (64-bit quantity), the shift count is given by the low-order six bits of the second operand (second operand & 0x3f).

If the first operand is an [int](#) or [long](#), the right-shift is an arithmetic shift (high-order empty bits are set to the sign bit). If the first operand is of type [uint](#) or [ulong](#), the right-shift is a logical shift (high-order bits are zero-filled).

User-defined types can overload the >> operator; the type of the first operand must be the user-defined type, and the type of the second operand must be [int](#). For more information, see [operator](#). When a binary operator is overloaded, the corresponding assignment operator, if any, is also implicitly overloaded.

Example

```
// cs_operator_right_shift.cs
using System;
class MainClass
{
    static void Main()
    {
        int i = -1000;
        Console.WriteLine(i >> 3);
    }
}
```

Output

-125

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

== Operator (C# Reference)

For predefined value types, the equality operator (==) returns true if the values of its operands are equal, **false** otherwise. For reference types other than [string](#), == returns **true** if its two operands refer to the same object. For the **string** type, == compares the values of the strings.

Remarks

User-defined value types can overload the == operator (see [operator](#)). So can user-defined reference types, although by default == behaves as described above for both predefined and user-defined reference types. If == is overloaded, != must also be overloaded. Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_equality.cs
using System;
class MainClass
{
    static void Main()
    {
        // Numeric equality: True
        Console.WriteLine((2 + 2) == 4);

        // Reference equality: different objects,
        // same boxed value: False.
        object s = 1;
        object t = 1;
        Console.WriteLine(s == t);

        // Define some strings:
        string a = "hello";
        string b = String.Copy(a);
        string c = "hello";

        // Compare string values of a constant and an instance: True
        Console.WriteLine(a == b);

        // Compare string references;
        // a is a constant but b is an instance: False.
        Console.WriteLine((object)a == (object)b);

        // Compare string references, both constants
        // have the same value, so string interning
        // points to same reference: True.
        Console.WriteLine((object)a == (object)c);
    }
}
```

Output

```
True
False
True
False
True
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

!= Operator (C# Reference)

The inequality operator (!=) returns false if its operands are equal, true otherwise. Inequality operators are predefined for all types, including string and object. User-defined types can overload the != operator.

Remarks

For predefined value types, the inequality operator (!=) returns true if the values of its operands are different, false otherwise. For reference types other than **string**, != returns true if its two operands refer to different objects. For the **string** type, != compares the values of the strings.

User-defined value types can overload the != operator (see [operator](#)). So can user-defined reference types, although by default != behaves as described above for both predefined and user-defined reference types. If != is overloaded, == must also be overloaded. Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_inequality.cs
using System;
class MainClass
{
    static void Main()
    {
        // Numeric inequality:
        Console.WriteLine((2 + 2) != 4);

        // Reference equality: two objects, same boxed value
        object s = 1;
        object t = 1;
        Console.WriteLine(s != t);

        // String equality: same string value, same string objects
        string a = "hello";
        string b = "hello";

        // compare string values
        Console.WriteLine(a != b);

        // compare string references
        Console.WriteLine((object)a != (object)b);
    }
}
```

Output

False
True
False
False

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

<= Operator (C# Reference)

All numeric and enumeration types define a "less than or equal" relational operator (<=) that returns **true** if the first operand is less than or equal to the second, **false** otherwise.

Remarks

User-defined types can overload the <= operator. For more information, see [operator](#). If <= is overloaded, >= must also be overloaded. Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_less_than_or_equal.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(1 <= 1.1);
        Console.WriteLine(1.1 <= 1.1);
    }
}
```

Output

True
True

See Also

Reference

[C# Operators](#)

[explicit \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

>= Operator (C# Reference)

All numeric and enumeration types define a "greater than or equal" relational operator, `>=` that returns **true** if the first operand is greater than or equal to the second, **false** otherwise.

Remarks

User-defined types can overload the `>=` operator. For more information, see [operator](#). If `>=` is overloaded, `<=` must also be overloaded. Operations on integral types are generally allowed on enumeration.

Example

```
// cs_operator_greater_than_or_equal.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine(1.1 >= 1);
        Console.WriteLine(1.1 >= 1.1);
    }
}
```

Output

True
True

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

+= Operator (C# Reference)

The addition assignment operator.

Remarks

An expression using the += assignment operator, such as

```
x += y
```

is equivalent to

```
x = x + y
```

except that `x` is only evaluated once. The meaning of the [+ operator](#) is dependent on the types of `x` and `y` (addition for numeric operands, concatenation for string operands, and so forth).

The += operator cannot be overloaded directly, but user-defined types can overload the [+ operator](#) (see [operator](#)).

The += operator is also used to specify a method that will be called in response to an event; such methods are called event handlers. Since event handlers are encapsulated within a delegate type, the use of the += operator in this context is referred to as *delegate concatenation*. For more information, see [event \(C# Reference\)](#) and [Delegates \(C# Programming Guide\)](#).

Example

```
// cs_operator_addition_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        //addition
        int a = 5;
        a += 6;
        Console.WriteLine(a);

        //string concatenation
        string s = "Micro";
        s += "soft";
        Console.WriteLine(s);
    }
}
```

Output

```
11
Microsoft
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

-= Operator (C# Reference)

The subtraction assignment operator.

Remarks

An expression using the -= assignment operator, such as

```
x -= y
```

is equivalent to

```
x = x - y
```

except that `x` is only evaluated once. The meaning of the [- operator](#) is dependent on the types of `x` and `y` (subtraction for numeric operands, delegate removal for delegate operands, and so forth).

The -= operator cannot be overloaded directly, but user-defined types can overload the [- operator](#) (see [operator](#)).

Example

```
// cs_operator_subtraction_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 5;
        a -= 6;
        Console.WriteLine(a);
    }
}
```

Output

```
-1
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

***= Operator (C# Reference)**

The binary multiplication assignment operator.

Remarks

An expression using the *= assignment operator, such as

```
x *= y
```

is equivalent to

```
x = x * y
```

except that `x` is only evaluated once. The [* operator](#) is predefined for numeric types to perform multiplication.

The *= operator cannot be overloaded directly, but user-defined types can overload the [* operator](#) (see [operator](#)).

Example

```
// cs_operator_multiplication_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 5;
        a *= 6;
        Console.WriteLine(a);
    }
}
```

Output

```
30
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

/= Operator (C# Reference)

The division assignment operator.

Remarks

An expression using the /= assignment operator, such as

```
x /= y
```

is equivalent to

```
x = x / y
```

except that `x` is only evaluated once. The [/ operator](#) is predefined for numeric types to perform division.

The /= operator cannot be overloaded directly, but user-defined types can overload the [/ operator](#) (see [operator](#)). On all compound assignment operators, overloading the binary operator implicitly overloads the equivalent compound assignment.

Example

```
// cs_operator_division_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 5;
        a /= 6;
        Console.WriteLine(a);
        double b = 5;
        b /= 6;
        Console.WriteLine(b);
    }
}
```

Output

```
0
0.8333333333333333
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

%= Operator (C# Reference)

The modulus assignment operator.

Remarks

An expression using the %= assignment operator, such as

```
x %= y
```

is equivalent to

```
x = x % y
```

except that `x` is only evaluated once. The [% operator](#) is predefined for numeric types to compute the remainder after division.

The %= operator cannot be overloaded directly, but user-defined types can overload the [% operator](#) (see [operator \(C# Reference\)](#)).

Example

```
// cs_operator_modulus_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 5;
        a %= 3;
        Console.WriteLine(a);
    }
}
```

Output

```
2
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

&= Operator (C# Reference)

The AND assignment operator.

Remarks

An expression using the **&=** assignment operator, such as

```
x &= y
```

is equivalent to

```
x = x & y
```

except that `x` is only evaluated once. The [& operator](#) performs a bitwise logical AND operation on integral operands and logical AND on **bool** operands.

The **&=** operator cannot be overloaded directly, but user-defined types can overload the binary [& operator](#) (see [operator](#)).

Example

```
// cs_operator_and_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 0x0c;
        a &= 0x06;
        Console.WriteLine("0x{0:x8}", a);
        bool b = true;
        b &= false;
        Console.WriteLine(b);
    }
}
```

Output

```
0x00000004
False
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

|= Operator (C# Reference)

The OR assignment operator.

Remarks

An expression using the |= assignment operator, such as

```
x |= y
```

is equivalent to

```
x = x | y
```

except that `x` is only evaluated once. The [| operator](#) performs a bitwise logical OR operation on integral operands and logical OR on `bool` operands.

The |= operator cannot be overloaded directly, but user-defined types can overload the [| operator](#) (see [operator](#)).

Example

```
// cs_operator_or_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 0x0c;
        a |= 0x06;
        Console.WriteLine("0x{0:x8}", a);
        bool b = true;
        b |= false;
        Console.WriteLine(b);
    }
}
```

Output

```
0x0000000e
True
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

^= Operator (C# Reference)

The exclusive-OR assignment operator.

Remarks

An expression of the form

```
x ^= y
```

is evaluated as

```
x = x ^ y
```

except that `x` is only evaluated once. The [^ operator](#) performs a bitwise exclusive-OR operation on integral operands and logical exclusive-OR on [bool](#) operands.

The ^= operator cannot be overloaded directly, but user-defined types can overload the [! operator](#) (see [operator](#)).

Example

```
// cs_operator_xor_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 0x0c;
        a ^= 0x06;
        Console.WriteLine("0x{0:x8}", a);
        bool b = true;
        b ^= false;
        Console.WriteLine(b);
    }
}
```

Output

```
0x0000000a
True
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

<<= Operator (C# Reference)

The left-shift assignment operator.

Remarks

An expression of the form

```
x <<= y
```

is evaluated as

```
x = x << y
```

except that `x` is only evaluated once. The [<< operator](#) shifts `x` left by the number of bits specified by `y`.

The `<<=` operator cannot be overloaded directly, but user-defined types can overload the [<< operator](#) (see [operator](#)).

Example

```
// cs_operator_left_shift_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 1000;
        a <<= 4;
        Console.WriteLine(a);
    }
}
```

Output

```
16000
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

>>= Operator (C# Reference)

The right-shift assignment operator.

Remarks

An expression of the form

```
x >>= y
```

is evaluated as

```
x = x >> y
```

except that `x` is only evaluated once. The [>> operator](#) shifts `x` right by an amount specified by `y`.

The `>>=` operator cannot be overloaded directly, but user-defined types can overload the [>> operator](#) (see [operator](#)).

Example

```
// cs_operator_right_shift_assignment.cs
using System;
class MainClass
{
    static void Main()
    {
        int a = 1000;
        a >>= 4;
        Console.WriteLine(a);
    }
}
```

Output

```
62
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

-> Operator (C# Reference)

The -> operator combines pointer dereferencing and member access.

Remarks

An expression of the form,

```
x->y
```

(where x is a pointer of type T^* and y is a member of T) is equivalent to,

```
(*x).y
```

The -> operator can be used only in [unmanaged code](#).

The -> operator cannot be overloaded.

Example

```
// cs_operator_dereferencing.cs
// compile with: /unsafe
using System;
struct Point
{
    public int x, y;
}

class MainClass
{
    unsafe static void Main()
    {
        Point pt = new Point();
        Point* pp = &pt;
        pp->x = 123;
        pp->y = 456;
        Console.WriteLine ( "{0} {1}", pt.x, pt.y );
    }
}
```

Output

```
123 456
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

?? Operator (C# Reference)

The ?? operator returns the left-hand operand if it is not null, or else it returns the right operand.

Remarks

A nullable type can contain a value, or it can be undefined. The ?? operator defines the default value to be returned when a nullable type is assigned to a non-nullable type. If you try to assign a nullable type to a non-nullable type without using the ?? operator, you will generate a compile-time error. If you use a cast, and the nullable type is currently undefined, an **InvalidOperationException** exception will be thrown.

For more information, see [Nullable Types \(C# Programming Guide\)](#).

Example

```
// nullable_type_operator.cs
using System;
class MainClass
{
    static int? GetNullableInt()
    {
        return null;
    }

    static string GetStringValue()
    {
        return null;
    }

    static void Main()
    {
        // ?? operator example.
        int? x = null;

        // y = x, unless x is null, in which case y = -1.
        int y = x ?? -1;

        // Assign i to return value of method, unless
        // return value is null, in which case assign
        // default value of int to i.
        int i = GetNullableInt() ?? default(int);

        string s = GetStringValue();
        // ?? also works with reference types.
        // Display contents of s, unless s is null,
        // in which case display "Unspecified".
        Console.WriteLine(s ?? "Unspecified");
    }
}
```

See Also

Reference

[C# Operators](#)

Concepts

[C# Programming Guide](#)

[Nullable Types \(C# Programming Guide\)](#)

Other Resources

[C# Reference](#)

C# Preprocessor Directives

This section discusses the C# language's preprocessor directives:

[#if](#)

[#else](#)

[#elif](#)

[#endif](#)

[#define](#)

[#undef](#)

[#warning](#)

[#error](#)

[#line](#)

[#region](#)

[#endregion](#)

[#pragma](#)

[#pragma warning](#)

[#pragma checksum](#)

While the compiler does not have a separate preprocessor, the directives described in this section are processed as if there was one; these directives are used to aid in conditional compilation. Unlike C and C++ directives, you cannot use these directives to create macros.

A preprocessor directive must be the only instruction on a line.

See Also

Tasks

[Conditional Methods Sample](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#if (C# Reference)

#if lets you begin a conditional directive, testing a symbol or symbols to see if they evaluate to **true**. If they do evaluate to **true**, the compiler evaluates all the code between the **#if** and the nearest **#endif** directive. For example,

```
#define DEBUG
// ...
#if DEBUG
    Console.WriteLine("Debug version");
#endif
```

You can use the operators **==** (equality), **!=** (inequality), **&&** (and), and **||** (or) to evaluate multiple symbols. You can also group symbols and operators with parentheses.

Remarks

#if, along with the **#else**, **#elif**, **#endif**, **#define**, and **#undef** directives, lets you include or exclude code based on the condition of one or more symbols. This can be most useful when compiling code for a debug build or when compiling for a specific configuration.

A conditional directive beginning with a **#if** directive must explicitly be terminated with a **#endif** directive.

#define lets you define a symbol, such that, by using the symbol as the expression passed to the **#if** directive, the expression will evaluate to **true**.

You can also define a symbol with the **/define** compiler option. You can undefine a symbol with **#undef**.

A symbol that you define with **/define** or with **#define** does not conflict with a variable of the same name. That is, a variable name should not be passed to a preprocessor directive and a symbol can only be evaluated by a preprocessor directive.

The scope of a symbol created with **#define** is the file in which it was defined.

Example

```
// preprocessor_if.cs
#define DEBUG
#define VC_V7
using System;
public class MyClass
{
    static void Main()
    {
#if (DEBUG && !VC_V7)
        Console.WriteLine("DEBUG is defined");
#elif (!DEBUG && VC_V7)
        Console.WriteLine("VC_V7 is defined");
#elif (DEBUG && VC_V7)
        Console.WriteLine("DEBUG and VC_V7 are defined");
#else
        Console.WriteLine("DEBUG and VC_V7 are not defined");
#endif
    }
}
```

Output

DEBUG and VC_V7 are defined

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#else (C# Reference)

#else lets you create a compound conditional directive, so that, if none of the expressions in the preceding [#if](#) or (optional) [#elif](#) directives to **true**, the compiler will evaluate all code between **#else** and the subsequent **#endif**.

Remarks

[#endif](#) must be the next preprocessor directive after **#else**. See [#if](#) for an example of how to use **#else**.

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#elif (C# Reference)

#elif lets you create a compound conditional directive. The **#elif** expression will be evaluated if neither the preceding **#if** nor any preceding, optional, **#elif** directive expressions evaluate to **true**. If a **#elif** expression evaluates to **true**, the compiler evaluates all the code between the **#elif** and the next conditional directive. For example:

```
#define VC7
//...
#if debug
    Console.WriteLine("Debug build");
#elif VC7
    Console.WriteLine("Visual Studio 7");
#endif
```

You can use the operators **==** (equality), **!=** (inequality), **&&** (and), and **||** (or), to evaluate multiple symbols. You can also group symbols and operators with parentheses.

Remarks

#elif is equivalent to using:

```
#else
#if
```

Using **#elif** is simpler, because each **#if** requires a **#endif**, whereas a **#elif** can be used without a matching **#endif**.

See [#if](#) for an example of how to use **#elif**.

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#endif (C# Reference)

#endif specifies the end of a conditional directive, which began with the [#if](#) directive. For example,

```
        #define DEBUG
// ...
#if DEBUG
    Console.WriteLine("Debug version");
#endif
```

Remarks

A conditional directive, beginning with a **#if** directive, must explicitly be terminated with a **#endif** directive. See [#if \(C# Reference\)](#) for an example of how to use **#endif**.

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#define (C# Reference)

#define lets you define a symbol, so that by using the symbol as the expression passed to the [#if](#) directive, the expression will evaluate to **true**. For example:

```
# define DEBUG
```

Remarks

Symbols can be used to specify conditions for compilation. You can test for the symbol with either [#if](#) or [#elif](#). You can also use the **conditional** attribute to perform conditional compilation.

You can define a symbol, but you cannot assign a value to a symbol. The **#define** directive must appear in the file before you use any instructions that are not also directives.

You can also define a symbol with the [/define](#) compiler option. You can undefine a symbol with [#undef](#).

A symbol that you define with **/define** or with **#define** does not conflict with a variable of the same name. That is, a variable name should not be passed to a preprocessor directive and a symbol can only be evaluated by a preprocessor directive.

The scope of a symbol created with **#define** is the file in which it was defined.

See [#if](#) for an example of how to use **#define**.

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#undef (C# Reference)

#undef lets you undefine a symbol, such that, by using the symbol as the expression in a **#if** directive, the expression will evaluate to **false**.

A symbol can be defined either with the **#define** directive or the **/define** compiler option. The **#undef** directive must appear in the file before you use any statements that are not also directives.

Example

```
// preprocessor_undef.cs
// compile with: /d:DEBUG
#undef DEBUG
using System;
class MyClass
{
    static void Main()
    {
#if DEBUG
        Console.WriteLine("DEBUG is defined");
#else
        Console.WriteLine("DEBUG is not defined");
#endif
    }
}
```

Output

DEBUG is not defined

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#warning (C# Reference)

#warning lets you generate a level one warning from a specific location in your code. For example:

```
#warning Deprecated code in this method.
```

Remarks

A common use of **#warning** is in a conditional directive. It is also possible to generate a user-defined error with [#error \(C# Reference\)](#).

Example

```
// preprocessor_warning.cs
// CS1030 expected
#define DEBUG
class MainClass
{
    static void Main()
    {
#if DEBUG
#warning DEBUG is defined
#endif
    }
}
```

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#error (C# Reference)

#error lets you generate an error from a specific location in your code. For example:

```
#error Deprecated code in this method.
```

Remarks

A common use of **#error** is in a conditional directive.

It is also possible to generate a user-defined warning with [#warning \(C# Reference\)](#).

Example

```
// preprocessor_error.cs
// CS1029 expected
#define DEBUG
class MainClass
{
    static void Main()
    {
#if DEBUG
#error DEBUG is defined
#endif
    }
}
```

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#line (C# Reference)

#line lets you modify the compiler's line number and (optionally) the file name output for errors and warnings. This example shows how to report two warnings associated with line numbers. The `#line 200` directive forces the line number to be 200 (although the default is #7). The other line (#9) follows the usual sequence as a result of the default `#line` directive.

```
class MainClass
{
    static void Main()
    {
#line 200
        int i;    // CS0168 on line 200
#line default
        char c;  // CS0168 on line 9
    }
}
```

Remarks

The **#line** directive might be used in an automated, intermediate step in the build process. For example, if lines were removed from the original source code file, but you still wanted the compiler to generate output based on the original line numbering in the file, you could remove lines and then simulate the original line numbering with **#line**.

The **#line hidden** directive hides the successive lines from the debugger, such that when the developer steps through the code, any lines between a **#line hidden** and the next **#line** directive (assuming that it is not another **#line hidden** directive) will be stepped over. This option can also be used to allow ASP.NET to differentiate between user-defined and machine-generated code. Although ASP.NET is the primary consumer of this feature, it is likely that more source generators will make use of it.

A **#line hidden** directive does not affect file names or line numbers in error reporting. That is, if an error is encountered in a hidden block, the compiler will report the current file name and line number of the error.

The **#line filename** directive specifies the file name you want to appear in the compiler output. By default, the actual name of the source code file is used. The file name must be in double quotation marks ("").

A source code file can have any number of **#line** directives.

Example 1

The following example shows how the debugger ignores the hidden lines in the code. When you run the example, it will display three lines of text. However, when you set a break point, as shown in the example, and hit F10 to step through the code, you will notice that the debugger ignores the hidden line. Notice also that even if you set a break point at the hidden line, the debugger will still ignore it.

```
// preprocessor_linehidden.cs
using System;
class MainClass
{
    static void Main()
    {
        Console.WriteLine("Normal line #1."); // Set break point here.
#line hidden
        Console.WriteLine("Hidden line.");
#line default
        Console.WriteLine("Normal line #2.");
    }
}
```

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

#region (C# Reference)

#region lets you specify a block of code that you can expand or collapse when using the [outlining](#) feature of the Visual Studio Code Editor. For example:

```
        #region MyClass definition
public class MyClass
{
    static void Main()
    {
    }
}
#endregion
```

Remarks

A **#region** block must be terminated with a [#endregion](#) directive.

A **#region** block cannot overlap with a [#if](#) block. However, a **#region** block can be nested in a **#if** block, and a **#if** block can be nested in a **#region** block.

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#endregion (C# Reference)

#endregion marks the end of a [#region](#) block. For example:

```
        #region MyClass definition
class MyClass
{
    static void Main()
    {
    }
}
#endregion
```

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#pragma (C# Reference)

#pragma is used to give the compiler special instructions for the compilation of the file in which it appears

```
#pragma pragma-name pragma-arguments
```

Parameters

pragma-name

The name of a recognized pragma.

pragma-arguments

Pragma-specific arguments.

See Also

Reference

[C# Preprocessor Directives](#)

[#pragma warning \(C# Reference\)](#)

[#pragma checksum \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#pragma warning (C# Reference)

#pragma warning may be used to enable or disable certain warnings.

```
#pragma warning disable warning-list
#pragma warning restore warning-list
```

Parameters

warning-list

A comma-separated list of warning numbers. Enter the numbers alone, without the "CS" prefix.

When no warning numbers are specified, **disable** disables all warnings and **restore** enables all warnings.

Example

```
// pragma_warning.cs
using System;

#pragma warning disable 414, 3021
[CLSCompliant(false)]
public class C
{
    int i = 1;
    static void Main()
    {
    }
}
#pragma warning restore 3021
[CLSCompliant(false)] // CS3021
public class D
{
    int i = 1;
    public static void F()
    {
    }
}
```

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

#pragma checksum (C# Reference)

May be used to generate checksums for source files to aid with debugging ASP.NET pages.

```
#pragma checksum "filename" "{guid}" "checksum bytes"
```

Parameters

"filename"

The name of the file that requires monitoring for changes or updates.

"{guid}"

The Globally Unique Identifier (GUID) for the file.

"checksum_bytes"

The string of hexadecimal digits representing the bytes of the checksum. Must be an even number of hexadecimal digits. An odd number of digits results in a compile-time warning, and the directive is then ignored.

Remarks

The Visual Studio debugger uses a checksum to ensure that it always finds the right source. The compiler computes the checksum for a source file, and then emits the output to the program database (PDB) file. The debugger then uses the PDB to compare against the checksum that it computes for the source file.

This solution does not work for ASP.NET projects, because the computed checksum is for the generated source file, rather than the .aspx file. To address this problem, **#pragma checksum** provides checksum support for ASP.NET pages.

When you create an ASP.NET project in Visual C#, the generated source file contains a checksum for the .aspx file, from which the source is generated. The compiler then writes this information into the PDB file.

If the compiler encounters no **#pragma checksum** directive in the file, it computes the checksum and writes the value to the PDB file.

Example

```
class TestClass
{
    static int Main()
    {
        #pragma checksum "file.cs" "{3673e4ca-6098-4ec1-890f-8fceb2a794a2}" "{012345678AB}"
    }
    // New checksum
}
```

See Also

Reference

[C# Preprocessor Directives](#)

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

C# Compiler Options

The compiler produces executable (.exe) files, dynamic-link libraries (.dll), or code modules (.netmodule).

Every compiler option is available in two forms: **-option** and **/option**. The documentation only shows the **/option** form.

In This Section

[Command-Line Building](#)

Information about building a Visual C# application from the command line.

[How to: Build from the Command Line](#)

Provides steps for running vsvars32.bat to enable command-line builds.

[Deployment of C# Applications](#)

Describes options for deploying C# applications.

[C# Compiler Options Listed by Category](#)

A categorical listing of the compiler options.

[C# Compiler Options Listed Alphabetically](#)

An alphabetical listing of the compiler options.

[Compiler Errors](#)

A reference to the errors generated by the C# compiler.

Related Sections

[How to: Set Project Properties \(C#, J#\)](#)

Setting properties that govern how your project is compiled, built, and debugged. Includes information about custom build steps in Visual C# projects.

[Default and Custom Builds](#)

Information on build types and configurations.

[Preparing and Managing Builds](#)

Procedures for building within the Visual Studio development environment.

Command-Line Building

You can invoke the C# compiler by typing the name of its executable file (csc.exe) on the command line. If you use the Visual Studio Command Prompt (available as a shortcut on the start menu under Visual Studio Tools), all the necessary environment variables are set for you. Otherwise, you might need to adjust your path in order to invoke csc.exe to be invoked from any subdirectory on your computer. If you do not use the Visual Studio Command Prompt, it is necessary to run vsvars32.bat to set the appropriate environment variables to support command line builds. For more information on vsvars32.bat, see [How to: Build from the Command Line](#).

If you are working on a computer that only has the .NET Framework SDK, you can use the C# compiler at the command line if you use the **SDK Command Prompt**, which is available from the **Microsoft .NET Framework SDK** menu option.

To build from the development environment, see [Preparing and Managing Builds](#).

The csc.exe executable is usually located in the Microsoft.NET\Framework\<version> folder under the system directory. Its location may vary depending on the exact configuration on any individual computer. Multiple versions of this executable will be present on the machine if more than one version of the .NET Framework is installed on the machine. For more information about such installations, see [Installing Multiple Versions of the .NET Framework](#).

This topic covers the following:

[Rules for Command-Line Syntax](#)

[Sample Command Lines](#)

[Differences Between C# Compiler and C++ Compiler Output](#)

Rules for Command-Line Syntax

The C# compiler uses the following rules when interpreting arguments given on the operating system command line:

- Arguments are delimited by white space, which is either a space or a tab.
- The caret character (^) is not recognized as an escape character or delimiter. The character is handled completely by the command-line parser in the operating system before being passed to the argv array in the program.
- A string surrounded by double quotation marks ("string") is interpreted as a single argument, regardless of white space contained within. A quoted string can be embedded in an argument.
- A double quotation mark preceded by a backslash (\") is interpreted as a literal double quotation mark character (").
- Backslashes are interpreted literally, unless they immediately precede a double quotation mark.
- If an even number of backslashes is followed by a double quotation mark, one backslash is placed in the argv array for every pair of backslashes, and the double quotation mark is interpreted as a string delimiter.
- If an odd number of backslashes is followed by a double quotation mark, one backslash is placed in the argv array for every pair of backslashes, and the double quotation mark is "escaped" by the remaining backslash, causing a literal double quotation mark (") to be placed in argv.

Sample Command Lines

- Compiles File.cs producing File.exe:

```
csc File.cs
```

- Compiles File.cs producing File.dll:

```
csc /target:library File.cs
```

- Compiles File.cs and creates My.exe:

```
csc /out:My.exe File.cs
```

- Compiles all of the C# files in the current directory, with optimizations on and defines the DEBUG symbol. The output is File2.exe:

```
csc /define:DEBUG /optimize /out:File2.exe *.cs
```

- Compiles all of the C# files in the current directory producing a debug version of File2.dll. No logo and no warnings are displayed:

```
csc /target:library /out:File2.dll /warn:0 /nologo /debug *.cs
```

- Compiles all of the C# files in the current directory to Something.xyz (a DLL):

```
csc /target:library /out:Something.xyz *.cs
```

Differences Between C# Compiler and C++ Compiler Output

There are no object (.obj) files created as a result of invoking the C# compiler; output files are created directly. As a consequence of this, the C# compiler does not need a linker.

See Also

Reference

[C# Compiler Options Listed Alphabetically](#)

[C# Compiler Options Listed by Category](#)

Other Resources

[C# Compiler Options](#)

How to: Set Environment Variables

The vsvars32.bat file sets the appropriate environment variables to enable command line builds. For more information about vsvars32.bat, see the following Knowledge Base article:

- [Q248802 : Vcvars32.bat Generates Out of Environment Message](#)

If the current version of Visual Studio is installed on a computer that also has a previous version of Visual Studio, you should not run vsvars32.bat or vcvars32.bat from different versions in the same command window.

To run VSVARS32.BAT

1. At the command prompt, change to the Common7\Tools subdirectory of your installation.
2. Run VSVARS32.bat by typing `VSVARS32`.

⚠Caution
VSVARS32.bat can vary from machine to machine. Do not replace a missing or damaged VSVARS32.bat file with a VSVARS32.bat from another machine. Rerun setup to replace the missing file.

See Also

Concepts

[Command-Line Building](#)

Deployment of C# Applications

After you have completed building your C# application, the next step is to distribute it. C# is a .NET language; therefore, distributing any C# executables to other machines requires the .NET Framework to be installed on each execution machine (and possibly other dependencies specific to your application). You have a variety of options available for distributing the .NET Framework. For an overview, see [Redistributing the .NET Framework](#).

Moving completed applications to other computers is generally referred to as deployment. Microsoft's development environment provides mechanisms for deployment; for more information, see [Deploying Applications and Components](#).

If you build and distribute mainly from the command line, you might need to consider other methods of deployment and redistributing dependencies.

See Also

Concepts

[Command-Line Building](#)

C# Compiler Options Listed by Category

The following compiler options are sorted by category. For an alphabetical list, see [C# Compiler Options Listed Alphabetically](#).

Optimization

Option	Purpose
/filealign	Specifies the size of sections in the output file.
/optimize	Enables/disables optimizations.

Output Files

Option	Purpose
/doc	Specifies an XML file where processed documentation comments are to be written.
/out	Specifies the output file.
/pdb	Specifies the file name and location of the .pdb file.
/platform	Specify the output platform.
/target	Specifies the format of the output file using one of four options: /target:exe / target:library / target:module / target:winexe

.NET Framework Assemblies

Option	Purpose
/addmodule	Specifies one or more modules to be part of this assembly.
/delaysign	Instructs the compiler to add the public key but to leave the assembly unsigned.
/keycontainer	Specifies the name of the cryptographic key container.
/keyfile	Specifies the filename containing the cryptographic key.
/lib	Specifies the location of assemblies referenced by means of /reference .
/nostdlib	Instructs the compiler not to import the standard library (mscorlib.dll).
/reference	Imports metadata from a file that contains an assembly.

Debugging/Error Checking

Option	Purpose
/bugreport	Creates a file that contains information that makes it easy to report a bug.
/checked	Specifies whether integer arithmetic that overflows the bounds of the data type will cause an exception at run time.
/debug	Instruct the compiler to emit debugging information.
/errorreport	Sets error reporting behavior.
/fullpaths	Specifies the absolute path to the file in compiler output.
/nowarn	Suppresses the compiler's generation of specified warnings.

/warn	Sets the warning level.
/warnaserror	Promotes warnings to errors.

Preprocessor

Option	Purpose
/define	Defines preprocessor symbols.

Resources

Option	Purpose
/linkresource	Creates a link to a managed resource.
/resource	Embeds a .NET Framework resource into the output file.
/win32icon	Specifies an .ico file to insert into the output file.
/win32res	Specifies a Win32 resource to insert into the output file.

Miscellaneous

Option	Purpose
@	Specifies a response file.
/?	Lists compiler options to stdout.
/baseaddress	Specifies the preferred base address at which to load a DLL.
/codepage	Specifies the code page to use for all source code files in the compilation.
/help	Lists compiler options to stdout.
/langversion	Specifies what version of the language to use.
/main	Specifies the location of the Main method.
/noconfig	Instructs the compiler not to compile with csc.rsp.
/nologo	Suppresses compiler banner information.
/recurse	Searches subdirectories for source files to compile.
/unsafe	Enables compilation of code that uses the unsafe keyword.
/utf8output	Displays compiler output using UTF-8 encoding.

Obsolete Options

/incremental	Enables incremental compilation.
------------------------------	----------------------------------

See Also

Tasks

[How to: Set Environment Variables](#)

Reference

[C# Compiler Options Listed Alphabetically](#)

Other Resources

[C# Compiler Options](#)

C# Compiler Options Listed Alphabetically

The following compiler options are sorted alphabetically. For a categorical list, see [C# Compiler Options Listed By Category](#).

Option	Purpose
@	Reads a response file for more options.
/?	Displays a usage message to stdout.
/addmodule	Links the specified modules into this assembly
/baseaddress	Specifies the base address for the library to be built.
/bugreport	Creates a 'Bug Report' file. This file will be sent along with any crash information if used with /errorreport:prompt or /errorreport:send.
/checked	Causes the compiler to generate overflow checks.
/codepage	Specifies the codepage to use when opening source files.
/debug	Emits debugging information.
/define	Defines conditional compilation symbols.
/delaysign	Delay-signs the assembly using only the public portion of the strong name key.
/doc	Specifies an XML Documentation file to generate.
/errorreport	Specifies how to handle internal compiler errors: prompt, send, or none. The default is none.
/filealign	Specifies the alignment used for output file sections.
/fullpaths	Causes the compiler to generate fully qualified paths.
/help	Displays a usage message to stdout.
/incremental	Enables incremental compilation [obsolete].
/keycontainer	Specifies a strong name key container.
/keyfile	Specifies a strong name key file.
/langversion	Specifies language version modes: ISO-1 or Default.
/lib	Specify additional directories to search in for references.
/linkresource	Links the specified resource to this assembly.
/main	Specifies the type that contains the entry point (ignore all other possible entry points).
/noconfig	Instructs the compiler not to auto include CSC.RSP file.

/nologo	Suppresses compiler copyright message.
/nostdlib	Instructs the compiler not to reference standard library (mscorlib.dll).
/nowarn	Disables specific warning messages
/optimize	Enables/disables optimizations.
/out	Specifies the output file name (default: base name of file with main class or first file).
/pdb	Specifies the file name and location of the .pdb file.
/platform	Limits which platforms this code can run on: x86, Itanium, x64, or anycpu. The default is anycpu.
/recurse	Includes all files in the current directory and subdirectories according to the wildcard specifications.
/reference	References metadata from the specified assembly files.
/resource	Embeds the specified resource.
/target	Specifies the format of the output file using one of four options: /target:exe / /target:library / /target:module / /target:winexe
/unsafe	Allows unsafe code.
/utf8output	Outputs compiler messages in UTF-8 encoding.
/warn	Sets the warning level (0-4).
/warnaserror	Reports specific warnings as errors.
/win32icon	Uses this icon for the output.
/win32res	Specifies the Win32 resource file (.res).

See Also

Tasks

[How to: Set Environment Variables](#)

Reference

[C# Compiler Options Listed by Category](#)

Other Resources

[C# Compiler Options](#)

@ (Specify Response File) (C# Compiler Options)

The @ option lets you specify a file that contains compiler options and source code files to compile.

```
@response_file
```

Arguments

response_file

A file that lists compiler options or source code files to compile.

Remarks

The compiler options and source code files will be processed by the compiler just as if they had been specified on the command line.

To specify more than one response file in a compilation, specify multiple response file options. For example:

```
@file1.rsp @file2.rsp
```

In a response file, multiple compiler options and source code files can appear on one line. A single compiler option specification must appear on one line (cannot span multiple lines). Response files can have comments that begin with the # symbol.

Specifying compiler options from within a response file is just like issuing those commands on the command line. See [Building from the Command Line](#) for more information.

The compiler processes the command options as they are encountered. Therefore, command line arguments can override previously listed options in response files. Conversely, options in a response file will override options listed previously on the command line or in other response files.

C# provides the csc.rsp file, which is located in the same directory as the csc.exe file. See [/noconfig](#) for more information on csc.rsp.

This compiler option cannot be set in the Visual Studio development environment, nor can it be changed programmatically.

Example

The following are a few lines from a sample response file:

```
# build the first output file
/target:exe /out:MyExe.exe source1.cs source2.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/addmodule (Import Metadata) (C# Compiler Options)

This option adds a module that was created with the `target:module` switch to the current compilation.

```
/addmodule:file[;file2]
```

Arguments

file, *file2*

An output file that contains metadata. The file cannot contain an assembly manifest. To import more than one file, separate file names with either a comma or a semicolon.

Remarks

All modules added with **/addmodule** must be in the same directory as the output file at run time. That is, you can specify a module in any directory at compile time but the module must be in the application directory at run time. If the module is not in the application directory at run time, you will get a [TypeLoadException](#).

file cannot contain an assembly. For example, if the output file was created with [/target:module](#), its metadata can be imported with **/addmodule**.

If the output file was created with a **/target** option other than **/target:module**, its metadata cannot be imported with **/addmodule** but can be imported with [/reference](#).

This compiler option is unavailable in Visual Studio; a project cannot reference a module. In addition, this compiler option cannot be changed programmatically.

Example

Compile source file `input.cs` and add metadata from `metad1.netmodule` and `metad2.netmodule` to produce `out.exe`:

```
csc /addmodule:metad1.netmodule;metad2.netmodule /out:out.exe input.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/baseaddress (Specify Base Address of DLL) (C# Compiler Options)

The **/baseaddress** option lets you specify the preferred base address at which to load a DLL.

```
/baseaddress:address
```

Arguments

address

The base address for the DLL. This address can be specified as a decimal, hexadecimal, or octal number.

Remarks

The default base address for a DLL is set by the .NET Framework common language runtime.

Be aware that the lower-order word in this address will be rounded. For example, if you specify 0x11110001, it will be rounded to 0x11110000.

To complete the signing process for a DLL, use -R of SN.EXE.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Click the **Advanced** button.
4. Modify the **DLL Base Address** property.

To set this compiler option programmatically, see [BaseAddress](#).

See Also

Other Resources

[C# Compiler Options](#)

/bugreport (Report a Problem)

Specifies that debug information should be placed in a file for later analysis.

```
/bugreport:file
```

Arguments

file

The name of the file that you want to contain your bug report.

Remarks

The **/bugreport** option specifies that the following information should be placed in *file*:

- A copy of all source code files in the compilation.
- A listing of the compiler options used in the compilation.
- Version information about your compiler, run time, and operating system.
- Referenced assemblies and modules, saved as hexadecimal digits, except assemblies that ship with the .NET Framework and SDK.
- Compiler output, if any.
- A description of the problem, which you will be prompted for.
- A description of how you think the problem should be fixed, which you will be prompted for.

If this option is used with **/errorreport:prompt** or **/errorreport:send**, the information in the file will be sent to Microsoft Corporation.

Because a copy of all source code files will be placed in *file*, you might want to reproduce the suspected code defect in the shortest possible program.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

Notice that contents of the generated file expose source code that could result in inadvertent information disclosure.

See Also

Reference

[/errorreport \(Set Error Reporting Behavior\) \(C# Compiler Options\)](#)

Other Resources

[C# Compiler Options](#)

/checked (Check Integer Arithmetic) (C# Compiler Options)

The **/checked** option specifies whether an integer arithmetic statement that is not in the scope of the [checked](#) or [unchecked](#) keywords and that results in a value outside the range of the data type will cause a run-time exception.

```
/checked[+ | <U>-</U>]
```

Remarks

An integer arithmetic statement that is in the scope of the **checked** or **unchecked** keywords is not subject to the effect of the **/checked** option.

If an integer arithmetic statement that is not in the scope of the **checked** or **unchecked** keywords results in a value outside the range of the data type, and **/checked+ (/checked)** is used in the compilation, then that statement will cause an exception at run time. If **/checked-** is used in the compilation, then that statement will not cause an exception at run time.

The default value for this option is **/checked-**, so you can get the same effect by simply omitting the option altogether. One scenario for using **/checked-** is in building large applications: sometimes automated tools are used to build such applications, and the tools might automatically set **/checked** to **+**: in this situation, you can override the global default by specifying **/checked-**.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For more information, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Click the **Advanced** button.
4. Modify the **Check for arithmetic overflow/underflow** property.

To access this compiler option programmatically, see [CheckForOverflowUnderflow](#).

Example

Compile `t2.cs` and specify that an integer arithmetic statement that is not in the scope of the **checked** or **unchecked** keywords and results in a value outside the range of the data type will cause an exception at run time.

```
csc t2.cs /checked
```

See Also

Other Resources

[C# Compiler Options](#)

/codepage (Specify Code Page for Source Code Files) (C# Compiler Options)

This option specifies which codepage to use during compilation if the required page is not the current default codepage for the system.

```
/codepage:id
```

Arguments

id

The id of the code page to use for all source code files in the compilation.

Remarks

If you compile one or more source code files that were not created to use the default code page on your computer, you can use the **/codepage** option to specify which code page should be used. **/codepage** applies to all source code files in your compilation.

If the source code files were created with the same codepage that is in effect on your computer or if the source code files were created with UNICODE or UTF-8, you need not use **/codepage**.

See [GetCPInfo](#) for information on how to find which code pages are supported on your system.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

See Also

Other Resources

[C# Compiler Options](#)

/debug (Emit Debugging Information) (C# Compiler Options)

The **/debug** option causes the compiler to generate debugging information and place it in the output file or files.

```
/debug[+ | <U>-</U>]  
/debug:{<U>full</U> | pdbonly}
```

Arguments

+ | -

Specifying **+**, or just **/debug**, causes the compiler to generate debugging information and place it in a program database (.pdb file). Specifying **-**, which is in effect if you do not specify **/debug**, causes no debug information to be created.

full | pdbonly

Specifies the type of debugging information generated by the compiler. The full argument, which is in effect if you do not specify **/debug:pdbonly**, enables attaching a debugger to the running program. Specifying **pdbonly** allows source code debugging when the program is started in the debugger but will only display assembler when the running program is attached to the debugger.

Remarks

Use this option to create debug builds. If **/debug**, **/debug+**, or **/debug:full** is not specified, you will not be able to debug the output file of your program.

If you use **/debug:full**, be aware that there is some impact on the speed and size of JIT optimized code and a small impact on code quality with **/debug:full**. We recommend **/debug:pdbonly** or no PDB for generating release code.

Note

One difference between **/debug:pdbonly** and **/debug:full** is that with **/debug:full** the compiler emits a [DebuggableAttribute](#), which is used to tell the JIT compiler that debug information is available. Therefore, you will get an error if your code contains the **DebuggableAttribute** set to false if you use **/debug:full**.

For more information on how to configure the debug performance of an application, see [Making an Image Easier to Debug](#).

To change the location of the .pdb file, see [/pdb \(Specify Debug Symbol File\) \(C# Compiler Options\)](#).

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For more information, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Click the **Advanced** button.
4. Modify the **Debug Info** property.

For information on how to set this compiler option programmatically, see [DebugSymbols](#).

Example

Place debugging information in output file `app.pdb`:

```
csc /debug /out:app.pdb test.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/define (Preprocessor Definition) (C# Compiler Options)

The **/define** option defines *name* as a symbol in your program.

```
/define:name[;name2]
```

Arguments

name, *name2*

The name of one or more symbols that you want to define.

Remarks

The **/define** option has the same effect as using a **#define** preprocessor directive in your source file. A symbol remains defined until an **#undef** directive in the source file removes the definition or the compiler reaches the end of the file.

You can use symbols created by this option with **#if**, **#else**, **#elif**, and **#endif** to compile source files conditionally.

/d is the short form of **/define**.

You can define multiple symbols with **/define** by using a semicolon or comma to separate symbol names. For example:

```
/define:DEBUG;TUESDAY
```

The C# compiler itself defines no symbols or macros that you can use in your source code; all symbol definitions must be user-defined.

Note

The C# **#define** does not allow a symbol to be given a value, as in languages such as C++. For example, **#define** cannot be used to create a macro or to define a constant. If you need to define a constant, use an **enum** variable. If you want to create a C++ style macro, consider alternatives such as generics. Since macros are notoriously error-prone, C# disallows their use but provides safer alternatives.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For more information, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Modify the **Conditional Compilation Symbols** property.

For information on how to set this compiler option programmatically, see [DefineConstants](#).

Example

```
// preprocessor_define.cs
// compile with: /define:xx
// or uncomment the next line
// #define xx
using System;
public class Test
{
    public static void Main()
    {
        #if (xx)
            Console.WriteLine("xx defined");
        #else
            Console.WriteLine("xx not defined");
        #endif
    }
}
```

See Also

Other Resources

[C# Compiler Options](#)

/delaysign (Delay Sign the Assembly) (C# Compiler Options)

This option causes the compiler to reserve space in the output file so that a digital signature can be added later.

```
/delaysign[ + | - ]
```

Arguments

+ | -

Use **/delaysign-** if you want a fully signed assembly. Use **/delaysign+** if you only want to place the public key in the assembly. The default is **/delaysign-**.

Remarks

The **/delaysign** option has no effect unless used with [/keyfile](#) or [/keycontainer](#).

When you request a fully signed assembly, the compiler hashes the file that contains the manifest (assembly metadata) and signs that hash with the private key. The resulting digital signature is stored in the file that contains the manifest. When an assembly is delay signed, the compiler does not compute and store the signature, but reserves space in the file so the signature can be added later.

For example, using **/delaysign+** allows a tester to put the assembly in the global cache. After testing, you can fully sign the assembly by placing the private key in the assembly using the [Assembly Linker](#) utility.

For more information, see [Creating and Using Strong-Named Assemblies](#) and [Delay Signing an Assembly](#).

To set this compiler option in the Visual Studio development environment

1. Open the **Properties** page for the project. For more information, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Signing** property page.
3. Modify the **Delay sign only** property.

For information on how to set this compiler option programmatically, see [DelaySign](#).

See Also

Other Resources

[C# Compiler Options](#)

/doc (Process Documentation Comments) (C# Compiler Options)

The **/doc** option allows you to place documentation comments in an XML file.

```
/doc:file
```

Arguments

file

The output file for XML, which is populated with the comments in the source code files of the compilation.

Remarks

In source code files, documentation comments that precede the following can be processed and added to the XML file:

- Such user-defined types as a [class](#), [delegate](#), or [interface](#)
- Such members as a field, [event](#), [property](#), or method

The source code file that contains Main is output first into the XML.

To use the generated .xml file for use with the [IntelliSense](#) feature, let the file name of the .xml file be the same as the assembly you want to support and then make sure the .xml file is in the same directory as the assembly. Thus, when the assembly is referenced in the Visual Studio project, the .xml file is found as well. See [Supplying Code Comments](#) and for more information.

Unless you compile with [/target:module](#), *file* will contain <assembly> </assembly> tags specifying the name of the file containing the assembly manifest for the output file of the compilation.

Note

The /doc option applies to all input files; or, if set in the Project Settings, all files in the project. To disable warnings related to documentation comments for a specific file or section of code, use [#pragma warning](#).

See [Recommended Tags for Documentation Comments](#) for ways to generate documentation from comments in your code.

See [XML Documentation Sample](#) for a sample.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Modify the **XML Documentation File** property.

For information on how to set this compiler option programmatically, see [DocumentationFile](#).

See Also

Tasks

[XML Documentation Sample](#)

Other Resources

[C# Compiler Options](#)

/errorreport (Set Error Reporting Behavior) (C# Compiler Options)

This option provides a convenient way to report a C# internal compiler error to Microsoft.

```
/errorreport:{ none | prompt | queue | send }
```

Arguments

none

Reports about internal compiler errors will not be collected or sent to Microsoft.

prompt

Prompts you to send a report when you receive an internal compiler error. **prompt** is the default when compiling an application in the development environment.

queue

Queues the error report. When you log in with administrator privileges, a pop-up window opens and enables you to report any failures since the last time you were logged in. You will not be prompted to send reports for failures more than once every three days. **queue** is the default when compiling an application at the command line.

send

Automatically sends reports of internal compiler errors to Microsoft. To enable this option, you must first agree to Microsoft's data collection policy. The first time you specify **/errorreport:send** on a computer, a compiler message will refer you to a Web site that contains Microsoft's data collection policy.

Remarks

An internal compiler error (ICE) results when the compiler cannot process a source code file. When an ICE occurs, the compiler does not produce an output file or any useful diagnostic that you can use to fix your code.

In previous releases, when you got an ICE, you were encouraged to call Microsoft technical support to report the problem. With **/errorreport**, you can provide ICE information directly to the Visual C# team. Your error reports can help improve future compiler releases.

A user's ability to send reports depends on machine and user policy permissions.

For more information on error debugger, see [Description of the Dr. Watson for Windows \(Drwtsn32.exe\) Tool](#).

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For more information, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Click the **Advanced** button.
4. Modify the **Internal Compiler Error Reporting** property.

For information on how to set this compiler option programmatically, see [ErrorReport](#).

See Also

Other Resources

[C# Compiler Options](#)

/filealign (Specify Section Alignment) (C# Compiler Options)

The **/filealign** option lets you specify the size of sections in your output file.

```
/filealign:number
```

Arguments

number

A value that specifies the size of sections in the output file. Valid values are 512, 1024, 2048, 4096, and 8192. These values are in bytes.

Remarks

Each section will be aligned on a boundary that is a multiple of the **/filealign** value. There is no fixed default. If **/filealign** is not specified, the common language runtime picks a default at compile time.

By specifying the section size, you affect the size of the output file. Modifying section size may be useful for programs that will run on smaller devices.

Use [DUMPBIN](#) to see information about sections in your output file.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Click the **Advanced** button.
4. Modify the **File Alignment** property.

For information on how to set this compiler option programmatically, see [FileAlignment](#).

See Also

Other Resources

[C# Compiler Options](#)

/fullpaths (Specify Fully Qualified Path in Compiler Output)

(C# Compiler Options)

The **/fullpaths** option causes the compiler to specify the full path to the file when listing compilation errors and warnings.

```
/fullpaths
```

Remarks

By default, errors and warnings that result from compilation specify the name of the file in which an error was found. The **/fullpaths** option causes the compiler to specify the full path to the file.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

See Also

Other Resources

[C# Compiler Options](#)

/help, /? (Compiler Command-Line Help) (C# Compiler Options)

This option sends a listing of compiler options, and a brief description of each option, to stdout.

```
/help  
/?
```

Remarks

If this option is included in a compilation, no output file will be created and no compilation will take place.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

See Also

Other Resources

[C# Compiler Options](#)

/keycontainer (Specify Strong Name Key Container) (C# Compiler Options)

Specifies the name of the cryptographic key container.

```
/keycontainer:string
```

Arguments

Term	Definition
string	The name of the strong name key container.

Remarks

When the /keycontainer option is used, the compiler creates a sharable component by inserting a public key from the specified container into the assembly manifest and signing the final assembly with the private key. To generate a key file, type sn -k *file* at the command line. sn -i installs the key pair into a container.

If you compile with /target:module, the name of the key file is held in the module and incorporated into the assembly when you compile this module into an assembly with /addmodule.

You can also specify this option as a custom attribute (System.Reflection.AssemblyKeyNameAttribute) in the source code for any Microsoft intermediate language (MSIL) module.

You can also pass your encryption information to the compiler with /keyfile. Use /delaysign if you want the public key added to the assembly manifest but want to delay signing the assembly until it has been tested.

For more information, see [Creating and Using Strong-Named Assemblies](#) and [Delay Signing an Assembly](#).

To set this compiler option in the Visual Studio development environment

- This compiler option is not available in the Visual Studio development environment.

You can programmatically access this compiler option with [AssemblyKeyContainerName](#).

See Also

Other Resources

[C# Compiler Options](#)

/keyfile (Specify Strong Name Key File) (C# Compiler Options)

Specifies the filename containing the cryptographic key.

```
/keyfile:file
```

Arguments

Term	Definition
<i>file</i>	The name of the file containing the strong name key.

Remarks

When this option is used, the compiler inserts the public key from the specified file into the assembly manifest and then signs the final assembly with the private key. To generate a key file, type `sn -k file` at the command line.

If you compile with **/target:module**, the name of the key file is held in the module and incorporated into the assembly that is created when you compile an assembly with [/addmodule](#).

You can also pass your encryption information to the compiler with [/keycontainer](#). Use [/delaysign](#) if you want a partially signed assembly.

In case both `/keyfile` and `/keycontainer` are specified (either by command line option or by custom attribute) in the same compilation, the compiler will first try the key container. If that succeeds, then the assembly is signed with the information in the key container. If the compiler does not find the key container, it will try the file specified with `/keyfile`. If that succeeds, the assembly is signed with the information in the key file and the key information will be installed in the key container (similar to `sn -i`) so that on the next compilation, the key container will be valid.

Note that a key file might contain only the public key.

For more information, see [Creating and Using Strong-Named Assemblies](#) and [Delay Signing an Assembly](#).

To set this compiler option in the Visual Studio development environment

1. Open the **Properties** page for the project. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Signing** property page.
3. Modify the **Choose a strong name key file** property.

You can programmatically access this compiler option with [AssemblyOriginatorKeyFile](#).

See Also

Other Resources

[C# Compiler Options](#)

/langversion (Conformant Syntax) (C# Compiler Options)

Causes the compiler to only accept syntax that is included in the ISO/IEC 23270:2003 C# language specification.

```
/langversion:option
```

Arguments

option

If *option* is **ISO-1** the compiler will give an error for any syntax that is not in the ISO/IEC 23270:2003 C# language specification. If *option* is **default** the compiler will accept all valid language syntax. **/langversion:default** is the default.

Remarks

Version 1.0 of the C# specification represents the functionality available with **/langversion:ISO-1**.

<http://msdn.microsoft.com/vcsharp/programming/language/default.aspx> contains all specifications as Microsoft Word files.

Metadata referenced by your C# application is not subject to **/langversion** compiler option.

Each version of the C# compiler contains extensions to the language specification, so **/langversion** does not give you the equivalent functionality of a previous version of the compiler.

Regardless of which **/langversion** setting you use, you will use the current version of the common language runtime to create your .exe or .dll.

One exception is friend assemblies and [/moduleassemblyname \(Specify Friend Assembly for Module\) \(C# Compiler Option\)](#), which work under **/langversion:ISO-1**.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Click the **Advanced** button.
4. Modify the **Language Version** property.

For information on how to set this compiler option programmatically, see [LanguageVersion](#).

See Also

Other Resources

[C# Compiler Options](#)

/lib (Specify Assembly Reference Locations) (C# Compiler Options)

The **/lib** option specifies the location of assemblies referenced by means of the [/reference \(Import Metadata\) \(C# Compiler Options\)](#) option.

```
/lib:dir1[,dir2]
```

Arguments

dir1

A directory for the compiler to look in if a referenced assembly is not found in the current working directory (the directory from which you are invoking the compiler) or in the common language runtime's system directory.

dir2

One or more additional directories to search in for assembly references. Separate additional directory names with a comma, and without white space between them.

Remarks

The compiler searches for assembly references that are not fully qualified in the following order:

1. Current working directory. This is the directory from which the compiler is invoked.
2. The common language runtime system directory.
3. Directories specified by **/lib**.
4. Directories specified by the LIB environment variable.

Use **/reference** to specify an assembly reference.

/lib is additive; specifying it more than once appends to any prior values.

An alternative to using **/lib** is to copy into the working directory any required assemblies; this will allow you to simply pass the assembly name to **/reference**. You can then delete the assemblies from the working directory. Since the path to the dependent assembly is not specified in the assembly manifest, the application can be started on the target computer and will find and use the assembly in the global assembly cache.

Because the compiler can reference the assembly does not imply the common language runtime will be able to find and load the assembly at runtime. See [How the Runtime Locates Assemblies](#) for details on how the runtime searches for referenced assemblies.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Property Pages** dialog box. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **References Path** property page.
3. Modify the contents of the list box.

For information on how to set this compiler option programmatically, see [ReferencePath](#).

Example

Compile t2.cs to create an .exe file. The compiler will look in the working directory and in the root directory of the C drive for assembly references.

```
csc /lib:c:\ /reference:t2.dll t2.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/linkresource (Link to .NET Framework Resource) (C# Compiler Options)

Creates a link to a .NET Framework resource in the output file; the resource file is not placed in the output file, in contrast to the [/resource](#) option which does embed a resource file in the output file.

```
/linkresource:filename[,identifier[,accessibility-modifier]]
```

Arguments

filename

The .NET Framework resource file to which you want to link from the assembly.

identifier (optional)

The logical name for the resource; the name used to load the resource. The default is the name of the file.

accessibility-modifier (optional)

The accessibility of the resource: public or private. The default is public.

Remarks

By default, linked resources are public in the assembly when created with the C# compiler. To make the resources private, specify **private** as the accessibility modifier. No other modifier other than **public** or **private** is allowed.

/linkresource requires one of the [/target](#) options other than **/target:module**.

If *filename* is a .NET Framework resource file created, for example, by [Resgen.exe](#) or in the development environment, it can be accessed with members in the [System.Resources](#) namespace. For more information, see [System.Resources.ResourceManager](#). For all other resources, use the **GetManifestResource*** methods in the [Assembly](#) class to access the resource at run time.

The file specified in *filename* can be any format. For example, you may want to make a native DLL part of the assembly, so it can be installed into the global assembly cache and accessed from managed code in the assembly. The second of the following examples shows how to do this. You can do the same thing in the Assembly Linker. The third of the following examples shows how to do this. For more information, see [Assembly Linker \(Al.exe\)](#) and [Working with Assemblies and the Global Assembly Cache](#).

/linkres is the short form of **/linkresource**.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

Example

Compile `in.cs` and link to resource file `rf.resource`:

```
csc /linkresource:rf.resource in.cs
```

Compile `A.cs` into a DLL, link to a native DLL `N.dll`, and put the output in the Global Assembly Cache (GAC). In this example, both `A.dll` and `N.dll` will reside in the GAC.

```
csc /linkresource:N.dll /t:library A.cs  
gacutil -i A.dll
```

This example does the same thing as the preceding one, but by using Assembly Linker options.

```
csc /t:module A.cs  
al /out:A.dll A.netmodule /link:N.dll  
gacutil -i A.dll
```

[Assembly Linker \(Al.exe\)](#)

Other Resources

[C# Compiler Options](#)

[Working with Assemblies and the Global Assembly Cache](#)

/main (Specify Location of Main Method) (C# Compiler Options)

This option specifies the class that contains the entry point to the program, if more than one class contains a **Main** method.

```
/main:class
```

Arguments

class

The type that contains the **Main** method.

Remarks

If your compilation includes more than one type with a [Main](#) method, you can specify which type contains the **Main** method that you want to use as the entry point into the program.

This option is for use when compiling an .exe file.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Application** property page.
3. Modify the **Startup object** property.

To set this compiler option programmatically, see [StartupObject](#).

Example

Compile `t2.cs` and `t3.cs`, specifying that the **Main** method will be found in `Test2`:

```
csc t2.cs t3.cs /main:Test2
```

See Also

Other Resources

[C# Compiler Options](#)

/moduleassemblyname (Specify Friend Assembly for Module) (C# Compiler Option)

Specifies an assembly whose non-public types a .netmodule can access.

```
/moduleassemblyname:assembly_name
```

Arguments

assembly_name

The name of the assembly whose non-public types the .netmodule can access.

Remarks

/moduleassemblyname should be used when building a .netmodule, and where the following conditions are true:

- The .netmodule needs access to non-public types in an existing assembly.
- You know the name of the assembly into which the .netmodule will be built.
- The existing assembly has granted friend assembly access to the assembly into which the .netmodule will be built.

For more information on building a .netmodule, see

[/target:module \(Create Module to Add to Assembly\) \(C# Compiler Options\)](#).

For more information on friend assemblies, see [Friend Assemblies \(C# Programming Guide\)](#).

This option is not available from within the development environment; it is only available when compiling from the command line.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

Example

This sample builds an assembly with a private type, and that gives friend assembly access to an assembly called csman_an_assembly.

```
// moduleassemblyname_1.cs
// compile with: /target:library
using System;
using System.Runtime.CompilerServices;

[assembly:InternalsVisibleTo ("csman_an_assembly")]

class An_Internal_Class
{
    public void Test()
    {
        Console.WriteLine("An_Internal_Class.Test called");
    }
}
```

This sample builds a .netmodule that accesses a non-public type in the assembly moduleassemblyname_1.dll. By knowing that this .netmodule will be built into an assembly called csman_an_assembly, we can specify **/moduleassemblyname**, allowing the .netmodule to access non-public types in an assembly that has granted friend assembly access to csman_an_assembly.

```
// moduleassemblyname_2.cs
// compile with: /moduleassemblyname:csman_an_assembly /target:module /reference:moduleasse
mblyname_1.dll
class B {
    public void Test() {
        An_Internal_Class x = new An_Internal_Class();
        x.Test();
    }
}
```

```
}  
}
```

This code sample builds the assembly `csman_an_assembly`, referencing the previously-built assembly and `.netmodule`.

```
// csman_an_assembly.cs  
// compile with: /addmodule:moduleassemblyname_2.netmodule /reference:moduleassemblyname_1.  
dll  
class A {  
    public static void Main() {  
        B bb = new B();  
        bb.Test();  
    }  
}
```

Output

`An_Internal_Class.Test` called

See Also

Other Resources

[C# Compiler Options](#)

/noconfig (Ignore csc.rsp) (C# Compiler Options)

The **/noconfig** option tells the compiler not to compile with the csc.rsp file, which is located in and loaded from the same directory as the csc.exe file.

```
/noconfig
```

Remarks

The csc.rsp file references all the assemblies shipped with the .NET Framework. The actual references that the Visual Studio .NET development environment includes depend on the project type.

You can modify the csc.rsp file and specify additional compiler options that should be included in every compilation from the command line with csc.exe (except the **/noconfig** option).

The compiler processes the options passed to the **csc** command last. Therefore, any option on the command line overrides the setting of the same option in the csc.rsp file.

If you do not want the compiler to look for and use the settings in the csc.rsp file, specify **/noconfig**.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

See Also

Other Resources

[C# Compiler Options](#)

/nologo (Suppress Banner Information) (C# Compiler Options)

The **/nologo** option suppresses display of the sign-on banner when the compiler starts up and display of informational messages during compiling.

```
/nologo
```

Remarks

This option is not available from within the development environment; it is only available when compiling from the command line.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

See Also

Other Resources

[C# Compiler Options](#)

/nostdlib (Do Not Import Standard Library) (C# Compiler Options)

/nostdlib prevents the import of mscorlib.dll, which defines the entire System namespace.

```
/nostdlib[<U>+</U> | -]
```

Remarks

Use this option if you want to define or create your own System namespace and objects.

If you do not specify **/nostdlib**, mscorlib.dll will be imported into your program (same as specifying **/nostdlib-**). Specifying **/nostdlib** is the same as specifying **/nostdlib+**.

To set this compiler option in the Visual Studio development environment

1. Open the **Properties** page for the project. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** properties page.
3. Click the **Advanced** button.
4. Modify the **Do not reference mscorlib.dll** property.

For information on how to set this compiler option programmatically, see [NoStdLib](#).

See Also

Other Resources

[C# Compiler Options](#)

/nowarn (Suppress Specified Warnings) (C# Compiler Options)

The **/nowarn** option lets you suppress the compiler from showing one or more warnings. Separate multiple warning numbers with a comma.

```
/nowarn:number1[,number2,...]
```

Arguments

number1, *number2*

Warning number(s) that you want the compiler to suppress.

Remarks

You should only specify the numeric part of the warning identifier. For example, if you want to suppress CS0028, you could specify `/nowarn:28`.

The compiler will silently ignore warning numbers passed to **/nowarn** that were valid in previous releases, but that have been removed from the compiler. For example, CS0679 was valid in the compiler in Visual Studio .NET 2002 but was subsequently removed.

To set this compiler option in the Visual Studio development environment

1. Open the **Properties** page for the project. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Modify the **Suppress Warnings** property.

For information on how to set this compiler option programmatically, see [DelaySign](#).

See Also

Other Resources

[C# Compiler Options](#)

/optimize (Enable/Disable Optimizations) (C# Compiler Options)

The **/optimize** option enables or disables optimizations performed by the compiler to make your output file smaller, faster, and more efficient.

```
/optimize[+ | <U>-</U>]
```

Remarks

/optimize also tells the common language runtime to optimize code at runtime.

By default, optimizations are disabled. Specify **/optimize+** to enable optimizations.

When building a module to be used by an assembly, use the same **/optimize** settings as those of the assembly.

/o is the short form of **/optimize**.

It is possible to combine the **/optimize** and [/debug](#) options.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Modify the **Optimize Code** property.

For information on how to set this compiler option programmatically, see [Optimize](#).

Example

Compile `t2.cs` and enable compiler optimizations:

```
csc t2.cs /optimize
```

See Also

Other Resources

[C# Compiler Options](#)

/out (Set Output File Name) (C# Compiler Options)

The **/out** option specifies the name of the output file.

```
/out:filename
```

Arguments

filename

The name of the output file created by the compiler.

Remarks

On the command line, it is possible to specify multiple output files for your compilation. The compiler expects to find one or more source code files following the **/out** option. Then, all source code files will be compiled into the output file specified by that **/out** option.

Specify the full name and extension of the file you want to create.

If you do not specify the name of the output file:

- An .exe will take its name from the source code file that contains the **Main** method.
- A .dll or .netmodule will take its name from the first source code file.

A source code file used to compile one output file cannot be used in the same compilation for the compilation of another output file.

When producing multiple output files in a command-line compilation, keep in mind that only one of the output files can be an assembly and that only the first output file specified (implicitly or explicitly with **/out**) can be the assembly.

Any modules produced as part of a compilation become files associated with any assembly also produced in the compilation. Use [ildasm.exe](#) to view the assembly manifest to see the associated files.

The **/out** compiler option is required in order for an exe to be the target of a friend assembly. For more information see [Friend Assemblies \(C# Programming Guide\)](#).

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Application** property page.
3. Modify the **Assembly name** property.

To set this compiler option programmatically: the [OutputFileName](#) is a read-only property, which is determined by a combination of the project type (exe, library, and so forth) and the assembly name. Modifying one or both of these properties will be necessary to set the output file name.

Example

Compile `t.cs` and create output file `t.exe`, as well as build `t2.cs` and create module output file `mymodule.netmodule`:

```
csc t.cs /out:mymodule.netmodule /target:module t2.cs
```

See Also

Reference

[Friend Assemblies \(C# Programming Guide\)](#)

Other Resources

[C# Compiler Options](#)

/pdb (Specify Debug Symbol File) (C# Compiler Options)

The **/pdb** compiler option specifies the name and location of the debug symbols file.

```
/pdb:filename
```

Arguments

filename

The name and location of the debug symbols file.

Remarks

When you specify [/debug \(Emit Debugging Information\) \(C# Compiler Options\)](#), the compiler will create a .pdb file in the same directory where the compiler will create the output file (.exe or .dll) with a file name that is the same as the name of the output file.

/pdb allows you to specify a non-default file name and location for the .pdb file.

This compiler option cannot be set in the Visual Studio development environment, nor can it be changed programmatically.

Example

Compile `t.cs` and create a .pdb file called `tt.pdb`:

```
csc /debug /pdb:tt t.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/platform (Specify Output Platform) (C# Compiler Options)

Specifies which version of the common language runtime (CLR) can run the assembly.

```
/platform:string
```

Parameters

string

x86, Itanium, x64, or anycpu (default).

Remarks

- x86 compiles your assembly to be run by the 32-bit, x86-compatible common language runtime.
- Itanium compiles your assembly to be run by the 64-bit common language runtime on a computer with an Itanium processor.
- x64 compiles your assembly to be run by the 64-bit common language runtime on a computer that supports the AMD64 or EM64T instruction set.
- anycpu (default) compiles your assembly to run on any platform.

On a 64-bit Windows operating system:

- Assemblies compiled with **/platform:x86** will execute on the 32 bit CLR running under WOW64.
- Executables compiled with the **/platform:anycpu** will execute on the 64 bit CLR.
- DLLs compiled with the **/platform:anycpu** will execute on the same CLR as the process into which it is being loaded.

For more information about developing an application to run on a Windows 64-bit operating system, see [64-bit Applications](#).

To set this compiler option in the Visual Studio development environment

1. Open the **Properties** page for the project. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Modify the **Platform target** property.

Note **/platform** is not available in the development environment in Visual C# Express.

For information on how to set this compiler option programmatically, see [PlatformTarget](#).

Example

The following example shows how to use the **/platform** option to specify that the application should only be run by the 64-bit CLR on a 64-bit Windows operating system for Itanium.

```
csc /platform:Itanium myItanium.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/recurse (Find Source Files in Subdirectories) (C# Compiler Options)

The /recurse option enables you to compile source code files in all child directories of either the specified directory (*dir*) or of the project directory.

```
/recurse:[dir\]file
```

Arguments

dir (optional)

The directory in which you want the search to begin. If this is not specified, the search begins in the project directory.

file

The file(s) to search for. Wildcard characters are allowed.

Remarks

The **/recurse** option lets you compile source code files in all child directories of either the specified directory (*dir*) or of the project directory.

You can use wildcards in a file name to compile all matching files in the project directory without using **/recurse**.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

Example

Compiles all C# files in the current directory:

```
csc *.cs
```

Compiles all of the C# files in the dir1\dir2 directory and any directories below it and generates dir2.dll:

```
csc /target:library /out:dir2.dll /recurse:dir1\dir2\*.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/reference (Import Metadata) (C# Compiler Options)

The **/reference** option causes the compiler to import [public \(C# Reference\)](#) type information in the specified file into the current project, thus enabling you to reference metadata from the specified assembly files.

```
/reference:[alias=]filename  
/reference:filename
```

Arguments

filename

The name of a file that contains an assembly manifest. To import more than one file, include a separate **/reference** option for each file.

alias

A valid C# identifier that will represent a root namespace that will contain all namespaces in the assembly.

Remarks

To import from more than one file, include a **/reference** option for each file.

The files you import must contain a manifest; the output file must have been compiled with one of the [/target \(Specify Output File Format\) \(C# Compiler Options\)](#) options other than [/target:module \(Create Module to Add to Assembly\) \(C# Compiler Options\)](#).

/r is the short form of **/reference**.

Use [/addmodule \(Import Metadata\) \(C# Compiler Options\)](#) to import metadata from an output file that does not contain an assembly manifest.

If you reference an assembly (Assembly A) that references another assembly (Assembly B), you will need to reference Assembly B if:

- A type you use from Assembly A inherits from a type or implements an interface from Assembly B.
- You invoke a field, property, event, or method that has a return type or parameter type from Assembly B.

Use [/lib \(Specify Assembly Reference Locations\) \(C# Compiler Options\)](#) to specify the directory in which one or more of your assembly references is located. The **/lib** topic also discusses the directories in which the compiler searches for assemblies.

In order for the compiler to recognize a type in an assembly, and not in a module, it needs to be forced to resolve the type, which you can do by defining an instance of the type. There are other ways to resolve type names in an assembly for the compiler: for example, if you inherit from a type in an assembly, the type name will then be recognized by the compiler.

Sometimes it is necessary to reference two different versions of the same component from within one assembly. To do this, use the alias suboption on the **/reference** switch for each file to distinguish between the two files. This alias will be used as a qualifier for the component name, and will resolve to the component in one of the files.

The csc response (.rsp) file, which references commonly used .NET Framework assemblies, is used by default. Use [/noconfig \(Ignore csc.rsp\) \(C# Compiler Options\)](#) if you do not want the compiler to use csc.rsp.

For more information, see [Add Reference Dialog Box](#).

Example

This example shows how to use the [extern alias \(C# Reference\)](#) feature.

You compile the source file and import metadata from `grid.dll` and `grid20.dll`, which have been compiled previously. The two DLLs contain separate versions of the same component, and you use two **/reference** with alias options to compile the source file. The options look like this:

```
/reference:GridV1=grid.dll and /reference:GridV2=grid20.dll
```

This sets up the external aliases "GridV1" and "GridV2," which you use in your program by means of an `extern` statement:

```
extern GridV1;
```

```
extern GridV2;  
// Using statements go here.
```

Once this is done, you can refer to the grid control from grid.dll by prefixing the control name with GridV1, like this:

```
GridV1::Grid
```

In addition, you can refer to the grid control from grid20.dll by prefixing the control name with GridV2 like this:

```
GridV2::Grid
```

See Also

Other Resources

[C# Compiler Options](#)

/resource (Embed Resource File to Output) (C# Compiler Options)

Embeds the specified resource into the output file.

```
/resource:filename[,identifier[,accessibility-modifier]]
```

Arguments

filename

The .NET Framework resource file you want to embed in the output file.

identifier (optional)

The logical name for the resource; the name used to load the resource. The default is the name of the file name.

accessibility-modifier (optional)

The accessibility of the resource: public or private. The default is public.

Remarks

Use [/linkresource](#) to link a resource to an assembly and not place the resource file in the output file.

By default, resources are public in the assembly when created with the C# compiler. To make the resources private, specify **private** as the accessibility modifier. No other accessibility other than **public** or **private** is allowed.

If *filename* is a .NET Framework resource file created, for example, by [Resgen.exe](#) or in the development environment, it can be accessed with members in the [System.Resources](#) namespace (see [System.Resources.ResourceManager](#) for more information). For all other resources, use the **GetManifestResource*** methods in the [Assembly](#) class to access the resource at run time.

/res is the short form of **/resource**.

The order of the resources in the output file is determined from the order specified on the command line.

To set this compiler option in the Visual Studio development environment

1. Add a resource file to your project.
2. Select the file you want to embed in Solution Explorer.
3. Select **Build Action** for the file in the Properties window.
4. Set **Build Action** to **Embedded Resource**.

For information on how to set this compiler option programmatically, see [BuildAction](#).

Example

Compile `in.cs` and attach resource file `rf.resource`:

```
csc /resource:rf.resource in.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/target (Specify Output File Format) (C# Compiler Options)

The **/target** compiler option can be specified in one of four forms:

/target:exe

To create an .exe file.

/target:library

To create a code library.

/target:module

To create a module.

/target:winexe

To create a Windows program.

Unless you specify **/target:module**, **/target** causes a .NET Framework assembly manifest to be placed in an output file. For more information, see [Assemblies in the Common Language Runtime](#) and [Global Attributes](#).

The assembly manifest is placed in the first .exe output file in the compilation or in the first DLL, if there is no .exe output file. For example, in the following command line, the manifest will be placed in `1.exe`:

```
csc /out:1.exe t1.cs /out:2.netmodule t2.cs
```

The compiler creates only one assembly manifest per compilation. Information about all files in a compilation is placed in the assembly manifest. All output files except those created with **/target:module** can contain an assembly manifest. When producing multiple output files at the command line, only one assembly manifest can be created and it must go into the first output file specified on the command line. No matter what the first output file is (**/target:exe**, **/target:winexe**, **/target:library** or **/target:module**), any other output files produced in the same compilation must be modules (**/target:module**).

If you create an assembly, you can indicate that all or part of your code is CLS compliant with the [CLSCompliant](#) attribute.

```
// target_clscompliant.cs
[assembly:System.CLSCompliant(true)]    // specify assembly compliance

[System.CLSCompliant(false)]    // specify compliance for an element
public class TestClass
{
    public static void Main() {}
}
```

For more information about setting this compiler option programmatically, see [OutputType](#).

See Also

Other Resources

[C# Compiler Options](#)

/target:exe (Create a Console Application) (C# Compiler Options)

The **/target:exe** option causes the compiler to create an executable (EXE), console application.

```
/target:exe
```

Remarks

The **/target:exe** option is in effect by default. The executable file will be created with the .exe extension.

Use [/target:winexe](#) to create a Windows program executable.

Unless otherwise specified with the [/out](#) option, the output file name takes the name of the input file that contains the [Main](#) method.

When specified at the command line, all files up to the next **/out** or **/target:module** option are used to create the .exe file.

One and only one **Main** method is required in the source code files that are compiled into an .exe file. The [/main](#) compiler option lets you specify which class contains the **Main** method, in cases where your code has more than one class with a **Main** method.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Application** property page.
3. Modify the **Output type** property.

For information on how to set this compiler option programmatically, see [OutputType](#).

Example

Each of the following command lines will compile `in.cs`, creating `in.exe`:

```
csc /target:exe in.cs
csc in.cs
```

See Also

Reference

[/target \(Specify Output File Format\) \(C# Compiler Options\)](#)

Other Resources

[C# Compiler Options](#)

/target:library (Create a Code Library) (C# Compiler Options)

The **/target:library** option causes the compiler to create a dynamic-link library (DLL) rather than an executable file (EXE).

```
/target:library
```

Remarks

The DLL will be created with the .dll extension.

Unless otherwise specified with the [/out](#) option, the output file name takes the name of the first input file.

When specified at the command line, all files up to the next **/out** or **/target:module** option are used to create the .dll file.

When building a .dll file, a [Main](#) method is not required.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Application** property page.
3. Modify the **Output type** property.

For information on how to set this compiler option programmatically, see [OutputType](#).

Example

Compile `in.cs`, creating `in.dll`:

```
csc /target:library in.cs
```

See Also

Reference

[/target \(Specify Output File Format\) \(C# Compiler Options\)](#)

Other Resources

[C# Compiler Options](#)

/target:module (Create Module to Add to Assembly) (C# Compiler Options)

This option causes the compiler to not generate an assembly manifest.

```
/target:module
```

Remarks

By default, the output file created by compiling with this option will have an extension of .netmodule.

A file that does not have an assembly manifest cannot be loaded by the .NET Framework common language runtime. However, such a file can be incorporated into the assembly manifest of an assembly by means of [/addmodule](#).

If more than one module is created in a single compilation, [internal](#) types in one module will be available to other modules in the compilation. When code in one module references **internal** types in another module, then both modules must be incorporated into an assembly manifest, by means of **/addmodule**.

Creating a module is not supported in the Visual Studio development environment.

For information on how to set this compiler option programmatically, see [OutputType](#).

Example

Compile `in.cs`, creating `in.netmodule`:

```
csc /target:module in.cs
```

See Also

Reference

[/target \(Specify Output File Format\) \(C# Compiler Options\)](#)

Other Resources

[C# Compiler Options](#)

/target:winexe (Create a Windows Program) (C# Compiler Options)

The **/target:winexe** option causes the compiler to create an executable (EXE), Windows program.

```
/target:winexe
```

Remarks

The executable file will be created with the .exe extension. A Windows program is one that provides a user interface from either the .NET Framework library or with the Win32 APIs.

Use [/target:exe](#) to create a console application.

Unless otherwise specified with the [/out](#) option, the output file name takes the name of the input file that contains the [Main](#) method.

When specified at the command line, all files until the next **/out** or [/target](#) option are used to create the Windows program.

One and only one **Main** method is required in the source code files that are compiled into an .exe file. The [/main](#) option lets you specify which class contains the **Main** method, in cases where your code has more than one class with a **Main** method.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Application** property page.
3. Modify the **Output type** property.

For information on how to set this compiler option programmatically, see [OutputType](#).

Example

Compile `in.cs` into a Windows program:

```
csc /target:winexe in.cs
```

See Also

Reference

[/target \(Specify Output File Format\) \(C# Compiler Options\)](#)

Other Resources

[C# Compiler Options](#)

/unsafe (Enable Unsafe Mode) (C# Compiler Options)

The **/unsafe** compiler option allows code that uses the [unsafe](#) keyword to compile.

```
/unsafe
```

Remarks

For more information about unsafe code, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Select the **Allow Unsafe Code** check box.

For information about how to set this compiler option programmatically, see [AllowUnsafeBlocks](#).

Example

Compile `in.cs` for unsafe mode:

```
csc /unsafe in.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/utf8output (Display Compiler Messages with UTF-8) (C# Compiler Options)

The **/utf8output** option displays compiler output using UTF-8 encoding.

```
/utf8output
```

Remarks

In some international configurations, compiler output cannot correctly be displayed in the console. In these configurations, use **/utf8output** and redirect compiler output to a file.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

See Also

Other Resources

[C# Compiler Options](#)

/warn (Specify Warning Level) (C# Compiler Options)

The **/warn** option specifies the warning level for the compiler to display.

```
/warn:option
```

Arguments

option

The warning level you want displayed for the compilation: Lower numbers show only high severity warnings; higher numbers show more warnings. Valid values are 0-4:

Warning level	Meaning
0	Turns off emission of all warning messages.
1	Displays severe warning messages.
2	Displays level 1 warnings plus certain, less-severe warnings, such as warnings about hiding class members.
3	Displays level 2 warnings plus certain, less-severe warnings, such as warnings about expressions that always evaluate to true or false .
4 (the default)	Displays all level 3 warnings plus informational warnings.

Remarks

To get information about an error or warning, you can look up the error code in the Help Index. For other ways to get information about an error or warning, see [How to: Locate Help for Compiler Errors](#).

Use **/warnaserror** to treat all warnings as errors. Use **/nowarn** to disable certain warnings.

/w is the short form of **/warn**.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Modify the **Warning Level** property.

For information on how to set this compiler option programmatically, see [WarningLevel](#).

Example

Compile `in.cs` and have the compiler only display level 1 warnings:

```
csc /warn:1 in.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/warnaserror (Treat Warnings as Errors) (C# Compiler Options)

The **/warnaserror+** option treats all warnings as errors

```
/warnaserror[<U>+</U> | -][:warning-list]
```

Remarks

Any messages that would ordinarily be reported as warnings are instead reported as errors, and the build process is halted (no output files are built).

By default, **/warnaserror-** is in effect, which causes warnings to not prevent the generation of an output file. **/warnaserror**, which is the same as **/warnaserror+**, causes warnings to be treated as errors.

Optionally, if you want only a few specific warnings to be treated as errors, you may specify a comma-separated list of warning numbers to treat as errors.

Use [/warn](#) to specify the level of warnings that you want the compiler to display. Use [/nowarn](#) to disable certain warnings.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** page. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Build** property page.
3. Modify the **Treat Warnings As Errors** property.

To set this compiler option programmatically, see [TreatWarningsAsErrors](#).

Example

Compile `in.cs` and have the compiler display no warnings:

```
csc /warnaserror in.cs  
csc /warnaserror:642,649,652 in.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/win32icon (Import an .ico File) (C# Compiler Options)

The **/win32icon** option inserts an .ico file in the output file, which gives the output file the desired appearance in the Windows Explorer.

```
/win32icon:filename
```

Arguments

filename

The .ico file that you want to add to your output file.

Remarks

An .ico file can be created with the [Resource Compiler](#). The Resource Compiler is invoked when you compile a Visual C++ program; an .ico file is created from the .rc file.

See [/linkresource](#) (to reference) or [/resource](#) (to attach) a .NET Framework resource file. See [/win32res](#) to import a .res file.

To set this compiler option in the Visual Studio development environment

1. Open the project's **Properties** pages. For details, see [How to: Set Project Properties \(C#, J#\)](#).
2. Click the **Application** property page.
3. Modify the **Application icon** property.

For information on how to set this compiler option programmatically, see [ApplicationIcon](#).

Example

Compile `in.cs` and attach an .ico file `rf.ico` to produce `in.exe`:

```
csc /win32icon:rf.ico in.cs
```

See Also

Other Resources

[C# Compiler Options](#)

/win32res (Import a Win32 Resource File) (C# Compiler Options)

The **/win32res** option inserts a Win32 resource in the output file.

```
/win32res:filename
```

Arguments

filename

The resource file that you want to add to your output file.

Remarks

A Win32 resource file can be created with the [Resource Compiler](#). The Resource Compiler is invoked when you compile a Visual C++ program; a .res file is created from the .rc file.

A Win32 resource can contain version or bitmap (icon) information that would help identify your application in the Windows Explorer. If you do not specify **/win32res**, the compiler will generate version information based on the assembly version.

See [/linkresource](#) (to reference) or [/resource](#) (to attach) a .NET Framework resource file.

This compiler option is unavailable in Visual Studio and cannot be changed programmatically.

Example

Compile `in.cs` and attach a Win32 resource file `rf.res` to produce `in.exe`:

```
csc /win32res:rf.res in.cs
```

See Also

Other Resources

[C# Compiler Options](#)

How to: Locate Help for Compiler Errors

Note

The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose Import and Export Settings on the Tools menu. For more information, see [Visual Studio Settings](#).

All C# compiler errors have corresponding topics that explain why an error is generated, and in some cases, how to fix the error. To get help on a particular error message, try one of the following:

Procedure

To find help for an error

- Click the error number in the [Output Window](#), and press F1.

-or-

Type the error number in the **Look for** box in the Index.

-or-

Type the error number in the **Search** page.

See Also

Other Resources

[C# Compiler Options](#)

C# Compiler Errors

This section contains C# compiler errors listed numerically. All C# compiler errors have corresponding topics that explain why an error is generated, and in some cases, how to fix the error. To get Help on a particular error message, try one of the following:

- Click the error number in the [Output Window](#), and press F1.
- Type the error number in the **Look for** box in the Index.
- Type the error number in the **Search** page.
- Locate the error in the Contents list.

Note

The dialog boxes and menu commands you see might differ from those described in Help depending on your active settings or edition. To change your settings, choose Import and Export Settings on the Tools menu. For more information, see [Visual Studio Settings](#).

See Also

Other Resources

[C# Compiler Options](#)

Compiler Error CS0001

Error Message

Internal compiler error

Try to determine if the compiler is failing because of its inability to parse unexpected syntax. If this is not the case, try [Getting Help from Microsoft Product Support Services \(Visual Studio\)](#).

Compiler Error CS0003

Error Message

Out of memory

The compiler was unable to allocate enough virtual memory to complete compilation. Close all unnecessary applications and compile again.

You may also want to increase the pagefile size and make sure you have free disk space.

This error may also appear as the result of having mismatched versions of the .NET Framework SDK and the C# compiler or one or more corrupt files that support the C# compiler; reinstall Visual Studio.

Compiler Error CS0004

Warning treated as error

You compiled with the `/warnaserror` compiler option, which causes the compiler to generate warnings as errors.

Compiler Error CS0005

Error Message

Compiler option 'compiler_option' must be followed by an argument

Some compiler options require parameters. If you do not pass the arguments required by the compiler option, CS0005 is generated.

For more information, see [C# Compiler Options](#).

Compiler Error CS0006

Error Message

Metadata file 'dll_name' could not be found

The program was compiled and explicitly passed the name of a file that contained metadata; however, the .dll was not found. For more information, see [/reference \(Import Metadata\) \(C# Compiler Options\)](#).

Compiler Error CS0007

Error Message

Unexpected common language runtime initialization error — 'description'

This error occurs if the runtime could not be loaded. This could occur if the version of the common language runtime that the compiler attempts to load is not present on the machine, or if the common language runtime installation or configuration is corrupt.

This can happen if the `csc.exe.config` file was changed. This file is configured during setup and should not be changed. If there is a possibility that the `csc.exe.config` file was changed, check the file to make sure that the version of the runtime specified in the file is present on the machine. If the correct version is present, it may be corrupted. Reinstall the common language runtime.

Compiler Error CS0008

Error Message

Unexpected error reading metadata from file 'file' — 'description'

A DLL was successfully opened for retrieving metadata but is corrupted, such that data could not be read from it. For more information, see [/reference \(Import Metadata\) \(C# Compiler Options\)](#).

Compiler Error CS0009

Error Message

Metadata file 'file' could not be opened — 'description'

The file specified with the [/reference](#) compiler option does not contain valid metadata.

Compiler Error CS0011

Error Message

The base class or interface 'class' in assembly 'assembly' referenced by type 'type' could not be resolved

A class that was imported from a file with **/reference**, is derived from a class or implements an interface that is not found. This can occur if a required DLL is not also included in the compilation with **/reference**.

For more information, see [Add Reference Dialog Box](#) and [/reference \(Import Metadata\) \(C# Compiler Options\)](#).

Example

```
// CS0011_1.cs
// compile with: /target:library

public class Outer
{
    public class B { }
}
```

The second file creates a DLL that defines a class `C` that is derived from the class `B` that was created in the previous example.

```
// CS0011_2.cs
// compile with: /target:library /reference:CS0011_1.dll
// post-build command: del /f CS0011_1.dll
public class C : Outer.B {}
```

The third file replaces the DLL created by the first step, and omits the definition of the inner class `B`.

```
// CS0011_3.cs
// compile with: /target:library /out:cs0011_1.dll
public class Outer {}
```

Finally, the fourth file references the class `C` defined in the second example, which is derived from class `B`, and which is now missing.

The following sample generates CS0011.

```
// CS0011_4.cs
// compile with: /reference:CS0011_1.dll /reference:CS0011_2.dll
// CS0011 expected

class M
{
    public static void Main()
    {
        C c = new C();
    }
}
```

See Also

Reference

[Add Reference Dialog Box](#)

[/reference \(Import Metadata\) \(C# Compiler Options\)](#)

Compiler Error CS0012

Error Message

The type 'type' is defined in an assembly that is not referenced. You must add a reference to assembly 'assembly'.

The definition for a referenced type was not found. This could occur if a required .DLL file is not included in the compilation. For more information, see [Add Reference Dialog Box](#) and [/reference \(Import Metadata\) \(C# Compiler Options\)](#).

The following sequence of compilations will result in CS0012:

```
// cs0012a.cs
// compile with: /target:library
public class A {}
```

Then:

```
// cs0012b.cs
// compile with: /target:library /reference:cs0012a.dll
public class B
{
    public static A f()
    {
        return new A();
    }
}
```

Then:

```
// cs0012c.cs
// compile with: /reference:cs0012b.dll
class C
{
    public static void Main()
    {
        object o = B.f();    // CS0012
    }
}
```

You could resolve this CS0012 by compiling with `/reference:b.dll;a.dll`.

Compiler Error CS0013

Error Message

Unexpected error writing metadata to file 'file' — 'description'

The .NET Framework common language runtime failed to emit metadata. Check to make sure that the path is correct and that the disk is not full. If the problem persists, you may need to repair or reinstall Visual Studio and/or the .NET Framework.

Compiler Error CS0014

Error Message

Required file 'file' could not be found

A file is needed by the compiler but is not on the system. Make sure that the path is correct. If the file is a Visual Studio system file, then you may need to repair your installation or else remove and reinstall Visual Studio completely.

The name of type 'type' is too long

The fully qualified name of a user-defined type must be less than or equal to 2048 characters.

The following sample generates CS0015:

```
// CS0015.cs
namespace
{
    namespace
    {
        namespace
        {
            public class
            {
                public static void Main()
                {
                }
            }
        }
    }
}
```

Compiler Error CS0016

Error Message

Could not write to output file 'file' — 'reason'

The compiler could not write to an output file. Check the path to the file to make sure it exists. If a previously built file is already at the location, make sure it is writeable, and that no process currently has a lock on the file. For example, make sure your executable is not running when you attempt to build.

Compiler Error CS0017

Error Message

Program 'output file name' has more than one entry point defined: function

A program can only have one [Main](#) method.

To resolve this error, you can either delete all Main methods in your code, except one, or you can use the [/main](#) compiler option to specify which Main method you want to use.

The following sample generates CS0017:

```
// CS0017.cs
public class clx
{
    static public void Main()
    {
    }
}

public class cly
{
    public static void Main()    // CS0017, delete one Main or use /main
    {
    }
}
```

Compiler Error CS0019

Error Message

Operator 'operator' cannot be applied to operands of type 'type' and 'type'

A binary operator is operating on data types for which it was not designed. For example, you cannot use the `||` operator on strings.

Example

In this example, conditional logic must be specified outside the [ConditionalAttribute](#). You can only pass one predefined symbol to the **ConditionalAttribute**.

The following sample generates CS0019.

```
// CS0019.cs
// compile with: /target:library
using System.Diagnostics;
public class MyClass
{
    [ConditionalAttribute("DEBUG" || "TRACE")]    // CS0019
    public void TestMethod() {}

    // OK
    [ConditionalAttribute("DEBUG")]
    public void TestMethod2() {}
}
```

See Also

Reference

[Operators \(C# Programming Guide\)](#)

Compiler Error CS0020

Error Message

Division by constant zero

An expression uses a literal (not variable) value of zero in the denominator of a division operation. Division by zero is not defined, and is therefore invalid.

The following sample generates CS0020:

```
// CS0020.cs
namespace x
{
    public class b
    {
        public static void Main()
        {
            1 / 0;    // CS0020
        }
    }
}
```

See Also

Reference

[Operators \(C# Programming Guide\)](#)

Compiler Error CS0021

Error Message

Cannot apply indexing with [] to an expression of type 'type'

An attempt was made to access a value through an indexer on a data type that does not support [Indexers \(C# Programming Guide\)](#).

You may get CS0021 when trying to use an indexer in a C++ assembly. In this case, decorate the C++ class with the **DefaultMember** attribute so the C# compiler knows which indexer is the default. The following sample generates CS0021.

Example

This file compiles to a .dll file—with the **DefaultMember** attribute commented out—in order to generate the error.

```
// CPP0021.cpp
// compile with: /clr /LD
using namespace System::Reflection;
// Uncomment the following line to resolve
//[DefaultMember("myItem")]
public ref class MyClassMC
{
    public:
    property int myItem[int]
    {
        int get(int i){ return 5; }
        void set(int i, int value) {}
    }
};
```

The following is the C# file that calls the .dll file. This file attempts to access the class via an indexer, but because no member was declared as the default indexer to be used, the error is generated.

```
// CS0021.cs
// compile with: /reference:CPP0021.dll
public class MyClass
{
    public static void Main()
    {
        MyClassMC myMC = new MyClassMC();
        int j = myMC[1]; // CS0021
    }
}
```

Compiler Error CS0022

Error Message

Wrong number of indices inside [], expected 'number'

An array-access operation specified the incorrect number of dimensions within the square brackets. For more information, see [Arrays \(C# Programming Guide\)](#).

Example

The following sample generates CS0022:

```
// CS0022.cs
public class MyClass
{
    public static void Main()
    {
        int[] a = new int[10];
        a[0] = 0;    // single-dimension array
        a[0,1] = 9;  // CS0022, the array does not have two dimensions
    }
}
```

Compiler Error CS0023

Error Message

Operator 'operator' cannot be applied to operand of type 'type'

An attempt was made to apply an operator to a variable whose type was not designed to work with the operator. For more information, see [Data Types \(C# Programming Guide\)](#) and [C# Operators](#).

The following sample generates CS0023:

```
// CS0023.cs
namespace x
{
    public class a
    {
        public static void Main()
        {
            string s = "hello";
            s = -s;    // CS0023, minus operator not allowed on strings
        }
    }
}
```


Compiler Error CS0026

Error Message

Keyword 'this' is not valid in a static property, static method, or static field initializer

The [this \(C# Reference\)](#) keyword refers to an object, which is an instance of a type. Since static methods are independent of any instance of the containing class, the "this" keyword is meaningless and is therefore not allowed. For more information, see [Static Classes and Static Class Members \(C# Programming Guide\)](#) and [Objects \(C# Programmers Reference\)](#).

Example

The following example generates CS0026:

```
// CS0026.cs
public class A
{
    public static int i = 0;

    public static void Main()
    {
// CS0026
        this.i = this.i + 1;
        // Try the following line instead:
        // i = i + 1;
    }
}
```

Compiler Error CS0027

Error Message

Keyword 'this' is not available in the current context

The [this \(C# Reference\)](#) keyword was found outside of a property, method, or constructor.

To fix this error, either modify the statement to eliminate use of the **this** keyword, and/or move part or all of the statement inside a property, method, or constructor.

The following example generates CS0027:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace ConsoleApplication3
{
    class MyClass
    {
        int err1 = this.Fun() + 1; // CS0027

        public int Fun()
        {
            return 10;
        }

        public void Test()
        {
            // valid use of this
            int err = this.Fun() + 1;
            Console.WriteLine(err);
        }

        public static void Main()
        {
            MyClass c = new MyClass();
            c.Test();
        }
    }
}
```

Compiler Error CS0029

Error Message

Cannot implicitly convert type 'type' to 'type'

The compiler requires an explicit conversion. For example, you may need to cast an r-value to be the same type as an l-value. Or, you must provide conversion routines to support certain operator overloads.

Conversions must occur when assigning a variable of one type to a variable of a different type. When making an assignment between variables of different types, the compiler must convert the type on the right-hand side of the assignment operator to the type on the left-hand side of the assignment operator. Take the following the code:

```
int i = 50;
long lng = 100;
i = lng;
```

`i = lng;` makes an assignment, but the data types of the variables on the left and right-hand side of the assignment operator don't match. Before making the assignment the compiler is implicitly converting the variable `lng`, which is of type `long`, to an `int`. This is implicit because no code explicitly instructed the compiler to perform this conversion. The problem with this code is that this is considered a narrowing conversion, and the compiler does not allow implicit narrowing conversions because there could be a potential loss of data.

A narrowing conversion exists when converting to a data type that occupies less storage space in memory than the data type we are converting from. For example, converting a `long` to an `int` would be considered a narrowing conversion. A `long` occupies 8 bytes of memory while an `int` occupies 4 bytes. To see how data loss can occur, consider the following sample:

```
int i = 50;
long lng = 3147483647;
i = lng;
```

The variable `lng` now contains a value that cannot be stored in the variable `i` because it is too large. If we were to convert this value to an `int` type we would be losing some of our data and the converted value would not be the same as the value before the conversion.

A widening conversion would be the opposite of a narrowing conversion. With widening conversions, we are converting to a data type that occupies more storage space in memory than the data type we are converting from. Here is an example of a widening conversion:

```
int i = 50;
long lng = 100;
lng = i;
```

Notice the difference between this code sample and the first. This time the variable `lng` is on the left-hand side of the assignment operator, so it is the target of our assignment. Before the assignment can be made, the compiler must implicitly convert the variable `i`, which is of type `int`, to type `long`. This is a widening conversion since we are converting from a type that occupies 4 bytes of memory (an `int`) to a type that occupies 8 bytes of memory (a `long`). Implicit widening conversions are allowed because there is no potential loss of data. Any value that can be stored in an `int` can also be stored in a `long`.

We know that implicit narrowing conversions are not allowed, so to be able to compile this code we need to explicitly convert the data type. Explicit conversions are done using casting. Casting is the term used in C# to describe converting one data type to another. To get the code to compile we would need to use the following syntax:

```
int i = 50;
long lng = 100;
i = (int) lng;    // cast to int
```

The third line of code tells the compiler to explicitly convert the variable `lng`, which is of type `long`, to an `int` before making the assignment. Remember that with a narrowing conversion, there is a potential loss of data. Narrowing conversions should be used with caution and even though the code will compile you may get unexpected results at run-time.

This discussion has only been for value types. When working with value types you work directly with the data stored in the

variable. However, the .NET Framework also has reference types. When working with reference types you are working with a reference to a variable, not the actual data. Examples of reference types would be classes, interfaces and arrays. You cannot implicitly or explicitly convert one reference type to another unless the compiler allows the specific conversion or the appropriate conversion operators are implemented.

The following sample generates CS0029:

```
// CS0029.cs
public class MyInt
{
    private int x = 0;

    // Uncomment this conversion routine to resolve CS0029
    /*
    public static implicit operator int(MyInt i)
    {
        return i.x;
    }
    */

    public static void Main()
    {
        MyInt myInt = new MyInt();
        int i = myInt; // CS0029
    }
}
```

See Also

Concepts

[Conversion Operators \(C# Programming Guide\)](#)

Compiler Error CS0030

Error Message

Cannot convert type 'type' to 'type'

You must provide conversion routines to support certain operator overloads. For more information, see [Conversion Operators \(C# Programming Guide\)](#).

The following sample generates CS0030:

```
// CS0030.cs
namespace x
{
    public class iii
    {
        /*
        public static implicit operator iii(int aa)
        {
            return null;
        }

        public static implicit operator int(iii aa)
        {
            return 0;
        }
        */

        public static iii operator ++(iii aa)
        {
            return (iii)0;    // CS0030
            // uncomment the conversion routines to resolve CS0030
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0031

Error Message

Constant value 'value' cannot be converted to a 'type'

An attempt was made to assign a value to a variable whose type cannot store the value. For more information, see [Data Types \(C# Programming Guide\)](#).

The following sample generates CS0031:

```
// CS0031.cs
namespace x
{
    public class a
    {
        public static void Main()
        {
            byte i = 512;    // CS0031, 512 cannot fit in byte
        }
    }
}
```

Compiler Error CS0034

Error Message

Operator 'operator' is ambiguous on operands of type 'type1' and 'type2'

An operator was used on two objects and the compiler found more than one conversion. Conversions need to be unique, so you either need to make a cast or to make one of the conversions explicit. For more information, see

[Conversion Operators \(C# Programming Guide\)](#).

The following sample generates CS0034:

```
// CS0034.cs
public class A
{
    // allows for the conversion of A object to int
    public static implicit operator int (A s)
    {
        return 0;
    }

    public static implicit operator string (A i)
    {
        return null;
    }
}

public class B
{
    public static implicit operator int (B s)
    // one way to resolve this CS0034 is to make one conversion explicit
    // public static explicit operator int (B s)
    {
        return 0;
    }

    public static implicit operator string (B i)
    {
        return null;
    }

    public static implicit operator B (string i)
    {
        return null;
    }

    public static implicit operator B (int i)
    {
        return null;
    }
}

public class C
{
    public static void Main ()
    {
        A a = new A();
        B b = new B();
        b = b + a;    // CS0034
        // another way to resolve this CS0034 is to make a cast
        // b = b + (int)a;
    }
}
```

Compiler Error CS0035

Error Message

Operator 'operator' is ambiguous on an operand of type 'type'

The compiler has more than one available conversion and does not know which to choose before applying the operator. For more information, see [Templated User Defined Conversions](#) and [Conversion Operators \(C# Programming Guide\)](#).

The following sample generates CS0035:

```
// CS0035.cs
class MyClass
{
    private int i;

    public MyClass(int i)
    {
        this.i = i;
    }

    public static implicit operator double(MyClass x)
    {
        return (double) x.i;
    }

    public static implicit operator decimal(MyClass x)
    {
        return (decimal) x.i;
    }
}

class MyClass2
{
    static void Main()
    {
        MyClass x = new MyClass(7);
        object o = - x;    // CS0035
        // try a cast:
        // object o = - (double)x;
    }
}
```


Compiler Error CS0036

Error Message

An out parameter cannot have the '[In]' attribute

Currently, the **In** attribute is not permitted on an [out](#) parameter.

The following sample generates CS0036:

```
// CS0036.cs

using System;
using System.Runtime.InteropServices;

public class MyClass
{
    public static void TestOut([In] out char TestChar)    // CS0036
    // try the following line instead
    // public static void TestOut(out char TestChar)
    {
        TestChar = 'b';
        Console.WriteLine(TestChar);
    }

    public static void Main()
    {
        char i;           //variable to receive the value
        TestOut(out i);    // the arg must be passed as out
        Console.WriteLine(i);
    }
}
```

Compiler Error CS0037

Error Message

Cannot convert null to 'type' because it is a value type

The compiler cannot assign null to a value type; null can only be assigned to a [reference type](#) or to a Nullable type. [struct](#) is a value type. For more information, see [Nullable Types \(C# Programming Guide\)](#).

The following sample generates CS0037:

```
// CS0037.cs
public struct s
{
}

class a
{
    public static void Main()
    {
        int i = null;    // CS0037
        s ss = null;     // CS0037
    }
}
```

Compiler Error CS0038

Error Message

Cannot access a nonstatic member of outer type 'type1' via nested type 'type2'

A field in a class is not automatically available to a nested class. To be available to a nested class, the field must be [static](#). Otherwise, you must create an instance of the outer class. For more information, see [Nested Types \(C# Programming Guide\)](#).

The following sample generates CS0038:

```
// CS0038.cs
class OuterClass
{
    public int count;
    // try the following line instead
    // public static int count;

    class InnerClass
    {
        void func()
        {
            // or, create an instance
            // OuterClass class_inst = new OuterClass();
            // int count2 = class_inst.count;
            int count2 = count;    // CS0038
        }
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0039

Error Message

Cannot convert type 'type1' to 'type2' via a built-in conversion

A conversion with the [as \(C# Reference\)](#) operator is allowed by inheritance, reference conversions, and boxing conversions. For more information, see [Conversion Operators \(C# Programming Guide\)](#).

Example

The following example generates CS0039.

```
// CS0039.cs
using System;
class A
{
}
class B: A
{
}
class C: A
{
}
class M
{
    static void Main()
    {
        A a = new C();
        B b = new B();
        C c;

        // This is valid; there is a built-in reference
        // conversion from A to C.
        c = a as C;

        //The following generates CS0039; there is no
        // built-in reference conversion from B to C.
        c = b as C; // CS0039
    }
}
```

Compiler Error CS0040

Error Message

Unexpected debug information initialization error — 'reason'

This error can occur when using the [/debug](#) compiler option and indicates that the compiler was unable to write to the .pdb file. Possible resolutions to this error include reinstalling Visual Studio, ensuring that the compiler has write access to a file or directory, or not compiling with /debug.

Compiler Error CS0041

Error Message

Unexpected error writing debug information to file 'file' — 'reason'

This error can occur when using the [/debug](#) compiler option. If you encounter this error, try deleting the PDB files in the bin directory and recompiling. If you are still encountering this error, you may need to repair or reinstall Visual Studio. As a last resort, try [Getting Help from Microsoft Product Support Services \(Visual Studio\)](#).

Compiler Error CS0042

Error Message

Unexpected error creating debug information file 'file' — 'reason'

The compiler could not create the debug information; *reason* contains additional information on the error condition.

Compiler Error CS0043

Error Message

PDB file 'file' has an incorrect or out-of-date format. Delete it and rebuild.

Delete the .pdb file for this compilation and recompile.

Compiler Error CS0050

Error Message

Inconsistent accessibility: return type 'type' is less accessible than method 'method'

The return type and each of the types referenced in the formal parameter list of a method must be at least as accessible as the method itself. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

Example

The following sample generates CS0050 because no accessibility modifier is supplied for `MyClass` and its accessibility therefore defaults to **private**.

```
// CS0050.cs
class MyClass //accessibility defaults to private
// try the following line instead
// public class MyClass
{
}

public class MyClass2
{
    public static MyClass MyMethod()    // CS0050
    {
        return new MyClass();
    }

    public static void Main() { }
}
```

Compiler Error CS0051

Error Message

Inconsistent accessibility: parameter type 'type' is less accessible than method 'method'

The return type and each of the types referenced in the formal parameter list of a method must be at least as accessible as the method itself. Make sure the types used in method signatures are not accidentally private due to the omission of the **public** modifier. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

Example

The following sample generates CS0051:

```
// CS0051.cs
public class A
{
    // Try making B public since F is public
    // B is implicitly private here
    class B
    {
    }

    public static void F(B b) // CS0051
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0052

Error Message

Inconsistent accessibility: field type 'type' is less accessible than field 'field'

The type of a field cannot be less accessible than the field itself because all public constructs must return a publicly accessible object.

Example

The following sample generates CS0052:

```
// CS0052.cs
class MyClass
// try the following line instead
// public class MyClass
{
}

public class MyClass2
{
    public MyClass mf;    // CS0052
}

public class MyClass3
{
    public static void Main()
    {
    }
}
```

See Also

Reference

[C# Keywords](#)

[Access Modifiers \(C# Reference\)](#)

[Accessibility Levels \(C# Reference\)](#)

[Modifiers \(C# Reference\)](#)

Compiler Error CS0053

Error Message

Inconsistent accessibility: property type 'type' is less accessible than property 'property'

A public construct must return a publicly accessible object. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

The following sample generates CS0053:

```
// CS0053.cs
class MyClass //defaults to private accessibility
// try the following line instead
// public class MyClass
{
}

public class MyClass2
{
    public MyClass myProperty    // CS0053
    {
        get
        {
            return new MyClass();
        }
        set
        {
        }
    }
}

public class MyClass3
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0054

Error Message

Inconsistent accessibility: indexer return type 'type' is less accessible than indexer 'indexer'

A public construct must return a publicly accessible object. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

The following sample generates CS0054:

```
// CS0054.cs
class MyClass
// try the following line instead
// public class MyClass
{
}

public class MyClass3
{
    public MyClass this[int i]    // CS0054
    {
        get
        {
            return new MyClass();
        }
    }
}

public class MyClass2
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0055

Error Message

Inconsistent accessibility: parameter type 'type' is less accessible than indexer 'indexer'

A public construct must return a publicly accessible object. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

The following sample generates CS0055:

```
// CS0055.cs
class MyClass //defaults to private accessibility
// try the following line instead
// public class MyClass
{
}

public class MyClass2
{
    public int this[MyClass myClass]    // CS0055
    {
        get
        {
            return 0;
        }
    }
}

public class MyClass3
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0056

Error Message

Inconsistent accessibility: return type 'type' is less accessible than operator 'operator'

A public construct must return a publicly accessible object. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

The following sample generates CS0056:

```
// CS0056.cs
class MyClass
// try the following line instead
// public class MyClass
{
}

public class A
{
    public static implicit operator MyClass(A a)    // CS0056
    {
        return new MyClass();
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0057

Error Message

Inconsistent accessibility: parameter type 'type' is less accessible than operator 'operator'

A public construct must return a publicly accessible object. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

The following sample generates CS0057:

```
// CS0057.cs
class MyClass //defaults to private accessibility
// try the following line instead
// public class MyClass
{
}

public class MyClass2
{
    public static implicit operator MyClass2(MyClass iii)    // CS0057
    {
        return new MyClass2();
    }

    public static void Main()
    {
    }
}
```


Compiler Error CS0058

Error Message

Inconsistent accessibility: return type 'type' is less accessible than delegate 'delegate'

A public construct must return a publicly accessible object. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

The following sample generates CS0058 because no access modifier is applied to MyClass and therefore it is given private accessibility by default:

```
// CS0058.cs
class MyClass
// try the following line instead
// public class MyClass
{
}

public delegate MyClass MyClassDel();    // CS0058

public class A
{
    public static void Main()
    {
    }
}
```

See Also

Reference

[private \(C# Reference\)](#)

Compiler Error CS0059

Error Message

Inconsistent accessibility: parameter type 'type' is less accessible than delegate 'delegate'

The return type and each of the types referenced in the formal parameter list of a method must be at least as accessible as the method itself. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

Example

The following sample generates CS0059:

```
// CS0059.cs
class MyClass //defaults to private accessibility
// try the following line instead
// public class MyClass
{
}

public delegate void MyClassDel( MyClass myClass);    // CS0059

public class Program
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0060

Error Message

Inconsistent accessibility: base class 'class1' is less accessible than class 'class2'

Class accessibility should be consistent between the base class and inherited class.

The following sample generates CS0060:

```
// CS0060.cs
class MyClass
// try the following line instead
// public class MyClass
{
}

public class MyClass2 : MyClass    // CS0060
{
    public static void Main()
    {
    }
}
```

See Also

Reference

[Access Modifiers \(C# Programming Guide\)](#)

Compiler Error CS0061

Error Message

Inconsistent accessibility: base interface 'interface 1' is less accessible than interface 'interface 2'

A [public](#) construct must return a publicly accessible object.

Interface accessibility cannot be narrowed in a derived interface. For more information, see [Interfaces \(C# Programming Guide\)](#) and [Access Modifiers \(C# Programming Guide\)](#).

The following sample generates CS0061.

```
// CS0061.cs
// compile with: /target:library
internal interface A {}
public interface AA : A {} // CS0061

// OK
public interface B {}
internal interface BB : B {}

internal interface C {}
internal interface CC : C {}
```

Compiler Error CS0065

Error Message

'event' : event property must have both add and remove accessors

An event that is not a field must have both access methods.

The following sample generates CS0065:

```
// CS0065.cs
using System;
public delegate void EventHandler(object sender, int e);
public class MyClass
{
    public event EventHandler Click    // CS0065,
    {
        // to fix, uncomment the add and remove definitions
        /*
        add
        {
            Click += value;
        }
        remove
        {
            Click -= value;
        }
        */
    }

    public static void Main()
    {
    }
}
```

See Also

Concepts

[Events \(C# Programming Guide\)](#)

Compiler Error CS0066

Error Message

'event': event must be of a delegate type

The event keyword requires a [delegate](#) type. For more information, see [Events \(C# Programming Guide\)](#) and [Delegates \(C# Programming Guide\)](#).

The following sample generates CS0066:

```
// CS0066.cs
using System;

public class EventHandler
{
}

// to fix the error, remove the event declaration and the
// EventHandler class declaration, and uncomment the following line
// public delegate void EventHandler();

public class a
{
    public event EventHandler Click;    // CS0066

    private void TestMethod()
    {
        if (Click != null)
            Click();
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0068

Error Message

'event': event in interface cannot have initializer

An event in an interface cannot have an initializer. For more information, see [Interfaces \(C# Programming Guide\)](#).

The following sample generates CS0068:

```
// CS0068.cs

delegate void MyDelegate();

interface I
{
    event MyDelegate d = new MyDelegate(M.f);    // CS0068
    // try the following line instead
    // event MyDelegate d2;
}

class M
{
    event MyDelegate d = new MyDelegate(M.f);

    public static void f()
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0069

Error Message

An event in an interface cannot have add or remove accessors

You cannot define an event's accessor functions in an [interface](#). For more information, see [Events \(C# Programming Guide\)](#) and [Interfaces \(C# Programming Guide\)](#).

The following sample generates CS0069:

```
// CS0069.cs
// compile with: /target:library

public delegate void EventHandler();

public interface a
{
    event EventHandler Click { remove {} }    // CS0069
    event EventHandler Click2;    // OK
}
```


Compiler Error CS0070

Error Message

The event 'event' can only appear on the left hand side of += or -= (except when used from within the type 'type')

Outside of the class it is defined in, an [event](#) can only add or subtract references. For more information, see [Events \(C# Programming Guide\)](#).

The following sample generates CS0070:

```
// CS0070.cs
using System;
public delegate void EventHandler();

public class A
{
    public event EventHandler Click;

    public static void OnClick()
    {
        EventHandler eh;
        A a = new A();
        eh = a.Click;
    }

    public static void Main()
    {
    }
}

public class B
{
    public int Foo ()
    {
        EventHandler eh = new EventHandler(A.OnClick);
        A a = new A();
        eh = a.Click;    // CS0070
        // try the following line instead
        // a.Click += eh;
        return 1;
    }
}
```

Compiler Error CS0071

Error Message

An explicit interface implementation of an event must use property syntax

When explicitly implementing an [event](#) that was declared in an interface, you must manually provide the **add** and **remove** event accessors that are typically provided by the compiler. The accessor code can connect the interface event to another event in your class (shown below), or to its own delegate type. For more information, see

[How to: Implement Interface Events \(C# Programming Guide\)](#).

Example

The following sample generates CS0071.

```
// CS0071.cs
public delegate void MyEvent(object sender);

interface ITest
{
    event MyEvent Clicked;
}

class Test : ITest
{
    event MyEvent ITest.Clicked; // CS0071

    // try the following code instead
    /*
    private MyEvent clicked;

    event MyEvent ITest.Clicked
    {
        add
        {
            clicked += value;
        }
        remove
        {
            clicked -= value;
        }
    }
    */
    public static void Main() { }
}
```

Compiler Error CS0072

Error Message

'event' : cannot override; 'method' is not an event

An [event](#) can only override another event. For more information, see [Events \(C# Programming Guide\)](#).

The following sample generates CS0072:

```
// CS0072.cs
delegate void MyDelegate();

class Test1
{
    public virtual event MyDelegate MyEvent;
    public virtual void VMeth()
    {
    }

    public void FireAway()
    {
        if (MyEvent != null)
            MyEvent();
    }
}

class Test2 : Test1
{
    public override event MyDelegate VMeth // CS0072
    // uncomment the following lines to resolve
    // public override event MyDelegate MyEvent
    {
        add
        {
            VMeth += value;
            // MyEvent += value;
        }
        remove
        {
            VMeth -= value;
            // MyEvent -= value;
        }
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0073

Error Message

An add or remove accessor must have a body

An **add** or **remove** keyword in an [event](#) definition must have a body. For more information, see [Events \(C# Programming Guide\)](#).

The following sample generates CS0073:

```
// CS0073.cs
delegate void del();

class Test
{
    public event del MyEvent
    {
        add;    // CS0073
        // try the following lines instead
        // add
        // {
        //     MyEvent += value;
        // }
        remove
        {
            MyEvent -= value;
        }
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0074

Error Message

'event': abstract event cannot have initializer

If an [event](#) is marked as **abstract**, it cannot be initialized. For more information, see [Events \(C# Programming Guide\)](#).

The following sample generates CS0074:

```
// CS0074.cs
delegate void D();

abstract class Test
{
    public abstract event D e = null;    // CS0074
    // try the following line instead
    // public abstract event D e;

    public static void Main()
    {
    }
}
```

Compiler Error CS0075

Error Message

To cast a negative value, you must enclose the value in parentheses

If you are casting using a keyword that identifies a predefined type, then you do not need parentheses. Otherwise, you must put the parentheses because $(x) - y$ will not be considered a cast expression. From the C# Specification, Section 7.6.6:

From the disambiguation rule it follows that, if x and y are identifiers, $(x)y$, $(x)(y)$, and $(x)(-y)$ are cast-expressions, but $(x)-y$ is not, even if x identifies a type. However, if x is a keyword that identifies a predefined type (such as `int`), then all four forms are cast-expressions (because such a keyword could not possibly be an expression by itself).

The following code generates CS0075:

```
// CS0075
namespace MyNamespace
{
    enum MyEnum { }
    public class MyClass
    {
        public static void Main()
        {
            // To fix the error, place the negative
            // values below in parentheses
            int i = (System.Int32) - 4; //CS0075
            MyEnum e = (MyEnum) - 1;    //CS0075
            System.Console.WriteLine(i); //to avoid warning
            System.Console.WriteLine(e); //to avoid warning
        }
    }
}
```

See Also

Reference

[Casting \(C# Programming Guide\)](#)

Compiler Error CS0076

Error Message

The enumerator name 'value__' is reserved and cannot be used

An item in an [enumeration](#) cannot have an identifier called **value__**.

Compiler Error CS0077

Error Message

The as operator must be used with a reference type ('type' is a value type)

The [as](#) operator was passed a [value type](#). Because **as** can return [null](#), it can only be passed [reference types](#).

The following sample generates CS0077:

```
// CS0077.cs
using System;

class C
{
}

struct S
{
}

class M
{
    public static void Main()
    {
        object o1, o2;
        C c;
        S s;

        o1 = new C();
        o2 = new S();

        s = o2 as S; // CS0077, S is not a reference type.
        // try the following line instead
        // c = o1 as C;
    }
}
```


Compiler Error CS0079

Error Message

The event 'event' can only appear on the left hand side of += or -=

An [event](#) was called incorrectly. For more information, see [Events \(C# Programming Guide\)](#) and [Delegates \(C# Programming Guide\)](#).

The following sample generates CS0079:

```
// CS0079.cs
using System;

public delegate void MyEventHandler();

public class Class1
{
    private MyEventHandler _e;

    public event MyEventHandler Pow
    {
        add
        {
            _e += value;
            Console.WriteLine("in add accessor");
        }
        remove
        {
            _e -= value;
            Console.WriteLine("in remove accessor");
        }
    }

    public void Handler()
    {
    }

    public void Fire()
    {
        if (_e != null)
        {
            Pow(); // CS0079
            // try the following line instead
            // _e();
        }
    }

    public static void Main()
    {
        Class1 p = new Class1();
        p.Pow += new MyEventHandler(p.Handler);
        p._e();
        p.Pow += new MyEventHandler(p.Handler);
        p._e();
        p._e -= new MyEventHandler(p.Handler);
        if (p._e != null)
        {
            p._e();
        }
        p.Pow -= new MyEventHandler(p.Handler);
        if (p._e != null)
        {
            p._e();
        }
    }
}
```


Compiler Error CS0080

Error Message

Constraints are not allowed on non-generic declarations

The syntax found may only be used in a generic declaration to apply constraints to the type parameter. For more information, see [Generics \(C# Programming Guide\)](#).

The following sample generates CS0080 because MyClass is not a generic class and Foo is not a generic method.

```
namespace MyNamespace
{
    public class MyClass where MyClass : System.IDisposable // CS0080    //the following li
ne shows an example of correct syntax
    //public class MyClass<T> where T : System.IDisposable
    {
        public void Foo() where Foo : new() // CS0080
        //the following line shows an example of correct syntax
        //public void Foo<U>() where U : struct
        {
        }
    }

    public class Program
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0081

Error Message

Type parameter declaration must be an identifier not a type

When you declare a generic method or type, specify the type parameter as an identifier, for example "T" or "inputType". When client code calls the method, it supplies the type, which replaces each occurrence of the identifier in the method or class body. For more information, see [Generic Type Parameters \(C# Programming Guide\)](#).

```
// CS0081.cs
class MyClass
{
    public void F<int>() {}    // CS0081
    public void F<T>(T input) {}    // OK

    public static void Main()
    {
        MyClass a = new MyClass();
        a.F<int>(2);
        a.F<double>(.05);
    }
}
```

See Also

Concepts

[Generics \(C# Programming Guide\)](#)

Compiler Error CS0100

Error Message

The parameter name 'parameter name' is a duplicate

A method declaration used the same parameter name more than once. Parameter names must be unique in a method declaration. For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0100:

```
// CS0100.cs
namespace x
{
    public class a
    {
        public static void f(int i, char i)    // CS0100
        // try the following line instead
        // public static void f(int i, char j)
        {
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0101

Error Message

The namespace 'namespace' already contains a definition for 'type'

A [namespace](#) has duplicate identifiers. Rename or delete one of the duplicate identifiers. For more information, see [Namespaces \(C# Programming Guide\)](#)

The following sample generates CS0101:

```
// CS0101.cs
namespace MyNamespace
{
    public class MyClass
    {
        static public void Main()
        {
        }
    }

    public class MyClass    // CS0101
    {
    }
}
```

Compiler Error CS0102

Error Message

The type 'type name' already contains a definition for 'identifier'

A class contains multiple declarations for identifiers with the same name at the same scope. To fix the error, rename the duplicate identifiers.

Example

The following sample generates CS0102.

```
// CS0102.cs
// compile with: /target:library
namespace MyApp
{
    public class MyClass
    {
        string s = "Hello";
        string s = "Goodbye";    // CS0102

        public void GetString()
        {
            string s = "Hello again";    // method scope, no error
        }
    }
}
```

Compiler Error CS0103

Error Message

The name 'identifier' does not exist in the current context

An attempt was made to use a name that does not exist in the class, [namespace](#), or scope. Check the spelling of the name and check your using statements and assembly references to make sure that the name you are trying to use is available. One common mistake is to declare a variable within a loop or a try block and then attempt to access it from an enclosing code block or another code block, as shown in the following example.

The following sample generates CS0103:

```
// CS0103.cs
using System;

class MyClass
{
    public static void Main()
    {
        // MyClass conn = null;
        try
        {
            MyClass conn = new MyClass(); // delete this line
            // and uncomment the following line and the line above the try
            // conn = new MyClass();
        }
        catch(Exception e)
        {
            if (conn != null) // CS0103
                Console.WriteLine("{0}", e);
        }
    }
}
```


Compiler Error CS0104

Error Message

'reference' is an ambiguous reference between 'identifier' and 'identifier'

Your program contains [using](#) directives for two namespaces and your code references a name that appears in both namespaces.

The following sample generates CS0104:

```
// CS0104.cs
using x;
using y;

namespace x
{
    public class Test
    {
    }
}

namespace y
{
    public class Test
    {
    }
}

public class a
{
    public static void Main()
    {
        Test test = new Test();    // CS0104, is Test in x or y namespace?
        // try the following line instead
        // y.Test test = new y.Test();
    }
}
```

Compiler Error CS0106

Error Message

The modifier 'modifier' is not valid for this item

A class or interface member was marked with an invalid access modifier. The following examples describe some of these invalid modifiers:

- The **static** and **public** modifiers are not permitted on interface methods.
- The **public** keyword is not allowed on an explicit interface declaration. In this case, remove the **public** keyword from the explicit interface declaration.
- The **abstract** keyword is not allowed on an explicit interface declaration because an explicit interface implementation can never be overridden.

In prior releases of Visual Studio, the **static** modifier was not permitted on a class, but **static** classes are allowed starting with Microsoft Visual Studio 2005.

For more information, see [Interfaces \(C# Programming Guide\)](#)

Example

The following sample generates CS0106.

```
// CS0106.cs
namespace MyNamespace
{
    interface I
    {
        void m();
        static public void f();    // CS0106
    }

    public class MyClass
    {
        public void I.m() {}    // CS0106
        public static void Main() {}
    }
}
```

Compiler Error CS0107

Error Message

More than one protection modifier

Only one access modifier ([public](#), [private](#), [protected](#), or [internal](#)) is allowed on a class member, with the exception of except **internal protected**.

The following sample generates CS0107:

```
// CS0107.cs
public class C
{
    private internal void f()    // CS0107, delete private or internal
    {
    }

    public static int Main()
    {
        return 1;
    }
}
```

Compiler Error CS0110

Error Message

The evaluation of the constant value for 'const declaration' involves a circular definition

The declaration of a [const](#) variable (a) cannot reference another const variable (b) that also references (a).

The following sample generates CS0110:

```
// CS0110.cs
namespace MyNamespace
{
    public class A
    {
        public static void Main()
        {
        }
    }

    public class B : A
    {
        public const int i = c + 1;    // CS0110, c already references i
        public const int c = i + 1;
        // the following line would be OK
        // public const int c = 10;
    }
}
```

See Also

Reference

[Constants \(C# Programming Guide\)](#)

Compiler Error CS0111

Error Message

Type 'class' already defines a member called 'member' with the same parameter types

CS0111 occurs if a class contains two member declarations with the same name and parameter types. For more information, see [Methods \(C# Programming Guide\)](#).

Example

The following sample generates CS0111.

```
// CS0111.cs
class A
{
    void Test() { }
    public static void Test(){}    // CS0111

    public static void Main() {}
}
```

Compiler Error CS0112

Error Message

A static member 'function' cannot be marked as override, virtual or abstract

Any method declaration that uses the **override**, **virtual**, or **abstract** keyword cannot also use the **static** keyword.

For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0112:

```
// CS0112.cs
namespace MyNamespace
{
    abstract public class MyClass
    {
        public abstract void Foo();
    }
    public class MyClass2 : MyClass
    {
        override public static void Foo()    // CS0112, remove static keyword
        {
        }
        public static int Main()
        {
            return 0;
        }
    }
}
```

Compiler Error CS0113

Error Message

A member 'function' marked as override cannot be marked as new or virtual

It is mutually exclusive to mark a method with the [new](#) and [override](#) keywords.

The following sample generates CS0113:

```
// CS0113.cs
namespace MyNamespace
{
    abstract public class MyClass
    {
        public abstract void Foo();
    }

    public class MyClass2 : MyClass
    {
        override new public void Foo()    // CS0113, remove new keyword
        {
        }

        public static int Main()
        {
            return 0;
        }
    }
}
```

Compiler Error CS0115

Error Message

'function' : no suitable method found to override

A method was marked as an override, but the compiler found no method to override. For more information, see [override \(C# Reference\)](#), and [Knowing When to Use Override and New Keywords \(C# Programming Guide\)](#).

Example

The following sample generates CS0115. You can resolve CS0115 in one of two ways:

- Remove the **override** keyword from the method in `MyClass2`.
- Use `MyClass1` as a base class for `MyClass2`.

```
// CS0115.cs
namespace MyNamespace
{
    abstract public class MyClass1
    {
        public abstract int f();
    }

    abstract public class MyClass2
    {
        public override int f()    // CS0115
        {
            return 0;
        }

        public static void Main()
        {
        }
    }
}
```


Compiler Error CS0116

Error Message

A namespace does not directly contain members such as fields or methods

Inside a [namespace](#), the compiler only accepts classes, structures, unions, enumerations, interfaces, and delegates. This error is often generated by developers from a C/C++ background who forget that in C#, methods and variables must be declared and defined within a struct or class. For more information, see [General Structure of a C# Program \(C# Programming Guide\)](#).

Example

The following sample generates CS0116:

```
// CS0116.cs
namespace x
{
    using System;

    // method must be in class/struct
    void Method(string str) // CS0116
    {
        Console.WriteLine(str);
    }
}
```

Compiler Error CS0117

'type' does not contain a definition for 'identifier'

This error occurs when a reference is made to a member that does not exist for the data type.

Several common situations can generate this error:

- Calling a method that does not exist.
- Using the **Item** property followed by an indexer.
- Calling a qualified method when a class name and its enclosing namespace name are the same.
- Calling an interface written in a language that supports static members inside interfaces.

The following sample generates CS0117.

```
// CS0117_1.cs
namespace MyNamespace
{
    public class MyClass
    {
        public static void Main()
        {
            int i;
            i = i.get();    // CS0117
        }
    }
}
```

Example

In this example, the **Item** property is used with an indexer. In C#, you can use a property or an indexer to access a member, but not both. The following sample generates CS0117.

```
// CS0117_2.cs
using System;
using System.Collections;
class Test
{
    public static void Main()
    {
        ArrayList al = new ArrayList();
        al.Add( new Test() );
        Console.WriteLine("{0}", al.Item[0]);    // CS0117
        Console.WriteLine("{0}", al[0]);        // OK
    }
}
```

CS0017 also occurs if you use a library written in a language that allows static members in interfaces, and you try to access the static member from C#.

```
// CS0117_3.jsl
// compile with: /target:library
public interface IMyJSharpInterface
{
    static int MyStaticMember = 0;
    void NonStaticMember();
}
```

The following sample generates CS0117.

```
// CS0117_4.cs
// compile with: /reference:CS0117_3.dll
class MyCSharpClass : IMyJSharpInterface
{
    public void NonStaticMember() {}

    public static void Main()
    {
        IMyJSharpInterface myObj = new MyCSharpClass();
        myObj.NonStaticMember();
        int i = myObj.MyStaticMember;    // CS0117
    }
}
```

Compiler Error CS0118

Error Message

'construct1_name' is a 'construct1' but is used like a 'construct2'

The compiler detected a situation in which a construct was used in some erroneous way or a disallowed operation was attempted on a construct. Some common examples include:

- An attempt to instantiate a namespace (rather than a class)
- An attempt to call a field (rather than a method)
- An attempt to use a type as a variable
- An attempt to use an extern alias as a type.

To resolve this error, make sure the operation you are performing is valid for the type you are performing the operation on.

Example

The following sample generates CS0118.

```
// CS0118.cs
// compile with: /target:library
namespace MyNamespace
{
    class MyClass
    {
        // MyNamespace not a class
        MyNamespace ix = new MyNamespace ();    // CS0118
    }
}
```

Compiler Error CS0119

Error Message

'construct1_name' is a 'construct1', which is not valid in the given context.

The compiler detected an unexpected construct such as the following:

- A class constructor is not a valid test expression in a conditional statement.
- A class name was used instead of an instance name to refer to an array element.
- A method identifier is used as if it were a struct or class

Example

The following sample generates CS0119.

```
// CS0119.cs
using System;
public class MyClass
{
    public static void Test() {}

    public static void Main()
    {
        Console.WriteLine(Test.x);    // CS0119
    }
}
```

Compiler Error CS0120

Error Message

An object reference is required for the nonstatic field, method, or property 'member'

In order to use a non-static field, method, or property, you must first create an object instance. For more information, see [Instance Constructors \(C# Programming Guide\)](#).

The following sample generates CS0120:

```
// CS0120_1.cs
public class MyClass
{
    // Non-static field
    public int i;
    // Non-static method
    public void f(){}
    // Non-static property
    int Prop
    {
        get
        {
            return 1;
        }
    }

    public static void Main()
    {
        i = 10;    // CS0120
        f();       // CS0120
        int p = Prop;    // CS0120
        // try the following lines instead
        // MyClass mc = new MyClass();
        // mc.i = 10;
        // mc.f();
        // int p = mc.Prop;
    }
}
```

CS0120 will also be generated if there is a call to a non-static method from a static method, as follows:

```
// CS0120_2.cs
// CS0120 expected
using System;

public class MyClass
{
    public static void Main()
    {
        TestCall();    // CS0120
        // To call a non-static method from Main,
        // first create an instance of the class.
        // Use the following two lines instead:
        // MyClass anInstanceofMyClass = new MyClass();
        // anInstanceofMyClass.TestCall();
    }

    public void TestCall()
    {
    }
}
```

Similarly, a static method cannot call an instance method unless you explicitly give it an instance of the class, as follows:

```
// CS0120_3.cs
using System;

public class MyClass
{
    public static void Main()
    {
        do_it("Hello There");    // CS0120
    }

    private void do_it(string sText)
    // You could also add the keyword static to the method definition:
    // private static void do_it(string sText)
    {
        Console.WriteLine(sText);
    }
}
```

Compiler Error CS0121

Error Message

The call is ambiguous between the following methods or properties: 'method1' and 'method2'

Due to implicit conversion, the compiler was not able to call one form of an overloaded method. You can resolve this error in the following ways:

- Specify the method parameters in such a way that implicit conversion does not take place.
- Remove all overloads for the method.
- Cast to proper type before calling method.

The following sample generates CS0121:

```
// CS0121.cs
public class C
{
    void f(int i, double d)
    {
    }

    void f(double d, int i)
    {
    }

    public static void Main()
    {
        C c = new C();

        c.f(1, 1);    // CS0121
        // try the following line instead
        // c.f(1, 1.0);
        // or
        // c.f(1.0, 1);
        // or
        // c.f(1, (double)1);    // cast and specify which method to call
    }
}
```


Compiler Error CS0122

Error Message

'member' is inaccessible due to its protection level

The [access modifier](#) for a class member prevents accessing the member. For more information, see [Access Modifiers \(C# Programming Guide\)](#).

One cause of this (not shown in the sample below) can be omitting the **/out** compiler flag on the target of a friend assembly. For more information, see [Friend Assemblies \(C# Programming Guide\)](#) and [/out \(Set Output File Name\) \(C# Compiler Options\)](#)

Example

The following sample generates CS0122:

```
// CS0122.cs
public class MyClass
{
    // Make public to resolve CS0122
    void Foo()
    {
    }
}

public class MyClass2
{
    public static int Main()
    {
        MyClass a = new MyClass();
        // Foo is private
        a.Foo();    // CS0122
        return 0;
    }
}
```

Compiler Error CS0123

Error Message

No overload for 'method' matches delegate 'delegate'

An attempt to create a delegate failed because the correct signature was not used. Instances of a delegate must be declared with the same signature as the delegate declaration.

You can resolve this error by adjusting either the method or delegate signature. For more information, see [Delegates \(C# Programming Guide\)](#).

The following sample generates CS0123.

```
// CS0123.cs
delegate void D();
delegate void D2(int i);

public class C
{
    public static void f(int i) {}

    public static void Main()
    {
        D d = new D(f);    // CS0123
        D2 d2 = new D2(f); // OK
    }
}
```

Compiler Error CS0126

Error Message

An object of a type convertible to 'type' is required

A return statement is present, but the statement does not return a value of the expected type. For more information, see [Properties \(C# Programming Guide\)](#).

The following sample generates CS0126:

```
// CS0126.cs
public class a
{
    public int i
    {
        set
        {
        }
        get
        {
            return;    // CS0126, specify a value to return
        }
    }
}
```

Compiler Error CS0127

Error Message

Since 'function' returns void, a return keyword must not be followed by an object expression

A method with a [void](#) return type cannot return a value. For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0127:

```
// CS0127.cs
namespace MyNamespace
{
    public class MyClass
    {
        public int hiddenMember2
        {
            get
            {
                return 0;
            }
            set    // CS0127, set has an implicit void return type
            {
                return 0;    // remove return statement to resolve this CS0127
            }
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0128

Error Message

A local variable named 'variable' is already defined in this scope

The compiler detected declarations of two local variables with the same name. For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0128:

```
// CS0128.cs
namespace MyNamespace
{
    public class MyClass
    {
        public static void Main()
        {
            char i;
            int i;    // CS0128
        }
    }
}
```

Compiler Error CS0131

Error Message

The left-hand side of an assignment must be a variable, property or indexer

In an assignment statement, the value of the right-hand side is assigned to the left-hand side. The left-hand side must be a variable, property, or indexer.

To fix this error, make sure that all operators are on the right-hand side and that the left-hand side is a variable, property, or indexer. For more information, see [Statements, Expressions, and Operators \(C# Programming Guide\)](#).

Example

The following sample generates CS0131.

```
// CS0131.cs
public class MyClass
{
    public int i = 0;
    public void MyMethod()
    {
        i++ = 1;    // CS0131
        // try the following line instead
        // i = 1;
    }
    public static void Main() { }
}
```

This error can also occur if you attempt to perform arithmetic operations on the left hand side of an assignment operator, as in the following example.

```
// CS0131b.cs
public class C
{
    public static int Main()
    {
        int a = 1, b = 2, c = 3;
        if (a + b = c) // CS0131
        // try this instead
        // if (a + b == c)
            return 0;
        return 1;
    }
}
```

Compiler Error CS0132

Error Message

'constructor' : a static constructor must be parameterless

A [static](#) constructor cannot be declared with one or more parameters. For more information, see [Constructors \(C# Programming Guide\)](#).

The following sample generates CS0132:

```
// CS0132.cs
namespace MyNamespace
{
    public class MyClass
    {
        public MyClass(int i)
        {
        }
    }

    public class MyClass2 : MyClass
    {
        static MyClass2(int i)    // CS0132
        {
        }
    }
}
```

Compiler Error CS0133

Error Message

The expression being assigned to 'variable' must be constant

A [const](#) variable cannot take as its value an expression that is not constant. For more information, see [Constants \(C# Programming Guide\)](#).

The following sample generates CS0133:

```
// CS0133.cs
public class MyClass
{
    public const int i = c;    // CS0133, c is not constant
    public static int c = i;
    // try the following line instead
    // public const int i = 6;

    public static void Main()
    {
    }
}
```


Compiler Error CS0134

Error Message

'variable' is of type 'type'. A const of reference type other than string can only be initialized with null.

A constant-expression is an expression that can be fully evaluated at compile-time. Because the only way to create a non-null value of a reference-type is to apply the new operator, and because the new operator is not permitted in a constant-expression, the only possible value for constants of reference-types other than string is null.

If you encounter this error by attempting to create a [const](#) string array, the solution is to make the array [readonly](#), and initialize it in the constructor.

Example

The following example generates CS0134.

```
// CS0134.cs
// compile with: /target:library
class MyTest {}

class MyClass
{
    const MyTest test = new MyTest();    // CS0134

    //OK
    const MyTest test2 = null;
    const System.String test3 = "test";
}
```

Compiler Error CS0135

Error Message

'declaration1' conflicts with the declaration 'declaration2'

The compiler does not allow hiding names, which commonly leads to logic errors in your code.

The following sample generates CS0135:

```
// CS0135.cs
public class MyClass2
{
    public static int i = 0;

    public static void Main()
    {
        {
            int i = 4;
            i++;
        }
        i = 0;    // CS0135
    }
}
```

Compiler Error CS0136

Error Message

A local variable named 'var' cannot be declared in this scope because it would give a different meaning to 'var', which is already used in a 'parent or current/child' scope to denote something else

A variable declaration hides another declaration that would otherwise be in scope. Rename the variable that is declared on the line that generated CS0136.

The following sample generates CS0136:

```
// CS0136.cs
namespace MyNamespace
{
    public class MyClass
    {
        public static void Main()
        {
            int i = 0;
            {
                char i = 'a';    // CS0136, hides int i
            }
            i++;
        }
    }
}
```

Compiler Error CS0138

Error Message

A using namespace directive can only be applied to namespaces; 'type' is a type not a namespace

A [using](#) directive can only take the name of a namespace as a parameter. For more information, see [Namespaces \(C# Programming Guide\)](#).

The following sample generates CS0138:

```
// CS0138.cs
using System.Object;    // CS0138
```

Compiler Error CS0139

Error Message

No enclosing loop out of which to break or continue

A break or continue statement was encountered outside of a loop.

For more information, see [Jump Statements](#).

The following sample generates CS0139 twice:

```
// CS0139.cs
namespace x
{
    public class a
    {
        public static void Main()
        {
            continue; // CS0139
            break;     // CS0139
        }
    }
}
```

Compiler Error CS0140

Error Message

The label 'label' is a duplicate

A label with the same name appeared twice. For more information, see [goto \(C# Reference\)](#).

The following sample generates CS0140:

```
// CS0140.cs
namespace MyNamespace
{
    public class MyClass
    {
        public static void Main()
        {
            label1: int i = 0;
            label1: int j = 0;    // CS0140, comment this line to resolve
            goto label1;
        }
    }
}
```

Compiler Error CS0143

Error Message

The type 'class' has no constructors defined

There is no appropriate constructor available.

The following sample generates CS0143:

```
// CS0143.cs
class MyClass
{
    static public void Main ()
    {
        double d = new double(4.5);    // CS0143
    }
}
```

Compiler Error CS0144

Error Message

Cannot create an instance of the abstract class or interface 'interface'

You cannot create an instance of an [abstract](#) class or an [interface](#). For more information, see [Interfaces \(C# Programming Guide\)](#).

The following sample generates CS0144:

```
// CS0144.cs
interface MyInterface
{
}
public class MyClass
{
    public static void Main()
    {
        MyInterface myInterface = new MyInterface ();    // CS0144
    }
}
```


Compiler Error CS0145

Error Message

A const field requires a value to be provided

You must initialize a [const](#) variable. For more information, see [Constants \(C# Programming Guide\)](#).

The following sample generates CS0145:

```
// CS0145.cs
class MyClass
{
    const int i;    // CS0145
    // try the following line instead
    // const int i = 0;

    public static void Main()
    {
    }
}
```

Compiler Error CS0146

Error Message

Circular base class dependency involving 'class1' and 'class2'

The inheritance list for a class includes a direct or indirect reference to itself. A class cannot inherit from itself. For more information, see [Inheritance \(C# Programming Guide\)](#).

The following sample generates CS0146:

```
// CS0146.cs
namespace MyNamespace
{
    public interface InterfaceA
    {
    }

    public class MyClass : InterfaceA, MyClass2
    {
        public void Main()
        {
        }
    }

    public class MyClass2 : MyClass    // CS0146
    {
    }
}
```

Compiler Error CS0148

Error Message

The delegate 'delegate' does not have a valid constructor

You imported and used a managed program (one that uses the .NET Framework common language runtime) that was created with another compiler. That compiler allowed an ill-formed [delegate](#) constructor. For more information, see [Delegates \(C# Programming Guide\)](#).

Compiler Error CS0149

Error Message

Method name expected

When creating a [delegate](#), specify a method. For more information, see [Delegates \(C# Programming Guide\)](#).

The following sample generates CS0149:

```
// CS0149.cs
using System;

delegate string MyDelegate(int i);

class MyClass
{
    // class member-field of the declared delegate type
    static MyDelegate dt;

    public static void Main()
    {
        dt = new MyDelegate(17.45);    // CS0149
        // try the following line instead
        // dt = new MyDelegate(Func2);
        F(dt);
    }

    public static string Func2(int j)
    {
        Console.WriteLine(j);
        return j.ToString();
    }

    public static void F(MyDelegate myFunc)
    {
        myFunc(8);
    }
}
```

Compiler Error CS0150

Error Message

A constant value is expected

A variable was found where a constant was expected. For more information, see [switch \(C# Reference\)](#).

The following sample generates CS0150:

```
// CS0150.cs
namespace MyNamespace
{
    public class MyClass
    {
        public static void Main()
        {
            int i = 0;
            int j = 0;

            switch(i)
            {
                case j:    // CS0150, j is a variable int, not a constant int
                           // try the following line instead
                           // case 1:
            }
        }
    }
}
```

Compiler Error CS0151

Error Message

A value of an integral type expected

A variable was used in a situation where an integral data type was required. For more information, see [Data Types \(C# Programming Guide\)](#).

Example

This error can occur when there is no conversion or if the available implicit conversions result in an ambiguous situation. The following sample generates CS0151.

```
// CS0151.cs
public class MyClass
{
    public static implicit operator int (MyClass aa)
    {
        return 0;
    }

    public static implicit operator long (MyClass aa)
    {
        return 0;
    }

    public static void Main()
    {
        MyClass a = new MyClass();

        // Compiler cannot choose between int and long
        switch (a) // CS0151
        // try the following line instead
        // switch ((int)a)
        {
            case 1:
                break;
        }
    }
}
```

Compiler Error CS0152

Error Message

The label 'label' already occurs in this switch statement

A label was repeated in a [switch](#) statement. For more information, see [switch \(C# Reference\)](#).

The following sample generates CS0152:

```
// CS0152.cs
namespace MyNamespace
{
    public class MyClass
    {
        public static void Main()
        {
            int i = 0;

            switch (i)
            {
                case 1:
                    i++;
                    return;

                case 1:    // CS0152, two case 1 statements
                    i++;
                    return;
            }
        }
    }
}
```

Compiler Error CS0153

Error Message

A goto case is only valid inside a switch statement

An attempt was made to use [switch](#) syntax outside of a **switch** statement. For more information, see [switch \(C# Reference\)](#).

The following sample generates CS0153:

```
// CS0153.cs
public class a
{
    public static void Main()
    {
        goto case 5;    // CS0153
    }
}
```


Compiler Error CS0154

Error Message

The property or indexer 'property' cannot be used in this context because it lacks the get accessor

An attempt to use a [property](#) failed because no get accessor method was defined in the property. For more information, see [Fields \(C# Programming Guide\)](#).

Example

The following sample generates CS0154:

```
// CS0154.cs
public class MyClass2
{
    public int i
    {
        set
        {
        }
        // uncomment the get method to resolve this error
        /*
        get
        {
            return 0;
        }
        */
    }
}

public class MyClass
{
    public static void Main()
    {
        MyClass2 myClass2 = new MyClass2();
        int j = myClass2.i;    // CS0154, no get method
    }
}
```

Compiler Error CS0155

Error Message

The type caught or thrown must be derived from System.Exception

An attempt was made to pass a data type that does not derive from **System.Exception** into a [catch](#) block. Only data types that derive from **System.Exception** can be passed into a **catch** block. For more information, see [Exception Handling Statements](#) and [Exceptions and Exception Handling \(C# Programming Guide\)](#).

The following sample generates CS0155:

```
// CS0155.cs
using System;

namespace MyNamespace
{
    public class MyClass2
        // try the following line instead
        // public class MyClass2 : Exception
    {
    }
    public class MyClass
    {
        public static void Main()
        {
            try
            {
            }
            catch (MyClass2)    // CS0155, resolves if you derive MyClass2 from Exception
            {
            }
        }
    }
}
```

Compiler Error CS0156

Error Message

A throw statement with no arguments is not allowed outside of a catch clause

A [throw](#) statement with no parameters can only appear in a **catch** clause that takes no parameters.

For more information, see [Exception Handling Statements](#) and [Exceptions and Exception Handling \(C# Programming Guide\)](#).

The following sample generates CS0156:

```
// CS0156.cs
using System;

namespace MyNamespace
{
    public class MyClass2 : Exception
    {
    }

    public class MyClass
    {
        public static void Main()
        {
            try
            {
                throw;    // CS0156
            }

            catch(MyClass2)
            {
                throw;    // this throw is valid
            }
        }
    }
}
```

Compiler Error CS0157

Error Message

Control cannot leave the body of a finally clause

All of the statements in a [finally](#) clause must execute. For more information, see [Exception Handling Statements](#) and [Exceptions and Exception Handling \(C# Programming Guide\)](#).

The following sample generates CS0157:

```
// CS0157.cs
using System;
namespace MyNamespace
{
    public class MyClass2 : Exception
    {
    }

    public class MyClass
    {
        public static void Main()
        {
            try
            {
            }

            finally
            {
                return;    // CS0157, cannot leave finally clause
            }
        }
    }
}
```

Compiler Error CS0158

Error Message

The label 'label' shadows another label by the same name in a contained scope

A label in an inner scope hides a label with the same name in an outer scope. For more information, see [goto \(C# Reference\)](#).

The following sample generates CS0158:

```
// CS0158.cs
namespace MyNamespace
{
    public class MyClass
    {
        public static void Main()
        {
            goto lab1;
            lab1:
            {
                lab1:
                goto lab1;    // CS0158
            }
        }
    }
}
```

Compiler Error CS0159

Error Message

No such label 'label' within the scope of the goto statement

The label referenced by the [goto](#) statement could not be found within the scope of the **goto** statement.

The following sample generates CS0159:

```
// CS0159.cs
public class Class1
{
    public static void Main()
    {
        int i = 0;

        switch (i)
        {
            case 1:
                goto case 3;    // CS0159, case 3 label does not exist
            case 2:
                break;
        }
        goto NOWHERE;    // CS0159, NOWHERE label does not exist
    }
}
```

Compiler Error CS0160

Error Message

A previous catch clause already catches all exceptions of this or of a super type ('type')

A series of **catch** statements needs to be in decreasing order of derivation. For example, the most derived objects must appear first.

For more information, see [Exception Handling Statements](#) and [Exceptions and Exception Handling \(C# Programming Guide\)](#).

The following sample generates CS0160:

```
// CS0160.cs
public class MyClass2 : System.Exception {}
public class MyClass
{
    public static void Main()
    {
        try {}

        catch(System.Exception) {}    // Second-most derived; should be second catch
        catch(MyClass2) {}           // CS0160 Most derived; should be first catch
    }
}
```

Compiler Error CS0161

Error Message

'method': not all code paths return a value

A method that returns a value must have a **return** statement in all code paths. For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0161:

```
// CS0161.cs
public class Test
{
    public static int Main() // CS0161
    {
        int i = 10;
        if (i < 10)
        {
            return i;
        }
        else
        {
            // uncomment the following line to resolve
            // return 1;
        }
    }
}
```


Compiler Error CS0163

Error Message

Control cannot fall through from one case label ('label') to another

When a [case statement](#) contains one or more statements and is followed by another **case** statement, one of the following statements must explicitly terminate the **case**:

- **return**
- **goto**
- **break**

If you want to implement "fall through" behavior, use `goto case #`. For more information, see [switch \(C# Reference\)](#)

The following sample generates CS0163:

```
// CS0163.cs
public class MyClass
{
    public static void Main()
    {
        int i = 0;

        switch (i)    // CS0163
        {
            case 1:
                i++;
                // uncomment one of the following lines to resolve
                // return;
                // break;
                // goto case 3;

            case 2:
                i++;
                return;

            case 3:
                i = 0;
                return;
        }
    }
}
```

Compiler Error CS0165

Error Message

Use of unassigned local variable 'var'

The C# compiler does not allow the use of uninitialized variables. If the compiler detects the use of a variable that might not have been initialized, it generates CS0165. For more information, see [Fields \(C# Programming Guide\)](#).

Use [new](#) to create an instance of an object or assign a value.

The following sample generates CS0165:

```
// CS0165.cs
using System;

class MyClass
{
    public int i;
}

class MyClass2
{
    public static void Main(string [] args)
    {
        int i, j;
        if (args[0] == "test")
        {
            i = 0;
        }

        /*
        // to resolve, either initialize the variables when declared
        // or provide for logic to initialize them, as follows:
        else
        {
            i = 1;
        }
        */

        j = i;    // CS0165, i might be uninitialized

        MyClass myClass;
        myClass.i = 0;    // CS0165
        // use new as follows
        // MyClass myClass = new MyClass();
        // myClass.i = 0;
    }
}
```

Compiler Error CS0167

Error Message

The delegate 'delegate' is missing the Invoke method

You imported and used a managed program (one that uses the .NET Framework common language runtime) that was created with another compiler. That compiler allowed an ill-formed [delegate](#). Therefore, the **Invoke** method was not available. For more information, see [Delegates \(C# Programming Guide\)](#).

Use of possibly unassigned field 'field'

The following sample generates CS0170:

[illegible]

Compiler Error CS0171

Error Message

Field 'field' must be fully assigned before control leaves the constructor

A constructor in a [struct](#) must initialize all fields in the struct. For more information, see [Constructors \(C# Programming Guide\)](#).

The following sample generates CS0171:

```
// CS0171.cs
struct MyStruct
{
    MyStruct(int initField)    // CS0171
    {
        // i = initField;      // uncomment this line to resolve this error
    }
    public int i;
}

class MyClass
{
    public static void Main()
    {
        MyStruct aStruct = new MyStruct();
    }
}
```

Compiler Error CS0172

Error Message

Type of conditional expression cannot be determined because 'type1' and 'type2' implicitly convert to one another

In a conditional statement, you must be able to convert the types on either side of the `:` operator. Also, there cannot be mutual conversion routines; you only need one conversion. For more information, see

[Conversion Operators \(C# Programming Guide\)](#).

The following sample generates CS0172:

```
// CS0172.cs
public class Square
{
    public class Circle
    {
        public static implicit operator Circle(Square aa)
        {
            return null;
        }

        public static implicit operator Square(Circle aa)
        // using explicit resolves this error
        // public static explicit operator Square(Circle aa)
        {
            return null;
        }
    }

    public static void Main()
    {
        Circle aa = new Circle();
        Square ii = new Square();
        object o = (1 == 1) ? aa : ii;    // CS0172
        // the following cast would resolve this error
        // (1 == 1) ? aa : (Circle)ii;
    }
}
```

Compiler Error CS0173

Error Message

Type of conditional expression cannot be determined because there is no implicit conversion between 'class1' and 'class2'

Conversions between classes are useful when you want objects of different classes to work with the same code. However, two classes that will work together cannot have mutual and redundant conversions.

To resolve CS0173, verify that there is one and only one implicit conversion between *class1* and *class2*, regardless of which direction the conversion is in and regardless of which class the conversion is in. For more information, see [Implicit Numeric Conversions Table \(C# Reference\)](#) and [Conversion Operators \(C# Programming Guide\)](#)

The following sample generates CS0173:

```
// CS0173.cs
public class C {}
public class A {}

public class MyClass
{
    public static void F(bool b)
    {
        A a = new A();
        C c = new C();
        object o = b ? a : c; // CS0173
    }

    public static void Main()
    {
        F(true);
    }
}
```

Compiler Error CS0174

Error Message

A base class is required for a 'base' reference

This error occurs only when the .NET Framework common language runtime compiles the source code for the **System.Object** class, which is the only class that has no base class.

Compiler Error CS0175

Error Message

Use of keyword 'base' is not valid in this context

The [base \(C# Reference\)](#) keyword must be used to specify a particular member of the base class. For more information, see [Constructors \(C# Programming Guide\)](#).

The following sample generates CS0175:

```
// CS0175.cs
using System;
class BaseClass
{
    public int TestInt = 0;
}

class MyClass : BaseClass
{
    public static void Main()
    {
        MyClass aClass = new MyClass();
        aClass.BaseTest();
    }

    public void BaseTest()
    {
        Console.WriteLine(base); // CS0175
        // Try the following line instead:
        // Console.WriteLine(base.TestInt);
        base = 9;    // CS0175

        // Try the following line instead:
        // base.TestInt = 9;
    }
}
```

Compiler Error CS0176

Error Message

Static member 'member' cannot be accessed with an instance reference; qualify it with a type name instead

Only a class name can be used to qualify a [static](#) variable; an instance name cannot be a qualifier. For more information, see [Static Classes and Static Class Members \(C# Programming Guide\)](#).

The following sample generates CS0176:

```
// CS0176.cs
public class MyClass2
{
    public static int ii;
}

public class a
{
    public static void Main()
    {
        MyClass2 myClass2 = new MyClass2 ();
        int i = myClass2.ii;    // CS0176
        // try the following line instead
        // int i = MyClass2.ii;
    }
}
```

Compiler Error CS0177

Error Message

The out parameter 'parameter' must be assigned to before control leaves the current method

A parameter marked with the [out](#) keyword was not assigned a value in the method body. For more information, see [Passing Parameters \(C# Programming Guide\)](#)

The following sample generates CS0177:

```
// CS0177.cs
public class MyClass
{
    public static void Foo(out int i)    // CS0177
    {
        // uncomment the following line to resolve this error
        // i = 0;
    }

    public static void Main()
    {
        int x = -1;
        Foo(out x);
    }
}
```

Compiler Error CS0178

Error Message

Invalid rank specifier: expected ',' or ']'

An array initialization was ill-formed. For example, when specifying the array dimensions, you can specify the following:

- A number in brackets
- Empty brackets
- A comma enclosed in brackets

For more information, see [Arrays \(C# Programming Guide\)](#) and the C# specification ([C# Language Specification](#)) section on array initializers.

Example

The following sample generates CS0178.

```
// CS0178.cs
class MyClass
{
    public static void Main()
    {
        int a = new int[5][,][5];    // CS0178
        int[,] b = new int[3,2];    // OK

        int[][] c = new int[10][];
        c[0] = new int[5][5];    // CS0178
        c[0] = new int[2];    // OK
        c[1] = new int[2]{1,2};    // OK
    }
}
```

Compiler Error CS0179

Error Message

'member' cannot be extern and declare a body

When a class member is marked [extern](#), it means that the member's definition is located in another file. Therefore, a class member marked as **extern** cannot be defined in the class. Either remove the **extern** keyword or delete the definition. For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0179:

```
// CS0179.cs
public class MyClass
{
    public extern int ExternMethod(int aa)    // CS0179
    {
        return 0;
    }
    // try the following line instead
    // public extern int ExternMethod(int aa);

    public static void Main()
    {
    }
}
```

Compiler Error CS0180

Error Message

'member' cannot be both extern and abstract

The [abstract](#) and [extern](#) keywords are mutually exclusive. The **extern** keyword means that the member is defined outside the file, and **abstract** means that the implementation is provided in a derived class. For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0180:

```
// CS0180.cs
namespace MyNamespace
{
    public class MyClass
    {
        public extern abstract int Foo(int a);    // CS0180

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0182

Error Message

An attribute argument must be a constant expression, typeof expression or array creation expression

An argument to an attribute was not specified correctly. For more information, see [Global Attributes \(C# Programming Guide\)](#).

Example

The following sample generates CS0182:

```
// CS0182.cs
public class MyClass
{
    static string s = "Test";

    [System.Diagnostics.ConditionalAttribute(s)]    // CS0182
    // try the following line instead
    // [System.Diagnostics.ConditionalAttribute("Test")]
    void NonConstantArgumentToConditional()
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0185

Error Message

'type' is not a reference type as required by the lock statement

The [lock](#) statement can only evaluate reference types. For more information, see [Thread Synchronization \(C# Programming Guide\)](#) and [Reference Types](#).

Example

The following sample generates CS0185:

```
// CS0185.cs
public class MainClass
{
    public static void Main ()
    {
        lock (1)    // CS0185
        // try the following lines instead
        // MainClass x = new MainClass();
        // lock(x)
        {
        }
    }
}
```


Compiler Error CS0186

Error Message

Use of null is not valid in this context

The following sample generates CS0186:

```
// CS0186.cs
using System;
using System.Collections;

class MyClass
{
    static void Main()
    {
        // Each of the following lines generates CS0186:
        foreach (int i in null) {}    // CS0186
        foreach (int i in (IEnumerable) null) { };    // CS0186
    }
}
```

Compiler Error CS0188

Error Message

The 'this' object cannot be used before all of its fields are assigned to

All fields in a **struct** have to be assigned by a constructor before the constructor can call a method in the **struct**. For more information, see [Using Structs \(C# Programming Guide\)](#).

The following sample generates CS0188:

```
// CS0188.cs
// compile with: /t:library
namespace MyNamespace
{
    class MyClass
    {
        struct S
        {
            public int a;

            void Foo()
            {
            }

            S(int i)
            {
                // a = i;
                Foo(); // CS0188
            }
        }
        public static void Main()
        { }
    }
}
```

Compiler Error CS0191

Error Message

A readonly field cannot be assigned to (except in a constructor or a variable initializer)

A [readonly](#) field can only take an assignment in a constructor or at declaration. For more information, see [Constructors \(C# Programming Guide\)](#).

CS0191 also results if the **readonly** field is [static](#) and the constructor is not marked **static**.

Example

The following sample generates CS0191.

```
// CS0191.cs
class MyClass
{
    public readonly int TestInt = 6; // OK to assign to readonly field in declaration

    MyClass()
    {
        TestInt = 11; // OK to assign to readonly field in constructor
    }

    public void TestReadOnly()
    {
        TestInt = 19; // CS0191
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0192

Error Message

A readonly field cannot be passed ref or out (except in a constructor)

A field (variable) marked with the [readonly](#) keyword cannot be passed either to a [ref](#) or [out](#) parameter except inside a constructor. For more information, see [Fields \(C# Programming Guide\)](#).

CS0192 also results if the **readonly** field is [static](#) and the constructor is not marked **static**.

Example

The following sample generates CS0192.

```
// CS0192.cs
class MyClass
{
    public readonly int TestInt = 6;
    static void TestMethod(ref int testInt)
    {
        testInt = 0;
    }

    MyClass()
    {
        TestMethod(ref TestInt);    // OK
    }

    public void PassReadOnlyRef()
    {
        TestMethod(ref TestInt);    // CS0192
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0193

Error Message

The * or -> operator must be applied to a pointer

The * or -> operator was used with a nonpointer type. For more information, see [Pointer types \(C# Programming Guide\)](#).

The following sample generates CS0193:

```
// CS0193.cs
using System;

public struct Age
{
    public int AgeYears;
    public int AgeMonths;
    public int AgeDays;
}

public class MyClass
{
    public static void SetAge(ref Age anAge, int years, int months, int days)
    {
        anAge->Months = 3;    // CS0193, anAge is not a pointer
        // try the following line instead
        // anAge.AgeMonths = 3;
    }

    public static void Main()
    {
        Age MyAge = new Age();
        Console.WriteLine(MyAge.AgeMonths);
        SetAge(ref MyAge, 22, 4, 15);
        Console.WriteLine(MyAge.AgeMonths);
    }
}
```

Compiler Error CS0196

Error Message

A pointer must be indexed by only one value

A pointer cannot have multiple indices. For more information, see [Pointer types \(C# Programming Guide\)](#)

The following sample generates CS0196:

```
// CS0196.cs
public class MyClass
{
    public static void Main ()
    {
        int *i = null;
        int j = 0;
        j = i[1,2];    // CS0196
        // try the following line instead
        // j = i[1];
    }
}
```

Compiler Warning (level 1) CS0197

Error Message

Passing 'argument' as ref or out or taking its address may cause a runtime exception because it is a field of a marshal-by-reference class

Any class that derives, directly or indirectly, from [MarshalByRefObject](#) is a marshal-by-reference class. Such a class can be marshaled by reference across process and machine boundaries. Thus, instances of this class could be proxies to remote objects. You cannot pass a field of a proxy object as [ref](#) or [out](#). So, you cannot pass fields of such a class as **ref** or **out**, unless the instance is [this](#), which can not be a proxy object.

Example

The following sample generates CS0197.

```
// CS0197.cs
// compile with: /W:1
class X : System.MarshalByRefObject
{
    public int i;
}

class M
{
    public int i;
    static void AddSeventeen(ref int i)
    {
        i += 17;
    }

    static void Main()
    {
        X x = new X();
        x.i = 12;
        AddSeventeen(ref x.i);    // CS0197

        // OK
        M m = new M();
        m.i = 12;
        AddSeventeen(ref m.i);
    }
}
```

Compiler Error CS0198

Error Message

A static readonly field cannot be assigned to (except in a static constructor or a variable initializer)

A [readonly](#) variable must have the same [static](#) usage as the constructor in which you want to initialize it. For more information, see [Static Constructors \(C# Programming Guide\)](#).

The following sample generates CS0198:

```
// CS0198.cs
class MyClass
{
    public static readonly int TestInt = 6;

    MyClass()
    {
        TestInt = 11;    // CS0198, constructor is not static and readonly field is
    }

    public static void Main()
    {
    }
}
```


Compiler Error CS0199

Error Message

A static readonly field cannot be passed ref or out (except in a static constructor)

A [readonly](#) variable must have the same [static](#) usage as the constructor in which you want to pass it as a [ref](#) or [out](#) parameter. For more information, see [Passing Parameters \(C# Programming Guide\)](#).

Example

The following sample generates CS0199:

```
// CS0199.cs
class MyClass
{
    public static readonly int TestInt = 6;

    static void TestMethod(ref int testInt)
    {
        testInt = 0;
    }

    MyClass()
    {
        TestMethod(ref TestInt);    // CS0199, TestInt is static
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0200

Error Message

Property or indexer 'property' cannot be assigned to — it is read only

An attempt was made to assign a value to a [property](#), but the property does not have a set accessor. Resolve the error by adding a set accessor. For more information, see [How to: Declare and Use Read/Write Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS0200:

```
// CS0200.cs
public class MainClass
{
    // private int _mi;
    int I
    {
        get
        {
            return 1;
        }

        // uncomment the set accessor and declaration for _mi
        /*
        set
        {
            _mi = value;
        }
        */
    }

    public static void Main ()
    {
        MainClass II = new MainClass();
        II.I = 9;    // CS0200
    }
}
```

Compiler Error CS0201

Error Message

Only assignment, call, increment, decrement, and new object expressions can be used as a statement

The compiler generates an error when it encounters a meaningless statement.

Example

The following sample generates CS0201.

```
// CS0201.cs
public class MainClass
{
    public static void Main()
    {
        2 * 3;    // CS0201
    }
}
```

The following sample generates CS0201.

```
// CS0201_b.cs
// compile with: /target:library
public class MyList<T>
{
    public void Add(T x)
    {
        int i = 0;
        if ( (object)x == null)
        {
            checked(i++);    // CS0201

            // OK
            checked {
                i++;
            }
        }
    }
}
```

Compiler Error CS0202

Error Message

foreach requires that the return type 'type' of 'type.GetEnumerator()' must have a suitable public MoveNext method and public Current property

A [GetEnumerator](#) function, used to enable the use of foreach statements, cannot return a pointer or array; it must return an instance of a class that is able to act as an enumerator. The proper requirements to serve as an enumerator include a public Current property and a public MoveNext method.

Note

In C# 2.0, the compiler will automatically generate Current and MoveNext for you. For more information, see the code example in [Generic Interfaces \(C# Programming Guide\)](#).

The following sample generates CS0202:

```
// CS0202.cs

public class C1
{
    public int Current
    {
        get
        {
            return 0;
        }
    }

    public bool MoveNext ()
    {
        return false;
    }

    public static implicit operator C1 (int c1)
    {
        return 0;
    }
}

public class C2
{
    public int Current
    {
        get
        {
            return 0;
        }
    }

    public bool MoveNext ()
    {
        return false;
    }

    public C1[] GetEnumerator ()
    // try the following line instead
    // public C1 GetEnumerator ()
    {
        return null;
    }
}

public class MainClass
```

```
{
    public static void Main ()
    {
        C2 c2 = new C2();

        foreach (C1 x in c2)    // CS0202
        {
            System.Console.WriteLine(x.Current);
        }
    }
}
```

Compiler Error CS0204

Error Message

Only 65535 locals are allowed

The limit of 65535 local variables has been exceeded in a method.

Compiler Error CS0205

Error Message

Cannot call an abstract base member: 'method'

You cannot call an [abstract](#) method because it does not have a method body. For more information, see [Abstract and Sealed Classes and Class Members \(C# Programming Guide\)](#).

The following sample generates CS0205:

```
// CS0205.cs
abstract public class MyClass
{
    abstract public void mf();
}

public class MyClass2 : MyClass
{
    public override void mf()
    {
        base.mf();    // CS0205, delete this line
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0206

Error Message

A property or indexer may not be passed as an out or ref parameter

A [property](#) is not available to be passed as a [ref](#) or [out](#) parameter. For more information, see [Passing Parameters \(C# Programming Guide\)](#).

Example

The following sample generates CS0206:

```
// CS0206.cs
public class MyClass
{
    public static int P
    {
        get
        {
            return 0;
        }
        set
        {
        }
    }

    public static void MyMeth(ref int i)
    // public static void MyMeth(int i)
    {
    }

    public static void Main()
    {
        MyMeth(ref P);    // CS0206
        // try the following line instead
        // MyMeth(P);    // CS0206
    }
}
```


Compiler Error CS0208

Error Message

Cannot take the address of, get the size of, or declare a pointer to a managed type ('type')

Even when used with the [unsafe](#) keyword, taking the address of a managed object, getting the size of a managed object, or declaring a pointer to a managed type is not allowed. For more information, see

[Unsafe Code and Pointers \(C# Programming Guide\)](#).

Example

The following sample generates CS0208:

```
// CS0208.cs
// compile with: /unsafe

class S
{
    public int a = 98;
}

public class MyClass
{
    unsafe public static int Main()
    {
        S s = new S(); // S is managed
        S * s2 = &s;    // CS0208
        return 1;
    }
}
```

Compiler Error CS0209

Error Message

The type of locals declared in a fixed statement must be a pointer type

The variable that you declare in a [fixed statement](#) must be a pointer. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample generates CS0209:

```
// CS0209.cs
// compile with: /unsafe

class Point
{
    public int x, y;
}

public class MyClass
{
    unsafe public static void Main()
    {
        Point pt = new Point();

        fixed (int i)    // CS0209
        {
        }
        // try the following lines instead
        /*
        fixed (int* p = &pt.x)
        {
        }
        fixed (int* q = &pt.y)
        {
        }
        */
    }
}
```

Compiler Error CS0210

Error Message

You must provide an initializer in a fixed or using statement declaration

You must declare and initialize the variable in a [fixed statement](#). For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample generates CS0210:

```
// CS0210a.cs
// compile with: /unsafe

class Point
{
    public int x, y;
}

public class MyClass
{
    unsafe public static void Main()
    {
        Point pt = new Point();

        fixed (int i)    // CS0210
        {
        }
        // try the following lines instead
        /*
        fixed (int* p = &pt.x)
        {
        }
        fixed (int* q = &pt.y)
        {
        }
        */
    }
}
```

The following sample also generates CS0210 because the [using statement](#) has no initializer.

```
// CS0210b.cs

using System.IO;
class Test
{
    static void Main()
    {
        using (StreamWriter w) // CS0210
        // Try this line instead:
        // using (StreamWriter w = new StreamWriter("TestFile.txt"))
        {
            w.WriteLine("Hello there");
        }
    }
}
```

Compiler Error CS0211

Error Message

Cannot take the address of the given expression

You can take the address of fields, local variables, and indirection of pointers, but you cannot take, for example, the address of the sum of two local variables. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample generates CS0211:

```
// CS0211.cs
// compile with: /unsafe

public class MyClass
{
    unsafe public void mf()
    {
        int a = 0, b = 0;
        int *i = &(a + b);    // CS0211, the addition of two local variables
        // try the following line instead
        // int *i = &a;
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0212

Error Message

You can only take the address of an unfixed expression inside of a fixed statement initializer

For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample shows how to take the address of an unfixed expression. The following sample generates CS0212.

```
// CS0212a.cs
// compile with: /unsafe /target:library

public class A {
    public int iField = 5;

    unsafe public void mf() {
        A a = new A();
        int* ptr = &a.iField;    // CS0212
    }

    // OK
    unsafe public void mf2() {
        A a = new A();
        fixed (int* ptr = &a.iField) {}
    }
}
```

The following sample also generates CS0212 and shows how to resolve the error:

```
// CS0212b.cs
// compile with: /unsafe /target:library
using System;

public class MyClass
{
    unsafe public void mf()
    {
        // Null-terminated ASCII characters in an sbyte array
        sbyte[] sbArr1 = new sbyte[] { 0x41, 0x42, 0x43, 0x00 };
        sbyte* pAsciiUpper = &sbArr1[0];    // CS0212
        // To resolve this error, delete the previous line and
        // uncomment the following code:
        // fixed (sbyte* pAsciiUpper = sbArr1)
        // {
        //     String szAsciiUpper = new String(pAsciiUpper);
        // }
    }
}
```

Compiler Error CS0213

Error Message

You cannot use the fixed statement to take the address of an already fixed expression

A local variable in an [unsafe](#) method or a parameter is already fixed (on the stack), so you cannot take the address of either of these two variables in a [fixed](#) expression. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

Example

The following sample generates CS0213.

```
// CS0213.cs
// compile with: /unsafe
public class MyClass
{
    unsafe public static void Main()
    {
        int i = 45;
        fixed (int *j = &i) { } // CS0213
        // try the following line instead
        // int* j = &i;

        int[] a = new int[] {1,2,3};
        fixed (int *b = a)
        {
            fixed (int *c = b) { } // CS0213
            // try the following line instead
            // int *c = b;
        }
    }
}
```

Compiler Error CS0214

Error Message

Pointers and fixed size buffers may only be used in an unsafe context

Pointers can only be used with the [unsafe](#) keyword. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample generates CS0214:

```
// CS0214.cs
// compile with: /target:library /unsafe
public struct S
{
    public int a;
}

public class MyClass
{
    public static void Test()
    {
        S s = new S();
        S * s2 = &s;    // CS0214
        s2->a = 3;       // CS0214
        s.a = 0;
    }

    // OK
    unsafe public static void Test2()
    {
        S s = new S();
        S * s2 = &s;
        s2->a = 3;
        s.a = 0;
    }
}
```

Compiler Error CS0215

Error Message

The return type of operator True or False must be bool

User-defined [true](#) and [false](#) operators must have a return type of [bool](#). For more information, see [Overloadable Operators \(C# Programming Guide\)](#).

The following sample generates CS0215:

```
// CS0215.cs
class MyClass
{
    public static int operator true (MyClass MyInt)    // CS0215
    // try the following line instead
    // public static bool operator true (MyClass MyInt)
    {
        return true;
    }

    public static int operator false (MyClass MyInt)   // CS0215
    // try the following line instead
    // public static bool operator false (MyClass MyInt)
    {
        return true;
    }

    public static void Main()
    {
    }
}
```


Compiler Error CS0216

Error Message

The operator 'operator' requires a matching operator 'missing_operator' to also be defined

A user-defined [true](#) operator requires a user-defined [false](#) operator, and vice versa. For more information, see [Operators \(C# Programming Guide\)](#).

The following sample generates CS0216:

```
// CS0216.cs
class MyClass
{
    public static bool operator true (MyClass MyInt)    // CS0216
    {
        return true;
    }

    // to resolve, uncomment the following operator definition
    /*
    public static bool operator false (MyClass MyInt)
    {
        return true;
    }
    */

    public static void Main()
    {
    }
}
```

Compiler Error CS0217

Error Message

In order to be applicable as a short circuit operator a user-defined logical operator ('operator') must have the same return type as the type of its 2 parameters.

If you define an operator for a user-defined type, and then try to use the operator as a short-circuit operator, the user-defined operator must have parameters and return values of the same type. For more information on short-circuit operators, see [&& Operator](#) and [|| Operator](#).

The following sample generates CS0217:

```
// CS0217.cs
using System;

public class MyClass
{
    public static bool operator true (MyClass f)
    {
        return false;
    }

    public static bool operator false (MyClass f)
    {
        return false;
    }

    public static implicit operator int(MyClass x)
    {
        return 0;
    }

    public static int operator & (MyClass f1, MyClass f2)    // CS0217
    // try the following line instead
    // public static MyClass operator & (MyClass f1, MyClass f2)
    {
        return new MyClass();
    }

    public static void Main()
    {
        MyClass f = new MyClass();
        int i = f && f;
    }
}
```

See Also

Reference

[Overloadable Operators \(C# Programming Guide\)](#)

Compiler Error CS0218

Error Message

The type ('type') must contain declarations of operator true and operator false

If you define an operator for a user-defined type, and then try to use the operator as a short-circuit operator, the user-defined operator must have operator true and operator false defined. For more information on short-circuit operators, see [&& Operator](#) and [|| Operator](#).

The following sample generates CS0218:

```
// CS0218.cs
using System;
public class MyClass
{
    // uncomment these operator declarations to resolve this CS0218
    /*
    public static bool operator true (MyClass f)
    {
        return false;
    }

    public static bool operator false (MyClass f)
    {
        return false;
    }
    */

    public static implicit operator int(MyClass x)
    {
        return 0;
    }

    public static MyClass operator & (MyClass f1, MyClass f2)
    {
        return new MyClass();
    }

    public static void Main()
    {
        MyClass f = new MyClass();
        int i = f && f;    // CS0218, requires operators true and false
    }
}
```

See Also

Concepts

[Conversion Operators \(C# Programming Guide\)](#)

Compiler Error CS0220

Error Message

The operation overflows at compile time in checked mode

An operation was detected by [checked](#), which is the default, and a data loss resulted. Either correct the inputs to the assignment or use [unchecked](#) to resolve this error. For more information, see [Checked and Unchecked \(C# Reference\)](#).

The following sample generates CS0220:

```
// CS0220.cs
using System;

class TestClass
{
    const int x = 1000000;
    const int y = 1000000;

    public int MethodCh()
    {
        int z = (x * y);    // CS0220
        return z;
    }

    public int MethodUnCh()
    {
        unchecked
        {
            int z = (x * y);
            return z;
        }
    }

    public static void Main()
    {
        TestClass myObject = new TestClass();
        Console.WriteLine("Checked : {0}", myObject.MethodCh());
        Console.WriteLine("Unchecked: {0}", myObject.MethodUnCh());
    }
}
```

Compiler Error CS0221

Error Message

Constant value 'value' cannot be converted to a 'type' (use 'unchecked' syntax to override)

An assignment operation that would result in a data loss was detected by [checked](#), which is on by default. Either correct the assignment or use [unchecked](#) to resolve this error. For more information, see [Checked and Unchecked \(C# Reference\)](#).

The following sample generates CS0221:

```
// CS0221.cs
public class MyClass
{
    public static void Main()
    {
        // unchecked
        // {
        int a = (int)0xFFFFFFFF;    // CS0221
        a++;
        // }
    }
}
```

Compiler Error CS0225

Error Message

The params parameter must be a single dimensional array

When using the [params](#) keyword, you must specify a single-dimensional array of the data type. For more information, see [Methods \(C# Programming Guide\)](#).

Example

The following sample generates CS0225:

```
// CS0225.cs
public class MyClass
{
    public static void TestParams(params int a)    // CS0225
    // try the following line instead
    // public static void TestParams(params int[] a)
    {
    }

    public static void Main()
    {
        TestParams(1);
    }
}
```

Compiler Error CS0227

Error Message

Unsafe code may only appear if compiling with /unsafe

If source code contains the [unsafe](#) keyword, then the [/unsafe](#) compiler option must also be used. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample, when compiled without **/unsafe**, will generate CS0227:

```
// CS0227.cs
public class MyClass
{
    unsafe public static void Main()    // CS0227
    {
    }
}
```

Compiler Error CS0228

Error Message

'type' does not contain a definition for 'member', or it is not accessible

You may have replaced the system version of **System.Hashtable**, **System.String**, or **System.Array**, and the replacement does not contain *member*.

Otherwise, you may need to repair or reinstall Visual Studio.

Compiler Error CS0229

Error Message

Ambiguity between 'member1' and 'member2'

Members of different interfaces have the same name. If you want to keep the same names, you must qualify the names. For more information, see [Interfaces \(C# Programming Guide\)](#).

Note

In some cases, this ambiguity can be resolved by providing an explicit prefix to the identifier via a [using](#) alias.

Example

The following example generates CS0229:

```
// CS0229.cs

interface IList
{
    int Count
    {
        get;
        set;
    }

    void Counter();
}

interface Icounter
{
    double Count
    {
        get;
        set;
    }
}

interface IListCounter : IList , Icounter {}

class MyClass
{
    void Test(IListCounter x)
    {
        x.Count = 1; // CS0229
        // Try one of the following lines instead:
        // ((IList)x).Count = 1;
        // or
        // ((Icounter)x).Count = 1;
    }

    public static void Main() {}
}
```

Compiler Error CS0230

Error Message

Type and identifier are both required in a foreach statement

A [foreach](#) statement was poorly formed.

The following sample generates CS0230:

```
// CS0230.cs
using System;

class MyClass
{
    public static void Main()
    {
        int[] myarray = new int[3] {1,2,3};

        foreach (int in myarray)    // CS0230
        // try the following line instead
        // foreach (int x in myarray)
        {
            Console.WriteLine(x);
        }
    }
}
```

Compiler Error CS0231

Error Message

A params parameter must be the last parameter in a formal parameter list.

The [params](#) parameter supports a variable numbers of arguments and must be after all other parameters. For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0231:

```
// CS0231.cs
class Test
{
    public void TestMethod(params int[] p, int i) {} // CS0231
    // To resolve the error, use the following line instead:
    // public void TestMethod(int i, params int[] p) {}

    static void Main()
    {
    }
}
```

Compiler Error CS0233

Error Message

'identifier' does not have a predefined size, therefore sizeof can only be used in an unsafe context (consider using `System.Runtime.InteropServices.Marshal.SizeOf`)

The [sizeof](#) operator can only be used for types that are compile-time constants. If you are getting this error, make sure that the size of the identifier can be determined at compile time. If it cannot, then use [SizeOf](#) instead of **sizeof**.

Example

The following example generates CS0233:

```
// CS0233.cs
using System;
using System.Runtime.InteropServices;

[StructLayout(LayoutKind.Sequential)]
public struct S
{
    public int a;
}

public class MyClass
{
    public static void Main()
    {
        S myS = new S();
        Console.WriteLine(sizeof(S));    // CS0233
        // Try the following line instead:
        // Console.WriteLine(Marshal.SizeOf(myS));
    }
}
```

Compiler Error CS0234

Error Message

The type or namespace name 'name' does not exist in the namespace 'namespace' (are you missing an assembly reference?)

A type was expected. Possible reasons include:

- An assembly that contains the definition of a type was not referenced in the compilation; use [/reference \(Import Metadata\)](#) to specify the assembly.
- You passed a variable name to the [typeof](#) operator.

See [Add Reference Dialog Box](#) for information on how to add a reference in the development environment.

The following sample generates CS0234:

```
// CS0234.cs
public class C
{
    public static void Main()
    {
        System.DateTime x = new System.DateTim();    // CS0234
        // try the following line instead
        // System.DateTime x = new System.DateTime();
    }
}
```

Compiler Error CS0236

Error Message

A field initializer cannot reference the nonstatic field, method, or property 'field'

Instance fields cannot be used to initialize other instance fields outside of a method. If you are attempting to initialize a variable outside of a method, consider performing the initialization inside the class constructor. For more information, see [Methods \(C# Programming Guide\)](#).

The following sample generates CS0236:

```
// CS0236.cs
public class MyClass
{
    public int i = 5;
    public int j = i; // CS0236
    public int k;      // initialize in constructor

    MyClass()
    {
        k = i;
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0238

Error Message

'member' cannot be sealed because it is not an override

[sealed](#) was used on a member that was not also marked with [override](#). For more information, see [Inheritance \(C# Programming Guide\)](#).

The following sample generates CS0238:

```
// CS0238.cs
abstract class MyClass
{
    public abstract void f();
}

class MyClass2 : MyClass
{
    public static void Main()
    {
    }

    public sealed void f() // CS0238
    // Try the following definition instead:
    // public override sealed void f()
    {
    }
}
```

Compiler Error CS0239

Error Message

'member' : cannot override inherited member 'inherited member' because it is sealed

A member cannot [override](#) a [sealed](#) inherited member. For more information, see [Checked and Unchecked \(C# Reference\)](#).

The following sample generates CS0239:

```
// CS0239.cs
abstract class MyClass
{
    public abstract void f();
}

class MyClass2 : MyClass
{
    public static void Main()
    {
    }

    public override sealed void f()
    {
    }
}

class MyClass3 : MyClass2
{
    public override void f()    // CS0239
    // Try the following definition instead:
    // public new void f()
    {
    }
}
```


Compiler Error CS0241

Error Message

Default parameter specifiers are not permitted

[Method parameters](#) cannot have default values. Use method overloads if you want to achieve the same effect. For more information, see [Passing Parameters \(C# Programming Guide\)](#).

Example

The following sample generates CS0241. In addition, the sample shows how to simulate, with overloading, a method with default arguments.

```
// CS0241.cs
public class A
{
    public void Test(int i = 9) {}    // CS0241
}

public class B
{
    public void Test() { Test(9); }
    public void Test(int i) {}
}

public class C
{
    public static void Main()
    {
        B x = new B();
        x.Test();
    }
}
```

Compiler Error CS0242

Error Message

The operation in question is undefined on void pointers

Incrementing a void pointer is not allowed. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample generates CS0242:

```
// CS0242.cs
// compile with: /unsafe
class TestClass
{
    public unsafe void Test()
    {
        void * p = null;
        p++;    // CS0242, incrementing a void pointer not allowed
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0243

Error Message

The Conditional attribute is not valid on 'method' because it is an override method

The [Conditional](#) attribute is not allowed on a method that is marked with the [override](#) keyword. For more information, see [Knowing When to Use Override and New Keywords \(C# Programming Guide\)](#).

The compiler never binds to override methods; it only binds to the base method and the common language runtime calls the override, as appropriate.

The following sample generates CS0243:

```
// CS0243.cs
// compile with: /target:library
public class MyClass
{
    public virtual void M() {}
}

public class MyClass2 : MyClass
{
    [System.Diagnostics.Conditional("MySymbol")]    // CS0243
    // remove Conditional attribute or remove override keyword
    public override void M() {}
}
```

Compiler Error CS0244

Error Message

Neither "is" nor "as" are valid on pointer types

The [is](#) and [as](#) keywords are not valid for use on pointer types. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample generates CS0244:

```
// CS0244.cs
// compile with: /unsafe

class UnsafeTest
{
    unsafe static void SquarePtrParam (int* p)
    {
        bool b = p is object;    // CS0244 p is pointer
    }

    unsafe public static void Main()
    {
        int i = 5;
        SquarePtrParam (&i);
    }
}
```

Compiler Error CS0245

Error Message

Destructors and object.Finalize cannot be called directly. Consider calling IDisposable.Dispose if available.

For more information, see [Programming Essentials for Garbage Collection](#) and [Destructors \(C# Programming Guide\)](#).

The following sample generates CS0245:

```
// CS0245.cs
using System;
using System.Collections;

class MyClass // : IDisposable
{
    /*
    public void Dispose()
    {
        // cleanup code goes here
    }
    */

    void m()
    {
        this.Finalize();    // CS0245
        // this.Dispose();
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0246

Error Message

The type or namespace name 'type/namespace' could not be found (are you missing a using directive or an assembly reference?)

A type was not found. You may have forgotten to reference ([/reference](#)) the assembly that contains the type or you may not have correctly qualified its use with the [using directive](#).

There are several reasons for this error:

1. The name of the type or namespace you are trying to use may be misspelled (including the correct case). Without the correct name the compiler is unable to find the definition for the type or namespace you have referred to in your code. This occurs most often because C# is case-sensitive and the correct casing has not been used when referring to the type. For example, `Dataset ds;` will generate CS0246; notice the s in Dataset is not capitalized.
2. If the error is for a namespace name, you may not have referenced ([/reference](#)) the assembly containing the namespace. For example, your code might contain `using Accessibility;`. However, if your project doesn't reference the assembly Accessibility.dll then you will get CS0246. See [Add Reference Dialog Box](#) for information on how to add a reference in the development environment.
3. If the error is for a type name, you may not have the proper [using](#) directive, or you have not fully qualified the name of the type. Consider the following line of code: `DataSet ds;`. To be able to use the DataSet type you would need to do two things. First, you need a reference to the assembly that contains the definition for the DataSet type. Second, you need a using directive for the namespace where DataSet is located. For example, because DataSet is located in the **System.Data** namespace, you would need the following statement at the beginning of your code: `using System.Data;`.

The second step is not required. However, if you omitted this step then it would require that you fully qualify the DataSet type when referring to it. Fully qualifying it means that you use the namespace and type each time you refer to it in your code. So, if you decided to skip the second step you would need to change your declaration code above to:

```
System.Data.DataSet ds;
```

4. If the error is for a non-type, you may have used a variable or something else when a type was expected. For example, in the **is** statement, if you use a Type object rather than an actual type, you will get this error.

The following sample generates CS0246:

```
// CS0246.cs
// using System.Diagnostics;

public class MyClass
{
    [Conditional("A")]    // CS0246, uncomment using directive to resolve
    public void Test()
    {
    }

    public static void Main()
    {
    }
}
```

Here's an example where an object of type Type was used where an actual type was expected (case 4 above):

```
// CS0246b.cs
using System;

class C
{
    public bool supports(object o, Type t)
    {
        if (o is t)    // CS0246 - t is not a type
        {
        }
    }
}
```

```
        return true;
    }
    return false;
}

public static void Main()
{
}
}
```

Compiler Error CS0247

Error Message

Cannot use a negative size with stackalloc

A negative number was passed to a [stackalloc](#) statement.

The following sample generates CS0247:

```
// CS0247.cs
// compile with: /unsafe
public class MyClass
{
    unsafe public static void Main()
    {
        int *p = stackalloc int [-30];    // CS0247
    }
}
```


Compiler Error CS0248

Error Message

Cannot create an array with a negative size

An array size was specified with a negative number. For more information, see [Arrays \(C# Programming Guide\)](#).

Example

The following sample generates CS0248:

```
// CS0248.cs
class MyClass
{
    public static void Main()
    {
        int[] myArray = new int[-3] {1,2,3};    // CS0248, pass a nonnegative number
    }
}
```

Compiler Error CS0249

Error Message

Do not override object.Finalize. Instead, provide a destructor.

Use destructor syntax to specify instructions to execute when your object is destroyed.

For more information, see [Destructor Syntax in C# and C++](#).

The following sample generates CS0249:

```
// CS0249.cs
class MyClass
{
    protected override void Finalize()    // CS0249
    // try the following line instead
    // ~MyClass()
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0250

Error Message

Do not directly call your base class Finalize method. It is called automatically from your destructor.

A program cannot attempt to force cleanup of base class resources.

See [Finalize Methods and Destructors](#) for more information.

The following sample generates CS0250

```
// CS0250.cs
class B
{
}

class C : B
{
    ~C()
    {
        base.Finalize();    // CS0250
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0254

Error Message

The right hand side of a fixed statement assignment may not be a cast expression

The right side of a [fixed](#) expression may not use a cast. For more information, see [Unsafe Code and Pointers \(C# Programming Guide\)](#).

The following sample generates CS0254:

```
// CS0254.cs
// compile with: /unsafe
class Point
{
    public uint x, y;
}

class FixedTest
{
    unsafe static void SquarePtrParam (int* p)
    {
        *p *= *p;
    }

    unsafe public static void Main()
    {
        Point pt = new Point();
        pt.x = 5;
        pt.y = 6;

        fixed (int* p = (int*)&pt.x)    // CS0254
        // try the following line instead
        // fixed (uint* p = &pt.x)
        {
            SquarePtrParam ((int*)p);
        }
    }
}
```

Compiler Error CS0255

Error Message

stackalloc may not be used in a catch or finally block

The [stackalloc](#) keyword may not be used in a [catch](#) or [finally](#) block. For more information, see [Exceptions and Exception Handling \(C# Programming Guide\)](#).

The following sample generates CS0255:

```
// CS0255.cs
// compile with: /unsafe
using System;

public class TestTryFinally
{
    public static unsafe void Test()
    {
        int i = 123;
        string s = "Some string";
        object o = s;

        try
        {
            // Conversion is not valid; o contains a string not an int
            i = (int) o;
        }

        finally
        {
            Console.WriteLine("i = {0}", i);
            int* fib = stackalloc int[100];    // CS0255
        }
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0260

Error Message

Missing partial modifier on declaration of type 'type'; another partial declaration of this type exists

This error indicates that more than one class declaration with the same name was found, and at least one but not all of these declarations was declared partial. If your intention is to define a class in several parts, all those parts must be declared with the keyword **partial**. This error also occurs if you create a new class that accidentally has the same name as a partial class declared elsewhere in the same namespace.

The following sample generates CS0260:

```
// CS0260.cs
class C // CS0260
{
}

partial class C
{
}
```

Compiler Error CS0261

Error Message

Partial declarations of 'type' must be all classes, all structs, or all interfaces

This error occurs if a partial type is declared as a different type of construct in various places. For more information, see [Partial Class Definitions \(C# Programming Guide\)](#).

The following sample generates CS0261:

```
// CS0261.cs
partial class A // CS0261 - A declared as a class here, but as a struct below
{
}

partial struct A
{
}
```

Compiler Error CS0262

Error Message

Partial declarations of 'type' have conflicting accessibility modifiers

This error occurs if a partial type has inconsistent modifiers such as public, private, protected, internal, or abstract. These modifiers must be consistent in all partial declarations for that type. For more information, see

[Partial Class Definitions \(C# Programming Guide\)](#).

Example

The following sample generates CS0262:

```
// CS0262.cs
class A
{
    public partial class C // CS0262
    {
    }
    private partial class C
    {
    }
}
```


Compiler Error CS0263

Error Message

Partial declarations of 'type' must not specify different base classes

When defining a type in partial declarations, specify the same base types in each of the partial declarations. For more information, see [Partial Class Definitions \(C# Programming Guide\)](#).

The following sample generates CS0263:

```
// CS0263.cs
// compile with: /target:library
class B1
{
}

class B2
{
}
partial class C : B1 // CS0263 - is the base class B1 or B2?
{
}

partial class C : B2
{
}
```

Compiler Error CS0264

Error Message

Partial declarations of 'type' must have the same type parameter names in the same order

This error occurs if you are defining a generic type in partial declarations and the type parameters are not consistent in name or order throughout all of the partial declarations. To get rid of this error, check the type parameters for each partial declaration and make sure the same name and order of parameters is used. For more information, see

[Partial Class Definitions \(C# Programming Guide\)](#) and [Generic Type Parameters \(C# Programming Guide\)](#).

Example

The following example generates CS0264.

```
// CS0264.cs

partial class MyClass<T> // CS0264
{
}

partial class MyClass <MyType>
{
}
```

Compiler Error CS0265

Error Message

Partial declarations of 'type' have inconsistent constraints for type parameter 'type parameter'

This error happens when you define a generic class as a partial class, so that its partial definitions occur in more than one place, and the constraints on the generic type are inconsistent or different in two or more places. If you specify the constraints in more than one place, they must all be identical. The easiest solution is to specify the constraints in one place, and omit them everywhere else. For more information, see [Partial Class Definitions \(C# Programming Guide\)](#) and [Constraints on Type Parameters \(C# Programmers Reference\)](#).

The following code generates error CS0265.

Example

In this code, the partial class definitions are all in a single file, but they could also be spread across multiple files.

```
// CS0265.cs
public class GenericsErrors
{
    interface IFace1 { }
    interface IFace2 { }
    partial class PartialBadBounds<T> where T : IFace1 { } // CS0265
    partial class PartialBadBounds<T> where T : IFace2 { }
}
```

Compiler Error CS0266

Error Message

Cannot implicitly convert type 'type1' to 'type2'. An explicit conversion exists (are you missing a cast?)

This error occurs if you have code that attempts to convert two types that cannot be implicitly converted, such as in an assignment of a base type to a derived type that is missing an explicit cast. For more information, see [Conversion Operators \(C# Programming Guide\)](#).

The following sample generates CS0266:

```
// CS0266.cs
class MyClass
{
    public static void Main()
    {
        object obj = "MyString";
        // Cannot implicitly convert 'object' to 'MyClass'
        MyClass myClass = obj; // CS0266
        // Try this line instead
        // MyClass c = ( MyClass )obj;
    }
}
```

Compiler Error CS0267

Error Message

The partial modifier can only appear immediately before 'class', 'struct', or 'interface'

The placement of the **partial** modifier was incorrect in the declaration of the class, struct or interface. To fix the error, reorder the modifiers. For more information, see [Partial Class Definitions \(C# Programming Guide\)](#).

The following sample generates CS0267:

```
// CS0267.cs
public partial class MyClass
{
    public MyClass()
    {
    }
}

partial public class MyClass // CS0267
// Try this line instead:
// public partial class MyClass
{
    public void Foo()
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0268

Error Message

Imported type 'type' is invalid. It contains a circular base class dependency.

This error occurs if a type imported from another language has a circular base class dependency. Such a type cannot be used in a C# program. To resolve this error, check types imported from other languages in any referenced assemblies or modules.

Compiler Error CS0269

Error Message

Use of unassigned out parameter 'parameter'

The compiler could not verify that the out parameter was assigned a value before it was used; its value may be undefined when assigned. Be sure to initialize **out** parameters before accessing the value. If you need to use the value of the variable passed in, use a **ref** parameter instead. For more information, see [Passing Parameters \(C# Programming Guide\)](#).

Example

The following sample generates CS0269:

```
// CS0269.cs
class C
{
    public static void F(out int i)
    // Try this instead:
    // public static void F(ref int i)
    {
        int k = i; // CS0269
        i = 1;
    }

    public static void Main()
    {
        int myInt;
        F(out myInt);
    }
}
```

This could also occur if the initialization of a variable occurs in a try block, which the compiler is unable to verify will execute successfully:

```
// CS0269b.cs
class C
{
    public static void F(out int i)
    {
        try
        {
            // Assignment occurs, but compiler can't verify it
            i = 1;
        }
        catch
        {
        }

        int k = i; // CS0269
        i = 1;
    }

    public static void Main()
    {
        int myInt;
        F(out myInt);
    }
}
```

Compiler Error CS0270

Error Message

Array size cannot be specified in a variable declaration (try initializing with a 'new' expression)

This error occurs when a size is specified as part of an array declaration. To resolve, use the [new Operator](#) expression.

The following example generates CS0270:

```
// CS0270.cs
// compile with: /t:module

public class Test
{
    int[10] a;    // CS0270
    // To resolve, use the following line instead:
    // int[] a = new int[10];
}
```


Compiler Error CS0271

Error Message

The property or indexer 'property/indexer' cannot be used in this context because the get accessor is inaccessible

This error occurs when you try to access an inaccessible **get** accessor. To resolve this error, increase the accessibility of the accessor, or change the calling location. For more information, see [Accessor Accessibility](#) and [Properties \(C# Programming Guide\)](#).

The following example generates CS0271:

```
// CS0271.cs
public class MyClass
{
    public int Property
    {
        private get { return 0; }
        set { }
    }

    public int Property2
    {
        get { return 0; }
        set { }
    }
}

public class Test
{
    public static void Main(string[] args)
    {
        MyClass c = new MyClass();
        int a = c.Property;    // CS0271
        int b = c.Property2;   // OK
    }
}
```

Compiler Error CS0272

Error Message

The property or indexer 'property/indexer' cannot be used in this context because the set accessor is inaccessible

This error occurs when the **set** accessor is not accessible to the program code. To resolve this error, increase the accessibility of the accessor, or change the calling location. For more information, see

[Asymmetric Accessor Accessibility \(C# Programming Guide\)](#).

Example

The following example generates CS0272:

```
// CS0272.cs
public class MyClass
{
    public int Property
    {
        get { return 0; }
        private set { }
    }
}

public class Test
{
    static void Main()
    {
        MyClass c = new MyClass();
        c.Property = 10;          // CS0272
        // To resolve, remove the previous line
        // or use an appropriate modifier on the set accessor.
    }
}
```

Compiler Error CS0273

Error Message

The accessibility modifier of the 'property_accessor' accessor must be more restrictive than the property or indexer 'property'

The accessibility modifier of the set/get accessor must be more restrictive than the property or indexer 'property/indexer'

This error occurs when you declare a property or indexer with an access modifier that is less restrictive than the access modifier on one of its accessors. To resolve this error, use the appropriate access modifier on either the property or the set accessor. For more information, see [Accessor Accessibility](#).

Example

This sample contains an internal property with an internal set method. The following sample generates CS0273.

```
// CS0273.cs
// compile with: /target:library
public class MyClass
{
    internal int Property
    {
        get { return 0; }
        internal set {} // CS0273
        // try the following line instead
        // private set {}
    }
}
```

Compiler Error CS0274

Error Message

Cannot specify accessibility modifiers for both accessors of the property or indexer 'property/indexer'

This error occurs when you declare a property or indexer with access modifiers on both its accessors. To resolve this error, use an access modifier on only one of the two accessors. For more information, see [Accessor Accessibility](#).

The following example generates CS0274:

```
// CS0274.cs
public class MyClass
{
    public int Property    // CS0274
    {
        public get { return 0; }
        protected set { }
    }
}
```

Compiler Error CS0275

Error Message

'accessor': accessibility modifiers may not be used on accessors in an interface

This error occurs when you use an access modifier on any one of the accessors of a property or indexer in an interface. To resolve, remove the access modifier.

Example

The following example generates CS0275:

```
// CS0275.cs
public interface MyInterface
{
    int Property
    {
        get;
        internal set;    // CS0275
    }
}
```

Compiler Error CS0276

Error Message

'property/indexer': accessibility modifiers on accessors may only be used if the property or indexer has both a get and a set accessor

This error occurs when you declare a property or indexer with one accessor only, and use an access modifier on the accessor. To resolve, remove the access modifier or add another accessor.

Example

The following example generates CS0276:

```
// CS0276.cs
public class MyClass
{
    public int Property
    {
        protected set { }    // CS0276
    }
    public int Property2
    {
        internal get { }    // CS0276
    }
}
```

Compiler Error CS0277

Error Message

'class' does not implement interface member 'accessor'. 'class accessor' is not public

This error occurs when you try to implement a property of an interface, but the implementation of the property accessor in the class is not public. Methods that implement interface members need to have public accessibility. To resolve, remove the access modifier on the property accessor.

Example

The following example generates CS0277:

```
// CS0277.cs
public interface MyInterface
{
    int Property
    {
        get;
        set;
    }
}

public class MyClass : MyInterface    // CS0277
{
    public int Property
    {
        get { return 0; }
        // Try this instead:
        //set { }
        protected set { }
    }
}
```

Compiler Error CS0281

Error Message

Friend access was granted to 'AssemblyName1', but the output assembly is named 'AssemblyName2'. Try adding a reference to 'AssemblyName1' or changing the output assembly name to match.

Friend access is a new common language runtime (CLR) feature that enables an assembly to see another assembly's non-public types. This error occurs when the assembly granting friend access specifies the wrong name for the grantee assembly. For more information, see [Friend Assemblies \(C# Programming Guide\)](#).

Example

The following sequence of code samples will generate CS0281.

The files used to create the strong named assemblies are generated as follows:

- `sn -d CS0281.snk`
- `sn -k CS0281.snk`
- `sn -i CS0281.snk CS0281.snk`
- `sn -pc CS0281.snk key.publickey`
- `sn -tp key.publickey`

```
// CS0281.cs
// compile with: /target:library /keyfile:CS0281.snk
public class A {}
```

```
// CS0281_b.cs
// compile with: /target:library /keyfile:CS0281.snk /reference:CS0281.dll
[assembly:System.Runtime.CompilerServices.InternalsVisibleTo("CS0281 , PublicKey=0024000004
8000009400000006020000002400005253413100040000010001004b2d4d56af7c50be2fcbbf97cb880b9e73ad8
4467a587191fef63aad118a96cecf9d508cd679c907b6e20f71684300bdc2c0a851019af0c96b29bf8f1339753
276041aef6d67db46139e6348b3a12f29537b4dc6c2c19829df2c9ed6803f3c63c3b84cfa2728849386aea575c54
3a5f70fa85793d2946f15f7fe1ccb0c5e8fe0")]
class B : A {}
```

The following sample generates CS0281.

Notice that this sample creates an output file with the same name as the output file in the first sample. To resolve, do not change the assembly attributes of the component and add class C.

```
// CS0281_c.cs
// compile with: /target:library /out:CS0281.dll /keyfile:CS0281.snk /reference:CS0281_b.dl
1
// CS0281 expected
[assembly:System.Reflection.AssemblyVersion("3")]
[assembly:System.Reflection.AssemblyCulture("en-us")]
class C : B {}
public class A {}
```


Compiler Error CS0283

Error Message

The type 'type' cannot be declared const

The type specified in a constant declaration must be **byte**, **char**, **short**, **int**, **long**, **float**, **double**, **decimal**, **bool**, **string**, an enum-type, or a reference type that is assigned a value of null. Each constant-expression must yield a value of the target type or of a type that can be converted to the target type by implicit conversion.

Example

The following example generates CS0283.

```
// CS0283.cs
struct MyTest
{
}
class MyClass
{
    // To resolve the error but retain the "const-ness",
    // change const to readonly.
    const MyTest test = new MyTest();    // CS0283

    public static int Main() {
        return 1;
    }
}
```

Compiler Error CS0304

Error Message

Cannot create an instance of the variable type 'type' because it does not have the new() constraint

This error occurs if you use `new` to create an instance of a type variable when that type variable doesn't specify the [new\(\) constraint](#). You cannot use `new` to directly invoke the constructor for an unknown type unless the `new()` constraint is used to guarantee the existence of a default constructor. If you cannot use the `new` constraint, consider using a [typeof](#) expression to access the desired constructor.

The following sample generates CS0304:

```
// CS0304.cs
// compile with: /target:library
class C<T>
{
    T t = new T();    // CS0304
}
```

The `new` statement of this form is also not allowed in class methods:

```
// CS0304_2.cs
// compile with: /target:library
class C<T>
{
    public void f()
    {
        T t = new T();    // CS0304
    }
}
```

Compiler Error CS0305

Error Message

Using the generic type 'generic type' requires 'number' type arguments

This error occurs when the expected number of type arguments was not found. To resolve C0305, use the required number of type arguments.

Example

The following sample generates CS0305.

```
// CS0305.cs
public class MyList<T> {}
public class MyClass<T> {}

class MyClass
{
    public static void Main()
    {
        MyList<MyClass, MyClass> list1 = new MyList<MyClass>();    // CS0305
        MyList<MyClass> list2 = new MyList<MyClass>();    // OK
    }
}
```

Compiler Error CS0306

Error Message

The type 'type' may not be used as a type argument

The type used as a type parameter is not allowed. This could be because the type is a pointer type.

The following example generates CS0306:

```
// CS0306.cs
class C<T>
{
}

class M
{
    // CS0306 - int* not allowed as a type parameter
    C<int*> f;
}
```

Compiler Error CS0307

Error Message

The 'construct' 'identifier' cannot be used with type arguments

The construct named was not a type or a method, the only constructs that can take generic arguments. Remove the type arguments in angle brackets. If a generic is needed, declare your generic construct as a generic type or method.

The following sample generates CS0307:

```
// CS0307.cs
class C
{
    public int P { get { return 1; } }
    public static void Main()
    {
        C c = new C();
        int p = c.P<int>(); // CS0307 - C.P is a property
        // Try this instead
        // int p = c.P;
    }
}
```

Compiler Error CS0308

Error Message

The non-generic type-or-method 'identifier' cannot be used with type arguments.

The method or type is not generic, but it was used with type arguments. To avoid this error, remove the angled brackets and type arguments, or redeclare the method or type as a generic method or type.

The following example generates CS0308:

```
// CS0308a.cs
class MyClass
{
    public void F() {}
    public static void Main()
    {
        F<int>(); // CS0308 - F is not generic.
        // Try this instead:
        // F();
    }
}
```

The following example also generates CS0308. To resolve the error, use the directive "using System.Collections.Generic."

```
// CS0308b.cs
// compile with: /t:library
using System.Collections;
// To resolve, uncomment the following line:
// using System.Collections.Generic;
public class MyStack<T>
{
    // Store the elements of the stack:
    private T[] items = new T[100];
    private int stack_counter = 0;

    // Define the iterator block:
    public IEnumerator<T> GetEnumerator() // CS0308
    {
        for (int i = stack_counter - 1 ; i >= 0; i--)
            yield return items[i];
    }
}
```

Compiler Error CS0309

Error Message

The type 'typename' must be convertible to 'constraint type' in order to use it as parameter 'parameter' in the generic type or method 'generic'

When using a generic class or method, constraints placed on the generic type using the [where](#) keyword must be observed. A violation of the constraints results in this error. To fix the error, a different type should be passed in to the generic class or method, or the constraints should be changed.

The following sample generates CS0309 since B does not implement I and C<T> specifies in its constraints that T must implement I:

```
// CS0309.cs
using System;

interface I
{
}

class C<T> where T : I
{
}

class B
{
}

class CMain
{
    public static void Main()
    {
        Console.WriteLine(new C<B>()); // CS0309
    }
}
```

Compiler Error CS0310

Error Message

The type 'typename' must have a public parameterless constructor in order to use it as parameter 'parameter' in the generic type or method 'generic'

The generic type or method defines a new constraint in its where clause, so any type must have a public parameterless constructor in order to be used as a type argument for that generic type or method. To avoid this error, make sure that the type has the correct constructor, or modify the constraint clause of the generic type or method.

Example

The following sample generates CS0310:

```
// CS0310.cs
using System;

class G<T> where T : new()
{
    T t;

    public G()
    {
        t = new T();
        Console.WriteLine(t);
    }
}

class B
{
    private B() { }
    // Try this instead:
    // public B() { }
}

class CMain
{
    public static void Main()
    {
        G<B> g = new G<B>();    // CS0310
        Console.WriteLine(g.ToString());
    }
}
```


Compiler Error CS0400

Error Message

The type or namespace name 'identifier' could not be found in the global namespace (are you missing an assembly reference?)

The identifier scoped with the global scope operator (::) was not found in the global namespace. You may be missing an assembly reference that contains the identifier, or the identifier may be declared in a class or namespace other than the global namespace. This error could also occur if a globally scoped identifier is not declared or is misspelled.

To avoid this error, locate the declaration of the identifier, verify the correct spelling, and if the declaration is in a separate assembly, make sure that you have the appropriate assembly reference. If the identifier is declared inside another type or namespace, use the fully-qualified name after the ::. The following sample generates CS0400:

```
// CS0400.cs
class C
{
    public static void Main()
    {
        // CS0400 - D could not be found
        // in the global namespace.
        global::D d = new global::D();
    }
}
```

Compiler Error CS0401

Error Message

The new() constraint must be the last constraint specified

When using multiple constraints, list all other constraints before the new() constraint.

Example

The following sample generates CS0401.

```
// CS0401.cs
// compile with: /target:library
using System;
class C<T> where T : new(), IDisposable {} // CS0401

class D<T> where T : IDisposable
{
    static void F<U>() where U : new(), IDisposable{} // CS0401
}
```

Compiler Error CS0403

Error Message

Cannot convert null to type parameter 'type parameter' because it could be a value type. Consider using default('T') instead.

You cannot assign null to the unknown type named because it might be a value type, which does not allow null assignment. If your generic class is not intended to accept value types, use the class type constraint. If it can accept value types, such as the built-in types, you may be able to replace the assignment to null with the expression `default(T)`, as shown in the following example.

Example

The following sample generates CS0403.

```
// CS0403.cs
// compile with: /target:library
class C<T>
{
    public void f()
    {
        T t = null; // CS0403
        T t2 = default(T); // OK
    }
}

class D<T> where T : class
{
    public void f()
    {
        T t = null; // OK
    }
}
```

Compiler Error CS0404

Error Message

'<' unexpected : attributes cannot be generic

Generic type parameters are not allowed in attributes. Remove the type parameter and angled brackets.

The following sample generates CS0404:

```
// CS0404.cs
[MyAttrib<int>] // CS0404
class C
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0405

Error Message

Duplicate constraint 'constraint' for type parameter 'type parameter'

Two of the constraints on the generic declaration are identical. To get rid of the error, remove the duplicate.

The following sample generates CS0405:

```
// CS0405.cs
interface I
{
}

class C<T> where T : I, I // CS0405.cs
{
}
```

Compiler Error CS0406

Error Message

The class type constraint 'constraint' must come before any other constraints

When a generic type or method has a class type constraint, that constraint must be listed first. To avoid this error, move the class type constraint to the beginning of the constraint list.

Example

The following sample generates CS0406.

```
// CS0406.cs
// compile with: /target:library
interface I {}
class C {}
class D<T> where T : I, C {}    // CS0406
class D2<T> where T : C, I {} // OK
```

Compiler Error CS0407

Error Message

'return-type method' has the wrong return type

The method was not compatible with the delegate type. The argument types matched, but the return type was not the correct return type for that delegate. To avoid this error, use a different method, change the method's return type, or change the delegate's return type.

Example

The following sample generates CS0407:

```
// CS0407.cs
public delegate int MyDelegate();

class C
{
    MyDelegate d;

    public C()
    {
        d = new MyDelegate(F); // OK: F returns int
        d = new MyDelegate(G); // CS0407 - G doesn't return int
    }

    public int F()
    {
        return 1;
    }

    public void G()
    {
    }

    public static void Main()
    {
        C c1 = new C();
    }
}
```

Compiler Error CS0409

Error Message

A constraint clause has already been specified for type parameter 'type parameter'. All of the constraints for a type parameter must be specified in a single where clause.

Multiple constraint clauses (where clauses) were found for a single type parameter. Remove the extraneous where clause, or correct the where clauses so that a unique type parameter in each clause.

```
// CS0409.cs
interface I
{
}

class C<T1, T2> where T1 : I where T1 : I // CS0409 - T1 used twice
{
}
```


Compiler Error CS0410

Error Message

No overload for 'method' has the correct parameter and return types

This error occurs if you try to instantiate a delegate with a function that has the wrong parameter types. The parameter types of the delegate must match the function that you are assigning to the delegate.

Example

The following example generates CS0410:

```
// CS0410.cs
// compile with: /langversion:ISO-1

class Test
{
    delegate void D(double d );
    static void F(int i) { }

    static void Main()
    {
        D d = new D(F);  // CS0410
    }
}
```

Compiler Error CS0411

Error Message

The type arguments for method 'method' cannot be inferred from the usage. Try specifying the type arguments explicitly.

This error occurs if you call a generic method without explicitly providing the type arguments and the compiler cannot infer which type arguments are intended. To avoid this error, add the intended type arguments in angle brackets.

Example

The following sample generates CS0411:

```
// CS0411.cs
class C
{
    void G<T>()
    {
    }

    public static void Main()
    {
        G(); // CS0411
        // Try this instead:
        // G<int>();
    }
}
```

Other possible error cases include when the parameter is `null`, which has no type information:

```
// CS0411b.cs
class C
{
    public void F<T>(T t) where T : C
    {
    }

    public static void Main()
    {
        C c = new C();
        c.F(null); // CS0411
    }
}
```

Another case occurs when a conversion is involved on one of multiple parameters:

```
// CS0411c.cs
class C
{
    void F<T>(T t1, T t2)
    {
    }

    public static void Main()
    {
        C c = new C();
        c.F(1, 2L); // CS0411 -- is T int or long?
    }
}
```

Compiler Error CS0412

Error Message

'generic': a parameter or local variable cannot have the same name as a method type parameter

There is a name conflict between the type parameter of a generic method and a local variable in the method or one of the method's parameters. To avoid this error, rename any conflicting parameters or local variables.

Example

The following sample generates CS0412:

```
// CS0412.cs
using System;

class C
{
    // Parameter name is the same as method type parameter name
    public void G<T>(int T) // CS0412
    {
    }
    public void F<T>()
    {
        // Method local variable name is the same as method type
        // parameter name
        double T = 0.0; // CS0412
        Console.WriteLine(T);
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0413

Error Message

The type parameter 'type parameter' cannot be used with the 'as' operator because it does not have a class type constraint nor a 'class' constraint

This error occurs if a generic type uses the `as` operator, but that generic type does not have a class type constraint. The `as` operator is only allowed with reference types, so the type parameter must be constrained to guarantee that it is not a value type. To avoid this error, use a class type constraint or a reference type constraint.

This is because the `as` operator could return `null`, which is not a possible value for a value type, and the type parameter must be treated as a value type unless it is a class type constraint or a reference type constraint.

Example

The following sample generates CS0413.

```
// CS0413.cs
// compile with: /target:library
class A {}
class B : A {}

class CMain
{
    A a = null;
    public void G<T>()
    {
        a = new A();
        System.Console.WriteLine (a as T); // CS0413
    }

    // OK
    public void H<T>() where T : A
    {
        a = new A();
        System.Console.WriteLine (a as T);
    }
}
```

Compiler Error CS0415

Error Message

The 'IndexerName' attribute is valid only on an indexer that is not an explicit interface member declaration

This error occurs if you use an `IndexerName` attribute on an indexer that was an explicit implementation of an interface. This error may be avoided by removing the interface name from the declaration of the indexer, if possible. For more information, see the [IndexerNameAttribute Class](#).

The following sample generates CS0415:

```
// CS0415.cs
using System;
using System.Runtime.CompilerServices;

public interface IA
{
    int this[int index]
    {
        get;
        set;
    }
}

public class A : IA
{
    [IndexerName("Item")] // CS0415
    int IA.this[int index]
    // Try this line instead:
    // public int this[int index]
    {
        get { return 0; }
        set { }
    }

    static void Main()
    {
    }
}
```

Compiler Error CS0416

Error Message

'type parameter': an attribute argument cannot use type parameters

A type parameter was used as an attribute argument, which is not allowed. Use a non-generic type.

The following sample generates CS0416:

```
// CS0416.cs
public class MyAttribute : System.Attribute
{
    public MyAttribute(System.Type t)
    {
    }
}

class G<T>
{
    [MyAttribute(typeof(G<T>))] // CS0416
    public void F()
    {
    }
}
```

Compiler Error CS0417

Error Message

'identifier': cannot provide arguments when creating an instance of a variable type

This error occurs if a call to the new operator on a type parameter has arguments. The only constructor that can be called using the new operator on an unknown parameter type is a constructor with no arguments. If you need to call another constructor, consider using a class type constraint or interface constraint.

Example

The following example generates CS0417:

```
// CS0417
class C<T> where T : new()
{
    T type = new T(1);    // CS0417
}
```

Compiler Error CS0418

Error Message

'class name': an abstract class cannot be sealed or static

An abstract class cannot be used to create objects unless it is derived from, so it makes no sense to be sealed. An abstract class cannot meaningfully be static either; abstract classes are designed to support an object hierarchy that will use the abstract class as a base.

Example

The following sample generates CS0418:

```
// CS0418.cs
public abstract sealed class C // CS0418
{
}

sealed static class S // CS0418
{
}

public class MyClass
{
    public static void Main()
    {
    }
}
```


Compiler Error CS0423

Error Message

Since 'class' has the ComImport attribute, 'method' must be extern or abstract

Specifying the ComImport attribute implies that the implementation for the class is to be imported from a COM module. Additional methods may not be defined.

The following sample generates CS0423:

```
// CS0423.cs

using System.Runtime.InteropServices;

[
    ComImport,
    Guid("7ab770c7-0e23-4d7a-8aa2-19bfad479829")
]
class ImageProperties
{
    public static void Main() // CS0423
    {
        ImageProperties i = new ImageProperties();
    }
}
```

Compiler Error CS0424

Error Message

'class': a class with the ComImport attribute cannot specify a base class

Specifying the [ComImportAttribute](#) attribute implies that the implementation for the class is to be imported from a COM module. Additional methods or fields inherited from the base class are not allowed to be added to the implementation defined in the COM module.

The following sample generates CS0424:

```
// CS0424.cs
// compile with: /target:library
using System.Runtime.InteropServices;
public class A {}

[ ComImport, Guid("7ab770c7-0e23-4d7a-8aa2-19bfad479829") ]
class B : A {}    // CS0424 error
```

Compiler Error CS0425

Error Message

The constraints for type parameter 'type parameter' of method 'method' must match the constraints for type parameter 'type parameter' of interface method 'method'. Consider using an explicit interface implementation instead.

This error occurs if a virtual generic method is overridden in a derived class and the constraints on the method in the derived class do not match the constraints on the method in the base class. To avoid this error, make sure the `where` clause is identical in both declarations, or implement the interface explicitly.

Example

The following example generates CS0425:

```
// CS0425.cs

class C1
{
}

class C2
{
}

interface IBase
{
    void F<ItemType>(ItemType item) where ItemType : C1;
}

class Derived : IBase
{
    public void F<ItemType>(ItemType item) where ItemType : C2 // CS0425
    {
    }
}

class CMain
{
    public static void Main()
    {
    }
}
```

The constraints do not have to be a literal match, as long as the set of constraints has the same meaning. For example, the following is okay:

```
// CS0425b.cs

interface J<Z>
{
}

interface I<S>
{
    void F<T>(S s, T t) where T: J<S>, J<int>;
}

class C : I<int>
{
    public void F<X>(int s, X x) where X : J<int>
    {
    }

    public static void Main()
```

}

{
}

Compiler Error CS0426

Error Message

The type name 'identifier' does not exist in the type 'type'

The type name could not be found in the type specified. Check the spelling of the names used, and verify that the type referenced has the expected member.

The following sample generates CS0426:

```
// CS0426.cs

class C
{
}

class D
{
    public static void Main()
    {
        C.A a; // CS0426
    }
}
```

Compiler Error CS0428

Error Message

Cannot convert method group 'Identifier' to non-delegate type 'type'. Did you intend to invoke the method?

This error occurs when converting a method group to a non-delegate type, or attempting to invoke a method without using parentheses.

Example

The following sample generates CS0428:

```
// CS0428.cs

delegate object Del1();
delegate int Del2();

public class C
{
    public static C Method() { return null; }
    public int Foo() { return 1; }

    public static void Main()
    {
        C c = Method; // CS0428, C is not a delegate type.
        int i = (new C()).Foo; // CS0428, int is not a delegate type.

        Del1 d1 = Method; // OK, assign to the delegate type.
        Del2 d2 = (new C()).Foo; // OK, assign to the delegate type.
        // or you might mean to invoke method
        // C c = Method();
        // int i = (new C()).Foo();
    }
}
```

Compiler Error CS0430

Error Message

The extern alias 'alias' was not specified in a /reference option

This error occurs when extern Alias is encountered but Alias was not specified as a reference on the command line. To resolve CS0430, compile with **/reference**.

Example

```
// CS0430_a.cs
// compile with: /target:library
public class MyClass {}
```

Compiling with **/reference:MyType=cs0430_a.dll** to refer to the DLL created in the previous sample resolves this error. The following sample generates CS0430.

```
// CS0430_b.cs
extern alias MyType;    // CS0430
public class Test
{
    public static void Main() {}
}
```

Compiler Error CS0431

Error Message

Cannot use alias 'identifier' with '::' since the alias references a type. Use '.' instead.

You used "::" with an alias that references a type. To resolve this error, use the "." operator.

The following sample generates CS0431:

```
// CS0431.cs
using A = Outer;

public class Outer
{
    public class Inner
    {
        public static void Meth() {}
    }
}

public class MyClass
{
    public static void Main()
    {
        A::Inner.Meth();    // CS0431
        A.Inner.Meth();    // OK
    }
}
```


Compiler Error CS0432

Error Message

Alias 'identifier' not found

This error occurs when you use "::" to the right of an identifier that is not an alias. To resolve the error, use "." instead.

The following example generates CS0432:

```
// CS0432.cs
namespace A {
    public class B {
        public static void Meth() { }
    }
}

public class Test
{
    public static void Main()
    {
        A::B.Meth();    // CS0432
        // To resolve, use the following line instead:
        // A.B.Meth();
    }
}
```

Compiler Error CS0433

Error Message

The type TypeName1 exists in both TypeName2 and TypeName3

Two different assemblies referenced in your application contain the same namespace and type, which produces ambiguity.

To resolve this error, use the alias feature of the [/reference \(Import Metadata\) \(C# Compiler Options\)](#) compiler option or do not reference one of your assemblies.

Example

This code creates the DLL with the first copy of the ambiguous type.

```
// CS0433_1.cs
// compile with: /target:library
namespace TypeBindConflicts
{
    public class AggPubImpAggPubImp {}
}
```

This code creates the DLL with the second copy of the ambiguous type.

```
// CS0433_2.cs
// compile with: /target:library
namespace TypeBindConflicts
{
    public class AggPubImpAggPubImp {}
}
```

The following example generates CS0433.

```
// CS0433_3.cs
// compile with: /reference:cs0433_1.dll /reference:cs0433_2.dll
using TypeBindConflicts;
public class Test
{
    public static void Main()
    {
        AggPubImpAggPubImp n6 = new AggPubImpAggPubImp();    // CS0433
    }
}
```

The following example shows how you can use the alias feature of the **/reference** compiler option to resolve this CS0433 error.

```
// CS0433_4.cs
// compile with: /reference:cs0433_1.dll /reference:TypeBindConflicts=cs0433_2.dll
using TypeBindConflicts;
public class Test
{
    public static void Main()
    {
        AggPubImpAggPubImp n6 = new AggPubImpAggPubImp();
    }
}
```

Compiler Error CS0434

Error Message

The namespace NamespaceName1 in NamespaceName2 conflicts with the type TypeName1 in NamespaceName3

This error occurs when the imported type and the imported namespace have the same fully qualified name. When that name is referenced, the compiler is unable to distinguish between the two.

The following code generates error CS0434.

Example

This code creates the first copy of the type with the identical fully qualified name.

```
// CS0434_1.cs
// compile with: /t:library
namespace TypeBindConflicts
{
    namespace NsImpAggPubImp
    {
        public class X { }
    }
}
```

This code creates the second copy of the type with the identical fully qualified name.

```
// CS0434_2.cs
// compile with: /t:library
namespace TypeBindConflicts {
    // Conflicts with another import (import2.cs).
    public class NsImpAggPubImp { }
    // Try this instead:
    // public class UniqueClassName { }
}
```

This code references the type with the identical fully qualified name.

```
// CS0434.cs
// compile with: /r:cs0434_1.dll /r:cs0434_2.dll
using TypeBindConflicts;
public class Test
{
    public TypeBindConflicts.NsImpAggPubImp.X n2 = null; // CS0434
}
```

Compiler Error CS0438

Error Message

The type 'type' in 'module_1' conflicts with the namespace 'namespace' in 'module_2'.

This error occurs when a type in a source file conflicts with a namespace in another source file. This typically happens when one or both come from an added module. To resolve, rename the type or the namespace that caused the conflict.

The following example generates CS0438:

Compile this file first:

```
// CS0438_1.cs
// compile with: /target:module
public class Util
{
    public class A { }
}
```

Then compile this file:

```
// CS0438_2.cs
// compile with: /target:module
namespace Util
{
    public class A { }
}
```

And then compile this file:

```
// CS0438_3.cs
// compile with: /addmodule:CS0438_1.netmodule /addmodule:CS0438_2.netmodule
using System;
public class Test
{
    public static void Main() {
        Console.WriteLine(typeof(Util.A));    // CS0438
    }
}
```

Compiler Error CS0439

Error Message

An extern alias declaration must precede all other namespace elements

This error occurs when an **extern** declaration comes after something else, such as a **using** declaration, in the same namespace. The **extern** declarations must come before all other namespace elements.

Example

The following example generates CS0439:

```
// CS0439.cs
using System;

extern alias MyType;    // CS0439
// To resolve the error, make the extern alias the first line in the file.

public class Test
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0441

Error Message

'class': a class cannot be both static and sealed

This error occurs when you declare a class that is both static and sealed. Static classes are inherently sealed, so the sealed modifier is not necessary. To resolve, use one modifier only.

The following example generates CS0441:

```
// CS0441.cs  
sealed static class MyClass { }    // CS0441
```

Compiler Error CS0442

Error Message

'Property': abstract properties cannot have private accessors

This error occurs when you use the access modifier "private" to modify an abstract accessor. To resolve, use a different access modifier, or make the property non-abstract.

Example

The following sample generates CS0442:

```
// CS0442.cs
public abstract class MyClass
{
    public abstract int AbstractProperty
    {
        get;
        private set;    // CS0442
        // Try this instead:
        // set;
    }
}
```

Compiler Error CS0443

Error Message

Syntax error, value expected

This error occurs when you reference an array without specifying a value for the array index.

Example

The following code generates CS0443.

```
// CS0443.cs
using System;
class MyClass
{
    public static void Main()
    {
        int[,] x = new int[1,5];
        if (x[] == 5) {} // CS0443
        // if (x[0, 0] == 5) {}
    }
}
```


Compiler Error CS0445

Error Message

Cannot modify the result of an unboxing conversion

The result of an unboxing conversion is a temporary variable, and the compiler prevents you from modifying such variables because any modification would go away when the temporary goes away. To fix this, use a value type to store the intermediate expression. You will then be able to assign a value to the value type.

The following code generates CS0455:

```
// CS0445.CS
public struct Point
{
    public int x;
    public static void SetX(object obj, int x)
    {
        ((Point)obj).x = x; // CS0445
    }
}
class UnboxingTest{public static void Main(){}}
```

Compiler Error CS0446

Error Message

Foreach cannot operate on a 'Method or Delegate'. Did you intend to invoke the 'Method or Delegate'?

This error is caused by specifying a method without parentheses or an anonymous method without parentheses in the part of the **foreach** statement where you would normally put a collection class. Note that it is valid, though unusual, to put a method call in that location, if the method returns a collection class.

Example

The following code will generate CS0446.

```
// CS0446.cs
using System;
class Tester
{
    static void Main()
    {
        int[] intArray = new int[5];
        foreach (int i in M) { } // CS0446
    }
    static void M() { }
```

Compiler Error CS0447

Error Message

Attributes cannot be used on type arguments, only on type parameters

This error occurs when you apply an attribute to a type argument that occurs in an invocation statement. It is acceptable to apply an attribute to a type parameter in a class or method declaration statement such as the following:

```
class C<[some attribute] T> {...}
```

The following line of code will generate this error. It is assumed that the class `C`, defined in the previous line of code, has a static method called `MyStaticMethod`.

```
C<[some attribute] T>.MyStaticMethod();
```

Example

The following code generates error CS0447.

```
// CS0447.cs
using System;
namespace Test41
{
    public interface I<A>
    {
        void Meth<B>();
    }
    public class B : I<int>
    {
        void I<[Test] int>.Meth<X>() { } // CS0447
    }
}
```

Compiler Error CS0448

Error Message

The return type for ++ or -- operator must be the containing type or derived from the containing type

When you override the ++ or -- operators, they must return the same type as the containing type, or return a type that is derived from the containing type.

Example

The following sample generates CS0448.

```
// CS0448.cs
class C5
{
    public static int operator ++(C5 c) { return null; }    // CS0448
    public static C5 operator --(C5 c) { return null; }    // OK
    public static void Main() {}
}
```

The following sample generates CS0448.

```
// CS0448_b.cs
public struct S
{
    public static S? operator ++(S s) { return new S(); }    // CS0448
    public static S? operator --(S s) { return new S(); }    // CS0448
}

public struct T
{
    // OK
    public static T operator --(T t) { return new T(); }
    public static T operator ++(T t) { return new T(); }

    public static T? operator --(T? t) { return new T(); }
    public static T? operator ++(T? t) { return new T(); }

    public static void Main() {}
}
```

Compiler Error CS0449

Error Message

The 'class' or 'struct' constraint must come before any other constraints

The constraints on the type parameter of a generic type or method must occur in a specific order: **class** or **struct** must be first, if present, then any interface constraints, and finally any constructor constraints. This error is caused by the **class** or **struct** constraint not appearing first. To resolve this error, reorder the constraint clauses.

Example

The following sample generates CS0449.

```
// CS0449.cs
// compile with: /target:library
interface I {}
public class C4
{
    public void F1<T>() where T : class, struct, I {}    // CS0449
    public void F2<T>() where T : I, struct {}        // CS0449
    public void F3<T>() where T : I, class {}         // CS0449

    // OK
    public void F4<T>() where T : class {}
    public void F5<T>() where T : struct {}
    public void F6<T>() where T : I {}
}
```

Compiler Error CS0450

Error Message

'Type Parameter Name': Cannot specify both a constraint class and the 'class' or 'struct' constraint

If a type parameter is constrained by the struct type constraint, it is logically contradictory for it to also be constrained by a specific class type, since struct and class are mutually exclusive categories. If a type parameter is constrained by a specific class type constraint, then it is by definition constrained by the class type constraint, and so specifying the class type constraint is redundant.

Example

```
// CS0450.cs
// compile with: /t:library
public class GenericsErrors
{
    public class B { }
    public class G3<T> where T : struct, B { } // CS0450
// To resolve, use the following line instead:
// public class G3<T> where T : B { }
}
```

Compiler Error CS0451

Error Message

The 'new()' constraint cannot be used with the 'struct' constraint

When specifying constraints on the type of a generic, the **new()** constraint may only be used with class type constraints, interface type constraints, reference type constraints, and type parameter constraints, but not with value type constraints.

Example

The following sample generates CS0451.

```
// CS0451.cs
using System;
public class C4
{
    public void F4<T>() where T : struct, new() {}    // CS0451
}

// OK
public class C5
{
    public void F5<T>() where T : struct {}
}

public class C6
{
    public void F6<T>() where T : new() {}
}
```

Compiler Error CS0452

Error Message

The type 'type name' must be a reference type in order to use it as parameter 'parameter name' in the generic type or method 'identifier of generic'

This error occurs when you pass a value type such as a **struct** or **int** as a parameter to a generic type or method that has a reference type constraint.

Example

The following code generates error CS0452.

```
// CS0452.cs
using System;
public class BaseClass<S> where S : class { }
public class Derived1 : BaseClass<int> { } // CS0452
public class Derived2<S> : BaseClass<S> where S : struct { } // CS0452
```


Compiler Error CS0453

Error Message

The type 'Type Name' must be a non-nullable value type in order to use it as parameter 'Parameter Name' in the generic type or method 'Generic Identifier'

This error occurs when you use a non-value type argument in instantiating a generic type or method which has the **value** constraint on it. It can also occur when you use a nullable value type argument. See the last two lines of code in the following example.

Example

The following code generates this error.

```
// CS0453.cs
using System;
public class HV<S> where S : struct { }
public class H1 : HV<string> { }           // CS0453
public class H2 : HV<H1> { }              // CS0453
public class H3<S> : HV<S> where S : class { } // CS0453
public class H4 : HV<int?> { }             // CS0453
public class H5 : HV<Nullable<Nullable<int>>> { } // CS0453
```

Compiler Error CS0454

Error Message

Circular constraint dependency involving 'Type Parameter 1' and 'Type Parameter 2'

This error arises because at some point one type parameter refers to another, and the second refers back to the first. To fix this error, break the circular dependency by removing one of the constraints. Note that the circular constraint dependency can be indirect.

Example

The following code generates error CS0454.

```
// CS0554
using System;
public class GenericsErrors
{
    public class G4<T> where T : T { } // CS0454
}
```

The following example demonstrates a circular dependency between two type constraints.

```
public class Gen<T,U> where T : U where U : T // CS0454
{
}
```

Compiler Error CS0455

Error Message

Type parameter 'Type Parameter Name' inherits conflicting constraints 'Constraint Name 1' and 'Constraint Name 2'

Two common ways to get this error are to set up constraints so that the type parameter derives from two non-related classes, or so that it derives from a class type or reference type constraint and a **struct** type or value type constraint. To resolve this error, remove the conflict from your inheritance hierarchy.

Example

The following code generates error CS0455.

```
// CS0455.cs
using System;

public class GenericsErrors {
    public class B { }
    public class B2 { }
    public class G6<T> where T : B { public class N<U> where U : B2, T { } } // CS0455
}
```

Compiler Error CS0456

Error Message

Type parameter 'Type Parameter Name 1' has the 'struct' constraint so 'Type Parameter Name 1' cannot be used as a constraint for 'Type Parameter Name 2'

Value type constraints are implicitly sealed so those constraints cannot be used as constraints on a second type parameter. This is because value types cannot be overridden. To resolve this error, put a value type constraint directly on the second type parameter, instead of doing so indirectly by means of the first type parameter.

Example

The following sample generates CS0456.

```
// CS0456.cs
// compile with: /target:library
public class GenericsErrors
{
    public class G5<T> where T : struct
    {
        public class N<U> where U : T {}    // CS0456
        public class N2<U> where U : struct {}    // OK
    }
}
```

Compiler Error CS0457

Error Message

Ambiguous user defined conversions 'Conversion method name 1' and 'Conversion method name 2' when converting from 'type name 1' to 'type name 2'

Two conversion methods are applicable, and the compiler is unable to decide which one to use.

One scenario that can cause this error is as follows:

- You want to convert class A to class B where A and B are unrelated.
- A is derived from A0, and there is a method that converts from A0 to B.
- B has a subclass B1 and there is a method that converts from A to B1.

The compiler will weight the two conversion methods equally, because the first conversion provides the best destination type, and the second conversion provides the best source type. Since the compiler will be unable to choose, this error will be generated. To resolve, write a new explicit method converting A to B.

Another scenario that causes this error is if there are two methods that convert A to B. To fix, specify which conversion to use through an explicit cast.

Example

The following sample generates CS0457.

```
// CS0457.cs
using System;
public class A { }

public class G0 { }
public class G1<R> : G0 { }

public class H0 {
    public static implicit operator G0(H0 h) {
        return new G0();
    }
}
public class H1<R> : H0 {
    public static implicit operator G1<R>(H1<R> h) {
        return new G1<R>();
    }
}

public class Test
{
    public static void F0(G0 g) { }
    public static void Main()
    {
        H1<A> h1a = new H1<A>();
        F0(h1a);    // CS0457
    }
}
```

Compiler Error CS0459

Error Message

Cannot take the address of a read-only local variable

There are three common scenarios in the C# language that generate read-only local variables: **foreach**, **using**, and **fixed**. In each of these cases, you are not allowed to write to the read-only local variable, or to take its address. This error is generated when the compiler realizes you are trying to take the address of a read-only local variable.

Example

The following example generates CS0459 when an attempt is made to take the address of a read-only local variable in a **foreach** loop and in a **fixed** statement block.

```
// CS0459.cs
// compile with: /unsafe

class A
{
    public unsafe void M1()
    {
        int[] ints = new int[] { 1, 2, 3 };
        foreach (int i in ints)
        {
            int *j = &i; // CS0459
        }

        fixed (int *i = &_i)
        {
            int **j = &i; // CS0459
        }
    }

    private int _i = 0;
}
```

Compiler Error CS0460

Error Message

Constraints for override and explicit interface implementation methods are inherited from the base method, so they cannot be specified directly

When a generic method that is part of a derived class overrides a method in the base class, you may not specify constraints on the overridden method. The override method in the derived class inherits its constraints from the method in the base class.

Example

The following sample generates CS0460.

```
// CS0460.cs
// compile with: /target:library
class BaseClass
{
    BaseClass() { }
}

interface I
{
    void F1<T>() where T : BaseClass;
    void F2<T>() where T : struct;
    void F3<T>() where T : BaseClass;
}

class ExpImpl : I
{
    void I.F1<T>() where T : BaseClass {}    // CS0460
    void I.F2<T>() where T : class {}      // CS0460
}
```

Compiler Error CS0462

Error Message

The inherited members 'member1' and 'member2' have the same signature in type 'type', so they cannot be overridden

This error arises with the introduction of generics. Normally, you cannot have two versions of a method in a class with the same signature. But with generics, you can specify a generic method that might duplicate another method if it is instantiated with a particular type.

Example

When `C<int>` is instantiated, two versions of the method `F` are created with the same signature, so the override in class `D` cannot decide which one to apply the override to.

The following sample generates CS0462.

```
// CS0462.cs
// compile with: /target:library
class C<T>
{
    public virtual void F(T t) {}
    public virtual void F(int t) {}
}

class D : C<int>
{
    public override void F(int t) {}    // CS0462
}
```


Compiler Error CS0463

Error Message

Evaluation of the decimal constant expression failed with error: 'error'

This error occurs when a constant decimal expression overflows at compile time.

Typically you get overflow errors at run time. In this case, you defined the constant expression in such a way that the compiler could evaluate the result and know that an overflow would happen.

Example

The following code generates error CS0463.

```
// CS0463.cs
using System;
class MyClass
{
    public static void Main()
    {
        const decimal myDec = 7900000000000000000000000000.0m + 7900000000000000000000000000
000.0m; // CS0463
        Console.WriteLine(myDec.ToString());
    }
}
```

Compiler Error CS0466

Error Message

'method1' should not have a params parameter since 'method2' does not

You cannot use **params** parameter on a class member if the implemented interface doesn't use it.

Example

The following sample generates CS0466.

```
// CS0466.cs
interface I
{
    void F1(params int[] a);
    void F2(int[] a);
}

class C : I
{
    void I.F1(params int[] a) {}
    void I.F2(params int[] a) {} // CS0466
    void I.F2(int[] a) {} // OK

    public static void Main()
    {
        I i = (I) new C();

        i.F1(new int[] {1, 2} );
        i.F2(new int[] {1, 2} );
    }
}
```

Compiler Error CS0468

Error Message

Ambiguity between type 'type1' and type 'type2'

This error is generated when there are two types in the assembly being compiled that have the same fully qualified name. This could occur if they are both in added modules or one is in an added module and one in source.

Compiler Error CS0470

Error Message

Method 'method' cannot implement interface accessor 'accessor' for type 'type'. Use an explicit interface implementation.

This error is generated when an accessor is trying to implement an interface. Explicit interface implementation must be used.

Example

The following sample generates CS0470.

```
// CS0470.cs
// compile with: /target:library

interface I
{
    int P { get; }
}

class MyClass : I
{
    public int get_P() { return 0; }    // CS0470
    public int P2 { get { return 0;} } // OK
}
```

Compiler Error CS0471

Error Message

The variable 'variable' is not a generic method. If you intended an expression list, use parentheses around the < expression.

This error is generated when your code contains an expression list without parentheses.

Example

The following example generates CS0471.

```
// CS0471.cs
// compile with: /t:library
class Test
{
    public void F(bool x, bool y) {}
    public void F1()
    {
        int a = 1, b = 2, c = 3;
        F(a<b, c>(3));    // CS0471
        // To resolve, try the following instead:
        // F((a<b), c>(3));
    }
}
```

Compiler Error CS0500

Error Message

'class member' cannot declare a body because it is marked abstract

An [abstract](#) method cannot contain its implementation.

The following sample generates CS0500:

```
// CS0500.cs
namespace x
{
    abstract public class clx
    {
        abstract public void f(){}    // CS0500
        // try the following line instead
        // abstract public void f();
    }

    public class cly
    {
        public static int Main()
        {
            return 0;
        }
    }
}
```

Compiler Error CS0501

Error Message

'member function' must declare a body because it is not marked abstract or extern

Nonabstract methods must have implementations.

The following sample generates CS0501:

```
// CS0501.cs
// compile with: /target:library
public class clx
{
    public void f();    // CS0501 declared not defined
    public void g() {} // OK
}
```

Compiler Error CS0502

Error Message

'member' cannot be both abstract and sealed

A class member cannot be both [abstract](#) and [sealed](#).

The following sample generates CS0502:

```
// CS0502.cs
public class B
{
    abstract public void F();
}

public class C : B
{
    abstract sealed override public void F()    // CS0502
    {
    }
}

public class CMain
{
    public static void Main()
    { }
}
```


Compiler Error CS0503

Error Message

The abstract method 'method' cannot be marked virtual

It is redundant to mark a member method as both [abstract](#) and [virtual](#) because **abstract** implies **virtual**.

The following sample generates CS0503:

```
// CS0503.cs
namespace x
{
    abstract public class clx
    {
        abstract virtual public void f();    // CS0503
    }
}
```

Compiler Error CS0504

Error Message

The constant 'variable' cannot be marked static

If a variable is [const](#), it is also [static](#). If you want a **const** and **static** variable, just declare that variable as **const**; if all you want is a **static** variable, just mark it **static**.

The following sample generates CS0504:

```
// CS0504.cs
namespace x
{
    abstract public class clx
    {
        static const int i = 0;    // CS0504, cannot be both static and const
        abstract public void f();
    }
}
```

Compiler Error CS0505

Error Message

'member1': cannot override because 'member2' is not a function

A class declaration attempted to override a non-method in a base class. Overrides must match the member type. If a method with the same name as a method in a base class is desired, use [new](#) (and not [override](#)) on the method declaration in the base class.

The following sample generates CS0505:

```
// CS0505.cs
// compile with: /target:library
public class clx
{
    public int i;
}

public class cly : clx
{
    public override int i() { return 0; }    // CS0505
}
```

Compiler Error CS0506

Error Message

'function1' : cannot override inherited member 'function2' because it is not marked "virtual", "abstract", or "override"

A method was overridden that was not explicitly marked as **virtual**, **abstract**, or **override**.

The following sample generates CS0506:

```
// CS0506.cs
namespace MyNameSpace
{
    abstract public class ClassX
    {
        public int i = 0;

        public int f()
        {
            return 0;
        }
        // Try the following definition for f() instead:
        // abstract public int f();
    }

    public class ClassY : ClassX
    {
        public override int f()    // CS0506
        {
            return 0;
        }

        public static int Main()
        {
            return 0;
        }
    }
}
```

Compiler Error CS0507

Error Message

'function1' : cannot change access modifiers when overriding 'access' inherited member 'function2'

An attempt was made to change the access specification in a method override.

Example

The following sample generates CS0507.

```
// CS0507.cs
abstract public class clx
{
    virtual protected void f() {}
}

public class cly : clx
{
    public override void f() {}    // CS0507
    public static void Main() {}
}
```

CS0507 can also occur if a class attempts to override a method marked as **protected internal** defined in referenced metadata. In this situation, the overriding method should be marked as **protected**.

```
// CS0507_b.cs
// compile with: /target:library
abstract public class clx
{
    virtual protected internal void f() {}
}
```

The following sample generates CS0507.

```
// CS0507_c.cs
// compile with: /reference:cs0507_b.dll
public class cly : clx
{
    protected internal override void f() {}    // CS0507
    // try the following line instead
    // protected override void f() {}    // OK

    public static void Main() {}
}
```

Compiler Error CS0508

Error Message

'Type 1': return type must be 'Type 2' to match overridden member 'Member Name'

An attempt was made to change the return type in a method override. To resolve this error, make sure both methods declare the same return type.

Example

The following sample generates CS0508.

```
// CS0508.cs
// compile with: /target:library
abstract public class Clx
{
    public int i = 0;
    // Return type is int.
    abstract public int F();
}

public class Cly : Clx
{
    public override double F()
    {
        return 0.0;    // CS0508
    }
}
```

Compiler Error CS0509

Error Message

'class1' : cannot derive from sealed type 'class2'

A [sealed](#) class cannot act as a [base](#) class. Structs are sealed by default.

The following sample generates CS0509:

```
// CS0509.cs
// compile with: /target:library
sealed public class clx {}
public class cly : clx {}    // CS0509
```

Compiler Error CS0513

Error Message

'function' is abstract but it is contained in nonabstract class 'class'

A method cannot be an [abstract](#) member of a nonabstract class.

The following sample generates CS0513:

```
// CS0513.cs
namespace x
{
    public class clx
    {
        abstract public void f();    // CS0513, abstract member of nonabstract class
    }
}
```


Compiler Error CS0514

Error Message

'constructor' : static constructor cannot have an explicit 'this' or 'base' constructor call

Calling **this** in the static constructor is not allowed because the static constructor is called automatically before creating any instance of the class. Also, static constructors are not inherited, and cannot be called directly.

For more information, see [this \(C# Reference\)](#) and [base \(C# Reference\)](#).

Example

The following example generates CS0514:

```
// CS0514.cs
class A
{
    static A() : base(0) // CS0514
    {
    }

    public A(object o)
    {
    }
}

class B
{
    static B() : this(null) // CS0514
    {
    }

    public B(object o)
    {
    }
}
```

Compiler Error CS0515

Error Message

'function' : access modifiers are not allowed on static constructors

A static constructor cannot have an [access modifier](#).

Example

The following sample generates CS0515:

```
// CS0515.cs
public class Clx
{
    public static void Main()
    {
    }
}

public class Clz
{
    public static Clz()    // CS0515, remove public keyword
    {
    }
}
```

Compiler Error CS0516

Error Message

Constructor 'constructor' can not call itself

A program cannot recursively call constructors.

The following sample generates CS0516:

```
// CS0516.cs
namespace x
{
    public class clx
    {
        public clx() : this()    // CS0516, delete "this()"
        {
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0517

Error Message

'class' has no base class and cannot call a base constructor

CS0517 can occur only when the .NET Framework common language runtime compiles the source code for the object class, which is the only class that has no base class.

Compiler Error CS0518

Error Message

Predefined type 'type' is not defined or imported

The main cause for this problem is that the project is not importing mscorlib.dll, which defines the entire System namespace. This can be caused by one of the following:

- The [/nostdlib](#) option from the command line compiler has been specified. The /nostdlib option prevents the import of mscorlib.dll. Use this option if you want to define or create a user-specific System namespace.
- An incorrect mscorlib.dll is referenced.
- A corrupt Visual Studio .NET or .NET Framework common language runtime installation exists.
- Residual components from an earlier installation that are incompatible with the latest installation remain.

To resolve this problem, take one of the following actions:

- Do not specify the /nostdlib option from the command line compiler.
- Make sure that the project refers to the correct mscorlib.dll.
- Reinstall the .NET Framework common language runtime (if the previous solutions do not solve the problem).

Compiler Error CS0520

Error Message

Predefined type 'name' is declared incorrectly

One or more files needed by the compiler were not found. Reinstall the .NET Framework common language runtime.

Compiler Error CS0522

Error Message

'constructor' : structs cannot call base class constructors

A [struct](#) cannot call a base class constructor; remove the call to the base class constructor.

The following sample generates CS0522:

```
// CS0522.cs
public class clx
{
    public clx(int i)
    {
    }

    public static void Main()
    {
    }
}

public struct cly
{
    public cly(int i):base(0)    // CS0522
    // try the following line instead
    // public cly(int i)
    {
    }
}
```

Compiler Error CS0523

Error Message

Struct member 'struct2 field' of type 'struct1' causes a cycle in the struct layout

The definitions of two structs include recursive references. Change the [struct](#) definitions such that each does not define itself on the other. This limitation applies only to structs, since structs are value types. If using recursive references, declare your types as classes.

The following sample generates CS0523:

```
// CS0523.cs
// compile with: /target:library
struct RecursiveLayoutStruct1
{
    public RecursiveLayoutStruct2 field;
}

struct RecursiveLayoutStruct2
{
    public RecursiveLayoutStruct1 field;    // CS0523
}
```


Compiler Error CS0524

Error Message

'type' : interfaces cannot declare types

An [interface](#) cannot contain a user-defined type; it should contain only methods and properties.

Example

The following sample generates CS0524:

```
// CS0524.cs
public interface Clx
{
    public class Cly    // CS0524, delete user-defined type
    {
    }
}
```

Compiler Error CS0525

Error Message

Interfaces cannot contain fields

An [interface](#) can contain methods and properties but not fields.

The following sample generates CS0525:

```
// CS0525.cs
namespace x
{
    public interface clx
    {
        public int i;    // CS0525
    }
}
```

Compiler Error CS0526

Error Message

Interfaces cannot contain constructors

Constructors cannot be defined for [interfaces](#). A method is considered a constructor if it has the same name as the class and no return type.

The following sample generates CS0526:

```
// CS0526.cs
namespace x
{
    public interface clx
    {
        public clx()    // CS0526
        {
        }
    }

    public class cly
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0527

Error Message

Type 'type' in interface list is not an interface

It is possible for a [struct](#) or [interface](#) to inherit from another interface but not from any other type.

The following sample generates CS0527:

```
// CS0527.cs
// compile with: /target:library
public struct clx : int {}    // CS0527 int not an interface
```

Compiler Error CS0528

Error Message

'interface' is already listed in interface list

An interface-inheritance list includes a duplicate. An [interface](#) can only be specified once in the inheritance list.

The following sample generates CS0528:

```
// CS0528.cs
namespace x
{
    public interface a
    {
    }

    public class b : a, a    // CS0528
    {
        public void Main()
        {
        }
    }
}
```

Compiler Error CS0529

Error Message

Inherited interface 'interface1' causes a cycle in the interface hierarchy of 'interface2'

The inheritance list for an [interface](#) includes a direct or indirect reference to itself. An interface cannot inherit from itself.

The following sample generates CS0529:

```
// CS0529.cs
namespace x
{
    public interface a
    {
    }

    public interface b : a, c
    {
    }

    public interface c : b    // CS0529, b inherits from c
    {
    }
}
```

Compiler Error CS0531

Error Message

'member' : interface members cannot have a definition

Methods that are declared in an [interface](#) must be implemented in a class that inherits from it and not in the interface itself.

The following sample generates CS0531:

```
// CS0531.cs
namespace x
{
    public interface clx
    {
        int xclx()    // CS0531, cannot define xclx
        // Try the following declaration instead:
        // int xclx();
        {
            return 0;
        }
    }

    public class cly
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0533

Error Message

'derived-class member' hides inherited abstract member 'base-class member'

A base [class](#) method is hidden. Check the syntax of your declaration to see if it is correct.

For more information, see [base](#).

The following sample generates CS0533:

```
// CS0533.cs
namespace x
{
    abstract public class a
    {
        abstract public void f();
    }

    abstract public class b : a
    {
        new abstract public void f();    // CS0533
        // try the following lines instead
        // override public void f()
        // {
        // }

        public static void Main()
        {
        }
    }
}
```


Compiler Error CS0534

Error Message

'function1' does not implement inherited abstract member 'function2'

A class is required to implement all the [abstract](#) members in the base class, unless the class is also abstract.

The following sample generates CS0534:

```
// CS0534.cs
namespace x
{
    abstract public class clx
    {
        public abstract void f();
    }

    public class cly : clx    // CS0534, no override for clx::f
    {
        // uncomment the following sample override to resolve CS0534
        // override public void f()
        // {
        // }

        public static int Main()
        {
            return 0;
        }
    }
}
```

Compiler Error CS0535

Error Message

'class' does not implement interface member 'member'

A [class](#) derived from an [interface](#), but the class did not implement one or more of the interface's members. A class must implement all members of interfaces from which it derives or else be declared **abstract**.

Example

The following sample generates CS0535.

```
// CS0535.cs
public interface A
{
    void F();
}

public class B : A {}    // CS0535 A::F is not implemented

// OK
public class C : A {
    public void F() {}
    public static void Main() {}
}
```

The following sample generates CS0535.

```
// CS0535_b.cs
using System;
class C : IDisposable {}    // CS0535

// OK
class D : IDisposable {
    void IDisposable.Dispose() {}
    public void Dispose() {}

    static void Main() {
        using (D d = new D()) {}
    }
}
```

Compiler Error CS0536

Error Message

'class' does not implement interface member 'interface member'. 'class member' is static, not public, or has the wrong return type

'class' does not implement interface member 'member1'. 'member2' is either static, not public, or has the wrong return type.

The compiler did not detect an [interface](#) member implementation. A declaration may be present that almost implements the interface member. Check for the following syntax errors in the declaration for the interface member:

- **public** keyword is omitted.
- Return type does not match.
- **static** keyword is present.

The following sample generates CS0536:

```
// CS0536.cs
public interface a
{
    void f();
}

public class b : a
{
    public static int f()    // CS0536
    // try the following line instead
    // public void f()
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0537

Error Message

The class System.Object cannot have a base class or implement an interface

CS0537 occurs when rebuilding the [System](#) class libraries, and where [Object](#) derives from another class. You are very unlikely to encounter this error. If you do, do not derive **Object** from a class or interface: it is the root of the entire .NET Framework class hierarchy, and as such, does not derive from anything else.

Compiler Error CS0538

Error Message

'name' in explicit interface declaration is not an interface

An attempt was made to explicitly declare an [interface](#), but an interface was not specified.

The following sample generates CS0538:

```
// CS0538.cs
interface MyIFace
{
    void F();
}

public class MyClass
{
    public void G()
    {
    }
}

class C: MyIFace
{
    void MyIFace.F()
    {
    }

    void MyClass.G()    // CS0538, MyClass not an interface
    {
    }
}
```

Compiler Error CS0539

Error Message

'member' in explicit interface declaration is not a member of interface

An attempt was made to explicitly declare an [interface](#) member that does not exist. You should either delete the declaration or change it so that it refers to a valid interface member.

The following sample generates CS0539:

```
// CS0539.cs
namespace x
{
    interface I
    {
        void m();
    }

    public class clx : I
    {
        void I.x()    // CS0539
        {
        }

        public static int Main()
        {
            return 0;
        }
    }
}
```

Compiler Error CS0540

Error Message

'interface member' : containing type does not implement interface 'interface'

You attempted to implement an interface member in a [class](#) that does not derive from the [interface](#). You should either delete the implementation of the interface member or add the interface to the base-class list of the class.

Example

The following sample generates CS0540.

```
// CS0540.cs
interface I
{
    void m();
}

public class Clx
{
    void I.m() {}    // CS0540
}

// OK
public class Cly : I
{
    void I.m() {}
    public static void Main() {}
}
```

The following sample generates CS0540.

```
// CS0540_b.cs
using System;
class C {
    void IDisposable.Dispose() {}    // CS0540
}

class D : IDisposable {
    void IDisposable.Dispose() {}
    public void Dispose() {}

    static void Main() {
        using (D d = new D()) {}
    }
}
```

Compiler Error CS0541

Error Message

'declaration' : explicit interface declaration can only be declared in a class or struct

An explicit [interface](#) declaration was found outside a [class](#) or [struct](#).

The following sample generates CS0541:

```
// CS0541.cs
namespace x
{
    interface IFace
    {
        void F();
    }

    interface IFace2 : IFace
    {
        void IFace.F();    // CS0541
    }
}
```


Compiler Error CS0542

Error Message

'user-defined type' : member names cannot be the same as their enclosing type

A name was used more than once in the same construct. This error might be caused by inadvertently putting a return type on a constructor.

The following sample generates CS0542:

```
// CS0542.cs
class F
{
    // Remove void from F() to resolve the problem.
    void F()    // CS0542, same name as the class
    {
    }
}

class MyClass
{
    public static void Main()
    {
    }
}
```

If your class is named 'Item' and has an indexer declared as `this`, you may get this error. A default indexer is given the name 'Item' in the emitted code, creating the conflict.

```
// CS0542b.cs
class Item
{
    public int this[int i]    // CS0542
    {
        get
        {
            return 0;
        }
    }
}

class CMain
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0543

Error Message

'enumeration' : the enumerator value is too large to fit in its type

A value that was assigned to an element in an [enumeration](#) is outside the range of the data type.

The following sample generates CS0543:

```
// CS0543.cs
namespace x
{
    enum I : byte
    {a = 255, b, c}    // CS0543
    public class clx
    {
        public static int Main()
        {
            return 0;
        }
    }
}
```

Compiler Error CS0544

Error Message

'property override': cannot override because 'non-property' is not a property

An attempt was made to override a nonproperty data type as a [property](#), which is not allowed.

The following sample generates CS0544:

```
// CS0544.cs
// compile with: /target:library
public class a
{
    public int i;
}

public class b : a
{
    public override int i {    // CS0544
        // try the following line instead
        // public new int i {
            get
            {
                return 0;
            }
        }
    }
}
```

Compiler Error CS0545

Error Message

'function' : cannot override because 'property' does not have an overridable get accessor

An attempt was made to define an override for a property accessor when the base class has no such definition to override. You can resolve this error by:

- Adding a **set** accessor in the base class.
- Removing the **set** accessor from the derived class.
- Hiding the base class property by adding the **new** keyword in derived class's property.
- Making the base class property **virtual**.

For more information, see [Using Properties \(C# Programming Guide\)](#) and [Using Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS0545.

```
// CS0545.cs
// compile with: /target:library
// CS0545
public class a
{
    public virtual int i
    {
        set {}

        // Uncomment the following line to resolve.
        // get { return 0; }
    }
}

public class b : a
{
    public override int i
    {
        get { return 0; }
        set {} // OK
    }
}
```

Compiler Error CS0546

Error Message

'accessor' : cannot override because 'property' does not have an overridable set accessor

An attempt to override one of the accessor methods for a property failed because the accessor cannot be overridden. You can resolve this error by:

- Adding a **set** accessor in the base class.
- Removing the **set** accessor from the derived class.
- Hiding the base class property by adding the [new](#) keyword in derived class's property.
- Making the base class property [virtual](#).

For more information, see [Property Declaration](#) and [Using Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS0546.

```
// CS0546.cs
// compile with: /target:library
public class a
{
    public virtual int i
    {
        get
        {
            return 0;
        }
    }

    public virtual int i2
    {
        get
        {
            return 0;
        }

        set {}
    }
}

public class b : a
{
    public override int i
    {
        set {}    // CS0546 error no set
    }

    public override int i2
    {
        set {}    // OK
    }
}
```

Compiler Error CS0547

Error Message

'property' : property or indexer cannot have void type

`void` is invalid as a return value for a property.

For more information, see [Properties](#).

The following sample generates CS0547:

```
// CS0547.cs
public class a
{
    public void i    // CS0547
    // Try the following declaration instead:
    // public int i
    {
        get
        {
            return 0;
        }
    }
}

public class b : a
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0548

Error Message

'property' : property or indexer must have at least one accessor

A property must have at least one accessor (get or set) method.

For more information, see [Property Declaration](#) and [Using Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS0548.

```
// CS0548.cs
// compile with: /target:library
public class b
{
    public int MyProp {}    // CS0548

    public int MyProp2    // OK
    {
        get
        {
            return 0;
        }
        set {}
    }
}
```

Compiler Error CS0549

Error Message

'function' is a new virtual member in sealed class 'class'

A [sealed class](#) cannot be used as a base class. Therefore, it is useless to have a virtual method in sealed class.

The following sample generates CS0549:

```
// CS0549.cs
// compile with: /target:library
sealed public class MyClass
{
    virtual public void TestMethod() {}    // CS0549
    public void TestMethod2() {}         // OK
}
```


Compiler Error CS0550

Error Message

'accessor' adds an accessor not found in interface member 'property'

The implementation of a property in a derived class contains an accessor that was not specified in the base interface.

For more information, see [Using Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS0550.

```
// CS0550.cs
namespace x
{
    interface ii
    {
        int i
        {
            get;
            // add the following accessor to resolve this CS0550
            // set;
        }
    }

    public class a : ii
    {
        int ii.i
        {
            get
            {
                return 0;
            }
            set {} // CS0550 no set in interface
        }

        public static void Main() {}
    }
}
```

Compiler Error CS0551

Error Message

Explicit interface implementation 'implementation' is missing accessor 'accessor'

A class that explicitly implements an interface's property must implement all the accessors that the interface defines.

For more information, see [Property Declaration](#) and [Using Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS0551.

```
// CS0551.cs
// compile with: /target:library
interface ii
{
    int i
    {
        get;
        set;
    }
}

public class a : ii
{
    int ii.i { set {} }    // CS0551

    // OK
    int ii.i
    {
        set {}
        get { return 0; }
    }
}
```

Compiler Error CS0552

Error Message

'conversion routine' : user defined conversion to/from interface

You cannot create a user-defined conversion to or from an interface. If you need the conversion routine, resolve this error by making the interface a class or derive a class from the interface.

The following sample generates CS0552:

```
// CS0552.cs
public interface ii
{
}

public class a
{
    // delete the routine to resolve CS0552
    public static implicit operator ii(a aa) // CS0552
    {
        return new ii();
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0553

Error Message

'conversion routine' : user defined conversion to/from base class

User-defined conversions to values of a base class are not allowed; you do not need such an operator.

The following sample generates CS0553:

```
// CS0553.cs
namespace x
{
    public class ii
    {
    }

    public class a : ii
    {
        // delete the conversion routine to resolve CS0553
        public static implicit operator ii(a aa) // CS0553
        {
            return new ii();
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0554

Error Message

'conversion routine' : user defined conversion to/from derived class

User-defined conversions to values of a derived class are not allowed; you do not need such an operator.

See chapter 6 in the C# language specification for more information on user-defined conversions.

The following sample generates CS0554:

```
// CS0554.cs
namespace x
{
    public class ii
    {
        // delete the conversion routine to resolve CS0554
        public static implicit operator ii(a aa) // CS0554
        {
            return new ii();
        }
    }

    public class a : ii
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0555

Error Message

User-defined operator cannot take an object of the enclosing type and convert to an object of the enclosing type

User-defined conversions to values of the enclosing class are not allowed; you do not need such an operator.

The following sample generates CS0555:

```
// CS0555.cs
public class MyClass
{
    // delete the following operator to resolve this CS0555
    public static implicit operator MyClass(MyClass aa)    // CS0555
    {
        return new MyClass();
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0556

Error Message

User-defined conversion must convert to or from the enclosing type

A user-defined conversion routine must convert to or from the class that contains the routine.

The following sample generates CS0556:

```
// CS0556.cs
namespace x
{
    public class ii
    {
        public class iii
        {
            public static implicit operator int(byte aa)    // CS0556
            // try the following line instead
            // public static implicit operator int(iii aa)
            {
                return 0;
            }
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0557

Error Message

Duplicate user-defined conversion in type 'class'

Duplicate conversion routines are not allowed in a class.

The following example generates CS0557:

```
// CS0557.cs
namespace x
{
    public class ii
    {
        public class iii
        {
            public static implicit operator int(iii aa)
            {
                return 0;
            }

            // CS0557, delete duplicate
            public static explicit operator int(iii aa)
            {
                return 0;
            }
        }

        public static void Main()
        {
        }
    }
}
```


Compiler Error CS0558

Error Message

User-defined operator 'operator' must be declared static and public

Both the **static** and **public** access [modifiers](#) must be specified on user-defined operators.

The following sample generates CS0558:

```
// CS0558.cs
namespace x
{
    public class ii
    {
        public class iii
        {
            static implicit operator int(iii aa)    // CS0558, add public
            {
                return 0;
            }
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0559

Error Message

The parameter type for ++ or -- operator must be the containing type

The method declaration for an operator overload must follow certain guidelines. For the ++ and -- operators, it is required that the parameter be of the same type as the type in which the operator is being overloaded.

Example

The following sample generates CS0559:

```
// CS0559.cs
// compile with: /target:library
public class iii
{
    public static implicit operator int(iii x)
    {
        return 0;
    }

    public static implicit operator iii(int x)
    {
        return null;
    }

    public static int operator ++(int aa)    // CS0559
    // try the following line instead
    // public static iii operator ++(iii aa)
    {
        return (iii)0;
    }
}
```

The following sample generates CS0559.

```
// CS0559_b.cs
// compile with: /target:library
public struct S
{
    public static S operator ++(S? s) { return new S(); }    // CS0559
    public static S operator --(S? s) { return new S(); }    // CS0559
}

public struct T
{
    // OK
    public static T operator --(T t) { return new T(); }
    public static T operator ++(T t) { return new T(); }

    public static T? operator --(T? t) { return new T(); }
    public static T? operator ++(T? t) { return new T(); }
}
```

Compiler Error CS0562

Error Message

The parameter of a unary operator must be the containing type

The method declaration for an operator overload must follow certain guidelines. For more information, see [Overloadable Operators](#) and [Operator Overloading Sample](#).

Example

The following sample generates CS0562:

```
// CS0562.cs
public class iii
{
    public static implicit operator int(iii x)
    {
        return 0;
    }

    public static implicit operator iii(int x)
    {
        return null;
    }

    public static iii operator +(int aa)    // CS0562
    // try the following line instead
    // public static iii operator +(iii aa)
    {
        return (iii)0;
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0563

Error Message

One of the parameters of a binary operator must be the containing type

The method declaration for an [operator overload](#) must follow certain guidelines. For more information, see [Overloadable Operators](#) and [Operator Overloading Sample](#).

Example

The following sample generates CS0563:

```
// CS0563.cs
public class iiii
{
    public static implicit operator int(iiii x)
    {
        return 0;
    }
    public static implicit operator iiii(int x)
    {
        return null;
    }
    public static int operator +(int aa, int bb)    // CS0563
    // Use the following line instead:
    // public static int operator +(int aa, iiii bb)
    {
        return 0;
    }
    public static void Main()
    {
    }
}
```

Compiler Error CS0564

Error Message

Overloaded shift operator must have the type of the first operand be the containing type, and the type of the second operand must be **int**.

You attempted to overload a shift operator (<< or >>) with incorrectly typed operands. The first operand must be the type and the second operand must be of the type **int**.

The following sample generates CS0564:

```
// CS0564.cs
using System;
class C
{
    public static int operator << (C c1, C c2) // CS0564
    // To correct, change second operand to int, like so:
    // public static int operator << (C c1, int c2)
    {
        return 0;
    }
    static void Main()
    {
    }
}
```

Compiler Error CS0567

Error Message

Interfaces cannot contain operators

Operators are not permitted in [interface](#) definitions.

The following sample generates CS0567:

```
// CS0567.cs
interface IA
{
    int operator +(int aa, int bb);    // CS0567
}

class Sample
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0568

Error Message

Structs cannot contain explicit parameterless constructors

Each [struct](#) already has a default constructor that initializes the object to zero. Therefore, the constructors that you can create for a struct must take one or more parameters.

The following sample generates CS0568:

```
// CS0568.cs
public struct ClassY
{
    public int field1;
    public ClassY(){} // CS0568, cannot have no param constructor
    // Try following instead:
    // public ClassY(int i)
    // {
    //     field1 = i;
    // }
}

public class ClassX
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0569

Error Message

'method2' : cannot override 'method1' because it is not supported by the language

This error occurs when you derive from a base class that was written in another language and when the compiler does not support the method that you are attempting to override.

Compiler Error CS0570

Error Message

'class' is not supported by the language

This error occurs when using imported metadata that was generated by another compiler. Your code attempted to use a class member that the compiler cannot process.

Example

The following C++ program uses an attribute, `RequiredAttributeAttribute`, which may not be consumed by other languages.

```
// CPP0570.cpp
// compile with: /clr /LD

using namespace System;
using namespace System::Runtime::CompilerServices;

namespace CS0570_Server {
    [RequiredAttributeAttribute(Int32::typeid)]
    public ref struct Scenario1 {
        int intVar;
    };

    public ref struct CS0570Class {
        Scenario1 ^ sc1_field;

        property virtual Scenario1 ^ sc1_prop {
            Scenario1 ^ get() { return sc1_field; }
        }

        Scenario1 ^ sc1_method() { return sc1_field; }
    };
};
```

The following sample generates CS0570.

```
// CS0570.cs
// compile with: /reference:CPP0570.dll
using System;
using CS0570_Server;

public class C {
    public static int Main() {
        CS0570Class r = new CS0570Class();
        r.sc1_field = null;    // CS0570
        object o = r.sc1_prop;    // CS0570
        r.sc1_method();    // CS0570
    }
}
```

Compiler Error CS0571

Error Message

'function' : cannot explicitly call operator or accessor

Certain operators have internal names. For example, **op_Increment** is the internal name of the ++ operator. You should not use or explicitly call such method names.

The following sample generates CS0571:

```
// CS0571.cs
public class MyClass
{
    public static MyClass operator ++ (MyClass c)
    {
        return null;
    }

    public static int prop
    {
        get
        {
            return 1;
        }
        set
        {
        }
    }
}

public static void Main()
{
    op_Increment(null);    // CS0571
    // use the increment operator as follows
    // MyClass x = new MyClass();
    // x++;

    set_prop(1);           // CS0571
    // try the following line instead
    // prop = 1;
}
}
```

Compiler Error CS0572

Error Message

'type' : cannot reference a type through an expression; try 'path_to_type' instead

An attempt was made to access a member of a class through an identifier, which is not permitted in this situation.

The following sample generates CS0572:

```
// CS0572.cs
using System;
class C
{
    public class Inner
    {
        public static int v = 9;
    }
}

class D : C
{
    public static void Main()
    {
        C cValue = new C();
        Console.WriteLine(cValue.Inner.v);    // CS0572
        // try the following line instead
        // Console.WriteLine(C.Inner.v);
    }
}
```

Compiler Error CS0573

Error Message

'field declaration' : cannot have instance field initializers in structs

You cannot initialize an instance field of a [struct](#). Fields of value types will be initialized to their default values, and reference-type fields will be initialized to **null**.

Example

The following sample generates CS0573:

```
// CS0573.cs
namespace x
{
    public class clx
    {
        public static void Main()
        {
        }
    }

    public struct cly
    {
        clx a = new clx(); // CS0573
        // clx a;           // OK
        int i = 7;          // CS0573
        // int i;           // OK
    }
}
```

Compiler Error CS0574

Error Message

Name of destructor must match name of class

The name of a destructor must be the class name preceded by a tilde (~).

The following sample generates CS0574:

```
// CS0574.cs
namespace x
{
    public class iiii
    {
        ~iiii()    // CS0574
        {
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0575

Error Message

Only class types can contain destructors

A [struct](#) cannot contain a destructor.

The following sample generates CS0575:

```
// CS0575.cs
namespace x
{
    public struct iiii
    {
        ~iiii()    // CS0575
        {
        }

        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0576

Error Message

Namespace 'namespace' contains a definition conflicting with alias 'identifier'

An attempt was made to use the same namespace twice.

Example

The following sample generates CS0576:

```
// CS0576.cs
using SysIO = System.IO;
public class SysIO
{
    public void MyMethod() {}
}

public class Test
{
    public static void Main()
    {
        SysIO.Stream s;    // CS0576
    }
}
```

Compiler Error CS0577

Error Message

The Conditional attribute is not valid on 'function' because it is a constructor, destructor, operator, or explicit interface implementation

Conditional cannot be applied to the specified methods.

For example, you cannot use some attributes on an explicit interface definition. The following sample generates CS0577:

```
// CS0577.cs
// compile with: /target:library
interface I
{
    void m();
}

public class MyClass : I
{
    [System.Diagnostics.Conditional("a")]    // CS0577
    void I.m() {}
}
```


Compiler Error CS0578

Error Message

The Conditional attribute is not valid on 'function' because its return type is not void

[ConditionalAttribute](#) cannot be applied to a method that has a return type other than **void**. The reason for this is that any other return type for a method may be needed by another part of your program.

Example

The following sample generates CS0578. To resolve this error, you must either delete **ConditionalAttribute**, or you must change the return value of the method to **void**.

```
// CS0578.cs
// compile with: /target:library
public class MyClass
{
    [System.Diagnostics.ConditionalAttribute("a")]    // CS0578
    public int TestMethod()
    {
        return 0;
    }
}
```

Compiler Error CS0579

Error Message

Duplicate 'attribute' attribute

It is not possible to specify the same attribute more than once unless the attribute specifies **AllowMultiple=true** in its [AttributeUsage](#).

Example

The following example generates CS0579.

```
// CS0579.cs
using System;
public class MyAttribute : Attribute
{
}

[AttributeUsage(AttributeTargets.All,AllowMultiple=true)]
public class MyAttribute2 : Attribute
{
}

public class z
{
    [MyAttribute, MyAttribute]      // CS0579
    public void zz()
    {
    }

    [MyAttribute2, MyAttribute2]   // OK
    public void zzz()
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0582

Error Message

The Conditional not valid on interface members

ConditionalAttribute is not valid on an interface member.

The following sample generates CS0582:

```
// CS0582.cs
// compile with: /target:library
using System.Diagnostics;
interface MyIFace
{
    [ConditionalAttribute("DEBUG")]    // CS0582
    void zz();
}
```

Compiler Error CS0583

Error Message

Internal Compiler Error ('address'): likely culprit is 'culprit'

Please create a bug report file with the '/bugreport' option, and submit the report to your assigned problem reporting contact.

Use the [/bugreport](#) compiler option to capture the problem.

Compiler Error CS0584

Error Message

Internal Compiler Error: stage 'stage' symbol 'symbol'

Try to determine if the compiler is failing due to its inability to parse unexpected syntax. If that is not the case, try [Getting Help from Microsoft Product Support Services \(Visual Studio\)](#).

Compiler Error CS0585

Error Message

Internal Compiler Error: stage 'stage'

Try to determine if the compiler is failing due to its inability to parse unexpected syntax. If that is not the case, try [Getting Help from Microsoft Product Support Services \(Visual Studio\)](#).

Compiler Error CS0586

Error Message

Internal Compiler Error: stage 'stage'

Try to determine if the compiler is failing due to its inability to parse unexpected syntax. If that is not the case, try [Getting Help from Microsoft Product Support Services \(Visual Studio\)](#).

Compiler Error CS0587

Error Message

Internal Compiler Error: stage 'stage'

Try to determine if the compiler is failing due to its inability to parse unexpected syntax. If this is not the case, try [Getting Help from Microsoft Product Support Services \(Visual Studio\)](#).

Compiler Error CS0588

Error Message

Internal Compiler Error: stage 'LEX'

Try to determine if the compiler is failing due to its inability to parse unexpected syntax. If this is not the case, try [Getting Help from Microsoft Product Support Services \(Visual Studio\)](#).

Compiler Error CS0589

Error Message

Internal Compiler Error: stage 'PARSE'

Try to determine if the compiler is failing due to its inability to parse unexpected syntax. If that is not the case, try [Getting Help from Microsoft Product Support Services \(Visual Studio\)](#).

Compiler Error CS0590

Error Message

User-defined operators cannot return void

The purpose of a user-defined operator is to return an object.

The following sample generates CS0590:

```
// CS0590.cs
namespace x
{
    public class a
    {
        public static void operator+(a A1, a A2)    // CS0590
        {
        }

        // try the following user-defined operator
        /*
        public static a operator+(a A1, a A2)
        {
            return A2;
        }
        */

        public static int Main()
        {
            return 1;
        }
    }
}
```

Compiler Error CS0591

Error Message

Invalid value for argument to 'attribute' attribute

An attribute was passed either an invalid argument or two mutually exclusive arguments.

Example

The following sample generates CS0591:

```
// CS0591.cs
using System;

[AttributeUsage(0)]    // CS0591
class I: Attribute
{
}

public class a
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0592

Error Message

Attribute 'attribute' is not valid on this declaration type. It is valid on 'type' declarations only.

An attribute was applied to a declaration for which it was not intended.

Example

The following sample generates CS0592:

```
// CS0592.cs
using System;

[AttributeUsage(AttributeTargets.Interface)]
public class MyAttribute : Attribute
{
}

[MyAttribute]
public class A // CS0592, MyAttribute is not valid for a class
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0594

Error Message

Floating-point constant is outside the range of type 'type'

A value was assigned to a floating-point variable that is too large for the variables of this data type. See [Integral Types Table](#) for information about the range of values allowed in data types.

The following sample generates CS0594:

```
// CS0594.cs
namespace MyNamespace
{
    public class MyClass
    {
        public static void Main()
        {
            float f = 6.777777777777E400;    // CS0594, value too large
        }
    }
}
```

Compiler Error CS0596

Error Message

The Guid attribute must be specified with the ComImport attribute

The **Guid** attribute must be present when using the **ComImport** attribute.

The following sample generates CS0596:

```
// CS0596.cs
using System.Runtime.InteropServices;

namespace x
{
    [ComImport]    // CS0596
    // try the following line to resolve this CS0596
    // [ComImport, Guid("00000000-0000-0000-0000-000000000001")]
    public class a
    {
    }

    public class b
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS0599

Error Message

Invalid value for named attribute argument 'argument'

An invalid argument was passed to an attribute.

Compiler Error CS0601

Error Message

The `DllImport` attribute must be specified on a method marked 'static' and 'extern'

The **`DllImport`** attribute was used on a method that did not have the correct access keywords.

The following sample generates CS0601:

```
// CS0601.cs
using System.Runtime.InteropServices;
using System.Text;

public class C
{
    [DllImport("KERNEL32.DLL")]
    extern int GetCurDirectory(int bufSize, StringBuilder buf);    // CS0601
    // Try the following line instead:
    // static extern int GetCurDirectory(int bufSize, StringBuilder buf);
}

public class MainClass
{
    public static void Main ()
    {
    }
}
```

Compiler Error CS0609

Error Message

Cannot set the IndexerName attribute on an indexer marked override

The name attribute (**IndexerNameAttribute**) cannot be applied to an indexed property that is an override. For more information, see [Indexers](#).

The following sample generates CS0609:

```
// CS0609.cs
using System;
using System.Runtime.CompilerServices;

public class idx
{
    public virtual int this[int iPropIndex]
    {
        get
        {
            return 0;
        }
        set
        {
        }
    }
}

public class MonthDays : idx
{
    [IndexerName("MonthInfoIndexer")] // CS0609, delete to resolve this CS0609
    public override int this[int iPropIndex]
    {
        get
        {
            return 0;
        }
        set
        {
        }
    }
}

public class test
{
    public static void Main( string[] args )
    {
    }
}
```

Compiler Error CS0610

Error Message

Field or property cannot be of type 'type'

There are some types that cannot be used as fields or properties. These types include **System.ArgIterator** and **System.TypedReference**.

The following sample generates CS0610 as a result of using **System.TypedReference** as a field:

```
// CS0610.cs
public class MainClass
{
    System.TypedReference i;    // CS0610
    public static void Main ()
    {
    }

    public static void Test(System.TypedReference i)    // OK
    {
    }
}
```

Compiler Error CS0611

Error Message

Array elements cannot be of type 'type'

There are some types that cannot be used as the type of an array. These types include **System.TypedReference** and **System.ArgIterator**.

The following sample generates CS0611 as a result of using **System.TypedReference** as an array element:

```
// CS0611.cs
public class a
{
    public static void Main()
    {
        System.TypedReference[] ao = new System.TypedReference [10];    // CS0611
        // try the following line instead
        // int[] ao = new int[10];
    }
}
```

Compiler Error CS0616

Error Message

'class' is not an attribute class

An attempt was made to use a non-attribute class in an attribute block. All the attribute types need to be inherited from [System.Attribute](#).

Example

The following sample generates CS0616.

```
// CS0616.cs
// compile with: /target:library
[CMYClass(i = 5)]    // CS0616
public class CMYClass {}
```

The following sample shows how you might define an attribute:

```
// CreateAttrib.cs
// compile with: /target:library
using System;

[AttributeUsage(AttributeTargets.Class|AttributeTargets.Interface)]
public class MyAttr : Attribute
{
    public int Name = 0;
    public int Count = 0;

    public MyAttr (int iCount, int sName)
    {
        Count = iCount;
        Name = sName;
    }
}

[MyAttr(5, 50)]
class Class1 {}

[MyAttr(6, 60)]
interface Interface1 {}
```

Compiler Error CS0617

Error Message

'reference' is not a valid named attribute argument. Named attribute arguments must be fields which are not readonly, static, or const, or read-write properties which are public and not static.

An attempt was made to access a [private](#) member of an attribute class.

Example

The following sample generates CS0617.

```
// CS0617.cs
using System;

[AttributeUsage(AttributeTargets.Struct |
                AttributeTargets.Class |
                AttributeTargets.Interface)]
public class MyClass : Attribute
{
    public int Name;

    public MyClass (int sName)
    {
        Name = sName;
        Bad = -1;
        Bad2 = -1;
    }

    public readonly int Bad;
    public int Bad2;
}

[MyClass(5, Bad=0)] class Class1 {}    // CS0617
[MyClass(5, Bad2=0)] class Class2 {}
```

Compiler Error CS0619

Error Message

'member' is obsolete: 'text'

A class member was marked with the [Obsolete](#) attribute, such that an error will be issued when the class member is referenced.

The following sample generates CS0619:

```
// CS0619.cs
using System;

public class C
{
    [Obsolete("Use newMethod instead", true)]    // generates an error on use
    public static void m()
    {
    }

    // this is the method you should be using
    public static void newMethod()
    {
    }
}

class MyClass
{
    public static void Main()
    {
        C.m();    // CS0619
    }
}
```

Compiler Error CS0620

Error Message

Indexers can't have void type

The return type of an [indexer](#) cannot be **void**. An indexer must return a value.

The following sample generates CS0620:

```
// CS0620.cs
class MyClass
{
    public static void Main()
    {
        MyClass test = new MyClass();
        System.Console.WriteLine(test[2]);
    }

    void this [int intI]    // CS0620, return type cannot be void
    {
        get
        {
            // will need to return some value
        }
    }
}
```


Compiler Error CS0621

Error Message

'member' : virtual or abstract members cannot be private

Private **virtual** or **abstract** members are not allowed.

The following sample generates CS0621:

```
// CS0621.cs
abstract class MyClass
{
    private virtual void DoNothing1()    // CS0621
    {
    }

    private abstract void DoNothing2();  // CS0621

    public static void Main()
    {
    }
}
```

Compiler Error CS0622

Error Message

Can only use array initializer expressions to assign to array types. Try using a new expression instead.

Syntax that is appropriate to initialize an array was used in the declaration of a non-array.

Example

The following sample generates CS0622:

```
// CS0622.cs
using System;

public class Test
{
    public static void Main ()
    {
        Test t = { new Test() };    // CS0622
        // Try the following instead:
        // Test[] t = { new Test() };
    }
}
```

Compiler Error CS0623

Error Message

Array initializers can only be used in a variable or field initializer. Try using a new expression instead.

An array was initialized incorrectly.

Compiler Error CS0625

Error Message

'field': instance field types marked with StructLayout(LayoutKind.Explicit) must have a FieldOffset attribute

When a struct is marked with an explicit **StructLayout** attribute, all fields in the struct must have the [FieldOffset](#) attribute. For more information, see [StructLayoutAttribute Class](#).

The following sample generates CS0625:

```
// CS0625.cs
// compile with: /target:library
using System;
using System.Runtime.InteropServices;

[StructLayout(LayoutKind.Explicit)]
struct A
{
    public int i;    // CS0625 not static; an instance field
}

// OK
[StructLayout(LayoutKind.Explicit)]
struct B
{
    [FieldOffset(5)]
    public int i;
}
```

Compiler Error CS0629

Error Message

Conditional member 'member' cannot implement interface member 'base class member' in type 'Type Name'

The [Conditional](#) attribute cannot be used on the implementation of an interface.

The following sample generates CS0629:

```
// CS0629.cs
interface MyInterface
{
    void MyMethod();
}

public class MyClass : MyInterface
{
    [System.Diagnostics.Conditional("debug")]
    public void MyMethod()    // CS0629, remove the Conditional attribute
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0631

Error Message

ref and out are not valid in this context

The declaration for an [indexer](#) cannot include the use of [ref](#) or [out](#) parameters.

Example

The following sample generates CS0631:

```
// CS0631.cs
public class MyClass
{
    public int this[ref int i]    // CS0631
    // try the following line instead
    // public int this[int i]
    {
        get
        {
            return 0;
        }
    }
}

public class MyClass2
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0633

Error Message

The argument to the 'attribute' attribute must be a valid identifier

Any argument that you pass to the [ConditionalAttribute](#) or [IndexerNameAttribute](#) attributes must be a valid identifier. This means that it may not contain characters such as "+" that are illegal when they occur in identifiers.

Example

This example illustrates CS0633 using the **ConditionalAttribute**. The following sample generates CS0633.

```
// CS0633a.cs
#define DEBUG
using System.Diagnostics;
public class Test
{
    [Conditional("DEB+UG")]    // CS0633
    // try the following line instead
    // [Conditional("DEBUG")]
    public static void Main() { }
}
```

This example illustrates CS0633 using the **IndexerNameAttribute**.

```
// CS0633b.cs
// compile with: /target:module
#define DEBUG
using System.Runtime.CompilerServices;
public class Test
{
    [IndexerName("Invalid+Identifier")]    // CS0633
    // try the following line instead
    // [IndexerName("DEBUG")]
    public int this[int i]
    {
        get { return i; }
    }
}
```

Compiler Error CS0634

Error Message

'arg' : argument only valid for marshal of type System.Interop.UnmanagedType.CustomMarshaller

One of the arguments passed to the **Marshal** attribute can only be used when the marshal format is **System.InteropServices.UnmanagedType.CustomMarshaller**.

Compiler Error CS0635

Error Message

'attribute' : System.Interop.UnmanagedType.CustomMarshaller requires named arguments ComType and Marshal

The **ComType** and **Marshal** arguments must be specified when the marshal format is **System.InteropServices.UnmanagedType.CustomMarshaller**.

Compiler Error CS0636

Error Message

The `FieldOffset` attribute can only be placed on members of types marked with the `StructLayout(LayoutKind.Explicit)`

You must use the **`StructLayout(LayoutKind.Explicit)`** attribute on the struct declaration, if it contains any members marked with the **`FieldOffset`** attribute. For more information, see [FieldOffset](#).

The following sample generates CS0636:

```
// CS0636.cs
using System;
using System.Runtime.InteropServices;

// To resolve the error, uncomment the following line:
// [StructLayout(LayoutKind.Explicit)]
struct Worksheet
{
    [FieldOffset(4)]public int i;    // CS0636
}

public class MainClass
{
    public static void Main ()
    {
    }
}
```

Compiler Error CS0637

Error Message

The FieldOffset attribute is not allowed on static or const fields

The [FieldOffset](#) attribute cannot be used on fields marked with [static](#) or [const](#).

The following sample generates CS0637:

```
// CS0637.cs
using System;
using System.Runtime.InteropServices;

[StructLayout(LayoutKind.Explicit)]
public class MainClass
{
    [FieldOffset(3)]    // CS0637
    public static int i;
    public static void Main ()
    {
    }
}
```

Compiler Error CS0641

Error Message

'attribute' : attribute is only valid on classes derived from System.Attribute

An attribute was used that can only be used on a class that derives from **System.Attribute**.

The following sample generates CS0641:

```
// CS0641.cs
using System;

[AttributeUsage(AttributeTargets.All)]
public class NonAttrClass    // CS0641
// try the following line instead
// public class NonAttrClass : Attribute
{
}

class MyClass
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0643

Error Message

'arg' duplicate named attribute argument

A parameter, *arg*, on a user-defined attribute was specified twice. For more information, see [Attributes \(C# Programming Guide\)](#).

Example

The following sample generates CS0643:

```
// CS0643.cs
using System;
using System.Runtime.InteropServices;

[AttributeUsage(AttributeTargets.Class)]
public class MyAttribute : Attribute
{
    public MyAttribute()
    {
    }

    public int x;
}

[MyAttribute(x = 5, x = 6)]    // CS0643, error setting x twice
// try the following line instead
// [MyAttribute(x = 5)]
class MyClass
{
}

public class MainClass
{
    public static void Main ()
    {
    }
}
```

Compiler Error CS0644

Error Message

'class1' cannot inherit from special class 'class2'

Classes cannot explicitly inherit from any of the following base classes:

- **System.Enum**
- **System.ValueType**
- **System.Delegate**
- **System.Array**

These are used as implicit base classes by the compiler. For example, **System.ValueType** is the implicit base class of structs.

The following sample generates CS0644:

```
// CS0644.cs
class MyClass : System.ValueType    // CS0644
{
}
```

Compiler Error CS0645

Error Message

Identifier too long

A class name or other identifier can be no longer than 512 characters.

Compiler Error CS0646

Error Message

Cannot specify the DefaultMember attribute on a type containing an indexer

If a class or other type specifies **System.Reflection.DefaultMemberAttribute**, it cannot contain an indexer. For more information, see [Properties](#).

The following sample generates CS0646:

```
// CS0646.cs
// compile with: /target:library
[System.Reflection.DefaultMemberAttribute("x")] // CS0646
class MyClass
{
    public int this[int index] // an indexer
    {
        get
        {
            return 0;
        }
    }

    public int x = 0;
}

// OK
[System.Reflection.DefaultMemberAttribute("x")]
class MyClass2
{
    public int prop
    {
        get
        {
            return 0;
        }
    }

    public int x = 0;
}

class MyClass3
{
    public int this[int index] // an indexer
    {
        get
        {
            return 0;
        }
    }

    public int x = 0;
}
```


Compiler Error CS0647

Error Message

Error emitting 'attribute' attribute -- 'reason'

The following sample generates CS0647:

```
// CS0647.cs
using System.Runtime.InteropServices;

[Guid("z")] // CS0647, incorrect uuid format.
// try the following line instead
// [Guid("00000000-0000-0000-0000-000000000001")]
public class MyClass
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0648

Error Message

'type' is a type not supported by the language

Metadata that was generated from another language, possibly C++, contained a type that was not marked as managed. Information about this type is included in the metadata, but the type is not available to programs written in C#.

Compiler Error CS0650

Error Message

Syntax error, bad array declarator. To declare a managed array the rank specifier precedes the variable's identifier. To declare a fixed size buffer field, use the fixed keyword before the field type.

An array was declared incorrectly. Be aware that the syntax for a fixed size buffer is different from that of an array.

Example

The following sample generates CS0650.

```
// CS0650.cs
public class MyClass
{
    public static void Main()
    {
        int myarray[2];    // CS0650

        // OK
        int[] myarray2 = new int[2] {1,2};
        myarray2[0] = 0;
    }
}
```

Compiler Error CS0653

Error Message

Cannot apply attribute class 'class' because it is abstract

An [abstract](#) custom attribute class cannot be used as an attribute.

The following sample generates CS0653:

```
// CS0653.cs
using System;

public abstract class MyAttribute : Attribute
{
}

public class My2Attribute : MyAttribute
{
}

[My]    // CS0653
// try the following line instead
// [My2]
class MyClass
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0655

Error Message

'parameter' is not a valid named attribute argument because it is not a valid attribute parameter type

See [Attributes \(C# Programming Guide\)](#) for a discussion of valid parameter types for an attribute.

Example

The following sample generates CS0655:

```
// CS0655.cs
using System;

class MyAttribute : Attribute
{
    // decimal is not valid attribute parameter type
    public decimal d = 0;
    public int e = 0;
}

[My(d = 0)]    // CS0655
// Try the following line instead:
// [My(e = 0)]
class C
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0656

Error Message

Missing compiler required member 'object.member'

One of the following problems exists:

- Your installation of the common language runtime is corrupt.
- You have a reference to an assembly that defines a type that is also found in the common language runtime. However, your assembly's type is not defined the way the C# compiler expects.

Check your references to ensure that you are using the correct version of the common language runtime.

Compiler Warning (level 1) CS0657

Error Message

'attribute modifier' is not a valid attribute location for this declaration. Valid attribute locations for this declaration are 'location'

The compiler found an attribute modifier in an invalid location. See [Attribute Targets](#) for more information.

The following sample generates CS0657:

```
// CS0657.cs
// compile with: /target:library
public class TestAttribute : System.Attribute {}
[return: Test]    // CS0657 return not valid on a class
class Class1 {}
```

Compiler Error CS0662

Error Message

'method' cannot specify only Out attribute on a ref parameter. Use both In and Out attributes, or neither.

An interface method has a parameter that uses [ref](#) with just the [Out](#) attribute. A **ref** parameter that uses the **Out** attribute must also use the [In](#) attribute.

The following sample generates CS0662:

```
// CS0662.cs
using System.Runtime.InteropServices;

interface I
{
    void method([Out] ref int i);    // CS0662
    // try one of the following lines instead
    // void method(ref int i);
    // void method([Out, In]ref int i);
}

class test
{
    public static void Main()
    {
    }
}
```


Compiler Error CS0663

Error Message

Cannot define overloaded methods that differ only on ref and out.

Methods that differ only on their use of [ref](#) and [out](#) on a parameter are not allowed.

The following sample generates CS0663:

```
// CS0663.cs
class TestClass
{
    public static void Main()
    {
    }

    public void Test(ref int i)
    {
    }

    public void Test(out int i)    // CS0663
    {
    }
}
```

Compiler Error CS0664

Error Message

Literal of type double cannot be implicitly converted to type 'type'; use an 'suffix' suffix to create a literal of this type

An assignment could not be completed; use a suffix to correct the instruction. The documentation for each type identifies the corresponding suffix for the type.

The following sample generates CS0664:

```
// CS0664.cs
class M
{
    static void Main()
    {
        decimal m = 1.0;    // CS0664
        // try the following line instead
        // decimal m = 1.0M;
        System.Console.WriteLine(m);
    }
}
```

Compiler Error CS0666

Error Message

'member' : new protected member declared in struct

A [struct](#) cannot be **abstract** and is always implicitly **sealed**. For more information, see [Inheritance \(C# Programming Guide\)](#).

Example

The following sample generates CS0666:

```
// CS0666.cs
class M
{
    static void Main()
    {
    }
}

struct S
{
    protected int x;    // CS0666
}
```

Compiler Error CS0667

Error Message

The feature 'invalid feature' is deprecated. Please use 'valid feature' instead'.

The feature you are attempting to use is now deprecated. Update your code to use the valid feature instead.

Compiler Error CS0668

Error Message

Two indexers have different names; the `IndexerName` attribute must be used with the same name on every indexer within a type

The values passed to the **`IndexerName`** attribute must be the same for all indexers in a type. For more information on the **`IndexerName`** attribute, see [IndexerNameAttribute Class](#).

The following sample generates CS0668:

```
// CS0668.cs
using System;
using System.Runtime.CompilerServices;

class IndexerClass
{
    [IndexerName("IName1")]
    public int this [int index]    // indexer declaration
    {
        get
        {
            return index;
        }
        set
        {
        }
    }

    [IndexerName("IName2")]
    public int this [string s]    // CS0668, change IName2 to IName1
    {
        get
        {
            return int.Parse(s);
        }
        set
        {
        }
    }

    void Main()
    {
    }
}
```

Compiler Error CS0669

Error Message

A class with the `ComImport` attribute cannot have a user-defined constructor

The COM interop layer in the common language runtime supplies the constructor for [ComImport](#) classes. Consequently, a COM object can be used as a managed object in the runtime.

The following sample generates CS0669:

```
// CS0669.cs
using System.Runtime.InteropServices;
[ComImport, Guid("00000000-0000-0000-0000-000000000001")]
class TestClass
{
    TestClass()    // CS0669, delete constructor to resolve
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0670

Error Message

Field cannot have void type

A field was declared to be of type [void](#).

The following sample generates CS0670:

```
// CS0670.cs
class C
{
    void f;    // CS0670
    // try the following line instead
    // public int f;

    public static void Main()
    {
        C myc = new C();
        myc.f = 0;
    }
}
```

Compiler Error CS0673

Error Message

System.Void cannot be used from C# -- use typeof(void) to get the void type object.

System.Void cannot be used in C#.

The following sample generates CS0673:

```
// CS0673.cs
class MyClass
{
    public static void Main()
    {
        System.Type t = typeof(System.Void);    // CS0673
        // try the following line instead
        // System.Type t = typeof(void);
    }
}
```


Compiler Error CS0674

Error Message

Do not use 'System.ParamArrayAttribute'. Use the 'params' keyword instead.

The C# compiler does not allow the use of [System.ParamArrayAttribute](#); use [params](#) instead.

The following sample generates CS0674:

```
// CS0674.cs
using System;
public class MyClass
{
    public static void UseParams([ParamArray] int[] list)    // CS0674
    // try the following line instead
    // public static void UseParams(params int[] list)
    {
        for ( int i = 0 ; i < list.Length ; i++ )
            Console.WriteLine(list[i]);
        Console.WriteLine();
    }

    public static void Main()
    {
        UseParams(1, 2, 3);
    }
}
```

Compiler Error CS0677

Error Message

'variable': a volatile field can not be of the type 'type'

Fields declared with the **volatile** keyword must be one of the following types:

- Any reference type
- Any pointer type (in an **unsafe** context)
- The types **sbyte**, **byte**, **short**, **ushort**, **int**, **uint**, **char**, **float**, **bool**
- Enum types based on any of the above types

The following sample generates CS0677:

```
// CS0677.cs
class TestClass
{
    private volatile long i;    // CS0677

    public static void Main()
    {
    }
}
```

Compiler Error CS0678

Error Message

'variable': a field can not be both volatile and readonly

Use of the [volatile](#) and [readonly](#) keywords is mutually exclusive.

The following sample generates CS0678:

```
// CS0678.cs
using System;

class TestClass
{
    private readonly volatile int i;    // CS0678
    // try the following line instead
    // private volatile int i;

    public static void Main()
    {
    }
}
```

Compiler Error CS0681

Error Message

The modifier 'abstract' is not valid on fields. Try using a property instead

You cannot make a field abstract. You can, however, have an abstract property that accesses the field.

Example

The following sample generates CS0681:

```
// CS0681.cs
// compile with: /target:library
abstract class C
{
    abstract int num; // CS0681
}
```

Try the following code instead:

```
// CS0681b.cs
// compile with: /target:library
abstract class C
{
    public abstract int num
    {
        get;
        set;
    }
}
```

Compiler Error CS0682

Error Message

'type1' cannot implement 'type2' because it is not supported by the language

This error occurs when you try to implement an interface written in another language and the compiler does not support the interface.

Compiler Error CS0683

Error Message

""explicitmethod' explicit method implementation cannot implement 'method' because it is an accessor

The following sample generates CS0683:

```
// CS0683.cs
interface IExample
{
    int Test { get; }
}

class CExample : IExample
{
    int IExample.get_Test() { return 0; } // CS0683
    int IExample.Test { get{ return 0; } } // correct
}
```

Compiler Error CS0685

Error Message

Conditional member 'member' cannot have an out parameter

When using the [ConditionalAttribute](#) attribute on a method, that method may not have an out parameter. This is because the value of the variable used for the out parameter would not be defined in the case that the method call is compiled to nothing. To avoid this error, remove the out parameter from the conditional method declaration, or don't use the Conditional Attribute.

Example

The following sample generates CS0685:

```
// CS0685.cs
using System.Diagnostics;

class C
{
    [Conditional("DEBUG")]
    void trace(out int i) // CS0685
    {
        i = 1;
    }
}
```

Compiler Error CS0686

Error Message

Accessor 'accessor' cannot implement interface member 'member' for type 'type'. Use an explicit interface implementation.

Suggested: This error can occur when implementing an interface that contains method names which conflict with the auto-generated methods associated with a property or event. The get/set methods for properties are generated as `get_property` and `set_property`, and the add/remove methods for events are generated as `add_event` and `remove_event`. If an interface contains either of these methods, a conflict occurs. To avoid this error, implement the methods using an explicit interface implementation. To do this, specify the function as:

```
Interface.get_property() { /* */ }
Interface.set_property() { /* */ }
```

Example

The following sample generates CS0686:

```
// CS0686.cs
interface I
{
    int get_P();
}

class C : I
{
    public int P
    {
        get { return 1; } // CS0686
    }
}
// But the following is valid:
class D : I
{
    int I.get_P() { return 1; }
    public static void Main() {}
}
```

This error can also occur when declaring events. The event construct automatically generates the `add_event` and `remove_event` methods, which could conflict with the methods of the same name in an interface, as in the following sample:

```
// CS0686b.cs
using System;

interface I
{
    void add_OnMyEvent(EventHandler e);
}

class C : I
{
    public event EventHandler OnMyEvent
    {
        add { } // CS0686
        remove { }
    }
}

// Correct (using explicit interface implementation):
class D : I
{
    void I.add_OnMyEvent(EventHandler e) {}
}
```



```
} public static void Main() {}
```

Compiler Error CS0687

Error Message

The namespace alias qualifier '::' always resolves to a type or namespace so is illegal here. Consider using '.' instead.

This error occurs if you used something which the parser interpreted as a type in an unexpected place. A type or namespace name is valid only in a member access expression, using the member access (.) operator. This could occur if you used the global scope operator (::) in another context.

Example

The following sample generates CS0687:

```
// CS0687.cs

using M = Test;
using System;

public class Test
{
    public static int x = 77;

    public static void Main()
    {
        Console.WriteLine(M::x); // CS0687
        // To resolve use the following line instead:
        // Console.WriteLine(M.x);
    }
}
```

Compiler Error CS0689

Error Message

Cannot derive from 'identifier' because it is a type parameter

Base classes or interfaces for generic classes cannot be specified by a type parameter. Derive from a specific class or interface, or a specific generic class instead, or include the unknown type as a member.

The following sample generates CS0689:

```
// CS0689.cs
class A<T> : T    // CS0689
{
}
```

Compiler Error CS0690

Error Message

Input file 'file' contains invalid metadata.

You are able to open the metadata file, but due to some localization problem, the metadata was corrupted. This is similar to error [CS0009](#), except that you are able to open the metadata file.

A PE (Process Executable) is an executable file.

Compiler Error CS0692

Error Message

Duplicate type parameter 'identifier'

The same name may not be used more than once in a type parameter list. Rename or remove the duplicate type parameter(s).

Example

The following sample generates CS0692:

```
// CS0692.cs
// compile with: /target:library
class C <T, A, T>    // CS0692
{
}

class D <T, T>      // CS0692
{
}
```

Compiler Error CS0694

Error Message

Type parameter 'identifier' has the same name as the containing type, or method

You must use a different name for the type parameter since the type parameter's name cannot be identical to the type or method name that contains the type parameter.

Example

The following sample generates CS0694.

```
// CS0694.cs
// compile with: /target:library
class C<C> {}    // CS0694
```

In addition to the above case involving a generic class, this error may occur with a method:

```
// CS0694_2.cs
// compile with: /target:library
class A
{
    public void F<F>(F arg);    // CS0694
}
```

Compiler Error CS0695

Error Message

'generic type' cannot implement both 'generic interface' and 'generic interface' because they may unify for some type parameter substitutions

This error occurs when a generic class implements more than one parameterization of the same generic interface, and there exists a type parameter substitution which would make the two interfaces identical. To avoid this error, implement only one of the interfaces, or change the type parameters to avoid the conflict.

The following sample generates CS0695:

```
// CS0695.cs
// compile with: /target:library

interface I<T>
{
}

class G<T1, T2> : I<T1>, I<T2> // CS0695
{
}
```

Compiler Error CS0698

Error Message

A generic type cannot derive from 'class' because it is an attribute class

Any class that derives from an attribute class is an attribute. Attributes are not allowed to be generic types.

The following sample generates CS0698:

```
// CS0698.cs
class C<T> : System.Attribute // CS0698
{
}
```


Compiler Error CS0699

Error Message

'generic' does not define type parameter 'identifier'

A type parameter was used in a generic definition that was not found in the declaration list of the type parameters for that generic. This can happen if the name used for the type parameter was inconsistent.

The following sample generates CS0699:

```
// CS0699.cs
class C<T> where U : I    // CS0699 - U is not a valid type parameter
{
}
```

Compiler Error CS0701

Error Message

'identifier' is not a valid constraint. A type used as a constraint must be an interface, a non-sealed class or a type parameter.

This error occurs if a sealed type is used as a constraint. To resolve this error, use only non-sealed types as constraints.

Example

The following sample generates CS0701.

```
// CS0701.cs
// compile with: /target:library
class C<T> where T : System.String {} // CS0701
class D<T> where T : System.Attribute {} // OK
```

Compiler Error CS0702

Error Message

Constraint cannot be special class 'identifier'

The following types may not be used as constraints: `System.Array`, `System.Delegate`, `System.Enum`, or `System.ValueType`.

Example

The following sample generates CS0702:

```
// CS0702.cs
class C<T> where T : System.Array // CS0702
{
}
```

Compiler Error CS0703

Error Message

Inconsistent accessibility: constraint type 'identifier' is less accessible than 'identifier'

A constraint may not force the generic parameter to be less accessible than the generic class itself. In the following example, while the generic class `C<T>` is declared public, the constraint attempts to force `T` to implement an internal interface. Even if this were allowed, only clients with internal access would be able to create the parameter for the class, so in effect, the class could be used only by clients with internal access.

To get rid of this error, make sure the access level of the generic class is not less restrictive than any classes or interfaces that appear in the bounds.

The following sample generates CS0703:

```
// CS0703.cs
internal interface I {}
public class C<T> where T : I // CS0703 - I is internal; C<T> is public
{
}
```

Compiler Error CS0704

Error Message

Cannot do member lookup in 'type' because it is a type parameter

An inner type cannot be specified through a type parameter. Try using the desired type explicitly.

Example

The following sample generates CS0704:

```
// CS0704.cs
class B
{
    public class I { }
}

class C<T> where T : B
{
    T.I f; // CS0704 - member lookup on type parameter
    // Try this instead:
    // B.I f;
}

class CMain
{
    public static void Main() {}
}
```

Compiler Error CS0706

Error Message

Invalid constraint type. A type used as a constraint must be an interface, a non-sealed class or a type parameter.

This error occurs when an invalid construct is used in a constraint clause. To avoid this error, use an interface or non-sealed class instead of the construct that caused the error.

Example

The following sample generates CS0706.

```
// CS0706.cs
// compile with: /target:library
class A {}
class C<T> where T : int[] {} // CS0706
class D<T> where T : A {} // OK
```

Compiler Error CS0708

Error Message

'field': cannot declare instance members in a static class

This error occurs if you declare a non-static member in a class that is declared static. It is not possible to create instances of static classes, so instance variables would not be meaningful. The **static** keyword should be applied to all members of static classes.

The following sample generates CS0708:

```
// CS0708.cs
// compile with: /target:library
public static class C
{
    int i; // CS0708
    static int j; // OK
}
```

Compiler Error CS0709

Error Message

'derived class': cannot derive from static class 'base class'

A static class cannot be instantiated or derived from. That is, a static class cannot be a base class of any other class.

Example

The following sample generates CS0709.

```
// CS0709.cs
// compile with: /target:library
public static class Base {}
public class Derived : Base {}    // CS0709
```


Compiler Error CS0710

Error Message

Static classes cannot have instance constructors

A static class cannot be instantiated, hence it has no need for constructors. To avoid this error, remove any constructors from static classes, or if you really want to construct instances, make the class non-static.

The following sample generates CS0710:

```
// CS0710.cs
public static class C
{
    public C() // CS0710
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0711

Error Message

Static classes cannot contain destructors

A static class cannot be instantiated, hence it has no need for constructors or destructors. To avoid this error, remove any destructors from static classes, or, if you really want to construct and destroy instances, make the class non-static.

The following sample generates CS0711:

```
// CS0711.cs
public static class C
{
    ~C() // CS0711
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0712

Error Message

Cannot create an instance of the static class 'static class'

It is not possible to create instances of static classes. Static classes are designed to contain static fields and methods, but may not be instantiated.

Example

The following sample generates CS0712:

```
// CS0712.cs
public static class SC
{
}

public class CMain
{
    public static void Main()
    {
        SC sc = new SC(); // CS0712
    }
}
```

Compiler Error CS0713

Error Message

Static class 'static type' cannot derive from type 'type'. Static classes must derive from object.

If this were allowed, the static class would inherit methods and non-static members from the base class, so it would not be static. Therefore, it is not allowed.

The following sample generates CS0713:

```
// CS0713.cs
public class Base
{
}

public static class Derived : Base // CS0713
{
}

public class CMain
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0714

Error Message

'static type' : static classes cannot implement interfaces

Interfaces may define non-static methods on objects and hence may not be implemented by static classes. To resolve this error, make sure your class does not attempt to implement any interfaces.

Example

The following sample generates CS0714:

```
// CS0714.cs
interface I
{
}

public static class C : I // CS0714
{
    public static void Main()
    {
    }
}
```

Compiler Error CS0715

Error Message

'static class' : static classes cannot contain user defined operators

User defined operators operate on instances of classes. Static classes cannot be instantiated, so it is not possible to create instances for operators to act upon. Hence, user defined operators are not allowed for static classes.

The following sample generates CS0715:

```
// CS0715.cs
public static class C
{
    public static C operator+(C c) // CS0715
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0716

Error Message

Cannot convert to static type 'type'

This error occurs if your code uses a cast to convert to a static type. Since it is not possible for an object to be an instance of a static type, casting to a static type can never be a meaningful cast.

Example

The following sample generates CS0716:

```
// CS0716.cs

public static class SC
{
    static void F() { }
}

public class Test
{
    public static void Main()
    {
        object o = new object();
        System.Console.WriteLine((SC)o); // CS0716
    }
}
```

Compiler Error CS0717

Error Message

'static class': static classes cannot be used as constraints

Static classes cannot be extended as they only contain static members and not instance members. Because they cannot be extended, this makes them useless as type parameters and constraints, as no type can exist that is a specialization of a static class.

Example

The following sample generates CS0717:

```
// CS0717.cs

public static class SC
{
    public static void F()
    {
    }
}

public class G<T> where T : SC // CS0717
{
    public static void Main()
    {
    }
}
```


Compiler Error CS0718

Error Message

'type': static types cannot be used as generic arguments

Because a static type cannot be instantiated, it cannot be used as a generic argument. To resolve this error, remove the static type from the generic argument.

Example

The following sample generates CS0718:

```
// CS0718.cs
public static class SC
{
    public static void F()
    {
    }
}

public class G<T>
{
}

public class CMain
{
    public static void Main()
    {
        G<SC> gsc = new G<SC>(); // CS0718
    }
}
```

Compiler Error CS0719

Error Message

'type': array elements cannot be of static type

An array of static type does not make sense since array elements are instances, but it is not possible to create instances of static types.

The following sample generates CS0719:

```
// CS0719.cs
public static class SC
{
    public static void F()
    {
    }
}

public class CMain
{
    public static void Main()
    {
        SC[] sca = new SC[10]; // CS0719
    }
}
```

Compiler Error CS0720

Error Message

'static class': cannot declare indexers in a static class

Indexers are not meaningful in static classes, since they can only be used with instances, and it is not possible to create instances of a static type.

Example

The following sample generates CS0720:

```
// CS0720.cs

public static class Test
{
    public int this[int index] // CS0720
    {
        get { return 1; }
        set {}
    }

    static void Main() {}
}
```

Compiler Error CS0721

Error Message

'type': static types cannot be used as parameters

A static type is not meaningful as a parameter. Since no instances of static types may be created, no instance could ever be passed as a parameter.

The following sample generates CS0721:

```
// CS0721.cs
public static class SC
{
}

public class CMain
{
    public void F(SC sc) // CS0721
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0722

Error Message

'type': static types cannot be used as return types

A static type as a return type is not meaningful since instances of static types cannot be created.

The following sample generates CS0722:

```
// CS0722.cs
public static class SC
{
}

public class CMain
{
    public SC F() // CS0722
    {
        return null;
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS0723

Error Message

Cannot declare variable of static type 'type'

Instances of static types cannot be created.

The following sample generates CS0723:

```
// CS0723.cs
public static class SC
{
}

public class CMain
{
    public static void Main()
    {
        SC sc = null;  // CS0723
    }
}
```

Compiler Error CS0724

Error Message

A throw statement with no arguments is not allowed inside of a finally clause nested inside of the innermost catch clause

The following example generates CS0724 because of the **throw** statement inside the **finally** clause block.

Example

The following example generates CS0724.

```
// CS0724.cs
using System;

class X
{
    static void Test()
    {
        try
        {
            throw new Exception();
        }
        catch
        {
            try
            {
            }
            finally
            {
                throw; // CS0724
            }
        }
    }

    static void Main()
    {
    }
}
```

Compiler Error CS0726

Error Message

'format specifier' is not a valid format specifier

This error occurs in the debugger. When you type a variable name into one of the debugger windows, you can follow it with a comma, and then a format specifier. Examples are: `myInt, h` or `myString, nq`. This error arises when the compiler does not recognize the [Format Specifiers in C#](#).

Compiler Error CS0727

Error Message

Invalid format specifier

This error occurs in the debugger. When you type a variable name into one of the debugger windows, you can follow it with a comma, and then a format specifier. Examples are: myInt, h; or myString,nq. This error arises when the compiler is completely unable to parse what you typed in. To resolve this error, retype the variable name, optionally followed by a comma and a [valid Format Specifier](#).

Compiler Error CS0729

Error Message

Type 'type' is defined in this assembly, but a type forwarder is specified for it

You cannot use a type forwarder for a type defined in the same assembly.

Example

The following sample generates CS0729.

```
// CS0729.cs
// compile with: /target:library
using System.Runtime.CompilerServices;
[assembly:TypeForwardedTo(typeof(TestClass))] // CS0729
class TestClass {}
```

Compiler Error CS0730

Error Message

Cannot forward type 'type' because it is a nested type of 'type'

This error is generated when you try to forward a nested class.

Example

The following sample generates CS0730. It consists of two source files. First, compile the library file `CS0730a.cs`, and then compile the file `CS0730.cs` referencing the library file.

```
// CS0730a.cs
// compile with: /t:library
public class Outer
{
    public class Nested {}
}
```

```
// CS0730.cs
// compile with: /t:library /r:CS0730a.dll
using System.Runtime.CompilerServices;

[assembly:TypeForwardedToAttribute(typeof(Outer.Nested))] // CS0730

[assembly:TypeForwardedToAttribute(typeof(Outer))] // OK
```

Compiler Error CS0731

Error Message

The type forwarder for type 'type' in assembly 'assembly' causes a cycle

This error can only occur with improperly formed imported metadata. It cannot occur with only C# source.

Example

The following sample generates CS0731. The example consists of three files:

1. Circular.il
2. Circular2.il
3. CS0731.cs

First compile the .il files as libraries, and then compile the .cs code referencing the two files.

```
// Circular.il
// compile with: /DLL /out=Circular.dll
.assembly extern circular2
{
    .ver 0:0:0:0
}
.assembly extern circular3
{
    .ver 0:0:0:0
}
.assembly extern mscorlib
{
    .publickeytoken = (B7 7A 5C 56 19 34 E0 89 )    // .z\V.4..
    .ver 2:0:0:0
}
.assembly Circular
{
    .custom instance void [mscorlib]System.Runtime.CompilerServices.CompilationRelaxationsAttribute::.ctor(int32) = ( 01 00 08 00 00 00 00 00 )
    .hash algorithm 0x00008004
    .ver 0:0:0:0
}
.class extern forwarder Circular.Referenced.TypeForwarder
{
    .assembly extern circular2
}
.module Circular.dll
// MVID: {880C2329-C915-42A0-83E9-9D10C3E6DBD0}
.imagebase 0x00400000
.file alignment 0x00000200
.stackreserve 0x00100000
.subsystem 0x0003        // WINDOWS_CUI
.corflags 0x00000001     // ILONLY
// Image base: 0x04E40000
// ===== CLASS MEMBERS DECLARATION =====
.class public abstract auto ansi sealed beforefieldinit User
    extends [mscorlib]System.Object
{
    .method public hidebysig static class [circular2]Circular.Referenced.TypeForwarder
        F() cil managed
    {
        .maxstack 1
        newobj instance void [circular2]Circular.Referenced.TypeForwarder::.ctor()
        ret
    }
}
```

```

// Circular2.il
// compile with: /DLL /out=Circular2.dll
.assembly extern mscorlib
{
    .publickeytoken = (B7 7A 5C 56 19 34 E0 89 )    // .z\V.4..
    .ver 2:0:0:0
}
.assembly extern Circular
{
    .ver 0:0:0:0
}
.assembly circular2
{
    .custom instance void [mscorlib]System.Runtime.CompilerServices.CompilationRelaxationsAttribute::.ctor(int32) = ( 01 00 08 00 00 00 00 00 )
    .hash algorithm 0x00008004
    .ver 0:0:0:0
}
.class extern forwarder Circular.Referenced.TypeForwarder
{
    .assembly extern Circular
}
.module circular2.dll
// MVID: {8B3BE5C8-DBE1-49C4-BC72-DF35F0387C21}
.imagebase 0x00400000
.file alignment 0x00000200
.stackreserve 0x00100000
.subsystem 0x0003          //  WINDOWS_CUI
.corflags 0x00000001      //  ILONLY
// Image base: 0x04E40000

```

```

// CS0731.cs
// compile with: /reference:circular.dll /reference:circular2.dll
// CS0731 expected
class A {
    public static void Main() {
        User.F();
    }
}

```

Compiler Error CS0733

Error Message

Cannot forward generic type, 'GenericType<>'

Example

The following example generates CS0733. Compile the first file as a library, and then reference it when you compile the second file.

```
// CS0733a.cs
// compile with: /target:library
public class GenericType<T>
{
}
```

```
// CS0733.cs
// compile with: /target:library /r:CS0733a.dll
[assembly: System.Runtime.CompilerServices.TypeForwardedTo(typeof(GenericType<int>))] //
CS0733
```

Compiler Error CS0734

Error Message

The `/moduleassemblyname` option may only be specified when building a target type of 'module'

The compiler option **`/moduleassemblyname`** should only be used when building a .netmodule. See [/moduleassemblyname \(Specify Friend Assembly for Module\) \(C# Compiler Option\)](#) for more information.

For more information on building a .netmodule, see [/target:module \(Create Module to Add to Assembly\) \(C# Compiler Options\)](#).

Example

The following sample generates CS0734. To resolve, add **`/target:module`** to the compilation.

```
// CS0734.cs
// compile with: /moduleassemblyname:A
// CS0734 expected
public class Test {}
```

Compiler Error CS1001

Error Message

Identifier expected

You did not supply an identifier. For example, when declaring an enum, you must specify members.

The following sample generates CS1001:

```
// CS1001.cs
public class clx
{
    enum splitch : int
    {
        'a', 'b' // CS1001, 'a' is not a valid int identifier
    };

    public static void Main()
    {
    }
}
```

Parameter names are required even if the compiler doesn't use them, for example, in an interface definition. These parameters are required so that programmers who are consuming the interface know something about the what the parameters mean.

```
// CS1001-2.cs
// compile with: /target:library
interface IMyTest
{
    void TestFunc1(int, int); // CS1001
}

class CMyTest : IMyTest
{
    void IMyTest.TestFunc1(int a, int b)
    {
    }
}
```


Compiler Error CS1002

Error Message

; expected

The compiler detected a missing semicolon.

The following sample generates CS1002:

```
// CS1002.cs
namespace x
{
    abstract public class clx
    {
        int i    // CS1002, missing semicolon

        public static int Main()
        {
            return 0;
        }
    }
}
```

Compiler Error CS1003

Error Message

Syntax error, 'char' expected

The compiler will generate this error for any one of several error conditions. Review your code to find the syntax error.

The following sample generates CS1003:

```
// CS1003.cs
public class b
{
    public static void Main()
    {
        int[] a;
        a();    // CS1003
    }
}
```

Compiler Error CS1004

Error Message

Duplicate 'modifier' modifier

A duplicate modifier, such as an **access** modifier, was detected.

The following sample generates CS1004:

```
// CS1004.cs
public class clx
{
    public public static void Main()    // CS1004, two public keywords
    {
    }
}
```

Compiler Error CS1007

Error Message

Property accessor already defined

When you declare a [property](#), you must also declare its accessor methods. However, a property cannot have more than one **get** accessor method or more than one **set** accessor method.

Example

The following sample generates CS1007:

```
// CS1007.cs
public class clx
{
    public int MyProperty
    {
        get
        {
            return 0;
        }
        get    // CS1007, this is the second get method
        {
            return 0;
        }
    }

    public static void Main() {}
}
```

Compiler Error CS1008

Error Message

Type byte, sbyte, short, ushort, int, uint, long, or ulong expected

Certain data types, such as [enums](#), can only be declared to hold data of specified types.

The following sample generates CS1008:

```
// CS1008.cs
abstract public class clx
{
    enum splitch : char    // CS1008, char not valid type for enums
    {
        x, y, z
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1009

Error Message

Unrecognized escape sequence

An unexpected character followed a backslash (\) in a [string](#). The compiler expects one of the valid escape characters; see [Character Escapes](#) for more information.

The following sample generates CS1009:

```
// CS1009-a.cs
class MyClass
{
    static void Main()
    {
        string a = "\m";    // CS1009
        // try the following line instead
        // string a = "\t";
    }
}
```

A common case for this error is using the backslash character in a file name, for example:

```
string filename = "c:\myFolder\myFile.txt";
```

To resolve this error, use "\\" or the @-quoted string literal, as shown in the following example:

```
// CS1009-b.cs
class MyClass
{
    static void Main()
    {
        string filename = "c:\myFolder\myFile.txt";    // CS1009
        // try the one of the following lines instead
        // string filename = "c:\\myFolder\\myFile.txt";
        // string filename = @"c:\myFolder\myFile.txt";
    }
}
```

Compiler Error CS1010

Error Message

Newline in constant

A [string](#) was not properly delimited.

The following sample generates CS1010:

```
// CS1010.cs
class Sample
{
    static void Main()
    {
        string a = "Hello World;    // CS1010, add end quote
    }
}
```

Compiler Error CS1011

Error Message

Empty character literal

A [char](#) was declared and initialized to a null. The initialization of a **char** must specify a character.

The following sample generates CS1011:

```
// CS1011.cs
class Sample
{
    public char CharField = '';    // CS1011
}
```


Compiler Error CS1012

Error Message

Too many characters in character literal

An attempt was made to initialize a [char](#) constant with more than one character.

CS1012 can also occur when doing data binding. For example the following line will give an error:

```
<%# DataBinder.Eval(Container.DataItem, 'doctitle') %>
```

Try the following line instead:

```
<%# DataBinder.Eval(Container.DataItem, "doctitle") %>
```

The following sample generates CS1012:

```
// CS1012.cs
class Sample
{
    static void Main()
    {
        char a = 'xx';    // CS1012
        char a2 = 'x';    // OK
        System.Console.WriteLine(a2);
    }
}
```

Compiler Error CS1013

Error Message

Invalid number

The compiler detected a malformed number. For more information on integral types, see the [Integral Types Table \(C# Reference\)](#).

Example

The following sample generates CS1013:

```
// CS1013.cs
class Sample
{
    static void Main()
    {
        int i = 0x;    // CS1013
    }
}
```

Compiler Error CS1014

Error Message

A get or set accessor expected

A method declaration was found in a property declaration. You can only declare **get** and **set** methods in a property.

For more information on properties, see [Using Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS1014.

```
// CS1014.cs
// compile with: /target:library
class Sample
{
    public int TestProperty
    {
        get
        {
            return 0;
        }
        int z;    // CS1014  not get or set
    }
}
```

Compiler Error CS1015

Error Message

An object, string, or class type expected

An attempt was made to pass a predefined data type into a [catch](#) block. Only data types that derive from [System.Exception](#) can be passed into a **catch** block. For more information on exceptions, see [Exception Handling Statements \(C# Reference\)](#).

Example

The following sample generates CS1015:

```
// CS1015.cs
class Sample
{
    static void Main()
    {
        try
        {
        }
        catch(int)    // CS1015, int is not derived from System.Exception
        {
        }
    }
}
```

Compiler Error CS1016

Error Message

Named attribute argument expected

Unnamed attribute arguments must appear before the named arguments.

Example

The following sample generates CS1016:

```
// CS1016.cs
using System;

[AttributeUsage(AttributeTargets.Class)]
public class HelpAttribute : Attribute
{
    public HelpAttribute(string url)    // url is a positional parameter
    {
        m_url = url;
    }

    public string Topic = null;        // Topic is a named parameter
    private string m_url = null;
}

[HelpAttribute(Topic="Samples", "http://intranet/inhouse")]    // CS1016
// try the following line instead
//[HelpAttribute("http://intranet/inhouse", Topic="Samples")]
public class MainClass
{
    public static void Main ()
    {
    }
}
```

Compiler Error CS1017

Error Message

Try statement already has an empty catch block

A **catch** block that does not take any parameters must be the last in a series of **catch** blocks. For more information on exceptions, see [Exception Handling Statements \(C# Reference\)](#).

Example

The following sample generates CS1017:

```
// CS1017.cs
using System;

namespace x
{
    public class b : Exception
    {
    }

    public class a
    {
        public static void Main()
        {
            try
            {
            }

            catch    // CS1017, must be last catch
            {
            }

            catch(b)
            {
                throw;
            }
        }
    }
}
```

Compiler Error CS1018

Error Message

Keyword 'this' or 'base' expected

The compiler encountered an incomplete constructor declaration.

Example

The following example generates CS1018, and suggests several ways to resolve the error:

```
// CS1018.cs
public class C
{
}

public class a : C
{
    public a(int i)
    {
    }

    public a () :    // CS1018
    // possible resolutions:
    // public a () resolves by removing the colon
    // public a () : base() calls C's default constructor
    // public a () : this(1) calls the assignment constructor of class a
    {
    }

    public static int Main()
    {
        return 1;
    }
}
```

Compiler Error CS1019

Error Message

Overloadable unary operator expected

You have a unary operator that returns a value of another class. If you want this conversion, you need to do an **implicit** or **explicit** cast.

The following sample generates CS1019:

```
// CS1019.cs
public class ii
{
    int i
    {
        get
        {
            return 0;
        }
    }
}

public class a
{
    public static a operator ii(a aa)    // CS1019
    // try the following line instead
    //public static a operator ++(a aa)
    {
        return new a();
    }

    public static void Main()
    {
    }
}
```


Compiler Error CS1020

Error Message

Overloadable binary operator expected

An attempt was made to define an [operator overload](#), but the operator was not a binary operator, which takes two parameters.

The following sample generates CS1020:

```
// CS1020.cs
public class iii
{
    public static int operator ++(iii aa, int bb)    // CS1020, change ++ to +
    {
        return 0;
    }

    public static void Main()
    {
    }
}
```

Integral constant is too large

The following sample generates CS1021:

```
// CS1021.cs  
enum F : int  
{  
    a=(int)259000000000000000000000 // CS1021  
}  
  
public class clx  
{  
    public static void Main()  
    {  
    }  
}
```

Compiler Error CS1022

Error Message

Type or namespace definition, or end-of-file expected

A source-code file does not have a matching set of braces.

The following sample generates CS1022:

```
// CS1022.cs
namespace x
{
}
} // CS1022
```

Compiler Error CS1023

Error Message

Embedded statement cannot be a declaration or labeled statement

An embedded statement, such as the statements following an **if** statement, can contain neither declarations nor labeled statements.

The following sample generates CS1023 twice:

```
// CS1023.cs
public class a
{
    public static void Main()
    {
        if (1)
            int i;      // CS1023, declaration is not valid here

        if (1)
            xx : i++;    // CS1023, labeled statement is not valid here
    }
}
```

Compiler Error CS1024

Error Message

Preprocessor directive expected

A line began with the pound symbol (#), but the subsequent string was not a valid [preprocessor directive](#).

The following sample generates CS1024:

```
// CS1024.cs
#import System    // CS1024
```

Compiler Error CS1025

Error Message

Single-line comment or end-of-line expected

A line with a [preprocessor directive](#) cannot have a multiline comment.

The following sample generates CS1025:

```
#if true /* hello
*/      // CS1025
#endif   // this is a good comment
```

CS1025 could also occur if you attempt some invalid preprocessor directive, as follows:

```
// CS1025.cs
#define a

class Sample
{
    static void Main()
    {
        #if a 1    // CS1025, invalid syntax
            System.Console.WriteLine("Hello, World!");
        #endif
    }
}
```

Compiler Error CS1026

Error Message

) expected

An incomplete statement was found.

A common cause of this error is placing a statement, rather than an expression, within an inline expression in an ASP.NET page. For example, the following is incorrect:

```
<%=new TimeSpan(DateTime.Now.Ticks - new DateTime(2001, 1, 1).Ticks).Days;%>
```

The following is correct:

```
<%=new TimeSpan(DateTime.Now.Ticks - new DateTime(2001, 1, 1).Ticks).Days %>
```

It is interpreted as follows:

```
<% Response.Write(new TimeSpan(DateTime.Now.Ticks - new DateTime(2001, 1, 1).Ticks).Days);  
%>
```

The following example generates CS1026:

```
// CS1026.cs  
#if (a == b    // CS1026, add closing )  
#endif  
  
class x  
{  
    public static void Main()  
    {  
    }  
}
```

Compiler Error CS1027

Error Message

#endif directive expected

A matching **#endif** [preprocessor directive](#) was not found for a specified **#if** directive. Or, the compiler may have found a **#endregion** directive when there was no matching **#region** directive inside a **#if** block.

The following sample generates CS1027:

```
// CS1027.cs
#if true    // CS1027, uncomment next line to resolve
// #endif

namespace x
{
    public class clx
    {
        public static void Main()
        {
        }
    }
}
```


Compiler Error CS1028

Error Message

Unexpected preprocessor directive

A [preprocessor directive](#) was found but not expected.

For example, a **#endif** was found with no preceding **#if**.

The following sample generates CS1028:

```
// CS1028.cs
#endif    // CS1028, no matching #if
namespace x
{
    public class clx
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS1029

Error Message

#error: 'text'

Displays the text of an error defined with the [#error](#) directive.

The following sample shows how to create a user-defined error:

```
// CS1029.cs
class Sample
{
    static void Main()
    {
        #error Let's give an error here    // CS1029
    }
}
```

Compiler Error CS1031

Error Message

Type expected

A type parameter is expected.

Example

The following sample generates CS1031:

```
// CS1031.cs
namespace x
{
    public class ii
    {
    }

    public class a
    {
        public static operator +(a aa)    // CS1031
        // try the following line instead
        // public static ii operator +(a aa)
        {
            return new ii();
        }

        public static void Main()
        {
            e = new base;    // CS1031, not a type
            e = new this;    // CS1031, not a type
            e = new ();      // CS1031, not a type
        }
    }
}
```

Compiler Error CS1032

Error Message

Cannot define/undefine preprocessor symbols after first token in file

The **#define** and **#undef** [preprocessor directives](#) must be used at the beginning of a program, before any other keywords, such as those used in the namespace declaration.

The following sample generates CS1032:

```
// CS1032.cs
namespace x
{
    public class clx
    {
        #define a    // CS1032, put before namespace
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS1033

Error Message

Compiler limit exceeded: File cannot exceed 'number' lines

A source-code file exceeded the maximum allowable number of lines that the compiler can process. To resolve this error, create two or more source-code files from the original file. The maximum number of lines is 268,435,454 lines. If you are using /debug, using more than 16,707,566 will result in corrupted debug information.

Compiler Error CS1034

Error Message

Compiler limit exceeded: Line cannot exceed 'number' characters

The limit for the number of characters allowed on a line is 16,777,214.

Compiler Error CS1035

Error Message

End-of-file found, '*/' expected

An opening comment delimiter was not matched with a closing delimiter.

The following sample generates CS1035:

```
// CS1035.cs
public class a
{
    public static void Main()
    {
    }
}
/*    // CS1035, needs closing comment
```

Compiler Error CS1036

Error Message

(or . expected

The XML in a [doc](#) comment was badly formed.

Compiler Error CS1037

Error Message

Overloadable operator expected

When specifying a comment with [/doc](#), the compiler encountered an invalid link.

Compiler Error CS1038

Error Message

#endregion directive expected

A [#region](#) directive did not have a matching [#endregion](#) directive.

The following sample generates CS1038:

```
// CS1038.cs
#region testing

public class clx
{
    public static void Main()
    {
    }
}
// CS1038
// uncomment the next line to resolve
// #endregion
```

Compiler Error CS1039

Error Message

Unterminated string literal

The compiler detected an ill-formed [string](#) literal.

Example

The following sample generates CS1039. To resolve the error, add the terminating quotation mark.

```
// CS1039.cs
public class MyClass
{
    public static void Main()
    {
        string b = @"hello, world;    // CS1039
    }
}
```

Compiler Error CS1040

Error Message

Preprocessor directives must appear as the first non-whitespace character on a line

A [preprocessor directive](#) was found on a line and was not the first token on the line. A directive must be the first token on the line.

The following sample generates CS1040:

```
// CS1040.cs
/* Define a symbol, X */ #define X    // CS1040

// try the following two lines instead
// /* Define a symbol, X */
// #define X

public class MyClass
{
    public static void Main()
    {
    }
}
```

Compiler Error CS1041

Error Message

Identifier expected, 'keyword' is a keyword

A reserved word for the C# language was found where an identifier was expected. Replace the [keyword](#) with a user-specified identifier.

Example

The following sample generates CS1041:

```
// CS1041a.cs
class MyClass
{
    public void f(int long)    // CS1041
    // Try the following instead:
    // public void f(int i)
    {
    }

    public static void Main()
    {
    }
}
```

When you are importing from another programming language that does not have the same set of reserved words, you can modify the reserved identifier with the @ prefix, as demonstrated in the following sample.

An identifier with an @ prefix is called a verbatim identifier.

```
// CS1041b.cs
class MyClass
{
    public void f(int long)    // CS1041
    // Try the following instead:
    // public void f(int @long)
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1043

Error Message

{ or ; expected

A property accessor was declared incorrectly. For more information, see [Using Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS1043.

```
// CS1043.cs
// compile with: /target:library
public class MyClass
{
    public int DoSomething
    {
        get return 1;    // CS1043
        set {}
    }

    // OK
    public int DoSomething2
    {
        get { return 1;}
    }
}
```

Compiler Error CS1044

Error Message

Cannot use more than one type in a for, using, fixed, or declaration statement

The compiler found an invalid statement.

The following sample generates CS1044:

```
// CS1044.cs
using System;

public class MyClass : IDisposable
{
    public void Dispose()
    {
        Console.WriteLine("Res1.Dispose()");
    }

    public static void Main()
    {
        using (MyClass mc1 = new MyClass(),
              MyClass mc2 = new MyClass())    // CS1044, remove an instantiation
        {
        }
    }
}
```

Compiler Error CS1055

Error Message

An add or remove accessor expected

If your [event](#) is not declared as a field, it must define both **add** and **remove** accessor functions.

The following sample generates CS1055:

```
// CS1055.cs
delegate void del();
class Test
{
    public event del MyEvent
    {
        int i;    // CS1055
        // uncomment accessors and delete previous line to resolve
        // add
        // {
        //     MyEvent += value;
        // }
        // remove
        // {
        //     MyEvent -= value;
        // }
    }

    public static void Main()
    {
    }
}
```


Compiler Error CS1056

Error Message

Unexpected character 'character'

The C# compiler encountered an unexpected character, and is unable to identify the token currently being processed. For example, if the compiler encounters a Euro-character in the middle of processing an identifier, it will be unable to classify the identifier, since a Euro-character would only be valid inside a string, and the compiler knows it is not processing a string.

Compiler Error CS1501

Error Message

No overload for method 'method' takes 'number' arguments

A call was made to a class method, but there is no form of the method that takes the necessary number of arguments.

CS1501 could occur if you are calling a method on a class in a referenced assembly and if that method has default values on one or more of its parameters. C# does not let you create methods with a default value on a parameter, but another language that targets the runtime might. If a parameter (in a method in a referenced assembly) has a default value, you must still call the method and explicitly pass all parameters.

CS1501 could occur when you allocate memory for a delegate; you must also specify the name of the method that the delegate represents, as shown in the second example below.

CS1501 also occurs if you attempt to instantiate a derived class that inherits from a base class that has no default constructor but does have a non-default constructor that takes one or more arguments. To correct the error, either supply a default constructor for the base class, or use the [base \(C# Reference\)](#) keyword to call the appropriate base class constructor, as shown in the third example below. For more information, see [Using Constructors \(C# Programmers Reference\)](#).

Example

The following sample generates CS1501.

```
// CS1501.cs
public class a
{
    // declare the following constructor to resolve this CS1501
    /*
    public a(int i)
    {
    }
    */

    public static void Main()
    {
        a aa = new a(2);    // CS1501
    }
}
```

The following sample generates CS1501.

```
// CS1501a.cs
using System;

delegate string func(int i);    // declare the delegate

class a
{
    static func dt;    // class member-field of the declared delegate type

    public static void Main()
    {
        dt = new func();    // CS1501
        // try the following line instead
        // dt = new func(z);    // this works
        x(dt);
    }

    public static string z(int j)
    {
        Console.WriteLine(j);
        return j.ToString();
    }
}
```

```
public static void x(func hello)
{
    hello(8);
}
}
```

The following sample generates CS1501.

```
// CS1501b.cs
class Base
{
    public Base(string s)
    {
    }
}

class Derived : Base
{ // CS1501
    public Derived(string s)
    {
    }

    // Try this instead:
    // public Derived(string s) : base(s)
    // {
    // }

    public static void Main()
    {
    }
}
```

Compiler Error CS1502

Error Message

The best overloaded method match for 'declaration' has some invalid arguments

This error occurs when the argument types being passed to the method do not match the parameter types of that method. If the called method is overloaded, then none of the overloaded versions has a signature that matches the argument types being passed.

To resolve this problem, do one of the following:

- Double-check the types of the arguments being passed. Make sure that they match the arguments of the method being called.
- If appropriate, convert any mismatched parameters using the [System.Convert Class](#).
- If appropriate, cast any mismatched parameters to match the type that the method is expecting.
- If appropriate, define another overloaded version of the method to match the parameter types that are being sent.

The following sample generates CS1502:

```
// CS1502.cs
namespace x
{
    public class a
    {
        public a(char i)
        // try the following constructor instead
        // public a(int i)
        {
        }

        public static void Main()
        {
            a aa = new a(2222);    // CS1502
        }
    }
}
```

Compiler Error CS1503

Error Message

Argument 'arg': cannot convert from 'type1' to 'type2'

The type of one argument in a method does not match the type that was passed when the class was instantiated.

See [CS1502](#) for a discussion on how to resolve this error.

Compiler Error CS1504

Error Message

Source file 'file' could not be opened ('reason')

A source file could not be opened or read by the compiler. The file may be locked by another application, or there could be some other operating system problem. The message includes the operating system's reason for why the compiler could not open or read the file.

Compiler Error CS1507

Error Message

Cannot link resource file 'file' when building a module

[/linkresource](#) was used in the same compilation with [/target:module](#), which is not allowed. For example, the following options would generate CS1507:

```
csc /linkresource:rf.resource /target:module in.cs
```

Embedding resources ([/resource](#)), however, is allowed.

Compiler Error CS1508

Error Message

Resource identifier 'identifier' has already been used in this assembly

In a compilation, the same identifier (***identifier***) was passed to two or more [/resource](#) or [/linkresource](#) compiler options.

For example, the following options would generate CS1508:

```
/resource:anyfile.bmp,DuplicatIdent /linkresource:a.bmp,DuplicatIdent
```


Compiler Error CS1509

Error Message

Referenced file 'file' is not an assembly; use '/addmodule' option instead

An output file (output file 1), produced in a compilation that used [/target:module](#) (does not have an assembly manifest), was specified to [/reference](#). So, rather than appending an assembly to the assembly for the current program, the metadata information in output file 1 will be added to the assembly for the current program.

Compiler Error CS1510

Error Message

A ref or out argument must be an assignable variable

Only a variable can be passed as a [ref](#) parameter in a method call. A **ref** value is similar to passing a pointer.

Example

The following sample generates CS1510:

```
// CS1510.cs
public class C
{
    public static int j = 0;

    public static void mf(ref int j)
    {
        j++;
    }

    public static void Main ()
    {
        mf (ref 2);    // CS1510, can't pass a number as a ref parameter
        // try the following to resolve the error
        // mf (ref j);
    }
}
```

Compiler Error CS1511

Error Message

Keyword 'base' is not available in a static method

The [base](#) keyword was used in a [static](#) method. **base** can only be called in an instance constructor, instance method, or instance accessor.

Example

The following sample generates CS1511.

```
// CS1511.cs
// compile with: /target:library
public class A
{
    public int j = 0;
}

class C : A
{
    public void Method()
    {
        base.j = 3;    // base allowed here
    }

    public static int StaticMethod()
    {
        base.j = 3;    // CS1511
        return 1;
    }
}
```

Compiler Error CS1512

Error Message

Keyword 'base' is not available in the current context

The [base](#) keyword was used outside of a method, property, or constructor.

The following example generates CS1512:

```
// CS1512.cs
using System;

class Base {}

class CMyClass : Base
{
    private String xx = base.ToString();    // CS1512
    // Try putting this initialization in the constructor instead:
    // public CMyClass()
    // {
    //     xx = base.ToString();
    // }

    public static void Main()
    {
        CMyClass z = new CMyClass();
    }
}
```

Compiler Error CS1513

Error Message

} expected

The compiler expected a closing curly brace (}) that was not found.

The following sample generates CS1513:

```
// CS1513
namespace y    // CS1513, no close curly brace
{
    class x
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS1514

Error Message

{ expected

The compiler expected an opening curly brace ({) that was not found.

The following sample generates CS1514:

```
// CS1514.cs
namespace x
// CS1514, no open curly brace
```

Compiler Error CS1515

Error Message

'in' expected

In a [foreach, in](#) statement, the "in" part is missing.

Example

The following sample generates CS1515:

```
using System;

class Driver
{
    static void Main()
    {
        int[] arr = new int[] {1, 2, 3};

        // try the following line instead
        // foreach (int x in arr)
        foreach (int x arr)      // CS1515, "in" is missing
        {
            Console.WriteLine(x);
        }
    }
}
```

Compiler Error CS1517

Error Message

Invalid preprocessor expression

The compiler encountered an invalid preprocessor expression.

For more information, see [Preprocessor Directives](#).

The following sample shows some valid and invalid preprocessor expressions:

```
// CS1517.cs
#if symbol      // OK
#endif
#if !symbol     // OK
#endif
#if (symbol)    // OK
#endif
#if true       // OK
#endif
#if false      // OK
#endif
#if 1          // CS1517
#endif
#if ~symbol     // CS1517
#endif
#if *          // CS1517
#endif

class x
{
    public static void Main()
    {
    }
}
```


Compiler Error CS1518

Error Message

Expected class, delegate, enum, interface, or struct

A declaration was found that is not supported in a [namespace](#). Inside a namespace, the compiler accepts only classes, structs, enums, interfaces, namespaces, and delegates.

Example

The following sample generates CS1518:

```
// CS1518.cs
namespace x
{
    sealed class c1 {};           // OK
    namespace f2 {};             // OK
    sealed f3 {};                // CS1518
}
```

Compiler Error CS1519

Error Message

Invalid token 'token' in class, struct, or interface member declaration

Any [class](#), struct, or interface member declaration that contains invalid modifiers before the type will generate this error. To fix the error, remove the invalid modifiers.

The following sample generates CS1519:

```
// CS1519.cs
public class IMyInterface
{
    checked void f4();    // CS1519
    // Remove "checked" from the line above.
    lock void f5();       // CS1519
    // Remove "lock" from the line above
    namespace;           // CS1519
    // The line above should be removed entirely.
    int i;                // OK
}
```

Compiler Error CS1520

Error Message

Class, struct, or interface method must have a return type

A method that is declared in a class, struct, or interface must have an explicit return type.

The following sample generates CS1520:

```
// CS1520a.cs
public class x
{
    f7()    // CS1520, needs return type
    // try the following definition
    // void f7()
    {
    }

    public static void Main()
    {
    }
}
```

Alternatively, this error might be encountered when the case of a constructor's name differs from that of the class or struct declaration, as in the following sample:

```
// CS1520b.cs
public class Class1
{
    public class1()    // CS1520, incorrect case
    {
    }
    static void Main()
    {
    }
}
```

Compiler Error CS1521

Error Message

Invalid base type

A [base](#) class specification was ill formed.

The following sample generates CS1521:

```
// CS1521.cs
class CMyClass
{
    public static void Main()
    {
    }
}

class CMyClass1 : CMyClass    // OK
{
}

class CMyClass2 : CMyClass[]  // CS1521
{
}

class CMyClass3 : CMyClass*   // CS1521
{
}
```

Compiler Error CS1524

Error Message

Expected catch or finally

A **try** block must be followed by a **catch** or **finally** block.

For more information on exceptions, see [Exception Handling Statements \(C# Reference\)](#).

Example

The following sample generates CS1524:

```
// CS1524.cs
class x
{
    public static void Main()
    {
        try
        {
            // Code here
        }
        catch
        {
        }
        try
        {
            // Code here
        }
        finally
        {
        }
        try
        {
            // Code here
        }
    }
    // CS1524, missing catch or finally
}
```

Compiler Error CS1525

Error Message

Invalid expression term 'character'

The compiler detected an invalid character in an expression.

The following sample generates CS1525:

```
// CS1525.cs
class x
{
    public static void Main()
    {
        int i = 0;
        i = i +    // OK - identifier
        'c' +      // OK - character
        (5) +      // OK - parenthesis
        [ +        // CS1525, operator not a valid expression element
        throw +    // CS1525, keyword not allowed in expression
        void;      // CS1525, void not allowed in expression
    }
}
```

An empty label can also generate CS1525, as in the following sample:

```
// CS1525b.cs
using System;
public class MyClass
{
    public static void Main()
    {
        goto FoundIt;
        FoundIt:    // CS1525
        // Uncomment the following line to resolve:
        // System.Console.WriteLine("Hello");
    }
}
```

Compiler Error CS1526

Error Message

A new expression requires () or [] after type

The [new](#) operator, used to dynamically allocate memory for an object, was not specified correctly.

Example

The following sample shows how to use **new** to allocate space for an array and an object.

```
// CS1526.cs
public class y
{
    public static int i = 0;
    public int myi = 0;
}

public class z
{
    public static void Main()
    {
        y py = new y;    // CS1526
        y[] aoys = new y[10];    // Array of Ys

        for (int i = 0; i < aoys.Length; i++)
            aoys[i] = new y();    // an object of type y
    }
}
```

Compiler Error CS1527

Error Message

Namespace elements cannot be explicitly declared as private, protected, or protected internal

Type declarations in a namespace can have either [public](#) or [internal](#) access. If no accessibility is specified, **internal** is the default.

The following sample generates CS1527:

```
// CS1527.cs
namespace Sample
{
    private class C1 {};           // CS1527
    protected class C2 {};        // CS1527
    protected internal class C3 {}; // CS1527
}
```


Compiler Error CS1528

Error Message

Expected ; or = (cannot specify constructor arguments in declaration)

A reference to a class was formed as if an object to the class was being created. For example, there was an attempt to pass a variable to a constructor. Use the [new](#) operator to create an object of a class.

The following sample generates CS1528:

```
// CS1528.cs
using System;

public class B
{
    public B(int i)
    {
        _i = i;
    }

    public void PrintB()
    {
        Console.WriteLine(_i);
    }

    private int _i;
}

public class mine
{
    public static void Main()
    {
        B b(3);    // CS1528, reference is not an object
        // try one of the following
        // B b;
        // or
        // B bb = new B(3);
        // bb.PrintB();
    }
}
```

Compiler Error CS1529

Error Message

A using clause must precede all other namespace elements except extern alias declarations

A [using](#) clause must appear first in a namespace.

Example

The following sample generates CS1529:

```
// CS1529.cs
namespace X
{
    namespace Subspace
    {
        using Microsoft;

        class SomeClass
        {
        };

        using Microsoft;           // CS1529, place before class definition
    }

    using System.Reflection;       // CS1529, place before namespace 'Subspace'
}

using System;                     // CS1529, place at the beginning of the file
```

Compiler Error CS1530

Error Message

Keyword new not allowed on namespace elements

It is not necessary to specify the [new](#) keyword on any construct that is in a [namespace](#).

The following sample generates CS1530:

```
// CS1530.cs
namespace a
{
    new class i    // CS1530
    {
    }

    // try the following instead
    class ii
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Error CS1534

Error Message

Overloaded binary operator 'operator' takes two parameters

The definition of a binary [overloadable operator](#) must take two parameters.

The following sample generates CS1534:

```
// CS1534.cs
class MyClass
{
    public static MyClass operator - (MyClass MC1, MyClass MC2, MyClass MC3)    // CS1534
    // try the following line instead
    // public static MyClass operator - (MyClass MC1, MyClass MC2)
    {
        return new MyClass();
    }

    public static int Main()
    {
        return 1;
    }
}
```

Compiler Error CS1535

Error Message

Overloaded unary operator 'operator' takes one parameter

The definition of a unary [overloadable operator](#) must take one parameter.

Example

The following sample generates CS1535:

```
// CS1535.cs
class MyClass
{
    // uncomment the method parameter to resolve CS1535
    public static MyClass operator ++ (/*MyClass MC1*/)    // CS1535
    {
        return new MyClass();
    }

    public static int Main()
    {
        return 1;
    }
}
```

Compiler Error CS1536

Error Message

Invalid parameter type void

It is not necessary or valid to specify a [void](#) parameter other than a void pointer.

The following sample generates CS1536:

```
// CS1536.cs
class a
{
    public static int x( void )    // CS1536
    // try the following line instead
    // public static int x()
    {
        return 0;
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1537

Error Message

The using alias 'alias' appeared previously in this namespace

You defined a symbol twice as an alias for a namespace. A symbol can only be defined once.

The following sample generates CS1537:

```
// CS1537.cs
namespace x
{
    using System;
    using Object = System.Object;
    using Object = System.Object;    // CS1537, delete this line to resolve
    using System = System;
}
```

Compiler Error CS1540

Error Message

Cannot access protected member 'member' via a qualifier of type 'type1'; the qualifier must be of type 'type2' (or derived from it)

Although a derived [class](#) can access protected members of its base class, it cannot do so through an instance of the base class.

The following sample generates CS1540:

```
// CS1540.cs
public class Base
{
    protected void func()
    {
    }
}

public class Derived : Base
{
    public static void test(Base anotherInstance)
    // the method declaration could be changed as follows
    // public static void test(Derived anotherInstance)
    {
        anotherInstance.func();    // CS1540
    }
}

public class Tester : Derived
{
    public static void Main()
    {
        Base pBase = new Base();
        // the allocation could be changed as follows
        // Derived pBase = new Derived();
        test(pBase);
    }
}
```


Compiler Error CS1541

Error Message

Invalid reference option: 'symbol' — cannot reference directories

The compiler detected an attempt to specify a directory rather than a specific file. For example, when you use the [/reference](#) compiler option, you must specify a file; it is not possible to specify a directory.

For example, passing `/reference:c:\` to the compiler would generate CS1541.

Compiler Error CS1542

Error Message

'dll' cannot be added to this assembly because it already is an assembly; use '/R' option instead

The file that was referenced with the [/addmodule](#) compiler option was not built with [/target:module](#); use [/reference](#) to reference the file in this compilation.

Compiler Error CS1545

Error Message

Property, indexer, or event 'property' is not supported by the language; try directly calling accessor methods 'set accessor' or 'get accessor'

Your code is consuming an object that has a default [indexer](#) and tried to use the indexed syntax. To resolve this error, call the property's **get** or **set** accessor method.

Example

```
// CPP1545.cpp
// compile with: /clr /LD
// a Visual C++ program
using namespace System;
public ref struct Employee {
    Employee( String^ s, int d ) {}

    property String^ name {
        String^ get() {
            return nullptr;
        }
    }
};

public ref struct Manager {
    property Employee^ Report [String^] {
        Employee^ get(String^ s) {
            return nullptr;
        }

        void set(String^ s, Employee^ e) {}
    }
};
```

The following sample generates CS1545.

```
// CS1545.cs
// compile with: /r:CPP1545.dll

class x {
    public static void Main() {
        Manager Ed = new Manager();
        Employee Bob = new Employee("Bob Smith", 12);
        Ed.Report( Bob.name ) = Bob;    // CS1545
        Ed.set_Report( Bob.name, Bob);  // OK
    }
}
```

Compiler Error CS1546

Error Message

Property, indexer, or event 'property' is not supported by the language; try directly calling accessor method 'accessor'

Your code is consuming an object that has a default indexed property and tried to use the indexed syntax. To resolve this error, call the property's accessor method. For more information on indexers and properties, see [Indexers \(C# Programming Guide\)](#) and [Defining and Using Properties](#).

The following sample generates CS1546.

Example

This code sample consists of a .cpp file, which compiles to a .dll, and a .cs file, which uses that .dll. The following code is for the .dll file, and defines a property to be accessed by the code in the .cs file.

```
// CPP1546.cpp
// compile with: /clr /LD
using namespace System;
public ref class MCPP
{
public:
    property int Prop [int,int]
    {
        int get( int i, int b )
        {
            return i;
        }
    }
};
```

This is the C# file.

```
// CS1546.cs
// compile with: /r:CPP1546.dll
using System;
public class Test
{
    public static void Main()
    {
        int i = 0;
        MCPP mcpp = new MCPP();
        i = mcpp.Prop(1,1); // CS1546
        // Try the following line instead:
        // i = mcpp.get_Prop(1,1);
    }
}
```

Compiler Error CS1547

Error Message

Keyword 'void' cannot be used in this context

The compiler detected an invalid use of the [void](#) keyword.

The following sample generates CS1547:

```
// CS1547.cs
public class MyClass
{
    void BadMethod()
    {
        void i;    // CS1547, cannot have variables of type void
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1548

Error Message

Cryptographic failure while signing assembly 'assembly' — 'reason'

CS1548 occurs when assembly signing fails. This is usually due to an invalid key file name, an invalid key file path, or a corrupt key file.

To fully sign an assembly, you must provide a valid key file that contains information about the public and private keys. To delay sign an assembly, you must select the **Delay sign only** check box and provide a valid key file that contains information about the public key information. The private key is not necessary when an assembly is delay-signed.

For more information, see [How to: Sign an Assembly \(Visual Studio\)](#), [/keyfile \(Specify Strong Name Key File\) \(C# Compiler Options\)](#) and [/delaysign \(Delay Sign the Assembly\) \(C# Compiler Options\)](#).

When creating an assembly, the C# compiler calls into a utility called al.exe. If there is a failure in the assembly creation, the reason for the failure is reported by al.exe. See [Al.exe Tool Errors and Warnings](#) and search that topic for the text reported by the compiler in 'reason'.

See Also

Tasks

[How to: Sign an Assembly \(Visual Studio\)](#)

Compiler Error CS1549

Error Message

Appropriate cryptographic service not found

An [RSA](#) compatible cryptographic service provider was not installed.

Compiler Error CS1551

Error Message

Indexers must have at least one parameter

An [indexer](#) that takes no arguments was declared.

The following sample generates CS1551:

```
// CS1551.cs
public class MyClass
{
    int intI;

    int this[]    // CS1551
    // try the following line instead
    // int this[int i]
    {
        get
        {
            return intI;
        }
        set
        {
            intI = value;
        }
    }

    public static void Main()
    {
    }
}
```


Compiler Error CS1552

Error Message

Array type specifier, [], must appear before parameter name

The position of the array type specifier is after the variable name in the array declaration.

Example

The following sample generates CS1552:

```
// CS1552.cs
public class C
{
    public static void Main(string args[])    // CS1552
    // try the following line instead
    // public static void Main(string [] args)
    {
    }
}
```

Compiler Error CS1553

Error Message

Declaration is not valid; use 'modifier operator <dest-type> (...)' instead

The return type for an [operator](#) must immediately precede the parameter list, and *modifier* is either **implicit** or **explicit**.

The following sample generates CS1553:

```
// CS1553.cs
class MyClass
{
    public static int implicit operator (MyClass f)    // CS1553
    // try the following line instead
    // public static implicit operator int (MyClass f)
    {
        return 6;
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1554

Error Message

Declaration is not valid; use '<type> operator op (...)' instead

The return type for a user-defined [operator](#) must appear before the keyword operator.

The following sample generates CS1554:

```
// CS1554.cs
class MyClass
{
    public static operator ++ MyClass (MyClass f)    // CS1554
    // try the following line instead
    // public static MyClass operator ++ (MyClass f)
    {
        return new MyClass ();
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1555

Error Message

Could not find 'class' specified for Main method

A class was specified to the [/main](#) compiler option, but the class name was not found in the source code.

Compiler Error CS1556

Error Message

'construct' specified for Main method must be a valid class or struct

The [/main](#) compiler option was passed an identifier that was not a class name.

Compiler Error CS1557

Error Message

Cannot use 'class' for Main method because it is in a different output file

The [/main](#) compiler option was specified for one output file in a multi-output file compilation. However, the class was not found in the source code for the /main compilation; it was found in a source code file for one of the other output files in the compilation.

Compiler Error CS1558

Error Message

'class' does not have a suitable static Main method

The [/main](#) compiler option specified a class in which to look for a **Main** method. However, the [Main](#) method was not defined correctly.

The following example generates CS1558 because of invalid return type.

```
// CS1558.cs
// compile with: /main:MyNamespace.MyClass

namespace MyNamespace
{
    public class MyClass
    {
        public static float Main()
        {
            return 0.0; // CS1558 because the return type is a float.
        }
    }
}
```

Compiler Error CS1559

Error Message

'object' is imported and cannot be used as an entry point for program 'program'

An invalid class was specified to the `/main` compiler option; the class cannot be used as a location for the `Main` method.

Compiler Error CS1560

Error Message

Invalid filename specified for preprocessor directive. Filename is too long or not a valid filename

The file name that was specified with [#line \(C# Reference\)](#) exceeded _MAX_PATH (256 characters) or the line on which **#line** was found exceeded 2000 characters.

Example

The following sample generates CS1560.

```
// cs1560.cs
using System;
class MyClass
{
    public static void Main()
    {
        Console.WriteLine("Normal line #1.");
        #line 21 "MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile12345
67890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile
1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890.
txt"    // CS1560
    }
}
```

Compiler Error CS1561

Error Message

Output filename is too long or invalid

The output file name cannot be longer than 256 characters, and may not contain invalid characters, such as the Euro symbol, a question mark, or a backslash, among others.

Compiler Error CS1562

Error Message

Outputs without source must have the /out option specified

The compilation could create an output file, but there was no source code file as input from which the name of the output file could be implied. For example, you may be trying to compile a metadata- or resource-only file.

Use the [/out](#) compiler option to specify the name of the output file.

Compiler Error CS1563

Error Message

Output 'output file' does not have any source files

The [/out](#) compiler option was specified, but no source code files follow. You should either not use **/out**, or you should specify the source code files following **/out**.

Compiler Error CS1565

Error Message

Conflicting options specified: Win32 resource file; Win32 icon

It is not valid to specify both the [/win32res](#) compiler option and the [/win32ico](#) compiler option in the same compilation.

Compiler Error CS1566

Error Message

Error reading resource file 'file' — 'reason'

The compiler had trouble with the file name passed to the [/resource](#) compiler option.

Compiler Error CS1567

Error Message

Error generating Win32 resource: 'file'

Your compilation either used the [/win32icon](#) compiler option or did not use [/win32res](#), which causes the compiler to generate a file that contains resource information, but the compiler was unable to create the file due to insufficient disk space or some other error.

If you are unable to resolve the file-generation problem, you could use [/win32res](#), which does not generate a file that contains resource information.

Compiler Error CS1569

Error Message

Error generating XML documentation file 'Filename' ('reason')

When attempting to write the XML documentation to the file named in the message, an error occurred for the reason specified. The reason may be something like "network drive not found," or "access denied." Often, the reason will suggest what needs to be done to correct the error. For example, if the error says "access denied," you would verify that you have write permission on the file.

Example

```
// CS1569.cs
// compile with: /doc:CS1569.xml
// post-build command: attrib +r CS1569.xml
class Test
{
    /// <summary>Test.</summary>
    public static void Main() {}
}
```

The previous sample generated an .xml file that was then set to read only. This sample attempts to write to the same file. The following sample generates CS1569.

```
// CS1569_a.cs
// compile with: /doc:CS1569.xml
// CS1569 expected
class Test
{
    /// <summary>Test.</summary>
    public static void Main() {}
}
```


Compiler Error CS1575

Error Message

A stackalloc expression requires [] after type

The size of the requested allocation, with [stackalloc](#), must be specified in square brackets.

The following sample generates CS1575:

```
// CS1575.cs
// compile with: /unsafe
public class MyClass
{
    unsafe public static void Main()
    {
        int *p = stackalloc int (30);    // CS1575
        // try the following line instead
        // int *p = stackalloc int [30];
    }
}
```

Compiler Error CS1576

Error Message

The line number specified for #line directive is missing or invalid

The compiler detected an error with the value passed to the [#line](#) directive.

The following sample generates CS1576:

```
// CS1576.cs
public class MyClass
{
    static void Main()
    {
        #line "abc.sc"           // CS1576
        // try the following line instead
        // #line 101 "abc.sc"
        intt i; // error will be reported on line 101
    }
}
```

Compiler Error CS1577

Error Message

Assembly generation failed — reason

The assembly-generation part of the compilation failed. See the errors documentation for the alink utility ([Al.exe Tool Errors and Warnings](#)) for more information.

Compiler Error CS1578

Error Message

Filename, single-line comment or end-of-line expected

After a [#line](#) directive, only a file name (in double quotation marks) or a single-line comment is allowed.

The following sample generates CS1578:

```
// CS1578.cs
class MyClass
{
    static void Main()
    {
        #line 101 abc.cs    // CS1578
        // try the following line instead
        // #line 101 "abc.cs"
        intt i;            // error will be reported on line 101
    }
}
```

Compiler Error CS1579

Error Message

foreach statement cannot operate on variables of type 'type1' because 'type2' does not contain a public definition for 'identifier'

To iterate through a collection using the [foreach](#) statement, the collection must meet the following requirements:

- It must be an interface, class or struct.
- It must include a public [GetEnumerator](#) method that returns a type.
- The return type must contain a public property named [Current](#), and a public method named [MoveNext](#).
- For more information, see [How to: Access a Collection Class with foreach \(C# Programming Guide\)](#).

Example

In this sample, [foreach](#) is not able to iterate through the collection because there is no **public GetEnumerator** method in `MyCollection`.

The following sample generates CS1579.

```
// CS1579.cs
using System;
public class MyCollection
{
    int[] items;
    public MyCollection()
    {
        items = new int[5] {12, 44, 33, 2, 50};
    }

    // Delete the following line to resolve.
    MyEnumerator GetEnumerator()

    // Uncomment the following line to resolve:
    // public MyEnumerator GetEnumerator()
    {
        return new MyEnumerator(this);
    }

    // Declare the enumerator class:
    public class MyEnumerator
    {
        int nIndex;
        MyCollection collection;
        public MyEnumerator(MyCollection coll)
        {
            collection = coll;
            nIndex = -1;
        }

        public bool MoveNext()
        {
            nIndex++;
            return(nIndex < collection.items.GetLength(0));
        }

        public int Current
        {
            get
            {
                return(collection.items[nIndex]);
            }
        }
    }
}
```

```
public static void Main()
{
    MyCollection col = new MyCollection();
    Console.WriteLine("Values in the collection are:");
    foreach (int i in col)    // CS1579
    {
        Console.WriteLine(i);
    }
}
```

Compiler Error CS1583

Error Message

'file' is not a valid Win32 resource file

A file was passed to the [/win32res](#) compiler option that was not a valid resource file.

Compiler Error CS1585

Error Message

Member modifier 'keyword' must precede the member type and name

The access specifier in a method signature did not occur in the correct location.

The following sample generates CS1585:

```
// CS1585.cs
public class Class1
{
    public void static Main(string[] args)    // CS1585
    // try the following line instead
    // public static void Main(string[] args)
    {
    }
}
```


Compiler Error CS1586

Error Message

Array creation must have array size or array initializer

An array was declared incorrectly.

The following sample generates CS1586:

```
// CS1586.cs
using System;
class MyClass
{
    public static void Main()
    {
        int[] a = new int[];    // CS1586
        // try the following line instead
        int[] b = new int[5];
    }
}
```

Compiler Error CS1588

Error Message

Cannot determine common language runtime directory -- 'reason'

The compiler cannot retrieve the directory that the .NET Framework common language runtime is installed in. This error indicates that the common language runtime is improperly installed.

Compiler Error CS1593

Error Message

Delegate 'del' does not take 'number' arguments

The number of arguments passed to a [delegate](#) invocation does not agree with the number of parameters in the delegate declaration.

The following sample generates CS1593:

```
// CS1593.cs
using System;
delegate string func(int i);    // declare delegate

class a
{
    public static void Main()
    {
        func dt = new func(z);
        x(dt);
    }

    public static string z(int j)
    {
        Console.WriteLine(j);
        return j.ToString();
    }

    public static void x(func hello)
    {
        hello(8, 9);    // CS1593
        // try the following line instead
        // hello(8);
    }
}
```

Compiler Error CS1594

Error Message

Delegate 'delegate' has some invalid arguments

The type of an argument passed to a [delegate](#) invocation does not agree with the type of the parameter in the delegate declaration.

The following sample generates CS1594:

```
// CS1594.cs
using System;
delegate string func(int i);    // declare delegate

class a
{
    public static void Main()
    {
        func dt = new func(z);
        x(dt);
    }

    public static string z(int j)
    {
        Console.WriteLine(j);
        return j.ToString();
    }

    public static void x(func hello)
    {
        hello("8");    // CS1594
        // try the following line instead
        // hello(8);
    }
}
```

Compiler Error CS1597

Error Message

Semicolon after method or accessor block is not valid

Semicolons are not needed (or allowed) to end a method or accessor block.

The following sample generates CS1597:

```
// CS1597.cs
class TestClass
{
    public static void Main()
    {
    };    // CS1597, remove semicolon
}
```

Compiler Error CS1599

Error Message

Method or delegate cannot return type 'type'

Some types in the base class library, for example, [TypedReference](#) and [ArgIterator](#) cannot be used as return types.

The following sample generates CS1599:

```
// CS1599.cs
using System;

class MyClass
{
    public static void Main()
    {
    }

    public TypedReference Test1()    // CS1599
    {
        return null;
    }

    public ArgIterator Test2()      // CS1599
    {
        return null;
    }
}
```

Compiler Error CS1600

Error Message

Compilation cancelled by user

A compilation with the C# compiler was cancelled while using the Visual Studio IDE.

Compiler Error CS1601

Error Message

Method or delegate parameter cannot be of type 'type'

Some types in the base class library, for example, [TypedReference](#) and [ArgIterator](#) cannot be used as parameter types with the [ref](#) or [out](#) keywords.

The following sample generates CS1601:

```
// CS1601.cs
using System;

class MyClass
{
    public void Test1 (ref TypedReference t)    // CS1601
    {
    }

    public void Test2 (out ArgIterator t)      // CS1601
    {
    }
}
```


Compiler Error CS1604

Error Message

Cannot assign to 'variable' because it is read-only

An assignment was made to a read-only variable. To avoid this error, do not assign or attempt to modify to this variable in this context.

Compiler Error CS1605

Error Message

Cannot pass 'var' as a ref or out argument because it is read-only

A variable passed as a [ref](#) or [out](#) parameter is expected to be modified in the called method. Therefore, it is not possible to pass a read-only parameter as **ref** or **out**.

Compiler Error CS1606

Error Message

Assembly signing failed; output may not be signed -- reason

The assembly was produced, but when the compiler attempted to finish signing it, there was a failure.

Compiler Error CS1608

Error Message

The Required attribute is not permitted on C# types

[RequiredAttributeAttribute](#) is not allowed on types defined in C# code.

Compiler Error CS1609

Error Message

Modifiers cannot be placed on event accessor declarations

Modifiers can only be placed on event declarations and not on the event accessor declarations. For more information, see [Using Properties \(C# Programming Guide\)](#).

Example

The following sample generates CS1609.

```
// CS1609.cs
// compile with: /target:library
delegate int Del();
class A
{
    public event Del MyEvent
    {
        private add {}    // CS1609
        // try the following line instead
        // add {}
        remove {}
    }
}
```

Compiler Error CS1611

Error Message

The params parameter cannot be declared as ref or out

The keywords [ref](#) or [out](#) cannot be used with the [params](#) keyword.

The following sample generates CS1611:

```
// CS1611.cs
public class MyClass
{
    public static void Test(params ref int[] a)    // CS1611, remove ref
    {
    }

    public static void Main()
    {
        Test(1);
    }
}
```

Compiler Error CS1612

Error Message

Cannot modify the return value of 'expression' because it is not a variable

An attempt was made to modify a value type that was the result of an intermediate expression. Because the value is not persisted, the value will be unchanged.

To resolve this error, store the result of the expression in an intermediate value, or use a reference type for the intermediate expression.

Example

The following code generates error CS1612.

```
// CS1612.cs
public struct MyStruct
{
    public int Width;
}

public class ListView
{
    public MyStruct Size
    {
        get { return new MyStruct(); }
    }
}

public class MyClass
{
    public MyClass()
    {
        ListView lvi;
        lvi = new ListView();
        lvi.Size.Width = 33; // CS1612
        // Use this instead:
        // MyStruct temp = lvi.Size;
        // temp.Width = 33;
    }

    public static void Main() {}
}
```

Compiler Error CS1613

Error Message

The managed coclass wrapper class 'class' for interface 'interface' cannot be found (are you missing an assembly reference?)

An attempt was made to instantiate a COM object from an interface. The interface has the **ComImport** and **CoClass** attributes, but the compiler cannot find the type given for the **CoClass** attribute.

To resolve this error, you can try one of the following:

- Add a reference to the assembly that has the coclass (most of the time the interface and coclass should be in the same assembly). See [/reference](#) or [Add Reference Dialog Box](#) for information.
- Fix the **CoClass** attribute on the interface.

The following sample demonstrates correct usage of **CoClassAttribute**:

```
// CS1613.cs
using System;
using System.Runtime.InteropServices;

[Guid("1FFD7840-E82D-4268-875C-80A160C23296")]
[ComImport()]
[CoClass(typeof(A))]
public interface IA{}
public class A : IA {}

public class AA
{
    public static void Main()
    {
        IA i;
        i = new IA(); // This is equivalent to new A().
                     // because of the CoClass attribute on IA
    }
}
```


Compiler Error CS1614

Error Message

'attribute' is ambiguous; use either '@attribute' or 'attributeAttribute'

The compiler has encountered an ambiguous attribute specification.

For convenience, the C# compiler allows you to specify **ExampleAttribute** as just `[Example]`. However, ambiguity arises if an attribute class named **Example** exists along with **ExampleAttribute**, because the compiler cannot tell if `[Example]` refers to the **Example** attribute or the **ExampleAttribute** attribute. To clarify, use `[@Example]` for the **Example** attribute and `[ExampleAttribute]` for **ExampleAttribute**.

The following sample generates CS1614:

```
// CS1614.cs
using System;

// Both of the following classes are valid attributes with valid
// names (MySpecial and MySpecialAttribute). However, because the lookup
// rules for attributes involves auto-appending the 'Attribute' suffix
// to the identifier, these two attributes become ambiguous; that is,
// if you specify MySpecial, the compiler can't tell if you want
// MySpecial or MySpecialAttribute.

public class MySpecial : Attribute {
    public MySpecial() {}
}

public class MySpecialAttribute : Attribute {
    public MySpecialAttribute() {}
}

class MakeAWarning {
    [MySpecial()] // CS1614
                  // Ambiguous: MySpecial or MySpecialAttribute?
    public static void Main() {
    }

    [MySpecial()] // This isn't ambiguous, it binds to the first attribute above.
    public static void NoWarning() {
    }

    [MySpecialAttribute()] // This isn't ambiguous, it binds to the second attribute above.
    public static void NoWarning2() {
    }

    [MySpecialAttribute()] // This is also legal.
    public static void NoWarning3() {
    }
}
```

Compiler Error CS1615

Error Message

Argument 'number' should not be passed with the 'keyword' keyword

One of the keywords **ref** or **out** was used when the function did not take a **ref** or **out** parameter for that argument. To resolve this error, remove the incorrect keyword and use the appropriate keyword that matches the function declaration, if any.

The following sample generates CS1615:

```
// CS1615.cs
class C
{
    public void f(int i) {}
    public static void Main()
    {
        int i = 1;
        f(ref i); // CS1615
    }
}
```

Compiler Error CS1617

Error Message

Invalid option 'option' for /langversion; must be ISO-1 or Default

This error occurs if you used the [/langversion](#) command line switch or project setting but didn't specify a valid language option. To resolve this error, check the command line syntax or project setting and change it to one of the listed options.

For example, compiling with `csc /langversion:ISO` will generate error CS1617.

Compiler Error CS1618

Error Message

Cannot create delegate with 'method' because it has a Conditional attribute

You cannot create a delegate with a conditional method because the method might not exist in some builds.

The following sample generates CS1618:

```
// CS1618.cs
using System;
using System.Diagnostics;

delegate void del();

class MakeAnError {
    public static void Main() {
        del d = new del(ConditionalMethod);    // CS1618
        // Invalid because on builds where DEBUG is not set,
        // there will be no "ConditionalMethod".
    }
    // To fix the error, remove the next line:
    [Conditional("DEBUG")]
    public static void ConditionalMethod()
    {
        Console.WriteLine("Do something only in debug");
    }
}
```

Compiler Error CS1619

Error Message

Cannot create temporary file 'filename' -- reason

The compiler was unable to create a temporary file for the given reason (for example, the disk is full).

Compiler Error CS1620

Error Message

Argument 'number' must be passed with the 'keyword' keyword

This error occurs if you are passing an argument to a function that takes a [ref](#) or [out](#) parameter and you don't include the **ref** or **out** keyword at the point of call, or you include the wrong keyword. The error text indicates the appropriate keyword to use and which argument caused the failure.

The following sample generates CS1620:

```
// CS1620.cs
class C
{
    void f(ref int i) {}
    public static void Main()
    {
        int x = 1;
        f(out x); // CS1620 - f takes a ref parameter, not an out parameter
        // Try this line instead:
        // f(ref x);
    }
}
```

Compiler Error CS1621

Error Message

The yield statement can not be used inside anonymous method blocks

The yield statement cannot be in an anonymous method block in an iterator.

Example

The following sample generates CS1621:

```
// CS1621.cs

using System.Collections;

delegate object MyDelegate();

class C : IEnumerable
{
    public IEnumerator GetEnumerator()
    {
        MyDelegate d = delegate
        {
            yield return this; // CS1621
            return this;
        };
        d();
        // Try this instead:
        // MyDelegate d = delegate { return this; };
        // yield return d();
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1622

Error Message

Cannot return a value from an iterator. Use the yield return statement to return a value, or yield break to end the iteration.

An iterator is a special function that returns a value via the yield statement rather than the return statement. For more information, see [iterators](#).

The following sample generates CS1622:

```
// CS1622.cs
// compile with: /target:library
using System.Collections;

class C : IEnumerable
{
    public IEnumerator GetEnumerator()
    {
        return (IEnumerator) this; // CS1622
        yield return this; // OK
    }
}
```


Compiler Error CS1623

Error Message

Iterators cannot have ref or out parameters

This error occurs if an iterator method takes a **ref** or **out** parameter. To avoid this error, remove the **ref** or **out** keyword from the method signature.

Example

The following sample generates CS1623:

```
// CS1623.cs
using System.Collections;

class C : IEnumerable
{
    public IEnumerator GetEnumerator()
    {
        yield return 0;
    }

    // To resolve the error, remove ref
    public IEnumerator GetEnumerator(ref int i) // CS1623
    {
        yield return i;
    }

    // To resolve the error, remove out
    public IEnumerator GetEnumerator(out float f) // CS1623
    {
        f = 0.0F;
        yield return f;
    }
}
```

Compiler Error CS1624

Error Message

The body of 'accessor' cannot be an iterator block because 'type' is not an iterator interface type

This error occurs if an iterator accessor is used but the return type is not one of the iterator interface types: [IEnumerable](#), [IEnumerable](#), [IEnumerator](#), [IEnumerator](#). To avoid this error, use one of the iterator interface types as a return type.

Example

The following sample generates CS1624:

```
// CS1624.cs
using System;
using System.Collections;

class C
{
    public int Iterator
    // Try this instead:
    // public IEnumerable Iterator
    {
        get // CS1624
        {
            yield return 1;
        }
    }
}
```

Compiler Error CS1625

Error Message

Cannot yield in the body of a finally clause

A yield statement is not allowed in the body of a finally clause. To avoid this error, move the yield statement out of the finally clause.

The following sample generates CS1625:

```
// CS1625.cs
using System.Collections;

class C : IEnumerable
{
    public IEnumerator GetEnumerator()
    {
        try
        {
        }
        finally
        {
            yield return this; // CS1625
        }
    }
}

public class CMain
{
    public static void Main() { }
}
```

Compiler Error CS1626

Error Message

Cannot yield a value in the body of a try block with a catch clause

A yield statement is not allowed in a try block if there is a catch clause associated with the try block. To avoid this error, move the yield statement out of the catch clause.

The following sample generates CS1626:

```
// CS1626.cs
using System.Collections;

class C : IEnumerable
{
    public IEnumerator GetEnumerator()
    {
        try
        {
            yield return this; // CS1626
        }
        catch
        {
        }
    }
}

public class CMain
{
    public static void Main() { }
}
```

Compiler Error CS1627

Error Message

Expression expected after yield return

This error occurs if **yield** is used without an expression. To avoid this error, insert the appropriate expression in the statement.

The following sample generates CS1627:

```
// CS1627.cs
using System.Collections;

class C : IEnumerable
{
    public IEnumerator GetEnumerator()
    {
        yield return;    // CS1627
        // To resolve, add the following line:
        // yield return 0;
    }
}

public class CMain
{
    public static void Main() { }
}
```

Compiler Error CS1628

Error Message

Cannot use ref or out parameter 'parameter' inside an anonymous method block

This error occurs if you use a ref or out parameter inside an anonymous method block. To avoid this error, use a local variable or some other construct.

The following sample generates CS1628:

```
// CS1628.cs

delegate int MyDelegate();

class C
{
    public static void F(ref int i)
    {
        MyDelegate d = delegate { return i; }; // CS1628
        // Try this instead:
        // int tmp = i;
        // MyDelegate d = delegate { return tmp; };
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1629

Error Message

Unsafe code may not appear in iterators

The C# language specification does not allow unsafe code in iterators.

The following sample generates CS1629:

```
// CS1629.cs
// compile with: /unsafe
using System.Collections.Generic;
class C
{
    IEnumerator<int> IteratorMeth() {
        int i;
        unsafe // CS1629
        {
            int *p = &i;
            yield return *p;
        }
    }
}
```

Compiler Error CS1630

Error Message

Invalid option 'option' for /errorreport; must be prompt, send, queue, or none

The command line option [/errorreport](#) must be followed by **prompt**, **send**, **queue**, or **none**, specifying what action you want taken when an internal compiler error occurs.

Compiler Error CS1631

Error Message

Cannot yield a value in the body of a catch clause

The yield statement is not allowed from within the body of a catch clause. To avoid this error, move the yield statement outside the body of the catch clause.

The following sample generates CS1631:

```
// CS1631.cs
using System;
using System.Collections;

public class C : IEnumerable
{
    public IEnumerator GetEnumerator()
    {
        try
        {
        }
        catch(Exception e)
        {
            yield return this; // CS1631
        }
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1632

Error Message

Control cannot leave the body of an anonymous method

This error occurs if a jump statement (**break**, **goto**, **continue**, etc.) attempts to move control out of an anonymous method block. An anonymous method block is a function body and can only be exited by a return statement or by reaching the end of the block.

The following sample generates CS1632:

```
// CS1632.cs
// compile with: /target:library
delegate void MyDelegate();
class MyClass
{
    public void Test()
    {
        for (int i = 0 ; i < 5 ; i++)
        {
            MyDelegate d = delegate {
                break;    // CS1632
            };
        }
    }
}
```

Compiler Error CS1637

Error Message

Iterators cannot have unsafe parameters or yield types

Check the argument list of the iterator and the type of any yield statements to verify that you are not using any unsafe types.

Example

The following sample generates CS1637:

```
// CS1637.cs
// compile with: /unsafe
using System.Collections;

public unsafe class C
{
    public IEnumerator Iterator1(int* p) // CS1637
    {
        yield return null;
    }
}
```

Compiler Error CS1638

Error Message

'identifier' is a reserved identifier and cannot be used when ISO language version mode is used

When the ISO language compatibility option is specified by the **/langversion** compiler switch, any identifier with double underscores anywhere in the identifier will produce this error. To avoid this error, eliminate any identifiers with double underscores, or do not use the ISO-1 language version option.

Example

The following sample generates CS1638:

```
// CS1638.cs
// compile with: /langversion:ISO-1
class bad__identifer // CS1638 (double underscores are not ISO compliant)
{
}

// Try this instead:
//class GoodIdentifier
//{
//}

class CMain
{
    public static void Main() { }
}
```

See Also

Reference

[/langversion \(Conformant Syntax\) \(C# Compiler Options\)](#)

Compiler Error CS1639

Error Message

The managed coclass wrapper class signature 'signature' for interface 'interface' is not a valid class name signature

This error occurs if the metadata for the interface is invalid. This should never occur if the interface was generated using C# or a supported .NET language. Check with the vendor of the interface or verify that the interface assembly was generated successfully.

Compiler Error CS1640

Error Message

foreach statement cannot operate on variables of type 'type' because it implements multiple instantiations of 'interface', try casting to a specific interface instantiation

The type inherits from two or more instances of `IEnumerator<T>`, which means there is not a unique enumeration of the type that **foreach** could use. Specify the type of `IEnumerator<T>` or use another looping construct.

Example

The following sample generates CS1640:

```
// CS1640.cs

using System;
using System.Collections;
using System.Collections.Generic;

public class C : IEnumerable, IEnumerable<int>, IEnumerable<string>
{
    IEnumerator<int> IEnumerable<int>.GetEnumerator()
    {
        yield break;
    }

    IEnumerator<string> IEnumerable<string>.GetEnumerator()
    {
        yield break;
    }

    IEnumerator IEnumerable.GetEnumerator()
    {
        return (IEnumerator)((IEnumerable<string>)this).GetEnumerator();
    }
}

public class Test
{
    public static int Main()
    {
        foreach (int i in new C()){}    // CS1640

        // Try specifying the type of IEnumerable<T>
        // foreach (int i in (IEnumerable<int>)new C()){
        return 1;
    }
}
```

Compiler Error CS1641

Error Message

A fixed size buffer field must have the array size specifier after the field name

Unlike regular arrays, fixed size buffers require a constant size to be specified at the declaration point. To resolve this error, add a positive integer literal or a constant positive integer and put the square brackets after the identifier.

The following sample generates CS1641:

```
// CS1641.cs
// compile with: /unsafe /target:library
unsafe struct S {
    fixed int [] a; // CS1641

    // OK
    fixed int b [10];
    const int c = 10;
    fixed int d [c];
}
```

Compiler Error CS1642

Error Message

Fixed size buffer fields may only be members of structs.

This error occurs if you use a fixed size buffer field in a **class**, instead of a **struct**. To resolve this error, change the **class** to a **struct** or declare the field as an ordinary array.

Example

The following sample generates CS1642.

```
// CS1642.cs
// compile with: /unsafe /target:library
unsafe class C
{
    fixed int a[10];    // CS1642
}

unsafe struct D
{
    fixed int a[10];
}

unsafe class E
{
    public int[] a = null;
}
```


Compiler Error CS1643

Error Message

Anonymous method of type 'type': not all code paths return a value

This error occurs if a delegate body does not have a return statement, or has a return statement that the compiler is unable to verify will be reached. In the example below, the compiler does not attempt to predict the result of the branching condition in order to verify that the anonymous method block always returns a value.

Example

The following sample generates CS1643:

```
// CS1643.cs
delegate int MyDelegate();

class C
{
    static void Main()
    {
        MyDelegate d = delegate
        {
            int i = 0;
            if (i == 0)
                return 1;
        };
    }
}
```

Compiler Error CS1644

Error Message

Feature 'feature' cannot be used because it is not part of the standardized ISO C# language specification

This error occurs if you specified the [/langversion](#) option ISO-1 and the code you are compiling uses features that are not part of the ISO 1.0 standard. To resolve this error, do not use any of the new C# compiler features with the ISO-1 compatibility option. See [What's New in the C# 2.0 Language and Compiler](#) for a list of the new C# compiler features.

The following sample generates CS1644:

```
// CS1644.cs
// compile with: /langversion:ISO-1 /target:library
class C<T> {}    // CS1644
```

Compiler Error CS1646

Error Message

Keyword, identifier, or string expected after verbatim specifier: @

See string literals for the usage of the verbatim specifier '@'. The verbatim specifier is only allowed before a string, keyword or identifier. To resolve this error, remove the @ symbol from any inappropriate place or add the intended string, keyword or identifier.

The following sample generates CS1646:

```
// CS1646
class C
{
    int i = @5; // CS1646
    // Try this line instead:
    // int i = 5;
}
```

Compiler Error CS1647

Error Message

An expression is too long or complex to compile near 'code'

There was a stack overflow in the compiler processing your code. To resolve this error, simplify your code. If your code is valid, contact [Product Support](#).

Compiler Error CS1648

Error Message

Members of readonly field 'identifier' cannot be modified (except in a constructor or a variable initializer)

This error occurs when you attempt to modify a member of a field which is readonly where it is not allowed to be modified. To resolve this error, limit assignments to readonly fields to the constructor or variable initializer, or remove the readonly keyword from the declaration of the field.

The following sample generates CS1648:

```
// CS1648.cs
public struct Inner
{
    public int i;
}

class Outer
{
    public readonly Inner inner = new Inner();
}

class D
{
    static void Main()
    {
        Outer outer = new Outer();
        outer.inner.i = 1; // CS1648
    }
}
```

Compiler Error CS1649

Error Message

Members of readonly field 'identifier' cannot be passed ref or out (except in a constructor)

This error occurs if you pass a variable to a function that is a member of a **readonly** field as a **ref** or **out** argument. Since **ref** and **out** parameters may be modified by the function, this is not allowed. To resolve this error, remove the **readonly** keyword on the field, or do not pass the members of the **readonly** field to the function. For example, you might try creating a temporary variable which can be modified and passing the temporary as a **ref** argument, as shown in the following example.

Example

The following sample generates CS1649:

```
// CS1649.cs
public struct Inner
{
    public int i;
}

class Outer
{
    public readonly Inner inner = new Inner();
}

class D
{
    static void f(ref int iref)
    {
    }

    static void Main()
    {
        Outer outer = new Outer();
        f(ref outer.inner.i); // CS1649
        // Try this code instead:
        // int tmp = outer.inner.i;
        // f(ref tmp);
    }
}
```

Compiler Error CS1650

Error Message

Fields of static readonly field 'identifier' cannot be assigned to (except in a static constructor or a variable initializer)

This error occurs when you attempt to modify a member of a field which is readonly and static where it is not allowed to be modified. To resolve this error, limit assignments to readonly fields to the constructor or variable initializer, or remove the **readonly** keyword from the declaration of the field.

```
// CS1650.cs
public struct Inner
{
    public int i;
}

class Outer
{
    public static readonly Inner inner = new Inner();
}

class D
{
    static void Main()
    {
        Outer.inner.i = 1;  // CS1650
    }
}
```

Compiler Error CS1651

Error Message

Fields of static readonly field 'identifier' cannot be passed ref or out (except in a static constructor)

This error occurs if you pass a variable to a function that is a member of a static readonly field as a ref argument. Since ref parameters may be modified by the function, this is not allowed. To resolve this error, remove the **readonly** keyword on the field, or do not pass the members of the readonly field to the function. For example, you might try creating a temporary variable which can be modified and passing the temporary as a ref argument, as shown in the following example.

The following sample generates CS1651:

```
// CS1651.cs
public struct Inner
{
    public int i;
}

class Outer
{
    public static readonly Inner inner = new Inner();
}

class D
{
    static void f(ref int iref)
    {
    }

    static void Main()
    {
        f(ref Outer.inner.i); // CS1651
        // Try this instead:
        // int tmp = Outer.inner.i;
        // f(ref tmp);
    }
}
```


Compiler Error CS1652

Error Message

Cannot modify members of 'identifier' because it is read-only

This error occurs when you attempt to modify members of a variable which is read-only because of its context.

Compiler Error CS1653

Error Message

Cannot pass fields of 'identifier' as a ref or out argument because it is read-only

This error occurs when you have a variable which is read-only because of its context and you attempt to pass members of this variable as a ref or out parameter.

Compiler Error CS1654

Error Message

Cannot modify members of 'variable' because it is a 'read-only variable type'

This error occurs when you attempt to modify members of a variable which is read-only because it is in a special construct.

Example

The following example generates error CS1654.

```
// CS1654.cs
using System;
using System.Collections;

public struct Test : IEnumerable
{
    private int index;
    public int Index
    {
        get { return index; }
        set { index = value; }
    }

    public IEnumerator GetEnumerator()
    {
        for(int i = 0; i < 10; i++)
            yield return this;
        yield break;
    }
}

public class Repro
{
    static int Main()
    {
        int i = 0;
        Test t = new Test();
        foreach (Test tt in t)
        {
            tt.Index = i++;    // CS1654
        }
        return 1;
    }
}
```

Compiler Error CS1655

Error Message

Cannot pass fields of 'variable' as a ref or out argument because it is a 'readonly variable type'

This error occurs if you are attempting to pass a member of a [foreach](#) variable, a [using](#) variable, or a [fixed](#) variable to a function as a ref or out argument. Because these variables are considered read-only in these contexts, this is not allowed.

The following sample generates CS1655:

```
// CS1655.cs
struct S
{
    public int i;
}

class CMain
{
    static void f(ref int iref)
    {
    }

    public static void Main()
    {
        S[] sa = new S[10];
        foreach(S s in sa)
        {
            CMain.f(ref s.i); // CS1655
        }
    }
}
```

Compiler Error CS1656

Error Message

Cannot assign to 'variable' because it is a 'read-only variable type'

This error occurs when an assignment to variable occurs in a read-only context. Read-only contexts include [foreach](#) iteration variables, [using](#) variables, and [fixed](#) variables. To resolve this error, avoid assignments to a statement variable in **using** blocks, **foreach** statements, and **fixed** statements.

Example

The following sample generates CS1656:

```
// CS1656.cs
// compile with: /unsafe
using System;

class C : IDisposable
{
    public void Dispose() { }
}

class CMain
{
    unsafe public static void Main()
    {
        using (C c = new C())
        {
            c = new C(); // CS1656
        }

        foreach (object o in new string[] { "1", "2" })
        {
            o = "10"; // CS1656
        }

        int[] ary = new int[] { 1, 2, 3, 4 };
        fixed (int* p = ary)
        {
            p = null; // CS1656
        }
    }
}
```

Compiler Error CS1657

Error Message

Cannot pass 'variable' as a ref or out argument because it a 'readonly variable type'

This error occurs when a variable is passed as a [ref](#) or [out](#) argument in a context in which that variable is readonly. Readonly contexts include [foreach](#) iteration variables, [using](#) variables, and **fixed** variables. To resolve this error, do not call functions that take the **foreach**, **using** or **fixed** variable as a **ref** or **out** parameter in **using** blocks, **foreach** statements, and **fixed** statements.

Example

The following sample generates CS1657:

```
// CS1657.cs
using System;
class C : IDisposable
{
    public int i;
    public void Dispose() {}
}

class CMain
{
    static void f(ref C c)
    {
    }
    static void Main()
    {
        using (C c = new C())
        {
            f(ref c); // CS1657
        }
    }
}
```

The following code illustrates the same problem in a **fixed** statement:

```
// CS1657b.cs
// compile with: /unsafe
unsafe class C
{
    static void F(ref int* p)
    {
    }

    static void Main()
    {
        int[] a = new int[5];
        fixed(int* p = a) F(ref p); // CS1657
    }
}
```

Compiler Error CS1660

Error Message

Cannot convert anonymous method block to type 'type' because it is not a delegate type

This error occurs if you try to assign or otherwise convert an anonymous method block to a type which is not a delegate type.

The following sample generates CS1660:

```
// CS1660.cs
delegate int MyDelegate();
class C {
    static void Main()
    {
        int i = delegate { return 1; }; // CS1660
        // Try this instead:
        // MyDelegate myDelegate = delegate { return 1; };
        // int i = myDelegate();
    }
}
```

Compiler Error CS1661

Error Message

Cannot convert anonymous method block to delegate type 'delegate type' because the specified block's parameter types do not match the delegate parameter types

This error occurs if, in an anonymous method definition, the parameter types of the anonymous method do not match the delegate parameter types. Check the number of parameters, the parameter types, and any ref or out parameters and verify an exact match.

The following sample generates CS1661:

```
// CS1661.cs

delegate void MyDelegate(int i);

class C
{
    public static void Main()
    {
        MyDelegate d = delegate(string s) { }; // CS1661
    }
}
```


Compiler Error CS1662

Error Message

Cannot convert anonymous method block to delegate type 'delegate type' because some of the return types in the block are not implicitly convertible to the delegate return type

This error occurs if the anonymous method block's return statement had a type that was not implicitly convertible to the return type of the delegate.

The following sample generates CS1662:

```
// CS1662.cs

delegate int MyDelegate(int i);

class C
{
    public static void Main()
    {
        MyDelegate d = delegate(int i) { return 1.0; }; // CS1662
        // Try this instead:
        // MyDelegate d = delegate(int i) { return (int)1.0; };
    }
}
```

Compiler Error CS1663

Error Message

Fixed size buffer type must be one of the following: bool, byte, short, int, long, char, sbyte, ushort, uint, ulong, float or double

A fixed sized buffer may not be any type other than those listed. To avoid this error, use another type or do not use a fixed array.

Example

The following sample generates CS1663.

```
// CS1663.cs
// compile with: /unsafe /target:library

unsafe struct C
{
    fixed string ab[10];    // CS1663
}
```

Compiler Error CS1664

Error Message

Fixed size buffer of length 'length' and type 'type' is too big

The maximum size of a fixed-size buffer (as determined by the length multiplied by the element size) is $2^{31} = 268435455$.

Compiler Error CS1665

Error Message

Fixed size buffers must have a length greater than zero

This error occurs if a fixed size buffer is declared with a zero or negative size. The length of a fixed size buffer must be a positive integer.

Example

The following sample generates CS1665.

```
// CS1665.cs
// compile with: /unsafe /target:library
struct S
{
    public unsafe fixed int A[0];    // CS1665
}
```

Compiler Error CS1666

Error Message

You cannot use fixed size buffers contained in unfixed expressions. Try using the fixed statement.

This error occurs if you use the fixed sized buffer in an expression involving a class that is not itself fixed. The runtime is free to move the unfixed class around to optimize memory access, which could lead to errors when using the fixed sized buffer. To avoid this error, use the **fixed** keyword on the statement.

Example

The following sample generates CS1666.

```
// CS1666.cs
// compile with: /unsafe /target:library
unsafe struct S
{
    public fixed int buffer[1];
}

unsafe class Test
{
    S field = new S();

    private bool example1()
    {
        return (field.buffer[0] == 0);    // CS1666 error
    }

    private bool example2()
    {
        // OK
        fixed (S* p = &field)
        {
            return (p->buffer[0] == 0);
        }
    }

    private bool example3()
    {
        S local = new S();
        return (local.buffer[0] == 0);
    }
}
```

Compiler Error CS1667

Error Message

Attribute 'attribute' is not valid on property or event accessors. It is valid on 'declaration type' declarations only.

This error occurs if you use an attribute on a property or event accessor, when it should be on the property or event itself. This error could occur with the attributes [CLSCompliantAttribute](#), [ConditionalAttribute](#), and [ObsoleteAttribute](#).

Example

The following sample generates CS1670:

```
// CS1667.cs
using System;

public class C
{
    private int i;

    //Try this instead:
    //[Obsolete]
    public int ObsoleteProperty
    {
        [Obsolete] // CS1667
        get { return i; }
        set { i = value; }
    }

    public static void Main()
    {
    }
}
```

Compiler Error CS1670

Error Message

params is not valid in this context

A number of C# features are incompatible with variable argument lists, and do not allow the **params** keyword, including the following:

- Parameter lists of anonymous methods
- Overloaded operators

Example

The following sample generates CS1670:

```
// CS1670.cs
public class C
{
    public bool operator +(params int[] paramsList) // CS1670
    {
        return false;
    }

    static void Main()
    {
    }
}
```

Compiler Error CS1671

Error Message

A namespace declaration cannot have modifiers or attributes

Modifiers are not meaningful when applied to a namespace, so they are not allowed.

The following sample generates CS1671:

```
// CS1671.cs
public namespace NS // CS1671
{

}
```


Compiler Error CS1672

Error Message

Invalid option 'option' for /platform; must be anycpu, x86, Itanium or x64

The options specify the processor type; use one of the forms listed.

Compiler Error CS1673

Error Message

Anonymous methods inside structs cannot access instance members of 'this'. Consider copying 'this' to a local variable outside the anonymous method and using the local instead

The following sample generates CS1673:

```
// CS1673.cs
delegate int MyDelegate();

public struct S
{
    int member;

    public int F(int i)
    {
        member = i;
        // Try assigning to a local variable
        // S s = this;
        MyDelegate d = delegate()
        {
            i = this.member; // CS1673
            // And use the local variable instead of "this"
            // i = s.member;
            return i;
        };
        return d();
    }
}

class CMain
{
    public static void Main()
    {
    }
}
```

Compiler Error CS1674

Error Message

'T': type used in a using statement must be implicitly convertible to 'System.IDisposable'

The [using Statement](#) is intended to be used to ensure the disposal of an object at the end of the **using** block, thus, only types which are disposable may be used in such a statement. For example, value types are not disposable, and type parameters which are not constrained to be classes may not be assumed to be disposable.

Example

The following sample generates CS1674.

```
// CS1674.cs
class C
{
    public static void Main()
    {
        int a = 0;
        a++;

        using (a) {}    // CS1674
    }
}
```

The following sample generates CS1674.

```
// CS1674_b.cs
using System;
class C {
    public void Test() {
        using (C c = new C()) {}    // CS1674
    }
}

// OK
class D : IDisposable {
    void IDisposable.Dispose() {}
    public void Dispose() {}

    public static void Main() {
        using (D d = new D()) {}
    }
}
```

The following case illustrates the need for a class type constraint to guarantee that an unknown type parameter is disposable. The following sample generates CS1674.

```
// CS1674_c.cs
// compile with: /target:library
using System;
public class C<T>
// Add a class type constraint that specifies a disposable class.
// Uncomment the following line to resolve.
// public class C<T> where T : IDisposable
{
    public void F(T t)
    {
        using (t) {}    // CS1674
    }
}
```

Compiler Error CS1675

Error Message

Enums cannot have type parameters

To resolve this error, remove the type parameter from the **enum** declaration.

Example

The following sample generates CS1675:

```
// CS1675.cs
enum E<T> // CS1675
{
}

class CMain
{
    public static void Main()
    {
    }
}
```

Compiler Error CS1676

Error Message

Parameter 'number' must be declared with the 'keyword' keyword

This error occurs when the parameter type modifier in an anonymous method is different from that used in the declaration of the delegate you are casting the method to.

The following sample generates CS1676:

```
// CS1676.cs
delegate void E(ref int i);
class Errors
{
    static void Main()
    {
        E e = delegate(out int i) { };    // CS1676
        // To resolve, use the following line instead:
        // E e = delegate(ref int i) { };
    }
}
```

Compiler Error CS1677

Error Message

Parameter 'number' should not be declared with the 'keyword' keyword

This error occurs when the parameter type modifier in an anonymous method does not match that used in the declaration of the delegate, to which you are casting the method.

Example

The following sample generates CS1677:

```
// CS1677.cs
delegate void D(int i);
class Errors
{
    static void Main()
    {
        D d = delegate(out int i) { };    // CS1677
        // To resolve, use the following line instead:
        // D d = delegate(int i) { };

        D d = delegate(ref int j){}; // CS1677
        // To resolve, use the following line instead:
        // D d = delegate(int j){};
    }
}
```

Compiler Error CS1678

Error Message

Parameter 'number' is declared as type 'type1' but should be 'type2'

This error occurs when the parameter type in an anonymous method is different from the declaration of the delegate you are casting the method to.

The following sample generates CS1678:

```
// CS1678
delegate void D(int i);
class Errors
{
    static void Main()
    {
        D d = delegate(string s) { };    // CS1678
        // To resolve, use the following line instead:
        // D d = delegate(int s) { };
    }
}
```

Compiler Error CS1679

Error Message

Invalid extern alias for '/reference'; 'identifier' is not a valid identifier

When using the external assembly alias feature of the **/reference** option, the text that follows **/reference:** and that precedes the '=' must be a valid C# identifier or keyword according to the C# Language Specification.

To correct this error, change text before the "=" to a valid C# identifier or keyword.

Example

The following example generates CS1679.

```
// CS1679.cs
// compile with: /reference:123$BadIdentifier%=System.dll
class TestClass {
    static void Main()
    {
    }
}
```


Compiler Error CS1680

Error Message

Invalid reference alias option: 'alias=' -- missing filename.

This error occurs when you use the `alias` feature with the **/reference** compiler option without specifying a valid file name.

The following sample generates CS1680.

```
// CS1680.cs
// compile with: /reference:alias=
// CS1680 expected
// To resolve, specify the name of a file with an assembly manifest
class MyClass {}
```

Compiler Error CS1681

Error Message

You cannot redefine the global extern alias

The global alias is already defined to include all unaliased references and therefore cannot be redefined.

Example

The following sample generates CS1681.

```
// CS1681.cs
// compile with: /reference:global=System.dll
// CS1681 expected

// try this instead: /reference:System.dll
class A
{
    static void Main() {}
}
```

Compiler Error CS1686

Error Message

Local 'variable' or its members cannot have their address taken and be used inside an anonymous method block

This error is generated when you use a variable, and attempt to take its address, and one of these actions is done inside an anonymous method.

Example

The following sample generates CS1686.

```
// CS1686.cs
// compile with: /unsafe /target:library
class MyClass
{
    public unsafe delegate int * MyDelegate();

    public unsafe int * Test()
    {
        int j = 0;
        MyDelegate d = delegate { return &j; };    // CS1686
        return &j;    // OK
    }
}
```

Compiler Error CS1688

Error Message

Cannot convert anonymous method block without a parameter list to delegate type 'delegate' because it has one or more out parameters

The compiler allows parameters to be omitted from an anonymous method block in most cases. This error arises when the anonymous method block does not have a parameter list, but the delegate has an *out* parameter. The compiler does not allow this situation because it would need to ignore the presence of the *out* parameter, which is unlikely to be the correct behavior.

Example

The following code generates error CS1688.

```
// CS1688.cs
using System;
delegate void OutParam(out int i);
class ErrorCS1676
{
    static void Main()
    {
        OutParam o;
        o = delegate // CS1688
            // Try this instead:
            // o = delegate(out int i)
            {
                Console.WriteLine("");
            };
    }
}
```

Compiler Error CS1689

Error Message

Attribute 'Attribute Name' is only valid on methods or attribute classes

This error only occurs with the **ConditionalAttribute** attribute. As the message states, this attribute can only be used on methods or attribute classes. For example, trying to apply this attribute to a class will generate this error.

Example

The following sample generates CS1689.

```
// CS1689.cs
// compile with: /target:library
[System.Diagnostics.Conditional("A")]    // CS1689
class MyClass {}
```

Compiler Warning (Level 1) CS1690

Error Message

Accessing a member on 'member' may cause a runtime exception because it is a field of a marshal-by-reference class

This warning occurs when you try to call a method, property, or indexer on a member of a class that derives from [MarshalByRefObject](#), and the member is a value type. To resolve the warning, copy the member into a local variable and call the method on that variable.

The following sample generates CS1690:

```
// CS1690.cs
using System;

class WarningCS1690: MarshalByRefObject
{
    int i = 5;

    public static void Main()
    {
        WarningCS1690 e = new WarningCS1690();
        e.i.ToString();    // CS1690

        // OK
        int i = e.i;
        i.ToString();
        e.i = i;
    }
}
```

Compiler Error CS1703

Error Message

An assembly with the same identity "Assembly Name" has already been imported. Try removing one of the duplicate references.

The compiler removes references that have the same path and file name, but it is possible that the same file exists in two places, or that you forgot to change the version number. This error points out that two references have the same assembly identity and thus the compiler has no way of distinguishing between them in metadata. Either remove one of the redundant references, or make the references unique somehow, such as by incrementing the assembly version number.

The following code generates error CS1703.

Example

This code creates assembly A in the .\bin1 directory.

Save this example in a file named CS1703a1.cs, and compile it with the following flags: `/t:library /out:.\bin1\cs1703.dll /keyfile:key.snk`

```
using System;
public class A { }
```

This code creates a copy of assembly A in the .\bin2 directory.

Save this example in a file named CS1703a2.cs, and compile it with the following flags: `/t:library /out:.\bin2\cs1703.dll /keyfile:key.snk`

```
using System;
public class A { }
```

This code references the assembly A in the two prior modules.

Save this example in a file named CS1703ref.cs, and compile it with the following flags: `/t:library /r:A2=.\bin2\cs1703.dll /r:A1=.\bin1\cs1703.dll`

```
extern alias A1;
extern alias A2;
```

Compiler Error CS1704

Error Message

An assembly with the same simple name 'Assembly Name' has already been imported. Try removing one of the references or sign them to enable side-by-side.

This error points out that two references have the same assembly identity because the assemblies in question lack strong names, they were not signed, and thus the compiler has no way of distinguishing between them in metadata. Thus, the run time ignores the version and culture assembly name properties. The user should remove the redundant reference, rename one of the references, or provide a strong name for them.

Example

This sample creates an assembly and saves it to the root directory.

```
// CS1704_a.cs
// compile with: /target:library /out:c:\\cs1704.dll
public class A {}
```

This sample creates an assembly with the same name as the previous sample, but saves it to a different location.

```
// CS1704_b.cs
// compile with: /target:library /out:cs1704.dll
public class A {}
```

This sample attempts to reference both assemblies. The following sample generates CS1704.

```
// CS1704_c.cs
// compile with: /target:library /r:A2=cs1704.dll /r:A1=c:\\cs1704.dll
// CS1704 expected
extern alias A1;
extern alias A2;
```


Compiler Error CS1705

Error Message

Assembly 'AssemblyName1' uses 'TypeName' which has a higher version than referenced assembly 'AssemblyName2'

You are referencing a type that has a higher version number than the version number in a referenced assembly.

For example, you have two assemblies, A and B. A references a class `myClass` that was added to assembly B in version 2.0. But the reference to assembly B specifies version 1.0. The compiler has unification rules for binding references, and a reference to version 2.0 cannot be satisfied by version 1.0.

Example

This sample consists of four code modules:

- Two DLLs that are identical except for a version attribute.
- A DLL that references them.
- A client.

The following is the first of the identical DLLs.

```
// CS1705_a.cs
// compile with: /target:library /out:c:\\cs1705.dll /keyfile:mykey.snk
[assembly:System.Reflection.AssemblyVersion("1.0")]
public class A
{
    public void M1() {}
    public class N1 {}
    public void M2() {}
    public class N2 {}
}

public class C1 {}
public class C2 {}
```

The following is version 2.0 of the assembly, as specified by the [AssemblyVersionAttribute](#) attribute.

```
// CS1705_b.cs
// compile with: /target:library /out:cs1705.dll /keyfile:mykey.snk
using System.Reflection;
[assembly:AssemblyVersion("2.0")]
public class A
{
    public void M2() {}
    public class N2 {}
    public void M1() {}
    public class N1 {}
}

public class C2 {}
public class C1 {}
```

Save this example in a file named `CS1705ref.cs` and compile it with the following flags: `/t:library /r:A2=.\\bin2\\CS1705a.dll /r:A1=.\\bin1\\CS1705a.dll`

```
// CS1705_c.cs
// compile with: /target:library /r:A2=c:\\CS1705.dll /r:A1=CS1705.dll
extern alias A1;
extern alias A2;
using a1 = A1::A;
using a2 = A2::A;
using n1 = A1::A.N1;
```

```

using n2 = A2::A.N2;
public class Ref
{
    public static a1 A1() { return new a1(); }
    public static a2 A2() { return new a2(); }
    public static A1::C1 M1() { return new A1::C1(); }
    public static A2::C2 M2() { return new A2::C2(); }
    public static n1 N1() { return new a1.N1(); }
    public static n2 N2() { return new a2.N2(); }
}

```

The following sample references version 1.0 of the CS1705.dll assembly. But the statement `Ref.A2().M2()` references the A2 method in the class in CS1705_c.dll, which returns an a2, which is aliased to A2::A, and A2 references version 2.0 via an **extern** statement, thus causing a version mismatch.

The following sample generates CS1705.

```

// CS1705_d.cs
// compile with: /reference:c:\\CS1705.dll /reference:CS1705_c.dll
// CS1705 expected
class Tester
{
    static void Main()
    {
        Ref.A1().M1();
        Ref.A2().M2();
    }
}

```

Compiler Error CS1706

Error Message

Expression cannot contain anonymous methods

You cannot insert an anonymous method inside an expression.

To correct this error

- Use a regular **delegate** in the expression.

Example

The following example generates CS1706.

```
// CS1706.cs
using System;

delegate void MyDelegate();
class MyAttribute : Attribute
{
    public MyAttribute(MyDelegate d) { }
}

// Anonymous Method in Attribute declaration is not allowed.
[MyAttribute(delegate{/* anonymous Method in Attribute declaration */})] // CS1706
class Program
{
}
```

Compiler Error CS1708

Error Message

Fixed size buffers can only be accessed through locals or fields

A new feature in C# 2.0 is the ability to define an in-line array inside of a **struct**. Such arrays can only be accessed via local variables or fields, and may not be referenced as intermediate values on the left-hand side of an expression. Also, the arrays cannot be accessed by fields that are **static** or **readonly**.

To resolve this error, define an array variable, and assign the in-line array to the variable. Or, remove the **static** or **readonly** modifier from the field representing the in-line array.

Example

The following sample generates CS1708.

```
// CS1708.cs
// compile with: /unsafe
using System;

unsafe public struct Foo
{
    public fixed char name[10];
}

public unsafe class C
{
    public Foo UnsafeMethod()
    {
        Foo myFoo = new Foo();
        return myFoo;
    }

    static void Main()
    {
        C myC = new C();
        myC.UnsafeMethod().name[3] = 'a'; // CS1708
        // Uncomment the following 2 lines to resolve:
        // Foo myFoo = myC.UnsafeMethod();
        // myFoo.name[3] = 'a';

        // The field cannot be static.
        C._foo1.name[3] = 'a'; // CS1708

        // The field cannot be readonly.
        myC._foo2.name[3] = 'a'; // CS1708
    }

    static readonly Foo _foo1;
    public readonly Foo _foo2;
}
```

Compiler Error CS1713

Error Message

Unexpected error building metadata name for type Typename1—'Reason'

This error is often caused by an internal compiler error. Making some minor changes to your code, such as shortening the length of the name, and then recompiling, may resolve this error.

Compiler Error CS1714

Error Message

The base class or interface of TypeName1 could not be resolved or is invalid

You are implementing TypeName1 from a class or interface that either could not be resolved, for example, the compiler was unable to locate it, or is invalid. The solution is to determine which of these two cases it is, and either more correctly specify the location of the type, or fix any compiler errors in the base class.

Compiler Error CS1715

Error Message

'Type1': type must be 'Type2' to match overridden member 'MemberName'

This error is the same as [Compiler Error CS0508](#), except that CS0508 now only applies to methods that have return types, while CS1715 applies to properties and indexers that only have 'types' instead of 'return types'.

Example

The following code generates CS1715.

```
// CS1715.cs
abstract public class Base
{
    abstract public int myProperty
    {
        get;
        set;
    }
}

public class Derived : Base
{
    int myField;
    public override double myProperty // CS1715
    // try the following line instead
    // public override int myProperty
    {
        get { return myField; }
        set { myField = value; }
    }

    public static void Main()
    {
        Derived d = new Derived();
        d.myProperty = 5;
    }
}
```

Compiler Error CS1716

Error Message

Do not use 'System.Runtime.CompilerServices.FixedBuffer' attribute. Use the 'fixed' field modifier instead.

This error arises in an unsafe code section that contains a fixed-size array declaration similar to a field declaration. Do not use this attribute. Instead, use the keyword **fixed**.

Example

The following example generates CS1716.

```
// CS1716.cs
// compile with: /unsafe
using System;
using System.Runtime.CompilerServices;

public struct UnsafeStruct
{
    [FixedBuffer(typeof(int), 4)] // CS1716
    unsafe public int aField;
    // Use this single line instead of the above two lines.
    // unsafe public fixed int aField[4];
}

public class TestUnsafe
{
    static int Main()
    {
        UnsafeStruct us = new UnsafeStruct();
        unsafe
        {
            if (us.aField[0] == 0)
                return us.aField[1];
            else
                return us.aField[2];
        }
    }
}
```


Compiler Error CS1719

Error Message

Error reading Win32 resource file 'File Name' -- 'reason'

An attempt to read the Win32 resource file failed for the reason given in the error, typically something like "file not found" or "access denied." This error is resolved by correcting the problem described by the reason.

Compiler Error CS1721

Error Message

Class 'class' cannot have multiple base classes: 'class_1' and 'class_2'

The most common cause of this error message is attempting to use multiple inheritance. A class in C# may only inherit from one class: the remaining types following the class name in the class declaration must be interfaces.

Example

The following sample generates CS1721.

```
// CS1721.cs
public class A {}
public class B {}
public class MyClass : A, B {}    // CS1721
```

Compiler Error CS1722

Error Message

Base class 'class' must come before any interfaces

When specifying a class to inherit from and interfaces to implement, the class name must be specified first.

Example

The following sample generates CS1722.

```
// CS1722.cs
// compile with: /target:library
public class A {}
interface I {}

public class MyClass : I, A {}    // CS1722
public class MyClass2 : A, I {}   // OK
```

Compiler Error CS1724

Error Message

Value specified for the argument to 'System.Runtime.InteropServices.DefaultCharSetAttribute' is not valid

This error is generated by an invalid argument to the [DefaultCharSetAttribute](#) class.

Example

The following example generates CS1724.

```
// CS1724.cs
using System.Runtime.InteropServices;
// To resolve, replace 42 with a valid CharSet value.
[module:DefaultCharSetAttribute((CharSet)42)] // CS1724
class C {

    [DllImport("F.Dll")]
    extern static void FW1Named();

    static void Main() {}
}
```

Compiler Error CS1728

Error Message

Cannot bind delegate to 'member' because it is a member of 'type'

You cannot bind delegates to members of **Nullable** types.

Example

The following sample generates CS1728.

```
// CS1728.cs
// compile with: /W:2
class Test
{
    delegate T GetT<T>();
    delegate T GetT1<T>(T t);

    delegate bool E(object o);
    delegate int I();
    delegate string S();

    static void Main()
    {
        int? x = null;
        int? y = 5;

        GetT<int> d1 = x.GetValueOrDefault;    // CS1728
        GetT<int> d2 = y.GetValueOrDefault;    // CS1728
        GetT1<int> d3 = x.GetValueOrDefault;    // CS1728
        GetT1<int> d4 = y.GetValueOrDefault;    // CS1728
    }
}
```

Compiler Error CS1900

Error Message

Warning level must be in the range 0-4

The `/warn` compiler option can only take one of five possible values (0, 1, 2, 3, or 4). Any other value passed to `/warn` will result in CS1900.

The following sample generates CS1900:

```
// CS1900.cs
// compile with: /W:5
// CS1900 expected
class x
{
    public static void Main()
    {
    }
}
```

Compiler Error CS1902

Error Message

Invalid option 'option' for /debug; must be full or pdbonly

An invalid option was passed to the [/debug](#) compiler option.

The following sample generates CS1902:

```
// CS1902.cs
// compile with: /debug:x
// CS1902 expected
class x
{
    public static void Main()
    {
    }
}
```

Compiler Error CS1906

Error Message

Invalid option 'option'; Resource visibility must be either 'public' or 'private'

This error indicates an invalid [/resource \(Embed Resource File to Output\)](#) or [/linkresource \(Link to .NET Framework Resource\)](#) command line option. Check the syntax of the **/resource** or **/linkresource** command line option, and make sure that the accessibility modifier used is either **public** or **private**.

Compiler Error CS1908

Error Message

The type of the argument to the DefaultValue attribute must match the parameter type

This error is generated when you use the wrong argument for the [DefaultValueAttribute](#) attribute value. Use a value that matches the parameter type.

Example

The following sample generates CS1908.

```
// CS1908.cs
// compile with: /target:library
using System.Runtime.InteropServices;

public interface ISomeInterface
{
    void Bad([Optional] [DefaultValue("true")] bool b);    // CS1908

    void Good([Optional] [DefaultValue(true)] bool b);    // OK
}
```

Compiler Error CS1909

Error Message

The DefaultValue attribute is not applicable on parameters of type 'type'

CS1909 is generated when you use a `DefaultValue` attribute that is not applicable on the parameter type.

Example

The following sample generates CS1909.

```
// CS1909.cs
// compile with: /target:library
using System.Runtime.InteropServices;

public interface ISomeInterface
{
    void Test1([DefaultValue(new int[] {1, 2})] int[] arr1);    // CS1909

    void Test2([DefaultValue("Test String")] string s);      // OK
}
```

Compiler Error CS1910

Error Message

Argument of type 'type' is not applicable for the DefaultValue attribute

For parameters whose type is object, the argument of the [DefaultParameterValueAttribute](#) must be **null**, an integral type, a floating point, **bool**, **string**, **enum**, or **char**. The argument can not be of type [Type](#) or any array type.

Example

The following sample generates CS1910.

```
// CS1910.cs
// compile with: /target:library
using System.Runtime.InteropServices;

public interface MyI
{
    void Test([DefaultParameterValue(typeof(object))] object o);    // CS1910
}
```

Compiler Error CS2000

Error Message

Compiler initialization failed unexpectedly

This error indicates an initialization failure.

Use setup to repair or reinstall Visual Studio or the .NET Framework SDK.

If the error persists, contact product support.

Compiler Error CS2001

Error Message

Source file 'file' could not be found

A source file name was passed to the compiler, but could not be located. Check the spelling of the file name and the location of the file.

Compiler Error CS2003

Error Message

Response file 'file' included multiple times

A response file was passed to the compiler more than once. A response file can only be passed to the compiler once per output file.

For more on response files, see [@ \(Specify Response File\)](#).

Compiler Error CS2005

Error Message

Missing file specification for 'option' option

A [compiler option](#) was only partially specified.

For example, when using [/recurse](#), you must specify the file to search: ***/recurse:filename.cs***.

Example

The following sample generates CS2005.

```
// CS2005.cs
// compile with: /recurse:
// CS2005 expected
class x
{
    public static void Main() {}
}
```

Compiler Error CS2006

Error Message

Command-line syntax error: Missing 'text' for 'option' option

The syntax for *option* requires additional text. For information, see [Compiler Options](#).

Compiler Error CS2007

Error Message

Unrecognized command-line option: 'option'

The compiler was passed a string that was not a [compiler option](#), even though it began with a forward slash (/).

The following sample generates CS2007:

```
// CS2007.cs
// compile with: /recur
// CS2007 expected
class x
{
    public static void Main() {}
}
```

Compiler Error CS2008

Error Message

No inputs specified

The compiler was invoked and compiler options were specified, but no source-code files were passed.

Compiler Error CS2011

Error Message

Unable to open response file 'file'

A response file was specified in a compilation, but the compiler was unable to locate and open the file.

For more on response files, see [@ \(Specify Response File\)](#).

Compiler Error CS2012

Error Message

Cannot open 'file' for writing

While using the `/bugreport:file` compiler option, the file could not be opened for writing. Make sure you specified a valid file name and that the file is not read-only.

Compiler Error CS2013

Error Message

Invalid image base number 'value'

An invalid value (not a number) was passed to the [/baseaddress](#) compiler option.

The following sample generates CS2013:

```
// CS2013.cs
// compile with: /target:library /baseaddress:x
// CS2013 expected
class MyClass
{
}
```

Compiler Error CS2015

Error Message

'file' is a binary file instead of a text file

A file was passed to the compiler that was a binary file. The compiler expects a source code file.

Compiler Error CS2016

Error Message

Code page 'codepage' is invalid or not installed

The [/codepage](#) compiler option was passed an invalid value.

The following sample generates CS2016:

```
// CS2016.cs
// compile with: /codepage:x
// CS2016 expected
class MyClass
{
    public static void Main()
    {
    }
}
```

Compiler Error CS2017

Error Message

Cannot specify /main if building a module or library

You cannot specify a main entry point when you are building a [/target:library](#).

The following sample generates CS2017:

```
// CS2017.cs
// compile with: /main:MyClass /target:library
// CS2017 expected
class MyClass
{
    public static void Main()
    {
    }
}
```


Compiler Error CS2018

Error Message

Unable to find messages file 'cscmsgs.dll'

The .dll file that contains the compiler's error and warning messages was not found. This file must be present in the same directory as the other compiler support files.

Compiler Error CS2019

Error Message

Invalid target type for /target: must specify 'exe', 'winexe', 'library', or 'module'

The [/target](#) compiler option was used, but an invalid parameter was passed. To resolve this error, recompile the program using the form of the **/target** option that is appropriate to your output file.

The following sample generates CS2017:

```
// CS2019.cs
// compile with: /target:libra
// CS2019 expected
class MyClass
{
}
```

Compiler Error CS2020

Error Message

Only the first set of input files can build a target other than 'module'

In a multi-output compilation, the first output file must be built with [/target:exe](#), [/target:winexe](#), or [/target:library](#). Any subsequent output files must be built with [/target:module](#).

Compiler Error CS2021

Error Message

File name 'file' is too long or invalid

All file names passed to the C# compiler must be no longer than **_MAX_PATH** (defined in a Windows header file). the compiler will give this error in the following situations:

- A file name (including the path) is longer than **_MAX_PATH**.
- The file name contains invalid characters.
- The file name contains wildcards where wildcards are not allowed (such as in resource file names).

Compiler Error CS2022

Error Message

Options '/out' and '/target' must appear before source file names

The [/out \(Set Output Filename\)](#) and [/target \(Specify Output File Format\)](#) compiler options, when specified on the command line, must precede the source code files.

Compiler Error CS2024

Error Message

Invalid file section alignment number '#'

An invalid value was passed to the [/filealign](#) compiler option.

The following sample generates CS2024:

```
// CS2024.cs
// compile with: /filealign:ex
// CS2024 expected
class MyClass
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS2029

Error Message

Invalid value for '/define'; 'identifier' is not a valid identifier

This warning occurs if the value used in the [/define](#) option has some invalid characters.

Compiler Error CS2032

Error Message

Character 'character' is not allowed on the command-line or in response files

Response files and the command line options for csc.exe are not allowed to contain low-ASCII control characters in the range of 0-31, or the pipe (|) character. You cannot ordinarily generate this error directly from the command line because the command line processor filters out invalid characters, as does the integrated development environment (IDE). You can generate this error using a response file, as follows.

To generate this error

1. Create a file named CS2032.rsp that contains the line, `/target:exe /out:cs|2032.exe cs2032.cs`, in **My Documents**.
2. Create a file named cs2032.cs that contains anything at all in **My Documents**.
3. Click **Start**, then point to **All Programs**, point to **Microsoft Visual Studio 2005**, point to **Visual Studio 2005 Tools**, and then click **Visual Studio Command Prompt**.

The **Visual Studio Command Prompt** window opens.

4. In the **Visual Studio Command Prompt** window, change the current directory to **My Documents**.
5. Run the following from the **Visual Studio Command Prompt**: `csc @cs2032.rsp`
6. The CS2032 error message appears.

7.

1.

Compiler Error CS2033

Error Message

Cannot create short filename 'filename' when a long filename with the same short filename already exists

Compile any C# file with a name longer than eight characters. Then compile another file with the short version of the preceding file name, such as the first six characters of the name plus "~1." The second compile will generate this error.

To resolve this error, rename the short file name to one that does not conflict with the long file name.

Compiler Error CS2034

Error Message

A /reference option that declares an extern alias can only have one filename. To specify multiple aliases or filenames, use multiple /reference options.

To specify two aliases and/or file names, use two **/reference** options, like this:

Example

The following code will generate error CS2034.

```
// CS2034.cs
// compile with: /r:A1=cs2034a1.dll;A2=cs2034a2.dll
// to fix, compile with: /r:A1=cs2034a1.dll /r:A2=cs2034a2.dll
// CS2034
extern alias A1;
extern alias A2;
using System;
```

Compiler Error CS2035

Error Message

Command-line syntax error: Missing ';<number>' for 'compiler_option' option

Some compiler options require a value.

Example

The following sample generates CS2035.

```
// CS2035.cs
// compile with: /baseaddress
// CS2035 expected
```

Compiler Warning (level 1) CS3027

Error Message

'type_1' is not CLS-compliant because base interface 'type_2' is not CLS-compliant

A non-CLS compliant type cannot be a base type for a type that is CLS compliant.

Example

The following sample contains an interface with a method that uses a non-CLS compliant type in its signature, making the type non-CLS compliant.

```
// CS3027.cs
// compile with: /target:library
public interface IBase
{
    void IMethod(uint i);
}
```

The following sample generates CS3027.

```
// CS3027_b.cs
// compile with: /reference:CS3027.dll /target:library /W:1
[assembly:System.CLSCompliant(true)]
public interface IDerived : IBase {}
```

Compiler Error CS5001

Error Message

Program 'program' does not contain a static 'Main' method suitable for an entry point

This error occurs when no [Main](#) method is found in the code that produces an executable file. This error also occurs if the entry point function, **Main**, is defined with the wrong case, such as lower-case **main**, or not declaring **Main** as static.

Example

The following example generates CS5001.

```
// CS5001.cs
// CS5001 expected
public class a
{
    // Uncomment the following line to resolve.
    // public static void Main() {}
}
```

Compiler Error CS0025

Error Message

Standard library file 'file' could not be found

A file that is needed by the compiler was not found. Check that the path is correct and that the file exists.

If the file is a Visual Studio system file, you may need to repair your Visual Studio installation, or reinstall it completely.

Compiler Warning (level 1) CS0183

Error Message

The given expression is always of the provided ('type') type

If a conditional statement always evaluates to **true**, then you do not need a conditional statement. This warning occurs when you try to evaluate a type using the **is** operator. If the evaluation is a value type, then the check is unnecessary.

The following sample generates CS0183:

```
// CS0183.cs
// compile with: /W:1
using System;
public class Test
{
    public static void F(Int32 i32, String str)
    {
        if (str is Object)           // OK
            Console.WriteLine( "str is an object" );
        else
            Console.WriteLine( "str is not an object" );

        if (i32 is Object)  // CS0183
            Console.WriteLine( "i32 is an object" );
        else
            Console.WriteLine( "i32 is not an object" ); // never reached
    }

    public static void Main()
    {
        F(0, "CS0183");
        F(120, null);
    }
}
```

Compiler Warning (level 1) CS0184

Error Message

The given expression is never of the provided ('type') type

The expression can never be **true** because the variable you are testing is neither declared as ***type*** nor derived from ***type***.

The following sample generates CS0184:

```
// CS0184.cs
// compile with: /W:1
class MyClass
{
    public static void Main()
    {
        int i = 0;
        if (i is string)    // CS0184
            i++;
    }
}
```


Compiler Warning (level 1) CS0420

Error Message

'identifier': a reference to a volatile field will not be treated as volatile

A volatile field should not normally be passed using a **ref** or **out** parameter, since it will not be treated as volatile within the scope of the function. There are exceptions to this, such as when calling an interlocked API. As with any warning, you may use the [#pragma warning](#) to disable this warning in those rare cases where you are intentionally using a volatile field as a reference parameter.

The following sample generates CS0420:

```
// CS0420.cs
// compile with: /W:1
using System;

class TestClass
{
    private volatile int i;

    public void TestVolatile(ref int ii)
    {
    }

    public static void Main()
    {
        TestClass x = new TestClass();
        x.TestVolatile(ref x.i);    // CS0420
    }
}
```

Compiler Warning (level 1) CS0465

Error Message

Introducing a 'Finalize' method can interfere with destructor invocation. Did you intend to declare a destructor?

This warning occurs when you create a class with a method whose signature is `public virtual void Finalize`.

If such a class is used as a base class and if the deriving class defines a destructor, the destructor will override the base class `Finalize` method, not [Finalize](#).

Example

The following sample generates CS0465.

```
// CS0465.cs
// compile with: /target:library
class A
{
    public virtual void Finalize() {}    // CS0465
}

// OK
class B
{
    ~B() {}
}
```

Compiler Warning (level 1) CS0602

Error Message

The feature 'old_feature' is deprecated. Please use 'new_feature' instead

A language feature used in your code (*old_feature*) is still supported, but that support may be removed in a future release. Instead, you should use the recommended syntax (*new_feature*).

Compiler Warning (level 1) CS0612

Error Message

'member' is obsolete

The class designer marked a member with the [Obsolete](#) attribute. This means that the member might not be supported in a future version of the class.

The following sample shows how accessing an obsolete member generates CS0612:

```
// CS0612.cs
// compile with: /W:1
using System;

class MyClass
{
    [Obsolete]
    public static void ObsoleteMethod()
    {
    }

    [Obsolete]
    public static int ObsoleteField;
}

class MainClass
{
    static public void Main()
    {
        MyClass.ObsoleteMethod();    // CS0612 here: method is deprecated
        MyClass.ObsoleteField = 0;    // CS0612 here: field is deprecated
    }
}
```

Compiler Warning (level 1) CS0626

Error Message

Method, operator, or accessor 'method' is marked external and has no attributes on it. Consider adding a `DllImport` attribute to specify the external implementation

A method marked **extern** should also be marked with an attribute, for example, the [DllImport](#) attribute.

The attribute specifies where the method is implemented. At run time, the program will need this information.

The following sample generates CS0626:

```
// CS0626.cs
// compile with: /warnaserror
using System.Runtime.InteropServices;

public class MyClass
{
    static extern public void mf(); // CS0626
    // try the following line
    // [DllImport("mydll.dll")] static extern public void mf();

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS0658

Error Message

'attribute modifier' is not a recognized attribute location. All attributes in this block will be ignored.

An invalid attribute modifier was specified. See [Attribute Targets](#) for more information.

The following sample generates CS0658:

```
// CS0658.cs
using System;
public class TestAttribute : Attribute{}
[badAttributeLocation: Test]    // CS0658, badAttributeLocation is invalid
class ClassTest
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS0672

Error Message

Member 'member1' overrides obsolete member 'member2'. Add the Obsolete attribute to 'member1'

The compiler found an **override** to a method marked as **obsolete**. However, the overriding method was not itself marked as obsolete. The overriding method will still generate [CS0612](#), if called.

Review your method declarations and explicitly indicate whether a method (and all of its overrides) should be marked **obsolete**.

The following sample generates CS0672:

```
// CS0672.cs
// compile with: /W:1
class MyClass
{
    [System.Obsolete]
    public virtual void ObsoleteMethod()
    {
    }
}

class MyClass2 : MyClass
{
    public override void ObsoleteMethod()    // CS0672
    {
    }
}

class MainClass
{
    static public void Main()
    {
    }
}
```

Compiler Warning (level 1) CS0684

Error Message

'interface' interface marked with 'CoClassAttribute' not marked with 'ComImportAttribute'

If you specify **CoClassAttribute** on an interface, you must also specify **ComImportAttribute**.

The following sample generates CS0684:

```
// CS0684.cs
// compile with: /W:1
using System;
using System.Runtime.InteropServices;

[CoClass(typeof(C))] // CS0684
// try the following line instead
// [CoClass(typeof(C)), ComImport]
interface I
{
}

class C
{
    static void Main() {}
}
```


Compiler Warning (level 1) CS0688

Error Message

'method1' has a link demand, but overrides or implements 'method2' which does not have a link demand. A security hole may exist.

The link demand set up on the derived class method can easily be circumvented by calling the base class method. To close the security hole, the base class method needs to also use the link demand. For more information, see [Demand vs. LinkDemand](#).

Example

The following sample generates CS0688. To resolve the warning without modifying the base class, remove the security attribute from the overriding method. This will not solve the security problem.

```
// CS0688.cs
// compile with: /W:1
using System;
using System.Security.Permissions;

class Base
{
    //Uncomment the following line to close the security hole
    //[FileIOPermission(SecurityAction.LinkDemand, All=@"C:\\")]
    public virtual void DoScaryFileStuff()
    {
    }
}

class Derived: Base
{
    [FileIOPermission(SecurityAction.LinkDemand, All=@"C:\\")] // CS0688
    public override void DoScaryFileStuff()
    {
    }
    static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS1030

Error Message

#warning: 'text'

Displays the text of a warning defined with the [#warning](#) directive.

The following sample shows how to create a user-defined warning:

```
// CS1030.cs
class Sample
{
    static void Main()
    {
        #warning Let's give a warning here    // CS1030
    }
}
```

Compiler Warning (level 1) CS1200

Error Message

The feature 'invalid feature' is deprecated. Please use 'valid feature' instead'.

The feature you are attempting to use is now deprecated. Update your code to use the valid feature instead.

Compiler Warning (level 1) CS1201

Error Message

The feature 'invalid feature' is deprecated. Please use 'valid feature' instead.

The feature you are attempting to use is now deprecated. Update your code to use the valid feature instead.

Compiler Warning (level 1) CS1202

Error Message

The feature 'invalid feature' is deprecated. Please use 'valid feature' instead'.

The feature you are attempting to use is now deprecated. Update your code to use the valid feature instead.

Compiler Warning (level 1) CS1203

The feature 'invalid feature' is deprecated. Please use 'valid feature' instead'.

The feature you are attempting to use is now deprecated. Update your code to use the valid feature instead.

Compiler Warning (level 1) CS1522

Error Message

Empty switch block

The compiler detected a [switch](#) block with no **case** or **default** statement. A **switch** block must have one or more **case** or **default** statements.

The following sample generates CS1522:

```
// CS1522.cs
// compile with: /W:1
using System;
class x
{
    public static void Main()
    {
        int i = 6;

        switch(i)    // CS1522
        {
            /*
            case (5):
                Console.WriteLine("5");
                return;
            default:
                Console.WriteLine("not 5");
                return;
            */
        }
    }
}
```

Compiler Warning (level 1) CS1570

Error Message

XML comment on 'construct' has badly formed XML — 'reason'

When using [/doc](#), any comments in the source code must be in XML. Any error with your XML markup will generate CS1570. For example:

- If you are passing a string to a **cref**, such as in an [<exception>](#) tag, the string must be enclosed in double quotation marks.
- If you are using a tag, such as [<seealso>](#), which does not have a closing tag, you must specify a forward slash before the closing angle bracket.
- If you need to use a greater-than or less-than symbol in the text of description, you need to represent them with **>** or **<**.
- The file or path attribute on an [<include>](#) tag was missing or improperly formed.

The following sample generates CS1570:

```
// CS1570.cs
// compile with: /W:1
namespace ns
{
    // the following line generates CS1570
    /// <summary> returns true if < 5 </summary>
    // try this instead
    // /// <summary> returns true if &lt;5 </summary>

    public class MyClass
    {
        public static void Main ()
        {
        }
    }
}
```


Compiler Warning (level 1) CS1574

Error Message

XML comment on 'construct' has cref attribute 'item' that could not be resolved

A string passed to a cref tag, for example, within an <exception> tag, referred to a member that is not available within the current build environment. The string that you pass to a cref tag must be the syntactically correct name of a member or field.

For more information, see [Recommended Tags for Documentation Comments](#).

The following sample generates CS1574:

```
// CS1574.cs
// compile with: /W:1 /doc:x.xml
using System;

/// <exception cref="System.Console.WriteLine">An exception class.</exception> // CS1574
// instead, uncomment and try the following line
// /// <exception cref="System.Console.WriteLine">An exception class.</exception>
class EClass : Exception
{
}

class TestClass
{
    public static void Main()
    {
        try
        {
        }
        catch(EClass)
        {
        }
    }
}
```

Compiler Warning (level 1) CS1580

Error Message

Invalid type for parameter 'parameter number' in XML comment cref attribute

When attempting to reference an overload form of a method, the compiler detected a syntax error. Typically, this indicates that the parameter name, and not the type, was specified. A malformed line will appear in the generated XML file.

The following sample generates CS1580:

```
// CS1580.cs
// compile with: /W:1 /doc:x.xml
using System;

/// <seealso cref="Test(i)"/>    // CS1580
// try the following line instead
// /// <seealso cref="Test(int)"/>
public class MyClass
{
    /// <summary>help text</summary>
    public static void Main()
    {
    }

    /// <summary>help text</summary>
    public void Test(int i)
    {
    }

    /// <summary>help text</summary>
    public void Test(char i)
    {
    }
}
```

Compiler Warning (level 1) CS1581

Error Message

Invalid return type in XML comment cref attribute

When attempting to reference a method, the compiler detected an error due to an invalid return type.

Example

The following sample generates CS1581:

```
// CS1581.cs
// compile with: /W:1 /doc:x.xml

/// <summary>help text</summary>
public class MyClass
{
    /// <summary>help text</summary>
    public static void Main()
    {
    }

    /// <summary>help text</summary>
    public static explicit operator int(MyClass f)
    {
        return 0;
    }
}

/// <seealso cref="MyClass.explicit operator intt(MyClass)"/> // CS1581
// try the following line instead
// /// <seealso cref="MyClass.explicit operator int(MyClass)"/>
public class MyClass2
{
}
```

Compiler Warning (level 1) CS1584

Error Message

XML comment on 'member' has syntactically incorrect cref attribute 'invalid_syntax'

One of the parameters passed to a tag for documentation comments has invalid syntax. For more information, see [Recommended Tags for Documentation Comments \(C# Programming Guide\)](#).

Example

The following sample generates CS1584.

```
// CS1584.cs
// compile with: /W:1 /doc:CS1584.xml
/// <remarks>Test class</remarks>
public class Test
{
    /// <remarks>Called in <see cref="Test.Mai()">.</remarks>    // CS1584
    // try the following line instead
    // /// <remarks>Called in <see cref="Test.Main()">.</remarks>
    public static void Test2() {}

    /// <remarks>Main method</remarks>
    public static void Main() {}
}
```

Compiler Warning (level 1) CS1589

Error Message

Unable to include XML fragment 'fragment' of file 'file' -- reason

The syntax (*fragment*) of a `<include>` tag, which referenced a file (*file*), was incorrect for the specified ***reason***.

A malformed line will be placed in the generated XML file.

The following sample generates CS1589:

```
// CS1589.cs
// compile with: /W:1 /doc:CS1589_out.xml

/// <include file='CS1589.doc' path='MyDocs/MyMembers[@name="test"]/' />    // CS1589
// try the following line instead
// /// <include file='CS1589.doc' path='MyDocs/MyMembers[@name="test"]/*' />
class Test
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS1590

Error Message

Invalid XML include element -- Missing file attribute

A path or doc attribute, passed to the [<include>](#) tag, was missing or incomplete.

The following sample generates CS1590:

```
// CS1590.cs
// compile with: /W:1 /doc:x.xml

/// <include path='MyDocs/MyMembers[@name="test"]/*' />    // CS1590
// try the following line instead
// /// <include file='x.doc' path='MyDocs/MyMembers[@name="test"]/*' />
class Test
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS1592

Error Message

Badly formed XML in included comments file -- 'reason'

A problem, reported as ***reason***, was found in the file specified by the `<include>` tag.

Compiler Warning (level 1) CS1598

Error Message

XML parser could not be loaded for the following reason: 'reason'. The XML documentation file 'file' will not be generated.

The [/doc](#) option was specified, but the compiler could not find and load msxml3.dll. Make sure that the file msxml3.dll is installed and registered.

Compiler Warning (level 1) CS1607

Error Message

Assembly generation -- reason

A warning was generated from the assembly-creation phase of the compilation.

If you are building a 64-bit application on a 32-bit platform, you must ensure that 64-bit versions of all referenced assemblies are installed on the target platform.

All x86-specific common language runtime (CLR) assemblies have 64-bit counterparts (every CLR assembly will exist on all platforms). Therefore, you can safely ignore CS1607 for CLR assemblies.

For more information, see [Al.exe Tool Errors and Warnings](#).

Compiler Warning (level 1) CS1616

Error Message

Option 'option' overrides attribute 'attribute' given in a source file or added module

This warning occurs if the assembly attributes [AssemblyKeyFile](#) or [AssemblyKeyName](#) found in source conflict with the [/keyfile](#) or [/keycontainer](#) command line option or key file name or key container specified in the Project Properties.

For the example below, assume you have a key file named `cs1616.snk`. This file could be generated with the command line:

```
sn -k CS1616.snk
```

The following sample generates CS1616:

```
// CS1616.cs
// compile with: /keyfile:cs1616.snk
using System.Reflection;

// To fix the error, remove the next line
[assembly: AssemblyKeyFile("cs1616b.snk")] // CS1616

class C
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS1633

Error Message

Unrecognized #pragma directive

The pragma used was not one of the known pragmas supported by the C# compiler. To resolve this error, use only pragmas supported.

The following sample generates CS1633:

```
// CS1633.cs
// compile with: /W:1
#pragma unknown // CS1633

class C
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS1634

Error Message

Expected disable or restore

This error occurs if a `#pragma warning` clause is badly formed, such as if `disable` or `restore` was omitted. For more information, see the [#pragma warning](#) topic.

Example

The following sample generates CS1634:

```
// CS1634.cs
// compile with: /W:1

#pragma warning    // CS1634
// Try this instead:
// #pragma warning disable 0219

class MyClass
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS1635

Error Message

Cannot restore warning 'warning code' because it was disabled globally

This warning occurs if you use the **/nowarn** command line option or project setting to disable a warning for the entire compilation unit, but you use `#pragma warning restore` to attempt to restore that warning. To resolve this error, remove the **/nowarn** command line option or project setting, or remove the `#pragma warning restore` for any warnings you are disabling via the command line or project settings. For more information, see the [#pragma warning](#) topic.

The following sample generates CS1635:

```
// CS1635.cs
// compile with: /w:1 /nowarn:162

enum MyEnum {one=1,two=2,three=3};

class MyClass
{
    public static void Main()
    {
#pragma warning disable 162

        if (MyEnum.three == MyEnum.two)
            System.Console.WriteLine("Duplicate");

#pragma warning restore 162
    }
}
```

Compiler Warning (level 1) CS1645

Error Message

Feature 'feature' is not part of the standardized ISO C# language specification, and may not be accepted by other compilers

The feature you are using is not part of the ISO standard. Code using this feature may not compile on other compilers.

```
// CS1645.cs
// compile with: /W:1 /t:module /langversion:ISO-1
[assembly:System.CLSCompliant(false)]
// To supress the warning use the switch: /nowarn:1645
[module:System.CLSCompliant(false)]    // CS1645
class Test
{
}
```

Compiler Warning (level 1) CS1658

Error Message

'warning text'. See also error 'error code'

The compiler emits this warning when it overrides an error with a warning. For information about the problem, refer to the error mentioned. To find the appropriate error from within the Visual Studio IDE, use the index. For example, if the text above reads "See also error 'CS1037'," look for CS1037 in the index.

Example

The following example generates CS1658.

```
// CS1658.cs
// compile with: /doc:x.xml
// CS1584 expected
/// <summary>
/// </summary>
public class C
{
    /// <see cref="C.F(params object[])"/> // CS1658
    public static void M()
    {
    }

    /// <summary>
    /// </summary>
    public void F(params object[] o)
    {
    }

    static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS1682

Error Message

Reference to type 'type' claims it is nested within 'nested type', but it could not be found

This error arises when you import references that do not agree with other references or with code you have written. A common way to get this error is to write code that refers to a class in metadata, and then you either delete that class or modify its definition.

Example

```
// CS1682.cs
// compile with: /target:library /keyfile:mykey.snk
public class A {
    public class N1 {}
}
```

```
// CS1682_b.cs
// compile with: /target:library /reference:CS1682.dll
using System;
public class Ref {

    public static A A1() {
        return new A();
    }

    public static A.N1 N1() {
        return new A.N1();
    }
}
```

```
// CS1682_c.cs
// compile with: /target:library /keyfile:mykey.snk /out:CS1682.dll
public class A {
    public void M1() {}
}
```

The following sample generates CS1682.

```
// CS1682_d.cs
// compile with: /reference:CS1682.dll /reference:CS1682_b.dll /W:1
// CS1682 expected
class Tester {
    static void Main()
    {
        Ref.A1().M1();
    }
}
```


Compiler Warning (level 1) CS1683

Error Message

Reference to type 'Type Name' claims it is defined in this assembly, but it is not defined in source or any added modules

This error can occur when you are importing an assembly that contains a reference back to the assembly you are currently compiling, but the assembly being compiled contains nothing matching the reference. One way to get to this situation is to compile your assembly, which initially does contain the member that the assembly being imported is referencing. Then you update your assembly, mistakenly removing the members that the imported assembly is referencing.

Compiler Warning (level 1) CS1684

Error Message

Reference to type 'Type Name' claims it is defined in 'Namespace', but it could not be found

This error can be caused by a reference inside one namespace referring to a type that it says exists inside a second namespace, but the type does not exist. For example, mydll.dll says that type `A` exists inside yourdll.dll, but no such type exists inside yourdll.dll. One possible cause of this error is that the version of yourdll.dll you are using is too old and `A` has not yet been defined.

The following sample generates CS1684.

Example

```
// CS1684_a.cs
// compile with: /target:library /keyfile:CS1684.key
public class A {
    public void Test() {}
}

public class C2 {}
```

```
// CS1684_b.cs
// compile with: /target:library /r:cs1684_a.dll
// post-build command: del /f CS1684_a.dll
using System;
public class Ref
{
    public static A GetA() { return new A(); }
    public static C2 GetC() { return new C2(); }
}
```

We now rebuild the first assembly, leaving out the definition of the class `C2` not to be defined in the recompilation.

```
// CS1684_c.cs
// compile with: /target:library /keyfile:CS1684.key /out:CS1684_a.dll
public class A {
    public void Test() {}
}
```

This module references the second module by means of the identifier `Ref`. But the second module contains a reference to the class `C2`, which no longer exists because of the compilation in the previous step, and therefore the CS1684 error message is returned from the compilation of this module.

```
// CS1684_d.cs
// compile with: /reference:cs1684_a.dll /reference:cs1684_b.dll
// CS1684 expected
class Tester
{
    public static void Main()
    {
        Ref.GetA().Test();
    }
}
```

Compiler Warning (level 1) CS1685

Error Message

The predefined type 'System.type name' is defined in multiple assemblies in the global alias; using definition from 'File Name'

This error occurs when a predefined system type such as `System.int32` is found in two assemblies. One way this can happen is if you are referencing `mscorlib` from two different places, such as trying to run the .Net Framework versions 1.0 and 1.1 side-by-side.

The compiler will use the definition from only one of the assemblies. The compiler searches only global aliases, does not search libraries defined **/reference**. If you have specified **/nostdlib**, the compiler will search for [Object](#), and in the future start all searches for predefined types in the file where it found **Object**.

Compiler Warning (level 1) CS1687

Error Message

Source file has exceeded the limit of 16,707,565 lines representable in the PDB, debug information will be incorrect

The PDB and debugger have some limitations about how big a file can be. If the source file is too big, the debugger will not behave properly beyond that limit. The user should either not emit debug information for that file by possibly using `#line hidden`, or they should find a way to shrink the file, possibly by splitting the file into multiple files. They might want to use the **partial** keyword to split up a large class.

Compiler Warning (level 1) CS1691

Error Message

'number' is not a valid warning number

A number that was passed to the [#pragma warning](#) preprocessor directive was not a valid warning number. Verify that the number represents a warning, not an error or another sequence of characters.

Example

The following example generates CS1691.

```
// CS1691.cs
public class C
{
    int i = 1;
    public static void Main()
    {
        C myC = new C();
#pragma warning disable 151 // CS1691
// Try the following line instead:
// #pragma warning disable 1645
        myC.i++;
#pragma warning restore 151 // CS1691
// Try the following line instead:
// #pragma warning restore 1645
    }
}
```

Compiler Warning (level 1) CS1692

Error Message

Invalid number

A number of preprocessor directives, such as **#pragma** and **#line**, use numbers as parameters. One of these numbers is invalid because it is too big, in the wrong format, contains illegal characters, and so on. To correct this error, correct the number.

Example

The following example generates CS1692.

```
// CS1692.cs

#pragma warning disable a // CS1692
// Try this instad:
// #pragma warning disable 1691

class A
{
    static void Main()
    {
    }
}
```

Invalid filename specified for preprocessor directive. Filename is too long or not a valid filename.

This warning occurs when using the **#pragma checksum** preprocessor directive. The file name specified is longer than 256 characters. To resolve this warning, use a shorter file name.

Example

The following sample generates CS1694.

```
// cs1694.cs
#pragma checksum "MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890MyFile1234567890.txt" {00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F} // CS1694
class MyClass {}
```

Compiler Warning (level 1) CS1695

Error Message

Invalid #pragma checksum syntax; should be #pragma checksum "filename" "{XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX}" "XXXX..."

You should rarely encounter this error since the checksum is generally inserted at run time if you are generating code by means of the Code Dom API.

However, if you were to type in this **#pragma** statement and mistype either the GUID or checksum, you would get this error. The syntax checking by the compiler does not validate that you typed in a correct GUID, but it does check for the right number of digits and delimiters, and that the digits are hexadecimal. Likewise, it verifies that the checksum contains an even number of digits, and that the digits are hexadecimal.

Example

The following example generates CS1695.

```
// CS1695.cs

#pragma checksum "12345" // CS1695

public class Test
{
    static void Main()
    {
    }
}
```


Compiler Warning (level 1) CS1696

Error Message

Single-line comment or end-of-line expected

The compiler requires a preprocessor directive to be followed by an end-of-line terminator or by a single-line comment. The compiler has finished processing a valid preprocessor directive, and has encountered something that violates this syntax constraint.

Example

The following sample generates CS1696.

```
// CS1696.cs
class Test
{
    public static void Main()
    {
        #pragma warning disable 1030;219    // CS1696
        #pragma warning disable 1030    // OK
    }
}
```

Compiler Warning (level 1) CS1697

Error Message

Different checksum values given for 'file name'

You have specified more than one checksum for a given file. The debugger uses the checksum value to determine which file to debug when there is more than one file in a project with the same name. Most users will not encounter this error, but if you are writing an application that generates code, you may run into it. To resolve this error, ensure that you generate the checksum only once for any given code file.

Compiler Warning (level 1) CS1699

Error Message

Use command line option "compiler_option" or appropriate project settings instead of "attribute_name"

In order to sign an assembly, it is necessary to specify a key file. Prior to Microsoft Visual C# 2005, you specified the key file using CLR attributes in source code. These attributes are now deprecated.

Beginning in Microsoft Visual C# 2005, you should use the **Signing Page** of the **Project Designer** or the Assembly Linker to specify the key file.

The **Signing Page** of the **Project Designer** is the preferred method; for more information, see [Signing Page, Project Designer](#) and [Managing Assembly and Manifest Signing](#).

The [How to: Sign an Assembly with a Strong Name](#) uses the following compiler options:

- [/keyfile \(Specify Strong Name Key File\) \(C# Compiler Options\)](#) instead of the [AssemblyKeyFileAttribute](#) attribute.
- [/keycontainer \(Specify Strong Name Key Container\) \(C# Compiler Options\)](#) instead of [AssemblyKeyNameAttribute](#).
- [/delaysign \(Delay Sign the Assembly\) \(C# Compiler Options\)](#) instead of [AssemblyDelaySignAttribute](#).

These attributes have been deprecated for the following reasons:

- There were security issues due to the attributes being embedded in the binary files produced by the compiler. Everyone who had your binary also had the keys stored in it.
- There were usability issues due to the fact that the path specified in the attributes was relative to the current working directory, which could change in the integrated development environment (IDE), or to the output directory. Thus, most times the key file is likely to be `..\..\mykey.snk`. Attributes also make it more difficult for the project system to properly sign satellite assemblies. When you use the compiler options instead of these attributes, you can use a fully qualified path and file name for the key without anything being embedded in the output file; the project system and source code control system can properly manipulate that full path when projects are moved around; the project system can maintain a project-relative path to the key file, and still pass a full path to the compiler; other build programs can more easily sign outputs by passing the proper path directly to the compiler instead of generating a source file with the correct attributes.
- Using attributes with friend assemblies can hamper compiler efficiency. When you use attributes, the compiler does not know what the key is when it has to decide whether or not to grant friendship and so it has to guess. At the end of compilation, the compiler is able to verify the guess once it finally knows the key. When the key file is specified with a compiler option, the compiler can immediately decide whether to grant friendship.

Example

The following sample generates CS1699. To resolve the error, remove the attribute and compile with **/delaysign**.

```
// CS1699.cs
// compile with: /target:library
[assembly:System.Reflection.AssemblyDelaySign(true)] // CS1699
```

See Also

Tasks

[How to: Sign an Assembly with a Strong Name](#)

Reference

[Signing Page, Project Designer](#)

Other Resources

[Managing Assembly and Manifest Signing](#)

Compiler Warning (level 1) CS1707

Error Message

Delegate 'DelegateName' bound to 'MethodName1' instead of 'MethodName2' because of new language rules

C# 2.0 implements new rules for binding a delegate to a method. Additional information is considered that was not looked at in the past. This warning indicates that the delegate is now bound to a different overload of the method than it was previously bound to. You may wish to verify that the delegate really should be bound to 'MethodName1' instead of 'MethodName2'.

For a description of how the compiler determines which method to bind a delegate to, see

[Covariance and Contravariance in Delegates \(C# Programming Guide\)](#).

Compiler Warning (level 1) CS1709

Error Message

Filename specified for preprocessor directive is empty

You have specified a preprocessor directive that includes a file name, but that file is empty. To resolve this warning, put the needed content into the file.

Example

The following example generates CS1709.

```
// CS1709.cs
class Test
{
    static void Main()
    {
        #pragma checksum "" "{406EA660-64CF-4C82-B6F0-42D48172A799}" "" // CS1709
    }
}
```

Compiler Warning (level 1) CS1720

Error Message

Expression will always cause a `System.NullReferenceException` because the default value of 'generic type' is null

If you write an expression involving the default of a generic type variable that is a reference type (for example, a class), this error will occur. Consider the following expression:

```
default(T).ToString()
```

Since `T` is a reference type, its default value is null, and so attempting to apply the [ToString](#) method to it will throw a [NullReferenceException](#).

Example

The class reference constraint on type `T` ensures that `T` is a reference type.

The following sample generates CS1720.

```
// CS1720.cs
using System;
public class Tester
{
    public static void GenericClass<T>(T t1) where T : class
    {
        Console.WriteLine(default(T).ToString()); // CS1720
    }
    public static void Main() {}
}
```

Compiler Warning (level 1) CS1723

Error Message

XML comment on 'param' has cref attribute 'attribute' that refers to a type parameter

This error is generated by an XML comment that refers to a type parameter.

Example

The following example generates CS1723.

```
// CS1723.cs
// compile with: /t:library /doc:filename.XML
///<summary>A generic list class.</summary>
///<see cref="T" />    // CS1723
// To resolve comment the previous line.
public class List<T>
{
}
```

Compiler Warning (level 1) CS1911

Error Message

Access to member 'member' through a 'base' keyword from an anonymous method or iterator results in unverifiable code.

Calling virtual functions with the **base** keyword inside the method body of an iterator or anonymous methods will result in unverifiable code. Unverifiable code will fail to run in a partial trust environment.

One resolution for CS1911 is to move the virtual function call to a helper function.

Example

The following sample generates CS1911.

```
// CS1911.cs
// compile with: /W:1
using System;

delegate void D();
delegate D RetD();

class B {
    protected virtual void M() {
        Console.WriteLine("B.M");
    }
}

class Der : B {
    protected override void M() {
        Console.WriteLine("D.M");
    }

    void Test() { base.M(); }
    D Test2() { return new D(base.M); }

    public D CallBaseM() {
        return delegate () { base.M(); };    // CS1911

        // try the following line instead
        // return delegate () { Test(); };
    }

    public RetD DelToBaseM() {
        return delegate () { return new D(base.M); };    // CS1911

        // try the following line instead
        // return delegate () { return Test2(); };
    }
}

class Program {
    public static void Main() {
        Der der = new Der();
        D d = der.CallBaseM();
        d();
        RetD rd = der.DelToBaseM();
        rd();
    }
}
```


Compiler Warning (level 1) CS2002

Error Message

Source file 'file' specified multiple times

A source file name was passed to the compiler more than once. You can only specify a file once to the compiler to build an output file.

The following sample generates CS2002:

```
// CS2002.cs
// compile with: CS2002.cs
public class A
{
    public static void Main(){}
}
```

To generate the error, compile the example with the command line:

```
csc CS2002.cs CS2002.cs
```

Compiler Warning (level 1) CS2014

Error Message

Compiler option 'old option' is obsolete, please use 'new option' instead

The form of the compiler option is deprecated. See [C# Compiler Options](#) for more information.

Compiler Warning (level 1) CS2023

Error Message

Ignoring /noconfig option because it was specified in a response file

The [/noconfig](#) compiler option was specified in a response file, which is not allowed.

Compiler Warning (level 1) CS3000

Error Message

Methods with variable arguments are not CLS-compliant

The arguments used in the method expose features that are not in the Common Language Specifications (CLS). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#).

The following example generates the warning CS3000.

```
// CS3000.cs
// compile with: /target:library
// CS3000 expected
[assembly:System.CLSCompliant(true)]

public class Test
{
    public void AddABunchOfInts( __arglist ) {}    // CS3000
}
```

Compiler Warning (level 1) CS3001

Error Message

Argument type 'type' is not CLS-compliant

A [public](#), [protected](#), or **protected internal** method must accept a parameter whose type is compliant with the Common Language Specification (CLS). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3001:

```
// CS3001.cs

[assembly:System.CLSCompliant(true)]
public class a
{
    public void bad(ushort i)    // CS3001
    {
    }

    private void OK(ushort i)    // OK, private method
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3002

Error Message

Return type of 'method' is not CLS-compliant

A [public](#), [protected](#), or **protected internal** method must return a value whose type is compliant with the Common Language Specification (CLS). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3002:

```
// CS3002.cs

[assembly:System.CLSCompliant(true)]
public class a
{
    public ushort bad()    // CS3002, public method
    {
        ushort a;
        a = ushort.MaxValue;
        return a;
    }

    private ushort OK()    // OK, private method
    {
        ushort a;
        a = ushort.MaxValue;
        return a;
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3003

Error Message

Type of 'variable' is not CLS-compliant

A [public](#), [protected](#), or **protected internal** variable must be of a type that is compliant with the Common Language Specification (CLS). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3003:

```
// CS3003.cs

[assembly:System.CLSCompliant(true)]
public class a
{
    public ushort a1;    // CS3003, public variable
    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3004

Error Message

Mixed and decomposed Unicode characters are not CLS-compliant

Only composed UNICODE characters are allowed in [public](#), [protected](#), or **protected internal** identifiers in order to be compliant with the Common Language Specification (CLS). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Compiler Warning (level 1) CS3005

Error Message

Identifier 'identifier' differing only in case is not CLS-compliant

A [public](#), [protected](#), or **protected internal** identifier, which differs from another **public**, **protected**, or **protected internal** identifier only in the case of one or more letters, is not compliant with the Common Language Specification (CLS). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3003:

```
// CS3005.cs

using System;

[assembly:CLSCompliant(true)]
public class a
{
    public static int a1 = 0;
    public static int A1 = 1;    // CS3005

    public static void Main()
    {
        Console.WriteLine(a1);
        Console.WriteLine(A1);
    }
}
```

Compiler Warning (level 1) CS3006

Error Message

Overloaded method 'method' differing only in ref or out, or in array rank, is not CLS-compliant

A method does not cannot be overloaded based on the [ref](#) or [out](#) parameter and still comply with the Common Language Specification (CLS). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3006. To resolve this warning, comment out the assembly-level attribute or remove one of the method definitions.

```
// CS3006.cs

using System;

[assembly: CLSCompliant(true)]
public class MyClass
{
    public void f(int i)
    {
    }

    public void f(ref int i)    // CS3006
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3007

Error Message

Overloaded method 'method' differing only by unnamed array types is not CLS-compliant

This error occurs if you have an overloaded method that takes a jagged array and the only difference between the method signatures is the element type of the array. To avoid this error, consider using a rectangular array rather than a jagged array; use an additional parameter to disambiguate the function call; rename one or more of the overloaded methods; or, if CLS Compliance is not needed, remove the [CLSCompliantAttribute](#) attribute. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3007:

```
// CS3007.cs
[assembly: System.CLSCompliant(true)]
public struct S
{
    public void F(int[][] array) { }
    public void F(byte[][] array) { } // CS3007
    // Try this instead:
    // public void F1(int[][] array) {}
    // public void F2(byte[][] array) {}
    // or
    // public void F(int[, ] array) {}
    // public void F(byte[, ] array) {}
}
```

Compiler Warning (level 1) CS3008

Error Message

Identifier 'identifier' is not CLS-compliant

A [public](#), [protected](#), or **protected internal** identifier breaks compliance with the Common Language Specification (CLS) if it begins with an underscore character (_). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3008:

```
// CS3008.cs

using System;

[assembly:CLSCompliant(true)]
public class a
{
    public static int _a = 0; // CS3008
    // OK, private
    // private static int _a1 = 0;

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3009

Error Message

'type': base type 'type' is not CLS-compliant

A base type was marked as not having to be compliant with the Common Language Specification (CLS) in an assembly that was marked as being CLS compliant. Either remove the attribute that specifies the assembly is CLS compliant or remove the attribute that indicates the type is not CLS compliant. For more information on CLS Compliance, see

[Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3009:

```
// CS3009.cs

using System;

[assembly:CLSCompliant(true)]
[CLSCompliant(false)]
public class B
{
}

public class C : B    // CS3009
{
    public static void Main () {}
}
```

Compiler Warning (level 1) CS3010

Error Message

'member': CLS-compliant interfaces must have only CLS-compliant members

In an assembly marked with `[assembly:CLSCompliant(true)]`, an interface contains a member marked with `[CLSCompliant(false)]`. Remove one of the Common Language Specification (CLS) compliance attributes. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3010:

```
// CS3010.cs

using System;

[assembly:CLSCompliant(true)]
public interface I
{
    [CLSCompliant(false)]
    int mf();    // CS3010
}

public class C : I
{
    public int mf()
    {
        return 1;
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3011

Error Message

'member': only CLS-compliant members can be abstract

A class member cannot be both [abstract](#) and non-compliant with the Common Language Specification (CLS). The CLS specifies that all class members shall be implemented. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3011:

```
// CS3011.cs

using System;

[assembly:CLSCompliant(true)]
public abstract class I
{
    [CLSCompliant(false)]
    public abstract int mf();    // CS3011

    // OK
    [CLSCompliant(false)]
    public void mf2()
    {
    }
}

public class C : I
{
    public override int mf()
    {
        return 1;
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3012

Error Message

You must specify the CLSCompliant attribute on the assembly, not the module, to enable CLS compliance checking

In order for a module to be compliant with the Common Language Specification (CLS) through `[module:System.CLSCompliant(true)]`, it must be built with the [/target:module](#) compiler option. For more information on the CLS, see [Common Language Specification](#).

Example

The following example, when built without **/target:module**, generates CS3012:

```
// CS3012.cs
// compile with: /W:1

[module:System.CLSCompliant(true)]    // CS3012
public class C
{
    public static void Main()
    {
    }
}
```


Compiler Warning (level 1) CS3013

Error Message

Added modules must be marked with the CLSCompliant attribute to match the assembly

A module that was compiled with the [/target:module](#) compiler option was added to a compilation with [/addmodule](#). However, the module's compliance with the Common Language Specification (CLS) does not agree with the CLS state of the current compilation.

CLS compliance is indicated with the module attribute. For example, `[module:CLSCompliant(true)]` indicates that the module is CLS compliant, and `[module:CLSCompliant(false)]` indicates that the module is not CLS compliant. The default is `[module:CLSCompliant(false)]`. For more information on the CLS, see [Common Language Specification](#).

Compiler Warning (level 1) CS3014

Error Message

'member' cannot be marked as CLS compliant because the assembly does not have a CLSCompliant attribute

In a source code file where compliance with the Common Language Specification (CLS) was not specified, a construct in the file was marked as being CLS compliant. This is not allowed. To resolve this warning, add an assembly level CLS compliant attribute to the file (in the example below, uncomment the line containing the assembly level attribute). For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3014:

```
// CS3014.cs

using System;

// [assembly:CLSCompliant(true)]
public class I
{
    [CLSCompliant(true)]    // CS3014
    public void mf()
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3015

Error Message

'method signature' has no accessible constructors which use only CLS-compliant types

To be compliant with the Common Language Specification (CLS), the argument list of an attribute class cannot contain an array. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following sample generates C3015.

```
// CS3015.cs
// compile with: /target:library
using System;

[assembly:CLSCompliant(true)]
public class MyAttribute : Attribute
{
    public MyAttribute(int[] ai) {}    // CS3015
    // try the following line instead
    // public MyAttribute(int ai) {}
}
```

Compiler Warning (level 1) CS3016

Error Message

Arrays as attribute arguments is not CLS-compliant

It is not compliant with the Common Language Specification (CLS) to pass an array to an attribute. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3016:

```
// CS3016.cs

using System;

[assembly : CLSCompliant(true)]
[C(new int[] {1, 2})] // CS3016
// try the following line instead
// [C()]
class C : Attribute
{
    public C()
    {
    }

    public C(int[] a)
    {
    }

    public static void Main ()
    {
    }
}
```

Compiler Warning (level 1) CS3017

Error Message

You cannot specify the CLSCompliant attribute on a module that differs from the CLSCompliant attribute on the assembly

This warning occurs if you have a assembly CLSCompliant attribute that conflicts with a module CLSCompliant attribute. An assembly that is CLS compliant cannot contain modules that are not CLS compliant. To resolve this warning, make sure the assembly and module CLSCompliant attributes are either both true or both false, or remove one of the attributes. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following example generates CS3017:

```
// CS3017.cs
// compile with: /target:module

using System;

[module: CLSCompliant(true)]
[assembly: CLSCompliant(false)] // CS3017
// Try this line instead:
// [assembly: CLSCompliant(true)]
class C
{
    static void Main() {}
}
```

Compiler Warning (level 1) CS3018

Error Message

'type' cannot be marked as CLS-Compliant because it is a member of non CLS-compliant type 'type'

This warning occurs if a nested class with the `CLSCompliant` attribute set to **true** is declared as a member of a class declared with the `CLSCompliant` attribute set to **false**. This is not allowed, since a nested class cannot be CLS-compliant if it is a member of an outer class that is not CLS-compliant. To resolve this warning, remove the `CLSCompliant` attribute from the nested class, or change it from **true** to **false**. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#) and [Common Language Specification](#).

Example

The following sample generates CS3018.

```
// CS3018.cs
// compile with: /target:library
using System;

[assembly: CLSCompliant(true)]
[CLSCompliant(false)]
public class Outer
{
    [CLSCompliant(true)]    // CS3018
    public class Nested {}

    // OK
    public class Nested2 {}

    [CLSCompliant(false)]
    public class Nested3 {}
}
```

Compiler Warning (level 1) CS3022

Error Message

CLSCompliant attribute has no meaning when applied to parameters. Try putting it on the method instead.

Method parameters are not checked for CLS Compliance, since the CLS Compliance rules apply to methods and type declarations.

Example

The following sample generates CS3022:

```
// CS3022.cs
// compile with: /W:1

using System;

[assembly: CLSCompliant(true)]
[CLSCompliant(true)]
public class C
{
    public void F([CLSCompliant(true)] int i)
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 1) CS3023

Error Message

CLSCompliant attribute has no meaning when applied to return types. Try putting it on the method instead.

Function return types are not checked for CLS Compliance, since the CLS Compliance rules apply to methods and type declarations.

Example

The following example generates warning CS3023:

```
// C3023.cs

[assembly:System.CLSCompliant(true)]
public class Test
{
    [return:System.CLSCompliant(true)] // CS3023
    // Try this instead:
    // [method:System.CLSCompliant(true)]
    public static int Main()
    {
        return 0;
    }
}
```


Compiler Warning (level 1) CS3026

Error Message

CLS-compliant field 'field' cannot be volatile

A volatile variable should not be CLS compliant.

Example

The following example generates CS3026.

```
// CS3026.cs
[assembly:System.CLSCompliant(true)]
public class Test
{
    public volatile int v0 =0;    // CS3026
    // To resolve remove the CLS-CCompliant attribute.
    public static void Main() { }
}
```

Compiler Warning (level 1) CS5000

Error Message

Unknown compiler option '/option'

An invalid [compiler option](#) was specified.

Compiler Warning (level 2) CS0108

Error Message

'member1' hides inherited member 'member2'. Use the new keyword if hiding was intended.

A variable was declared with the same name as a variable in a base class. However, the [new](#) keyword was not used. This warning informs you that you should use **new**; the variable is declared as if **new** had been used in the declaration.

The following sample generates CS0108:

```
// CS0108.cs
// compile with: /W:2
using System;

namespace x
{
    public class clx
    {
        public int i = 1;
    }

    public class cly : clx
    {
        public static int i = 2;    // CS0108, use the new keyword
        // the compiler parses the previous line as if you had specified:
        // public static new int i = 2;

        public static void Main()
        {
            Console.WriteLine(i);
        }
    }
}
```

Compiler Warning (level 2) CS0114

Error Message

'function1' hides inherited member 'function2'. To make the current method override that implementation, add the override keyword. Otherwise add the new keyword.

A declaration in a class conflicts with a declaration in a base class such that the base class member will be hidden.

For more information, see [base](#).

The following sample generates CS0114:

```
// CS0114.cs
// compile with: /W:2 /warnaserror
abstract public class clx
{
    public abstract void f();
}

public class cly : clx
{
    public void f() // CS0114, hides base class member
    // try the following line instead
    // override public void f()
    {
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 2) CS0162

Error Message

Unreachable code detected

The compiler detected code that will never be executed.

The following sample generates CS0162:

```
// CS0162.cs
// compile with: /W:2
public class A
{
    public static void Main()
    {
        goto lab1;
        {
            // The following statements cannot be reached:
            int i = 9;    // CS0162
            i++;
        }
        lab1:
        {
        }
    }
}
```

Compiler Warning (level 2) CS0164

Error Message

This label has not been referenced

A label was declared but never used.

The following sample generates CS0164:

```
// CS0164.cs
// compile with: /W:2
public class a
{
    public int i = 0;

    public static void Main()
    {
        int i = 0;    // CS0164
        l1: i++;
        // the following lines resolve this error
        // if(i < 10)
        //     goto l1;
    }
}
```

Compiler Warning (level 2) CS0251

Error Message

Indexing an array with a negative index (array indices always start at zero)

Do not use a negative number to index into an array.

The following sample generates CS0251:

```
// CS0251.cs
// compile with: /W:2
class MyClass
{
    public static void Main()
    {
        int[] myarray = new int[] {1,2,3};
        try
        {
            myarray[-1]++;    // CS0251
            // try the following line instead
            // myarray[1]++;
        }
        catch (System.IndexOutOfRangeException e)
        {
            System.Console.WriteLine("{0}", e);
        }
    }
}
```

Compiler Warning (level 2) CS0252

Error Message

Possible unintended reference comparison; to get a value comparison, cast the left hand side to type 'type'

The compiler is doing a reference comparison. If you want to compare the value of strings, cast the left side of the expression to *type*.

The following sample generates CS0252:

```
// CS0252.cs
// compile with: /W:2
using System;

class MyClass
{
    public static void Main()
    {
        string s = "11";
        object o = s + s;

        bool b = o == s;    // CS0252
        // try the following line instead
        // bool b = (string)o == s;
    }
}
```


Compiler Warning (level 2) CS0253

Error Message

Possible unintended reference comparison; to get a value comparison, cast the right hand side to type 'type'

The compiler is doing a reference comparison. If you want to compare the value of strings, cast the right side of the expression to *type*.

The following sample generates CS0253:

```
// CS0253.cs
// compile with: /W:2
using System;
class MyClass
{
    public static void Main()
    {
        string s = "11";
        object o = s + s;

        bool c = s == o;    // CS0253
        // try the following line instead
        // bool c = s == (string)o;
    }
}
```

Compiler Warning (level 2) CS0278

Error Message

'type' does not implement the 'pattern name' pattern. 'method name' is ambiguous with 'method name'.

There are several statements in C# that rely on defined patterns, such as **foreach** and **using**. For example, **foreach** relies on the collection class implementing the "enumerable" pattern.

CS0278 can occur if the compiler is unable to make the match due to ambiguities. For example, the "enumerable" pattern requires that there be a method called **MoveNext**, and your code might contain two methods called **MoveNext**. The compiler will attempt to find an interface to use, but it is recommended that you determine and resolve the cause of the ambiguity.

For more information, see [How to: Access a Collection Class with foreach \(C# Programming Guide\)](#).

Example

The following sample generates CS0278.

```
// CS0278.cs
using System.Collections.Generic;
public class myTest
{
    public static void TestForeach<W>(W w)
        where W: IEnumerable<int>, IEnumerable<string>
    {
        foreach (int i in w) {}    // CS0278
    }
}
```

Compiler Warning (level 2) CS0279

Error Message

'type name' does not implement the 'pattern name' pattern. 'method name' is either static or not public.

There are several statements in C# that rely on defined patterns, such as **foreach** and **using**. For example, **foreach** relies on the collection class implementing the enumerable pattern. This error occurs when the compiler is unable to make the match due to a method being declared **static** or not **public**. Methods in patterns are required to be instances of classes, and to be public.

Example

The following example generates CS0279:

```
// CS0279.cs

using System;
using System.Collections;

public class myTest : IEnumerable
{
    IEnumerator IEnumerable.GetEnumerator()
    {
        yield return 0;
    }

    internal IEnumerator GetEnumerator()
    {
        yield return 0;
    }

    public static void Main()
    {
        foreach (int i in new myTest()) {} // CS0279
    }
}
```

Compiler Warning (level 2) CS0280

Error Message

'type' does not implement the 'pattern name' pattern. 'method name' has the wrong signature.

Two statements in C#, **foreach** and **using**, rely on predefined patterns, "collection" and "resource" respectively. This warning occurs when the compiler cannot match one of these statements to its pattern due to a method's incorrect signature. For example, the "collection" pattern requires that there be a method called [MoveNext](#) which takes no parameters and returns a **boolean**. Your code might contain a **MoveNext** method that has a parameter or perhaps returns an object.

The "resource" pattern and **using** provide another example. The "resource" pattern requires the [Dispose](#) method; if you define a property with the same name, you will get this warning.

To resolve this warning, ensure that the method signatures in your type match the signatures of the corresponding methods in the pattern, and ensure that you have no properties with the same name as a method required by the pattern.

Example

The following sample generates CS0280.

```
// CS0280.cs
using System;
using System.Collections;

public class ValidBase: IEnumerable
{
    IEnumerator IEnumerable.GetEnumerator()
    {
        yield return 0;
    }

    internal IEnumerator GetEnumerator()
    {
        yield return 0;
    }
}

class Derived : ValidBase
{
    // field, not method
    new public int GetEnumerator;
}

public class Test
{
    public static void Main()
    {
        foreach (int i in new Derived()) {}    // CS0280
    }
}
```

Compiler Warning (level 2) CS0435

Error Message

The namespace 'namespace' in 'file_2' conflicts with the imported type 'type' in 'file_1'. Using the namespace.

This warning is issued when a namespace in a source file (file_2) conflicts with an imported type in file_1. The compiler uses the one in the source file.

The following example generates CS0435:

Compile this file first:

```
// CS0435_1.cs
// compile with: /t:library
public class Util
{
    public class A { }
}
```

Then, compile this file:

```
// CS0435_2.cs
// compile with: /r:CS0435_1.dll

using System;

namespace Util
{
    public class A { }
}

public class Test
{
    public static void Main()
    {
        Console.WriteLine(typeof(Util.A)); // CS0435
    }
}
```

Compiler Warning (level 2) CS0436

Error Message

The type 'type' in 'file_2' conflicts with the imported type 'type'. Using the one in 'file_2'

This warning is issued when a type in a source file (file_2) conflicts with an imported type in file_1. The compiler uses the one in the source file.

Example

```
// CS0436_a.cs
// compile with: /target:library
public class A {
    public void Test() {
        System.Console.WriteLine("CS0436_a");
    }
}
```

The following example generates CS0436.

```
// CS0436_b.cs
// compile with: /reference:CS0436_a.dll
// CS0436 expected
public class A {
    public void Test() {
        System.Console.WriteLine("CS0436_b");
    }
}

public class Test
{
    public static void Main()
    {
        A x = new A();
        x.Test();
    }
}
```

Compiler Warning (level 2) CS0437

Error Message

The type 'type' in 'file_2' conflicts with the imported namespace 'namespace' in 'file_1'. Using the type.

This warning is issued when a type in a source file, file_2, conflicts with an imported namespace in file_1. The compiler uses the type in the source file.

Example

```
// CS0437_a.cs
// compile with: /target:library
namespace Util
{
    public class A {
        public void Test() {
            System.Console.WriteLine("CS0437_a.cs");
        }
    }
}
```

The following sample generates CS0437.

```
// CS0437_b.cs
// compile with: /reference:CS0437_a.dll /W:2
// CS0437 expected
class Util
{
    public class A {
        public void Test() {
            System.Console.WriteLine("CS0437_b.cs");
        }
    }
}

public class Test
{
    public static void Main()
    {
        Util.A x = new Util.A();
        x.Test();
    }
}
```

Compiler Warning (level 2) CS0440

Error Message

Defining an alias named 'global' is ill-advised since 'global::' always references the global namespace and not an alias

This warning is issued when you define an alias named global.

Example

The following example generates CS0440:

```
// CS0440.cs
// Compile with: /W:1

using global = MyClass;    // CS0440
class MyClass
{
    static void Main()
    {
        // Note how global refers to the global namespace
        // even though it is redefined above.
        global::System.Console.WriteLine();
    }
}
```


Compiler Warning (level 2) CS0444

Error Message

Predefined type 'type name 1' was not found in 'System namespace 1' but was found in 'System namespace 2'

A predefined object such as [Int32](#) was not found where the compiler expected to find it, but instead found it in 'System namespace 2'.

The error could indicate that the .NET Framework is installed incorrectly. To fix this, reinstall the .NET Framework.

If you are writing your own base class libraries, you might also encounter this error. In this case, to resolve the error, rebuild mscorlib.

Compiler Warning (level 2) CS0458

Error Message

The result of the expression is always 'null' of type 'type name'

This warning is caused by a **nullable** expression that always results in **null**.

The following code generates warning CS0458.

Example

This example illustrates a number of the different operations with **nullable** types that will cause this error.

```
// CS0458.cs
using System;
public class Test
{
    public static void Main()
    {
        int a = 5;
        int? b = a + null;    // CS0458
        int? qa = 15;
        b = qa + null;        // CS0458
        b -= null;            // CS0458
        int? qa2 = null;
        b = qa2 + null;       // CS0458
        qa2 -= null;          // CS0458
    }
}
```

Compiler Warning (level 2) CS0464

Error Message

Comparing with null of type 'type' always produces 'false'

This warning is produced when you perform a comparison between a nullable variable and null, and the comparison is not == or !=. To resolve this error, verify if you really want to check a value for **null**. A comparison like `i == null` can be either true or false. A comparison like `i > null` is always false.

Example

The following sample generates CS0464.

```
// CS0464.cs
class MyClass
{
    public static void Main()
    {
        int? i = 0;
        if (i < null) ;    // CS0464

        i++;
    }
}
```

Compiler Warning (level 2) CS0467

Error Message

Ambiguity between method 'method' and non-method 'non-method'. Using method group.

Inherited members with same signature, from different interfaces, cause ambiguity error.

Example

The following example generates CS0467.

```
// CS0467.cs
interface IList
{
    int Count { get; set; }
}
interface ICounter
{
    void Count(int i);
}

interface IListCounter : IList, ICounter {}

class Driver
{
    void Test(IListCounter x)
    {
        x.Count = 1;
        x.Count(1);    // CS0467
        // To resolve change the method name "Count" to another name.
    }

    static void Main()
    {
    }
}
```

Compiler Warning (level 2) CS0618

Error Message

'member' is obsolete: 'text'

A class member was marked with the [Obsolete](#) attribute, such that a warning will be issued when the class member is referenced.

The following sample generates CS0618:

```
// CS0618.cs
// compile with: /W:2
using System;

public class C
{
    [Obsolete("Use newMethod instead", false)]    // warn if referenced
    public static void m2()
    {
    }

    public static void newMethod()
    {
    }
}

class MyClass
{
    public static void Main()
    {
        C.m2();    // CS0618
    }
}
```

Compiler Warning (level 2) CS0652

Error Message

Comparison to integral constant is useless; the constant is outside the range of type 'type'

The compiler detected a comparison between a constant and a variable where the constant is out of the range of the variable.

The following sample generates CS0652:

```
// CS0652.cs
// compile with: /W:2
public class Class1
{
    private static byte i = 0;
    public static void Main()
    {
        short j = 256;
        if (i == 256)    // CS0652, 256 is out of range for byte
            i = 0;
    }
}
```

Compiler Warning (level 2) CS0728

Error Message

Possibly incorrect assignment to local 'variable' which is the argument to a using or lock statement. The Dispose call or unlocking will happen on the original value of the local.

There are several scenarios where **using** or **lock** blocks will result in a temporary leak of resources. Here is one example:

```
thisType f = null;

using (f)
{
    f = new thisType();
    ...
}
```

In this case, the original value, such as null, of the variable `thisType` will be disposed of when the **using** block finishes executing, but the `thisType` object created inside the block will not be, although it will eventually get garbage collected.

To resolve this error, use the following form:

```
using (thisType f = new thisType())
{
    ...
}
```

In this case, the newly allocated `thisType` object will be disposed of.

Example

The following code will generate warning CS0728.

```
// CS0728.cs

using System;
public class ValidBase : IDisposable
{
    public void Dispose() { }
}

public class Logger
{
    public static void dummy()
    {
        ValidBase vb = null;
        using (vb)
        {
            vb = null; // CS0728
        }
        vb = null;
    }
    public static void Main() { }
}
```

Compiler Warning (level 2) CS1571

Error Message

XML comment on 'construct' has a duplicate param tag for 'parameter'

When using the [/doc](#) compiler option, multiple comments were found for the same method parameter. Remove one of the duplicate lines.

The following sample generates CS1571:

```
// CS1571.cs
// compile with: /W:2 /doc:x.xml

/// <summary>help text</summary>
public class MyClass
{
    /// <param name='Int1'>Used to indicate status.</param>
    /// <param name='Char1'>An initial.</param>
    /// <param name='Int1'>Used to indicate status.</param> // CS1571
    public static void MyMethod(int Int1, char Char1)
    {
    }

    /// <summary>help text</summary>
    public static void Main ()
    {
    }
}
```


Compiler Warning (level 2) CS1572

Error Message

XML comment on 'construct' has a param tag for 'parameter', but there is no parameter by that name

When using the [/doc](#) compiler option, a comment was specified for a parameter that does not exist for the method. Change the value passed to the name attribute or remove one of the comment lines.

The following sample generates CS1572:

```
// CS1572.cs
// compile with: /W:2 /doc:x.xml

/// <summary>help text</summary>
public class MyClass
{
    /// <param name='Int1'>Used to indicate status.</param>
    /// <param name='Char1'>Used to indicate status.</param>
    /// <param name='Char2'>??</param> // CS1572
    public static void MyMethod(int Int1, char Char1)
    {
    }

    /// <summary>help text</summary>
    public static void Main ()
    {
    }
}
```

Compiler Warning (level 2) CS1587

Error Message

XML comment is not placed on a valid language element

Recommended tags for documentation comments are not allowed on all language elements. For example, a tag is not allowed on a namespace. For more information on XML comments, see

[Recommended Tags for Documentation Comments \(C# Programming Guide\)](#).

Example

The following sample generates CS1587:

```
// CS1587.cs
// compile with: /W:2 /doc:x.xml

/// <summary>test</summary>    // CS1587, tag not allowed on namespace
namespace MySpace
{
    class MyClass
    {
        public static void Main()
        {
        }
    }
}
```

Compiler Warning (level 2) CS1668

Error Message

Invalid search path 'path' specified in 'path string' -- 'system error message'

The path supplied to [/lib](#) at the command line was not valid, or a path in the LIB environment variable is invalid. Check the path used to verify that it exists and is accessible. The error message in single quotes is the error returned from the operating system.

One known cause of this error is when Visual Studio .NET 2002 or Visual Studio .NET 2003 are uninstalled from a machine that also has Visual Studio 2005 installed. Use the following procedure to fix the error:

1. Right-click **My Computer** and then click **Properties**.
2. Click the **Advanced** tab and then click **Environment Variables**.
3. Click the LIB entry in the top pane and make sure it points to the v2.0 lib files.

If you installed Visual Studio 2005 to the default location, the full path is C:\Program Files\Microsoft Visual Studio 8\SDK\v2.0\Lib.

Compiler Warning (level 2) CS1698

Error Message

Circular assembly reference 'AssemblyName1' does not match the output assembly name 'AssemblyName2'. Try adding a reference to 'AssemblyName1' or changing the output assembly name to match.

CS1698 occurs when an assembly reference is incorrect. This can happen if a referenced assembly is recompiled. To resolve, do not replace an assembly that itself is a dependency of an assembly you are referencing.

Example

```
// CS1698_a.cs
// compile with: /target:library /keyfile:mykey.snk
[assembly:System.Reflection.AssemblyVersion("2")]
public class CS1698_a {}
```

```
// CS1698_b.cs
// compile with: /target:library /reference:CS1698_a.dll /keyfile:mykey.snk
public class CS1698_b : CS1698_a {}
```

The following sample generates CS1698.

```
// CS1698_c.cs
// compile with: /target:library /out:cs1698_a.dll /reference:cs1698_b.dll /keyfile:mykey.s
nk
// CS1698 expected
[assembly:System.Reflection.AssemblyVersion("3")]
public class CS1698_c : CS1698_b {}
public class CS1698_a {}
```

Compiler Warning (level 2) CS1701

Error Message

Assuming assembly reference "Assembly Name #1" matches "Assembly Name #2", you may need to supply runtime policy

The two assemblies differ in release and/or version number. For unification to occur, you must specify directives in the application's .config file, and you must provide the correct strong name of an assembly, as demonstrated in the following example code.

Example

The following multifile sample references an assembly using two different external aliases. This first sample builds the older version of the code that creates assembly CS1701_d.

```
// CS1701_a.cs
// compile with: /target:library /out:cs1701_d.dll /keyfile:mykey.snk
using System.Reflection;
[assembly:AssemblyVersion("1.0")]
public class A {
    public void M1() {}
}

public class C1 {}
```

This is the code that creates the newer version of assembly CS1701_d. Note that it compiles into a different directory than the older version, necessary since the output files have the same names.

```
// CS1701_b.cs
// compile with: /target:library /out:c:\\cs1701_d.dll /keyfile:mykey.snk
using System.Reflection;
[assembly:AssemblyVersion("2.0")]
public class A {
    public void M2() {}
    public void M1() {}
}

public class C2 {}
public class C1 {}
```

This sample sets up the external aliases A1 and A2.

```
// CS1701_c.cs
// compile with: /target:library /reference:A2=c:\\cs1701_d.dll /reference:A1=cs1701_d.dll

extern alias A1;
extern alias A2;
// using System;
using a1 = A1::A;
using a2 = A2::A;

public class Ref {
    public static a1 A1() { return new a1(); }
    public static a2 A2() { return new a2(); }

    public static A1::C1 M1() { return new A1::C1(); }
    public static A2::C2 M2() { return new A2::C2(); }
}
```

This sample calls methods using two different aliases of A. The following sample generates C1701.

```
// CS1701_d.cs
// compile with: /reference:c:\\CS1701_d.dll /reference:CS1701_c.dll
```

```
// CS1701 expected
class Tester {
    public static void Main() {
        Ref.A1().M1();
        Ref.A2().M2();
    }
}
```

Compiler Warning (level 2) CS1710

Error Message

XML comment on 'type' has a duplicate typeparam tag for 'parameter'

The documentation of a generic type includes a duplicate tag for the type parameter.

Example

The following code will cause warning CS1710 to appear.

```
// CS1710.cs
// To resolve this warning, delete one of the duplicate <typeparam>'s.
using System;
class Stack<ItemType>
{
}

///<typeparam name="MyType">can be an int</typeparam>
///<typeparam name="MyType">can be an int</typeparam>
class MyStackWrapper<MyType>
{
    // Open constructed type Stack<MyType>.
    Stack<MyType> stack;
    public MyStackWrapper(Stack<MyType> s)
    {
        stack = s;
    }
}

class CMain
{
    public static void Main()
    {
        // Closed constructed type Stack<int>.
        Stack<int> stackInt = new Stack<int>();
        MyStackWrapper<int> MyStackWrapperInt =
            new MyStackWrapper<int>(stackInt);
    }
}
```

Compiler Warning (level 2) CS1711

Error Message

XML comment on 'type' has a typeparam tag for 'parameter', but there is no type parameter by that name

The documentation of a generic type includes a tag for the type parameter that has the wrong name.

Example

The following code generates warning CS1711.

```
// cs1711.cs
// compile with: /doc:cs1711.xml
// CS1711 expected
using System;
///
```


Compiler Warning (level 2) CS3019

Error Message

CLS compliance checking will not be performed on 'type' because it is not visible from outside this assembly.

This warning occurs when a type or a member that has the [CLSCompliantAttribute](#) attribute is not visible from another assembly. To resolve this error, remove the attribute on any classes or members that are not visible from the other assembly, or make the type or members visible. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#).

Example

The following sample generates CS3019:

```
// CS3019.cs
// compile with: /W:2

using System;

[assembly: CLSCompliant(true)]

// To fix the error, remove the next line
[CLSCompliant(true)] // CS3019
class C
{
    [CLSCompliant(false)] // CS3019
    void Foo()
    {
    }

    static void Main()
    {
    }
}
```

See Also

Concepts

[Common Language Specification](#)

Compiler Warning (level 2) CS3021

Error Message

'type' does not need a CLSCompliant attribute because the assembly does not have a CLSCompliant attribute

This warning occurs if `[CLSCompliant(false)]` appears on a class in an assembly which does not have an assembly-level CLSCompliant attribute set to true (i.e., the line `[assembly: CLSCompliant(true)]`). Since the assembly is not declaring itself CLS compliant, there is no need for anything within the assembly to declare itself non-compliant, since it is assumed to be non-compliant. For more information on CLS Compliance, see [Writing CLS-Compliant Code](#).

To get rid of this warning, remove the attribute or add the assembly level attribute.

Example

The following example generates CS3021:

```
// CS3021.cs
using System;
// Uncomment the following line to declare the assembly CLS Compliant,
// and avoid the warning without removing the attribute on the class.
//[assembly: CLSCompliant(true)]

// Remove the next line to avoid the warning.
[CLSCompliant(false)]           // CS3021
public class C
{
    public static void Main()
    {
    }
}
```

See Also

Concepts

[Common Language Specification](#)

Compiler Warning (level 3) CS0067

Error Message

The event 'event' is never used

An [event](#) was declared but never used in the class in which it was declared.

The following sample generates CS0067:

```
// CS0067.cs
// compile with: /W:3
using System;
delegate void MyDelegate();

class MyClass
{
    public event MyDelegate evt;    // CS0067
    // uncomment TestMethod to resolve this CS0067
    /*
    private void TestMethod()
    {
        if (evt != null)
            evt();
    }
    */
    public static void Main()
    {
    }
}
```

Compiler Warning (level 3) CS0105

Error Message

The using directive for 'namespace' appeared previously in this namespace

A [namespace](#), which should only be declared once, was declared more than once; remove all duplicate namespace declarations.

The following sample generates CS0105:

```
// CS0105.cs
// compile with: /W:3
using System;
using System;    // CS0105

public class a
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 3) CS0168

Error Message

The variable 'var' is declared but never used

The compiler warns when a variable is declared but not used.

The following sample generates two CS0168 warnings:

```
// CS0168.cs
// compile with: /W:3
public class clx
{
    public int i;
}

public class clz
{
    public static void Main()
    {
        int j = 0;    // CS0168, uncomment the following line
        // j++;
        clx a;        // CS0168, try the following line instead
        // clx a = new clx();
    }
}
```

Compiler Warning (level 3) CS0169

Error Message

The private field 'class member' is never used

A private variable was declared but never referenced. A common way to generate this warning is when you declare a private member of a class and do not use it.

The following sample generates CS0169:

```
// compile with: /W:3
using System;
public class ClassX
{
    int i;    // CS0169, i is not used anywhere

    public static void Main()
    {
    }
}
```

Compiler Warning (level 3) CS0219

Error Message

The variable 'variable' is assigned but its value is never used

The compiler issues a level-three warning, when you declare and assign a variable, but do not use it.

The following sample generates CS0219:

```
// CS0219.cs
// compile with: /W:3
public class MyClass
{
    public static void Main()
    {
        int a = 0;    // CS0219
    }
}
```

Compiler Warning (level 3) CS0282

Error Message

There is no defined ordering between fields in multiple declarations of partial class or struct 'type'. To specify an ordering, all instance fields must be in the same declaration.

To resolve this error, put all member variables in a single partial class definition.

A common way to get this error is by having a partial **struct** defined in more than one place, with some of the member variables in one definition, and some in another.

The following code generates CS0282.

Example

This code contains one description of a **struct**. Compile these two modules together in a single step by means of the command:

```
csc /target:library cs0282_1.cs cs0282_2.cs
```

```
partial struct A
{
    int i;
}
```

This code contains a conflicting description of the same **struct**.

```
partial struct A
{
    int j;
}
```


Compiler Warning (level 3) CS0414

Error Message

The private field 'field' is assigned but its value is never used

This warning occurs if you have a private field in a type that is assigned to, but that value is never subsequently read. The unnecessary assignment could impact performance.

The following sample generates CS0414:

```
// CS0414
// compile with: /W3
class C
{
    private int i = 1;  // CS0414

    public static void Main()
    { }
}
```

Compiler Warning (level 3) CS0419

Error Message

Ambiguous reference in cref attribute: 'Method Name1'. Assuming 'Method Name2', but could have also matched other overloads including 'Method Name3'.

In an XML documentation comment in the code, a reference could not be resolved. This could occur if the method is overloaded, or if two different identifiers with the same name are found. To resolve the warning, use a qualified name to disambiguate the reference, or include the specific overload in parentheses.

The following sample generates CS0419.

```
// cs0419.cs
// compile with: /doc:x.xml /W:3
interface I
{
    /// text for F(void)
    void F();
    /// text for F(int)
    void F(int i);
}
/// text for class MyClass
public class MyClass
{
    /// <see cref="I.F"/>
    public static void MyMethod(int i)
    {
    }
    /* Try this instead:
    /// <see cref="I.F(int)"/>
    public static void MyMethod(int i)
    {
    }
    */
    /// text for Main
    public static void Main ()
    {
    }
}
```

Compiler Warning (level 2) CS0469

Error Message

The 'goto case' value is not implicitly convertible to type 'type'

When you use **goto case**, there must be an implicit conversion from the value of the goto case to the type of the switch.

Example

The following sample generates CS0469.

```
// CS0469.cs
// compile with: /W:2
class Test
{
    static void Main()
    {
        char c = (char)180;
        switch (c)
        {
            case (char)127:
                break;

            case (char)180:
                goto case 127;    // CS0469
                // try the following line instead
                // goto case (char) 127;
        }
    }
}
```

Compiler Warning (level 3) CS0642

Error Message

Possible mistaken empty statement

A semicolon after a conditional statement may cause your code to execute differently than intended.

You can use **/nowarn** compiler option or **#pragma warning** to disable this warning; see [/nowarn \(Suppress Specified Warnings\) \(C# Compiler Options\)](#) or [#pragma warning \(C# Reference\)](#) for more information.

The following sample generates CS0642:

```
// CS0642.cs
// compile with: /W:3
class MyClass
{
    public static void Main()
    {
        int i;

        for (i = 0; i < 10; i += 1);    // CS0642 semicolon intentional?
        {
            System.Console.WriteLine (i);
        }
    }
}
```

Compiler Warning (level 3) CS0659

Error Message

'class' overrides Object.Equals(object o) but does not override Object.GetHashCode()

The compiler detected an override of the **Equals** function but no override for **GetHashCode**. An override of **Equals** implies that you also want to override **GetHashCode**.

For more information, see

- [Hashtable](#).
- [Guidelines for Implementing Equals and the Equality Operator \(==\)](#)
- [Implementing the Equals Method](#)
- [GetHashCode](#)

The following sample generates CS0659:

```
// CS0659.cs
// compile with: /W:3 /target:library
class Test
{
    public override bool Equals(object o) { return true; }    // CS0659
}

// OK
class Test2
{
    public override bool Equals(object o) { return true; }
    public override int GetHashCode() { return 0; }
}
```

Compiler Warning (level 3) CS0660

Error Message

'class' defines operator == or operator != but does not override Object.Equals(object o)

The compiler detected the user-defined equality or inequality operator, but no override for the **Equals** function. A user-defined equality or inequality operator implies that you also want to override the **Equals** function.

The following sample generates CS0660:

```
// CS0660.cs
// compile with: /W:3 /warnaserror
class Test    // CS0660
{
    public static bool operator == (object o, Test t)
    {
        return true;
    }

    // uncomment the Equals function to resolve
    // public override bool Equals(object o)
    // {
    //     return true;
    // }

    public override int GetHashCode()
    {
        return 0;
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 3) CS0661

Error Message

'class' defines operator == or operator != but does not override Object.GetHashCode()

The compiler detected the user-defined equality or inequality operator, but no override for the **GetHashCode** function. A user-defined equality or inequality operator implies that you also want to override the **GetHashCode** function.

The following sample generates CS0661:

```
// CS0661.cs
// compile with: /W:3
class Test    // CS0661
{
    public static bool operator == (object o, Test t)
    {
        return true;
    }

    public static bool operator != (object o, Test t)
    {
        return true;
    }

    public override bool Equals(object o)
    {
        return true;
    }

    // uncomment the GetHashCode function to resolve
    // public override int GetHashCode()
    // {
    //     return 0;
    // }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 3) CS0665

Error Message

Assignment in conditional expression is always constant; did you mean to use == instead of = ?

A conditional expression used the `= operator` and not the `== operator`.

The following sample generates CS0665:

```
// CS0665.cs
// compile with: /W:3
class Test
{
    public static void Main()
    {
        bool i = false;

        if (i = true)    // CS0665
            // try the following line instead
            // if (i == true)
            {
            }

        System.Console.WriteLine(i);
    }
}
```


Compiler Warning (level 3) CS0675

Error Message

Bitwise-or operator used on a sign-extended operand; consider casting to a smaller unsigned type first

The compiler implicitly widened and sign-extended a variable, and then used the resulting value in a bitwise OR operation. This can result in unexpected behavior.

The following sample generates CS0675:

```
// CS0675.cs
// compile with: /W:3
using System;

public class sign
{
    public static void Main()
    {
        int hi = 1;
        int lo = 1;
        long value = (((long)hi) << 32) | lo;           // CS0675
        // try the following line instead
        // long value = (((long)hi) << 32) | ((uint)lo); // correct
    }
}
```

Compiler Warning (level 3) CS0693

Error Message

Type parameter 'type parameter' has the same name as the type parameter from outer type 'type'

An inner class type parameter hides the outer class type parameter of the same name. To avoid this situation, use a different name for one of the type parameters.

Example

The following sample generates CS0693.

```
// CS0693.cs
// compile with: /W:3 /target:library
class Outer<T>
{
    class Inner<T> {}    // CS0693
    // try the following line instead
    // class Inner<U> {}
}
```

Compiler Warning (level 3) CS1700

Error Message

Assembly reference Assembly Name is invalid and cannot be resolved

This warning indicates that an attribute, such as [InternalsVisibleToAttribute](#), was not specified correctly.

For more information, see [Friend Assemblies \(C# Programming Guide\)](#).

Example

The following sample generates CS1700.

```
// CS1700.cs
// compile with: /target:library
using System.Runtime.CompilerServices;
[assembly:InternalsVisibleTo("app2, Retargetable=f")] // CS1700
[assembly:InternalsVisibleTo("app2")] // OK
```

Compiler Warning (level 3) CS1702

Error Message

Assuming assembly reference "Assembly Name #1" matches "Assembly Name #2", you may need to supply runtime policy

The two assembly references have differing build and/or revision numbers, so will not automatically unify. You may need to supply run-time policy to force unification by using directives in the application .config file.

Compiler Warning (level 3) CS1717

Error Message

Assignment made to same variable; did you mean to assign something else?

This warning occurs when you assign a variable to itself, such as `a = a`.

Several common mistakes can generate this warning:

- Writing `a = a` as the condition of an **if** statement, such as `if (a = a)`. You probably meant to say `if (a == a)`, which is always true, so you could write this more concisely as `if (true)`.
- Mistyping. You probably meant to say `a = b`.
- In a constructor where the parameter has the same name as the field, not using the **this** keyword: you probably meant to say `this.a = a`.

Example

The following sample generates CS1717.

```
// CS1717.cs
// compile with: /W:3
public class Test
{
    public static void Main()
    {
        int x = 0;
        x = x;    // CS1717
    }
}
```

Compiler Warning (level 3) CS1718

Error Message

Comparison made to same variable; did you mean to compare something else?

If you meant to compare to something else, then you should simply correct the statement.

But another possibility is that you were testing for true or false, and were doing so by statements such as `if (a == a) (true)` or `if (a < a) (false)`. It is better to simply say `if (true)` or `if (false)`. There are two reasons for this:

- It is simpler: it is always clearer to simply say what you mean.
- It helps avoid confusion: a new feature of C# 2.0 is nullable value types, which are analogous to the value **null** in T-SQL, the programming language used by SQL Server. Developers familiar with T-SQL might be concerned about the effect of nullable types on expressions such as `if (a == a)`, because of the use of ternary logic in T-SQL. If you use **true** or **false**, you avoid this possible confusion.

Example

The following code generates warning CS1718.

```
// CS1718.cs
using System;
public class Tester
{
    public static void Main()
    {
        int i = 0;
        if (i == i) { // CS1718.cs
            //if (true) {
                i++;
            }
        }
    }
}
```

Compiler Warning (level 4) CS0028

Error Message

'function declaration' has the wrong signature to be an entry point

The method declaration for **Main** was invalid: it was declared with an invalid signature. `Main` must be declared as static and it must return either `int` or `void`. For more information, see [Main\(\) and Command Line Arguments \(C# Programming Guide\)](#).

The following sample generates CS0028:

```
// CS0028.cs
// compile with: /W:4 /warnaserror
public class a
{
    public static double Main(int i)    // CS0028
    // Try the following line instead:
    // public static void Main()
    {
    }
}
```

Compiler Warning (level 4) CS0078

Error Message

The 'l' suffix is easily confused with the digit '1' -- use 'L' for clarity

The compiler warns when it detects a cast to long using a lowercase l instead of an uppercase L.

The following sample generates CS0078:

```
// CS0078.cs
// compile with: /W:4
class test {
    public static void TestL(long i)
    {
    }

    public static void TestL(int i)
    {
    }

    public static void Main()
    {
        TestL(25l);    // CS0078
        // try the following line instead
        // TestL(25L);
    }
}
```


Compiler Warning (level 4) CS0109

Error Message

The member 'member' does not hide an inherited member. The new keyword is not required

A class declaration included the [new](#) keyword even though the declaration does not override an existing declaration in a base class. You can delete the **new** keyword.

The following sample generates CS0109:

```
// CS0109.cs
// compile with: /W:4
namespace x
{
    public class a
    {
        public int i;
    }

    public class b : a
    {
        public new int i;
        public new int j;    // CS0109
        public static void Main()
        {
        }
    }
}
```

Compiler Warning (level 4) CS0402

Error Message

'identifier' : an entry point cannot be generic or in a generic type

The entry point was found in a generic type. To remove this warning, implement Main in a non-generic class or struct.

```
// CS0402.cs
// compile with: /W:4
class C<T>
{
    public static void Main() // CS0402
    {

    }
}

class CMain
{
    public static void Main() {}
}
```

Compiler Warning (level 4) CS0422

Error Message

The `/incremental` option is no longer supported

Incremental compilation (**`/incr`** or **`/incremental`**) is not supported in Visual C# 2005.

Compiler Warning (level 4) CS0429

Error Message

Unreachable expression code detected

This error occurs whenever part of an expression in your code is unreachable. In the following example, the condition `false && myTest()` meets this criteria because the `myTest()` method will never get evaluated due to the fact that the left side of the `&&` operation is always false. As soon as the `&&` operator evaluates the **false** statement as false, it stops the evaluation, and will never evaluate the right side.

Example

The following code generates CS0429.

```
// CS0429.cs
public class cs0429
{
    public static void Main()
    {
        if (false && myTest()) // CS0429
            // Try the following line instead:
            // if (true && myTest())
            {
            }
        else
        {
            int i = 0;
            i++;
        }
    }

    static bool myTest() { return true; }
}
```

Compiler Warning (level 4) CS0628

Error Message

'member' : new protected member declared in sealed class

A [sealed](#) class cannot introduce a [protected](#) member because no other class will be able to inherit from the **sealed** class and use the **protected** member.

The following sample generates CS0628:

```
// CS0628.cs
// compile with: /W:4
sealed class C
{
    protected int i;    // CS0628
}

class MyClass
{
    public static void Main()
    {
    }
}
```

Compiler Warning (level 4) CS0649

Error Message

Field 'field' is never assigned to, and will always have its default value 'value'

The compiler detected an uninitialized private or internal field declaration that is never assigned a value.

The following sample generates CS0649:

```
// CS0649.cs
// compile with: /W:4
using System.Collections;

class MyClass
{
    Hashtable table; // CS0649
    // You may have intended to initialize the variable to null
    // Hashtable table = null;

    // Or you may have meant to create an object here
    // Hashtable table = new Hashtable();

    public void Func(object o, string p)
    {
        // Or here
        // table = new Hashtable();
        table[p] = o;
    }

    public static void Main()
    {
    }
}
```

Compiler Warning (level 4) CS1573

Error Message

Parameter 'parameter' has no matching param tag in the XML comment for 'parameter' (but other parameters do)

When using the `/doc` compiler option, a comment was specified for some but not all parameters in a method. You may have forgotten to enter a comment for these parameters.

The following sample generates CS1573:

```
// CS1573.cs
// compile with: /W:4
public class MyClass
{
    /// <param name='Int1'>Used to indicate status.</param>
    // enter a comment for Char1?
    public static void MyMethod(int Int1, char Char1)
    {
    }

    public static void Main ()
    {
    }
}
```

Compiler Warning (level 4) CS1591

Error Message

Missing XML comment for publicly visible type or member 'Type_or_Member'

The `/doc` compiler option was specified, but one or more constructs did not have comments.

The following sample generates CS1591:

```
// CS1591.cs
// compile with: /W:4 /doc:x.xml

/// text
public class Test
{
    // /// text
    public static void Main()    // CS1591, remove "///" from previous line
    {
    }
}
```


Compiler Warning (level 4) CS1610

Error Message

Unable to delete temporary file 'file' used for default Win32 resource -- resource

When using the [/win32res](#) compiler option and when your **%TEMP%** directory does not have DELETE permission, this warning indicates that the compiler could not delete a temporary file that it created.

Make sure that you have read/write/delete permissions for the **%TEMP%** directory.

If necessary, you can manually delete these files and there will be no harm to C# or any of your programs.

Compiler Warning (level 4) CS1712

Error Message

Type parameter 'type parameter' has no matching typeparam tag in the XML comment on 'type' (but other type parameters do)

The documentation of a generic type is missing a **typeparam** tag. For more information, see [<typeparam> \(C# Programming Guide\)](#).

Example

The following code generates warning CS1712. To resolve this error, add a **typeparam** tag for the type parameter S.

```
// CS1712.cs
// compile with: /doc:cs1712.xml
using System;
class Test
{
    public static void Main() {}
    /// <param name="j"> This is the j parameter.</param>
    /// <typeparam name="T"> This is the T type parameter.</typeparam>
    public void bar<T,S>(int j) {} // CS1712
}
```

Compiler Error CS1725

Error Message

Friend assembly reference 'reference' is invalid. InternalsVisibleTo declarations cannot have a version, culture, public key token, or processor architecture specified.

You cannot add a version culture in a friend assembly reference. Partial classes should be visible to friend assemblies.

Example

The following sample generates CS1725.

```
// CS1725.cs
// compile with: /target:library
using System.Runtime.CompilerServices;
[assembly:InternalsVisibleTo("partial01,version=1.1.0.0")]    // CS1725
// try the following line instead
// [assembly:InternalsVisibleTo("partial01")]

partial class TestClass
{
    public static string strBar = "my string";
}
```

Compiler Error CS1726

Error Message

Friend assembly reference 'reference' is invalid. Strong-name signed assemblies must specify a public key in their InternalsVisibleTo declarations.

A strong name signed assembly can only grant friend assembly access, made with the [InternalsVisibleToAttribute](#), to other strongly signed assemblies.

To resolve CS1726, either sign (give a strong name to) the assembly to which you want to grant friend access, or don't grant friend access.

For more information, see [Friend Assemblies \(C# Programming Guide\)](#).

Example

The following sample generates CS1726.

```
// CS1726.cs
// compile with: /keyfile:CS1726.key /target:library
using System.Runtime.CompilerServices;
[assembly:InternalsVisibleTo("UnsignedAssembly")] // CS1726
// try the following line instead
// [assembly:InternalsVisibleTo("SignedAssembly, PublicKey=00240000048000009400000006020000
00240000525341310004000001000100031d7b6f3abc16c7de526fd67ec2926fe68ed2f9901afbc5f1b6b428bf6
cd9086021a0b38b76bc340dc6ab27b65e4a593fa0e60689ac98dd71a12248ca025751d135df7b98c5f9d09172f7
b62dabdd302b2a1ae688731ff3fc7a6ab9e8cf39fb73c60667e1b071ef7da5838dc009ae0119a9cbff2c581fc0f
2d966b77114b2c4")]

class A {}
```

Compiler Warning (level 2) CS0472

Error Message

The result of the expression is always 'value1' since a value of type 'value2' is never equal to 'null' of type 'value3'

The compiler should warn if you use an operator with a constant null value.

Example

The following sample generates CS0472.

```
public class Test
{
    public static int Main()
    {
        int i = 5;
        int counter = 0;

        // Comparison:
        if (i == null) // CS0472
        // To resolve, use a valid value for i.
            counter++;
        return counter;
    }
}
```

Compiler Error CS1727

Error Message

Cannot send error report automatically without authorization. Please visit " to authorize sending error report.

The Web site listed in the error text explains how to enable automatic error reporting for Visual Studio 2005 command line tools.

Example

The following sample generates CS1727.

```
// CS1727.cs
// compile with: /errorreport:send
// CS1727 expected
class Test {}
```

Compiler Error CS0735

Error Message

Invalid type specified as an argument for TypeForwardedTo attribute

The following sample generates CS0735.

```
// CS735.cs
// compile with: /target:library
using System.Runtime.CompilerServices;
[assembly:TypeForwardedTo(typeof(int[]))] // CS0735
[assembly:TypeForwardedTo(typeof(string))] // OK
```

Compiler Error CS1057

Error Message

'member': static classes cannot contain protected members"

This error is generated by declaring a protected member inside a static class.

Example

The following example generates CS1057.

```
// CS1057.cs
using System;

static class Class1
{
    protected static int x;    // CS1057
    public static void Main()
    {
    }
}
```


Compiler Warning (level 1) CS1058

Error Message

A previous catch clause already catches all exceptions. All non-CLS exceptions thrown will be wrapped in a `System.Runtime.CompilerServices.RuntimeWrappedException`

This attribute causes CS1058 if a `catch()` block has no specified exception type after a `catch (System.Exception e)` block. The warning advises that the `catch()` block will not catch any exceptions.

A `catch()` block after a `catch (System.Exception e)` block can catch non-CLS exceptions if the

RuntimeCompatibilityAttribute is set to false in the AssemblyInfo.cs file: `[assembly:`

`RuntimeCompatibilityAttribute(WrapNonExceptionThrows = false)]`. If this attribute is not set explicitly to false, all thrown non-CLS exceptions are wrapped as Exceptions and the `catch (System.Exception e)` block catches them. For more information, see [How to: Catch a non-CLS Exception](#).

Example

The following example generates CS1058.

```
// CS1058.cs
// CS1058 expected
using System.Runtime.CompilerServices;

// the following attribute is set to true by default in the C# compiler
// set to false in your source code to resolve CS1058
[assembly: RuntimeCompatibilityAttribute(WrapNonExceptionThrows = false)]

class TestClass
{
    static void Main()
    {
        try {}

        catch (System.Exception e) {
            System.Console.WriteLine("Caught exception {0}", e);
        }

        catch {}    // CS1058. This line will never be reached.
    }
}
```

C# Terminology

access modifier

A keyword, such as [private](#), [protected](#), [internal](#), or [public](#), that restricts access to a type or type member. For more information, see [Access Modifiers](#).

accessible member

A member that can be accessed by a given type. An accessible member for one type is not necessarily accessible to another type. For more information, see [Access Modifiers](#) and [Friend Assemblies](#).

accessor

A method that sets or retrieves the value of a private data member value that is associated with a property. Read-write properties have **get** and **set** accessors. Properties that are read-only have only a **get** accessor. For more information, see [Properties](#).

anonymous method

An anonymous method is a code block that is passed as a parameter to a [delegate](#). For more information, see [Anonymous Methods](#).

base class

A [class](#) that is inherited by another 'derived' class. For more information, see [Inheritance](#).

call stack

The series of method calls leading from the beginning of the program to the statement currently being executed at run time.

class

A data type that describes an object. Classes contain both data, and the methods for acting on the data. For more information, see [Classes](#).

constructor

A special method on a class or struct that initializes the objects of that type. For more information, see [Constructors](#).

delegate

A delegate is a type that references a method. Once a delegate is assigned a method, it behaves exactly like that method. For more information, see [Delegates](#).

derived class

A [class](#) that uses inheritance to gain, augment, or modify the behavior and data of another 'base' class. For more information, see [Inheritance](#).

destructor

A special method on a [class](#) or [struct](#) that prepares the instance for destruction by the system. For more information, see [Destructors](#).

event

A member of a [class](#) or [struct](#) that sends notifications of a change. For more information, see [Events](#).

field

A data member of a [class](#) or [struct](#) that is accessed directly.

generics

Generics allow you to define a [class](#) and or method that are defined with a type parameter. When client code instantiates the type, it specifies a particular type as an argument. For more information, see [Generics](#).

IDE

Integrated Development Environment. The application that provides the unified user interface for the various development tools including the compiler, debugger, code editor, and designers.

immutable type

A type whose instance data, fields and properties, does not change after the instance is created. Most value types are immutable.

inaccessible member

A member that cannot be accessed by a given type. An inaccessible member for one type is not necessarily inaccessible to another type. For more information, see [Access Modifiers](#).

inheritance

C# supports inheritance, so a [class](#) that derives from another class, known as the base class, inherits the same methods and properties. Inheritance involves base classes and derived classes. For more information, see [Inheritance](#).

interface

A type that contains only the signatures of public methods, events, and delegates. An object that inherits the interface must implement all of the methods and events defined in the interface. Classes or structs may inherit any number of interfaces. For more information, see [Interfaces](#)

iterator

An iterator is a method that enables consumers of a [class](#) that contains a collection or array to use [foreach, in \(C# Reference\)](#) to iterate through that collection or array.

member

A field, property, method, or event declared on a [class](#) or [struct](#).

method

A named code block that provides behavior for a [class](#) or [struct](#).

mutable type

A type whose instance data, fields and properties, can be changed after the instance is created. Most [Reference Types](#) are mutable.

nested type

A type declared within the declaration of another type.

object

An instance of a [class](#). An object exists in memory, and has data and methods that act on the data. For more information, see [Objects, Classes, and Structs](#).

property

A data member accessed by means of an accessor. For more information, see [Properties](#).

refactoring

Reusing previously entered code. The Visual C# Express Code Editor can intelligently reformat code to, for example, turn a block of highlight code into a method. For more information, see [Refactoring](#).

reference type

A data type. A variable declared as a reference type point to a location where data is stored. For more information, see [Reference Types](#).

static

A class or method declared as static exists without first being instantiated using the keyword **new**. `Main()` is a static method. For more information, see [Static Classes and Static Class Members](#).

struct

A compound data type that is typically used to contain a few variables that have some logical relationship. Structs can also contain methods and events. Structs do not support inheritance but they do support interfaces. A struct is a [value type](#), while a [class](#) is a [reference type](#). For more information, see [Structs](#).

value type

A value type is a data type that is allocated on the stack, as opposed to a reference type which is allocated on the heap. The

A value type is a data type that is allocated on the stack, as opposed to a reference type which is allocated on the heap. The [built-in types](#), including the numeric types as well as the struct type and the nullable type, are all value types. The [class](#) type and [string](#) type are [reference types](#). For more information, see [Value Types \(C# Reference\)](#).

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

C# Language Specification

Versions 1.2 and 2.0 of the C# Language Specifications are the authoritative source for C# grammar and syntax. They contain detailed information on all aspects of the language, including many points not covered in the Visual C# product documentation.

The 1.2 specification discusses features that were added to the language prior to Visual C# 2005 and the 2.0 specification discusses features that were added for Visual C# 2005.

The C# Language Specifications are available in Microsoft Word format from the following locations:

- On MSDN online at the [Visual C# Developer Center](#).
- In Visual Studio, in the VC#\Specifications\1033\ directory under your Microsoft Visual Studio 2005 installation directory.

If you do not have Microsoft Word installed on your computer, you can view, copy, and print the Word version of the specification with the free [Word Viewer 2003](#).

The C# Language Specification is also available as a book published by [Addison Wesley](#).

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[C# Reference](#)

Visual C# Samples

You can access sample code by browsing the sample abstracts in this section. Each abstract contains a link to open or copy the sample's files. In addition, the .NET Framework SDK includes technology and application samples, and QuickStart tutorials that demonstrate .NET Framework features and Visual C# code.

The QuickStart Tutorials are the fastest way to understand what the .NET Framework technology offers leading-edge developers. The QuickStarts are a series of samples and supporting documentation designed to quickly acquaint you with the syntax, architecture, and power of Visual Studio and the .NET Framework. The QuickStart tutorials include samples on the ASP.NET and Windows Forms applications, in addition to many how-to applications that cover the most compelling features of the .NET Framework technology.

To access the QuickStarts, click **Start**, point to **Programs**, point to **Microsoft .NET Framework SDK v2.0**, and then click **QuickStart Tutorials**. A Web page of the "Microsoft .NET Framework SDK QuickStart Tutorials" application is displayed. To run the QuickStarts, follow the directions on this page, which sets up the samples database and completes the installation. For more information, see [Samples and QuickStarts](#).

 Note
Visual C# Express users will see the following message when you open the Visual Studio solution (.sln) file for each of these samples: " Solution Folders are not supported in this version of Visual Studio. Solution Folder 'Solution Items' will be displayed as unavailable. " Although this folder is not available in Visual C# Express, you can still build and run the projects.

In This Section

Introductory Samples

Anonymous Delegates Sample	Demonstrates the use of unnamed delegates to reduce application complexity.
Arrays Sample	Shows how to use arrays.
Collection Classes Sample	Shows how to make non-generic collection classes that can be used with the foreach statement.
Generics Sample (C#)	Shows how to make generic collection classes that can be used with the foreach statement.
Command Line Parameters Sample	Demonstrates simple command-line processing and array indexing.
Conditional Methods Sample	Demonstrates conditional methods, which provide a powerful mechanism by which calls to methods can be included or omitted depending on whether a symbol is defined.
Delegates Sample	Shows how delegates are declared, mapped, and combined.
Events Sample	Shows how to use events in C#.
Explicit Interface Implementation Sample	Demonstrates how to explicitly implement interface members.
Hello World Sample	A Hello World application.
Indexers Sample	Shows how to use array notation to access an object.
Indexed Properties Sample	Shows how to implement a class that uses indexed properties. Indexed properties allow you to use a class that represents an array-like collection of several different kinds of things.

Properties Sample	Shows how properties are declared and used; also demonstrates abstract properties.
Structs Sample	Shows how to use structs in C#.
Operator Overloading Sample	Shows how user-defined classes can overload operators.
User-Defined Conversions Sample	Shows how to define conversions to and from user-defined types.
Versioning Sample	Demonstrates versioning in C# through the use of the override and new keywords.
Yield Sample	Demonstrates the yield keyword to filter items in a collection.

Intermediate and Advanced Samples

Attributes Sample	Shows how to create custom attribute classes, use them in code, and query them through reflection.
COM Interop Part 1 Sample	Shows how to use C# to interoperate with COM objects.
COM Interop Part 2 Sample	Shows how to use a C# server with a C++ COM client.
Libraries Sample	Shows how to use compiler options to create a DLL from multiple source files; also, how to use the library in other programs.
Nullable Sample	Demonstrates value types that can be set to null.
OLE DB Sample	Demonstrates how to use a Microsoft Access database from C#. It shows how you can create a dataset and add tables to it from a database.
Partial Types Sample	Demonstrates how classes and structures can be defined in multiple C# source-code files.
Platform Invoke Sample	Shows how to call exported DLL functions from C#.
Security Sample	Discusses .NET Framework security and shows two ways to modify security permissions in C#: using permission classes and permission attributes.
Threading Sample	Demonstrates various thread activities such as creating and executing a thread, synchronizing threads, interacting between threads, and using a thread pool.
Unsafe Code Sample	Shows how to use pointers.
XML Documentation Sample	Shows how to document code by using XML.

Related Sections

[Samples and QuickStarts](#) | [Visual C# Walkthroughs](#)

Hello World Sample

[Download sample](#)

This sample shows several versions of a Hello World program in C#.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample files in Solution Explorer

1. Click **Download Sample**.

The File Download message box appears.

2. Click **Open**, and on the left column of the zip folder, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory the files will be extracted to, or click **Next** again.

4. Make sure the **Show extracted files** check box is selected, and click **Finish**.

5. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the project solution is not trusted. Click **OK** to continue.

To build and run the Hello World samples within Visual Studio

1. Open the solution (HelloWorld.sln).
2. In **Solution Explorer**, right-click the HelloWorld1 project, and click **Set as StartUp Project**.
3. On the **Debug** menu, click **Start Without Debugging**.
4. Press any key to close HelloWorld1.
5. In **Solution Explorer**, right-click the HelloWorld2 project, and click **Set as StartUp Project**.
6. On the **Debug** menu, click **Start Without Debugging**.
7. Press any key to close HelloWorld2.
8. In **Solution Explorer**, right-click the HelloWorld3 project, and click **Set as StartUp Project**.
9. In **Solution Explorer**, right-click the HelloWorld3 project, and click **Properties**.
10. Open the **Configuration Properties** folder, and click **Debug**.
11. In the **Command Line Arguments** property, type **A B C D** and click **OK**.
12. On the **Debug** menu, click **Start Without Debugging**.
13. Press any key to close HelloWorld3.
14. In **Solution Explorer**, right-click the HelloWorld4 project, and click **Set as StartUp Project**.
15. On the **Debug** menu, click **Start Without Debugging**.
16. Press any key to close HelloWorld4.

To build and run the Hello World samples from the command line

1. Use the **Change Directory** command to change to the HelloWorld directory.

2. Type the following:

```
cd HelloWorld1  
csc Hello1.cs  
Hello1
```

3. Type the following:

```
cd ..\HelloWorld2  
csc Hello2.cs  
Hello2
```

4. Type the following:

```
cd ..\HelloWorld3  
csc Hello3.cs  
Hello3 A B C D
```

5. Type the following:

```
cd ..\HelloWorld4  
csc Hello4.cs  
Hello4
```

See Also

Concepts

[Hello World -- Your First Program \(C# Programming Guide\)](#)

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Command Line Parameters Sample

[Download sample](#)

This sample shows how the command line can be accessed and two ways of accessing the array of command-line parameters.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Command Line Parameters samples within Visual Studio

1. In **Solution Explorer**, right-click the CmdLine1 project, and click **Set as StartUp Project**.
2. In **Solution Explorer**, right-click the project, and click **Properties**.
3. Open the **Configuration Properties** folder, and click **Debug**.
4. In the **Command Line Arguments** property, type the command-line parameters, and click **OK**. (See the tutorial for an example.)
5. On the **Debug** menu, click **Start Without Debugging**.
6. Repeat the preceding steps for CmdLine2.

To build and run the Command Line Parameters samples from the command line

1. Use the **Change Directory** command to change to the CmdLine1 directory.
2. Type the following:

```
csc cmdline1.cs  
cmdline1 A B C
```

3. Use the **Change Directory** command to change to the CmdLine2 directory.
4. Type the following:

```
csc cmdline2.cs  
cmdline2 John Paul Mary
```

See Also

Tasks

[How to: Display Command Line Arguments \(C# Programming Guide\)](#)

[How to: Access Command-Line Arguments Using foreach \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Arrays Sample

[Download sample](#)

This sample describes and shows how arrays work in C#. For more information, see [Arrays \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Arrays sample within Visual Studio

- On the **Debug** menu, click **Start Without Debugging**.

To build and run the Arrays sample from the Command Line

- Type the following at the command prompt:

```
csc arrays.cs  
arrays
```

See Also

Reference

[Passing Arrays as Parameters \(C# Programming Guide\)](#)

[Passing Arrays Using ref and out \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Properties Sample

[Download sample](#)

This sample shows how properties are an integral part of the C# programming language. It demonstrates how properties are declared and used. See [Properties](#) for more information.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Properties samples within Visual Studio

1. In **Solution Explorer**, right-click the Person project and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for shapetest.

To build and run the Properties samples from the Command Line

1. Use the **Change Directory** command to change to the person directory.
2. Type the following:

```
csc person.cs
person
```

3. Use the **Change Directory** command to change to the shapetest directory.
4. Type the following:

```
csc abstractshape.cs shapes.cs shapetest.cs
shapetest
```

See Also

Tasks

[How to: Declare and Use Read/Write Properties \(C# Programming Guide\)](#)

[How to: Define Abstract Properties \(C# Programming Guide\)](#)

Reference

[Comparison Between Properties and Indexers \(C# Programming Guide\)](#)

Concepts[C# Programming Guide](#)**Other Resources**[Visual C# Samples](#)

Libraries Sample

[Download sample](#)

This sample shows how to create and use a DLL in C#.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Libraries sample within Visual Studio

1. In **Solution Explorer**, right-click the FunctionTest project, and click **Set as StartUp Project**.
2. In **Solution Explorer**, right-click the FunctionTest project, and click **Properties**.
3. Open the **Configuration Properties** folder and click **Debug**.
4. In the **Command Line Arguments** property, enter 3 5 10.
5. Click **OK**.
6. On the **Debug** menu, click **Start Without Debugging**. This will automatically build the library in Functions and execute the program.

To build and run the Libraries sample from the command line

1. Use the **Change Directory** command to change to the Functions directory.
2. Type the following:

```
csc /target:library /out:Functions.dll Factorial.cs DigitCounter.cs
```

3. Use the **Change Directory** command to change to the FunctionTest directory.
4. Type the following:

```
copy ..\Functions\Functions.dll .  
csc /out:FunctionTest.exe /R:Functions.DLL FunctionClient.cs  
FunctionTest 3 5 10
```

See Also

Tasks

[How to: Create and Use C# DLLs \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Versioning Sample

[Download sample](#)

This sample demonstrates versioning in C# through the use of the **override** and **new** keywords. Versioning helps maintain compatibility between base and derived classes as they evolve. For additional information, see [Versioning with the Override and New Keywords \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Versioning sample within Visual Studio

- On the **Debug** menu, click **Start Without Debugging**.

To build and run the Versioning sample from the Command Line

- Type the following at the command prompt:

```
csc versioning.cs
versioning
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Collection Classes Sample

[Download sample](#)

This sample shows how to implement a collection class that can be used with the **foreach** statement. For more information, see [Collection Classes \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Collection Classes samples within Visual Studio

1. In **Solution Explorer**, right-click the CollectionClasses1 project and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for CollectionClasses2.

To build and run the Collection Classes samples from the command line

1. Use the **Change Directory** command to change to the CollectionClasses1 directory.
2. Type the following:

```
csc tokens.cs
tokens
```

3. Use the **Change Directory** command to change to the CollectionClasses2 directory.
4. Type the following:

```
csc tokens2.cs
tokens2
```

See Also

Tasks

[How to: Access a Collection Class with foreach \(C# Programming Guide\)](#)

Reference

[System.Collections.Generic](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

[Commonly Used Collection Types](#)

Structs Sample

[Download sample](#)

This sample presents the syntax and usage of structs. It also covers the important differences between classes and structs. See [Objects, Classes and Structs \(C# Programming Guide\)](#) for more information.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Structs samples within Visual Studio

1. In **Solution Explorer**, right-click the Struct1 project and click **Set as StartUp Project**.
2. From the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for Struct2.

To build and run the Structs samples from the Command Line

1. Use the **Change Directory** command to change to the Struct1 directory.
2. Type the following:

```
csc struct1.cs
struct1
```

3. Use the **Change Directory** command to change to the Struct2 directory.
4. Type the following:

```
csc struct2.cs
struct2
```

See Also

Tasks

[How to: Know the Difference Between Passing a Struct and Passing a Class Reference to a Method \(C# Programming Guide\)](#)

Reference

[struct \(C# Reference\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Indexers Sample

[Download sample](#)

This sample shows how C# classes can declare indexers to provide array-like access to the classes.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Indexers sample within Visual Studio

1. In **Solution Explorer**, right-click the indexers project, and click **Properties**.
2. Open the **Configuration Properties** folder, and click **Debug**.
3. In the **Command Line Arguments** property, enter `..\..\Test.txt`.
4. Click **OK**.
5. On the **Debug** menu, click **Start Without Debugging**.

To build and run the Indexers sample from the command line

1. To compile the sample program, type the following at the command prompt:

```
csc indexer.cs
```

2. The sample program reverses the bytes in a file given as a command-line argument. For example, to reverse the bytes in Test.txt and see the result, issue the following commands:

```
indexers Test.txt  
type Test.txt
```

3. To change the reversed file back to normal, run the program on the same file again.

See Also

Reference

[Using Indexers \(C# Programming Guide\)](#)

[Indexers in Interfaces \(C# Programming Guide\)](#)

[Comparison Between Properties and Indexers \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Indexed Properties Sample

[Download sample](#)

This sample shows how C# classes can declare indexed properties to represent an array-like collection of different kinds of things. For more information, see [Properties \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Indexed Properties sample within Visual Studio

- On the **Debug** menu, click **Start Without Debugging**.

To build and run the Indexed Properties sample from the command line

- Type the following at the command prompt:

```
csc indexedproperty.cs
indexedproperty
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

User-Defined Conversions Sample

[Download sample](#)

This sample shows how to define conversions to or from classes or structures, and how such conversions are used. See [Conversion Operators \(C# Programming Guide\)](#) for more information.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the User-Defined Conversions samples within Visual Studio

1. In **Solution Explorer**, right-click the Conversion1 project and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for Conversion2.

To build and run the User-Defined Conversions samples from the Command Line

1. Use the **Change Directory** command to change to the Conversion1 directory.
2. Type the following:

```
csc conversion.cs
conversion
```

3. Use the **Change Directory** command to change to the Conversion2 directory.
4. Type the following:

```
csc structconversion.cs
structconversion
```

See Also

Tasks

[How to: Implement User-Defined Conversions Between Structs \(C# Programming Guide\)](#)

Reference

[Implicit Numeric Conversions Table \(C# Reference\)](#)

[Explicit Numeric Conversions Table \(C# Reference\)](#)

Concepts[C# Programming Guide](#)**Other Resources**[Visual C# Samples](#)

Generics Sample (C#)

[Download sample](#)

This sample shows how to create a custom generic list class with a single type parameter, and how to implement **IEnumerable<T>** to enable **foreach** iteration over the contents of the list. The sample also shows how client code creates an instance of the class by specifying a type argument, and how constraints on the type parameter enable additional operations to be performed on the type arguments.

For an example of a generic collection class that implements an iterator block, see [How to: Create an Iterator Block for a Generic List \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Generics sample within Visual Studio

- On the **Debug** menu, click **Start Without Debugging**.

To build and run the Generics sample from the command line

- Type the following at the command prompt:

```
csc generics.cs
generics
```

Comments

This sample is provided for demonstration purposes and is not intended to be used without modification in production code. For production quality code, it is strongly recommended that you use the collection classes in the [System.Collections.Generic](#) namespace whenever possible.

See Also

Reference

[System.Collections.Generic](#)

Concepts

[Generics \(C# Programming Guide\)](#)

Other Resources

[Visual C# Samples](#)

Operator Overloading Sample

[Download sample](#)

This sample demonstrates how user-defined classes can overload operators. See [C# Operators](#) for more information.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Operator Overloading samples within Visual Studio

1. In **Solution Explorer**, right-click the Complex project and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for Dbbool.

To build and run the Operator Overloading samples from the Command Line

1. Use the **Change Directory** command to change to the Complex directory.
2. Type the following:

```
csc complex.cs
complex
```

3. Use the **Change Directory** command to change to the Dbbool directory.
4. Type the following:

```
csc dbbool.cs
dbbool
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Delegates Sample

[Download sample](#)

This sample demonstrates the delegate types. It shows how to map delegates to static and instance methods, and how to combine them to create multicast delegates.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Delegates samples within Visual Studio

1. In **Solution Explorer**, right-click the Delegates1 project, and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for Delegates2.

To build and run the Delegates samples from the command line

1. Use the **Change Directory** command to change to the Delegates1 directory.
2. Type the following:

```
csc bookstore.cs
bookstore
```

3. Use the **Change Directory** command to change to the Delegates2 directory.
4. Type the following:

```
csc compose.cs
compose
```

See Also

Tasks

[How to: Combine Delegates \(Multicast Delegates\)\(C# Programming Guide\)](#)

Reference

[Using Delegates \(C# Programming Guide\)](#)

Concepts

[Delegates \(C# Programming Guide\)](#)

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Events Sample

[Download sample](#)

This sample shows how to declare, invoke, and configure events in C#. For more information, see [Events \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Events samples within Visual Studio

1. In **Solution Explorer**, right-click the Events1 project, and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for Events2.

To build and run the Events samples from the command line

1. Use the **Change Directory** command to change to the Events1 directory.
2. Type the following:

```
csc events1.cs
events1
```

3. Use the **Change Directory** command to change to the Events2 directory.
4. Type the following:

```
csc events2.cs
events2
```

See Also

Tasks

[How to: Subscribe to and Unsubscribe from Events \(C# Programming Guide\)](#)

[How to: Publish Events that Conform to .NET Framework Guidelines \(C# Programming Guide\)](#)

[How to: Raise Base Class Events in Derived Classes \(C# Programming Guide\)](#)

[How to: Implement Interface Events \(C# Programming Guide\)](#)

[How to: Use a Dictionary to Store Event Instances \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Explicit Interface Implementation Sample

[Download sample](#)

This sample demonstrates how to explicitly implement interface members and how to access those members from interface instances. For background information, see [Interfaces \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Explicit Interface Implementation samples within Visual Studio

1. In **Solution Explorer**, right-click the ExplicitInterface1 project and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for ExplicitInterface2.

To build and run the Explicit Interface Implementation samples from the Command Line

1. Use the **Change Directory** command to change to the ExplicitInterface1 directory.
2. Type the following:

```
csc explicit1.cs
explicit1
```

3. Use the **Change Directory** command to change to the ExplicitInterface2 directory.
4. Type the following:

```
csc explicit2.cs
explicit2
```

See Also

Tasks

[How to: Explicitly Implement Interface Members \(C# Programming Guide\)](#)

[How to: Explicitly Implement Interface Members with Inheritance \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Conditional Methods Sample

[Download sample](#)

This sample demonstrates conditional methods, which provide a powerful mechanism by which calls to methods can be included or omitted depending on whether a symbol is defined.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Conditional Methods sample within Visual Studio

1. In **Solution Explorer**, right-click the project and click **Properties**.
2. Open the Configuration Properties folder, and click **Debug**.
3. Set the **Command Line Arguments** property to **A B C**.
4. In the Configuration Properties folder, click **Build**.
5. Modify the **Conditional Compilation Constants** property. For example, add or delete DEBUG. Click **OK**.
6. On the **Debug** menu, click **Start Without Debugging**.

To build and run the Conditional Methods sample from the command line

- To include the conditional method, compile and run the sample program by typing the following at the command prompt:

```
csc CondMethod.cs tracetest.cs /d:DEBUG
tracetest A B C
```

See Also

Reference

[Conditional \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

XML Documentation Sample

[Download sample](#)

This sample shows how to use XML to document code. See [XML Documentation Comments \(C# Programming Guide\)](#) for additional information.

Security Note
This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build the XML Documentation sample within Visual Studio

1. In **Solution Explorer**, right-click the project and click **Properties**.
2. Open the Configuration Properties folder and click **Build**.
3. Set the XML Documentation File property to XMLsample.xml.
4. On the **Build** menu, click **Build**. The XML output file will be in the debug directory.

To build the XML Documentation sample from the Command Line

1. To generate the sample XML documentation, type the following at the command prompt:

```
csc XMLsample.cs /doc:XMLsample.xml
```

2. To see the generated XML, issue the following command:

```
type XMLsample.xml
```

See Also

Tasks

[How to: Use the XML Documentation Features \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Platform Invoke Sample

[Download sample](#)

This sample demonstrates how to call platform invokes (exported DLL functions) from C#. See [Interoperability \(C# Programming Guide\)](#) for more information.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Platform Invoke samples within Visual Studio

1. In **Solution Explorer**, right-click the PinvokeTest project and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for Marshal and Pinvoke.

To build and run the Platform Invoke samples from the Command Line

1. Use the **Change Directory** command to change to the PinvokeTest directory.
2. Type the following:

```
csc PinvokeTest.cs
PinvokeTest
```

3. Use the **Change Directory** command to change to the Marshal directory.
4. Type the following:

```
csc Marshal.cs
Marshal
```

5. Use the **Change Directory** command to change to the Pinvoke directory.
6. Type the following:

```
csc logfont.cs pinvoke.cs
pinvoke
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

COM Interop Part 1 Sample

[Download sample](#)

This sample demonstrates how a C# program can interoperate with an unmanaged COM component.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the COM Interop Part 1 samples within Visual Studio

1. In **Solution Explorer**, right-click the Interop1 project, and click **Set as StartUp Project**.
2. In **Solution Explorer**, right-click the Interop1 project, and click **Properties**.
3. Open the **Configuration Properties** folder, and click **Debug**.
4. In the Command-Line Arguments property, enter an AVI file such as `c:\winnt\clock.avi`.
5. Click **OK**.
6. On the **Debug** menu, click **Start Without Debugging**.
7. Repeat the preceding steps for Interop2.

If the QuartzTypeLib.dll included with Interop1 is out of date

1. In **Solution Explorer**, open **References** for Interop1.
2. Right-click **QuartzTypeLib**, and click **Remove**.
3. Right-click **References**, and click **Add Reference**.
4. On the **COM** tab, select the component named, "ActiveMovie control type library."
5. Click **Select**, and then click **OK**.
6. Rebuild Interop1.

Note

Adding a reference to the component does the same thing as invoking `tlbimp` at the command line to create QuartzTypeLib.dll (as shown below).

To build and run the COM Interop Part 1 samples from the command line

1. Use the **Change Directory** command to change to the Interop1 directory.
2. Type the following:

```
tlbimp %windir%\system32\quartz.dll /out:QuartzTypeLib.dll  
csc /r:QuartzTypeLib.dll interop1.cs  
interop1 %windir%\clock.avi
```

3. Use the **Change Directory** command to change to the Interop2 directory.
4. Type the following:

```
csc interop2.cs  
interop2 %windir%\clock.avi
```

See Also

Reference

[Interoperability \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

COM Interop Part 2 Sample

[Download sample](#)

This sample demonstrates using a C# server with a C++ COM client.

Note

Visual C++ must be installed to compile this sample.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the COM Interop Part 2 sample within Visual Studio

1. In **Solution Explorer**, right-click the COMClient project, and click **Properties**.

2. Open the **Configuration Properties** folder, and click **Debug**.

3. In the **Command Line Arguments** property, enter a name.

4. Click **OK**.

5. On the **Debug** menu, click **Start Without Debugging**.

To build and run the COM Interop Part 2 sample from the command line

1. Use the **Change Directory** command to change to the COMInteropPart2\COMClient directory.

2. Copy the C# server code into the COMClient directory:

```
copy ..\CSharpServer\CSharpServer.cs
```

3. Compile the server:

```
csc /target:library CSharpServer.cs  
regasm CSharpServer.dll /tlb:CSharpServer.tlb
```

4. Compile the client (make sure the path and environment variables are set properly with vcvars32.bat):

```
cl COMClient.cpp
```

5. Run the client:

```
COMClient friend
```

See Also

Reference

[Interoperability \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Attributes Sample

[Download sample](#)

This sample demonstrates how to create custom attribute classes, use them in code, and query them through reflection. For more information on attributes, see [Attributes \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Attributes sample within Visual Studio

- On the **Debug** menu, click **Start Without Debugging**.

To build and run the Attributes sample from the Command Line

- Type the following at the command prompt:

```
csc AttributesTutorial.cs
AttributesTutorial
```

See Also

Tasks

[How to: Create a C/C++ Union Using Attributes \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Security Sample

Download sample

This sample demonstrates how to modify security permissions through permission classes and permission attributes. For additional information, see [Security \(C# Programming Guide\)](#).

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Security samples within Visual Studio

1. In **Solution Explorer**, right-click the Security1 project and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for Security2 and Security3.

To build and run the Security samples from the Command Line

1. Use the **Change Directory** command to change to the Security1 directory.
2. Type the following:

```
csc ImperativeSecurity.cs
ImperativeSecurity
```

3. Use the **Change Directory** command to change to the Security2 directory.
4. Type the following:

```
csc DeclarativeSecurity.cs
DeclarativeSecurity
```

5. Use the **Change Directory** command to change to the Security3 directory.
6. Type the following:

```
csc SuppressSecurity.cs
SuppressSecurity
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Threading Sample

Download sample

This sample demonstrates the following threading techniques. See [Threading \(C# Programming Guide\)](#) for more information.

- Creating, launching, and terminating a thread
- Using a thread pool
- Thread synchronization and interaction

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Threading samples within Visual Studio

1. In **Solution Explorer**, right-click the ThreadStartStop project and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. Repeat the preceding steps for ThreadPool, and ThreadSync.

To build and run the Threading samples from the Command Line

1. Use the **Change Directory** command to change to the Threads directory.
2. Type the following:

```
cd ThreadStartStop
csc ThreadStartStop.cs
ThreadStartStop
```

3. Type the following:

```
cd ..\ThreadPool
csc ThreadPool.cs
ThreadPool
```

4. Type the following:

```
cd ..\ThreadSync
csc ThreadSync.cs
ThreadSync
```

See Also

Tasks

[How to: Create and Terminate Threads \(C# Programming Guide\)](#)

[How to: Use a Thread Pool \(C# Programming Guide\)](#)

[How to: Synchronize a Producer and a Consumer Thread \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

Unsafe Code Sample

[Download sample](#)

This sample demonstrates how to use unmanaged code (code using pointers) in C#.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Unsafe Code samples within Visual Studio

1. In **Solution Explorer**, right-click the FastCopy project, and click **Set as StartUp Project**.
2. On the **Debug** menu, click **Start Without Debugging**.
3. In **Solution Explorer**, right-click the ReadFile project, and click **Set as StartUp Project**.
4. In **Solution Explorer**, right-click the ReadFile project, and click **Properties**.
5. Open the Configuration Properties folder, and click **Debug**.
6. In the Command Line Arguments property, enter `..\..\ReadFile.cs`.
7. Click **OK**.
8. On the **Debug** menu, click **Start Without Debugging**.
9. In **Solution Explorer**, right-click the PrintVersion project, and click **Set as StartUp Project**.
10. On the **Debug** menu, click **Start Without Debugging**.

To build and run the Unsafe Code samples from the Command Line

1. Use the **Change Directory** command to change to the Unsafe directory.
2. Type the following:

```
cd FastCopy
csc FastCopy.cs /unsafe
FastCopy
```

3. Type the following:


```
cd ..\ReadFile  
csc ReadFile.cs /unsafe  
ReadFile ReadFile.cs
```

4. Type the following:

```
cd ..\PrintVersion  
csc PrintVersion.cs /unsafe  
PrintVersion
```

See Also

Tasks

[How to: Use Pointers to Copy an Array of Bytes \(C# Programming Guide\)](#)

[How to: Use the Windows ReadFile Function \(C# Programming Guide\)](#)

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

OLE DB Sample

[Download sample](#)

This sample demonstrates how to use a Microsoft Access database from C#. It shows how you can create a dataset and add tables to it from a database. The BugTypes.mdb database used in the sample program is a Microsoft Access 2000 .mdb file.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the OLE DB sample within Visual Studio

- On the **Debug** menu, click **Start Without Debugging**.

Note

If you are building the solution in Release mode, copy BugTypes.mdb into the \bin\release folder.

To build and run the OLE DB sample from the Command Line

- Type the following at the command prompt:

```
csc oledbsample.cs
oledbsample
```

See Also

Concepts

[C# Programming Guide](#)

Other Resources

[Visual C# Samples](#)

[Accessing Data \(Visual Studio\)](#)

Yield Sample

[Download sample](#)

This sample shows how to create a list class that implements **IEnumerable<int>** and the **yield** keyword to enable **foreach** iteration over the contents of the list. Two properties are defined, one returning the odd numbers, the other returning the even numbers.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Yield Code sample within Visual Studio

- On the **Debug** menu, click **Start Without Debugging**.

To build and run the Yield Code sample from the Command Line

1. Use the **Change Directory (cd)** command to change to the **Yield** directory.
2. Type the following:

```
csc Yield.cs
Yield
```

See Also

Reference

[yield \(C# Reference\)](#)

Concepts

[Iterators \(C# Programming Guide\)](#)

Other Resources

[Visual C# Samples](#)

[Generics Sample \(C#\)](#)

Anonymous Delegates Sample

[Download sample](#)

This sample demonstrates using an anonymous delegate to calculate a salary bonus for an employee. Using an anonymous delegate simplifies the program because there is no need to define a separate method.

The data for each employee is stored in an object containing personal details as well as a delegate that references the algorithm required to calculate the bonus. By defining the algorithm by means of a delegate, the same method can be used to perform the bonus calculation, irrespective of how it is actually calculated. Also of note, a local variable, multiplier, becomes a captured outer variable because it is referenced in a delegated calculation.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the AnonymousDelegates Code sample within Visual Studio

- On the **Debug** menu, click **Start Without Debugging**.

To build and run the AnonymousDelegates Code sample from the Command Line

1. Use the **Change Directory (cd)** command to change to the AnonymousDelegates directory.
2. Type the following:

```
csc AnonymousDelegates.cs
AnonymousDelegates
```

See Also

Reference

[Anonymous Methods \(C# Programming Guide\)](#)

Concepts

[Delegates \(C# Programming Guide\)](#)

Other Resources

[Visual C# Samples](#)

[C# Reference](#)

Partial Types Sample

[Download sample](#)

This sample demonstrates the use of partial types, which enable a class or struct to be defined in two or more C# files. This allows multiple programmers to work on different portions of a class in parallel, and for different facets of a complex class to be kept in separate files.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the PartialTypes Code sample within Visual Studio

- From the **Debug** menu, click **Start Without Debugging**.

To build and run the PartialTypes Code sample from the Command Line

1. Use the **Change Directory (cd)** command to change to the **PartialTypes** directory.
2. Type the following:

```
csc PartialTypes.cs
PartialTypes
```

See Also

Reference

[partial \(C# Reference\)](#)

[Partial Class Definitions \(C# Programming Guide\)](#)

Other Resources

[Visual C# Samples](#)

Nullable Sample

[Download sample](#)

This sample demonstrates how to use nullable types. This feature allows value types to have an uninitialized, or empty, state similar to how reference types can be set to **null**.

Security Note

This sample code is provided to illustrate a concept and should not be used in applications or Web sites, as it may not illustrate the safest coding practices. Microsoft assumes no liability for incidental or consequential damages should the sample code be used for purposes other than as intended.

To open the sample file in Solution Explorer

1. Click **Download Sample**.

The **File Download** message box appears.

2. Click **Open**, and on the left column of the zip folder window, click **Extract all files**.

The **Extraction Wizard** opens.

3. Click **Next**. You can change the directory that the files will be extracted to, and then click **Next** again.

Make sure that the **Show extracted files** check box is selected, and click **Finish**.

4. Double-click the sample's .sln file.

The sample solution is displayed in **Solution Explorer**. You might get a security warning that says the solution location is not trusted. Click **OK** to continue.

To build and run the Nullable Code sample within Visual Studio

1. Open Nullable.sln, the solution file, either by double-clicking on the file from Windows Explorer, or by clicking **Open** on the **File** menu.
2. On the **Debug** menu, click **Start Without Debugging**.

To build and run the Nullable Code sample from the command line

1. Use the **Change Directory (cd)** command to change to the \Boxing, \Basics, or \Operator subdirectory under the Nullable directory.
2. For the Boxing sample, type the following:

```
csc Boxing.cs
Boxing
```

For the Basics sample, type the following:

```
csc Basics.cs
Basics
```

For the Operator sample, type the following:

```
csc Operator.cs
Operator
```

See Also

Concepts

[Nullable Types \(C# Programming Guide\)](#)

Other Resources

[Visual C# Samples](#)